OpenText™ Service Manager

Integrate

Version : 9.81

PDF Generated on : May 23, 2026

© Copyright 2026 Open Text

Table of Contents

1. Integrate 1
1.1. Service Manager integration methods and tools 2
1.1.1. Web Services 5
1.1.1.1. Service Manager Web Services 6
1.1.1.1.1. What's a Web Service? 7
1.1.1.1.1.1. Understand the Service Manager Web Services 8
1.1.1.1.1.2. Web Services basics 9
1.1.1.1.1.3. Add or change Web Services 10
1.1.1.1.2. Introduction to Web Services in Service Manager 11
1.1.1.1.2.1. Web Services and Service Manager 12
1.1.1.1.2.2. Web Services naming conventions for SOAP 13
1.1.1.1.2.3. Web Services security considerations 14
1.1.1.1.2.4. Valid URLs for Service Manager 15
1.1.1.1.2.5. Service Manager Web Services URLs 16
1.1.1.1.2.6. Configure the WSDL field definitions 17
1.1.1.2. Publish Service Manager data using WS API 21
1.1.1.2.1. Things to consider prior to publishing data 22
1.1.1.2.1.1. Publishing Service Manager applications as Web Services 23
1.1.1.2.1.2. When to use Web Services 24
1.1.1.2.1.3. Can I use the out-of-box Web Services? 25
1.1.1.2.1.4. What items do I need to expose? 26
1.1.1.2.1.5. What data types should I use for SOAP? 29
1.1.1.2.1.6. What methods do I need? 32
1.1.1.2.1.7. Manage records with Web Services requests 33
1.1.1.2.1.8. Are there any security considerations? 34
1.1.1.2.1.9. What are released Web Services? 35
1.1.1.2.1.10. Enable SSL encryption for published Web Services 36

1.1.1.2.2. Example: Publishing request processes for the PPM integration 38
1.1.1.2.2.1. Create the display option 39
1.1.1.2.2.2. Set up the Request Management category 41
1.1.1.2.2.3. Create the new Process 42
1.1.1.2.2.4. Set up the State record 43
1.1.1.2.2.5. Update the format control record 44
1.1.1.2.2.6. Set up the extaccess record 45
1.1.1.2.3. List: Web Services available in the Service Manager Web Services API 51
1.1.1.2.4. Field names in the extaccess record 55
1.1.1.2.5. Create dedicated Web Services listeners 58
1.1.1.2.6. Data conversion between Service Manager and SOAP Web Services 60
1.1.1.2.7. Example: Publish the Terminate Change functionality by using Web Services 61
1.1.1.2.7.1. Create a display option 62
1.1.1.2.7.2. Create a new process 63
1.1.1.2.7.3. Set up a State record 65
1.1.1.2.7.4. Set up an extaccess record 66
1.1.1.2.7.5. Execute a request via SOAP Web Services 67
1.1.1.2.7.6. Response to a request via SOAP Web Services 74
1.1.1.2.7.7. Execute a request via RESTful Web Services 77
1.1.1.2.7.8. Response to a request by using RESTful Web Services 78
1.1.1.2.8. Publish a table as a Web Service 80
1.1.1.2.9. Expose a table with more than one Web Service 83
1.1.1.2.10. Remove a Document Engine display action from the Web Services API 84
1.1.1.2.11. Remove a Service Manager field from the Web Services API 85
1.1.1.2.12. Sample client for Web Services SM7 URL 86
1.1.1.2.13. Command line arguments for the Axis2 sample application 89
1.1.1.2.14. Add an external access action to the Web Services 91
1.1.1.3. Web services authentication 93

1.1.1.4. SOAP API 94
1.1.1.4.1. Web Services Description Language (WSDL) 95
1.1.1.4.1.1. Basic operations in WSDL files 96
1.1.1.4.1.2. Service Manager WSDL files 98
1.1.1.4.1.3. Types of Web Services in Service Manager 99
1.1.1.4.1.4. WSDL document structure 100
1.1.1.4.1.5. Change example to use the cookie 107
1.1.1.4.1.6. Example: Use Keep-Alive with .Net Web Services Studio 109
1.1.1.4.2. Consume a Service Manager Web Service 113
1.1.1.4.2.1. Dynamic and static Web Services clients 115
1.1.1.4.2.2. Updating Service Manager tables 116
1.1.1.4.2.3. Requirements for developing custom Web Services clients 117
1.1.1.4.2.4. Checklist: Creating a custom Web Services client 118
1.1.1.4.2.5. Sample Web Services client for sc62server PWS URL 120
1.1.1.4.2.6. Command line arguments for the .NET sample application 123
1.1.1.4.2.7. Command line arguments for the Axis sample application 125
1.1.1.4.2.8. Using query syntax 127
1.1.1.4.2.9. Retrieving data from Service Manager 133
1.1.1.4.2.10. Example: Retreiving data from Service Manager via a Web service 135
1.1.1.4.2.11. Retrieve data from Service Manager using Pagination 141
1.1.1.4.2.12. Retrieve data from Service Manager for Optimistic Locking 151
1.1.1.4.2.13. Web Services examples in the RUN directory 153
1.1.1.4.2.14. Special considerations for using Keep-Alive with Service Manager 165
1.1.1.4.2.15. Use SSL to consume Service Manager Web Services 167
1.1.1.4.2.16. Attachment handling 168
1.1.1.4.3. Consume an external Web Service 181
1.1.1.4.3.1. Use the WSDL2JS utility 182
1.1.1.4.3.2. Best practices for writing a JavaScript to consume a Web Service 184

1.1.1.4.3.3. Date/Time handling 185
1.1.1.4.3.4. Example: Interface to another system 186
1.1.1.4.3.5. Web Services with a proxy server 211
1.1.1.4.3.6. Connecting to a secure Web Service 212
1.1.1.4.3.7. Use SSL connections to connect to a Web Service 214
1.1.1.4.3.8. Web Services connections through a firewall 217
1.1.1.5. The RESTful API 218
1.1.1.5.1. Service Document 219
1.1.1.5.2. Consuming Service Manager RESTful API 220
1.1.1.5.2.1. RESTful syntax 221
1.1.1.5.2.2. RESTful commands 224
1.1.1.5.2.3. RESTful queries 226
1.1.1.5.2.4. Resource representations 230
1.1.1.5.2.5. Enable a resource for REST 232
1.1.1.5.2.6. RESTful capability word 234
1.1.1.5.2.7. HTTP header 235
1.1.1.5.2.8. HTTP response codes 236
1.1.1.5.2.9. OOB resource reference example 237
1.1.1.5.3. Attachment operations using the RESTful API 242
1.1.1.6. Web Services debugging parameters 248
1.1.1.7. Web Services special parameters 252
1.1.1.8. Testing your WSDL with a SOAP UI 253
1.1.1.9. Running Web Services on a dedicated port (servlet) 254
1.1.1.10. Debugging SOAP errors 255
1.1.1.11. Debugging RESTful API 258
1.1.1.12. Syntax for entity references in xml 259
1.1.1.13. Definitions, acronyms, and abbreviations 260
1.1.1.14. Web Services resources 261

1.1.2. Connect-It 262
1.1.3. ServiceCenter Automation (SCAuto) 263
1.1.3.1. ServiceCenter Automate Software Development Kit 264
1.1.3.1.1. Utility programs 273
1.1.3.1.2. Developing applications 279
1.1.3.1.3. C language API 290
1.1.4. Event Services 302
1.1.4.1. Event Services overview 303
1.1.4.1.1. Supported external products 304
1.1.4.1.2. Communication with external systems 305
1.1.4.1.3. Event Services routing 306
1.1.4.1.4. Routing events with external products 307
1.1.4.1.5. Event Services tables 310
1.1.4.1.6. Event Services workflow 312
1.1.4.2. Event Services operations 314
1.1.4.2.1. Registered events 315
1.1.4.2.1.1. View registered events 316
1.1.4.2.1.2. Add a registered event 317
1.1.4.2.2. Input events 318
1.1.4.2.2.1. Input event processing 319
1.1.4.2.2.2. View Event Services input events 320
1.1.4.2.3. Output events 321
1.1.4.2.3.1. Output event processing 322
1.1.4.2.3.2. View Event Services output events 323
1.1.4.2.4. Generic events 324
1.1.4.2.5. E-mail events 325
1.1.4.3. Mapping events 326
1.1.4.3.1. Event mapping process 327

1.1.4.3.1.1. View event maps 329
1.1.4.3.2. Using event maps 330
1.1.4.3.2.1. Mapping an array of structures 331
1.1.4.3.2.2. Mapping to multiple tables 332
1.1.4.3.2.3. Using expressions in event maps 335
1.1.4.3.2.4. Mapping and Configuration Management 336
1.1.4.3.2.5. Considerations for Configuration Management maps 337
1.1.4.3.2.6. Rules for building maps 338
1.1.4.3.3. Filtering events 339
1.1.4.3.3.1. View event filters 340
1.1.4.3.3.2. Event blocking 341
1.1.4.3.3.3. Selecting filters 343
1.1.4.4. Event Services and Change Management 345
1.1.4.4.1. Change Management input events 346
1.1.4.4.1.1. Change Management eventin fields 347
1.1.4.4.1.2. Change Management input event registration 348
1.1.4.4.2. External information string 349
1.1.4.4.2.1. External information string fields 350
1.1.4.4.2.2. External information string actions 351
1.1.4.4.3. Synchronizing with an external system 352
1.1.4.4.3.1. Acknowledgement events 353
1.1.4.4.4. Change Management output events 355
1.1.4.4.5. Example: Open a change 356
1.1.4.5. Event agents 358
1.1.4.5.1. Event scheduling 359
1.1.4.5.2. View event schedules 360
1.1.4.5.3. Agent status 361
1.1.4.5.4. View Event Services agent status 362

1.1.4.5.5. View system startup information 363
1.1.4.6. Email solutions based on Event Services 364
1.1.4.7. Format Control and eventout records 365
1.1.4.7.1. Generating email messages 366
1.1.4.8. Out-of-box events 367
1.1.4.8.1. Change Management events 368
1.1.4.8.2. Configuration Management events 369
1.1.4.8.3. Incident Management events 374
1.1.4.8.4. Service Desk events 376
1.1.4.8.5. Service Level Management events 377
1.1.4.8.6. Standard events 378
1.1.5. Dynamic Data Exchange (DDE) 391
1.1.5.1. Architecture 392
1.1.5.2. DDE client 393
1.1.5.2.1. DDE RAD panel 394
1.1.5.2.2. FrameRestore option 395
1.1.5.2.3. PassFocus option 396
1.1.5.2.4. Process panel 397
1.1.5.2.5. Structure support option 398
1.1.5.2.6. Usage notes 399
1.1.5.3. DDE server 400
1.1.5.3.1. Executes 401
1.1.5.3.2. SystemEvents file 402
1.1.5.3.3. Requests and pokes 403
1.1.5.3.4. DDE example 404
1.1.5.3.5. Access the script panel 405
1.1.5.3.6. Access system events records 406
1.1.6. Integration Manager 407

1.1.6.1. Add or delete an integration instance 410
1.1.6.1.1. Integration Instance Information fields 412
1.1.6.1.2. Edit an integration instance 418
1.1.6.1.3. Integration Instance Mapping 420
1.1.6.1.3.1. Add or delete field mappings 422
1.1.6.1.3.2. Edit field mappings 423
1.1.6.1.3.3. Configure a callback 424
1.1.6.1.3.4. Configure value mappings 426
1.1.6.1.3.5. Configure a condition 428
1.1.6.1.3.6. Pre script 430
1.1.6.1.3.7. Post script 431
1.1.6.1.3.8. Use placeholders 432
1.1.6.2. Enable or disable an integration instance 437
1.1.6.3. Monitor integration instance status 439
1.1.6.4. Monitor failover tasks 440
1.1.6.5. Monitor SMIS task log 442
1.1.6.6. Using LW-SSO with integrations 444
1.1.6.6.1. Configure LW-SSO on the Service Manager server 446
1.1.6.6.2. Configure LW-SSO on the Service Manager Web tier 449
1.1.6.6.3. Configure LW-SSO in Application Performance Management (APM) 453
1.1.6.6.4. Configure LW-SSO in Operations Orchestration (OO) 455
1.1.7. BDM Mapping Management 456
1.1.7.1. Field Mapping 458
1.1.7.2. Value Mapping 471
1.1.7.3. Atom Mapping 473
1.1.7.4. Customizing BDM mapping configuration 475
1.1.7.5. BDM RESTful API 477
1.1.8. Case Exchange framework 479

1.1.8.1. Case Exchange framework features 481
1.1.8.2. Enable Case Exchange with another system 487
1.1.8.2.1. General procedure 488
1.1.8.2.1.1. Recommendations 489
1.1.8.2.1.2. Configure Incident environment 490
1.1.8.2.1.3. Create a new integration template 491
1.1.8.2.1.4. Add and enable a Case Exchange integration instance 492
1.1.8.2.1.5. Configuration details 494
1.1.8.2.1.6. Configure Case Exchange Rule Sets 511
1.1.8.2.1.7. Invoke Case Exchange Rule Sets 512
1.1.8.2.2. Enable Case Exchange with SMAX 516
1.1.8.2.2.1. The Pull mechanism 517
1.1.8.2.2.2. Configure SSL between Service Manager and SMAX 519
1.1.8.2.2.3. Add and enable an integration instance 520
1.1.8.2.2.4. Create and invoke Rule Sets 523
1.1.8.2.2.5. Specify Service Manager as an external system in SMAX 526
1.1.8.2.2.6. Test the integration 529
1.1.8.2.3. Enable Case Exchange between two Service Manager systems 532
1.1.8.2.3.1. Connection mechanisms 533
1.1.8.2.3.2. Using the Pull mechanism 536
1.1.8.2.3.3. Using the Push mechanism 544
1.1.8.3. Incident exchange scenarios 552
1.1.8.4. Common user tasks 574
1.1.8.5. System administrator tasks 576
1.1.8.6. Case Exchange reference material 578
1.1.8.6.1. Edit the query and pagination query strings 579
1.1.8.6.2. Specify additional path 592
1.1.8.6.3. Entity path 593

1.1.8.6.4. Technical hints 595
1.1.8.6.5. Dealing with HTML tags from SMAX 597
1.1.8.6.6. Out-of-box functions for additional scripts 600
1.1.8.6.7. Limitations and known issues 606
1.1.9. Computer Telephony Integration (CTI) with the Web client 607
1.1.9.1. Use the new CTI method 609
1.1.9.1.1. How the new CTI method works 610
1.1.9.1.2. How to use the new CTI method 613
1.1.9.1.3. How to test the new CTI method 621
1.1.9.2. Use URI 624
1.1.10. Integrations and extacess records 629
1.2. Change Configuration and Release Management (CCRM) 630
1.3. SAP Solution Manager 631
1.3.1. Install and administer Service Manager Exchange with SAP Solution Manager 632
1.3.1.1. Existing fragmented Incident Management workflow 633
1.3.1.2. Deployment scenarios 635
1.3.1.3. Install and configure SMSSMEX 637
1.3.1.3.1. Install SMSSMEX 638
1.3.1.3.2. Configuring Tomcat 641
1.3.1.3.3. Setting up database 642
1.3.1.3.3.1. Oracle 643
1.3.1.3.3.2. MS-SQL 645
1.3.1.3.4. Configuring ovictex.properties 649
1.3.1.3.5. Configuring File ovictexInternal.properties 651
1.3.1.3.6. External Helpdesks 652
1.3.1.3.7. Configuring FieldMapping.xml 654
1.3.1.3.8. Verifying Configuration 655
1.3.1.3.9. Deploying on WebLogic 656

1.3.1.3.10. Starting/Stopping SMSSMEX (Tomcat only) 657
1.3.1.4. Upgrading to SMSSMEX V1.40 658
1.3.1.5. Customizing Service Manager 660
1.3.1.5.1. Configuring the denyattachwithoutext parameter 661
1.3.1.5.2. Importing Customizations Using Unload 662
1.3.1.5.3. Enabling the Integration 663
1.3.1.5.4. Creating a Service Manager User for Web Service 664
1.3.1.5.5. Configuring Incident Form (Classic/Hybrid Only) 666
1.3.1.5.6. Configuring Incident Form (Codeless Only) 668
1.3.1.5.7. Configuring WSDL Mapping 670
1.3.1.5.8. Adding Instance in SMIS and Configuring Parameters 671
1.3.1.6. Configuring SAP Solution Manager 673
1.3.1.6.1. Prerequisites 674
1.3.1.6.2. Configuring SAP Solution Manager External Service Desk interface 676
1.3.1.6.2.1. Release Web Service 678
1.3.1.6.2.2. Assign Roles to the Communication User 680
1.3.1.6.2.3. Create HTTP Connection 681
1.3.1.6.2.4. Create a Logical Port 683
1.3.1.6.2.5. Configure Interface to SAP Solution Manager Service Desk 685
1.3.1.6.2.6. Define Value Mapping for the Service Desk Interface 686
1.3.1.6.2.7. Define Extended Interface Mapping for Service Desk 687
1.3.1.6.2.8. Get SAP Solution Manager Service Port 688
1.3.1.6.3. Solution Manager tracing 689
1.3.1.6.3.1. Enable tracing 690
1.3.1.6.3.2. Download Trace File 691
1.3.1.7. Configuring security 692
1.3.1.7.1. Security between SAP Solution Manager and Tomcat 693
1.3.1.7.1.1. Configure SAP Solution Manager for SSL 694

1.3.1.7.1.2. Set up SSL between SAP and SMSSMEX 703
1.3.1.7.2. Security Between Service Manager and SMSSMEX 707
1.3.1.7.2.1. Configure Service Manager for SSL 708
1.3.1.7.2.2. Configure SMSSMEX for SSL Communication with Service Manager 709
1.3.1.8. Licensing 710
1.3.1.9. Status Page 711
1.3.1.10. Incident exchange details 713
1.3.1.10.1. Database Tables 714
1.3.1.10.2. Tools 719
1.3.1.10.3. Field mapping configuration 720
1.3.1.10.3.1. Types of Mapping 721
1.3.1.10.3.2. Structure of FieldMapping XML file 722
1.3.1.10.3.3. Composite Field Mapping 723
1.3.1.10.3.4. Field Value Mapping 726
1.3.1.10.3.5. Field Mapping Schema 727
1.3.1.10.3.6. Default field mapping file and customization 732
1.3.1.10.4. Person Synchronization Details 740
1.3.1.10.5. SMSSMEX Version 742
1.3.1.11. Installing and Configuring SAPCRYPLIB 743
1.3.1.12. Logging 744
1.3.1.13. Deploying Button Icons 745
1.3.1.14. SAP System Landscape Directory Registration 746
1.3.2. Purpose of document 748
1.3.2.1. Prerequisites 749
1.3.2.2. Architecture 750
1.3.2.3. Use cases 752
1.3.2.4. User scenarios 756
1.3.2.4.1. Set up SAP Instance CI in Service Manager 757

1.3.2.4.2. SM 9.x to SAP Solution Manager 7.1 759
1.3.2.4.2.1. Create an incident in Service Manager 760
1.3.2.4.2.2. Open a new incident to send to SAP 762
1.3.2.4.2.3. Open the incident in Solution Manager 764
1.3.2.4.2.4. Check updates in Service Manager 767
1.3.2.4.2.5. Synchronize new information with Solution Manager 768
1.3.2.4.2.6. Check updates from Service Manager 769
1.3.2.4.2.7. Update the incident status in Solution Manager 770
1.3.2.4.2.8. Close the incident in Service Manager 771
1.3.2.4.2.9. Check the incident's status in Solution Manager 772
1.3.2.4.3. SAP Solution Manager 7.1 to SM 9.x 773
1.3.2.4.3.1. Create incident in Solution Manager 774
1.3.2.4.3.2. Send Solution in Service Manager 778
1.3.2.4.3.3. Check the solution from Service Manager 781
1.3.2.4.3.4. Send the incident back to Service Manager 782
1.3.2.4.3.5. Send the incident back to Solution Manager again 783
1.3.2.4.3.6. Close the incident in Solution Manager 785
1.3.2.4.3.7. Check the incident's status in Service Manager 786
1.4. Project and Portfolio Management Center (PPM) 787
1.4.1. Configuring Service Manager 788
1.4.1.1. Add an integration instance 789
1.4.1.2. Configure the Service Manager Web tier server URL 791
1.4.2. Configuring PPM Center 792
1.4.2.1. Configuring the PPM Workbench 793
1.4.2.2. Configuring the PPM Server 809
1.4.3. Upgrading SM Service Catalog-PPM Center Project Proposal Integration Solution 812
1.4.4. Customize SM Project Proposal Catalog Item Fields 813
1.5. Application Lifecycle Management (ALM) 819

1.5.1. Prerequisites 821
1.5.2. Configure the settings in ALM 825
1.5.3. Configure the settings in Service Manager 831
1.5.4. Manage mappings 842
1.5.5. Verify the integration 846
1.6. Universal CMDB (UCMDB) 851
1.6.1. Introduction 852
1.6.1.1. Who should read the SM-UCMDB integration documentation 853
1.6.1.2. Supported adapters 854
1.6.1.3. Purpose of the integration 855
1.6.1.3.1. Supported use cases 856
1.6.1.3.1.1. Enabling ITIL processes 857
1.6.1.3.1.2. Managing planned Changes 858
1.6.1.3.1.3. Managing unplanned Changes 859
1.6.1.3.1.4. Retrieving Service Manager record information 860
1.6.1.3.1.5. Retrieving the actual state of UCMDB CIs 861
1.6.1.3.1.6. Accessing SM CI information in UCMDB 862
1.6.1.3.1.7. Retrieving UCMDB primary CI change history in SM 863
1.6.1.3.1.8. Support of Automated Service Modeling (ASM) 864
1.6.1.3.1.9. Centralized CI visualization and impact analysis 865
1.6.1.3.2. Core features 870
1.6.1.3.2.1. Push 871
1.6.1.3.2.2. Federation 872
1.6.1.3.2.3. Population 873
1.6.1.4. How CI information is synchronized between UCMDB and Service Manager 874
1.6.1.4.1. CI display name field in Service Manager 875
1.6.1.4.2. CI information usage 876
1.6.1.4.3. High-level components of the integration 877

1.6.1.4.4. Relationships between integration components 878
1.6.1.4.5. What information is stored in UCMDB 879
1.6.1.4.6. What information is stored in Service Manager 880
1.6.1.5. Discovery Event Manager 881
1.6.1.5.1. Discovery Event Manager rules 882
1.6.1.5.1.1. Discovery Event Manager rule options 883
1.6.1.5.1.2. Add a rule in Discovery Event Manager 886
1.6.1.5.1.3. View, modify, or delete rules in Discovery Event Manager 887
1.6.1.5.1.4. Customize Changes in Discovery Event Manager 889
1.6.1.5.1.5. Customize Incidents in Discovery Event Manager 890
1.6.1.5.2. Discovery Event Manager managed fields 891
1.6.1.5.2.1. Add a managed field in Discovery Event Manager 892
1.6.1.5.2.2. View, modify, or delete a managed field in Discovery Event Manager 894
1.6.1.5.3. Discovery Event Manager change open process 895
1.6.2. Using the ServiceManagerEnhancedAdapter9.41 adapter 897
1.6.2.1. How to upgrade your integration 898
1.6.2.1.1. Task 1. Close all SM unplanned Changes and Incidents created for CI modifications 899
1.6.2.1.2. Task 2. Upgrade Service Manager 900
1.6.2.1.3. (Optional) Task 3. Copy data to the joinnode table 901
1.6.2.1.4. Task 4. Update the DEM rules in Service Manager 902
1.6.2.1.5. Task 5. Enable the Restful API for custom CI types in Service Manager 903
1.6.2.1.6. Task 6. Upgrade UCMDB 907
1.6.2.1.7. Task 7: Update the configurations for custom CI types in UCMDB 908
1.6.2.1.8. Task 8. Upgrade field mapping 917
1.6.2.1.9. Task 9. Reconfigure an integration point 921
1.6.2.1.10. Task 10. Run a Full Push to resynchronize CI names 922
1.6.2.1.11. Task 11. Restore the DEM rules in Service Manager 923
1.6.2.2. Integration setup 924

1.6.2.2.1. Integration requirements 926
1.6.2.2.2. Integration setup overview 929
1.6.2.2.3. Service Manager setup 930
1.6.2.2.3.1. How to create an integration user account 931
1.6.2.2.3.2. How to add UCMDB connection information 932
1.6.2.2.4. Universal CMDB setup 953
1.6.2.2.4.1. How to create an integration point in UCMDB 954
1.6.2.2.5. Populating UCMDB with Service Manager CI data 961
1.6.2.2.5.1. How to define population jobs in UCMDB 962
1.6.2.2.5.2. View Service Manager CI data in UCMDB 966
1.6.2.2.5.3. How to schedule CI population jobs 967
1.6.2.2.6. Pushing UCMDB CI data to Service Manager 968
1.6.2.2.6.1. How to define data push jobs in UCMDB 969
1.6.2.2.6.2. How to schedule data push jobs 974
1.6.2.2.6.3. How to view UCMDB CI data in Service Manager 976
1.6.2.2.6.4. Retrieving UCMDB primary CI change history in SM 977
1.6.2.2.7. Federating Service Manager Ticket Data to UCMDB 978
1.6.2.2.7.1. Federation Queries 979
1.6.2.2.7.2. Examples of using federation 980
1.6.2.3. Multi-tenancy (Multi-company) setup 993
1.6.2.3.1. Multi-tenancy (Multi-company) support 994
1.6.2.3.1.1. Implementing multi-tenancy in the UCMDB-SM integration 995
1.6.2.3.1.2. Mandanten SM security layer 996
1.6.2.3.1.3. What multi-tenant Information is stored in UCMDB 997
1.6.2.3.1.4. What multi-tenant information is stored in Service Manager 998
1.6.2.3.1.5. Synchronization of company records 999
1.6.2.3.1.6. Tenant-specific Discovery Event Manager (DEM) rules 1005
1.6.2.3.2. Multi-tenancy requirements 1007

1.6.2.3.3. Setting up the multi-tenancy integration in UCMDB 1008
1.6.2.3.3.1. How to install a separate data flow probe for each tenant 1009
1.6.2.3.3.2. How to start tenant-specific data flow probes 1011
1.6.2.3.3.3. How to configure IP ranges for tenant-specific data flow probes 1012
1.6.2.3.4. Setting up the multi-tenancy integration in Service Manager 1013
1.6.2.3.4.1. How to start the schedule process 1015
1.6.2.3.4.2. How to configure the Service Manager System Information Record and Integration Manager 1016
1.6.2.3.4.3. How to start the process schedule 1018
1.6.2.3.4.4. How to add tenant-specific UCMDB user ID and password values 1019
1.6.2.3.4.5. How to add UCMDB customer ID values to existing companies 1020
1.6.2.3.4.6. How to synchronize existing companies from Service Manager to UCMDB 1021
1.6.2.3.4.7. How to view whether company information is in UCMDB 1022
1.6.2.3.4.8. How to resynchronize an existing company with UCMDB 1023
1.6.2.3.4.9. How to inactivate a synchronized company 1024
1.6.2.3.4.10. How to reactivate an inactive company 1025
1.6.2.3.4.11. How to add tenant-specific DEM rules 1026
1.6.2.3.4.12. How to add tenant-specific UCMDB user ID and password values 1027
1.6.2.3.4.13. How to add UCMDB customer ID values to existing companies 1028
1.6.2.3.4.14. How to Synchronize existing companies from Service Manager to UCMDBs 1029
1.6.2.3.4.15. How to view whether company information is in UCMDB 1030
1.6.2.3.4.16. How to resynchronize an existing company with UCMDB 1031
1.6.2.3.4.17. How to inactivate a synchronized company 1032
1.6.2.3.4.18. How to reactivate an inactive company 1033
1.6.2.3.4.19. How to add tenant-specific DEM rules 1034
1.6.2.4. Standards and best practices 1035
1.6.2.4.1. UCMDB-SM configuration best practices 1036

1.6.2.4.1.1. Centralized CI type management 1037
1.6.2.4.1.2. Bi-directional data synchronization recommendations 1039
1.6.2.4.1.3. Push scheduling recommendations 1041
1.6.2.4.1.4. Push in clustered environments 1043
1.6.2.4.1.5. Initial load configurations 1045
1.6.2.4.1.6. How to configure Differential or Delta Load DEM rules 1050
1.6.2.4.1.7. Fault detection and recovery for push 1051
1.6.2.4.1.8. How to enable Lightweight Single Sign-On (LW-SSO) configuration 1052
1.6.2.4.2. Frequently asked questions 1053
1.6.2.4.2.1. When is a new CI created in Service Manager? 1054
1.6.2.4.2.2. Can I analyze the reason for a CI deletion in SM? 1055
1.6.2.4.2.3. How do I monitor relationship changes between UCMDB and SM? 1056
1.6.2.4.2.4. What kinds of relationships are pushed from UCMDB to SM? 1057
1.6.2.4.2.5. What is a Root CI node? 1058
1.6.2.4.2.6. What is a Root relationship? 1059
1.6.2.4.2.7. What's the “Virtual-Compound” relationship type used in a UCMDB-SM integration query? 1060
1.6.2.4.2.8. When do I need the population feature? 1061
1.6.2.4.2.9. Can I populate physically deleted CIs from SM to UCMDB? 1062
1.6.2.4.2.10. How do I keep the outage dependency setting of a CI relationship in SM? 1063
1.6.2.4.2.11. How do I create an XML configuration file? 1066
1.6.2.4.2.12. How do I use the Load Fields button to add multiple managed fields? 1067
1.6.2.4.2.13. What is the purpose of the element in the population configuration file (smPopConf.xml)? 1068
1.6.2.4.2.14. Can I populate sub-item deletions? 1069
1.6.2.4.2.15. What will happen if a population job fails or succeeds with warnings? 1070
1.6.2.5. Tailoring the integration 1071
1.6.2.5.1. Integration architecture 1072

1.6.2.5.1.1. Integration class model 1073
1.6.2.5.1.2. Integration queries 1074
1.6.2.5.1.3. Service Manager Web Services 1083
1.6.2.5.1.4. Service Manager reconciliation rules 1089
1.6.2.5.1.5. Service Manager Discovery Event Manager rules 1091
1.6.2.5.2. Integration tailoring options 1094
1.6.2.5.2.1. How to enable ASM support 1095
1.6.2.5.2.2. How to update the integration adapter configuration File (sm.properties) 1096
1.6.2.5.2.3. How to add DEM reconciliation rules 1105
1.6.2.5.2.4. How to add Discovery Event Manager rules 1108
1.6.2.5.2.5. How to add a CI relationship type to the integration for data push 1112
1.6.2.5.2.6. How to add a CI attribute to the integration for data push 1118
1.6.2.5.2.7. How to add a CI type to the integration for data push 1131
1.6.2.5.2.8. How to add a CI type, attribute, or relationship type to the integration for population 1145
1.6.2.5.2.9. How to enable or disable UCMDB ID pushback for a CI type 1146
1.6.2.5.2.10. How to add an attribute of a supported CI type for federation 1149
1.6.2.5.2.11. How to enable the automatic deletion of CI relationships from UCMDB 1155
1.6.3. Using the ServiceManagerAdapter9.41 adapter 1159
1.6.3.1. How to upgrade your integration 1160
1.6.3.2. Integration setup 1166
1.6.3.2.1. Integration requirements 1168
1.6.3.2.2. Integration setup overview 1171
1.6.3.2.3. Service Manager setup 1172
1.6.3.2.3.1. How to create an integration user account 1173
1.6.3.2.3.2. How to add UCMDB connection information 1174
1.6.3.2.4. Universal CMDB Setup 1195
1.6.3.2.4.1. How to create an integration point in UCMDB 1196

1.6.3.2.4.2. How to update the time zone and date format for the integration adapter 1201
1.6.3.2.5. Populating UCMDB with Service Manager CI data 1203
1.6.3.2.5.1. How to define population jobs in UCMDB 1204
1.6.3.2.5.2. View Service Manager CI data in UCMDB 1207
1.6.3.2.5.3. How to schedule CI population jobs 1208
1.6.3.2.6. Pushing UCMDB CI data to Service Manager 1209
1.6.3.2.6.1. How to define data push jobs in UCMDB 1210
1.6.3.2.6.2. How to schedule data push jobs 1215
1.6.3.2.6.3. How to view UCMDB CI data in Service Manager 1217
1.6.3.2.7. Federating Service Manager ticket data to UCMDB 1218
1.6.3.2.7.1. Federation queries 1219
1.6.3.2.7.2. Examples of using federation 1220
1.6.3.3. Multi-tenancy (Multi-company) setup 1231
1.6.3.3.1. Multi-tenancy (Multi-company) support 1232
1.6.3.3.1.1. Implementing multi-tenancy in the UCMDB-SM integration 1233
1.6.3.3.1.2. Mandanten SM security layer 1234
1.6.3.3.1.3. What multi-tenant Information is stored in UCMDB 1235
1.6.3.3.1.4. What multi-tenant information is stored in Service Manager 1236
1.6.3.3.1.5. Synchronization of company records 1237
1.6.3.3.1.6. Tenant-specific Discovery Event Manager (DEM) rules 1243
1.6.3.3.2. Multi-tenancy requirements 1245
1.6.3.3.3. Setting up the multi-tenancy integration in UCMDB 1246
1.6.3.3.4. Setting up the multi-tenancy integration in Service Manager 1250
1.6.3.4. Standards and best practices 1252
1.6.3.4.1. UCMDB-SM configuration best practices 1253
1.6.3.4.1.1. Bi-directional data synchronization recommendations 1254
1.6.3.4.1.2. Push scheduling recommendations 1256
1.6.3.4.1.3. Push in clustered environments 1257

1.6.3.4.1.4. Initial load configurations 1259
1.6.3.4.1.5. How to configure differential/delta load DEM rules 1264
1.6.3.4.1.6. Fault detection and recovery for push 1265
1.6.3.4.1.7. How to enable Lightweight Single Sign-On (LW-SSO) configuration 1266
1.6.3.4.2. Frequently asked questions 1267
1.6.3.4.2.1. When is a new CI created in Service Manager? 1268
1.6.3.4.2.2. Can I analyze the reason for a CI deletion in SM? 1269
1.6.3.4.2.3. How do I monitor relationship changes between UCMDB and SM? 1270
1.6.3.4.2.4. What kinds of relationships are pushed from UCMDB to SM? 1271
1.6.3.4.2.5. What's a Root CI Node 1272
1.6.3.4.2.6. What's a Root relationship 1273
1.6.3.4.2.7. What's the "friendlyType" specified in an XSLT file 1274
1.6.3.4.2.8. What's the “Virtual-Compound” relationship type used in a UCMDB-SM integration query? 1275
1.6.3.4.2.9. When do I need the population feature? 1276
1.6.3.4.2.10. Can I populate physically deleted CIs from SM to UCMDB? 1277
1.6.3.4.2.11. How do I keep the outage dependency setting of a CI relationship in SM? 1278
1.6.3.4.2.12. How do I create an XSL transformation file 1281
1.6.3.4.2.13. How do I use the Load Fields button to add multiple managed fields? 1284
1.6.3.4.2.14. What's the purpose of the Element in a population XSLT file 1285
1.6.3.4.2.15. Can I populate sub-item deletions? 1286
1.6.3.4.2.16. What will happen if a population job fails or succeeds with warnings? 1287
1.6.3.4.3. Known issues and limitations 1288
1.6.3.5. Tailoring the integration 1293
1.6.3.5.1. Integration architecture 1294
1.6.3.5.1.1. Integration class model 1295
1.6.3.5.1.2. Integration queries 1296
1.6.3.5.1.3. Service Manager Web Services 1305

1.6.3.5.1.4. Service Manager reconciliation rules 1311
1.6.3.5.1.5. Service Manager Discovery Event Manager rules 1313
1.6.3.5.2. Integration tailoring options 1316
1.6.3.5.2.1. How to enable ASM support 1317
1.6.3.5.2.2. How to update the integration adapter configuration File (sm.properties) 1318
1.6.3.5.2.3. How to add DEM reconciliation rules 1326
1.6.3.5.2.4. Add Discovery Event Manager rules 1329
1.6.3.5.2.5. How to add a CI attribute to the integration for data push 1333
1.6.3.5.2.6. How to add a CI type to the integration for data push 1346
1.6.3.5.2.7. How to add a CI type's relationship types to the integration for data push 1368
1.6.3.5.2.8. How to Add a CI Attribute to the Integration for Population 1374
1.6.3.5.2.9. How to add a CI type to the integration for population 1378
1.6.3.5.2.11. How to customize UCMDB ID pushback for a CI type 1400
1.6.3.5.2.12. How to add custom queries to integration population jobs 1402
1.6.3.5.2.13. How to add an attribute of a supported CI type for federation 1403
1.6.4. CMS UI 1410
1.6.5. CMS Gateway 1412
1.7. Operations Orchestration (OO) 1413
1.7.1. Operations Orchestration integration setup 1415
1.7.1.1. Add an Operations Orchestration integration 1416
1.7.1.2. Enable SSL connection from Service Manager to Operations Orchestration 1424
1.7.1.3. Enable LW-SSO for the Operations Orchestration integration 1432
1.7.2. Manage OO flows from Knowledge Management 1433
1.7.2.1. Operations Orchestration flow synchronization rules 1435
1.7.2.2. Operations Orchestration flow detail form fields 1437
1.7.2.3. Add OO flow links to a knowledge document 1440
1.7.2.4. Remove OO flow links from a knowledge document 1443

1.7.3. Automated resolution of incidents 1444
1.7.3.1. Launch OO flows from an incident 1445
1.7.3.2. View OO flow execution results from an incident 1447
1.7.4. Automated deployment of changes 1448
1.7.4.1. Add OO flow links to a new change record 1449
1.7.4.2. Add OO flow links to an existing change record 1453
1.7.4.3. Update OO flow links in a change record 1457
1.7.4.4. Remove OO flow links from a change record 1458
1.7.4.5. Manually launch change flows 1459
1.7.4.6. Automatically launch change flows upon approval 1461
1.7.4.7. Automatically launch change flows on a schedule (for a new Change) 1462
1.7.4.8. Automatically launch change flows on a schedule (for an existing change) 1464
1.7.4.9. View execution results of change flows 1466
1.7.5. Automated implementation of change tasks 1467
1.7.5.1. Enable OO flow automation for change tasks 1468
1.7.5.2. Add an OO flow link to a change task 1469
1.7.5.3. Execute an OO flow from a change task 1474
1.7.6. Automated fulfillment of request tasks 1476
1.7.6.1. Enable OO flow automation for request tasks 1477
1.7.6.2. Add an OO flow link to a request task 1478
1.7.6.3. Execute an OO flow from a request task 1483
1.7.7. Example: launching automated implementation for change and request tasks 1485
1.8. Application Performance Management (APM) 1489
1.8.1. Incident Exchange (OMi - SM) integration 1490
1.8.1.1. Incident Exchange (OMi - SM) integration setup 1491
1.8.1.1.1. Create user accounts for the Incident Exchange (OMi - SM) integration 1493
1.8.1.1.2. Configure the Service Manager server as a connected server in OMi 1494
1.8.1.1.3. Configure an event forwarding rule in OMi 1498

1.8.1.1.4. Enable incident drill-down from OMi Event Browser 1499
1.8.1.1.5. Configure SSL for the Incident Exchange (OMi - SM) integration 1500
1.8.1.1.6. Configure the Instance Count in the SMOMi integration template 1501
1.8.1.1.7. Add an integration instance for each OMi server 1502
1.8.1.1.8. Enable LW-SSO for the Incident Exchange (OMi - SM) integration 1512
1.8.1.1.9. Configure automatic closure for OMi incidents 1514
1.8.1.1.10. Change the default assignment group for OMi incidents 1519
1.8.1.2. Synchronization of incident changes back to OMi 1520
1.8.1.3. Working with the Incident Exchange (OMi - SM) integration 1521
1.8.1.3.1. View related OMi event details from an incident 1522
1.8.1.3.2. Mark an incident for automatic closure 1523
1.8.2. APM Business Impact Report (BIR) 1524
1.8.2.1. APM Business Impact Report integration setup 1525
1.8.2.1.1. Add a Business Impact Report integration 1526
1.8.2.1.2. Enable LW-SSO for the Business Impact Report (BIR) integration 1529
1.8.2.1.3. Enable multi-tenancy for the APM Business Impact Report integration 1530
1.8.2.2. Launch a Business Impact Report from an incident 1532
1.8.3. SM-APM downtime synchronization 1533
1.8.3.1. SM-APM downtime synchronization setup 1535
1.8.3.1.1. Add an instance in Service Manager Integration Suite (SMIS) 1537
1.8.3.1.2. Tailor Service Manager to handle phase change 1543
1.8.3.1.3. Set up integration in APM 1544
1.8.3.1.4. Set up integration in Universal CMDB 1545
1.8.3.1.5. Verify the SM-APM downtime synchronization setup 1546
1.9. Operations Bridge Manager (OBM) 1549
1.9.1. Incident Exchange (OBM - SM) integration 1553
1.9.1.1. Incident Exchange (OBM - SM) integration setup 1554
1.9.1.1.1. Create user accounts for the Incident Exchange (OBM-SM) integration 1556

1.9.1.1.2. Configure the Service Manager server as a connected server in OBM 1557
1.9.1.1.3. Add custom attributes and map to SM fields 1561
1.9.1.1.4. Configure an event forwarding rule in OBM 1562
1.9.1.1.5. Enable event drill-down from Service Manager into OBM 1563
1.9.1.1.6. Enable incident drill-down from the OBM Event Browser 1565
1.9.1.1.7. Configure SSL for the Incident Exchange (OBM - SM) integration 1567
1.9.1.1.8. Configure the Instance Count in the SMOMi integration template 1569
1.9.1.1.9. Add an integration instance for each OBM server 1570
1.9.1.1.10. Enable LW-SSO for the Incident Exchange (OBM - SM) integration 1580
1.9.1.1.11. Configure automatic closure for OBM incidents 1582
1.9.1.1.12. Change the default assignment group for OBM incidents 1587
1.9.1.1.13. Test the connection 1588
1.9.1.1.14. Synchronize attributes 1590
1.9.1.2. Tips for customizing groovy scripts 1592
1.9.1.3. Synchronization of incident changes back to Operations Bridge Manager (OBM) 1596
1.9.1.4. Working with the Incident Exchange (OBM - SM) integration 1597
1.9.1.4.1. Drill down to the OBM event details from an incident 1598
1.9.1.4.2. Mark an incident for automatic closure 1599
1.9.2. Downtime exchange between OBM and Service Manager 1600
1.9.2.1. Downtime Exchange integration setup 1606
1.9.2.1.1. Enable OBM to send downtime events to Service Manager 1607
1.9.2.1.2. Integrate Service Manager downtime with OBM 1609
1.9.2.1.3. Add an integration instance in Service Manager 1611
1.9.2.1.4. Tailor Service Manager to handle phase change 1616
1.9.2.1.5. Verify the OBM-SM downtime synchronization setup 1617
1.10. Service Management Automation X (SMAX) 1619
1.11. Asset Manager 1621
1.12. ALM Octane 1624

1.12.1. Configure the settings in ALM Octane 1626
1.12.2. Configure the settings in Service Manager 1630
1.12.3. Manage mappings 1640
1.12.4. Verify the integration 1646
1.13. Microsoft Teams 1649
1.13.1. Set up the integration for war room experience 1653
1.13.2. Set up agent bot 1678
1.13.3. Set up virtual agent 1688
1.13.4. Set up Teams notification 1695
1.14. Atlassian Jira 1700
1.14.1. Configure the settings in Jira 1702
1.14.2. Configure the settings in Service Manager 1705
1.14.3. Manage mappings 1717
1.14.4. Verify the integration 1723

1. Integrate

You can extend the services available to your Service Manager installation by setting up integration with other products. Out-of-the-box, Service Manager allows you to integrate the following products without needing any additional integration software.

OpenText integrations

You can integrate Service Manager with the following OpenText products.

- Application Lifecycle Management (ALM)
- ALM Octane
- Application Performance Management (APM)
- Operations Bridge Manager (OBM)
- Operations Orchestration (OO)
- Project and Portfolio Management Center (PPM)
- Service Management Automation X (SMAX)
- UCMDB

Third-party integrations

You can integrate Service Manager with the following third-party products.

- Atlassian Jira
- SAP Solution Manager
- Microsoft Teams
- Third-party survey tools

You can also integrate data from other sources using tools such as Connect-It, SCAuto, Web Services API, and Case Exchange. Contact the software support for information or instructions on integrating data from sources outside of Service Manager.

Related topics

Integration Manager
UCMDB
Operations Orchestration (OO)
Application Performance Management (APM)
Computer Telephony Integration (CTI) with the Web client
Web Services
Dynamic Data Exchange (DDE)
Case Exchange framework

1.1. Service Manager integration methods and tools

In addition to the integrations provided by Service Manager and other OpenText products, you can use several utilities and methodologies to create your own Service Manager integrations.

Method or Tool	Description
Web Services	Allows you to connect to and consume external Web Services and to publish Service Manager tables as Web Services. For example, you might use web services to query external Web Services to validate an email address or a phone number when updating a contact record or automatically perform an Internet search using the brief description of the Service Desk interaction.
Connect-It (CIT)	Provides connectors that synchronize static data elements by simple data copying and by data transformation from source to destination. For example, you could use Connect-It to synchronize employee data from Asset Manager to Service Manager through Web Services.
ServiceCenter Automation (SCAuto)	Consists of a collection of automation products that enable external applications to integrate with Service Manager. For example, you might use SCAuto to open records, notify someone that their record has closed, or manage email between external mail programs.
Event Services	Is a background processor that Service Manager uses to receive information generated from and to send information back to external systems. exchange of dynamic, "event-based" information. For example, you must use Event Services with JavaMail, Connect-It, SCMail, and SCAuto.
Dynamic Data Exchange (DDE)	Provides DDE client support in a Service Manager RAD (Rapid Application Development) application. For example, you might create a script to call Service Manager from a Windows application such as Microsoft® Excel.

Method or Tool	Description
BDM Mapping Management	Allows you to configure BTO Data Modle (BDM) mapping records. BDM is intended to be used as a standard data model for integrations between BTO products, for example, Service Manager and Application Performance Management (APM). In Service Manager, a BDM mapping is a mapping between a Service Manager object (associated with a file in Service Manager) and a BDM object (predefined in BDM). A BDM mapping consists of three parts: field mapping, value mapping, and atom mapping.

Related topics

Field Mapping
Value Mapping
Atom Mapping

1.1.1. Web Services

Service Manager Web Services provide interoperability between software applications running on disparate platforms. By using Simple Object Access Protocol (SOAP) technology to establish a standard messaging framework, web services enable developers to access objects within multiple and customized data fields. Open architecture allows for real-time integration and exposure of objects to external applications.

Key features and benefits:

- Uses open standards and protocols
- Allows reuse of services and components
- Can combine multiple web services
- Uses HTTP/HTTPS to work through common firewall security measures without requiring changes to the firewall filtering rules
- Facilitates open integration to other systems
- Provides a standard framework with a consistent and dynamic application program interface (API) architecture for process-based integration

Related topics

Capability words

1.1.1.1. Service Manager Web Services

Service Manager Web Services provide the ability to communicate and integrate with applications in an open and efficient manner. Web Services provide the ability to use a third-party application inside Service Manager, manipulate Service Manager data inside your custom application, or transfer data among separate Service Manager systems.

1.1.1.1.1. What's a Web Service?

The formal definition (according to www.w3c.org) is that a Web service is a software application identified by a Uniform Resource Identifier (URI), whose interfaces and binding are capable of being defined, described, and discovered by XML artifacts and supports direct interactions with other software applications using XML-based messages via Internet-based protocols.

A Web service is a software system designed to support interoperable application to application interaction over a network. It has an interface described in a machine-processable format (specifically WSDL). Other systems interact with Web Services in a manner prescribed by its description using SOAP messages, typically conveyed using HTTP with an XML serialization in conjunction with other Web-related standards.

There are two major classes of Web Services:

- REST-compliant Web Services: The primary purpose of the service is to manipulate XML representations of Web resources using a uniform set of "stateless" operations.
- Arbitrary Web Services: The service may expose an arbitrary set of operations.

Both these Web Services use URIs to identify resources and use Web protocols (such as HTTP and SOAP) and XML data formats for messaging.

1.1.1.1.1.1. Understand the Service Manager Web Services

Every Web service published by Service Manager is a document-literal service. The documents which are used for the requests and replies are derived from the dbdict definition of a single Service Manager file and published via the fields section of the extaccess record.

Each field in the Service Manager data model must be understood in the context of the business logic for the application that defines the data. Before approaching any Web Services consumption project, it is important to understand the data model that's implemented within the Service Manager instance you are targeting. Because Service Manager allows you to add new fields, change the validation of fields or make fields mandatory, every Service Manager implementation will have a slightly different data model and business logic, and each difference has to be reflected in the published Web Service to ensure successful processing.

Web service definitions are maintained in the Web Service Configuration Utility. In this utility, you can see how file names such as probsummary are aliased to Incident, how fields within files can be exposed for purposes of Web Services; and how they are aliased to more appropriate names. Finally, the Web Service Configuration Utility is where XML schema data types such as dateTime can be applied to individual fields. The default type is string, but Service Manager fields can be mapped to various XML schema types if needed.

1.1.1.1.1.2. Web Services basics

The basic Web Services architecture includes the following:

- Publishing - Publishing a Web Service means enabling a Web Service user (consumer) to locate the service description and instructing the consumer how they should interact with the Web Service.
- Consuming (client) - A client is software that's able to use (consume) a Web Service. A service consumer issues one or more queries to the directory to locate a service and determine how to communicate with that service.
- Service - A collection of EndPoints that provides the servicing of the consumers request.
- EndPoint (port) - An EndPoint indicates a specific location for accessing a Web Service using a specific protocol and data format.

1.1.1.1.1.3. Add or change Web Services

Follow these steps:

1. Modify an existing extaccess record if the Web Service connecting through it needs to get additional information or add a new extaccess record if you want to expose a table to a new Web Service and don't want to interfere with existing Web Service applications.

   Note

   Writing expressions in extaccess: The extaccess tool uses the same file variables as the Document Engine. For example, a file variable that holds the current record is $L.file, and the copy of the record before modifications is $L.file.save.

2. For SOAP, rebuild or build the Service Object by re-running WSDL2JS if Service Manager is the consumer.
3. Modify the calling applications if actions, field names, or object name have changed, and a calling application refers to them.

1.1.1.1.2. Introduction to Web Services in Service Manager

The published out-of-box ITIL®-based processes for Web Services are:

- Service Desk
- Incident Management
- Problem Management
- Knowledge Management
- Configuration Management
- Change Management
- Service Catalog
- Service Level Management
- Request Fulfillment

Note

Table-based Web services are still available in Service Manager when needed. Each Web Service application can have a “different view” of the defined services, but the underneath logical flow is still controlled by same Service Manager applications. To avoid validation failure, make sure all the required fields are always exposed.

To publish a Service Manager Web Service, you create one extaccess record per table that you want to publish in that service. Each Web Service application can have a different view of the defined services but the underneath logical flow is still controlled by same Service Manager applications. To avoid validation failure, make sure all the required fields are always exposed on all extaccess records for the table. To add or modify an extaccess record:

Click **Tailoring** > **Web Services** > **Web Service Configuration**.

For SOAP, you may need the allowwsdlretrieval parameter in the sm.ini to be able to view Service Manager WSDL.

Caution

Changes to a specific extaccess record affect any client that's currently consuming the Web Service created by that record. If you modify this configuration, be sure to test all other applications that consume the same Web Service and address possible issues immediately. To avoid issues stemming from different applications using the same Web Service, create a unique extaccess record for each Web Service application, so that each application has a unique Web Service to consume. A single table can be represented in multiple extaccess records.

1.1.1.1.2.1. Web Services and Service Manager

A Web Service enables one application to access the functionality of another application using SOAP operations (XML-based transactions) or RESTful operations, regardless of differences in their operating system platform, application language, or tool set. Service Manager supports two types of Web Services features:

- Connecting to and consuming external Web Services
- Publishing Service Manager fields and methods as Web Services

For SOAP, the Service Manager server now offers out-of-box functionality to connect to and consume external Web Services. When you connect to an external Web service, Service Manager retrieves the Web Service Description Language (WSDL) for the service. You can then write custom JavaScript to use JavaScript functions generated by Web Services and send and receive messages to the remote Web Services. For example, you might query external Web Services to:

- Validate an email address or a phone number when updating a contact record.
- Automatically fill in the time zone of a contact in a Service Desk interaction based on the location given.
- Automatically perform a search for solutions using the brief description of the Service Desk interaction.

The out-of-box Service Manager includes a bundle of published tables, fields, and display actions collectively known as the Service Manager Web Services. The Service Manager Web Services includes Web Services for all the applications and uses an ITIL-compliant naming convention to refer to the Web Service object. The use of ITIL-compliant service and object names allows Web Services developers to create custom Web Services without needing to be familiar with the Service Manager database layer. To consume Service Manager tables, fields, and display actions, you must grant an operator the SOAP API or RESTful API capability word.

You can use the Service Manager Web Services to integrate applications and automate transactions. For example, you might want to publish a Web Service that enables another application or process to:

- Automatically open, update, escalate, resolve, or close Service Manager incidents.
- Automatically add or update a configuration item.

In addition to the tables, fields, and display actions available though the Service Manager Web Services, you can customize the Web Services available from Service Manager by adding, changing, or removing your own tables, fields, and display actions. When you customize the Web Services, Service Manager creates a new version of the Service Manager Web Services. Afterwards, any custom Web Services clients you create access this new version of the Web Service.

1.1.1.1.2.2. Web Services naming conventions for SOAP

The request and response names use the literal strings of the Action Name and Object Name defined in the extaccess record. The name of the Request and Response methods within Service Manager's Web Services are constructed by combining the Action Name with the Object Name and Request or Response.

Note

These names are case sensitive.

For example, the method to add a new incident to the system is:

Action Name	Object Name	Request	Response
Create	Incident	CreateIncidentRequest	CreateIncidentResponse

If your Object Name for the Incident object starts with a lower case "i" (incident) the request is CreateincidentRequest and the response is CreateincidentResponse.

1.1.1.1.2.3. Web Services security considerations

The Service Manager server requires that each Web Service request provide a valid operator name and password combination. These must be supplied in a standard HTTP Basic Authorization header. The Web Service toolkits universally support this authentication mechanism. Use SSL if you are concerned about the possibility of someone using a network monitoring tool to discover passwords. Basic Authorization by itself doesn't encrypt the password; it simply encodes it using Base 64.

Note

Only ASCII operator names are supported in Service Manager Web Service integrations. When Service Manager is handling an incoming Web Service request, the authorization string is decoded by BASE64Decoder. Service Manager uses the decoded string value to construct a UTF-8 string that's used in the RTE. However, the authorization string is in the header and Service Manager doesn't know the charset or encoding of the underling string value, which is BASE64 encoded. Therefore, if the underlying string value is not UTF-8, Web Service clients will fail to connect to Service Manager. In Service Manager, when fetching an operator from the database, no matter what collation the database uses, the operator finally will get a UTF-8 operator value. However, even if users put the same value in the authorization header, the operator name may differ because of the charset/encoding issue.

In addition to having a valid login, the operator must have the SOAP API or RESTful API capability word to access the Web Services. If the Web Service request doesn't contain valid authorization information, then the server sends a response message containing `401 (Unauthorized)`. If the request is valid, then the server sends a response message containing the results of your Web Services operation. The response message contains only the information the operator is allowed to see. The security settings of the user's profile, Mandanten security settings, and conditions defined in the Document Engine are maintained by all Web Services.

1.1.1.1.2.4. Valid URLs for Service Manager

The Service Manager publishes two different URLs for SOAP:

http://<server>:<port>/SM/7

This URL contains similar functionality as sc62server/PWS, except that it uses MTOM attachments.

http://<server>:<port>/sc62server/PWS

This URL provides complete functionality and despite the name sc62server, it is a fully implemented Service Manager7 Web Services interface using MIME attachments.

The Service Manager also publishes below URLs for REST:

http://<server>:<port>/SM/9/rest

1.1.1.1.2.5. Service Manager Web Services URLs

Service Manager support the Web Services at both URLs for SOAP API. If you already use the SC62 server , continue to use it. If you are starting to create a new Web service, use the SM/7 server. You can continue to use the methods, which are still applicable other than the following.

- Any new objects added to Service Manager 9.61, such as the new required fields in Incident Management, will not be available to existing Web Services.
- If you have an SOA broker application, BPEL orchestration engine, or Web Services middleware application configured between the deployed SOAP client application and ServiceCenter or Service Manager application. If so, the orchestration scenario or middleware can be modified to work as a mediator between the old and the new version of the IncidentManagement WSDL.
- MIME – If you use the legacy Web Services URL, then the server uses MIME to encode attachments.
- MTOM/XOP – If you use the Service Manager Web Services URL, then the server uses MTOM/XOP to encode attachments.

Service Manager also supports one URL for RESTful API.

1.1.1.1.2.6. Configure the WSDL field definitions

Use the Web Service Configuration Utility to define the fields that will be passed from Service Manager to the Web Service. The Service Manager fields are taken directly from the database dictionary.

Field	Description
Service Name	The name of the Web Service you want to use to publish theService Manager table. You can reuse the same Web Service name to publish multiple tables. Since this name becomes part of a URL, the name must consist of alphanumeric characters valid for URLs. The name can't consist of URL reserved characters such as spaces, slashes, or colons.
Released	You should consider any web service with the Released option selected as the supported version of the Web Service in Service Manager. While it is possible to clear the Released option and edit or delete the Web Service, we recommend that you assign the service a different name and work on that copy of the web service instead. When the Released option is selected, the external access definition remains read-only.
Name	The name of the Service Manager table that will be published as a Web Service.
Deprecated	Web Services marked as deprecated aren't supported.
Object Name	The name you want to use to identify the Service Manager table in the Web service. Since this name becomes part of the WSDL, the name must consist of alphanumeric characters valid for XML. The name can't consist of XML reserved characters such as brackets (<) and (>), colons (:), or quotation marks (").

Allowed Actions tab field definitions

Use this tab to enter the Service Manager Document Engine display actions you want to globally enable for this table.

Field	Description
Allowed Actions	Click to see the list of allowable display actions for the Service Manager table you have selected for the Web Service.
Action Names	The name used to identify the display action in the Web service as an operation. Since this name becomes part of the WSDL, the name must consist of alphanumeric characters valid for XML. The name can't consist of XML reserved characters such as brackets (<) and (>), colons (:), or quotation marks (").
Action Type	The type for each of the Document Engine display actions that are defined for this table. Click the drop-down icon to see a list of valid type values. • Create only actions will only create new records. • Update only actions will only update existing records. • Merge actions will update the record if it exists and create it if it doesn't exist. • Application Pass Through actions will perform custom actions defined in External Access Actions.
Custom Action To...	Create a custom action for the Service Manager table you have selected for the Web Service.

Expressions tab field definitions

Use this tab to enter system language expressions that run before the display action that runs as part of the Web Service.

Field	Description
Expressions	Call a custom action created in External Access Actions.

Fields tab definitions

Use this tab to set the fields, captions, and field types.

Field	Description
Field	The Service Manager field name that's published by the Web Services Configuration Utility.
Caption	The name that Service Manager displays for the associated Field in the Web Service.
Type	The data type that the Web Services API will convert field data to for Web Services access.

RESTful tab field definitions

Use this tab to enter the RESTful API related configurations.

Field	Description
RESTful Enabled	If it is selected, RESTful API is available for this service. If it's not selected, RESTful API is unavailable for this service.
Attachment Enabled	If it is selected, attachment is supported by RESTFful API for this service. If it's not selected, attachment is unsupported by RESTful API for this service.
Resource Collection Name	This is the name of the Resource Collection. For example, you may specify the group of incidents from the probsummary table as "incidents".
Resource Name	This is the name of the individual Resource. For example, you may specify that any individual incident from the probsummary table be referred to as an "Incident".
Unique Keys	This field specifies one or more fields that will function as a unique identifier for a Resource from the Resource Collection. For example, {ID} in single resource query URI, http://<server>:<port>/SM/9/rest/incidents/{ID}
Max Records Returned in Query	This is the max number of records returned in every single query when the number of records is huge. By default, the value is 1000.
Query Authorization	This is the query privilege for this service.
Resource Collection Action	This field represents the default action for resource collection.
Resource Actions	This field specifies the action to take when an individual resource is part of a POST, PUT, DELETE command. These actions are specified in the **Allowed Actions** tab of the **External Access Definition**.

1.1.1.2. Publish Service Manager data using WS API

To publish Service Manager data via Web Services, use the Web Service Configuration tool to expose files and methods to add, update, or delete Service Manager records. The consumer of this data can be a custom C# or Java program or an interface program such as Connect-It as well as another Service Manager system.

To expose a set of Service Manager tables as a Web Service, click **Tailoring > Web Services > External Access Actions** and create or update the related extaccess record for each of the tables.

1.1.1.2.1. Things to consider prior to publishing data

Before publishing Service Manager data via a Web Service, there are several things to consider. When investigated thoroughly, each of the following items will serve to improve the organization and performance of the Web Services.

1.1.1.2.1.1. Publishing Service Manager applications as Web Services

You can publish Service Manager applications as Web Services and create new integration points between the Service Manager server and external applications.

You can customize the Web Services that Service Manager publishes by adding or removing tables, fields, and display options, from the list of objects available to the Web Services. In addition, you can create alias names for each of these options that only appear in the Web Services but we recommend that you don't do this. You can also specify the XML schema data type you want the Service Manager server to use when publishing data to a SOAP Web service.

For your custom Web Services client to access Service Manager Web Services, it must present a valid operator record name and password with each request. Furthermore, the operator must have the SOAP API or RESTful API capability word as part of the security profile.

1.1.1.2.1.2. When to use Web Services

Web Services enable user-driven integrations with any application that supports Web Services.

- Web Services can be used for any table and external applications that support the technology.
- Web Services can be used to view data from an external source or copy data from one system to another.

1.1.1.2.1.3. Can I use the out-of-box Web Services?

The ITIL-standard Web Services provided with Service Manager should be used whenever possible. They have been tested extensively and are well documented, which makes them easier to use. If you are interested in using one of these Web Services, we recommend that you don't modify the out-of-box extaccess records. Instead, always create your own copy if you need to add actions or fields. If the changes are unique, create a copy of the extaccess record(s) involved first and name the service differently; for example, IncidentManagementForPortal rather than just IncidentManagement, and make your changes against the new set of extaccess records.

1.1.1.2.1.4. What items do I need to expose?

Only expose required fields and fields that are necessary for the actions exposed. Expose only those actions that are required for the consumer to perform their duty. All actions that are exposed need to be able to run in background without user interaction.

Though an entire table and even an entire system can be exposed via Web Services, doing so would affect performance and confuse users. Only the data that's needed by a client should be exposed. This prevents excess traffic and decreases the amount of storage that your client may need to use.

Publish a Document Engine display action in the Web Services API

You must have the SysAdmin capability words to use this procedure.

The Service Manager Web Services API allows you to publish any Document Engine display action as part of a Web Service.

1. Sign on to Service Manager as a System Administrator.
2. Click **Tailoring** > **Web Services** > **Web Service Configuration**. Service Manager displays the External Access Definition form.
3. In the Name field, type the name of the Service Manager table or join file whose display actions you want to publish.
4. Click **Search**. The External Access Definition record for the table opens.
5. Double-click the applicable object name entry. The External Access Definition form opens.
6. Click an empty cell from the Allowed Actions array and select the Document Engine display action you want to publish from the list.

   Note

   If a join file is chosen, the allowed actions for the join file are controlled by the initialization expressions in the `ext.init` Process record (click **Tailoring** > **Document Engine** > **Processes**).

   Note

   You must first have created the necessary Document Engine records, states, objects, and display actions, for a custom display action to appear in this list.

7. In the Action Names field next to the allowed action, type the name you want Service Manager to display for the action in the Web Services API.

> **Note**
>
> The name you type for this field becomes the alias name for the display action and becomes part of the Web Service WSDL. For example, if you type Create for the add action of the Incident object, then the WSDL operation becomes CreateIncident and the WSDL messages are CreateIncidentRequest and CreateIncidentResponse.

> **Caution**
>
> Since this name becomes part of the WSDL, the name must consist of alphanumeric characters valid for XML. The name can't include XML-reserved characters such as brackets (< & >), colons (:), or quotation marks (" & ").

8. In the Action Type field, select the conditions where this action will be valid.
   - To limit the action to new records, select the Create only type.
   - To limit the action to existing records, select the Update only type.
   - To make the action available for both new and existing records, select the Merge type.
9. Click **Save**.

Publish a Service Manager field in the Web Services API

You must have the SysAdmin capability words to use this procedure.

1. Sign in to Service Manager as a System Administrator.
2. Click **Tailoring** > **Web Services** > **Web Service Configuration**. Service Manager displays the External Access Definition form.
3. In the Service Name field, select the name of the Service Manager table or join file in which you want to rename fields.

> **Note**
>
> If a join file is chosen, the Fields tab lists all of the fields for all of the files in the join file.

4. Click **Search**. The web services record for that table opens.
5. Click the **Fields** tab.
6. In the Field field, type the name of the field for which you want to create an alias.

> **Note**
>
> To specify a compound field type such as an array of structure or an array of characters, you must use a special syntax.

7. In the Caption field, type the name (alias) you want the field to have in the Web Services API.

> **Caution**
>
> Since this name becomes part of the WSDL the name must consist of alphanumeric characters valid for XML. The name can't consist of XML reserved characters such as brackets (< & >), colons (:), or quotation marks (“ & ”).

8. In the Type column, select a data type override, if any, you want the field to have in the Web Services API.

> **Note**
>
> If you leave the Type field blank, Service Manager uses the default mapping to determine the data type. Any data type you select overrides the default mapping. The default is StringType.

9. Click **Save**.

1.1.1.2.1.5. What data types should I use for SOAP?

Service Manager has a more lenient data typing policy than the XML schema data typing policy used for Web Services. Certain field types in Service Manager can correspond to multiple data types in the XML schema data type policy. For example, the Service Manager data type decimal could be a decimal, a floating number, or an integer in the XML schema data type policy.

In addition, the actual formatting of data varies between Service Manager and XML schema data types. This is especially true of Service Manager date/time fields that use a different order than XML schema dates. Because some Web Services may require changes to field data format, you can now define the XML Schema data type you want Service Manager to convert the field's data to when you publish the field as part of a web service.

For outbound data, the Service Manager server automatically converts Service Manager data to the format you select in the data policy record for the Service Manager field. For inbound data, the Service Manager server automatically converts the XML schema data to the Service Manager field's listed data type format.

The services, objects, and fields published in the Service Manager Web Services API already have the proper XML schema data mappings listed in the Web Services definition (extaccess record). If the extaccess record doesn't list a data type mapping, then the Web Services API treats the field data as a string.

The following table lists the available SOAP API data types and their Service Manager equivalents.

SOAP API Data Type	Service Manager Data Type
Base64Type	used for binary data
BooleanType	Boolean
ByteType	Decimal
DateTimeType	Date/Time
DateType	Date/Time
TimeType	Date/Time
DurationType	Date/Time
DecimalType	Decimal
DoubleType	Decimal
IntType	Decimal
LongType	Decimal
ShortType	Decimal
FloatType	Decimal
StringType	Text

Caution

- Always map Service Manager date/time fields to the XML schema dateTime or to one of the related XML schema date or time types.
- Do not map a Service Manager logic field which is an expression or evaluate type like `'null(assigned.to in $L.file)'` to any SOAP API data type in the Web Services definition.

Otherwise, these fields will cause errors when you consume the service.

Caution

When integrating with Service Manager, array data should be broken into multi-elements by separator "\r." This is because Service Manager uses "\r" as the separator between array elements. When a string that contains "\r" is retrieved from the Service Manager system, it is decoded as an array with multiple elements separated by "\r". For this reason, when integrating other applications (for example, UCMDB) with Service Manager through web services, array data should be broken into multi-elements by separator "\r" before the data is encoded and sent to the Service Manager system. For example, if an array contains elements "aabb" and "ccdd", it should be sent to Service Manager as the following:

```
<ns:Comments type=\"Array\">
<ns:Comments mandatory=\"\" readonly=\"\">aabb</ns:Comments>
<ns:Comments mandatory=\"\" readonly=\"\">ccdd</ns:Comments>
</ns:Comments>
```

You can define the data type you want Service Manager to convert field data to when publishing it as a Web Service. These data types are consistent with XML schema data types.

1. Click **Tailoring > Database Manager**.
2. In the Table field, type **extaccess** and click **Search**. The External Access Definition record opens.
3. In the Name field, select the name of the Service Manager table whose exposed field you want to define datatypes for.
4. Click **Search**. The External Access Definition record for the table opens.
5. On the Fields tab, find the field that you want to define the data type for.
6. In the Type column for that field, either type the data type or select a data type from the predefined list in the drop-down list.

   Note

   The data type you select for this field becomes an XML schema data type in the web services WSDL.

   Caution

   You must also specify a Field name in API value when you set a data type value. Data type validation depends upon the existence of an alias name.

7. Click **Save**.

1.1.1.2.1.6. What methods do I need?

By default, any operation that's a part of the Document Engine for a table can be made available in the table's Web service. If you need additional methods, add them to the Document Engine first so that Service Manager has a process to follow when performing them. If you have methods in the Document Engine that you don't want exposed, delete them from the allowed actions array in the extaccess table.

Note

All actions performed from Web Services have to run without user interaction in Service Manager. It's not possible to prompt the user for more information when that user is a Web Services consumer.

1.1.1.2.1.7. Manage records with Web Services requests

An implementer can send a Web Services request to Service Manager that will create a new record, update an existing record, or merge two records. These actions are defined by selecting a value in the Action Type field on the Allowed Actions tab of the extaccess record. The following is a description of the expected behavior for each of the values in the drop-down list.

Create only

The server uses Create Semantics to initialize the file variable, fill it with the data from the Web Services request, and pass it to the se.external.action RAD application.

Update only

The server uses Update Semantics to select the matching record before calling the se.external.action RAD application. The server returns an error if it doesn't find a matching record.

Merge

The server attempts to select the record. If it finds the record, it changes the action to Update and calls the se.external.action RAD application. If the server fails to find the record, it changes the action to Create and calls the se.external.action RAD application. If either the Update or Create action is missing, the se.external.action returns a 70 – invalid action error message.

If there is no value specified in the Action Type field, the server uses Update Semantics. The only exception is when the Action Name specified is Create, in which case the server uses Create Semantics.

1.1.1.2.1.8. Are there any security considerations?

After you have exposed data by using Web Services, any client consuming the Web Service you are publishing has access to that data. If there are certain fields that you want to restrict from specific clients, create a different Web Service with those fields removed and have these clients consume that data.

1.1.1.2.1.9. What are released Web Services?

The Web Services delivered out-of-box with Service Manager are read-only and marked with the released option in the external access definition form. You should consider any Web Service with the released option selected as the supported version of the Web Service in Service Manager. While it is possible to clear the released option and edit or delete the Web Service, we recommend that you instead work on a copy of the Web Service that you give it a different name. While the released option is selected the external access definition remains read-only.

1.1.1.2.1.10. Enable SSL encryption for published Web Services

If you want external Web Services clients to use an SSL connection with the Service Manager server, you must provide them with the CA certificate for the Service Manager server. If you purchased a server certificate, copy the CA certificate from the CA certificate keystore provided with your purchased certificate. If you generated your own server certificate by using a self-signed private CA certificate, copy the CA certificate from your private CA certificate keystore instead.

Note

We recommend that you don't use the Service Manager sample server CA certificate because the sample certificate uses a common name (CN) for the server which will not match your actual server name. The best practice is to purchase or create a valid certificate for the Service Manager server in order to establish an SSL-encrypted connection with external web service clients.

1. Copy the keystore that contains the CA certificate that signed your server's certificate and send it to the systems running the external Web Services clients. Out-of-box, Service Manager uses a sample CA certificates keystore as part of the Web tier.

   Note

   We recommend using a CA certificate that you created or purchased instead of the default Service Manager CA certificate.

2. Import the CA certificate of the Service Manager system into the CA certificate keystore of the external Web Services client. You may use a tool like keytool to import the Service Manager CA certificate.
3. Configure the external Web Services client to use the updated CA certificate keystore. Follow the instructions for your Web Services client to set the path to the CA certificate keystore.
4. Update the endpoint URL that the external Web Services client uses to include the HTTPS protocol.

   For example, `https://myserver.mydomain.com:13443/SM/7/ws` for SOAP and `https://myserver.mydomain.com:13443/SM/9/rest` for RESTful.

   Follow the instructions for your Web Service client to update the endpoint URL.

Note

The endpoint URL must use the Service Manager server's common name (CN) as defined in the server certificate. For example, if the server certificate uses the name myserver.mydomain.com, then the endpoint URL must also use the name myserver.mydomain.com.

Note

If you want external Web Services clients to download the Service Manager Web Services WSDL, point them to a URL using the following format:

https://myserver.mydomain.com:13443/SM/7/<Service Name>.wsdl

1.1.1.2.2. Example: Publishing request processes for the PPM integration

In this example, we prepare the data from the ocmq file to integrate to Project and Portfolio Management (PPM) by using Web Services. We will assume that as part of a project, a new employee needs to be hired and the hiring process (approvals and workflow) will be done within the Service Manager Request Management Module.

Since Request Management is not published as a Web Service, we will need to start by creating some customized Display options and Processes. Once these work in the Windows client, we will include them in the newly-created extaccess record.

Since the goal is to publish just the possibility to start the new hire process, a lot of the required information will be hard-coded in the request creation, to minimize overhead.

1.1.1.2.2.1. Create the display option

To create the display option, follow these steps:

1. Sign in to Service Manager as a System Administrator.
2. Click **Tailoring** > **Database Manager** and open the `displayoption` table.
3. Search for an available display option for the `rmq.main.display` display screen in the range from 200 to 2000.
4. Create a new display option with the following values:

Field	Value
Screen ID	`rmq.main.display`
Action	`CreateNewHire`
Unique ID	`rmq.main.display_CreateNewHire`
GUI option	`500`
Text Option	`500`
Default Label	`Create New Hire Request`
Bank	`3`
Condition	`true`

5. Add the new record.

Field	Value
Screen ID	rmq.main.display
Action	CreateNewHire
Unique ID	rmq.main.display_CreateNewHire
GUI option	500
Text Option	500
Default Label	Create New Hire Request
Bank	3
Condition	true

6. Add the new record.

1.1.1.2.2.2. Set up the Request Management category

For the background processing to work correctly, the category has to have `Assign Number Before Commit` selected, so the flag is set to true.

Note

In this example, set the hr category flag to `true`.

Field	Value
Name	hr
Description	Human Resources
Availability	true
Assign Number Before Commit?	Select this option
Phases - Phase Name	Condition
Initial Quote	true
Quote Approval	true
Working	true
Customer follow-up	true

1.1.1.2.2.3. Create the new Process

The new `rmq.open.newhire` Process will first prepare the `ocmcowork` record, and then open the new quote with a single line item for the New Employee bundle.

The new Process will need the following Initial JavaScript:

```
system.vars.$L_work=new SCFile("ocmcowork");
```

On the RAD tab, enter the following information:

Expressions before 1st RAD:

```
$L.part.no={100};$L.quantities={1};$L.item.quantity=1
```

1st RAD:

```
–      svcCat.build.work.file.sub      –       Condition: true

–      names                           –        $L.part.no
–      record                          –        $L.work
–      numbers                         –        $L.quantities
–      number1                         –        $L.item.quantity
```

Expressions before 2nd RAD:

```
if (filename($L.file)="ocmq") then ($fileq=$L.file)
```

2nd RAD:

```
–       rmq.open         –       Condition: true
–       file             –       $L.file
–       second.file      –       $L.object
–       text             –       $L.exit
–       boolean1         –       true
–       cond.input       –       false
–       record           –       $L.work
```

1.1.1.2.2.4. Set up the State record

The rmq.view State record has to link the new display option to the new Process record. Add the following line:

```
– CreateNewHire – rmq.open.newhire – true –false
```

1.1.1.2.2.5. Update the format control record

Disable the `ocml.bld.smry` subroutine call on the ocmq format control record that runs on display.

1.1.1.2.2.6. Set up the extaccess record

In the default Service Manager system, Request Management tables aren't published as a Web Service.



Note

Enter all fields that need to be exposed, and then create field captions for those fields. The field captions can't be XML-reserved characters and can't contain spaces. Use of mixed case is supported.

To create a new Web Service, do the following:

1. Sign in to Service Manager as a System Administrator.
2. Click **Tailoring** > **Web Services** > **Web Service Configuration**.
3. To publish Request Management Quotes, create a new record with the following information:

Field	Value
Service Name	RequestManagement **Note** Type the name of the web service that you want to use to publish this table may be comprised of multiple Service Manager tables. The name you type in this field becomes the alias name for the service and it becomes part of the web service URL. For example, when you type RequestManagement as the service name, then the WSDL you publish will be called RequestManagement.wsdl. The name can't contain URL-reserved characters, such as spaces, slashes, or colons.
Name	ocmq

Field	Value
Object Name	Quote **Note** Type the name you want to use to identify the table. This name becomes the alias name for the table, and then becomes part of the web service WSDL. For example, when you type **Quote** as the object name, then the SOAP operations for this table include Quote as part of the WSDL element, such as UpdateQuote, CreateQuote, and DeleteQuote. The name can't consist of XML-reserved characters, such as brackets (< and >), colons (:), or quotation marks (" and '). Never use "CamelCase" (mixed case) notation in the Object name, as this creates an incorrect or missing filename when calling the web service via Service Manager. As a workaround, you can use a tool that lets you modify the XML to include the filename in the SOAP body request. However, Service Manager and some other tools don't allow modifications.

Field	Value
Allowed Actions	The Allowed Actions have to match the action field in the Display Option, and in the display action field in the State record. Only options that have a true condition will be available through the web service interface. Operator privileges will be checked to ensure security.
1st entry	save
2nd entry	GenNewHire
Action Names	Type the name you want to use in the Web Services application program interface (API) to identify the Document Engine display actions for this table. The name you type for this field becomes the alias name for the display action, and then becomes part of the web service WSDL. The only action that can be used to add a record to a Service Manager table is Create. Updating actions can be named to fit the action. Foe example, if you type Create for the add action of the Quote object, then the WSDL operation becomes CreateQuote and the WSDL message is CreateQuoteRequest. The name can't consisit of XML-reserved characters, such as brackets(< and >), colons (:), or quotation marks (" and ').
1st entry	Update
2nd entry	Create
Action Type	
1st entry	Merge

Field	Value
2nd entry	Create only
Expressions tab	if null(number in $L.file) then ($L.mode="add")

4. On the Fields tab, type the following:

Field	Caption	Type
priority	Priority	StringType
requested.for	Requestor	StringType
requestor.dept	RequestingDepartment	StringType
reason	Reason	StringType
location	Location	StringType
hire.type	HireType	StringType
requested.date	StartDate	DateTimeType
requestor.fname	NewEmployeeFirstName	StringType
requestor.lname	NewEmployeeLastName	StringType
category	Category	StringType
current.phase	Phase	StringType
number	Number	StringType

5. If you want to use RESTful API, on the **RESTful** tab, type the following:

Field	Value
RESTful Enabled	Checked
Attachment Enabled	Checked if you want to use attachment later.
Resource Collection Name	quotes
Resource Name	Quote
Unique Keys	number
Max Records Returned in Query	1000
Query Authorization	true
Resource Collection Actions: POST	Create
Resource Actions:POST	Update
Resource Actions: PUT	Update

When your Web service is set up, it is ready to be consumed by a custom client. Windows and Web clients are unaffected by changes you make to the extaccess table. The operator's application profile is used to determine which tables the user can access, and which actions the user can perform.

1.1.1.2.3. List: Web Services available in the Service Manager Web Services API

The Service Manager Web Services includes ITIL-compliant Web Services. The following table lists some of those web services. To see all the Web Services that are ITIL-compliant, use Web Service Configuration in Tailoring (**Tailoring** > **Web Services** > **Web Service Configuration**) and then do a true search. This will list all of the out-of-box services.



Note

This is the out-of-box list for SOAP.

Web Service	URL to access WSDL	Service Manager objects (tables) published
Change Management	ChangeManagement.wsdl	Change (cm3r), ChangeTask (cm3t)
Configuration Management	ConfigurationManagement.wsdl	Company (company), Contact (contacts), Department (dept), Device (device), DeviceParent (deviceparent), Computer (joincomputer), DisplayDevice (joindisplaydevice), Furnishing (joinfurnishings), HandHeldDevice (joinhandhelds), MainFrame (joinmainframe), NetworkDevice (joinnetworkcomponents), OfficeElectronic (joinofficeelectronics), SoftwareLicense (joinsoftwarelicense), StorageDevice (joinstorage), TelecommunicationDevice (jointelecom), Location (location), Model (model), InstalledSoftware (pcsoftware), Vendor (vendor)
Incident Management	IncidentManagement.wsdl	Incident (probsummary), Incident Task (imTask)
Problem Management	ProblemManagement.wsdl	Problem (rootcause)

Web Service	URL to access WSDL	Service Manager objects (tables) published
Service Desk	ServiceDesk.wsdl	Interaction (incidents) **Note** As of Service Manager 9.41, the "Continue" action is introduced and it only applies to the standalone streamlined interactions. If you apply "Continue" to a streamlined interaction that's already related to a fulfillment record, the action will fail with an error message. The “add.escalate.incident" action supports both the “Streamlined Service Desk” workflow and other previous interaction workflows.

Web Service	URL to access WSDL	Service Manager objects (tables) published
Service Level Management	ServiceLevelManagement.wsdl	ServiceEntry (serviceent), SLA (sla), ActiveSLA (slaactive), AssignedSLA (slaassigned), SLA Control (slacontrol), MonthlySLA (slamonthly), MonthlySLALag (slamonthlylag), SLAResponse (slaresponse)
Request Fulfillment	RequestManagement.wsdl	Request (request), Request Task (requestTask)

For RESTful, you can find the out-of-box list from the Service Document, `http://<server>:<port>/SM/9/rest`.

1.1.1.2.4. Field names in the extaccess record

Implementers can change the field name and data type of a Service Manager field when they publish the field as part of a Web Service. To change the field name and data type of a Service Manager field, the implementer must specify the Service Manager field in the extaccess record using one of the formats listed in the following table.

Type of Service Manager field	Format required to specify field	Example field listing fromthe Web Services API
All primitive fields	field.name	initial.impact
array **field**	field.name	misc.array1
structure **field 1** **field 2** **field 3**	structure.name,field.name.1 structure.name,field.name.2 structure.name,field.name.3	header,agreement.id header,approval.status header,assigned.to
array structure **field 1** **field 2** **field 3**	array.name[field.name.1] array.name[field.name.2] array.name[field.name.3]	affected.ci[ci.assign.group] affected.ci[ci.device.name] affected.ci[ci.device.type]
structure1 structure2 **field 1** **field 2** **field 3**	structure.name.1,structure.name.2,field.name.1 structure.name.1,structure.name.2,field.name.2 structure.name.1,structure.name.2,field.name.3	*<no example available>*

Type of Service Manager field	Format required to specify field	Example field listing fromthe Web Services API
structure 1 array structure 2 **field 1** **field 2** **field 3**	structure.name.1,array.name[field.name.1] structure.name.1,array.name[field.name.2] structure.name.1,array.name[field.name.3]	*<no example available>*

1.1.1.2.5. Create dedicated Web Services listeners

A Service Manager system configured for vertical or horizontal scaling uses a Load Balancer to redirect client connection requests to an available Service Manager process. A system that also has many Web Services may need a Load Balancer for multiple nodes. Service Manager's Web Services don't support HTTP redirect, and will fail to clean up the resources on the Service Manager loadBalancer process, if the loadBalancer port is used as the endpoint URL. For this reason, we recommend creating one or more Service Manager processes dedicated to Web Services requests. You can then configure any external Web service clients to connect directly to the dedicated Service Manager processes. If your system needs a load balancer, use a hardware load balancer to balance between a set of servlets with the `debugnode` parameter.

1. Sign in to the host running Service Manager with an administrator account.
2. Stop the Service Manager server.

   Note

   It's not necessary to stop and start the Service Manager server to add a new port. You can add the line to the `sm.cfg` file while the system is running and start that same port from a command prompt manually.

3. Open the sm.cfg file, and create a dedicated Service Manager process to listen for Web Services requests using the `-debugnode` parameter. For example, the following entries create a dedicated process listening on ports 13085 and 13445.

   ```
   sm -httpPort:13080 -loadbalancer
   sm -httpPort:13081 -httpsPort:13443
   sm -httpPort:13083 -httpsPort:13444
   sm -httpPort:13085 -httpsPort:13445 -debugnode
   ```

   Note

   The `debugnode` parameter tells the Service Manager Load Balancer not to forward any client connection requests to this Service Manager process. Only clients that directly connect to the process can access it.

4. Restart the Service Manager server.
5. Configure any external web service clients to connect directly to the Service Manager processes running in debugnode. For example, set the endpoint URL to `http://<fully qualified host name>:13085/SM/7/<Service Name>` for normal connections and set the

URL to `https://<fully qualified host name>:13445/SM/7/<Service Name>` for SSL-encrypted connections.

1.1.1.2.6. Data conversion between Service Manager and SOAP Web Services

Service Manager has a more lenient data typing policy than the XML schema data typing policy used for Web Services. Certain field types in Service Manager can correspond to multiple data types in the XML schema data type policy. For example, the Service Manager data type decimal could be a decimal, a floating number, or an integer in the XML schema data type policy.

In addition, the actual formatting of data varies between Service Manager and XML schema data types. This is especially true of Service Manager date/time fields that use a different order than XML schema dates. Because some Web Services may require changes to field data format, you can now define the XML Schema data type you want Service Manager to convert the field's data to when you publish the field as part of a web service.

For outbound data, the Service Manager server automatically converts Service Manager data to the format you select in the data policy record for the Service Manager field. For inbound data, the Service Manager server automatically converts the XML schema data to the Service Manager field's listed data type format.

For example, the Service Manager Web Services API publishes the Service Manager field closed.time as ClosedTime in the IncidentManagement service. The Web Services API converts the outbound Service Manager data into the appropriate ISO 8601 date format for XML schema. When the Web Service responds, the Web Service API converts the ISO-formatted date back into a Service Manager date format. Here is an example: the conversion between 1994-11-05T08:15:30-05:00 and November 5, 1994, 8:15:30 am, US Eastern Standard Time by the Web Services API.

The services, objects, and fields published in the Service Manager Web Services API already have the proper XML schema data mappings listed in the Web Services definition (extaccess record). If the extaccess record doesn't list a data type mapping, then the Web Services API treats the field data as a string field. Typically, you only need to add or change a Web Services API data type mapping to publish custom fields you have added to Service Manager as Web Services objects.



Caution

Changing the Web Services API data type mappings for existing fields in the Service Manager Web Services API may result in data mismatch errors.

1.1.1.2.7. Example: Publish the Terminate Change functionality by using Web Services

In the default Service Manager system, the Terminate Change functionality is not published via Web Services.

To publish the Terminate Change functionality, follow the steps described in this section.

1.1.1.2.7.1. Create a display option

Add a Display Option record to eliminate the prompt for a closure code and closing comments.

Field	Value
Screen ID	cm.view.display
Modifies Record	Leave blank
Action	terminatebg
Unique ID	cm.view.display_terminatebg
GUI option	6
Balloon Help (If Option < 200)	Terminate Change
Text Option	6
Default Label	Terminate Background
Bank	1
Text Alternative	Leave blank
Condition	`evaluate($L.tableAccess.close) and open in $L.file=true and nullsub($G.ess, false)=false and (category in $L.file="Release Management" and ($phasepntr=3 or $phasepntr=2 or $phasepntr=1))`
User Condition	`$G.bg=true`
RAD tab	
PreRad Expressions subtab	`$terminate.release=true`

1.1.1.2.7.2. Create a new process

Enter the following values in the Process Definition record to create the terminate.release.bg Process record.

Field	Value
Process Name	terminate.release.bg
Run in Window?	Select this option

RAD tab

Expressions evaluated before RAD call

```
$L.file.vars={$L.category, $L.phase, $L.fc, $L.fc.master}
if (index(current.phase in $L.file, phases in
$L.category)=lng(denull(phases in $L.category))) then ($L.last=true)
else ($L.last=false)
```

Field	Value
RAD Application	sla.confirm.outage
Condition	$L.last and enable in$G.sla.environment
Parameter Names	file
Parameter Values	$L.file

Expressions evaluated before RAD call

```
$L.terminated.parent.name=number in
$L.file;$terminate.ok=true;$terminate.
release=true
```

Field	Value
RAD Application	cm3.close.child.tasks
Condition	true
Parameter Names	name
Parameter Values	$L.terminated.parent.name

Expressions evaluated before RAD call

```
$phasepnt=7;current.phase in $L.file="Verification"
status in $file="terminated"
```

Field	Value
RAD Application	cm.close
Condition	true
Parameter Names	record second.file boolean.1 prompt
Parameter Values	$L.file $L.object $L.bg $L.exit

1.1.1.2.7.3. Set up a State record

Add the following entries to the cm.view State record.

- Display Action: terminatebg
- Process Name: terminate.release.bg
- Condition: true
- Save First: (leave blank)

1.1.1.2.7.4. Set up an extaccess record

Follow these steps:

1. Update the extaccess record to expose this function. Select the extaccess record with Name = cm3r. Type **TerminateChange** in **Object Name** and click **Add**.
2. Add the following entries to the extaccess record:
    - Allowed Actions: terminatebg
    - Actions: Terminate
    - Action Type(Leave Blank)
3. If you want to use RESTful API, on RESTful tab, type the following:

Field	Value
RESTful Enabled	Checked
Attachment Enabled	Checked if you want to use attachment later.
Resource Collection Name	terminatechanges
Resource Name	TerminateChange
Unique Keys	header,number
Max Records Returned in Query	1,000
Query Authorization	lioption("Change Management") and (index("SysAdmin", $lo.ucapex)>0 or index("ChMAdmin", $lo.ucapex)>0 or index("change request", $lo.ucapex)>0)
Resource Collection Actions: POST	Create
Resource Actions:POST	Update
Resource Actions:PUT	Update

1.1.1.2.7.5. Execute a request via SOAP Web Services

Execute the following request by using SOAP Web Services.

Note

The change number has to be a change of the Release Management category. ClosingComments and ClosureCode are required fields for terminating a Release Management change.

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:pws="http://servicecenter.peregrine.com/PWS"
xmlns:com="http://servicecenter.peregrine.com/PWS/Common">
<soapenv:Header/>
<soapenv:Body>
<pws:TerminateChangeRequest attachmentInfo="?" attachmentData="?"
ignoreEmptyElements="true">
<pws:model query="">
<pws:keys query="">
<!--Optional:-->
<pws:ChangeNumber type="String" mandatory="?"
readonly="?">C10027</pws:ChangeNumber>
</pws:keys>
<pws:instance query="" uniquequery="?" recordid="?">
<pws:header type="Structure">
<!--Optional:-->
<pws:ChangeNumber type="String" mandatory="?"
readonly="?"></pws:ChangeNumber>
<!--Optional:-->
<pws:Category type="String" mandatory="?"
readonly="?"></pws:Category>
<!--Optional:-->
<pws:Status type="String" mandatory="?"
readonly="?">terminated</pws:Status>
<!--Optional:-->
<pws:ApprovalStatus type="String" mandatory="?"
readonly="?"></pws:ApprovalStatus>
<!--Optional:-->
<pws:RequestedBy type="String" mandatory="?"
readonly="?"></pws:RequestedBy>
<!--Optional:-->
<pws:AssignedTo type="String" mandatory="?"
readonly="?"></pws:AssignedTo>
<!--Optional:-->
<pws:Coordinator type="String" mandatory="?"
readonly="?"></pws:Coordinator>
<!--Optional:-->
<pws:CoordinatorPhone type="String" mandatory="?"
readonly="?"></pws:CoordinatorPhone>
<!--Optional:-->
<pws:PlannedStartDate type="DateTime" mandatory="?"
readonly="?"></pws:PlannedStartDate>
<!--Optional:-->
```

```
<pws:PlannedEndDate type="DateTime" mandatory="?"
readonly="?"></pws:PlannedEndDate>
<!--Optional:-->
<pws:Reason type="String" mandatory="?"
readonly="?"></pws:Reason>
<!--Optional:-->
<pws:CurrentPhase type="String" mandatory="?"
readonly="?"></pws:CurrentPhase>
<!--Optional:-->
<pws:RiskAssessment type="String" mandatory="?"
readonly="?"></pws:RiskAssessment>
<!--Optional:-->
<pws:Priority type="String" mandatory="?"
readonly="?"></pws:Priority>
<!--Optional:-->
<pws:DateEntered type="DateTime" mandatory="?"
readonly="?"></pws:DateEntered>
<!--Optional:-->
<pws:Open type="Boolean" mandatory="?"
readonly="?"></pws:Open>
<!--Optional:-->
<pws:BackoutDuration type="Duration" mandatory="?"
readonly="?"></pws:BackoutDuration>
<!--Optional:-->
<pws:CloseTime type="DateTime" mandatory="?"
readonly="?"></pws:CloseTime>
<!--Optional:-->
<pws:ForeignID type="String" mandatory="?"
readonly="?"></pws:ForeignID>
<!--Optional:-->
<pws:RFCType2 type="String" mandatory="?"
readonly="?"></pws:RFCType2>
<!--Optional:-->
<pws:Company type="String" mandatory="?"
readonly="?"></pws:Company>
<!--Optional:-->
<pws:BriefDescription type="String" mandatory="?"
readonly="?"></pws:BriefDescription>
<!--Optional:-->
<pws:Subcategory type="String" mandatory="?"
readonly="?"></pws:Subcategory>
<!--Optional:-->
<pws:SLAAgreementID type="Int" mandatory="?"
readonly="?"></pws:SLAAgreementID>
</pws:header>
```

```
<pws:description.structure type="Structure">
<!--Optional:-->
<pws:Description type="Array">
<!--Zero or more repetitions:-->
<pws:Description type="String" mandatory="?"
readonly="?"></pws:Description>
</pws:Description>
<!--Optional:-->
<pws:Justification type="Array">
<!--Zero or more repetitions:-->
<pws:Justification type="String" mandatory="?"
readonly="?"></pws:Justification>
</pws:Justification>
<!--Optional:-->
<pws:BackoutMethod type="Array">
<!--Zero or more repetitions:-->
<pws:BackoutMethod type="String" mandatory="?"
readonly="?"></pws:BackoutMethod>
</pws:BackoutMethod>
</pws:description.structure>
<pws:middle type="Structure">
<!--Optional:-->
<pws:ConfigurationItem type="String" mandatory="?"
readonly="?"></pws:ConfigurationItem>
<!--Optional:-->
<pws:Location type="String" mandatory="?"
readonly="?"></pws:Location>
<!--Optional:-->
<pws:Misc1 type="String" mandatory="?"
readonly="?"></pws:Misc1>
<!--Optional:-->
<pws:Misc2 type="String" mandatory="?"
readonly="?">pass</pws:Misc2>
<!--Optional:-->
<pws:Misc3 type="String" mandatory="?"
readonly="?"></pws:Misc3>
<!--Optional:-->
<pws:Misc4 type="String" mandatory="?"
readonly="?"></pws:Misc4>
<!--Optional:-->
<pws:Misc5 type="String" mandatory="?"
readonly="?"></pws:Misc5>
<!--Optional:-->
<pws:Misc6 type="String" mandatory="?"
readonly="?"></pws:Misc6>
```

```
<!--Optional:-->
<pws:Misc7 type="String" mandatory="?"
readonly="?"></pws:Misc7>
<!--Optional:-->
<pws:Misc8 type="String" mandatory="?"
readonly="?"></pws:Misc8>
<!--Optional:-->
<pws:Misc9 type="String" mandatory="?"
readonly="?"></pws:Misc9>
<!--Optional:-->
<pws:Misc10 type="String" mandatory="?"
readonly="?"></pws:Misc10>
<!--Optional:-->
<pws:OutageStart type="DateTime" mandatory="?"
readonly="?"></pws:OutageStart>
<!--Optional:-->
<pws:OutageEnd type="DateTime" mandatory="?"
readonly="?"></pws:OutageEnd>
<!--Optional:-->
<pws:ScheduledOutageStart type="DateTime" mandatory="?"
readonly="?"></pws:ScheduledOutageStart>
<!--Optional:-->
<pws:ScheduledOutageEnd type="DateTime" mandatory="?"
readonly="?"></pws:ScheduledOutageEnd>
<!--Optional:-->
<pws:ActualOutageStart type="DateTime" mandatory="?"
readonly="?"></pws:ActualOutageStart>
<!--Optional:-->
<pws:ActualOutageEnd type="DateTime" mandatory="?"
readonly="?"></pws:ActualOutageEnd>
<!--Optional:-->
<pws:MiscArray1 type="Array">
<!--Zero or more repetitions:-->
<pws:MiscArray1 type="String" mandatory="?"
readonly="?"></pws:MiscArray1>
</pws:MiscArray1>
<!--Optional:-->
<pws:MiscArray2 type="Array">
<!--Zero or more repetitions:-->
<pws:MiscArray2 type="String" mandatory="?"
readonly="?"></pws:MiscArray2>
</pws:MiscArray2>
<!--Optional:-->
<pws:MiscArray3 type="Array">
<!--Zero or more repetitions:-->
```

```
<pws:MiscArray3 type="String" mandatory="?"
readonly="?">test passed</pws:MiscArray3>
</pws:MiscArray3>
<!--Optional:-->
<pws:Assets type="Array">
<!--Zero or more repetitions:-->
<pws:Assets type="String" mandatory="?"
readonly="?"></pws:Assets>
</pws:Assets>
<!--Optional:-->
<pws:EstimateDescription type="String" mandatory="?"
readonly="?"></pws:EstimateDescription>
<!--Optional:-->
<pws:EstimatePrice type="String" mandatory="?"
readonly="?"></pws:EstimatePrice>
<!--Optional:-->
<pws:ActualCost type="String" mandatory="?"
readonly="?"></pws:ActualCost>
<!--Optional:-->
<pws:ActualPrice type="String" mandatory="?"
readonly="?"></pws:ActualPrice>
</pws:middle>
<pws:close type="Structure">
<!--Optional:-->
<pws:CompletionCode type="Decimal" mandatory="?"
readonly="?">1</pws:CompletionCode>
<!--Optional:-->
<pws:ClosingComments type="Array">
<!--Zero or more repetitions:-->
<pws:ClosingComments type="String" mandatory="?"
readonly="?">Terminating Change</pws:ClosingComments>
</pws:ClosingComments>
</pws:close>
<!--Optional:-->
<pws:Urgency type="String" mandatory="?"
readonly="?"></pws:Urgency>
<!--Optional:-->
<pws:InitialAssessment type="String" mandatory="?"
readonly="?"></pws:InitialAssessment>
<!--Optional:-->
<pws:attachments>
<!--Zero or more repetitions:-->
<com:attachment
href="Service_Manager:9.61_Codeless/webservicesguide/concepts/?"
contentId="?" action=""
```

```
name="?" type="?" len="?" charset="?" attachmentType="?"/>
</pws:attachments>
</pws:instance>
<!--Optional:-->
<pws:messages>
<!--1 or more repetitions:-->
<com:message type="String" mandatory="?" readonly="?"
severity="?" module="?"></com:message>
</pws:messages>
</pws:model>
</pws:TerminateChangeRequest>
</soapenv:Body>
</soapenv:Envelope>
```

1.1.1.2.7.6. Response to a request via SOAP Web Services

The response to a request by using SOAP Web Services is as follows:

```
<SOAP-ENV:Envelope xmlns:SOAP
-ENV="http://schemas.xmlsoap.org/soap/envelope/">
<SOAP-ENV:Body>
<TerminateChangeResponse message="Success" returnCode="0"
schemaRevisionDate="2008-05-21" schemaRevisionLevel="5"
status="SUCCESS"
xsi:schemaLocation="http://servicecenter.peregrine.com/PWS http://<sm
server>.americas.hpqcorp.net:13701/sc62server/ws/Change.xsd"
xmlns="http://servicecenter.peregrine.com/PWS"
xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<model>
<keys>
<ChangeNumber type="String">C10027</ChangeNumber>
</keys>
<instance recordid="C10027 - test"
uniquequery="header,number=&quot;C10027&quot;">
<header type="Structure">
<ChangeNumber type="String">C10027</ChangeNumber>
<Category type="String">Release Management</Category>
<Status type="String">terminated</Status>
<ApprovalStatus type="String">approved</ApprovalStatus>
<RequestedBy type="String">ALSTON, LOU</RequestedBy>
<Coordinator type="String">CM 3</Coordinator>
<Reason type="String">problem</Reason>
<CurrentPhase type="String">Verification</CurrentPhase>
<Priority type="String">1</Priority>
<DateEntered type="DateTime">2008-05-
27T16:34:26+00:00</DateEntered>
<Open type="Boolean">false</Open>
<BackoutDuration
type="Duration">P0DT0H0M0S</BackoutDuration>
<CloseTime type="DateTime">2008-05-
27T16:34:26+00:00</CloseTime>
<Company type="String">advantage</Company>
<BriefDescription type="String">test</BriefDescription>
</header>
<description.structure type="Structure">
<Description type="Array">
<Description type="String">test</Description>
</Description>
</description.structure>
<middle type="Structure">
```

```
<Location type="String">North America</Location>
<Misc2 type="String">pass</Misc2>
<MiscArray3 type="Array">
<MiscArray3 type="String">test passed</MiscArray3>
</MiscArray3>
</middle>
<close type="Structure">
<CompletionCode type="Decimal">1</CompletionCode>
<ClosingComments type="Array">
<ClosingComments type="String">Terminating
Change</ClosingComments>
</ClosingComments>
</close>
<Urgency type="String">1</Urgency>
<InitialAssessment type="String">1</InitialAssessment>
</instance>
</model>
<messages>
<cmn:message type="String">Audit Record successfully recorded
and added.</cmn:message>
<cmn:message type="String">Change C10027 Phase Verification
Closed by System Administrator.</cmn:message>
</messages>
</TerminateChangeResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>END
```

1.1.1.2.7.7. Execute a request via RESTful Web Services

Execute the following request by using RESTful Web Services.

Note

The change number must be a change of the Release Management category. **ClosingComments** and **ClosureCode** are required fields for terminating a Release Management change.

POST an action terminate to a change which is in **Evaluation & Change Closure** phase. The json data resembles:

```
{ "TerminateChange":    {
"close":{
"ClosingComments":"closureComments tested",
"ClosureCode":"1"
}
}
}
```

1.1.1.2.7.8. Response to a request by using RESTful Web Services

The response to a request by using RESTful Web Services resembles the following:

```
{
"Messages": [],
"ReturnCode": 0,
"TerminateChange":    {
"Impact": "4",
"RequestedEndDate": "2007-10-23T21:06:00+00:00",
"Service": "Applications",
"Urgency": "1",
"close":       {
"ClosingComments": ["closureComments tested"],
"ClosureCode": 1
},
"description.structure": {"Description": ["Check and clean system on
virusses."]},
"header":       {
"ApprovalStatus": "approved",
"AssignmentGroup": "Application",
"BackoutDuration": "P0DT0H0M0S",
"Category": "Maintenance",
"ChangeCoordinator": "Change.Coordinator",
"ChangeID": "C10019",
"Company": "advantage",
"DateEntered": "2013-07-02T09:16:29+00:00",
"InitiatedBy": "BERRY, ELLIS",
"Open": true,
"Phase": "Evaluation & Change Closure",
"PlannedEnd": "2008-10-19T18:00:00+00:00",
"PlannedStart": "2008-10-12T18:00:00+00:00",
"Priority": "2",
"RiskAssessment": "5",
"Status": "initial",
"Subcategory": "Maintenance",
"Title": "Multiple virusses"
},
"middle":       {
"Assets": ["Norton Anti-Virus"],
"ConfigurationItem": "Norton Anti-Virus",
"Location": "North America"
}
}
}
```

1.1.1.2.8. Publish a table as a Web Service

You must have the SysAdmin capability words to use this procedure.

1. Login to Service Manager as a System Administrator.
2. Click **Tailoring** > **Database Manager**.
3. In the Table field, type extaccess.
4. Click **Search**.
5. In the Name field, select the name of the Service Manager table or join file you want to publish as a Web Service.

   Caution

   Type the name of the table as it is defined in the database dictionary.

   Note

   Only valid Service Manager table names appear in the list. This list includes the names of tables that don't physically reside in the database, but are defined in memory at run time based on join definitions and relationship information in joindef and erddef records respectively.

6. In the Service Name field, type the name of the Web Service you want to use to publish this table. You can reuse the same Web Service name to publish multiple tables, as long the combination of Service Name and Object Name is unique.

   Caution

   Since this name becomes part of a URL, the name must consist of alphanumeric characters that are valid for URLs. The name can't consist of URL reserved characters such as spaces, slashes, or colons.

   Note

   The name you type in this field becomes the alias name for service and becomes part of the Web Service URL. For example, if you type IncidentManagement for the service name, then SOAP applications must include IncidentManagement.wsdl in the URL to access this service.

7. In the Object Name field, type the name you want to use to identify this table.

> **Note**
>
> The name is unique and can't be used by other Web Services definitions.

> **Note**
>
> The name you type in this field becomes the alias name for the table and becomes part of the Web Service WSDL. For example, if you type Incident for the object name, then the SOAP operations for this table include Incident as part of the WSDL element (such as RetrieveIncident, CreateIncident, and ResolveIncident).

> **Caution**
>
> Since this name becomes part of the WSDL, the name must consist of alphanumeric characters valid for XML. The name can't consist of XML reserved characters such as brackets (< & >), colons (:), or quotation marks (" & ').

8. In the Allowed Actions array, select the Service Manager Document Engine display actions you want to globally enable for this table.

> **Note**
>
> Each table has its own set of display actions allowed as defined in the Service Manager Document Engine. Enabling or disabling the display actions from this field only determines whether the display action is available through the Web Services API. Service Manager still validates the operator credentials supplied with each Web Service request to ensure that the operator has sufficient privileges to perform the display action. Click the array field to see a list of allowable display actions for the table you select.

> **Note**
>
> If a join file is chosen, the allowed actions for the join file come from the primary table of the join.

9. In the Action Names field, type the name you want to use in the Web Services API to identify the Document Engine display actions for this table.

> **Note**
>
> The name you type for this field becomes the alias name for the display action and becomes part of the Web Service WSDL. For example, if you type Create for the add action of the Incident object, then the WSDL operation becomes CreateIncident and the WSDL messages are CreateIncidentRequest and CreateIncidentResponse.

> **Caution**
>
> Since this name becomes part of the WSDL, the name must consist of alphanumeric characters valid for XML. The name can't consist of XML reserved characters such as brackets (< & >), colons (:), or quotation marks (" & ').

10. If you want to use RESTful API, you need to configure RESTful tab field too. Type the name of Resource Collection and Resource you want to use, and set the Unique Key and default actions.
11. Click **Add**.

Users can now access this Service Manager table from a custom or third-party Web Service client and use the actions you have enabled.

1.1.1.2.9. Expose a table with more than one Web Service

User role: System Administrator

An implementer can define multiple Web Service definition records with different names for a given table or join file, and have different fields and actions exposed for each.

1. Click **Tailoring** > **Web Services** > **Web Service Configuration Utility**.
2. In the Table field, type extaccess, and then click **Search**. The External Access Definition form opens.
3. In the Name field, select or type the name of the table or join file for which you want to create a copy of the extaccess record, and then click **Search**. The record opens.
4. Change the Service Name to the name of the Web Service you want to use to publish the Service Manager table.

   > **Note**
   >
   > The combination of Service Name and Object Name must be unique to this record. The combination can't exist anywhere else in the system.

5. Change the Object Name to the name you want to use to identify the Service Manager table in the Web Services API.
6. On the Fields tab, change the fields that are exposed and modify the Caption and Type information, if necessary.

   > **Note**
   >
   > If a join file is chosen, the Fields tab lists all the fields for all the files in that join file.

7. On the Allowed Actions tab, change the actions, if necessary.
8. On the Expressions tab, add expressions, if necessary.
9. On the RESTful tab, add RESTful API configurations, if necessary.
10. Click **Add**.

The new extaccess record is added to the system. When you view the exposed Object (for example, WSDL for SOAP) for both Web Services, they should display with the applicable actions and fields, as defined in each extaccess record.

1.1.1.2.10. Remove a Document Engine display action from the Web Services API

You must have the **SysAdmin** capability word to use this procedure.

The Service Manager Web Services Configuration Utility allows you to remove any Document Engine display action you published as part of a Web service.

1. Click **Tailoring > Web Services > Web Service Configuration**.
2. In the Service Name field, type the name of the service.
3. In the Name field, type the name of the Service Manager table whose display actions you want to remove.
4. From the Allowed Actions array, select the Document Engine display action you want to remove from the list.
5. Clear the Allowed Actions field with the Backspace key.
6. Click **Save**.

1.1.1.2.11. Remove a Service Manager field from the Web Services API

You must have the **SysAdmin** capability words to use this procedure.

1. Click **Tailoring > Web Services > Web Service Configuration Utility**.
2. In the Table field, type **extaccess** and click **Search**. The External Access Definition record opens.
3. In the Name field, select the name of the Service Manager table in which you want to remove fields.
4. Click **Search**. The Web Services record for that table opens.
5. On the Fields tab, find the fields you want to remove and make the value that's currently there NULL.
6. In the Caption column, make the value NULL for the field you want to remove.
7. In the Type column, make the value NULL for the field you want to remove.
8. Click **Save**.

1.1.1.2.12. Sample client for Web Services SM7 URL

The Service Manager server includes a sample Web Services client application for the http://servername:port_number/SM/7/service_name.wsdl. The sample application was created for Apache™ Axis2 (version 1.7.9). If you have Axis 1.7.9 and Apache™ Ant installed, you can review and update the source code of the sample application as well as generate updated proxy code to test the Service Manager Web Services functionality. The Apache Axis2 sample is written in Java. The sample client application is included with the server installation in the following folder:

<Service Manager server installation folder>\webservices\sample\sm7webservice

The sample includes the source code for the client applications as well as support files for the Web Services development environment. The Apache Axis2 jar files are included and they are located under the "lib" folder. A set of the batch files that you can use to run each class are located under the "bin" folder and you can run each class from the Windows command prompt after you have compiled the sample Java. You can use the sample application as an example of how to create your own custom Web Services client applications.



Note

All the sample applications use a command line interface. To see the usage information for the command line interface, change to "bin" folder, type xxxSample where xxxSample is the batch file name of the sample application.

The Apache Axis2 sample client application assumes that you have a Service Manager server instance running from the local host. If this is not the case, you can change the server host name and port number using the sample's command line interface.

Each of the sample folders includes a readme file that contains valuable information about using the sample application found in each of the sample folder.

The sample client application contains examples of how to send the MTOM attachments to the Service Manager server.

Configuration Management sample

The sample client applications contain the following classes for Configuration Management. Refer to the sample application source code for comments on the usage of each class.

Field	Description
ConfigurationManagementServiceUtility	• Provides the CreateService method to initialize an object for the service. • Provides the InitServiceAuthentication method to send the host name, communications port, operator name, and operator password with each SOAP request.
CreateContactSample	Creates a contact record with the supplied parameters.
DeleteContactSample	Deletes the contact record listed in the supplied parameters.
RetrieveContactSample	Retrieves a single contact record matching the supplied parameters.
UpdateContactSample	Updates a contact record with the supplied parameters.

Incident Management sample

The sample client applications contain the following classes for Incident Management. Refer to the sample application source code for comments on the usage of each class.

Class	Description
CloseIncidentSample	Closes an incident record with the supplied parameters.
CreateIncidentSample	Creates an incident record with the supplied parameters.
IncidentManagementServiceUtility	• Provides the CreateService method to initialize an object for the service. • Provides the InitServiceAuthentication method to send the host name, communications port, operator name, and operator password with each SOAP request.
ResolveIncidentSample	Resolves an incident record matching the supplied parameters.
RetrieveIncidentListSample	Retrieves multiple incident records matching the supplied parameters.
RetrieveIncidentSample	Retrieves a single incident record matching the supplied parameters.
UpdateIncidentSample	Updates an incident record with the supplied parameters.

1.1.1.2.13. Command line arguments for the Axis2 sample application

The Axis2 sample application runs from the command prompt using Java. After you have compiled the Axis2 sample into an executable class files, you can perform configuration and incident management tasks with the following arguments.

Note

To see the usage information for the Axis2 sample application, type ClassName where ClassName is the name of a sample application class.

Configuration Management

The following commands invoke Configuration Management functionality. These examples assume you are using the batch files provided with the Axis2 sample application to automatically set the class path and call the proper executable class.

Operation	Command-line example
Create contact	CreateContactSample -name "FALCON, MERLINE2" -fullname "MERLINE2 FALCON"
Delete contact	DeleteContactSample DeleteContactSample -name "FALCON, MERLINE2"
Retrieve contact	RetrieveContactSample RetrieveContactSample -name "FALCON, MERLINE2"
Update Contact	UpdateContactSample UpdateContactSample -name "FALCON, MERLINE2" -email "fmerline2@Advantage.com"

Incident Management

The following commands invoke Incident Management functionality. These examples assume you are using the batch files provided with the Axis2 sample application to automatically set the class path and call the proper executable class.

Operation	Command-line example
Close incident	CloseIncidentSample -incidentId IM10136 -solution "Developer fix problem" -closeCode "User Closure"
Create incident	CreateIncidentSample -title "WS Sample title" -category incident -area hardware -subArea "hardware failure" -description "sample WS IM desc" -urgency 1 -impact 1 -service Applications -assignmentGroup Application
Create incident with attachment(s)	CreateIncidentSample -title "WS Sample title" -category incident -area hardware -subArea "hardware failure" -description "sample WS IM desc" -urgency 1 -impact 1 -service Applications -assignmentGroup Application -attachment 101.jpg:README.txt
Resolve incident	ResolveIncidentSample -incidentId IM10136 -solution "Developer fix problem"
Retrieve incident list	RetrieveIncidentListSample -incidentId IM10011:IM10021:IM10031
Retrieve incident	RetrieveIncidentSample -incidentId IM10041
Update incident	UpdateIncidentSample -incidentId IM10060 -journalUpdates "this is a update from SM webservice sample"

The CreateIncicentSample and UpdateIncidentSample classes can send MTOM attachments to Service Manager server. The command line argument is -attachment file_01:file_02. You can send more than one attachment to Service Manager server. Be sure to place the attachments in the <SM_installation_directory>\webservices\sample\sm7webservices\Axis2Sample\bin\resources directory.

1.1.1.2.14. Add an external access action to the Web Services

You must have the **SysAdmin** capability words to use this procedure.

1. Click **Tailoring** > **Web Services** > **External Access Actions**. Service Manager displays the External Access Actions form.
2. In **External Action ID**, type a unique ID name.
3. In **RAD/ScriptLibrary.function**, type the name of the RAD or JavaScript function you want to make available as a custom action in the Web Services API.

   Note

   To specify a script from the Script Library, use the following format:

   <script name>.<function name>

   For example, Approval.buildAllStatus.

4. In **Type**, select RAD if your custom action is a RAD function or select JavaScript if your custom action is a JavaScript.
5. In **Description**, type the name you want the custom action to have.

   Note

   Service Manager displays the name you type here as the Custom Action to Perform in the External Access Defintion form.

   The type you select determines what Parameters array Service Manager displays. If you select RAD, Service Manager displays an array with Parameter Names and Parameter Values fields. If you select JavaScript, Service Manager displays an array with only the Parameter Values field.
6. Type any required input parameters of the RAD function or JavaScript in the parameters array.

   RAD functions require values in both the Parameter Names and Parameter Values fields. Each RAD function has its own list of required RAD parameters names. RAD parameter values are typically system variables such as $L.file or$L.exit. You can type RAD function parameters in any order.

   JavaScript parameters only require the Parameter Values field, but require you to type them in the same order as the JavaScript function expects them. For example, the

buildAllStatus function of the Approval script expects the following parameters in the following order:

record
fApprovalDef
keepRoleOld
keepRoleNew
tokens
tokenToDescription

7. Click **Add** to create your custom Web Services action.

1.1.1.3. Web services authentication

Both The RESTful and SOAP API framework support the following authentication methods:

- HTTP Basic Authentication
- CAC (Common Access Card)
- TSO (Trusted Sign On)

Implementing TSO for custom web services clients

Do the following to use TSO authentication for custom web services clients:

- Enable two-way SSL between the web service client and the Service Manager server, so that they trust each other.
- Add parameter **trustedsignon:1** to the integration servlet.
- Create a dedicated integration account for the web service client to log in to the Service Manager server.
- Add a “Pragma: TrustedSignOn” header in the request



Note

- You still need to use the Basic Authentication to pass the username of integration user to SM Server
- When the custom HTTP header “Pragma: TrustedSignOn” is present, the SM Server checks only the existence of the integration user on Service Manager side and skips password checking because the presence of the header indicates the client has been authenticated by an external source such as Active Directory, Kerberos, or a custom Single Sign-On third-party server. This document assumes that you have taken the necessary steps to ensure that your custom web services client has been successfully pre-authenticated before any transactions with the Service Manager web services API begin.

1.1.1.4. SOAP API

This chapter introduces the SOAP API used in Service Manager.

1.1.1.4.1. Web Services Description Language (WSDL)

The W3C describes WSDL in the W3C Note 15 March 2001 as "*WSDL is an XML format for describing network services as a set of endpoints operating on messages containing either document-oriented or procedure-oriented information. The operations and messages are described abstractly, and then bound to a concrete network protocol and message format to define an endpoint. Related concrete endpoints are combined into abstract endpoints (services). WSDL is extensible to allow description of endpoints and their messages regardless of what message formats or network protocols are used to communicate, however, the only bindings described in this document describe how to use WSDL in conjunction with SOAP 1.1, HTTP GET/POST, and MIME*." In other words, the WSDL defines a URL endpoint that publishes objects and methods usable against the publishing application. These objects and methods can then be used to communicate to that application.

1.1.1.4.1.1. Basic operations in WSDL files

Each Web Service that Service Manager publishes has a set of valid operations that an administrator can enable or disable for custom Web Services clients. The list of valid Web Service operations comes from two sources:

- The Document Engine display actions defined for each Service Manager table
- The common operations available to all Web Services

The <Operation Name> is the alias name of the Service Manager display option as defined in the Web Services Configuration Utility. The <Object Name> is the alias name of the Service Manager table as published in the Web Service. Use a Request message to send SOAP operations to the Service Manager server. The Service Manager server uses a Response message to send its reply to the SOAP operation.

You can see the list of available Document Engine display actions for each table in the extaccess table. The Service Manager server converts each published display action into a separate <operation> element in the Web Services Definition Language (WSDL).

For example, the Resolve operation for the Incident object translates to the ResolveIncidentRequest SOAP message. The Service Manager server replies with a ResolveIncidentResponse SOAP message. Any custom Web Services client you create must be able to generate these SOAP message requests and understand the SOAP message response.

In addition to application-specific display actions, there are common operations available to all Service Manager Web Services objects. Just as with display options, the Service Manager server converts each common operation into a separate <operation> element in the Web Services Definition Language (WSDL). The following common messages are always available.

- **Retrieve<Object>Request** – retrieves a single record detail matching the value of the <keys> element or query attribute, for example an Incident record.
- **Retrieve<Object>KeysList** – retrieves a list of keys matching the value of query attribute.
- **Retrieve<Object>List** – retrieves a list of objects matching the value of query attribute.

The following common messages but aren't always available.

- **Update<Object>Request** – updates a single record matching the value of the <keys> element or query attribute with the new values defined in the <instance> element
- **Delete<Object>Request** * – deletes a single record matching the value of the <keys> element
- **Create<Object>Request** – adds a single record with the values defined in the <instance> element

- The IncidentManagement Web Service doesn't offer the delete operation. To retrieve a single incident record, you can use the RetrieveIncident operation.

For more information about Web Services and WSDL, see the W3C Web site.

1.1.1.4.1.2. Service Manager WSDL files

You can view the Web Services Description Language (WSDL) for any Service Manager Web Service by navigating to one of the following URLs:

Version	URL	Supports
Backwards compatibility for ServiceCenter 6.2 servlet mode	http://*<servername>*:*<port_number>*/sc62server/PWS/*<service_name>*.wsdl	MIME attachments
Service Manager	http://*<servername>*:*<port_number>*/SM/7/*<service_name>*.wsdl	MTOM attachments

For example, type http://myserver:13080/SM/7/IncidentManagement.wsdl to view the Incident Management service WSDL from myserver.

The server also responds to requests with `?WSDL` as the file extension. For example, http://myserver:13080/SM/7/IncidentManagement?wsdl

The Service Manager server automatically generates a WSDL whenever it receives an HTTP *get* request for WSDL. Service Manager WSDLs use XML Schema definitions to describe literal Web Services. Service Manager is able to serve two different versions of the WSDL for a given service:

- ServiceCenter 6.2 WSDL files for backwards compatibility. The API described in these WSDL files is deprecated. See the ServiceCenter 6.2 documentation for more information.
- Service Manager WSDL . New applications should use this WSDL.



Note

To avoid receiving the "Invalid XML schema: Element <xs:import> is not allowed at this location under element <xs:schema>" error when viewing any multiple object WSDL (for example, ConfigurationManagement.wsdl), disable the validation in the SOAP tool you are using before loading the WSDL and creating a Web Service request.

The XML document which describes a particular Service Manager record (such as a Change or Incident) is wrapped in an outer document called a "model". The model is nothing more than a container for separating the actual data (the “instance” part) from the "keys" part, which is metadata about the fields that make up the primary key of the object.

1.1.1.4.1.3. Types of Web Services in Service Manager

The types of Web Services supported by Service Manager are as follows:

- Service Manager 9.x URL supporting the W3C Message Transmission Optimization Mechanism (MTOM) attachments, which is a method of efficiently sending binary data to and from Web Services. MTOM is usually used with XML-binary Optimized Packaging (XOP).

  http://<SM Server>:<SM port>/SM/7/<service name>.wsdl

  > **Note**
  >
  > AXIS2 supports MTOM.

- Service Manager 9.x URL supporting Multipurpose Internet Mail Extensions (MIME), which is an Internet standard that extends the format of email to support MIME attachments. MIME's use has grown beyond describing the content of email to describing content type in general, including for the Web.

  http://<SM Server>:<SM port>/sc62server/PWS/<service name>.wsdl

Which URL to use depends on the consumer side. You also need to consider whether it supports MTOM or MIME. For example, Microsoft applications tend to support MIME.

1.1.1.4.1.4. WSDL document structure

A WSDL document is simply a **set of definitions**. There is a **definitions** element at the root and definitions inside. A WSDL document defines **services** as collections of network endpoints or **ports**. In a WSDL document, the abstract definition of endpoints and messages is separated from their concrete network deployment or data format bindings. This allows the reuse of abstract definitions: **messages**, which are abstract descriptions of the data being exchanged, and **port types** which are abstract collections of **operations**. The concrete protocol and data format specifications for a particular port type constitutes a reusable **binding**. A port is defined by associating a network address with a reusable binding. A collection of ports defines a service.

A WSDL document uses the following elements in the definition of network services:

- **Types**: a container for data type definitions using some type system (such as XSD).
- **Message**: an abstract, typed definition of the data being communicated.
- **Operation**: an abstract description of an action supported by the service.
- **Port Type**: an abstract set of operations supported by one or more endpoints.
- **Binding**: a concrete protocol and data format specification for a particular port type.
- **Port**: a single endpoint defined as a combination of a binding and a network address.
- **Service**: a collection of related endpoints.

XML header

The XML header specifies the XML version number, and optionally the character encodings, as part of a grammar document's XML declaration on the first line of the document.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
```

Namespace definitions

XML namespaces are used for providing uniquely named elements and attributes in an XML document

The following section of the Service Manager IncidentManagement wsdl shows the namespace definitions.

```
- <definitions targetNamespace="http://servicecenter.peregrine.com/PWS"
xmlns="http://schemas.xmlsoap.org/wsdl/"
xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common"
xmlns:http="http://schemas.xmlsoap.org/wsdl/http/"
xmlns:mime="http://schemas.xmlsoap.org/wsdl/mime/"
xmlns:ns="http://servicecenter.peregrine.com/PWS"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://schemas.xmlsoap.org/wsdl/
http://schemas.xmlsoap.org/wsdl/">
```

Operation section

The following section of the example Service Manager IncidentManagement wsdl shows the operation section used to define each individual action supported by the service.

```
- <operation name="RetrieveIncident">
<documentation />
<input message="ns:RetrieveIncidentRequest" />
<output message="ns:RetrieveIncidentResponse" />
</operation>
</portType>
```

Messages section

The following section of the example Service Manager IncidentManagement wsdl shows the messages section used to define the data being communicated.

```
- <message name="RetrieveIncidentRequest">
<part element="ns:RetrieveIncidentRequest"
name="RetrieveIncidentRequest" />
</message>
- <message name="RetrieveIncidentResponse">
<part element="ns:RetrieveIncidentResponse"
name="RetrieveIncidentResponse" />
</message>
</message>
```

Types section

The following section of the example Service Manager IncidentManagement wsdl shows the definition of the data, including data types, that's being communicated between the consumer

and Service Manager.

```
- <types>
- <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
attributeFormDefault="unqualified" elementFormDefault="qualified"
targetNamespace="http://servicecenter.peregrine.com/PWS"
version="2007-04-14 Rev 1"
xmlns="http://servicecenter.peregrine.com/PWS"
xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common">
<xs:import namespace="http://servicecenter.peregrine.com/PWS/Common"
schemaLocation="http://server:13080/sc62server/PWS/Common.xsd" />
- <xs:complexType name="IncidentKeysType">
- <xs:sequence>
<xs:element minOccurs="0" name="IncidentID" nillable="true"
type="cmn:StringType" />
</xs:sequence>
<xs:attribute name="query" type="xs:string" use="optional" />
</xs:complexType>
- <xs:complexType name="IncidentInstanceType">
- <xs:sequence>
<xs:element minOccurs="0" name="IncidentID" nillable="true"
type="cmn:StringType" />
- <xs:element minOccurs="0" name="IncidentDescription">
- <xs:complexType>
- <xs:complexContent>
- <xs:extension base="cmn:ArrayType">
- <xs:sequence>
<xs:element maxOccurs="unbounded" minOccurs="0"
name="IncidentDescription" type="cmn:StringType" />
</xs:sequence>
</xs:extension>
</xs:complexContent>
</xs:complexType>
</xs:element>
- <xs:complexType name="IncidentModelType">
- <xs:sequence>
<xs:element name="keys" type="IncidentKeysType" />
<xs:element name="instance" type="IncidentInstanceType" />
<xs:element minOccurs="0" name="messages" type="cmn:MessagesType" />
</xs:sequence>
<xs:attribute name="query" type="xs:string" use="optional" />
</xs:complexType>
- <xs:element name="RetrieveIncidentRequest">
- <xs:complexType>
- <xs:sequence>
<xs:element name="model" type="IncidentModelType" />
```

```
</xs:sequence>
<xs:attribute name="attachmentInfo" type="xs:boolean" use="optional" />
<xs:attribute name="attachmentData" type="xs:boolean" use="optional" />
<xs:attribute default="true" name="ignoreEmptyElements"
type="xs:boolean" use="optional" />
</xs:complexType>
</xs:element>
- <xs:element name="RetrieveIncidentResponse">
- <xs:complexType>
- <xs:sequence>
<xs:element name="model" type="IncidentModelType" />
<xs:element minOccurs="0" name="messages" type="cmn:MessagesType" />
</xs:sequence>
<xs:attribute name="status" type="cmn:StatusType" use="required" />
<xs:attribute name="message" type="xs:string" use="required" />
<xs:attribute name="schemaRevisionDate" type="xs:date" use="required"
/>
<xs:attribute name="schemaRevisionLevel" type="xs:int" use="required"
/>
<xs:attribute name="returnCode" type="xs:decimal" use="optional" />
<xs:attribute name="query" type="xs:string" use="optional" />
</xs:complexType>
</xs:element>
</types>
```

Nillable attribute

The nillable attribute specifies whether or not an explicit NULL value can be assigned to the element. True enables an instance of the element to have the NULL attribute set to true. The NULL attribute is defined as part of the XML Schema namespace for instances. The default is false. This attribute is optional.

The nillable attribute is analogous to the SQL concept of NULL and is useful for dealing with the ambiguity that may otherwise surround an empty XML element value. With SQL there is a difference between a NULL value and a column containing a varchar of length zero. Similarly, in an XML schema there is a difference between an XML element containing no text value and one which is explicitly marked with xsi:nil=”true”.

Unless the XML schema indicates that an XML element is nillable, you can't specify the nillable attribute for the element.

The following sample code with the nillable attribute can be found in the schema definition section:

```
<xs:element minOccurs="0" name="IncidentID" nillable="true"
type="cmn:StringType" />
```

Port type

The port defines the connection point to a Web Service. The follow section of the example Service Manager IncidentManagement WSDL shows the port type section, which includes the set of operations allowed by the endpoint.

```
- <portType name="IncidentManagement">
```

Binding section

The following section of the example Service Manager IncidentManagement WSDL shows the binding section used to define a protocol and defined data formats for a particular port type.

```
- <binding name="IncidentManagement" type="ns:IncidentManagement">
<soap:binding style="document"
transport="http://schemas.xmlsoap.org/soap/http" />
- <operation name="RetrieveIncident">
<soap:operation soapAction="Retrieve" style="document" />
- <input>
<soap:body use="literal" />
</input>
- <output>
<soap:body use="literal" />
</output>
</operation>
- <operation name="RetrieveIncidentKeysList">
<soap:operation soapAction="RetrieveKeysList" style="document" />
- <input>
<soap:body use="literal" />
</input>
- <output>
<soap:body use="literal" />
</output>
</operation>
</binding>
```

Service section

The Service section describes one or more concrete endpoints where the functionality of the service can be found.

The following example Service Manager IncidentManagement WSDL shows the service section, which is a collection of endpoints (In this example, just IncidentManagement).

```
- <service name="IncidentManagement">
```

Port section

The following section of the example Service Manager IncidentManagement WSDL shows the port section, which is a single endpoint defined as a combination of a binding and a network address.

```
- <port binding="ns:IncidentManagement" name="IncidentManagement">
<soap:address location="http://server:13080/sc62server/ws" />
</port>
</service>
</definitions>
```

1.1.1.4.1.5. Change example to use the cookie

To change the Keep-Alive example to use the cookie, you can modify the following methods.

The `createService()` method in the IncidentManagementServiceUtility.java file:

```
public static IncidentManagementStub createService(Map arguments)
throws Exception
{
String host = (String) arguments.get(ARGUMENT_HOST);
String port = (String) arguments.get(ARGUMENT_PORT);
String address = "http://" + host + ":" + port + "/SM/7/ws";
IncidentManagementStub stub = new IncidentManagementStub(address);
stub._getServiceClient().getOptions().setManageSession(true);
stub._getServiceClient().getOptions().setProperty
(HTTPConstants.REUSE_HTTP_CLIENT,true);
// set connection: close
//Header hdr = new Header(HTTPConstants.HEADER_CONNECTION,
HTTPConstants.HEADER_CONNECTION_CLOSE);
//ArrayList<Header> headers = new ArrayList<Header>();
//headers.add(hdr);
//stub._getServiceClient().getOptions().setProperty
(HTTPConstants.HTTP_HEADERS, headers);
stub._getServiceClient().getOptions().setProperty
(Constants.Configuration.ENABLE_MTOM,
Constants.VALUE_TRUE);
ServiceUtility.initServiceAuthentication(stub, arguments);
return stub;
}
```

The `createIncidents()` method in the CreateIncidentSample.java file:

```
public void createIncidents() throws Exception, IOException
{
/* Open a port to the Incident Management Web Service */
IncidentManagementStub stub =
IncidentManagementServiceUtility.createService(arguments);
int totalIM = 10;
/* Create details about the new incident */
for (int i = 1; i <= totalIM; i++)
{
if (i == totalIM)
{
// close the connection if this is the last request
Header hdr = new Header(HTTPConstants.HEADER_CONNECTION,
HTTPConstants.HEADER_CONNECTION_CLOSE);
ArrayList<Header> headers = new ArrayList<Header>();
headers.add(hdr);
stub._getServiceClient().getOptions().setProperty
(HTTPConstants.HTTP_HEADERS, headers);
}
createIncident(stub);
}
return;
}
```

The client is responsible for echoing back this value in a Cookie header in all subsequent POST requests. If the client fails to do this, the servlet container will quickly run out of sessions.

If a client request causes any Service Manager Server error or exception then this session will be terminated by the Service Manager Server. Once this happens the current JSESSIONID becomes invalid and a new JSESSIONID will be returned on the following client request. The SOAP client should echo back the new JSESSIONID for the subsequent requests to avoid the user login/logout overhead and dangling sessions saturation.

1.1.1.4.1.6. Example: Use Keep-Alive with .Net Web Services Studio

To use Keep-Alive with .Net Web Services Studio, perform the following actions.

First, make the following configurations:

- set "AllowWriteStreamBuffering" to True
- set "BasicAuthUsername" to falcon
- set "KeepAlive" to True
- set "UserCookieContainer" to True

Next, execute a RetrieveIncident action and search for the incident with the number IM1001.

- When you click **Send**, the "Set-Cookie" and "Connection" headers can be seen in the response window.
- Click **Send** again, only the "Connection" header can be seen in the response.

In the sm.log, these two requests (one per send) will belong to one session, meaning they have the same Process ID (Thread ID) combination.

```
2052(  6096) 05/05/2008 15:30:31  RTE I Using "utalloc" memory manager
2052(  6096) 05/05/2008 15:30:31  RTE I Process sm 7.01.048
System: 13080 (0x784dfb00) on PC running Windows
XP Professional (5.1 Build 2600) from server (127.0.0.1)
2052(  6096) 05/05/2008 15:30:31  RTE I Connected to SOAP client
at 127.0.0.1
2052(  6096) 05/05/2008 15:30:31
RTE I Attaching to resources with key 0x784dfb00
2052(  6096) 05/05/2008 15:30:31
RTE I Info: SQL State: 01000-5701
Message: [Microsoft][SQL Native Client][SQL Server]Changed database
context to 'sm701'.
2052(  6096) 05/05/2008 15:30:31  RTE I Info: SQL State: 01000-5703
Message: [Microsoft][SQL Native Client][SQL Server]Changed language
setting
to us_english.
2052(  6096) 05/05/2008 15:30:31  RTE I sqmssqlExec info statement= SQL
CONNECT
2052(  6096) 05/05/2008 15:30:31  RTE I Connection established to
dbtype 'sqlserver' database 'sm701' user 'sm7'
2052(  6096) 05/05/2008 15:30:31  RTE I Connected to Data source
'sm701' SQL server 'server\SQLEXPRESS' version: 9.0.3042
Using database 'sm701' as user 'sm7'
2052(  6096) 05/05/2008 15:30:31  RTE I MS SQL Server collation
'Latin1_General_BIN', varchar codepage 1252, comparison 0:
case sensitive, accent sensitive
2052(  6096) 05/05/2008 15:30:31  RTE I Thread
912DAAD51D1B0A53B251147F6665B7EE initialization done.
```

First execution of .Net Web Services Studio

The following codes show an example of the first execution of code when using Keep-Alive with .Net Web Services Studio.

```
2052(  6096) 05/05/2008 15:30:31  RTE D Parsing request document:
<?xml version="1.0" encoding="utf-8"?><soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<soap:Body>
<RetrieveIncidentRequest
xmlns="http://servicecenter.peregrine.com/PWS">
<model query="">
<keys query="">
<IncidentID/>
</keys>
<instance query="" recordid="" uniquequery="">
<IncidentID>IM1001</IncidentID>
</instance>
<messages>
<message xmlns="http://sc62server/PWS/Common"
module=""/>
</messages>
</model>
</RetrieveIncidentRequest>
</soap:Body>
</soap:Envelope>
2052(  6096) 05/05/2008 15:30:31  RTE D Done parsing request document
2052(  6096) 05/05/2008 15:30:31  RTE D doCardinalOperation entered for
SOA Mode 2 operation 1 - Retrieve
2052(  6096) 05/05/2008 15:30:31  RTE D Calling loginAuthenticate
with user=falcon and password=########
2052(  6096) 05/05/2008 15:30:31  RTE D Authentication succeeded
2052(  6096) 05/05/2008 15:30:31  RTE D Calling agend()
2052(  6096) 05/05/2008 15:30:31  RTE D Calling agstart()
2052(  6096) 05/05/2008 15:30:31  RTE D Calling login with user=falcon
and password=########
2052(  6096) 05/05/2008 15:30:32  RTE I User falcon logged in.
Already licensed
2052(  6096) 05/05/2008 15:30:32  RTE D Login succeeded
2052(  6096) 05/05/2008 15:30:32  RTE D Setting uname to falcon
2052(  6096) 05/05/2008 15:30:32  RTE D Operation will be carried out
on
file probsummary
2052(  6096) 05/05/2008 15:30:32  RTE D doQuery using query string
number="IM1001"
2052(  6096) 05/05/2008 15:30:33  RTE D doGet query returned 1
```

```
2052(  6096) 05/05/2008 15:30:33  RTE D SOA revision time is 2005-03-15
2052(  6096) 05/05/2008 15:30:33  RTE D SOA revision level is 0
```

Second execution of .Net Web Services Studio

The following codes show an example of the second execution of code when using Keep-Alive with .Net Web Services Studio.

```
2052(  6096) 05/05/2008 15:33:40  RTE D Parsing request document:
<?xml version="1.0" encoding="utf-8"?><soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<soap:Body>
<RetrieveIncidentRequest
xmlns="http://servicecenter.peregrine.com/PWS">
<model query="">
<keys query="">
<IncidentID/>
</keys>
<instance query="" recordid="" uniquequery="">
<IncidentID>IM1001</IncidentID>
</instance>
<messages>
<message xmlns="http://servicecenter.peregrine.com/PWS/Common"
module=""/>
</messages>
</model>
</RetrieveIncidentRequest>
</soap:Body>
</soap:Envelope>
2052(  6096) 05/05/2008 15:33:40  RTE D Done parsing request document
2052(  6096) 05/05/2008 15:33:40  RTE D doCardinalOperation entered for
SOA Mode 2 operation 1 - Retrieve
2052(  6096) 05/05/2008 15:33:40  RTE D User falcon is already logged
in for this process - skipping login processing
2052(  6096) 05/05/2008 15:33:40  RTE D Operation will be carried out
on file probsummary
2052(  6096) 05/05/2008 15:33:40  RTE D doQuery using query string
number="IM1001"
2052(  6096) 05/05/2008 15:33:40  RTE D doGet query returned 1
2052(  6096) 05/05/2008 15:33:41  RTE D SOA revision time is 2005-03-15
2052(  6096) 05/05/2008 15:33:41  RTE D SOA revision level is 0
```

1.1.1.4.2. Consume a Service Manager Web Service

A Service Manager Web service can be consumed by a custom client or by an application that directly consumes Web Services, such as Service Manager or Connect-It.

General Information

A Web Service development tool kit that can generate a complete Web service application from a `.wsdl` file is required to create a custom client that can access the Service Manager Web service. A good understanding of Web Services and SOAP versions 1.1 or 1.2 is also recommended.



Note

Service Manager users and application designers can choose any third-party Web Services development tool kit. However, Service Manager publishes only the WSDL files for the Web Service. Troubleshooting the client application is the responsibility of the application developer, and outside the scope of Service Manager Customer Support.

Use the steps below as a guide to create your custom Web Service client.

1. Publish the Service Manager tables that you want your client to access. You can use the Service Manager Web Services out-of-the-box or customize the extaccess records to meet your needs.
2. Obtain a Web Services client development tool that can create a complete Web Service application, such as Microsoft .NET or Apache Axis, or obtain a tool that generates a complete Web Service application by evaluating the target WSDL file.
3. Browse to the URL of your Service Manager server and download the WSDL files for the services you want your custom clients to use. Use your Web Services client development tool to browse the WSDL and determine which features you want your custom client to use. The URL of your server must include the port and the Web service name.
   For example:
   http://<Service Manager server>:<httpPort (use a dedicated port, don't use loadBalancer port)>/SM/9/PWS/IncidentManagement.wsdl connects to the Service Manager server host on the specified port and requests the IncidentManagement WSDL.
4. Use your Web Services client development tool to generate the programming language client code (classes) that will invoke the Service Manager Web services. Tools such as .NET wsdl.exe or Axis wsdl2java generate client code that can be used to invoke the Service Manager Web service from the WSDL. Your custom Web Services client invokes the client code rather than the WSDL directly.
5. Write a client application in the appropriate language of your client development tool. For example, .NET requires either Microsoft Visual C# or Visual Basic®, and Axis requires

Java.

1.1.1.4.2.1. Dynamic and static Web Services clients

Tools such as Visual Studio or .NET allow for simple creation of Web Service clients from a WSDL. These clients are static Web Service consumers and have to be rebuilt every time the WSDL changes. To get around the tedious work of rebuilding the client code for every WSDL change (new fields, new methods, new objects), you can create dynamic Web Services clients. These clients read the WSDL each time they use it and dynamically refer to the objects and methods within.

When an external client consumes Service Manager data, the client code can be written for dynamic or static WSDL consumption. When Service Manager consumes external data, it uses static consumption always.

What happens if an exposed table is changed?

The WSDL for a service doesn't change automatically as a result of making tailoring changes such as adding a new field to a table. Only if you include the new field in the Web Services API by adding it to the extaccess record will the new field be exposed.

If you change the caption (alias name) by which a field is exposed in a Web Service, you are going to have to modify and recompile any SOAP client applications which reference this field. You can rename the internal Service Manager field names, even for fields which are exposed via Web Services, without impacting deployed Web Services, as long as you don't change the alias name by which the field is known to Web Services.

Finally, if you add a new field, make the new field a required field and you have previously deployed Web Services applications which don't populate this field, you must provide tailoring in the server to generate a valid default value for the field when a value is not provided. Otherwise, inserts and updates via Web Services will fail because the new field has not been populated when the record goes through validation.

1.1.1.4.2.2. Updating Service Manager tables

By design, the Service Manager server expects that the client application will specify only those fields to be updated. It ignores missing or empty elements in the update request. If you specify a new value to update a field and that field is an array, ensure that you match the number of new values for the array elements to the number of existing array elements; otherwise, the number of elements in the array will dynamically resize to contain only the new values.

You can code a global attribute on the request element called ignoreEmptyElements and set it to true or false. If you specify ignoreEmptyElements=false, any missing or empty element in the update request causes the named field to be cleared to null values.

If you want to clear a specific field, specify xsi:nil=true as an element attribute.

1.1.1.4.2.3. Requirements for developing custom Web Services clients

You can create custom Web Services clients to access the Service Manager Web Services API. If you choose to create a custom Web Services client, ensure that you review the statement of technical support for custom Web Services clients, and that you have the following skills and tools:

- A good understanding of the W3C recommendation for SOAP version 1.1 or 1.2. Service Manager supports both versions, but recommends SOAP version 1.2.
- A Web Service development tool kit that can generate a complete Web service application from a .wsdl file.
- Familiarity with the debughttp server parameter and the HTTP.LOG it generates.



Note

There are several Web services development tool kits that you can use to develop custom Web Services clients, such as Microsoft Visual Studio .NET™, Systinet WASP™, Glue™, Apache Axis™, or Sun Web Services Developer Pack™.

In order to support custom Web Services client connections to Service Manager you need:

- An installed Service Manager server instance (Your custom Web services clients can connect to the normal server listener port)
- A list of the Service Manager tables and actions you want to permit access to (you can grant or deny access from the extaccess table)

1.1.1.4.2.4. Checklist: Creating a custom Web Services client

You can create custom Web Services client applications to connect and conduct transactions with the Service Manager Web service. Any custom clients you create must be able to send and receive from the Service Manager server valid SOAP messages.

1. Publish the Service Manager tables to which you want the custom client to connect as Web Services. You can use Service Manager Web Services API out-of-the-box, or customize the Web Services to meet your business needs.
2. Obtain a Web Services client development tool that can create a complete Web service application, such as Microsoft .NET™ or Apache Axis™, or obtain a tool that generates a complete Web Service application by evaluating the target WSDL file, such as GotDotNet™ WebServiceStudio™.
3. Browse to the URL of your Service Manager server and download the WSDL files for the services you want your custom clients to use. Use your Web Services client development tool to browse the WSDL and determine which features you want your custom client to use.

   Note

   The URL of your server must include the listener port and the Web service name. For example, http://smserver:13081/IncidentManagement.wsdl connects to the smserver host on port 13081 and requests the IncidentManagement WSDL.

   Caution

   Do not use the Load Balancer listener port for all incoming Web Services requests. Instead, dedicate one or more Service Manager server processes to serve Web Services requests by adding the "debugnode" parameter to the process you wish to dedicate to serve Web Services requests.
4. Use your Web Services client development tool to generate the programming language client code that invokes the Service Manager Web Services for the Service Manager Web Services. Tools such as .NET wsdl.exe or Axis wsdl2java can generate client code that can be used to invoke the Service Manager Web Service from the WSDL.
5. Write a client application in the appropriate language of your client development tool. For example, .NET requires Microsoft C#™ or Visual Basic™; Axis requires Java.



Tip

The Service Manager installation DVD contains source code for several sample Web Services client applications you can use as templates for your own custom clients. The source code includes Axis and .NET examples.



Note

There are many Web Service application development tools available such as Microsoft Visual Studio .NET™, Systinet WASP™, Glue™, Apache Axis™, or Sun Web Services Developer Pack™. Service Manager users and application designers can choose any third-party tool with the understanding that the company publishes only the WSDL files for the web service. Troubleshooting the client application is the responsibility of the application developer, and outside the scope of Service Manager Customer Support.

Technical support for custom Web Services clients

Custom Web Services clients and any code or scripting that you add to interface with the Service Manager products are outside the scope of the company's product suite and aren't covered under maintenance and support contracts. Ensure that you have full access to the appropriate resources to assist you with training, debugging, and maintaining any code that you add to your Service Manager environment.

We provide a working example database and several sample Web Services clients that can help you troubleshoot your custom clients and determine where errors occur.

1.1.1.4.2.5. Sample Web Services client for sc62server PWS URL

The Service Manager server includes two sample Web Services client applications for the http://servername:port_number/sc62server/PWS/service_name.wsdl. One was created for Apache™ Axis and the other for Microsoft™ Visual Studio .NET. If you have one of these two Web Services development tools installed, you can review and update the source code of the sample applications as well as generate updated proxy code to test the Service Manager Web Services functionality. The Apache Axis samples are written in Java while the Microsoft .NET samples are written in C#. The sample client applications are included with the server installation in the following folders:

- <Service Manager server installation folder>\webservices\sample\sc62webservices
  - AxisSample
  - DotNetSample

Each sample includes the source code for the client applications as well as support files for the Web Services development environment. The Apache Axis sample also includes a library of Axis jar files as well as batch files that you can use to run each class from the Windows command prompt after you have compiled the sample Java. You can use the sample applications as examples of how to create your own custom Web Services client applications.



Note

All the sample applications use a command line interface. To see the usage information for the command line interface, type: **dotNetSample -example ClassName** where ClassName is the name of the sample application class.

The Apache Axis sample client applications assume that you have a Service Manager server instance running from the local host. If this is not the case, you can change the server host name and port number using the sample's command line interface.

The Microsoft .Net sample client applications assume that you have a Service Manager server instance running from the local host. If this is not the case, you can change the server host name and port number using the sample's command line interface or from Visual Studio .NET's Web reference URL.



Caution

To use attachments with .Net samples, you must install Microsoft Web Services Enhancements (WSE) 2.0 SP2. Be sure to select the "Visual Studio Developer" option during installation. If you add WSE2 after building the examples, you must delete the old reference files ("reference.cs" and "reference.map"), update the web references, and then rebuild the sample applications.

Each of the sample folders includes a readme file that contains valuable information about using the sample application found in each of the sample folder.

Configuration Management sample

The sample client applications contain the following classes for Configuration Management. Refer to the sample application source code for comments on the usage of each class.

Field	Description
ConfigurationManagementServiceUtility	• Provides the CreateService method to initialize an object for the service. • Provides the InitServiceAuthentication method to send the host name, communications port, operator name, and operator password with each SOAP request. • Provides the InitServiceforAttachments method to initialize the service to handle MIME attachments.
CreateContactSample	Creates a contact record with the supplied parameters.
DeleteContactSample	Deletes the contact record listed in the supplied parameters.
RetrieveContactSample	Retrieves a single contact record matching the supplied parameters.
UpdateContactSample	Updates a contact record with the supplied parameters.

Incident Management sample

The sample client applications contain the following classes for Incident Management. Refer to the sample application source code for comments on the usage of each class.

Class	Description
CloseIncidentSample	Closes an incident record with the supplied parameters.
CreateIncidentSample	Creates an incident record with the supplied parameters.
IncidentManagementServiceUtility	• Provides the CreateService method to initialize an object for the service. • Provides the InitServiceAuthentication method to send the host name, communications port, operator name, and operator password with each SOAP request. • Provides the InitServiceforAttachments method to initialize the service to handle MIME attachments.
ResolveIncidentSample	Resolves an incident record matching the supplied parameters.
RetrieveIncidentListSample	Retrieves multiple incident records matching the supplied parameters.
RetrieveIncidentSample	Retrieves a single incident record matching the supplied parameters.
UpdateIncidentSample	Updates an incident record with the supplied parameters.

1.1.1.4.2.6. Command line arguments for the .NET sample application

The .NET sample application runs from the Windows command prompt. After you have compiled the .NET sample into an executable, you can perform configuration and incident management tasks with the following arguments.

Note

To see the usage information for the .NET sample application, type dotNetSample -example ClassName where ClassName is the name of a sample application class.

The following commands invoke Configuration Management functionality.

Operation	Command-line example
Create contact	dotnetsample -example CreateContact -name sneveau -lastName Neveau -firstName Sophie -workPhone "(858) 481-5000" -extension 3573 -fullname "Sophie Neveau"
Delete contact	dotNetSample -example DeleteContact -name sneveau -lastName Neveau -firstName Sophie
Retrieve contact	dotNetSample -example RetrieveContact -name "FALCON, JENNIFER"
Update Contact	dotNetSample -example UpdateContact -name "FALCON, JENNIFER" -workPhone "(858) 481-5000" -extension 3573

The following commands invoke Incident Management functionality.

Operation	Command-line example
Close incident	dotNetSample -example CloseIncident -incidentId IM10001 -closeCode "User Closer" -resolution "Problem disappeared"
Create incident	dotNetSample -example CreateIncident -title ".NET sample brief description" -category incident -problemType "not specified" -description ".NET sample incident" -severity 1 -subCategory data -productType "storage limit exceeded" -initialImpact 1 -primaryAssignmentGroup "Operating System Support (South America)" -service "Printing (Africa)"
Resolve incident	dotNetSample -example ResolveIncident -incidentId IM10006 -resolution "Problem disappeared"
Retrieve incident list	dotNetSample -example RetrieveIncidentList -incidentId IM10001:IM10002
Retrieve incident	dotNetSample -example RetrieveIncident -incidentId IM10001
Update incident	dotNetSample -example UpdateIncident -incidentId IM10006 -journalUpdates "User provided more information"

1.1.1.4.2.7. Command line arguments for the Axis sample application

The Axis sample application runs from the command prompt using Java. After you have compiled the Axis sample into an executable class file, you can perform configuration and incident management tasks with the following arguments.

Note

To see the usage information for the Axis sample application, type ClassName where ClassName is the name of a sample application class.

Configuration Management

The following commands invoke Configuration Management functionality. These examples assume you are using the batch files provided with the Axis sample application to automatically set the class path and call the proper executable class.

Operation	Command-line example
Create contact	CreateContactSample -name sneveau -fullname "Sophie Neveau"
Delete contact	DeleteContactSample -username falcon -name "sneveau"
Retrieve contact	RetrieveContactSample -name "FALCON, JENNIFER"
Update Contact	UpdateContactSample -name "FALCON, JENNIFER"

Incident Management

The following commands invoke Incident Management functionality. These examples assume you are using the batch files provided with the Axis sample application to automatically set the class path and call the proper executable class.

Operation	Command-line example
Close incident	CloseIncidentSample -incidentId IM10001 -closeCode "User Closer" -resolution "Problem disappeared"
Create incident	CreateIncidentSample -briefDescription "Java sample brief description" -category incident -incidentDescription "This is a description" -severity 1 -subCategory hardware -productType "missing or stolen" -initialImpact 1 -service Applications -primaryAssignmentGroup Network
Resolve incident	ResolveIncidentSample -incidentId IM10006 -resolution "Problem disappeared"
Retrieve incident list	RetrieveIncidentListSample -incidentId IM10001:IM10002
Retrieve incident	RetrieveIncidentSample -incidentId IM1001
Update incident	UpdateIncidentSample -incidentId IM10006 -journalUpdates "User provided more information"

1.1.1.4.2.8. Using query syntax

As shown in the example above, Service Manager supports queries using a special query syntax with special characters such as # ("starts with"), or relational operators such as > or < preceding an actual data value. With Web Services this syntax is available for string data as well. If the field is of a type other than string (for example an integer or dateTime type) and you are using a strongly typed programming language such as Java or C# to write your client code, you will not be able to leverage this feature, since the special characters would not be acceptable data types for these fields. To generate queries with this syntax on all types of fields, fill in the query=”xxx” section as shown below.

The request

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:pws="http://servicecenter.peregrine.com/PWS"
 xmlns:com="http://servicecenter.peregrine.com/PWS/Common">
   <soapenv:Header/>
   <soapenv:Body>
      <pws:RetrieveIncidentKeysListRequest attachmentInfo="?"
       attachmentData="?" ignoreEmptyElements="true">
         <pws:model query="">
            <pws:keys query=" update.time>'05/01/08'">
               <pws:IncidentID type="String" mandatory="?"
readonly="?">
               </pws:IncidentID>
            </pws:keys>
            <pws:instance query=" " uniquequery="?" recordid="?">
               <pws:IncidentID type="String" mandatory="?"
readonly="?">
               </pws:IncidentID>
               <pws:Category type="String" mandatory="?" readonly="?">
               </pws:Category>
               <pws:OpenTime type="DateTime" mandatory="?"
readonly="?">
               </pws:OpenTime>
               <pws:OpenedBy type="String" mandatory="?" readonly="?">
               </pws:OpenedBy>
               <pws:severity type="String" mandatory="?" readonly="?">
               </pws:severity>
               <pws:UpdatedTime type="DateTime" mandatory="?"
                readonly="?"></pws:UpdatedTime>
               <pws:PrimaryAssignmentGroup type="String" mandatory="?"
                readonly="?"></pws:PrimaryAssignmentGroup>
               <pws:ClosedTime type="DateTime" mandatory="?"
                readonly="?"></pws:ClosedTime>
               <pws:ClosedBy type="String" mandatory="?" readonly="?">
               </pws:ClosedBy>
               <pws:ClosureCode type="String" mandatory="?"
readonly="?">
               </pws:ClosureCode>
               <pws:ConfigurationItem type="String" mandatory="?"
                readonly="?"></pws:ConfigurationItem>
               <pws:Location type="String" mandatory="?" readonly="?">
               </pws:Location>
               <pws:IncidentDescription type="Array">
                  <pws:IncidentDescription type="String" mandatory="?"
```

```
                    readonly="?"></pws:IncidentDescription>
                </pws:IncidentDescription>
                <pws:Resolution type="Array">
                   <pws:Resolution type="String" mandatory="?"
                    readonly="?"></pws:Resolution>
                </pws:Resolution>
                <pws:AssigneeName type="String" mandatory="?"
readonly="?">
                </pws:AssigneeName>
                <pws:Contact type="String" mandatory="?" readonly="?">
                </pws:Contact>
                 <pws:JournalUpdates type="Array">
                   <pws:JournalUpdates type="String" mandatory="?"
                    readonly="?"></pws:JournalUpdates>
                </pws:JournalUpdates>
                <pws:AlertStatus type="String" mandatory="?"
readonly="?">
                </pws:AlertStatus>
                <pws:ContactLastName type="String" mandatory="?"
                 readonly="?"></pws:ContactLastName>
                <pws:ContactFirstName type="String" mandatory="?"
                 readonly="?"></pws:ContactFirstName>
                <pws:Company type="String" mandatory="?" readonly="?">
                </pws:Company>
                <pws:BriefDescription type="String" mandatory="?"
                 readonly="?"></pws:BriefDescription>
                <pws:TicketOwner type="String" mandatory="?"
readonly="?">
                </pws:TicketOwner>
                <pws:UpdatedBy type="String" mandatory="?" readonly="?">
                </pws:UpdatedBy>
                <pws:IMTicketStatus type="String" mandatory="?"
                 readonly="?"></pws:IMTicketStatus>
                <pws:Subcategory type="String" mandatory="?"
readonly="?">
                </pws:Subcategory>
                <pws:SLAAgreementID type="Decimal" mandatory="?"
                 readonly="?"></pws:SLAAgreementID>
                <pws:SiteCategory type="String" mandatory="?"
readonly="?">
                </pws:SiteCategory>
                <pws:ProductType type="String" mandatory="?"
readonly="?">
                </pws:ProductType>
                <pws:ProblemType type="String" mandatory="?"
```

```
readonly="?">
               </pws:ProblemType>
               <pws:ResolutionFixType type="String" mandatory="?"
                readonly="?"></pws:ResolutionFixType>
               <pws:UserPriority type="String" mandatory="?"
readonly="?">
               </pws:UserPriority>
               <pws:Solution type="Array">
                  <pws:Solution type="String" mandatory="?"
readonly="?">
                  </pws:Solution>
               </pws:Solution>
               <pws:InitialImpact type="String" mandatory="?"
                readonly="?"></pws:InitialImpact>
               <pws:folder type="String" mandatory="?" readonly="?">
               </pws:folder>
            </pws:instance>
            <pws:messages>
                <com:message type="String" mandatory="?"
                 readonly="?" severity="?" module="?">?</com:message>
            </pws:messages>
         </pws:model>
      </pws:RetrieveIncidentKeysListRequest>
   </soapenv:Body>
</soapenv:Envelope>
```

The response

```
<SOAP-ENV:Envelope
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
   <SOAP-ENV:Body>
      <RetrieveIncidentKeysListResponse message="Success"
       query="" returnCode="0" schemaRevisionDate="2007-04-14"
       schemaRevisionLevel="1" status="SUCCESS"
       xsi:schemaLocation="http://servicecenter.peregrine.com/PWS
       http://<sm server>.americas.hpqcorp.net:13701/sc62server/ws/
       Incident.xsd"
       xmlns="http://servicecenter.peregrine.com/PWS"
       xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common"
       xmlns:xsd="http://www.w3.org/2001/XMLSchema"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
         <keys>
            <IncidentID type="String">IM10055</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10063</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10070</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10077</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10090</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10115</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10116</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10117</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10118</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10119</IncidentID>
         </keys>
         <keys>
```

```
            <IncidentID type="String">IM10120</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10121</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10122</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10123</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10124</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10125</IncidentID>
         </keys>
      </RetrieveIncidentKeysListResponse>
   </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```



Note

The field names in such directly entered query strings reflect either the actual field names (such as update.time) or the Caption (such as UpdateTime). Clients who want to submit an expert/advanced query should use either the query attribute on the <keys> element or the query attribute on the <instance> element. Both are provided because some requests don't define any <instance> element. During SOAP API request processing, the server will look first at the <keys> element and, if there is no query attribute there, will look at the <instance> element. Query attributes defined on any other element are never consulted during inbound SOAP request processing.

1.1.1.4.2.9. Retrieving data from Service Manager

Retrieval methods aren't defined in the extaccess record. The following list shows the methods for retrieval that are available and under which circumstances to use each one:

- `Retrieve<FileName>` - Used if only one record will be returned. Throws a fault if multiple records are returned.
- `Retrieve<FileName>KeysList` - Retrieves the list of unique keys (which doesn't have to be the unique key of the Service Manager dbdicts). The list can either be passed as an array to the `Retrieve<FileName>List` method, or looped through to pass to the `Retrieve<FileName>` method.
- `Retrieve<FileName>List` - Retrieves a list of records with information that was gathered either in the `Retrieve<FileName>KeysList` method or by passing in a query directly through the instance block. This method expects an array of keys unless the query approach is used.



Note

When retrieving data from a single table rather than a Service such as the contacts table, request the WSDL for the alias name defined in `extaccess`, such as "Contact" (singular form, upper-case "C") rather than for `contacts` (the actual file name).

There are different approaches to retrieving a list of records. When developing a custom client there are actually two separate methods that can be used to retrieve list data.

The first approach uses the following steps:

1. Send the data query (such as `<open.time>>6/30/05</open.time>`) to the `RetrieveKeysList` method.
2. The result is a list of records where each record contains only the "primary key" (such as Incident ID) for those records that match the query.
3. You can either provide the list to the `RetrieveList` method and receive all records defined by the list in a single XML document, or loop through the list, one record at a time, calling `Retrieve` once for each record by key.

The second approach uses these steps:

1. Send the data query (such as `< open.time>>6/30/05</open.time>`) directly to the `RetrieveList` method.
2. This single method call returns the entire result set (all fields for all records matching the query) in a single XML response.

Note

The second approach returns the entire query result set in one method call. If the result set is large, use the first approach to increase performance.

1.1.1.4.2.10. Example: Retreiving data from Service Manager via a Web service

The simplest way to perform retrieval operations is via Query-by-example (QBE). This is done by creating an instance of a particular kind of object (such as an Incident) and populating one or more fields with values to determine the result set. You have to only supply values for the fields on which you wish to select.

The instance is then passed into a RetrieveXXXKeysList request. In a program, such as the sample programs provided with Service Manager, you would be assigning values to properties or calling setter methods on various Java or C# or other objects. In the following example, we submit a RetrieveIncidentKeysList object, supplying a value for OpenedBy and UpdatedBy. In this example, we will use Service Manager query syntax to find all incidents where the OpenedBy element starts with “fal” as well as pass a literal value for the UpdatedBy field.

We get back a RetrieveIncidentKeysListResponse object listing the primary keys of the matching Incident objects.

Combining multiple values in this QBE style selection connects the query terms with AND. To create a query using OR, supply an expert query as a string in the <pws:instance query="xxx" ...> area.

The request

```
<?xml version="1.0" encoding="UTF-8" ?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:pws="http://servicecenter.peregrine.com/PWS"
xmlns:com="http://servicecenter.peregrine.com/PWS/Common">
   <soapenv:Header/>
   <soapenv:Body>
      <pws:RetrieveIncidentKeysListRequest attachmentInfo="?"
      attachmentData="?" ignoreEmptyElements="true">
         <pws:model query="">
            <pws:keys query="">
               <pws:IncidentID type="String" mandatory="?"
readonly="?">
               </pws:IncidentID>
            </pws:keys>
            <pws:instance query="" uniquequery="?" recordid="?">
               <pws:IncidentID type="String" mandatory="?"
readonly="?">
               </pws:IncidentID>
               <pws:Category type="String" mandatory="?" readonly="?">
               </pws:Category>
               <pws:OpenTime type="DateTime" mandatory="?"
readonly="?">
               </pws:OpenTime>
               <pws:OpenedBy type="String" mandatory="?"
readonly="?">#fal
               </pws:OpenedBy>
               <pws:severity type="String" mandatory="?" readonly="?">
               </pws:severity>
               <pws:UpdatedTime type="DateTime" mandatory="?"
               readonly="?"></pws:UpdatedTime>
               <pws:PrimaryAssignmentGroup type="String" mandatory="?"
               readonly="?"></pws:PrimaryAssignmentGroup>
               <pws:ClosedTime type="DateTime" mandatory="?"
readonly="?">
               </pws:ClosedTime>
               <pws:ClosedBy type="String" mandatory="?" readonly="?">
               </pws:ClosedBy>
               <pws:ClosureCode type="String" mandatory="?"
readonly="?">
               </pws:ClosureCode>
               <pws:ConfigurationItem type="String" mandatory="?"
               readonly="?"></pws:ConfigurationItem>
               <pws:Location type="String" mandatory="?" readonly="?">
```

```
                </pws:Location>
                <pws:IncidentDescription type="Array">
                   <pws:IncidentDescription type="String" mandatory="?"
                   readonly="?"></pws:IncidentDescription>
                </pws:IncidentDescription>
                <pws:Resolution type="Array">
                   <pws:Resolution type="String" mandatory="?"
                   readonly="?"></pws:Resolution>
                </pws:Resolution>
                <pws:AssigneeName type="String" mandatory="?"
readonly="?">
                </pws:AssigneeName>
                <pws:Contact type="String" mandatory="?" readonly="?">
                </pws:Contact>
                 <pws:JournalUpdates type="Array">
                   <pws:JournalUpdates type="String" mandatory="?"
                   readonly="?"></pws:JournalUpdates>
                </pws:JournalUpdates>
                <pws:AlertStatus type="String" mandatory="?"
readonly="?">
                </pws:AlertStatus>
                <pws:ContactLastName type="String" mandatory="?"
                readonly="?"></pws:ContactLastName>
                <pws:ContactFirstName type="String" mandatory="?"
                readonly="?"></pws:ContactFirstName>
                <pws:Company type="String" mandatory="?" readonly="?">
                </pws:Company>
                <pws:BriefDescription type="String" mandatory="?"
                readonly="?"></pws:BriefDescription>
                <pws:TicketOwner type="String" mandatory="?"
readonly="?">
                </pws:TicketOwner>
                <pws:UpdatedBy type="String" mandatory="?"
                readonly="?">falcon</pws:UpdatedBy>
                <pws:IMTicketStatus type="String" mandatory="?"
                readonly="?"></pws:IMTicketStatus>
                <pws:Subcategory type="String" mandatory="?"
readonly="?">
                </pws:Subcategory>
                <pws:SLAAgreementID type="Decimal" mandatory="?"
                readonly="?"></pws:SLAAgreementID>
                <pws:SiteCategory type="String" mandatory="?"
readonly="?">
                </pws:SiteCategory>
                <pws:ProductType type="String" mandatory="?"
```

```
readonly="?">
               </pws:ProductType>
               <pws:ProblemType type="String" mandatory="?"
readonly="?">
               </pws:ProblemType>
               <pws:ResolutionFixType type="String" mandatory="?"
               readonly="?"></pws:ResolutionFixType>
               <pws:UserPriority type="String" mandatory="?"
readonly="?">
               </pws:UserPriority>
               <pws:Solution type="Array">
                  <pws:Solution type="String" mandatory="?"
readonly="?">
                  </pws:Solution>
               </pws:Solution>
               <pws:InitialImpact type="String" mandatory="?"
              readonly="?"></pws:InitialImpact>
               <pws:folder type="String" mandatory="?" readonly="?">
               </pws:folder>
            </pws:instance>
            <pws:messages>
                <com:message type="String" mandatory="?" readonly="?"
                severity="?" module="?">?</com:message>
            </pws:messages>
         </pws:model>
      </pws:RetrieveIncidentKeysListRequest>
</soapenv:Envelope>
```

The response

```
<?xml version="1.0" encoding="utf-16"?>
<SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
   <SOAP-ENV:Body>
      <RetrieveIncidentKeysListResponse message="Success" query=""
       returnCode="0" schemaRevisionDate="2007-04-14"
       schemaRevisionLevel="1" status="SUCCESS"
       xsi:schemaLocation="http://servicecenter.peregrine.com/PWS
       http://<sm server>.americas.hpqcorp.net:13701/sc62server/ws/
       Incident.xsd" xmlns="http://servicecenter.peregrine.com/PWS"
       xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common"
       xmlns:xsd="http://www.w3.org/2001/XMLSchema"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
         <keys>
            <IncidentID type="String">IM10001</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10004</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10009</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10016</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10027</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10038</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10049</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10060</IncidentID>
         </keys>
         <keys>
            <IncidentID type="String">IM10061</IncidentID>
         </keys>
      </RetrieveIncidentKeysListResponse>
   </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Having retrieved a list of <keys> elements we can now retrieve these Incidents using a RetrieveIncidentList request, by supplying the collection of keys elements in that request.

You can submit a variable number of <keys> elements in a RetrieveXXXList request, subject only to your program's ability to handle large XML responses. Java client programs can sometimes run out of memory if the server returns very large responses.

To help prevent the RetrieveXXXListRequest Web Service from causing a Java Heap Space out-of-memory error when retrieving a list of records, the following applies for the response of a RetrieveXXXList request:

- When there is neither a start attribute nor a count attribute, return all records/keys.
- When there is a valid start value but no count attribute, return all records starting from the start attribute.
- When there is a valid start attribute and valid count attribute, return the number of keys/records starting from the start attribute.
- When there is a negative start attribute, return from the first record.
- When there is a negative count attribute, return one record.
- When the start attribute is bigger than the total number of records/keys, no record is returned.

1.1.1.4.2.11. Retrieve data from Service Manager using Pagination

Retrieving a list of data objects may result in a very large XML response document that could cause performance and memory utilization issues either on the client or the server. To avoid these problems, Service Manager supports the use of pagination, the process of returning pages of data instead of one large response.

To enable pagination, a Web service request can make use of these additional attributes:

- count – number of records/keys to return. By default all records are returned. This attribute indicates you want to use pagination. (Optional attribute)
- start – the starting record/key number. By default a retrieve request will start at record 0. (Optional attribute)
- handle – a record-list handle returned on a previous retrieve request that specified a count.

When pagination is used, the Web service does the following:

- If there is not a 'start' or 'count' attribute, the service returns all records or keys.
- With a valid 'start' value and 'no count' attribute, the service returns all records starting from the 'start' attribute.
- With a valid 'start' attribute and valid 'count' attribute, the service returns the number of keys or records starting from the 'start' attribute.
- With a negative 'start' attribute, the service returns records from the first record and creates a warning in log file.
- With a negative 'count' attribute, the service returns one record and creates a warning message in the log file.
- With the 'start' attribute greater than the total number of records or keys, the service returns no records and creates a message in the error log file.

Example: Use Web Service with pagination to retrieve data from Service Manager

In the previous example the Web Service returned a list of eight Incident keys. To illustrate pagination, this example limits the number of records to returned to four. This requires the use of the additional attributes count and start.

For example to limit the response to a maximum of eight records starting at the first record:

Request with pagination

```
<?xml version="1.0" encoding="UTF-8" ?><soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:pws="http://servicecenter.peregrine.com/PWS"
xmlns:com="http://servicecenter.peregrine.com/PWS/Common">
    <soapenv:Header/>
     <soapenv:Body>
        <pws:RetrieveIncidentKeysListRequest attachmentInfo="?"
          attachmentData="?" ignoreEmptyElements="true" count=8
start=0>
           <pws:model query="">
           <pws:keys query="">
           <pws:IncidentID type="String" mandatory="?"
             readonly="?">
           </pws:IncidentID>
           </pws:keys>
        <pws:instance query="" uniquequery="?" recordid="?">
        <pws:IncidentID type="String" mandatory="?"
          readonly="?">
        </pws:IncidentID>
        <pws:Category type="String" mandatory="?" readonly="?">
        </pws:Category>
        <pws:OpenTime type="DateTime" mandatory="?"
        readonly="?">
        </pws:OpenTime>
        <pws:OpenedBy type="String" mandatory="?"
          readonly="?">#fal
        </pws:OpenedBy>
        <pws:severity type="String" mandatory="?" readonly="?">
        </pws:severity>
        <pws:UpdatedTime type="DateTime" mandatory="?"
           readonly="?"></pws:UpdatedTime>
        <pws:PrimaryAssignmentGroup type="String" mandatory="?"
        readonly="?"></pws:PrimaryAssignmentGroup>
        <pws:ClosedTime type="DateTime" mandatory="?"
        readonly="?">
        </pws:ClosedTime>
        <pws:ClosedBy type="String" mandatory="?" readonly="?">
        </pws:ClosedBy>
        <pws:ClosureCode type="String" mandatory="?"
        readonly="?">
        </pws:ClosureCode>
        <pws:ConfigurationItem type="String" mandatory="?"
          readonly="?"></pws:ConfigurationItem>
        <pws:Location type="String" mandatory="?" readonly="?">
```

```
        </pws:Location>
        <pws:IncidentDescription type="Array">
        <pws:IncidentDescription type="String" mandatory="?"
        readonly="?"></pws:IncidentDescription>
        </pws:IncidentDescription>
        <pws:Resolution type="Array">
        <pws:Resolution type="String" mandatory="?"
          readonly="?"></pws:Resolution>
        </pws:Resolution>
        <pws:AssigneeName type="String" mandatory="?"
        readonly="?">
        </pws:AssigneeName>
        <pws:Contact type="String" mandatory="?" readonly="?">
        </pws:Contact>
        <pws:JournalUpdates type="Array">
        <pws:JournalUpdates type="String" mandatory="?"
          readonly="?"></pws:JournalUpdates>
        </pws:JournalUpdates>
        <pws:AlertStatus type="String" mandatory="?"
        readonly="?">
        </pws:AlertStatus>
        <pws:ContactLastName type="String" mandatory="?"
          readonly="?"></pws:ContactLastName>
        <pws:ContactFirstName type="String" mandatory="?"
          readonly="?"></pws:ContactFirstName>
        <pws:Company type="String" mandatory="?" readonly="?">
        </pws:Company>
        <pws:BriefDescription type="String" mandatory="?"
        readonly="?"></pws:BriefDescription>
        <pws:TicketOwner type="String" mandatory="?"
          readonly="?">
        </pws:TicketOwner>
        <pws:UpdatedBy type="String" mandatory="?"
          readonly="?">falcon</pws:UpdatedBy>
        <pws:IMTicketStatus type="String" mandatory="?"
        readonly="?"></pws:IMTicketStatus>
        <pws:Subcategory type="String" mandatory="?"
        readonly="?">
        </pws:Subcategory>
        <pws:SLAAgreementID type="Decimal" mandatory="?"
        readonly="?"></pws:SLAAgreementID>
        <pws:SiteCategory type="String" mandatory="?"
        readonly="?">
        </pws:SiteCategory>
        <pws:ProductType type="String" mandatory="?"
```

```
        readonly="?">
      </pws:ProductType>
      <pws:ProblemType type="String" mandatory="?"
      readonly="?">
      </pws:ProblemType>
      <pws:ResolutionFixType type="String" mandatory="?"
        readonly="?"></pws:ResolutionFixType>
      <pws:UserPriority type="String" mandatory="?"
        readonly="?">
      </pws:UserPriority>
      <pws:Solution type="Array">
      <pws:Solution type="String" mandatory="?"
      readonly="?">
      </pws:Solution>
      </pws:Solution>
      <pws:InitialImpact type="String" mandatory="?"
      readonly="?"></pws:InitialImpact>
      <pws:folder type="String" mandatory="?" readonly="?">
      </pws:folder>
      </pws:instance>
      <pws:messages>
        <com:message type="String" mandatory="?" readonly="?"
         severity="?" module="?">?</com:message>
      </pws:messages>
      </pws:model>
      </pws:RetrieveIncidentKeysListRequest>
      </soapenv:Envelope>
```

Response with pagination

The response indicates that eight records were returned (count=8). In addition the response indicates that there are more records to retrieve (more=1).

```
<?xml version="1.0" encoding="utf-16"?>
<SOAP-ENV:Envelope xmlns:SOAPENV="
http://schemas.xmlsoap.org/soap/envelope/">
<SOAP-ENV:Body>
<RetrieveIncidentKeysListResponse count=8 more=1
handle=probsummary4d67db480000c0082000f1a0 message="Success" query=""
returnCode="0" schemaRevisionDate="2007-04-14"
schemaRevisionLevel="1" status="SUCCESS"
xsi:schemaLocation="http://servicecenter.peregrine.com/PWS
http://<sm server>.americas.hpqcorp.net:13701/sc62server/ws/
Incident.xsd" xmlns="http://servicecenter.peregrine.com/PWS"
xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<keys>
<IncidentID type="String">IM10001</IncidentID>
</keys>
<keys>
<IncidentID type="String">IM10004</IncidentID>
</keys>
<keys>
<IncidentID type="String">IM10009</IncidentID>
</keys>
<keys>
<IncidentID type="String">IM10016</IncidentID>
</keys>
<keys>
<IncidentID type="String">IM10027</IncidentID>
</keys>
<keys>
<IncidentID type="String">IM10038</IncidentID>
</keys>
<keys>
<IncidentID type="String">IM10049</IncidentID>
</keys>
<keys>
<IncidentID type="String">IM10060</IncidentID>
</keys>
</RetrieveIncidentKeysListResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Next pagination request

To retrieve the next page of records the request would increment the start attribute by the previous count and supply the handle returned on the previous response.

This request is for the next set of records.

```
<?xml version="1.0" encoding="UTF-8" ?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:pws="http://servicecenter.peregrine.com/PWS"
xmlns:com="http://servicecenter.peregrine.com/PWS/Common">
<soapenv:Header/>
<soapenv:Body>
<pws:RetrieveIncidentKeysListRequest attachmentInfo="?"
attachmentData="?" ignoreEmptyElements="true" count=8  start=8 handle=
probsummary4d67db480000c0082000f1a0>
<pws:model query="">
<pws:keys query="">
<pws:IncidentID type="String" mandatory="?"
readonly="?">
</pws:IncidentID>
</pws:keys>
<pws:instance query="" uniquequery="?" recordid="?">
<pws:IncidentID type="String" mandatory="?"
readonly="?">
</pws:IncidentID>
<pws:Category type="String" mandatory="?" readonly="?">
</pws:Category>
<pws:OpenTime type="DateTime" mandatory="?"
readonly="?">
</pws:OpenTime>
<pws:OpenedBy type="String" mandatory="?"
readonly="?">#fal
</pws:OpenedBy>
<pws:severity type="String" mandatory="?" readonly="?">
</pws:severity>
<pws:UpdatedTime type="DateTime" mandatory="?"
readonly="?"></pws:UpdatedTime>
<pws:PrimaryAssignmentGroup type="String" mandatory="?"
readonly="?"></pws:PrimaryAssignmentGroup>
<pws:ClosedTime type="DateTime" mandatory="?"
readonly="?">
</pws:ClosedTime>
<pws:ClosedBy type="String" mandatory="?" readonly="?">
</pws:ClosedBy>
<pws:ClosureCode type="String" mandatory="?"
readonly="?">
</pws:ClosureCode>
<pws:ConfigurationItem type="String" mandatory="?"
readonly="?"></pws:ConfigurationItem>
```

```
<pws:Location type="String" mandatory="?" readonly="?">
</pws:Location>
<pws:IncidentDescription type="Array">
<pws:IncidentDescription type="String" mandatory="?"
readonly="?"></pws:IncidentDescription>
</pws:IncidentDescription>
<pws:Resolution type="Array">
<pws:Resolution type="String" mandatory="?"
readonly="?"></pws:Resolution>
</pws:Resolution>
<pws:AssigneeName type="String" mandatory="?"
readonly="?">
</pws:AssigneeName>
<pws:Contact type="String" mandatory="?" readonly="?">
</pws:Contact>
<pws:JournalUpdates type="Array">
<pws:JournalUpdates type="String" mandatory="?"
readonly="?"></pws:JournalUpdates>
</pws:JournalUpdates>
<pws:AlertStatus type="String" mandatory="?"
readonly="?">
</pws:AlertStatus>
<pws:ContactLastName type="String" mandatory="?"
readonly="?"></pws:ContactLastName>
<pws:ContactFirstName type="String" mandatory="?"
readonly="?"></pws:ContactFirstName>
<pws:Company type="String" mandatory="?" readonly="?">
</pws:Company>
<pws:BriefDescription type="String" mandatory="?"
readonly="?"></pws:BriefDescription>
<pws:TicketOwner type="String" mandatory="?"
readonly="?">
</pws:TicketOwner>
<pws:UpdatedBy type="String" mandatory="?"
readonly="?">falcon</pws:UpdatedBy>
<pws:IMTicketStatus type="String" mandatory="?"
readonly="?"></pws:IMTicketStatus>
<pws:Subcategory type="String" mandatory="?"
readonly="?">
</pws:Subcategory>
<pws:SLAAgreementID type="Decimal" mandatory="?"
readonly="?"></pws:SLAAgreementID>
<pws:SiteCategory type="String" mandatory="?"
readonly="?">
</pws:SiteCategory>
```

```
<pws:ProductType type="String" mandatory="?"
readonly="?">
</pws:ProductType>
<pws:ProblemType type="String" mandatory="?"
readonly="?">
</pws:ProblemType>
<pws:ResolutionFixType type="String" mandatory="?"
readonly="?"></pws:ResolutionFixType>
<pws:UserPriority type="String" mandatory="?"
readonly="?">
</pws:UserPriority>
<pws:Solution type="Array">
<pws:Solution type="String" mandatory="?"
readonly="?">
</pws:Solution>
</pws:Solution>
<pws:InitialImpact type="String" mandatory="?"
readonly="?"></pws:InitialImpact>
<pws:folder type="String" mandatory="?" readonly="?">
</pws:folder>
</pws:instance>
<pws:messages>
<com:message type="String" mandatory="?" readonly="?"
severity="?" module="?">?</com:message>
</pws:messages>
</pws:model>
</pws:RetrieveIncidentKeysListRequest>
</soapenv:Envelope>
```

Next pagination response

The response indicates that one record was returned (count=1). In addition, the response indicates that there are no more records to retrieve (more=0).

```
<?xml version="1.0" encoding="utf-16"?>
<SOAP-ENV:Envelope xmlns:SOAPENV="
http://schemas.xmlsoap.org/soap/envelope/">
<SOAP-ENV:Body>
<RetrieveIncidentKeysListResponse count=1 more=0
handle=probsummary4d67db480000c0082000f1a0 message="Success" query=""
returnCode="0" schemaRevisionDate="2007-04-14"
schemaRevisionLevel="1" status="SUCCESS"
xsi:schemaLocation="http://servicecenter.peregrine.com/PWS
http://<sm server>.americas.hpqcorp.net:13701/sc62server/ws/
Incident.xsd" xmlns="http://servicecenter.peregrine.com/PWS"
xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<keys>
<IncidentID type="String">IM10061</IncidentID>
</keys>
</RetrieveIncidentKeysListResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

1.1.1.4.2.12. Retrieve data from Service Manager for Optimistic Locking

You can retrieve data via a Web service for later use in an update using optimistic locking by using an additional attribute:

- updatecounter – a boolean value specifying if the response should include the attribute updatecounter that can be used on a subsequent update request to optimistically lock the record. The value returned in the updatecounter attribute can be specified on a subsequent update request as the value of the updatecontraint attribute. If the value of the updateconstraint matches the value in the database the update is allowed. If the updateconstraint value doesn't match the value in the database the update is rejected with a return code of 51 and the message “Record modified since last retrieved”

For example to retrieve the data with the updatecounter.

Request with updatecounter

```
<?xml version="1.0" encoding="UTF-8" ?>
 <soapenv:Envelope
 xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:pws="http://servicecenter.peregrine.com/PWS"
 xmlns:com="http://servicecenter.peregrine.com/PWS/Common">
 <soapenv:Header/>
 <soapenv:Body>
 <pws:RetrieveIncidentKeysListRequest attachmentInfo="?"
 attachmentData="?" ignoreEmptyElements="true">
 <pws:model query="" updatecounter=true>
 …
 …
…
```

Response with updatecounter

The response indicates the updatecounter for each record returned.

```
<?xml version="1.0" encoding="utf-16"?>
 <SOAP-ENV:Envelope xmlns:SOAPENV="
 http://schemas.xmlsoap.org/soap/envelope/">
 <SOAP-ENV:Body>
 <RetrieveIncidentKeysListResponse count=8 more=1
 handle=probsummary4d67db480000c0082000f1a0 message="Success" query=""
 returnCode="0" schemaRevisionDate="2007-04-14"
 schemaRevisionLevel="1" status="SUCCESS"
 xsi:schemaLocation="http://servicecenter.peregrine.com/PWS
 http://<sm server>.americas.hpqcorp.net:13701/sc62server/ws/
 Incident.xsd"
 xmlns="http://servicecenter.peregrine.com/PWS"
 xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common"
 xmlns:xsd="http://www.w3.org/2001/XMLSchema"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
 <keys updatecounter=”7”>
 <IncidentID type="String">IM10001</IncidentID>
 </keys>
 <keys updatecounter =”45”>
 <IncidentID type="String">IM10004</IncidentID>
 </keys>
 <keys updatecounter =”9”>
 <IncidentID type="String">IM10009</IncidentID>
 </keys>
 <keys updatecounter =”13”>
 <IncidentID type="String">IM10016</IncidentID>
 </keys>
 …
 …
…
```

1.1.1.4.2.13. Web Services examples in the RUN directory

All valid Web Services examples for Axis and .NET are contained in the https://portal.microfocus.com/s/article/KM000003929 knowledge document.

Both the AxisSample and the DotNetSample directories contain documents with setup instructions. In AxisSample this document is readme.txt. In DotNetSample the document is WebServices README.doc.

The AxisSample\bin directory contains a selection of batch files that can be run directly, if JDK 1.4 or higher and Apache Ant are both installed on the machine. The DotNet samples have to be compiled before running.

Note

Axis 1.x defaults to using its own httpSender class which is not compatible with HTTP 1.1 Keep-Alive. Axis 1.x must be configured to use the commons-httpclient jar file in order to get keep-alive behavior. Running the Web Service without Keep-Alive negatively impacts performance. The Web Service client needs to be Cookie aware so that the servlet container session is maintained. You need to use Axis 2 that provides HTTP 1.1 keep-alive support and cookie-aware behavior.

Example: Retrieving Service Manager Release Management changes into a text file using Connect-It

1. Create the Service Manager Web Services Connector with the following connection parameter settings. (All other settings remain the defaults.)
   - **Server name:** <server>:<httpPort (use a dedicated port, not the loadBalancer port)>
   - **Context Path:** sc62server/PWS
   - **Service name:** Change Management
   - Enter the SysAdmin userID and password
2. Click **Test** to verify that the connection works.
3. Click **Finish** to save the changes made to the new connector.
4. In our example, the Web Service simply fills information into a delimited text field. The settings for that connector are as follows:
   - **Name:** Changes
   - **Processing Mode:** write
   - **Connection protocol:** Local / network files

- Enter a folder name and decide whether to create a separate file for each record retrieved or write all records into one file (recommended).
- On the next screen, decide whether to append to the same file (recommended) or overwrite with each run, or how many files to keep.
- Enter the path to the descriptor file (see below) or create a new descriptor file.

5. Click **Finish** to create and save the connector.

   To create a description file for the text output file (comma delimited text in our example), the following code is a sample .dsc file that can be used for retrieving change information:

```
{ TextFileFormat SMChange
        Extension=
        FormatType=Delimited
        EscapeChar="\\"
        Quote="\""
        Extracolumn=1
        WriteColumn=1
        Delimiter=,
        { String "Change Number"
                UserType=Default
        }
        { String "Category"
                UserType=Default
        }
        { String "Status"
                UserType= Default
        }
        { String "Approval Status"
                UserType=Default
        }
        { String "Requested By"
                UserType=Default
        }
        { String "Assigned To"
                UserType= Default
        }
        { String "Coordinator"
                UserType=Default
        }
        { String "Coordinator Phone"
                UserType=Default
        }
        { TimeStamp "Planned Start Date"
                UserType=TimeStamp
        }
        { TimeStamp "Planned End Date"
                UserType=TimeStamp
        }
        { String "Reason"
                UserType=Default
        }
        { String "Current Phase"
                UserType= Default
        }
```

```
        { String "Risk Assessment"
                UserType=Default
        }
}
```

6. Finally the source (Web Services) data and the target data (Delimited Text file) must be mapped, in this case based on the matching names.
7. Because Web Services need to be prompted to produce output, another text file connector must be created that helps create the request sent to the Web service. This text file connector is defined as follows:

   • **Processing Mode:** read

   • **Connection Protocol:** Local/Network files

   • **Location:** Read files, file name: <path and filename>

   • Upon successful processing, leave the file in the folder.

   Use this .dsc file. Enter <path and filename> and click Find (magnifying glass) to create or modify a description file.
8. Select a document type. Click the down arrow and enter SMChange.
9. Click **Next**.
10. Select a file for the preview. Accept the default <path and filename from step 6> and click **Next**.
11. Select the appropriate delimiter.
12. Enter the information on the screen:
    - Write the column headers: checked
    - Do not generate errors if a line contains... : checked
    - Number of skipped lines: 0
    - Quote character: "
    - Start of the comment line: //
    - Escape character: /
13. Click **Next**.
14. Enter the column names and type: for example, Change Number - text
15. Click **Finish** to create and save the connector.
16. Now create a mapping between this text file and the Service Manager Web Service by connecting the two.
17. Click the first text connector and click **Produce Now** (the F5 button) to fill information from the cm3r file in Service Manager into a delimited text file via Web Services.

Example: Getting change information from another Service Manager system

This example retrieves change information from the cm3r file and uses the Web Service to create a new change.

1. Click **Tailoring** > **Web Services** > **Run WSDL to JS**.
2. In the “Please enter the URL for the WSDL file” field, enter the following:

   http://<serenade>:<port number>/sc62server/PWS/ChangeManagement.wsdl
3. Click **Proceed** and then **Add** to create the ChangeManagement JavaScript record in the ScriptLibrary.

   Note

   Service Manager always uses the user name and password you provided to access the remote Web Service unless you override the values at run time. For example, create custom JavaScript to use the currently logged in user's credentials instead of the user name and password you provided to access the remote Web Service.

```
        function ChangeManagement( )
        {
        this.location = new String(
"http://hostname:13080/sc62server/ws" );
        this.user = null;
        this.password = null;
        this.connectTimeOut = 10;
        this.sendTimeOut = 10;
        this.recvTimeOut = 10;
        this.soapEnvelope = null;
        this.soapBody = null;
        this.soapHeader = null;
        this.attachments = new Array();
        this.resultXML = null;
        this.invoke = invoke;
        […]
```

   Note

   The namespace specified in “this.location” doesn't have to be resolvable.

   The user can be filled in here (this.user), or in the invoking script. As a best practice, fill in the user in the invoking script.
4. Now that the Script has been added to the ScriptLibrary, you need to write another JavaScript that can be used to retrieve the change or create a new change. Enter the JavaScript code as listed below:

```
function RetrieveChangeKeysList(query)
{
try
{
var ChangeMgmtService=new system.library.ChangeManagement.
ChangeManagement();
////////////////////////////////////////////////////////
//              set Connection information (optional)
////////////////////////////////////////////////////////
ChangeMgmtService.user = "falcon";
ChangeMgmtService.password = "";
////////////////////////////////////////////////////////
//      create the request object
////////////////////////////////////////////////////////
var RetrieveChangeListRequest = new system.library.
ChangeManagement.RetrieveChangeKeysListRequest();
////////////////////////////////////////////////////////
//              Request Data Fill Section
////////////////////////////////////////////////////////////////////
/////
if (query!=null)
{
RetrieveChangeListRequest.model.instance.query=query;
}
else
print("Please enter a valid Query statement.")
////////////////////////////////////////////////////////////////////
/////
//     Invoke and final processing
////////////////////////////////////////////////////////////////////
/////
var RetrieveChangesResponse = ChangeMgmtService.invoke(
RetrieveChangeListRequest );

if ( RetrieveChangesResponse.isFault() )
{
print( RetrieveChangesResponse.messages.getContent() );
return(-1);
}
else
{
print("Success")
print(RetrieveChangesResponse.keys[0].ChangeNumber.getValue())
print(RetrieveChangesResponse.keys[1].ChangeNumber.getValue())
```

```
return(RetrieveChangesResponse.keys);
}

}
catch( e )
{
print( e );
}

}
function RetrieveChange(number)
{
try
{
var ChangeMgmtService=new
system.library.ChangeManagement.ChangeManagement();
////////////////////////////////////////////////////////////////
/////
//              set Connection information (optional)
////////////////////////////////////////////////////////////////
/////
ChangeMgmtService.user = "falcon";
ChangeMgmtService.password = "";

////////////////////////////////////////////////////////////////
/////
//              create the request object
////////////////////////////////////////////////////////////////
/////
var RetrieveChangeRequest = new system.library.ChangeManagement.
```

```
RetrieveChangeRequest();
////////////////////////////////////////////////////////////////////
/////
//     Request Data Fill Section
////////////////////////////////////////////////////////////////////
/////
if (number!=null)
{
RetrieveChangeRequest.model.instance.header.ChangeNumber.setValue(n
umber);
}
else
print("Please pass in a valid Change Number.");
////////////////////////////////////////////////////////////////////
/////
//     Invoke and final processing
////////////////////////////////////////////////////////////////////
/////
var RetrieveChangeResponse = ChangeMgmtService.invoke(
RetrieveChangeRequest );

if ( RetrieveChangeResponse.isFault() )
{
print( RetrieveChangeResponse.messages.getContent() );
return(-1);
}
else
{
print("Success")
return(RetrieveChangeResponse.model.instance);
}

}
catch( e )
{
print( e );
}

}
```

5. Create a JavaScript in the same ScriptLibrary record that reads a change record and then creates a new record in the other Service Manager system. Enter the Java Script shown below:

```
function CreateChangeFromChange(change)
 {
        try
        {
                var ChangeMgmtService=new
system.library.ChangeManagement.
                ChangeManagement();


//////////////////////////////////////////////////////////////////////
//////
 //             set Connection information (optional)

//////////////////////////////////////////////////////////////////////
//////
                ChangeMgmtService.user = "falcon";
                ChangeMgmtService.password = "";


//////////////////////////////////////////////////////////////////////
//////
 //             create the request object

//////////////////////////////////////////////////////////////////////
//////
                var CreateChangeRequest = new
system.library.ChangeManagement.
                CreateChangeRequest();



//////////////////////////////////////////////////////////////////////
//
 //     Request Data Fill Section

//////////////////////////////////////////////////////////////////////
//

                if (change!=null)
                {

CreateChangeRequest.model.instance.header.Category.

setValue(change.header.Category.getValue());
```

```
CreateChangeRequest.model.instance.header.RequestedBy.

setValue(change.header.RequestedBy.getValue());

CreateChangeRequest.model.instance.header.Reason.
                        setValue(change.header.Reason.getValue());

CreateChangeRequest.model.instance.header.Coordinator.

setValue(change.header.Coordinator.getValue());

CreateChangeRequest.model.instance.InitialAssessment.

setValue(change.InitialAssessment.getValue());
                        CreateChangeRequest.model.instance.Urgency.
                        setValue(change.Urgency.getValue());

CreateChangeRequest.model.instance.ReleaseType.
                        setValue(change.ReleaseType.getValue());

                        for (var i=0;
i<change.description_structure.
                        Description.Description.length; i++)
                        {

CreateChangeRequest.model.instance.description_structure.
                        Description.Description_newInstance().

setValue(change.description_structure.Description.
                        Description[i].getValue());
                        }
                }
                else
                        print("Please pass in a valid Change
Request.");



////////////////////////////////////////////////////////////////
///
 //     Invoke and final processing

////////////////////////////////////////////////////////////////
///
```

```
                var CreateChangeResponse = ChangeMgmtService.
                invoke( CreateChangeRequest );

                if ( CreateChangeResponse.isFault() )
                {
            print( CreateChangeResponse.messages.getContent() );
            return(-1);
                }
                else
                {
                        print("Success")

return(CreateChangeResponse.model.instance.header.
                        ChangeNumber.getValue());
                }

        }
        catch( e )
        {
                print( e );
        }
 }


////////////////////////////////////////////////////////////////////
////
 // Test Call

////////////////////////////////////////////////////////////////////
////
 var rc_Code_List=RetrieveChangeKeysList("category=\"" + "Release
Management"
         + "\"" + "and number <=\"" + "C10027" + "\"");

 //var rc_Code_List=RetrieveChangeKeysList("number>=\"" + "C10026"
+ "\"");

 if (rc_Code_List != -1)
 {
        for (var i = 0; i < rc_Code_List.length; i+=1)
        {
                var
rc_Code=RetrieveChange(rc_Code_List[i].ChangeNumber.
                getValue());
                var rc_Create=CreateChangeFromChange(rc_Code);
```

```
        }
}
```

Example to close an existing incident record

To run an example of closing an existing Incident Record from the Axis2Sample directory, follow these steps:

1. Follow the instructions in the Axis README.txt file located in the Axis2Sample\resources directory.
2. Enter the following in the DOS command prompt to close an incident:

```
C:\scs\sm920\server\webservices\sample\sm7webservices\
Axis2Sample\bin>CloseIncidentSample -host <sm server>
-port <sm port> -username falcon -incidentId IM10001
-closeCode Fault -resolution "It works now"
```



Note

If the username and password aren't entered, they default to “falcon” with no password.

1.1.1.4.2.14. Special considerations for using Keep-Alive with Service Manager

A Service Manager user session starts when the Service Manager server receives the first request from a SOAP client and ends when the SOAP client closes the HTTP connection. The user login process is performed in the first SOAP client request and the user logout process is performed when the SOAP client ends this Service Manager session. A SOAP client can reduce the login and logout overhead by enabling HTTP persistent connections, also called HTTP keep-alive. If you want to use HTTP 1.1 Keep-Alive connections with a SOAP API client, the SOAP API client must support cookies. When Service Manager responds to the first POST request from the SOAP API client, it includes a Set Cookie header that conveys the servlet container sessionid to the client.

- Configure the SOAP stack with which the SOAP API client is written to support cookies. Axis and .NET can both be configured to do this.
- If the SOAP toolkit supports HTTP 1.1 Keep-Alive but not cookies, you can arrange for the application to echo back the JSESSIONID value in a Cookie header by adding code to the client application to manually create the HTTP header on the second and subsequent requests



Note

- In HTTP/1.1, persistent connections are the default behavior of any connection.
- If you use HTTP 1.0 you have to manually set the HTTP header “connection” to keep-alive.
- A SOAP client ends a Service Manager session by sending a request with the HTTP header “connection” set to “close”. If a close request is never received by the Service Manager server then this session is terminated by the Service Manager server when the webservices_sessiontimeout time is reached.

Keep-Alive example for Service Manager

The following shows an example of the code for using Keep-Alive with Service Manager.



Note

In `Keep-Alive: timeout=60`, the unit is second.

```
================================================================
client request:
POST /SM/9/ws HTTP/1.1
accept: application/xop+xml, text/xml image/gif, image/jpeg, *; q=.2,
*/*; q=.2
authorization: Basic ZmFsY29uOg==
soapaction: "Create"
connection: Keep-Alive
.....
SM server response:
HTTP/1.1 200
Set-Cookie: JSESSIONID=ED61093038F9FF8CE9CF44E34C9366AC; Path=/SM
Keep-Alive: timeout=60, max=100
Connection: Keep-Alive
Content-Type: text/xml;charset=utf-8
Content-Length: 2112
.....
```

Then you need to echo back the JSESSIONID for each client use the following:

```
POST /SM/9/ws HTTP/1.1
accept: application/xop+xml, text/xml image/gif, image/jpeg, *; q=.2,
*/*; q=.2
authorization: Basic ZmFsY29uOg==
soapaction: "Retrieve"
connection: Keep-Alive
cookie: JSESSIONID=ED61093038F9FF8CE9CF44E34C9366AC; Path=/SM;
....
================================================================
```



Note

If you use Axis2 to implement the client then Axis2 can maintain the session by calling the setManageSession(true).

There is an Axis2 example in SM 9.x installation directory:

(...\Server\webservices\sample\sm9webservices\Axis2Sample)

1.1.1.4.2.15. Use SSL to consume Service Manager Web Services

Provide the client keystore and the client keystore password to the Web Service consumer. This consumer will need to enter this information into the proper location on the client. For example, in SOAPUI, you can enter this information in the **File** > **Preferences** > **SSL Settings** section. To find out how to create the client keystore, refer to the Service Manager Trusted Sign-on White paper. Ensure that the client's cacerts file contains the information on the authority that signed the keystore.

1.1.1.4.2.16. Attachment handling

What do the following error messages mean?

`Error Message: Warning: incoming add attachment request 1 has no href attribute`

`Error: unable to match incoming add attachment request 1 with no href attribute to an attachment part`

They indicate that the <attachment> elements in the XML in the SOAP requests don't have a href or contentId attribute value. The same value is supposed to be in the MIME message part as the Content-ID: value. In SOAP with attachments, we need a correlation between the XML element/attributes that describe the attachment, and the actual binary or base64 attachment content which is in a MIME message. This correlation is typically a unique ID specified in an href or Content-ID attribute.

The Service Manager server deliberately allows requests that omit the href or contentId and attempts to match up the XML and the attachment parts.

We report the missing href or contentId value with a message in the >sm.log file, as follows:

`Warning: incoming add attachment request 1 has no href attribute`

The server first tries to get an href or contentId value out of the XML; if it succeeds, it finds the associated MIME attachment by looking for a MIME message part whose id has the same value. If there is no href or Content-ID, the server tries to match up the <attachment> element with a particular attachment part. This assumes that there is a one-to-one correspondence between <attachment> elements and attachment parts and uses the index of the DOM node of the <attachment> element as an index into the array of binary attachment parts.

This strategy doesn't work when there are miscellaneous white-space nodes in the DOM document, because the index number of the DOM node for the <attachment> element is greater than it would otherwise be.

Service Manager allows requests with no href or content-id

The reason Service Manager allows requests with no href or Content-ID is that with some tools and toolkits it is difficult to arrange for the unique id of an attachment part to be the same in the XML as in the binary attachment part. Though this is trivial when using .NET, when using Axis, the Java code would generate a unique cid value in the MIME message part dynamically during message serialization. Unless you write code to set up a handler to participate in serialization (via a callback), it is impossible to match the value in the XML to the value in the MIME message part. To prevent these problems, Service Manager:

- Relaxed the schema such that href was not strictly required (use=optional).
- Added an alternative, optional attribute called Content-ID, which is used instead of href when serving responses containing attachments to Axis clients.

- Added code to try to guess the href value that should be present in the XML, if it is missing. If we are processing the Nth <attachment> element (the Nth DOM Node within the set of DOM children for the <attachments> element, where the <attachment> element has neither an href nor a Content-ID attribute), Service Manager tries to look at the attachment part with the same index value to check whether the name, length, and type match. If the number of DOM Node children under <attachments> doesn't match the number of attachment parts, Service Manager can't process the attachment. It prints the following error in the sm.ini file:

  ```
  Error: unable to match incoming add attachment request 2 with no
  href
  attribute to an attachment part
  ```

  This message says "attachment request 2", which seems to be incorrect; because there is only 1 <attachment> element, it should apparently be "attachment request 1." However, the attachment element is the second DOM child node of <attachments> due to the white space text present as DOM child node 1; the first child node of <attachments> is white space that may be ignored.

  The workaround is either don't serialize with pretty-printing (such as adding white space nodes to make the XML easier to read for the human eye) when sending requests to Service Manager, or write code that ensures that requests containing attachment operations have either an href or Content-ID attribute on the <attachment> element.

  Supported attachment types in Service Manager are MIME and MTOM. We often get the question if the consumer doesn't support these attachment types, if the SYSATTACHMENTS file can be exposed to get the attachments out of Service Manager. This is unsupported. The attachments are compressed and cut into <=32K pieces and can't easily be read from an outside source. A workaround that customers use frequently is receiving the attachment with the parent record, for example via a RetrieveIncident request, and then transforming it into base64 and sending to the consuming application where it can be transformed into the required format.

Sample script to send a record with attachments within Service Manager

The sample script below sends a record with attachments from Service Manager to ServiceManager. First, you will need to have a generated JavaScript for both Service Managers: Service Manager 1, called IncidentManagement, and Service Manager 2, called IncidentManagementTarget.

Note

The lines in bold font perform the attachment handling.

```
try
{
        //STEP 1 --> RETRIEVE AN EXISTING INCIDENT HAVING ATTACHMENT(S)
//THIS EXAMPLE USE "IM10010", PLEASE ADD ONE OR MORE ATTACHMENTS TO
THIS INCIDENT IF NO
        var imService = new
lib.IncidentManagement.IncidentManagement();
        var request = new
lib.IncidentManagement.RetrieveIncidentRequest();
        request.model.instance.IncidentID.setValue( "IM10010" );
        request.attachmentInfo = true;
        request.attachmentData = true;
        imService.user = "falcon";
        imService.pwd = "";
        var incidentResp = imService.invoke( request );
        if ( incidentResp.isFault() )
        {
                print ( incidentResp.detail );
        }
        else
        {
                print( "SM OOB Test: Retrieved incident:" +
incidentResp.model.instance.IncidentID.getValue());
        }


       //STEP2 -->CREATE A NEW INCIDENT BASED ON THE RETRIEVED INCIDENT
//USING 'IncidentManagement' WEB SERVICE


        var imService2 = new
lib.IncidentManagementTarget.IncidentManagement();
        var attach
        var newAttach = new Array();
        for (var i=0;
i<incidentResp.model.instance.attachments.attachment.length; i++)
        {
                attach = imService.attachments[i];
                attach.href = "test_"+i;
                attach.action="add";
                newAttach.push( attach );
        }
        var createIM = new
lib.IncidentManagementTarget.CreateIncidentRequest();
```

```
        imService2.setAttachments( newAttach );
        imService2.user = "falcon";
        imService2.pwd = "";
        createIM.attachmentData = true;
        createIM.attachmentInfo = true;


createIM.model.instance.OpenedBy.setValue(incidentResp.model.instance.O
penedBy.getValue());

createIM.model.instance.Title.setValue(incidentResp.model.instance.Titl
e.getValue());

createIM.model.instance.Category.setValue(incidentResp.model.instance.C
ategory.getValue());

createIM.model.instance.Area.setValue(incidentResp.model.instance.Area.
getValue());

createIM.model.instance.Subarea.setValue(incidentResp.model.instance.Su
barea.getValue());

createIM.model.instance.Impact.setValue(incidentResp.model.instance.Imp
act.getValue());

createIM.model.instance.Urgency.setValue(incidentResp.model.instance.Ur
gency.getValue());

createIM.model.instance.AssignmentGroup.setValue(incidentResp.model.ins
tance.AssignmentGroup.getValue());

createIM.model.instance.Service.setValue(incidentResp.model.instance.Se
rvice.getValue());

        for (var i=0;i<newAttach.length;i++)
        {
                var attachmentXml    =
createIM.model.instance.attachments.attachment_newInstance();
                attachmentXml.action = newAttach[i].action;
                attachmentXml.name   = newAttach[i].name;
                attachmentXml.type   = newAttach[i].type;
                attachmentXml.len    = newAttach[i].value.length;
                attachmentXml.attachmentType = attach.attachmentType;
        }
        for (var
```

```
i=0;i<incidentResp.model.instance.Description.Description.length; i++)
        {
                newDescriptionLine =
createIM.model.instance.Description.Description_newInstance();

newDescriptionLine.setValue(incidentResp.model.instance.Description.Des
cription[i].getValue());
        }


        response = imService2.invoke(createIM);
        if ( response.isFault() )
        {
                print ( "SM OOB Test: Outgoing request failed with
error:" + response.detail );
        }
        else
        {
                if ( response.messages.message.length; i++ )
                {
                        print ( "SM OOB Test: Outgoing request returned
message:"
                                        +
response.messages.message[i].getValue() );
                }
        }
print("Incident"  + incidentResp.model.instance.IncidentID.getValue()
             + "was retrieved from" + imService.getHost() + ":" +
imService.getPort()
             + "and copied to new incident "
             + response.model.instance.IncidentID.getValue()
             + "on" + imService2.getHost() + ":" +
imService2.getPort());
}
catch( e )
{
        print( "SM OOB Test: Outgoing request returned message: "
                             + response.messages.message[i].getValue() );
);
}
```

Attachment operations using the SOAP API

Service Manager supports all typical attachment operations using its web services (SOAP API) or REST API. These operations include: get attachment information, get (single) attachment,

delete (single) attachment, insert an attachment, and so on.

The following topics describe how to perform typical attachment operations by using the web services (SOAP) API. It provides information about the services published by Service Manager, and explains requests of an external application being sent to Service Manager, as well as the responses to expect.

Tip

This topic describes attachment operations that you can perform using the Service Manager web services (SOAP API). For information about attachment operations using the REST API, see Attachment operations using the RESTful API.

When writing Web services requests, the WSDL files published by Service Manager on the published services contain the API information. These files are auto-generated from the **extaccess** dbdict. They can be displayed in a browser using a URL like any one of the following:

- http://localhost:13080/sc62server/PWS/IncidentManagement.wsdl
- http://localhost:13080/SM/9/IncidentManagement.wsdl

Note

When the sc62server URL is used, the attachment format is MIME; when the SM/9 URL is used, the attachment format is MTOM.

Tip

Testing web services with tools is quite easy. To avoid unnecessary complexity, the SoapUI tool is used for the example below.

Both the request and response tag names include the operation name (for example, Retrieve or Update), and the target object name (for example, Incident) in namespace http://schemas.hp.com/SM/9.

The following is an example of a request tag (the prefix **ns** refers to the namespace definition in the <soapenv> tag):

```
<ns:RetrieveIncidentRequest>
```

The following is an example of a response tag:

```
<RetrieveIncidentResponse>
```

The operation is also reflected in the HTTP header property named SOAPAction. For example:

```
SOAPAction: "Update"
```

If the request tag and the SOAPAction header don't match, the response shows an error message:

```
<faultstring>A CXmlApiException was raised in native code : error 16 :
scxmlapi(16) - Invalid or missing file name in XML
request</faultstring>
```

Retrieve attachment information

Set up a **Retrieve** request and set the **attachmentInfo** attribute in the request tag to **true**.

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ns="http://schemas.hp.com/SM/9"
xmlns:com="http://schemas.hp.com/SM/9/Common"
xmlns:xm="http://www.w3.org/2005/05/xmlmime">
<soapenv:Header/>
<soapenv:Body>
<ns:RetrieveIncidentRequest attachmentInfo="true">
<ns:model>
<ns:keys><ns:IncidentID>IM10003</ns:IncidentID></ns:keys>
<ns:instance></ns:instance>
</ns:model>
</ns:RetrieveIncidentRequest>
</soapenv:Body>
</soapenv:Envelope>
```

The response contains the complete Incident data (shortened in the example below) and the attachment information:

```
<SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
<SOAP-ENV:Body>
<RetrieveIncidentResponse message="Success" returnCode="0"
schemaRevisionDate="2016-05-10" schemaRevisionLevel="0"
status="SUCCESS"
xsi:schemaLocation="http://servicecenter.peregrine.com/PWS
http://bra03sm25.eu.hpecorp.net:13080/sc62server/Incident.xsd"
xmlns="http://servicecenter.peregrine.com/PWS"
xmlns:cmn="http://servicecenter.peregrine.com/PWS/Common"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<model>
<keys>
<IncidentID type="String">IM10003</IncidentID>
</keys>
<instance recordid="IM10003 - Not able to print"
uniquequery="number=&quot;IM10003&quot;">
<IncidentID type="String">IM10003</IncidentID>
[..]
<attachments>
<cmn:attachment href="cid:551bb5d00003a005805a1278" len="19779"
name="cwc_labels_ru.utf8" type="application/octet-stream"
upload.by="falcon" upload.date="04/01/15 03:09:36"/>
<cmn:attachment href="cid:551bb5d00008c006805a1278" len="12842"
name="cwc_labels_en.properties" type="application/octet-stream"
upload.by="falcon" upload.date="04/01/15 03:09:36"/>
<cmn:attachment href="cid:551bb5d0000b8007805a1278" len="14606"
name="cwc_labels_de.utf8" type="application/octet-stream"
upload.by="falcon" upload.date="04/01/15 03:09:36"/>
<cmn:attachment href="cid:5668051d000e901f808612a0" len="21579"
name="cimultirel.unl" type="application/octet-stream"
upload.by="falcon" upload.date="12/09/15 03:40:29"/>
<cmn:attachment href="cid:566808350013605b809bd6c0" len="27944"
name="cimultirel_v2.1.unl" type="application/octet-stream"
upload.by="falcon" upload.date="12/09/15 03:53:41"/>
</attachments>
</instance>
</model>
</RetrieveIncidentResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

In the attachment information section, the **href** attribute contains the content id for each attachment.

> **Note**
>
> When the **attachmentData** attribute in the request tag is set to **true**, the response contains all file attachments. If **attachmentData** is set to **true**, the attachment information is always included in the response, regardless of the **attachmentInfo** attribute.

Retrieve a single attachment

In the attachments tag, specify the attachment using the **href** attribute and set the attribute action to **get**. In the request tag, the **attachmentData** attribute must exist and must be set to **true**.

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ns="http://schemas.hp.com/SM/9"
xmlns:com="http://schemas.hp.com/SM/9/Common"
xmlns:xm="http://www.w3.org/2005/05/xmlmime">
<soapenv:Header/>
<soapenv:Body>
<ns:RetrieveIncidentRequest attachmentData="true">
<ns:model>
<ns:keys><ns:IncidentID>IM10003</ns:IncidentID></ns:keys>
<ns:instance>
<attachments>
<attachment href="cid:551bb5d00003a005805a1278" action="get"/>
</attachments>
</ns:instance>
</ns:model>
</ns:RetrieveIncidentRequest>
</soapenv:Body>
</soapenv:Envelope>
```

Now the attachment appears below the XML response in Soap UI.

The raw message showing the file contents is put into a separate HTTP message part:

Note: In `Keep-Alive: timeout=60`, the unit is second.

```
HTTP/1.1 200 OK
Server: Apache-Coyote/1.1
Set-Cookie: JSESSIONID=7A8FB01D34C0CF736842A48724DF3FB5;
Path=/sc62server/; HttpOnly
Keep-Alive: timeout=60, max=100
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: multipart/related; boundary="----
=_Part_56_30479565.1495112821461"; type="text/xml"
Transfer-Encoding: chunked
Date: Thu, 18 May 2017 13:07:01 GMT
------=_Part_56_30479565.1495112821461
Content-Type: text/xml; charset=utf-8
<SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/"><SOAP-ENV:Body>
<RetrieveIncidentResponse message="Success" returnCode="0"
[..]
------=_Part_56_30479565.1495112821461
Content-Type: application/octet-stream
Content-Location: cwc_labels_ru.utf8
Content-ID: cid:551bb5d00003a005805a1278
# <!-- (C) Copyright Hewlett-Packard Company, L.P. All Rights Reserved.
-->
AboutPage.Title=Ðž Ð¿Ñ€Ð¾Ð³Ñ€Ð°Ð¼Ð¼Ð½Ð¾Ð¼ Ð¾Ð±ÐµÑ‘Ð¿ÐµÑ‡ÐµÐ½Ð¸Ð¸ HP
[..]
```

Delete an attachment

To remove an attachment, set the action for the attachment to **remove**.

Any manipulation of an attached file requires a successful update to the target record, and therefore all validations must be passed. For this reason, the example request updates some incidents field as well.

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ns="http://schemas.hp.com/SM/9"
xmlns:com="http://schemas.hp.com/SM/9/Common"
xmlns:xm="http://www.w3.org/2005/05/xmlmime">
<soapenv:Header/>
<soapenv:Body>
<ns:UpdateIncidentRequest attachmentInfo="true" attachmentData="false">
<ns:model>
<ns:keys>
<ns:IncidentID>IM10003</ns:IncidentID>
</ns:keys>
<ns:instance>
<Category>incident</Category>
<Area>hardware</Area>
<Subarea>hardware failure</Subarea>
<JournalUpdates>
<JournalUpdates>Drop attachment</JournalUpdates>
</JournalUpdates>
<ns:attachments>
<com:attachment href="cid:551bb5d00003a005805a1278" action="remove"/>
</ns:attachments>
</ns:instance>
</ns:model>
</ns:UpdateIncidentRequest>
</soapenv:Body>
</soapenv:Envelope>
```

Attach a new file to an incident

Attaching a file also uses the **attachments** tag. The action attribute is set to **add**. The **href** attribute refers to the HTTP message of the request itself. Therefore, the **href** attribute received in the response may differ from that.

The file name for the attachment is set by the **name** attribute, and the content type by the **type** attribute. Therefore, the actual file name on the source system is not sent in the request.

An **update** request looks like the following:

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ns="http://schemas.hp.com/SM/9"
xmlns:com="http://schemas.hp.com/SM/9/Common"
xmlns:xm="http://www.w3.org/2005/05/xmlmime">
<soapenv:Header/>
<soapenv:Body>
<ns:UpdateIncidentRequest attachmentInfo="true">
<ns:model>
<ns:keys>
<ns:IncidentID>IM10003</ns:IncidentID>
</ns:keys>
<ns:instance>
<Category>incident</Category>
<Area>hardware</Area>
<Subarea>hardware failure</Subarea>
<JournalUpdates>
<JournalUpdates>Attach a file</JournalUpdates>
</JournalUpdates>
<ns:attachments>
<com:attachment href="cid:soapui.log" name="soapui.log"
type="application/octet-stream" action="add"/>
</ns:attachments>
</ns:instance>
</ns:model>
</ns:UpdateIncidentRequest>
</soapenv:Body>
</soapenv:Envelope>
```

Then, you receive the following attachment information in the response, which lists both the previously existing attachments and the new file attachment:

```
[..]
<attachments>
<cmn:attachment href="cid:551bb5d00008c006805a1278" len="12842"
name="cwc_labels_en.properties" type="application/octet-stream"
upload.by="falcon" upload.date="04/01/15 03:09:36"/>
<cmn:attachment href="cid:551bb5d0000b8007805a1278" len="14606"
name="cwc_labels_de.utf8" type="application/octet-stream"
upload.by="falcon" upload.date="04/01/15 03:09:36"/>
<cmn:attachment href="cid:5668051d000e901f808612a0" len="21579"
name="cimultirel.unl" type="application/octet-stream"
upload.by="falcon" upload.date="12/09/15 03:40:29"/>
<cmn:attachment href="cid:566808350013605b809bd6c0" len="27944"
name="cimultirel_v2.1.unl" type="application/octet-stream"
upload.by="falcon" upload.date="12/09/15 03:53:41"/>
<cmn:attachment href="cid:591da42c0029800280680d00" len="79002"
name="soapui.log" type="application/octet-stream" upload.by="falcon"
upload.date="05/18/17 07:39:56"/>
</attachments>
[..]
```

1.1.1.4.3. Consume an external Web Service

You can configure Service Manager to connect to and exchange information with remote Web Services. This functionality allows Service Manager to act as a client to other servers that publish Web Services. Service Manager uses JavaScript™ to create and format the proper SOAP messages.



Note

For a production application that needs services that aren't available within your corporate intranet (such as postal address verification, email address verification, and currency conversions) we recommend that you investigate offerings from established for-fee Web Services vendors. Although there are a lot of free and demo Web Services, we don't recommend basing a production application on such services, since availability of the service is not guaranteed. Several Web sites such as www.xmethods.net publish lists of available free and fee-based Web Services. (Be sure to click the full list button to see the complete list of Web Services.)

1.1.1.4.3.1. Use the WSDL2JS utility

The WSDL2JS (Web Services description language to JavaScript) utility translates the operations and types in the WSDL into objects, methods and functions in JavaScript that can be called from another JavaScript record.

The WSDL2JS utility is a JavaScript script library record named SOAP. It is written based on the W3C specifications for WSDL to interpret the content of the WSDL.

To consume a Web Service from Service Manager perform the following steps.

1. Obtain the URL to the Web Service's WSDL file.
2. Examine the WSDL either as a text file, or using a third-party graphical WSDL analysis tool to determine what functions, inputs, and formats the Web service expects. Some third-party Web Services tools allow you to experiment interactively with Web Services. We recommend that you familiarize yourself with the Web Service using such a tool before beginning any Service Manager JavaScript work.
3. Execute the Run WSDL to JS wizard to obtain and convert the Web service's WSDL into JavaScript.
4. Verify the WSDL to JS output. For details, see *Verify the WSDL to JS output* below.
5. Review and modify the timeout values in the JavaScript generated by the utility according to your integration needs. For example, you may want to increase the timeout values as needed.
6. Write custom JavaScript to call the JavaScript functions generated by the WSDL to JS wizard. These functions will enable you to create and send the SOAP messages required to interact with the Web service. We recommend that you write a short “standalone” script and invoke it from the Script Library utility to test it prior to implementing the JavaScript call from Format Control, Triggers, or Display Options. After you have determined and debugged the JavaScript code required to invoke the service, you can then integrate the script with your Service Manager application.
7. Tailor your Service Manager application to invoke your custom JavaScript when you want to connect to a remote a Web service. Usually Web Services are invoked from the Document Engine, Format Control, Links, Display application, or from similar tailoring tools.

Verify the WSDL to JS output

Generated JavaScript must end with

```
// Ensure that material in lib.SOAP is available
lib.SOAP.init();
/// End ----------------
```

All defined types and operations must be represented by a function such as

```
this.SOAPOperations[ "UpdateIncident" ] = new soap_Operation(
"UpdateIncident", "Update", "document", "UpdateIncidentRequest",
"UpdateIncidentResponse" );
function UpdateIncidentRequest( )
```

Or -

```
this.ProductType= new StringType();
•<code>functionStringType( val)
```

If any of these definitions are missing, report this to customer support with an unload of the generated JavaScript, the WSDL in text format, and all imported xsd files.

1.1.1.4.3.2. Best practices for writing a JavaScript to consume a Web Service

Never modify the JavaScript that's automatically generated by WSDL2JS unless you are specifically instructed to do so by Service Manager Customer Support. To invoke the Web service, write a JavaScript record that calls the functions generated by WSDL2JS. The JavaScript that invokes an external Web service should perform the following tasks:

1. Create the Service Object.
2. Create the Request Object.
3. Fill the Request Object with information that defines the request.
4. Invoke the Service Object and pass in the Request Object.
5. Return either the Response Object, an instance of the Response Object, or a specific value of that instance.
6. Perform error handling to test each response. Use try {...}, throw {...}, catch {...}, and the isFault function.

As a best practice, don't reuse the names of variables and functions in the calling JavaScript that are the names of variables and functions in the generated script. This can help avoid confusion.



Caution

Never use the “new” keyword on a subordinate object unless it is an array. Unlike conventionally compiled applications that invoke a Web service, the generated function objects described in this document already use “new” when instantiating all children, so it is unnecessary to do so in the calling JavaScript. The only exception is for arrays, where you use the newInstance() function to generate the array and fill its elements.

1.1.1.4.3.3. Date/Time handling

The self-written JavaScript is responsible for correct formatting of xml schema dateTime fields. The WSDL2JS-generated functions don't reformat the values assigned to them to convert them into the correct format. If a field in an outgoing SOAP message is defined as a dateTime, and the script writer assigns a value to the field, it needs to be a valid XML Schema dateTime, duration or other date/time string value. It can't be a Service Manager datetime string nor should it be a JavaScript dateTime string. To get the valid XML Schema date/time, the script writer should use the XMLDate global object. For example:

```
// get today’s date/time from Javascript Date() object and store in a
new XMLDate object
var xmlDt = new XMLDate( new Date() );
// coerce the datetime value stored in the XMLDate object to ISO/XML
schema format
print( xmlDt.getISODateTimeString() );
```

There are a variety of methods for the XMLDate object that you can look up in the white paper with the title of *JavaScript Programmers Guide.pdf*.

The constructor for XMLDate can handle several different formats. You can pass it a string, a number of milliseconds, or a JS Date object (as in the example above).

1.1.1.4.3.4. Example: Interface to another system

This is an example for interfacing the Service Manager with PPM Demand service via Web Services.

When writing an interface to a different system, it is very important to understand the data structure and the methods available, as well as understand how to interpret the generated JavaScript code. In this example, we will not only discuss the methods and fields published by the WSDL, but reading the generated code to successfully write the invoking code as well.

1. Determine the correct URL to enter into the WSDL to JS tool, check with the PPM administrator.
2. In Service Manager, go to Menu Navigator and click **Utilities** > **Tools** > **Web Services**. Click **Run WSDL to JS**.
3. Enter the URL to the WSDL, such as http://<hostname>:8080/itg/ppmservices/DemandService?wsdl.
4. Click **Proceed**.
5. Click **Add** to add the new ScriptLibrary record called DemandService.
6. Write an interfacing JavaScript record in the ScriptLibrary called DemandServiceInvoke.

Generated JavaScript interfaces

This section helps provide a general understanding of how the generated JavaScript interfaces with the invoking JavaScript. As a best practice, find the proper objects and methods using a tool such as SoapUI and test the Web Service there prior to writing the invoking script. It should not be necessary to interpret the generated code when taking that approach.

Check these sections in the “master” JavaScript to write the calling JavaScript:

The first line in the master code gives the name of the main function or Service Object to call in the calling JavaScript:

```
function DemandService( )
{
this.location = new String( "http://<ppm server>:15000/itg/
ppmservices/DemandService" );
```

Create a request for a new project

Find the function that creates the desired result; in this case create a request for a new project:

```
this.SOAPOperations[ "createRequest" ] = new soap_Operation
   ( "createRequest", "urn:createRequest", "document", "createRequest",
   "createRequestResponse" );
 …
 function createRequest(  )
 {
this.$$nsPrefix = "ns1";
this.$$elementFormDefault = "qualified";
this.$$attributes = new Array();
this.$$xmlNames = new Array();
this.$$objNames = new Array();
this.$$minOccurs = new Array();
this.$$refs = new Array();
this.getName = getName;
this.getXmlName = getXmlName;
this.setContent = setContent;
this.addContent = addContent;
this.getContent = getContent;
this.isFault = isFault;
this.$$elementChildren = new Array();
this.$$name = "createRequest";
this.$$xmlNames[ "createRequest" ] = "createRequest";
this.xmlns_ns1 = new String("http://mercury.com/ppm/dm/service/1.0");
this.$$attributes.push( "xmlns_ns1" );
this.$$xmlNames["xmlns_ns1"] = "xmlns:ns1";
this.$$objNames["xmlns:ns1"] = "xmlns_ns1";
this.requestObj = new Request();
this.$$elementChildren.push( "requestObj" );
}
```

The structure of the request

The bold $$elementChildren.push section shows that the requestObj Child element is of type Request(). To find the structure of the Request, find the definition of that function in the generated code.

```
function Request(  )
{
this.$$nsPrefix = "ns8";
this.$$elementFormDefault = "qualified";
this.$$attributes = new Array();
this.$$xmlNames = new Array();
this.$$objNames = new Array();
this.$$minOccurs = new Array();
this.$$refs = new Array();
this.getName = getName;
this.getXmlName = getXmlName;
this.setContent = setContent;
this.addContent = addContent;
this.getContent = getContent;
this.isFault = isFault;
this.$$elementChildren = new Array();
this.$$name = "Request";
this.$$xmlNames[ "Request" ] = "Request";
this.$$minOccurs[ "id" ] = 0;
this.id = new xs_string();
this.$$elementChildren.push( "id" );
this.requestType = new xs_string();
this.$$elementChildren.push( "requestType" );
this.simpleFields = new Array();  // of SimpleField
this.simpleFields.$$nsPrefix = "ns8"
this.simpleFields_newInstance = function()
{
var newLen = this.simpleFields.push( new SimpleField() );
return this.simpleFields[ newLen-1 ];
}
this.$$elementChildren.push( "simpleFields" );
this.tables = new Array();  // of Table
this.tables.$$nsPrefix = "ns8"
this.tables_newInstance = function()
{
var newLen = this.tables.push( new Table() );
return this.tables[ newLen-1 ];
}
this.$$elementChildren.push( "tables" );
this.notes = new Array();  // of Note
this.notes.$$nsPrefix = "ns8"
this.notes_newInstance = function()
{
var newLen = this.notes.push( new Note() );
```

```
return this.notes[ newLen-1 ];
}
this.$$elementChildren.push( "notes" );
this.fieldChangeNotes = new Array();  // of FieldChangeNote
this.fieldChangeNotes.$$nsPrefix = "ns8"
this.fieldChangeNotes_newInstance = function()
{
var newLen = this.fieldChangeNotes.push( new FieldChangeNote() );
return this.fieldChangeNotes[ newLen-1 ];
}
this.$$elementChildren.push( "fieldChangeNotes" );
this.URLReferences = new Array();  // of URLReference
this.URLReferences.$$nsPrefix = "ns8"
this.URLReferences_newInstance = function()
{
var newLen = this.URLReferences.push( new URLReference() );
return this.URLReferences[ newLen-1 ];
}
this.$$elementChildren.push( "URLReferences" );
this.remoteReferences = new Array();  // of RemoteReference
this.remoteReferences.$$nsPrefix = "ns8"
this.remoteReferences_newInstance = function()
{
var newLen = this.remoteReferences.push( new RemoteReference() );
return this.remoteReferences[ newLen-1 ];
}
this.$$elementChildren.push( "remoteReferences" );
}
```

Request object

The Request object can contain simple Fields, Notes, Field Change Notes, Tables etc. This example examines these fields.

```
function SimpleField(  )
{
this.$$nsPrefix = "ns8";
this.$$elementFormDefault = "qualified";
this.$$attributes = new Array();
this.$$xmlNames = new Array();
this.$$objNames = new Array();
this.$$minOccurs = new Array();
this.$$refs = new Array();
this.getName = getName;
this.getXmlName = getXmlName;
this.setContent = setContent;
this.addContent = addContent;
this.getContent = getContent;
this.isFault = isFault;
this.$$elementChildren = new Array();
this.$$name = "SimpleField";
this.$$xmlNames[ "SimpleField" ] = "SimpleField";
this.token = new xs_string();
this.token.$$nsPrefix = "ns7"
this.$$refs[ "token" ] = true;
this.$$elementChildren.push( "token" );
this.stringValue = new Array();  // of xs_string
this.stringValue.$$nsPrefix = "ns8"
this.stringValue_newInstance = function()
{
var newLen = this.stringValue.push( new xs_string() );
return this.stringValue[ newLen-1 ];
}
this.$$elementChildren.push( "stringValue" );
this.$$minOccurs[ "dateValue" ] = 0;
this.dateValue = new xs_dateTime();
this.$$elementChildren.push( "dateValue" );
}
```

Simple fields

Simple fields consist of tokens (of type xs_string), as well as instances of string Values (where each element is of type xs_string).

Check the xs_string() function

When checking the xs_string() function, you will find that the JavaScript uses the setValue function to fill the elements with data.

```
function xs_string( val )
{
this.$$nsPrefix = "xsd";
this.$$elementFormDefault = "qualified";
this.$$attributes = new Array();
this.$$xmlNames = new Array();
this.$$objNames = new Array();
this.$$minOccurs = new Array();
this.$$refs = new Array();
this.getValue = getValue;
this.setValue = setValue;
this.$$value = val;
this.xsi_type = new String("xsd:string");
this.$$attributes.push( "xsi_type" );
this.$$xmlNames["xsi_type"] = "xsi:type";
this.$$objNames["xsi:type"] = "xsi_type";
}
function setValue( value )
{
this.$$value = value;
}
```

Check expected parameters in invoke() function

Check which parameters the invoke() function expects.

```
function invoke( requestObj, headerObj, bEmitXsiTypeAttributes )
```

In this case, the headerObj and the bEmitXsiTypeAttributes are optional. They are “nullsub’ed” in the JavaScript code), so the requestObj is the only required argument.

Check the syntax for the Response function

```
function createRequestResponse(  )
{
  this.$$nsPrefix = "ns1";
  this.$$elementFormDefault = "qualified";
  this.$$attributes = new Array();
  this.$$xmlNames = new Array();
  this.$$objNames = new Array();
  this.$$minOccurs = new Array();
  this.$$refs = new Array();
  this.getName = getName;
  this.getXmlName = getXmlName;
  this.setContent = setContent;
  this.addContent = addContent;
  this.getContent = getContent;
  this.isFault = isFault;
  this.$$elementChildren = new Array();
  this.$$name = "createRequestResponse";
  this.$$xmlNames[ "createRequestResponse" ] = "createRequestResponse";
  this.$$xmlNames["_return"] = "return";
  this.$$objNames["return"] = "_return";
  this._return = new RemoteReference();
  this.$$elementChildren.push( "_return" );
}
```

Use getValue

Use getValue (or a similarly defined function) to read the result of the request.

```
function getValue( )
{
return this.$$value;
}
```

Write the invoking JavaScript code

Now that the generated JavaScript gave information on the structure of the code to use for invoke, write the invoking JavaScript code. In this case, the invoking code gets passed in information from a Service Catalog Item. This information is then processed and passed through to the PPM Web Service.

```
function CreateDemandRequest(CartItem)
{
        try
        {
////////////////////////////////////////////////////////////////////
//    Initialization Section
////////////////////////////////////////////////////////////////////
//              first, initialize the Service Object for this
JavaScript
                var DemandService=new
system.library.DemandService.DemandService();

//              set Connection information (optional)
                DemandService.user = "admin";
                DemandService.password = "admin";
//              DemandService.location="http://<ppm server>:15000
/itg/ppmservices/DemandService";
                DemandService.location="http://localhost:15001
/itg/ppmservices/DemandService";
//              create the request object
                var RequestDemRequest =
new system.library.DemandService.createRequest();
////////////////////////////////////////////////////////////////////
/
//     Data Fill Section
////////////////////////////////////////////////////////////////////
/
// Data from Cart Item
                var PlannedStart=system.library.xmlFill.
getValue(CartItem.options, "PlannedStart");
                var PlannedEnd=system.library.xmlFill.
getValue(CartItem.options, "PlannedEnd");
                var ProjectName=system.library.xmlFill.
getValue(CartItem.options, "ProjectName");
                var ProjectManager=system.library.xmlFill.getValue
(CartItem.options, "ProjectManager");
                var Region=system.library.xmlFill.getValue
(CartItem.options, "Region");
                var ProjectType=system.library.xmlFill.getValue
(CartItem.options, "ProjectType");
// Fill into Web Request
                RequestDemRequest.requestObj.requestType.setValue
( "PFM - Proposal" );
```

```
RequestDemRequest.requestObj.simpleFields_newInstance();


RequestDemRequest.requestObj.simpleFields[0].token.setValue
("REQ.VP.KNTA_PLAN_START_DATE");
                var
String0=RequestDemRequest.requestObj.simpleFields[0]
.stringValue_newInstance();
               String0.setValue(PlannedStart)

RequestDemRequest.requestObj.simpleFields_newInstance();


RequestDemRequest.requestObj.simpleFields[1].token.setValue
("REQ.VP.KNTA_PLAN_FINISH_DATE");
               var
String1=RequestDemRequest.requestObj.simpleFields[1]
.stringValue_newInstance();
               String1.setValue(PlannedEnd)


RequestDemRequest.requestObj.simpleFields_newInstance();

RequestDemRequest.requestObj.simpleFields[2].token.setValue
("REQ.VP.KNTA_PROJECT_NAME");
               var
String2=RequestDemRequest.requestObj.simpleFields[2]
.stringValue_newInstance();
               String2.setValue(ProjectName)

RequestDemRequest.requestObj.simpleFields_newInstance();
               RequestDemRequest.requestObj.simpleFields[3].token
.setValue("REQ.VP.KNTA_PROJECT_MANAGER");
               var
String3=RequestDemRequest.requestObj.simpleFields[3]
.stringValue_newInstance();
               String3.setValue(ProjectManager)

RequestDemRequest.requestObj.simpleFields_newInstance();
               RequestDemRequest.requestObj.simpleFields[4].token
.setValue("REQ.VP.KNTA_REGION");
               var
String4=RequestDemRequest.requestObj.simpleFields[4]
.stringValue_newInstance();
```

```
                String4.setValue(Region)


RequestDemRequest.requestObj.simpleFields_newInstance();

RequestDemRequest.requestObj.simpleFields[5].token.setValue
("REQ.VP.KNTA_PROJECT_TYPE");
                var
String5=RequestDemRequest.requestObj.simpleFields[5]
.stringValue_newInstance();
                String5.setValue(ProjectType)
//              var ProjectNotes=RequestDemRequest.requestObj.
notes_newInstance();
//              ProjectNotes.content.setValue("notes");
RequestDemRequest.requestObj.simpleFields[4].token.getValue())
////////////////////////////////////////////////////////////////////
//
//     Invoke and final processing
////////////////////////////////////////////////////////////////////
//
                var DemandResponse = DemandService.invoke
( RequestDemRequest );

                if ( DemandResponse.isFault() )
                {
print( DemandResponse.faultcode.getValue() );
print( DemandResponse.faultstring.getValue() );
print( DemandResponse.detail.getValue() );
return("Failure");
                }
                else
                {
                        print("Success")
                        return("Success");
                }

        }
        catch( e )
        {
                print( e );
        }

}
////////////////////////////////////////////////////////////////////
/
```

```
// Test Call
//////////////////////////////////////////////////////////////////
//
//var rc_Code=CreateDemandRequest(CartItem);
```

Determine the structure of the request and response

To determine the structure of the request and response, it is very helpful to look at both the request and response using a tool such as SOAP UI. The PPM WSDL shown below generated the request and response in the next section using SOAP UI.

PPM WSDL:

```
- <wsdl:definitions xmlns:ds="http://mercury.com/ppm/dm/service/1.0"
xmlns:http="http://schemas.xmlsoap.org/wsdl/http/"
xmlns:mime="http://schemas.xmlsoap.org/wsdl/mime/"
xmlns:soap12="http://schemas.xmlsoap.org/wsdl/soap12/"
xmlns:wsaw="http://www.w3.org/2006/05/addressing/wsdl"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:dmMsg="http://mercury.com/ppm/dm/msg/1.0"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
targetNamespace="http://mercury.com/ppm/dm/msg/1.0">
<wsdl:documentation>DemandService</wsdl:documentation>
- <wsdl:types>
- <xs:schema xmlns:dm="http://mercury.com/ppm/dm/1.0"
xmlns:xs="http://www.w3.org/2001/XMLSchema"
xmlns:common="http://mercury.com/ppm/common/1.0"
attributeFormDefault="qualified" elementFormDefault="qualified"
targetNamespace="http://mercury.com/ppm/dm/service/1.0">
<xs:import namespace="http://mercury.com/ppm/dm/1.0"
schemaLocation="DemandService?xsd=xsd0" />
- <xs:element name="createRequest">
- <xs:complexType>
- <xs:sequence>
<xs:element name="requestObj" type="dm:Request" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="createRequestResponse">
- <xs:complexType>
- <xs:sequence>
<xs:element name="return" nillable="true" type="dm:RemoteReference" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="getRequests">
- <xs:complexType>
- <xs:sequence>
<xs:element maxOccurs="unbounded" name="requestIds"
nillable="true" type="dm:Identifier" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="getRequestsResponse">
- <xs:complexType>
- <xs:sequence>
<xs:element maxOccurs="unbounded" name="return"
```

```
nillable="true" type="dm:Request" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="addRequestNotes">
- <xs:complexType>
- <xs:sequence>
<xs:element name="requestId" type="dm:Identifier" />
<xs:element maxOccurs="unbounded" name="notes" type="common:Note" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="addRequestNotesResponse">
- <xs:complexType>
- <xs:sequence>
<xs:element name="return" nillable="true" type="xs:int" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="deleteRequests">
- <xs:complexType>
- <xs:sequence>
<xs:element maxOccurs="unbounded" name="requestIds"
type="dm:Identifier" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="deleteRequestsResponse">
- <xs:complexType>
- <xs:sequence>
<xs:element name="return" nillable="true" type="xs:int" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="setRequestRemoteReferenceStatus">
- <xs:complexType>
- <xs:sequence>
<xs:element name="receiver" type="dm:Identifier" />
<xs:element name="source" type="dm:Identifier" />
<xs:element name="status" type="xs:string" />
<xs:element maxOccurs="unbounded" name="fields" nillable="true"
type="dm:SimpleField" />
</xs:sequence>
</xs:complexType>
</xs:element>
```

```
- <xs:element name="setRequestRemoteReferenceStatusResponse">
- <xs:complexType>
- <xs:sequence>
<xs:element name="return" nillable="true" type="xs:int" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="setRequestFields">
- <xs:complexType>
- <xs:sequence>
<xs:element name="requestId" type="dm:Identifier" />
<xs:element maxOccurs="unbounded" name="fields"
type="dm:SimpleField" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="setRequestFieldsResponse">
- <xs:complexType>
- <xs:sequence>
<xs:element name="return" nillable="true" type="xs:int" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="executeWFTransitions">
- <xs:complexType>
- <xs:sequence>
<xs:element name="receiver" type="dm:Identifier" />
<xs:element name="transition" type="xs:string" />
</xs:sequence>
</xs:complexType>
</xs:element>
- <xs:element name="executeWFTransitionsResponse">
- <xs:complexType>
- <xs:sequence>
<xs:element name="return" nillable="true" type="xs:string" />
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>
</wsdl:types>
- <wsdl:message name="setRequestRemoteReferenceStatusMessage">
<wsdl:part name="part1" element="ds:setRequestRemoteReferenceStatus" />
</wsdl:message>
- <wsdl:message name="setRequestRemoteReferenceStatusResponseMessage">
<wsdl:part name="part1"
```

```
element="ds:setRequestRemoteReferenceStatusResponse" />
</wsdl:message>
- <wsdl:message name="addRequestNotesMessage">
<wsdl:part name="part1" element="ds:addRequestNotes" />
</wsdl:message>
- <wsdl:message name="addRequestNotesResponseMessage">
<wsdl:part name="part1" element="ds:addRequestNotesResponse" />
</wsdl:message>
- <wsdl:message name="createRequestMessage">
<wsdl:part name="part1" element="ds:createRequest" />
</wsdl:message>
- <wsdl:message name="createRequestResponseMessage">
<wsdl:part name="part1" element="ds:createRequestResponse" />
</wsdl:message>
- <wsdl:message name="deleteRequestsMessage">
<wsdl:part name="part1" element="ds:deleteRequests" />
</wsdl:message>
- <wsdl:message name="deleteRequestsResponseMessage">
<wsdl:part name="part1" element="ds:deleteRequestsResponse" />
</wsdl:message>
- <wsdl:message name="setRequestFieldsMessage">
<wsdl:part name="part1" element="ds:setRequestFields" />
</wsdl:message>
- <wsdl:message name="setRequestFieldsResponseMessage">
<wsdl:part name="part1" element="ds:setRequestFieldsResponse" />
</wsdl:message>
- <wsdl:message name="getRequestsMessage">
<wsdl:part name="part1" element="ds:getRequests" />
</wsdl:message>
- <wsdl:message name="getRequestsResponseMessage">
<wsdl:part name="part1" element="ds:getRequestsResponse" />
</wsdl:message>
- <wsdl:message name="executeWFTransitionsMessage">
<wsdl:part name="part1" element="ds:executeWFTransitions" />
</wsdl:message>
- <wsdl:message name="executeWFTransitionsResponseMessage">
<wsdl:part name="part1" element="ds:executeWFTransitionsResponse" />
</wsdl:message>
- <wsdl:portType name="DemandServicePortType">
- <wsdl:operation name="setRequestRemoteReferenceStatus">
<wsdl:input message="dmMsg:setRequestRemoteReferenceStatusMessage"
wsaw:Action="urn:setRequestRemoteReferenceStatus" />
<wsdl:output message=
"dmMsg:setRequestRemoteReferenceStatusResponseMessage"
wsaw:Action="http://mercury.com/ppm/dm/msg/1.0/
```

```
DemandServicePortType/setRequestRemoteReferenceStatus" />
</wsdl:operation>
- <wsdl:operation name="addRequestNotes">
<wsdl:input message="dmMsg:addRequestNotesMessage"
wsaw:Action="urn:addRequestNotes" />
<wsdl:output message="dmMsg:addRequestNotesResponseMessage"
wsaw:Action="http://mercury.com/ppm/dm/msg/1.0
/DemandServicePortType/addRequestNotesResponse" />
</wsdl:operation>
- <wsdl:operation name="createRequest">
<wsdl:input message="dmMsg:createRequestMessage"
wsaw:Action="urn:createRequest" />
<wsdl:output message="dmMsg:createRequestResponseMessage"
wsaw:Action="http://mercury.com/ppm/dm/msg/1.0/
DemandServicePortType/createRequestResponse" />
</wsdl:operation>
- <wsdl:operation name="deleteRequests">
<wsdl:input message="dmMsg:deleteRequestsMessage"
wsaw:Action="urn:deleteRequests" />
<wsdl:output message="dmMsg:deleteRequestsResponseMessage"
wsaw:Action="http://mercury.com/ppm/dm/msg/1.0
/DemandServicePortType/deleteRequestsResponse" />
</wsdl:operation>
- <wsdl:operation name="setRequestFields">
<wsdl:input message="dmMsg:setRequestFieldsMessage"
wsaw:Action="urn:setRequestFields" />
<wsdl:output message="dmMsg:setRequestFieldsResponseMessage"
wsaw:Action="http://mercury.com/ppm/dm/msg/1.0
/DemandServicePortType/setRequestFieldsResponse" />
</wsdl:operation>
- <wsdl:operation name="getRequests">
<wsdl:input message="dmMsg:getRequestsMessage"
wsaw:Action="urn:getRequests" />
<wsdl:output message="dmMsg:getRequestsResponseMessage"
wsaw:Action="http://mercury.com/ppm/dm/msg/1.0
/DemandServicePortType/getRequestsResponse" />
</wsdl:operation>
- <wsdl:operation name="executeWFTransitions">
<wsdl:input message="dmMsg:executeWFTransitionsMessage"
wsaw:Action="urn:executeWFTransitions" />
<wsdl:output message="dmMsg:executeWFTransitionsResponseMessage"
wsaw:Action="http://mercury.com/ppm/dm/msg/1.0
/DemandServicePortType/executeWFTransitionsResponse" />
</wsdl:operation>
</wsdl:portType>
```

```
- <wsdl:binding name="DemandServiceSOAP11Binding"
type="dmMsg:DemandServicePortType">
<soap:binding transport="http://schemas.xmlsoap.org/soap/http"
style="document" />
- <wsdl:operation name="setRequestRemoteReferenceStatus">
<soap:operation soapAction="urn:setRequestRemoteReferenceStatus"
style="document" />
- <wsdl:input>
<soap:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="addRequestNotes">
<soap:operation soapAction="urn:addRequestNotes" style="document" />
- <wsdl:input>
<soap:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="createRequest">
<soap:operation soapAction="urn:createRequest" style="document" />
- <wsdl:input>
<soap:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="deleteRequests">
<soap:operation soapAction="urn:deleteRequests" style="document" />
- <wsdl:input>
<soap:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="setRequestFields">
<soap:operation soapAction="urn:setRequestFields" style="document" />
- <wsdl:input>
<soap:body use="literal" />
```

```
</wsdl:input>
- <wsdl:output>
<soap:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="getRequests">
<soap:operation soapAction="urn:getRequests" style="document" />
- <wsdl:input>
<soap:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="executeWFTransitions">
<soap:operation soapAction="urn:executeWFTransitions"
style="document" />
- <wsdl:input>
<soap:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap:body use="literal" />
</wsdl:output>
</wsdl:operation>
</wsdl:binding>
- <wsdl:binding name="DemandServiceSOAP12Binding"
type="dmMsg:DemandServicePortType">
<soap12:binding transport="http://schemas.xmlsoap.org/soap
/http" style="document" />
- <wsdl:operation name="setRequestRemoteReferenceStatus">
<soap12:operation soapAction="urn:setRequestRemoteReferenceStatus"
style="document" />
- <wsdl:input>
<soap12:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap12:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="addRequestNotes">
<soap12:operation soapAction="urn:addRequestNotes" style="document" />
- <wsdl:input>
<soap12:body use="literal" />
</wsdl:input>
- <wsdl:output>
```

```
<soap12:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="createRequest">
<soap12:operation soapAction="urn:createRequest" style="document" />
- <wsdl:input>
<soap12:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap12:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="deleteRequests">
<soap12:operation soapAction="urn:deleteRequests" style="document" />
- <wsdl:input>
<soap12:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap12:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="setRequestFields">
<soap12:operation soapAction="urn:setRequestFields" style="document" />
- <wsdl:input>
<soap12:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap12:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="getRequests">
<soap12:operation soapAction="urn:getRequests" style="document" />
- <wsdl:input>
<soap12:body use="literal" />
</wsdl:input>
- <wsdl:output>
<soap12:body use="literal" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="executeWFTransitions">
<soap12:operation soapAction="urn:executeWFTransitions"
style="document" />
- <wsdl:input>
<soap12:body use="literal" />
</wsdl:input>
```

```
- <wsdl:output>
<soap12:body use="literal" />
</wsdl:output>
</wsdl:operation>
</wsdl:binding>
- <wsdl:binding name="DemandServiceHttpBinding"
type="dmMsg:DemandServicePortType">
<http:binding verb="POST" />
- <wsdl:operation name="setRequestRemoteReferenceStatus">
<http:operation location="setRequestRemoteReferenceStatus" />
- <wsdl:input>
<mime:content type="text/xml" />
</wsdl:input>
- <wsdl:output>
<mime:content type="text/xml" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="addRequestNotes">
<http:operation location="addRequestNotes" />
- <wsdl:input>
<mime:content type="text/xml" />
</wsdl:input>
- <wsdl:output>
<mime:content type="text/xml" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="createRequest">
<http:operation location="createRequest" />
- <wsdl:input>
<mime:content type="text/xml" />
</wsdl:input>
- <wsdl:output>
<mime:content type="text/xml" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="deleteRequests">
<http:operation location="deleteRequests" />
- <wsdl:input>
<mime:content type="text/xml" />
</wsdl:input>
- <wsdl:output>
<mime:content type="text/xml" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="setRequestFields">
```

```
<http:operation location="setRequestFields" />
- <wsdl:input>
<mime:content type="text/xml" />
</wsdl:input>
- <wsdl:output>
<mime:content type="text/xml" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="getRequests">
<http:operation location="getRequests" />
- <wsdl:input>
<mime:content type="text/xml" />
</wsdl:input>
- <wsdl:output>
<mime:content type="text/xml" />
</wsdl:output>
</wsdl:operation>
- <wsdl:operation name="executeWFTransitions">
<http:operation location="executeWFTransitions" />
- <wsdl:input>
<mime:content type="text/xml" />
</wsdl:input>
- <wsdl:output>
<mime:content type="text/xml" />
</wsdl:output>
</wsdl:operation>
</wsdl:binding>
- <wsdl:service name="DemandService">
- <wsdl:port name="DemandServiceSOAP11port_http"
binding="dmMsg:DemandServiceSOAP11Binding">
<soap:address location="http://<ppm server>:15000/itg/ppmservices/
DemandService" />
</wsdl:port>
- <wsdl:port name="DemandServiceSOAP12port_http"
binding="dmMsg:DemandServiceSOAP12Binding">
<soap12:address location="http://<ppm server>:15000/itg/
ppmservices/DemandService" />
</wsdl:port>
- <wsdl:port name="DemandServiceHttpport1"
binding="dmMsg:DemandServiceHttpBinding">
<http:address location="http://<ppm server>:15000/itg/
ppmrest/DemandService" />
</wsdl:port>
```

```
</wsdl:service>
</wsdl:definitions>
```

PPM request

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/
  soap/envelope/" xmlns:ns="http://mercury.com/ppm/dm/service/1.0"
  xmlns:ns1="http://mercury.com/ppm/dm/1.0"
  xmlns:ns2="http://mercury.com/ppm/common/1.0">
   <soapenv:Header/>
   <soapenv:Body>
      <ns:createRequest>
         <ns:requestObj>
            <!--Optional:-->
            <ns1:id></ns1:id>
            <ns1:requestType></ns1:requestType>
            <!--Zero or more repetitions:-->
            <ns1:simpleFields>
               <ns2:token>REQ.VP.KNTA_PLAN_START_DATE</ns2:token>
               <!--Zero or more repetitions:-->
               <ns1:stringValue>May 2008</ns1:stringValue>
               <!--Optional:-->
               <ns1:dateValue></ns1:dateValue>
            </ns1:simpleFields>
            <!--1 or more repetitions:-->
            <ns1:tables>
               <ns2:token></ns2:token>
               <!--1 or more repetitions:-->
               <ns2:columns>
                  <ns2:token></ns2:token>
                  <!--1 or more repetitions:-->
                  <ns2:values></ns2:values>
                  <!--1 or more repetitions:-->
                  <ns2:dates></ns2:dates>
               </ns2:columns>
            </ns1:tables>
            <!--1 or more repetitions:-->
            <ns1:notes>
               <!--Optional:-->
               <ns2:author></ns2:author>
               <!--Optional:-->
               <ns2:creationDate></ns2:creationDate>
               <!--Optional:-->
               <ns2:content></ns2:content>
            </ns1:notes>
            <!--1 or more repetitions:-->
            <ns1:fieldChangeNotes>
               <!--Optional:-->
               <ns2:author></ns2:author>
```

```
               <!--Optional:-->
               <ns2:creationDate>?</ns2:creationDate>
               <!--Optional:-->
               <ns2:content></ns2:content>
               <ns1:fieldPrompt></ns1:fieldPrompt>
               <ns1:oldValue></ns1:oldValue>
               <ns1:newValue></ns1:newValue>
            </ns1:fieldChangeNotes>
            <!--1 or more repetitions:-->
            <ns1:URLReferences>
               <!--Optional:-->
               <ns1:addedBy></ns1:addedBy>
               <!--Optional:-->
               <ns1:creationDate></ns1:creationDate>
               <!--Optional:-->
               <ns1:description></ns1:description>
               <!--Optional:-->
               <ns1:name></ns1:name>
               <ns1:refURL></ns1:refURL>
            </ns1:URLReferences>
            <!--1 or more repetitions:-->
            <ns1:remoteReferences>
               <!--Optional:-->
               <ns1:addedBy></ns1:addedBy>
               <!--Optional:-->
               <ns1:creationDate></ns1:creationDate>
               <!--Optional:-->
               <ns1:description></ns1:description>
               <!--Optional:-->
               <ns1:name></ns1:name>
               <!--Optional:-->
               <ns1:displayURL></ns1:displayURL>
               <ns1:identifier>
                  <ns1:id></ns1:id>
                  <!--Optional:-->
                  <ns1:serverURL></ns1:serverURL>
               </ns1:identifier>
               <!--Optional:-->
               <ns1:status></ns1:status>
            </ns1:remoteReferences>
         </ns:requestObj>
      </ns:createRequest>
   </soapenv:Body>
</soapenv:Envelope>
```

PPM response

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/
  soap/envelope/">
   <soapenv:Header/>
   <soapenv:Body>
      <soapenv:Fault>
         <faultcode>INTERNAL ERROR</faultcode>
         <faultstring>Internal error has occurred while calling
          PPM Web Service. Please contact PPM support with the detail
          information if the problem persists. (KNTA-11186)
          Details: Missing 'T' separator in dateTime</faultstring>
         <detail>
            <exception:exceptionDetails xmlns:exception=
              "http://www.mercury.com/ppm/ws/exception">
               <exception:detail>Missing 'T' separator in dateTime
                </exception:detail>
            </exception:exceptionDetails>
         </detail>
      </soapenv:Fault>
   </soapenv:Body>
</soapenv:Envelope>
```

1.1.1.4.3.5. Web Services with a proxy server

It is possible to consume Web Services through a proxy server in Service Manager. The proxy server settings allow your Service Manager server to connect to remote sites and download the WSDL for the remote Web Services. The following parameters have to be added to the sm.ini file for the Web service to connect through a proxy server.

- JVMOptionX:-Dhttp.proxyHost=proxyserver.domain.company.com
- JVMOptionY:-Dhttp.proxyPort=<port number such as 8088>

You can also specify a list of hosts to bypass the proxy:

JVMOptionZ:-Dhttp.nonProxyHosts="*.americas.hpqcorp.net|localhost"

The http.nonProxyHosts property indicates the hosts which should connect directly and not through the proxy server. The value can be a list of hosts, each separated by a |, and in addition a wildcard character (*) can be used for matching.

The X, Y and Z represent three consecutive numbers. The first JVMOption in the sm.ini will be number 1, the next will be number 2 and 3. See the following example if these three are the only JVMOptions in your sm.ini file:

```
JVMOption1:-Dhttp.proxyHost=proxyserver.domain.company.com
JVMOption2:-Dhttp.proxyPort=<port number such as 8088>
JVMOption3:-Dhttp.nonProxyHosts="*.domain.company.com|localhost
```

In addition, you can also include the following parameters for SSL connection:

```
JVMOption4:-Dhttps.proxyHost=proxyserver.com
JVMOption5:-Dhttps.proxyPort=<port number for ssl>
```

1.1.1.4.3.6. Connecting to a secure Web Service

If you are consuming a secure Web service that requires mutual authentication from Service Manager application using Javascript and WSDL2JS, follow these steps:.

If you are consuming an SSL-protected Web service using Javascript in SM 7x, which uses java.xml.soap.SOAPConnection to send the request, the SSL configuration is done using Java key stores. Refer to the documentation for the list of sm.ini parameters required for SSL configuration.

When you consume a secure Web service or Web site from JavaScript all you need to do is use an https:// URL. There are no facilities for configuring SSL in WSDL2JS or in your script. The SSL communication that's initiated by the WSDL2JS-generated code relies on the SSL configuration that's in place for the server itself. The Service Manager server's server certificate in effect becomes the client certificate for the outbound request.

To supply a Basic Authorization header when consuming a Web service using JavaScript generated by WSDL2JS, basic authentication is supplied automatically if you supply userid and password values on the service object generated by WSDL2JS as shown in the below example:

```
var service = new system.library.IncidentManagement.
     IncidentManagement();
     service.user = "falcon";
     service.password = "";
     ...
```

If on the other hand, you are coding a REST-style GET directly in your script, you need to code it manually, because you have to code the HTTP request yourself. Add the following style code in the JavaScript to perform this:

```
// HTTP GET example with Basic Auth header
var headers = new Array();
try
{
if ( result.userid != undefined )
{
var authHeader = new Header();
authHeader.name = "Authorization";
authHeader.value = "Basic " + base64Encode
        ( result.userid + ":" + result.password );
headers.push( authHeader );
}
strWSDLsrc = doHTTPRequest( "GET", wsdlURL, headers, null,
        10, 10, 10 );
}
catch( e )
{
print( "WSDL request failed with exception " + e );
...
}
```

1.1.1.4.3.7. Use SSL connections to connect to a Web Service

Enable SSL encryption for published Web Services

If you want external Web Services clients to use an SSL connection with the Service Manager server, you must provide them with the CA certificate for the Service Manager server. If you purchased a server certificate, copy the CA certificate from the CA certificate keystore provided with your purchased certificate. If you generated your own server certificate by using a self-signed private CA certificate, copy the CA certificate from your private CA certificate keystore instead.

1. Copy the keystore that contains the CA certificate that signed your server's certificate, and send it to the systems running the external Web services clients.
2. Import the CA certificate of the Service Manager system into the CA certificate keystore of the external Web Services client. You may use a tool like keytool to import Service Manager's CA certificate.
3. Configure the external Web Services client to use the updated CA certificate keystore. Follow the instructions for your Web Services client to set the path to the CA certificate keystore.
4. Update the endpoint URL the external Web Services clients use to include the HTTPS protocol. For example, `https://smserver.yourdomain.net:13443/SM/7/ws`. Follow the instructions for your Web Services client to update the endpoint URL.

   Note:

   - The endpoint URL must use the Service Manager server's common name (CN) as defined in the server certificate. For example, if the server certificate uses the name myserver.mydomain.com, then the endpoint URL must also use the name myserver.mydomain.com.
   - If you want the external Web Service clients to download the Service Manager Web Service WSDL, point them to a URL using the following format:

     `https://smserver.yourdomain.net:13443/SM/7/<Service Name>.wsdl`

Enable SSL encryption for external Web Services

When using SSL connections to an external Web service, the Service Manager server acts like a client and must be set up accordingly. The Web service provider must send the root certificate or the certificate authority's (CA) certificate to the Service Manager administrator. If it is a certificate hierarchy, all certificates must be sent. Add this certificate to the Service Manager cacerts file using keytool.

For an anonymous SSL connection with an external Web Service using WSDL2JS, you need a root certificate file which includes the certificate for the CA that signed the remote Web Server's certificate.

When the root certificate file is saved, the following parameters must be added to sm.ini on the Service Manager server if they don't already exist. These parameters identify the name of the root certificate or authority's certificate as well as the Service Manager server's keystore.

Parameter	Description
-truststoreFile	The TrustStore file to use to validate client certificates. Default to the cacerts in the RUN\jre\security directory.
-truststorePass	Identifies the password to the keystore file containing the external Web Servics's CA certificate. The pass phrase for the TrustStore file
-keystoreFile	Identifies the keystore file containing the Service Manager's server's certificate and private key. Server keystore
-keystorePass	Identifies the password to the keystore file containing the Service Manager's certificate and private key. Pass phrase for server keystore.

To enable the SSL encryption, follow these steps:

1. Import the CA certificate of the external Web Service into the CA certificate file of the Service Manager system. You may use a tool like keytool to import the external Web Service's CA certificate.
2. Stop the Service Manager server.
3. Open the sm.ini file with a text editor.
4. Add the following parameters and their values:
    1. keystoreFile
    2. keystorePass
    3. truststoreFile:cacerts
    4. truststorePass
5. Save the sm.ini file.
6. Restart the Service Manager sever.
7. Login to Service Manager with SysAdmin privileges.
8. Click **Tailoring** > **Web Services** > **Run WSDL to JS**.
   Note: Alternatively, you can use the doHTTPRequest JavaScript method to send an HTTP request to the Web services API.

9. Update the endpoint URL to the external Web Service to include the HTTPS protocol. For example: `https://remote_server.remote_domain.com:13445/remote_service.wsdl`

If the `https://<fully qualified server path>:<portnumber>/<Service>.wsdl` connection doesn't work after you make these changes, it is possible that the distinguished name (DN) used to create the certificate is not identical to the fully qualified server path in the URL. Check which DN the certificate is using by asking the provider of the certificate. If it is different from the fully qualified path used in the URL, request a new certificate where the DN matches the URL. If this can't be done in a timely manner, the following workaround can be tested:

Go to the server's hosts file (which is located in etc/hosts on Linux systems, and located in c:\winnt\system32\drivers\etc\hosts on Windows systems). In the hosts file, add a line with the fully qualified name of the certificate and the IP address of the machine that runs the Web Service. For example:

mymachine.microfocus.com 10.2.5.77

where "mymachine.microfocus.com" is the distinguished name (DN) of the certificate and 10.2.5.77 is the IP address for the server that hosts the Web Service.



Note

This is a temporary workaround, and not a permanent fix. Once the new certificate is issued, that certificate should be put into the root certificate file, and the entry in the hosts file should be removed.



Caution

When you use SSL connections on high-frequency Web Services where many requests per second are made, performance is negatively impacted because an SSL handshake occurs for each SOAP request. The SSL handshake involves numerous TCP requests and responses as the two partners identify each other, negotiate the encryption algorithm, and perform other required tasks. To avoid this issue, ensure to use keep-alive connections. These will perform the handshake once and then SSL is set up for the length of the session.

1.1.1.4.3.8. Web Services connections through a firewall

If your Service Manager server is behind a firewall, you may need to configure a proxy server redirection to send and receive WSDL and SOAP requests. If your firewall uses the SOCKS protocol, then it can likely handle Web Services redirection requests transparently to the user. If your firewall doesn't recognize the SOCKS protocol, then you can install a dedicated redirector application for SOCKS traffic such as that generated by Web Services requests.

If you install a redirector application for your Web Services SOAP traffic, then you need to modify the URLs you use to connect to the remote Web Services. To download the remote WSDL, change the URL listed in the WSDL to JS wizard to point to the dedicated socket you have established for the remote Web Service. To send and receive SOAP messages to the Web Service, you can change the location object of your custom JavaScript to the dedicated socket you have established for the remote Web Service.

Example: dedicated socket connection

Define a dedicated socket on port 8888 to the Amazon Search Service using the following `proxyconnect` command of the `connect.c` application:

```
proxyconnect -p 8888 -S 192.168.1.254:1080
```

```
http://soap.amazon.com/onca/soap280
```

To obtain the WSDL for the Amazon Search Service through this example proxy connection, update the `WSDL to JS` URL to point to:

```
http://localhost:8888/soap/servlet/rpcrouter
```

To send to and to receive from the Amazon Search Service SOAP messages , you could update the custom calling script `AmazonSearchServiceTestwith` the following new line just after the `AmazonSearchService.ActorSearchRequest` class is initialized.

```
actorSearchRequest.location =
```

```
"http://localhost:8888/soap/servlet/rpcrouter"
```

1.1.1.5. The RESTful API

Service Manager also supports a Rest API Framework. You can use the Rest API Framework to support lightweight queries and operations on Service Manager data via a single URI. Using the Rest API Framework you can create an application that can perform Create, Read, Update, and Delete actions on Service Manager objects.

The Rest API Framework re-implements most of the functionality that the Service Manager SOAP implementation. Therefore, the Rest API Framework uses the same actions on objects as the SOAP implementation, and the implementation methods are similar.

For more information on the available actions, see the Allowed Actions tab field definitions in the Web Services Guide.

1.1.1.5.1. Service Document

Service Manager supports the automatic generation of RESTful Service Documents by providing an HTML (textl) representation.

RESTful Service Documents represent server-defined group of Collections used to initialize the process of creating and editing resources.

After **RESTful Enabled** is checked into **Web Service Configuration** for a Web service, its description will be generated.

At the top of the document, there is a group of all of Service Manager RESTful services. You can go to the detailed description of each service by clicking **Service Name**.

In the detail of the description, you can find the URI and supported HTTP Methods for each Resource Type. You can also find the supported actions' descriptions.

For example:

Change

Resource type	Supported Methods	URI
Resource Collection	GET/POST	http://localhost:14930/SM/9/rest/changes
Single Resource	GET/POST/PUT	http://localhost:14930/SM/9/rest/changes/{header,number}
Resource Attachment Collection	GET/POST/DELETE	http://localhost:14930/SM/9/rest/changes/{header,number}/attachments
Resource Single Attachment	GET/PUT/DELETE	http://localhost:14930/SM/9/rest/changes/{header,number}/attachments/{id}

Supported actions:

Action name	Supported Methods	URI
Update	POST	http://localhost:14930/SM/9/rest/changes/{header,number}/action/update
Retract	POST	http://localhost:14930/SM/9/rest/changes/{header,number}/action/retract
Reopen	POST	http://localhost:14930/SM/9/rest/changes/{header,number}/action/reopen
MoveToNextPhase	POST	http://localhost:14930/SM/9/rest/changes/{header,number}/action/movetonextphase
Deny	POST	http://localhost:14930/SM/9/rest/changes/{header,number}/action/deny
Close	POST	http://localhost:14930/SM/9/rest/changes/{header,number}/action/close
Approve	POST	http://localhost:14930/SM/9/rest/changes/{header,number}/action/approve
Create	POST	http://localhost:14930/SM/9/rest/changes/{header,number}/action/create

1.1.1.5.2. Consuming Service Manager RESTful API

A Service Manager Restful Web service can be consumed by a custom client or by an application that directly consumes Restful Web Services.

1.1.1.5.2.1. RESTful syntax

A RESTful query allows you to send a request or execute an operation by sending a single Universal Resource Identifier (URI) space to Service Manager. In general, the format for the URI resembles the following:

```
http://{host}:{port}/{initialPath}
```

This example uses the following placeholders:

- The {*host*} placeholder indicates the host or domain name on which the service is available.
- The {*port*} placeholder indicates the TCP port number on which the service is available.
- The {*initialPath*} placeholder indicates any initial path that's part of the URI for a given deployment. This might be the path to the deployment point on the given server. This value should start with **/SM/9/rest**.

Note

If the URL contains "\", it should be replaced with "%5C" before encoding. For example, for `Adobe\Reader`, it should be replaced with `Adobe%5CReader`, and then be encoded as `Adobe%255CReader`.

If the URL contains "/", it should be replaced with "%2F" first before encoding. For example, for `Adobe/Reader`, it should be replaced with `Adobe%2FReader`, and then be encoded as `Adobe%252FReader`.

Resource types

The Rest API Framework in Service Manager allows you to perform standard CRUD operations on resources. URI endpoints for resources are roughly divided into the following resource types:

- Resource Collection
- Resource Instance
- Resource with Actions
- Attachment Collection
- Attachment Instance

URI structure

The following table describes the structure of URIs that can be used in the Rest API Framework.

URI	Resource	Comments	Example
/	Service Document	This is the service document for an entire business service. Only the GET method is supported.	http://{host}:{port}/SM/9/rest
/{*resources*}	Resource Collection	This is the collection resources. Only the GET and POST methods are supported.	http://{host}:{port}/SM/9/rest/incidents
/{*resources*}/{*key*}	Resource Instance	This is an individual resource.	http://{host}:{port}/SM/9/rest/incidents/IM10001 If with multiple keys, it should be {key1}/{key2}/{key3}. If one of the key value is null, it may resemble: {key1}/null/{key3}
/{*resources*}/{*key*}/attachments	Attachments Collection	This is the attachment list for an individual resource.	http:/{host}:{port}/SM/9/rest/incidents/IM10001/attachments
/{*resources*}/{*key*}/attachments/{*attachment-id*}	Attachment	This is an individual attachment.	http://{host}:{port}/SM/9/rest/incidents/IM10002/attachments/cid:51dd0b6d0002c0042075d798
/{*resources*}/{*key*}/action/{*action*}	Resource With Actions	Only the POST method is supported.	POST http://{host}:{port}/SM/9/rest/incidents/IM10134/action/reopen

This table uses the following placeholders:

- The {*resources*} placeholder represents the resource key of the individual object.
- The {*attachment-id*} placeholder represents the ID of the attachment for an individual resource.
- The {*key*} placeholder represents any unique key specified in the extaccess record.
- The {*action*} placeholder represents any action specified in the Allowed Actions tab of the External Access Definition. It must be lowercase. For example, if the action in the Allowed Actions tab is "Reopen", the value of {action} should be "reopen".
- action, attachments and view are keywords that are used in the URI.

1.1.1.5.2.2. RESTful commands

The Rest API Framework supports the following HTTP commands:

- `GET`
- `POST`
- `PUT`
- `DELETE`
- `PATCH`

The functionality of these commands varies according to the type of object to which it's applied and the actions that are associated with that object. The following table shows example resources and illustrates how these commands are used:

Object	Example Commands	Result
Service Document	`GET /`	Returns all accessible URLs.
Query	`GET/<incidents>`	Returns all Incidents
	`GET/<incidents>/<id>`	Returns an Incident with the specified ID.
	`GET/<incidents>?query=<url-encoded-string>&sort=number:ascending`	Returns a subset of Incidents as specified by the URL encoded string, in ascending order.
Resources with Actions	`POST /<incidents>`	Creates an Incident
	`PUT /<incidents>/<id>`	Updates an specific Incident
	`POST /<incidents>/<id>/action/<action>`	Invokes a customized action on a specific Incident
	`PATCH /<incidents>/<id>`	Updates an specific Incident

1.1.1.5.2.3. RESTful queries

The Rest API Framework also supports several parameters to query resources to return different views of a resource or to filter for desired entries in a list. You can run a query on all resource types.

Example of a query:

- `/incidents //Query to gather all incidents`
- `/incidents?field1=value1&field2=value2 //Simple Query`
- `/incidents?query=<url-encoded-string>&sort=number:ascending //Service Manager Native Query`



Note

- The query string must use UTF-8 Encoding.
- The query string must use HTML URL Encoding.
- Datetime fields must use ISO standard formats.
  If you want to specify a time zone, you need to add the time zone with the following format:
  `yyyy-MM-ddTHH:mm:ss+hh:mm`
  For example: `2023-11-24T18:00:00+01:00`
  Here is an example query:
  `/SM/9/rest/incidents?query=open.time%3E%272019-11-11T18:00:00%2B01%3A00%27%20and%20open.time%3C%272019-11-11T23:59:59%2B01%3A00%27`
  Here is the example query string above being URL decoded for your reference:
  `/SM/9/rest/incidents?query=open.time>'2019-11-11T18:00:00+01:00' and open.time<'2019-11-11T23:59:59+01:00'`

The following table describes the parameters you can use:

Parameter Name	Type	Description:

query	native sm query	Field name could be either "Caption" or "Field".
		Examples:
		`Category="incident" and (Title="Desktop screen out of order" or Title="Network logon failure")`
		`Category="incident" and (brief.description="Desktop screen out of order" or brief.description="Network logon failure")`

Parameter Name	Type	Description:
sort	string	Returns the collection members in sorted order according to the arguments specified. Arguments are listed in pairs where the first argument of the pair specifies the attribute name on which to base the sort, and the second argument of the pair indicates whether to sort ascending or descending. More than one attribute can be specified on the sort list. The attributes `ascending` and `descending` can be used as well. `sort={primaryField}:{ascending|descending}[,{secondaryField}:{ascending|descending}...]` For example, `sort=Urgency` or `sort=severity` or `sort=Urgency:ascending,field2:descending` By default, the sortorder is ascending.
start	int	Indicates the index of the member that the collection response representation begins with.

Parameter Name	Type	Description:
count	int	Indicates the number of collection members to be included in the response. The minimum value for this parameter is 1. By default, its behavior is to return all members. The behavior is the same as `view=summary` if `count=0`.

view	string	Represents a collection.
		
		Supported values:
		
		`view=summary`
		
		`view=condense (default)`
		
		`view=expand`
		
		summary: It returns the number of members if there are any, it doesn't return the actual members.
		
		condense: Returns the value of unique key field, it doesn't return the whole record. This is the default behavior if the view query parameter is not specified.
		
		expand: It returns all the fields defined in the extaccess record.

1.1.1.5.2.4. Resource representations

The following sections describe the media types and supported commands for each resource.

Media types for an individual resource

The following table describes the supported commands and media types for an individual resource.

Action	Supported?	Supported Request Media Types	Supported Response Media Types
GET	Ö	-	application/json
POST	Ö	application/json	application/json
PUT	Ö	application/json	application/json
PATCH	-	-	-
DELETE	Ö	-	application/json

Resource collection media types

The following table describes the supported commands and media types for a resource collection.

Action	Supported?	Supported Request Media Types	Supported Response Media Types
GET	Ö	-	application/json
POST	Ö	application/json	application/json
PUT	-	-	-
PATCH	-	-	-
DELETE	-	-	-

Media types for an individual attachment

The following table describes the supported commands and media types for an individual attachment.

Action	Supported?	Supported Request Media Types	Supported Response Media Types
GET	Ö	-	The attachment binary
POST	-	-	-
PUT	Ö	The attachment binary	application/json
PATCH	-	-	-
DELETE	Ö	-	application/json

Attachment collection media types

The following table describes the supported commands and media types for an attachment collection.

Action	Supported?	Supported Request Media Types	Supported Response Media Types
GET	Ö	-	application/json
POST	Ö	The attachment binary	application/json
PUT	-	-	-
PATCH	-	-	-
DELETE	Ö	-	application/json

1.1.1.5.2.5. Enable a resource for REST

To enable a resource to use the RESTful API Framework, follow these steps:

1. Navigate to **Tailoring** > **Web Services** > **Web Service Configuration**.
2. If needed, enter the information in the **Service Name**, **Name**, or **Object Name** fields to specify the resource you want to enable for RESTful.
3. Click **Search**, and then select the appropriate resource from the list.
4. Under **External Access Definition**, click the RESTful API tab.
5. Select the **RESTful Enabled?** checkbox.
6. Specify the following required fields:

Field	Function
Resource Collection Name:	This is the name of the Resource Collection. For example, you may specify the group of incidents from the probsummary table as "incidents".
Resource Name:	This is the name of the individual Resource. For example, you may specify that any individual incident from the probsummary table be referred to as an "Incident".
Unique Keys	This field specifies one or more fields that will function as a unique identifier for a Resource from the Resource Collection.

7. If needed, specify the following additional fields:

Field	Function
Resource Collection Action **POST:**	This field represents the default action for resource collection.
Resource Actions **POST:** **PUT:** **DELETE:** **PATCH:**	This field specifies the action to take when an individual resource is part of a POST, PUT, DELETE command. These actions are specified in the **Allowed Actions** tab of the **External Access Definition**. Note: **PATCH** only support **Update** action

1.1.1.5.2.6. RESTful capability word

The Rest API Framework introduces the following capability word to Service Manager: RESTful API.

You MUST add this capability word to a user's operator record for a user to be able to log in and execute a RESTful API request.



Note

Existing OMi integration RESTful functions will not check it and keep unchanged. The rest actions go through document engine and follow the same permission process as normal client.

1.1.1.5.2.7. HTTP header

Request Header

`Authorization:`

`Keep-Alive:`

`Connnection:`

`Accept-Language:`

`Content-Type:` (Required for attachment POST/PUT action)

`Content-Disposition:` (Required for attachment POST/PUT action. Value is attachment;filename=filename; Semicolon is reserved for character separator, so it's not allowed in file name.)



Note

- Accept-Language is used for l10n.
- In order to use non-ASCII characters in the content disposition HTTP header, the web service client must support the RFC 6266 standard. For example, the web service client must add an HTTP header that resembles the following:

  `Content-Disposition: attachment;filename*=UTF-8%E6%99%93%E6%B9%984.txt`

Response Header

`code&msg: 200, 201, 400, 401, 403, 404, 500`

`Content-Type:`

`Content-Length:`

1.1.1.5.2.8. HTTP response codes

Unless otherwise specified, the system uses these HTTP status codes.

Code	Cause
200	Successful operation. Viewing a list or detail page will return this code if no error occurred.
201	Successful POST operations. Returned after a successful create or update of an object.
400	Bad request. This response code may indicate that the URI was formed incorrectly.
401	Unauthorized operations.
403	Request forbidden.
404	Page not found. The page or resource doesn't exist.
500	Internal server error. This response code may indicate a bug. Please contact Software Support.

Related topics

Network Working Group RFC 2616: Hypertext Transfer Protocol -- HTTP/1.1
Network Working Group RFC 2616 Section 10: Status Code Definitions

1.1.1.5.2.9. OOB resource reference example

The following topic provide detailed information on how specific resources are published (for example, Incident).

Web Service: Incident

External access definition

Field	Value
Service Name:	IncidentManagement
Name:	probsummary
Object Name:	Incident

Default RESTful settings

Field	Value
RESTful Enabled:	checked
Attachments Enabled:	checked
Resource Collection Name:	incidents
Resource Name:	Incident
Unique Keys:	number

HTTP resource collection actions

Command	Action	Result
GET		Retrieves a list of Incidents
POST	Create	Creates a new collection of resources

HTTP resource actions

Command	Action	Result
GET	-	Retrieves an individual Incident.
PUT:	Update	Saves the changes to the Incident record.
POST:	Update	Saves the changes to the Incident record.
DELETE:	-	
PATCH:	Update	Saves the changes to the Incident record.

Samples

Return sample of single incident query

```
{"Messages":[],
"ReturnCode":0,
"Incident":{
"UpdatedBy":"falcon",
"Status":"Closed",
"SLAAgreementID":168,
"Urgency":"3",
"Area":"failure",
"OpenTime":"2007-08-31T20:21:00+00:00",
"Location":"advantage/North America",
"ClosedTime":"2007-09-01T01:13:00+00:00",
"Title":"Printer malfunction",
"Subarea":"job failed",
"Solution":["Reset printer queue."],
"ClosedBy":"Incident.Analyst",
"OpenedBy":"Servicedesk.Manager",
"IncidentID":"IM10001",
"Assignee":"Incident.Analyst",
"Company":"advantage",
"Description":["Printjob keeps pending."],
"TicketOwner":"Servicedesk.Manager",
"ProblemType":"incident",
"AssignmentGroup":"Office Supplies (North America)",
"UpdatedTime":"2008-08-04T12:53:21+00:00",
"Service":"Printing (North America)",
"Impact":"4",
"ClosureCode":"Solved by Workaround",
"Category":"incident",
"AffectedCI":"adv-nam-printer-hr-5550"
}
}
```


Note

"Incident" is the Resource Name as defined in the extaccess record.

Return sample of the Incident list query

```
{
"@totalcount":136,
"@start":1,
"@count":10,
"Messages":[],
"content":[
{"Incident":{"IncidentID":"IM10001"}},
{"Incident":{"IncidentID":"IM10002"}},
{"Incident":{"IncidentID":"IM10003"}},
{"Incident":{"IncidentID":"IM10004"}},
{"Incident":{"IncidentID":"IM10005"}},
{"Incident":{"IncidentID":"IM10006"}},
{"Incident":{"IncidentID":"IM10007"}},
{"Incident":{"IncidentID":"IM10008"}},
{"Incident":{"IncidentID":"IM10009"}},
{"Incident":{"IncidentID":"IM10010"}}
],
"ReturnCode":0,
"ResourceName":"Incident"
}
```


Request sample of creating an Incident

```
{
        "Incident":
        {
          "AffectedCI" :"adv-nam-server-mail",
"AlertStatus" : "updated",
"Area" :"failure",
"Assignee" : "Incident.Analyst",
"AssignmentGroup": "Network",
"Category" : "incident",
"ClosureCode" : "Solved by Workaround",
"Company" : "advantage",
"Contact" : "FALCON, JENNIFER",
"ContactFirstName" :"FALCON",
"ContactLastName" : "JENNIFER",
"Description" : [ "test" ],
"Impact" : "2",
"JournalUpdates":
            [ "08/04/08 12:54:14 US/Mountain (falcon):",
"test",
"08/04/08 12:54:14 US/Mountain (falcon):",
"test"
],
"Location":"advantage/North America",
"OpenTime" : "2007-09-02T07:51:00+00:00",
"OpenedBy":"Jurr.Fleijs",
"ProblemType" : "incident",
"ResolutionFixType" : "incident",
"SLAAgreementID" : 168,
"Service" : "E-mail / Webmail (North America)",
"SiteCategory" : "incident",
"Solution" : ["Solution by rest api"],
"Status" : "Work In Progress",
"Subarea" :"function or feature not working",
"TicketOwner" : "Jurr.Fleijs",
"Title" : "test",
"UpdatedBy" : "problem",
"UpdatedTime" : "2008-08-04T12:54:26+00:00",
"Urgency" : "3",
"UserPriority": "3 - Average",
"explanation" : ["test"],
"folder" :"advantage"
        }
}
```

1.1.1.5.3. Attachment operations using the RESTful API

Service Manager supports all typical attachment operations using its web services (SOAP API) or RESTful API. These operations include: get attachment information, get (single) attachment, delete (single) attachment, insert attachment, and so on.

The following topics describes how to perform typical attachment operations by using the RESTful API. It provides information about the services published by Service Manager, and explains requests of an external application being sent to Service Manager, as well as the responses to expect.



Tip

This topic describes attachment operations that you can perform using the Service Manager REST API. For information about attachment operations using the Service Manager web services, see Attachment operations using the SOAP API.

When an external system sends a RESTful request, you can first look at the service document:

```
http://<SM server>:<port>/SM/9/rest
```

Information of RESTful services of all available objects can be found in the document retrieved. The following table contains the part returned for Incident.

Resource type	Supported methods	URI
Resource Collection	GET/POST	http://localhost:13080/SM/9/rest/incidents
Single Resource	GET/POST/PUT	http://localhost:13080/SM/9/rest/incidents/{number}
Resource Attachment Collection	GET/POST/DELETE	http://localhost:13080/SM/9/rest/incidents/{number}/attachments
Resource Single Attachment	GET/PUT/DELETE	http://localhost:13080/SM/9/rest/incidents/{number}/attachments/{id}

The target object of a request is determined by the URI, and the operation executed is determined by the HTTP method of the request. The table above lists supported methods.



Tip

The `Content-Disposition` request header is required for attachment POST/PUT action. Value is `attachment;filename=filename;` . Note that the semicolon is reserved for character separator and it's not allowed in file name. For example:

`Content-Disposition: attachment;filename*=UTF-8%E6%99%93%E6%B9%984.txt`

Examples:

- A GET request on the Resource Attachment Collection URI returns a JSON document with all the attachments IDs.
- A GET request on the Resource Single Attachment URI downloads the attached file identified by ID.
- A DELETE request on the Resource Single Attachment URI deletes the file attachment identified by ID.



Tip

You can enter a GET HTTP message in the address bar of the browser, which is comfortable for testing. In order that Service Manager accepts these requests, enable the **restaccessviabrowser:1** parameter for the servlet. For REST requests using other HTTP methods such as GET, testing in the browser address bar is impossible. You can use tools for web services testing or use available browser plugins for testing.

Retrieve attachment information

Execute a GET request on the Resource Attachment Collection of an incident identified by incident number.

http://localhost:13080/SM/9/rest/incidents/IM10003/attachments

The response received is a JSON document that resembles the following:

```
{
"@start": 1,
"@count": 29,
"ReturnCode": 0,
"Messages": [],
"ResourceName": "attachment",
"@totalcount": 29,
"content": [
{"attachment": {
"len": 2671,
"name": "test.jpg",
"href": "cid:676A50F922E10AC400006CC400003BA0",
"type": "image/jpeg",
"xmime:contentType": "image/jpeg"
}},
[ .. ]
}
```


In this example, incident IM10003 has 29 attachments, and for each the JSON document contains information such as the attachment id (href), length in bytes (len), file name (name), file type (type), and xmime type (xmime:contentType).

Download an attached file

With the attachment information retrieved, you can find the attachment id (href attribute) and then send a GET request to download the attachment file:

http://localhost:13080/SM/9/rest/incidents/IM10001/attachments/58ac7b3200242211802b29a8

If you test this using a browser, the browser should show that a file is being downloaded.

Delete an attachment

When sending the same URI with HTTP method DELETE instead of GET, the attachment is deleted.

This is impossible by using the browser address bar, and therefore either a tool or plugin is required, or implementation like HTML page with JavaScript implementation is required.

The following is a demonstration HTML page to demonstrate deleting an attachment by using a REST request.



Note

Your browser may not allow access because of security restrictions. To learn more, you can search for online information about "Cross-Origin Resource Sharing".

```
<!DOCTYPEl>
<html>
<head>
<script language="javascript">
function sendrequest(method, url, contenttype, user, password, sync,
request, callback)
{
// Create a XMLHttpRequest object
var ajax = null;
if(window.XMLHttpRequest){          //Google Chrome, Mozilla Firefox,
Opera, Safari, IE 10
ajax = new XMLHttpRequest();
}
else
{
return;
}
// Add data to ajax object
ajax.open( method, url, sync, user, password );
if (contenttype) ajax.setRequestHeader("Content-Type", contenttype);
if (!sync)
{
// Define how response will be processed
ajax.onreadystatechange = function()
{
var result = {};
switch (this.readyState)
{
case 0: result.status = "ReadyState: " + this.readyState; break;
case 1: result.status = "ReadyState: " + this.readyState; break;
case 2: result.status = "ReadyState: " + this.readyState; break;
case 3: result.status = "ReadyState: " + this.readyState; break;
case 4:
{
var result = {};
result.status = ajax.status;
result.response = ajax.responseText;
if (callback) callback(result);
break;
}
}
updatePage(result.status, result.response);
return result;
}
```

```
}
// send the request
if (request)  ajax.send( request );
else ajax.send();
if (sync)
{
var result = {};
result.status = ajax.status;
result.response = ajax.responseText;
updatePage(result.status, result.response);
if (callback) callback(result);
}
}
function updatePage(status, response)
{
document.getElementById("status").innerHTML=status;
document.getElementById("response").value=response;
}
</script>
</head>
<body>
<h1>Test SM RestfulAPI</h1>
Request: <textarea id="request" width="80" height="60"></textarea>
<p></p>
Status: <p id="status"></p>
Response: <textarea id="response" width="80" height="60"></textarea>
<script language="javascript">
// Defaults
var method = "GET"
var request = null;
var contenttype = null;
var sync = false;
// Credentials
var user = "falcon";
var password = "";
// TEST
var method = "DELETE";
var URL =
"http://localhost:13080/SM/9/rest/incidents/IM10001/attachments/58ac7b3
200242211802b29a8";
sendrequest(method, URL, contenttype, user, password, sync, request);
</script>
</body>
</html>
```

Attach a new file to an incident

Execute a POST request to attach a file to an incident identified by incident number.

Here is an example using ReadyAPI:

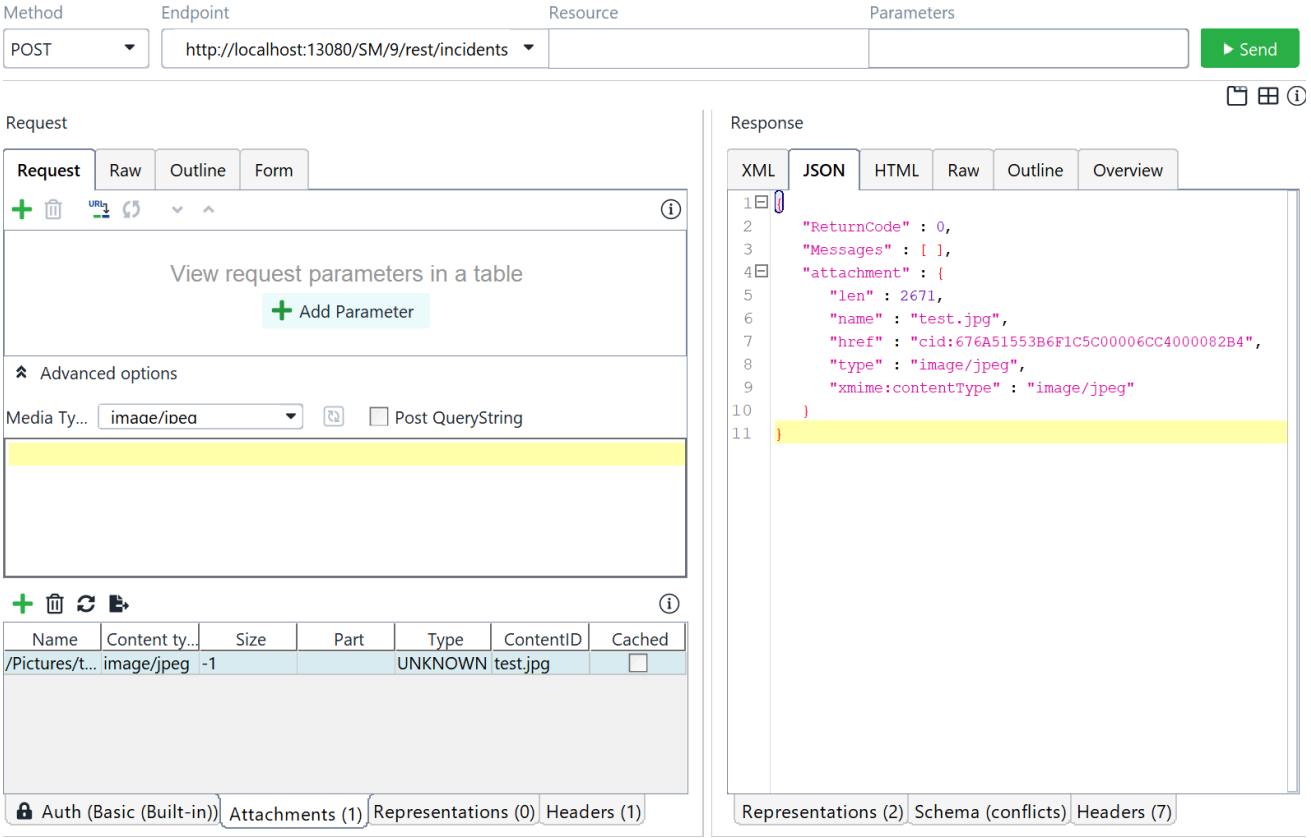
URL:
`http://<server>:<port>/SM/9/rest/incidents/<incidentid>/attachments`

Add a header:
Key: Content-Disposition
Value: `attachment;filename=<filename.filetype>`

Select a Media Type: `<content type>`

Attach a file to request.

If you have multiple attachments, add them one by one.



1.1.1.6. Web Services debugging parameters

Three parameters are most frequently used: debughttp:1, RTM:3, and debugdbquery:999. It may be useful to use the msglog:1 parameter to have all messages written to the sm.log as well, especially for Connect-It Web Services integrations. As a best practice, put these debug parameters on the dedicated Web Services port such as shown below:

```
sm -httpPort:13087 –debugnode –debughttp:1
```

The debughttp parameter

Add the debughttp in the server `sm.ini` file or in the dedicated servlet container line of the sm.cfg file, restart the Service Manager server and rerun the Web service application to invoke the debugging parameter.

For consuming Web Services, the debughttp parameter writes to two files in the Service Manager server log directory, http.log and writes additional information into the sm.log file.

An excerpt of the http.log file follows. (To determine which areas of the log file are for the Web service call, search for “sm7server/ws”. Regular client communication uses SOAP UI instead.)

```
POST /sm7webserver/ws HTTP/1.1
content-type: text/xml;charset=UTF-8
soapaction: "EnableNewEmployee"
user-agent: Jakarta Commons-HttpClient/3.0.1
host: <server>:<port>
content-length: 2033
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:pws="http://servicecenter.peregrine.com/PWS" xmlns:com=
"http://servicecenter.peregrine.com/PWS/Common">
<soapenv:Header/>
<soapenv:Body>
<pws:EnableNewEmployeeRequestQuoteRequest attachmentInfo="?
attachmentData="?" ignoreEmptyElements="true">
<pws:model query="?">
<pws:keys query="?">
<!--Optional:-->
<pws:number type="String" mandatory="?"
readonly="?">?</pws:number>
</pws:keys>
<pws:instance query="?" uniquequery="?" recordid="?">
<!--Optional:-->
<pws:Priority type="String" mandatory="?"
 readonly="?">?</pws:Priority>
<!--Optional:-->
<pws:Reason type="String" mandatory="?" readonly="?"
?</pws:Reason>
<!--Optional:-->
<pws:RequestingDepartment type="String" mandatory="?"
 readonly="?">?</pws:RequestingDepartment>
<!--Optional:-->
<pws:Requestor type="String" mandatory="?" readonly="?">?
</pws:Requestor>
<!--Optional:-->
<pws:Location type="String" mandatory="?" readonly="?">?
</pws:Location>
<!--Optional:-->
<pws:HireType type="String" mandatory="?" readonly="?">?
</pws:HireType>
<!--Optional:-->
<pws:attachments>
<!--Zero or more repetitions:-->
<com:attachment
href="Service_Manager:9.61_Codeless/webservicesguide/reference/?"
```

```
contentId="?" action="?
name="?" type="?" len="attachmentType="?"/>
</pws:attachments>
</pws:instance>
<!--Optional:-->
<pws:messages>
<!--1 or more repetitions:-->
<com:message type="String" mandatory="?" readonly="?"
severity="?" module="?">?</com:message>
</pws:messages>
</pws:model>
</pws:EnableNewEmployeeRequestQuoteRequest>
</soapenv:Body>
</soapenv:Envelope>
HTTP/1.1 401
Set-Cookie: JSESSIONID=94DCC5F90495E0202B84EFB1F998195A;
 Path=/sc62server
WWW-Authenticate: Basic realm="CASM"
Connection: close
Content-Type: textl;charset=utf-8
Content-Length: 40
Date: Wed, 21 May 2008 17:16:05 GMT
```

Interpreting the http.log

The http.log may contain encoded or compressed messages from the Windows or Web Client communication. See below to turn off SOAP compression and FastInfoset encoding. Web Services communications aren't encoded nor compressed.

The http.log contains information on all Processes/Threads that connect to the traced servlets. If more than one Process/Thread is traced, information in the log may overlap from different sessions, resulting in multiple POST or GET messages together rather than the POST – GET pair you would expect.

To turn off FastInfoset and compression for the clients, follow these steps:

1. On the shortcut starting the Windows client, add –DFastInfoset=false to the target.
2. The command line argument for the Service Manager Windows client is:

   -vmargs -option1 -option2
3. As an example, if -DFastInfoset=false is the only command line option ensure your command line is:

   -vmargs -DFastInfoset=false

4. Turn off Compress SOAP Messages on the File - Connect – Connections screen's Advanced tab.
5. For the Web client, turn off FastInfoset by setting the JAVA Option -DFastInfoset=false. As an example, to turn off FastInfoset on Tomcat 5.5 or later, follow these steps:
    1. Go to the Apache Tomcat x.x folder on the Start Menu.
    2. Start the Configure Tomcat application.
    3. Select the Java tab in the configuration dialog.
6. Turn off the Compress SOAP Messages by editing the ...\WEB-INF\web.xml file.

```
<!-- Compress network communication between the application server
and the HP Service Manager server -->
<init-param>
<param-name>compress_soap</param-name>
<param-value>false</param-value>
</init-param>
```

RTM:3 and debugdbquery:999

Sometimes the Web Service itself is working correctly, but actions performed by the Document Engine within Service Manager aren't performing as expected. (Error Message: soap_serve - Caught XML API exception scxmlapi(19) - Doc Engine call failed with cc -1)The RTM:3 and debugdbquery:999 parameters can expose such issues that occur within the application layer of Service Manager. The debugging information produced by these parameters can be found in the sm.log file in the Service Manager server RUN directory. It's not necessary to restart Service Manager to activate these debug parameters. Reconnecting the Web Service to Service Manager triggers the use of these debug parameters.

The allowwsdlretrieval parameter

This parameter is used to allow retrieval of the WSDL without having the SOAP UI license.

Contents

- 1 The debughttp parameter
- 2 Interpreting the http.log
- 3 RTM:3 and debugdbquery:999
- 4 The allowwsdlretrieval parameter

1.1.1.7. Web Services special parameters

The `internalURI` parameter enables you to load Web Service (WSDL) when your server can't access the Internet.

To work with Web Service without internet access, follow these steps:

1. Stop the Service Manager server.
2. Open the `sm.ini` file with a text editor. By default, the file is in the **`C:\Program Files\OpenText\Service Manager\Server\RUN`** folder on the Service Manager server.
3. Add the `internalURI` parameter into `sm.ini`.
4. Save your update.
5. Restart the Service Manager server.

Related topics

Enter a parameter in the sm.ini file

1.1.1.8. Testing your WSDL with a SOAP UI

To read the WSDL, go to **File > New Project** and enter a project name as well as the initial WSDL location and click on OK. The list of methods will be displayed on the left, the request is in the middle, the response on the right.

To pass authentication information, enter the Username and Password. If the password is blank, enter information and then remove the information again.

Note

SoapUI fills in each field value with a question mark symbol. For correct processing, remove these ? before submitting the request.

1.1.1.9. Running Web Services on a dedicated port (servlet)

To create a separate servlet within a horizontally or vertically scaled Service Manager system, add the debugnode parameter to the dedicated servlet container. The debugnode parameter stops the load balancer from distributing client load to this node. This can be used to set up a dedicated servlet for tracing and logging without adding an uncontrollable amount of load to that servlet. Another use is to create a dedicated servlet for a special purpose within the scaled solution. As an example, refer to the following `sm.cfg` file:

```
#load Balancer Port
sm -loadBalancer -httpPort:13080
#Ports for loadBalanced Connections
sm -httpPort:13081 -httpsPort:13082
sm -httpPort:13083 -httpsPort:13084
sm -httpPort:13085 -httpsPort:13086
#Port for Web Services
sm -httpPort:13087 -debugnode
```

Current limitations of running Web Services through the load balancer:

- The HTTP 307 redirect is not fully compliant with the specifications which can affect Web Services integrations through the Service Manager Load Balancer. The workaround is to connect directly to one of the Service Manager Application server servlets.
- Web Services through the Service Manager Load Balancer aren't possible when SSL is enabled on the server.
- Web Services through the Service Manager Load Balancer aren't possible for Web Services clients that can't handle a redirect.

1.1.1.10. Debugging SOAP errors

The best practice for troubleshooting SOAP errors is to start a new client connection process with a dedicated log file associated with it. Opening a new client connection process allows you to isolate any faulty client traffic from your regular client traffic.

1. To set up your system to debug SOAP traffic, do one of the following:

- Start HTTP debugging for the entire system. Type the following command on a single line in the sm.ini file and then save the file.

debughttp:1

Debug parameters in the sm.ini file affect all Service Manager processes and the log files record all send/receive messages. This method is not recommended for a busy server however, since you have to restart the server for the debugging parameter to take effect.

- Start a separate client connection process to troubleshoot your SOAP errors. Type the following command in the operating system command line:

sm -httpPort:unique portnumber -sslConnector:0 -debughttp:1
-log:../logs/debug.log

where

-httpPort identifies a port where Web Services clients can connect
-log defines a path to store the logs for this process

Normally, all client connection processes for a particular Service Manager installation use the parameter values listed in the sm.ini file. This means that all client connections share the same log file specified in the sm.ini file. By starting a new client connection process with a different log parameter value, you can isolate the logs for a particular group of clients. Choose a port number that's not likely to be used by any other process.

2. Recreate the error.
3. Review the http.log, sm.log and log files from the Web Services consumer or publisher that Service Manager is communicating with and server, and client log files for information about the SOAP error. The HTTP log is in the server's RUN folder. The server logs are in the path you specified with the log parameter. The client logs are located in the following paths:

- For a Web Services client, see your Web Services client log
- For a Web client, see the log filed specified in the log.properties file on the web tier system

- For a Windows client, see the .log on the Windows client system

SOAP messages: Debugging HTTP traffic problems

If, after reviewing the client logs, you discover that there is an error in the HTTP transfer of SOAP messages, you must manually enable the HTTP debugging option on the Service Manager server. This option allows you to trace all HTTP and SOAP messages between the Service Manager server and client. You can trace HTTP traffic in one of two ways.

- Trace all HTTP connections to the server
- Trace a dedicated connection to the server

To review all the HTTP traffic, you can enable the `debughttp` parameter from the Service Manager initialization file (`sm.ini`) file. This causes the server to record all messages sent from and to the server to the following log files.

- logs\sm.log
- RUN\TEST.log

This method of debugging SOAP messages traces all Service Manager processes, but significantly reduces system performance because the log files the server produces contain all HTTP traffic, including HTTP headers and attachments. For this reason it is recommended that you not enable this parameter on production systems, but rather in test environments only.

To review HTTP traffic use the -httpPort paremeter as in the sample below. In addition, you can create a dedicated log file for this connection. By starting a new Service Manager process and specifying a separate log parameter, you reduce the amount of system resources needed to produce debugging output. For example, you can enter the following command from the server OS command line to create a dedicated servlet and log file.

```
sm -httpPort:portnumber -debughttp:1 -log:../logs/debug.log
```

For *portnumber*, type a communications port number on which you want the server to use for SOAP requests. You can use the `-log` parameter to define a path to any log file you want.

SOAP messages: Debugging problems with RAD applications

If your review of the client logs reveals potential problems in the RAD applications, you can enable the logging of RAD application messages by adding the `rtm` startup parameter to the Service Manager initialization file. This parameter causes the server to record all application-generated messages to `sm.log` file. For example, to receive detailed information about the RAD applications, add the following command to the Service Manager initialization file (sm.ini).

rtm:3

You can use the RAD application logging messages to determine if any tailoring changes you made are the cause of SOAP faults.

Contents

- 1 SOAP messages: Debugging HTTP traffic problems
- 2 SOAP messages: Debugging problems with RAD applications

1.1.1.11. Debugging RESTful API

This section provides information about how to debug RESTful API issues.

Debugging parameters

The following three parameters are most frequently used for debugging RESTful API:

- debugrest

  Add debugrest in sm.ini or in the dedicated servlet container line of the sm.cfg file, restart the Service Manager and re-run the RESTful Web service application to invoke debugging parameter. It provides more detailed log trace for diagnostics.

- dao_threadsperprocess

  Similar to debugrest, it can be set in sm.ini or in the dedicated servlet container line of sm.cfg file.

  It means the maximum number of threads allowed concurrently running in the process for RESTful Web Service application. It can be defined in sm.ini and the default value is 80. It is better to start enough server threads to handle requests (suggest to maintain the buffer of 30%~ 40% spare capacities).

- dao_sessiontimeout

  Like debugrest, the dao_sessiontimeout parameter can be set in sm.ini or in the dedicated servlet container line of sm.cfg file.

  It indicates the seconds to wait before terminate the RESTful threads. It can be defined in sm.ini and the default value is 15 seconds. Unless the client sends subsequent requests within the timeout, the Service Manager server will recycle the session for re-use and re-allocate it on demand.

  If there is big divergence, it is recommended to connect several servlets with different thresholds.

It is also useful to use the msglog:1 parameter to have all messages written to the sm.log as well.

Web services with a proxy server

If you are using web services with a proxy server, incorrect proxy server settings will cause your web service calls to fail. For details, see Web Services with a proxy server.

1.1.1.12. Syntax for entity references in xml

Character represented	Entity Reference	xml code
>	greater than	>
<	less than	<
“	Quotation marks	"
&	ampersand	&
‘	apostrophy	'

1.1.1.13. Definitions, acronyms, and abbreviations

Term	Definition
Consuming	Using a Web Service by calling its methods, supplying the appropriate calling parameters
Publishing	Providing a service over the Web by making public the services operations and objects in a Web Service.
WSDL	Web Services Description Language, which is a standard, structured way of describing SOAP messages and Web Services
REST	Representational State Transfer.

1.1.1.14. Web Services resources

You can use the following resources to develop and publish your own Web Services.

The World Wide Web Consortium (W3C) has existed for almost ten years. Its objective is to develop common protocols and to recommend standards that promote Internet interoperability. There are over 400 member organizations who contribute to forming recommendations for standards and best practices among Internet developers. The W3C provides leadership in an array of Web technologies (including XML, HTML, and similar areas of interest) by creating working groups that gather and publish information and recommendations.

You can find the WSDL schema and SOAP schemas published and propagated by IBM and Microsoft at schemas.xmlsoap.org. The W3C has complete descriptions of the schema elements for both SOAP and WSDL. See the W3C Web site for the most recent working draft of SOAP and WSDL recommendations.

There are third party tool kits that simplify creating a Web Service. For example, Apache Axis and Microsoft Visual Studio .NET are development tool kits you can use to create a custom Web Services client directly from the Service Manager Web Services API WSDL.

If you are interested in examples of working Web service WSDL files, programmatic interfaces, tutorials, samples, and a list of available Web services, see the Xmethods Web site. Also see the resources listed below:

- *Service-Oriented Architecture : A Field Guide to Integrating XML and Web Services*, April 2004, Prentice Hall Publishing
- *Web Services: A Technical Introduction*, August 2004, Prentice Hall Publishing
- *Java Web Services*, March 2004, O'Reilly
- Apache Axis
- Microsoft Visual Studio .NET
- schemas.xmlsoap.org
- SOAP schemas
- World Wide Web Consortium
- Gzip Web site
- Apache Wink
- Representational state transfer
- Hypertext Transfer Protocol

1.1.2. Connect-It

Connect-It is an integration platform that exposes technology infrastructure data as common business objects. The Connect-It family of connectors leverages industry-standard protocols and connects with third-party information systems in order to integrate technology infrastructure products with external information systems to align business processes and synchronize relevant data.

Key features and benefits:

- Integrates ITSM solutions with almost any data source
- Leverages industry-standard protocols
- Helps Managers make informed business decisions
- Improves data quality and integrity
- Includes built-in best practice integrations
- Reduces implementation time
- Provides the power of an enterprise application integration (EAI) tool at the cost of a gateway

For more information, refer to the Connect-IT documentation available at the ITOM Practitioner Portal.



Note

The OOB design doesn't trigger activities when creating or updating data through an event (which is sent from CIT to SM). Hence, there is no log on the Activity tab.

1.1.3. ServiceCenter Automation (SCAuto)

SCAuto is a suite of interface products that integrate Service Manager to network and systems management tools. The SCAuto interface sends event messages to a Service Manager server over a TCP connection. SCAuto links service workflows in Service Manager to automated event collection tools, providing better and more proactive management and control. SCAuto allows for proactive incident prevention because network events can be detected often before a problem is noticed or reported by the business. You can use the SCAuto software development kit to build integrations to nearly application.

Key features and benefits:

- Integrates to nearly any application
- Uses an event-driven architecture
- Provides efficient resource overhead

Each SCAuto product requires a separate license. See the *ServiceCenter Automate (SCAuto) Software Development Kit (SDK)* for more information.

For more information about the supported operating systems and SM versions, see SCAuto SDK Support Matrix.

Related topics

Integrations
Service Manager email solutions

1.1.3.1. ServiceCenter Automate Software Development Kit

Introduction

Welcome to ServiceCenter Automate (SCAuto) Software Development Kit (SDK). This product is part of the suite of SCAuto interface products that integrates Service Manager with premier network and systems management tools. SCAuto SDK is used in the development of custom interfaces to Service Manager. It provides a simple application program interface (API) to interact with current Service Manager Rational Application Developer (RAD) applications.

Purpose

This guide describes how to implement SCAuto SDK for integration with Service Manager.

Audience

This guide is intended for system administrators who install and implement Service Manager.

Prerequisites

This guide assumes you have knowledge of the following:

- **Service Manager**
  Working knowledge of Service Manager applications, the Service Manager client-server environment, and the C programming language. For details, see the appropriate Service Manager documentation.
- **GUI environment**
  Working knowledge of a GUI or text-based environment.
- **Operating system**
  A thorough knowledge of the operating system where the product will be installed and implemented, as well as a basic understanding of Service Manager applications and Event Services.



Note
Service Manager installation requirements are specific to the machine where Service Manager is installed. For a list of requirements, see the respective installation guides.

SCAuto SDK Support Matrix

SCAuto SDK 9.80 supports the following operating systems with Service Manager version 9.80 and 9.7x.

Windows	• Windows Server 2019 • Windows Server 2022
Linux	• Red Hat Enterprise Linux 8.4, 8.6, 8.8, 8.10, 9.0, 9.2, 9.4 • Oracle Enterprise Linux 8.6, 8.8, 8.9, 8.10, 9.0, 9.3, 9.4 • Novell SUSE Linux Enterprise Server 15sp5 **Note:** All UNIX operating systems are deprecated from SCAuto SDK 9.70.

SCAuto overview

ServiceCenter Automate (SCAuto) provides Event Management services through a collection of automation products. These automation products enable you to integrate external applications with Service Manager. Generic Event Management services enable you to create meaningful events from their environment and applications. For example, you can create tickets, send out notifications after tickets are closed, manage network inventory automatically, and manage email between disparate external email programs. The intent is to enable communication from any external application with any Service Manager RAD application, without predetermining what that communication requires.

Listener

A dedicated Java-based listener (scautolistener) manages all connections between SCAuto and Service Manager. You can start scautolistener from the command line or by using an entry in the sm.cfg file. For details, see the Service Manager online help. For example, to start a listener on port 12690, you could use the following command:

```
sm -scautolistener:12690
```

After the listener starts, it accepts connections from SCAuto clients.



Note

While running, the listener appears with the user name `SCAutoListener-12690` from within Service Manager. You should stop it from there, if needed. You do not terminate existing clients when you stop scautolistener.

Internal Event Services

SCAuto products run on various supported platforms. Each product uses TCP/IP socket architecture to connect to a Listener that runs on the Service Manager server platform. After a connection is established with the Listener, a process on the server is created. The new process runs in a manner similar to that of an sm process in Service Manager. The new process reads and writes event records directly to the event files.

Service Manager events are written and read internally, using a set of RAD applications known as Event Services. Event Services provides functions such as event mapping and application scheduling within Service Manager. These services enable RAD applications to create events for external processing. They also enable external programs to schedule RAD applications asynchronously.

You can write or modify any RAD application to use Event Services to interact with external SCAuto applications.

External Event Services

Applications external to the Service Manager environment are provided with multiple options for interfacing with RAD applications.

File Monitor interface

The simplest option is a File Monitor interface, which is comprised of external event schedulers and event files. Using this option, Service Manager input events are written by the applications to an input event file. That file is read by an SCAuto external scheduler, and then forwarded to the Service Manager Event Manager. Service Manager output events are read from the Service Manager Event Manager by an SCAuto file monitor, and then placed in an application output events file. Each instance of an external scheduler is started for a different file, enabling you to batch events, create more parallelism, or have separation of external applications. Each file monitor can be started to process a different event or event list. This interface mode permits external programs to use standard file read-write privileges with a predefined record structure to communicate with Service Manager.

C function API

A second option is an application programming interface (API) and a run-time library that provide connection services and event services. Any program that can call a C function, and that can be called from a C environment, can utilize this API. Under this option, applications can connect to Service Manager with an `scauto_connect` call. You can then specify which events and Service Manager server your application requires. After connecting to Service Manager successfully, you can create, retrieve, and delete Service Manager events by issuing `scauto_create`, `scauto_query`, and `scauto_delete` calls. Before termination, your application issues an `scauto_disconnect` call. Applications can have multiple sessions with one or more Service Manager servers.

SDK

The SCAuto Software Development Kit (SDK) is used in the development of custom interfaces to Service Manager. It provides a simple API to interact with current Service Manager RAD

applications.

Platforms

The SCAuto SDK requires SCAuto Listener to be on the Service Manager server platform. Listener and an SDK application provide Event Management services, enabling external applications to be integrated with Service Manager. These generic services enable users to create meaningful events from their environment and legacy applications.



Note

To find the process for starting the listener for SCAuto, see the relevant Service Manager documentation.

SCAuto SDK is available for Microsoft Windows and most Linux platforms. The API is packaged as a C library on Linux and as DLLs on Windows. The Windows versions can also be used with Microsoft Visual Basic, Borland Delphi, or other products that can use DLL routines.

Protocols

SCAuto clients are not Service Manager clients. Rather, they use their own simpler network protocol to communicate with an Listener. The resulting applications are smaller. They do not need to be upgraded or recompiled to connect to newer Service Manager server versions. The TCP/IP network protocol is used to connect SCAuto SDK to Listener. TCP/IP must be installed and configured correctly on the systems where SCAuto applications are to be run.

Components

The SCAuto SDK includes the following optional component files:

- **Libraries**
  The link libraries and run-time libraries are included for some systems.
- **Header file**
  The scauto.h C header file is included for applications written in C.
- **Test program**
  A test program ensures the proper installation and communication between SCAuto and Listener.
- **Source code**
  The C source code is included for a sample SCAuto application.
- **File Monitor program**
  The SCAuto File Monitor program reads and routes system events. It can be run in several instances simultaneously.
- **Submit Monitor program**
  The SCAuto Submit Monitor program responds to Service Manager requests to run external programs.

Architecture

The general architecture is composed of an SCAuto SDK application and the supplied API. SCAuto SDK connects to Listener, which in turn connects to Service Manager. The SCAuto SDK application can then create events that are processed by an accompanying RAD application. The RAD application can also create events that can be retrieved and processed by the external SCAuto SDK application.

This event-driven technique allows asynchronous running of the external applications. This standard interface is currently being employed by SCAuto products.

Event Manager

Service Manager includes an Event Manager application. This application consists of an input event scheduler, as well as a set of standard tables for registration, mapping, and event handling. The input event scheduler polls the input events tables. It then routes new events to RAD applications by using registration and mapping tables. For details, see the ServiceCenter 6.1 Event Services Guide. The output event scheduler is called by any RAD application to create an event, and schedule it for external processing.



Note

In many cases, using these generic schedulers enables SCAuto implementation without affecting current production RAD applications. It also assists in new application development with a standard ready-made solution to many external interfacing requirements.

Installing SCAuto SDK

You can install SCAuto SDK on a Windows or Linux server. This section contains information about installation requirements and how to install
the SCAuto SDK.

This chapter provides the instructions for installing the SCAuto SDK in the Windows and Linux environments.

Topics in this chapter include:

- Windows Installation
- Linux Installation

Windows Installation

The bulk of the installation is done automatically.

Installing SCAuto SDK on Windows systems

Follow these steps to install the SCAuto SDK:

1. Log in to the Windows server as a user with local administrator privileges.
2. Insert the SCAuto installation media into the appropriate drive of the server.
   If you are installing on a system that has auto-run enabled, the installation media browser

starts automatically. If auto-run is disabled, follow these steps to start the installation media browser manually. The SCAuto SDK InstallShield wizard opens.
    1. Navigate to the installation media folder.
    2. Double-click setupwin32.exe.
3. Click **Next** to read and accept the licensing agreement.
4. Select the **I accept the terms in the License Agreement** option.
   The **Next** button becomes active.
5. Do one of the following:
    1. Click **Next** to accept the default installation folder.
       The default installation folder is:
       `C:\Program Files (x86)\OpenText\SCAuto SDK 9.80`
    2. Click **Browse** to choose a different installation location.

> **Note**
> You must install the SCAuto SDK in a folder containing only ASCII characters in the folder name. The SDK cannot start if installed in a folder with non-ASCII characters in the folder name.

    The installation process automatically adds a subfolder for the SCAuto SDK installed:
    - `\bin` contains executable programs
    - `\lib` contains the SCAuto SDK libraries
    - `\samples` contain sample source code and header files
6. Click **Install** to begin copying the installation files.
   You can stop the installation by clicking Cancel.
   A summary page opens when the installation is complete.
7. Click **Finish** to exit the wizard.

Uninstalling SCAuto Applications from Windows systems

Follow these steps to uninstall SCAuto SDK:

1. From the Windows main menu, click **Start > Settings > Control Panel > Add/Remove Programs**.
   The Add/Remove Programs dialog box opens.
2. Select SCAuto SDK and click **Change/Remove**. The SCAuto SDK InstallAnywhere wizard opens.
3. Click **Next**.
4. Review the uninstallation information.
5. Click **Next**.
6. Click **Uninstall** to remove the indicated features.
   You can stop the installation by clicking **Cancel**.
   A summary page opens when the installation is complete.
7. Click **Finish** to exit the Setup wizard.

Linux Installation

The bulk of the installation is done automatically for you.
Case sensitivity may be dependent on the Linux system that you are using.

Installing SCAuto SDK on Linux systems

Follow these steps to install the SCAuto applications:

> **Note**
> Before you install SCAuto SDK, you may need to run the following commands to install libstdc++ and glibc:
>
> - `yum install libstdc++.so.5`
> - `yum install glibc.i686`
> - `yum install zlib.i686`

1. Log on to the server.
2. Insert the SCAuto installation media into the appropriate drive of the server.
3. Mount the installation media.
4. CD to the mount location.
5. Run the appropriate setup script. For example, you could run:
   `./setupLinux.bin`
   The SCAuto SDK InstallShield wizard starts.
6. Click **Next** to read the licensing agreement.
   Console mode users chose 1 for **Next**, and press Enter to read and accept the licensing agreement.
7. Select the **I accept the terms in the License Agreement** option.
   Click **Next** to accept the licensing agreement.
   Console mode users press 1 to accept the terms of the license agreement. and press 0 to proceed.
   The SCAuto SDK Install Location screen opens showing the Directory Name.
8. Do one of the following:
   - Click **Next** to accept the default installation directory
   - Click **Browse** to choose a different installation location.
     The default installation folder is:
     `/opt/MicroFocus/SCAuto_SDK_9.80`

     > **Note**
     > You must install the SCAuto SDK in a directory containing only ASCII characters in the directory name. The SDK cannot start if installed in a directory with non-ASCII characters in the directory name.

     The installation process automatically adds a subdirectory for each SCAuto application installed:
   - `/bin` contains executable programs
   - `/lib` contains SCAuto SDK libraries

- `/samples` contains sample source code and header files
  Console users, specify a directory or press Enter to accept the default directory, and then press 1 to proceed to the next screen.
  The installation summary screen opens.

9. Click Install to start installing.
   You can stop the installation by clicking Cancel.
   Console users, press 1 to start installing.
   A summary page opens when the installation is complete.
10. Click Finish to exit the Setup wizard.
    Console users, press 3 to exit the wizard.

Uninstalling SCAuto SDK from Linux systems

Follow these steps to uninstall the SCAuto SDK:

1. CD to `<install_directory>/_uninst`
2. Run the uninstall program `./uninstall`.
   The SCAuto SDK InstallShield wizard opens.
3. Click **Next**.
4. Review the uninstallation information.
5. Click **Next**.
6. Click **Uninstall** to remove the indicated features.
   You can stop the installation by clicking **Cancel**.
   A summary page opens when the installation is complete.
7. Click **Finish** to exit the Setup wizard.

Utility programs

ServiceCenter Automatic (SCAuto) Software Development Kit (SDK) has two utility programs:

- File Monitor
- Submit Monitor

See Utility programs for more information.

Developing applications

To develop a ServiceCenter Automatic (SCAuto) Software Development Kit (SDK) application, see Developing applications.

C language API

The primary interface for ServiceCenter Automatic (SCAuto) Software Development Kit (SDK) consists of a small set of C functions, which are the same under all supported operating systems. See C language API for more information.

Troubleshooting

See Troubleshoot SCAuto SDK for tips of troubleshooting ServiceCenter Automatic (SCAuto) Software Development Kit (SDK).

1.1.3.1.1. Utility programs

ServiceCenter Automatic (SCAuto) Software Development Kit (SDK) has two utility programs:

- File Monitor
- Submit Monitor

File Monitor

The SCAuto File Monitor utility communicates with Service Manager through normal text files. External applications can create Service Manager input events by appending events to a file. File Monitor reads the events, and then forwards them to Service Manager Event Manager.

File Monitor reads Service Manager output events from Service Manager Event Manager, and places them in a file for external applications to read. You can start each instance of File Monitor on a different file. Doing so enables you to batch events, increase parallelism, and separate external applications. You can also start each instance of File Monitor to process a different list of events.

External applications use standard read-write operations to access File Monitor files, using a predefined record structure to communicate with Service Manager.

To operate File Monitor, you need a general understanding of Service Manager Event Services. For details, see Event Services.

Components

The SCAuto File Monitor utility includes the following components:

- `scfiled`
  File Monitor executable. On Windows, the file is called scfiled.exe.
- `scauto.msg`
  Optional component. Messages printed by the File Monitor and other SCAuto tools. You can customize this component for use with languages other than English.
- `scauto.dll`
  Windows-only component. SCAuto run-time library.

Command-line syntax

You can start the SCAuto File Monitor utility with the following syntax:

```
scfiled -input infile -output outfile -evlist eventlist
```

```
-server scauto server -id execution id [...]
```

The following parameters are allowed:

`-input <infile>`
Input events file. Events in this file are written to Service Manager. Youmust specify at

least one -input or -output parameter.

`-output <outfile>`

Output events file. Events from Service Manager are written to this file.

You must specify at least one -input or -output parameter.

`-evlist <event list>`

List of event types to be retrieved from Service Manager (only if the -output parameter is provided).

You can specify the event list as follows:

`"type"`

Retrieves a single event type.

`"(type1,type2,...)"`

Retrieves a list of event types.

`"all"`

Default. Retrieves all event types.

`-user <user name list>`

List of events that match the user name list.

You can specify the user name list as follows:

`"type"`

Retrieves a single user name type.

`"(type1,type2,...)"`

Retrieves a list of user name types.

`"all"`

Default. Retrieves all event types.

`-server <scauto server>`

Service name of the SCAuto server to which you want to connect. The default is scauto.

`-id <execution id>`

Execution ID for this Service Manager connection. This ID is used from within status in Service Manager to identify processes. This parameter is required. It has no default.

`-log <log file>`

Name of the file to which you want to log messages. The default is scfiled.log.

To log to the screen, use one of the following:

- *Linux*
  `/dev/tty`
- *Windows*
  `con`

`-checkpoint <file>`

Name of the file to store run-time checkpoints. This parameter enables File Monitor to continue where it left off when it was restarted. The default is scfiled.chk.

`-nocheckpoint`

Prevents a checkpoint file from being used.

`-noloop`
Terminates File Monitor when all pending events have been processed.
Normally, File Monitor sleeps and polls for new events.

`-sleep <secs>`
Number of seconds File Monitor sleeps before polling for new events. The default 10.

`-delete`

Causes Service Manager output events to be deleted when successfully written to the output file. Normally, the events are retained, and the checkpoint file is used to resynchronize when the File Monitor is restarted.

`-debug`
Switches on additional log messages.

`-msgfile`
Name of the message file to use. The default is scauto.msg.

Event Records

SCAuto Event Records are comprised of comma-delimited header data followed by the event fields. For descriptions of these fields, see **Event structure**.

Field Name	Max Length	Type
evtype	16	character
evtime	19	date
evsysseq	32	reserved
evusrseq	32	character
evsysopt	24	character
evuser	24	character
evpswd	24	character
evsepchar	1	character
evfields	32767	character

You can separate the header data and the event fields by a comma or a colon:

- Comma
  `evtype, evtime, evsysseq, evuserseq, evsysopt, evuser, evpswd, evsep char, evfields`

- Colon
  `evtype, evtime, evsysseq, evuserseq, evsysopt, evuser, evpswd, evsepchar:evfields`

Fields that are too long are truncated. However, exceeding the maximum length of evfields can be inefficient.

Example:
`pmo,06/11/1997 22:06:58,,866065020,,EventUser,,`

`^:logname^netname^866065020^^Testing Problem`

`Open^^^^^^^^^^^^^^^^^^`



Note
You do not need to fill in all fields. Fields that are not filled in are indicated by double commas (,,). In the example, the empty evsysseq, evsysopt, and evpswd fields are indicated by double commas.

Submit Monitor

The SCAuto Submit Monitor utility enables Service Manager to execute external commands through the Event Manager. The commands are executed on the same machine as that of the Submit Monitor. This machine may be a different machine from that of the Service Manager. You can use this utility to automate and customize Service Manager applications.

The Submit Monitor uses submit events. You specify the command and arguments to execute, as well as the name of the host machine on which the command should execute. For the command to execute, the local host name must match that specified in the event.

Components

The SCAuto Submit Monitor utility includes the following components:

`scsubmit`
: Submit Monitor executable. On Windows, the file is called scsubmit.exe.

`scauto.msg`
: Optional component. Messages printed by the Submit Monitor and other SCAuto tools. You can customize this component for use with languages other than English.

`scauto.dll`
: Windows-only component. SCAuto run-time library.

Command-line syntax

You can start the SCAuto Submit Monitor utility with the following syntax:
`scsubmit -server scauto server [...]`

The following parameters are allowed:

`-server <scauto server>`
Service name of the SCAuto server to which you want to connect. The default service name is scauto.

`-user <user name list>`
Only retrieve events that match user name.
You can specify the user name list as follows:
`"type"`
Retrieves a single user name type.
`"(type1,type2,...)"`
Retrieves a list of user name types.
`"all"`
Default. Retrieves all user name types.

`-log <log file>`
Name of file to which you want to log messages. The default is scsubmit.log.

`-sleep <secs>`
Number of seconds the Submit Monitor sleeps before polling for new events. The default is 10 seconds.

`-debug`
Switches on additional log messages.

`-msgfile`
Name of the message file to use. The default is scauto.msg.

Submitting events

With the SCAuto Submt Monitor utility, you can submit events with the following syntax:

```
hostname^command^arg1^arg2^...
```

The following parameter, command, and arguments are allowed:

`<Hostname>`
Name of the machine on which the command should be executed.

`<command><args>`
Command and arguments are joined together, separated by spaces, to compose the full command line to execute.



Tip
There is no special formatting involved. Also, there is no requirement that you place each argument in a separate field.

No error checking is performed on the executed commands. Error checking is performed by the command itself.

If the command does not redirect its output, Submit Monitor tries to append the output of the command to its own log file:

- *Linux*
  The executed command is placed in the background. An ampersand (&) is not added to the command.
- *Windows*
  The processing of submit events pauses until the executed command finishes. For this reason, you should write the command to execute quickly.

The executed command inherit the execution environment of Submit Monitor.
Make sure that Submit Monitor has access to the commands you intend to execute from Service Manager, as well as the capabilities to execute them.

1.1.3.1.2. Developing applications

This chapter describes how to develop a ServiceCenter Automatic (SCAuto) Software Development Kit (SDK) application:

- Sample application
- Building applications

Sample application

This section includes a sample SCAuto SDK application with annotations.

This sample is very similar to the sample application code that is included with SCAuto SDK.

Opening the session

When run, the application expects an -s server option on the command line:

```
/*
* SCAuto SDK API sample program
*
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <errno.h>
```

Defining prototypes

The application must include scauto.h, which defines prototypes for all of the SCAuto functions, as well as for event structure and error codes:

`#include "scauto.h"`

The session identifier used throughout this program is defined as follows:

```
#define SESSID "SCAuto_test"
char server[256];/* SCAuto Listener to connect to */
void cleanup(int);
void print_event(char *header, struct scauto_event *event);
void parms_init(int argc, char *argv[]);
void show_error(int code);
int
main (int argc, char *argv[])
{
```

Passing the event structure to and from calls

The event variable passes an event structure to and from SCAuto calls:

```
struct scauto_event *event, the_event;
int status;
event = &the_event;
parms_init(argc, argv);/* parse command line */
#ifdef UNIX
signal(SIGINT, cleanup);
signal(SIGTERM, cleanup);
signal(SIGQUIT, cleanup);
#endif
```

The status variable stores the return value of SCAuto calls.

Connecting to the remote SCAuto Listener

The application connects to the remote SCAuto Listener by using the scauto_connect function:

```
/* Connect to the remote SCAuto Listener */
printf("<<<<<< scauto_connect to SCAuto Listener >>>>>>\n");
status = scauto_connect(SESSID, server, "all", "all");
if (status != 0)
{
printf("%s: scauto_connect() failure.\n", argv[0]);
show_error(status);
exit(4);
}
printf("scauto_connect() ok.\n\n");
```

The first two parameters specify the session identifier and server to which to connect. The last two all parameters indicate that the application lists events
with any event type and from any user. The application also checks the return value from scauto_connect for errors. If it finds errors, it displays a diagnostic message.

Creating an event

To create an event, you must fill in an event structure:

```
/* Create an event */
printf("<<<<<<< Create events processing >>>>>>\n");\
strcpy(event->evtype, "email");
strcpy(event->evtime, "");/* leave blank and SCAuto Listener will fill
it in */
strcpy(event->evsysseq, "");/* this will be ignored */
strcpy(event->evusrseq, "12345678901234");
strcpy(event->evsysopt, "");
strcpy(event->evuser, "Testapi");
strcpy(event->evpswd, "");
event->evsepchar = '^';
event->evfields = "falcon^SCAuto/sdk^^Test SCAuto^Hey, this worked!";
/* utility routine to print out an event */
print_event("Create event", event);
```

Checking the return status for errors

The application passes the event to scauto_create, and then checks the return status for errors:

```
status = scauto_create(SESSID, event);
if (status != 0)
{
printf("%s: scauto_create_event() failure.\n", argv[0]);
show_error(status);
scauto_disconnect(SESSID);/* be sure to clean up! */
exit(4);
}w
printf("scauto_create() ok.\n\n");
```

Because an SCAuto connection is open, the application calls `scauto_disconnect` before exiting.

Retrieving events with wait processing

To retrieve an event, the application calls one of the following:

- `scauto_query`
  Retrives the first event in the eventout file. The event argument contains the event, if there is one.
- `scauto_query_ex`
  Enables you to specify a query string in P4 format and look for event that match the szQuery string.
  Example:

`szQuery = "evusrseq=\"12345678901234\""`

To get the matched event, you need to call scauto_get_next.

Retrieving the first event

To retrieve the first event, you use scauto_query and scauto_get_next:

```
printf("<<<<<<< Retrieve events with wait processing >>>>>>\n");
status = scauto_query(SESSID, "0000000000000000", "wait", event);
if (status != 0)
{
if (status != NOEVFND)
{
printf("%s: scauto_query() failure.\n", argv[0]);
show_error(status);
scauto_disconnect(SESSID);
exit(4);
}
else
{
printf("%s: No events found, exiting.\n", argv[0]);
scauto_disconnect(SESSID);
exit(4);
}
}
print_event("Retrieved event ok", event);
```

In this instance, the `0000000000000000` parameter is the starting `evsysseq` value on which to query. This value causes `scauto_query` to retrieve the first event in the eventout file. The application tests for `NOEVFND` explicitly because this value does not indicate an error. Normally, applications do not need to check for `NOEVFND` after calling `scauto_query` with the wait option.

Retrieving the next event

To retrieve the second event, you use scauto_get_next:

```
printf("\n<<<<<<< Get next event processing >>>>>>\n");
status = scauto_get_next(SESSID, event);
if (status == 0)
{
print_event("Retrieved event ok", event);
}
else
{
if(status != NOEVFND)
{
printf("%s: scauto_get_next() failure.\n", argv[0]);
show_error(status);
scauto_disconnect(SESSID);
exit(4);
}
else
printf("No more events found.\n");
}
}
```

You check for `NOEVFND` explicitly because it is a very common return code for `scauto_get_next`. If you want the application to continue and retrieve all events, you can put `scauto_get_next` into a loop. The application then checks the return status to determine when there are no more events to retrieve.

Finding a matching event

To find a matching event, you use scauto_query_ex and scauto_get_next:

```
// Query for the Event using the new Query API. Here we're doing a
query
// for event that has a matching evusrseq number of 12345678901234
status = scauto_query_ex(SESSID, "evusrseq=\"12345678901234\"","wait");
if (status != 0)
{
if (status != NOEVFND)
{
printf("%s: scauto_query_ex() failure.\n", argv[0]);
show_error(status);
scauto_disconnect(SESSID);
exit(4);
}
else
{
printf("%s: No events found, exiting.\n", argv[0]);
scauto_disconnect(SESSID);
exit(4);
}
}
```

In this instance, the question mark (?) query string is the following:

`evusrseq="12345678901234"`

If a match event is found, you call scauto_get_next to retrieve the event.

Retrieving a matching event

To retrieve a matching event from the eventout file, you call `scauto_get_next`:

```
// This gets the event out of the ServiceCenter EVENTOUT queue
printf("\n<<<<<<< Get next event processing >>>>>>\n");
status = scauto_get_next(SESSID, event);
if (status == 0)
{
print_event("Retrieved event ok", event);
}
else
{
if(status != NOEVFND)
{
printf("%s: scauto_get_next() failure.\n", argv[0]);
show_error(status);
scauto_disconnect(SESSID);
exit(4);
}
else
printf("No more events found.\n");
}
print_event("Scauto_query_ex Retrieved event ok", event);
```

Closing the session

When the application is ready to exit, or when it is finished with the SCAuto session, you disconnect the session:

```
/* Disconnect from SCAuto Listener */
printf("\n<<<<<<< Disconnect SCAuto >>>>>>\n");
status = scauto_disconnect(SESSID);
if (status != 0)
{
printf("%s: scauto_disconnect failed.\n", argv[0]);
show_error(status);
}
printf("\ntests completed ok\n\n");
return (0);
}
```

Explicitly disconnecting enables the remote Listener to close the session gracefully and free up any used resources.

Printing an event

To print an event, you run the following utility routine:

```
void print_event(char *header, struct scauto_event *event)
{
printf("%s: evtype=%s, evtime=%s, evsysseq=%s, evusrseq=%s, ", header,
event->evtype, event->evtime, event->evsysseq, event->evusrseq);
printf("evuser=%s, evpswd=%s, evsepchar=%c, ",
event->evuser, event->evpswd, event->evsepchar);
printf("evfields=%s\n", event->evfields);
}
```

Defining the server

To parse the command line for the -s server parameters, you use the following two routines:

```
void
usage(char *name)
{
printf("usage: %s -s server\n", name);
exit(0);
}
```

```
void
parms_init(int argc, char *argv[])
{
/* Getopt would make this cleaner, but it's not included on many
systems */
int i;
strcpy( server,"");
for (i = 1; i < argc; i++)
{
if (strcmp(argv[i], "-s") == 0 && argv[i+1])
{
i++;
strcpy(server, argv[i]);
}
else
usage(argv[0]);
}
/* verify that a server argument was given */
if(!server[0])
{
printf("no server specified.\n");
usage(argv[0]);
}
}
```

Disconnecting from the server

When the application receives a signal (as specified during initialization), it calls the following routine:

```
void
cleanup(int ignored)
{
scauto_disconnect(SESSID);
printf("terminated.\n");
exit(1);
}
```

This routine ensures that scauto_disconnect is called.

Printing a diagnostic

To print out an appropriate diagnostic when passed an SCAuto error code, you call the following routine:

```
void show_error(int code)
{
if (errno)
perror(scauto_error_string(code));
else
fprintf(stderr, "%s\n", scauto_error_string(code));
}
```

The function `scauto_error` does most of the work.

Building applications

To build an SCAuto application, you must link it with the appropriate SCAuto library, as well as with any operating system-dependent libraries. How you link the application to these libraries depends to a large extent on the operating system and compiler being used.

This section describes sample makefiles for some systems. The sample makefiles that come with SCAuto SDK may be more detailed.

Windows applications

The SCAuto library includes two components for import libraries:

- `scauto.lib`
  Import library for Microsoft Visual C++ only.
- `scauto.dll`
  DLL file for other compilers. You use this DLL file to build an import library at run time. For example, you can create an scauto.lib that works with Borland C with the command: `implib scauto.lib scauto.dll`

scauto.lib

On Windows, the makefile that builds the SCAuto application might look like this:

```
# This is a makefile for building the sample SCAuto application.
!include <win32.mak>
CFLAGS = $(cflags) $(cvarsmt) $(cdebug) -I..\lib
LFLAGS = $(conlflags) $(ldebug)
LIBS = $(conlibsmt) ..\lib\scauto.lib wsock32.lib
sdktest.exe: sdktest.obj
$(link) $(LFLAGS) /OUT:sdktest.exe sdktest.obj $(LIBS)
```

scauto.dll

To run properly, SCAuto applications for Windows must access the SCAuto DLL files. These DLL files are in the LIB subdirectory, labeled SCAUTO.DLL.

When an SCAuto application runs, it searches for these DLL files in the following places:

- Same directory as the application executable file
- Windows system directory:
  `\Windows\system`
  `\windows\system32`
- All directories specified in the `PATH` environment variable
  The same DLL files can be used by all SCAuto products. If you have previously installed an SCAuto product, these DLL files may already be set up correctly.
  If you plan to use a single SCAuto application, simply copy these DLL files to the directory that contains your SCAuto application executable.
  If you plan to develop multiple SCAuto applications, do one of the following:
  - Add the SCAUTO\LIB directory to your PATH environment variable.
  - Copy the DLL files to a directory names in the PATH variable.
  - Copy the DLL files to the Windows system directory.

Linux applications

By default, many Linux compilers do not accept ANSI C code. For this reason, make sure that the correct flags are passed on the command line to switch on
ANSI C mode. Some systems also require explicit links to network libraries. The SCAuto library is named libscauto.a.

On Linux, a makefile might look like this:

```
# A sample makefile for HPUX Linux.
# HPUX defines
# CFLAGS= -
Aa -
I../lib
LDFLAGS=
sdktest: sdktest.o
$(CC) $(CFLAGS) sdktest.o ../lib/libscauto.a $(LDFLAGS) -o sdktest
```

1.1.3.1.3. C language API

The primary interface for ServiceCenter Automatic (SCAuto) Software Development Kit (SDK) consists of a small set of C functions, which are the same under all supported operating systems. These functions enable you to create, delete, and read Service Manager events. To use the C functions effectively, you should have at least a conceptual understanding of events and the Event Manager.

One important concept used by the C functions is a session. An SCAuto SDK session corresponds to a connection with SCAuto Listener. Each SCAuto SDK session appears as a distinct session from within Service Manager. When a session is created, it is given an identifying name. This name is then passed as a parameter to subsequent SCAuto SDK function calls.

Each primary SCAuto SDK function takes a session name as a parameter, along with other needed parameters. If it succeeds, each function returns the integer 0. If it does not succeed, it returns a special error code. After each function call, you should check these error codes.

Each C program that uses SCAuto SDK function calls should include the C header file scauto.h. This header file declares prototypes for all SCAuto SDK functions. It also defines error codes that can be returned. The header file is set up for compilers that can process standard C (also known as ANSI or ISO C). Some compilers require that you specify a flag to switch on ANSI C compatibility mode. Other compilers use standard C by default.

Event structure

The structure definition for the scauto_event type looks like this:

```
struct scauto_event
{
char* evfields;
char evtype[17];
char evtime[20];
char evsysseq[33];
char evusrseq[33];
char evsysopt[25];
char evuser[25];
char evpswd[25];
char evsepchar;
};
```

This structure type is used for variables that hold an event definition. It is used to pass events to and retrieve events from SCAuto.

The fields in the structure definition correspond to fields of the same name within Service Manager Event Services. Except for evsepchar, all strings are standard null-terminated C

strings. The fields are defined as follows:

`evtype`
Type of event (for example, pmo for problem open, e-mail for electronic mail, and so on). For a list of standard event types, see Event Services. You can also create custom event types.

`evtime`
Time the event occurred. The time should be in the format that the event scheduler expects. This format varies, depending on the operator record of the event scheduler, as well as the defaults of Service Manager. Normally, the event scheduler uses UTC time (also known as Greenwich Mean Time) and the format MM/DD/YYYY HH:MM:SS. If the field is empty while scauto_create is called, it is filled in automatically.

`evsysseq`
Reserved sequence number for the event. This number is a unique keyed field. You can save the field and then use it in scauto_query to start the query after the last retrieved event. Because the number is unique, scauto_create ignores any values placed in the field and creates its own.

`evusrseq`
User-specified sequence number. This number may be used for any purpose the application needs. In particular, it may be used to track an input event to an output event. All standard events that create an output event copy the evusrseq field from the input event.

`evsysopt`
Reserved field.

`evuser`
User or application for which the event is scheduled.

`evpswd`
Application password, if required.

`evsepchar`
Application separator character to use for parsing evfields. Normally, this field is set to a caret (^) character. It should keep that value, if possible, for consistency.

`evfields`
All of the application-specific data. For the standard event types, all sub-fields within this field are separated by the character specified in evsepchar. The filed is not a fixed-length string. Rather, it is a pointer to an allocated string of any length. The program that allocates this value is also responsible for freeing the string.

Error codes

If they succeed, SCAuto SDK functions return 0. If they fail, they return an error code. Symbolic names for these error codes are defined in the **scauto.h** C header file.

For a complete list of errors in the SCAuto SDK, please refer to the **scauto.h** C header file.

Symbolic error codes

Each function description in Functions on page 49 lists several possible error codes that it may return. SCAuto SDK functions `scauto_error_string` and `scauto_error` can provide additional error information.

SCAuto SDK functions use the following symbolic error codes:

`NOCONN (800)`
No connection could be established.

`INVID (801)`
An invalid session identifier was passed as a parameter.

`LICENCE (803)`
License check failed.

`STORERR (807)`
Error while allocating memory.

`DUPEID (808)`
Session identifier was not unique.

`NOEVFND (809)`
No more events were found.

`NOQUERY (810)`
Function `scauto_query` was not called before the function `scauto_get_next`.

`SYNTAX (812)`
Network protocol error occurred.

`DELERR (813)`
Service Manager could not delete an event.

`INSERR (814)`
Service Manager could not create an event.

`BADREQ (816)`
Command was not understood by Listener. It is possible that the server needs to be upgraded.

Networking error codes

SCAuto SDK uses codes to indicate networking errors. For these errors, additional information is available from the scauto_error function.

SCAuto SDK uses the following networking error codes:

`NETERR_SOCKINIT (101)`
Error while initializing networking libraries. Make sure that networking libraries are installed.

`NETERR_GETHOST (102)`
Hostname lookup failed. Make sure that the hostname passed to `scauto_connect` is correct,

and it exists in the system hosts file.

`NETERR_GETSERV (103)`
Service name lookup failed. Make sure that the service name passed to `scauto_connect` is correct, and it exists in the system services file.

`NETERR_SOCKET (104)`
Could not create a network socket.

`NETERR_CONNECT (105)`
Connection to a remote Listener failed. Make sure that the server is running, and that the network is up.

`NETERR_BIND (106)`
Could not bind to a network address.

`NETERR_SOCKSEND (107)`
Error while sending data over the network.

`NETERR_SOCKRECV (108)`
Error while reading data from the network.

`NETERR_TIMEOUT (109)`
Timeout while waiting for server response. Timeouts occur if the SCAuto client tries to connect to a Service Manager server instead of an SCAuto server.

`NETERR_LOSTCONN (110)`
Remote connection closed or server process died.

Functions

This section lists SCAuto SDK functions, as well as the possible error codes for each function.

scauto_connect

NAME

`scauto_connect()` - connect to Service Manager.

SYNOPSIS

```
#include "scauto.h"

int scauto_connect(char *id, char *server, char *events, char *users);
```

DESCRIPTION

The `scauto_connect` function connects the application to Service Manager.
This function must be performed before any other SCAuto function that takes a session identifier as a parameter. Use `scauto_disconnect` to close and abandon all sessions opened with `scauto_connect`.

`id`
Name of the SCAuto session that is created. This name is used to uniquely identify the session

in subsequent SCAuto function calls.

`server`
Host name and service name of the Listener to which you want to connect.
Separate the hostname and service name with a single period (.). If you specify a service name without a hostname, the local machine is assumed.
For example, using joe.scauto1 with this parameter means that Listener is sought on the machine joe by using the service name scauto1.

`events`
Event type or list of event types to retrieve when calling subsequent `scauto_query` and `scauto_get_next` functions. A value of all causes queries to fetch all event types. You can specify multiple event types in the format `(type1,type2,...)`.

`user`
User name or list of user names to retrieve on subsequent queries. A value of all causes events owned by any user to be retrieved. You can specify multiple user names in the format (user1,user2,...).

`RETURN VALUE`
If it competes successfully, the scauto_connect function returns 0.
Otherwise, it returns an error code. To get extended error information, call scauto_error.

`ERRORS`
The scauto_connect function can return the following symbolic error codes:

`DUPEID`
ID passed is in use by another session.

`NOCONN`
Connection could not be created.

`STORERR`
Problem allocating memory.

`NETERR_CONNECT`
Problem connecting to a remote server. Make sure that the server is running, and that the network is up.

`NETERR_TIMEOUT`
Timeout waiting for a server response. Make sure that server refers to a SCAuto server, not a Service Manager server.

scauto_disconnect

NAME
`scauto_disconnect()` - disconnect from Service Manager.

SYNOPSIS
`#include "scauto.h"`

```
int scauto_disconnect(char *id);
```

DESCRIPTION
The `scauto_disconnect` function closes a connection to Service Manager, and de-allocates any resources used by the session.

`id`
Identifier for the session to be closed. This identifier must be the same as that passed to `scauto_connect`.

`RETURN VALUE`
If it completes successfully, the scauto_disconnect function returns 0.
Otherwise, it returns an error code.

`ERRORS`
The `scauto_disconnect` function can return the following error codes:

`NOCONN`
No previous connection was established.

`INVID`
No session with the name id was found.

scauto_query

NAME
`scauto_query()` - retrieve a Service Manager event.

SYNOPSIS

```
#include "scauto.h"
int scauto_query(char *id, char *startseq, char *options, struct scauto_event *result);
```

DESCRIPTION
The scauto_query function retrieves an event, starting with the sequence number specified by the startseq parameter. Only events that match the event types and user names specified in the scauto_connect call are retrieved. Only one event is retrieved with this function. To retrieve subsequent events, use scauto_get_next.

`id`
SCAuto session identifier.

`startseq`
Starting sequence number for queried events. Only events with higher sequence numbers (the evsysseq field) are retrieved. This number can be saved and used when restarting an application to resume processing events from where it left off.

`options`
A value of wait causes the query to wait until an event is available for retrieving. If no events are available, scauto_query immediately returns an error code of NOEVFND.

result
When scauto_query returns, the retrieved scauto_event structure as defined in scauto.h.
The wait option is ignored if used on Microsoft Windows 3.1.

RETURN VALUES
If it completes successfully, the scauto_query function returns 0, and the result points to a valid scauto_event structure. Otherwise, it returns a symbolic error code.

ERRORS
The scauto_query function can return the following error codes:

NOEVFND
No matching event was found.

INVID
Invalid session identifier was passed.

STORERR
Problem allocating memory.

scauto_query_ex

NAME
scauto_query_ex() - retrieve a Service Manager event based on the szQuery option.

SYNOPSIS
```
#include "scauto.h"

scauto_query_ex(char *id, char *szQuery, char *options);
```

DESCRIPTION
The scauto_query_ex function queries for an event by looking for the event that matches the szQuery string.



Note
To retrieve the event and any subsequent events, use scauto_get_next.

id
SCAuto session identifier.

szQuery
Query string (for example, "evusreq=\"12345678901234\"").

options
A value of wait causes the query to wait until an event is available for retrieving. If no events are available, scauto_query_ex immediately return an error code of NOEVFND.

RETURN VALUES
If the scauto_query_ex function completes successfully, it return 0.
Otherwise, it returns a symbolic error code.

ERRORS
The scauto_query_ex function can return the following error codes:

NOEVFND
No matching event was found.

INVID
Invalid session identifier was passed.

STORERR
Problem allocating memory.

scauto_get_next

NAME
scauto_get_next() - retrieve next Service Manager event.

SYNOPSIS
```
#include "scauto.h"

int scauto_get_next(char *id, struct scauto_event *result);
```

DESCRIPTION
The scauto_get_next function retrieves an event after an scauto_query or scauto_query_ex function call. For example, scauto_query is called to retrieve the first event, and scauto_get_next can then be repeatedly called to retrieve the remaining matching events. By contrast, scauto_query_ex is called to query for events, scauto_get_next is called to retrieve the first event, and then scauto_get_next can be repeatedly called to retrieve the remaining matching events. Typical usage is to call scauto_get_next repeatedly in a loop until it returns NOEVFND.

id
SCAuto session identifier.

result
Pointer to a retrieved event if scauto_get_next is successful.

RETURN VALUES
If the scauto_get_next function completes successfully, it returns 0, and result points to a valid scauto_event structure. Otherwise, it returns a symbolic error code.

ERRORS
The scauto_get_next function can return the following error codes:

NOEVFND
No matching event was found.

INVID
Invalid session identifier was passed.

`STORERR`
Problem allocating memory.

scauto_create

NAME
`scauto_create()` - create a Service Manager event.

SYNOPSIS
```
#include "scauto.h"

int scauto_create(char *id, struct scauto_event *event);
```

DESCRIPTION
The `scauto_create` function creates a new Service Manager event. The Event Manager in Service Manager processes the event, and executes the requested RAD applications.

`id`
Identifier for this session.

`event`
Pointer to an `scauto_event`. Unused fields should be initialized to null strings. The `evsysseq` field is ignored.

`RETURN VALUE`
If the `scauto_create` function completes successfully, it returns 0. Otherwise, it returns a symbolic error code.

`ERRORS`
The `scauto_create` function can return the following error codes:

`INVID`
Invalid session ID was passed.

`INSERR`
Service Manager returned an error when trying to create the event. Check the Service Manager log file for possible error messages.

scauto_delete

NAME
`scauto_delete()` - delete an event in Service Manager.

SYNOPSIS
```
#include "scauto.h"

int scauto_delete(char *id, struct scauto_event *event);
```

DESCRIPTION
The `scauto_delete` function deletes the event that matches the passed event parameter. The event is deleted from the Event Manager output events, not from the input events.

The event passed should be an event that was retrieved previously by using `scauto_query` or `scauto_get_next`. Because other applications may be using this event at the same time as your application, make sure that yours is the only application using this event record before deleting it.

`id`
SCAuto session ID.

`event`
Pointer to the event to delete.

`RETURN VALUE`
If the `scauto_delete` function completes successfully, it returns 0. Otherwise, it returns a symbolic error code.

`ERRORS`
The `scauto_delete` function can return the following error codes:

`INVID`
Invalid session ID was passed.

`NOEVFND`
Event to delete was not found.

`DELERR`
Service Manager returned an error while trying to delete the event. Check the Service Manager log file for possible error messages.

scauto_error

NAME
`scauto_error()` - return OS- or network-specific error code.

SYNOPSIS

```
#include "scauto.h"

int scauto_error(char * id);
```

DESCRIPTION
Sometimes, an SCAuto error occurs because of error indications in the operating system or network interface. These errors are saved for each SCAuto session, and can be retrieved with the `scauto_error` function. These codes can provide more detailed information than the error codes returned from SCAuto functions. The codes are invaluable when developing and debugging applications. On Linux operating systems, the errno value is saved. On Windows operating systems, the `GetLastError()` value is saved.

`id`
Identifier for the SCAuto session from which the error is to be retrieved.

`RETURN VALUE`
The appropriate saved error code, if any, is returned. If the previous SCAuto function did not

have an error, or if there was no appropriate error code to save, then 0 is returned. If the passed session ID is invalid, then -1 is returned.

`ERRORS`
The specific error codes returned are specific to the operating system used.

scauto_error_string

NAME
`scauto_error_string()` - return a descriptive string for an SCAuto error code.

SYNOPSIS
```
#include "scauto.h"

char *scauto_error_string(int error);
```

DESCRIPTION
The `scauto_error_string` function takes an SCAuto error code as a parameter, and returns a short descriptive string for that error. This function is intended for debugging convenience, as well as for providing descriptive error messages to end users. No locale is used. Currently, only English-language messages are provided in the library.

`error`
Code for which a description is returned.

`RETURN VALUE`
A null-terminated character string with no embedded carriage returns.

`ERRORS`
None.

scauto_version

NAME
`scauto_version()` - return string describing current SCAuto version.

SYNOPSIS
```
#include "scauto."

char *scauto_version(void);
```

DESCRIPTION
The `scauto_versio` function returns a string that describes the current SCAuto version as well as the date the SCAuto library or DLL was built. It is intended to be used for debugging or for presenting descriptive text on application startup.

RETURN VALUE
A null-terminated character string constant containing the version.

ERRORS
None.

1.1.4. Event Services

Service Manager Event Services is an interface between Service Manager and external systems. It is the preferred mechanism for interfacing Service Manager to external systems. It runs on Microsoft Windows and Linux platforms, doesn't require an existing client session to operate, and can be configured to run as a background processor.

Related topics

Event blocking
Event mapping process
Event scheduling
Event Services and Change Management
Generating email messages
Event Services and Service Manager e-mail
Event Services operations
Event Services routing
Event Services tables
Event Services workflow
View event schedules
View Event Services agent status
View Event Services input events
View Event Services output events
View registered events
Standard events

1.1.4.1. Event Services overview

Event Services provides standard applications to:

- Open, update, and close Service Manager incidents
- Open, update, and close Service Manager calls
- Add, update, and delete Configuration Items
- Open, update, approve, and close changes

Standard applications are available within Service Manager to generate outbound information, such as e-mails. Standard applications come with out-of-box forms, or you can tailor both forms and operations to meet your needs. You can also modify Event Services to perform any Service Manager operation on any Service Manager table.

Related topics

Input event processing
Output events
Output event processing

1.1.4.1.1. Supported external products

Service Manager Event Services interfaces with other internal and external products which include:

- Service Manager Mail (JavaMail), which sends Service Manager e-mail to other mail systems
- SCMail (for Unix) and SCMapi (for Windows), which supports bi-directional e-mail with other e-mail systems
- SCAutomate, which links Service Manager to external products such as IBM Tivoli® and Network Node Manager
- Customized SCAutomate applications
- Connect-It, which uses Event Services for input events and Service Manager for output events

Related topics

Communication with external systems
Event Services
Integrations
Synchronizing with an external system

1.1.4.1.2. Communication with external systems

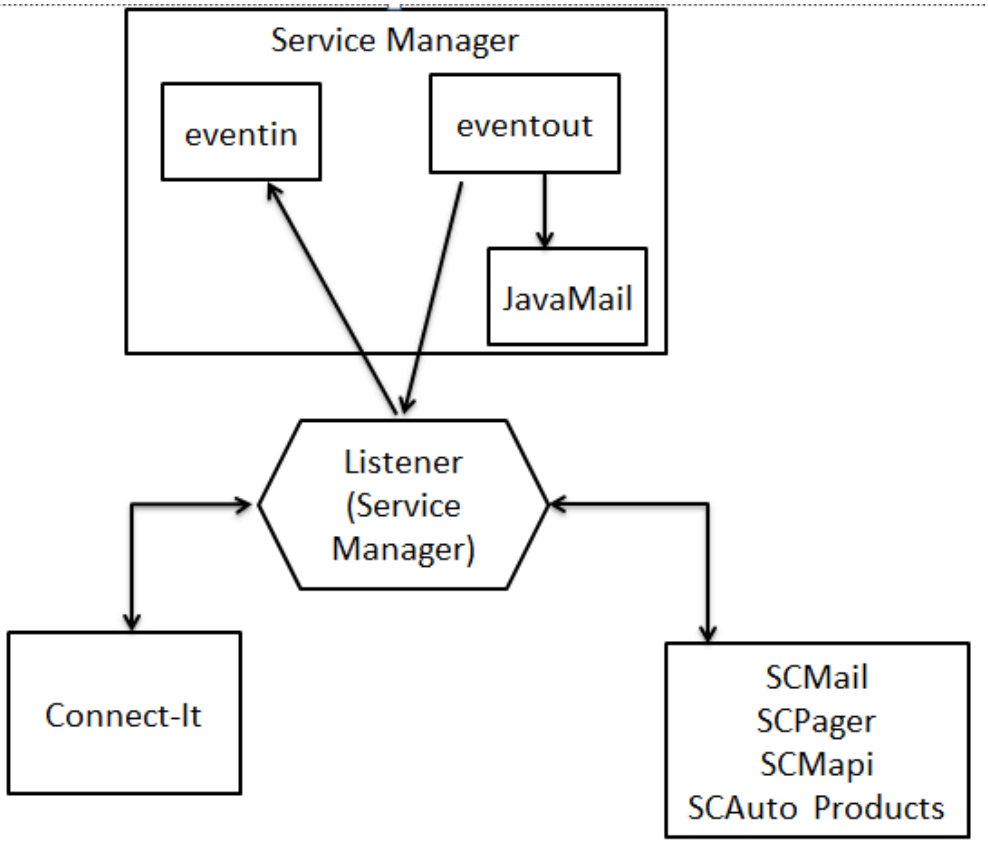
To accomplish the communication between products, you must establish a connection between Service Manager and the external system. Some type of TCP/IP connection is required, depending on the product and environment. The connection may also involve the sm listener.

Related topics

Event Services
Integrations

1.1.4.1.3. Event Services routing

Event Services routes Events entering and exiting Service Manager differently depending on the external system communicating with Service Manager. For some products, Event Services routes information in one direction only. For others, events flow in both directions.



Related topics

Event Services
Routing events with internal and external products

1.1.4.1.4. Routing events with external products

The following table describes how the following products route events using Event Services.

Product	Internal/External product	Routing direction	Description
Service Manager e-mail (JavaMail)	Internal	Outbound events only	Service Manager uses an eventout record to route information to JavaMail. JavaMail connects directly to the server, responds to, and processes only email eventout records.
Connect-It	External	Inbound events only	Connect-It establishes a client connection to the Service Manager server through a listener (sm) and information is routed bi-directionally through the connection. Connect-It requires inbound information to go through Event Services. However, outbound information can be read directly. Connect-It can read the outbound data from the eventout queue.

Product	Internal/External product	Routing direction	Description
SCMail **SCMap** **SCAutomate products**	External	Inbound and outbound events	These products establish a client connection to the Service Manager server through a listener (sm) and information is routed bi-directionally through the connection.

Related topics

Communication with external systems
Event Services routing
Output events

1.1.4.1.5. Event Services tables

Event Services uses the following tables.

Table name	Description
eventregister	Defines the events that exist in the system. Event registration records also specify the eventmaps used to process events and defines the RAD application used for processing.
eventin	Table used to move information into Service Manager from an external system. If a corresponding input eventregister record exists, external or internal applications can write records to the eventin table.
eventout	Table used to move information from Service Manager into an external system. A specific type of an eventout record can be written only if a corresponding output eventregister record exists.
eventmap	Defines how information is parsed. Eventmaps define individual fields and create condition statements for eventin and eventout records. Many eventmap records can exist for each eventregistration record.
eventfilter	Prevents duplicate events. Filters block incoming events based on defined criteria to prevent external systems from creating many eventin records for the same item in a short amount of time. Filters can block events by time frame, item, or location.

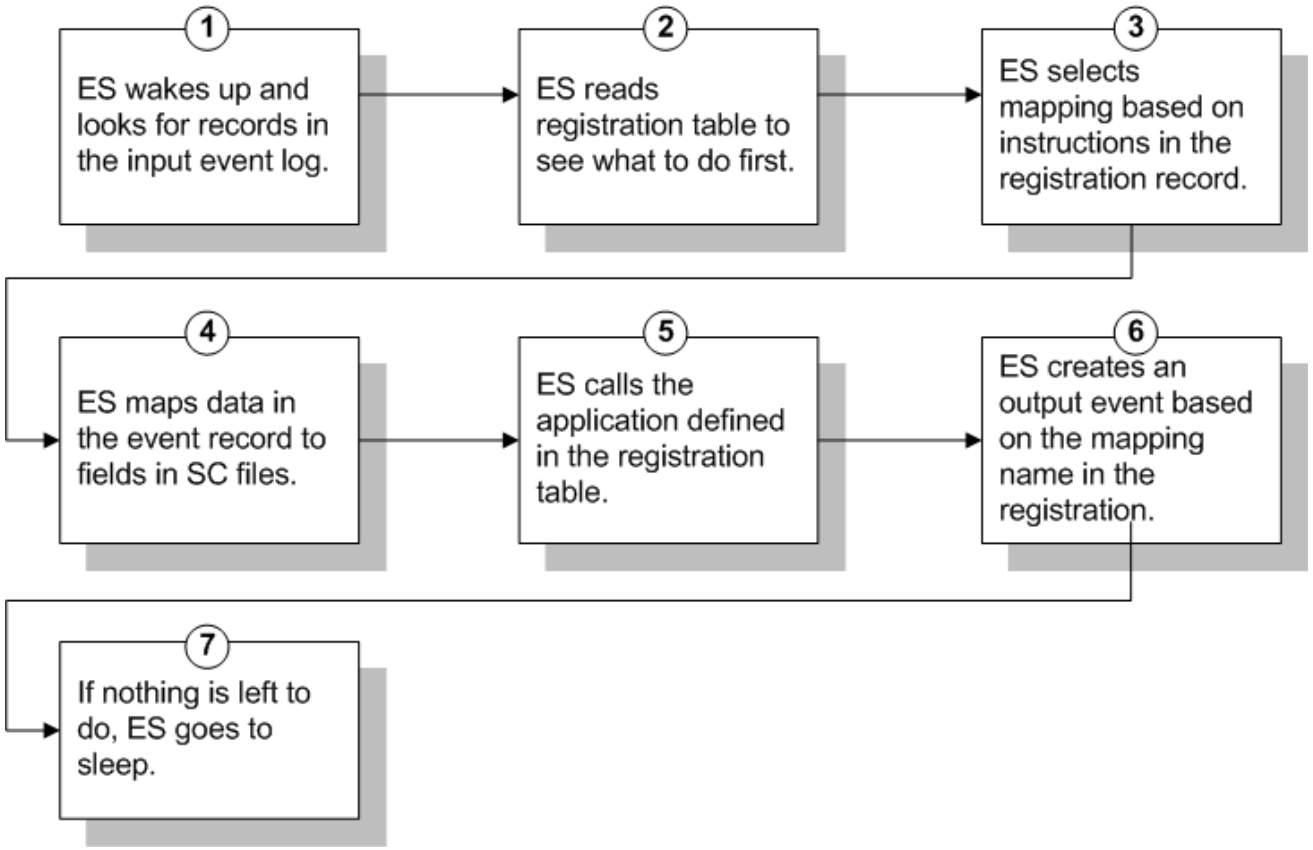
Related topics

Event Services
Event Services and Change Management
Generating email messages
Event Services and Service Manager e-mail
Event Services operations

Event Services routing
Event Services workflow

1.1.4.1.6. Event Services workflow

Service Manager Event Services has the following workflow:



Phase	Details related to the phase
1	Event Services (ES) uses a scheduler called event. You start and stop the event scheduler like any other Service Manager scheduler and process events in background or asynchronously.
2	The event registration file contains all of the information Event Services needs to determine what to do with each event.
3	Mapping records contain instructions to move data from the eventin record to fields in Service Manager files.
4	Based on instructions in the mapping records, a data structure is built.
5	A multi-purpose call routine is issued to the application named in the registration record, along with any necessary variables.
6	When the application has completed, an output event is created and added to the queue, if instructed by the registration.
7	If Event Services has nothing left to do, it sleeps for an interval, then reawakens to look for more work.

Related topics

Event scheduling
Event Services
Mapping events
Event Services tables

1.1.4.2. Event Services operations

Event Services has the following actions.

Action	Description
Review Agents	Opens the Event Scheduler and displays the details of scheduled system events.
Send Email	Initiates an e-mail event.
Write an Output Event	Initiates the event create script and prompts you to select the type of external event: Incident, Inventory or Generic (message).

Related topics

Agent status
E-mail events
Event Services

1.1.4.2.1. Registered events

Each event has a record in the eventregister table. Each event record includes a unique event code and a sequence number. A single event can start a series of applications. Each event record also contains initialization statements, mapping information and instructions for calling the Service Manager application. Registration is necessary for all input events that external applications process.

Example

Records in the eventregister table contain unique identification codes. The *pmo* code identifies opening an incident. When a pmo event occurs, Event Services calls the axces.apm application if the condition is true. The event parameters are passed by name and value. The Map Name identifies the event map to use when mapping the information into Service Manager tables.

The event registration record contains expression statements that create different queries, depending on the source of data. For example:

```
$ax.query.passed=nullsub("flag=true and network.name=\""+2 in $axces.fields+"\"", "false")
if (index("axmail", evuser in $axces)>0) then ($ax.query.passed=nullsub("flag=true and
logical.name=\""+1 in $axces.fields+"\"", "false"))
```

The query uses network.name to select open incidents for update. The SCAuto mail incident event uses logical.name.

The pmo event registration record instructs Event Services to:

- Select a record from the probsummary file (based on the query in $ax.query.passed).
- Map data from the eventin record ($axces) based on the incident open (evmap in $axces.register) map record.
- Open an incident.

In most standard Event Services input applications, the first two parameters passed are the event record and the name of the event map. An exception in standard Service Manager SCAuto applications is e-mail, which passes the mail record and the delimiter character.

Related topics

Event Services
Generating email messages
Input events
Mapping events
View registered events

1.1.4.2.1.1. View registered events

Applies to User Roles: System Administrator

To view registered events:

1. Click **Tailoring** > **Event Services** > **Registration**.
2. Click **Search**.
3. Select an event registration record to view its details.

Related topics

Event Services
Registered events
Add a registered event

1.1.4.2.1.2. Add a registered event

Applies to User Roles: System Administrator

To add a registered event:

1. Click **Tailoring** > **Event Services** > **Registration**.
2. Complete the top-portion of the Event Registration form.
3. Provide the appropriate information on each tab: Expressions, Basics, Application.
4. Click **Add**.

Note

You can delete any registered events added by administration users, but you can't delete any out-of-box events.

Related topics

Registered events
Mapping events
View registered events

1.1.4.2.2. Input events

The eventin table contains input event log records. It contains a record for every event detected, but not filtered, by SCAutomate external applications. Each record contains the event code, a unique system ID and a time stamp. During the input event, data passes to Service Manager in a character string using a delimiter character to separate fields.

Related topics

View Event Services input events

1.1.4.2.2.1. Input event processing

An external application inserts all records in the eventin table and external programs manipulate the eventin records. For example, SCAutomate supports an event named email. External sources can send electronic mail and pass it to Service Manager mail. The sources for electronic mail can be external e-mail systems, alert monitors, or other programs that can send messages. The external SCAutomate application packages the data in a standard format and stores it in the eventin file. Eventmap records specify the format of the data.

Processed records in the eventin table don't contain a First Expiration value. Normally, Event Services deletes event records after they are processed unless they are filtered or an exception occurs during processing. There is a condition set in the eventregister table that controls the delete flag.

If an error occurs due to Format Control processing, event processing terminates for that event and Event Services writes the specific error message to the eventin Messages log and to the Service Manager msglog file.

After you install and test SCAutomate, do one of the following:

- Set all delete flags in the eventregister records to true.
- Use the Service Manager purge/archive routines to schedule cleaning up the eventin table on a regular basis.

Related topics

Input events
View Event Services input events

1.1.4.2.2.2. View Event Services input events

Applies to User Roles: System Administrator

To view Event Services input events:

1. Click **Tailoring** > **Event Services** > **Input Events**.
2. Click **Search**.
3. Select an input event record.

Related topics

Event Services
Input events
Input event processing

1.1.4.2.3. Output events

The eventout table contains output event log records. It contains one record for each event processed by Event Services applications and instructions that the external software uses. This type of data passes to external applications in a character string with a delimiter character to separate fields.

Related topics

Output event processing
View Event Services output events

1.1.4.2.3.1. Output event processing

External programs manipulate the eventout records.

Expired events

Processed records in the eventout table don't contain an expiration date. Normally, Event Services deletes events from the eventout table after processing unless an error occurs. You can manipulate a flag in the external IPAS or SCAutomate software to cause record deletion after a read operation; however, because multiple SCAutomate processes can read the same record, it's not always feasible to delete the record after each read operation. We recommend that you use scheduled Service Manager purge/archive routines to clean up the eventout table on a regular basis.

Related topics

Output events
View Event Services output events

1.1.4.2.3.2. View Event Services output events

Applies to User Roles: System Administrator

To view Event Services output events:

1. Click **Tailoring** > **Event Services** > **Output Events**.
2. Click **Search**.
3. Select an output event record.

Related topics

Event Services
Output events
Output event processing

1.1.4.2.4. Generic events

Generic event options enable you to manage outgoing event records into Connect-It, including:

- Editing eventout information generation
- Exporting configuration records
- Exporting database dictionary structures

Related topics

E-mail events
Event Services
Input events
Mapping events
Out-of-box events
Output events
Registered events
Troubleshooting: Event Services

1.1.4.2.5. E-mail events

A standard e-mail event that Service Manager creates is the opening of a problem with a valid Contacts field. This event can notify individuals of a problem in their area of expertise. You also can create e-mail events using the User Services Send Mail function.

In addition to the standard creation of e-mail events in Service Manager, any RAD application can create an event. An example of this is implementing e-mail notification for problems that reach a specified status.

Related topics

Event Services

1.1.4.3. Mapping events

After creating an eventregister record for an input or output event, a mapping record in the eventmap table describes the process to manage the event. The event mapping record directs the event and its associated data to create results that are used within Service Manager or in an external application.

Mapping also allows complete flexibility of data manipulation during the mapping process. Because Event Services runs as a background task, no input/output routines are available for online validation with user feedback, but you can check field values and make substitutions based on processing statements.

Related topics

Change Management input event registration
Change Management output events
Event Services
View event maps

1.1.4.3.1. Event mapping process

The eventmap table contains event mapping information. There are two types of maps: input maps and output maps. Input maps have information about moving data from the External Information String (evfields) field for the eventin record to the target table. Output maps have information about moving data from the source file to the External Information String field for the eventout record.

The purpose of event mapping is to relate elements in a list to fields in a record. An external event, such as SCAutomate or SCAuto for NetView, passes data in the Service Manager eventin file to a field called fields. Each element is separated from the others with a delimiter, or separation character. In the following example, the ^ character separates the five fields.

```
john@microfocus^falcon^toby;al;joe^Meeting today^Tue 12 Aug
```

Internally, Event Services converts this string to a list ($axces.fields).

```
john@microfocus
falcon
toby;al;joe
Meeting today
Tue 12 Augcol
```

The event processor assumes that fields with a type of date/time are in the time zone of the Service Manager system (that is, the time zone defined in the System Wide Company Record). If the event background process has an operator record, that time zone for that operator is used. For synchronous processing, the session processing the event handles the date/time in the time zone where it is defined.

Mapping defines the link between the elements in the internal list (evlist) and fields in a Service Manager file. The first field, john@microfocus, is mapped to the user.to field for the mail file.

Standard event maps

Out-of-box event maps describe standard events. Changing the relative position of data in the information exchanged between Service Manager and the external applications may cause standard events to fail. It is better to create new maps for non-standard events rather than modifying existing maps.

Related topics

Change Management input events
Change Management output events
Event Services
Input event processing

Output event processing
Output events
Rules for building maps
View event maps
View registered events
Standard events

1.1.4.3.1.1. View event maps

Applies to User Roles: System Administrator

To view event maps:

1. Click **Tailoring** > **Event Services** > **Maps**.
2. Click **Search**.
3. Select an event map.

Related topics

Mapping events
Event mapping process

1.1.4.3.2. Using event maps

Each record in the eventmap table describes a single field for an input or output event.

For an input event, Event Services uses information in the eventmap record to map data from an external source that it can import into a Service Manager table.

For an output event, Event Services uses information in an eventmap record to map data in Service Manager tables into a sequence of delimited fields that it can export to external applications.

Example

When a Service Manager user sends e-mail, specific fields in the Service Manager mail file are populated. These include user.to, user.from, user.array, subject and text. The event map describes the information in these fields in a standard, defined sequence to ensure that the SCAutomate mail application can translate it to external programs. When SCAutomate receives mail from an external program and posts it to an eventin record, the Event Services event map describes how to populate the required fields in the Service Manager mail file.

For example, the user.from field in the Service Manager mail record is in the second position. Through mapping instructions, it becomes the second field in the delimited text string written to the eventin record. For output, Event Services puts the contents of the user.from field in the Service Manager mail record into the second position in the External Information String field of the eventout record and changes the Type field to output.

If you have custom mapping records for e-mail and they are deleted, Service Manager uses the out-of-box e-mail eventmap record.

Related topics

Event mapping process
Mapping events
Rules for building maps
Using expressions in event maps

1.1.4.3.2.1. Mapping an array of structures

The following example shows an event string that maps information into an existing Change Management task. The last portion of the string maps data to the parts and labor array of structures fields in the cm3t table. The vertical bar (|) array separator delimits each array. Each portion of the array with the | delimiter has another subdivision using the grave accent structure separator (`).

```
T^^update^^^
```

1.1.4.3.2.2. Mapping to multiple tables

Event Services also handles mapping to multiple tables. For example, SCAutomate for NetView and SCAutomate can send inventory information that's stored in more than one table. Configuration Management applications use two tables to describe each device: the entity table and the attribute table. The entity table is named device; the attribute file depends upon the device type, and is identified by the type field in the entity table. When Configuration Management information is gathered using discovery processes in external applications, Event Services passes the data to Service Manager using SCAutomate and updates both tables.

Example: Mapping to multiple tables

This example describes how to map to the device and attribute tables. Remember the roles of these Event Services tables:

- **The eventregister table:** Contains one record for every pre-defined event.
- **The eventmap table:** There are several out-of-box event maps for adding a new device named inventory add. Each inventory add map contains the instructions to map data for one field.

To view the inventory add map records:

1. Click **Tailoring** > **Event Services** > **Maps**.
2. Type `inventory add` in the blank **Map Name** field.
3. Click **Search**.

   Service Manager displays a list of out-of-box event maps named inventory add.
4. Select the inventory add event map record where the **Position** is 9 and the **Field Name** is type.
5. Click the **Expressions** tab. The following expression in the Post Map Instructions identifies the attribute table.

   ```
   $attribute.file=type in $axces.target
   ```

   This instruction sets the variable $attribute.file to the value in the type field of the device record.

If you view the first inventory add event map records, The **Sequence** is 1, and the **File Name** in the map record is device. Notice that each record identifies a different field to be mapped. Until all fields are mapped to the device file, the **Sequence** remains 1 and **File Name** remains device.

After the last field for the initial file is mapped, the record is added or updated and a new file is initialized based on the value of $attribute.file.

While $axces.target and$axces.field have special meaning within Event Services, $attribute.file is an arbitrary global variable name.

When all fields are mapped into the device file, the next map record has a Sequence of 2, the File Name is different and a Query is supplied.

- File Name now contains the value assigned to the $attribute.file variable.
- Query tells Event Services how to select the record to update from the file identified by $attribute.file. The query can be either a literal statement (as shown in the previous example) or a variable set in previous Post Map Instruction or Initialization fields.

The first mapping for the new file is logical.name, which is stored in Position 1 (as shown in the previous example) of the evfields array field, which is represented by the $axces.fields variable in the eventin record.

Subsequent map records move data from the eventin record to the new file. When updating an existing record, Event Services substitutes the value in the original record for a null value passed from the eventin record.

Build an event map

Applies to User Roles: System Administrator

To build an event map:

1. Click **Tailoring** > **Event Services** > **Build New Map** to open the Build Event Mapping form.
2. Type the map name and source table name, and click **Next**.
   Once you have selected a table, Service Managerdisplays a screen that allows you to build the event map for the table you selected. A list of field names and data types for the table you selected is displayed.
3. To delete a field before building the map, select the field from the list, and click **Remove Field**.

   Note

   If an array field is part of your mapping, delete the second instance of the field in the list presented when building a new map, leaving only the array field.
4. To view the details of the event map, click **Select Map**.
5. Click one of the following options to build a map:
   - **Build Input:** Builds the records that map information from the eventin file to the selected Service Manager table.
   - **Build Output:** Builds the records to map information from the selected file to a formatted string to be passed to SCAuto using the eventout table.

Related topics

Change Management input event registration
Change Management output events
Event Services
Mapping events
View event maps

1.1.4.3.2.3. Using expressions in event maps

You can check field values and make substitutions based on processing statements.

For example you can use the following expression to check field values and make substitutions based on processing.

```
if (logical.name in $axces.target="UNKNOWN")
then (logical.name in $axces.target=network.name in $axces.target)
```

or

```
if (logical.name in $axces.target=NULL
then (logical.name in $axces.target="?"+str(tod()))
```

In this example, the value the value in network.name replaces logical.name if logical.name is UNKNOWN. The second statement sets logical.name to a constant if it is NULL.

Other common uses for expressions are to set the value of a field to the current date and time and to calculate a value based on information in the record. Event Services applications handles data type and case conversions as long as the Field Type field is correctly identified and the data is written to the descriptor structure.

You can use a single Format Control record named login.event to establish initial global variables (such as lists of valid operators) when the event agent is started, just as you can for users when they log into Service Manager.

If you are writing data to a field whose name exists in more than one structure in a record, you must explicitly name the field. For example, if you add a field named assignment to the middle structure of your incident database dictionary record and you want to manipulate that field, you must identify it as middle,assignment. The field must exist in the target file before any instruction can manipulate it. Ensure the data type is correctly identified.

Related topics

Event mapping process
Mapping events
Rules for building maps
Using event maps

1.1.4.3.2.4. Mapping and Configuration Management

While Service Manager provides both an entity file (device) and attribute files (for example, server), it is unnecessary that both files exist to represent the characteristics of every device type. You can often fully describe a device using only the fields in the device file.

The map record for the type field (field #9 in standard events) defines how Service Manager selects and displays information about a device once the data is added. The type field in the device file refers directly to the associated attribute file of each device. If no attribute file associated with a device, the type field must contain device or be empty (NULL).

Similarly, the format.name field in the device record defines the name of the form that displays the device within Service Manager and, by extension, the name of the join file that temporarily stores information for review and update. The formatctrl record for the format name stored in the device record must contain device as the file name for all device types that don't have associated attribute files.

If an external agent detects an unknown device type, Service Manager processes the event, updating the device file with the information provided. If no attribute file exists for that device type, a Warning message is written to the Message list for the event but the device is still added or updated in the Service Manager data repository. If event mapping indicates processing in more than one table, but the number of fields passed to the event is less than the position of the first field in the second table, there is no attempt to open the second table.

Related topics

Considerations for Configuration Management maps
View event maps

1.1.4.3.2.5. Considerations for Configuration Management maps

While Service Manager provides both an entity file (device) and attribute files (for example, server), it's not necessary that both files exist to represent the characteristics of every device type. You can often fully describe a device using only the fields in the device file.

The map record for the type field (field #9 in standard events) defines how Service Manager selects and displays information about a device once the data is added. The type field in the device file refers directly to the associated attribute file of each device. If no attribute file associated with a device, the type field must contain device or be empty (NULL).

Similarly, the format.name field in the device record defines the name of the form that displays the device within Service Manager and, by extension, the name of the join file that temporarily stores information for review and update. The formatctrl record for the format name stored in the device record must contain device as the file name for all device types that don't have associated attribute files.

If an external agent detects an unknown device type, Service Manager processes the event, updating the device file with the information provided. If no attribute file exists for that device type, a Warning message is written to the Message list for the event but the device is still added or updated in the Service Manager data repository. If event mapping indicates processing in more than one table, but the number of fields passed to the event is less than the position of the first field in the second table, there is no attempt to open the second table.

Related topics

Change Management input event registration
Change Management output events

1.1.4.3.2.6. Rules for building maps

For best results when building new maps that use array fields, follow these guidelines.

- Select the first instance of any array fields (such as user.array in the mail file) so the proper type is built for the field.
- Only scalar and array fields can be directly mapped; all other types must be manipulated using expressions.

If possible, build maps first and then design external applications to use the maps.

Related topics

Event mapping process
Event Services
View event maps

1.1.4.3.3. Filtering events

Event filtering information is stored in the eventfilter file. This file instructs SCAuto when to block incoming events. If an event is not blocked, filters also can prevent opening incident records based on recurrence intervals and counts, and on incident intervals.

The number of filters available for external blocking is unlimited. The external process (SCAuto) reads the eventfilter file to select records with the same Event Code and User Name (or User Name=NULL) and with Block Events?=true until it finds one that satisfies the criteria for the event being processed. If none is found, the event is inserted in the eventin file.

Once records are added to the eventin file, Event Services assumes the filtering task using Internal Filters. Event Services first selects the filter with the same Event Code as that of the event being processed and with a Network Name of SCAuto. This filter must contain all internal blocking conditions. If an eventin record satisfies one of the Block Conditions, it is updated to reflect a Status of blocked. The event action (for example, incident open or inventory add) doesn't take place.

With incident open event types (pmo), the Additional incident Filters take effect if no blocking condition exists. This filtering mechanism is available only when opening new incidents.

Related topics

Event Services
Selecting filters

1.1.4.3.3.1. View event filters

Applies to User Roles: System Administrator

To view event filters:

1. Click **Tailoring > Event Services > Filters**.
2. Click **Search**.
3. Use the External Filters, Internal Filters, or Additional Incident Filters tab to view the appropriate filter types.

Related topics

Filtering events

1.1.4.3.3.2. Event blocking

The external SCAuto application use the External Filters tab of the filter record to prevent the insertion of eventin records in the Service Manager database. The contents of the User Name field must either match that of the external process or be empty (NULL).

The `Block Events?` condition must be set to true to prevent records from being added to the eventin file. The Start Blocking at and End Blocking at values are optional. However, they allow for a block to be placed over a specified time frame allowing a more customized administration.

Event blocking with conditions

You can also prevent the insertion of events for specific network devices, domain names and error types by using the Index, Value, and Condition fields. Use these fields independently or in conjunction with the Start Blocking at and End Blocking at fields to populate other fields on the form.

- Index refers to the position of the data in the event message.
- Value refers to the actual data contained at that position.

For example, a pmo event contains the following message:

```
hp^hp^^6 58916865^Node Down^^^^SNMP Trap(IPAS)^net.hware^^^^^^^^^^
```

The caret (^) character separates fields in the message. The first field, which references the logical name of the device, contains hp. To block the insertion of all incident open events reported for the device hp, type pmo in the Event Type field, 2 in the first Index field and hp in the first Value field.



Note

Only Index values of 2 or 3 are supported for incident open actions.

Example: Event blocking with conditions

This is an example of event blocking from two servers, microfocus and falcon.

To block incident open events from both microfocus and another server named dolphin, select or in the Condition field, 2 in the second Index field and dolphin in the second Value field on the External Filters tab. If you specify a condition (AND or OR), then you must complete both Index and both Value fields.

To prevent insertion of records in the eventin file, the Block Events? field must be true (checked).

To specify a time range for blocking, set the Start Blocking at and End Blocking at fields. In this example, use 08:00 for start blocking and 17:00 for end blocking.

With these settings, all inventory add (icma) events are blocked between 08:00 and 17:00 if they come from either the microfocus or dolphin server. This action avoids unnecessary adds and updates if installation activity is scheduled to occur on the network during this time.

Related topics

Event scheduling
Event Services
View event schedules
View Event Services agent status
View registered events

1.1.4.3.3.3. Selecting filters

Filters are selected using the following search criteria and in the order listed:

- The Event Type is the same as that of the event being processed . The Network Name is the same as the network name specified in the eventin record. The Cause Code is the same as the cause code specified in the eventin record.
- The Event Type is the same as that of the event being processed. and the Network Name is the same as the network name specified in the eventin record.
- The Event Type is the same as that of the event being processed and the Network Name is AXCES and the Cause Code is the same as the cause code specified in the eventin record.
- The Event Type is the same as that of the event being processed and the Network Name is AXCES.

Example: Selecting filters

This example demonstrates how you can set up filters on the Additional Incidents Filter tab to block events.

The following is the event example:

```
hp^hp^^6 58916865^Node Down^^^^SNMP
Trap(IPAS)^net.hware^^^^^^^^^^
```

The queries are:

```
evtype="pmo" and evnetnm="hp" and evcode="6 58916865"
evtype="pmo" and evnetnm="hp"
evtype="pmo" and evnetnm="AXCES" and evcode="6 58916865"
evtype="pmo" and evnetnm="AXCES"
```

You can permanently block problem open by entering a Network Name or Cause Code. This has the same effect as a Block Condition except that the status in the eventin record is filtered rather than blocked.

For example, using hp for Network Name and SNMP 2.0 for Cause Code. This prevents any events from server hp with cause code of SNMP 2.0 from opening a problem.

You can further refine the filter by entering an Event Interval and Recurrence Count. For example, if you set Event Interval to ten minutes and Recurrence Count to 3, the filter prevents any events from server hp with cause code of SNMP 2.0 from opening a problem unless three events are received within a ten minute interval.

Related topics

Event Services
Filtering events

1.1.4.4. Event Services and Change Management

The Change Management application of Service Manager is fully supported by Event Services. This allows users outside of the Service Manager system to perform all standard functionality of Change Management from an external system, for example, SAP or PeopleSoft. The Event Services implementation is bidirectional, allowing external systems to synchronize with the Service Manager system.

With Event Services, Service Manager uses input and output events to transfer data in and out of Change Management.

You should have an administrator level of knowledge of Change Management and Event Services to use Event Services with Change Management.

Related topics

Change Management eventin fields
Change Management input event registration
Change Management input events
Change Management output events
Event Services
Event Services tables
View event schedules
View Event Services agent status
View registered events

1.1.4.4.1. Change Management input events

A correctly formatted eventin record must be created within Service Manager to use an external system to produce an action within the Service Manager Change Management application. You can format the eventin record with an SCAutomate product.

Related topics

Change Management eventin fields

1.1.4.4.1.1. Change Management eventin fields

The eventin record fields specific to the Change Management implementation are:

Field	Description
Event Code (evtype)	Name of the corresponding Event Registration record to use for this event. This must always be cm3rin for changes and cm3tin for tasks.
User Name (evuser)	User name in this field is interpreted as the operator for this event. The Change Management environment used depends on which user is entered in this field.
External Information String (evfields)	Delimited data fields that correspond to a specific event mapping.

Related topics

Event Services
Event Services and Change Management
Synchronizing with an external system
View event maps

1.1.4.4.1.2. Change Management input event registration

Change Management uses events cm3rin and cm3tin for input events. One of these two event codes must appear in the eventin record, depending on whether the event is related to a change or a task.

Change Management events

1.1.4.4.2. External information string

The external information string, or EIS, is the evfields field of the eventin record. This field carries the specific data of the change or task into the Service Manager system. These fields are placed in a single string with a user-specified separation character (the default is the ^ character). The first four fields contain specific functions that determine which change/task is being processed and what action the system should take. These fields are passed in a specific order.

Sequence	Field Description
1	Change/Task number of the object to be acted upon. This field is blank when opening a change or task.
2	The foreign ID. This field is the identifier of the change or task used by the external system. This field is used if a different number is used outside of Service Manager.
3	Action Token indicates which logical action to take, either: open, update, close, reopen, approve, unapprove, disapprove.
4	The Change Group or operator performing an approval action. It is only used for approve, unapprove, or disapprove.

The data fields in the EIS contain field-level data that Event Services uses to populate the change or task record being processed. If the action performed is not an open, these fields write over any existing data in the change or task. If a field in the EIS is blank, the existing data in the change or task is used. The exact field that each piece of data corresponds to can be determined by examining the proper input event map for changes (cm3r) or tasks (cm3t).

Related topics

Event Services
External information string actions
External information string fields

1.1.4.4.2.1. External information string fields

The first two external information string (EIS) fields determine the unique identifier of the change or task both in Service Manager and in an external system (if applicable).

The first field contains the unique number that corresponds to the number field in the cm3r or cm3t database dictionary. This field is blank if the action is open.

The second field of the EIS corresponds to the foreign.id field of the change or task. This field specifies the unique identifier of the change or task in the external system that's sending the request. If the Service Manager number is not specified, the system attempts to find the correct record by comparing the foreign.id to this field.

Related topics

Event Services
External information string
External information string actions

1.1.4.4.2.2. External information string actions

Event Services uses the third field of the external information string (EIS) to determine what type of action to perform on the specific change or task specified by one of the first two fields. The following table describes the supported actions.

Action	Description
approve	Approve a change or task.
disapprove	Disapprove a change or task.
unapprove	Unapprove a change or task.
open	Create a new change or task.
update	Update an existing change or task.
close	Close current phase and advance to the next phase, if applicable.
reopen	Reopen a change or task in the current phase.

When the action is an approval action (either an approve, disapprove, or unapprove), the Change Management Group or Operator Name that's performing the approval action must be specified in the fourth field of the EIS. The group or operator specified must match one of the approval groups specified in the change or task record for the approval action to complete properly.

Related topics

Event Services
External information string
External information string fields

1.1.4.4.3. Synchronizing with an external system

When Service Manager is used with a separate external system, the changes and tasks must be synchronized between the two systems. Event Services supplies two methods of sending output to the external system for this task.

First, a simple acknowledgment can be sent to the external system. This acknowledgment contains enough data to map the Service Manager change or task number to the unique ID used in the external system, along with enough messages to determine if the input event was successful.

Alternatively, a complete output event may be sent to an external system in order to synchronize every piece of data between the two systems.

Related topics

Communication with external systems
Event Services
Integrations
Supported products

1.1.4.4.3.1. Acknowledgement events

The cm3rinac and cm3tinac event registrations are used as acknowledgement events to synchronize the unique numbers of each system when Service Manager is used with an external system.

Event Code	Input/Output	Event Map	Application	Description
cm3rinac	Output	cm3ack	axces.write	Used for changes.
cm3tinac	Output	cm3ack	axces.write	Used for tasks.

Both event types use the cm3ack event map definition. The following table shows the mapping passes the fields in the external information string (EIS) of the eventout record.

Sequence	Field description
1	The change or task number of the object being acknowledged.
2	The foreign ID. This is the identifier of the change or task used by the external system. This field is used if a different number is used outside of Service Manager.
3	Action Token indicating which action was performed on this object (open or update, for example).
4	The status of the eventin record created by the original event. This field may be used to determine if there were any errors encountered when processing the original event.
5	An array of up to five messages sent during the original event (example: Change 15 updated, Location XXX is invalid). These messages can be used to determine if a Format Control or validation error occurred during the original event.

The acknowledgment events can be turned on or off in the cm3rin or cm3tin Event Registration records by modifying the value associated with the boolean1 parameter on the application tab. When this parameter value is set to true an acknowledgment event is sent out each time an input event is processed, while a setting of false keeps the acknowledgment event from being sent.

Related topics

Change Management output events
Event Services
Output events
View Event Services output events
View registered events

1.1.4.4.4. Change Management output events

The standard output events for Change Management are triggered by the cm3messages file. When the change scheduler processes a cm3message, the value is checked in the Event Services Reg (axces.out) field in the corresponding cm3message record. If the value matches an output event (most likely cm3rout or cm3tout), that event is processed and an eventout record is written. This gives an administrator great flexibility when deciding what types of events (opens or alerts, for example) cause the output event to be written.

The output maps used for these events are cm3r and cm3t. These maps correspond to their related input maps with the exception of the third and fourth fields. The third field contains the name of the event that caused the event to process (for example, cm3r open or cm3t update). The fourth field is used as a place-holder to keep the data fields of the input and the output event synchronized and always contains the words not used.

Related topics

Change Management eventin fields

1.1.4.4.5. Example: Open a change

This example uses the following parameters:

- category for adv-nam-desk-101 - Hardware
- external foreign ID - CM01
- requested by - System.Administrator
- assigned to - Servicedesk.Agent

The change contains a simple description while letting all other fields use default values.

The event register has the following specific fields:

- evtype is cm3rin
- evuser is System.Administrator

The external information string (EIS) is:

```
^CM01^open^^^Hardware^^^System.Administrator^^^^Servicedesk.Agent^^^^^^
^^^^^^^^^^^^^^^^
^^^^^^^^Move HP Pavilion M9160Elite to Mike's
office.^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^adv-nam-desk-101^^^...
```

The field positions correspond to the cm3r input event map as follows:

Position	Field name	Value
2	foreign.id	CM01
3	actiondummy	Open
6	category	Hardware
9	requested.by	System.Administrator
13	assigned.to	Servicedesk.Agent
42	description	Move HP Pavilion M9160Elite to Mike's office.
76	logical.name	adv-nam-desk-101

Related topics

Change Management eventin fields
Change Management input event registration
Change Management input events
Change Management output events
Considerations for Configuration Management maps
Event Services
Event Services and Change Management
Change Management events

1.1.4.5. Event agents

Automatic monitors within Service Manager, known as agents, can be set to collect data and create events appropriately within the system. You can use the Event Scheduler to set up these agents, or you can activate them automatically or manually (by user input).

Related topics

Event scheduling
Event Services
View event schedules
View Event Services agent status
View registered events

1.1.4.5.1. Event scheduling

The schedule table contains a record for each SCAuto agent. It contains instructions indicating how often the agent reads a queue and which application to execute if the read returns records. You use the Event Schedule form to manage event agent operations.

When the event agent starts, the event schedule record must have a class of event (or the name you specify for the event scheduler) and must have an expiration earlier than the current time. Set the expiration to the current date and time before starting the scheduler.

Since the event scheduler is a serial process, you may want to have more than one scheduler read events in the event queue, particularly when inventory activity is high, preventing incident management activity.

Use the Query field to further define what type of event to select from the eventin file. The user-specified query entered in the schedule record is appended automatically to the default event scheduler query, evtime<=tod(), to form a more specific query. If the Query field is left blank, only the default query is applied.



Note

The system always places the time portion of the query in front of the user-specified query.

If you define a query for use against the eventin file, ensure it is fully-keyed for maximum performance.

The agent processor attempts to restart any applications that ended while running (that have a status of application running). If you change for one of your agents, ensure there are no other agents with the same schedule class and a status of application running.

Related topics

Agent status
Event agents
Filtering events
View event schedules

1.1.4.5.2. View event schedules

Applies to User Roles: System Administrator

To view event schedules:

1. Click **Tailoring> Event Services > Review Agents**.
2. Select an agent from the list to open the Event Scheduler for the agent.

Related topics

Event Services
Event scheduling

1.1.4.5.3. Agent status

From Event Services you can start and stop any SCAuto or event agent without respect to your client status as long as the Service Manager problem agent is active. Using this feature, agents are scheduled to start, and the problem agent is their activation agent. The specific agents controlled from this option include:

- Event
- Axces
- SCAuto Server
- JavaMail

Related topics

Event scheduling
Event Services
View event schedules
View Event Services agent status

1.1.4.5.4. View Event Services agent status

Applies to User Roles: System Administrator

To view Event Services agent status:

Click **Miscellaneous** > **System Status**.

The **event** process shows a Last Expiration time and an Idle time. The Last Expiration time is the initialization time for the agent; the Idle time is the amount of time elapsed since the agent last woke up to check for work.

If an agent is inactive, there is no Last Expiration or Idle time, and the Start button is available.

Related topics

Agent status
Event Services

1.1.4.5.5. View system startup information

Applies to User Roles: System Administrator

To view system startup information:

1. Click **System Administration > Ongoing Maintenance > System > Startup Information**.
2. Type `startup` in the Type field.
3. Click **Search**.

Related topics

System information record

1.1.4.6. Email solutions based on Event Services

The following email solutions are based on Event Services:

- Smart Email outbound email (also known as JavaMail)
- SCAuto Mail

For more information on these email solutions, see Service Manager email solutions.

1.1.4.7. Format Control and eventout records

When incidents are opened, updated or closed by Event Services, a record is written to the eventout file. This record contains information from the incident (described in the output eventmap record for the event) that's passed to an external process via the SCAuto/IPAS external interface. You can elect to write to the eventout file when Service Desk operators open and close records so that the information is passed to the external interface.

The axces.write application creates a character string of fields from a structure and writes them to eventout. An Event Registration record identifies the event type and the name of the Event Map records used to define which fields will be selected from the record. The application should be called as a Format Control Subroutine passing two parameters:

- The record from which data will be mapped.
- The Event Type, as defined in the Event Register.

To write to eventout on incident close, the Format Control record is attached to the `problem.equipment.close` form using the following values:

Field	Value
Application	axces.write
axces.write	true

Name	Value
record	$file
name	pmc

Note: This procedure is not specific to Incident Management. You can write eventout records for other applications, such as Configuration Management or Change Management.

Related topics

Format Control

1.1.4.7.1. Generating email messages

SCAutomate supports a generic e-mail function. Use the Format Control RAD function, message.fc, to write e-mail events to the eventout file.

Related topics

Event Services
Service Manager Programming

1.1.4.8. Out-of-box events

Service Manager delivers out-of-box events with the Event Services application. Service Manager Event Services uses these events to open, update, and close incidents or requests. They are also used to update configuration items or change requests. You can use event registration to view the details of each registered event, which are stored in the eventregister table.

Related topics

Change Management events
Configuration Management events
Incident Management events
Service Desk events
Service Level Management events
Standard events

1.1.4.8.1. Change Management events

Event	Event type	Description
cm3rin	input	Use this event for all incoming change events.
cm3rinac	output	This is sent if the write eventout is set to true. This returns failed events so the calling application receives notification when an error occurs.
cm3rout	output	This is created when a cm3 message is created and you enter cm3rout in axces.out.
cm3tin	input	Use this for all incoming change tasks.
cm3tinac	output	This is sent if the write eventout is set to true. This returns failed events so the calling application is notified when an error occurs.
cm3tout	output	This is created when a cm3 message fires and you enter cm3tout in axces.out.

You can use event registration to view the details of each registered event, which are stored in the eventregister table.

Related topics

View registered events

1.1.4.8.2. Configuration Management events

Event	Event type	Description
icma	input	This event adds or updates inventory items to the device file if filter criteria are satisfied.
ICMapplication	input	Service Manager inventory regulation event when a device of this type is added to the system.
ICMcomputer	input	Service Manager inventory regulation event when a device of this type is added to the system.
icmd	input	This event marks an inventory item for deletion if filter criteria are satisfied by placing inactive in the status field.
ICMdevice	input	Use this event type when you add data records to the device file. These events use the icm device eventmaps.
ICMdevicenode (1)	input	Use this event type to add or update information about a network node (a device that appears as a discrete item in a network) to the Inventory Configuration Management application. These events use the icm networkcomponents mappings.

Event	Event type	Description
ICMdevicenode (2)	input	If you use the ICMdevicenode event to send information to Service Manager, after the initial database operation completes, this secondary event registration causes a logging record to write to the eventout table.
ICMdisplaydevice	input	Service Manager inventory regulation event when a device of this type is added to the system.
ICMexample	input	Service Manager inventory regulation event when a device of this type is added to the system.
ICMfurnishings	input	Service Manager inventory regulation event when a device of this type is added to the system.
ICMhandhelds	input	Service Manager inventory regulation event when a device of this type is added to the system.
ICMmainframe	input	Service Manager inventory regulation event when a device of this type is added to the system.
ICMnetworkcomp	input	Service Manager inventory regulation event when a device of this type is added to the system.

Event	Event type	Description
ICMofficeelec	input	Service Manager inventory regulation event when a device of this type is added to the system.
ICMserver	input	Use this event type to add or update information about a server to the Inventory Configuration Management application. These events use the icm computer mappings (since servers are a subtype of computers, you can reuse the mappings.
ICMsoftwarelicense	input	Service Manager inventory regulation event when a device of this type is added to the system.
ICMstorage	input	Service Manager inventory regulation event when a device of this type is added to the system.
icmswa	input	This event adds or updates inventory items (that ServerView or StationView discovers) to the device file if filter criteria are satisfied.
icmswd	input	This event marks an inventory item (that ServerView or StationView discovers) for deletion in the pcfiles file if filter criteria are satisfied.

Event	Event type	Description
ICMtelecom	input	Service Manager inventory regulation event when a device of this type is added to the system.
icmu	input	This event updates inventory items if filter criteria are satisfied.
IND	input	An event that adds or updates inventory items to the device file.
prgma	input	This adds or updates a software inventory item to the pcfiles file that an external agent (other than ServerView or StationView) discovers if filter criteria are satisfied.
prgmd	input	This deletes a software inventory item from the pcfiles file that an external agent (other than ServerView or StationView) discovers if filter criteria are satisfied. The default updates the estatus field as deleted rather than removing the record from the database.

You can use event registration to view the details of each registered event, which are stored in the eventregister table.

Related topics

View registered events

1.1.4.8.3. Incident Management events

Event	Event type	Description
epmosmu	input	This event opens an incident from a call.
epmosmu	output	This event writes the record after an incident opens from a call.
pmc	input	This event closes an incident if filter criteria are met. It uses the same path as manually closing the operation.
pmc	output	This event writes after an incident is closed.
pmo	input	This event opens an incident if filter criteria are met. It uses the same path as manually opening the incident.
pmo	output	This event writes after an incident is opened.
pmu	input	This event updates an incident if filter criteria are met. It uses the same path as manually updating the incident.
pmu	output	This event writes after an incident is updated.

You can use event registration to view the details of each registered event, which are stored in the eventregister table.

Related topics

View registered events

1.1.4.8.4. Service Desk events

Event	Event type	Description
esmin	input	This event opens a call in Service Desk.
esmin	output	This event writes a record once a call opens in Service Desk.
smin	input	Service Desk incoming service request or help issue.
smout	output	This event writes once an incoming request or help issue enters the system.

You can use event registration to view the details of each registered event, which are stored in the eventregister table.

Related topics

View registered events

1.1.4.8.5. Service Level Management events

Event	Event type	Description
outageend	input	This event performs updates to outage records, which are part of the service level agreement (SLA) application.
outagestart	input	This event performs updates to outage records, which are part of the SLA application.
slaresponse	output	This is a request from an external application to enter a response time metric against a device with an SLA.

You can use event registration to view the details of each registered event, which are stored in the eventregister table.

Related topics

View registered events

1.1.4.8.6. Standard events

Service Manager event registration currently supports events enabling integration with ERP, SAP, and other external system interfaces. The events listed in the following table provide a brief description of the event.

Standard event	Event type	Description
approval	input	This event processes approvals for Request Management and Change Management.
approval	output	This event sends approvals for Request Management and Change Management.
CTSCPY (1)	output	This event to SAP uses eventmap cm3tctsc. It is a generated message to SAP to copy a CTS Transport from one system to another.
CTSCPY(2)	input	This event from SAP uses eventmap cm3tctsc. It processes the acknowledgment of SAP CTS Copy messages.
CTSIMP (1)	output	This event to SAP uses eventmap cm3tctsi. It builds a message to request a specific SAP Instance perform an Import of a given Transport.
CTSIMP (2)	input	This event from SAP uses eventmap cm3tctsi. It processes the Input acknowledge message from SAP regarding Import of Transport.
CTSIMP2	output	This event handles scheduling of Output Import events to SAP using eventmap cm3tctsi.

Standard event	Event type	Description
CTSRQCLS (1)	input	This event to SAP uses eventmap cm3rcts. It sends a message to a SAP instance instructing it to release a transport.
CTSRQLS (2)	output	This event from SAP uses eventmap cm3rcts. It is a message received message from SAP acknowledging a transport release.
CTSRQOPN (1)	output	This system event sent to SAP uses the cm3rcts eventmap. It sends a message to a SAP instance instructing it to open a SAP Transport Request with specific Service Manager-supplied data.
CTSRQOPN (2)	input	This event from SAP uses eventmap cm3rcts. It is a received message from SAP acknowledging Transport Request creation. It closes the first phase of the Change and updates fields with data returned from SAP.

Standard event	Event type	Description
CTSRQOPN (3)	input	This input event from SAP uses eventmap cm3rctso. It is a received message from SAP sent when a Transport Request opens on the SAP side without first opening within Service Manager. It causes a Change to open within Service Manager with data received from SAP.
CTSRQUPD (1)	output	This event to SAP uses eventmap cm3rcts. It sends a message to a SAP Instance to update specific Transport Request data elements.
CTSRQUPD (2)	input	This is an Input event from SAP. It uses eventmap cm3rcts. It is a received message from SAP sent when a Transport Request has been updated on the SAP side. It is either an acknowledgment of a Service Manager originated request or a notification of a SAP originated action.
CTSTKCLS (1)	output	This event to SAP uses eventmap cm3tcts. It sends a message to SAP to close a Transport Task within SAP.

Standard event	Event type	Description
CTSTKCLS (2)	input	This event from SAP uses eventmap cm3tcts. It is a received message from SAP that a Transport Task closed on the SAP side. It is either an acknowledgment of a Service Manager-originated request or a notification on a SAP-originated action.
CTSTKOPN (1)	output	This is event to SAP uses eventmap cm3tcts. It sends a message to SAP indicating that a Transport Task opened within SAP.
CTSTKOPN (2)	input	This event from SAP uses eventmap cm3tcts. It is an acknowledgment message from SAP indicating that a Transport Task closed on the SAP side.
CTSTKOPN (3)	input	This event from SAP uses eventmap cm3tctso. This message indicates that a Transport Task opened on the SAP side within the Change Management application using data supplied from the SAP system.
CTSTKUPD (1)	output	This event to SAP uses eventmap cm3tcts. It is sent from Service Manager to SAP to update a Transport Task on the SAP side to match changes on the Service Manager side.

Standard event	Event type	Description
CTSTKUPD (2)	input	This event from SAP uses eventmap cm3tcts. It is a message received from SAP when a Transport Task update on the SAP side. It can either be an acknowledgment of a Service Manager-originated update or notification of a SAP-originated update.
dbadd	input	This adds an item to a specified Service Manager file when you satisfy the filter criteria. It updates the file if the item already exists.
dbdel	input	This deletes an item from a specified Service Manager file if the filter criteria are satisfied.
dbupd	input	This updates an item to a specified Service Manager file when you satisfy the filter criteria.
email	output	This is the standard interface to convert Service Manager mail to standard e-mail format.
email	input	This is the standard interface to receive external e-mail and convert to Service Manager mail.

Standard event	Event type	Description
ERPHR (1)	input	This event establishes contact with the ERP system using eventmap contactserp.
ERPHR (2)	output	This event uses eventmap contactserp.
ERPSTATES (1)	input	This event determines the state of the ERP system using eventmap stateerp.
ERPSTATES (2)	input	This event uses eventmap stateerp.
gie	input	Both Asset Manager and Service Manager use the Generic Input Event (GIE).
HotNews	output	HotNews defines an eventout type of HotNews.
opera	input	This event adds or updates a new user to Service Manager if filter criteria are satisfied.
operd	input	This event deletes a user from Service Manager if filter criteria are satisfied.
operu	input	This event updates items specified in a Service Manager file if filter criteria are satisfied.
page	output	This event registration allows Service Manager to create eventout records with the evtype=page.

Standard event	Event type	Description
pageclose	input	This event uses a condition statement (evfiends in $axces)#"pm".
pageresp	input	This event updates an incident with an acknowledgment or message received as response to a page. It uses a condition statement (evfiends in $axces)#"pm".
pcsoftware	input	This event allows desktop inventory products to update Service Manager.
PSSDELETE	input	This event deletes selected records from a Service Manager file.
SALESQUOTE	input	This event moves an eventin record to the eventout file and changes the evtype.
SAPGRT	output	This goods receipt Output event calls no application. It submits receipt notification to SAP system for processing.
SAPGRT	input	This goods receipt event calls submits receipt notification to SAP system for processing.
SAPGTE	input	This event updates existing line items.

Standard event	Event type	Description
SAPHR (1)	input	Event processing edits to contact file originating in Service Manager, routed through SAP, and returned to Service Manager.
SAPHR (2)	output	This event routes contact file changes to SAP.
SAPHRMD	input	This event processes SAP-originating contacts file changes.
SAPORD	input	This sales order event from SAP breaks events into appropriate constituent parts.
SAPORD	output	This sales order event to SAP calls no application. It routes order information to SAP for processing.
sapordl (1)	output	This event routes order information to SAP for processing.
sapordl (2)	input	This event routes order information to SAP for processing.
SAPORDQ	input	This is the header component of the SAPORD event.
SAPQTE	input	This sales quote event from SAP breaks events into constituent parts.

Standard event	Event type	Description
SAPQTE	output	This sales quote event from Service Manager calls no routine.
sapqtel (1)	output	This event is the Output quote line item component of the SAPQTE event. It uses this registration to identify which eventmap to use for message formatting.
sapqtel (2)	input	This event is the detail portion of the SAPQTE event.
SAPQTEQ	input	This is the header component of the SAPQTE sales quote.
saprecl (1)	output	This event is the output goods receipt line item component of the SAPQTE event. It uses this registration to identify which eventmap to use for message formatting.
SAPREQ	input	This purchase requisition event is from SAP.
SAPREQ	output	This purchase requisition event from Service Manager calls no application.
sapreql (1)	output	This event is the output request line item component of SAPREQ. It uses this registration to identify which eventmap to use for message formatting.

Standard event	Event type	Description
sapreql (2)	input	This event is the detail portion of the SAPREQ event.
SAPREQO	input	This event is a SAPREQ component from SAP.
ScAcBrand	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager vendor file to the corresponding Asset Manager file.
ScAcCompany	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager company file to the corresponding Asset Manager file.
ScAcContacts	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager contacts file to the corresponding Asset Manager file.
ScAcDept	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager department file to the corresponding Asset Manager file.

Standard event	Event type	Description
ScAcDevice	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager device file to the corresponding Asset Manager file.
ScAcLocation	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager location file to the corresponding Asset Manager file.
ScAcModel	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager model file to the corresponding Asset Manager file.
ScAcModelBundle	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager model file to the corresponding Asset Manager file.
ScAcModelVendor	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager modelvendor file to the corresponding Asset Manager file.

Standard event	Event type	Description
ScAcVendor	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager vendor file to the corresponding Asset Manager file.
ScAcVendorBACK	input	This event allows Service Manager and Asset Manager to integrate data from the Service Manager vendor file to the corresponding Asset Manager file.
submit	output	This event submits a job for processing.
sysbull	input	This event adds a new System Bulletin to Service Manager if filter criteria are satisfied.

You can use event registration to view the details of each registered event, which are stored in the eventregister table.

Related topics

View registered events

1.1.5. Dynamic Data Exchange (DDE)

Create DDE client support in a Service Manager RAD (Rapid Application Development) application.

DDE support in Service Manager works two ways: create a DDE script to call against Service Manager from a Windows application or create a script using the DDE script panel to make a DDE call within Service Manager. The difference is which application originates the call. For example, a Service Manager client in a Windows environment can push information to Microsoft® Excel or Excel can pull from Service Manager.



Note

Prior to version 9.30, Service Manager used client-side DDE to export data to Excel. As of version 9.30, the Export to Excel functionality has been redesigned so that the Service Manager server process exports the data as a CSV (comma separated file) on the server side and transports the file to the client side. The Service Manager client saves the file on the client system and launches Microsoft Excel to display the content of the file. With this redesigned functionality, the Service Manager server and client performance are both improved by using file exchange and eliminating XML exchange for exported data.

Service Manager DDE server support provides an interface to applications outside Service Manager, allowing the use of DDE functions like poke and execute.

Related topics

DDE server
Usage notes
Executes

1.1.5.1. Architecture

In the case of Service Manager Telephony (standard system), the following occurs when a phone call comes in to the Windows client (Web Client):

- RTE (Java telephony applet) generates a ReceiveInteraction event

  Note

  In the Web client, the Java telephony applet prompts the user to save before receiving the call.

- RTE (Java telephony applet) passes the event to the System Event Handler (SEH)
- SEH starts the us.router RAD application in a new RAD thread

When you make a phone call, the RAD application sends a MakeCall event to the SEH, which invokes an RTE function (Java telephony applet).

While system events are normally communicated between the application layer and the RTE (Java telephony applet), they can also pass events between RAD applications and between RTE (Java telephony applet) functions.

To send a system event from RAD, use the event.send RAD Command panel. You must use -2 as the Thread ID. Fill in the Event Name with the name of the event and pass any parameters in the Names and Values arrays. Since each event is arbitrary, so are the parameters it requires.

When receiving a system event in a RAD application, use the event.name() and event.value() functions to get the event name and parameters for that event.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.2. DDE client

There are six different actions associated with a DDE client conversation that initiate from a DDE RAD panel. The actions are the standard DDE actions:

- Initiate
- Advise
- Request
- Poke
- Execute
- Terminate

Note

Constructing a DDE RAD application requires you follow RAD conventions. For a complete description of programming in RAD, along with requirements for any RAD application to function, refer to the System Language help in the Tailoring module.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.2.1. DDE RAD panel

The DDE RAD panel performs DDE commands. The RAD start panel sets variables referenced on this panel and later in the RAD application.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.2.2. FrameRestore option

The FrameRestore option directs Service Manager to take the focus when a DDE advise hot link updates. For example, if the FrameRestore parameter is added to an advise DDE RAD panel, a software telephone being used by an agent to receive a call takes the focus. This allows the agent to answer the call without searching for the soft phone application.

To activate the FrameRestore option in RAD, enter FrameRestore as the fifth input value parameter in the appropriate DDE advise panel.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.2.3. PassFocus option

The PassFocus option directs Service Manager to pass the focus when a DDE terminate command issues. For example, if the PassFocus parameter is added to a terminate DDE RAD panel, the focus stays on the receiving application and focus doesn't return to Service Manager. Without the Passfocus command, Service Manager gains the focus.

To activate the PassFocus option in RAD, type PassFocus as the second argument of a DDE terminate panel.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.2.4. Process panel

A process panel is a type of RAD panel that initializes or sets variables used later in your RAD application. Process panels also process expressions.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.2.5. Structure support option

The structure support option allows a DDE advise action to use a user-defined data format. For example, DF,129, UL, I, SZ33. All values are comma delimited.

Sample values:

Value	Description
DF	Identifies the string as a data format
129	Integer value for the clipboard data format value required for the advise action.
UL	Unsigned long integers
I	Integers
SZ33	Null terminated string 33 characters long

Related topics

Dynamic Data Exchange (DDE)

1.1.5.2.6. Usage notes

To use the DDE Server functionality, the Service Manager Windows client must be started and a user should be logged in. DDE doesn't automatically start the server (as OLE does). The user must be logged in so that the environment is set up for the Service Manager user.

The DDE server can also handle DDE client transactions that perform:

- Requests: Get the value of a named item and return it as a string
- Pokes: Set the value of a named item
- Executes: Ask the GUI to execute a transaction or set the focus to a named item

Use this functionality to script user interaction for common operations such as closing a record, which may require several fields to be filled in and several transactions to be made.



Note

This functionality works for the Windows client only. It doesn't work for the web client.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.3. DDE server

Integration with external Windows applications is achieved using the DDE server functionality of the Service Manager Windows client.

Applications that implement the Microsoft Windows DDE server permit external applications to get and set data, as well as execute commands. Typical use of get and set is to inspect and change data that's part of a document. For example, Excel allows the contents of spreadsheet cells to be read and written. Use the DDE execute facility to issue commands such as File > Save. There is little standardization between DDE server applications either in capabilities offered or the formatting of the commands sent over the DDE link.

DDE clients contact DDE servers using an application and topic name. The application name must be unique on the machine and the topic is typically the name of a document or the “Actions” topic.

Implementation - system events

System events permit Service Manager to react to events on client platforms external to the Service Manager system. System events are an arbitrary set of events that are sent to and from either RAD or the RTE. They are used to start new RAD applications.

DDE server implementation provides the DDE execute facility. Service Manager’s DDE service name is "HP_Service_Manager" and the topic name is "Actions". The DDE execute facility initiates Service Manager system events.

For example, the CTI implementation creates a system event to start a RAD application when the phone rings to handle the call. RAD programs written to extract the system event parameters and act upon them handle system events. With the DDE server functionality, an external application such as Excel, Access, or Delphi, connects to Service Manager using application Service Manager and topic Actions to trigger system events.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.3.1. Executes

Executes are the DDE mechanisms for requesting that an application process data or perform an action. Service Manager provides the Transact execute capability.

Transact

The Transact execute function directs Service Manager to execute a transaction as though a user had pressed a function key or button. The Transact execute function requires one operand that designates the number of the function key or button ID of the button that was pressed. This example in Visual Basic for Applications shows how to tell Service Manager that the “fill” key was pressed:

```
DDEExecute nChannel, “[Transact( ““9”” )]” ’issue a fill command
```

Related topics

Dynamic Data Exchange (DDE)

1.1.5.3.2. SystemEvents file

You define system events in the SystemEvents file. This file contains records that have an Event Name and a RAD Application name. When the system receives an event, the corresponding RAD application invokes a new RAD thread. The SystemEvents file contains all the events that invoke a RAD application. The events used to invoke a RTE function register themselves upon start-up of the client. If you edit a system event record, you must re-login before the system recognizes the changes.



Note

You can have multiple records for one event; each application invokes a separate RAD thread.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.3.3. Requests and pokes

Requests and Pokes are Dynamic Data Exchange (DDE) mechanisms for obtaining a copy of or setting the value of named items. Service Manager uses the widget's input property to name the item. For example, to set the user name on the Service Manager login screen, use the following DDE command (this example is in Visual Basic for Applications): `DDEPoke nChannel, "$user.id", "falcon"`

Requests and pokes take and return data of string type.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.3.4. DDE example

This example uses Visual Basic for Applications.

Note

Note: The Service Manager server must be running in order for the following example to work.

The format of the Service Manager execute command string for system events is:

`SystemEvent( event name, parameter name, parameter value, ...)`

Where *event name* corresponds to the event name in the Service Manager SystemEvents table and *parameter name / value pairs* are known to the RAD program.

```
Sub ReceiveInteraction()
        channel = DDEInitiate("Service Manager", "Actions")
        DDEExecute channel, "[SystemEvent(""ReceiveInteraction"",
""Caller Name"", ""KENTNER"" )]"
        DDETerminate channel
        End Sub
```

The example above:

- Initiates a conversation with the Service Manager system.
- Executes a DDE command starting the system event ReceiveInteraction, passing the parameter name Caller Name with the parameter value of Kentner.
- Terminates the conversation.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.3.5. Access the script panel

Applies to User roles: System Administrator

The DDE script panel, dde.script.g, assists in creating Service Manager DDE scripts for DDE calls inside Service Manager against an outside application such as Excel.

To access the DDE script panel, click **Tailoring** > **Tailoring Tools** > **DDE Script**.

Related topics

Dynamic Data Exchange (DDE)

1.1.5.3.6. Access system events records

Applies to User roles: System Administrator

To access system events records:

1. Type `db` in the command line, and click the Execute Command button. The Database Manager dialog box opens.
2. Type `SystemEvents` in the Form field.
3. Click **Search**. A blank system events record opens.
4. Type the name of the record you want to view, or click **Search** to display a record list of all events in the system.

Related topics

Dynamic Data Exchange (DDE)

1.1.6. Integration Manager

Integration Manager refers to a plug-in-based platform called Service Manager Integration Suite (SMIS), which can provide centralized management and configuration of all instances of integrations that are configurable in SMIS.

Accessing Integration Manager

To access the Service Manager Integration Manager, use the Windows or web client and open **Tailoring** > **Integration Manager** from the System Navigator.

Integration Manager (SMIS) provides the following buttons or context menu options, which enable integration delivery engineers and support engineers to manage integration instances:

- **Add**: Add a new integration instance.
- **Edit**: Edit an existing integration instance.
- **Clone**: Clone an existing integration instance.
- **Refresh**: Refresh the status of an integration instance.
- **Enable**: Enable an integration instance.
- **Disable**: Disable an integration instance.
- **Delete**: Delete an existing integration instance.
- **Task**: Monitor failover tasks of an integration instance during a specified period of time.
- **Log**: View task execution history.

Integration types

SMIS supports two types of integrations:

- Schedule-based: A schedule-based integration runs as a schedule in the background. When a schedule-based integration instance is enabled, a corresponding scheduler is started in Service Manager.
- UI-based: A UI-based integration can be invoked only in the UI.

Out-of-box integration templates

Out-of-box, SMIS comes with a set of predefined templates that you can use to set up integrations to the following third-party products, OpenText products, or Service Manager components:

Product	Template name
Another product that supports Case Exchange	• CaseExchangeDefaultTemplate • CaseExchangeSM_SAW (for integration to SMAX) • CaseExchangeSM_SM_Pull • CaseExchangeSM_SM_Push
Microsoft Teams	MicrosoftTeams
Application Lifecycle Management (ALM)	SMALM
External email servers	SmartEmail
Application Performance Management (APM)	• SMBSM_DOWNTIME (for SM-APM downtime synchronization) • SMOMi (for integration to Operations Manager i) • SMBIR (for integration to Business Impact Report)
Smart Analytics	• SMIDOL • SMIDOLAutoFill • SMIDOLOCR
Atlassian Jira software	SMJira
ALM Octane	SMOctane
Operations Orchestration (OO)	SMOO
Project and Portfolio Management Center (PPM)	SMPPM
SAP Solution Manager	SMSAP
Third-party survey tools	SMSurvey
Universal CMDB	SMtoUCMDB

Note

The out-of-box templates aren't provided as sample interfaces. You need to access the Integration Manager, set up the integrations through the Integration Template Selection wizard, and configure your system to interface with the OpenText products listed above.

Developing custom integrations using SMIS

SMIS is also a development platform that enables you to develop new integration templates of your own. You can then register these new integration templates in SMIS so that they are configurable in SMIS. For more information, see Service Manager Integration Suite (SMIS).

Related topics

Integrations
Service Manager Integration Suite (SMIS)
Add or delete an integration instance
Add a APM Business Impact Reports integration
Add an integration instance for each Operations Manager i (OMi) server
Add an Operations Orchestration integration
Edit an integration instance
Enable or disable an integration instance
Monitor integration instance status
Monitor failover tasks
Configure a callback

1.1.6.1. Add or delete an integration instance

Applies to User roles: System Administrator

Before you can use an integration that's configurable in Integration Manager, you must add (and then enable) an instance of this integration.

To add an integration instance:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select a template from the Integration Template list.
4. If you want to use the field mappings provided with the template, select **Import Mapping**.

   Note

   A template becomes unavailable when the number of integration instances based on this template reaches a maximum value predefined in the template.

5. Click **Next**. The Integration Instance Information page opens. Verify or complete the fields as necessary. For the description of each field on this page, see the related topics.

   Note

   Some fields are automatically populated with the information from the integration template. However, you can modify their values as necessary.

6. Click **Next**. The Integration Instance Parameters page opens. This page contains all predefined parameters for the integration: General Parameters and Secure Parameters. You may set confidential parameters on the **Secure Parameters** tab, and the parameter values will appear as asterisks (*) for data security.

   Note

   You can modify the predefined parameter values as necessary. However, it's not recommended to add new parameters, modify parameter names or categories.

7. Click **Next**. The Integration Instance Fields page opens. This page contains all predefined fields available for the integration:

   • **SM Fields:** Contains a list of fields at the Service Manager server side and you can map these fields to endpoint fields.

- **Endpoint Fields:** Contains a list of fields at the endpoint side and you can map these fields to Service Manager server fields.

8. Click **Next**. The Integration Instance Mapping page opens. This page contains all field mappings and value mappings between Service Manager server fields and endpoint fields. Review the mappings and make modifications to suit your needs if necessary.

   Note

   The field mappings are available if you selected **Import Mapping** in step 4. Otherwise, the **Field Mapping** tab is empty. You may need to add field mappings manually. For detailed instructions about how to configure mappings, see the related topics.

9. Click **Finish**. The system adds the new integration instance.

To delete an integration instance:

Note

You can only delete integration instances that are in Disabled status.

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select the integration instance you want to delete.
3. Click **Delete**. This will delete the integration instance record, the corresponding Field Mapping records, and the scheduler record (for **schedule-based** integrations only).

Related topics

- Integration Manager
- Service Manager Integration Suite (SMIS)
- Integration Instance Mapping
- Integration Instance Information fields
- Edit an integration instance
- Add or delete field mappings
- Edit field mappings
- Configure value mappings
- Enable or disable an integration instance
- Monitor integration instance status
- Monitor failover tasks
- Configure a callback

1.1.6.1.1. Integration Instance Information fields

When you add or edit an integration instance, you must verify or complete the following fields on the Integration Instance Information page. The sample values below are given for an instance of the Service Manager to Operations Orchestration integration.

Field	Sample Value	Description
Name (required)	SMOO	Name of the integration instance. This field is automatically populated with the information from the integration template and can be modified to suit your needs.
Version (required)	1.0	Version of the integration. This field is automatically populated.
Interval Time (s) (required)	300	Enter an interval (in seconds) at which the integration instance is scheduled to run. **Note:** This field is required only when **Category** is **Schedule-based**.
Max Retry Times (required)	3	Enter a maximum allowed number of retries when a scheduled run of the instance fails. **Note:** This field is required only when **Category** is **Schedule-based**
SM Server	localhost	Enter a display name that identifies the Service Manager server machine you are using.

Field	Sample Value	Description
Endpoint Server	OO2-SRV	Enter a display name that identifies the endpoint server machine. For example, the display name of the Operations Orchestration server machine for the SM to OO integration.

Field	Sample Value	Description
Log Level	INFO	Level of diagnostic information that the Service Manager server logs to the log file directory. The possible log levels are as follows: • DEBUG: Designates fine-grained informational events that are most useful to debug the integration instance. • INFO (default): Designates informational messages that highlight the progress of the integration instance at coarse-grained level. • WARNING: Designates potentially harmful situations. • ERROR: Designates error events that might still allow the integration instance to continue running. • OFF: Turns off logging of the integration instance. The log hierarchy is DEBUG < INFO < WARNING < ERROR. Selecting a log of a higher level can render logs of lower levels invalid.

Field	Sample Value	Description
Log File Directory	C:\ Service Manager\server\logs	Enter an absolute path that exists on the Service Manager server, where log files will be stored. **Note:** If you enter a directory that doesn't exist, the log files will get lost.
Category	Schedule-based	Indicates whether the integration instance is Schedule-based or UI-based.
Shared Scheduler		The scheduler that the integration instance uses to schedule background processes. Multiple integration instances can use a shared scheduler. When you create an integration template, you can define a default shared scheduler for the instances that are based on the template. All instances that are based on this template can use the same shared scheduler. However, if you create an instance that uses a different scheduler, SMIS creates a new scheduler accordingly.
Run at system startup	selected	Indicates whether the integration instance runs at system startup. If selected, the current integration instance automatically runs when the Service Manager server is started.

Field	Sample Value	Description
Description	OO flows linked to SM Knowledge Management.	Description of the integration instance. You can modify the pre-populated text to suit your needs.

Related topics

Integration Manager
Integration Instance Mapping
Edit an integration instance

1.1.6.1.2. Edit an integration instance

Applies to User roles: System Administrator

> **Note**
>
> You can only edit integration instances in the Disabled status. Enabled integration instances (in Running or Sleeping status) can be viewed only.

To edit an integration instance:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance with a Status of Disabled, and click **Edit**.
   - In the Windows client, click an integration instance to select it.
   - In the Web client, hold down the Shift key as you click an integration instance to select it.

   The Integration Instance Information wizard opens.
3. Make changes to the field values as necessary, and then click **Next**. The Integration Instance Parameters wizard opens.
4. Update the parameter values with your real data if necessary. However, it's not recommended to add a new parameter, or modify parameter names or category.
5. Click **Next**. The Integration Instance Fields page opens, displaying all the fields predefined in the integration template:
   - **SM Fields:** A list of fields in the Service Manager server side that can be mapped to endpoint fields.
   - **Endpoint Fields:** A list of fields in the endpoint side that can be mapped to Service Manager server fields.
6. If necessary, on the **SM Fields** and **Endpoint Fields** tabs, add new fields and specify a field type for each of them: string, date, number, or boolean.
7. Click **Next**. The Integration Instance Mapping page opens. Make changes to the field mappings and value mappings to suit your needs.
8. Click **Finish**. Changes made to the integration instance are saved.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Mapping
Integration Instance Information fields
Add or delete an integration instance
Add or delete field mappings
Edit field mappings

Configure value mappings
Enable or disable an integration instance
Monitor integration instance status
Monitor failover tasks
Configure a callback

1.1.6.1.3. Integration Instance Mapping

The mapping functionality of an integration instance converts values between Service Manager and the other product automatically. It is based on the following settings:

- Value mapping - Each value mapping is based on the value mapping group table.
- Field mapping - Map the value of a source field to the value of a target field and define the direction of the mapping.
- Default value - Default value is used in the following situations:
  - No value mapping group is defined for this field and the source field value is empty.
  - A value mapping group is defined for this field but the value can't be mapped within the value mapping group.
- Callback - SMIS provides four callbacks for complicated field mappings, which provide the flexibility to implement corresponding business logic.
- Condition - SMIS supports conditional value mappings and enables you to configure conditions.
- Pre Script - The pre script can initialize variables so that succeeding processes can use these variables.
- Post Script - If all the previous processes can't fulfill your need, you can use the post script to perform additional process and store results into the `mapObj` object.

SMIS processes the above settings in the following sequence:

Field mapping > Pre Script > Value mapping > Default Value > Callback > Post Script

If you selected **Import Mapping** when adding an integration instance, the mapping template with out-of-box lists of predefined field mappings and value mappings become available.

The out-of-box field mappings and value mappings can be regarded as the minimum default settings for an integration instance to run. However, you can configure these default settings as necessary to suit your needs.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Information fields
Pre script
Post script
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Edit field mappings
Configure value mappings
Enable or disable an integration instance
Monitor integration instance status

Monitor failover tasks
Configure a callback
Configure a condition

1.1.6.1.3.1. Add or delete field mappings

Applies to User roles: System Administrator

To add a new row of field mapping, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance with a status of Disabled. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. Select the **Field Mapping** tab.
5. In the **SM Field** column, click the first blank cell, and select a field from the list.
6. In the **Direction** column, select a direction from the list.
7. In the **Endpoint Field** column, click the first blank cell, and select a field from the list.
8. In the **SM Default** or **EP Default** column, enter a default value or use a place holder expression.

   For more information about place holder, see Use placeholders.
9. In the **SM Callback** or **EP Callback** column, enter a callback function.
10. In the **Value Mapping Group** column, specify a value mapping group if necessary.
11. Click **Finish**.

To delete a row of field mapping, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance with a status of Disabled. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. Select the **Field Mapping** tab.
5. Clear all the fields in the row you want to delete.
6. Click **Finish**.

Related topics

Integration Manager
Integration Instance Mapping
Use placeholders
Edit an integration instance
Configure a callback
Edit field mappings
Configure a condition
Integration Instance Information fields

1.1.6.1.3.2. Edit field mappings

Applies to User roles: System Administrator

To edit a field mapping of an integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance with a status of Disabled. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. Select the **Field Mapping** tab.
5. In the **SM Field** column or **Endpoint Field** column, click an entry you wish to edit or configure, and select a field from the list.

   Note

   If you want to add a new field that's unavailable in the list, you must first go to the Integration Instance Fields page, add this field, and specify a field type for it. There are four field types available: string, number, date, and boolean.

6. If necessary, click in the **Direction** field, and select a new direction from the list. A bidirectional direction means two mappings actually.
7. After you have finished modifying the field mappings, click **Finish**.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Mapping
Integration Instance Information fields
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Configure value mappings
Enable or disable an integration instance
Monitor integration instance status
Monitor failover tasks
Configure a callback
Configure a condition

1.1.6.1.3.3. Configure a callback

Applies to User roles: System Administrator

A callback is a piece of executable code that's passed as a parameter to other code. The Service Manager Integration Suite (SMIS) provides the following four callback functions:

Callback Function	Description
lookup	Looks up a record from a Service Manager table matching the query condition. If a matching record is found, the **lookup** callback returns the value of the specified field; if not, it either returns an empty value or ignores the current mapping. **Note** Multiple conditions should be separated with "||".
combine	Concatenates an array of values with a specified delimiter.
setValue	Sets values when the current action is among any combinations of **Insert**, **Update**, and **Close**. Otherwise, the **setValue** callback either returns an empty value or ignores the current mapping.
when	Return the value if any condition is met. In both the **condition** and **value** fields, use "${sm.field}" to get the value of SM field or use "${ep.field}" to get the value of the end point field.

Note

For more information about how to use placeholders (for example, ${sm.field} and ${ep.field}), see Use placeholders.

You configure callbacks for an integration instance from the Integration Instance Mapping page. You can edit or clear callbacks.

To edit a callback, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. On the **Field Mapping** tab, select a cell in the **SM Callback** or **EP Callback** column.
5. Click **Edit Callback**. The Callback Editor window opens.
6. Select a value from the **Callback Function** list. The parameters that display in the window vary with the callback function you selected: **lookup**, **combine**, **setValue**, or **when**.
7. Provide values for all the parameters. Click **OK** to save the callback.
8. Click **Finish**.

> **Note**
>
> The changes you made to the integration instance aren't saved until you click **Finish**.

To delete a callback, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. On the **Field Mapping** tab, select a cell in the **SM Callback** or **EP Callback** column.
5. Click **Clear Callback**.
6. Click **Yes** to confirm the deletion.
7. Click **Finish**.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Mapping
Integration Instance Information fields
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Edit field mappings
Configure value mappings
Enable or disable an integration instance
Monitor integration instance status
Monitor failover tasks
Configure a condition

1.1.6.1.3.4. Configure value mappings

Applies to User roles: System Administrator

You configure value mappings for an integration instance from the Integration Instance Mapping page.

> **Note**
>
> JavaScript is unsupported in the **SM Values** and **End Point Values** columns of the **Value Mapping** section. If JavaScript calculations are required, you have to add the JavaScript code into the **Pre Script** or **Post Script** section.

To edit a value mapping, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. Select the **Value Mapping** tab.
5. Click the entry you want to edit, and enter a new value.
6. Click **Finish**.

To add a value mapping, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. On the **Value Mapping** tab, click the fields in the first blank row, and enter values for the fields.
5. Click **Finish**.

To delete a value mapping, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. On the **Value Mapping** tab, clear all the fields in the row you want to delete.
5. Click **Finish**.

For more information about how to configure conditions for value mappings, see Configure a condition.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Mapping

Integration Instance Information fields
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Edit field mappings
Enable or disable an integration instance
Monitor integration instance status
Monitor failover tasks
Configure a callback
Configure a condition

1.1.6.1.3.5. Configure a condition

Applies to User roles: System Administrator

SMIS enables you to configure conditions for value mappings. You can set the **Condition** field of a value mapping entry to one of the following strings:

- `true`: The value mapping defined in this entry is always effective.
- `false`: The value mapping defined in this entry is always ineffective.
- A string that uses place holder: The value mapping defined in this entry is effective only when the condition is met.

  For more information about how to use place holder, see Use placeholders.



Note

- If the value for a mapping condition is blank, the value is set as `true` by default.
- JavaScript is unsupported in the **Condition** field of the value mapping. In case JavaScript calculations are required, you have to add the JavaScript code into the **Pre Script** or **Post Script** section.

When you configure value mappings and set conditions, make sure logic conflicts don't occur. If several conditions of a value mapping are met but a logic conflict occurs, only the last value mapping is effective. As in the following example, the value mapping result for the Status field in Service Manager is Pending rather than Open.

Value Mapping Group	SM Values	End Point Values	Description	Condition
Status	Open	Pending		${context.inbound} && ${context.newSMIncident}
Status	Pending	Pending		${context.inbound} && ${context.newSMIncident}

To add or edit a condition, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.

2. Select an integration instance. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. On the **Value Mapping** tab, select a cell in the **Condition** column.
5. Provide or edit the value for the condition.
6. Click **Finish** to save your changes.

To delete a condition, follow these steps:

1. Click **Tailoring** > **Integration Manage**r. Integration Instance Manager opens.
2. Select an integration instance. The Integration Instance Information wizard opens.
3. Click **Next** to get to the Integration Instance Mapping page in the wizard.
4. On the **Value Mapping** tab, select a cell in the **Condition** column.
5. Remove the condition value.
6. Click **Finish** to save your changes.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Mapping
Integration Instance Information fields
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Edit field mappings
Configure a callback
Configure value mappings
Enable or disable an integration instance
Monitor integration instance status
Monitor failover tasks

1.1.6.1.3.6. Pre script

Service Manager Integration Suite (SMIS) executes the pre script after field mapping and before value mapping. You can use the pre script to initialize variables so that SMIS can use these variables in value mapping, or execute additional calculations to influence the value mapping. You can follow normal Javascript syntax to compose the pre script. For the objects that the pre script can use, see Placeholder objects.

For example, the following pre script first verifies if the `properties.Description` field in the endpoint is not empty. If the field is not empty, the script calls a function and assigns the result of the function back to the `properties.Description` field.

```
if(ep['properties.Description']) ep['properties.Description'] =
lib.CaseExchange_SAWUtil.processHtmlText(param, context, 'description',
ep['properties.Description']);
```

It is unsupported to update the affected record by using JavaScript statements in the **Pre Script** section. Only the update of the record itself via the running SMIS instance is supported.

Related topics

Post script
Integration Instance Mapping

1.1.6.1.3.7. Post script

If all previous processes can't fulfill your need, you can write the post script to perform additional process and store results into the `mapObj` object. You can follow normal Javascript syntax to compose the post script. For the object that the post script can use, see Placeholder objects.

It is unsupported to update the affected record by using JavaScript statements in the **Post Script** section. Only the update of the record itself via the running SMIS instance is supported.

Related topics

Pre script
Integration Instance Mapping

1.1.6.1.3.8. Use placeholders

You can use placeholders in the following fields when you configure an integration instance:

- In field mapping: **Callback**, **SM Default**, and **EP Default**
- In value mapping: **Condition**

You can use the following placeholder syntax:

- `${(sm|ep|param|context|vars).fieldName}` if `fieldName` doesn't contain a "."
- `${(sm|ep|param|context|vars)["fieldName"]}` if `fieldName` contains a "."

You can use the `sm`, `ep`, `param`, `context`, and `vars` objects in a placeholder, see *Placeholder objects* below for details.

Placeholder allows you to use any Javascript syntax, or to call a function from the ScriptLibrary, for example, `lib.scriptlib.function`.

The following examples are some common use of placeholder:

- `${(sm.AssignmentGroup)}` represents the value of the **AssignmentGroup** field in Service Manager.
- `${(ep["AssignedGroup.Name"])}` represents the **AssignedGroup.Name** field in the end point.
- If you use `${context.inbound==true} && (!${context.newSMIncident})` in a condition, the condition is "The value of the **context.inbound** field is `true` and the value of the **context.newSMIncident** field is `false`."
- `${vars.$L_file.assignment}` refers to the field assignment of variable `vars.$L_file`.
- Use `${lib.<scriptlib>.<func>()}` to call a Java script.

Placeholder objects

You can use the following objects in a placeholder:

- `sm`: Refers to the value of a field in Service Manager. You can use either `fieldName` or `alias` to refer to the value of a Service Manager field.
- `ep`: Refers to the value of a field in the end point. You can use either `fieldName` or `alias` to refer to the value of an end point field.
- `param`: Refers to the value of a parameter that's defined in SMIS configuration.
- `context`: Refers to the task information as in the following table. `context` can store shared variable that you can use in other places, for example, callback, condition in value mappings, and post script.

Task information	Description
internalId	The ID of the record in Service Manager.
externalId	The ID of the record in the end point.
object	The table name of the object in Service Manager.
action	The action of an inbound or outbound request.
direction	The direction of the action: inbound or outbound.

Task information	Description

internalObject	Refers to a specific record in Service Manager. You can perform the following operations with `internalObject`:
	◦ Directly modify `internalObject` to update the record. This operation is usually used for inbound tasks. For example, the following code in Additional Script can directly specify the value of a field:
	`context.internalObject["resolution.code"]="Request Rejected"`
	It's not recommended that you directly modify the value of a field by manipulating `internalObject`, because this action causes an inconsistency between the activity log and the actual result. Therefore, `internalObject` is usually used to perform operations that doesn't change the record itself, for example, creating activity log, or building relationship with another record.
	◦ Retrieve certain record information through `internalObject` in the pre script or post script. For example, the following code retrieves the value of the **Resolution Code** field through `internalObject`:
	`if(context.action=="Cancel" && context.internalObject['resolution.code']=="Withdrawn by User") context.cancelledBySM = true;`

- `vars`: Refers to a global variable.
- `mapObj`: Refers to the last map object, which contains the final result of field mapping, pre script, and value mapping. You can only use `mapObj` in the post script. If you modify `mapObj` in the post script, your modification will overwrite the result of field mapping

and value mapping. That is, the value of the corresponding field in the record will be what you specified in the post script.

The following example scenario helps you understand the use of the `sm`, `ep`, and `mapObj` objects:

- Assume that you configure the field mapping between the following SM and endpoint fields:
  - SM field: `resolution.code`
  - Endpoint field: `properties.CompletionCode`
- Some of the value mapping configurations for these two fields are as follows:

SM field (`resolution.code`)	**Endpoint field (`properties.CompletionCode`)**
Automatically Closed	NoUserResponse
Not Reproducible	NotReproducible

- In an inbound task, the value of the `properties.CompletionCode` field is `NoUserResponse` in the endpoint.

In this scenario, after the field mapping (and before the execution of the pre script), the `sm` and `ep` objects are initialized as follows:

- ep["properties.CompletionCode"]="NoUserResponse"
- sm["resolution.code"]="NoUserResponse"

You can manipulate these objects in the pre script.

After the value mapping, the `mapObj` object is initialized as follows (if you don't change these objects in the pre script):

mapObj["resolution.code"]="Automatically Closed"

This is the result of the field mapping and value mapping. If you don't modify this result in the post script, the **Resolution Code** field of the record will be **Automatically Closed**.

However, if you modify the result by adding the following code in the post script:

mapObj["resolution.code"]="Not Reproducible"

The previous field mapping and value mapping result will be overwritten, and the **Resolution Code** field of the record will be **Not Reproducible**.

Related topics

Configure a condition
Edit field mappings

1.1.6.2. Enable or disable an integration instance

Applies to User Roles: System Administrator

To enable an integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance with a Status of Disabled, and click **Enable**.
3. In the prompt window, click **Yes**. The integration instance is enabled.

   If the integration instance is successfully enabled, its status changes to Running. However, if the integration instance is schedule-based, its status switches between Running and Sleeping:

   - **Running** indicates that the integration instance is processing data at a scheduled time interval.
   - **Sleeping** indicates that the integration instance has finished the data processing at a scheduled time interval. When a subsequent scheduled time interval is triggered, the integration instance status changes from Sleeping to Running.

To disable an integration instance, follow these steps:

> **Note**
>
> A schedule-based integration instance can be disabled only when it is in the Sleeping status.

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance with a Status of Running (UI-based) or Sleeping (schedule-based), and then click **Disable**.
3. In the prompt window, click **Yes**. The integration instance is disabled.

> **Note**
>
> Disabling a schedule-based integration instance terminates the corresponding scheduler of the integration instance. If the scheduler is not terminated immediately, wait for a while and click **Refresh** (Refresh Integration instance) to check the status. If the integration instance is still not disabled, repeat the Disable operation or go to **System Status** to manually terminate the corresponding process named after the integration instance and its ID.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Mapping
Integration Instance Information fields
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Edit field mappings
Configure value mappings
Monitor integration instance status
Monitor failover tasks
Configure a callback

1.1.6.3. Monitor integration instance status

Applies to User roles: System Administrator

To monitor integration instance status:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance.
3. Click **Refresh**. The status of the selected integration instance refreshes.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Add or delete an integration instance
Edit an integration instance
Enable or disable an integration instance
Monitor failover tasks

1.1.6.4. Monitor failover tasks

Applies to User roles: System Administrator

The Task Manager window lists all failed tasks of an integration instance. You can check to see whether a task failed or has expired, as well as its retry times. A task expires if the scheduler has performed the maximum number of retries for it. If a task has expired, Service Manager no longer processes it until you reset it or manually rerun it.

To monitor a failover task, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance.
3. Click **Task**. The Task Manager window opens.
4. To filter tasks by time range, select a date range in the **From** and **To** fields, and click **Filter**.
5. Double-click a record in the task list. The task details are shown.
6. (Optional) If the task is unexpired, click **Reset** to set **Retry Times** to **0** and **Expired** to **false**.

   Or

   If the task is expired, click **Run Now** to rerun it.

   Or

   For an outbound task, if the record is out-of-date (for example, the Incident has been updated after the task is created), click **Merge** to merge the latest field value of the record to the task.



Note

When a failed task is successfully done, it is removed from the task queue.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Mapping
Integration Instance Information fields
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Edit field mappings
Configure value mappings
Enable or disable an integration instance

Monitor integration instance status
Configure a callback

1.1.6.5. Monitor SMIS task log

When an integration instance is active, Service Manager logs all system messages of the integration in the Service Manager Integration Suite (SMIS) task log. A log entry in the SMIS task log always links to a task in the SMIS task queue. The SMIS task log allows an administrator to view the task history of an integration instance, and restart unsuccessful tasks when needed. For more information about how to enable an integration instance, see Enable or disable an integration instance.

To monitor the SMIS task log, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select an integration instance.
3. Click **Log**. The **Task Log** page opens.
4. To filter log entries by time range, select a date range in the **From** and **To** fields, and click **Filter**.
5. Double-click a record in the log list. The **Task Log Detail** page opens.

If the task is unsuccessful, you can continue the following steps to restart the task:

1. click the **View** button on the **Task Log Detail** page. The **Task Detail** page opens.
2. Click **Run Now**.

Purge the SMIS task log records

By default, the Service Manager Integration Suite (SMIS) task log keeps all task records even if these tasks are successfully processed and removed. Service Manager provides an out-of-box scheduler record to purge the SMIS task log. To use this scheduler record, follow these steps:

1. Click **Tailoring** > **Database Manager**.
2. Type schedule in the **Form** field and then click **Search**.
3. Open **schedule** from the record list. A blank Schedule File record is displayed.
4. Type background purge/archive SMISTaskLog in the **Name** field and then click **Search**.
5. Specify the **Expiration** field to define the start time of the purge task.
6. Click **Save**.

Related topics

Integration Manager
Service Manager Integration Suite (SMIS)
Integration Instance Mapping
Integration Instance Information fields
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Edit field mappings
Configure value mappings

Enable or disable an integration instance
Monitor integration instance status
Configure a callback
Monitor failover tasks

1.1.6.6. Using LW-SSO with integrations

Lightweight Single Sign-On (LW-SSO) is optional but highly recommended for some integrations. Enabling LW-SSO for integrations will bypass the login prompts when connecting two OpenText products.

Incoming UI LW-SSO

If LW-SSO is enabled in both the Service Manager Web tier and another OpenText product, users who have logged on to Service Manager are allowed to sign on to the other OpenText product through the web tier without a login prompt; however, you must enable LW-SSO in the Service Manager server additionally if you want users who have logged on to another product to directly sign on to Service Manager through LW-SSO. This is because the Service Manager server needs to trust the Web tier.

Incoming Web Services LW-SSO

If LW-SSO is enabled in the Service Manager server, other OpenText products can use a dedicated user account to access Service Manager Web Services without re-authentication.

Integration user accounts

For each integration, there might be two types of integration user accounts:

- Dedicated user account: a dedicated user account that one product uses to call Web Services of the end-point product through LW-SSO.
- End-user account: a user account that an end-user uses to log in to one product and then sign on to another product through LW-SSO.

These two types of integration user accounts must be created on both product sides. In addition, an integration user account must have the same user name (but can have different passwords) on the two sides.

You can use LW-SSO with the following integrations:

- Operations Orchestration (OO)
- Application Performance Management (APM) Operations Manager i
- Application Performance Management (APM) Business Impact Report

For details, see the specific information about how to configure LW-SSO in each of these products.

Related topics

Integration Manager
Operations Orchestration (OO)
Application Performance Management (APM)
Incident Exchange (OMi - SM) integration

APM Business Impact Report (BIR)
Configure LW-SSO in the Service Manager server
Configure LW-SSO in the Service Manager Web tier
Enable LW-SSO for the Operations Orchestration integration
Enable LW-SSO for the Incident Exchange (OMi - SM) integration

1.1.6.6.1. Configure LW-SSO on the Service Manager server

Applies to User roles: System Administrator

Service Manager servers support Lightweight Single Sign-On (LW-SSO). A Service Manager integration can pass an authentication token to Service Manager and doesn't require re-authentication. This simplifies the configuration of Single Sign-On for OpenText solutions by removing the need to use Symphony Adapter (which proxies LW-SSO-based authentication with the Service Manager Trusted Sign-On solution).

Enabling LW-SSO in the Service Manager server enables web service integrations from other OpenText products to bypass Service Manager authentication if the product user is already authenticated and a proper token is used; enabling LW-SSO in both the Service Manager server and web tier enables users to bypass the login prompts when launching the Service Manager web client from other OpenText applications.



Note

- LW-SSO version 2.5 is supported.
- If Service Manager is running in FIPS mode, ignore the steps described here and follow the instructions in Configure LW-SSO in the Server for FIPS mode instead.

To configure LW-SSO in the Service Manager server:

1. Go to the <Service Manager server installation path>/RUN folder, and open `lwssofmconf.xml` in a text editor.
2. Make sure that the `enableLWSSOFramework` attribute is set to `true` (default).
3. Change the domain value `yourdomain.net` to the domain name of your Service Manager server host.

   Note

   To use LW-SSO, your Service Manager web tier and server must be deployed in the same domain; therefore you should use the same domain name for the web tier and server. If you fail to do so, users who log in from another application to the web tier can log in but may be forcibly logged out after a while.

4. Set the `initString` value. This value MUST be the same with the LW-SSO setting of the other OpenText product you want to integrate with Service Manager.

5. Optionally, you can change attributes `paddingModeName`, `keySize`, `encodingMode`, `engineName`, and `cipherType`. However, you must make sure that they are the same with the LW-SSO setting of the other OpenText product that you want to integrate with Service Manager.

> **Caution**
>
> Do not change the other configurations, such as the content in tag `<restURLs>`, and the attribute of tag `<service>`.

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<lwsso-config
xmlns="http://www.hp.com/astsecurity/idmenablmentfw/lwsso/2.0">
<enableLWSSO enableLWSSOFramework="true"
enableCookieCreation="true" cookieCreationType="LWSSO" />
<web-service>
        <inbound>
          <restURLs>
                <url>.*7/ws.*</url>
                <url>.*sc62server/ws.*</url>
                <url>.*/ui.*</url>
                <url>.*SM/9/rest.*</url>
          </restURLs>
          <service service-type="rest" >
            <in-lwsso>
                <lwssoValidation>
                  <domain>yourdomain.net</domain>
                 <crypto cipherType="symmetricBlockCipher"
engineName="AES"
                  paddingModeName="CBC" keySize="256"
encodingMode="Base64Url"
                  initString="This is a shared secret passphrase"
</crypto>
                </lwssoValidation>
            </in-lwsso>
          </service>
         </inbound>
         <outbound/>
</web-service>
</lwsso-config>
```

Related topics

Using LW-SSO with integrations
Configure LW-SSO in the Service Manager Web tier
Configure LW-SSO in Application Performance Management (APM)
Configure LW-SSO in Operations Orchestration (OO)

1.1.6.6.2. Configure LW-SSO on the Service Manager Web tier

Applies to User Roles: System Administrator

If Lightweight Single Sign-On (LW-SSO) is enabled on the Service Manager Web tier, integrations from other OpenText products will bypass Service Manager authentication when launching the Service Manager Web client, provided that the OpenText product user is already authenticated and a proper token is used.

Note

- All products involved in LW-SSO must use the same user account.
- To enable users to launch the Web client from another OpenText product using LW-SSO, you must also enable LW-SSO in the Service Manager server.
- Once you have enabled LW-SSO in the web tier, web client users should use the web tier server's fully-qualified domain name (FQDN) in the login URL: `http://<myWebtierHostName>.<myDomain>:<port>/webtier-x.xx/index.do`
- This section describes the steps for non-FIPS mode. When the Service Manager Server is running in FIPS mode, the steps are slightly different. See Configure LW-SSO on the Web tier for FIPS mode.

The following procedure is provided as an example, assuming that the Service Manager Web tier is deployed on Tomcat.

To configure LW-SSO in the Service Manager Web tier:

1. Open the `<Tomcat>\webapps\<Service Manager Web tier>\WEB-INF\web.xml` file in a text editor.
2. Modify the `web.xml` file as follows:
    1. Set the `<serverHost>` parameter to the fully-qualified domain name of the Service Manager server.

        Note

        This is required to enable LW-SSO from the web tier to the server.

    2. Set the `<serverPort>` parameter to the communications port of the Service Manager server.
    3. Change the value of context parameter **isCustomAuthenticationUsed** to false.

```
<context-param>
<param-name>isCustomAuthenticationUsed</param-name>
<param-value>false</param-value>
</context-param>
```

4. Remove the comment tags (<!-- and -->) enclosing the following elements to enable LW-SSO authentication.

```
<!--
<filter>
<filter-name>LWSSO</filter-name>
<filter-
class>com.hp.sw.bto.ast.security.lwsso.LWSSOFilter</filter-
class>
</filter>
-->
......
<!--
<filter-mapping>
<filter-name>LWSSO</filter-name>
<url-pattern>/*</url-pattern>
</filter-mapping>
-->
```

5. Save the `web.xml` file.

3. Open the `<Tomcat>\webapps\<Service Manager Web tier>\WEB-INF\classes\lwssofmconf.xml` file in a text editor, and then update this file as follows:

   1. Set the value of `enableLWSSOFramework` to `true` (default is false).

```
<enableLWSSO
enableLWSSOFramework="true"
enableCookieCreation="true"
cookieCreationType="LWSSO"/>
```

   2. Set the `<domain>` parameter to the domain name of the server where you deploy your Service Manager Web tier. For example, if your Web tier's fully qualified domain name is `mywebtier.domain.hp.com`, then the domain portion is `domain.hp.com`.

> **Note**
>
> To use LW-SSO, your Service Manager web tier and server must be deployed in the same domain; therefore you should use the same domain name for the web tier and server. If you fail to do so, users who log in from another application (for example, Enterprise Collaboration) to the web tier can log in but may be forcibly logged out after a while.

3. Set the `initString` value. This value MUST be the same with the LW-SSO setting of the other OpenText product you want to integrate with Service Manager.
4. Check the `secureHTTPCookie` value (default: true).

> **Note**
>
> - If you set `secureHTTPCookie` to `true` (default), you must also set `secureLogin` in the web tier configuration file (`web.xml`) to `true` (default); if you set `secureHTTPCookie` to `false` , you can set `secureLogin` to either `true` or `false` . In a production environment, you are recommended to set both parameters to `true` .
> - If you don't want to use SSL, set both `secureHTTPCookie` and `secureLogin` to `false` .

5. Save the `lwssofmconf.xml` file.

4. Open the `<Tomcat>\webapps\<Service Manager Web tier>\WEB-INF\classes\application-context.xml` in a text editor.
5. Modify the `application-context.xml` file as follows:
   1. Add `lwSsoFilter` to the `filterChainProxy` bean:

      ```
      <sec:filter-chain pattern="/**" filters="securityContextPersistenceFilter,resourceFilter,lwSsoFilter,anonymousAuthFilter"/>
      ```

      If you need to enable web tier LW-SSO for integrations and also enable trusted sign-on for your web client users, add **lwSsoFilter** followed by **preAuthenticationFilter**, as shown in the following:

      ```
      <sec:filter-chain pattern="/**" filters="securityContextPersistenceFilter,resourceFilter,lwSsoFilter,preAuthenticationFilter,anonymousAuthFilter"/>
      ```

      For information about how to enable trusted sign-on in Service Manager, see Example: Enabling trusted sign-on.

   2. Uncomment the `lwSsoFilter` bean by removing the comment tags:

```
  <bean id="lwSsoFilter"
class="com.hp.ov.sm.client.webtier.lwsso.LwSsoPreAuthentication
Filter">
<property name="authenticationManager">
<ref bean="authenticationManager"/>
</property>
<property name="defaultRole">
<value>ROLE_PRE</value>
</property>
</bean>
```

3. Save the `application-context.xml` file.
6. Restart Tomcat so that the configuration takes effect.

Related topics

Using LW-SSO with integrations
Configure LW-SSO on the Service Manager server

1.1.6.6.3. Configure LW-SSO in Application Performance Management (APM)

If LW-SSO is enabled in both Service Manager and Application Performance Management (APM), users who have signed in to Service Manager are allowed to sign on to APM through the web tier without providing a user name and password.

To configure LW-SSO in APM:

1. Sign in to APM as a system administrator.
2. Click **Admin** > **Platform** > **Users and Permissions** > **Authentication Management**.
3. Check to see if the following two fields are correctly configured:
   - Token Creation Key (initString): Must be the same as the **initString** value specified in the Service Manager LW-SSO configuration (minimum length: 12 characters). For example, **smintegrationlwsso**.
   - Trusted Hosts/Domains: Must contain the domain name of the Service Manager Web tier server. For example, if your Service Manager Web tier's fully qualified domain name is mywebtier.domain.microfocus.com, then the domain name is **domain.microfocus.com**.

   If these two fields are correctly configured, LW-SSO is already enabled in your APM environment, and you can ignore the steps below. If not, proceed to the steps below.
4. Click **Configure**. The Authentication Management Wizard opens.
5. Click **Next**. The Single Sign-On Configuration page opens.
6. Do the following:
   - In the **Token Creation Key (initString)** field, enter a string of characters. For example, **smintegrationlwsso**.

     Note

     This value must be the same as the initString value in your Service Manager LW-SSO configuration.
   - In the **Trusted Hosts/Domains** column, add the domain name of the Service Manager Web tier server.
   - In the **Type** column, select **DNS** for the Service Manager Web tier server.
7. Click **Next** twice, and then click **Finish**.

   LW-SSO is now enabled in your APM environment.

Note

Note: For other settings not described above, keep the defaults. If you want to change these settings, click **Help** on the Single Sign-On Configuration wizard pages.

Related topics

Application Performance Management (APM)
APM Business Impact Report (BIR)
Incident Exchange (OMi - SM) integration
Using LW-SSO with integrations
APM Business Impact Report integration setup
Incident Exchange (OMi - SM) integration setup
Enable LW-SSO for the Business Impact Report (BIR) integration
Enable LW-SSO for the Incident Exchange (OMi - SM) integration

1.1.6.6.4. Configure LW-SSO in Operations Orchestration (OO)

Applies to User roles: System Administrator

If you have enabled Lightweight Single Sign-On (LW-SSO) in both Service Manager and Operations Orchestration, users who have logged on to Service Manager are allowed to sign on to Operations Orchestration through the web tier without providing a username and password.

> **Note**
>
> LW-SSO requires that the accounts used to log on to Operations Orchestration and Service Manager have the same account name (but can have different passwords).

To configure LW-SSO in Operations Orchestration, refer to Set up security using LW-SSO.

Related topics

- Integration Manager
- Operations Orchestration (OO)
- Using LW-SSO with integrations
- Operations Orchestration integration setup
- Add an Operations Orchestration integration
- Enable SSL connection from Service Manager to Operations Orchestration
- Enable LW-SSO for the Operations Orchestration integration

1.1.7. BDM Mapping Management

Applies to User Roles: System Administrator

The BTO Data Model (BDM) is intended to be used as a standard data model for integrations between Service Manager and Application Performance Management (APM). In Service Manager, a BDM mapping is a mapping between a Service Manager object (associated with a file in Service Manager) and a BDM object (predefined in BDM). A BDM mapping consists of the following parts: field mapping, value mapping, atom mapping, and configuration.



Note

Service Manager supports data mappings of the `incident` entity, which are compliant with three versions of BDM: 1.0, 1.1, and 1.2, respectively.

System administrators can create new BDM mapping configurations or customize existing BDM mapping configurations in Service Manager.

To configure a BDM mapping in Service Manager:

1. Go to **System Administration** > **Ongoing Maintenance** > **BDM Mapping Management**. The BDM mapping configuration search page opens.
2. Search for an existing BDM mapping configuration. For example, enter **incident** in the BDM Name field, select **1.2** in the Version field, and then click **Search**. The mapping configuration of BDM object **incident** is displayed.

   Or

   Add a new BDM mapping configuration. Follow these steps:

   a. In the BDM Name field, enter the name of a BDM object.

   b. In the Version field, select a version of BDM from the list.

   c. In the File field, click the Fill icon and then select the name of a file from the resulting list. This is the Service Manager file that you want to map to BDM.

   d. Click **Add**. The new BDM mapping record is added.
3. Configure the mapping information on the **Field Mapping**, **Value Mapping**, **Atom Mapping**, and **Configuration** tabs. For a detailed description of the information on each tab, see the related topics.

> **Note**
>
> On the Field Mapping tab, if you add a new field to the file associated with the Service Manager object, you must click the Fill icon for the File field to refresh the field list in the SM Object Field column.

4. (Optional) Click **Publish** to publish the BDM mapping. Once published, a BDM mapping record becomes read-only.

Related topics

Field Mapping
Value Mapping
Atom Mapping
Customizing BDM mapping configuration

1.1.7.1. Field Mapping

The **Field Mapping** tab of a BDM mapping defines the field mappings between a Service Manager object and a BDM object. This section describes how to configure the columns on this tab, and provides detailed information about the inner objects, functions, and variables that can be used to configure callbacks in the SM Callback and BDM Callback columns.

Field Mapping columns

The following table describes the columns on the **Field Mapping** tab.

Column Name	Description
SM Object Field	The name of a field in the Service Manager file, and can be selected from the list. **Note** Do not select the first item such as **[probsummary]** in the list, which references the name of the file that you want to map to BDM. The BDM mapping utility doesn't support complex structures at this time (for example, an array of arrays or an array within an array of structures).
Direction	Indicates the mapping direction of a field mapping, and has only three values to select from. • →: from SM Object Field to BDM Object Field/Attribute. • ←: from BDM Object Field/Attribute to SM Object Field. • ↔: in both directions.

Column Name	Description

BDM Object Field/Attribute	The name of a field in the BDM object, and should be in a format like XPath of XML.
	
	For example, the following XML segment:
	
	`<is_registered_for target_role='xxx'><target_global_id>xxx</target_global_id><target_type>xxx</target_type><configuration_item><id>xxx</id><type>xxx</type><description>xxx</description></configuration_item></is_registered_for>`
	
	should be formatted as:
	
	`is_registered_for/@target_role (@ means it is an attribute)`
	`is_registered_for/ target_global_id`
	`is_registered_for/ target_type`
	`is_registered_for/ configuration_item/ id`
	`is_registered_for/ configuration_item/ type`
	`is_registered_for/ configuration_item/ description`
	
	• You can use an array to represent repetitive elements. For example:

Column Name	Description
	<pre><users> <user> <name>xxx</name> <email>xxx</email> </user> <user> <name>xxx</name> <email>xxx</email> </user> ... </users></pre> Can be described as: <pre>users/user[0]/name users/user[0]/email users/user[1]/name users/user[1]/email</pre> • The namespace of a BDM field should be formatted as: <pre>ns:property1/ns:subproperty2</pre> **ns** is the alias of a namespace, and should be included in the **BDM Callback** of the **@xmlns** mapping entry: <pre>$xmlns({..., "ns":"http://www.xxx.com/xxx/xxx", ...})</pre> If the alias is omitted, then a default namespace configured in @xmlns will be used: <pre>'\$':"http://www.xxx.com/xxx/xxx"</pre>

Column Name	Description
SM Default Value	Defines the default value of the SM Object Field if there is no value in the corresponding BDM Object Field/Attribute when mapping the BDM object to the Service Manager object.
SM Callback	Enter custom JavaScript code, which will be invoked when mapping BDM Object Field/Attribute to SM Object Field. For information about how to configure callbacks, see the **Callbacks** section below. **Note** If no callback is configured, the BDM Object Field/Attribute will be directly mapped to the SM Object Field.
BDM Default Value	Defines the default value of the BDM Object Field/Attribute if there is no value in the corresponding SM Object Field when mapping the Service Manager object to the BDM object.
BDM Callback	Enter custom JavaScript code for mapping between the BDM Object Field/Attribute and the SM Object Field. When mapping the Service Manager object to the BDM object, the custom JavaScript code will be invoked. For more information about how to configure callbacks, see the **Callbacks** section below.
BDM Type	Select one from the list: String (default), Number, and DateTime. If no BDM Type is selected, the default type (String) will be used.

Column Name	Description
Value Mapping Group	The name of a value mapping group to which the BDM Object Field/Attribute and SM Object Field belong. A value mapping group contains a collection of enumerated BDM/SM value pairs. The Service Manager value and BDM value will be mapped according to the configuration on the **Value Mapping** tab.
Description	Enter a text description for this BDM mapping.

Callbacks

In the **SM Callback** and **BDM Callback** columns, you can enter custom JavaScript code for mapping between the Service Manager object and BDM object. Some inner objects, variables, and functions can be used in the custom JavaScript code. The key word **$** is reserved to identify these inner objects, variables, and functions.

Reserved key word ($)

The following rules apply when you use the reserved key word: **$**.

- If you want to use your own variable prefixed with '$', you must escape it with the character '\'. For example:

  ```
  var \$test = 'abc';
  ```

- You can also use inner variables and objects in a string. For example:

  ```
  var strTemp = "Incident Number: $sm.number";
  ```

  If you want to use a '$' in a string, you must escape it. For example:

  ```
  cost: 100 \$ ...
  ```

- In a regular expression, you don't need to escape '$' with '\':

  ```
  var regExpr = /^[Jj]ava[Ss]cript$/
  ```

  However, if you define with the RegExp object, you must escape '$', because '$' is in a string:

```
var regExpr = new RegExp( "^[Jj]ava[Ss]cript\$" );
```

Inner objects, functions, and variables

The following tables list the inner objects, functions, and variables.

Inner Objects

Object Name	Member Method Signature / Method Description

$sm
 	
The current Service Manager object.
	value(field)
	
	field: The name of a field of the Service Manager object. If you don't include the <field> parameter, the currently configured field is used. Subfields should be separated with a forward slash ('/').
	
	This function is used to extract the value of a field of the Service Manager object. If you don't include the <field> parameter, it will return the current field value. For example:
	
	`$sm.value( "id" )`
	
	`$sm.value( "resolution[0]" )`
	
	`$sm.value()`

$sm The current Service Manager object.	setValue(value, field) **value:** The value of the <field> to be set. **field:** The name of a field of the Service Manager object. If you don't include the <field> parameter, the currently configured field will be used. This function is used to set the value of a field of the Service Manager object. If you don't include the <field> parameter, it will return the current field value.

Object Name	Member Method Signature / Method Description

$bdm
 	
The current BDM object.
	value(field)
	
	field: The name of a field of the BDM object. If you don't include the <field> parameter, the currently configured field will be used. Subfields should be separated with a forward slash ("/").
	
	This function is used to extract the value of a field of the BDM object. If you don't include the <field> parameter, it will return the current field value. For example:
	
	`$bdm.value( "is_registered_for/configuration_item" )`
	
	`$bdm.value( "ns:users/ns:user[0]/name" )`
	
	`$bdm.value()`

$bdm The current BDM object.	setValue(value, field) **value:** The value to be set for the <field>. **field:** The name of a field of the BDM object. If you don't include the <field> parameter, the currently configured field will be used. **Note**: The field value must be the value in the BDM Object Field/Attribute column. This function is used to set the value of a field of the BDM object.

Object Name	Member Method Signature / Method Description
$ctx The current mapping context object that is used to hold some variables to be used in the whole mapping process.	set(key, value) **key:** The key to be set for the <value>. **value:** The value to be kept in the mapping context object for the key. This function is used to set a key-value pair in a mapping context object. For example: `$ctx.set("affected_ci", affected_ci)`
$ctx The current mapping context object that is used to hold some variables to be used in the whole mapping process.	get(key) **key:** The key in a key-value pair. This function is used to get the value of a key in a mapping context object. For example: `var affected_ci = $ctx.get("affected_ci")`

Inner functions

Function Signature	Description

$xmlns()	Used to define an xml namespace.
	
	Example:
	
	`$xmlns({"\$":"http://www.hp.com/2009/software/data_model",`
	
	`"bdm":" http://www.hp.com/2009/software/data_model"}).`
	
	In this example, **"\$":"http://www.hp.com/2009/software/data_model"** is a default namespace, where reserved key word **$** is escaped with a backslash ("\"); **"bdm":" http://www.hp.com/2009/software/data_model"** is another namespace, where **bdm** is the alias of the namespace. You can add more namespaces, but there is only one default namespace.

$afterString(str) **str:** A portion of the current mapping field value.	Used to return the portion of the current mapping field value that's after the string specified by the <str> parameter. Example: `$afterString("urn:x-hp:software:servicemanager:incident:id:")` If the current mapping field is 'global_id' of the BDM object, and its value is **urn:x-hp:software:servicemanager:incident:id:7B21A623-158B-45D4-9C7C-3D2821096A44**, the $afterString will return: **7B21A623-158B-45D4-9C7C-3D2821096A44**.

Function Signature	Description
$prefix(str) **str:** The string value to be prefixed before the current mapping field value.	Used to prefix a string before the current mapping field value. Example: If the current mapping field is 'id' of the Service Manager object and its value is "5A8EABC6-CA1B-499B-A230-4D6DE07487D8", `$prefix("urn:x-hp:software:servicemanager:incident:id:")` will return **urn:x-hp:software:servicemanager:incident:id:5A8EABC6-CA1B-499B-A230-4D6DE07487D8**.
$select(fileName, query) • **filename:** The name of a valid Service Manager file. • **query:** The Service Manager query language.	Used to do select the SM object from the SM DB. Example: `var requestedBy = $select("contacts",'operator.id="falcon"');`
$combine(delimiter) **delimiter:** A symbol or character used to separate the combined values.	Used to combine the current field values of the Service Manager/BDM object into one string.
combine(arr, delimiter) • **arr:** An array of values to be combined. • **delimiter:** A symbol or character used to separate the combined values.	Used to combine the specified array of values into one string. Example: `combine( ['a','b','c'], ',')`

Function Signature	Description
$isCreate()	Used to verify if the current operation has created an SM object in Service Manager (true) or not (false).
$isUpdate()	Used to verify if the current operation has updated an SM object in Service Manager (true) or not (false).

Inner variables

Variable Name	Description
$result	This variable is used to hold the returned result that will be assigned to the current target field of the Service Manager/BDM object if the custom JavaScript code can't generate a value (for example, the 'if' statement can't generate a value, and thus needs to use the $result variable to hold the result): `var affected_ci = $ctx.get("affected_ci"); if( affected_ci ) { $result = affected_ci['type']; }`

Related topics

BDM Mapping Management
Value Mapping
Atom Mapping
Customizing BDM mapping configuration

1.1.7.2. Value Mapping

The **Value Mapping** tab of a BDM mapping defines the value mappings between a Service Manager object and a BDM object.

The following describes the columns on this tab.

Column Name	Description
Value Mapping Group	The name of a value mapping group. A value mapping group contains a collection of enumerated BDM/SM value pairs.
BDM Values	The values of the BDM fields.
SM Values	The values of the Service Manager fields.
Description	Description of the value mapping.

For example, below is a value mapping named **impact**, and there are four mapping entries for this value mapping group:

Value Mapping Group	BDM Values	SM Values
impact	enterprise	1
impact	site-dept	2
impact	multiple-users	3
impact	users	4

The name **impact** can be filled in the **Value Mapping Group** column on the **Field Mapping** tab:

SM Object Field	Direction	BDM Object Field/ Attribute	SM Default Value	SM Callback	BDM Default Value	BDM Callback	BDM Type	Value Mapping Group
initial.impact	↔	impact_scope						impact

This means when mapping the Service Manager object to the BDM object, if the value of field `initial.impact` of the Service Manager object is 2, the value `2` will be mapped to `site-dept`

for the field `impact_scope` of the BDM object, and vice versa.

Related topics

BDM Mapping Management
Field Mapping
Atom Mapping
Customizing BDM mapping configuration

1.1.7.3. Atom Mapping

The **Atom Mapping** tab of a BDM mapping defines the required mappings between certain fields in a BDM object and Atom (an XML-based document format for web feeds). The Service Manager RESTful web service supports the standard Atom representation, which conforms to the Atom Publishing Protocol. According to the Atom Publishing Protocol, the following elements are required in an Atom 1.0 feed: **title**, **id**, and **updated**.

The following table describes the columns on the **Atom Mapping** tab.

Column Name	Description
BDM Object Field	The name of a valid field in the BDM object
Atom Object Field	The name of a valid Atom field.
Atom Object Field Type	The type of a valid Atom field.

Below is an example of an Atom mapping:

BDM Object Field	Atom Object Field	Atom Object Field Type
reference_number	title	String
last_modified_time	updated	DateTime
description	summary	String
global_id	id	String

In the resulting Atom xml representation below, the **id**, **updated**, **title** and **summary** elements will take the values of the BDM object fields **global_id**, **last_modified_time**, **reference_number** and **description**, respectively.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<entry xmlns="http://www.w3.org/2005/Atom"
xmlns:ns2="http://a9.com/-/spec/opensearch/1.1/"
xmlns:ns3="http://www.w3.org/1999/l">
<id>urn:x-hp:software:servicemanager:incident:id:7B21A623-158B-45D4-
9C7C-3D2821096A44</id>
<updated>2022-01-27T11:41:25.000+08:00</updated>
<title type="text">IM10008</title>
<summary type="text">Desktop DVD-drive makes strange noices</summary>
```

Related topics

BDM Mapping Management
Field Mapping
Value Mapping

1.1.7.4. Customizing BDM mapping configuration

System Administrators can define key-value pairs that can be shared by SM callbacks and BDM callbacks in BDM mapping records. With this functionality, administrators only need to update these key values in one place instead of in individual callbacks.

To customize BDM mapping configuration:

1. Sign in to the Windows client as a System Administrator.
2. Click **System Administration** > **Ongoing Maintenance** > **BDM Mapping Management**.
3. In the Version field, select a value. For example, **1.2**.
4. Click **Search**. The mapping record `incident` is displayed.
5. Select the **Configuration** tab. A two-column table is displayed. This table defines key-value pairs that can be used in BDM mapping callbacks.
   - **Name**: Defines the name of a key to be used in callbacks.
   - **Value**: Defines the value of the key.

   Out-of-box, there are three keys: Host, Port, and AppName.
6. Type key-value pairs in the table. For example, you can type values for the following out-of-box keys:
   - Host: **abc.domain.com**
   - Port: **8080**
   - AppName: **webtier-9.30**
7. To view an out-of-box example of using these keys:
   1. Select the **Field Mapping** tab, and locate the SM Object Field named **affected.item**.
   2. In this field section, locate the row in which the BDM Object Field/Attribute value is: **affects/configuration_item/smns:drilldown_url**
   3. Click the BDM Callback in the same row, right-click it and then select **Magnify**.

      The callback code displays in a pop-up text window.
   4. Find the following strings, which contain the three out-of-box key names.

      ```
      - $ctx.getMappingConfiguration()['Host']
      - $ctx.getMappingConfiguration()['Port']
      - $ctx.getMappingConfiguration()['AppName'])
      ```
8. To use a defined key in a callback:
   1. Select the **Field Mapping** tab, click a callback in the SM Callback or BDM Callback column. Right-click the callback and select **Magnify**.

The callback code displays in a pop-up text window.

2. Edit the callback code using the following syntax:

   `$ctx.getMappingConfiguration()['Host']`, where Host should be replaced with your key name.

3. Click **Save and Close**.

9. To test the values you have defined for the out-of-box keys:

   1. Send a REST request by launching this URL in a web browser: `http://<SM_serverHost>:<Port>/SM/7/rest/1.2/incident_list/reference_number/IM10002`.

      A login window is displayed.

   2. Type a valid Service Manager username/password (for example, System.Admin/blank password), and click **OK**.

      The REST request response is displayed.

   3. Locate this string: `<smns:drilldown_url>http://<hostname>:<port>/<AppName>/index.do`, where `<hostname>`, `<port>`, and `<AppName>` should be replaced with the values of Host, Port, and AppName, respectively.

Related topics

BDM Mapping Management
Field Mapping
Value Mapping
Atom Mapping
BDM RESTful API

1.1.7.5. BDM RESTful API

If you confiure the BDM Mapping, you can use below URL to view the BMD RESTful requests, URL examples like below:

Service Manager Incident (BDM 1.0)

Service Manager Incident Attachment List (BDM 1.0):	http://<server>:<port>/SM/7/rest/1.0/incident-list/reference-number/{incidentKey}/attachment-list
Service Manager Incident Attachment List (BDM 1.0):	http://<server>:<port>/SM/7/rest/1.0/incident-list/{incidentKey}/attachment-list
Service Manager Incident List (BDM 1.0):	http://<server>:<port>/SM/7/rest/1.0/incident-list
Service Manager Incident List (BDM 1.0):	http://<server>:<port>/SM/7/rest/1.0/incident-list/reference-number

Service Manager Incident (BDM 1.1)

Service Manager Incident Attachment List (BDM 1.1):	http://<server>:<port>/SM/7/rest/1.1/incident-list/reference-number/{incidentKey}/attachment-list
Service Manager Incident Attachment List (BDM 1.1):	http://<server>:<port>/SM/7/rest/1.1/incident-list/{incidentKey}/attachment-list
Service Manager Incident List (BDM 1.1):	http://<server>:<port>/SM/7/rest/1.1/incident-list
Service Manager Incident List (BDM 1.1):	http://<server>:<port>/SM/7/rest/1.1/incident-list/reference-number

Service Manager Incident (BDM 1.2)

Service Manager Incident Attachment List (BDM 1.2):	http://<server>:<port>/SM/7/rest/1.2/incident-list/reference-number/{incidentKey}/attachment-list
Service Manager Incident Attachment List (BDM 1.2):	http://<server>:<port>/SM/7/rest/1.2/incident-list/{incidentKey}/attachment-list
Service Manager Incident List (BDM 1.2):	http://<server>:<port>/SM/7/rest/1.2/incident-list
Service Manager Incident List (BDM 1.2):	http://<server>:<port>/SM/7/rest/1.2/incident-list/reference-number

1.1.8. Case Exchange framework

Service Manager Case Exchange is a solution to exchange data between two Service Manager systems or between Service Manager and Service Management Automation X (SMAX). The Case Exchange framework mainly facilitates the following operations:

- Sending and receiving data
- Viewing and processing the exchanged data in the native environment

To leverage Case Exchange capabilities, the systems to be connected must support REST-based Web Services in JSON format. SOAP-based Web Services aren't supported. The following chapters describe the functionalities of Case Exchange and their expected behaviors.

It is highly recommended that the personnel who implement Case Exchange with another system have the experience in the following Service Manager technologies:

- REST-based Web Services
- DBDICT
- Process Designer framework
- JavaScript
- JSON
- Service Manager Integration Suite (SMIS)

During the implementation, you can refer to the following documentation:

- Web Services
- Programming
- Integration Manager and SMIS
- Process Designer Framework



Note

The scope of support is limited to the behavior of APIs and product features documented. OpenText is responsible for the correct operation of those features and APIs. OpenText is not responsible for debugging scripts or code created to utilize these features to implement a solution. In order to receive the best possible support, OpenText asks customers to provide a reproducible unit test scenario that demonstrates the error or unexpected behavior of the API or documented feature.

Tip

Consider the following tips when you troubleshoot the Case Exchange integration:

- Incorrect configuration of the **Base URL**, **User Name**, and **Password** fields in the **General** tab usually causes authentication failure. In this scenario, 401 errors are recorded in the log file.
- You can enable the error-handling feature when you configure the integration instance. SMIS can automatically create a new Incident record if a Case Exchange task fails. The **Description** field of this Incident record contains the details of the failed task record. For more information about the error-handling feature, see Error handling.
- Due to a known limitation, the error message in the SMIS task log may be insufficient to identify the root cause of a failed task. For more information about this known limitation and the workaround, see Insufficient message for a failed Case Exchange task.

1.1.8.1. Case Exchange framework features

This section introduces the Case Exchange features.

Connector

Case Exchange is enabled by connectors, which can open, update, and close records in the system of another provider. Connectors can also perform the following tasks:

- Listen for events or updates related to record exchange.
- Take care of the physical communication to the system via REST-based web services in JSON format.
- Manage appropriate authentication and identity credentials required by the connected system.

Field mapping and value mapping

Field mapping and value mapping determine how the record data is transformed into the internal or external normalized case format.

Field mapping ensures that the data from one system is send to the correct field on the second system. To avoid interface failures the field names and the field name description should *not* contain any special characters.

Value mapping ensures the data is correctly transformed according to the rules required in each system. The value mapping supports expressions and calculations to ensure the data is correctly manipulated prior to sending or updating records in the local database.

For more information about field mapping and value mapping, see topics under the Integration Instance Mapping section.

Outbound trigger rules

One of the core functions of Case Exchange is automated data exchange between Service Manager and integrated systems. For example, Case Exchange can automate the following activities:

- Opening records
- Exchanging record updates between Service Manager and integrated systems
- Tracking the status of related records in Service Manager and in integrated systems

Service Manager has a dedicated rule type in Rule Sets that can trigger Case Exchange between Service Manager and integrated systems. Use one of the following methods to invoke a Rule Set:

- If Process Designer is implemented in Service Manager, invoke the Rule Set from workflows.

- If Process Designer is not implemented in Service Manager, you must create a trigger and add the following API in the trigger's script:

  ```
  lib.CaseExchange_RuleExecute.executeSingleRuleSet(record,oldrecord,RuleSetID)
  ```

For more information about how to invoke a Rule Set, see the Invoke Case Exchange Rule Sets section.

Attachment handling

You can configure Case Exchange to transfer attachments in exchanging records from one system to the other system. When the attachment handling functionality is enabled, Case Exchange handles attachments differently in different integration mechanisms.

Service Manager and Service Management Automation X (SMAX) integration

When a Service Manager (SM) system is integrated with SMAX, the Pull mechanism is used. If the attachment handling functionality is enabled in the SM system, Case Exchange handles the attachments as follows:

- Attachments added in SM are transferred to the corresponding records in SMAX.
- SM retrieves attachments from SMAX when it handles the next Case Exchange task that's initialized by SMAX.

For more information about the Pull mechanism for SM/SMAX integration, see The Pull mechanism.

Service Manager and Service Manager integration

When an SM system is integrated with another SM system, either the Pull mechanism or the Push mechanism is used.

When the Pull mechanism is used, Case Exchange works in the same way as SM/SMAX integration. That is, the SM system that has the attachment handling functionality enabled transfers attachments to the other SM system and retrieves attachments from that system when it handles the next Case Exchange task.

However, when the Push mechanism is used, the system that has the attachment handling functionality enabled only transfers attachments to the other system. It doesn't retrieve attachments from that system. When the functionality is enabled in both systems, attachments are transferred in both directions.

For more information about the Pull mechanism for SM/SM integration, see The Pull mechanism. For more information about the Push mechanism in this case, see The Push mechanism.

Additional considerations for attachment handling

In addition to the attachment handling functionality described for different SM/SMAX or SM/SM integration scenarios, you also need to note the following when you use the functionality:

- No attachments are transferred if the attachment handling functionality is disabled in both systems.
- Even if the attachment handling functionality is enabled, Case Exchange doesn't remove attachments from one system when you remove corresponding attachments from the other system.
- A system administrator can specify the type and maximum size of the attachment files that Case Exchange can transfer. In case the file type of an attachment doesn't match the listed file types or in case the file size of attachment exceeds the configured limit, Service Manager doesn't transfer the attachment.

For more information about how to configure the attachment handling feature, see Integration Instance Parameters - Attachment Handling.

Audit and logging

The Case Exchange framework provides different portals for administrators and users to view the Case Exchange activity log.

The Case Exchange framework tracks and records all transactions of exchanged request in the Service Manager Integration Suite (SMIS) task log. Administrators can access this log from SMIS. For more information, see Monitor SMIS task log.

When a Case Exchange integration instance is active and a Case Exchange task has occurred to an Incident record, users can view the **Case Exchange** section in the Incident record for the details of the Case Exchange information. The **Case Exchange** section contains the following information:

Details

Field name	Description
External Id	The ID of the related record in the integrated system.
External Status	The status of the related record in the integrated system.
Creation Time	The time when the Incident record is created.
Active	This field indicates if Case Exchange is active for this record. Possible values of this field are `true` and `false`: `true`: The update in one of the systems can be exchanged to the other system. `false`: The update in one of the systems *can't* be exchanged to the other system.
Originator System	The name of the integrated system where the record come from.
Integration Name	The name of the integration instance that transfers the record from the integrated system.

Log

Field name	Description
Date	The date and time when the task occurs. The link on the date and time can direct you to the **Task Log Details** page, which displays the detailed information of that Case Exchange task.
Status	The status of the task.
Message	A short description of the task.
Integration Name	The name of the integration instance that carries out the task.

The task log gives a first indication about potential issues of an error but doesn't contain detailed error messages, you can find the detailed information in the standard Service Manager log file (sm.log).

Error handling

Service Manager Integration Suite (SMIS) can automatically create a new Incident record for the error if all of the following conditions are met:

- The error handling feature is active in a Case Exchange integration instance.
- A Case Exchange task of this integration encounters an error.
- SMIS has performed the number of retries that's pre-defined, but still encounters error.

SMIS automatically sets the following fields in the record according to the configuration in the integration instance:

- Title
- Assignment Group
- Impact
- Urgency
- Category
- Area
- Subarea
- Affected Service

The error-handling feature works in the following manner:

- Case Exchange only creates one Incident record for each integration instance that has failed tasks.
- When the task fails for the first time, SMIS creates a new Incident record.
- When the task that has failed fails again, SMIS works in the following manner:
  - If the record for the previous failure is closed, SMIS creates a new Incident record.
  - If the record for the previous failure is not closed, SMIS doesn't create a new Incident record.

Ownership

At any time throughout the life cycle of a record in Case Exchange, only one system owns the record. The owner of the record has the responsibility to resolve the record or transfer the record to another organization if necessary.

The ownership of records in Case Exchange relies on process definition rather than tooling. It is highly recommended to define clear rules of ownership for each step in a case exchange environment prior to taking an interface live. These rules of ownership are required to avoid any potential data loss in case both parties update a record at the same time. OpenText is not responsible for issues during the case exchange in a situation where both parties are able to update a record at the same time.

See Recommendations for other suggestions for implementing Case Exchange.

1.1.8.2. Enable Case Exchange with another system

This section introduces the tasks that you need to perform to set up a Case Exchange integration between Service Manager and another system:

General procedure

Recommendations

Configure Incident environment

Create a new integration template

Add and enable a Case Exchange integration instance

Configure Case Exchange Rule Sets

Invoke Case Exchange Rule Sets

Enable Case Exchange with SMAX

The Pull mechanism

Add and enable an integration instance

Create and invoke Rule Sets

Specify Service Manager as an external system in SMAX

Test and troubleshoot the integration

Enable Case Exchange between two Service Manager systems

Connection mechanisms

Using the Pull mechanism

Using the Push mechanism

1.1.8.2.1. General procedure

The following topics describe the general procedure of enabling Case Exchange with another system.

Recommendations

Configure Incident environment

Create a new integration template

Add and enable a Case Exchange integration instance

Configure Case Exchange Rule Sets

Invoke Case Exchange Rule Sets

1.1.8.2.1.1. Recommendations

Before you set up the Case Exchange integration, consider the following recommendations for preparation and configuration:

- Decide about the general connection method, which can be either of the following:
    - Pull the data from the target system
    - The target system pushes the data into Service Manager
- Ensure that you have a clear picture about the workflows in the systems you want to integrate.
- Ensure that you know how to map the workflows for incoming and outgoing data.
- Ensure that the attachment settings are correctly configured.
- Ensure security settings are defined and correctly configured.
- Enable outbound events for easy usage in the Rule Sets.
- Test the configurations in a test environment before going live.
- Ensure the ownership of the records is clearly defined for each stage.
- Align on reopen, resolve, and closure rules, follow Processes and Best Practices .
- Perform enough performance tests, including test with attachments to ensure the system can handle the case exchange requirements of your environment.
- Don't underestimate the effort for the workflow and status mapping.

1.1.8.2.1.2. Configure Incident environment

We recommend that you enable the **Use Resolved Status** setting in the Service Manager system.

To do this, follow these steps:

1. Go to **Incident Management** > **Administration** > **Environment**.
2. Select **Use Resolved Status?**.
3. Click **Save**.
4. Click **OK**.

1.1.8.2.1.3. Create a new integration template

Applies to User roles: System Administrator

An appropriate integration template must be ready before you integrate Service Manager with another system. You can either create a new template, or use and modify an existing template during integration.

To create a new integration template, refer to Developing an Integration Template.

Service Manager provides out-of-box templates for Case Exchange integrations, for more information see Out-of-box integration templates.

1.1.8.2.1.4. Add and enable a Case Exchange integration instance

Applies to User roles: System Administrator

To add a Case Exchange integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Click **Add**. The **Integration Template Selection** wizard opens.
3. Select a Case Exchange template from the **Integration Template** list. For example, select **CaseExchangeDefaultTemplate**.
4. If you want to use the field mappings provided with the template, select **Import Mapping**.

   > **Note**
   >
   > A template becomes unavailable when the number of integration instances based on this template reaches the maximum value predefined in the **Instance Count** filed in the template. For more information about how to add or edit an integration template, see Service Manager Integration Suite (SMIS).

5. Click **Next**. The **Integration Instance Information** page opens. Verify or complete the fields as necessary. For the description of each field on this page, see the topics under Configuration details.
6. Click **Next**. The **Integration Instance Parameters** page opens. This page contains all predefined parameters for the integration.

   > **Note**
   >
   > You can modify the predefined parameter values as necessary. However, it's not recommended to add new parameters, modify parameter names or categories.

7. Click **Next**. The **Integration Instance Fields** page opens.
8. Enter the Service Manager fields and the endpoint fields in the following two tabs:

   • **SM Fields**: Enter the fields of the Service Manager system side to map to endpoint fields.
   • **Endpoint Fields**: Enter the fields of the endpoint system to map to Service Manager server fields.

9. Click **Next**. The **Integration Instance Mapping** page opens.

This page contains all field mappings and value mappings between Service Manager server fields and endpoint fields. For more information about how to configure this page, see topics under Integration Instance Mapping.

10. Click **Finish** to add the new integration instance.

For more information about how to enable an integration instance, see Enable or disable an integration instance.

Related topics

- Integration Manager
- Service Manager Integration Suite (SMIS)
- Integration Instance Mapping
- Integration Instance Information fields
- Edit an integration instance
- Add or delete field mappings
- Edit field mappings
- Configure value mappings
- Enable or disable an integration instance
- Monitor integration instance status
- Monitor failover tasks
- Configure a callback

1.1.8.2.1.5. Configuration details

This section provides detailed explanation to each field you need to configure when you set up the Case Exchange integration by using the out-of-box template: **CaseExchangeDefaultTemplate**.

Note

Currently, Case Exchange only supports the data transaction through RESTful APIs.

Integration Instance Information

For details about how to configure the **Integration Instance Information** page, see Integration Instance Information fields.

Integration Instance Parameters - General

The following table explains the fields under the **General** tab in the **Integration Instance Parameters** page.

Fields	Value	Description
Object	User-defined. For example, `Incident`.	The table name of the object in Service Manager.
Timeout	User-defined. For example: `30`.	This value specifies the time-out (in seconds) for each of these three activities: Connect, Send, and Receive.
Base URL	User-defined.	The base portion of the endpoint URL. The format of the URL is as follows: http(s)://*<host>.<domain>* For example, the Base URL for the Service Anywhere endpoint might be as follows: `https://<host name>.saas.hpe.com`
Login URL	User-defined.	The login URL of the endpoint. The format of the URL is as follows: `<Base URL>/auth/authentication-endpoint/authenticate/login`
Tenant Id	User-defined.	The Tenant Id of the endpoint system.
User name	User-defined.	The endpoint user name for the integration.

Fields	Value	Description
Password	User-defined.	The password of the user name. **Note** This field can't be blank. The endpoint user name must have a password. Otherwise, the connection verification may fail.

Integration Instance Parameters - Inbound

The following tables explain the fields under the **Inbound** tab on the **Integration Instance Parameters** page.

Fields	Value	Description
External Id Field	User-defined.	The relative path of the ID field in inbound JSON. This path is relative to the List path. The field name you entered must be a valid reference in the external system. For example, `properties.Id`.
External Status Field	User-defined.	The relative path of the Status field in inbound JSON. This path is relative to the List path. The field name you entered must be a valid reference in the external system. For example, `properties.Status`.
Internal Id Field	User-defined.	The name of the unique field of the Service Manager object. For example, enter **number** for the probsummary table. The field name you entered must be a valid dbdict field name in the selected Service Manager object.

If you select the **Activate Data Pulling** option, Service Manager pulls the data of the inbound tasks from the endpoint system. The following fields appear when you select this option.

Fields	Value	Description
Pulling From	User-defined.	The integration instance pulls all the records that are updated later than the time specified in this field from the end point. If you don't specify a time, the system uses the current time.
Time difference (s)	User-defined. Default: `0`.	The time difference (in seconds) between Service Manager and the endpoint system.
Relative URL	User-defined.	The relative URL specifies the resource to be exchanged in the endpoint system. The string in this field is directly appended after the string you configured in the **Base URL** field under the **General** tab.
Query	User-defined.	The additional query parameters. The string in this field is directly appended after the Relative URL that's configured in the previous field. For more information on how to edit the Query strings, see Edit the query and pagination query strings

Fields	Value	Description
List path	User-defined.	The root path of the list result in the JSON response from the endpoint for a pulling request. For example, the list path is `entities` in the following JSON response: `{ "entities": [ { "entity_type": "Incident", "properties": { "Id": "***", ... } }, ... ] }`
Pagination	Selected/not selected.	When you select this option, the result list from the endpoint is transferred in pages.
Page Size	User-defined.	Specify the maximum number of records in a page.
First Record Offset	User-defined.	This value defines the index of the first record in each page. Usually the value is `0` or `1`.

Fields	Value	Description
Pagination Query	User-defined.	Pagination parameters. The string in this field is directly appended after the query parameters that are configured under the **Inbound** tab. For more information on how to edit the Pagination Query strings, see Edit the query and pagination query strings

If you select the **Activate Data Pushing** option, the endpoint Service Manager system pushes the data of the inbound task to the native Service Manager system. This option only works for the integration between two Service Manager systems. The following fields appears when you select this option.

Fields	Value	Description
Incoming WebService Action	User-defined. For example, `Create` or `Update`.	The action name in the RESTful API of the endpoint Service Manager system. In an out-of-box Service Manager system, the action names are **Create** and **Update**.
SM Action to execute	User-defined. For example, `add` or `save`.	The action to be executed in the native Service Manager system for the corresponding incoming action. Possible values for this field are `add` and `save`.

Note

By default, the integration template uses the Web Service: **CaseExchange**. To view this Web Service, go to the following location in Service Manager and search for "CaseExchange":

Tailoring > **Web Service** > **Web Service Configuration**

Case Exchange only supports the Web Service inbound push call that **Action Type** is **Application Pass Through** and **Customer Action to Perform** is **CaseExchange.Pushing**. Any other type of inbound push features for Case Exchange via SMIS aren't supported.

Integration Instance Parameters - Outbound

The following table explains the fields under the **Outbound** tab in the **Integration Instance Parameters** page.

Fields	Value	Description
Asynchronous Processing	Selected/not selected.	If you select this option, the outbound tasks work in asynchronous mode. If you don't select this option, the creation or update of an object triggers the outbound task immediately.
Event	User-defined.	The name of the outbound action.
Action Type	User-defined.	The action type of this event in the endpoint. The available options for this field are **Create only** and **Update only**.
URL	User-defined.	The relative URL. The string in this field is directly appended after the base URL in the outbound request URL.
HTTP method	User-defined.	Specify the HTTP method.
Entity path	User-defined.	Specify the root path of the list result in the JSON response from the endpoint. For more information about entity path, see Entity path.

Fields	Value	Description
Parameter	User-defined.	Additional parameters. The string in this field is directly appended after the relative URL in the outbound request URL. For example, if you specify the parameter as `?entitytype=incident`, the outbound request URL will be: `<BaseURL><URL>?entitytype=incident` This field is needed if the endpoint only accepts parameters in the URL.
Additional path	User-defined.	Additional paths or elements that are mandatory in outbound JSON but aren't defined in field mapping or entity path. For more information about how to configure this field, see Specify additional path.

Integration Instance Parameters - Error Handling

The following table explains the fields under the **Error Handling** tab in the **Integration Instance Parameters** page.

Fields	Value	Description
Activate incident creation for Errors	Selected/not selected.	Enable or disable the error handling feature, which can automatically create an Incident record for a failed SMIS task.
Title	User-defined. Default: `SMIS Error`.	The title of the Incident records for error handling.
Assignment Group	User-defined.	Specify the Assignment Group of the Incident records for error handling.
Impact / Urgency	User-defined.	Specify the Impact and Urgency of the Incident records for error handling.
Category	User-defined.	Specify the Category of the Incident records for error handling.
Area	User-defined.	Specify the Area of the Incident records for error handling.
Subarea	User-defined.	Specify the Subarea of the Incident records for error handling.
Affected Service	User-defined.	Specify the Affected Service of the Incident records for error handling.

Integration Instance Parameters - Attachment Handling

The following table explains the fields under the **Attachment Handling** tab in the **Integration Instance Parameters** page.

All the parameters in this tab apply to inbound pull and outbound processes.

Fields	Value	Description
Activate attachment exchange	Selected/not selected.	Enable or disable attachment exchange.
Attachment REST URL	User-defined.	The REST API URL for endpoint attachments.
Unsupported File Extensions	User-defined. For example: `exe;rar;iso`.	Specify unsupported file types in the endpoint. Split multiple file types by using semicolons (;).
Timeout (s)	User-defined. Default: `20`.	This value specifies the time-out (in seconds) for each of these three attachment activities: Connect, Send, and Receive.
Size Limitation (kb)	User-defined. Default: `1024`	The endpoint limitation on the attachment size. This value is useful only when an attachment is sent to the endpoint.

Integration Instance Parameters - Additional Script

The following table explains the fields under the **Additional Script** tab on the **Integration Instance Parameters** page.

The Authentication section

Fields	Value	Description
Set the HTTP header	User-defined.	Specify an additional JavaScript function to set the HTTP header. If you don't specify this field, the default HTTP header only provides Basic Authentication and Content-Type.
Login to the external system	User-defined.	Specify additional JavaScript function for login if the endpoint requires special login process. If you don't specify this field, Case Exchange doesn't perform any login process and only sets the authentication information into the HTTP header

The Inbound section

Fields	Value	Description
Validate inbound response	User-defined.	Specify an additional JavaScript function to validate the response body of an inbound request. When the data comes in, this script validates the data before the data is pushed into the SMIS task queue. If you don't specify this field, Case Exchange only validates the response HTTP code and doesn't validate the response body.
Inbound & Push post processing activities	User-defined.	Specify an additional JavaScript function to perform post process for inbound tasks. This script performs post processing after SMIS processes the inbound task. If you don't specify this field, Case Exchange doesn't perform any post process for inbound tasks.

The Outbound section

Fields	Value	Description
Validate & Parse outbound response	User-defined.	Specify an additional JavaScript function to validate and parse the response body of outbound request. The function works in the following way: • If the validation succeeds, the function returns the parsed external ID. • If the validation fails, the function returns "False". If you don't specify this field, Case Exchange only validates the HTTP code and retrieves the ID according to the definitions in the **Inbound** tab.
Outbound post processing activities	User-defined.	Specify an additional JavaScript function to perform post process for outbound tasks. This script performs post processing after SMIS processes the outbound task. If you don't specify this field, Case Exchange doesn't perform any post process for outbound tasks.

The Attachment section

Fields	Value	Description
Set the HTTP header	User-defined.	Specify an additional JavaScript function to set the HTTP header for attachment. If you don't specify this field, the default HTTP header only provides Basic Authentication and Content-Type.
Set header string for the HTTP body	User-defined.	Specify an additional JavaScript function to assemble the header of the attachment request if the attachment exchange uses the multipart/form-data transfer.
Retrieve and parse Attachment Info	User-defined.	Specify an additional JavaScript function to retrieve attachment information, such as attachment name, type, size, and so on.
Parse response of attachment creation	User-defined.	Specify an additional JavaScript function to parse the response body of an outbound attachment request.

Fields	Value	Description
Update outbound Attachment Info	User-defined.	Specify an additional JavaScript function to update the attachment information to the record in the endpoint. The information can link the attachment with the record. You don't need to specify this field if the endpoint system can automatically update the attachment information.

1.1.8.2.1.6. Configure Case Exchange Rule Sets

Before you start to configure a Case Exchange Rule Set, make sure the configuration of the fields mapping in the related integration instance is complete.

The Case Exchange Rule Set is introduced so that customers can easily trigger Case Exchange outbound events.

It is unsupported to trigger Case Exchange activities by using Rule Sets that aren't provided.

For more information about how to configure Case Exchange Rule Sets, see the following topics:

- Create a rule set
- Add a Case Exchange rule

1.1.8.2.1.7. Invoke Case Exchange Rule Sets

This section introduces how to invoke Rule Sets to trigger automated data exchange for the Case Exchange integration.

Invoke Rule Sets from workflows

Applies to User roles: System Administrator

On a Service Manager system that has Process Designer implemented, you can invoke a Rule Set from a workflow phase.

To invoke a Rule Set from a workflow phase, follow these steps:

1. Click **Tailoring** > **Process Designer** > **Workflows** from the System Navigator. The workflows list opens.
2. Select the workflow in which you want to invoke a Rule Set.
3. Select the phase in which you want to invoke a Rule Set.
4. Add the Rule Set in the following tabs according to your needs:
   - The **On enter**, **On exit**, **Initialization**, **On display**, **On update**, or **After successful update** tab under the **Rule Sets** tab.
   - The **Actions** tab.
5. Save the workflow.

Invoke Rule Sets from triggers

Applies to User roles: System Administrator

If Process Designer is not implemented in Service Manager, you must invoke a Rule Set from a Service Manager trigger, by adding the following API in the trigger's script:

`lib.CaseExchange_RuleExecute.executeSingleRuleSet(record,oldrecord,RuleSetID)`

The following table explains the meaning of each parameter in this API.

Parameter	Description
record	The new record.
oldrecord	The old record.
RuleSetID	The ID of the Case Exchange Rule Set. For example, `CE_CREATE_INCIDENT`.

To add a trigger that invokes a Rule Set, follow these steps:

1. Click **Tailoring** > **Database Manager**. The Database Manager page opens.
2. Type triggers in the **Table** field, and then click **Search**. The Search Trigger Records page opens.
3. Complete the following fields:

Field	Description
Trigger Name	The name of the trigger.
Table Name	The file to be monitored by this trigger.
Trigger Type	Select the specific activity to react to: **1 - Before Add** - When adding a new record to this file, launch the application prior to committing the addition. **2 - After Add** - When adding a new record to this file, launch the application after committing the addition. **3 - Before Update** - When modifying a record in this file, launch the application prior to committing the change. **4 - After Update** - When modifying a record in this file, launch the application after committing the change. **5 - Before Delete** - When deleting a record in this file, launch the application prior to performing the deletion. **6 - After Delete** - When deleting a record in this file, launch the application after performing the deletion. **Note:** Only triggers of *type 4* (After Update), and *type 6* (After Delete) are supported by Cascade Updates.
Application	The RAD application to be launch by this trigger. If you only intend to invoke a Rule Set by using this trigger, leave this field empty.

Field	Description
Script	Call the following API by using the appropriate parameters: `lib.CaseExchange_RuleExecute.executeSingleRuleSet(record,oldrecord,RuleSetID)`

4. Click **Add**.

1.1.8.2.2. Enable Case Exchange with SMAX

This section provides detailed instructions on how to set up the Case Exchange integration for the Incident Management module between Service Management Automation X (SMAX) and Service Manager.

Before setting up the Case Exchange integration, refer to SMAX integration with Service Manager (SM) and review the compatibility information between Service Manager and Service Management Automation X.

Topics in this section include:

The Pull mechanism

Configure SSL between Service Manager and SMAX

Add and enable an integration instance

Create and invoke Rule Sets

Specify Service Manager as an external system in SMAX

Test the integration

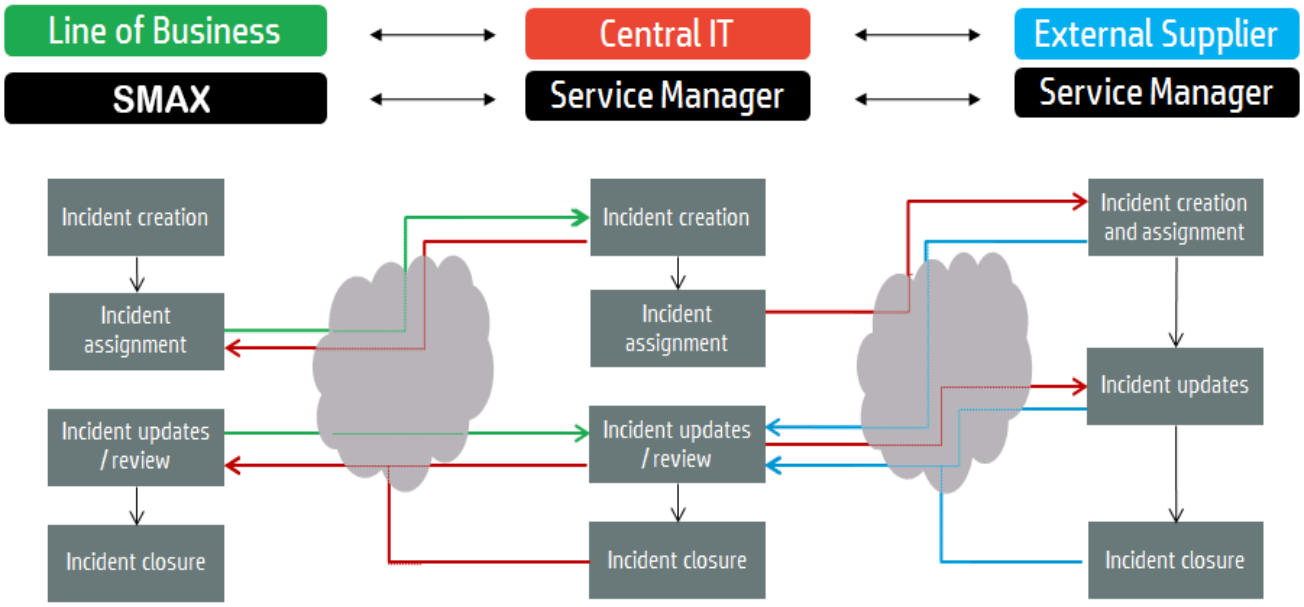
Related topics

Service Manager Integration Suite (SMIS)
Case Exchange framework
Case Exchange framework features
Enable Case Exchange with another system
Incident exchange scenarios
Common user tasks
System administrator tasks

1.1.8.2.2.1. The Pull mechanism

The Case Exchange integration between Service Manager and SMAX uses the Pull mechanism.

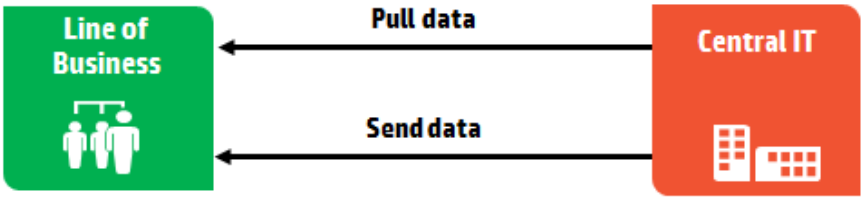
This section explains the Pull mechanisms between Line of Business (LOB) and Central IT (CIT) in the following typical ecosystem.



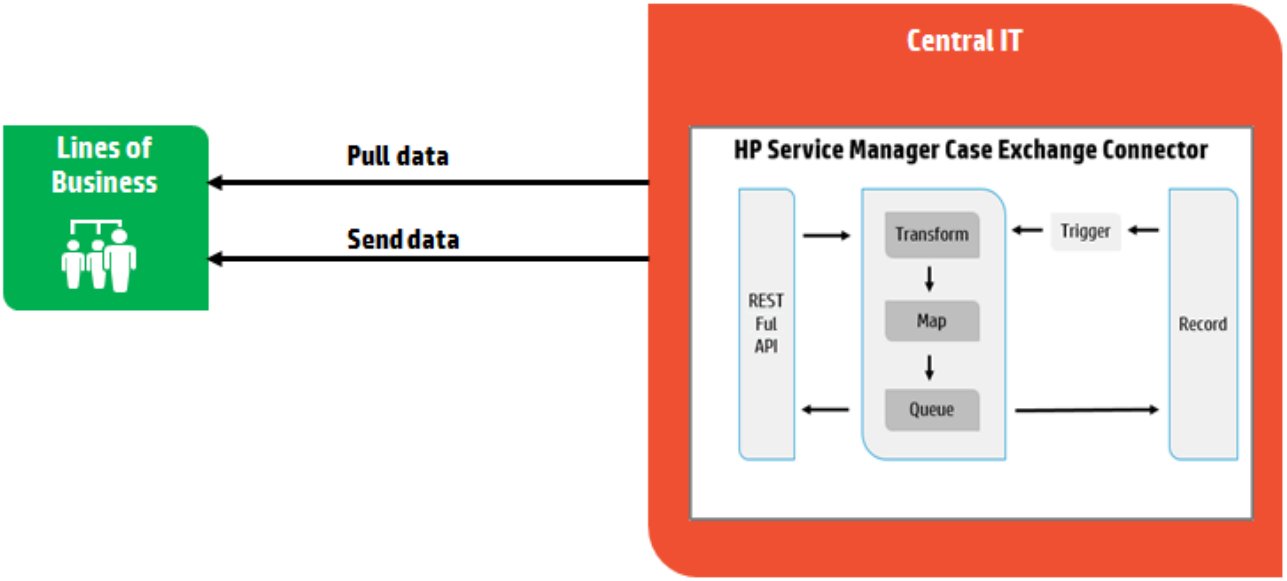
In the Pull mechanism, one system actively pulls data from and sends appropriate data to the other system.

> **Note**
>
> In the following diagrams, the arrows between the two systems represent the direction of the request, rather than the direction of the data flow.

In the following diagram, the CIT's system pulls data from and sends the updated data back to the LOB's system.



The following diagram shows the data flow of the Pull mechanism.


Central IT
Lines of Business
Pull data
Send data
HP Service Manager Case Exchange Connector
REST Ful API
Transform
Map
Queue
Trigger
Record

1.1.8.2.2.2. Configure SSL between Service Manager and SMAX

Service Manager accesses SMAX through HTTPS URLs. Follow these steps to configure the SSL settings between Service Manager and SMAX:

1. Get a CA certificate from the SMAX suite. For example, smaxca.crt.
   For detailed instructions, see Get the suite CA certificate.
2. Copy the cacerts (such as <Service Manager installation directory>\Server\RUN\jre\lib\security\cacerts) and smaxca.crt files to the <Service Manager installation directory>\Server\RUN\jre\bin directory.
3. In the bin directory, run the following command to import smaxca to cacerts.
   keytool -import -alias smaxca -keystore cacerts -file smaxca.crt
4. Copy the cacerts file from <Service Manager installation directory>\Server\RUN\jre\bin to <Service Manager installation directory>\Server\RUN.
5. Open the sm.ini file with a text editor, and then add the following parameter:
   `truststoreFile:cacerts`
   `truststorePass:<cacert's password>`

1.1.8.2.2.3. Add and enable an integration instance

Applies to User roles: System Administrator

The configuration of the Service Manager and SMAX integration is mainly done on the Service Manager side. However, you must do some configuration in SMAX before records can be exchanged between the two systems. The following subsections provide detailed instructions for adding and enabling the integration instance.

Note

The following instructions assume that you use the out-of-box settings of the **CaseExchangeSM_SAW** integration template. However, you can customize these out-of-box settings according to your need. For more information about how to customize the settings in the integration instance, see the following sections:

Add and enable a Case Exchange integration instance
Integration Instance Mapping

Add an integration instance in Service Manager

To add the Case Exchange integration between Service Manager and SMAX, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **CaseExchangeSM_SAW** from the **Integration Template** list, select the **Import Mapping** checkbox, and then click **Next**.
4. Complete the fields on the **Integration Instance Information** page as necessary, and then click **Next**.
5. In the **Integration Instance Parameters** page, configure the following settings on the **General** tab, and use the default setup for all the other settings.

   • **Base URL**: The base URL of the SMAX API. The format of the URL is: https://*<SMAX Server>*.

   • **Login URL**: The login URL of SMAX. The format of the URL is: *<Base URL>/auth/authentication-endpoint/authenticate/token*

   • **Tenant Id**: The Tenant Id of the SMAX system.

   • **User Name** and **Password**: The credentials of the SMAX account for this integration.

6. (Optional) Click the **Inbound** tab, and then configure the **Time difference (s)** setting in the integration instance if a time difference occurs between Service Manager and SMAX. For more information, see Time difference issue.
7. Click **Next**. The **Integration Instance Fields** page opens.
8. Modify the fields in the **SM Fields** and **Endpoint Fields** tabs as necessary. Otherwise, go to the next step.
9. Click **Next**. The **Integration Instance Mapping** page opens.
10. In the **Post Script** tab, update `10019` to the corresponding Service ID in SMAX in the following out-of-box code:

```
if (context.outbound) {
context.action = mapObj["ext_properties.Operation"];
//set the default value of required Primary Data when create the
ticket
if(context.action=="Create") {
mapObj["properties.RegisteredForActualService"]="10019";
        //set the default service
}
```

11. Click **Finish**.

Enable an integration instance in Service Manager

To enable the integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select the integration instance that you want to enable.
3. Click **Enable**.
4. Click **Yes**.

Configure an integration instance in SMAX

Before you can exchange records between Service Manager and SMAX, you must do the following configuration in SMAX:

1. Make sure your user account has the "SACM Integration" role assigned.
2. Add an external system.
   In Service Manager Case Exchange SMIS instance, an external system named "SM" is used by default for SMAX integration.



Note

For the external system defined in SMAX, if you use a name other than "SM", you must make the following changes in Service Manager when you add the integration instance:

- For the "Query" field on the **Inbound** tab, replace "SM" in "system=SM" with your new name
- For the values of the "Additional path" column on the **Outbound** tab, replace "SM" with your new name in "ExternalSystem":"SM"

3. Assign the external system to a group by using the **External system** field on the **Groups** page. This makes the group an external group.

After this configuration, you can select the external group for an incident record in the Incident Management module. Once an external group is selected, a new section, **External Assignment**, is then added to the incident page. You can then use that section to configure the record for data exchange.

For more information about the external systems, groups, or user account roles in SMAX, refer to the corresponding sections in the SMAX documentation.

1.1.8.2.2.4. Create and invoke Rule Sets

Service Manager doesn't provide out-of-box Rule Sets for the Case Exchange integration between Service Manager and SMAX, so you must create Rule Sets when you set up the integration. See the following topics for how to create and invoke Rule Sets:

- Create a rule set

  > **Note**
  >
  > When you create a Rule Set for incident exchange between Service Manager and SMAX, set the **Table name** field to **probsummary**.

- Add a Case Exchange rule
- Using the Condition Editor
- Invoke Case Exchange Rule Sets

Example Rule Sets

The following table provides example outbound rules that work in the out-of-box Service Manager system. You may modify these rules according to the workflow in your system.

Condition (RAD expression)	Event
assignment in $L.file=" <external_assignment_group>" and sysmoduser in $L.file~=" <smis_scheduler_name>" and jscall("CaseExchangeExternalReferencesDAO.getExternalID",number in $L.file)=""	Create
problem.status in $L.file= problem.status in $L.file.save and sysmoduser in $L.file~=" <smis_scheduler_name>" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Update
sysmoduser in $L.file=" <smis_scheduler_name>" **Note** This rule is used to acknowledge the Update action.	Update
assignment in $L.file=" <external_assignment_group>" and problem.status in $L.file~= problem.status in $L.file.save and resolution.code in $L.file#"Solved by" and sysmoduser in $L.file~="<smis_scheduler_name>" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Resolve
assignment in $L.file=" <external_assignment_group>" and problem.status in $L.file~= problem.status in $L.file.save and resolution.code in $L.file="Request Rejected" and sysmoduser in $L.file~="<smis_scheduler_name>" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Reject

Condition (RAD expression)	Event
assignment in $L.file=" <external_assignment_group>" and problem.status in $L.file~= problem.status in $L.file.save and resolution.code in $L.file="Withdrawn by User" and sysmoduser in $L.file~=" <smis_scheduler_name>" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Cancel

For more information about the **getExternalID** and **isExternalActive** functions, see Functions used in rule conditions.

1.1.8.2.2.5. Specify Service Manager as an external system in SMAX

To enable the Case Exchange integration with SMAX, you also need to specify Service Manager as an external system in the SMAX environment.

To do this, follow these steps:

1. Sign in to SMAX as a system administrator.
2. Click **My Dashboards**.
3. From the main menu, navigate to **Administration** > **Utilities** and then click **Integration**.
4. Select the **External Systems** tab, and then click **New**.
5. On the external system details page, enter information as described in the following table.

Field	Value
System id	SM **Note** This is the default value that's defined in the CaseExchangeSM_SAW integration template in Service Manager. To verify this value (`&system=SM`), locate the Query string on the **Inbound** tab of the Integration Instance Parameters page of this integration template. For more information, see Integration Instance Parameters - Inbound. If you have customized the Query string, be sure to enter a value that's consistent with your customized Query string.
Description	A short description that identifies the Service Manager system
Authorized user	Select your login account from the list of user accounts.

6. Click **Save**.
7. Create an Assignment Group for Service Manager.

    a. From the main menu, navigate to **Administration** > **Master Data**, and then click **People**.

    b. Click the **Groups** tab at the top of the page.

    c. Click **New**.

    d. On the new group page, enter information as described in the following table.

Field	Value
Name	Enter Central IT as the name for the group. This is a mandatory field.
User principal name	Enter Central IT for this field. This field is mandatory and must be unique.
Group type	Select **Organizational**.
Email	(Optional) Enter an email address for the group.
Owner	(Optional) Open the drop-down search window and search for the required person.



Note

If you want Service Manager to pull the Incidents, set either the **Expert group** or **Service desk group** value to **Central IT**. Additionally, when creating a new Incident, you must complete the title, description, impact, urgency, service, category, service desk group, and assignment group fields in the SMAX Incident form.

e. Click **Save** to add the new group to the list of groups.

8. Click the Central IT group from the list and then make the following edits.

   a. In the **External system** field, select **SM**.

   b. Locate the **System use definitions** section, and add the following group roles in the **Group roles** field: Incident Analyst, Incident Coordinator, Incident Manager, and Incident Process Owner.

   c. Click **Save** to save the group.

1.1.8.2.2.6. Test the integration

Once you have finished the above configurations in the Service Manager system, you can test the Case Exchange integration. The following procedure is an example that you can follow to test the basic functions of the integration:

1. In SMAX, create a new Incident record.
2. Wait for a while (depending on the value that's configured in the **Interval Time** field) and the Incident record should be exchanged to the Service Manager system.
3. In Service Manager, update the Incident record so that the update can trigger the outbound rule that's configured in Create and invoke Rule Sets.
4. The update to the Incident record should be exchanged to SMAX.

If all the above steps are successful, the integration between the two systems is properly configured.

If any of the above steps is unsuccessful, the following checklist can help you to solve the possible problems.

Symptoms	Checklist
In step 2, the Incident record doesn't appear in Service Manager.	Check the following areas in Service Manager: • Check the Service Manager Integration Suite (SMIS) task queue and task log. • Check the integration instance. • Is the integration instance active? • Is the information in the **General** tab correct? • Have you done any customization when you set up the integration instance? • Check the log file. For more information, refer to Review log file.
In step 4, the update in Service Manager can't be exchanged to SMAX.	Check the following areas in Service Manager: • Check the SMIS task queue and task log. • Have you done any customization when you set up the integration instance? • Check the outbound rule, and make sure that the update to the Incident record can trigger the outbound rule. See Create and invoke Rule Sets. • Check the log file. For more information, refer to Review log file.

Consider the following tips when you test the Case Exchange integration:

- Incorrect configuration of the **Base URL**, **User Name**, and **Password** fields in the **General** tab usually causes authentication failure. In this scenario, 401 errors are recorded in the log file.

- You can enable the error-handling feature when you configure the integration instance. SMIS can automatically create a new Incident record if a Case Exchange task fails. The **Description** field of this Incident record contains the details of the failed task record. For more information about the error-handling feature, see Error handling.
- Due to a known limitation, the error message in the SMIS task log may be insufficient to identify the root cause of a failed task. For more information about this known limitation and the workaround, see Insufficient message for a failed Case Exchange task.

1.1.8.2.3. Enable Case Exchange between two Service Manager systems

This section provides detailed instructions on how to set up the Case Exchange integration for the Incident Management module between two Service Manager systems.

Service Manager provides two integration templates for the Case Exchange integration between two Service Manager systems: **CaseExchangeSM_SM_Pull** and **CaseExchangeSM_SM_Push**. You can modify these template as necessary. To modify the integration template, follow the instructions in Service Manager Integration Suite (SMIS).

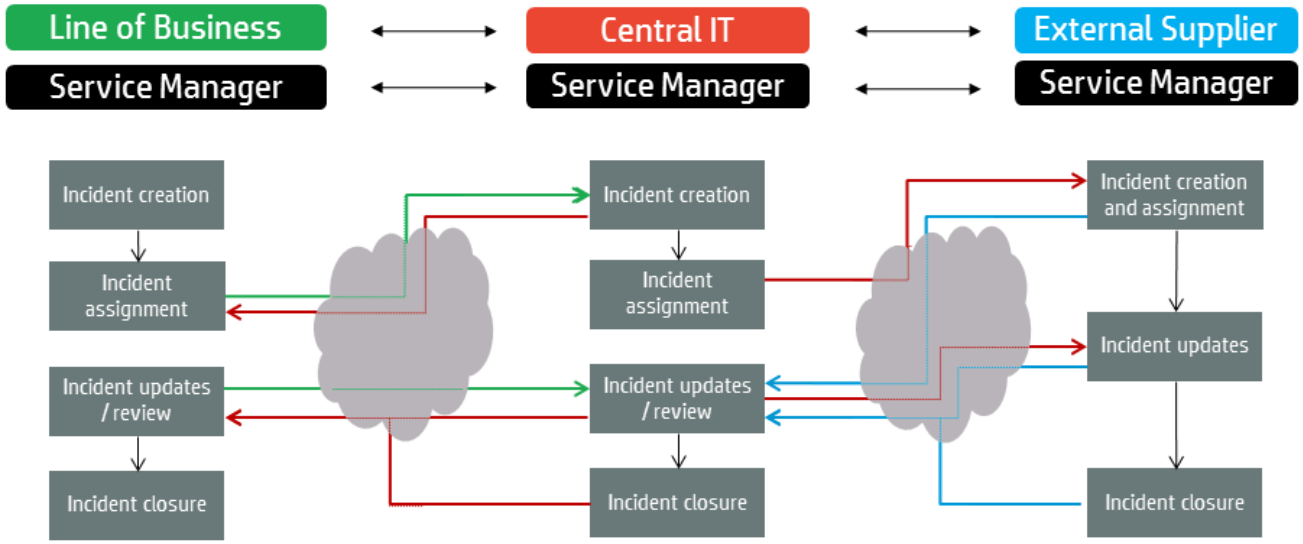
Before you start to set up the Case Exchange integration, we highly recommend that you read the following sections under Case Exchange framework:

- Case Exchange framework features: This section introduces the core features of the Case Exchange framework.
- Enable Case Exchange with another system : This section provides recommendations and general instructions on how to enable Case Exchange with another system.
- Incident exchange scenarios: This section introduces typical Case Exchange use cases.
- Common user tasks: This section introduces the common user tasks that apply to the Incident Management module in the Case Exchange integration.
- System administrator tasks: This section introduces the system administrator tasks in the Case Exchange integration.

1.1.8.2.3.1. Connection mechanisms

The Case Exchange integration between two Service Manager systems has two connection mechanisms: Pull and Push.

This section explains the two connection mechanisms between Line of Business (LOB) and Central IT (CIT) in the following typical ecosystem.



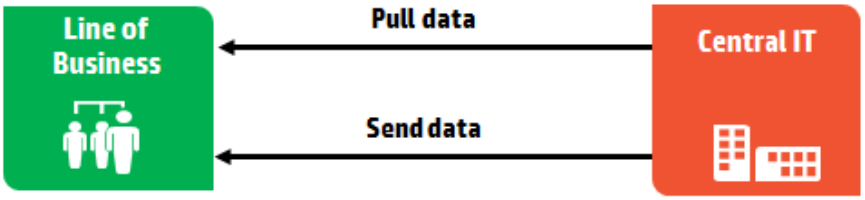
The Pull mechanism

In the Pull mechanism, one system actively pulls data from and sends appropriate data to the other system. The latter system doesn't need an integration instance.

In the following diagram, the Central IT (CIT) pulls data from and sends the updated data back to the Line of Business (LOB).

Note

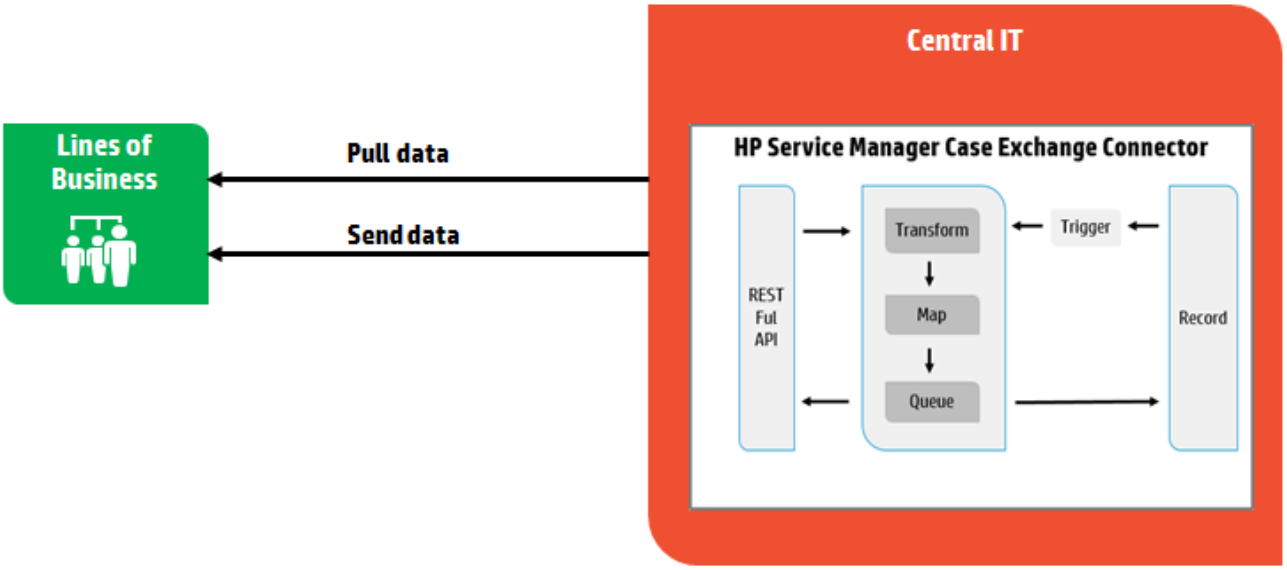
In the following diagrams, the arrows between the two systems represent the direction of the request, rather than the direction of the data flow.



The Pull mechanism has the following advantages and disadvantages.

Advantages	Disadvantages
• Only one integration account is required. That is, an integration account in the LOB's system. • You don't need to configure the Case Exchange logic in the LOB's system. • The CIT's system manages all use cases for Case Exchange. • Only the CIT's system needs configurations of field mapping and value mapping.	• The LOB has very little control over the pulled data.

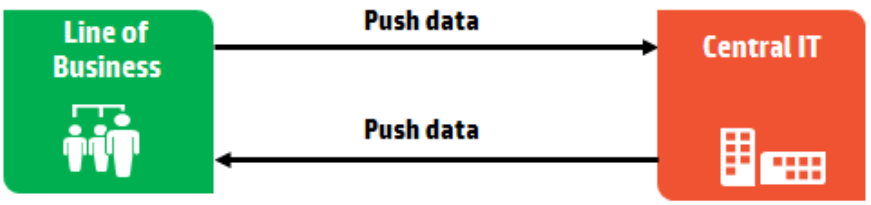
The following diagram shows the data flow of the Pull mechanism.



The Push mechanism

In the Push mechanism, both systems push data to each other. In this mechanism, both systems need an active integration instance.

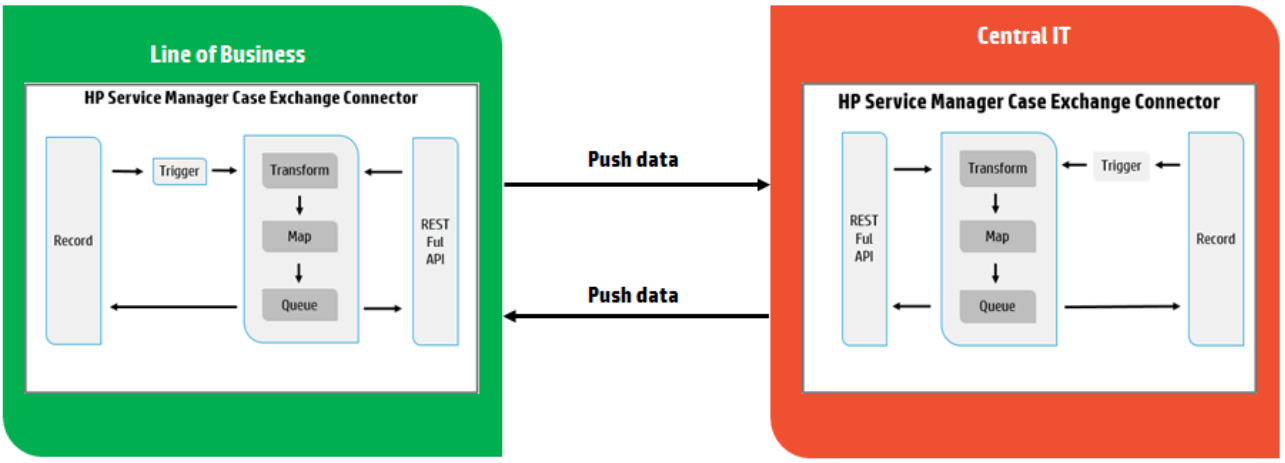
In the following diagram, the Line of Business (LOB) pushes data to the Central IT (CIT), and the Central IT pushes the updates back to the Line of Business.



The Push mechanism has the following advantages and disadvantages.

Advantages	Disadvantages
• The LOB defines the criteria to send data to the CIT. • The CIT receives data based on the LOB's decision. • Every party manages its own integration. • This mechanism provides flexibility on both sides. • The LOB can integrate with other Service Providers.	• Configuration is required in both systems. • Field mapping and data mapping are required in both systems.

The following diagram shows the data flow of the Push mechanism.



1.1.8.2.3.2. Using the Pull mechanism

This section describes how to set up the Case Exchange integration between two Service Manager systems (System 1 and System 2) by using the Pull mechanism.

In this mechanism, System 1 uses an integration instance that's based on the **CaseExchangeSM_SM_Pull** template, and System 2 doesn't need an integration instance.

Configure System 1

In System 1, you need to perform the following tasks.

Add and enable an integration instance

Applies to User roles: System Administrator

To add an integration instance in System 1, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **CaseExchangeSM_SM_Pull** from the **Integration Template** list, select the **Import Mapping** checkbox, and then click **Next**.
4. Complete the fields on the **Integration Instance Information** page as necessary, and then click **Next**.

   For more information about how to configure this page, see Integration Instance Information fields.
5. In the **Integration Instance Parameters** page, configure the following settings in the **General** tab and use the default setup for all the other settings.

   Note

   You can modify the other fields according to your needs. For more information about how to modify the fields in this page, see Configuration details

   Endpoint System runs Process Designer on the exchanged Object: Select this option if Process Designer is implemented in system 2. Otherwise, don't select this option.

   Base URL: The value of this field represents the base URL of the endpoint API. For example, `http://{endpoint server}:13080/SM/9`

   User Name and **Password**: The credentials of the integration account in system 2.

> **Note**
>
> • The **Password** field must not be empty.
> • This account must be properly configured in system 2. For more information, see *Configure the integration account*.

6. Click **Next**. The **Integration Instance Fields** page opens.
7. Modify the default entries in the **SM Fields** and **Endpoint Fields** if you have customized fields. Otherwise, go to the next step.

   > **Note**
   >
   > Endpoint field names must be the same as the caption names in web service **CEIncidentsPull**. For more information about the caption names configured in **CEIncidentsPull**, click **Tailoring** > **Web Service** > **Web Service Configuration** in Service Manager. In addition, you can refer to the out-of-box settings of the **Field Name** field in the **Endpoint Fields** tab.

8. Click **Next**. The **Integration Instance Mapping** page opens.
9. Configure field and value mappings if you have customized fields. Otherwise, go to the next step.

   For more information about how to edit the settings on this page, see the topics under Integration Instance Mapping.

10. Click **Finish**.

To enable the integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select the integration instance that you want to enable.
3. Click **Enable**.
4. Click **Yes**.

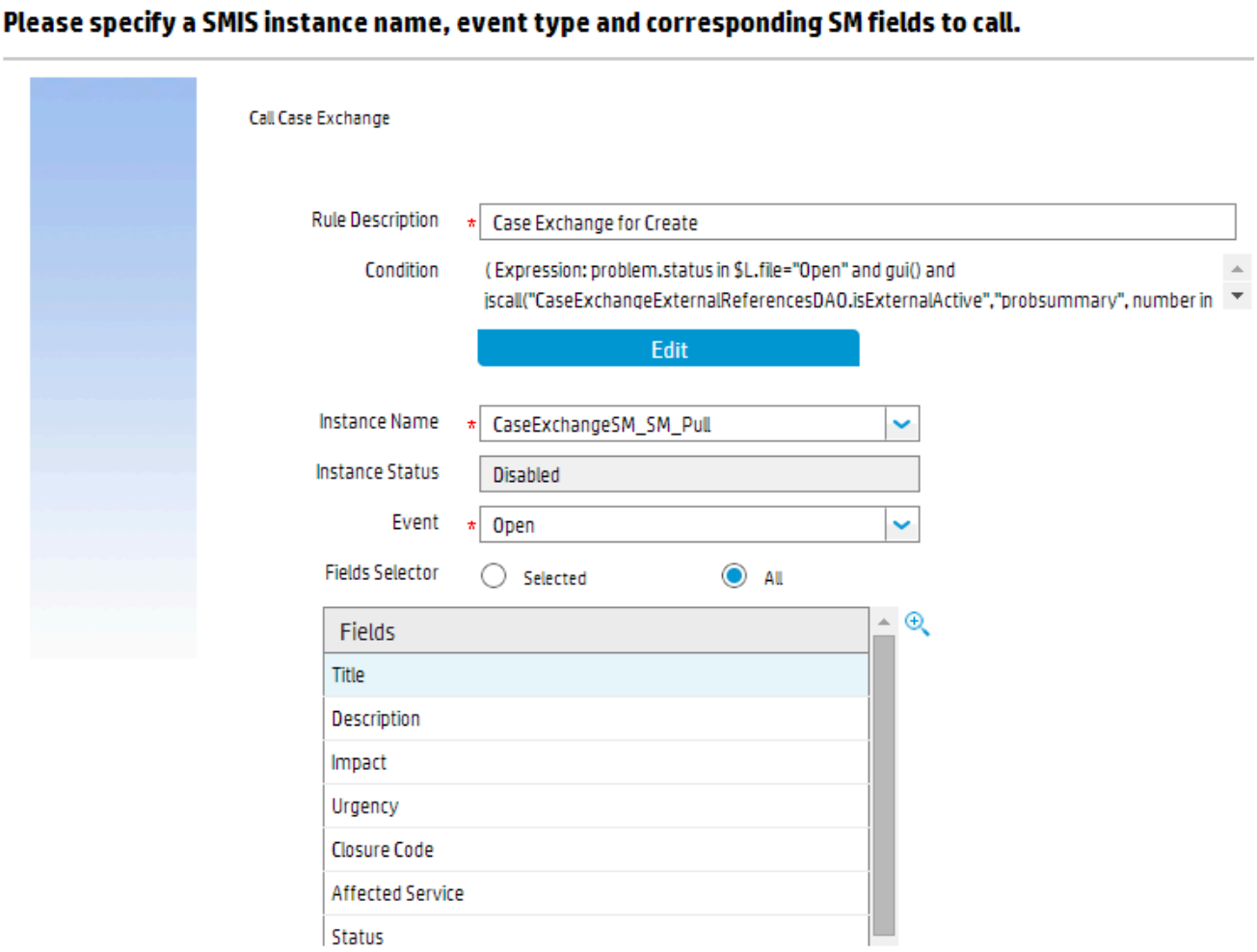
Create and invoke outbound rules

This section introduces how to create and invoke outbound rules when you use the Pull mechanism.

In Case Exchange, outbound rules implement business logic to drive a workflow or a process. Rules can help perform calculations, validate fields based on data or Rule Sets, set required fields, and more. A Rule Set uses role-based security.

Create outbound rules

The following screenshot shows an example outbound rule configuration for a Case Exchange integration.



The following table provides example outbound rules that work in the out-of-box Service Manager system. You may modify these rules according to the workflow in your system.

Note

When you create a Rule Set for Incident exchange, set the **Table name** field to `probsummary`.

Condition (RAD expression)	Event
assignment in $L.file="*<assignment group>*" and problem.status in $L.file isin {"Pending Customer", "Pending Vendor"} and sysmoduser in $L.file~="*<smis_scheduler_name>*" and jscall("CaseExchangeExternalReferencesDAO.getExternalID",number in $L.file)=""	Create
(not same(problem.status in $L.file, problem.status in $L.file.save) or $apm.activity isin {"Communication with customer","Communication with vendor"}) and not (problem.status in $L.file isin {"Rejected","Resolved","Closed"}) and sysmoduser in $L.file~="*<smis_scheduler_name>*" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Update
not same(problem.status in $L.file, problem.status in $L.file.save) and problem.status in $L.file isin {"Resolved","Closed"} and resolution.code in $L.file~="Request Rejected" and sysmoduser in $L.file~="*<smis_scheduler_name>*" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Resolve
not same(problem.status in $L.file, problem.status in $L.file.save) and problem.status in $L.file isin {"Rejected","Resolved","Closed"} and resolution.code in $L.file="Request Rejected" and sysmoduser in $L.file~="*<smis_scheduler_name>*" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Reject

Condition (RAD expression)	Event
(sysmoduser in $L.file="<*smis_scheduler_name*>")	Acknowledge

For more information about the **getExternalID** and **isExternalActive** functions, see Create a rule set

Invoke Rule Sets

If Process Designer is implemented in Service Manager, you can invoke Rule Sets from workflows. For more information, see Invoke Rule Sets from workflows.

If Process Designer is not implemented in Service Manager, you must invoke Rule Sets from triggers. The following table contains example configurations for two triggers: Incident update and Incident creation.

Trigger	Incident update	Incident creation
Trigger Name	case.exchange-update	case.exchange-creation
Table Name	probsummary	probsummary
Trigger Type	4 - After Update	2 - After Add
Script	lib.CaseExchange_RuleExecute.executeSingleRuleSet(record, oldrecord,"CERule4SM2SM")	lib.CaseExchange_RuleExecute.executeSingleRuleSet(record, oldrecord,"CERule4SM2SM")

Note

In this example, the Rule Set ID is `CERule4SM2SM`.

For more information about how to invoke Rule Sets from triggers, see Invoke Rule Sets from triggers.

Configure System 2

In System 2, you only need to configure the account for the Case Exchange integration.

Configure the integration account

System 1 needs the credentials of an account in System 2 to successfully enable the integration. This account must meet the following requirements:

- In case System 2 needs to control what data is pulled by System 1, System 2 can apply mandanten security on this account. For more information, see Mandanten file security.
- This account must have the **RESTful API** execution capability.

  For more information about how to configure this execution capability, see Configure the integration account.
- This account must have the rights to retrieve and update Incident records.
- This account must use the same **Time Zone** and **Date Format** settings as the endpoint Service Manager system. See also Time difference issue.

Test the integration

Once you have finished the above configurations in the Service Manager systems, you can test the Case Exchange integration. The following procedure is an example that you can follow to test the basic functions of the integration:

1. In System 2, create a new Incident record that's accessible by the integration account that's configured in *Configure the integration account*.
2. Wait for a while (depending on the value that's configured in the **Interval Time** field) and the Incident record should be exchanged to System 1.
3. In System 1, update the Incident record so that the update can trigger the outbound rule that's configured in *Create and invoke outbound rules*.
4. The update to the Incident record should be exchanged to System 2.

If all the above steps are successful, the integration between the two systems is properly configured.

If any of the above steps is unsuccessful, the following checklist can help you to solve the possible problems.

Symptoms	Checklist
In step 2, the Incident record doesn't appear in System 1.	• In System 1, check the Service Manager Integration Suite (SMIS) task queue and task log. • In System 1, check the integration instance. • Is the integration instance active? • Is the information in the **General** tab correct? • Have you done any customization when you set up the integration instance? • In System 1, check the log file. For more information about how to check the log file, see Review log file. • In System 2, check the account setting, and make sure that the Incident record is accessible by the integration account. see *Configure the integration account*.
In step 4, the update in System 1 can't be exchanged to System 2.	• In System 1, check the SMIS task queue and task log. • In System 1, have you done any customization when you set up the integration instance? • In System 1, check the outbound rule, and make sure that the update to the Incident record can trigger the outbound rule. See *Create and invoke outbound rules*. • Check the log file. For more information about how to check the log file, see Review log file.

Consider the following tips when you test the Case Exchange integration:

- Incorrect configuration of the **Base URL**, **User Name**, and **Password** fields in the **General** tab usually causes authentication failure. In this scenario, 401 errors are recorded in the log file.
- You can enable the error-handling feature when you configure the integration instance. SMIS can automatically create a new Incident record if a Case Exchange task fails. The **Description** field of this Incident record contains the details of the failed task record. For more information about the error-handling feature, see Error handling.
- Due to a known limitation, the error message in the SMIS task log may be insufficient to identify the root cause of a failed task. For more information about this known limitation and the workaround, see Insufficient message for a failed Case Exchange task.

1.1.8.2.3.3. Using the Push mechanism

This section describes how to set up the Case Exchange integration between two Service Manager systems (System 1 and System 2) by using the Push mechanism.

In this mechanism, both systems use an integration instance that's based on the **CaseExchangeSM_SM_Push** template. You need to perform the following configurations in both Service Manager systems:

Configure the integration account

Each system must have a properly configured account for the integration so that the other system can connect by using this account. This account must meet the following requirements:

- This account must have the **RESTful API** execution capability.
- This account must have the rights to retrieve and update Incident records.
- This account must use the same **Time Zone** and **Date Format** settings as the Service Manager systems. See also Time difference issue.

Add and enable an integration instance

Applies to User roles: System Administrator

To add an integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Click **Add**. The **Integration Template Selection** wizard opens.
3. Select **CaseExchangeSM_SM_Push** from the **Integration Template** list, select the **Import Mapping** checkbox, and then click **Next**.
4. Complete the fields on the **Integration Instance Information** page as necessary, and then click **Next**.

   For more information on how to configure this page, see Integration Instance Information fields.
5. On the **Integration Instance Parameters** page, configure the following settings and use the default setup for all the other settings.

   Note

   You can modify the other fields according to your needs. For more information about how to modify the fields on this page, see Configuration details

   a. On the **General** tab, configure the following fields or options:

Endpoint System runs Process Designer on the exchanged Object: Select this option if Process Designer is implemented in the endpoint system. Otherwise, don't select this option.

Base URL: The value of this field represents the base URL of the endpoint API. For example, `http://{endpoint server}:13080/SM/9`

User Name and **Password**: The credentials of the integration account in the endpoint system.

> **Note**
>
> • The **Password** field must not be empty.
> • This account must be properly configured in the endpoint system. For more information, see *Configure the integration account*.

b. On the **Outbound** tab, set the *<Endpoint_instance_ID>* in all rows of the **Additional path** column:

```
{"CEIncidentPush": {"InstanceID": "
<Endpoint_instance_ID>","InternalID":"${context.externalId}",
"ExternalID": "${context.internalId}",
"IncidentID":"${context.externalId}",
"ExternalStatus":"${context.internalObject['problem.status']}"}}
```

<Endpoint_instance_ID> is the ID of the SMIS integration instance in the other Service Manager system. You can find this ID under the **Id** column when you open Integration Manager in the other Service Manager system.

c. If you need to exchange attachments, select the **Activate attachment exchange** option on the **Attachment Handling** tab.

6. Click **Next**. The **Integration Instance Fields** page opens.
7. Modify the default entries in the **SM Fields** and **Endpoint Fields** if you have customized fields. Otherwise, go to the next step.

> **Note**
>
> Endpoint field names must be the same as the caption names in web service **CEIncidentsPush**. For more information about the caption names configured in **CEIncidentsPush**, click **Tailoring** > **Web Service** > **Web Service Configuration** in Service Manager. In addition, you can refer to the out-of-box settings of the **Field Name** field in the **Endpoint Fields** tab.

8. Click **Next**. The **Integration Instance Mapping** page opens.

9. Configure field and value mappings if you have customized fields. Otherwise, go to the next step.

   For more information about how to edit the settings on this page, see the topics under Integration Instance Mapping.

10. Click **Finish**.

To enable the integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Select the integration instance that you want to enable.
3. Click **Enable**.
4. Click **Yes**.

Create and invoke outbound rules

This section introduces how to create and invoke outbound rules when you use the Push mechanism.

In Case Exchange, outbound rules implement business logic to drive a workflow or a process. Rules can help perform calculations, validate fields based on data or Rule Sets, set required fields, and more. A Rule Set uses role-based security.

Create outbound rules

The following table provides example outbound rules that work in the out-of-box Service Manager system. You may modify these rules according to the workflow in your system.

For an example of the outbound rule configuration, see the screenshot in Create outbound rules.



Note

When you create a Rule Set for Incident exchange, set the **Table name** field to `probsummary`.

Condition (RAD expression)	Event
assignment in $L.file="*<assignment group>*" and problem.status in $L.file isin {"Pending Customer", "Pending Vendor"} and sysmoduser in $L.file~="*<smis_scheduler_name>*" and sysmoduser in $L.file~="*<endpoint_integration_user>*" and jscall("CaseExchangeExternalReferencesDAO.getExternalID",number in $L.file)=""	Create
(not same(problem.status in $L.file, problem.status in $L.file.save) or $apm.activity isin {"Communication with customer","Communication with vendor"}) and not (problem.status in $L.file isin {"Rejected","Resolved","Closed"}) and sysmoduser in $L.file~="*<smis_scheduler_name>*" and sysmoduser in $L.file~="*<endpoint_integration_user>*" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Update
not same(problem.status in $L.file, problem.status in $L.file.save) and problem.status in $L.file isin {"Resolved","Closed"} and resolution.code in $L.file~="Request Rejected" and sysmoduser in $L.file~="*<smis_scheduler_name>*" and sysmoduser in $L.file~="*<endpoint_integration_user>*" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Resolve

Condition (RAD expression)	Event
not same(problem.status in $L.file, problem.status in $L.file.save) and problem.status in $L.file isin {"Rejected","Resolved","Closed"} and resolution.code in $L.file="Request Rejected" and sysmoduser in $L.file~="*<smis_scheduler_name>*" and sysmoduser in $L.file~="*<endpoint_integration_user>*" and jscall("CaseExchangeExternalReferencesDAO.isExternalActive","probsummary",number in $L.file)=true	Reject
(sysmoduser in $L.file="*<smis_scheduler_name>*" or sysmoduser in $L.file="*<endpoint_integration_user>*") and (jscall("CaseExchangeExternalReferencesDAO.getExternalID",number in $L.file)="" or not same(problem.status in $L.file, problem.status in $L.file.save))	Acknowledge

Note

Be careful with the outbound conditions to avoid a loop between the integrated systems.

For more information about the **getExternalID** and **isExternalActive** functions, see Functions used in rule conditions.

For more information about how to create Rule Sets, see the following topics:

- Create a rule set
- Add a Case Exchange rule
- Using the Condition Editor

Invoke Rule Sets

If Process Designer is implemented in Service Manager, you can invoke Rule Sets from workflows. For more information, see Invoke Rule Sets from workflows.

If Process Designer is not implemented in Service Manager, you must invoke Rule Sets from triggers. The following table contains example configurations for two triggers: Incident update and Incident creation.

Trigger	Incident update	Incident creation
Trigger Name	case.exchange-update	case.exchange-creation
Table Name	probsummary	probsummary
Trigger Type	4 - After Update	2 - After Add
Script	lib.CaseExchange_RuleExecute.executeSingleRuleSet(record, oldrecord,"CERule4SM2SM")	lib.CaseExchange_RuleExecute.executeSingleRuleSet(record, oldrecord,"CERule4SM2SM")

Note

In this example, the Rule Set ID is `CERule4SM2SM`.

For more information about how to invoke Rule Sets from triggers, see Invoke Rule Sets from triggers.

Test the integration

Once you have finished the above configurations in the two Service Manager systems, you can test the Case Exchange integration. The following procedure is an example that you can follow to test the basic functions of the integration:

1. In System 2, create a new Incident record. This Incident record must be able to trigger the outbound rule that's configured in System 2. See *Create and invoke outbound rules*.
2. The Incident record should be exchanged to System 1.
3. In System 1, update the Incident record so that the update can trigger the outbound rule that's configured in System 1. See *Create and invoke outbound rules*.
4. The update to the Incident record should be exchanged to System 2.

If all the above steps are successful, the integration between the two systems is properly configured.

If any of the above steps is unsuccessful, the following checklist can help you to solve the possible problems.

Symptoms	Checklist
In step 2, the Incident record doesn't appear in System 1.	• In System 2, check the Service Manager Integration Suite (SMIS) task queue and task log. • In System 2, check the integration instance: • Is the integration instance active? • Is the information in the **General** tab correct? • Have you done any customization when you set up the integration instance? • In System 2, check the log file. For more information about how to check the log file, see Review log file. • In System 2, check the outbound rule, and make sure that the update to the Incident record can trigger the outbound rule. See *Create and invoke outbound rules*. • In System 1, check the account setting. See *Configure the integration account*.

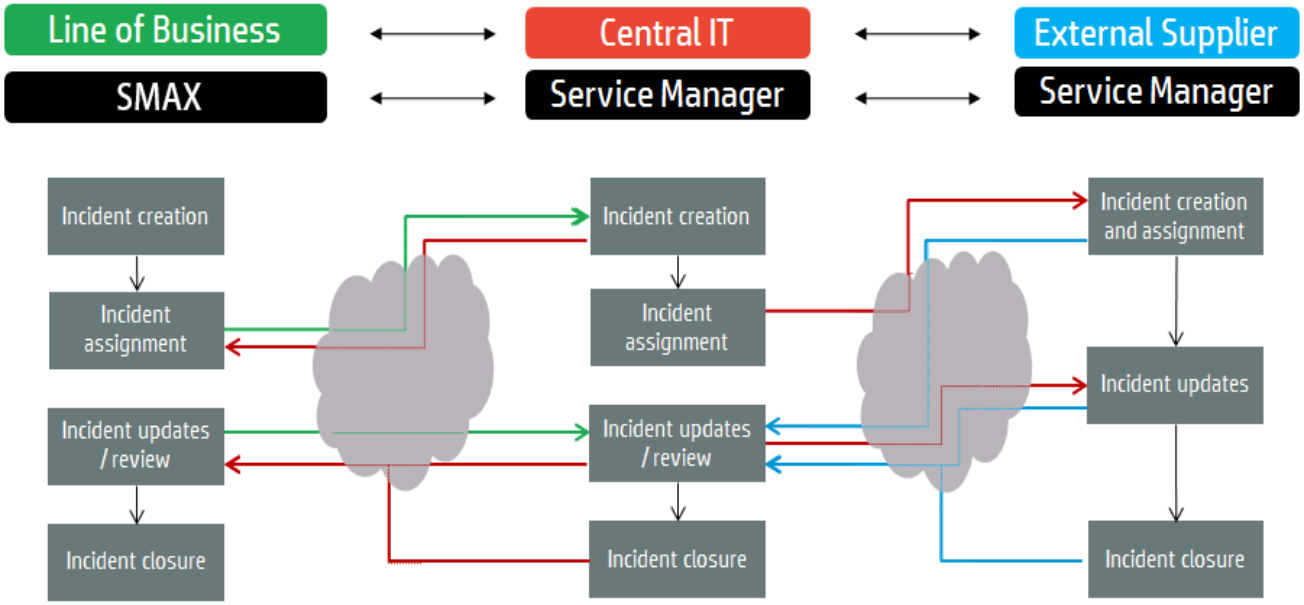
Symptoms	Checklist
In step 4, the update in System 1 can't be exchanged to System 2.	• In System 1, check the SMIS task queue and task log. • In System 1, have you done any customization when you set up the integration instance? • In System 1, check the outbound rule, and make sure that the update to the Incident record can trigger the outbound rule. See *Create and invoke outbound rules*. • In System 1, check the log file. For more information about how to check the log file, see Review log file. • In System 2, check the account setting. See *Configure the integration account*.

Consider the following tips when you test the Case Exchange integration:

- Incorrect configuration of the **Base URL**, **User Name**, and **Password** fields in the **General** tab usually causes authentication failure. In this scenario, 401 errors are recorded in the log file.
- You can enable the error-handling feature when you configure the integration instance. SMIS can automatically create a new Incident record if a Case Exchange task fails. The **Description** field of this Incident record contains the details of the failed task record. For more information about the error-handling feature, see Error handling.
- Due to a known limitation, the error message in the SMIS task log may be insufficient to identify the root cause of a failed task. For more information about this known limitation and the workaround, see Insufficient message for a failed Case Exchange task.

1.1.8.3. Incident exchange scenarios

This section describes various scenarios that are supported by Case Exchange in the Incident Management module. The following diagram shows a typical Case Exchange environment.



The description of the following scenarios uses the following simplifications to keep the scenarios generic:

- These scenarios refer to an Incident record in the target system as an "Incident". However, an Incident in Service Manager may correspond to an Incident, a Work Order, a Service Request, or something else in the connected system.
- The terminology used for the described scenarios is limited to the domain of Service Manager.

The following scenario diagrams in this section contain the following elements:

User actions:

——→ Actions that move a record from one stage to another. For example, open, assign, solve, close, and so on.

- - - - → Updates that don't move a record from one stage to another. For example, an update to description or attachments.

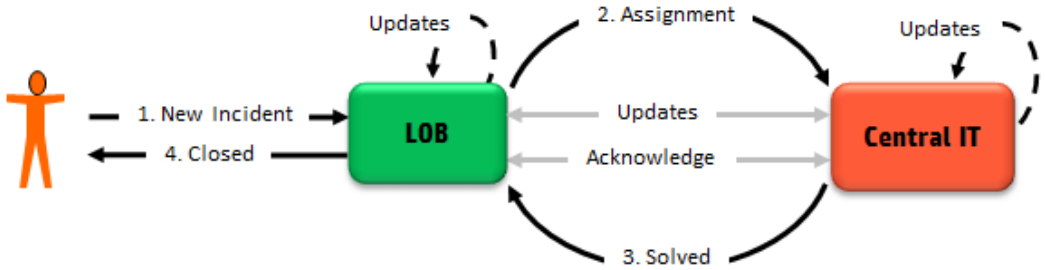
Background system actions:

←— Updates —→ Updates transferred to the connected systems.

←— Acknowledge —→ Always sent as soon data is exchanged (both directions).

Scenario 1

The Line of Business registers and assigns an Incident to the Central IT, who then solves the Incident.



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT solves the Incident.

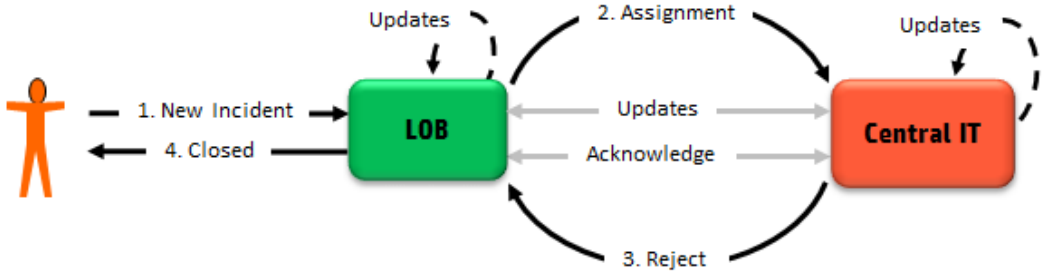
Action	Description
1 New Incident	An end user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT. With this assignment the Incident data is exchanged from the Line of Business to the Central IT.
Updates	Updates can happen on either side. Case Exchange transfers the update to the other side when needed.
3 Solved	The Central IT solves the Incident.
4 Closed	The Line of Business confirms the solution with the end user and closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 2

The Line of Business registers and assigns an Incident to the Central IT, who then rejects the Incident.



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT rejects the Incident.

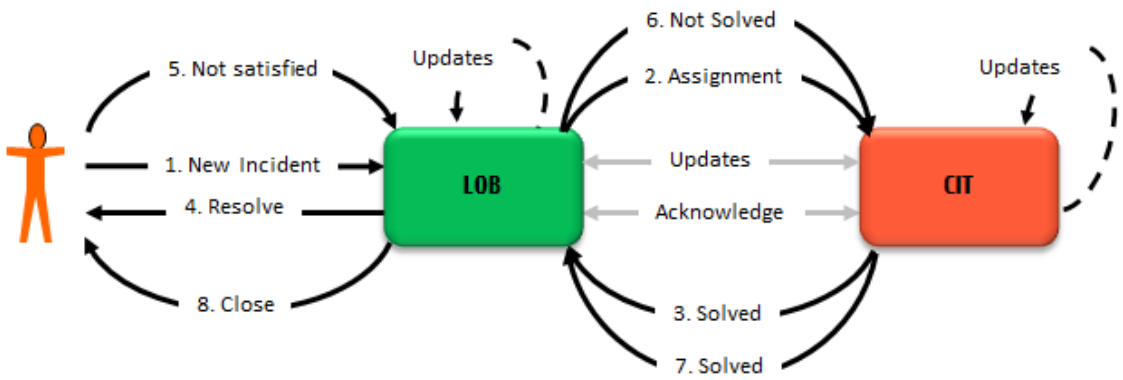
Action	Description
1 New Incident	An end user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
3 Reject	The Central IT sends a `Reject` update, which indicates the Incident is rejected (external Incident status). The responsibility is back to the Line of Business, who then solves the Incident internally.
Updates	Updates can happen on either side. Case Exchange transfers the update to the other side when needed.
4 Closed	The Line of Business confirms the solution with the end user and closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 3

The Line of Business registers an Incident and the Central IT solves the Incident. The solution doesn't satisfy the customer. The Central IT must solve the Incident again.



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT solves the Incident. The Line of Business confirms the solution with the end user, but the end user is not satisfied. Therefore, the Incident is reassigned (re-opened or re-created) to the Central IT.

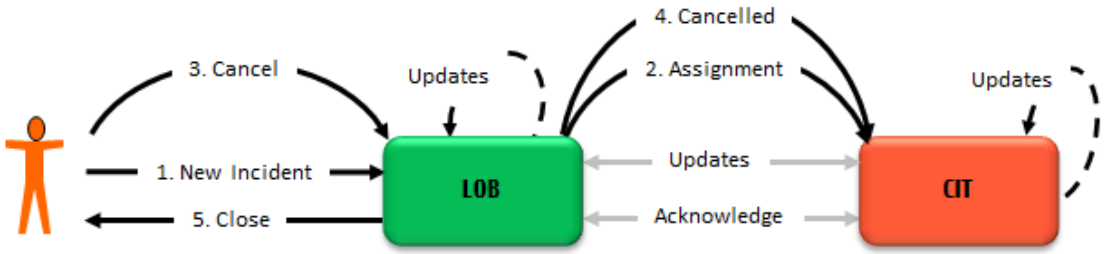
Action	Description
1 New Incident	An end user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
Updates	The Central IT analyzes and updates the Incident. Case Exchange transfers the update to the other side when needed.
3 Solved	The Central IT solves the Incident.
4 Resolve	The Line of Business confirms the solution with the end user for the closure of the Incident.
5 Not satisfied	The end user is not satisfied. The Incident needs to be re-opened or re-created.
6 Not solved	The Line of Business reassigns the Incident to the Central IT. In this case, Case Exchange handles the Incident as follows: • In case the Incident is **Closed** in the Central IT: If the Central IT doesn't support the re-open of an Incident, Case Exchange recreates the Incident. If the Central IT supports the re-open of a Incident, Case Exchange sets the Incident status to **Work in progress**. • In case the Incident is just **Resolved** in the Central IT, Case Exchange sets the status to **Work in progress**.
7 Solved	The Central IT solves the Incident.
8 Closed	The Line of Business confirms the solution with the end user and closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 4

The Line of Business registers and assigns an Incident to the Central IT. However, the Incident is no longer relevant for a customer (cancel and withdrawal).



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. However, the end user indicates that the issue no longer exists and cancels the Incident.

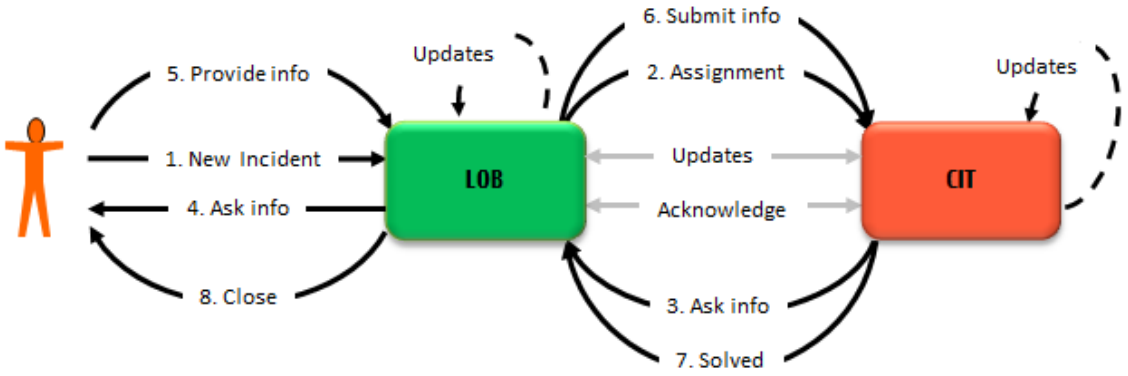
Action	Description
1 New Incident	An end user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
Updates	The Central IT analyzes and updates the Incident. Case Exchange transfers the update to the other side when needed.
3 Cancel	The end user cancels the Incident.
4 Cancelled	The Line of Business cancels the Incident. Once cancelled, the Incident is directly closed.
5 Closed	The Line of Business closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 5

The Line of Business registers and assigns an Incident to the Central IT. The Central IT asks for additional information (Pending Customer).



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assign the Incident to the Central IT. The Central IT requests for missing information and sets the status to **Pending Customer**. The Incident ownership is back to the Line of Business, who must collect the missing information.

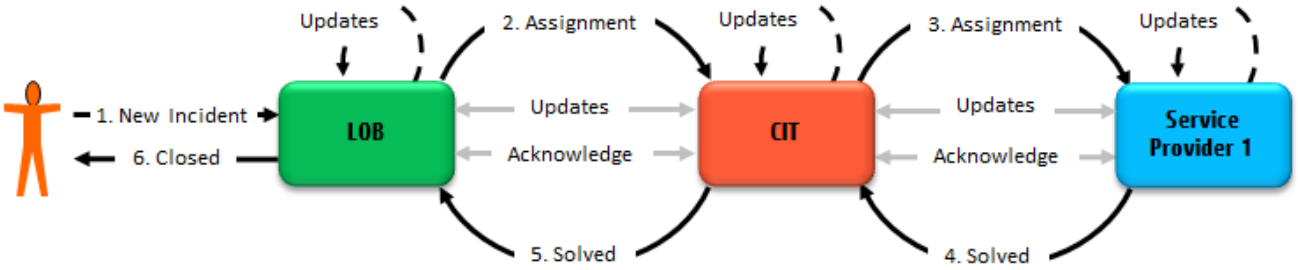
Action	Description
1 New Incident	An end user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
Updates	The Central IT analyzes and updates the Incident. Case Exchange transfers the update to the other side when needed.
3 Ask info	The Central IT assigns the Incident back to the Line of Business to request for additional information. The Incident status in the Central IT's system is **Pending customer**.
4 Ask info	The Line of Business receives the update and contacts the end user.
5 Provide info	The end user provides the information.
6 Submit info	The Line of Business updates the Incident, and then re-assigns the Incident to the Central IT.
Updates	Updates can happen on either side. Case Exchange transfers the update to the other side when needed.
7 Solved	The Central IT solves the Incident.
8 Closed	Service Desk confirms the solution with the end user and closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 6

The Line of Business registers an Incident, which is then assigned to the Central IT. The Central IT then assigns the Incident to Service Provider 1.



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT then assigns the record to Service Provider 1. Service Provider 1 solves the record.

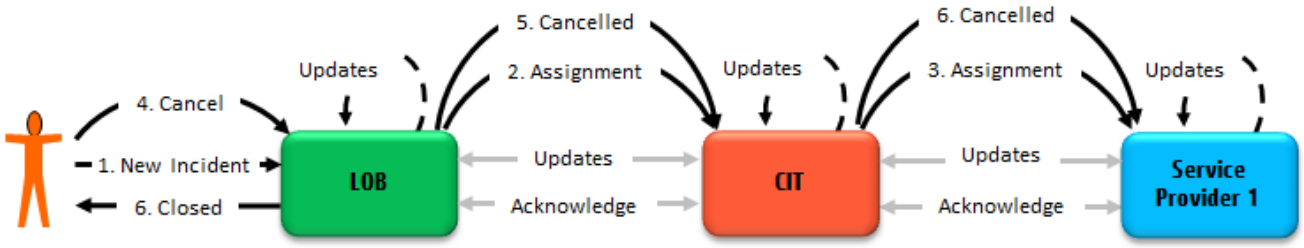
Action	Description
1 New Incident	An end user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
3 Assignment	The Central IT assigns the Incident to Service Provider 1.
Updates	Updates can happen on any side. Case Exchange transfers the update to other sides when needed.
4 Solved	Service Provider 1 solves the Incident. The responsibility is back to the Central IT, who reviews the resolution.
5 Solved	The Central IT passes the Incident back to the Line of Business.
6 Closed	The Line of Business confirms the solution with the end user and closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 7

The Line of Business registers and assigns an Incident to the Central IT. The Central IT then assigns the Incident to Service Provider 1. However, the Incident is no longer relevant for customer.



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT then assigns the Incident to Service Provider 1. However, the end user indicates that the issue no longer exists and cancels the Incident.

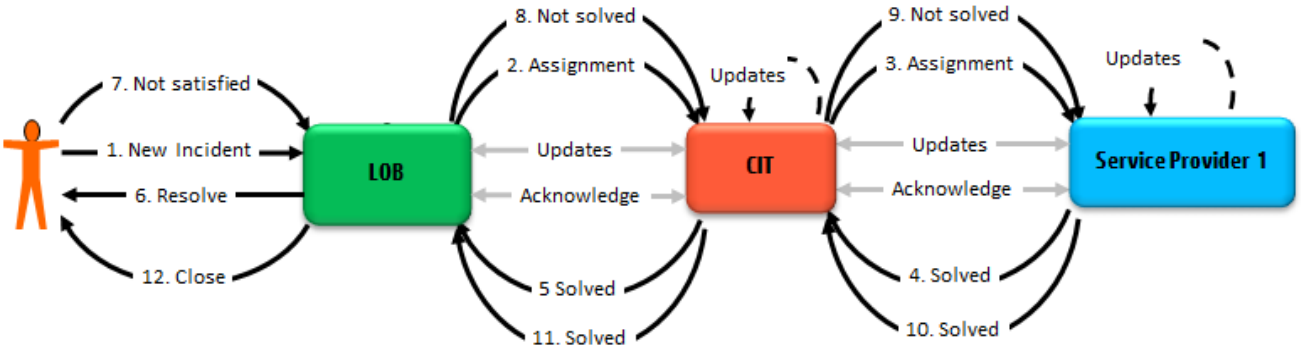
Action	Description
1 New Incident	A user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
3 Assignment	The Central IT assigns the Incident to Service Provider 1.
Updates	Updates can happen on any side. Case Exchange transfers the update to other sides when needed.
4 Cancel	The end user cancels the Incident.
5 Cancelled	The Line of Business sends the cancellation to the Central IT.
6 Cancelled	The Central IT sends the cancellation to Service Provider 1.
7 Closed	The Line of Business closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 8

The Line of Business registers and assigns an Incident to the Central IT. The Central IT assigns the Incident to the Service Provider 1. Service Provider 1 solves the Incident. The solution doesn't satisfy the customer, Service Provider 1 must solve the Incident again.



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT then assigns the Incident to Service Provider 1. Service Provider 1 solves the Incident and the solution is passed to the Line of Business. The Line of Business confirms the solution with the customer, but the customer is not satisfied. Therefore, the Incident is reassigned (re-opened or re-created) to the Service Provider 1.

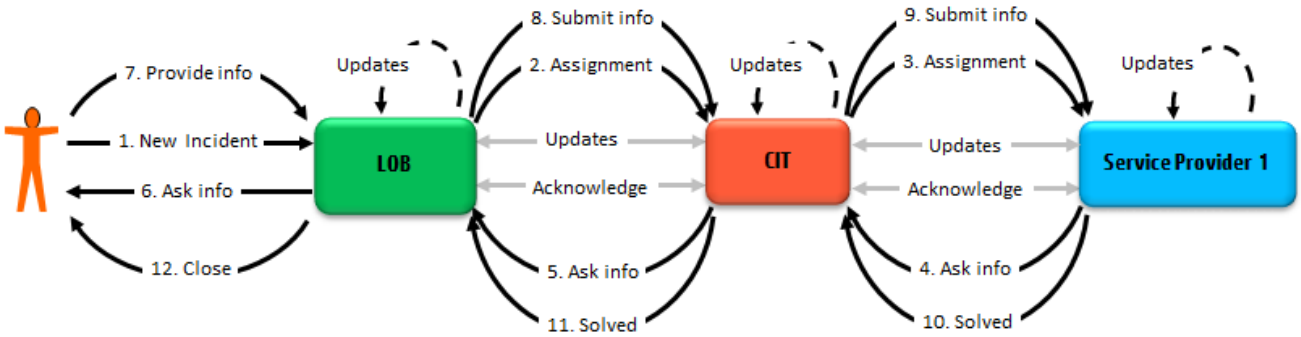
Action	Description
1 New Incident	An end user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
3 Assignment	The Central IT assigns the Incident to Service Provider 1.
Updates	Updates can happen on any side. Case Exchange transfers the update to other sides when needed.
4, 5 Solved	Service Provider 1 solves the Incident, and the solution is passed to the Line of Business.
6 Resolve	The Line of Business confirms the solution with the end user for the closure of the Incident.
7 Not satisfied	The end user is not satisfied.
8, 9 Not solved	The Incident is reassigned to Service Provider 1.
10, 11 Solved	Service Provider 1 solves the Incident, and the solution is passed to the Line of Business.
12 Closed	The Line of Business confirms the solution with the end user and closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 9

The Line of Business registers and assigns an Incident to the Central IT. The Central IT assigns the Incident to Service Provider 1. Service Provider 1 asks for additional information (Pending Customer).



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT assigns the Incident to Service Provider 1. Service Provider 1 requests for missing information and sets the status to **Pending Customer**. The Incident ownership is then back to the Central IT and then back to the Line of Business, who must collect the missing information.

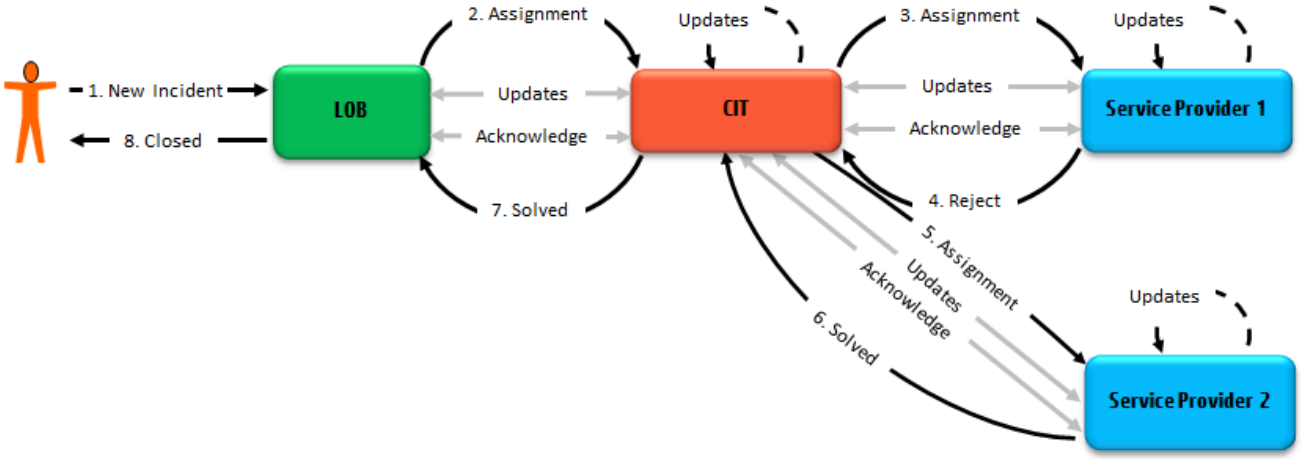
Action	Description
1 New Incident	An end user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
3 Assignment	The Central IT assigns the Incident to Service Provider 1.
4, 5 Ask info	The Service Provider 1 assigns the Incident back to the Central IT to request for additional information. The Incident is passed back to the Line of Business.
6 Ask info	The Line of Business receives the update and contacts the end user.
7 Provide info	The end user provides the information.
8, 9 Submit info	The Incident is updated and re-assigned to Service Provider 1.
Updates	Updates can happen on any side. Case Exchange transfers the update to other sides when needed.
10, 11 Solved	Service Provider 1 solves the Incident, and the solution is passed back to the Line of Business.
12 Closed	The Line of Business confirms the solution with the end user and closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 10

The Line of Business registers and assigns an Incident to the Central IT. The Central IT should assign the Incident to Service Provider 2, but incorrectly assigns the Incident to Service Provider 1.



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT incorrectly assigns the Incident to Service Provider 1, who is not responsible for the Incident and then rejects the Incident. The Central IT then assigns the Incident to Service Provider 2.

Action	Description
1 New Incident	A user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
3 Assignment	The Central IT assigns the Incident to Service Provider 1.
4 Reject	Service Provider 1 rejects the Incident. The Incident ownership is back to the Central IT, who reviews the update from Service Provider 1.
5 Assignment	The Central IT assigns the Incident to Service Provider 2.
6,7 Solved	Service Provider 2 solves the Incident, and the solution is passed to the Line of Business.
Updates	Updates can happen on either side. Case Exchange transfers the update to the other side when needed.
8 Closed	Line of Business confirms the solution with the end user and closes the Incident.

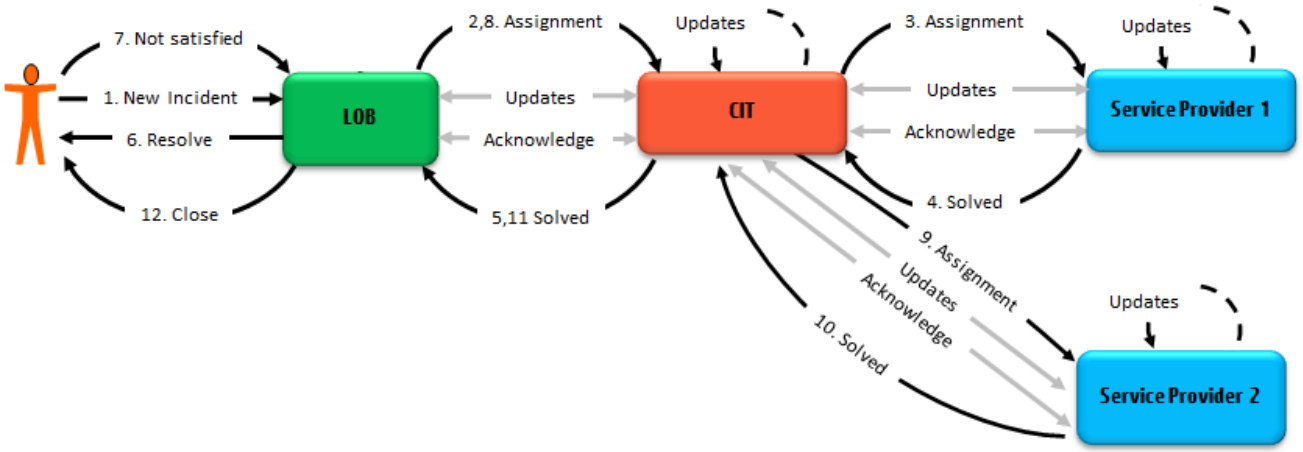
Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 11

The Line of Business registers and assigns an Incident to the Central IT. The Central IT assigns the Incident to Service Provider 1. Service Provider 1 provides an unsatisfactory solution. The

Central IT then assigns the Incident to Service Provider 2.



Scenario description

In this scenario, an end user creates an Incident. The Line of Business registers and assigns the Incident to the Central IT. The Central IT assigns the Incident to Service Provider 1, who then provides a solution that doesn't satisfy the end user. The Central IT then assigns the Incident to Service Provider 2.

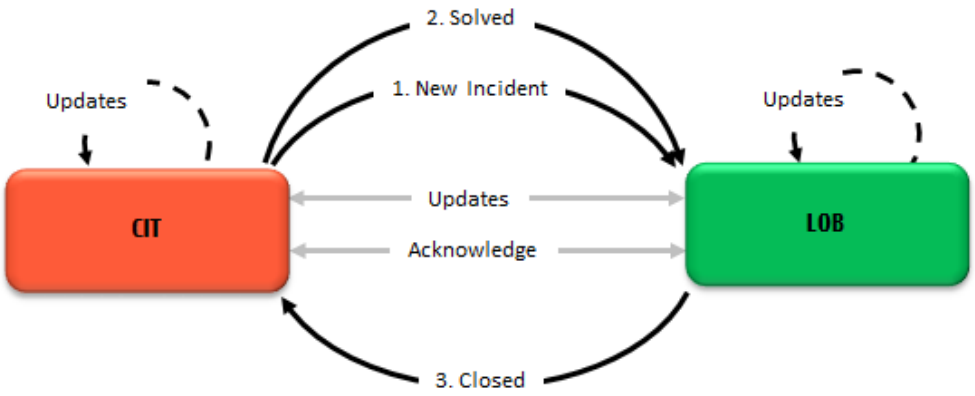
Action	Description
1 New Incident	A user calls the Line of Business to create a new Incident.
2 Assignment	The Line of Business assigns the Incident to the Central IT.
3 Assignment	The Central IT assigns the Incident to Service Provider 1.
Updates	Updates can happen on any side. Case Exchange transfers the update to other sides when needed.
4,5 Solve	Service Provider 1 solves the Incident, and the solution is passed to the Line of Business.
6 Resolve	The Line of Business confirms the solution with the end user for the closure of the Incident.
7 Not satisfied	The end user is not satisfied.
8,9 Assignment	The Incident is assigned to Service Provider 2.
10,11 Solved	Service Provider 2 solves the Incident, and the solution is passed to the Line of Business.
12 Closed	The Line of Business confirms the solution with the end user and closes the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 12

The Central IT detects and solves an Incident, and sends the Incident to the Line of Business.



Scenario description

In this scenario, the Central IT creates and solves an Incident, which affects the Line of Business. The Central IT sends the Incident to the Lind of Business for informational purposes. The Line of Business confirms the resolution of the Incident.

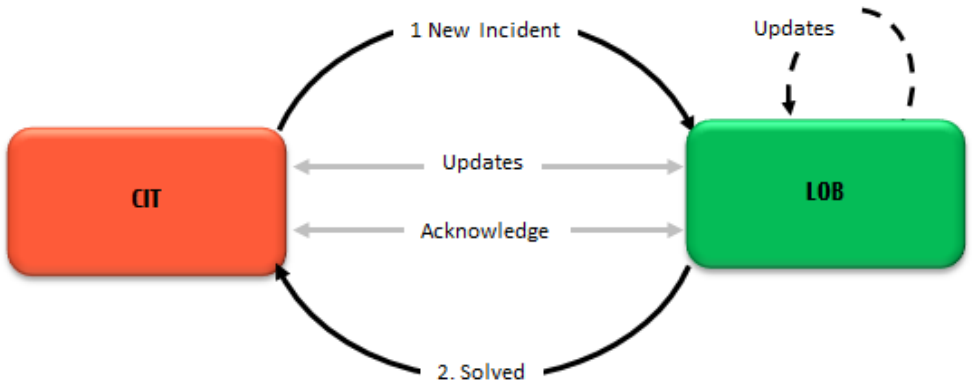
Action	Description
1 New Incident	The Central IT creates a new Incident and forward the Incident to the Line of Business.
2 Solved	The Central IT solves the Incident and sends a `Solved` update to the Line of Business.
3 Closed	Service Desk confirms the solution to close the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 13

The Central IT creates an Incident that must be solved by the Line of Business.



Scenario description

In this scenario, the Central IT creates an Incident, which has to be solved by the Line of Business.

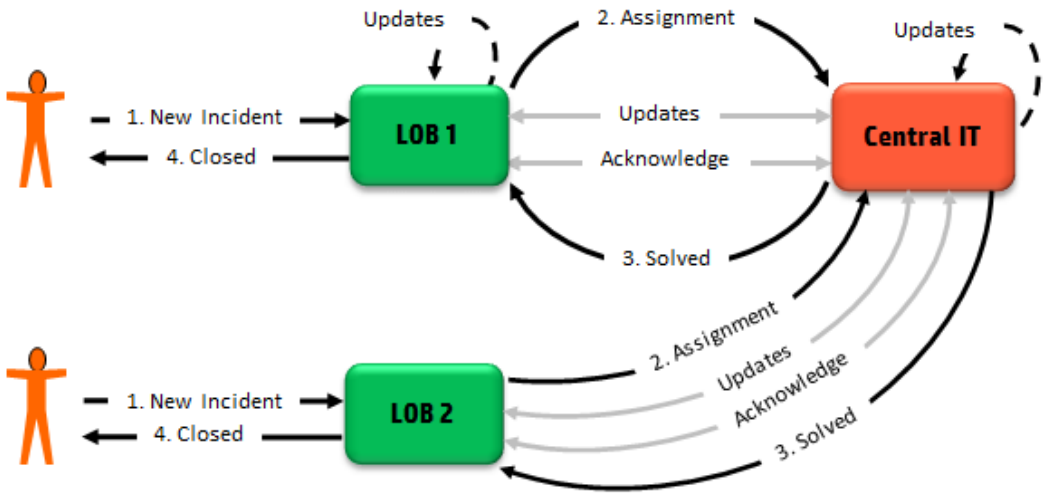
Action	Description
1 New Incident	The Central IT creates a new Incident and assigns the Incident to the Line of Business.
Updates	Updates are transferred between both systems.
2 Solved	The Line of Business solves the Incident.

Background

Action	Description
Updates	Updates in any of the systems are transferred to the other side.
Acknowledge	Each time data is transferred, an Acknowledgment has to happen.

Scenario 14

Two Line of Business systems are connected to the Central IT.



Scenario description

In this scenario, two Line of Business systems connect to the Central IT. The actions in the above scenarios (for example, pending customer, reject, customer dissatisfaction, and so on) are still valid.

1.1.8.4. Common user tasks

This section introduces the common user tasks that apply to the Incident Management module in the Case Exchange integration:

Update an incident

When Service Manager owns an Incident record, you can directly make any update you want. The configuration of the Case Exchange integration determines whether to transfer the update to the endpoint.

When the endpoint system owns a record, the Incident record isn't editable in Service Manager. However, you can still add additional information to the record, and Case Exchange can transfer the information to the record in the endpoint system. To do this, follow these steps:

1. Open the Incident record.
2. Click the **Activities** tab.
3. Choose an appropriate value in the **Update type** field, and then enter some information in the **Update** field.
4. Make sure the **Take back control** checkbox isn't selected.
5. Click **Finish**.

Case Exchange then transfers the information to the record in the endpoint. In Service Manager, you can view the information in the **Journal Updates** field.

Take back an incident

When the endpoint system owns a record, the related Incident record isn't editable in Service Manager. In case the endpoint system doesn't respond or something unforeseen happens, you may want to take back the ownership of the record. To take back the ownership of an Incident record, follow these steps:

1. Open the Incident record.
2. Click the **Activities** tab.
3. Choose an appropriate value in the **Update type** field, and then enter some information in the **Update** field.
4. Select the **Take back control** checkbox.
5. Click **Next**.

The ownership of the record then returns to Service Manager.

Check activity logs of an Incident record

You can view the log of the activities that occurred to an Incident record in two sections:

- The **Activity Logs** tab: This tab displays the log of all the activities that occurred to the Incident record, including the Case Exchange tasks.

- The **Case Exchange** tab: This tab only displays the log of all the Case Exchange tasks. This section is available only when a Case Exchange integration instance is active and the Incident record has a Case Exchange task.

  For more information about the **Case Exchange** tab, see Audit and logging.

To check the activity log of an Incident record, follow these steps:

1. Click **Incident Management** > **Search Incidents**.
2. Use search or advanced search to find one or more records.
3. Click the **Activity Logs** tab and check the activity log.

To check the log of Case Exchange tasks, follow these steps:

1. Click **Incident Management** > **Search Incidents**.
2. Use search or advanced search to find one or more records.
3. Click the **Case Exchange** tab, expand the **Log** subsection, and then check the log of the Case Exchange tasks.

Restart an unsuccessful task

Once a Case Exchange task occurs to an Incident record, the Incident record has a **Case Exchange** section that displays the history of all Case Exchange tasks. You can restart an unsuccessful Case Exchange task from the **Case Exchange** section. To do this, follow these steps:

> **Note**
>
> If you want to restart an unsuccessful Case Exchange task of an Incident record, you must have the Update right on that Incident record.

1. Click **Incident Management** > **Search Incidents**.
2. Use search or advanced search to find one or more records.
3. Select the Incident record.
4. Expand the **Case Exchange** section, and then expand **Log**.
5. Click the link on the date and time link of the unsuccessful Case Exchange task. The **Integration Task Log** page opens.
6. Click the **View** button after the **Task ID** field. The **Integration Failover Task Queue** page opens.
7. Click **Run Now**.

1.1.8.5. System administrator tasks

This section introduces the system administrator tasks in the Case Exchange integration:

Review log file

The standard Service Manager log file contains detailed information about the actions of a Case Exchange integration instance.

When you create the Case Exchange integration instance, you can define the following settings for the Service Manager log file:

- Log Level: Level of diagnostic information that the Service Manager server logs to the log file directory.
- Log File Directory: The location where log files will be stored. If you don't configure this option, the default log for this integration instance is `<Server home directory>/logs/sm.log`.

For more information, see Integration Instance Information fields.

Handle long-running tasks

Follow these steps to identify a "long-running" task:

1. Sign in to Service Manager as a system administrator.
2. Click **Tailoring** > **Integration Manager**.
3. Click **Log**, set the **Status** field to **In Progress**, and then click **Filter**.

If a task remains in the **In Progress** status for a long time, then this task is a "long-running" task.

Follow these steps to handle a "long-running" task:

1. Sign in to Service Manager as a system administrator.
2. Click **Tailoring** > **Integration Manager**.
3. Select the integration instance to which the "long-running" task belongs, and then click **Disable**.
4. Click **Miscellaneous** > **System Status**, and then wait until the scheduler for the integration instance disappears.



Note

You need to click the **Refresh Display** button to see the latest system status.

5. Go back to Integration Manager, select the corresponding integration instance, and then click **Task**.
6. Open the "long-running" task.
7. Perform one of the following actions according to your needs:
    - Click the **Merge** button and then manipulate the data in the task.

      This action only applies to the outbound tasks.
    - Select the **Expired** checkbox.

      In this way, the task will not be processed.
    - Click the **Delete** button to delete the task.

      In this way, you may have to ask the integration partner to resend the record. Be careful when you delete a task, this action may result in data loss.

1.1.8.6. Case Exchange reference material

This section provides the Case Exchange reference material.

Edit the query and pagination query

Specify additional path

Entity path

Technical hints

Dealing with HTML tags from Service Anywhere

Out-of-box functions for additional scripts

Limitations

1.1.8.6.1. Edit the query and pagination query strings

This topic provides specifications to explain how to configure the strings for the query parameter and pagination query parameter under the **Inbound** tab when you set up a Case Exchange integration instance. This topic includes the following collections.

Filter

The *filter* query parameter specifies a filtering condition for a resource collection. The service must return only resources that meet the specified condition. The following table lists the operators.

General Variables

Logical operator	Boolean operator	Description	Example
${*object*}	${*object*}	A placeholder for all parameters in SMIS configuration, in which the value can't be predefined but during running time or depends on other parameters. The *object* placeholder can be *sm*, *ep*, *param*, *context*,or *vars*. For more information about these objects, see Placeholder objects.	${param.lastPullingTime} ${vars.$epfieldsAlias}
Service Manager records	sm	Query the record in Service Manager.	Get the description of an incident in Service Manager: ${sm.action}
End point	ep	Query the record in the end point.	Get the description of an incident in the end point: ${ep['properties.description']}
Parameters	param	Query all parameters defined in the Service Manager configuration.	Get the object defined in the Service Manager configuration: ${param.object}

Logical operator	Boolean operator	Description	Example
Context information	context	Query all task-related information such as internal ID and external ID. For more information, see Placeholder objects.	Get the incident Id in Service Manager: ${context.internalId}
Variables	vars	Query all variables in Service Manager.	Get the current login users: ${vars.$lo_operator}

Special variables for Case Exchange

Logical operator	Boolean operator	Description	Example
Current page offset	$currentPageOffset	Query the index of the first record on the current page in all records. This variable is defined in the Case Exchange framework and is only used by the pagination parameter.	${vars.$currentPageOffset}

Logical Operators

Logical operator	Boolean operator	Description	Example
And	and	Evaluates to true if the left and right operands evaluate to true, otherwise evaluates to false.	http://domain/resource?filter=Id gt 1000 and Name eq 'Jake' http://domain/resource?filter=Id > 1000 and Name = 'Jake'
Or	or	Evaluates to true if at least one of the left and right operands evaluates to true, otherwise evaluates to false.	http://domain/resource?filter=Id gt 1000 or Name eq 'Jane' http://domain/resource?filter=Id > 1000 or Name = 'Jane'

Comparison Operators

Logical operator	Boolean operator	Description	Example
Equals	eq =	Evaluates to true if the left and right operands are equal, otherwise evaluates to false.	http://domain/resource?filter=Rank eq 2 http://domain/resource?filter=Rank = 2
Not Equals	ne !=	Evaluates to true if the left and right operands aren't equal, otherwise evaluates to false.	http://domain/resource?filter=Rank ne 2 http://domain/resource?filter=Rank != 2
Greater Than	gt >	Evaluates to true if the left operand is greater than the right operand, otherwise evaluates to false.	http://domain/resource?filter=Rank gt 2 http://domain/resource?filter=Rank > 2
Greater or Equals	ge >=	Evaluates to true if the left operand is greater than or equals to the right operand, otherwise evaluates to false.	http://domain/resource?filter=Rank ge 2 http://domain/resource?filter=Rank >= 2
Less Than	lt <	Evaluates to true if the left operand is less than the right operand, otherwise evaluates to false.	http://domain/resource?filter=Rank lt 2 http://domain/resource?filter=Rank < 2
Less or Equal	le <=	Evaluates to true if the left operand is less than or equals to the right operand, otherwise evaluates to false.	http://domain/resource?filter=Rank le 2 http://domain/resource?filter=Rank <= 2

Logical operator	Boolean operator	Description	Example
Between	btw	Evaluates to true if the leftmost operand is in the range defined by the two comma-separated operands in the parenthesis, otherwise evaluates to false. The first operand in the parenthesis represents the low limit of the range and the second operand represents the high limit of the range.	http://domain/resource?filter=LastUpdate btw(2012.01.01, 2013.01.01)
Not Between	not btw	Evaluates to true if the leftmost operand is not in the range defined by the two comma-separated operands in the parenthesis, otherwise evaluates to false. The first operand in the parenthesis represents the low limit of the range and the second operand represents the high limit of the range. Range boundaries are inclusive.	http://domain/resource?filter=LastUpdate not btw(2012.01.01, 2013.01.01)

Logical operator	Boolean operator	Description	Example
In	in	Evaluates to true if the leftmost operand is equal to one of the expressions in the parenthesis, otherwise evaluates to false.	http://domain/resource?filter=Id in(1,2,3)
Not In	not in	Evaluates to true if the leftmost operand is not equal to any of the expressions in the parenthesis, otherwise evaluates to false.	http://domain/resource?filter=Id not in(1, 2, 3)

Arithmetic Operators

Logical operator	Boolean operator	Description	Example
Add	add +	Evaluates to the sum of the left operand and the right operand in the parenthesis.	http://domain/resource?filter=PreviousRank gt (CurrentRank add 3) http://domain/resource?filter=PreviousRank > (CurrentRank + 3)
Subtract	sub -	Evaluates to the value that the left operand minus the right operand in the parenthesis.	http://domain/resource?filter=PreviousRank gt (CurrentRank sub 3) http://domain/resource?filter=PreviousRank > (CurrentRank - 3)
Multiply	mul *	Evaluates to the product that the left operand multiplies the right operand in the parenthesis.	http://domain/resource?filter=PreviousRank lt (CurrentRank mul 3) http://domain/resource?filter=PreviousRank < (CurrentRank * 3)

Logical operator	Boolean operator	Description	Example
Modulo	mod %	Evaluates to the remainder that the left operand divides the right operand in the parenthesis.	http://domain/resource?filter=Actual gt Planned and (Planned mod Actual) gt 5 http://domain/resource?filter=Actual > Planned and (Planned % Actual) > 5
Divide	div /	Evaluates to the value that the left operand divides the right operand in the parenthesis.	http://domain/resource?filter=PlannedHours div 10 http://domain/resource?filter=PlannedHours / 10

Parenthesis Operator

Logical operator	Boolean operator	Description	Example
Parenthesis	()	Like in math, parenthesis can be used around conditions and expressions to define priority.	http://domain/resource?filter=(x add y) mul (x sub z) http://domain/resource?filter=(x + y) * (x - z)

Unary Operators

Logical operator	Boolean operator	Description	Example
Unary Plus	+	Syntax: +number	N/A
Unary Minus	-	Syntax: -number	N/A

Layout

The *layout* query parameter specifies which properties or sub-structure of a data resource should be returned by a service. Similar to SQL, this parameter provides a comma-separated list of properties or multipart properties. The following table lists the operators.

Operator	Description	Example
Selecting Resource Properties	Resource properties are specified by a property name directly.	http://domain/resource?layout=Id,Name,Description
Selecting Related Resource Properties	Related resource properties are specified by a relationship name and a property name.	http://domain/Person?layout=PrimaryAddress.City
Selecting a Sub-Structure Using Multipart Properties	Resources that hold complex object structures can be queried for a sub-structure using multi-part property conventions.	Query a resource named “UserSetting” for a “Select” object, which is a sub-structure of a “Filter” object, which is a sub-structure of a “UserSetting” object. http://domain/UserSetting?layout=Settings.Filter.Select

Operator	Description	Example
Selecting Calculated Values	The query protocol supports function and Arithmetic operators.	**Note** Functions listed in examples below have to be supported by concrete services in order to be valid – these are only examples. • Query a “Formula” resource for the product of a property named “X” and a property named “Y”: http://domain/Formula?layout=X mul Y • Query a “Person” resource for “FirstName” and “LastName” and return a concatenation so “FirstName”, space (“ “) and “LastName”: http://domain/Person?layout=Concat(FirstName, Concat(‘ ’, LastName))

Size

The *size* parameter specifies the maximum number of resources requested to be returned.

Syntax: size=<integer>

Example: Fetch the first 10 persons with age lower than 30 in ascending order.

http://domain/Person?layout=FirstName,LastName,Email&filter=Age<30&order=Age asc&size=10

Skip

The *skip* parameter specifies how many resources should be skipped. In other words, it specifies the starting index of the returned result. When not specified, it is assumed to be zero, meaning that the first resource returned from the data-store is the first resource returned by the queried service.

Syntax: skip=<integer>

Example: Fetch persons, starting from person number 51.

http://domain/Person?layout=FirstName,LastName,Email&skip=50

1.1.8.6.2. Specify additional path

In case the outbound JSON request contains a path or element that's mandatory but is undefined in field mapping or entity path, you need to specify this path or element in the **Additional path** field.

For example, in the following scenario:

- An outbound JSON request contains the following content:

```
{
        "entities": [{
                "properties": {
                        "DisplayLabel": "sadf",
                        "Description": "sdf",
                        "ImpactScope": "Enterprise",
                        "Status": "Ready",
                        "Urgency": "TotalLossOfService",
                        "RegisteredForActualService": "10874",
                        "Category": "10707"
                },
                "entity_type": "Incident"
        }],
        "operation": "CREATE"
}
```

- The SMIS field mapping defines all the fields under `properties` : `DisplayLabel` , `Description` , `ImpactScope` , `Status` , `Urgency` , `RegisteredForActualService` , and `Category` .
- Field mapping and entity path don't define `entity_type` and `operation` .

In this scenario, you must define the `entity_type` and `operation` fields in the **Additional path** field by using the following string:

```
{"entities":[{"properties":{},
"entity_type":"Incident"}],"operation":"CREATE"}
```



Note

RESTful protocols other than JSON are *not* supported.

1.1.8.6.3. Entity path

In an outbound JSON request, entity path is the path above the endpoint field defined in the field mapping.

For example, in a Case Exchange integration instance, all endpoint fields are defined as `properties.****` in the field mapping. The following example is an outbound request body in the endpoint:

```
{
        "entities": [{
                "properties": {
                        "DisplayLabel": "Create New Incident",
                        "Description": "create the new incident
record",
                        "ImpactScope": "MultipleUsers",
                        "Urgency": "SlightDisruption",
                        "Solution": "",
                        "Id": "IM118216"
                },
                "ext_properties": {
                        "Operation": "Update",
                        "ExternalStatus": "Open",
                        "ExternalId": "IM45167",
                        "ExternalSystem": "SM",
                        "ExternalEntityType": "Incident"
                },
                "Comments": [{
                        "Body": "IM45167",
                        "CreatedTime": 1402038076398
                }],
                "entity_type": "Incident"
        }]
}
```

In this particular example, one of the endpoint fields is `properties.Displaylabel`, so the entity path is `entities[0]`.



Note

RESTful protocols other than JSON are *not* supported.

1.1.8.6.4. Technical hints

This section provides technical information that helps you better understand Case Exchange.

The `$G.CEOwnershipSM` variable

The `$G.CEOwnershipSM` variable is proprietary for Case Exchange. This variable identifies whether the native system has ownership of a record.

If Process Designer is not implemented, this variable is set in the **im.view.init** process record, and used in the **I/O (If RIO)** field of the **apm.edit.problem** display screen record.

If Process Designer is implemented, this variable is set in the **im.CEUpdate.init** Rule Set, which is injected to the **Incident** workflow under **Workflow Based Rule Sets** > **On display**.

Functions used in rule conditions

The example condition configurations in this document call the following functions:

CaseExchangeExternalReferencesDAO.getExternalID: This function judges if the record is exchanged to the external system. If the record is exchanged to the external system, the following expression returns true:

```
jscall("CaseExchangeExternalReferencesDAO.getExternalID",number in
$L.file) ~=""
```

CaseExchangeExternalReferencesDAO.isExternalActive: If a record is in an active Case Exchange life cycle, this function returns TRUE; otherwise, this function returns FALSE.

Time difference issue

In the Pull mechanism, date and time of the last Pull activity is the conclusive query filter to distinguish the new updates in the endpoint system, which will be retrieved in the next Pull activity.

Even the same time zone is applied for the integration, the time difference can occur between the two systems (for example, one system is one minute ahead of another system). Therefore, the system that initiates the Pull request must compensate the time difference between the two systems so that each Pull activity can retrieve the exact updates after the last Pull activity from the endpoint system.

To achieve this goal, you can configure the **Time difference (s)** setting in the integration instance.

Note

For each Pull activity, the time for the query filter is the time in the **Pulling From** field plus the time (in seconds) defined in the **Time difference (s)** field.

Contents

- 1 The $G.CEOwnershipSM variable
- 2 Functions used in rule conditions
- 3 Time difference issue

1.1.8.6.5. Dealing with HTML tags from SMAX

SMAX uses HTML tags for rich text in the **Description**, **Solution**, and **Discussion** fields of an Incident, for example, `<p>`, `<strong>`, `<ul>`, and so on. By default, Service Manager doesn't parse these tags, but displays these tags directly. To improve the readability of these fields in Service Manager, the out-of-box integration template (**CaseExchangeSM_SAW**) uses pre script to handle these tags in the following manner:

Inbound exchange:

- Automatically removes HTML tags.
- If the content contains an image and the address of the image doesn't start with "http://" or "https://", the image is downloaded and stored as an attachment, and the following description is appended:

  ```
  And Images are referenced as:
  (1 embedded image(s) <i><image_name></i> were downloaded as
  attachment, please check attachment section)
  ```

  If the download of an image fails, the following description is appended :

  ```
  And Images are referenced as:
  (1 embedded image(s) <i><image_name></i> were downloaded failed)
  ```

- If the content contains an image and the address of the image starts with "http://" or "https://", the image is not downloaded and the following description is appended:

  ```
  And Images are referenced as:
  (Images http://<i><domain_name_or_IP_address></i>/<i><image_name>
  </i> are from internet)
  ```

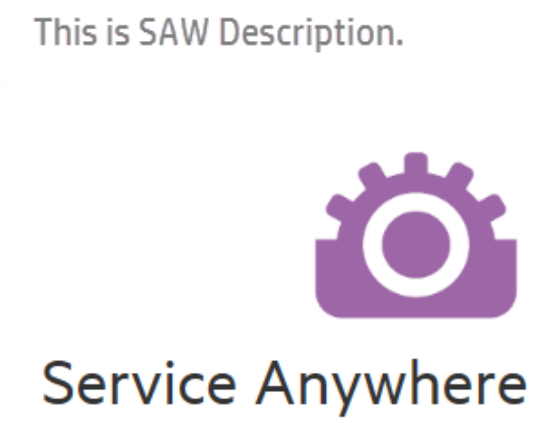
Outbound exchange:

If the content of a field is modified in Service Manager, then all the text in this field will be appended after the original content in SMAX as follows:

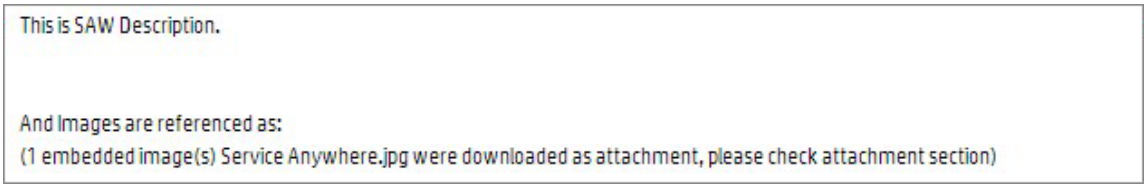
```
This is the original text in Service Anywhere.
#####SM START#####
This is the modified text in Service Manager.
#####SM END#####
```

The following example illustrates how this mechanism works in each Case Exchange transfer:

1. The **Description** field of an Incident has the following content in SMAX.



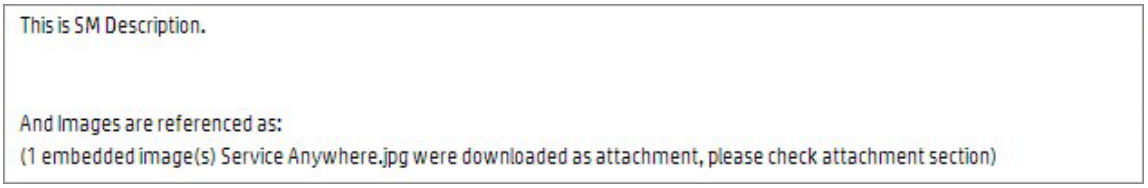
2. After the Incident is exchanged, the Description field has the following content in Service Manager.



And you can find the image in the **Attachments** section.



3. You modify this field in Service Manager as follows.



4. After your update is exchanged to SMAX, the **Description** field in SMAX contains the following information.

This is SAW Description.



Service Anywhere

SM START
This is SM Description.

And Images are referenced as:
(1 embedded image(s) Service Anywhere.jpg were downloaded as attachment, please check attachment section)
SM END

If you want to remove or modify this mechanism from the out-of-box integration template, remove or modify the following code in pre script:

```
//remove html tags from SAW
    if(ep['properties.Description']) ep['properties.Description'] =
lib.CaseExchange_SAWUtil.processHtmlText(param, context, 'description',
ep['properties.Description']);
    if(ep['properties.Solution']) ep['properties.Solution'] =
lib.CaseExchange_SAWUtil.processHtmlText(param, context, 'solution',
ep['properties.Solution']);
    if (ep['Comments'] && ep['Comments'].length > 0) {
        for (var i = 0; i < ep['Comments'].length; i++) {
            ep['Comments'][i]['Body'] =
lib.CaseExchange_SAWUtil.processHtmlText(param, context, 'comment',
ep['Comments'][i]['Body']);
        }
    }

//process the value which will be posted to SAW rich text field

if(sm['action'] && sm['action'] !== "") { if(sm['oldvalues']['action'])
sm['action'] = lib.CaseExchange_SAWUtil.processOutboundText(context,
'description', sm['action']); if(sm['oldvalues']['action'] ===
undefined) sm['action'] = null; } if(sm['resolution'] &&
sm['resolution'] !== "") { if(sm['oldvalues']['resolution'])
sm['resolution'] =
lib.CaseExchange_SAWUtil.processOutboundText(context, 'solution',
sm['resolution']); if(sm['oldvalues']['resolution'] === undefined)
sm['resolution'] = null; }
```

1.1.8.6.6. Out-of-box functions for additional scripts

When you set up the Case Exchange integration, you can specify functions in the **Additional Script** tab in the **Integration Instance Parameters** page to customize the Case Exchange integration in various different environments. This section explains the out-of-box functions that you can specify in the **Additional Script** tab.

Set the HTTP header - for authentication

Besides `username` and `password`, the endpoint system (SMAX) also requires `cookie` in the request header to perform authentication. The Javascript function below has the following purposes:

- Set `cookie` to the header for inbound requests
- Set additional 'Content-type' into the header for outbound requests

Out-of-box function:

CaseExchange_SAWUtil.getHttpHeader

The function has the following parameters:

- `direction`: The direction of the action. This parameter has only two values:
  - `lib.smis_Constants.MAPPING_DIRECTION_LEFTRIGHT()`
  - `lib.smis_Constants.MAPPING_DIRECTION_RIGHTLEFT()`

  You can call the `lib.CaseExchange_CommonLib.isOutbound(direction)` or `lib.CaseExchange_CommonLib.isInbound(direction)` function to judge if the action is inbound or outbound.
- `param`: All parameters defined in the SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax. If you change any value in this parameter, the change is saved to the SMIS configuration.

The function returns the following result:

Object: The HTTP Header object

Sign in to the external system

Besides user name and password, the endpoint system (SMAX) also requires a cookie in the request header to perform authentication. This cookie comes from another URL of a Software as a Service (SAAS) portal.

The function below performs the following tasks:

1. Sign in to the SAAS portal by using the username and password

2. Extract the cookie from the response
3. Set the cookie to `param`.

The Case Exchange framework then saves the cookie to the SMIS configuration. The succeeding calls can directly retrieve the cookie from the SMIS configuration until the cookie expires.

Out-of-box function:

CaseExchange_SAWUtil.doLoginRequest

The function has the following parameters:

- `param`: All parameters defined in the SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax. If you change any value in this parameter, the change is saved to the SMIS configuration.

The function returns the following result:

N/A

Validate inbound response

Besides the HTTP code in the GET REST response from SMAX, the HTTP body also contains the information of whether the current request is successful. The function below checks the response body to validate this request.

Out-of-box function:

CaseExchange_SAWUtil.validateInboundResponse

The function has the following parameters:

- `param`: All parameters defined in the SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax. The change to any value in this parameter is not saved to the SMIS configuration.
- `response`: The HTTP body of the REST response from endpoint.

The function returns the following result:

Boolean or String:

- `false` (Boolean): if the request is invalidated
- Id of the record in the endpoint (String): if the request is validated

Validate and parse outbound response

Besides the HTTP code in the POST REST response from SMAX, the HTTP body also contains information of whether the current request is successful. The function below checks the response body to validate this request and extract the Id if the request is validated.

Out-of-box function:

CaseExchange_SAWUtil.validateOutboundResponse

The function has the following parameters:

- `param` : All parameters defined in the SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax. The change to any value in this parameter is not saved to the SMIS configuration.
- `response` : The HTTP body of the REST response from endpoint.

The function returns the following result:

Object: The HTTP Header object

Inbound post-processing activities

The function below retrieves comments of an Incident record from SMAX and insert these comments into the activity log in Service Manager right after the Incident record is exchanged.

Out-of-box function:

CaseExchange_SAW_PostProcessing.postInbound

The function has the following parameters:

- `mapObj` : Final Service Manager value object after field mapping and value mapping.
- `sm` : Temporary Service Manager value object after copying basic data from end point according to field mapping.
- `ep` : Original value object from the end point.
- `context` : Context of the SMIS task. You can get a specific value by using the `context['internalId']` syntax.
- `param` : All parameters defined in SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax.

For more information about these parameters, see Placeholder objects.

The function returns the following result:

N/A

Outbound post-processing activities

The function below sends the activity log of the current Incident record to SMAX right after the Incident record is exchanged.

Out-of-box function:

CaseExchange_SAW_PostProcessing.postOutbound

The function has the following parameters:

- `mapObj` : Final Service Manager value object after field mapping and value mapping.

- `sm` : Temporary Service Manager value object after copying basic data from end point according to field mapping.
- `ep` : Original value object from the end point.
- `context` : Context of the SMIS task. You can get a specific value by using the `context['internalId']` syntax.
- `param` : All parameters defined in SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax.

For more information about these parameters, see Placeholder objects.

The function returns the following result:

N/A

Set the HTTP header - for attachment

Besides `cookie` in the HTTP header, the endpoint system (SMAX) also requires `fs_filename` to perform attachment exchange. The function below adds `cookie` and `fs_filename` into the HTTP header.

Out-of-box function:

CaseExchange_SAWUtil.getAttachHttpHeader

The function has the following parameters:

- `direction` : The direction of the action. This parameter has only two values:
  - `lib.smis_Constants.MAPPING_DIRECTION_LEFTRIGHT()`
  - `lib.smis_Constants.MAPPING_DIRECTION_RIGHTLEFT()`

  You can call the `lib.CaseExchange_CommonLib.isOutbound(direction)` or `lib.CaseExchange_CommonLib.isInbound(direction)` function to judge if the action is inbound or outbound.
- `param` : All parameters defined in the SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax. If you change any value in this parameter, the change is saved to the SMIS configuration.
- `attachObj` : For inbound activities, this is an instance of `CEAttachment` . For outbound activities, this is an instance of the `Attachment` object in Service Manager.

The function returns the following result:

Object: The HTTP Header object

Retrieve and parse attachment info

The function below performs the following tasks:

- Retrieves attachment information from the endpoint, such as file name, extension, size, and mime_type.

- Set all this information to the `CEAttachmentInfor` object and return the `CEAttachmentInfor` object in an array.

For SMAX, all attachment information comes from the REST response of Incident. For other endpoints, an additional REST request may be required to retrieve the information.

Out-of-box function:

CaseExchange_SAWUtil.getAttachmentInfor

The function has the following parameters:

- `mapObj` : Final Service Manager value object after field mapping and value mapping.
- `sm` : Temporary Service Manager value object after copying basic data from endpoint according to field mapping.
- `ep` : Original value object from the endpoint.
- `context` : Context of the SMIS task. You can get a specific value by using the `context['internalId']` syntax.
- `param` : All parameters defined in SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax.

For more information about these parameters, see Placeholder objects.

The function returns the following result:

Array: Array of the `CEAttachmentInfo` object

Parse response of attachment creation

Because the endpoint system (SMAX) needs an additional REST request to update attachment information after the creation of the attachment, the function below parses the endpoint response to the outbound attachment creation request.

Out-of-box function:

CaseExchange_SAWUtil.parseSAWCreateAttachResponse

The function has the following parameters:

- `response` : The HTTP body of the REST response from the endpoint.
- `action` : Action type to process attachment. Currently this parameter has only two values: `FETCH_ATTACHMENT` and `CREATE_ATTACHMENT` .
- `param` : All parameters defined in SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax.

The function returns the following result:

CEAttachmentInfo: An instance of `CEAttachmentInfo` parsed from the response

Update outbound Attachment Info

The function below calls the REST request to update attachment information. Before sending the information to the endpoint system (SMAX), new information should be merged to the existing one.

Out-of-box function:

CaseExchange_SAWUtil.updateAttachmentInfo

The function has the following parameters:

- `mapObj` : Final Service Manager value object after field mapping and value mapping.
- `sm` : Temporary Service Manager value object after copying basic data from endpoint according to field mapping.
- `ep` : Original value object from the endpoint.
- `context` : Context of the SMIS task. You can get a specific value by using the `context['internalId']` syntax.
- `param` : All parameters defined in SMIS configuration. You can get a specific parameter value by using the `param['object']` syntax.

For more information about these parameters, see Placeholder objects.

The function returns the following result:

Array: Array of the `CEAttachmentInfo` object

1.1.8.6.7. Limitations and known issues

This section introduces the limitations and known issues of the Case Exchange integration.

Insufficient message for a failed Case Exchange task

The SMIS task log doesn't contain sufficient information for a failed Case Exchange task (QCCR1E114322)

When a Case Exchange task fails, the SMIS task log doesn't contain sufficient information for you to troubleshoot the root cause of the failure.

Workaround

Use the following workaround if the SMIS task log doesn't contain sufficient information:

> **Note**
>
> Remember to adjust the **Log Level** setting in the SMIS integration instance before you debug the error. For more information about how to configure this setting, see Integration Instance Information fields.

- For an inbound task, the native system internally handles the exchange request from the endpoint system. In this scenario, you can find more information by using one of the following methods:
  - Check the latest message in the **sm.log** log file. For more information about how to view the **sm.log** file, see Review log file.
  - Run the failed task again and then check the messages in the message history box.
- For an outbound task, the native system sends the exchange request to the endpoint system and the endpoint system handles the request through RESTful APIs. In this scenario, you need a third-party tool (for example, SoapUI or Postman) to simulate and debug the exchange request.

  You need to collect the following information from the **Task Log Detail** page of the failed task to simulate and debug the exchange request in the third-party tool:
  - The request URL. You can find this URL in the latest message in the **Journal Messages** field.
  - The request data in the **Task Request Data** field.

  For more information about how to access the **Task Log Detail** page, see Monitor SMIS task log.

1.1.9. Computer Telephony Integration (CTI) with the Web client

The Service Manager web client can integrate with a Computer Telephony Integration (CTI) application in two ways: inbound and outbound. Inbound CTI can automate the creation of interactions, whereas outbound CTI can automate dialing.

The web client inbound CTI implementation provides the following method. To support inbound CTI, the web tier URL must have `?telephonyuser=1` appended.

Method	Technology	Supported functions	Supported browsers
New method	Uses native HTML and JavaScript instead of the CTI applet. Only Service Manager 9.50 or later supports this method. This method doesn't require the installation of any CTI support code the first time users connect using the telephony URL. For details, see Use the new CTI method.	• System event • Transact	Firefox, Chrome, and Edge

Important

Oracle has announced that the existing Applet support in Java SE 8 will continue through March 2019, after which it's subject to removal at any time. For more information, see the **Java Client Roadmap Update** white paper. By following the overall approach, it's highly advisable that the customers adopt the new method. You must upgrade both the Service Manager platform and the applications to version 9.50 or later.

Meanwhile, the web client outbound CTI implementation provides the following method:

Method	Technology	Supported functions	Supported browsers
URI protocols	Uses the URI (Uniform Resource Identifier) protocols to start telephone calls or dialogues. This method doesn't rely on the browser's ability to support Java applet technology. You must install a telephony client locally on your computer to handle and manage the calls or dialogues. For details, see Use URI protocols.	Transact	Firefox, Chrome, and Edge

1.1.9.1. Use the new CTI method

You are recommended to use the new Computer Telephony Integration (CTI) method instead of the legacy one, for the following reasons:

- The new method uses native HTML and JavaScript instead of the CTI applet, and therefore eliminates the dependency on your browser's ability to support Java applet technology.
- The new method can automate not only the creation and update of interactions and incidents, but also the action of a toolbar button or a More menu option in an active record detail form.

For more information about the two methods, see Computer Telephony Integration (CTI) with the Web client.

Click the following links to learn more about the new CTI method:

How the new CTI method works

How to use the new CTI method

How to test the new CTI method

1.1.9.1.1. How the new CTI method works

The new CTI implementation enables Service Manager to receive requests from the CTI application as system events. The out-of-box system events are described in the following table.



Tip

To access the system events, enter **db** in the Service Manager command line and press Enter to open the Database Manager, and then enter **SystemEvents** in the Table field and then click **Search**.

System event	Call application	Call script	Description
EditIncident	script.execute.event	EditIncident	Edit the specified incident
EditInteraction	script.execute.event	EditInteraction	Edit the specified interaction
ReceiveIncident	script.execute.event	ReceiveIncident	Create a new incident
ReceiveInteraction	script.execute.event	ReceiveInteraction	Create a new interaction
ReceiveInteractionStreamlined	script.execute.event	ReceiveInteractionStreamlined	Create a new streamlined interaction
ReceiveInteractionList	script.execute.event	ReceiveInteractionList	Search interactions based on the specified criteria

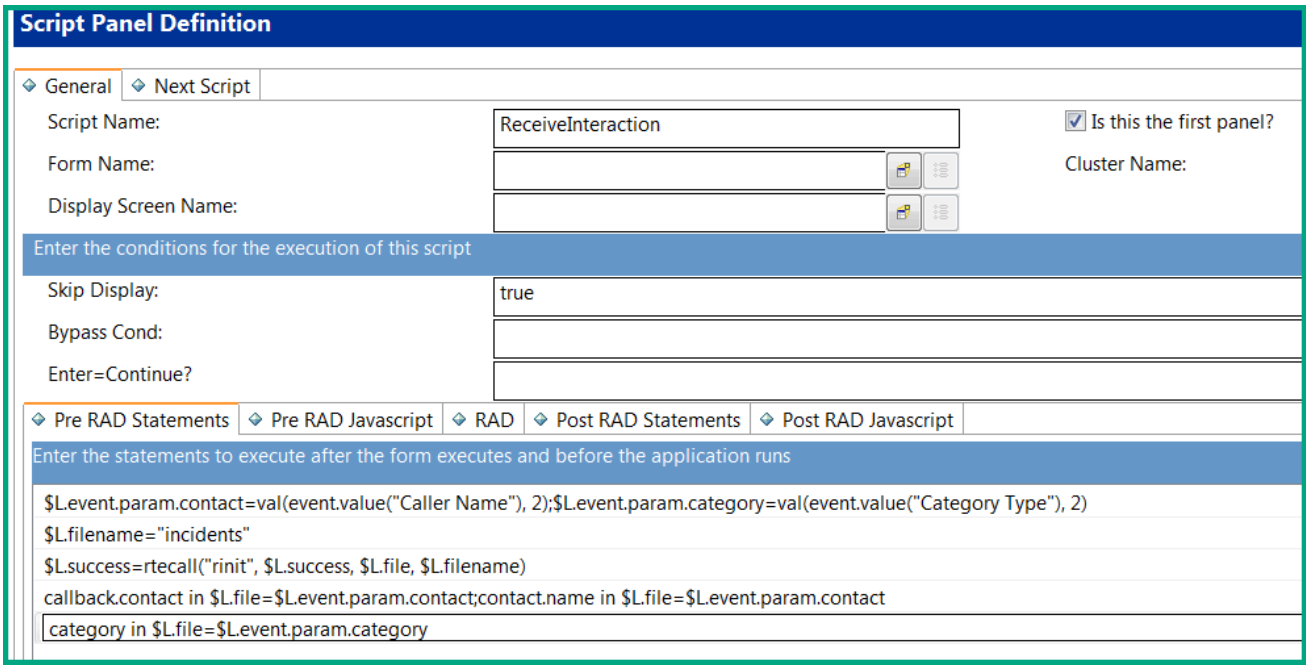
Each of these system events calls the script.execute.event RAD application, which calls a same-name script to fulfill the system event. These scripts are defined in the **scripts** table.



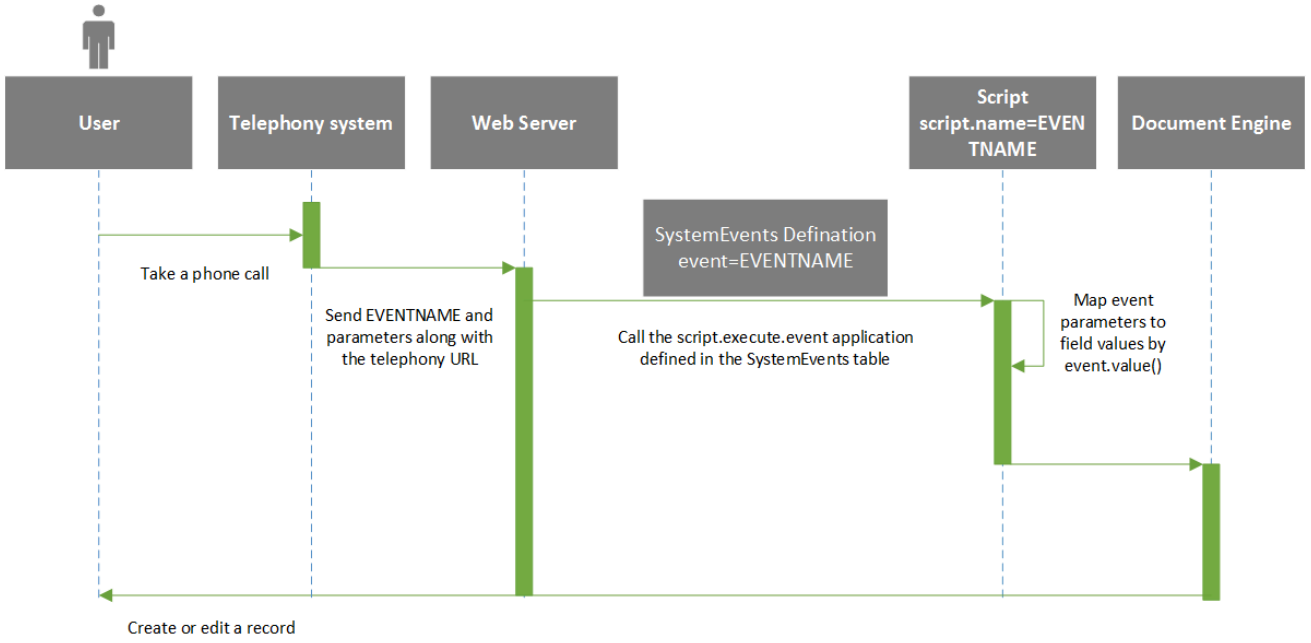
Tip

To access these scripts, enter **scripts** in the Service Manager command line, and then press Enter. To access a specific script, in the Script Name field, enter the script name (for example, **ReceiveInteraction**), and then click **Search**.

Each of these scripts defines the mapping between parameters and fields in the Interaction or Incident dbdict (**incidents** or **probsummary**). The following figure shows the mapping defined in the ReceiveInteraction script. For example, the **Caller Name** parameter is mapped to the **contact.name** field in the **incidents** dbdict.



The following diagram shows how the new CTI method works.



The working process is as follows:

1. The user is logged in to the Service Manager web tier client with the telephony URL. For example:

   ```
   http://smwebtierhost:8080/webtier-9.80/index.do?telephonyuser=1
   ```

> **Note**
>
> To take a telephony call, the user must be logged in to the Service Manager web client. Otherwise, the call is ignored.

2. The user's client system takes a telephone call.
3. The CTI application is triggered to call a VBA function (for example, CreateInteraction). The parameters defined in the function are sent along with the web tier telephony URL.

   The URL uses the following syntax:

   ```
   http://<SM web tier host>:<port>/<SM web tier name>/telephony.html?
   SystemEvent="EVENTNAME","Parameter1","Parameter1
   Value","Parameter2","Parameter2 Value", … "Parameterx", "Parameterx
   Value"
   ```

   For example:

   ```
   http://smwebtierhost:8080/webtier-9.80/telephony.html?
   SystemEvent="ReceiveInteraction","Caller Name","AGENT
   SERVICEDESK","Category Type","incident"
   ```

4. Service Manager receives the request as a system event (for example, a system event named ReceiveInteraction).
5. The web server calls the script.execute.event RAD application, which calls the script with the same name of the system event (for example, ReceiveInteraction).
6. The Document Engine creates a new record or edits an existing record based on the parameter-field mapping defined in the script.

> **Tip**
>
> For information about how to use and test this method, see How to use the new CTI method and How to test the new CTI method.

1.1.9.1.2. How to use the new CTI method

The new CTI implementation requires a system administrator to configure the CTI application to call a system event before each client system can take telephony calls.

The new CTI method also requires users to append **?telephonyuser=1** to the web tier client URL. When users access the telephony URL from a browser that supports Java applet technology, JVM is started to check the Java version installed on the client system. This checking is not needed for the new CTI method, which doesn't use the CTI applet. To skip this unnecessary checking, a system administrator must disable the legacy CTI method when using the new method.

Each client system doesn't require any configurations except that a browser setting must be enabled when Firefox is used as the browser.

The following describes the detailed steps for a system administrator and client users.

Disable the legacy CTI method

Perform the following steps:

1. In the Service Manager web tier's webtier.properties file, set the **enableLegacyTelephony** parameter to **false**:

   ```
   enableLegacyTelephony=false
   ```

   Note

   If this parameter is not specified, the default value (true) is used.

2. Restart the web tier application server.

(Optional) Edit or create system events

The new CTI implementation uses system events to create or edit records in Service Manager. Optionally, a system administrator can edit the out-of-box system events or create new ones to better suit your needs. For information about the out-of-box events, see How the new CTI method works.

You can edit the parameters in an out-of-box system event by updating the same-name script. To do this, follow these steps:

1. Enter **scripts** in the Service Manager command line, and then press Enter.
2. In the Script Name field, enter the script name (for example, **ReceiveInteraction**), and then click **Search**.

3. Click the **Pre RAD Statements** tab, and then update the parameters and map them to specific fields in the incidents or probsummary dbdict as appropriate.

You can also add new system events. To do this, follow these steps:

1. Enter **db** in the Service Manager command line, and press Enter. The Database Manager opens.
2. In the Table field, enter **SystemEvents**.
3. Click **Search**.
4. Select an existing record, update the name, and then click **Add**.
5. Save the new system event.
6. Enter **scripts** in the Service Manager command line, and then press Enter.
7. In the Script Name field, enter the name of the system event that you created, and then click **Add**.
8. On the **Pre RAD Statements** tab, define your parameters and map them to appropriate fields in the corresponding dbdict.
9. Save the script.

Next, you need to configure the CTI application to call a function that triggers a desired system event.

Configure the CTI application

A system administrator can perform the following tasks to configure the CTI application.

Configure the CTI application to create or edit a record

To create or edit a record (for example, create a new interaction) with the new method, a system administrator must configure the CTI application by adding the following functions to the telephony call VBA script:

```
Public Sub SendSystemEvent(params, browser)
baseUrl = "http://smwebtierhost:8080/sm/telephony.html?SystemEvent="
url = baseUrl & params
url = CreateObject("htmlfile").parentWindow.encodeURI(url)
If browser = "chrome" Then
OpenChrome (url)
Else
If browser = "firefox" Then
OpenFirefox (url)
Else
If browser = "edge" Then
OpenEdge (url)
Else
OpenIE (url)
End If
End If
End If
'MsgBox "Creates an Interaction"
Exit Sub
error_handler:
Debug.Print Err.Description
End Sub
Public Function OpenIE(url)
Set IE = CreateObject("InternetExplorer.Application")
IE.Visible = True
IE.Navigate (url)
End Function
Public Function OpenChrome(url)
Shell ("C:\Program Files\Google\Chrome\Application\chrome.exe" & " " &
url)
End Function
Public Function OpenFirefox(url)
Shell ("C:\Program Files\Mozilla Firefox\firefox.exe" & " " & url)
End Function
Public Function OpenEdge(url)
Set wshShell = CreateObject("WScript.Shell")
wshShell.Run Chr(34) & "C:\Windows\System32\cmd.exe" & Chr(34) & " /c
start microsoft-edge:" & url, 0
End Function
Sub ReceiveInteraction()
params = """ReceiveInteraction"",""Caller Name"",""AGENT,
SERVICEDESK"",""Category Type"",""incident"""
```

```
Call SendSystemEvent(params, "edge")
End Sub
```

Where you need to configure the following parameter values:

- Update the **baseUrl** parameter to http://<your web tier host>:<port>/<web tier name>.

  Example: http://smwebtierhost:8080/webtier-9.80

- At the end of the script above, **ReceiveInteraction** represents the out-of-box system event with the same name. You can replace this part with another system event and its parameters. For example, you can replace it with any of the following out-of-box system events:

  ```
  Sub ReceiveInteractionList()
  params = """ReceiveInteractionList"",""Caller Name"",""CAFFREY,
  AARON"",""Open Value"",""Assign"""
  Call SendSystemEvent(params, "chrome")
  End Sub
  Sub ReceiveInteractionStreamlined()
  params = """ReceiveInteractionStreamlined"",""Caller
  Name"",""CAFFREY, AARON"",""Category Type"",""incident"""
  Call SendSystemEvent(params, "chrome")
  End Sub
  Sub EditInteraction()
  params = """EditInteraction"",""Number"",""SD10002"""
  Call SendSystemEvent(params, "chrome")
  End Sub
  Sub ReceiveIncident()
  params = """ReceiveIncident"",""Caller Name"",""CAFFREY, AARON"""
  Call SendSystemEvent(params, "chrome")
  End Sub
  Sub EditIncident()
  params = """EditIncident"",""Number"",""IM10002"""
  Call SendSystemEvent(params, "chrome")
  End Sub
  ```

  In addition to the parameters defined in the system event script, you also need to specify a browser to send the system event: specify "chrome" for Chrome, "firefox" for Firefox, or "edge" for Edge.

- For Chrome, Firefox, or Edge, you need to update the browser installation path to your own in the **OpenChrome(url)** function, the **OpenFirefox(url)** function, or the **OpenEdge(url)** function, repectively.

> **Note**
>
> This configuration requires that all telephony web clients use the same browser and the same browser installation path. If this is not the case, you need to configure the CTI application to allow each user to specify their specific browser and browser installation path.

Configure the CTI application to execute a transaction

The new CTI method also enables the CTI application to execute the Transact capability, which performs the action of a toolbar button or **More** menu option on a record detail form, such as an Interaction or Incident record detail form.

To do this, call the following sub function:

```
Sub ButtonTransferCallFromCTI()
params = """Transact"",""3"""
Call SendSystemEvent(params, "chrome")
End Sub
```

Where:

- The Transact parameter value ("3" in the example above) represents the option ID of a button or menu option. You need to replace it with the option ID of your desired toolbar button or **More** menu option. For example, on the Interaction form, the **Back** button has an option ID of "3".

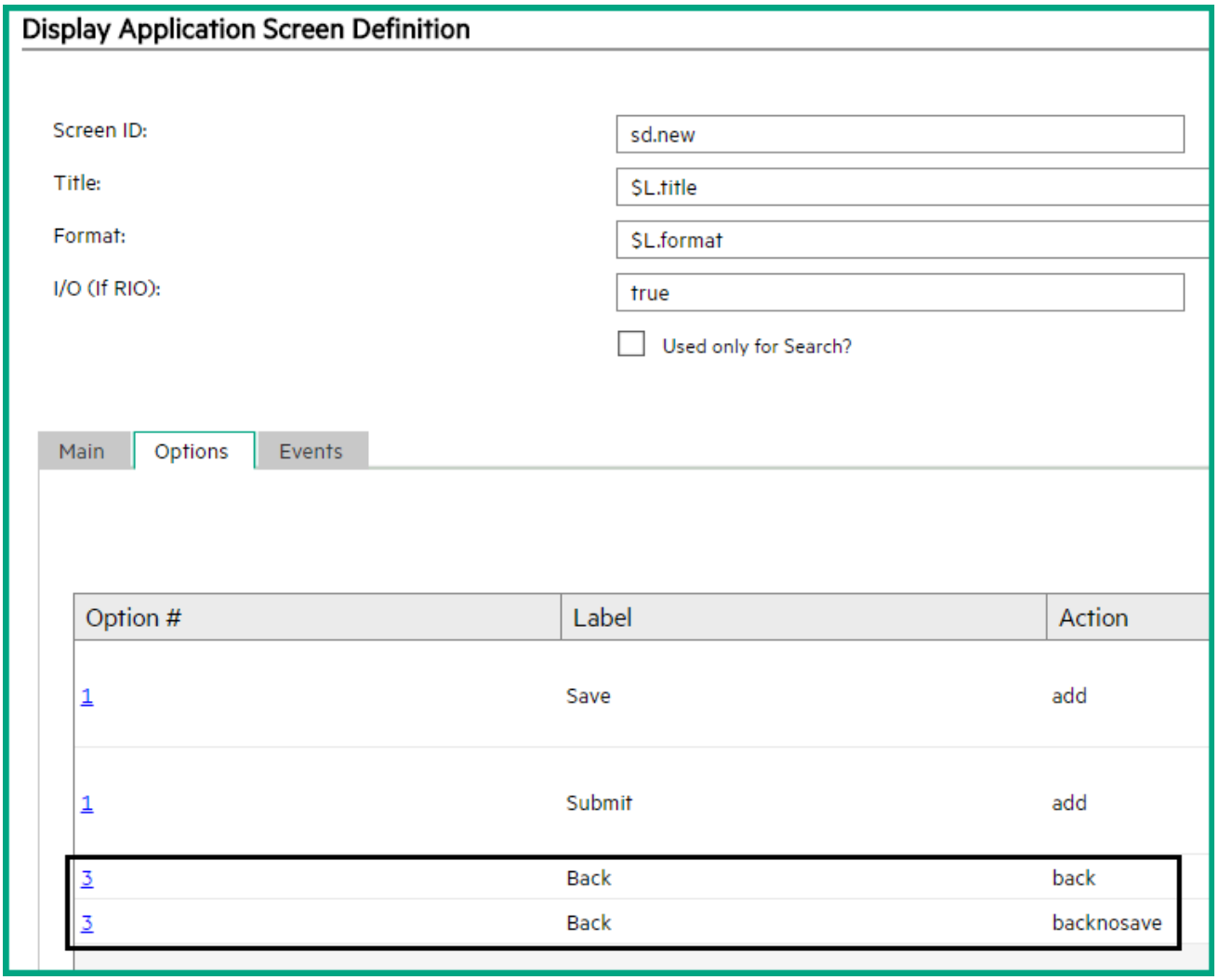
  > **Note**
  >
  > This function is supported for all modules, not only the Interaction module. Additionally, multi-level menu options are supported.

- The parameter value of "chrome" represents the browser that the web client is using. Change it to "firefox" if Firefox is used.

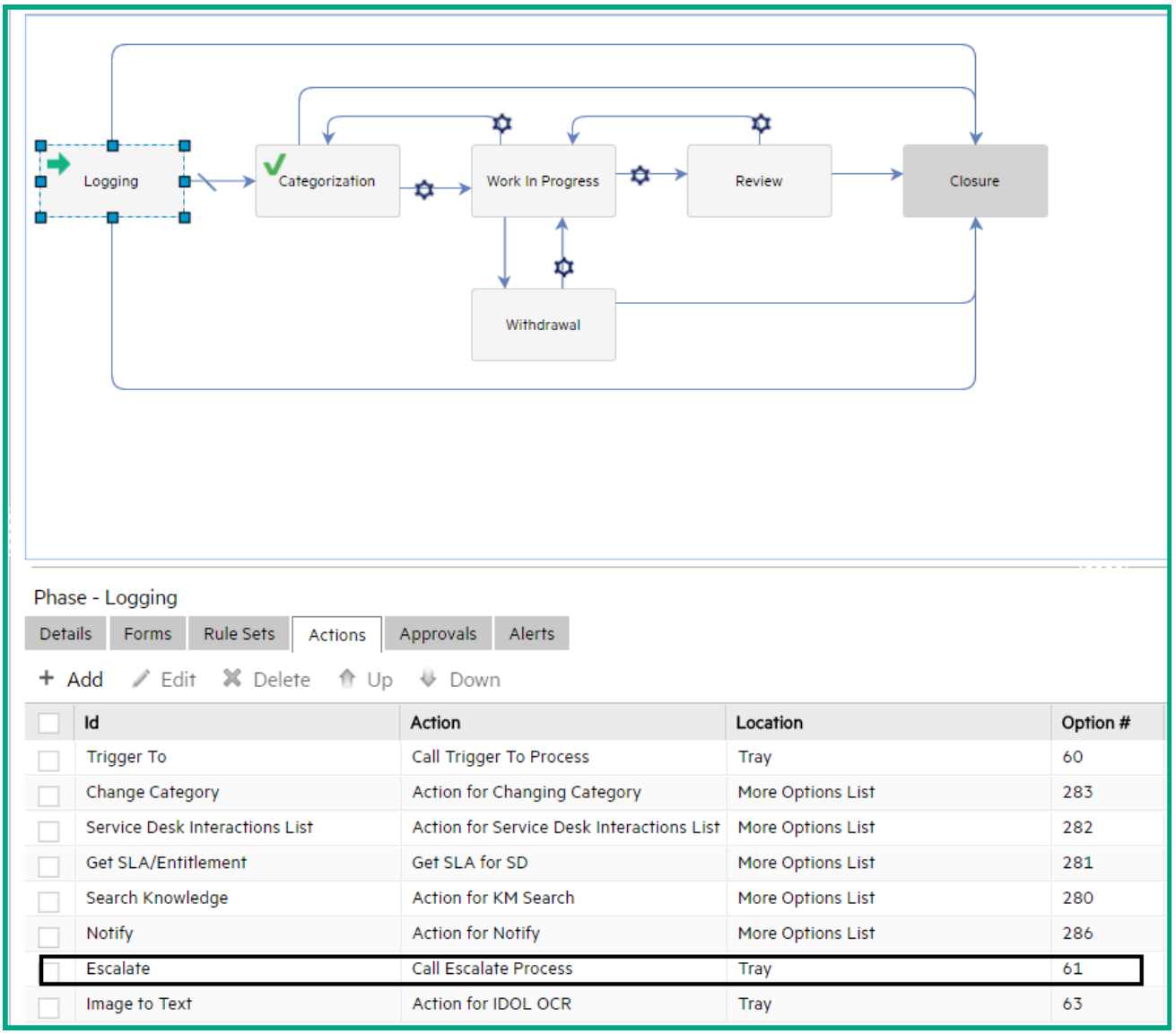
How to determine the option ID of a button or menu option

Each option ID is defined either in the corresponding Display Screen record or in the corresponding Process Designer workflow (the **Actions** tab). The following are two examples.

The following figure shows that the option ID of the **Back** button is defined in the **sd.new** Display Screen record.



The following figure shows that the option ID of the **Escalate** button is defined in the **ServiceDesk** workflow.



Configure the client system when using Firefox

The new CTI implementation doesn't require any configurations on each web client system. The user is only required to log in to the web tier client with **?telephonyuser=1** appended to the web tier client URL.

However, when using Firefox, when the client system takes a telephony call, a blank page opens and remains open. This could be annoying when the client system takes multiple calls. To avoid this from happening, enable the **dom.allow_scripts_to_close_windows** setting in Firefox. To do this, follow these steps:

1. In the Firefox address bar, type about:config.
2. Change the **dom.allow_scripts_to_close_windows** setting from **false** to **true**.

When this setting is true, the blank page is closed immediately after it opens.

Note

You can test the new CTI method using Microsoft Word and a web browser. For details, see How to test the new CTI method.

1.1.9.1.3. How to test the new CTI method

The following are example steps to test the new CTI method using Microsoft Word and Chrome. In this example, a new interaction will be created and the action of the **Back** button on the interaction detail page is executed.

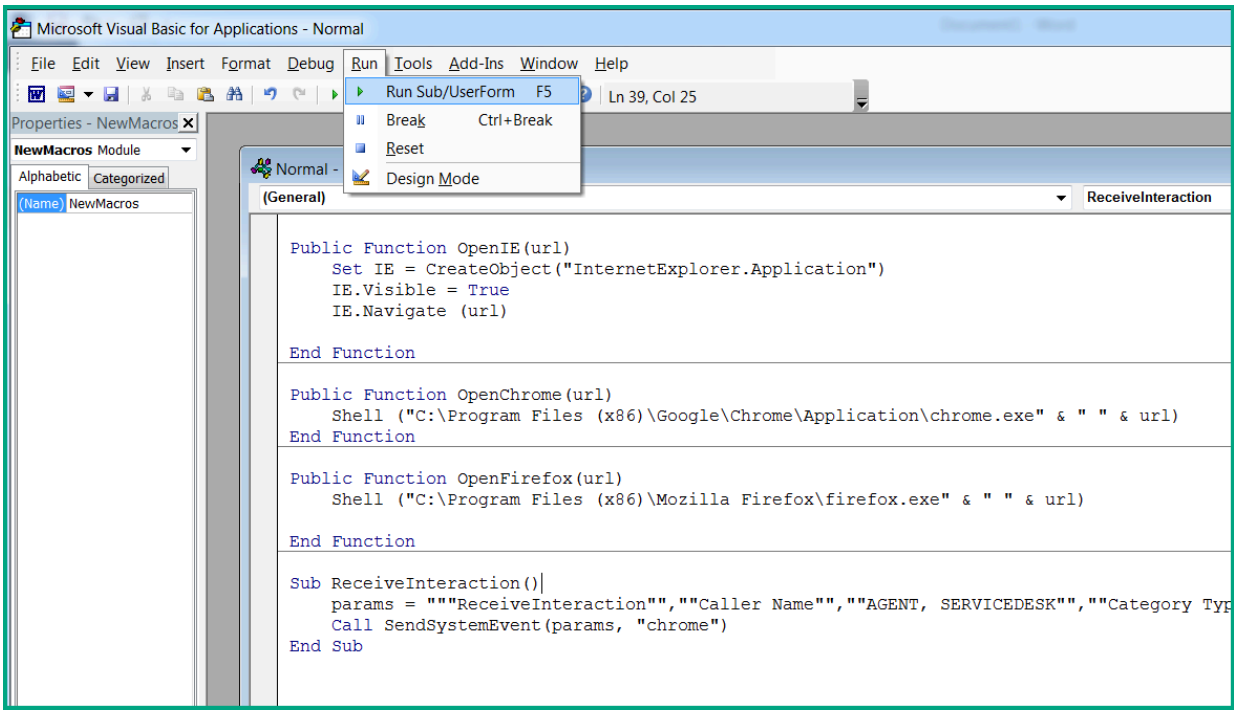
Tip

To learn more about the new CTI method, see How the new CTI method works and How to use the new CTI method.

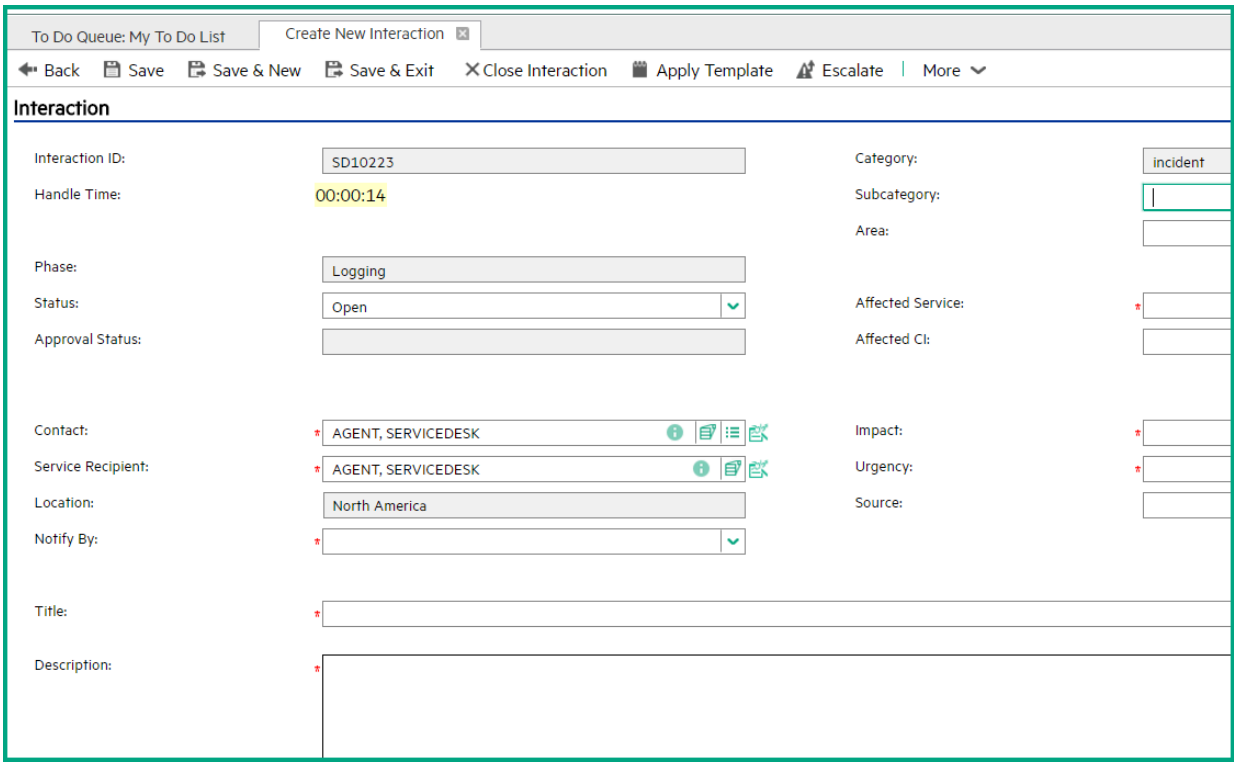
1. Open your web tier telephony URL in Chrome. For example:

   ```
   http://smwebtierhost:8080/webtier-9.70/index.do?telephonyuser=1
   ```

2. Sign in to the web tier client.
3. Open Microsoft Word on your system.
4. Open the macro editor from Word. For example, press ALT + F11 to open it.
5. Copy and paste the script above to the editor and update the parameters. For details, see Configure the CTI application.
6. In the macro editor, place your cursor in the **ReceiveInteraction** sub function area, and click **Run** > **Run Sub/UserForm**.



7. An interaction is automatically created.

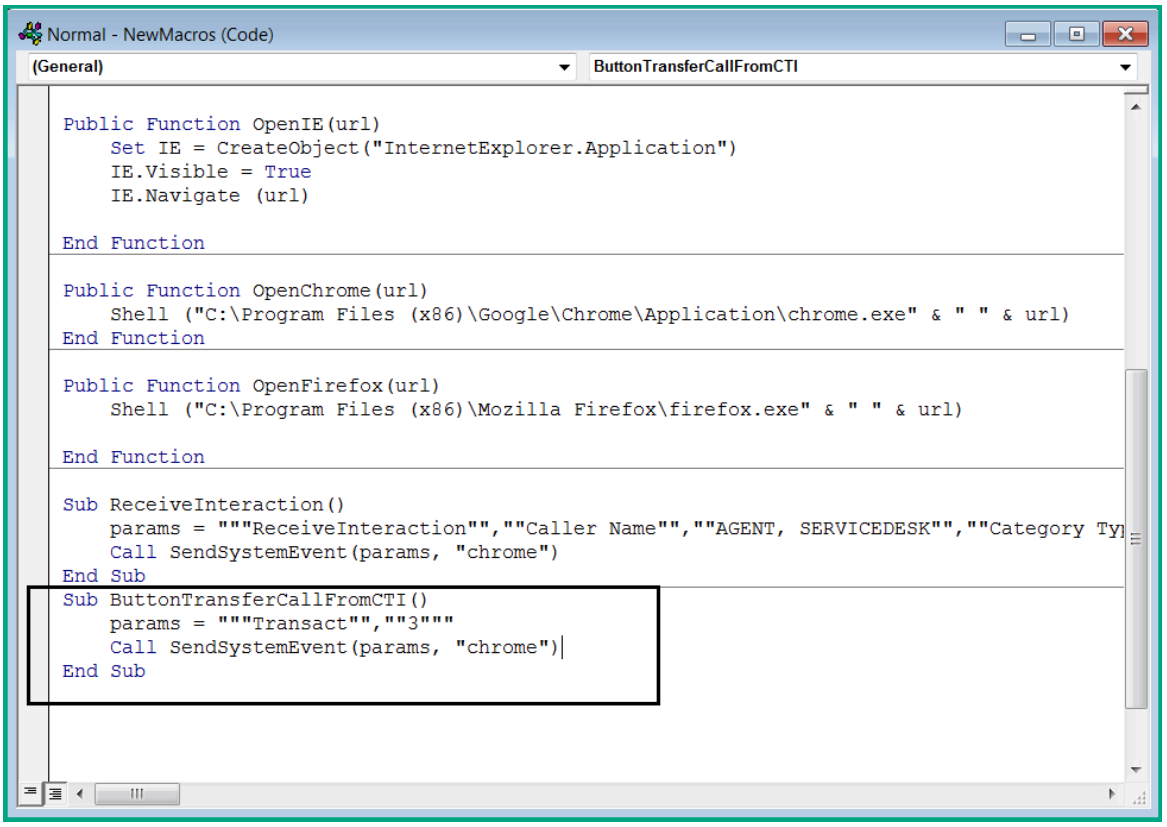


8. Call Service Manager with a System Event type (ButtonTransferCallFromCTI) to execute a **Back** action. To do this, follow these steps:

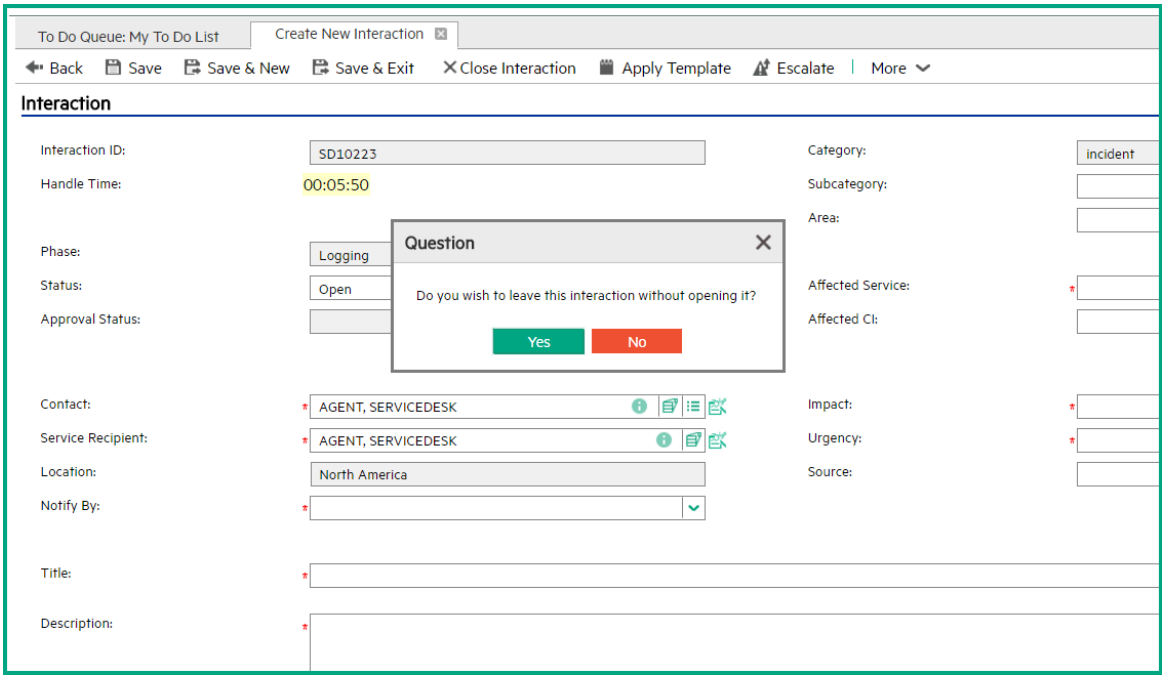
    1. Add the following sub function to the VBA script in Word:

    ```
    Sub ButtonTransferCallFromCTI()
    params = """Transact"",""3"""
    Call SendSystemEvent(params, "chrome")
    End Sub
    ```

    The following figure shows that the **ButtonTransferCallFromCTI** sub function is added.



2. Run the **ButtonTransferCallFromCTI** sub function.
3. A confirmation dialog opens, asking if you want to leave the interaction without creating it. This is the same dialog that would appear if you click the **Back** button in a new Interaction record.



1.1.9.2. Use URI

In addition to the HTTP and FTP protocols, Service Manager web client supports the `tel` and `sip` URI (Uniform Resource Identifier) protocols to initiate telephone calls or dialogues. This method doesn't rely on the browser's ability to support Java applet technology. To use this method, the following requirements must be met:

- Install a telephony client (such as Microsoft Lync) on your computer to handle and manage the calls or dialogues.
- Configure RAD expressions in the Service Manager system to trigger telephone calls or dialogues.

Example: How to use the tel URI protocol

The following are example steps to use the `tel` URI protocol with Microsoft Lync. In this example, you will add a new button to the contacts.g form, associate this button with the updated display option definitions, and then test how the `tel` URI protocol works.

1. Click **Tailoring** > **Database Manager**.
2. Type contacts in the Form field, and then press Enter. Service Manager displays a list of forms matching your search criteria.
3. Double-click contacts to open the contacts.g form.
4. Click **Search**. Service Manager displays a list of all contact records.

   Take `AARON, JIM` as an example. Click the **Contact Numbers** tab to verify that the Work field has a valid phone number. This value will be used in the later steps.
5. Click **Tailoring** > **Form Designer**.
6. Type contacts.g in the Form field, and then click **Search**. Service Manager opens the Contact Information form.
7. Click **Design**, and then add a button with the following sample properties:

Caption	Value
Visible	True
Caption	Tel
Button ID	1111
Enabled	True

8. Click **OK**, and then click **OK** again on the Contact Information form.

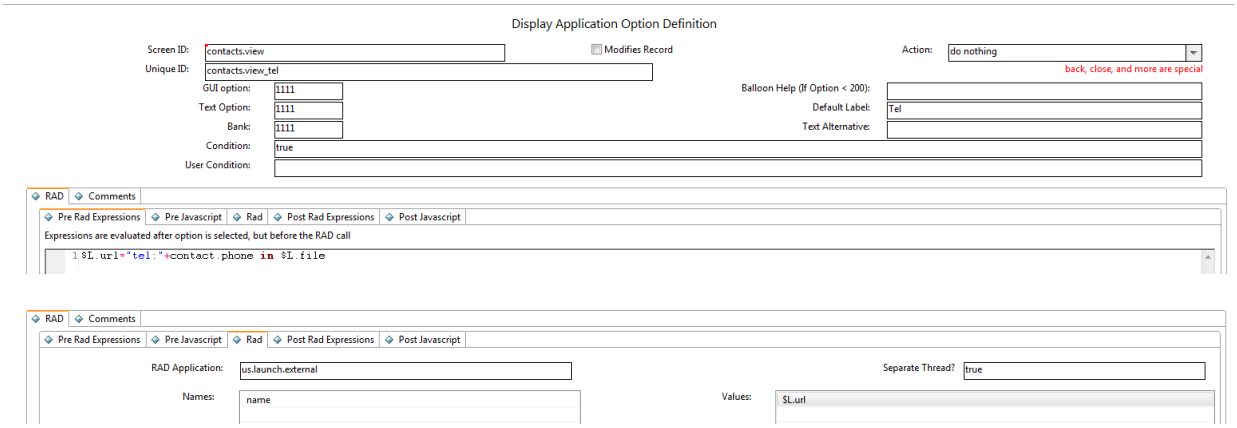
9. Click **Tailoring** > **Tailoring Tools** > **Display Options**.
10. Type contacts.view in the Screen ID field, and then click **Search**. Service Manager displays a list of display option definitions matching your search criteria.
11. Locate an option, and then enter the following information:
    - General information:

Field	Sample value
Unique ID	contacts.view_tel
Action	don'thing
GUI option	1111 **Note** You must enter the button ID you have assigned to the new button in *step 7*.
Default Label	Tel
Condition	true

    - Click the **Pre Rad Expressions** tab, and then add the `$L.url="tel:%22+contact.phone in $L.file` expression.
    - Click the **Rad** tab, and then enter the following information:

Field	Value
RAD Applications	us.launch.external
Separate Thread?	true
Values	$L.url

See the following screenshots as examples:



12. Click **Add** and **OK**.
13. Continue to follow *step 1* to *step 4* and locate the `AARON, JIM` contact record. Now you can click the **Tel** button to launch a Skype call with AARON, JIM using the `tel` URI protocol.

Example: How to use the sip URI protocol

The following are example steps to use the `sip` URI protocol with Microsoft Lync. In this example, you will add a new button to the contacts.g form, associate this button with the updated display option definitions, and then test how the `sip` URI protocol works.

1. Click **Tailoring** > **Database Manager**.
2. Type contacts in the Form field, and then press Enter. Service Manager displays a list of forms matching your search criteria.
3. Double-click contacts to open the contacts.g form.
4. Click **Search**. Service Manager displays a list of all contact records.

   Take `AARON, JIM` as an example. Click the **Business** tab to verify that the Email field has a valid email address. This value will be used in the later steps.

5. Click **Tailoring** > **Form Designer**.
6. Type contacts.g in the Form field, and then click **Search**. Service Manager opens the Contact Information form.
7. Click **Design**, and then add a button with the following sample properties:

Caption	Value
Visible	True
Caption	Tel
Button ID	1112
Enabled	True

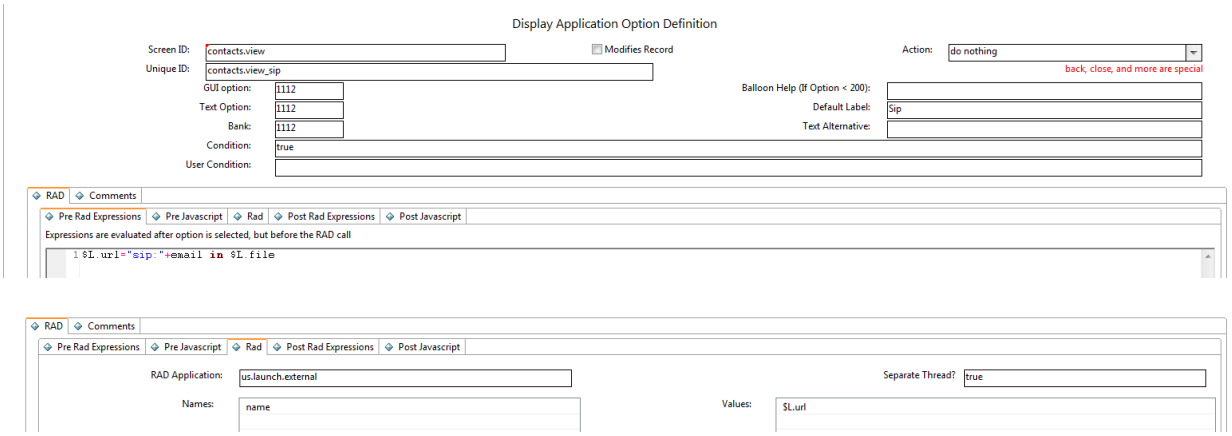
8. Click **OK**, and then click **OK** again on the Contact Information form.
9. Click **Tailoring** > **Tailoring Tools** > **Display Options**.
10. Type contacts.view in the Screen ID field, and then click **Search**. Service Manager displays a list of display option definitions matching your search criteria.
11. Locate an option, and then enter the following information:
    - General information:

Field	Sample value
Unique ID	contacts.view_sip
Action	don'thing
GUI option	1112 **Note** You must enter the button ID you have assigned to the new button in *step 7*.
Default Label	Tel
Condition	true

    - Click the **Pre Rad Expressions** tab, and then add the `$L.url="sip:%22+email in $L.file` expression.
    - Click the **Rad** tab, and then enter the following information:

Field	Value
RAD Applications	us.launch.external
Separate Thread?	true
Values	$L.url

    See the following screenshots as examples:



12. Click **Add** and **OK**.
13. Continue to follow *step 1* to *step 4* and locate the `AARON, JIM` contact record. Now you can click the **Sip** button to launch a Skype dialogue with AARON, JIM using the `sip` URI protocol.

1.1.10. Integrations and extaccess records

Integrations call existing processes from the processes they define instead of modifying an existing process.

In the situation where you need to define extaccess records that call display actions in the record's State there is a good chance that the process will need additional logic to properly function. The best practice for dealing with this situation is to create and call a new process rather than modifying an existing process with integration-specific logic.

For example, an integration wants to perform display action x, which calls Process (p), but needs to put integration specific logic into Process (p) in order to make it perform correctly. The best practice in this case is to:

- Create a new process record which performs the integration specific logic.
- Call the original Process (p) in the "Next Process" tab.
- Add a new action to the State record.
- Call that action from the extaccess record.

This will make the integration easier to maintain and debug.

When you add a new display action to the State record, the $L.action variable will most likely have to be updated. Usually the integration specific process record will need to update $L.action to match the display action being wrapped.

For example, an integration wants to call the "save" display action. A new process record called "integration.record.save" is created, which finishes by calling "record.save" in the next process tab. A new display action called "integration.save" is added to the State record and the extaccess record calls that in its Allowed Actions tab. The "integration.record.save" process record ensures that the $L.action variable is set to "save."

In some cases, other local variables may need to be set as well. Incident Management, for example, has a $L.mode variable that needs to be managed. Be sure to check both the existing State and Process record when developing a new process for references to variables that may need to be managed.

Related topics

Integrations
Display functions
Document Engine

1.2. Change Configuration and Release Management (CCRM)

Service Manager is part of the Change Configuration and Release Management (CCRM) solution. CCRM enables enterprise IT organizations to do the following:

- Provide the structure and formal workflow necessary to implement changes
- Reduce the risk of making changes by providing an accurate picture of IT infrastructure, as well as the impact that any change may have on IT services
- Enable early detection of unplanned changes
- Maintain an accurate record of IT infrastructure

To minimize incidents that result from unplanned or improperly planned changes to the organization, an IT organization must be able to respond in the following ways to change drivers:

- Proactively: to provide benefits to the organization, either through lower total cost of ownership (TCO), or by providing options that enable business to develop
- Reactively: to resolve errors that impact the level of services that are provided

CCRM offers a solution comprised of three integrated OpenText products:

- Service Manager: automates and standardizes the change process in accordance with ITIL best practices—from request through release and review.
- Release Control: plays a role at a specific point in the process as a decision-support tool; it includes risk analysis, impact assessment, collision detection, and forward schedule of changes.
- Universal CMDB: consists of a rich business-service-oriented data model with the built-in discovery of configuration items (CIs) and configuration item dependencies, visualization and mapping of business services, and tracking of configuration changes.

CCRM also offers an automation extension to automate change requests (through Operations Orchestration (OO)) and another extension to provide dashboard views of assigned request status as well as comparisons between current projects and staffing profiles (through PPM Center).

For more information about CCRM, see the CCRM documentation available with the Service Manager documentation from software support.

1.3. SAP Solution Manager

Businesses today increasingly rely on their mission-critical SAP applications. Disruptions in the SAP environment have a severe business impact. Keeping the system continuously available has never been more vital for success. In any SAP landscape, business process disruptions caused by an application or infrastructure incident must be proactively prevented. If disruptions do occur, they need to be quickly and efficiently resolved. OpenText and SAP have teamed up to solve this issue.

Incident management in enterprises today consists of disconnected incident management systems that often implement divergent processes. This situation diminishes collaboration within IT operations, lowers quality of service and productivity. The integration of SAP Solution Manager Service Desk with Service Manager provides a cohesive Incident and Service Request Management solution for the entire enterprise, resulting in higher enterprise availability, improved service quality and reduced IT costs.

Incident Exchange builds a dynamic link between Service Manager Software and SAP Solution Manager Service Desk and improves the Incident and Service Request Management Process throughout the entire enterprise Incident Exchange offers dynamic integration between Service Manager and SAP Solution Manager Service Desk for improved incident workflow.

The interface to exchange support messages between Service Manager and SAP Solution Manager Service Desk was designed and developed jointly by OpenText and SAP and is certified by SAP.

1.3.1. Install and administer Service Manager Exchange with SAP Solution Manager

This section describes the installation, configuration, administration and maintenance of Incident Exchange and the Incident Exchange web service. Information in this section is intended only for use by software consultants and application administrators that install and maintain Incident Exchange. We strongly recommend you to read this section carefully before installation and follow the instructions herein because SMSSMEX 1.40 significantly differs from the previous releases.

1.3.1.1. Existing fragmented Incident Management workflow

Performance monitoring of an SAP environment must include SAP and non-SAP applications.

SAP Solution Manager Service Desk

To monitor and manage SAP environments, IT operations management uses the SAP Solution Manager Service Desk to collect information about SAP systems and serves as an internal help desk for SAP installations. Users and administrators can create support messages from any SAP system. The messages are processed centrally in the Solution Manager Service Desk.

If the support message involves an SAP application, a solution may be available in the SAP Service Marketplace or from SAP Active Global Support or from the in-house SAP support team. But if the issue is not caused by the SAP application, the message will be forwarded to the administrators responsible for the non-SAP systems. The support call needs to be entered in a second or third service desk and tracked until resolved. In the meantime, the SAP Service Desk team waits for feedback before closing the call and informing the originator, who is temporarily left “in the dark”.

Service Manager

An incident can also be reported to the service desks monitoring non-SAP applications and infrastructure hardware and software. Many SAP customers have integrated these tasks in Service Manager, which is able to support nearly all IT application and infrastructure components.

If a support call, for example, pertains to a “printing issue from an SAP application” and the Service Manager team detects no issue with the printer hardware or software, the call will be forwarded to the SAP service desk team to check whether it is related to the SAP application. Again the service call must be re-entered in a service desk, in this case in the SAP Solution Manager Service Desk. Additional information or attachments regarding the error or error resolution must be forwarded manually. The Service Manager team has to wait for feedback before informing the requesting user and closing the call.

In both cases the disconnected service desks and the fragmented incident management workflow impede the service desk team’s ability to resolve problems. Disadvantages of this non-integrated workflow are:

- Only limited and often inconsistent information about the incident is available.
- It is difficult to monitor, track and report incidents or to work together toward resolution.
- Manual workarounds are required for the handover of incidents between the SAP and non-SAP service desks and for information updates.
- There is insufficient synchronization. The same incident may get reported, recorded and tracked in separate service desks, or the incidents may get lost or ‘dropped’.

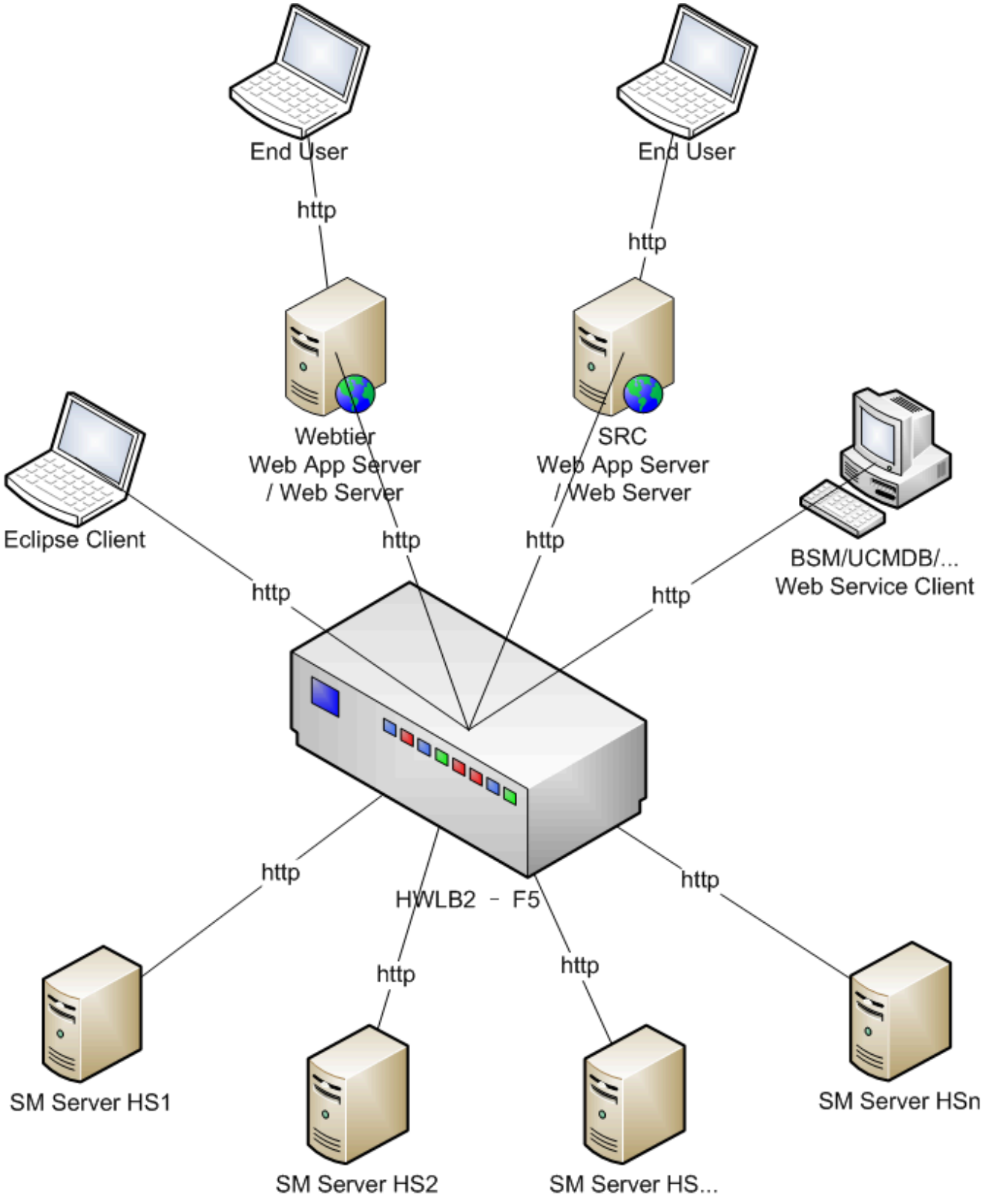
- Expertise about the interrelationships of SAP applications with non-SAP applications and other IT components is lost.

This results in productivity loss and reduced quality of service.

1.3.1.2. Deployment scenarios

High level overview

SMSSMEX integrates a single Service Manager server with multiple external help desk systems.



Components

The following diagram shows the component details.



- Service Manager Server is the OpenText service desk system.
- Service Manager DB provides persistent storage for Service Manager.
- SMSSMEX Client Code consists of RAD and Java scripts, table definitions and GUI formats. The SMSSMEX webservices are called from this client code.
- WebServer is a Tomcat Web Application Server or WebLogic Application Server that hosts the SMSSMEX WebService (deployed as a `.war` file).
- SMSSMEX WebService exposes the incident webservice of Service Manager in the SAP format and transfers client requests to SAP Solution Manager web services.
- SMSSMEX Database provides persistent storage for the SMSSMEX WebService.
- SAP Solution Manager is the Service Desk.

1.3.1.3. Install and configure SMSSMEX

This chapter describes how to install and configure SMSSMEX.

1.3.1.3.1. Install SMSSMEX

The Service Manager Exchange with SAP Solution Manager product DVD includes an autorun program for installation.

Prerequisites

It is NOT recommended to install SMSSMEX and Service Manager on the same server.

Install SMSSMEX

1. Sign in to the operation system as a super user.
2. The installer is in:
    - <SMSSMEX Release Package>\InstData\Windows\install.exe (Windows Server 2008 and 2012)
    - <SMSSMEX Release Package>\InstData\Linux\install.bin (Linux)
3. Run install.bin or install.exe. The Introduction dialog appears.
4. Click **Next**. The license agreement appears.
5. Select **I Accept the terms of License Agreement**.
6. Click **Next**. The Choose Install Folder page displays. For example, the default installation folder on Windows Server 2012 is C:\Program Files (x86)\Micro Focus\SMSSMEX.
7. Click **Next**. Review the summary information.
8. Click **Install**. The files are installed.
9. Click **Done** to close the installer when the Install Complete dialog appears.

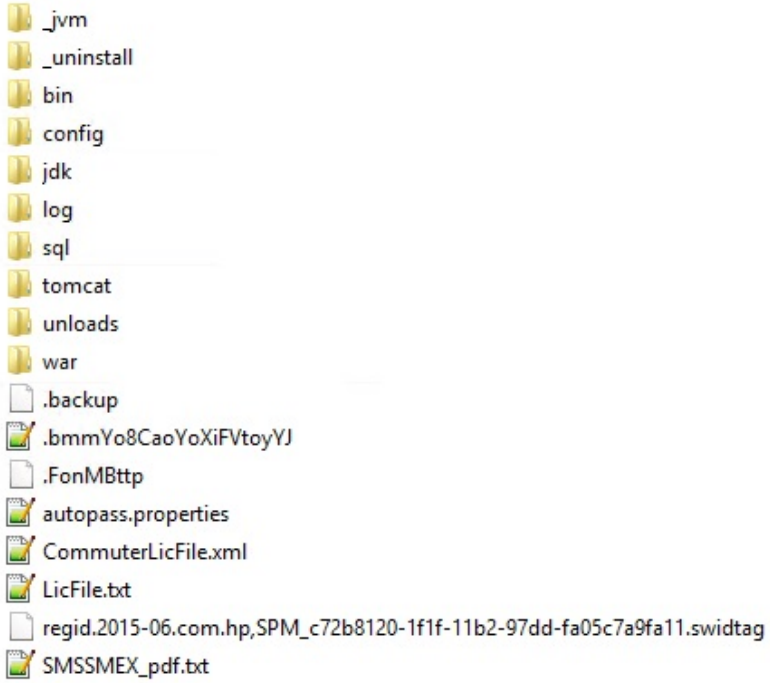
Uninstall SMSSMEX

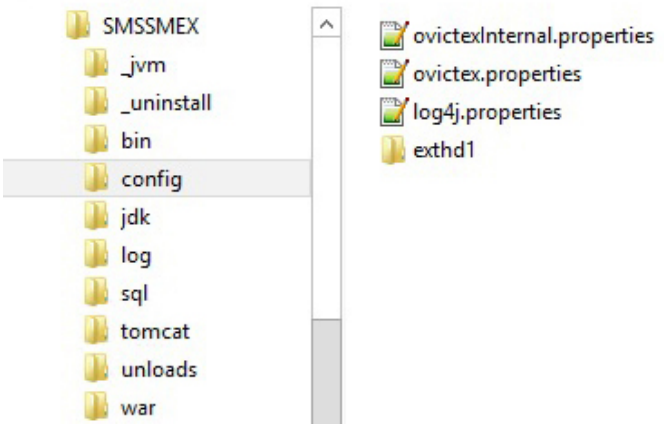
To uninstall SMSSMEX on Windows, execute <SMSSMEX_installDir>\Uninstall SMSSMEX\Uninstall SMSSMEX.exe or simply go to **Start** > **Programs** > **SMSSMEX** > **Uninstall SMSSMEX**.

To uninstall SMSSMEX on Unix, execute <SMSSMEX_installDir>/Uninstall SMSSMEX/Uninstall_SMSSMEX.

SMSSMEX installed files

After installation, the SMSSMEX folder has the following contents.


_jvm
_uninstall
bin
config
jdk
log
sql
tomcat
unloads
war
.backup
.bmmYo8CaoYoXiFVtoyYJ
.FonMBttp
autopass.properties
CommuterLicFile.xml
LicFile.txt
regid.2015-06.com.hp,SPM_c72b8120-1f1f-11b2-97dd-fa05c7a9fa11.swidtag
SMSSMEX_pdf.txt


SMSSMEX
_jvm
_uninstall
bin
config
jdk
log
sql
tomcat
unloads
war
ovictexInternal.properties
ovictex.properties
log4j.properties
exthd1

Directory	Content
bin	Executable commands and product description file
config	Web service configuration files The config\exthd1 folder stores the template for an external helpdesk configuration
sql	Database table creation/deletion scripts
unloads	Service Manager customization unload files
log	Log files
jdk	Internal JDK 11
tomcat	Tomcat 9
Uninstall SMSSMEX	Executable file for uninstallation

1.3.1.3.2. Configuring Tomcat

The connector for deploying the web service must be enabled. Uncomment the port specification in <SMSSMEX_installDir>\tomcat\conf\server.xml. For example:

```
<Connector port="8080"
redirectPort="8443"
maxThreads="150" minSpareThreads="25" maxSpareThreads="75"
enableLookups="false" acceptCount="100" debug="0"
connectionTimeout="20000" disableUploadTimeout="true" />
```

You can modify the ports as necessary.

1.3.1.3.3. Setting up database

This section describes how to setup the database.

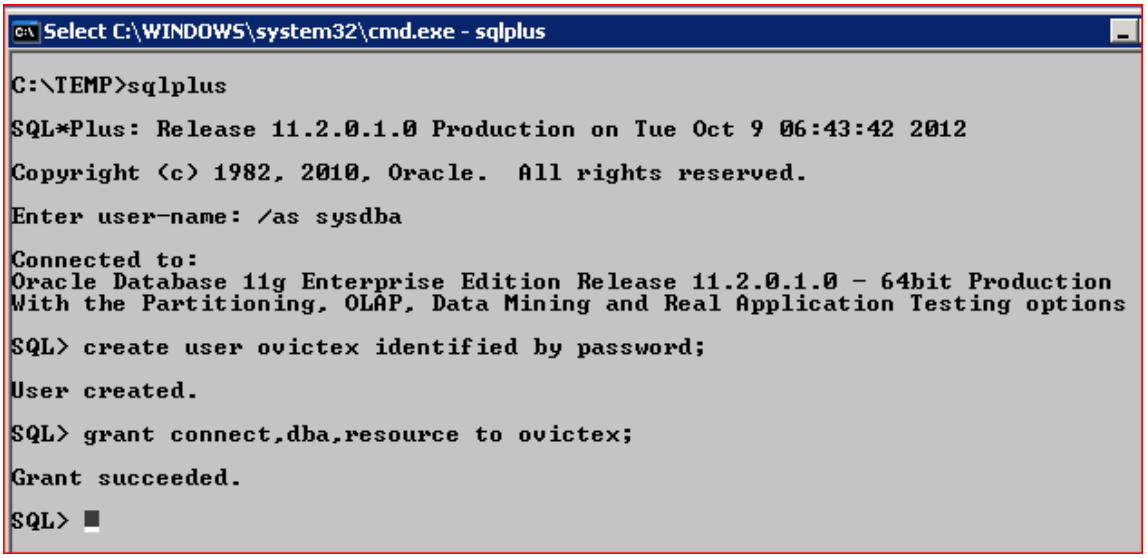
Note

The SMSSMEX web service uses a database to store metadata. The SMSSMEX web service must be able to read table `v$database` (Oracle) or execute function `SERVERPROPERTY('ProductVersion')` (SQLServer). These system tables are queried when validating the database connections.

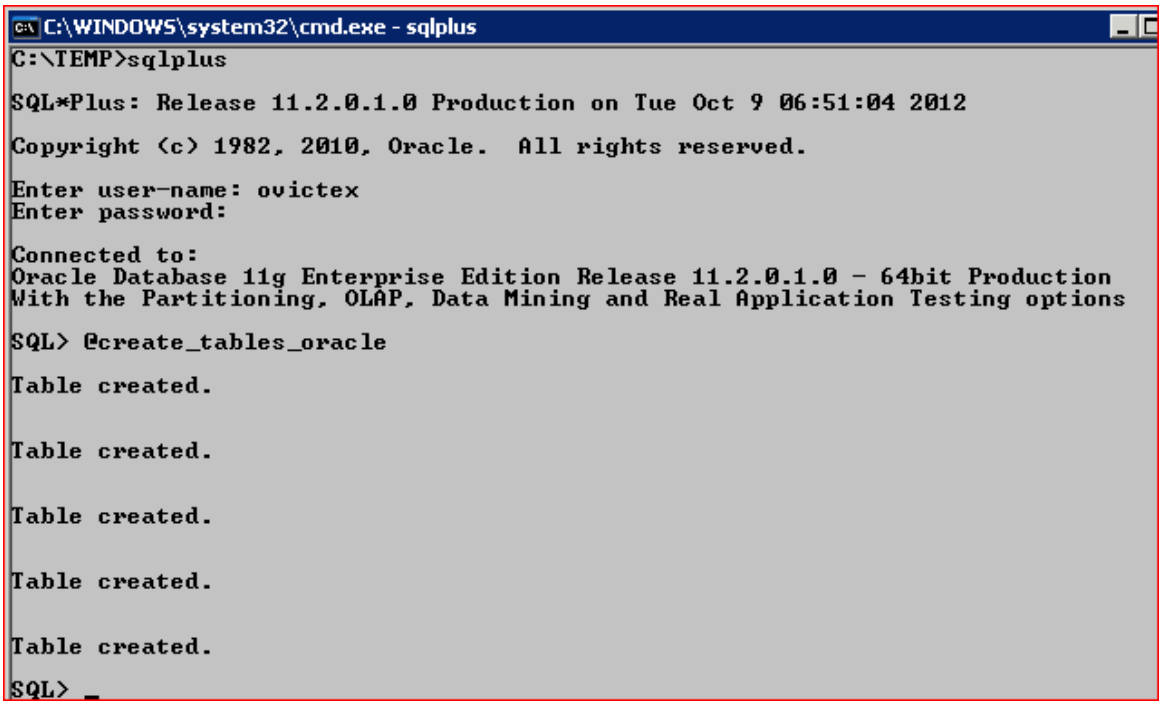
1.3.1.3.3.1. Oracle

To setup the Oracle database do the following:

1. Create a user.



2. Give the user the right to do a select on table `v$database`. This system table is queried by the SMSSMEX web service to validate database connections.
3. Login as the user and run the script create_tables_oracle.sql (log in from path <SMSSMEX_installDir>\sql so that the script is found). This creates all required tables.

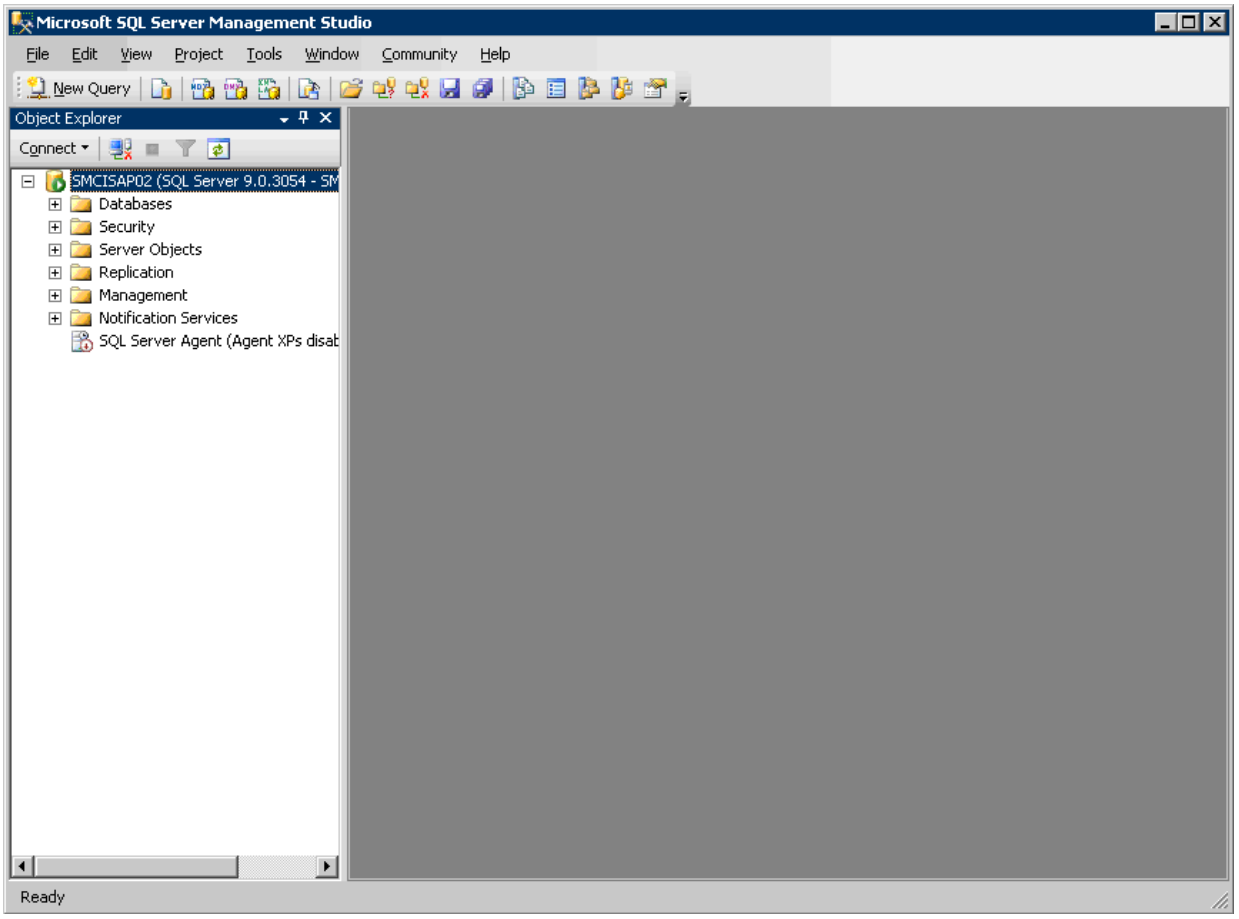


These tables are created within the schema of the database user (the tables are logically separated and don't interfere with each other).

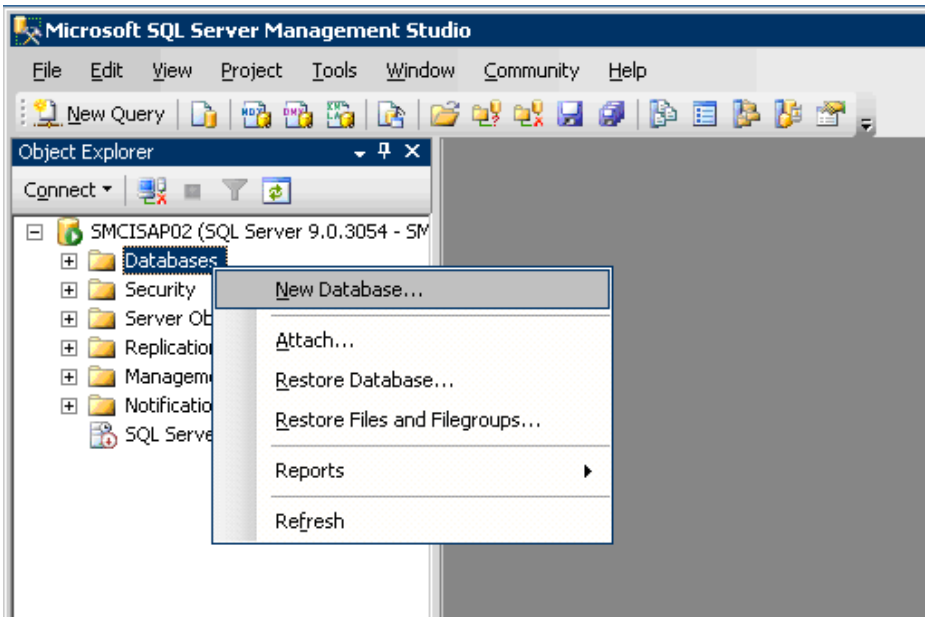
1.3.1.3.3.2. MS-SQL

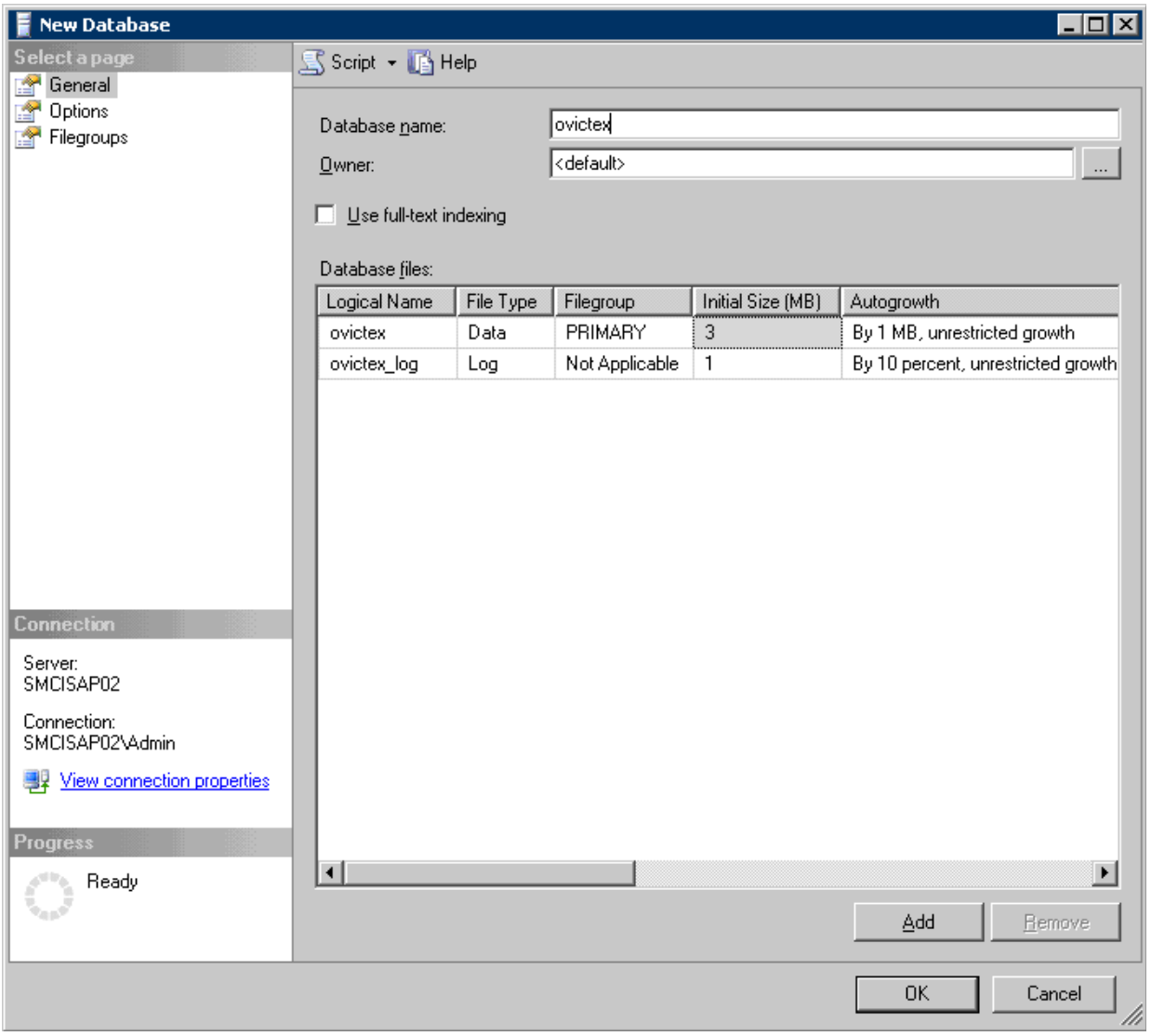
Do the following to create the required separate database for SMSSMEX tables:

1. Launch SQL Server Management Studio.

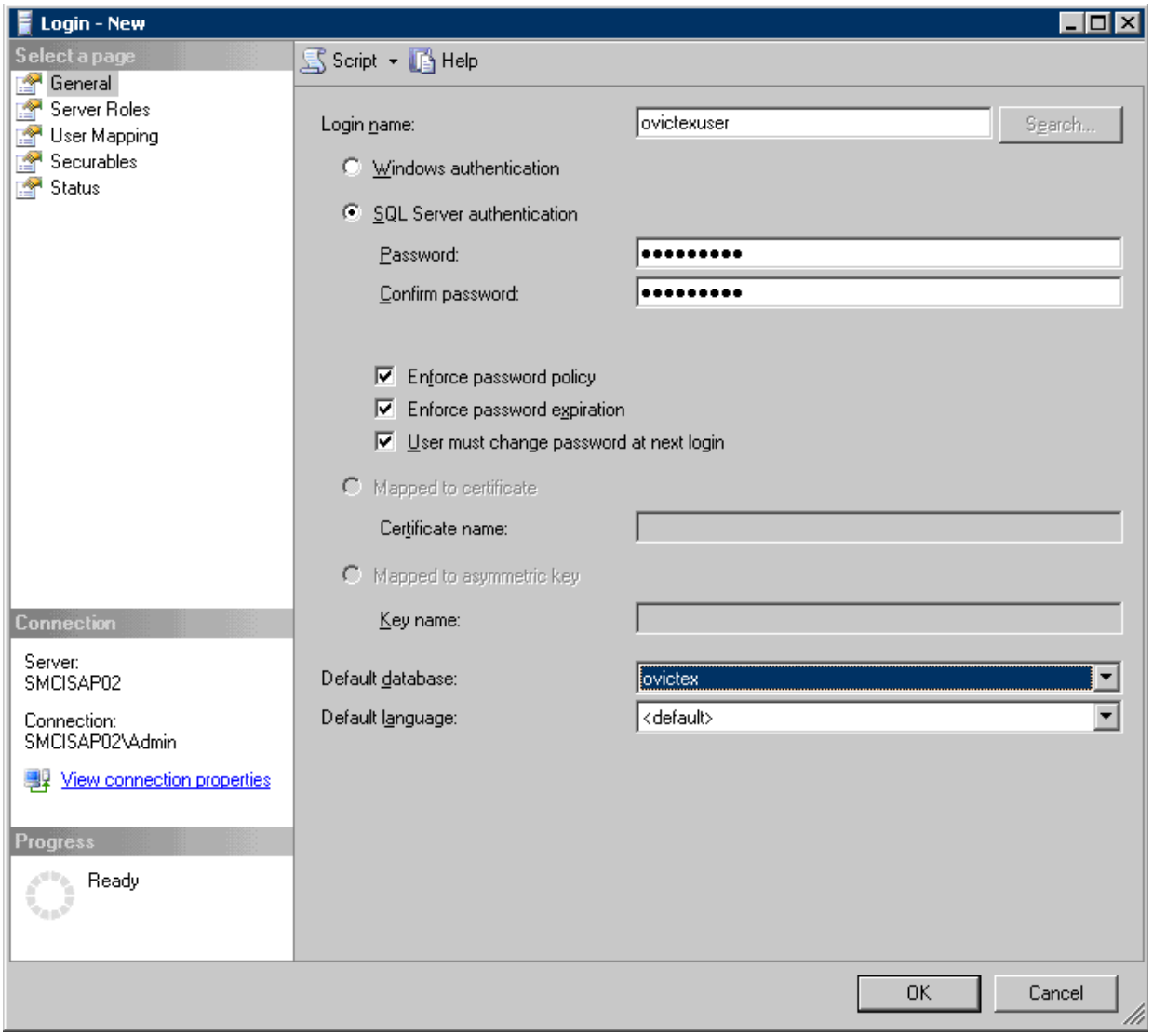


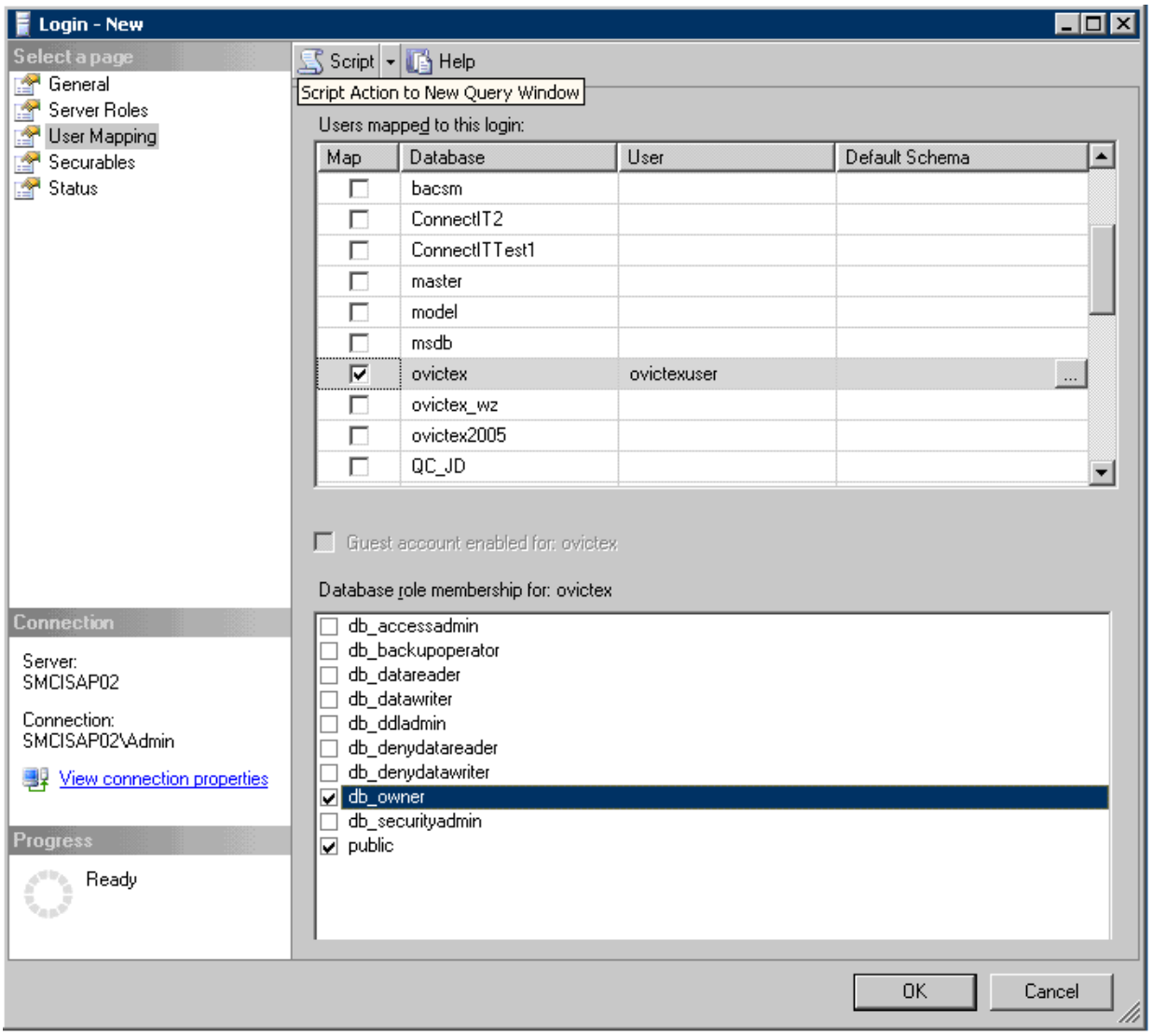
2. Create a new database (`ovictex`). Right-click on **Databases** and choose **New Database**.





3. Create a database user (`ovictexuser`) with permission for database `ovictex` . Right-click on **Security/Login** and select **New Login**.


Login - New
Select a page
General
Server Roles
User Mapping
Securables
Status
Script
Help
Login name:
ovictexuser
Search...
Windows authentication
SQL Server authentication
Password:
Confirm password:
Enforce password policy
Enforce password expiration
User must change password at next login
Mapped to certificate
Certificate name:
Mapped to asymmetric key
Key name:
Default database:
ovictex
Default language:
<default>
Connection
Server:
SMCISAP02
Connection:
SMCISAP02\Admin
View connection properties
Progress
Ready
OK
Cancel



4. Create the SMSSMEX tables.

    1. Click **New Query** on the toolbar and select database **ovictex**.
    2. Copy and execute the SQL scripts under folder <SMSSMEX_installDir>\sql\create_tables_sqlserver.sql.

1.3.1.3.4. Configuring ovictex.properties

File <SMSSMEX_installDir>/config/ovictex.properties must specify the local helpdesk installation. The file comments describe how to do this.

Caution

To configure the passwords, use command line application `<SMSSMEX_installDir>/bin/encryptPasswords.bat|sh` (don't enter the password directly in the file; passwords are stored in encrypted format). There are several sensitive fields that must be encrypted. These fields are discussed below. For more information about using `encryptPasswords.bat|sh`, see Tools.

The following parameters must be configured:

- Service Manager web service endpoint
- To connect to Service Manager:

```
sc.webservice.endpoint = http://<ServiceManager host>:
<Port>/sc62server/PWS
```

- The following are the required parameters:

  `sc.user=<web service endpoint access user name>`

  `sc.password=<encrypted password>`

  Note

  `sc.password` must be filled by `encryptPasswords.bat|sh`. SMSSMEX supports SSL connections to Service Manager, but the parameter values are different than above and additional parameters must be set (see Security between SAP Solution Manager and Tomcat).

- SMSSMEX database configuration information:

```
ovictex.db.type= <oracle | sqlserver>
ovictex.db.host=<database server address>
ovictex.db.port=<database server port number>
ovictex.db.instance=<sqlserver database server instance>
ovictex.db.name=<database name or oracle DB SID>
ovictex.db.user=<database user name>
ovictex.db.password=<database password>
```

> **Note**
>
> `ovictex.properties` contains examples. `ovictex.db.password` must be filled by `encryptPasswords.bat|sh` .

- One or more External Helpdesk instance names.
  - Parameters are `exthd.instances.id.<number>` , where `<number>` is a number {1,...,n}.
  - The first number must be 1 and each number must be greater than the previous.
  - `ExtHdInstanceName` differentiates multiple External Helpdesks and is the name of the subfolder in <SMSSMEX_installDir>/config and the ExtHd configuration file.
- Incident category in Service Manager.
- Property `ovhd.incident.informationlog.entry.separator` should be configured to a unique value that's not contained in messages exchanged between Helpdesks. By default, it is configured to "`----`". Service Manager must be configured to use this separator to append information to the Journal. If this separator is contained in a message then duplicate information could be sent to the external Helpdesk (no data is lost).

1.3.1.3.5. Configuring File ovictexInternal.properties

The property file for internal configurations is in the <SMSSMEX_installDir>/config directory of the SMSSMEX installation. There is typically no need to configure this file.

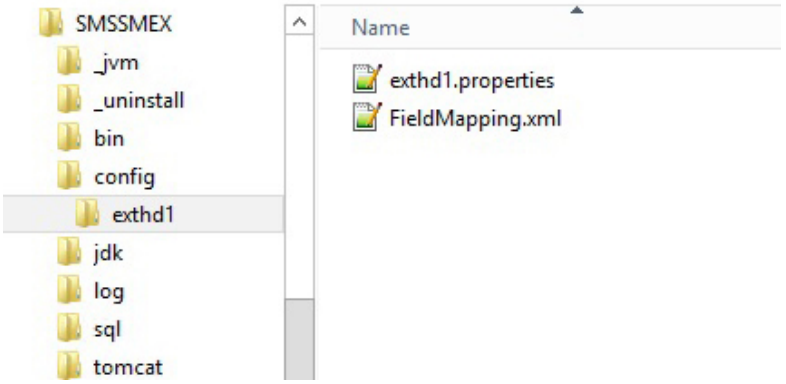
1.3.1.3.6. External Helpdesks

Main configuration file <SMSSMEX_installDir>/config/ovictex.properties must define all External Helpdesk Instances. For example:

- `exthd.instances.id.1 = exthd1`
- `exthd.instances.id.2 = SAP_exthd2`
- `exthd.instances.id.3 = NY200BM`

Each external helpdesk has the following configuration files:

- `<ExtHdInstanceName>.properties`
- `FieldMapping.xml`



Note

The same names (such as `exthd1`, `SAP_exthd2`, `NY200BM`) must be used for the names of subfolders with specific configuration file names. The names must not contain spaces or special characters. The default configuration comes with a defined `exthd1` sample External Helpdesk configuration.

To create a new instance:

1. Add a new line in the ovictex.properties file for the new ExtHd.

   `exthd.instances.id.2 = exthd2`

2. Create the new subfolder <SMSSMEX_installDir>/config/exthd2.
3. Copy the configuration files for `exthd1` to `exthd2`.
4. Rename <SMSSMEX_installDir>/config/exthd2/exthd1.properties to <SMSSMEX_installDir>/config/exthd2/exthd2.properties.
5. Make the required changes to the new files.

6. The following parameters must be configured in <ExtHdInstanceName>.properties:

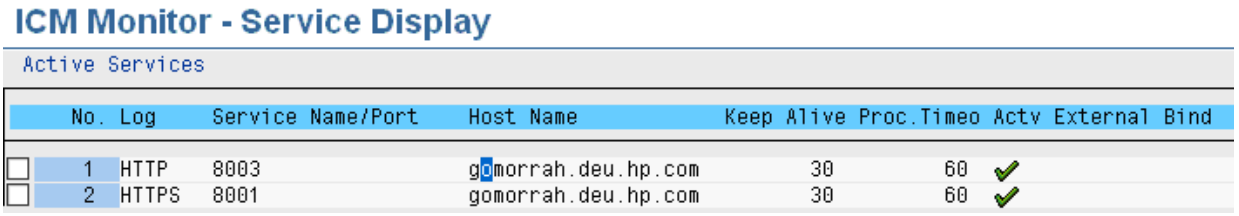
   `exthd.webservice.endpoint = http://<SolutionManager host>:<Port>/sap/bc/srt/rfc/sap/ICT_SERVICE_DESK_API?SAP-CLIENT=<SAP client number>`

   `exthd.webservice.authentication.scheme = BASIC`

   `exthd.webservice.authentication.username = <SAP client user name>`

   `exthd.webservice.authentication.password = <encrypted SAP client user password>`

   IMG activity guides you to SAP transaction **/nsmicm**. Select the activity in menu **Goto** > **Services**.



   This transaction shows the host and port for access to the SAP Solution Manager Service Desk web service. Specify the host/port in <ExtHdInstanceName>.properties as the endpoint entry.

   Note

   `exthd.webservice.authentication.password` must be filled by `encryptPasswords.bat|sh`.

1.3.1.3.7. Configuring FieldMapping.xml

The FieldMapping.xml file, which locates in the <SMSSMEX_installDir>/config/<ExtHdInstanceName>directory, must be adjusted to send/receive special/customized fields to/from the external Helpdesk. For detailed information see Field mapping configuration.

1.3.1.3.8. Verifying Configuration

Verify the configuration with the checker tool before trying to exchange incidents between Service Manager and SAP Solution Manager. The checker error messages are much more helpful for troubleshooting than Service Manager and Solution Manager error messages.

To execute the checker, run the following command:

`<SMSSMEX_installDir>/bin/checker.bat|sh`

Checker checks the environment, database and HTTP connections and configuration of Service Manager. No incidents are exchanged. The following are the possible results:

- OK
- ERROR (partial failure; checks that the check depends on have failed)
- FAIL (with troubleshooting recommendations)

You can re-run a check by passing the number of the check to the executable. You can also examine the Incident Exchange log messages or run a trace. For more information about using checker.bat|sh, see Tools.

1.3.1.3.9. Deploying on WebLogic

1. Before starting the WebLogic server, set an environment variable named "SMSSMEX_HOME" to the pathname where this application is installed.

   For example, if the WebLogic server is installed in the /opt/Micro Focus/SMSSMEX directory, set the environment variable to the following:

   ```
   $ export SMSSMEX_HOME=/opt/MicroFocus/SMSSMEX
   ```

2. Start the WebLogic server and launch the WebLogic administration console.
3. Deploy the ovictex.war file in the /opt/MicroFocus/SMSSMEX/war directory. See the following steps for an example:

   1. Select **Domain Structure > Deployments** and click **Install**.
   2. Use the Install Application Assistant to locate the ovictex.war file.
   3. Select **Install this deployment as an application** and click **Next** until last step.
   4. Click **Finish** to exit the installation wizard.

   For advanced configuration, refer to *BEA WebLogic Server Administration Console Online Help* for more information.

   Note

   We recommend that you deploy SMSSMEX on WebLogic 12 or later because there is a known issue QCCR1E128427 for WebLogic 11(version: 10.3.6.0).

1.3.1.3.10. Starting/Stopping SMSSMEX (Tomcat only)

Starting from Windows:

`<SMSSMEX_installDir>\bin\setup startup`

Stopping from Windows:

`<SMSSMEX_installDir>\bin\setup shutdown`

Starting from Linux:

`<SMSSMEX_installDir>/bin/setup.sh startup`

Stopping from Linux:

`<SMSSMEX_installDir>/bin/setup.sh shutdown`

1.3.1.4. Upgrading to SMSSMEX V1.40

Follow these stesp to upgrade to SMSSMEX v1.40 on Tomcat:

1. Stop the SMSSMEX old version.
2. Back up the following configuration files:
    - All files in the <SMSSMEX_installDir>\config folder.
    - (Optional) The <SMSSMEX_installDir>\tomcat\conf\server.xml file.

      Note

      If you have not customized the Tomcat configurations, skip this step.

    - Other configuration files which have been customized.
3. Backup the database.
4. Uninstall the SMSSMEX old version.
5. (Optional) Uninstall OpenView Autopass if it is used by SMSSMEX only. Otherwise, if another OpenText product on this computer also use Autopass for license management, don't uninstall Autopass.
6. Install SMSSMEX v1.40.
7. Configure SMSSMEX v1.40.

   You can copy parameter values from the backup configuration files to configure SMSSMEX v1.40. Do not just copy and replace ovictex.properties and FieldMapping.xml because these files have been updated in SMSSMEX v1.40.

   For example, some differences between the configuration files in SMSSMEX v1.10 and v1.40 are listed as follows:

    - SMSSMEX v1.40 adds the following codes to the ovictex.properties file:

      ```
      # default category
      ovhd.incident.default.category = incident
      ```

    - SMSSMEX v1.40 introduces the following code changes to the FieldMapping.xml file:

v1.10	v1.10 patch 2/v1.40
<FieldMapping ExtHDField="IctHead/ AgentId"> <OutOvHDField>AssigneeName< / OutOvHDField> <OutDataType>Person</OutDat aType> <InOvHDField>AssigneeName</ InOvHDField> <InDataType>Person</InDataT ype> </FieldMapping>	<FieldMapping ExtHDField="IctHead/ AgentId"> <OutOvHDField>Assignee</Out OvHDField> <OutDataType>Person</OutDat aType> <InOvHDField>Assignee</InOv HDField> <InDataType>Person</InDataT ype> </FieldMapping>
<FieldMapping ExtHDField="IctHead/ ShortDescription"> <OutOvHDField>BriefDescript ion</ OutOvHDField> <InOvHDField>BriefDescripti on</ InOvHDField> </FieldMapping>	<FieldMapping ExtHDField="IctHead/ ShortDescription"> <OutOvHDField>Title</OutOvH DField> <InOvHDField>Title</InOvHDF ield> </FieldMapping>
<FieldMappingExtHDField="Ic tIncident Statement/Text"> <OutOvHDField>Resolution</ OutOvHDField> <InOvHDField>Resolution</ InOvHDField> <KeyFieldOutVal>SU99</ KeyFieldOutVal> <KeyFieldInVal>SU01</KeyFie ldInVal> </FieldMapping>	<FieldMapping ExtHDField="IctIncidentStat ement/ Text"> <OutOvHDField>Solution</Out OvHDField> <InOvHDField>Solution</InOv HDField> <KeyFieldOutVal>SU99</KeyFi eldOutVal> <KeyFieldInVal>SU01</KeyFie ldInVal> </FieldMapping>

8. Manage the license. See License Management.
9. Start SMSSMEX v1.40.

1.3.1.5. Customizing Service Manager

This chapter describes the customization required for Service Manager for the SAP Solution Manager integration.

Note

Some configuration topics are required only for PD environment or for non-PD environment, whereas some are required for both. Unless otherwise noted in the step heading, a topic is required for both.

1.3.1.5.1. Configuring the denyattachwithoutext parameter

When the Service Manager attachment white list is enabled, you must add the *denyattachwithoutext* startup parameter to the sm.ini file. Otherwise, if the web service requests did not send the extension name of the attachement, the attachment is rejected and the request fails.

For more information, see Startup parameter: denyattachwithoutext.

1.3.1.5.2. Importing Customizations Using Unload

This section describes how to configure Service Manager using unload. Additional customization of Service Manager is later required for the integration.



Note

Ignore this topic if you are working with Service Manager applications version 9.5x or above.

Core unload

Unloads are used to transfer customizations from one Service Manager installation to another Service Manager installation. The Incident Exchange provides core unloads in <SMSSMEX1.30 Release Package>\unloads\SM9.34-9.4x\core.unl for Service Manager 9.34, 9.35, and 9.4x.

This unload contains new Service Manager records that are unique to Incident Exchange and don't override any existing Service Manager records.

Follow these steps to import the unload do the following:

1. On the Service Manager client select **Tailoring** > **Database Manager**.
2. Select **Import/Load** from the menu.
3. Select <SMSSMEX1.40 Release Package>\unloads\SM9.34-9.4x\core.unl for Service Manager 9.34, 9.35, and 9.4x.
4. Click **Load FG** to start the import.

Contents

- 1 Core unload

1.3.1.5.3. Enabling the Integration

Follow these steps to enable the Service Manager integration with SAP Solution Manager:

1. Sign in to Service Manager Windows client as a system administrator.
2. Type `sl` in the Service Manager command line field, and then press Enter.
3. Type `SMSAP_Enable` in the Name field, and then click **Search**.
4. (Optional) Locate the following codes:

   ```
   var incidentCategory = "incident";
   var isCustomizateSMFormat = true;
   var isGenerateIntegrationDemoData = true;
   var isGenerateIntegrationDemoUser = true;
   ```

   You can update the values as follows:

   - The value of `incidentCategory` must be consistent with that in the `ovictex.properties` file.
   - When setting `isCustomizateSMFormat` to `true`, the integration enablement script customizes the Service Manager forms and you can skip the steps as described in Configuring Incident form (Hybrid Only) and Configuring Incident Form (Codeless Only).
   - When setting `isGenerateIntegrationDemoData` to `true`, the integration enablement script generates the **sapinstance200** CI and the **sapinstance300** CI, and two CI relationships, respectively.
   - When setting `isGenerateIntegrationDemoUser` to `true`, the integration enablement script creates an operator named **ovictex** with the password of **Ovictex123**. You can skip the steps as described in Creating a Service Manager user for web service.
5. Click **Execute**. When the process completes, the Messages tab displays the `enable SM SAP integration done!` message.
6. Sign out and sign in again for the configuration to take effect.

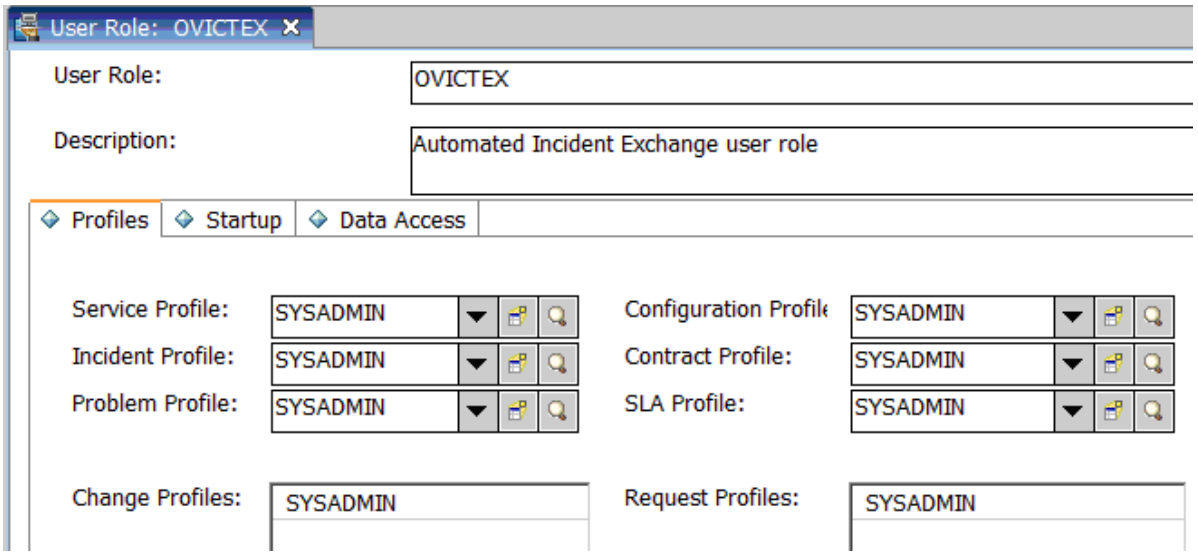


Note

The integration enablement script creates a new workflow named `Incident for SAP`, and adds necessary rule sets, actions, and transitions.

1.3.1.5.4. Creating a Service Manager User for Web Service

Incident Exchange uses one Service Manager user to connect to Service Manager web services. The user and the user role are for the integration only. The user requires the following permissions:



Do the following:

1. Sign in to Service Manager with a System Administrator account.
2. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **User Roles**.
3. Search for `system administrator`.

   > **Tip**
   >
   > In case your database is case-sensitive, try to use all lowercase search keywords instead of all UPPERCASE ones, or vice versa.

4. Enter `OVICTEX` as the User Role.
5. Change Description to Automated Incident Exchange user role.
6. Click **Add**.
7. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **User Quick Add Utility**.
8. Enter `ovictex, INCIDENT EXCHANGE, Incident, Exchange, ovictex@microfocus.com`.
9. Click **Next**.
10. Select **falcon** for **User to clone**.
11. Click **Finish**.
12. Click **Save**.

13. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Operators**, enter `ovictex` in the Login Name field, then click **Search**.
14. Change User Role to `OVICTEX`.
15. On the Security tab:
    1. Enter the operator password for Password.
    2. Clear the **Expire Password** checkbox.
    3. Check **Never Expire Password**.
16. Click **Save**.

1.3.1.5.5. Configuring Incident Form (Classic/Hybrid Only)

Incident Exchange must be integrated into the incident management workflow. The operator working on the incident must be able to control and trigger Incident Exchange. If more than one external helpdesk is connected to Service Manager, then the target system must be selected.

Note

If you are working with SM 9.40 Classic, you must apply the QCCR1E127035_SM940_SM940.unl unload file to fix QCCR1E127035. The unload file is available in the %SMSSMEX_HOME%\unloads\SM9.34-9.4x directory.

Follow these steps to configure incident forms in non-PD environment:

1. Open all Incident Forms that are parts of the Incident workflow.

   IM.open.incident

   IM.update.incident

   IM.close.incident

2. Embed the created subform on the Incident form in a new Notebook tab (or in a new section on Service Manager 9.2x and above).

3. Add a Notebook tab or section to the following forms:

   IM.open.incident

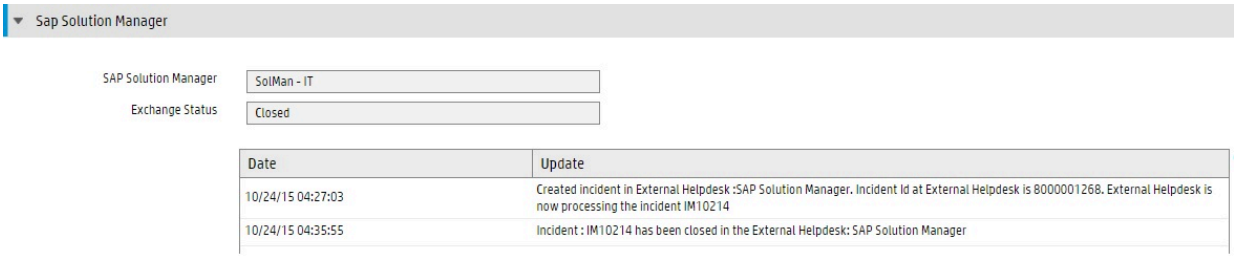
   IM.update.incident

   IM.close.incident

Property	Value
Caption	Sap Solution Manager
Visible condition	[$SMSAP]=true

4. Add a subform control to the SAP Solution Manager tab or section.

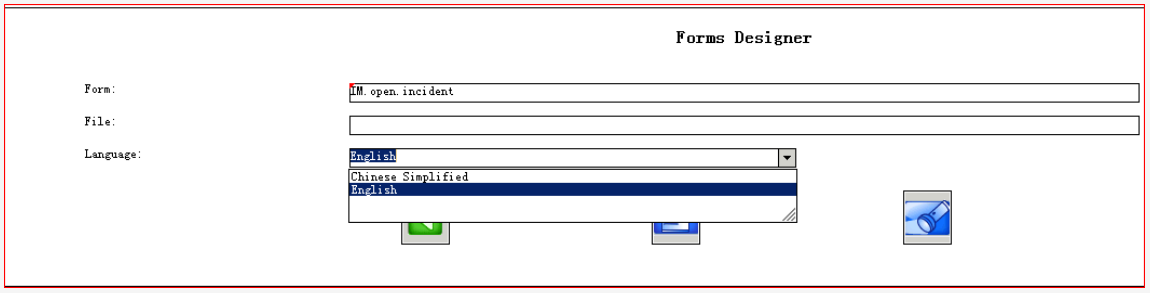
Property	Value
X	1
Y	0
Width	151
Height	28
Format	hp.sap.solution.sub



Note

If multiple Language packs are applied to Service Manager, do the following to update the incident related forms for other languages.

1. Copy the `hp.sap.solution.sub` form from English to other languages and perform translation.
2. Repeat *step 3* and *step 4* for `IM.open.incident`, `IM.update.incident` and `IM.close incident` for other languages.



1.3.1.5.6. Configuring Incident Form (Codeless Only)

To configure the incident form in PD environment, follow these steps to add a SAP Solution Manager tab for the following formats:

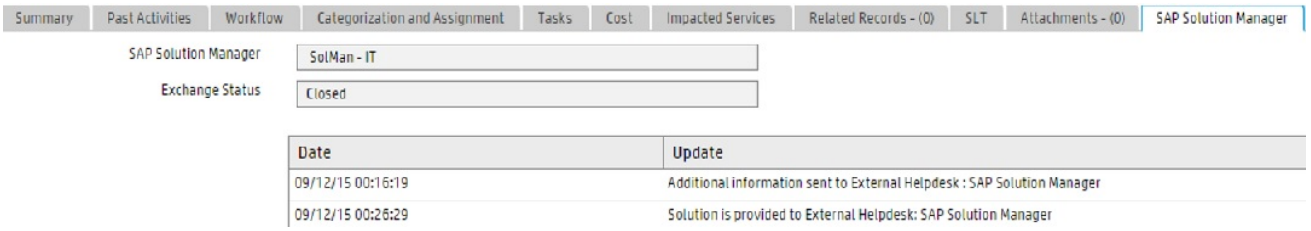
im.incident.categorization

im.incident.investigation

im.incident.recovery

im.incident.review

im.incident.closure



Note

If you are working with either SM 9.40 Codeless or SM 9.41 Hybrid, pay attention to the following items before adding add SAP Solution Manager tabs for various incident formats.

- If you are working with SM 9.40 Codeless, you must apply the QCCR1E127035_SM940_SM940.unl unload file to fix QCCR1E127035. The unload file is available in the %SMSSMEX_HOME%\unloads\SM9.34-9.4x directory.
- If you are upgrading SM 9.31 with PD3 to SM 9.41 Hybrid, you must apply a hotfix for QCCR1E128778 during the upgrade process. The hotfix is available from Software Support.

1. Click **Tailoring** > **Forms Designer**.
2. Type im.incident.categorization and click **Search**.
3. Add a new notebook tab with the following configurations:

Property	Value
Name	sapsolutionmanager
Visible	True
Visible condition	[$SMSAP]=true
Caption	Sap Solution Manager

4. Add a subform control to the SAP Solution Manager tab.

Property	Value
Format	hp.sap.solution.sub

5. Click **Save**.
6. Search for `im.incident.investigation`, `im.incident.recovery`, `im.incident.review`, and `im.incident.closure` respectively and repeat *step 3* to *step 5*.

1.3.1.5.7. Configuring WSDL Mapping

Configure the `IncidentManagement` WSDL Mapping table in WSDL Configuration of Service Manager as follow:

Field name	Caption
action	Description
assignee.name	Assignee
brief.description	Title
initial.impact	Impact
assignment	AssignmentGroup
product.type	ProductType
resolution	Solution
subcategory	SubCategory
severity	Urgency

1.3.1.5.8. Adding Instance in SMIS and Configuring Parameters

The SMSAP instance in SMIS integration is used to enable and disable the SMSAP integration. By configuring the SMSAP instance, you can also specify the integration parameters such as SMSSMEX accessing URL and SAP clients. After the SMSAP instance is enabled, the customization to Incident is visible and you can select a SAP Solution Manager client for Incident information exchange. If the SMSAP instance is disabled or removed, the Incident form customization for Incident exchange with SAP Solution Manager will be invisible to the end users. Note that the Incident information exchange from SAP SolMan to Service Manager will not be affected by SMSAP SMIS configuration. For more information about SMIS, refer to Integration Manager.

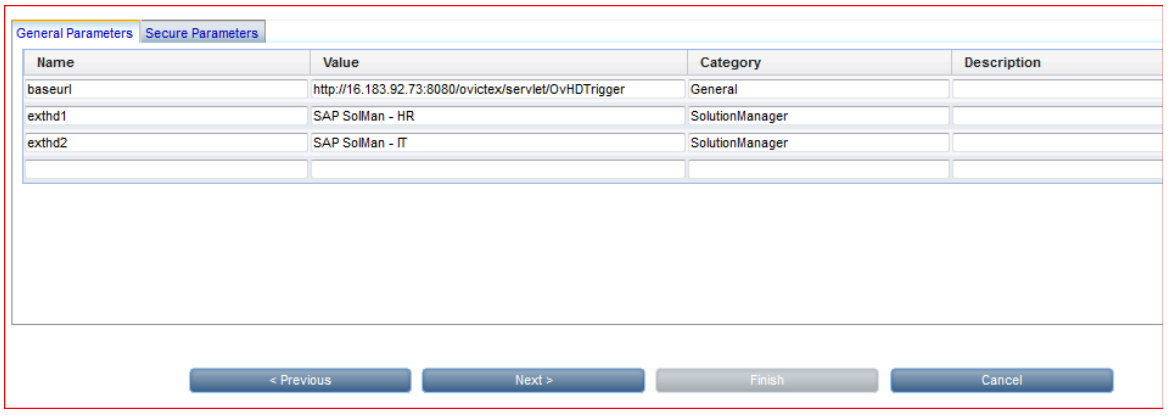
The SM-SAP integration operates only when the SMSAP instance in SMIS is enabled. Do the following to add a new instance in SMIS and configure the parameters:

1. Sign in to Service Manager with a System Administrator account.
2. Select **Tailoring** > **Integration Manager**. Integration Manager opens.
3. Click **Add** to open the wizard.
4. Select **SMSAP**, click **Next**.
5. Click **Next** to configure the parameters.
    1. Configure the baseurl for connection to SMSSMEX.

       Replace `<host>` and `<port>` according to the middleware installation.
    2. Configure SAP Solution Manager clients.

       Add general parameters for each of your SAP Solution Manager clients configured in SMSSMEX `ovictex.properties`. For example, `exthd.instances.id.1`, `exthd.instances.id.2` and so on.

       Note: Be sure to set your SAP Solution Manager client instance ID in the Name field, and `SolutionManager` in the Category field of each record. Value of each SAP Solution Manager client instance ID is the reference in Service Manager.



6. Click **Next** twice at the end of the wizard and click **Finish** to save the configurations.
7. Select the SMSAP instance and click **Enable** to enable the SM-SAP integration.

1.3.1.6. Configuring SAP Solution Manager

This chapter describes how to configure the SAP Solution Manager.

1.3.1.6.1. Prerequisites

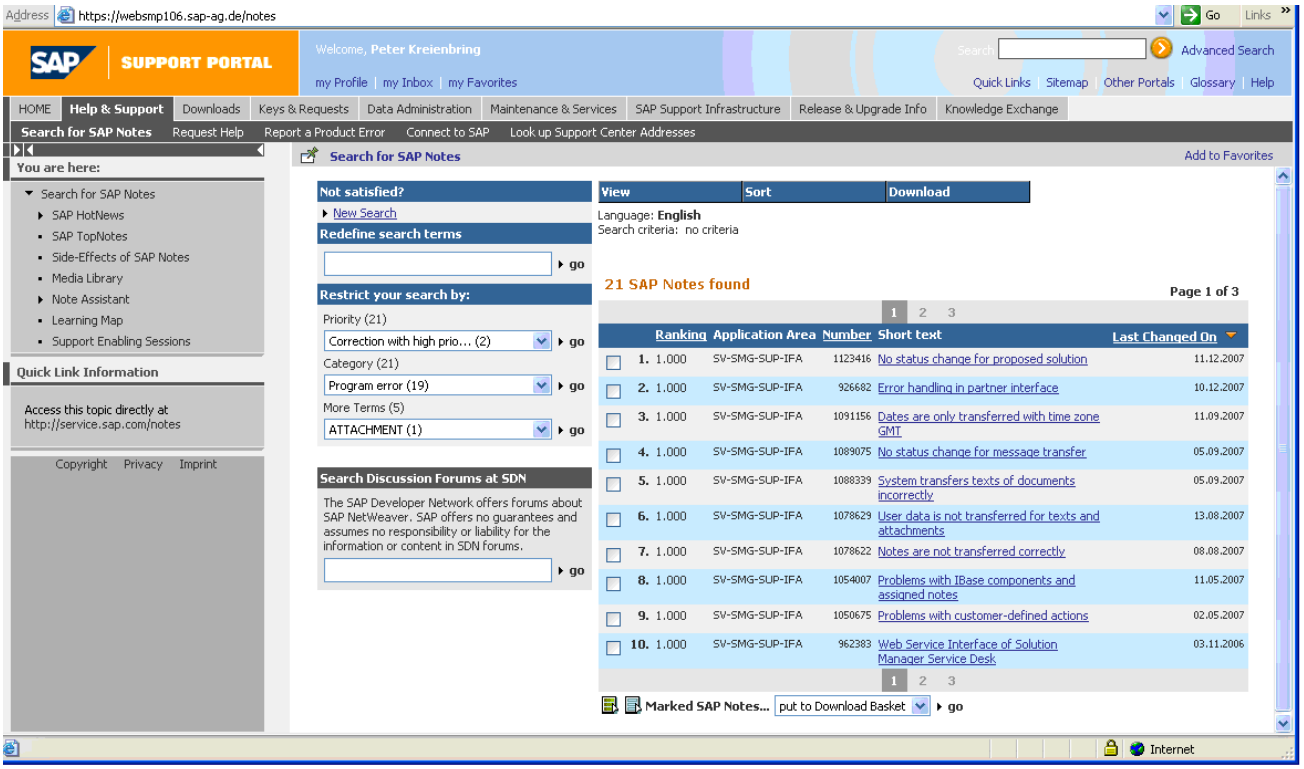
The prerequisites are:

- SAP Solution Manager 7.0 SP 12 (or higher) or SAP Solution Manager 7.1
- SAP Solution Manager SP12 if copying of business transaction SLFN for customization in a customer name space (for example ZLFN) is required
- SAP Solution Manager SP12. Required to copy a business transaction into a customer name space for customization (for example, to copy business transaction `SLFN` into customer name space `ZLFN`)
- Configured SAP Solution Manager Service Desk

Configured SAP Solution Manager Service Desk SSL encryption between SAP Solution Manager and Apache Tomcat requires:

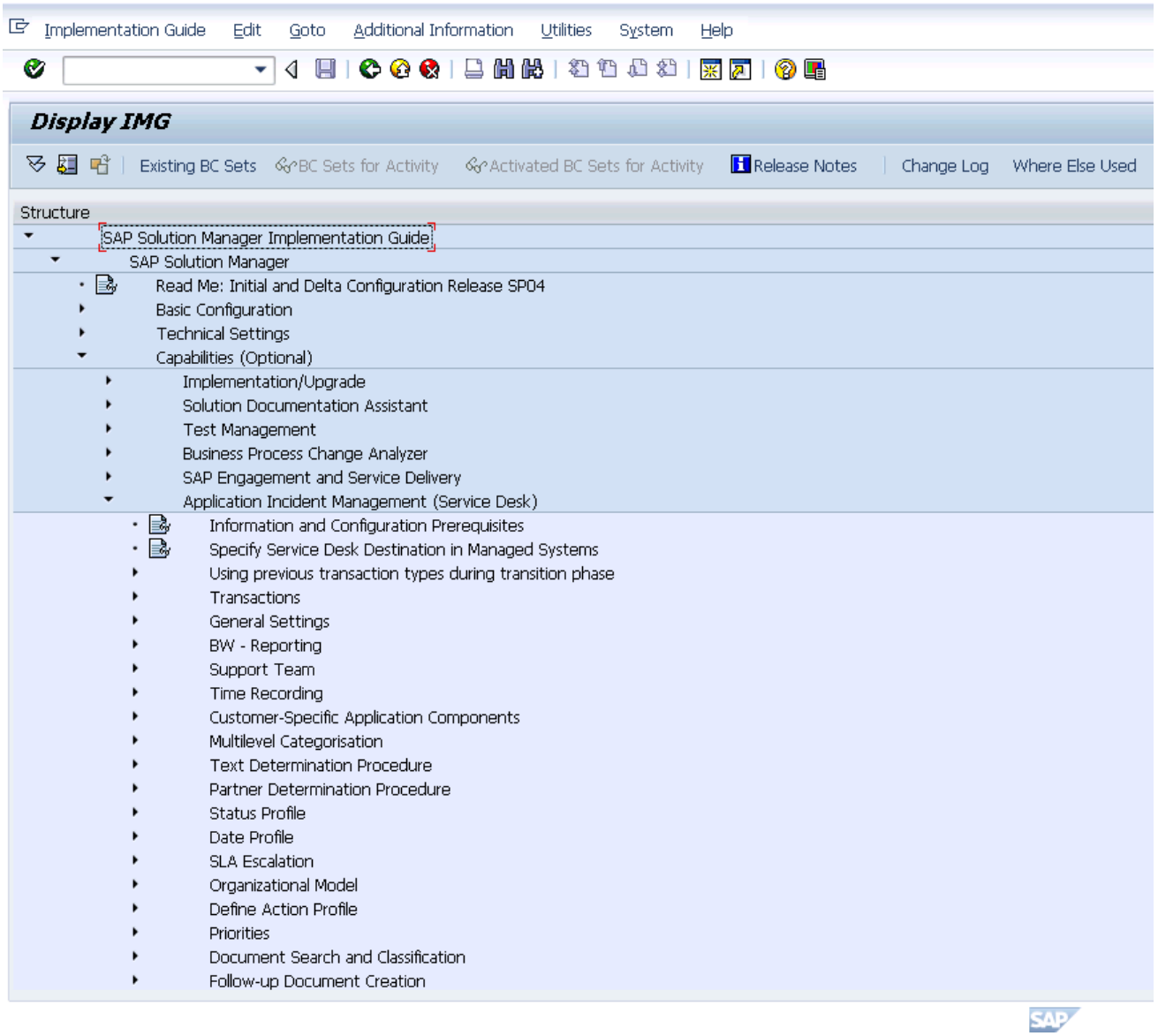
- Sapcryptolib 5.5.5C or higher
- SSL Server and SSL Client PSE
- SSL Server and SSL Client certificates trusted against a CA

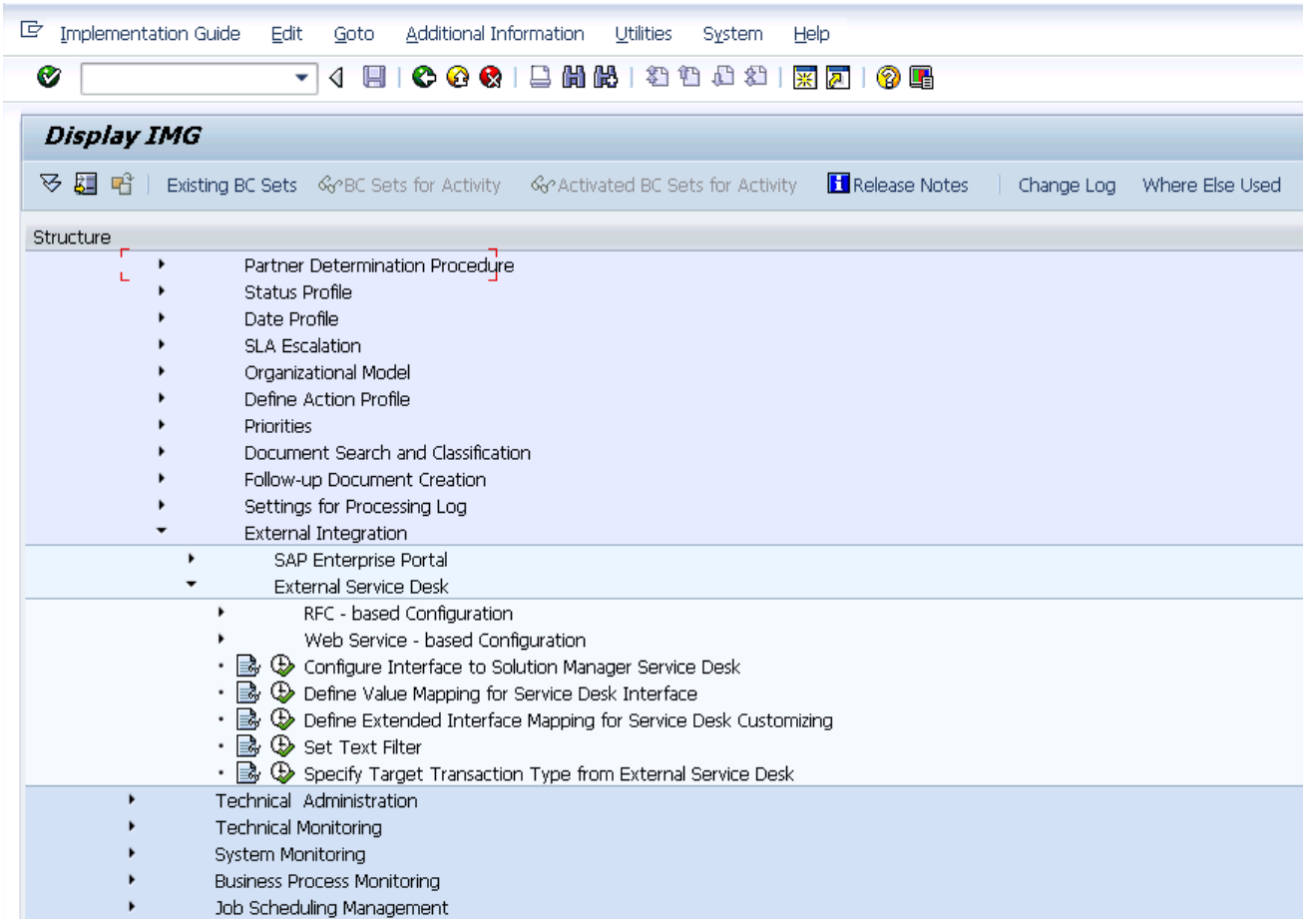
Integration with Service Manager requires implementation of the latest SAP notes (SAP application area SV-SMG-SUP-IFA) for the SP level stack of SAP Solution Manager. The following diagram shows the search results of SAP notes in the SAP Support Portal.



1.3.1.6.2. Configuring SAP Solution Manager External Service Desk interface

SAP provides the Implementation Guide “External Service Desk” for configuring the external help interface. The Implementation Guide is located in SAP transaction `/nspro` under path `\SAP Solution Manager ImplementationGuide\SAP Solution Manager\Capabilities (Optional)\Application Incident Management (Service Desk)\External Integration\ External Service Desk`. The following diagrams show the Implementation Guide for configuring the connection to the external Service Desk.



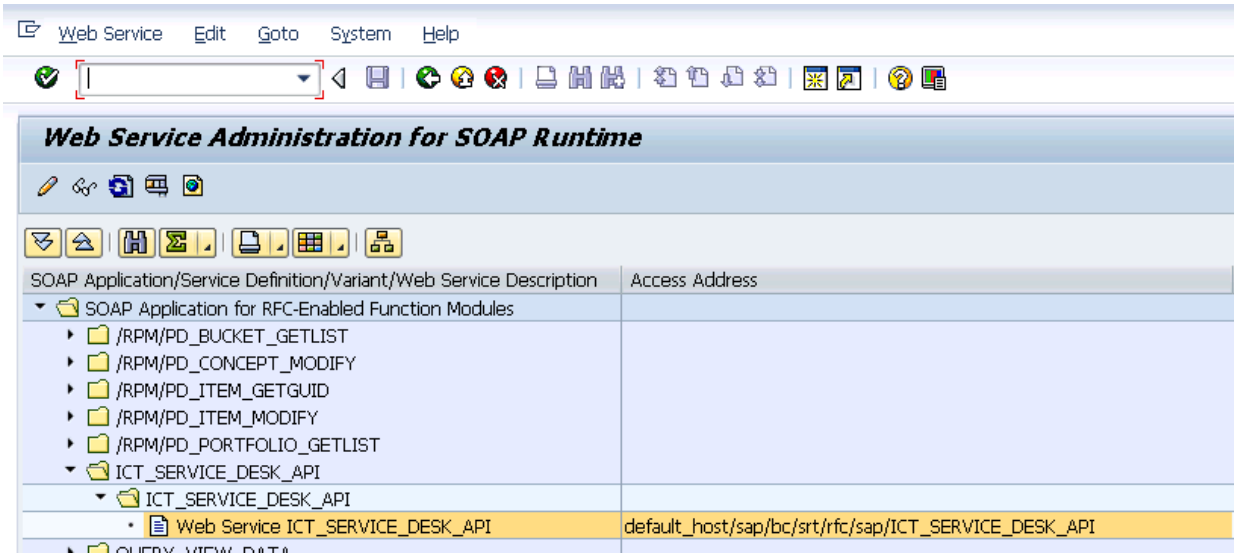


Click the leftmost text sign to view configuration steps. Click the clock sign to enter the corresponding transaction and edit the configuration.

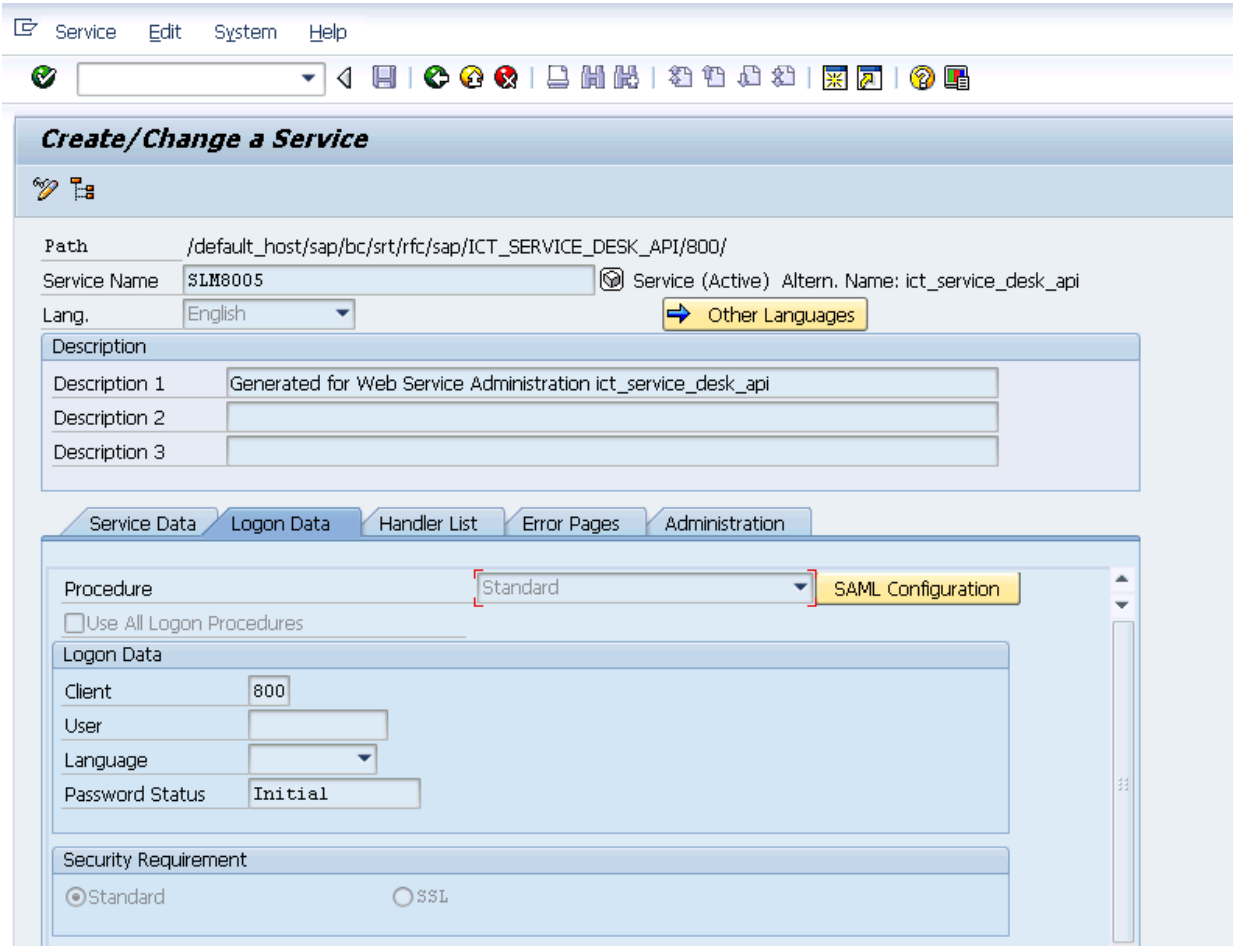
1.3.1.6.2.1. Release Web Service

The Incident Exchange Web Service is deactivated by default. It is required to release the WEB-Service in the Internet Communication Manager Service tree.

1. The transaction to release a Web Service is `/nwsconfig`. After the Web Service is released the WS is in SAP transaction `/nwsadmin`.



2. To determine the logon procedure of the Web Service for incoming requests, go to SAP transaction `/nsicf`.
3. Enter ICT_SERVICE_DESK_API as the service name.
4. Click **Execute** to execute the search.
5. Double-click the Service to edit or navigate to the path `/default_host/sap/bc/srt/rfc/sap/` and select **ICT_SERVICE_DESK_API**.
6. In the Logon tab of Create/Change a Service dialog , select **Standard**.



Note

The security section of this manual contains additional information for setting up SSL communications. Adding a user is not required. The Incident Exchange Web Service will use the user and password that's configured in the properties file for HTTP Basic authentication. This user must exist as an SAP user. It's not recommended to use a dialog user for this purpose.

1.3.1.6.2.2. Assign Roles to the Communication User

Configure an SAP user with permission to manage incidents in SAP Solution Manager Service Desk. Follow the instruction in the Implementation Guide and add the roles `SAP_SUPPDESK_PROCESS` and `SAP_SUPPDESK_INTERFACE` to the user. Exchanging a business partner with a default configuration interface requires the additional role `SAP_CRM_BUSINESS_PARTNER`.

To configure a user:

1. Select transaction `/nsu01`.
2. Input the name of the user.
3. Click **Display**. The user configuration transaction appears.



Note

- A person who is assigned to an incident in Service Manager but doesn't exist in Solution Manager will be created as a Business Partner when the incident is forwarded to Solution Manager. Without the business partner role `SAP_CRM_BUSINESS_PARTNER` the incident can't be created or updated in Solution Manager and the error code 99 appears.
- A communication user is recommended, but not necessary.

Sending support messages to SAP AGS requires assigning an SAP Support Portal contact to Solution Manager users who will communicate with the SAP Support Portal via RFC connections. The contact maintained corresponds to the S-user in the SAP Support Portal without “S”. See SAP Note 834534 and the SAP Solution Manager configuration guide for details of Solution Manager roles and authorizations.

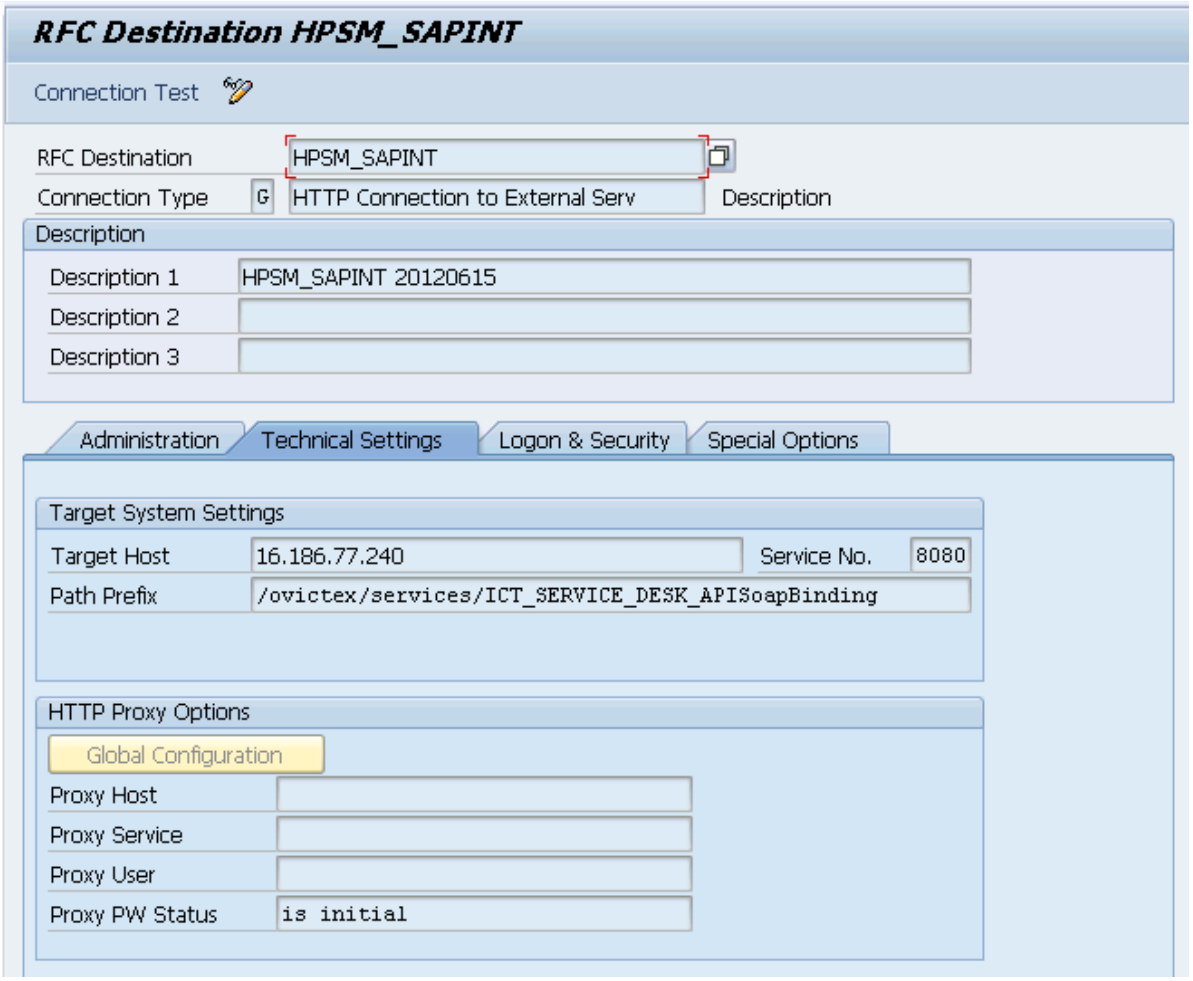
1.3.1.6.2.3. Create HTTP Connection

Define the endpoint of the SMSSMEX Web-Service for communication between SAP Solution Manager and Apache Tomcat.

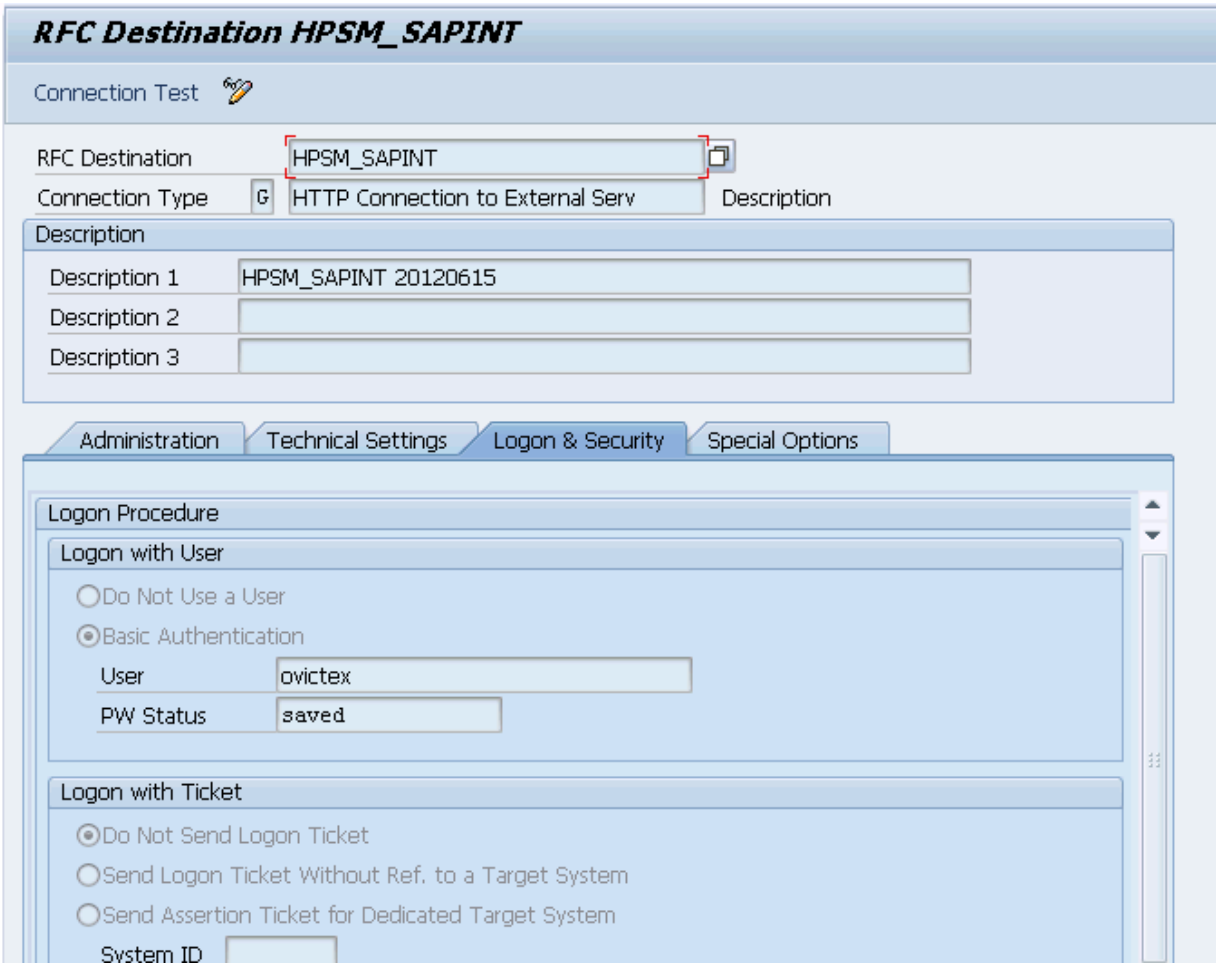
1. Select transaction `/nsm59`.
2. Create an RFC destination of type `G` (HTTP connection to external server).
3. Go to the **Technical settings** tab and specify the endpoint of the SMSSMEX Web-Service. The default is:

```
Target Host: <host>
Service No: <port>
Path Prefix: /ovictex/services/ICT_SERVICE_DESK_APISoapBinding
```

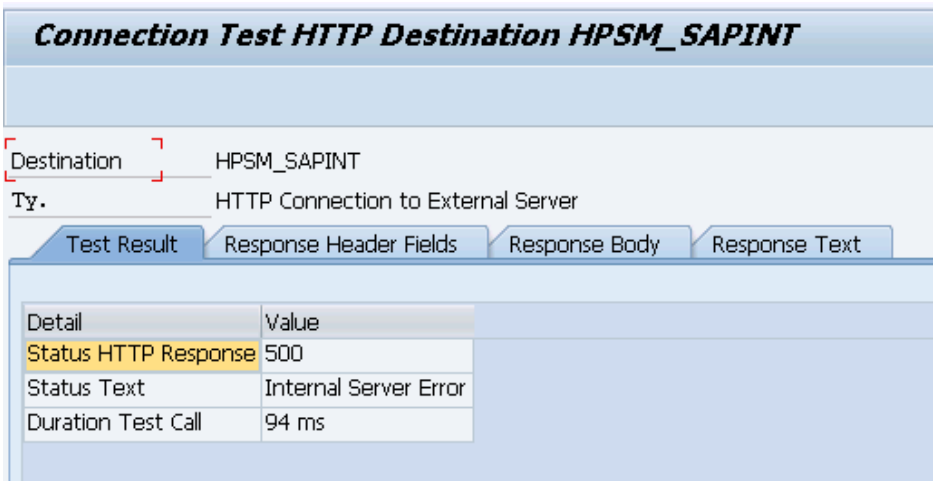
4. Add the endpoint in the RFC destination. Your network configuration may require specification of a proxy. The following example shows the RFC destination for host `itsamqavm130`.



5. In the Logon & Security tab, define the security settings for outgoing requests. Select **Basic Authentication** for HTTP basic authentication. Add the user and password specified in `ovictex.properties` for HTTP basic authentication. The more secure SSL communication configuration is described in the security chapter of the manual. You can also select **No Logon** which is the default selection for “Logon&Security”.



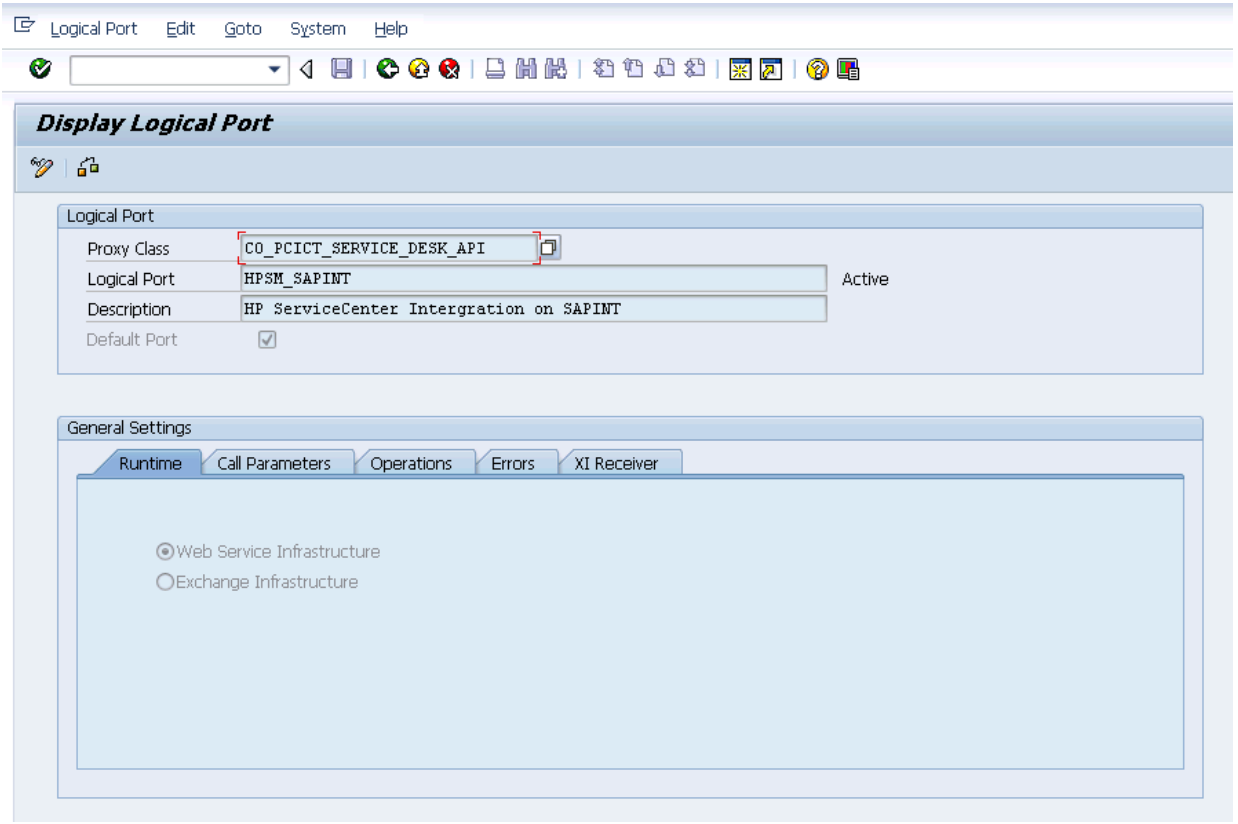
The following diagram shows the SMSSMEX Web service returning error 500. This result indicates the connection between SAP and SMSSMEX is established.



1.3.1.6.2.4. Create a Logical Port

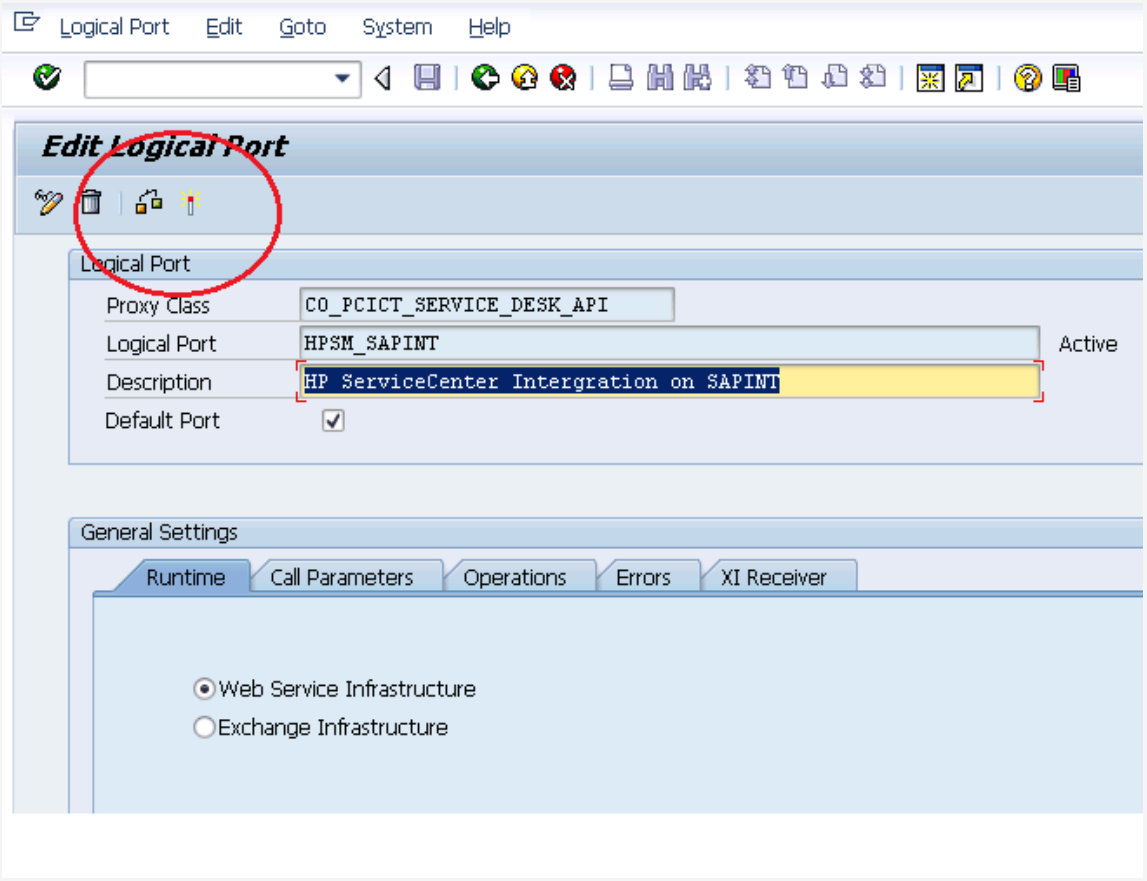
The logical port is the container that encapsulates the outgoing requests. Define the logical port as specified in the Implementation Guide instructions.

1. Go to transaction `/nlpconfig`.
2. Select **CO_PCICT_SERVICE_DESK_API** as the Proxy Class name.
3. In Call Parameters tab add the HTTP destination configured in the previous chapter.



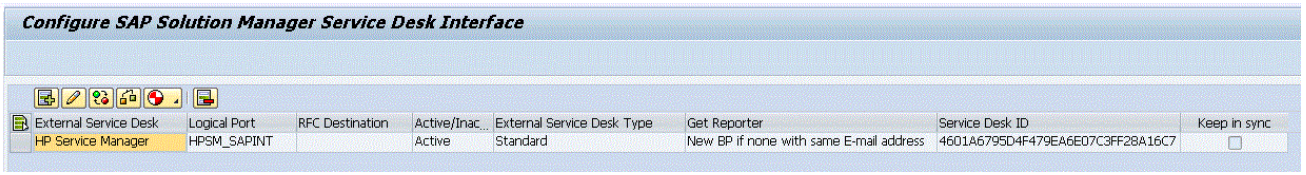
Note

The port must be activated. Click **Active** to activate the logical port.



1.3.1.6.2.5. Configure Interface to SAP Solution Manager Service Desk

This activity configures the interface between the SAP Solution Manager service desk and Service Manager. Follow the instructions in the Implementation Guide. The configuration requires that Apache Tomcat and the web service are configured and running. In this implementation step the SMSSMEX web service must deliver a unique Service Desk ID. If the Service Desk ID is changed, then the configuration must be repeated.



Use the **Check** button to verify the configuration. Any error message will be displayed in the output window. Use transaction `/nictconf` to jump to configuration transactions.

Note

Do not select the **Keep in sync** checkbox when configuring the interface to SAP Solution Manager service desk.

Note

If the check fails, try **Generate Default Mapping** > **Overwrite Old Values** and then run the check again. After configuration, click **Save** to save the configured interface.

1.3.1.6.2.6. Define Value Mapping for the Service Desk Interface

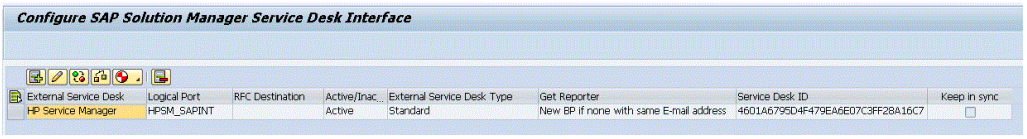
This IMG activity configures the value mapping between SAP Solution Manager Service Desk and SMSSMEX for ingoing and outgoing requests. Changing the default value mapping of the SAP Solution Manager is not required. If changes are necessary, use the field mapping file of the SMSSMEX configuration file. To change the default Mapping of the SAP Solution Manager, consult the instructions in the implementation guide.

1.3.1.6.2.7. Define Extended Interface Mapping for Service Desk

If SAP Solution Manager Service Desk is highly customized (not using standard SAP objects) then it might be necessary to change the interface mapping. The IMG activity instructions provide more information.

1.3.1.6.2.8. Get SAP Solution Manager Service Port

Go to SAP transaction `/nsmicm`. Select **Goto** > **Services**.



This transaction shows the host and the port required for access to the SAP Solution Manager Service Desk web service. Specify in `ovictex.properties` the host/port as the endpoint entry.

1.3.1.6.3. Solution Manager tracing

SolutionManager is able to trace incoming and outgoing web-service XML messages. The messages can be downloaded and used for failure analysis.

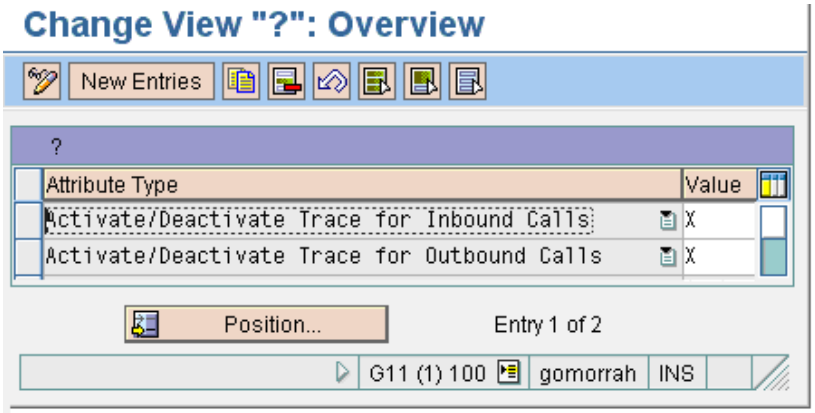
1.3.1.6.3.1. Enable tracing

To enable tracing, implement a SolutionManager Implementation Guide in transaction `/nspro`.

```
spro -> SAP Reference IMG ->
SAP SolutionManager Implementation Guide ->
SAP SolutionManager ->
Configuration ->
Scenario-Specific Settings ->
Service Desk ->
Connecting an External Service Desk ->
Define Extended Interface Mapping for Service Desk Customizing
```

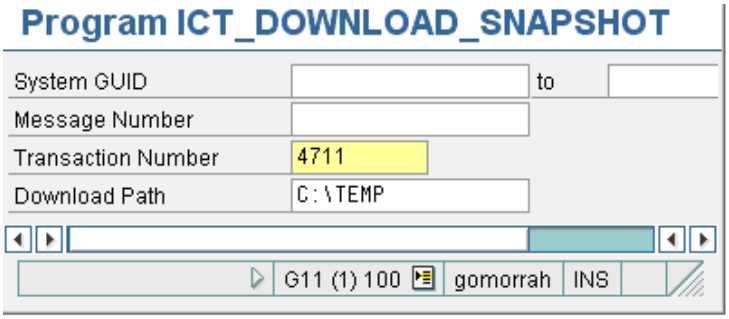
Add new entries to enable tracing for incoming and outgoing calls:

- `Activate/Deactivate Trace for Inbound Calls = X`
- `Activate/Deactivate Trace for Outbound Calls = X`



1.3.1.6.3.2. Download Trace File

To download the trace file, run `ict_download_snapshot` in transaction `/nse38`. Enter the Solution Manager incident id in the field `Transaction Number` and run the program (`F8`). The trace file will be downloaded to the local computer (for example, incident 4711 traces will be downloaded to C:/TEMP).



1.3.1.7. Configuring security

This chapter describe the required security configuration settings.

1.3.1.7.1. Security between SAP Solution Manager and Tomcat

This section describes the security configuration between SAP Solution Manager and Tomcat.

1.3.1.7.1.1. Configure SAP Solution Manager for SSL

This section describes how to configure SAP Solution Manager for SSL.

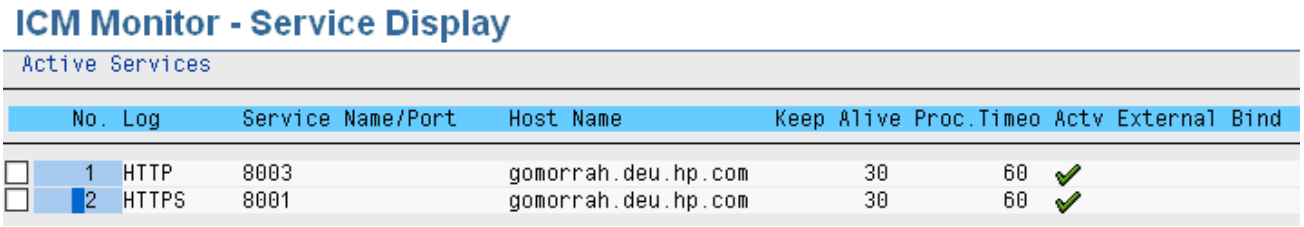
Checking SAP SSL configuration

SAP WEB AS doesn't support or allow self-signed certificates for communication between Solution Manager and the SMSSMEX Web Service. All certificates must be trusted against a CA.

Before configuring SSL for the External Help Desk interface, check if the WEB AS that hosts the SAP Solution Manager is configured for using SSL.

ICM (Internet Communication Manager) HTTPS service is required for SSL communication. Check if SSL communication is possible in SAP transaction `/nsmicm` (select menu entry **GOTO** and select **Services** or press SHIFT+F1).

If SSL communication is possible then an active HTTPS service that's listening to a port is visible. In the example below, the HTTPS port is 8001. This port must be configured in the SMSSMEX web service properties file.

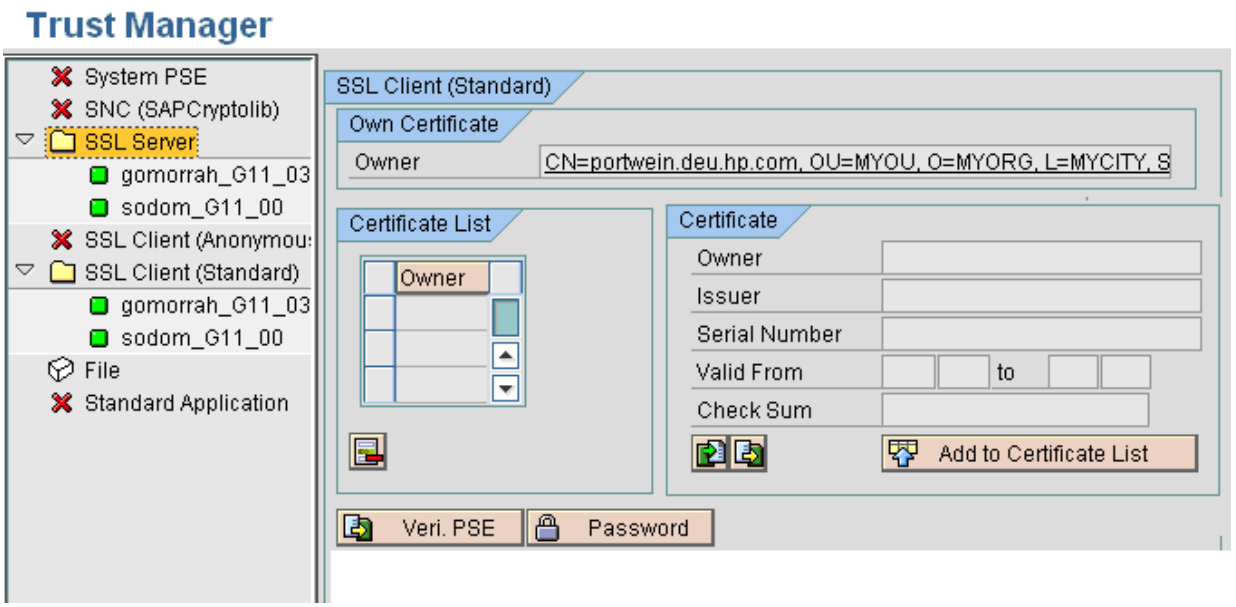


If an HTTPS service in the ICM monitor is not visible, then check the SSL Server configuration in Trust Manager. Start the Trust Manager with SAP transaction `/nstrust`.

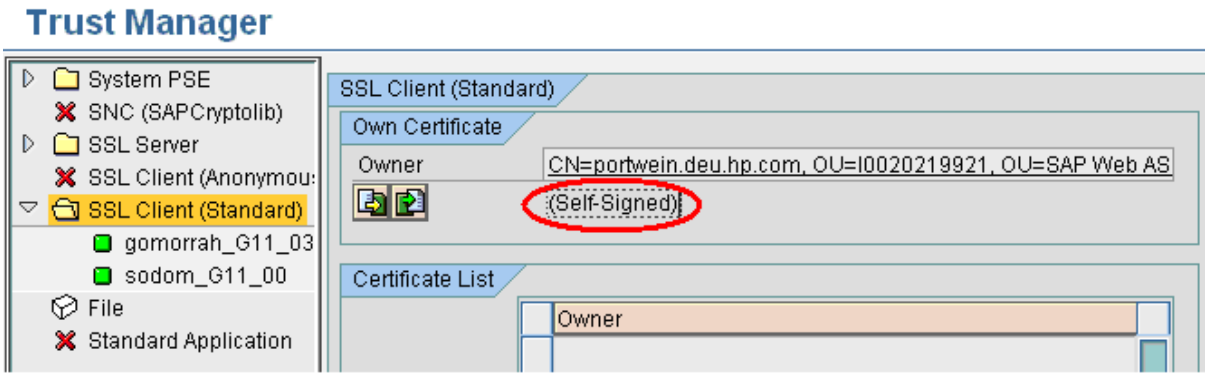
If the PSE entries SSL Server and SSL Client (Standard) aren't shown in the Trust Manager status section, then install and configure the SAP `sapcryptolib` library.

> **Note**
>
> Installing and configuring sapcryptolib requires a restart of the SAP WEB AS instance. The installation instructions are in the SAP online help. For more information, see Installing and configuring SAPCRYPLIB.

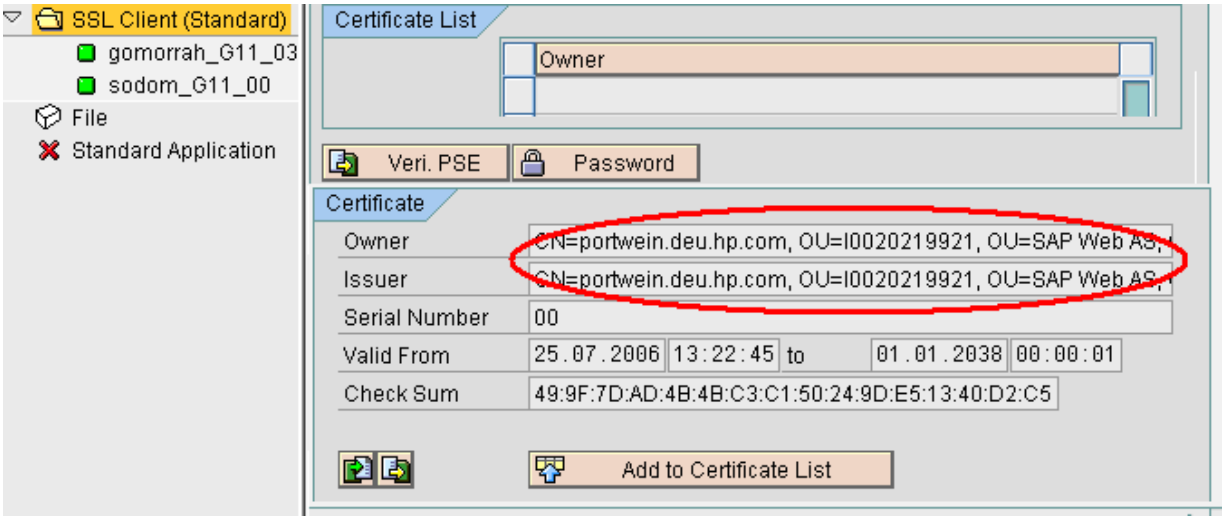
The following diagram shows the Trust Manager with the created PSE **SSL Server** and **SSL Client (Standard)**. The red X in front of the other PSE's indicates that the PSE's have not been created. The PSE **SSL Server** and **SSL Client (Standard)** must be created.



In the next diagram the certificate of the PSE **SSL Client (Standard)** is **Self Signed**. Self-signed certificates aren't supported for communication with Apache Tomcat (the certificate must be signed against a CA). If the certificate is signed, the **Self signed** certificate text will disappear.



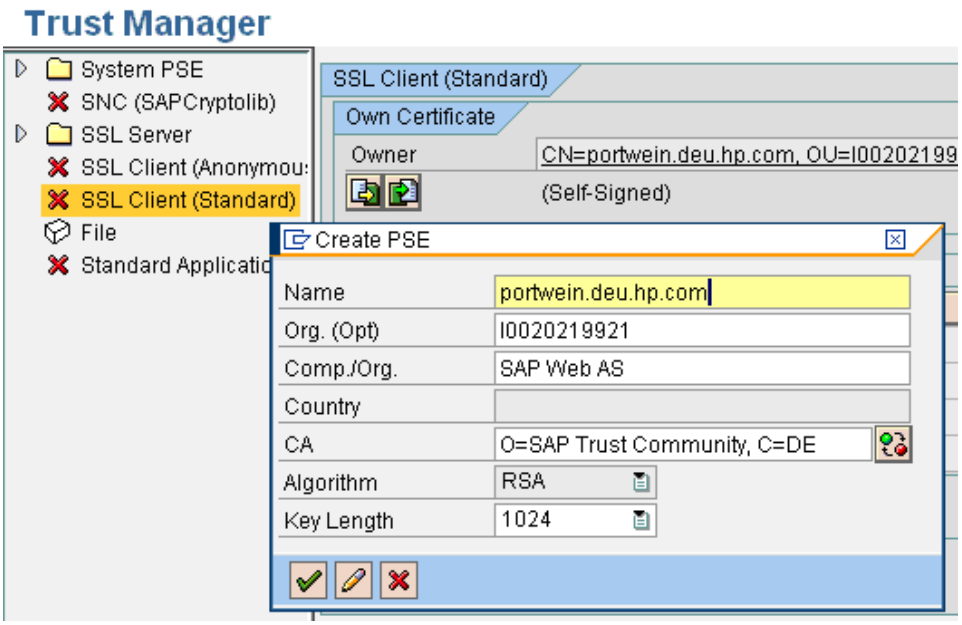
Check the certificate by double-clicking the **Owner** attribute. The certificate details are shown in the Certificate section. If the Owner and Issuer have the same DN the certificate is self-signed.



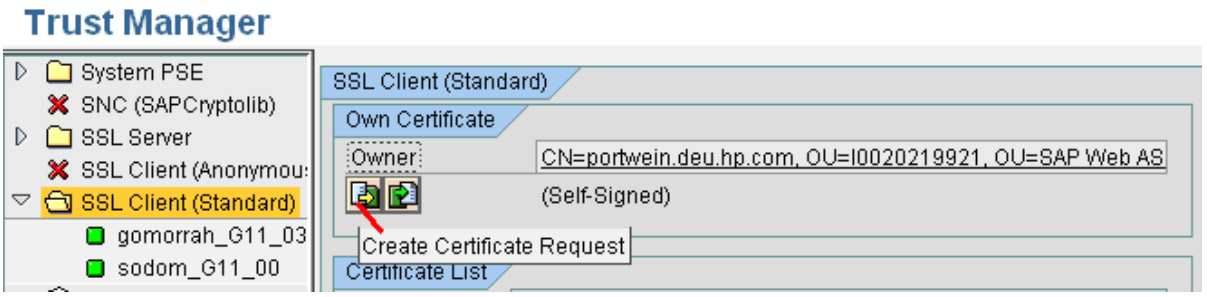
Creating a client PSE in Trust Manager

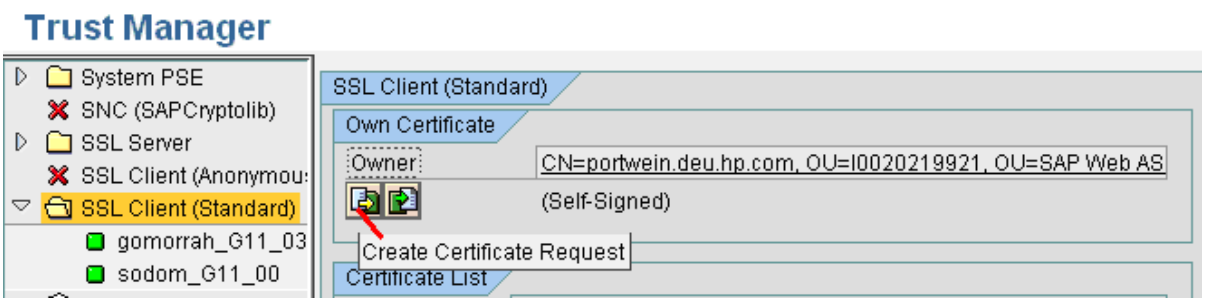
To create a client PSE in Trust Manager, do the following:

1. Start the Trust Manager.
2. Select the **PSE SSL Client (Standard)** in the status section of the Trust Manager.
3. Click **Create**.



4. For the CN (Name) enter the fully qualified hostname of the SAP WEB AS system. All other entries must not be changed. The key length should be 1024.
5. Save the settings.
6. Double-click **SSL Client (Standard)** in the status section. The `Own` certificate in the Own Certificate section is shown.
7. Click **Create Certificate Request**.





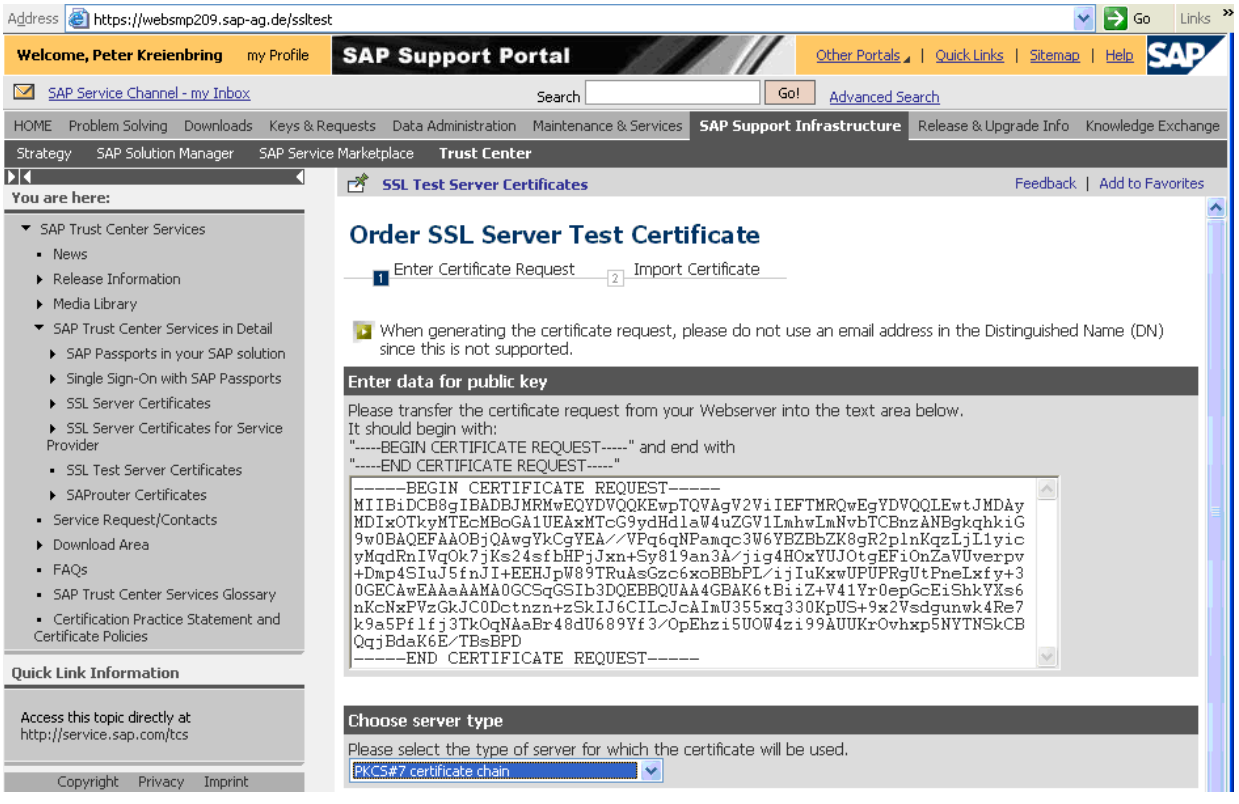
8. The Certification Request is shown. Copy the request to the Clipboard.

9. Certify the request with a CA.

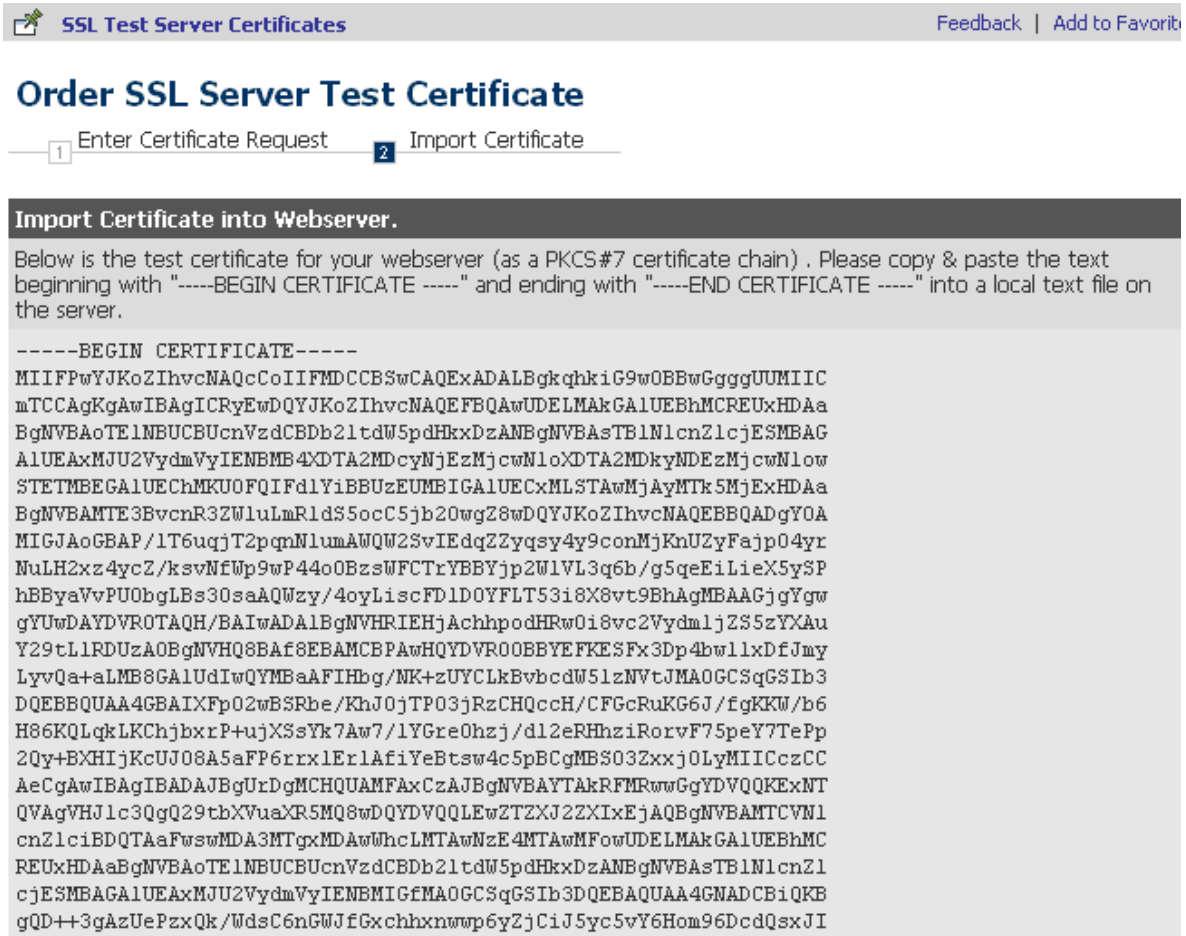


> **Note**
>
> SAP offers a two-month test period for signed certificates in the SAP Service Marketplace at http://www.service.sap.com/ssltest.

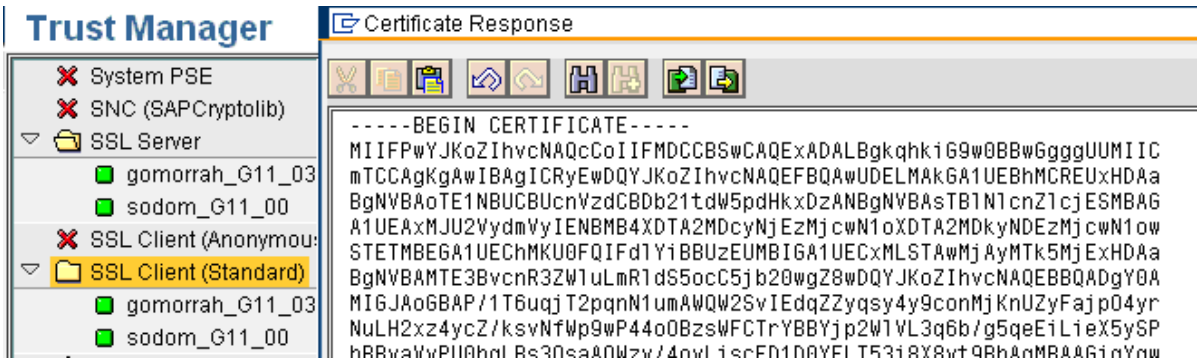
10. Request an SSL Server Test Certificate as shown in the following diagram (select the **PKCS#7** chain format).



11. Click **Continue**. The SSL Server Certificate is created.



12. Copy the Certificate response to your client PSE.

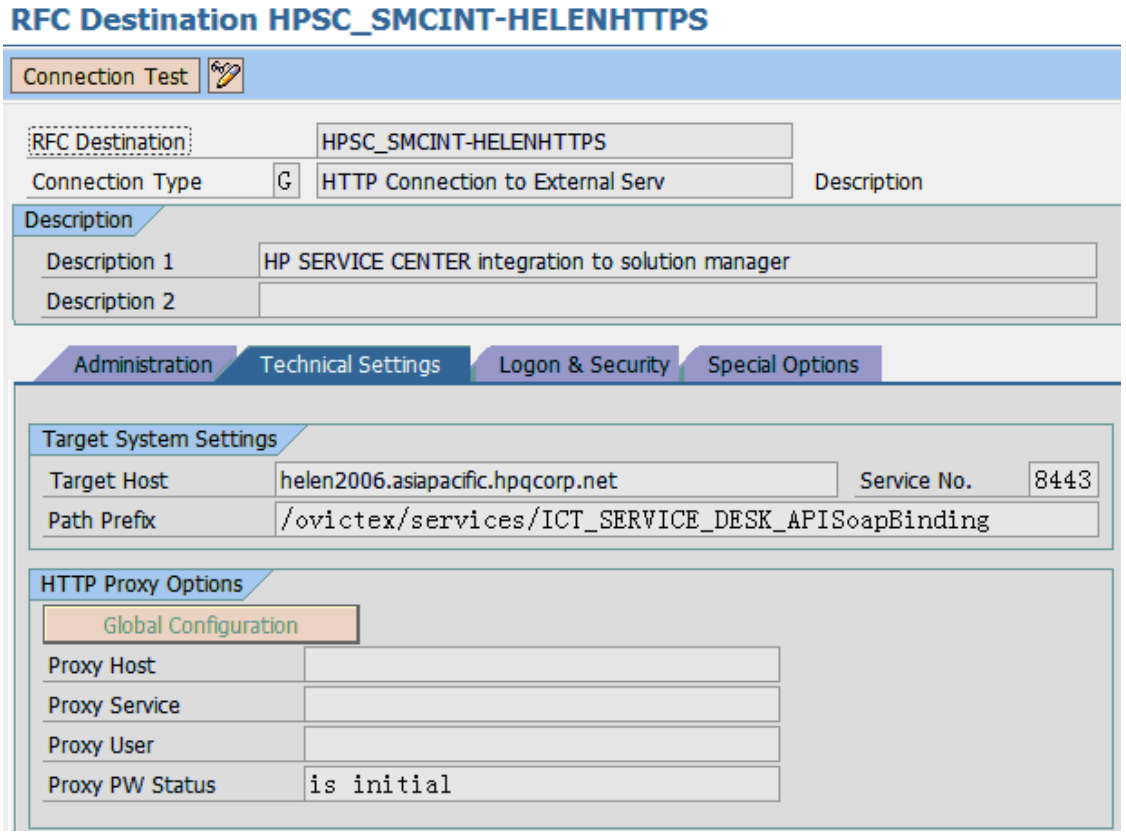


The certificate is now trusted against a CA. The required steps are different for other CA's. Contact the Trust Center for details. A certificate for the SSL Server PSE is also required.

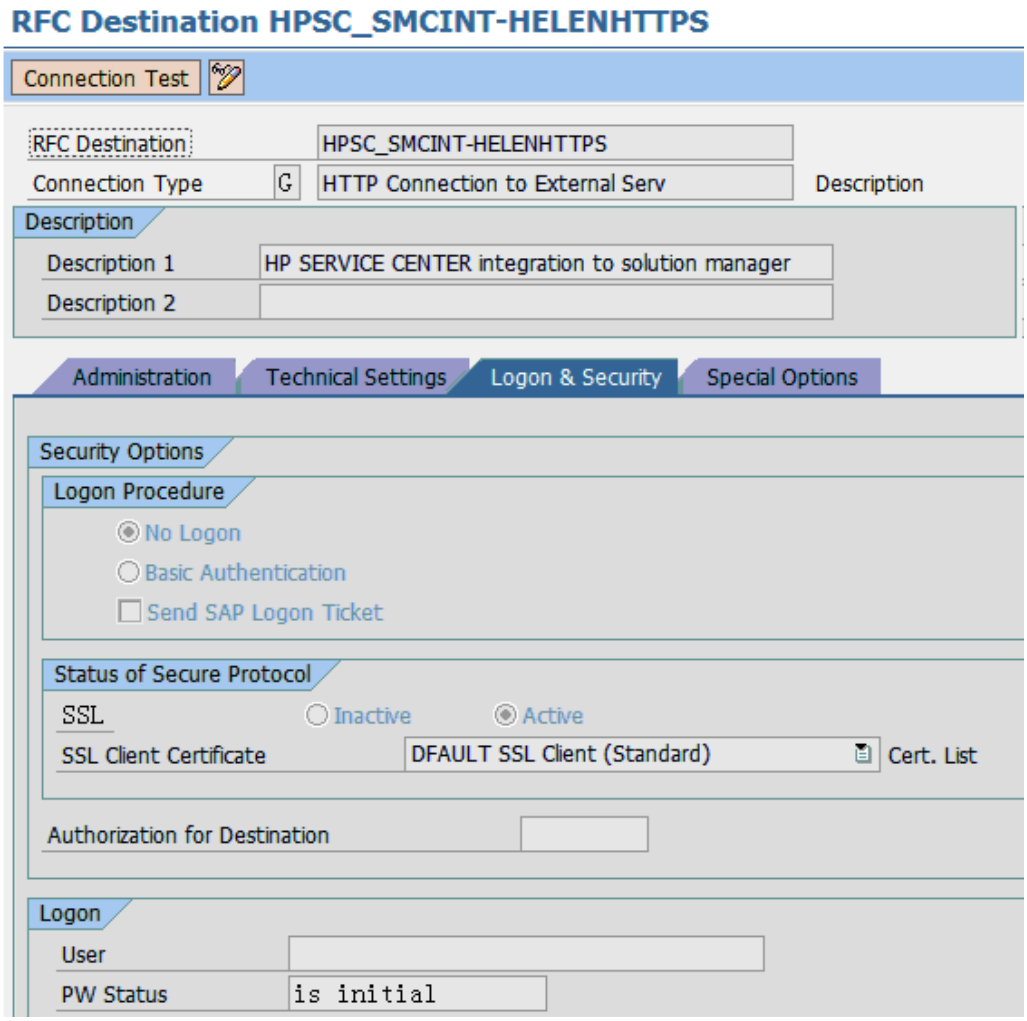
Setting up an outgoing connection in SAP Solution Manager

The outgoing connection from SAP Solution Manager to Apache Tomcat must be configured in SAP transaction `/nsm59`. Add a new or change an existing HTTP RFC destination with type G.

1. In SAP transaction `/nsm59` configure the HTTPS port of the Tomcat Server. A redirect from the HTTP port to the HTTPS port of Apache Tomcat will not work with the SAP WEB AS. The HTTPS port is defined in the `server.xml` configuration file of the Tomcat Server.
2. The SSL configuration of Apache Tomcat is switched off by default. Enable the configuration.
3. In the settings for the SSL HTTP connector, set the Tomcat default port for SSL communication to `8443`.
4. The diagram below shows the example configuration of the RFC Destination (in the `Target Host` field enter the server name (case sensitive) instead of the IP address).



5. In the Logon&Security tab of the RFC configuration define the logon procedure and the security protocol. Basic authorization with SSL communication and certificates is unsupported by Apache Tomcat. Set the Logon Procedure to `No Logon`.
6. In the security protocol status enable SSL and select a PSE from the certification list. SAP provides **PSE ANONYM SSL Client** and **DFAULT SSL Client (Standard)**.



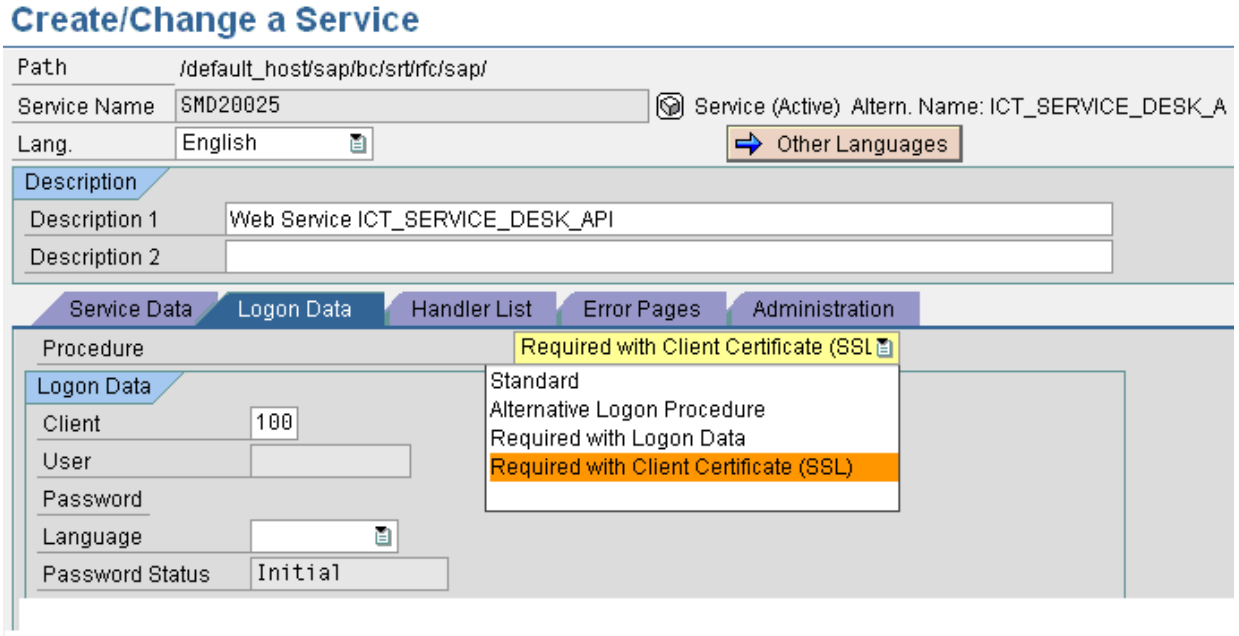
7. Check with the SAP Basis Administrator what client PSE should be used. In most cases this will be the PSE **SAP Client (Standard)**.
8. After assigning a client Certificate to the RFC destination, save the settings. The RFC destination is configured for using SSL with Apache Tomcat. A connection test will fail if the Server certificate in Apache Tomcat is not trusted against a CA.
9. Create a logical port (see Create a logical port).
10. Configure the interface between SAP Solution Manager Service Desk and Service Manager for the SSL outgoing connection (see Configure interface to SAP Solution Manager Service Desk).

Set up an incoming connection in SAP Solution Manager

Configure the incoming connection in the ICF Service tree in SAP transaction `/nsicf`.

1. In SAP transaction `/nsicf`, enter ICT_SERVICE_DESK_API as service name.
2. Execute the search of the service.
3. Double-click the Service to edit (or navigate to **/default_host/sap/bc/srt/rfc/sap/** and select **ICT_SERVICE_DESK_API**).
4. Open the **Create/Change a Service** dialog.
5. In the Logon tab, select **Required with client Certificates (SSL)**.

6. Save the settings. Service `ICT_SERVICE_DESK_API` is configured for SSL connection only. In this procedure, the lowest possible security level is specified. If **Required with Logon Data** is configured, then connecting via SSL and the client certificate is allowed.

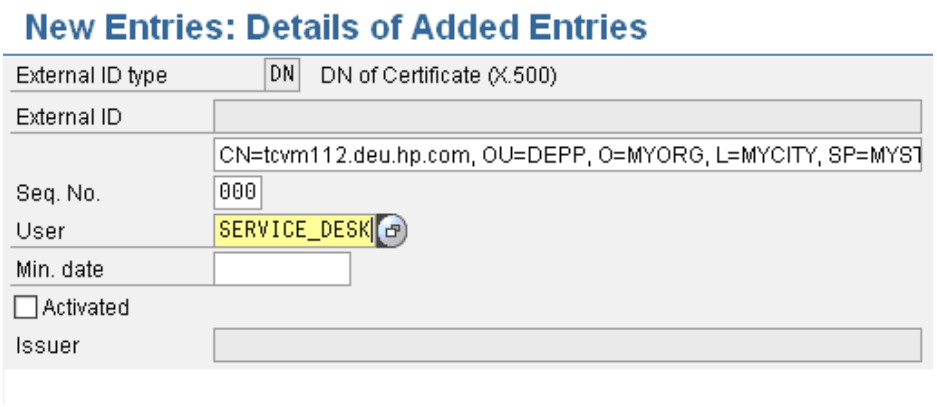


> **Note**
>
> For SSL communication, ensure that the ICM uses HTTPS.

Define the user mapping to the DN of the Certificate. The different ways of mapping are described in the SAP online help. Defining a user mapping to a DN is described below.

7. In SAP transaction `/nse16`, open the view `VUSREXTID` (enter VUSREXTID in the table `Name` field).
8. Select the Work Area **DN of Certificate X.500**.
9. In the user mapping dialog, as an external ID add the DN of the client certificate of Apache Tomcat (see Create keystore and truststore). Specify the exact DN of the certificate. For example:

   `CN=helen2006.asiapacific.hpqcorp.net, OU=TEST, O=GDCC, L=SH, SP=CN, C=CN`
10. For `Seq. No`, enter 000, 001... (for internal use only).
11. Assign the SAP user for the Web Service. This user has all the required permissions for managing incidents in SAP Solution Manager.


New Entries: Details of Added Entries
External ID type
DN
DN of Certificate (X.500)
External ID
CN=tcvm112.deu.hp.com, OU=DEPP, O=MYORG, L=MYCITY, SP=MYST
Seq. No.
000
User
SERVICE_DESK
Min. date
Activated
Issuer

1.3.1.7.1.2. Set up SSL between SAP and SMSSMEX

This section describes how to setup SSL between SAP and SMSSMEX.

Create keystore and truststore

SMSSMEX requires:

- Two separate stores that contain the certificates used to authenticate and encrypt communication.
- The following certificates:
  - Signed certificate with the long hostname of the SMSSMEX server in the CN section (for example, `CN="server.hp.com"`). This certificate must be mapped to an SAP user in SAP Solution Manager.
  - Certificate of the root CA used to sign the certificate of the SAP Solution Manager.
  - Certificate of the root CA used to sign the certificate of the SMSSMEX certificate.

The keystore must contain the following certificates:

- Root CA certificate used to sign the SMSSMEX certificate
- SMSSMEX certificate

The truststore must contain the root certificate used to sign the certificate of the SAP Solution Manager.

Any tool can be used to create and manage the key- and truststores. The following examples use the Java JDK tool `keytool` to create and import a signed certificate.

1. Create a self-signed certificate. The keypass and the storepass must be identical.

   `keytool -genkey -alias <alias> -keyalg RSA -keystore <keystorefile> -storepass <password> -keypass <password> -dname "CN=<serverhost>, OU=<MYOU>, O=<MYORG>, L=<MYCITY>, ST=<MYSTATE>, C=<MY>"`

   For example:

   `keytool -genkey -alias ovictex -keyalg RSA -keystore "C:\Program Files\Micro Focus\SMSSMEX\config\certs\ovictex.keystore" -storepass ovictex -keypass ovictex -dname "CN=helen2006.asiapacific.hpqcorp.net, OU=TEST, O=GDCC, L=SH, ST=CN, C=CN"`

2. Create a certificate request:

   `keytool -certreq -keystore <keystorefile> -alias <alias> -storepass <password>`

   For example:

`keytool -certreq -keystore "C:\Program Files\Micro Focus\SMSSMEX\config\certs\ovictex.keystore" -alias ovictex -storepass ovictex`

3. Use the resulting certificate request to acquire a signed certificate from SAP Web (https://websmp102.sap-ag.de/SSLTest) with chain PKCS#7. Copy the signed response `<filename>.p7b` (for example, `sap_rp.p7b`).
4. Download the root certificate file from https://tcs.mysap.com/invoke/tc/getCert?SAPServerCA.der.
5. Import the root certificate from the Certificate Authority (CA) into the keystore.

   `keytool -import -v -alias <alias2> -keystore <keystorefile> -storepass <password> -file <rootcertificatefile>`

   For example:

   `keytool -import -v -alias saproot -keystore "C:\Program Files\Micro Focus\SMSSMEX\config\certs\ovictex.keystore" -storepass ovictex -file "C:\Program Files\Micro Focus\SMSSMEX\config\certs\getCert.cer"`

6. Import the answer from the Certificate Authority into the keystore. Use the same keystore file and alias the request was created from.

   `keytool -import -v -alias <alias> -keystore <keystorefile> -storepass <password> -file <certificatefile>`

   For example:

   `keytool -import -v -alias ovictex -keystore "C:\Program Files\Micro Focus\SMSSMEX\config\certs\ovictex.keystore" -storepass ovictex -file "C:\Program Files\Micro Focus\SMSSMEX\config\certs\sap.p7b"`

   To import the certificates into the truststore, use the same command as in the step above, but instead of `<keystorefile>` use the filename of the truststore (if it doesn't exist, it will be created automatically). For example:

   `keytool -import -v -alias saproot -keystore "C:\Program Files\Micro Focus\SMSSMEX\config\certs\ovictex.truststore" -storepass ovictex -file "C:\Program Files\Micro Focus\SMSSMEX\config\certs\getCert.cer"`

Configure Tomcat SSL

To enable SSL with Tomcat, configure a new connector in the `server.xml` configuration file. The standard `server.xml` contains a connector definition that has been commented out. The following attributes are required:

```
port=<port>
scheme="https"
secure="true"
clientAuth="false"
sslProtocol = "TLS"
keystoreFile=<keystorefile>
keystorePass=<keystorepass>
truststoreFile=<truststorefile>
truststorePass=<truststorepass>
```

For example:

```
<Connector port="8443"
maxThreads="150" minSpareThreads="25" maxSpareThreads="75"
enableLookups="false" disableUploadTimeout="true"
acceptCount="100" debug="0" scheme="https" secure="true"
clientAuth="false" sslProtocol="TLS"
keystoreFile="C:/Program Files/Micro
Focus/SMSSMEX/config/certs/ovictex.keystore"
keystorePass="password"
truststoreFile="C:/Program Files/Micro
Focus/SMSSMEX/config/certs/ovictex.truststore"
truststorePass="password"
/>
```

Configure property files

Follow these steps:

1. Modify `exthd.properties`.
    1. In the SAP configuration files in property `exthd.webservice.endpoint` specify the new port (default is 8443) and use `https://` as the protocol. For example:

        `exthd.webservice.endpoint = https://watermelon.chn.hp.com:8001/sap/bc/srt/rfc/sap/ICT_SERVICE_DESK_API?sap-client=300`

    2. Set the `exthd.webservice.authentication.scheme` to HTTPS.
2. Add the following configuration entries in `ovictex.properties`:

    `<saphostname>.keystore=C:/Program Files/Micro Focus/SMSSMEX/certs/ovictex.keystore`
    `<saphostname>.keystore.password=~X1~H+7JAOrcX/R6kO5diPxV0w==`
    `<saphostname>.truststore=C:/Program Files/Micro`

```
Focus/SMSSMEX/certs/ovictex.truststore
<saphostname>.truststore.password=~X1~H+7JAOrcX/R6kO5diPxV0w==
```

For example:

```
watermelon.chn.hp.com.keystore= C:/Program Files/Micro
Focus/SMSSMEX/certs/ovictex.keystore
watermelon.chn.hp.com.keystore.password=~X1~H+7JAOrcX/R6kO5diPxV0w==
watermelon.chn.hp.com.truststore= C:/Program Files/Micro
Focus/SMSSMEX/certs/ovictex.truststore
watermelon.chn.hp.com.truststore.password=~X1~H+7JAOrcX/R6kO5diPxV0w==
```

1.3.1.7.2. Security Between Service Manager and SMSSMEX

This section describes how to configure security between Service Manager and SMSSMEX.

1.3.1.7.2.1. Configure Service Manager for SSL

For more information about how to configure Service Manager for SSL, refer to the *Service Manager Help Center* documentation.

1.3.1.7.2.2. Configure SMSSMEX for SSL Communication with Service Manager

To configure SMSSMEX for SSL communications with Service Manager, do the following:

1. Import the root CA into the trust keystore.

   `keytool -import -trustcacerts -alias <alias> -keystore <keystorefile> -file <certificatefile>`

   For example:

   `keytool -import -trustcacerts -alias scca -keystore ovictex.truststore -file mycacert.pem`

2. Configure `ovictex.properties`.

   1. Set `sc.webservice.endpoint`.

      `sc.webservice.endpoint = http://<smhostname>:<port>/sc62server/PWS`

   2. Add the following configuration entries in `ovictex.properties`.

      `<smhostname>.keystore=<ovictex keystore file>`
      `<smhostname>.keystore.password=<keystore password>`
      `<smhostname>.truststore=<ovictex truststore file>`
      `<smhostname>.truststore.password=<truststore password>`

      For example:

      ```
      sc.webservice.endpoint =
      http://SMCI02.chn.hp.com:13080/sc62server/PWS
      ……
      SMCI02.chn.hp.com.keystore=C:/Program Files/Micro
      Focus/SMSSMEX/config/certs/ovictex.keystore
      SMCI02.chn.hp.com.keystore.password=~X1~eD+6cy6OMNxdK9tcCQVBww=
      =
      SMCI02.chn.hp.com.truststore=C:/Program Files/Micro
      Focus/SMSSMEX/config/certs/ovictex.truststore
      SMCI02.chn.hp.com.truststore.password=~X1~eD+6cy6OMNxdK9tcCQVBw
      w==
      ```

      Note

      The `keystore.password` and `truststore.password` should use `<SMSSMEX_installDir>/bin/encryptPasswords.bat` to encrypt. For usage of `encryptPasswords.bat`, refer to Tools.

1.3.1.8. Licensing

This chapter describes licensing.

License Types

The following license types are available:

- InstantOn license provides full access to all features for 90 days.
- Permanent license is node-locked (restricted to a range of IP addresses).

License Management

Follow these steps to manage the license:

1. Rename the license file to LicFile.txt.
2. Copy and paste LicFile.txt to the SMSSMEX installation path, such as C:\Program Files (x86)\Micro Focus\SMSSMEX.

> **Note**
>
> The Instance On license is installed during SMSSMEX installation. However, considering the Windows 2012 Security Right Control feature, the Instance On license may not be successfully installed during installation. In this case, you must manually run the InstantOn.bat batch script in the bin folder to add the Instance On license to SMSSMEX or to replace the license file with the existing license data.

Contents

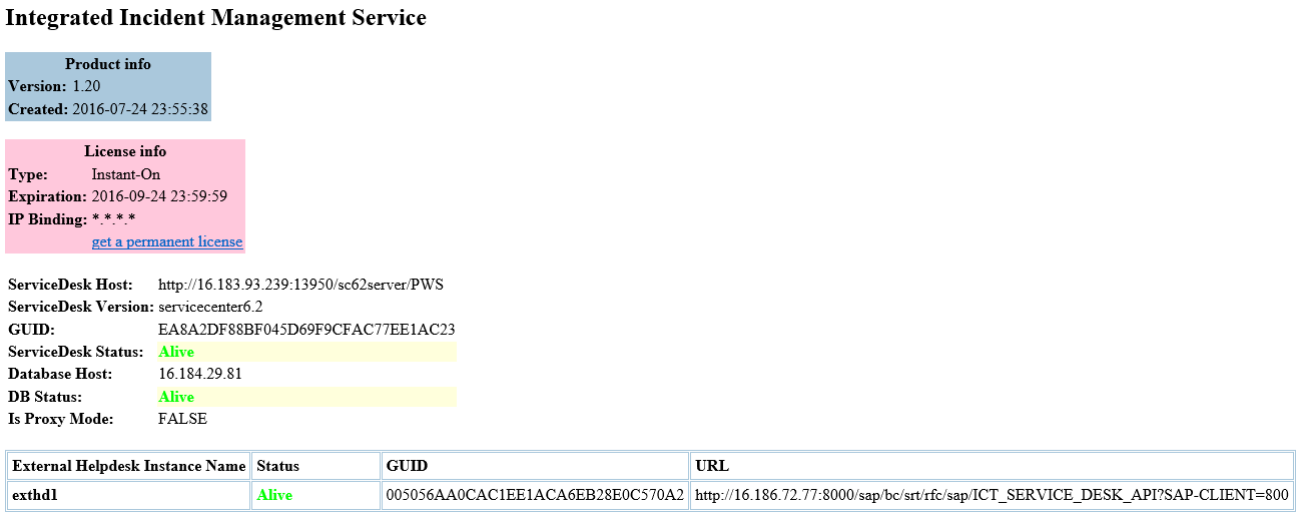
- 1 License Types
- 2 License Management

1.3.1.9. Status Page

The Incident Management Service provides a comprehensive overview of the status of the incident exchange systems and services and provides extensive information for troubleshooting. The URL of the status page is

`http://<hostname>:<port>/ovictex/servlet/OvHDTrigger?status`

The following is an example status page.



Integrated Incident Management Service

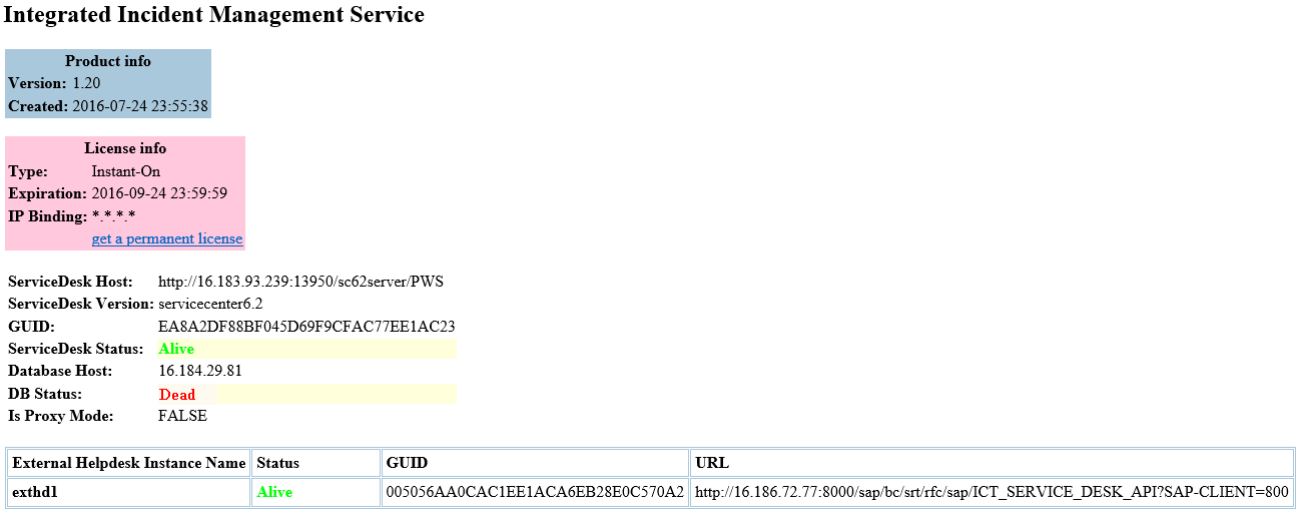
Product info
Version: 1.20
Created: 2016-07-24 23:55:38

License info
Type: Instant-On
Expiration: 2016-09-24 23:59:59
IP Binding: *.*.*.*
get a permanent license

ServiceDesk Host: http://16.183.93.239:13950/sc62server/PWS
ServiceDesk Version: servicecenter6.2
GUID: EA8A2DF88BF045D69F9CFAC77EE1AC23
ServiceDesk Status: Alive
Database Host: 16.184.29.81
DB Status: Alive
Is Proxy Mode: FALSE

External Helpdesk Instance Name	Status	GUID	URL
exthd1	Alive	005056AA0CAC1EE1ACA6EB28E0C570A2	http://16.186.72.77:8000/sap/bc/srt/rfc/sap/ICT_SERVICE_DESK_API?SAP-CLIENT=800

When a service becomes unavailable, the status changes from `Alive` to `Dead`.



Integrated Incident Management Service

Product info
Version: 1.20
Created: 2016-07-24 23:55:38

License info
Type: Instant-On
Expiration: 2016-09-24 23:59:59
IP Binding: *.*.*.*
get a permanent license

ServiceDesk Host: http://16.183.93.239:13950/sc62server/PWS
ServiceDesk Version: servicecenter6.2
GUID: EA8A2DF88BF045D69F9CFAC77EE1AC23
ServiceDesk Status: Alive
Database Host: 16.184.29.81
DB Status: Dead
Is Proxy Mode: FALSE

External Helpdesk Instance Name	Status	GUID	URL
exthd1	Alive	005056AA0CAC1EE1ACA6EB28E0C570A2	http://16.186.72.77:8000/sap/bc/srt/rfc/sap/ICT_SERVICE_DESK_API?SAP-CLIENT=800

Integrated Incident Management Service

Product info
Version: 1.20
Created: 2016-07-24 23:55:38

License info
Type: Instant-On
Expiration: 2016-09-24 23:59:59
IP Binding: *.*.*.*
get a permanent license

ServiceDesk Host: http://16.183.93.239:13950/sc62server/PWS
ServiceDesk Version: servicecenter6.2
GUID: EA8A2DF88BF045D69F9CFAC77EE1AC23
ServiceDesk Status: Alive
Database Host: 16.184.29.81
DB Status: Dead
Is Proxy Mode: FALSE

External Helpdesk Instance Name	Status	GUID	URL
exthd1	Dead	005056AA0CAC1EE1ACA6EB28E0C570A2	http://16.186.72.77:8000/sap/bc/srt/rfc/sap/ICT_SERVICE_DESK_API?SAP-CLIENT=800

1.3.1.10. Incident exchange details

This chapter provides incident exchange details.

1.3.1.10.1. Database Tables

The database tables required to operate the exchange service are created with the SQL scripts `create_tables_oracle.sql` or `create_tables_sqlserver.sql`.

Table	Field	Description
systemguid	systemguid	Unique system web service GUID

Table	Field	Description
tasklist	ovhdid	ID of incident that triggered the action
	action	Action for the incident (state transitions of status diagram). Can be ProcessIncident, AddInfo, AcceptIncidentProcessing, RejectIncidentSolution, VerifyIncidentSolution, or CloseIncident.
	startTimestamp	Creation timestamp of entry.
	enqueueTimestamp	Timestamp for ordering of tasks. Initial value is startTimestamp.
	earliestReadyTimestamp	Timestamp that specifies the earliest time when this entry can be processed. Empty means immediately. Task becomes ready only after this time.
	state	Task state. Can be 1=READY or 2=INPROCESS (task is processed already).
	tries	Number of attempts to complete this task.
	guid	GUID of the task to delete the correct entry in database.
	sapid	Name of external help desk instance that incident is exchanged with.

Table	Field	Description
runtimedata	incidentguid	GUID of exchanged incident
	ovhdincidentid	ID of incident in helpdesk managed by web service
	exthdincidentid	ID of incident in the external helpdesk
	requesterguid	System GUID of requester helpdesk for that incident
	providerguid	System GUID of provider helpdesk for that incident
	metadata	Incident state in statement diagram and role the ServiceDesk has for this incident (Requester or Provider). Stored in the same format used for the `Hidden_Meta_Data` field in ServiceDesk (such as `Requester:RequesterProcessing`, `Provider:Solution Provided`).
	infologid	Reference to multiple entries in runtimedata_infolog.
	attachmentid	Reference to multiple entries in runtimedata_attachments.
	lastchange	Timestamp of last change of entry.

Table	Field	Description
runtimedata_infolog	infologid	Key referenced from runtimedata.
	infologblock	Number of infolog block sent already.
runtimedata_attachments	attachmentid	Key referenced from runtimedata.
	filename	Filename of an attachment for incident.
	attachmentguid	GUID for attachment (also known by external helpdesk) to delete attachment.

1.3.1.10.2. Tools

There are several configuration tools in the installation \bin directory. Tool scripts are available for Windows (.bat) and Linux (.sh):

- `encryptPasswords` encrypts the passwords in the configuration file. All properties ending with `.password` must be configured with this tool. Use `-global` or `<instance key>` as a parameter.
  - `global`

    Encrypt a password in the global properties file (`ovictex.properties`). For example:

    `encryptPasswords.bat -global`
  - `<instance key>`

    Encrypt a password in the configuration file of a specific instance. For example:

    `encryptPasswords.bat exthd`
- `setup` is the setup script for Tomcat start/stop. For Tomcat:
  - Start: `Setup startup`
  - Shutdown: `Setup shutdown`
  - Start with debug mode: `Setup debug startup`
- `checker` checks the configuration in `ovictex.properties` and Service Manager configuration (see Verifying configuration for more information) .

1.3.1.10.3. Field mapping configuration

Incident exchange web service exchanges incident data as XML documents between Service Manager and the external Help Desk SAP Solution Manager. Incident exchange transforms the incident data in Service Manager to an XML message for SAP Solution Manager, and transforms Solution Manager data to an XML message for Service Manager. The transformation maps the field name in Service Manager to XML elements in Solution Manager while taking into account the following:

- Field names in Service Manager are usually different from the message element name.
- Service Manager field data type can differ from the message element data type.
- Not all message elements have corresponding data fields in Service Manager. Such fields are usually combined into a single log field called `Journal`.
- Some fields also require value mapping. For example, the possible values for the `Priority` field in Service Manager are `1 - Critical`, `2 - High`, `3 - Average`, `4 - Low`. The Solution Manager Priority can be `5`, `4`, `3`, `2`, `1`. These values must be specified in the `FieldValueMapping` configuration.
- Service Manager can assign customized fields to an Incident. These fields can be mapped to message elements.

A declarative field mapping file defines the mapping outlined above and :

- Enables the exchange of incident data between two helpdesks with reduced code size (the same code can handle any number of fields)
- Improves flexibility (mapping can be changed without changing code)
- Improves extensibility and customizability (a deployment-specific mapping can be added without changing code)
- Used to map incident data with an external help desk other than Solution Manager

1.3.1.10.3.1. Types of Mapping

The mapping file supports field mapping and field value mapping. Field mapping is simple (XML message element is a single value) or composite (multiple values such as an array).

1.3.1.10.3.2. Structure of FieldMapping XML file

The field mapping configuration is related to the `ICT_SERVICE_DESK_API` WSDL scheme defined by SAP Solution Manager. The mapping consists of field mapping and value mapping.

Field mapping includes:

- `IctHead`
- `IctIncidentAttachment`
- `IctIncidentSapNotes`
- `IctIncidentSolutions`
- `IctIncidentUrls`
- `IctIncidentStatement`
- `IctIncidentAdditionalInfo`

The following is a mapping file example:

```
<FieldMapping ExtHDField="IctHead/AgentId" >
  <OutOvHDField>Assignee</OutOvHDField>
  <OutDataType>Person</OutDataType>
  <InOvHDField>Assignee</InOvHDField>
  <InDataType>Person</InDataType>
</FieldMapping>
```

In the above example:

- Element `IctHead/AgentId` of SAP Solution Manager (sub-element `AgentId` of top level element `IctHead`) maps to the field `Assignee` exposed by the Service Manager IncidentManagement Web Service.
- Data types for the IN and OUT exchange modes are specified in the `InDataType` and `OutDataType` tags.
- `Person` type indicates that the Exchange must convert incoming data (to/from the Service Manager) to/from an internal `Person` type that corresponds with the `IctIncidentPerson` type of the SAP SolutionManager web service.
- `InDataType` and `OutDataType` tags declare types on the Service Manager side.

1.3.1.10.3.3. Composite Field Mapping

Composite field mapping maps a message element to a `OvHD` field depending upon the value of a sub-element (key) of the element `ExtHDKeyField` (`OvHD` and `ExtHD` are old terms; in this document, `OvHD` correspond to `Service Manager` and `ExtHD` correspond to `SAP Solution Manager`). A different value for the key defines mapping to a different Service Manager field. The following is a composite field mapping example.

```
<CompositeFieldMapping ExtHDField="IctIncidentStatement"
ExtHDKeyField="IctIncidentStatement/TextType">
<!-- For exchanging information log -->
<FieldMapping ExtHDField="IctIncidentStatement/Text" >
        <InDataType>InformationLog</InDataType>
        <OutDataType>InformationLog</OutDataType>
        <KeyFieldOutVal>SU99</KeyFieldOutVal>
        <KeyFieldInVal>SU99</KeyFieldInVal>
</FieldMapping>
<!-- for exchanging Solution Provided -->
<FieldMapping ExtHDField="IctIncidentStatement/Text">
        <OutOvHDField>Solution</OutOvHDField>
        <InOvHDField>Solution</InOvHDField>
        <KeyFieldOutVal>SU99</KeyFieldOutVal>
        <KeyFieldInVal>SU01</KeyFieldInVal>
</FieldMapping>
<!-- for exchanging CustomText01 (as example) -->
<FieldMapping ExtHDField="IctIncidentStatement/Text" >
        <InOvHDField>CustomText01</InOvHDField>
        <OutOvHDField>CustomText01</OutOvHDField>
        <KeyFieldOutVal>SU99</KeyFieldOutVal>
        <KeyFieldInVal>SU77</KeyFieldInVal>
</FieldMapping>
<!-- For sending custom fields from OVHD to external HD create an entry
as
the example below. Replace the place holder strings as per your
configuration -->
<!--
<FieldMapping ExtHDField="IctIncidentStatement/Text" >
        <OutOvHDField>USER_VISIBLE_FIELDNAME_FOR_THAT_CUSTOM_FIELD
        </OutOvHDField>
        <KeyFieldOutVal>TEXT_TYPE_AS_DEFINED_BY_USER_FOR_THIS_FIELD
        </KeyFieldOutVal>
</FieldMapping>
-->
</CompositeFieldMapping>
```

Element `IctIncidentStatement/Text` is mapped to the information log if the key element `IctIncidentStatement/TextType` is `SU99` or to `Resolution` field if the key element is `SU01` (for an incoming message).

This is used when a message has multiple occurrences of the same element that have different sub-element values. The sub-element is referred to as the key field. In the example above the

`IctIncidentStatement/TextType` element is the key field. For a composite field mapping, every instance of `FieldMapping` has a unique `KeyFieldInVal`.

1.3.1.10.3.4. Field Value Mapping

Field value mapping maps the values of a message element to the corresponding value of an OvHD field. The following is an example.

```
<FieldValueMapping Id="IctHead/Priority">
<ValueMapping OvHDValue="4" ExtHDValue="5"/>
<ValueMapping OvHDValue="4" ExtHDValue="4"/>
<ValueMapping OvHDValue="3" ExtHDValue="3"/>
<ValueMapping OvHDValue="2" ExtHDValue="2"/>
<ValueMapping OvHDValue="1" ExtHDValue="1"/>
</FieldValueMapping>
```

Note

Since both helpdesks priority lists can be configured, check the actual values in the field value mapping.

1.3.1.10.3.5. Field Mapping Schema

The RelaxNG Compact Schema of the mapping file is shown below.

```
default namespace =
  "http://schemas.hp.com/openview/incidentExchange/mapping"
start =
  element IncidentExchMapping {
    attribute targetNamespace { xsd:anyURI },
    element FieldMappings {
      (FieldMapping
       | element CompositeFieldMapping {
           attribute ExtHDField { string },
           attribute ExtHDKeyField { string },
           FieldMapping+
         })+
    } &
    element ValueMappings {
      element FieldValueMapping {
        attribute Id { string },
        element ValueMapping {
          attribute ExtHDValue { string },
          attribute OvHDValue { string }
        }+
      }
    }
  }
FieldMapping =
  element FieldMapping {
  ## field accessor in XML document using XPath like notation.
     Example:
  ## ExtHDField="IctHead/AgentId"
    attribute ExtHDField { string },
    attribute ValueMappingId { string }?,
    (element InOvHDField { string } &
    (element DefaultOutOvHDField { string }
     | element OutOvHDField { string })? &
    element InDataType { "InformationLog" | "Priority" | "Date" |
       "Attachment" | “OvCISearchKey” }? &
    element OutDataType { "Person" | "Priority" | "Date" |
       "Attachment" | “OvCISearchKey” }? &
    element KeyFieldOutVal { string }? &
    element KeyFieldInVal { string }? )
  }
```

The schema elements are described in the following table.

Schema element	Function
IncidentExchMapping	Top-level element of the mapping schema.
FieldMappings	Container element for all `FieldMapping` and `CompositeFieldMapping` elements.
ValueMappings	Container element for `FieldValueMapping` elements.
FieldMapping	Maps a message element to an `OvHD` field and includes type information for storing to and loading from OvHD. Optionally contains a reference to a `FieldValueMapping` element through attribute `ValueMappingId`. The value of this attribute must match the value of attribute `Id` in a `FieldValueMapping` element. When this reference is present, the information in the `FieldValueMapping` must be used to map field value.
CompositeFieldMapping	Maps a message element to a `OvHD` field depending upon the value of a sub-element (key) of the element. A different key value defines mapping to a different `OvHD` field. The `keyFieldInVal` must be unique for each individual field mapping within a composite field mapping.
ValueMapping	Maps the OvHD value of a message element to an ExtHD value.
ExtHDField	Field accessor in XML message document in XPATH like expression that identifies a specific field of exchanged incident information.
InOvHDField	Indicates the `OvHD` field name where information received from the external helpdesk is written for a specific `ExtHDField`.

Schema element	Function
OutOvHDField	Indicates the `OvHD` field name whose value is sent to the external helpdesk for a specific `ExtHDField`.
DefaultOutOvHDField	If this element appears in a field mapping then the value of this element is taken as the default value sent to the external helpdesk for a specific `ExtHDField`. For example, if a mapping `DefaultOutOvHDField` is specified as `DefaultUserId` and `OutDataType` is specified as `Person`, the default user ID will be sent to the external helpdesk for a specific `ExtHDField`.
InDataType	Datatype for storing the field value to `OvHD`.
OutDataType	Datatype for loading the field value from `OvHD`.
InDataType and OutDataType	Specifies the method to call for reading/writing information from/to the incident using the OvHDAccess layer. InDataType and `OutDataType` are optional elements. If not specified, then the field types are assumed to be String. Otherwise the following data types can be specified: • Priority: Priority of an incident • Date: A date field • Attachment: Refers to an attachment • Person: Indicates that the information is a person detail.
OvCISearchKey	Indicates the information is used as a search key for CI in OvHD.
InformationLog	Applicable only for InDataType. Indicates the information should be appended to the Information Log.

Schema element	Function
KeyFieldInVal	Value stored in OvHD for the element used as the key field.
KeyFieldOutVal	Value sent to ExtHD for the element used as the key field.

1.3.1.10.3.6. Default field mapping file and customization

This section provides default field mapping file and customization information.

Prerequisites

SMSSMEX operates with Service Manager based on the extended IncidentManagement Web Service and supports only the fields listed below (exposed in the Service Manager IncidentManagement WS).

Field	Type	Field	type
IncidentID	Text	Subcategory	Text
Category	Text	SLAAgreementID	Decimal
OpenTime	Datetime	PlannedEnd	Datetime
OpenedBy	Text	SiteCategory	Text
PriorityCode	Text	ProductType	Text
Severity	Text	ProblemType	Text
UpdatedTime	Datetime	ResolutionFixType	Text
PrimaryAssignment Group	Text	UserPriority	Text
ClosedTime	Datetime	Solution	Text
ClosedBy	Text	InitialImpact	Text
ClosureCode	Text	CustomText01	Text
ConfigurationItem	Text	CustomText02	Text
Location	Text	CustomText03	Text
IncidentDescription		CustomText04	Text
Resolution	Resolution	CustomText05	Text
Assignee	Text(OperatorID)	CustomText06	Text
Contact	Text(ContactID)	CustomText07	Text
JournalUpdates		CustomText08	Text
AlertStatus	Text	CustomText09	Text
ContactLastName	Text	CustomText10	Text
ContactFirstName	Text	SapSid	Text
Company	Text	SapClient	Text

Field	Type	Field	type
Title	Text	SapInstallationNumber	Text
TicketOwner	Text	HiddenMetaData	Text
UpdatedBy	Text	IsIncidentExchange	Boolean
IMTicketStatus	Text	attachments	Attachments

Add fields to fieldMapping.xml

The default field mapping file (provided with the incident exchange web service) doesn't include all fields from the web service and can be extended. Any additional field mapping can be included in section `IctIncidentStatement`. The following is an example:

```
<FieldMapping ExtHDField="IctIncidentStatement/Text">
 <OutOvHDField>SC_WS_FIELDNAME</OutOvHDField>
 <KeyFieldOutVal>SOLMAN_FIELD_TYPE </KeyFieldOutVal>
</FieldMapping>
<FieldMapping ExtHDField="IctIncidentStatement/Text">
 <OutOvHDField>CustomText09</OutOvHDField>
 <KeyFieldOutVal>SU99 </KeyFieldOutVal>
</FieldMapping>
```

In the above example, the custom field defined in Service Manager is sent to the external HD, so `KeyFieldOutVal` is defined at the external helpdesk. No `InOvHDField` or `KeyFieldInVal` is specified since the example only sends to the external helpdesk.

IN/OUT data exchange requires definition of `IN` and `OUT`:

```
<FieldMapping ExtHDField="IctIncidentStatement/Text">
 <OutOvHDField>SC_WS_FIELDNAME1</OutOvHDField>
 <InOvHDField>SC_WS_FIELDNAME2</InOvHDField>
 <KeyFieldOutVal> SOLMAN_FIELD_TYPE1 </KeyFieldOutVal>
 <KeyFieldInVal> SOLMAN_FIELD_TYPE2 </KeyFieldInVal>
</FieldMapping>
```

In this example, if the values of `SC_WS_FIELDNAME1` and `C_WS_FIELDNAME2` are the same, then the `OvHD` field is overwritten when information is sent from external helpdesk (1:1 field synchronization). For example:

```
<FieldMapping ExtHDField="IctIncidentStatement/Text">
 <OutOvHDField>CustomText09</OutOvHDField>
 <InOvHDField>CustomText09</InOvHDField>
 <KeyFieldOutVal SU01</KeyFieldOutVal>
```

```
 <KeyFieldInVal>SU01</KeyFieldInVal>
</FieldMapping>
```

In the following example, `CustomText08` updates field `ZZ08` in SAP Solution Manager, but `ZZ08` updates `CustomText09` in Service Manager (doesn't overwrite CustomText08).

```
<FieldMapping ExtHDField="IctIncidentStatement/Text">
 <OutOvHDField>CustomText08</OutOvHDField>
 <InOvHDField>CustomText09</InOvHDField>
 <KeyFieldOutVal ZZ08</KeyFieldOutVal>
 <KeyFieldInVal>ZZ08</KeyFieldInVal>
</FieldMapping>
```

Additional information

Section `IctIncidentAdditionalInfo` defines synchronization of CIs between SAP Solution Manager and Service Manager and defines the method for sending SAP Attributes from SAP Solution Manager.

Note

The first part of the mapping describes CI mapping handling and must not be changed.

```
<FieldMapping ExtHDField="IctIncidentAdditionalInfo/AddInfoValue" >
 <OutDataType>OvCISearchKey</OutDataType>
 <InDataType>OvCISearchKey</InDataType>
 <KeyFieldOutVal>SAPSystemID</KeyFieldOutVal>
 <KeyFieldInVal>SAPSystemID</KeyFieldInVal>
</FieldMapping>
<FieldMapping ExtHDField="IctIncidentAdditionalInfo/AddInfoValue" >
 <OutDataType>OvCISearchKey</OutDataType>
 <InDataType>OvCISearchKey</InDataType>
 <KeyFieldOutVal>SAPSystemClient</KeyFieldOutVal>
 <KeyFieldInVal>SAPSystemClient</KeyFieldInVal>
</FieldMapping>
<FieldMapping ExtHDField="IctIncidentAdditionalInfo/AddInfoValue" >
 <OutDataType>OvCISearchKey</OutDataType>
 <InDataType>OvCISearchKey</InDataType>
 <KeyFieldOutVal>SAPInstNo</KeyFieldOutVal>
 <KeyFieldInVal>SAPInstNo</KeyFieldInVal>
</FieldMapping>
```

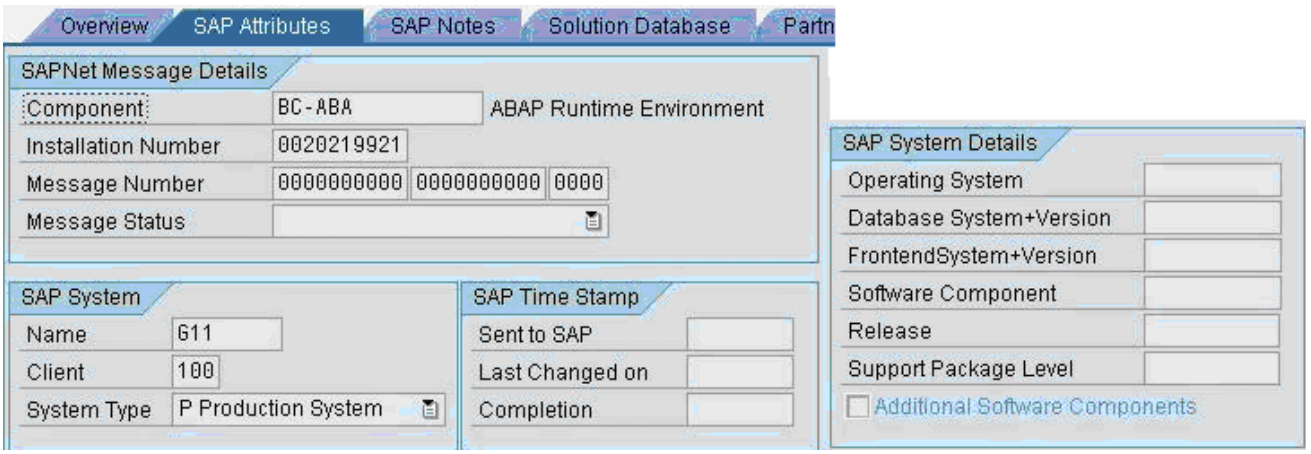
The following two attributes are used only when Solution Manager forwards an Incident to SAP Solution Manager.

```
<FieldMapping ExtHDField="IctIncidentAdditionalInfo/AddInfoValue" >
  <InDataType>InformationLog</InDataType>
  <KeyFieldInVal>SAPIncidentID</KeyFieldInVal>
</FieldMapping>
<FieldMapping ExtHDField="IctIncidentAdditionalInfo/AddInfoValue"
  ValueMappingId="IctIncidentAdditionalInfo/AddInfoValue    /SAPIncidentStatus" >
  <InDataType>InformationLog</InDataType>
  <KeyFieldInVal>SAPIncidentStatus</KeyFieldInVal>
</FieldMapping>
```

The only attributes that don't have read-only status in the SAP Solution Manager are CI attributes, allowing IN-mode mapping (from SAP Solution Manager to Service Manager). The following table defines the available attributes:

AttributeType	Description
SAPComponent	SAP Component (e.g. SV-SMG-SUP)
SAPSystemID	SAP System ID
SAPSystemClient	Client of SAP System
SAPCategory	Category of the Incident
SAPSystemType	SAP System Type
SAPInstNo	SAP Installation Number
SAPSubject	Subject of the Incident
SAPOperatingSystem	Operating System of SAP System
SAPDatabase	Database of SAP System
SAPFrontend	Frontendsystem and Version
SAPSoftwareComponent	Software Component
SAPSoftwareComponentRelease	Software Component Release
SAPSoftwareComponentPatch	Software Component Patch
SAPIncidentID	ID of the Incident at SAP (when forwarded to SAP)
SAPIncidentStatus	Status of the Incident at SAP (when forwarded to SAP)

In the SAP GUI most attributes are in the SAP Attributes tab.



The following example writes all incoming additional values of type `SAPDatabase` to the Journal in Service Manager:

```
<FieldMapping ExtHDField="IctIncidentAdditionalInfo/AddInfoValue" >
 <InDataType>InformationLog</InDataType>
 <KeyFieldInVal>SAPDatabase</KeyFieldInVal>
</FieldMapping>
```

The following example updates field `CustomText03`.

```
<FieldMapping ExtHDField="IctIncidentAdditionalInfo/AddInfoValue" >
 <InOvHDField>CustomText03</InOvHDField>
 <KeyFieldInVal>SAPDatabase</KeyFieldInVal>
</FieldMapping>
```

Changeable mappings

The following mappings can be modified.

Mapping	Description
IctHead/AgentID IctHead/ReporterID IctIncidentAttachment/PersonId IctIncidentStatement/PersonId	`OutOvHDField/InOvHDField` field name can be modified if the replacement field contains the ID of a Contact joined with a contact table that's exposed via ConfigurationManagement Web Service (defined in the default configuration). The `AssigneeName` field contains the `operator` name of Service Manager instead of the `contacts` name.
IctHead/ShortDescription	`OutOvHDField/InOvHDField` can be modified with any text field from Service Manager.
IctHead/RequestedBegin IctHead/RequestedEnd	Can be modified with any datetime field in the Service Manager.

Note

Required Mappings: The following mappings are required and must not be changed.

- `IctHead/Priority` (the value mapping for this field mapping can be changed)
- `IctIncidentSapNotes/item`
- `IctIncidentSolutions/item`
- `IctIncidentUrls/item`
- `IctIncidentAdditionalInfo/AddInfo` Value (first 3 mappings)

1.3.1.10.4. Person Synchronization Details

SAP Solution Manager to Service Manager

Persons sent from SAP Service Manager can be mapped to person fields in Service Manager. When Person details are received, the corresponding contact record is found in Service Manager by querying the Configuration Management Web Service. The resolved contact ID must be set in the mapped field. The exchange web service describes persons with the following fields:

- `Sex`
- `First name`
- `Last name`
- `Telephone`
- `Mobile phone`
- `Fax`
- `Email`

Fields that are used to find persons in Service Manager:

- `Email`
- `First name`
- `Last name`

Persons are searched by all three fields. If no matching person is found in Service Manager or duplicates are found, then a notification is added to the Journal. For example, an empty email causes the following message in response to Journal updates:

`Warning! Contact can't be found. Firstname,Lastname,Email fields should not be empty. Invalid contact: FirstName: "Nicholas" LastName: "Brown" Phone number: "(770) 954-4588" Fax number: "(770) 954-4590" ...`

SMSSMEX doesn't create Persons or Contacts. An operator-type lookup is enforced only for the `AssigneeName` field.

Mapping from Service Manager to SAP Solution Manager is performed in the same way. The ID of the `Person` field in the Service Manager is used to make an additional call to Configuration Management WS to get all details about the Person. The collected data is forwarded to the Solution Manager. In SAP Solution Manager the ID of the Person is checked. If the ID is

- Known: Solution Manager assigns an existing record to the Incident.
- Not known: Solution Manager tries to resolve a Person via the email field. If this is impossible, a new Person is created.

Contents

- 1 SAP Solution Manager to Service Manager

1.3.1.10.5. SMSSMEX Version

To find out the version of the SMSSMEX service in Tomcat, do one of the following:

- Open `<SMSSMEX_installDir>\tomcat\webapps\ovictex\WEB-INF\lib\ovictex.war` with a zip tool. The war file `MANIFEST.MF` file contains the version information.
- Go to the Status page.

1.3.1.11. Installing and Configuring SAPCRYPLIB

To install SAP Common Crypto Library (SAPCRYPLIB), do the following:

1. Download `SAPCRYLIB` from the SAP Software Downloads website.
2. Use `sapcar.exe` to extract the `*.sar` file:

   ```
   sapcar -xvf sarfile_name
   ```

3. Copy the extra files to `\usr\sap\[Instance folder]\DVEBMGS00\exe`.
4. In transaction `/nrz10` in the `Profile` field, select the profile with `prof.type` of "Instance profile".
5. Select **Extended maintenance** in Edit Profile.
6. Click **Change**.
7. Add the following parameters:

   ```
   ssf/name = SAPSECULIB
   ssf/ssfapi_lib = $(DIR_EXECUTABLE)\sapcrypto.dll
   ```

8. Restart the system.
9. Go to transaction `/nsmicm`.
10. Select the menu entry **GOTO** and select **Services** or press **SHIFT+F1**.
11. If the HTTPS port isn't listed, then configure the profile. Add or change the following parameter.

    ```
    icm/server_port_2 PROT=HTTPS,PORT=[SSL Port]
    ```

12. In the `/nsmicm` transaction, restart Internet Communication Manager (ICM) by selecting **Administration** > **ICM** > **Restart** > **Yes**.

1.3.1.12. Logging

The following describes the location of log files.

If you start `SMSSMEX` from Windows:

- **setup -startup**

  `%SMSSMEX_HOME%/logs/smssmex.log.<date>`

- Tomcat

  `%SMSSMEX_HOME%/tomcat/logs/smssmex.log.<date>`

If you start `SMSSMEX` from Linux:

- **setup.sh -startup**

  `%SMSSMEX_HOME%/logs/smssmex.log.<date>`

- Tomcat

  `%SMSSMEX_HOME%/tomcat/logs/smssmex.log.<date>`

1.3.1.13. Deploying Button Icons

SMSSMEX enhances the functionality of Service Manager by adding some buttons in incident form to trigger message exchange related actions. The icons for the buttons are provided additionally in the release package (under `<SMSSMEX Release Package>\icons` folder). You can deploy them to the Service Manager Client manually.

Service Manager has two client applications: Windows Client and Web Client. For each of the clients, the icons should be deployed separately.

Windows Client

Copy button icons from `<SMSSMEX Release Package>\icons` folder to `<Client_Home>\plugins\com.hp.ov.sm.client.eclipse.user_9.xx\src\resources\icons\obj16`

For more information, see Install.

Web Client

Copy the button icons from `<SMSSMEX Release Package>\icons` folder to `<WebApps_Root>\webtier-<version>\images\toolbar` directory.

1.3.1.14. SAP System Landscape Directory Registration

System Landscape Directory is the central information repository for your system landscape (Software Catalogue). It contains information about all installable and installed components in a system landscape. This section decribes how to register this integration into System Landscape Directory.



Note

The SAP System Landscape Directory Registration is a new feature in SMSSMEX v1.02. However, this feature is optional. If you don't deploy the SAP System Landscape Directory, the functionality of SMSSMEX v1.40 will not be affected.

Prerequisites

Service Landscape Directory is running.

Registering System Landscape Directory

1. Browse to the `<SMSSMEX Release Package` and copy the `SLDReg` folder to your computer.
2. Open the `SLDReg` folder. Modify the `HPSMISystem.properties` file according to the parameter descriptions in the file. For example, update the `ComputerName` variable to the host name which is running SMSSMEX.

   `ComputerName = <your computer name>`
3. Run the following command to compile XML file:

   java -cp SLDReg.jar com.hp.sm.sld.XMLGenerator

   After execution, HPSMI.xml is generated.
4. Run the following command to register System Landscape Directory:

   java -cp SLDReg.jar com.hp.sm.sld.Register <SLD_HOST> <SLD_HTTPPORT> <UserName> <Password>

   In this command:

   - `<SLD_HOST>` is the host name of the Service Landscape Directory server.
   - `<SLD_HTTPPORT>` is the http port of the service landscape directory service.
   - `<UserName>` is the name that you use to log in to the server.
   - `<Password>` is the password that you use to log in to the server.

Contents

- 1 Prerequisites
- 2 Registering System Landscape Directory

1.3.2. Purpose of document

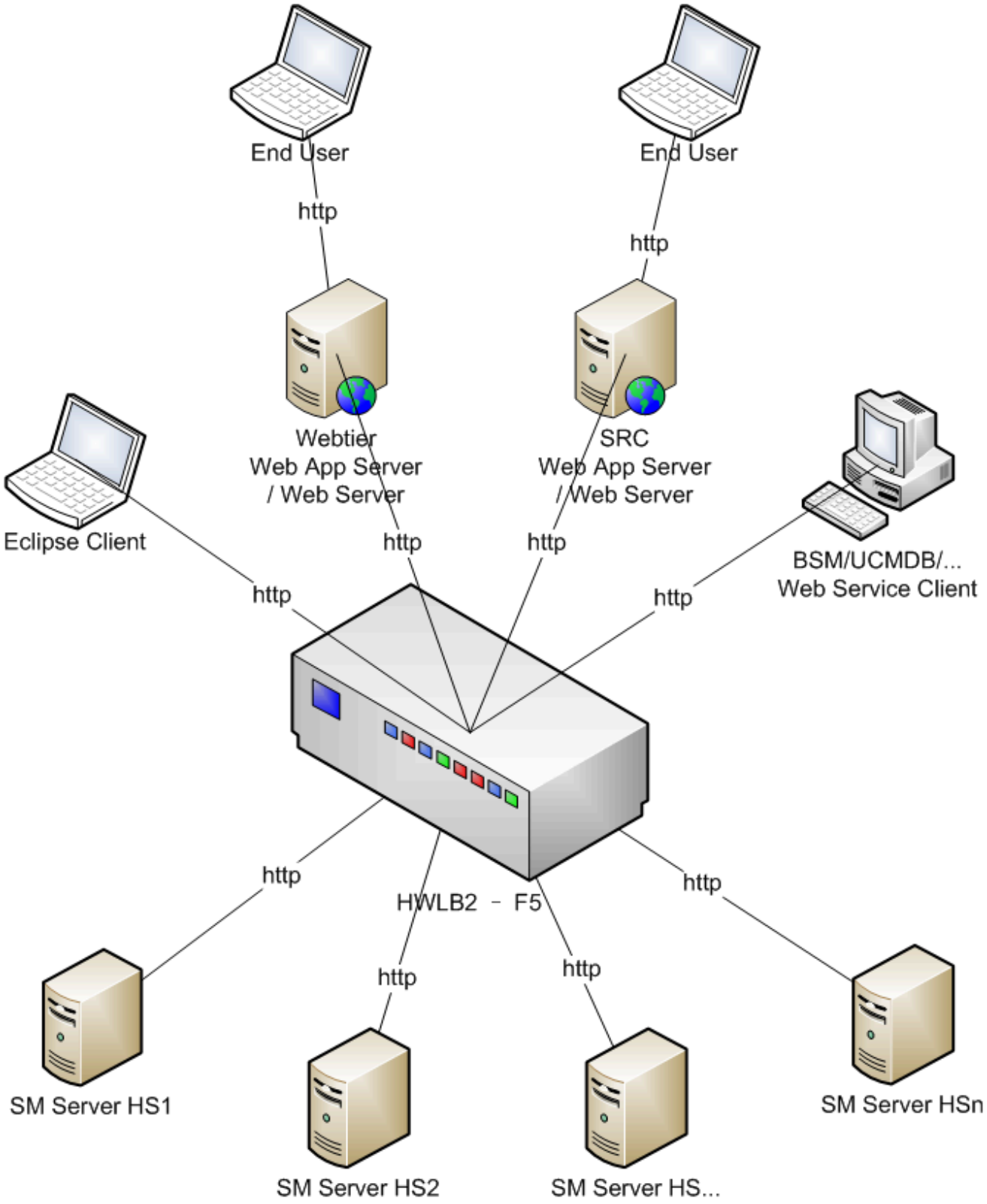
This section describes various Incident Exchange user scenarios between Service Manager and SAP Solution Manager Service Desk and is intended for the following audiences:

- Incident Analysts (and others involved in Incident Management, such as operators)
- Solution Manager User
- System Administrators (for installation and initial configuration)

1.3.2.1. Prerequisites

Refer to Incident Exchange (SM - SAP Solution Manager) for the supported component versions.

1.3.2.2. Architecture



SMSSMEX integrates a single Service Manager server with multiple external help desk systems.



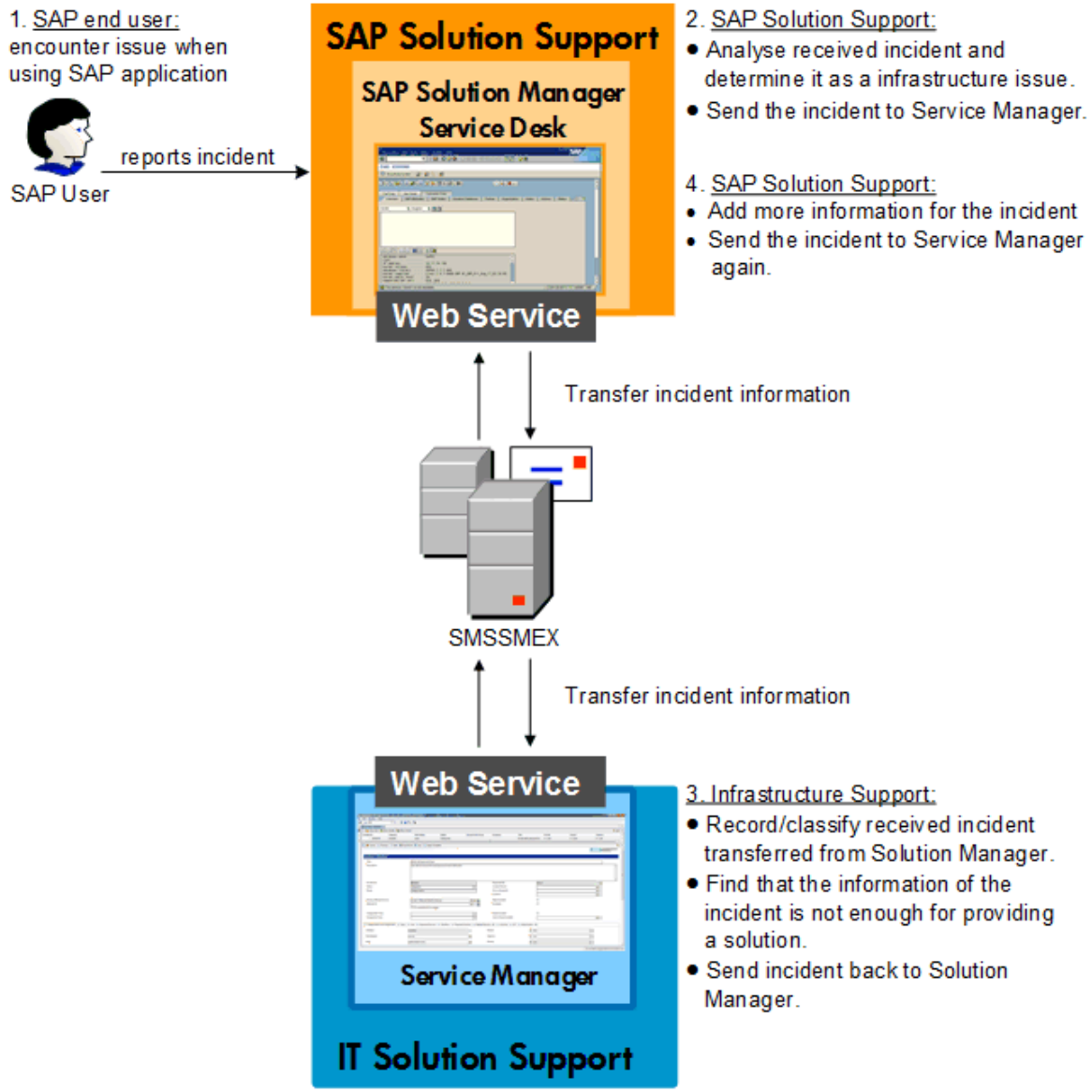
- Service Manager Server is the OpenText service desk system.
- Service Manager DB provides persistent storage for Service Manager.
- SMSSMEX Client Code consists of RAD and Java scripts, table definitions and GUI formats. The SMSSMEX webservices are called from this client code.
- WebServer is a Tomcat Web Application Server or WebLogic Application Server that hosts the SMSSMEX WebService (deployed as a .war file).
- SMSSMEX WebService exposes the incident webservice of Service Manager in the SAP format and transfers client requests to SAP Solution Manager webservices.
- SMSSMEX Database provides persistent storage for the SMSSMEX WebService.
- SAP Solution Manager is the Service Desk.

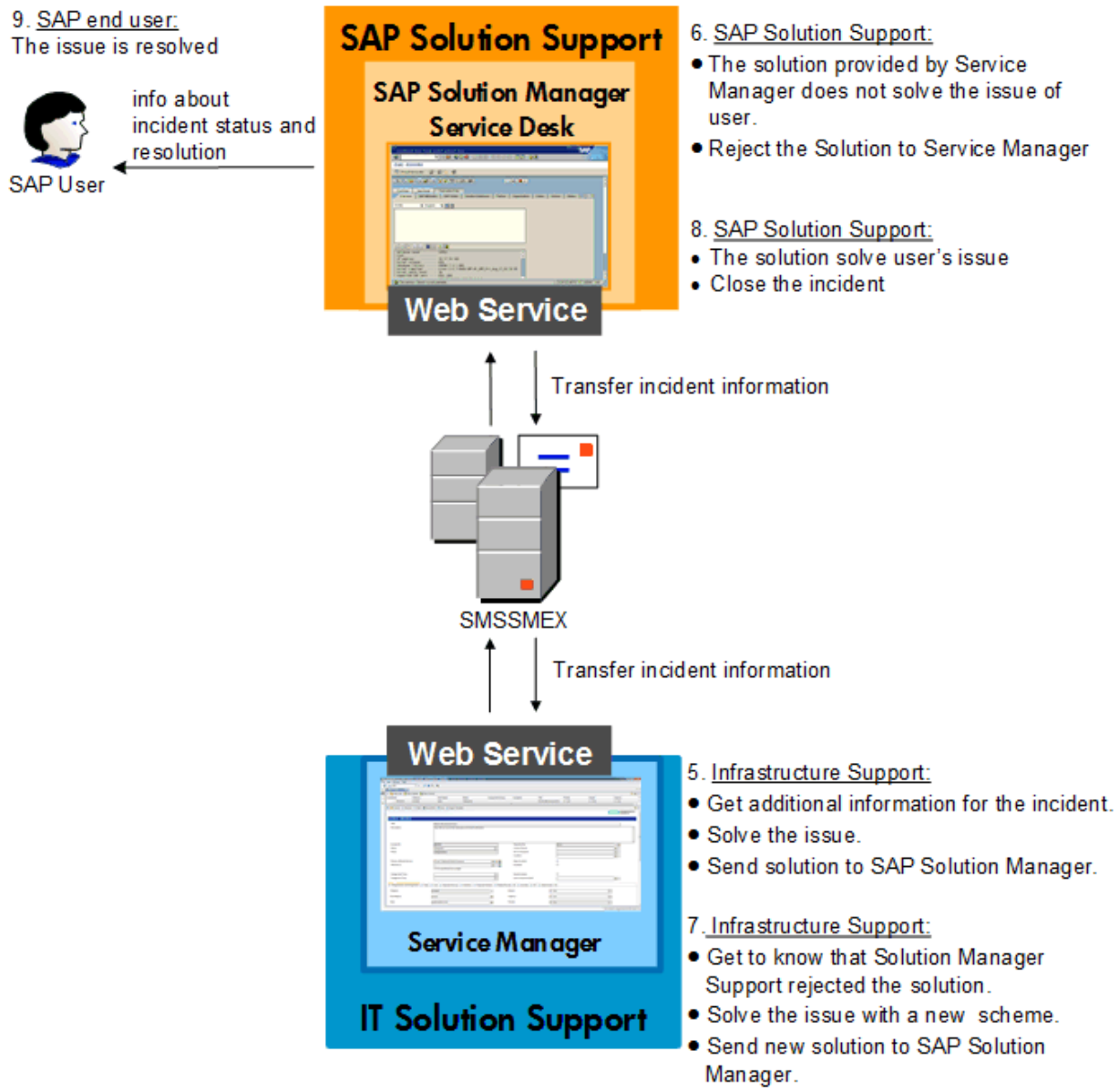
1.3.2.3. Use cases

This section discusses two use cases for Service Manager Exchange with SAP Solution Manager that demonstrate the integration scenarios.

Use Case 1: Incident Originates from Solution Manager

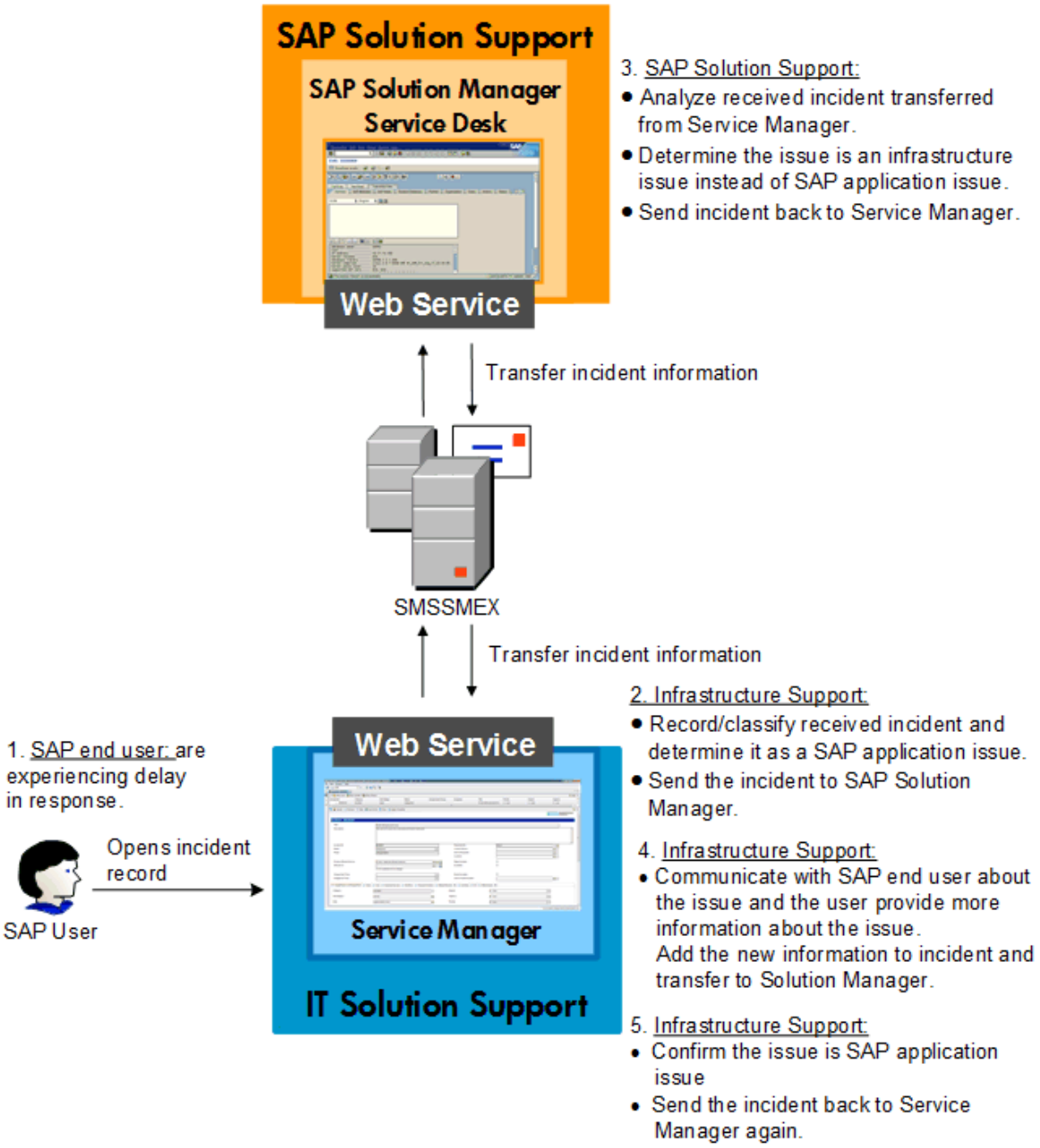
In this use case, a user reports an issue to SAP Solution Manager. The Solution Manager generates a new incident and sends the incident to Service Manager to request a solution for the issue.

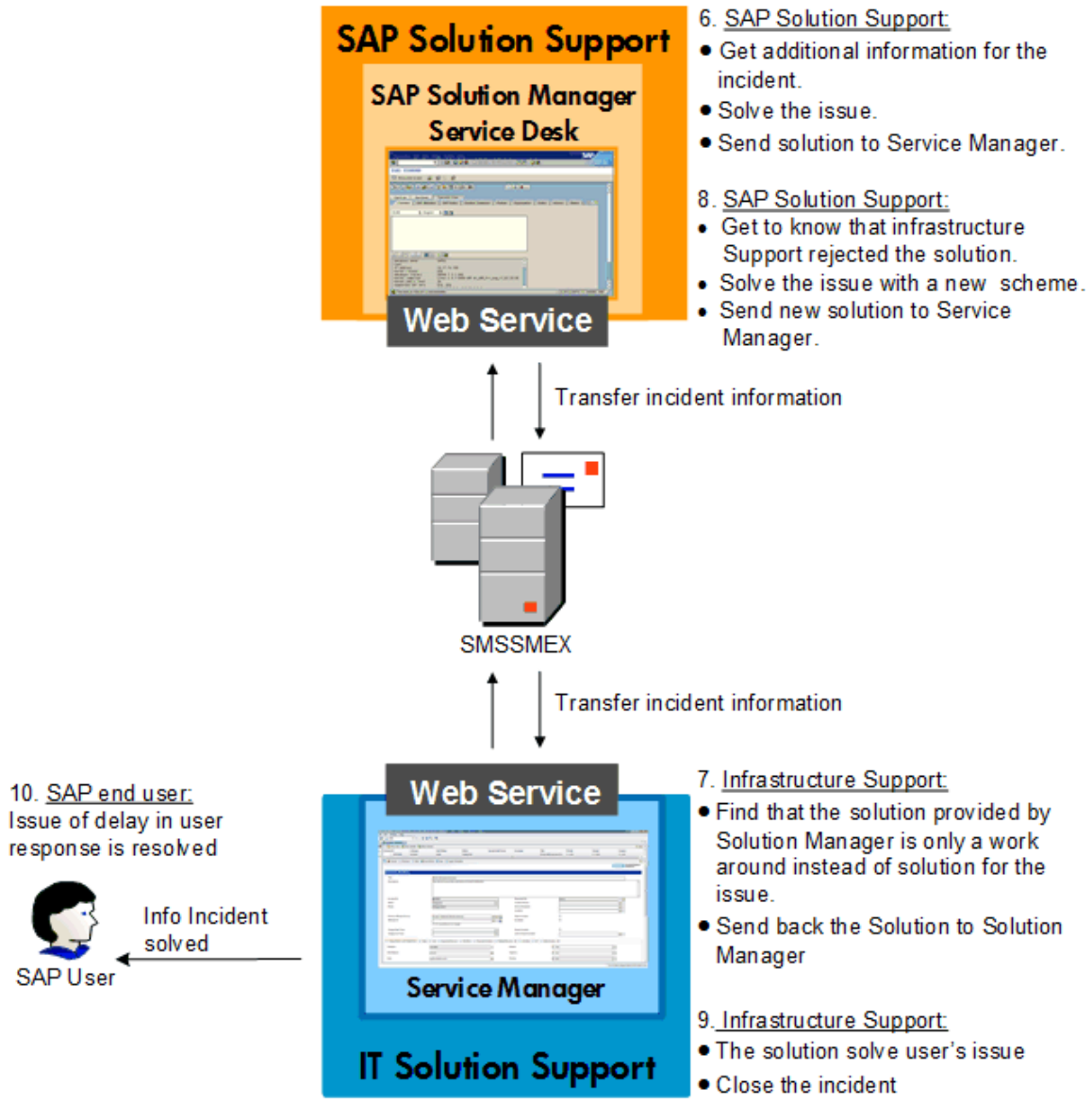




Use Case 2: Incident Originates from Service Manager

In this use case, the user issue is captured and sent to Service Manager. An incident is generated in Service Manager and is sent to Solution Manager to request a solution for the issue.


SAP Solution Support
SAP Solution Manager
Service Desk
Web Service
3. SAP Solution Support:
• Analyze received incident transferred from Service Manager.
• Determine the issue is an infrastructure issue instead of SAP application issue.
• Send incident back to Service Manager.
Transfer incident information
SMSSMEX
Transfer incident information
2. Infrastructure Support:
• Record/classify received incident and determine it as a SAP application issue.
• Send the incident to SAP Solution Manager.
1. SAP end user: are experiencing delay in response.
Opens incident record
SAP User
Web Service
Service Manager
IT Solution Support
4. Infrastructure Support:
• Communicate with SAP end user about the issue and the user provide more information about the issue.
Add the new information to incident and transfer to Solution Manager.
5. Infrastructure Support:
• Confirm the issue is SAP application issue
• Send the incident back to Service Manager again.



Contents

- 1 Use Case 1: Incident Originates from Solution Manager
- 2 Use Case 2: Incident Originates from Service Manager

1.3.2.4. User scenarios

Set up SAP Instance CI in Service Manager

SM 9.x to SAP Solution Manager 7.1

Create an incident in Service Manager

Open a new incident to send to SAP

Open the incident in Solution Manager

Check updates in Service Manager

Synchronize new information with Solution Manager

Check updates from Service Manager

Update the incident status in Solution Manager

Close the incident in Service Manager

Check the incident's status in Solution Manager

SAP Solution Manager 7.1 to SM 9.x

Create incident in Solution Manager

Send Solution in Service Manager

Check the solution from Service Manager

Send the incident back to Service Manager

Send the incident back to Solution Manager again

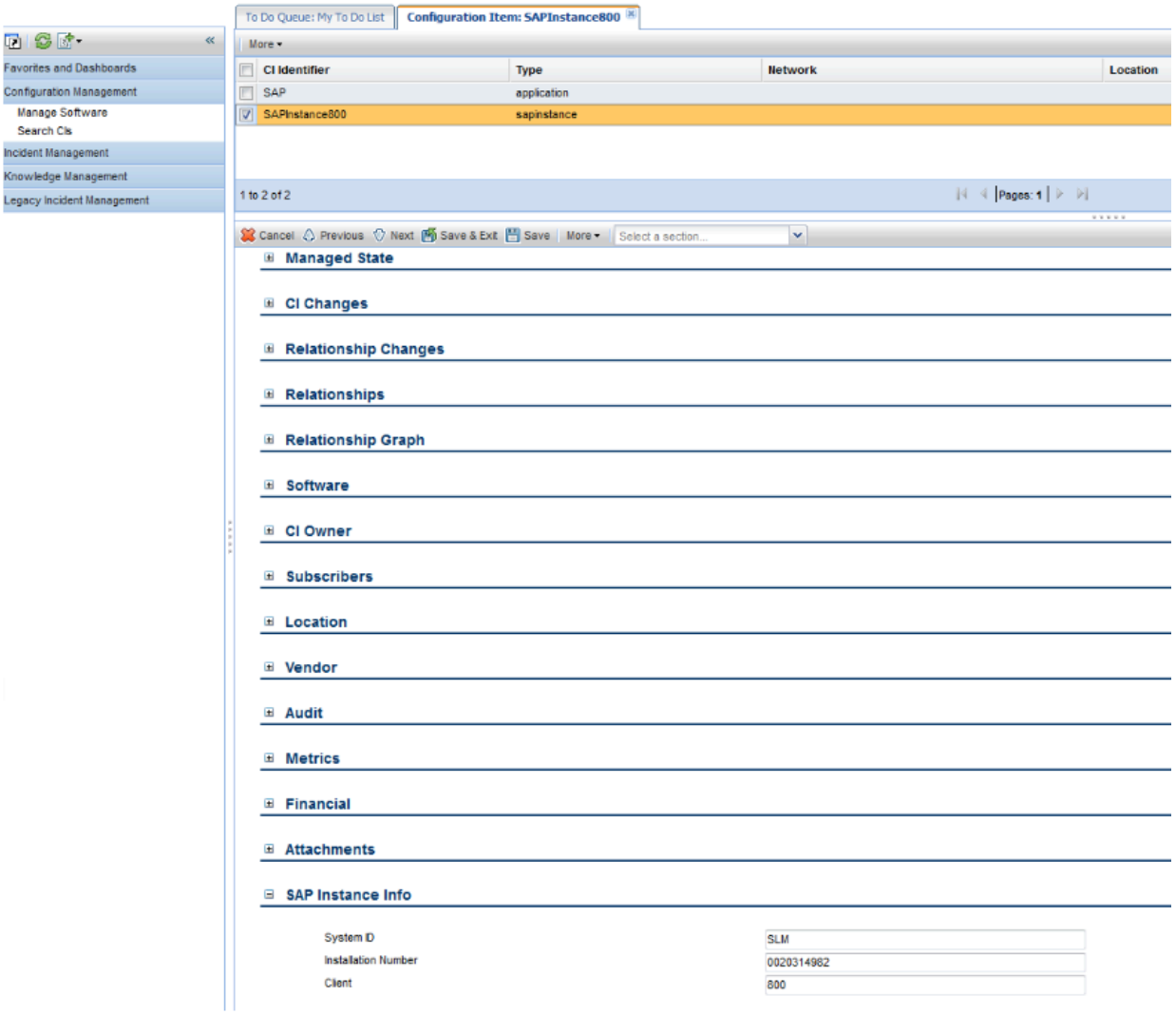
Close the incident in Solution Manager

Check the incident's status in Service Manager

1.3.2.4.1. Set up SAP Instance CI in Service Manager

Follow these steps:

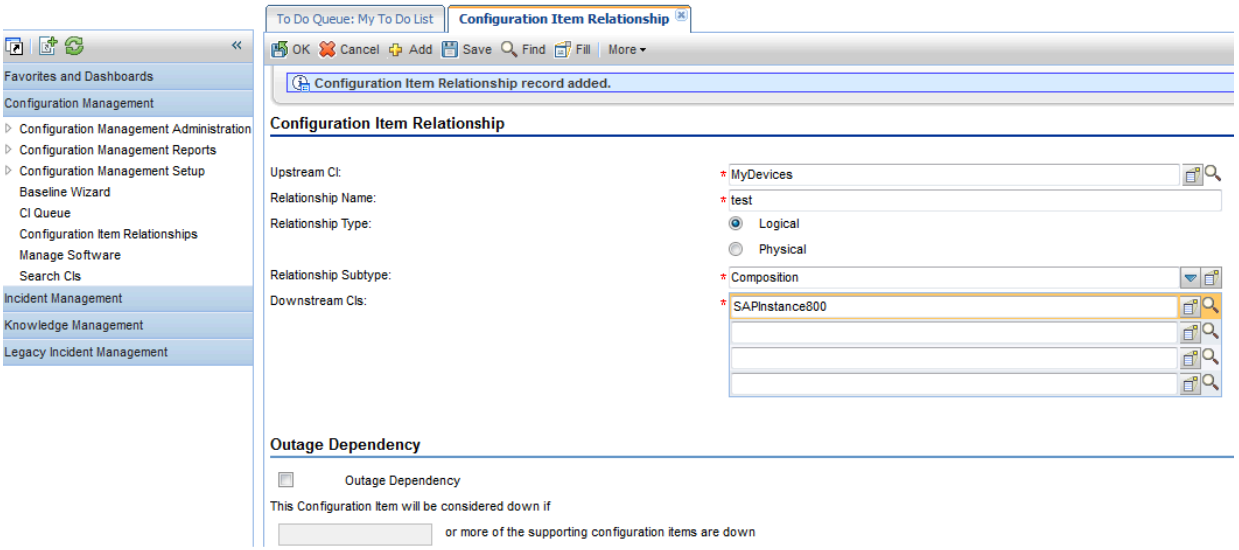
1. Sign in to Service Manager as **Config.Manager**.
2. Click **Configuration Management > Search CIs**.
3. Set **SAPInstance** for the Type field.
4. Click **New**.
5. Create a **SAPInstance** CI, and then enter SAP Instance Info.



To get System ID, Installation Number and Client information from SAP:

1. Sign in to Service Manager as **Config.Admin**.
2. Click **Configuration Management** > **Configuration Item Relationships**.
3. Select **MyDevices** as the Upstream CI.

4. Select the **SAPInstance** CI as the Downstream CI.
5. Fill in other fields.
6. Click **Add**.
7. Click **OK**.



1.3.2.4.2. SM 9.x to SAP Solution Manager 7.1

Create an incident in Service Manager

Open a new incident to send to SAP

Open the incident in Solution Manager

Check updates in Service Manager

Synchronize new information with Solution Manager

Check updates from Service Manager

Update the incident status in Solution Manager

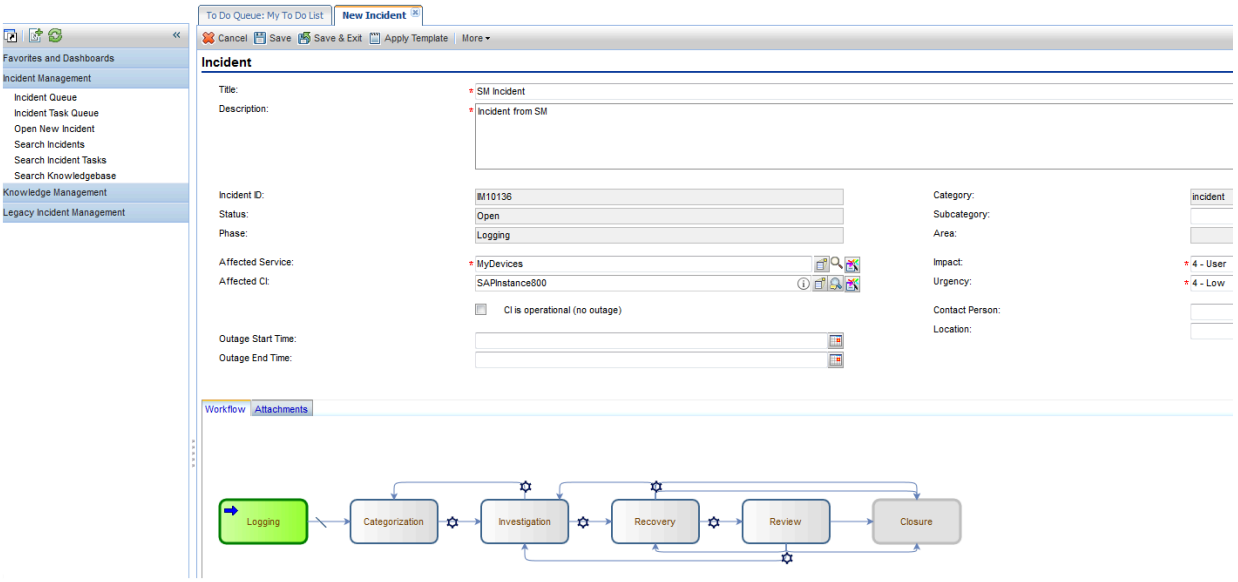
Close the incident in Service Manager

Check the incident's status in Solution Manager

1.3.2.4.2.1. Create an incident in Service Manager

Follow these steps:

1. Sign in to Service Manager as **Incident.Manager**.
2. Click **Incident Management** > **Open New Incident**. Click the Incident category. The incident ticket quick form opens.



3. Fill in required fields for the new incident as necessary:
    1. Click **Fill** to select a Subcategory, and then select an Area.
    2. Click **Fill** to select the applicable **Affected Service, "MyDevices"**.
    3. Click **Fill** to select the **Affected CI**. The CI describes the client information of Solution Manager, such as **SystemID**, **Installation Number**, and **Client**.

        Caution

        You should first select **Affected Service** and then select **Affected CI**. Please follow the sequence.

    4. Type a **Title** for the incident.

        Note

        The Default Impact and Priority values of the affected CI are automatically populated to the Impact and Urgency fields of the incident record. You can manually change these automatically populated values if needed.

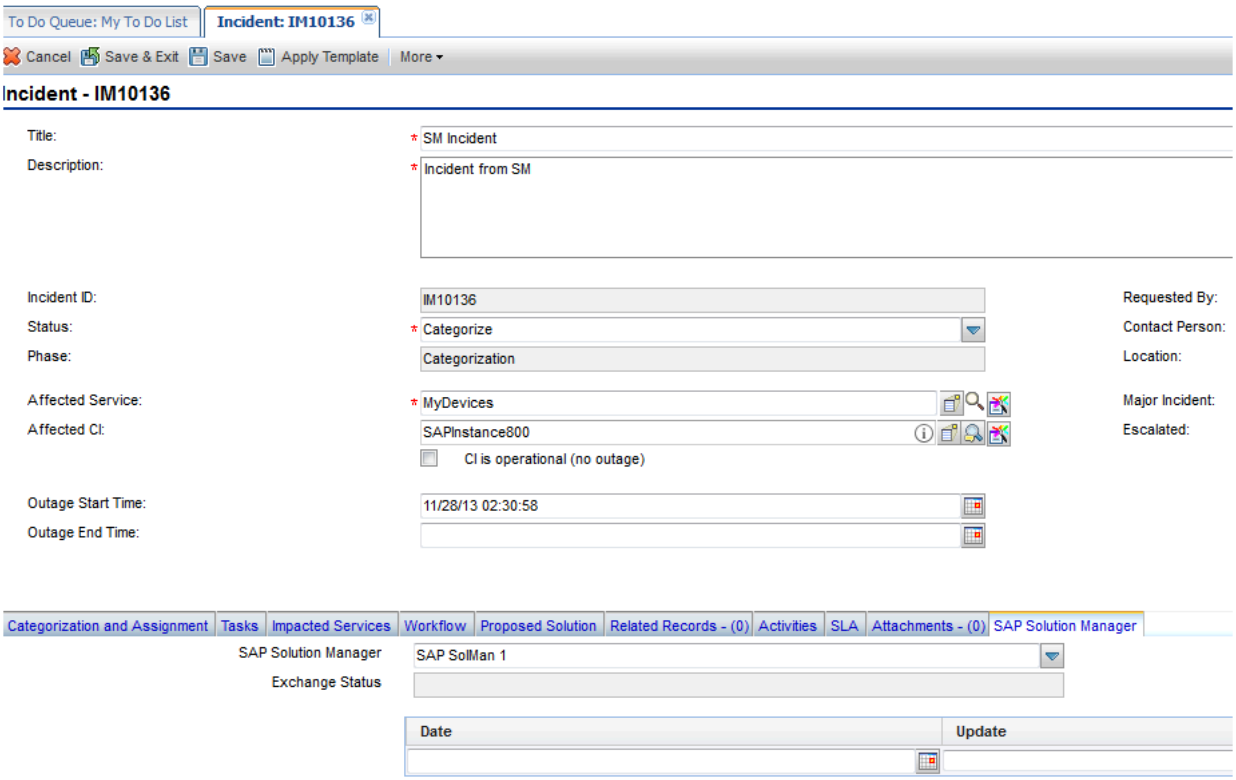
> **Caution**
>
> If you specify the Impact and Urgency values first and then specify an Affected CI with an empty Default Impact or Priority value, the system clears the Impact or Urgency value you selected for the incident record.

5. Type a **Description** for the incident.
6. Click **Search Knowledge** icon to see if the issue is already logged in the knowledge base.
7. Complete the required fields in the Incident Details section.
8. Complete the form with any other relevant information.

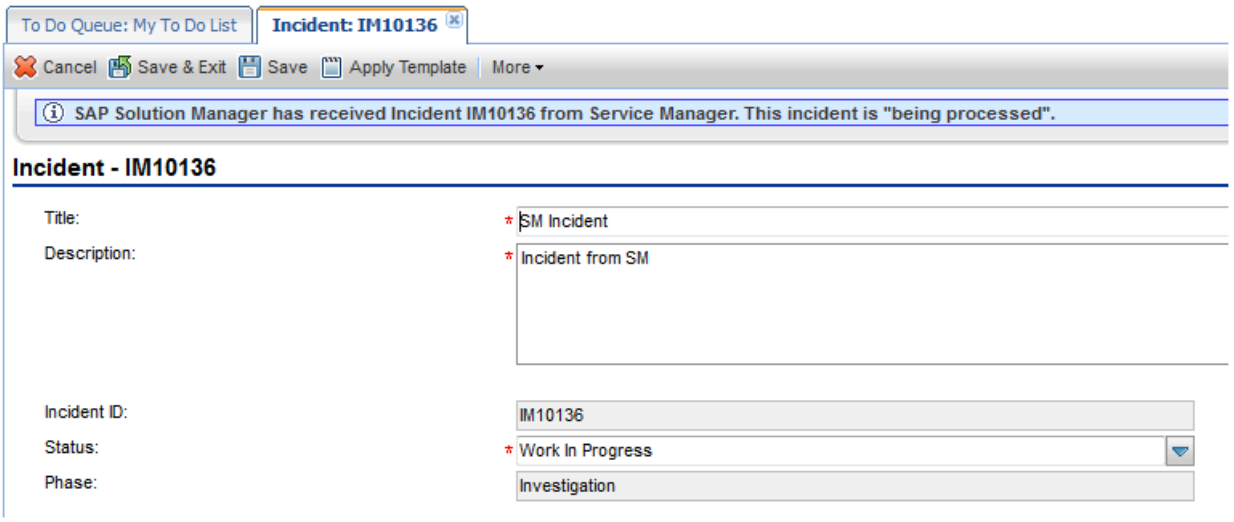
4. Click **Save&Exit** and then click **Open New Incident** to create an incident.

1.3.2.4.2.2. Open a new incident to send to SAP

1. Click **Incident Management > Search Incidents**. The Display Which Incident Tickets form opens.
2. In the Incident ID textbox, type the ID of the new incident created in step 1 and click **Search**. The incident opens.
3. Extend the **Sap Solution Manager** tab and select a Solution Manager client in the **SAP Solution Manager** drop-down list.

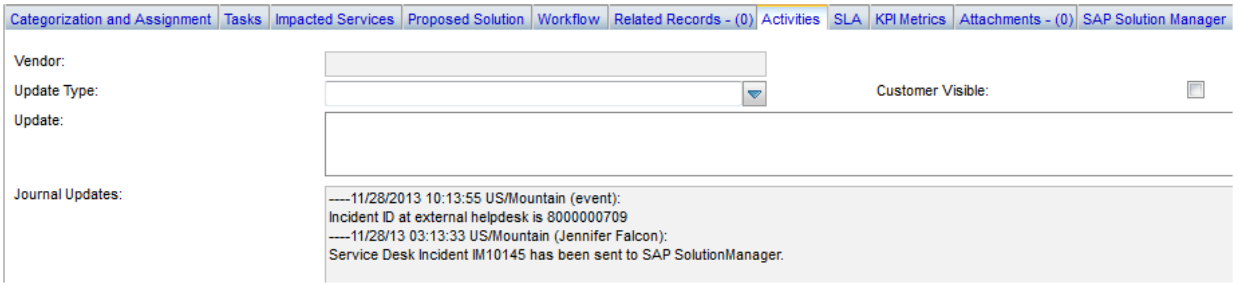


4. Update the Status field to **Work In Progress**, fill in other fields, and then click **Save**.
5. Click **Send Incident** to send the incident to SAP.
6. Service Manager receives the **Request accepted** message after the incident is sent to middleware successfully. Click **Save & Exit** to close the incident window.



7. After a few minutes, re-open the incident to check whether it is sent to SAP.

   If the incident is sent successfully, Service Manager receives the Incident ID message from SAP.

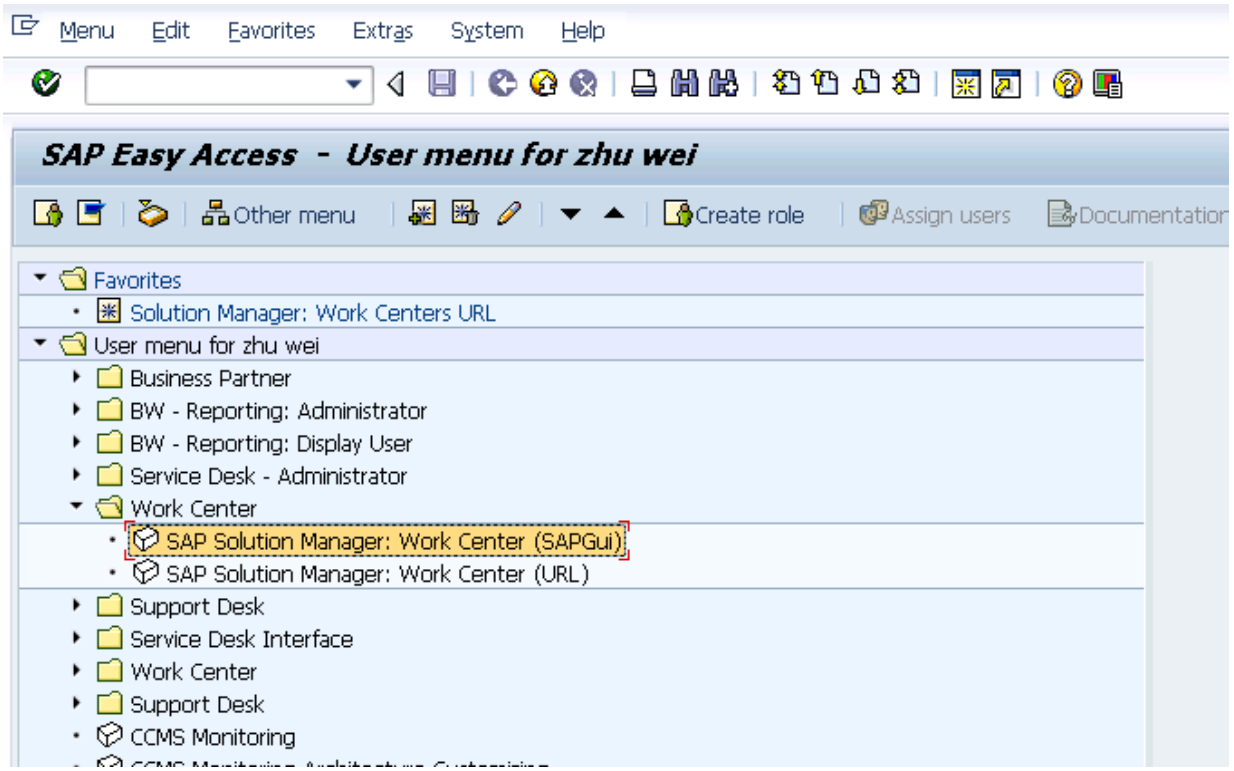


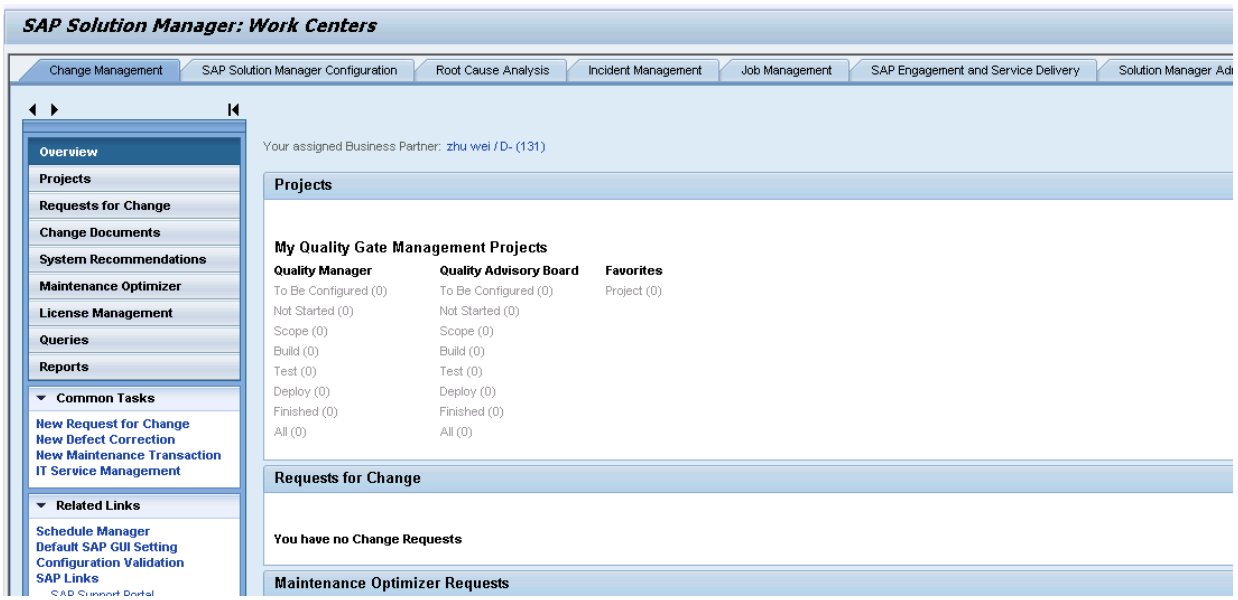
8. Click **Cancel** to close the incident window.

1.3.2.4.2.3. Open the incident in Solution Manager

Follow these steps:

1. Sign in to Solution Manager.
2. Click **Work Center** > **SAP Solution Manager: Work Center (SAP GUI)** to open Solution Manager Work Center.

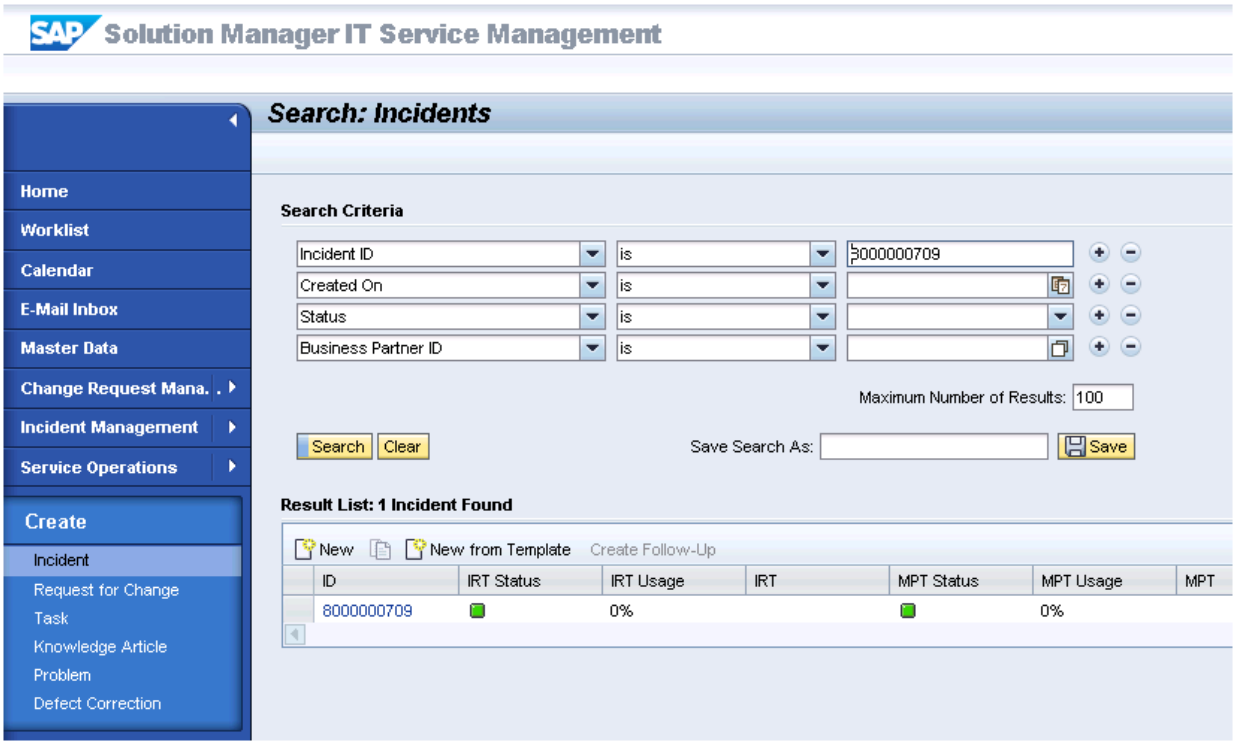




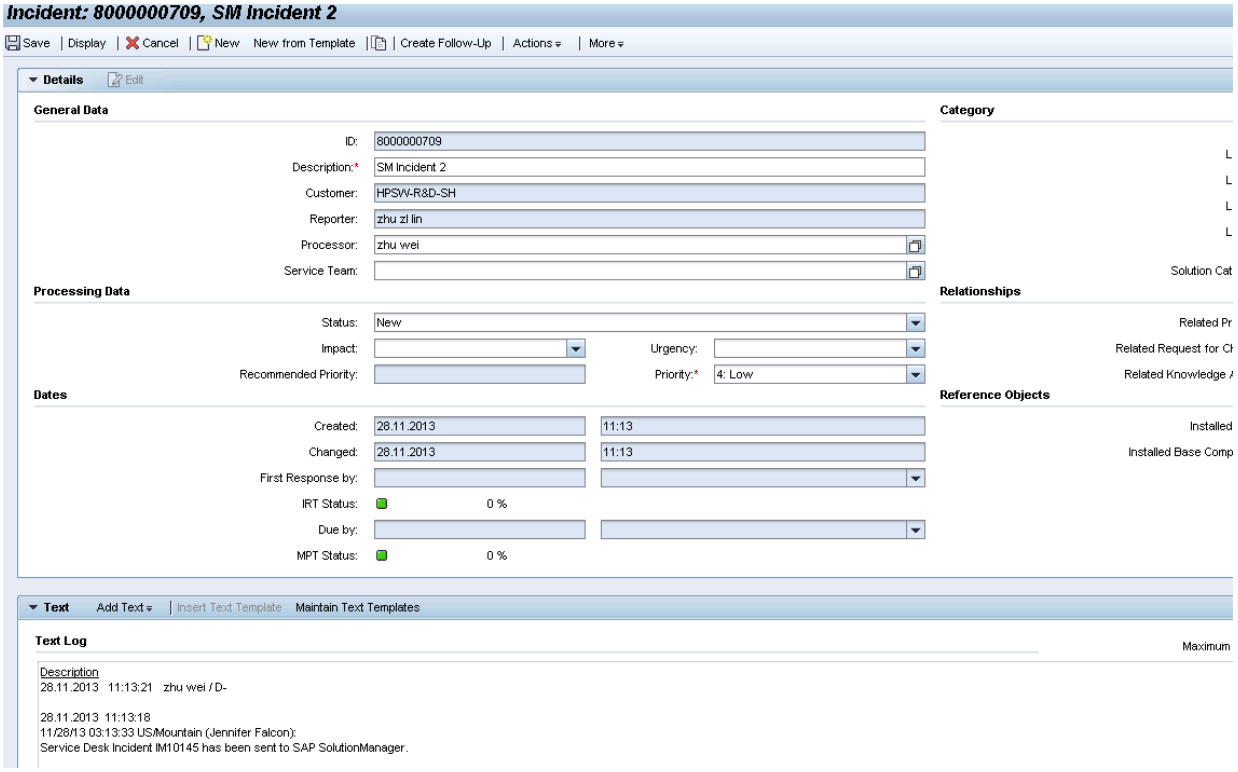
3. Click **Common Tasks** > **IT Service Management**. The SAP Solution Manager IT Service Management page opens in Explorer.



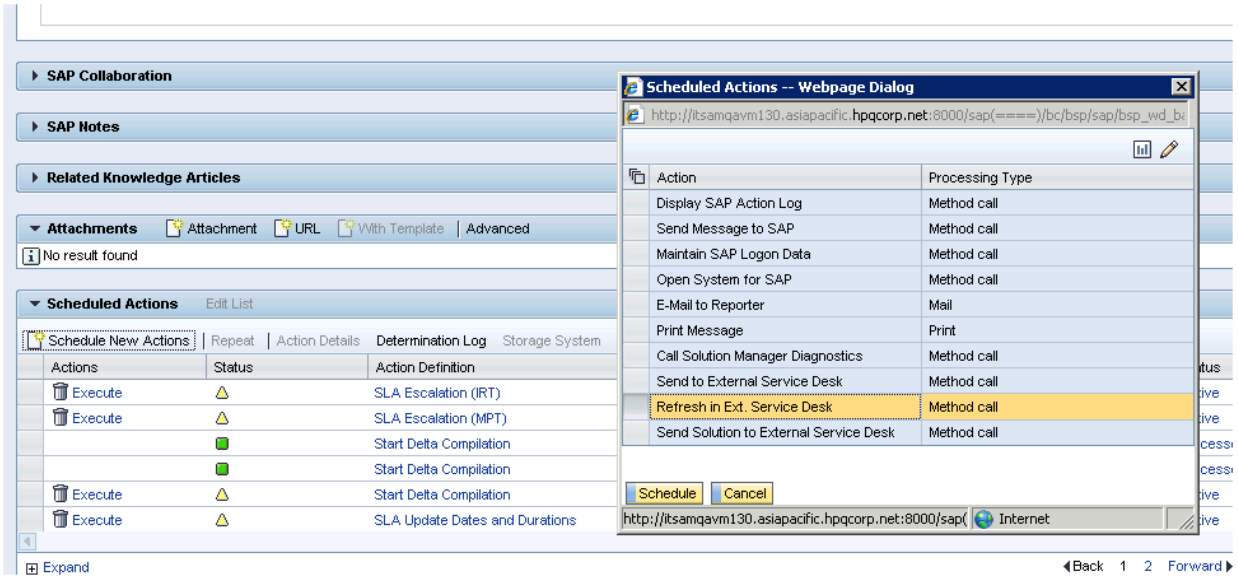
4. Click **Search** > **Incidents** to open the search window.
5. Type the incident ID in Search Criteria and then click **Search**. The Result List displays the incident.



6. Click the incident ID link to open the incident.
7. Fill in the required fields (Reporter) and change the Status to **In Process**.



8. Click **Add Text** in the Text drop-down section to add a description of the incident.
9. Add **Refresh in Ext. Service Desk** scheduled action.



10. Click **Save**. The incident information synchronizes with Service Manager automatically.

> **Note**
>
> Solution Manager saves the text field automatically and periodically. The text field saved automatically isn't synchronized with Service Manager.

11. Click **Display** to switch the incident to view mode.

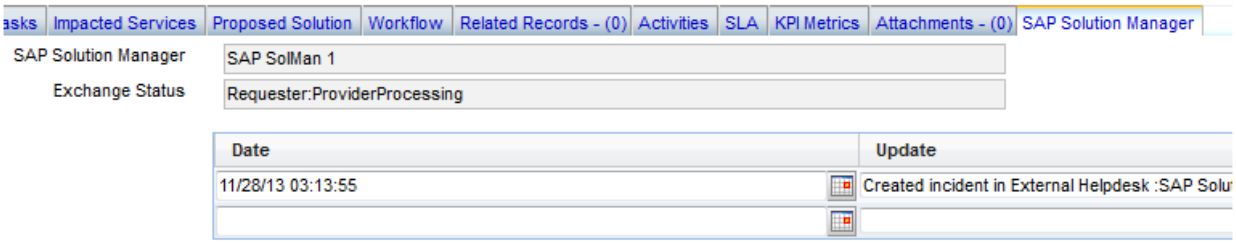
1.3.2.4.2.4. Check updates in Service Manager

Follow these steps:

1. Click **Incident Management > Search Incidents**. The Display Which Incident Tickets form opens.
2. In the Incident ID field, type the ID of the new incident created in step 1 and click **Search**. The incident opens.
3. Extend the **Sap Solution Manager** tab to view the incident's status from Hidden Metadata.

    • **Requester:** Indicates Service Manager has sent the incident. Solution Manager is the provider.

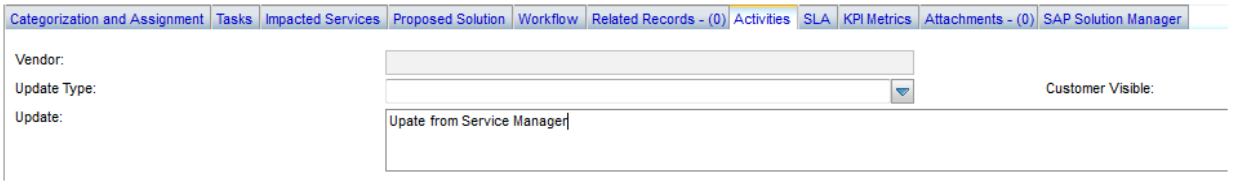
    • **ProviderProcessing:** Indicates Solution Manager is processing the incident.



4. Extend the **Activities** tab to view the message of Journal Updates. Service Manager receives the **Desc from Solution Manager** message from Solution Manager. The read-only Journal Updates field displays messages from Solution Manager every time it updates the incident.

1.3.2.4.2.5. Synchronize new information with Solution Manager

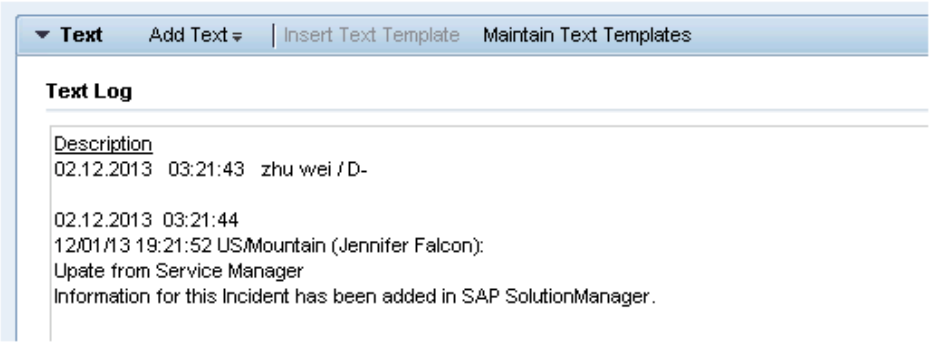
1. Extend Activities tab to input “Update from Service Manager” message into New Update textbox.



2. Click **Add Info** to synchronize with Solution Manager and then Service Manager receives the message “being processed”.
3. Click **Cancel** to close the incident window.

1.3.2.4.2.6. Check updates from Service Manager

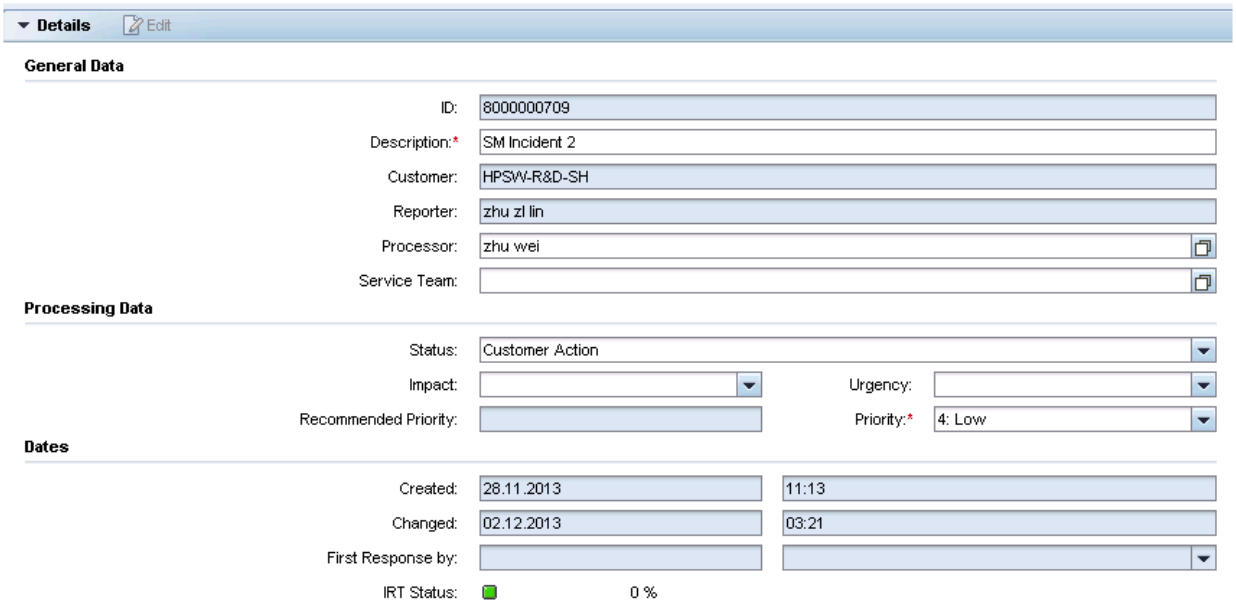
1. Search the incident and then open it.
2. As shown in the following screenshot, Solution Manager receives update from Service Manager.



1.3.2.4.2.7. Update the incident status in Solution Manager

1. Search the incident and open it.
2. Click **Edit** to switch to the edit mode.
3. Change the status to **Customer Action** and then save it.

   The incident's status must be changed to **Customer Action** or **Proposed Solution** in Solution Manager if the incident is closed in Service Manager.
4. Click **Display** to switch to the view mode.



1.3.2.4.2.8. Close the incident in Service Manager

Follow these steps:

1. Click **Incident Management** > **Search Incidents**. The Display Which Incident Tickets form opens.
2. In the Incident ID textbox, type the ID of the new incident created in step 1 and click **Search**. The incident opens.
3. Update Status to Resolved, and provide solution.
4. Click **Save**.
5. Click **Close SAP Incident** to close the incident in Service Manager.



Caution

The incident status in SAP Solution Manager MUST be **Customer Action** or **Proposed Solution**, then the incident can be closed in Service Manager side.

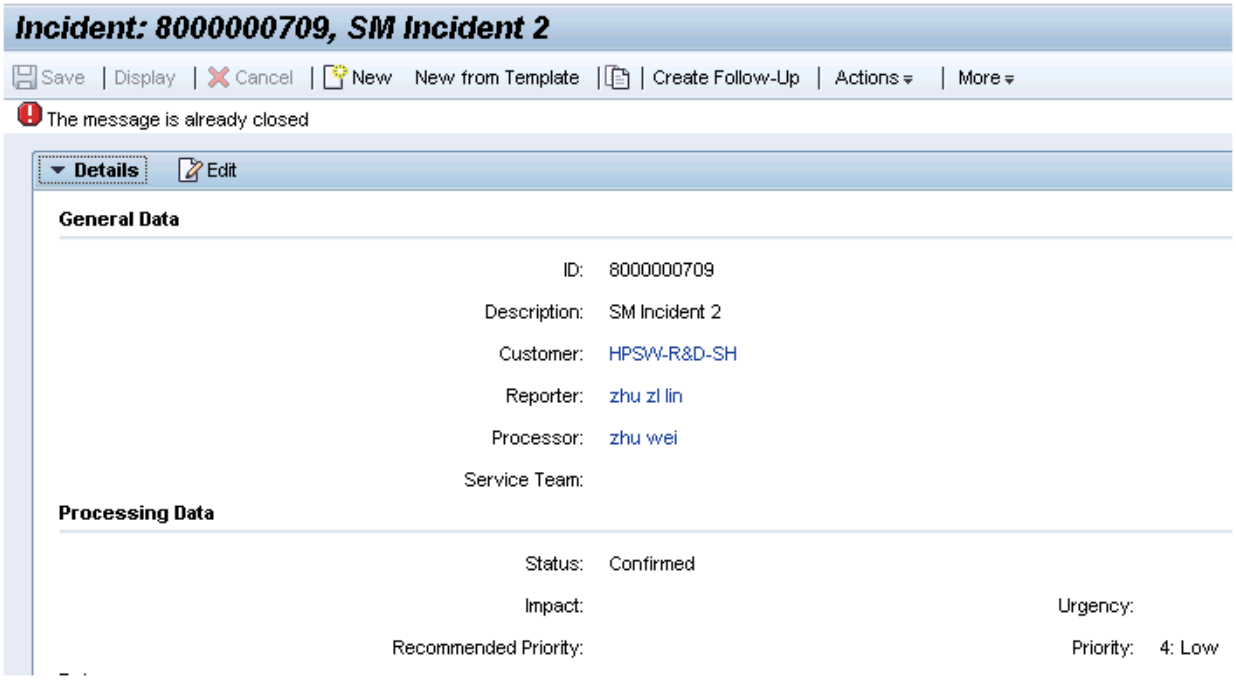


Note

Wait for a few minutes after you clicked the **Close SAP Incident** button. The Close Incident action is unsynchronized between Service Manager and SAP Solution Manager.

1.3.2.4.2.9. Check the incident's status in Solution Manager

1. Search the incident and then open it.
2. Solution Manager displays the message “The message is already closed”.
3. The incident’s status is changed to Confirmed.



1.3.2.4.3. SAP Solution Manager 7.1 to SM 9.x

Create incident in Solution Manager

Send Solution in Service Manager

Check the solution from Service Manager

Send the incident back to Service Manager

Send the incident back to Solution Manager again

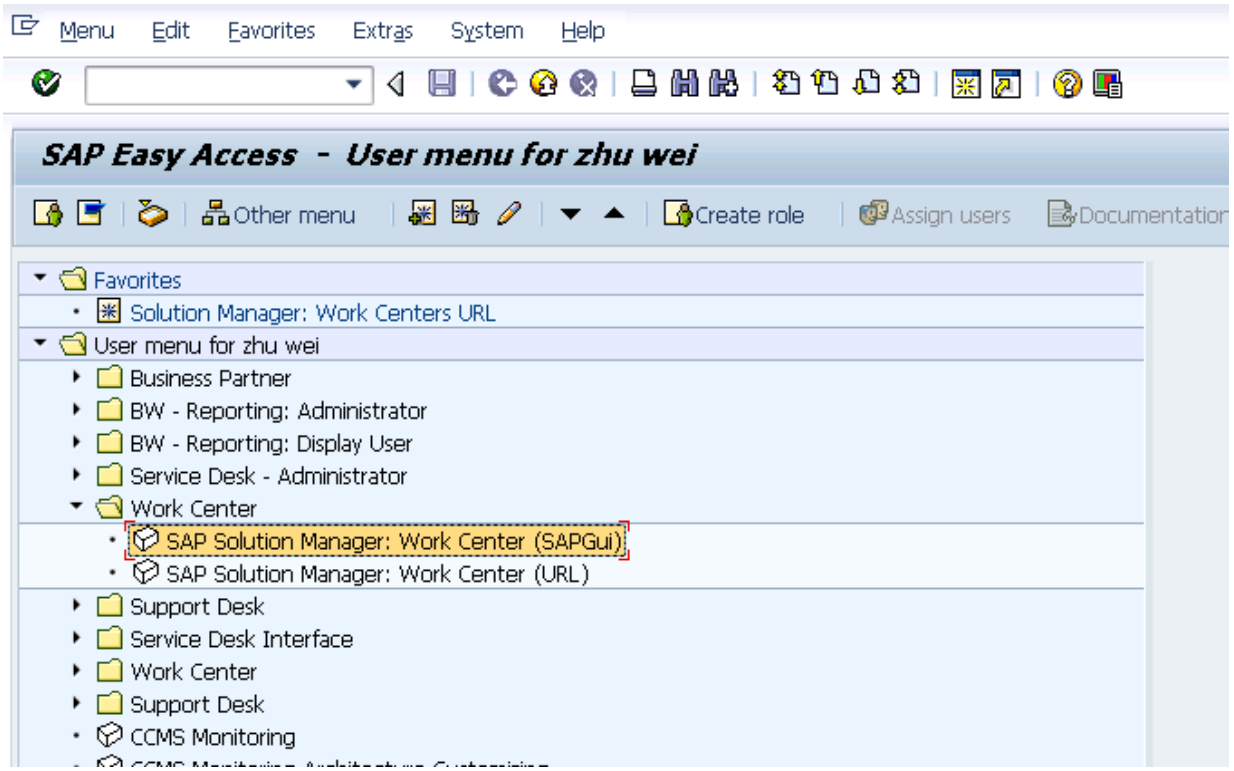
Close the incident in Solution Manager

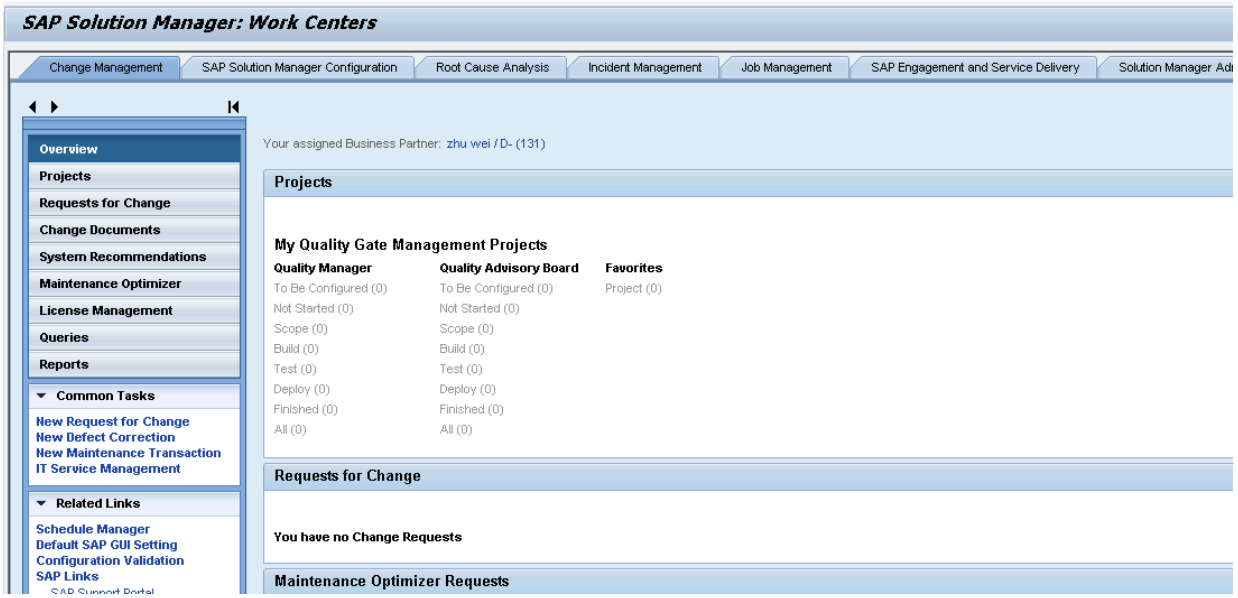
Check the incident's status in Service Manager

1.3.2.4.3.1. Create incident in Solution Manager

Follow these steps:

1. Sign in to Solution Manager.
2. Click **Work Center** > **SAP Solution Manager: Work Center (SAP GUI)** to open Solution Manager Work Center.

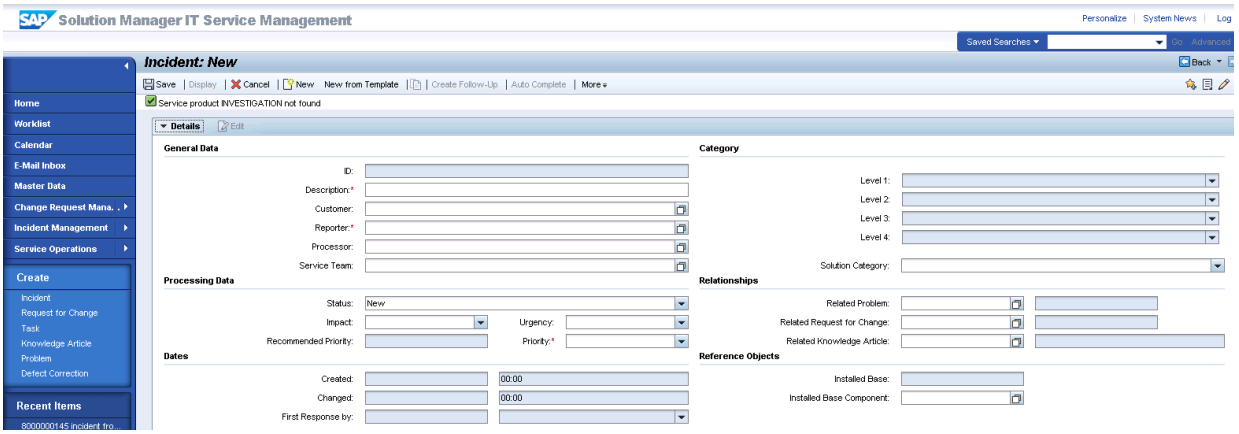




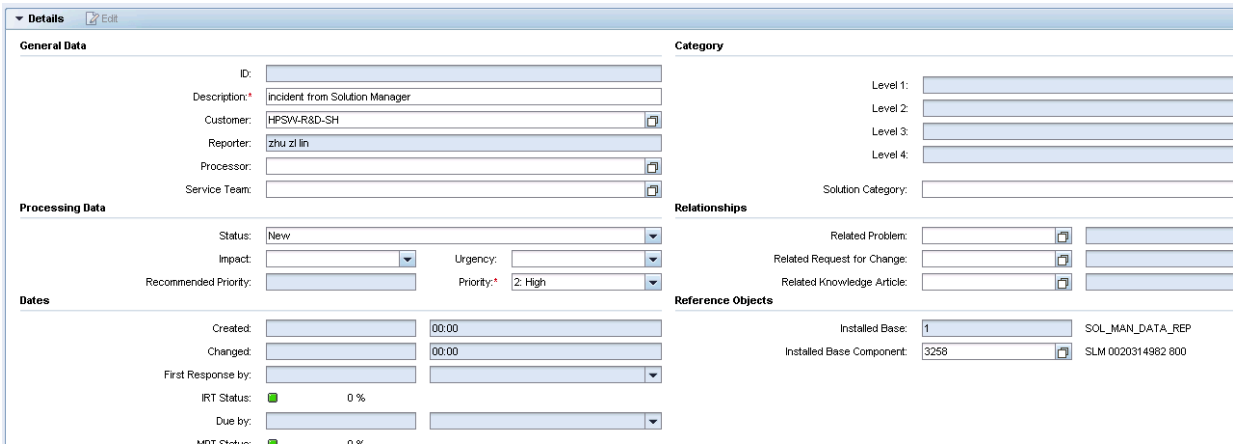
3. Click **Common Tasks** > **IT Service Management**. The SAP Solution Manager IT Service Management page opens in Explorer.



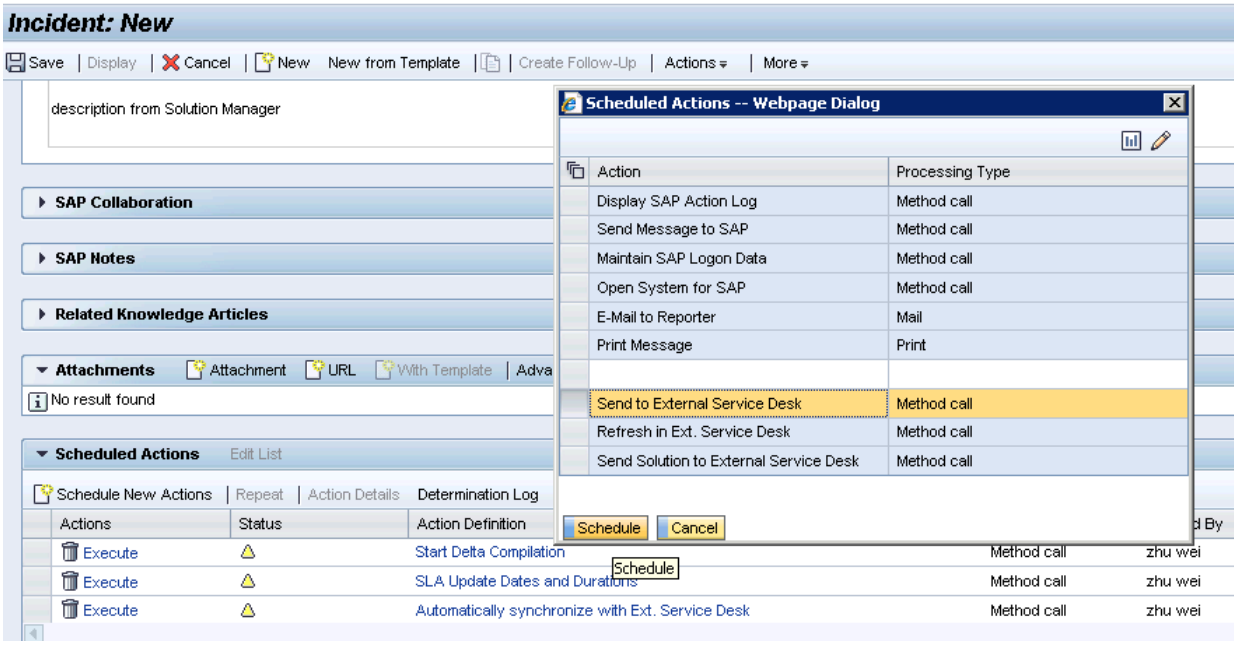
4. Click **Create** > **Incident**. The Incident: New page opens.



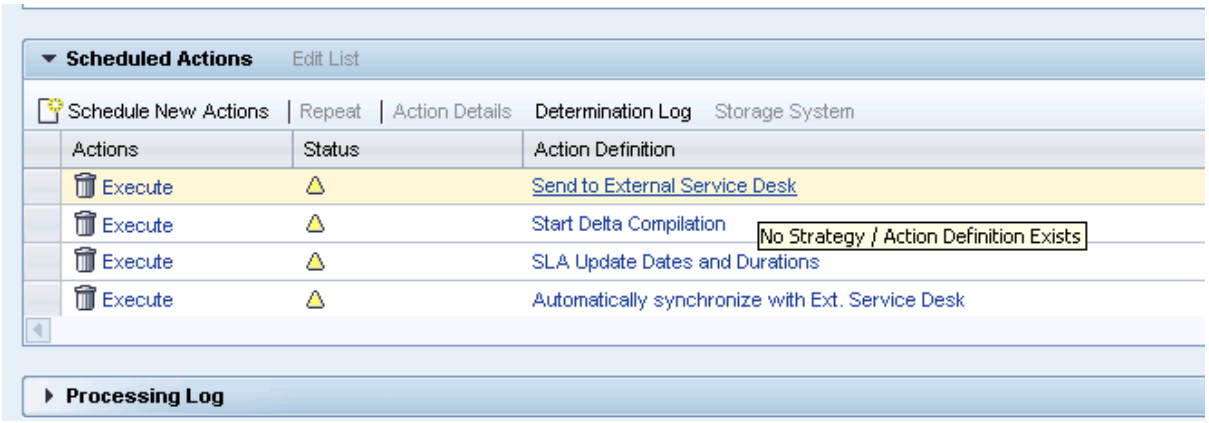
5. Fill in the required fields for the new incident as necessary:
   - Type a Description and a Reporter for the incident.
   - Select a Priority in the drop-down list.
   - Click **Fill** to select the Installed Base Component. This field describes the client information of Solution Manager, such as **SystemID**, **Installation Number**, and **Client**.
   - Complete the form with any other relevant information.



6. Click **Schedule new Action** in Schedule Actions drop-down section and select **Send to External Service Desk** from the list in the new window opened.



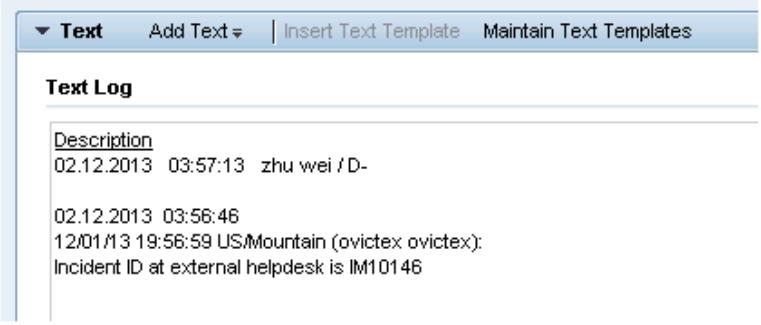
7. As shown in the following screenshot, the new action is waiting for execution in the action list.



8. Click **Save** to send the incident to Service Manager.
9. Click **Display** to switch the incident to view mode.

10. After a few minutes, open the incident again to check whether it's sent to Service Manager.

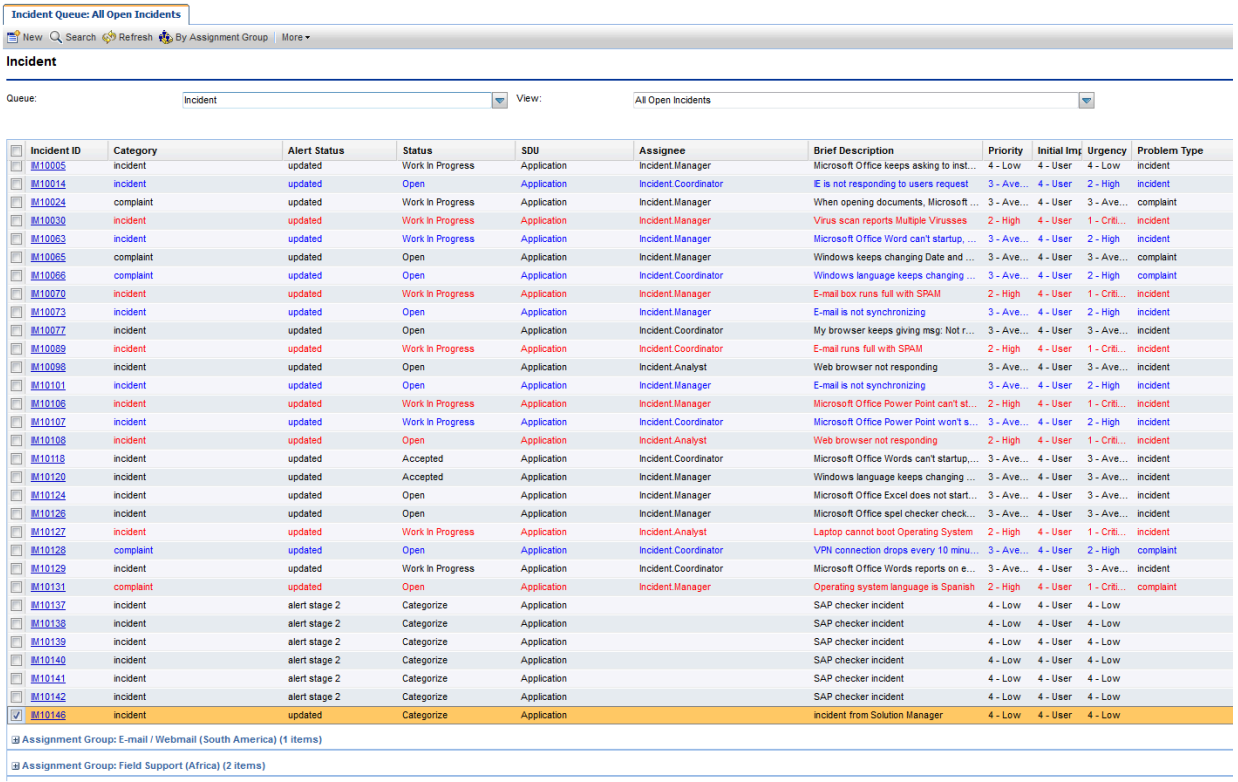
    If the incident is successfully sent, SAP receives the Incident ID message from Service Manager.



1.3.2.4.3.2. Send Solution in Service Manager

Follow these steps:

1. Sign in to Service Manager as Incident.Manager.
2. Click **Incident Management** > **Incident Queue**. Find the incident from Solution Manager in the Incident Queue form.



3. Open the incident and check the updated information from Solution Manager.
4. Extend the **Sap Solution Manager** tab to view the incident's status from Hidden Metadata.
   - **Provider:** Indicates the incident is requested by Solution Manager. Service Manager provides the solution to Solution Manager.
   - **ProviderProcessing:** Indicates the incident is being processed by Service Manager.

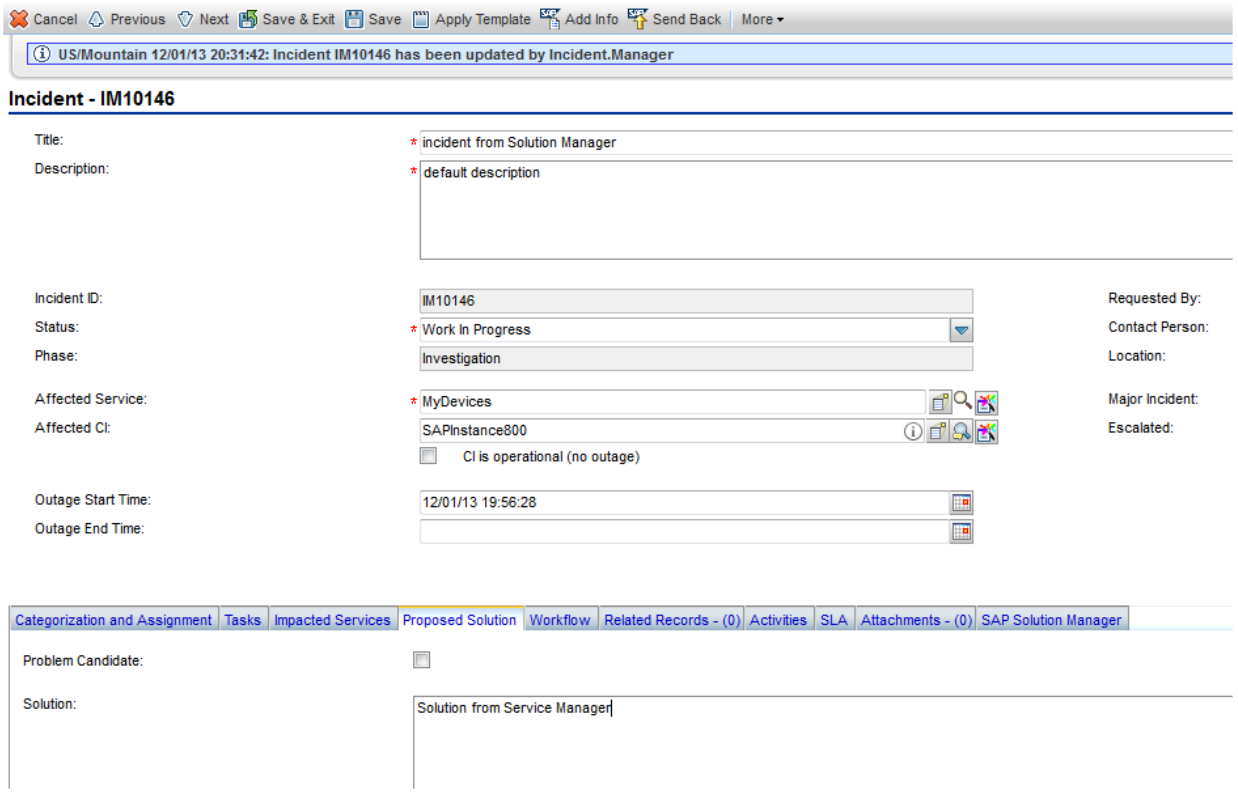


5. Extend the **Activities** tab to view the message of Journal Updates. Service Manager receives a message that displays the incident ID of Solution Manager. For example:

   Incident ID at external helpdesk is IM10146.

6. Update the status to Work In Progress and fill other fields.

7. Click **Save**.
8. Type solution in the **Solution** text box in the Proposed Solution tab.



9. Click **Save**.
10. Click **Send Solution** to send solution provided by Service Manager to Solution Manager.
11. Click **Cancel** to release the incident.
12. After a few minutes, click the Sap Solution Manager tab to view the incident's status from Hidden Metadata.

    - **SolutionProvided:** Indicates the incident has been sent with solution by Service Manager.



    And the log is recorded with the message "the solution is proved to Solution Manager" below the Hidden Metadata textbox. The Send Solution button and the Send Back button are unavailable and only the Add Info button is active.

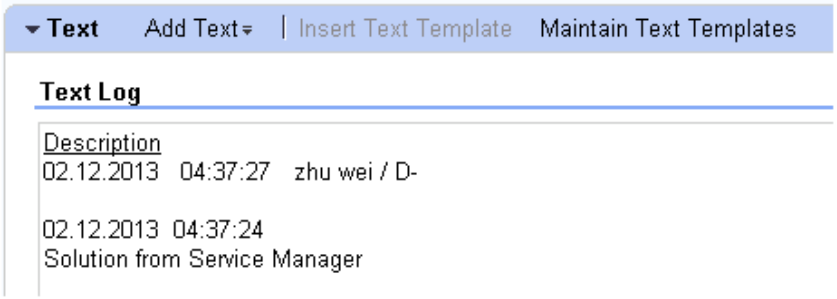
    - **Send Solution:** Service Manager provides a solution to Solution Manager. The action is unidirection. After the action, the incident in Service Manager can't send a second solution. Only adding new information into the incident is allowed. Otherwise, the incident will be sent back by Solution Manager.

- **Add Info:** Service Manager can synchronize with Solution Manager continually. The action is bidirection.
- **Send Back:** Service Manager rejects the incident from Solution Manager. The action is unidirection. After the action, the incident in Service Manager can't be sent back again to Solution Manager.

13. Click **Cancel** to close the incident window.

1.3.2.4.3.3. Check the solution from Service Manager

Search the incident and open it. As shown in the following screenshot, Solution Manager receives the message **Solution from Service Manager** from Service Manager.



1.3.2.4.3.4. Send the incident back to Service Manager

1. Search the incident and open it.
2. Click **Edit** to switch the incident to edit mode.
3. Change the status to **In Process**.
4. Create a New Scheduled Action **Send to External Service Desk** to send the incident back to Service Manager.
5. Click **Display** to switch the incident to view mode.

1.3.2.4.3.5. Send the incident back to Solution Manager again

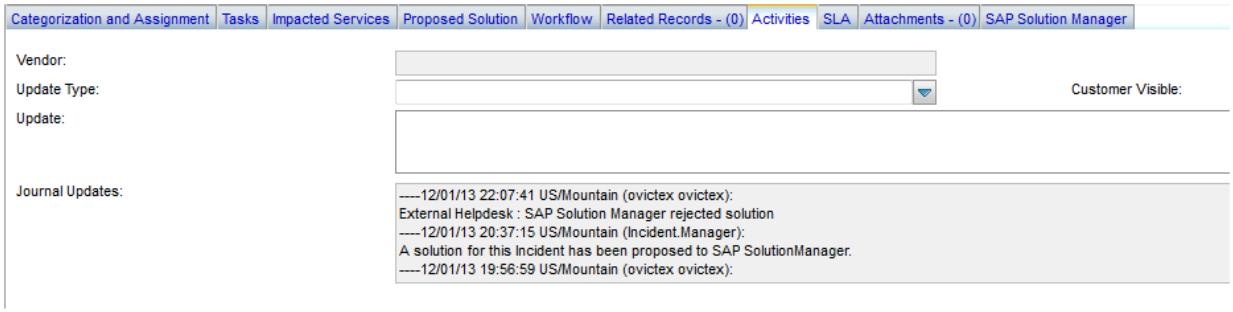
Follow these steps:

1. Sign in to Service Manager as **Incident.Manager**.
2. Click **Incident Management** > **Incident Queue**. Find the incident from Solution Manager in the Incident Queue form.
3. Open the incident and check the updated information from Solution Manager.
4. Extend the **Sap Solution Manager** tab to view the incident's status from Hidden Metadata.

   The Hidden Metadata field displays **Provider: ProviderProcessing**, which means Solution Manager is waiting for the solution provided by Service Manager.

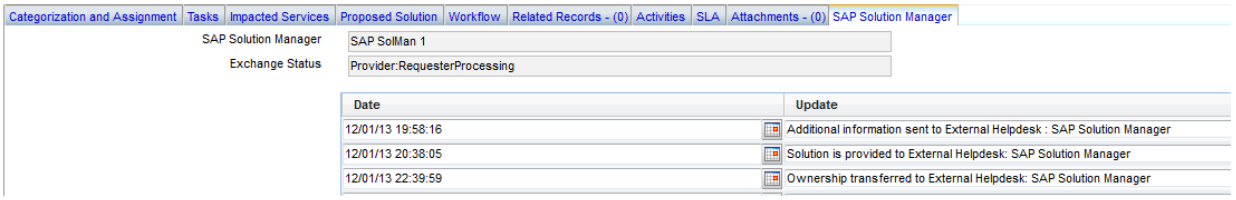


5. Extend **Activities** tab to view the comments when SAP rejects the solution.



6. Click **Send Back** to send the incident back to Solution Manager.
7. Click **Cancel** to release the incident in Service Manager.
8. After a few minutes, open the incident again and extend the **Sap Solution Manager** tab to view the incident's status from Hidden Metadata.

   The Hidden Metadata field displays **Provider: RequesterProcessing**, which means Solution Manager is processing the incident. Then Service Manager transfers the incident's ownership to Solution Manager.

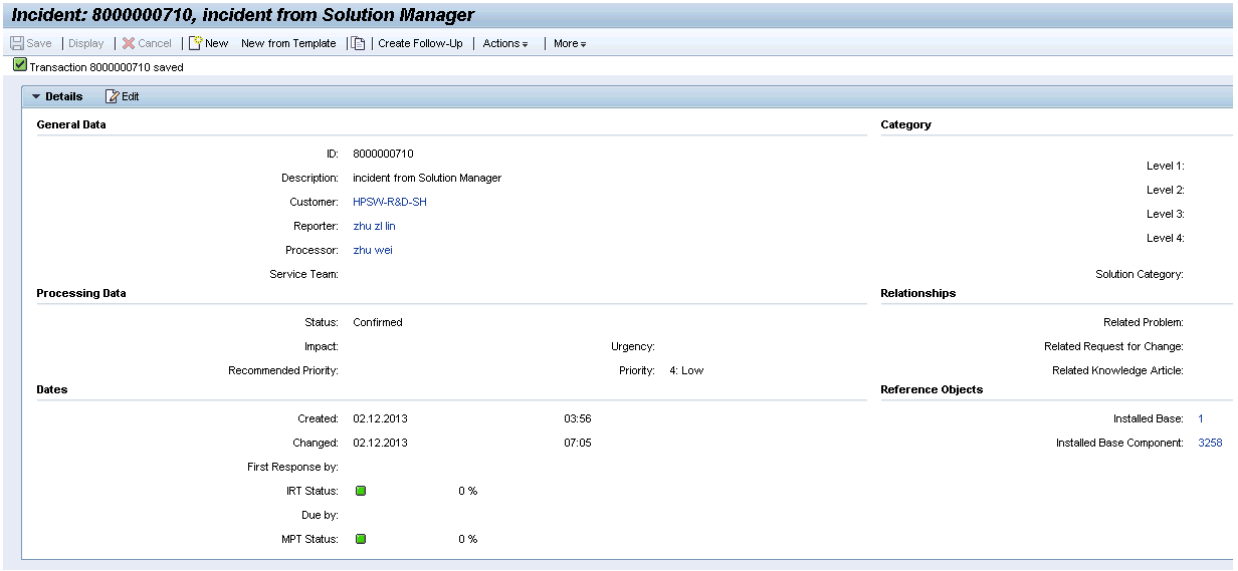


9. Click **Cancel** to close the incident window.

1.3.2.4.3.6. Close the incident in Solution Manager

1. Search the incident and open it.
2. Click **Edit** to switch the incident to edit mode.
3. Change the status to **Proposed Solution**.

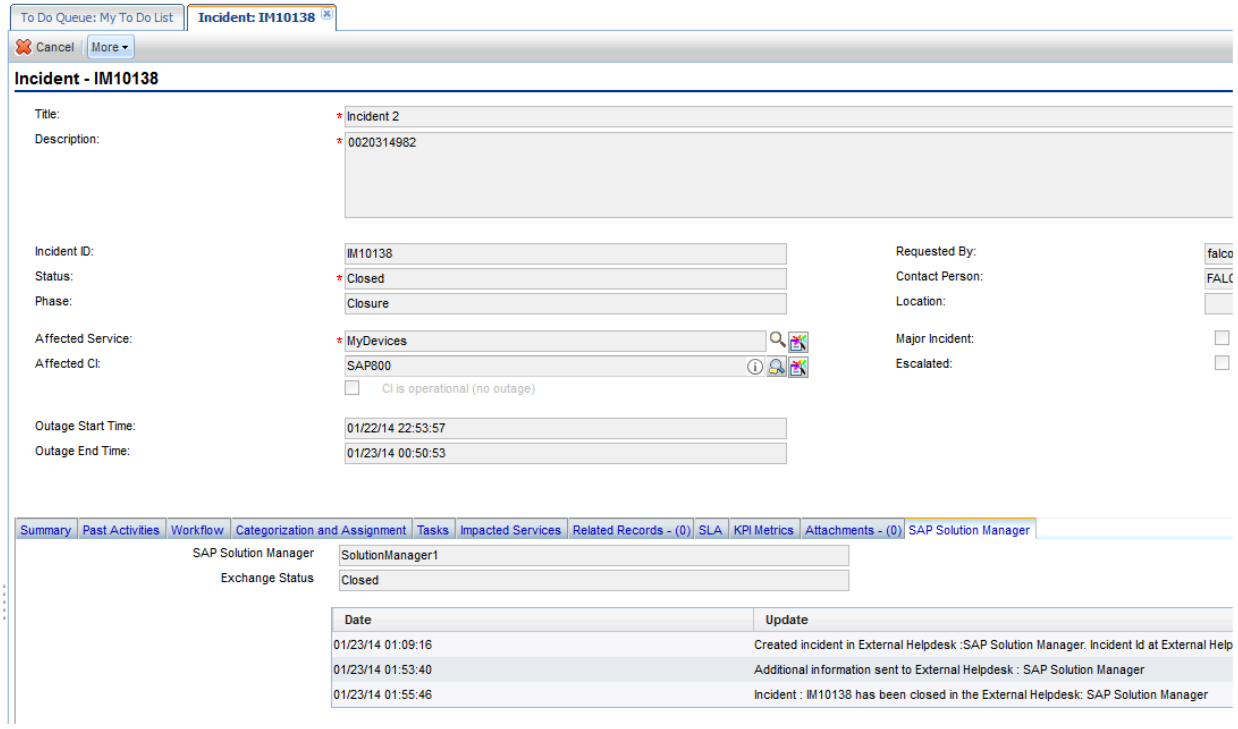
   The incident's status must be change to **Proposed Solution** or **Customer Action** before the incident is changed to **Confirmed**.
4. Change the status to **Confirmed**.
5. Click **Save** to close the incident.



1.3.2.4.3.7. Check the incident's status in Service Manager

Follow these steps:

1. Sign in to Service Manager as **Incident.Manager**.
2. Click **Incident Management** > **Incident Queue**. Find the incident from Solution Manager in the Incident Queue form.
3. Open the incident and check the updated information from Solution Manager.



The Status field is **Closed** and all buttons about SAP aren't available.

1.4. Project and Portfolio Management Center (PPM)

The SM-PPM Center Project Proposal Integration Solution aims to submit project proposals from Service Manager to PPM Center, and send the proposal's status (rejected or approved) from PPM Center to Service Manager.

This section includes the following topics.

Configuring Service Manager

Configuring PPM Center

Upgrading SM Service Catalog-PPM Center Project Proposal Integration Solution

Customizing SM Project Proposal Catalog Item Fields

1.4.1. Configuring Service Manager

This chapter provides configuration information on the Service Manager side. You need to set up an integration instance based on the SM-PPM integration template, and then add the Service Manager Web tier server URL to the system information record.

For the supported Service Manager versions, refer to the **Support Matrix** section on **Service Catalog to Project Proposal (SM - PPM)**.

1.4.1.1. Add an integration instance

To add a new integration instance for the SM Service Catalog-PPM Project Proposal Integration Solution, follow these steps:

1. Sign in to Service Manager as a System Administrator.
2. Go to **Tailoring** > **Integration Manager.**
3. Click **Add**. The Integration Template Selection page displays.
4. Select **SMPPM** from the **Integration Template** list, and then select the **Import Mapping** checkbox.
5. Click **Next**. The Integration Instance Information page displays.
6. In the Integration Instance Information page, some fields are already populated with default settings provided with the template. Review the values and make changes as necessary. Provide values for other required fields, including Interval Time, Max Retry Times, SM Server, Endpoint Server, Log Level, and Log File Directory.

   Note

   The Log File Directory must exist on the Service Manager server. Otherwise, the log files will get lost.

7. Click **Next**. The Integration Instance Parameters page displays.
8. In the **General Parameters** and **Secure Parameters** tabs, modify parameter values as described in the table below:

Parameter	Value
user	A PPM Center user account for Service Manager to connect to PPM Center, for example, **admin**.
PPMServerURL	`http://<PPM_server>:<port>` For example, `http://example.domain.com:8080`.
PPMRequestType	PPM Request Type, for example, PFM–Proposal.
password	Password of the PPM Center user account for Service Manager to connect to PPM Center.

9. Click **Next**. The Integration Instance Fields page displays.
10. In the Integration Instance Fields page, you can see pre-defined SM Fields and PPM Center (Endpoint) Fields provided with the integration instance template. You can modify or delete these fields or add new fields as necessary.
11. Click **Next**. The Integration Instance Mapping page displays.
12. In the Integration Field Mapping page, you can see pre-defined mappings between SM Fields and PPM Center (Endpoint) Fields.

    You can modify the pre-defined field mappings and value mappings between SM fields and PPM Center fields. You can also add your own field mappings and value mappings.

    For detailed instructions on configuring field mappings and value mappings, seeService Manager Integration Suite (SMIS).
13. Click **Finish**. The new integration instance is added.
14. With the new integration instance highlighted, click **Enable** to enable or disable the new SM Service Catalog-PPM Center Project Proposal Integration Solution instance.

1.4.1.2. Configure the Service Manager Web tier server URL

Follow these steps:

1. Sign in to Service Manager as a System Administrator.
2. Go to **System Administration** > **Base System Configuration** > **Miscellaneous** > **System Information Record**.
3. Click the **Active Integrations** tab.
4. In the **WebServer URL** field, enter the URL of the Service Manager Web tier application server. For example, http://16.187.191.84:8080/webtier980/index.do.

   Note

   Do not enter localhost as the Web tier server address even if the Web tier application server is on the same host machine as the Service Manager server is on. The Web tier server address must be the real hostname or IP address of the host machine.

5. Click **Save**.
6. Sign out and sign in to Service Manager again.

1.4.2. Configuring PPM Center

This chapter provides configuration information on the PPM Center side, including configuration of the PPM Workbench and the PPM Server.

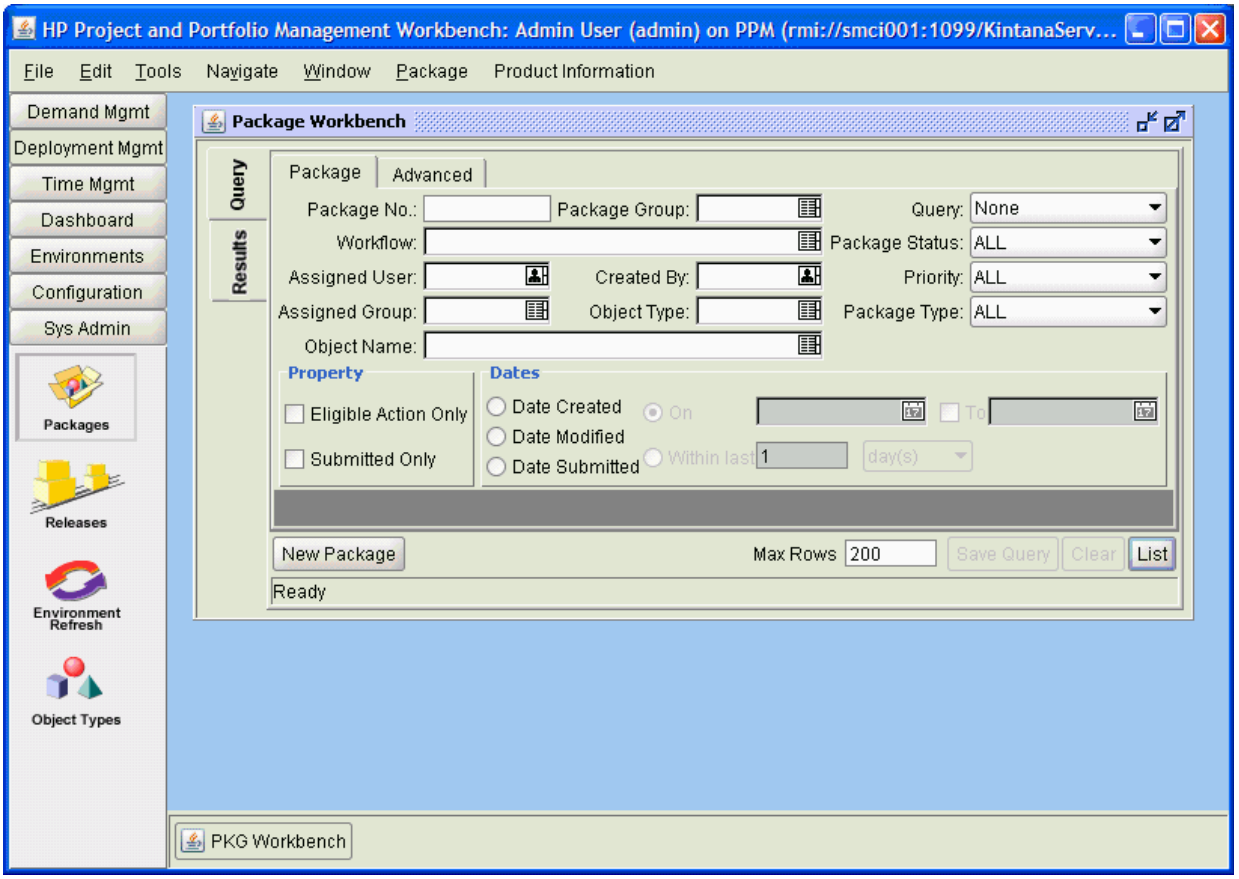
For the supported PPM Center versions, refer to the **Support Matrix** section on **Service Catalog to Project Proposal (SM - PPM)**.

If you encounter any problems when synchronizing the requests from PPM to SM, you can navigate to <PPM_HOME>\logs and then check the logs starting with `REQ_`. For example, <PPM_HOME>/logs/REQ_30244.

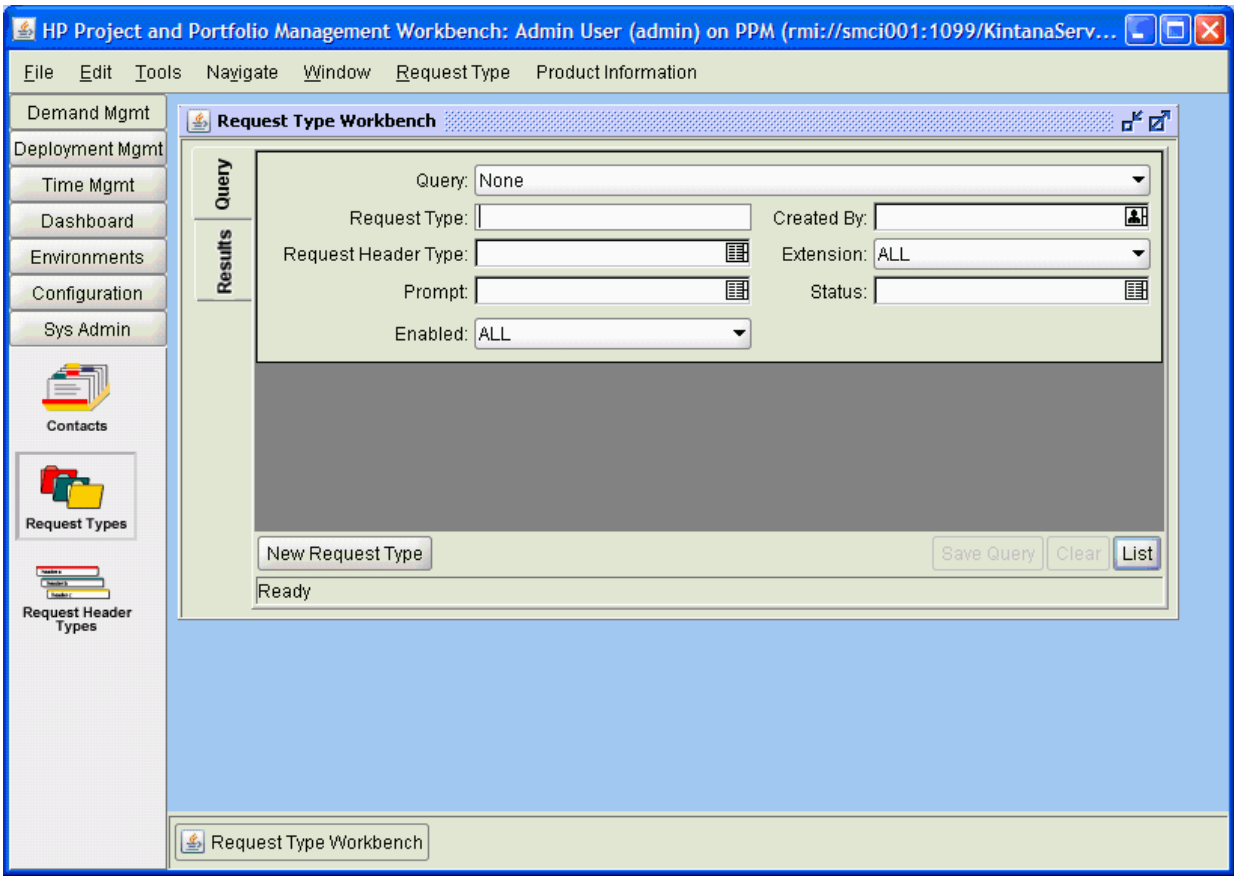
1.4.2.1. Configuring the PPM Workbench

Perform the following steps to configure the PPM Workbench:

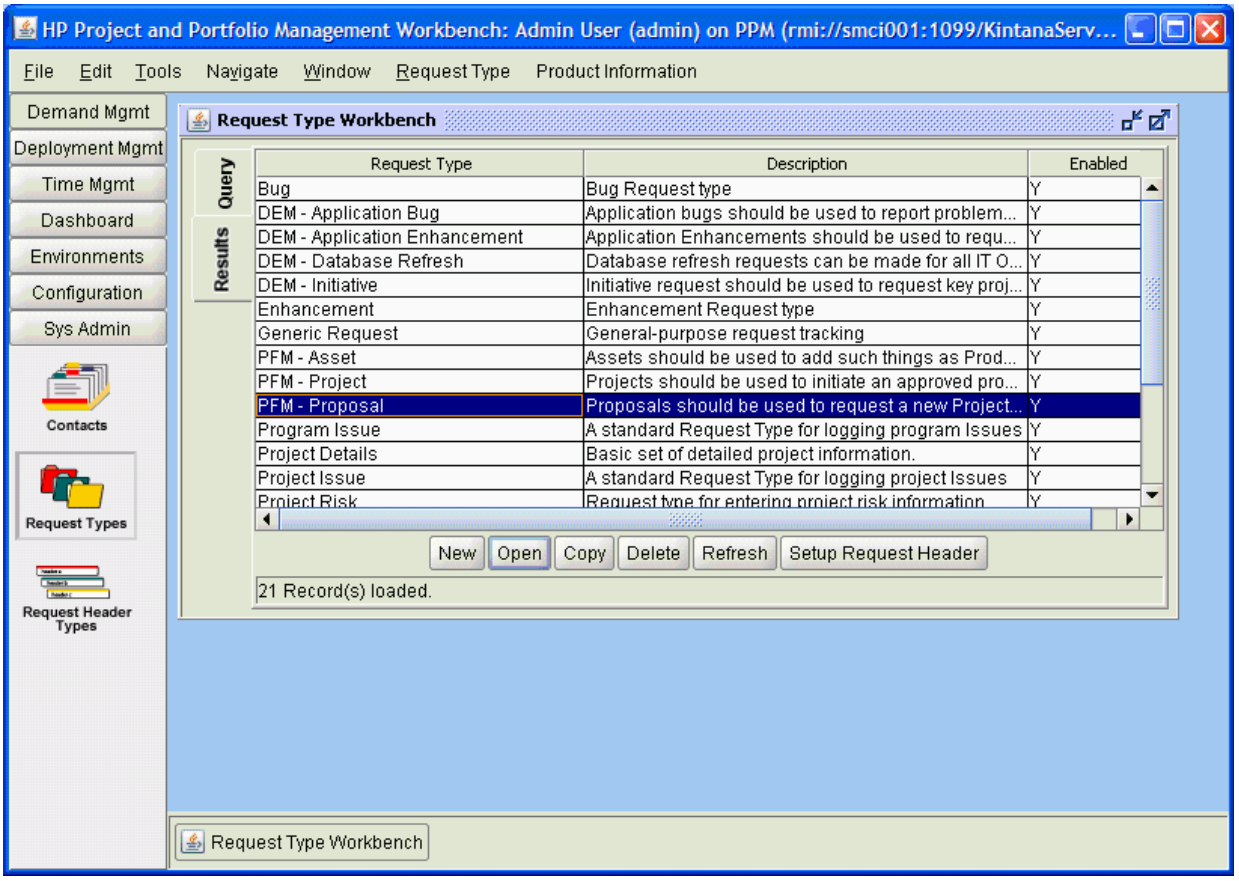
1. Sign in to PPM Center as an administrator.
2. In the navigation pane, go to **Administration** > **Open Workbench**. The Project and Portfolio Management Workbench application launches.



3. In the left navigation pane, click **Demand Mgmt** > **Requets Types**. The Request Type Workbench window opens.

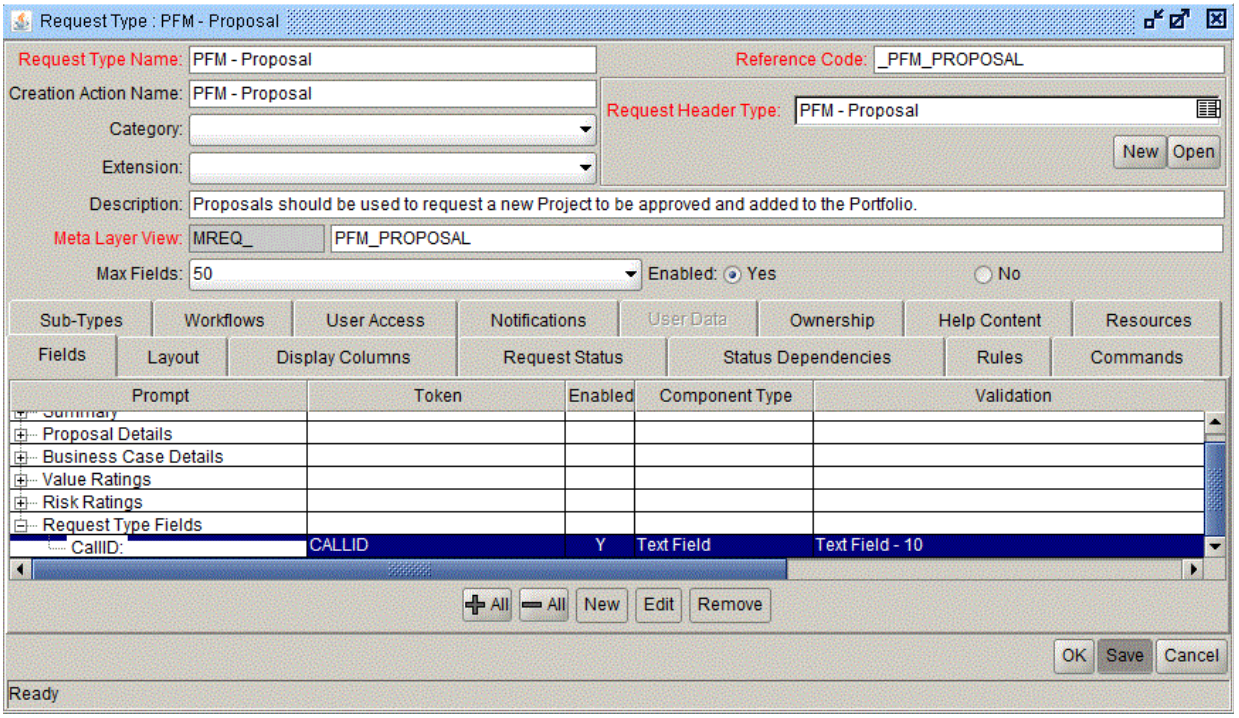


4. Click **List** to list all request types in PPM Center.

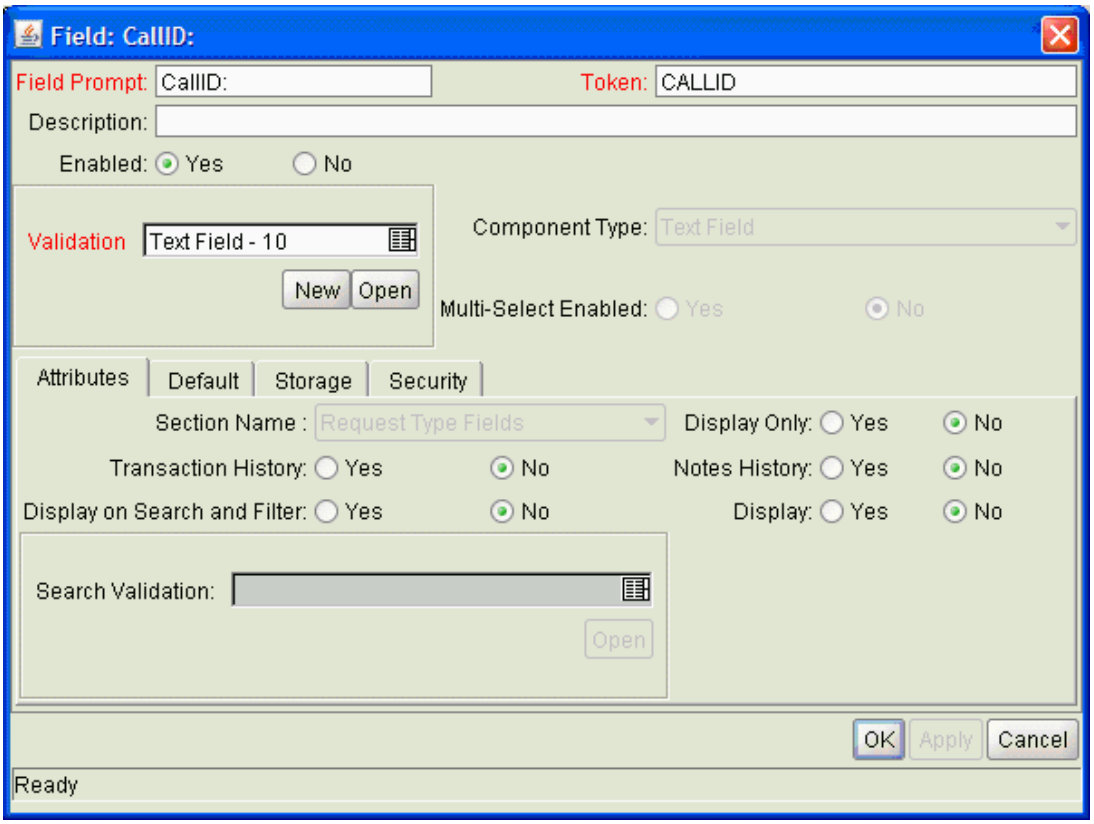


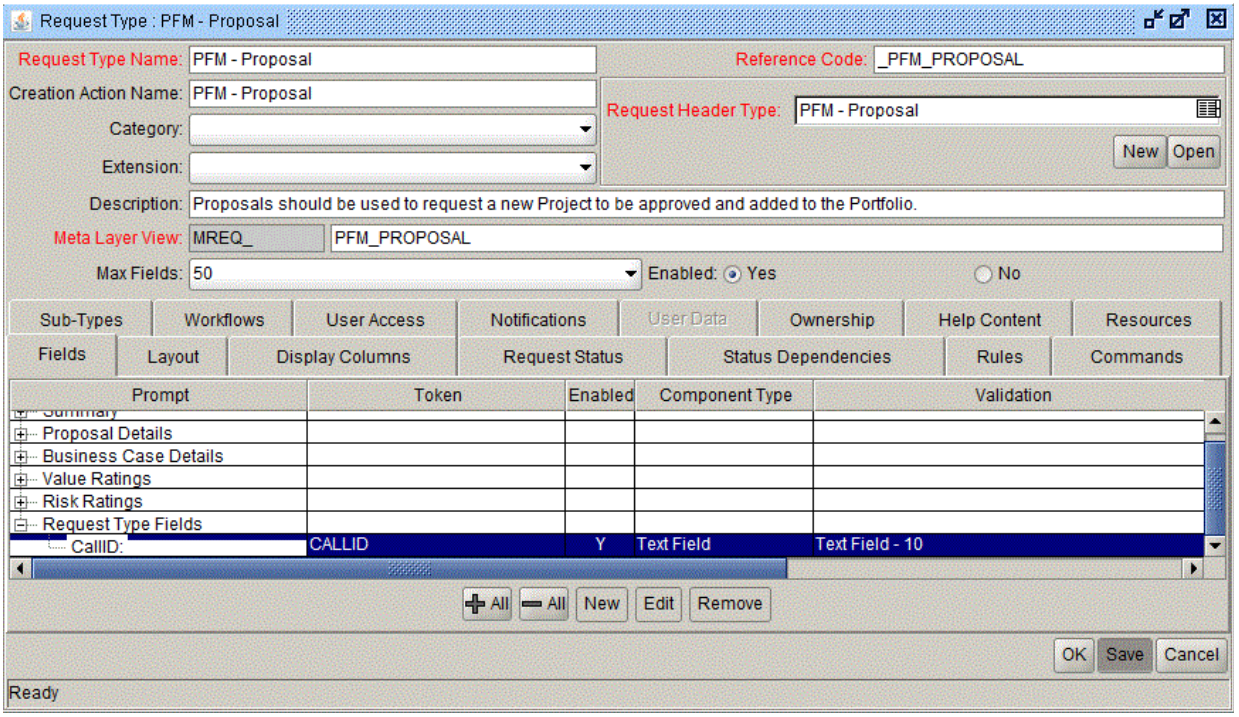
5. Double-click the **PFM–Proposal** entry, or select it and then click **Open**.

> **Note**
>
> If you can't find the PFM-Proposal entry, contact your PPM administrator for best practices of installation.

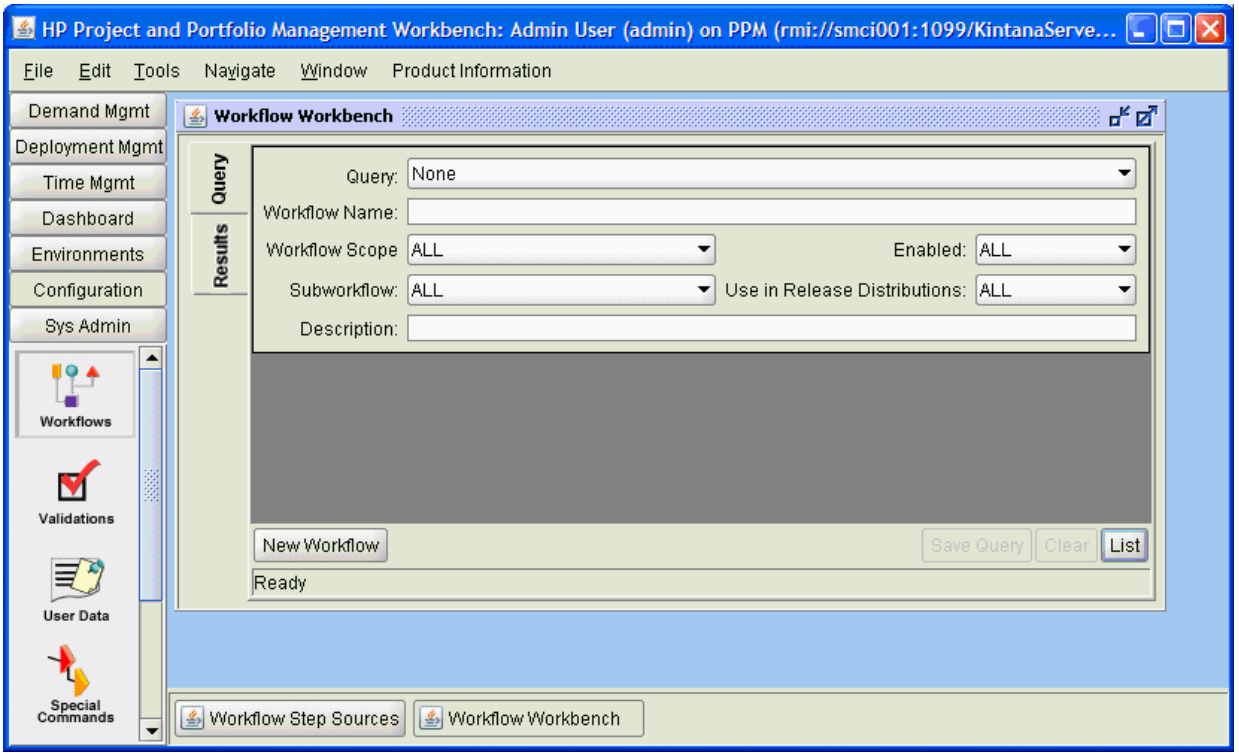


6. Go to the **Fields** tab and click **New**. Add a new field **CallID:** for the PFM–Proposal Request Type, and provide values for required fields as shown below.

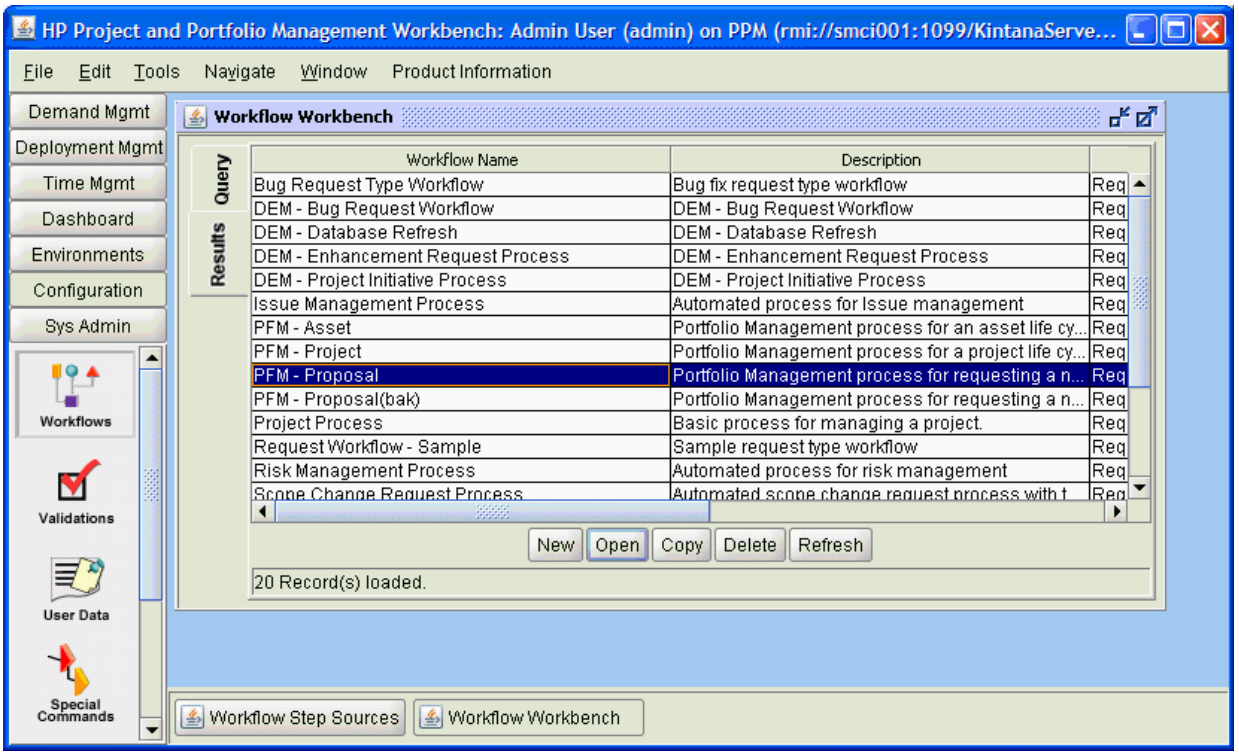


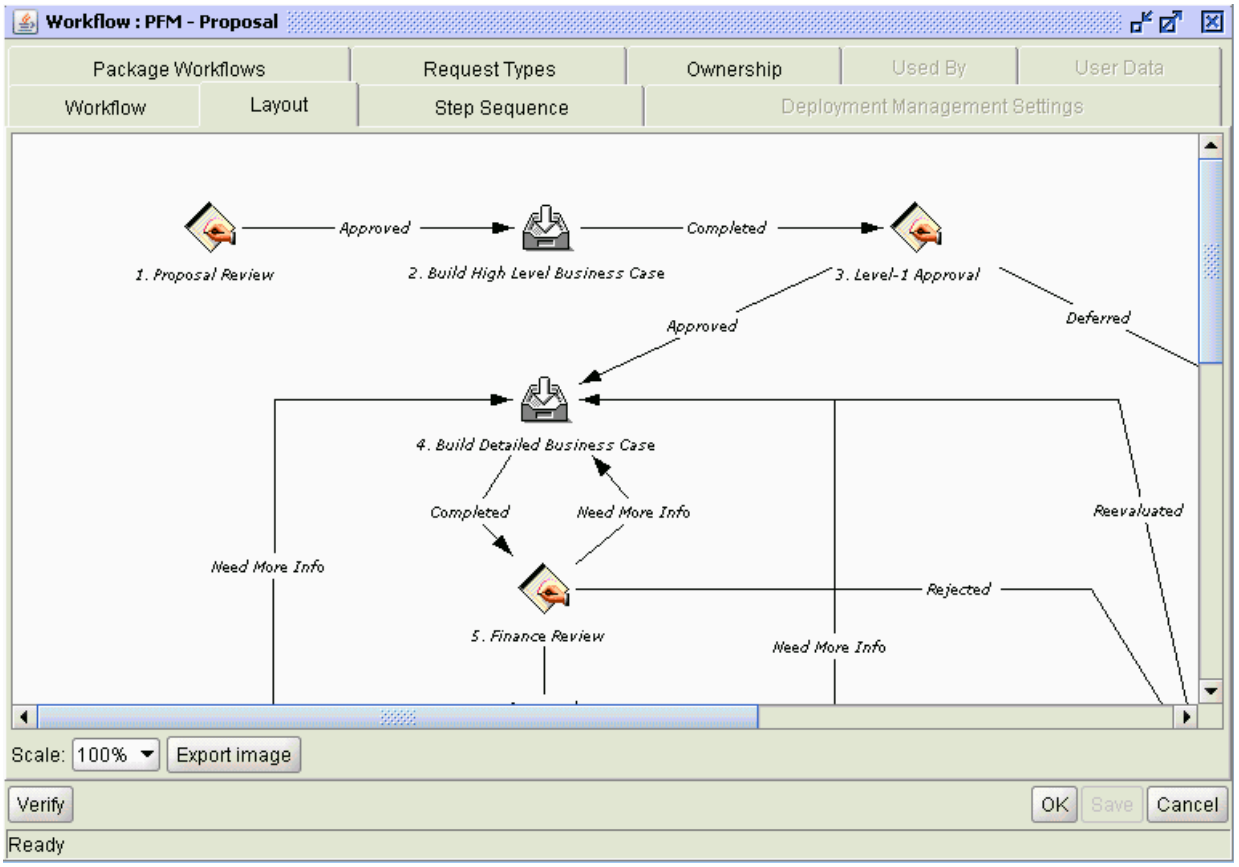


7. Click **OK** to save the modification.
8. In the left navigation pane, click **Configuration** > **Workflows** to launch the Workflow Workbench window.

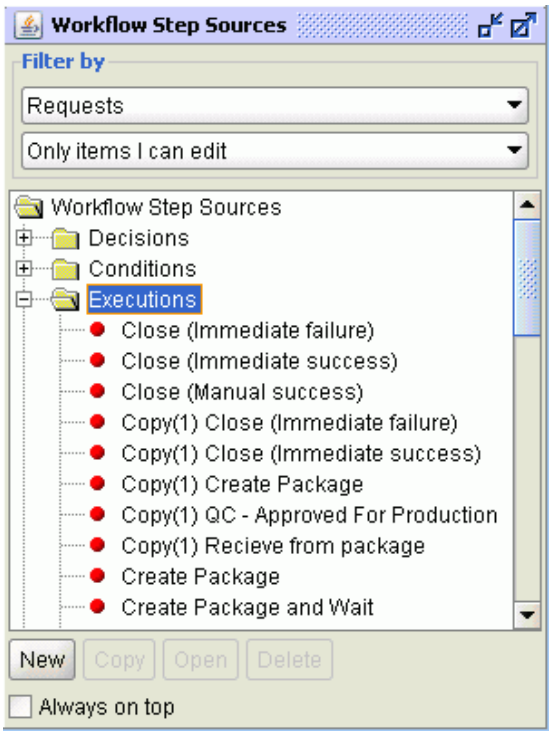


9. Click **List** to list all workflows. Locate the **PFM–Proposal** entry from the list, and double-click on it or click **Open**.

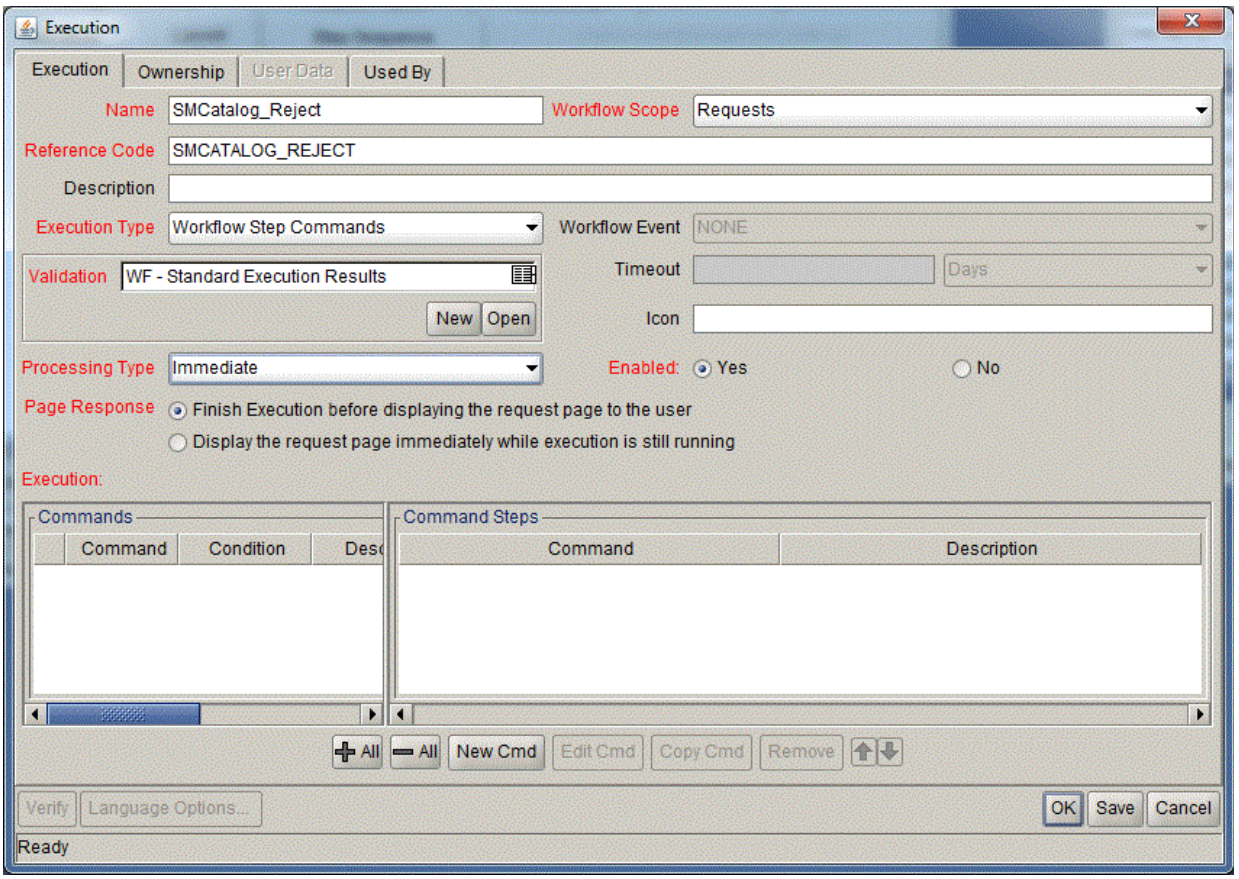




10. In the Workflow Step Sources sub-window, select the **Executions** node in the Workflow Step Sources item tree.



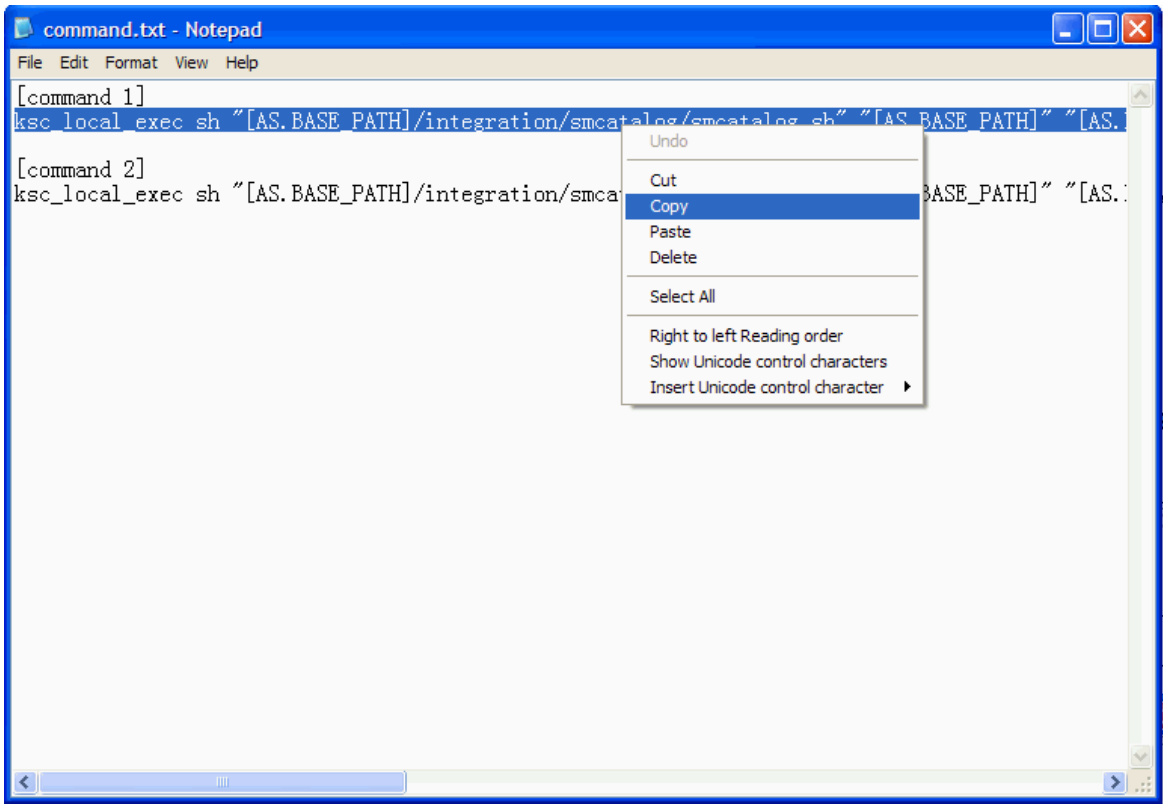
11. Click **New** to create a new workflow step source SMCatalog_Reject.

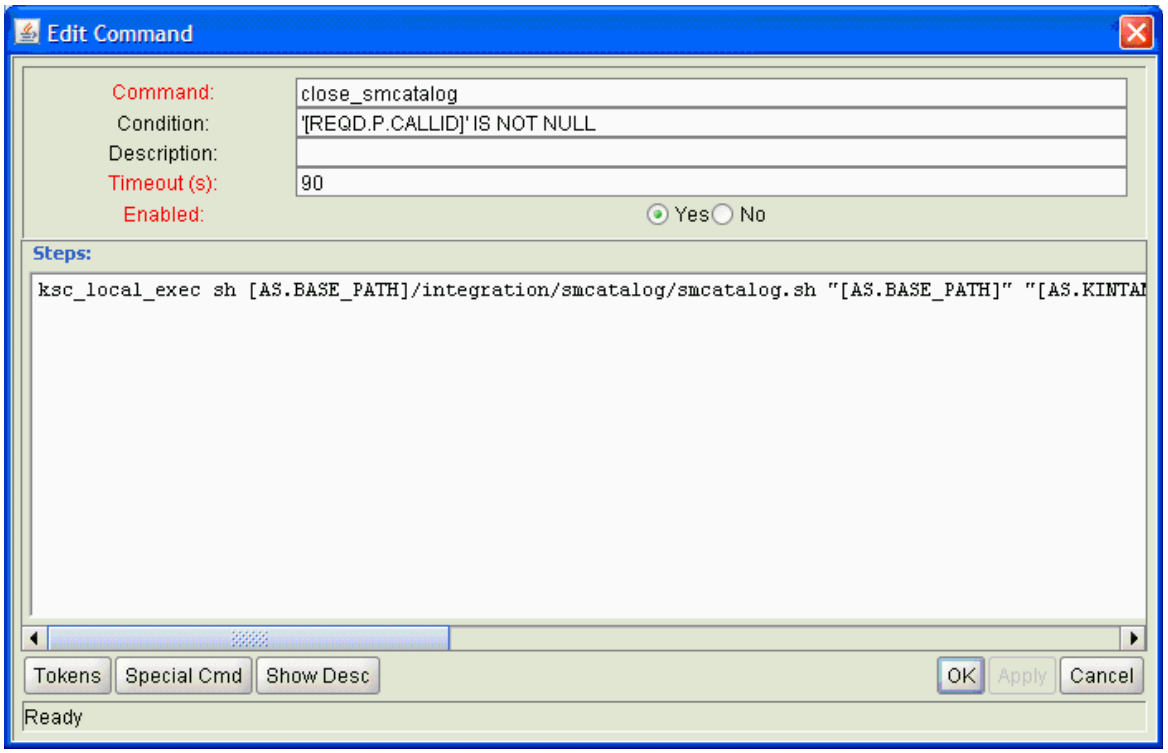


12. Click **New Cmd** to add a new command, and provide values for the corresponding fields:
    - **Command**: close_smcatalog
    - **Condition**: '[REQD.P.CALLID]' IS NOT NULL

- **Command Steps**: Locate the script with the `[command 1]` marker in `command.txt`, then copy and paste the entire section of script directly to this field.

> **Note**
>
> To facilitate your configuration effort, all scripts that may cause errors during your copy and paste operation are provided in `command.txt` under the `<SMPPMIntegration_installDir>\doc` directory.

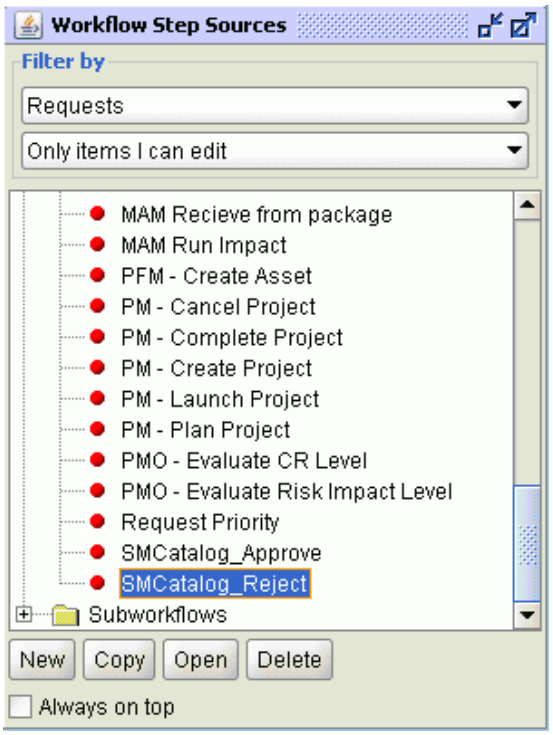




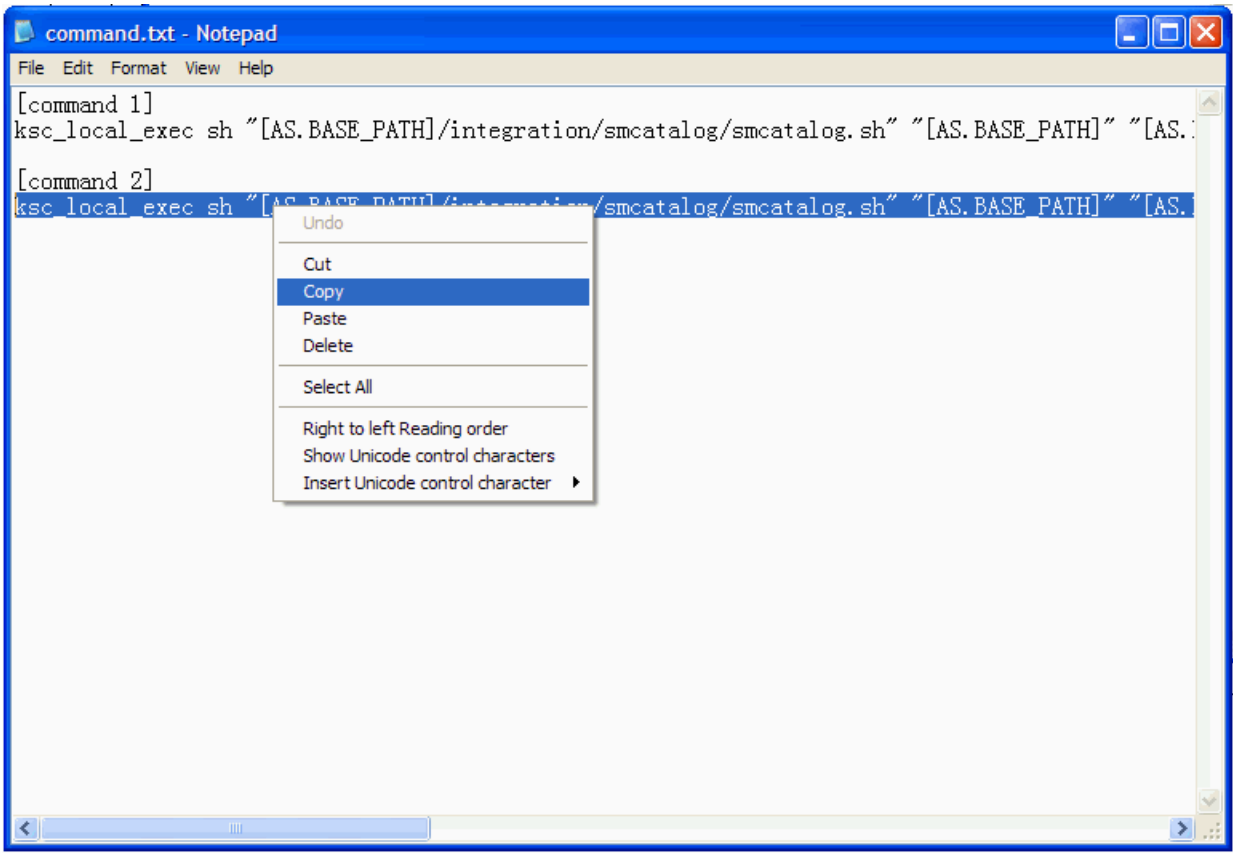
The following is the detailed script (`[command 1]` in `command.txt`):

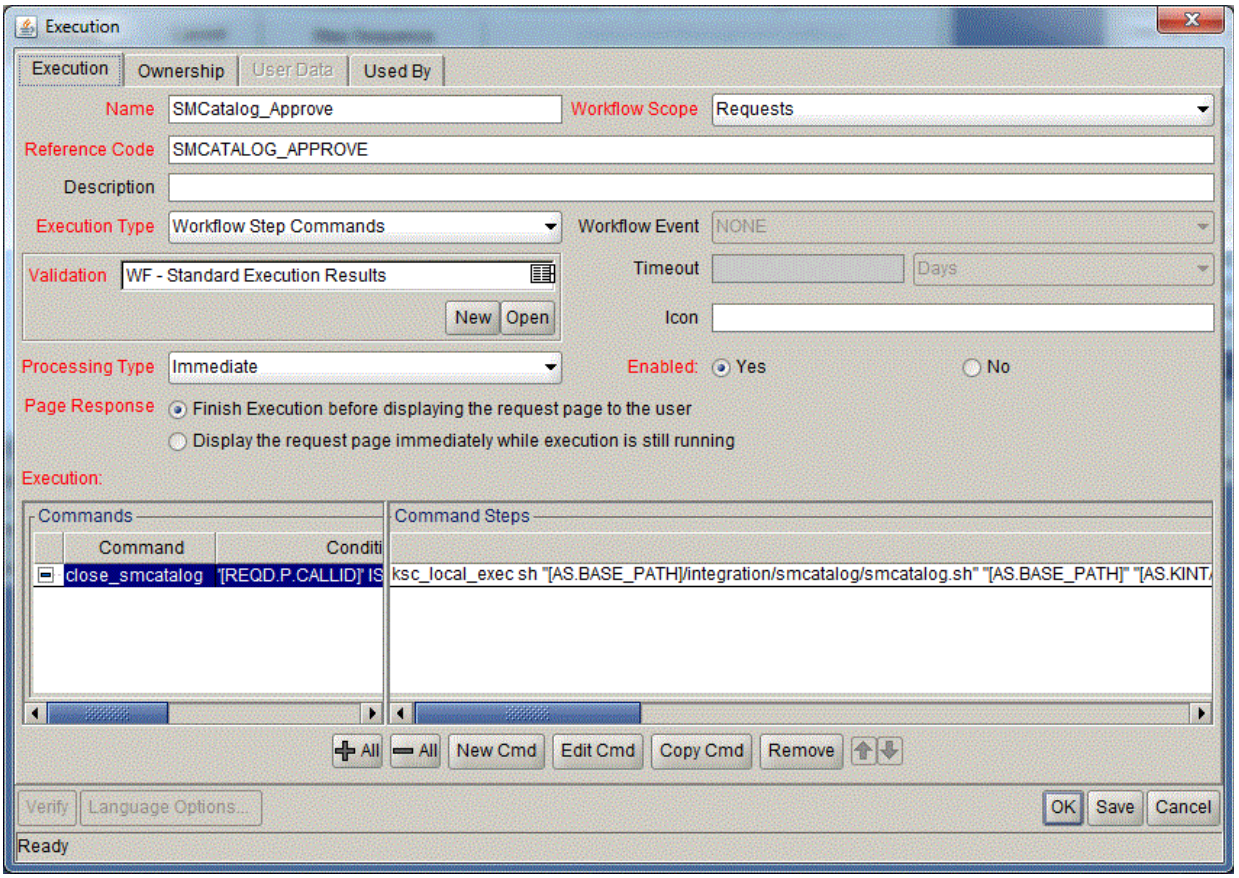
```
ksc_local_exec sh "
[AS.BASE_PATH]/integration/smcatalog/smcatalog.sh"
"[AS.BASE_PATH]" "[AS.KINTANA_SERVER_NAME]"
"[AS.SM_URL]/sc62server/PWS/ServiceDeskForPPM"
"[AS.SM_USERNAME]" "[AS.SM_PASSWORD]"
"[REQD.P.CALLID]" "Request Rejected"
"Proposal has been rejected."
```

13. In the Workflow Step Sources sub-window, select the newly added `SMCatalog_Reject` item.



14. Click **Copy** to create a new workflow step source `SMCatalog_Approve`. Locate the script with the `[command 2]` marker in `command.txt`, then copy and paste the entire section of script directly to the **Command Steps** field.

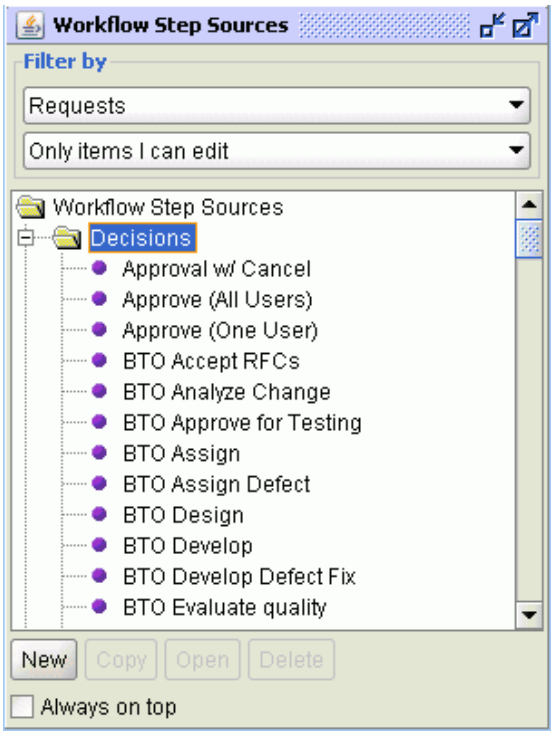




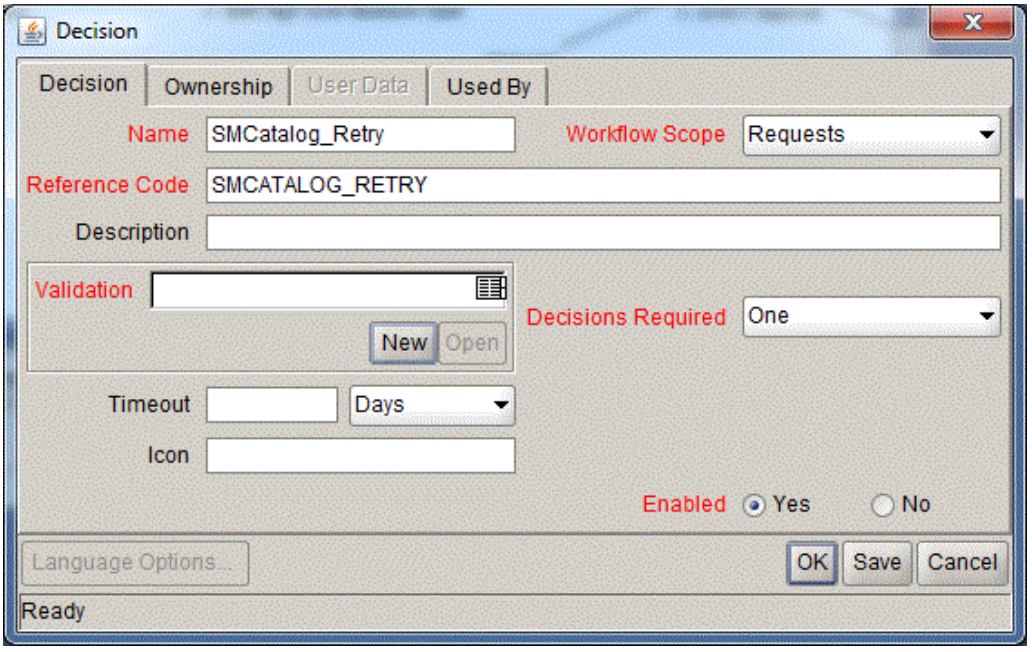
The following is the detailed script ([command 2] in command.txt):

```
ksc_local_exec sh "
[AS.BASE_PATH]/integration/smcatalog/smcatalog.sh"
"[AS.BASE_PATH]" "[AS.KINTANA_SERVER_NAME]"
"[AS.SM_URL]/sc62server/PWS/ServiceDeskForPPM"
"[AS.SM_USERNAME]" "[AS.SM_PASSWORD]"
"[REQD.P.CALLID]" "Proposal approved"
"Proposal has been approved."
```

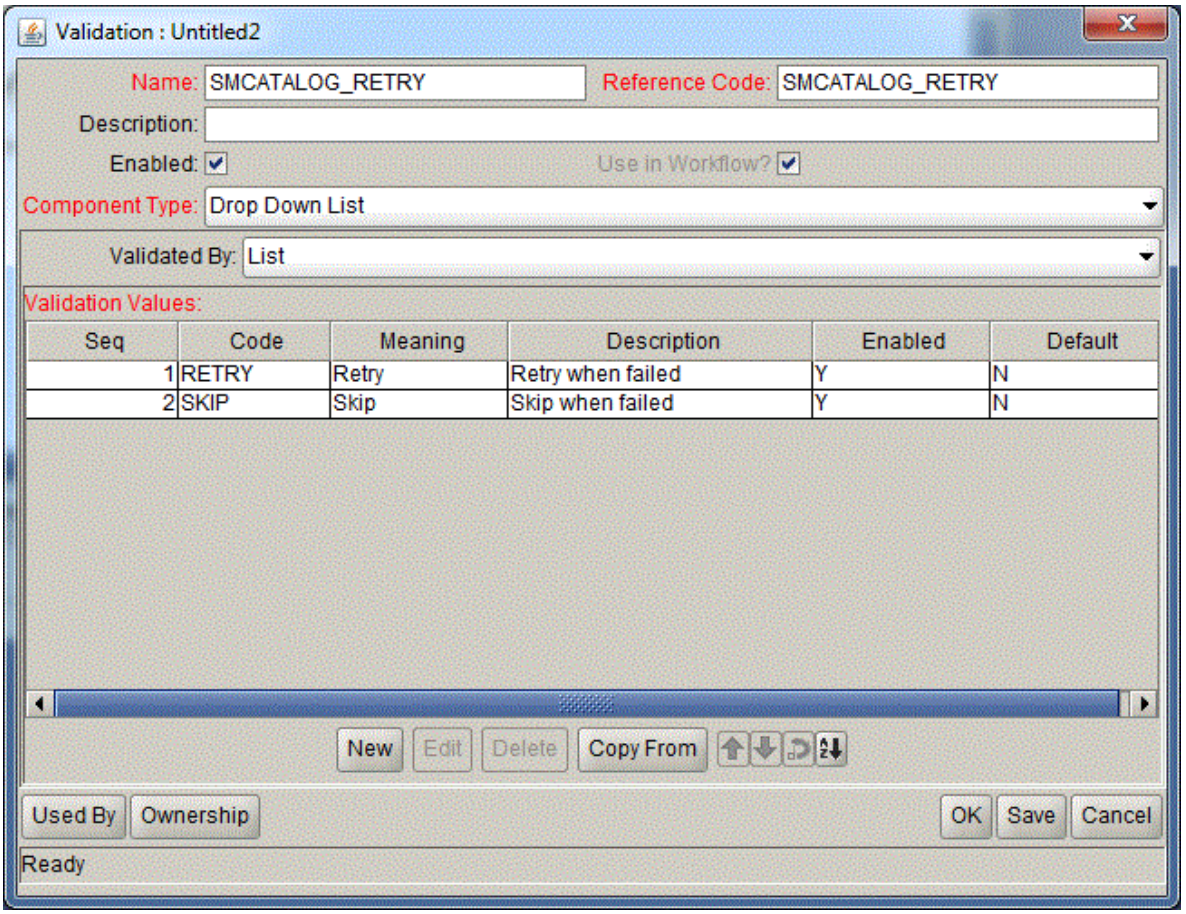
15. In the Workflow Step Sources sub-window, select the **Decisions** node.



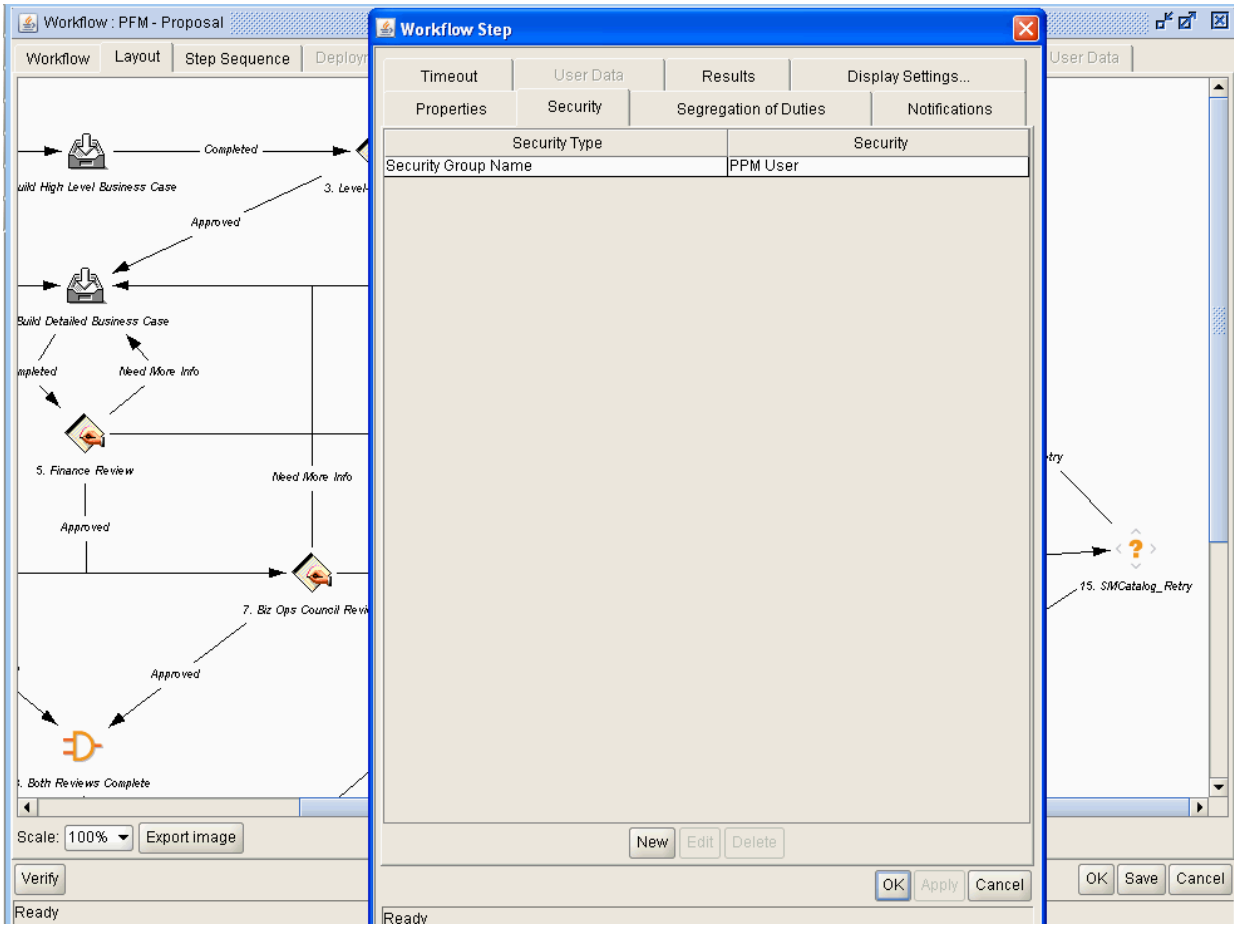
16. Click **New**. In the Decision dialogue box, create a new workflow step source SMCatalog_Retry.



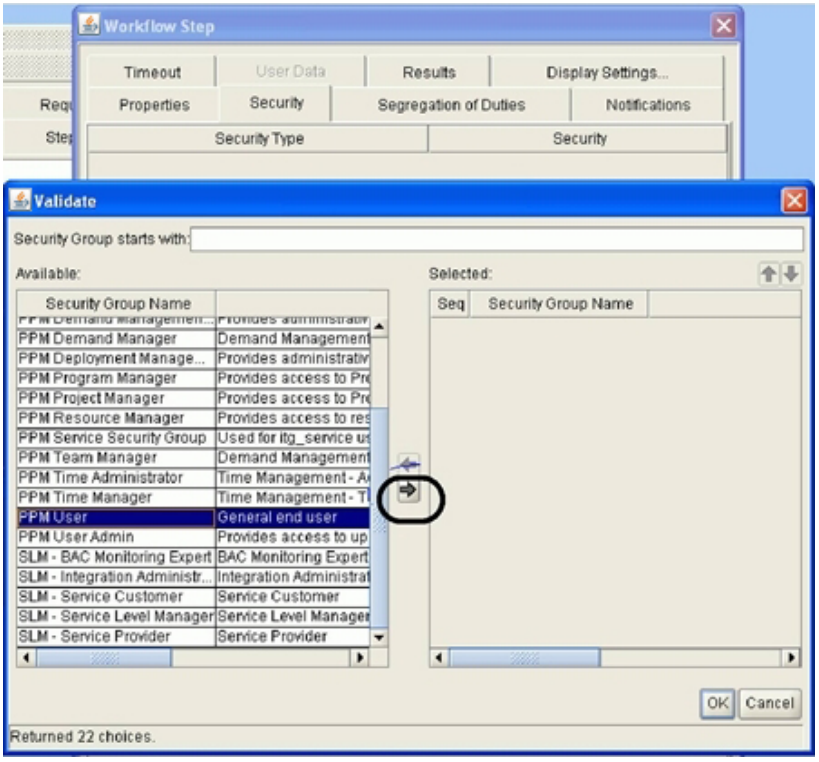
17. For the Validation field, click **New** to create a new validation SMCatalog_Retry for the SMCatalog_Retry workflow step resource, and click **New** to add two entries in Validation Values as shown below.



18. Click **OK** to close the Validation dialog.
19. Click **OK** to close the Decision dialog.
20. In the PFM–Proposal sub-window, drag-and-drop the newly added workflow step resources SMCatalog_Reject, SMCatalog_Approve, and SMCatalog_Retry from the Workflow Step Sources sub-window to the PFM–Proposal workflow. The Workflow Step window pops up.

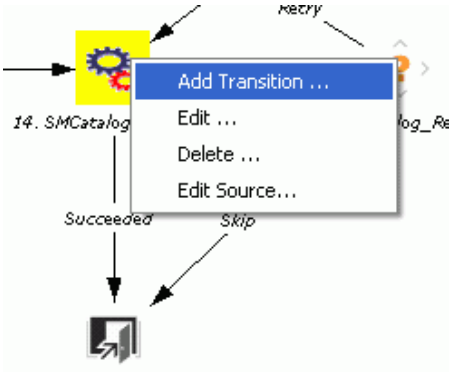


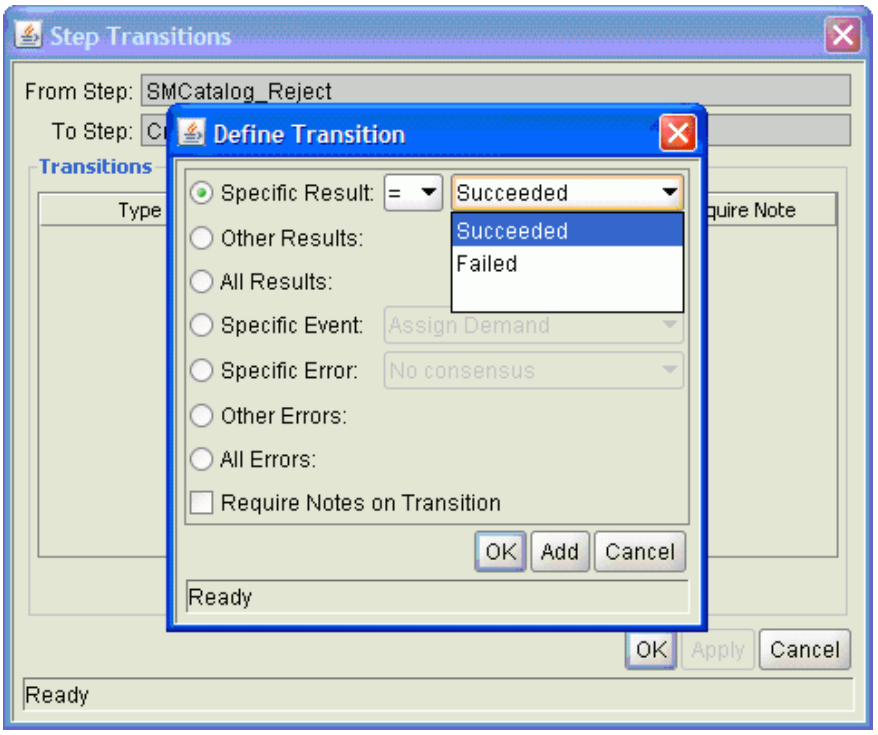
21. Click **Security** and then click **New**. For the Security Group field, select **PPM User**from the pop-up window and click the right arrow to add **PPM User** to Security Group.



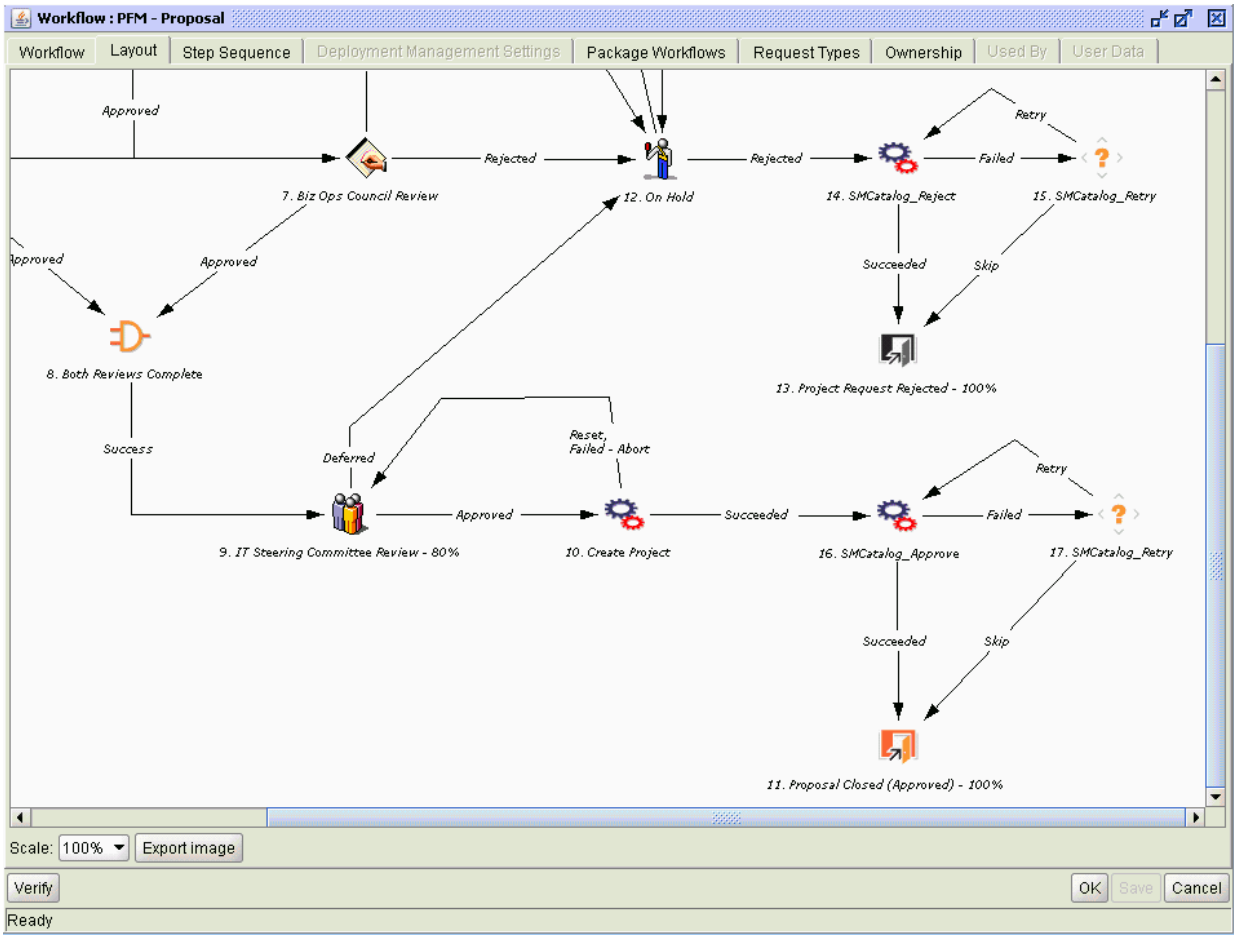
22. Click **OK** in the Validate and Workflow Step windows. PPM User is added to Security Group.

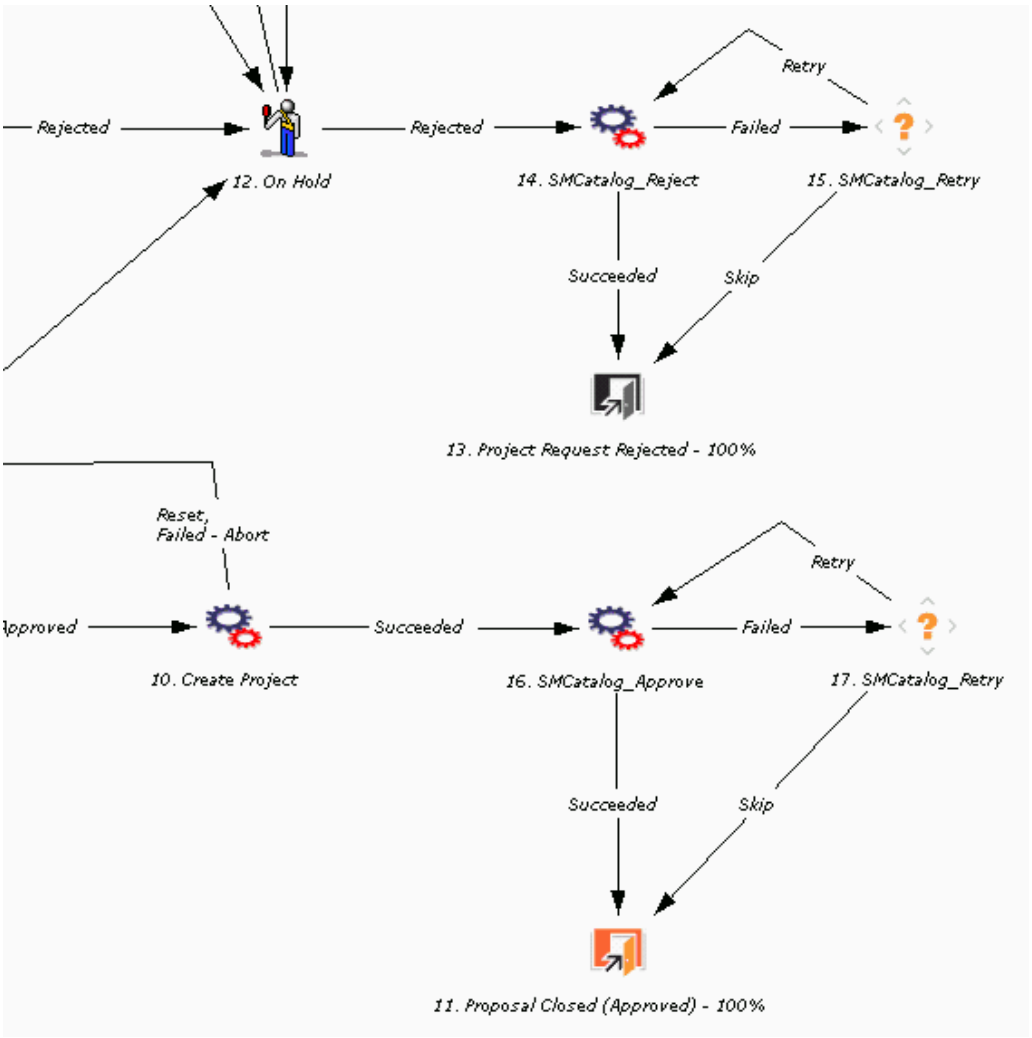
23. To add transitions between workflow steps, right-click a step, and select **Add Transition**from the menu.





24. After adding all the steps in the PFM–Proposal workflow, click **Save** to save the modification to the workflow. The configuration of the PPM Workbench is finished.


Workflow : PFM - Proposal
Workflow
Layout
Step Sequence
Deployment Management Settings
Package Workflows
Request Types
Ownership
Used By
User Data
Approved
7. Biz Ops Council Review
Rejected
12. On Hold
Rejected
14. SMCatalog_Reject
Failed
15. SMCatalog_Retry
Retry
Approved
Approved
8. Both Reviews Complete
Succeeded
Skip
13. Project Request Rejected - 100%
Success
Deferred
Reset,
Failed - Abort
9. IT Steering Committee Review - 80%
Approved
10. Create Project
Succeeded
16. SMCatalog_Approve
Failed
17. SMCatalog_Retry
Retry
Succeeded
Skip
11. Proposal Closed (Approved) - 100%
Scale: 100%
Export image
Verify
OK
Save
Cancel
Ready


Rejected
Rejected
Retry
Failed
12. On Hold
14. SMCatalog_Reject
15. SMCatalog_Retry
Succeeded
Skip
13. Project Request Rejected - 100%
Reset,
Failed - Abort
Approved
Succeeded
Retry
Failed
10. Create Project
16. SMCatalog_Approve
17. SMCatalog_Retry
Succeeded
Skip
11. Proposal Closed (Approved) - 100%

1.4.2.2. Configuring the PPM Server

Note

Be sure to complete the following preliminary tasks before configuring the PPM server:

1. Install AXIS2 (axis2-1.7.4) and ANT (apache-ant-1.9.7).
2. Configure `AXIS2_HOME`, `ANT_HOME`, and add `$AXIS2_HOME/bin`, `$ANT_HOME/bin` to PATH.
3. Install JDK and set JAVA_HOME to the location where the JDK is installed.
4. Make sure both the SM-PPM adapter and the PPM server use the same JAVA_HOME. Otherwise, the status synchronization from PPM to SM will fail with the following error message:

   `Exception in thread "main" java.lang.UnsupportedClassVersionError: com/hp/smppm/ws/CloseSMInteraction : Unsupported major.minor version 52.0`

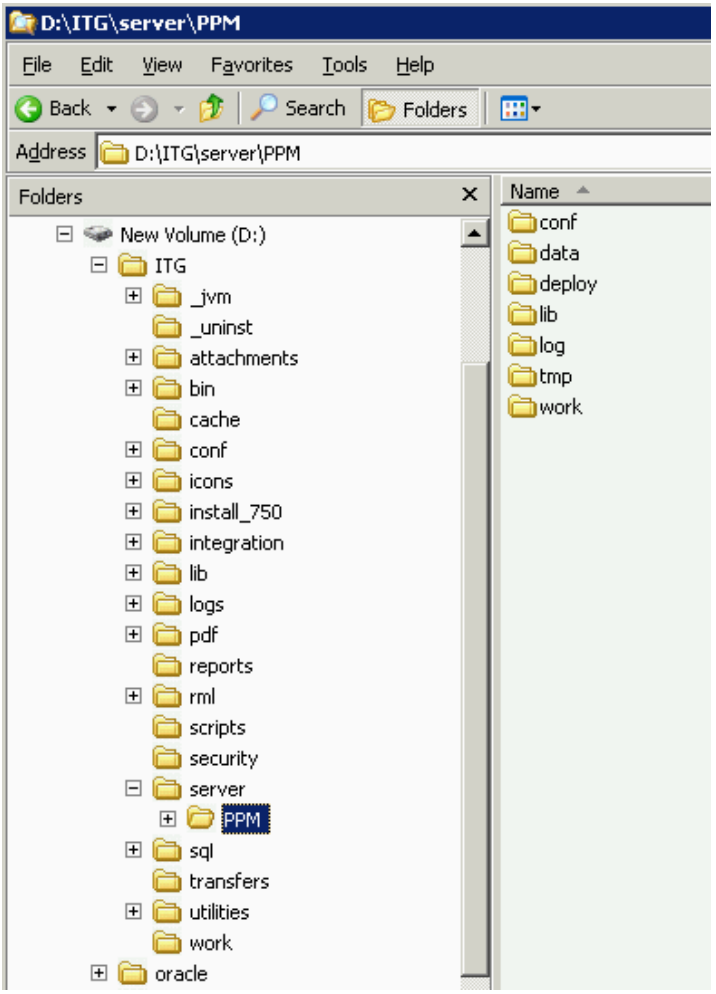
Perform the following steps to configure the PPM server:

1. Create a folder `smcatalog` under `<PPM_HOME>\integration`. For example, `C:\ITG\integration\smcatalog`.
2. Navigate to the <Release_Package>\SMPPM\PPM_configuration directory, and then copy the stub folder, smcatalog.sh, build.xml, WebServiceStubGenerator.sh, and properties.conf to the `smcatalog` folder.
3. Configure SM Web service URL and PPM Server in the properties.conf file.
   - **Configure SM Web service URL**: Update this value to http://<SM Server Address>:<port>/sc62server/PWS/ServiceDeskForPPM.wsdl. For example, `http://smserver01:13080/sc62server/PWS/ServiceDeskForPPM.wsdl`.
   - **Enter PPM Server Name**: Enter the PPM Server name.

Note

The PPM Server name can be found under `<PPM_HOME>\server\`. If the PPM server name can't be recognized, run the following command line before this step:

```
dos2unix -o *.sh
```



4. Run the **dos2unix properties.conf** command.
5. Run the **dos2unix WebServiceStubGenerator.sh** command.
6. Run the **dos2unix smcatalog.sh** command.
7. Run the **sh WebServiceStubGenerator.sh** command to generate SM WSDL stub.
8. Open the `server.conf` file in the PPM Center installation root path (for example, `c:\ITG\server.conf`).
    1. Set the `com.kintana.core.server.ENABLE_WEB_SERVICES` to `true` to support the Web Service function in PPM Center.

    ```
    com.kintana.core.server.ENABLE_WEB_SERVICES=true
    ```

2. Add the following to the end of the `server.conf` file:

```
com.kintana.core.server.SM_URL=http://<SM server address>:
<port>
```

```
com.kintana.core.server.SM_USERNAME=<SM user account>
```

```
com.kintana.core.server.SM_PASSWORD=<SM user account password>
```

Replace the italic content in angle brackets with the actual values of the Service Manager server and user account information which will be integrated with PPM Center.

Example

```
com.kintana.core.server.SM_URL=http://tvm001:13080
```

```
com.kintana.core.server.SM_USERNAME=falcon
```

```
com.kintana.core.server.SM_PASSWORD=1Qaz2wsx3edc
```



Note

Unexpected errors may occur if the SM user account password is blank.

9. Run the **./kStop.sh -now -user *<ppm_account>*** command, and then provide <ppm_password> as prompted. For example: **./kStop.sh -now -user admin**.

   You can run the **./kStatus.sh** command to check the status of stop process.

10. Run the **./kUpdateHtml.sh** command, and then run the **./kStart.sh** script to restart the PPM server for the configuration to take effect.

1.4.3. Upgrading SM Service Catalog-PPM Center Project Proposal Integration Solution

To upgrade the SM Service Catalog - Project and Portfolio Management (PPM) Center Project Proposal Integration Solution from v1.01 to v1.02, perform the following steps:

1. Sign in to PPM Center as an administrator.
2. Configure the PPM Workbench again. Refer to Configuring the PPM Workbench for detailed instructions.
3. Configure the PPM Server again. Refer to Configuring the PPM Server for detailed instructions.

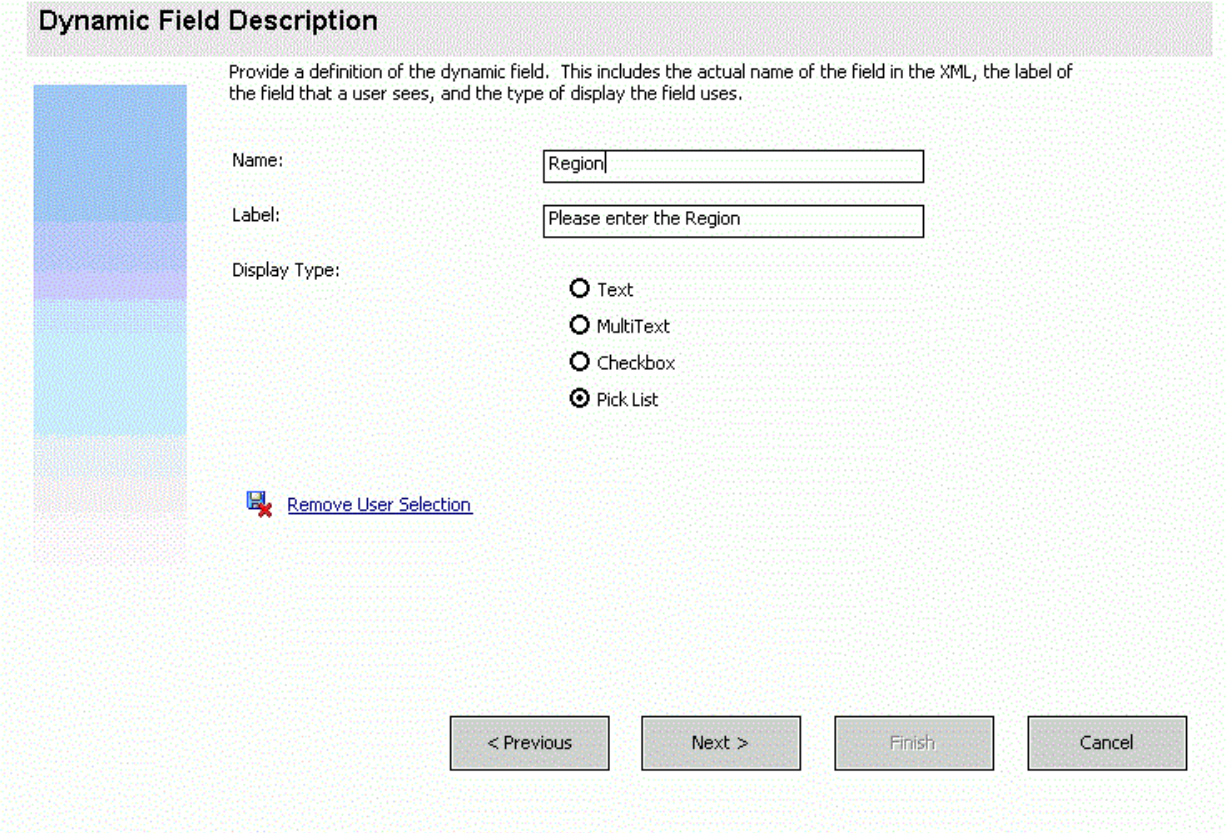
1.4.4. Customize SM Project Proposal Catalog Item Fields

Perform the following steps to customize SM Project Proposal Catalog Item fields:

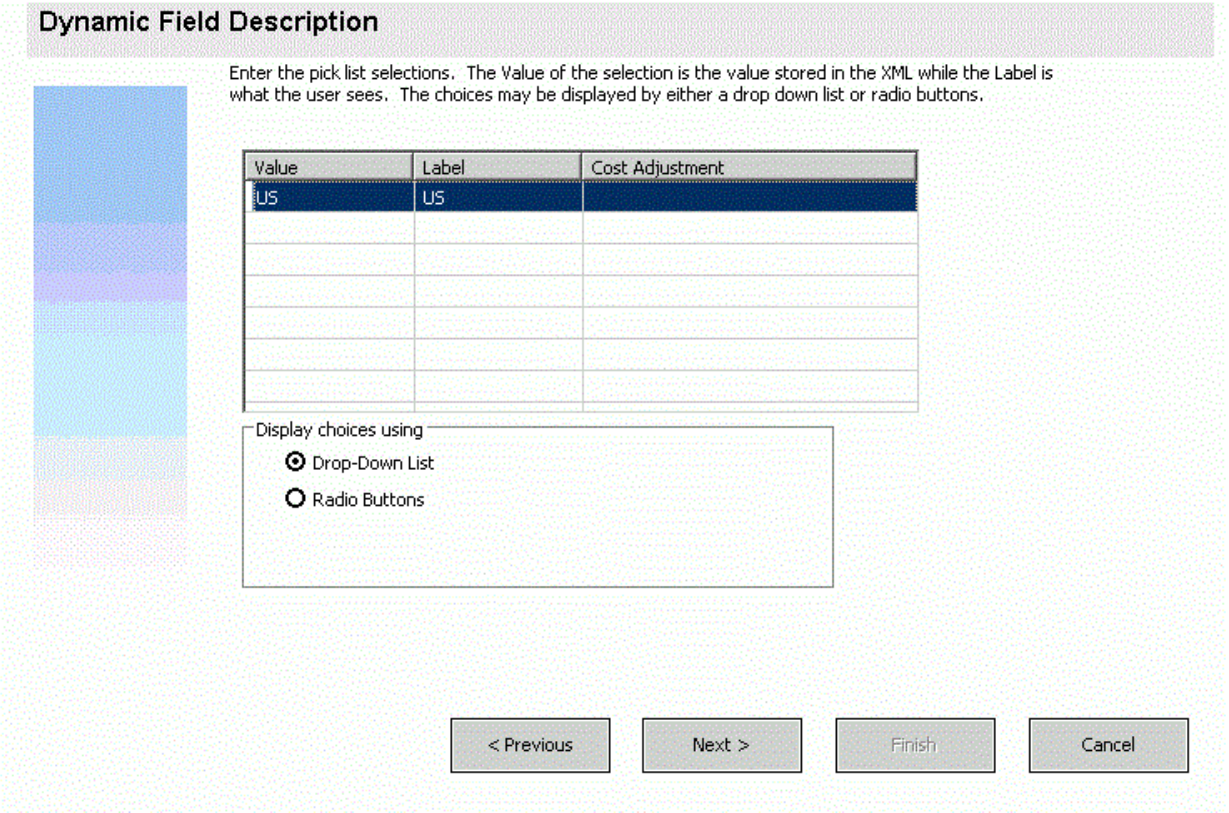
1. Sign in to Service Manager as a System Administrator.
2. Go to **Service Catalog** > **Administration** > **Manage Catalog**. The Search Catalog Item Definitions form opens.
3. Type Project Proposal in the Name field, and click **Search**. The Project Proposal Catalog Item Definition tab window displays. Click the **User Selections** tab.
4. On this page, you can click menu items on the right side of the window to edit fields of the Project Proposal Catalog Item, or simply double-click the entry you want to edit.

   For example, if you need to add a **Region** field, perform the following steps:

   a. Click **Add User Selection**, and then enter values as required.



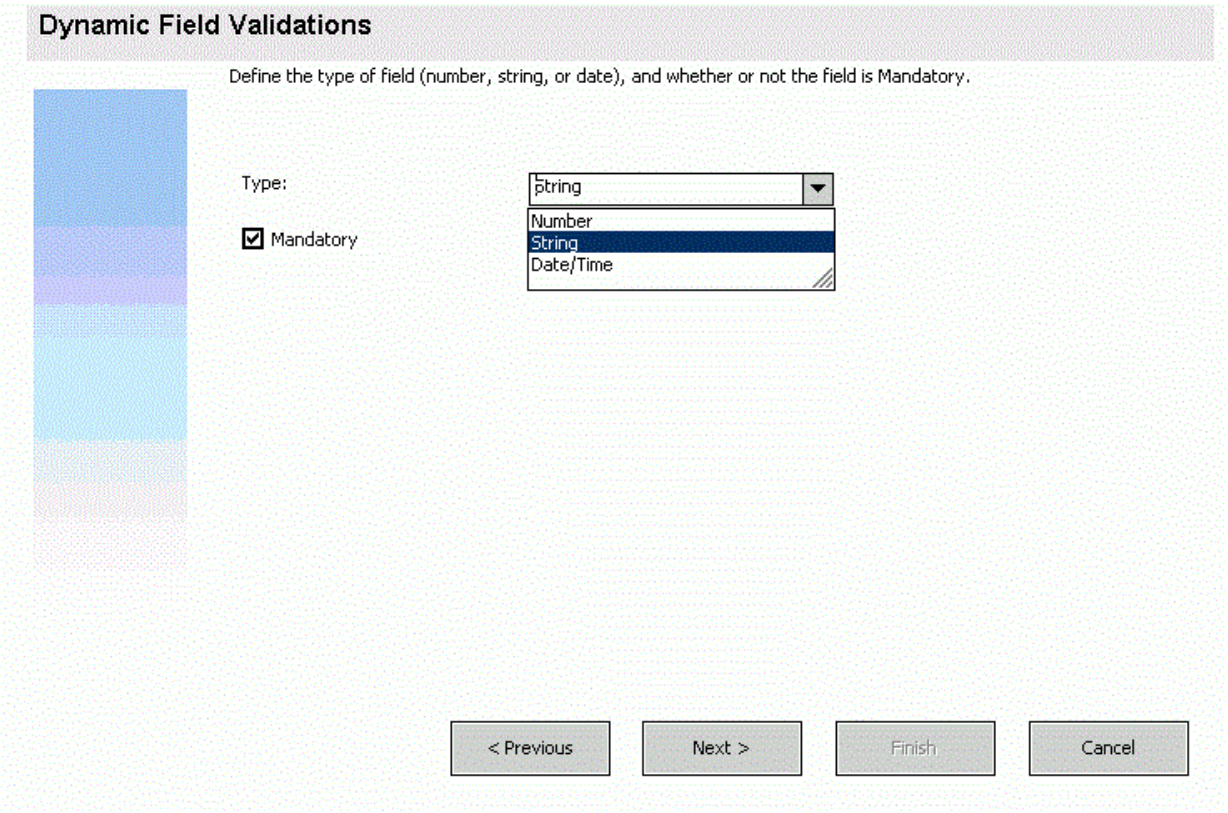
   b. Click **Next**. In the value list table, and then enter optional values for the Region field. The values added will be available in the Project Proposal Catalog Item drop-down list for users to select.



Note

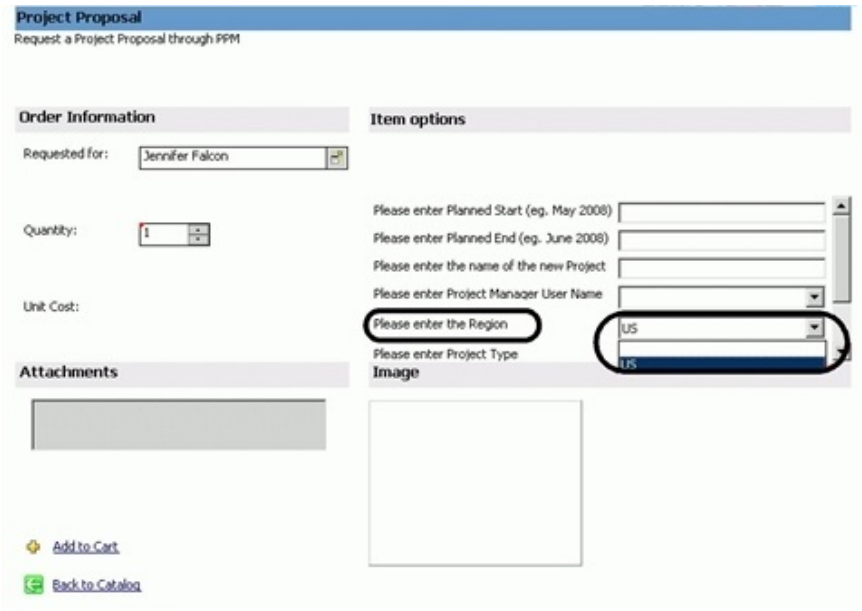
The value provided in the Value column must be the same as that in the Project and Portfolio Management Center (PPM) field on the web page.

c. Click **Next**, and select a type from the **Type** drop-down list. You may also select the **Mandatory** checkbox as necessary.

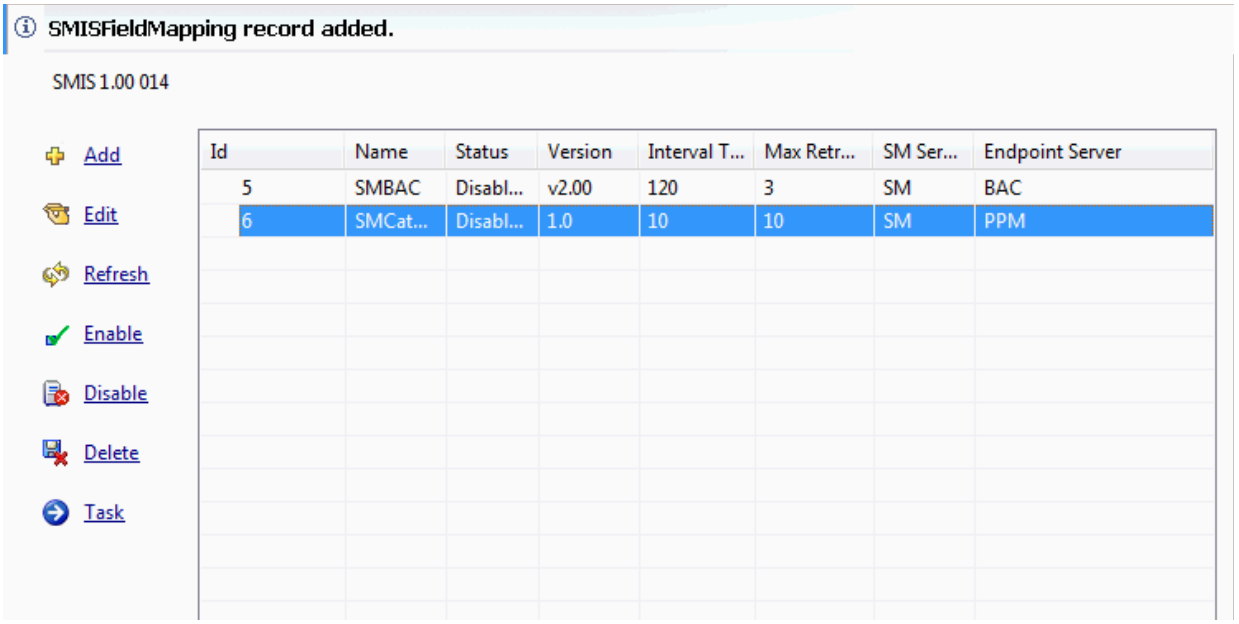


d. Click **Next** to finish the wizard.

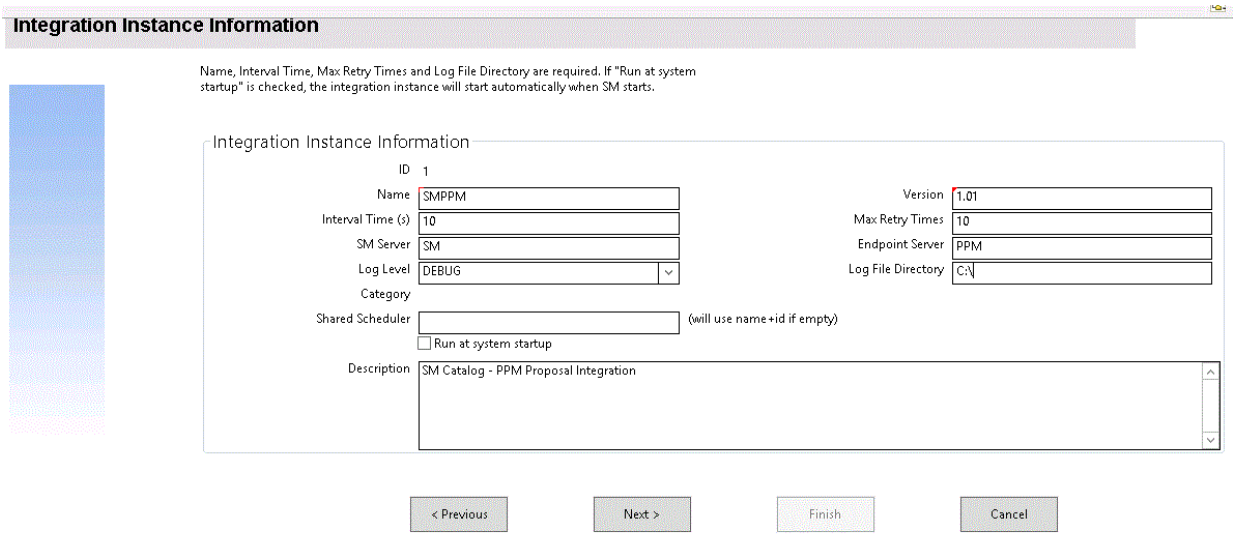
e. Click **Save**. Now the system adds the **Please enter the Region** field in the Project Proposal Catalog Item.



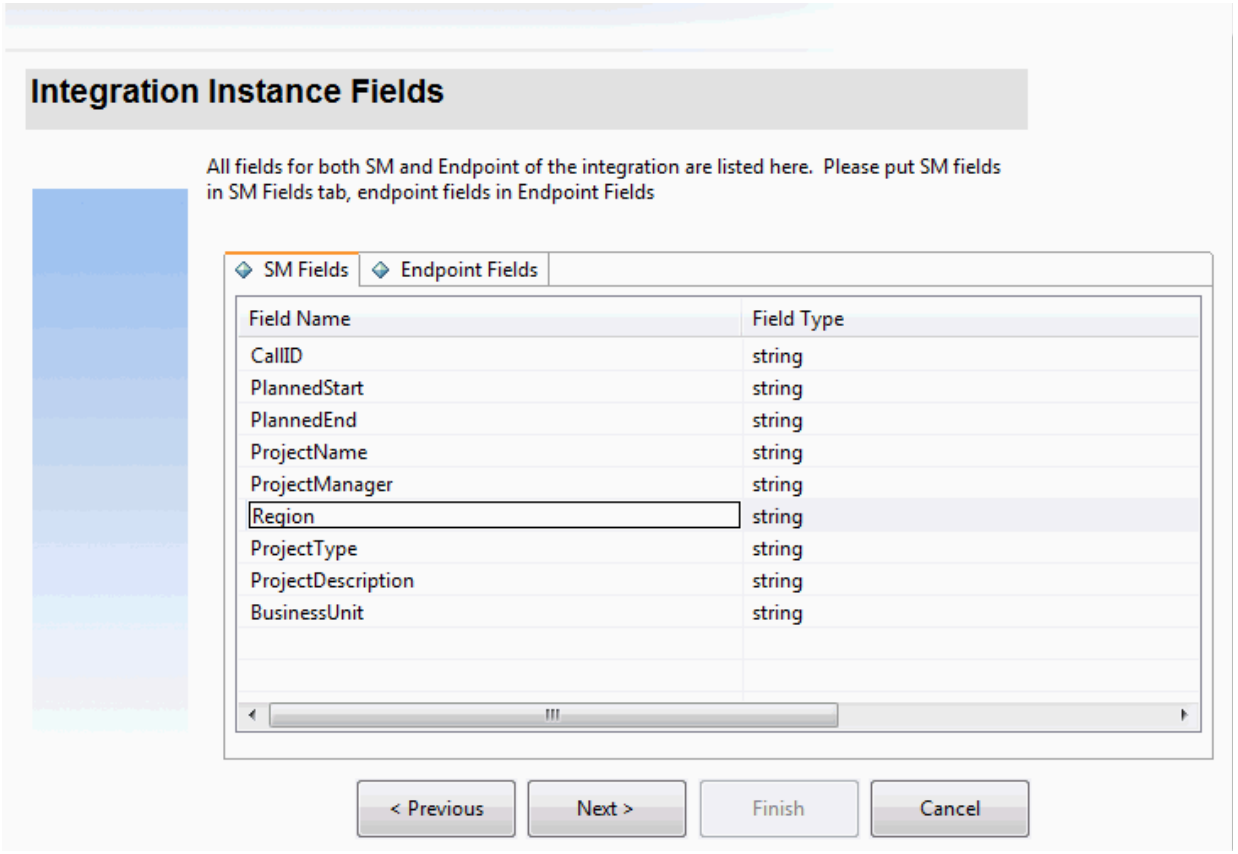
5. After the custom fields are added to the Project Proposal Catalog Item, configure the fields mapping in the Service Manager Integration Suite (SMIS) framework. To do so, go to **Tailoring** > **Integration Manager**.



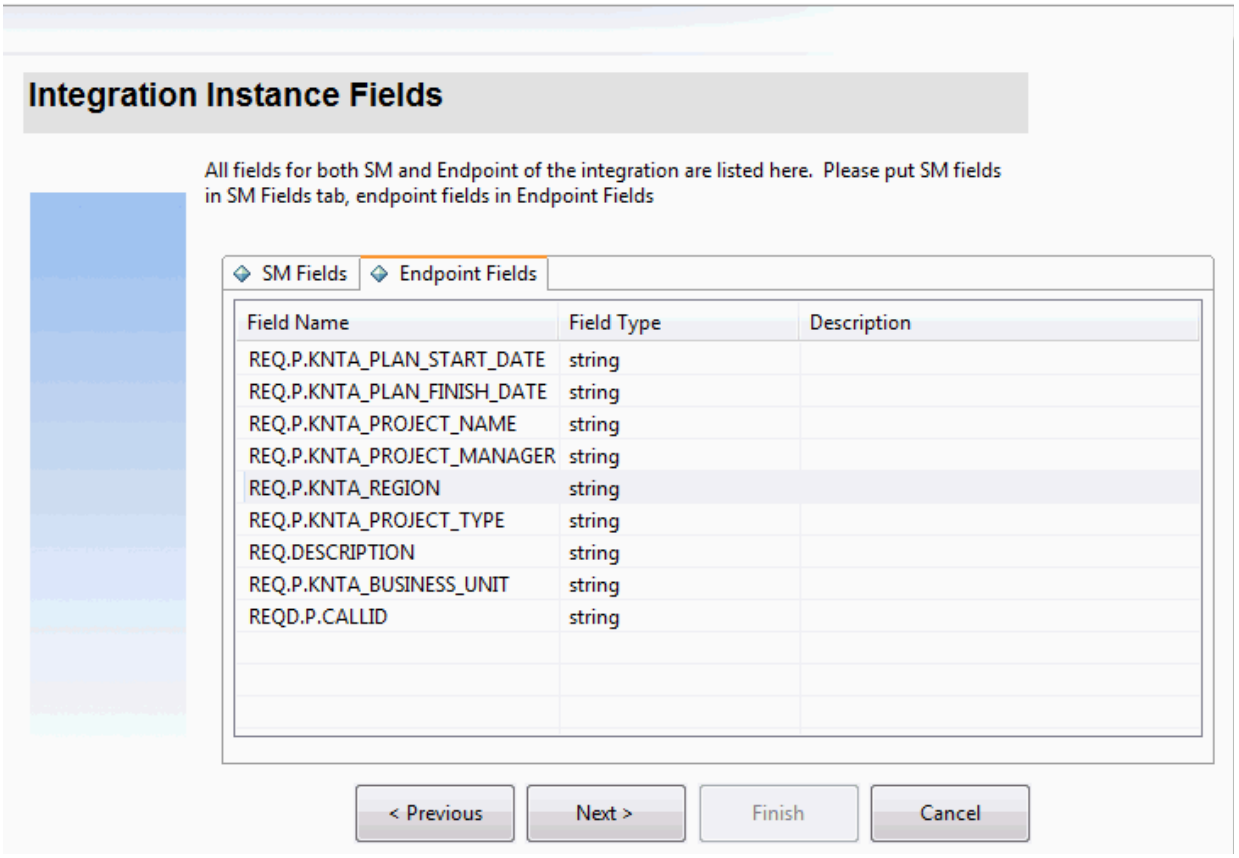
6. Double-click the **SMCatalog_PPMProjectProposal** entry.



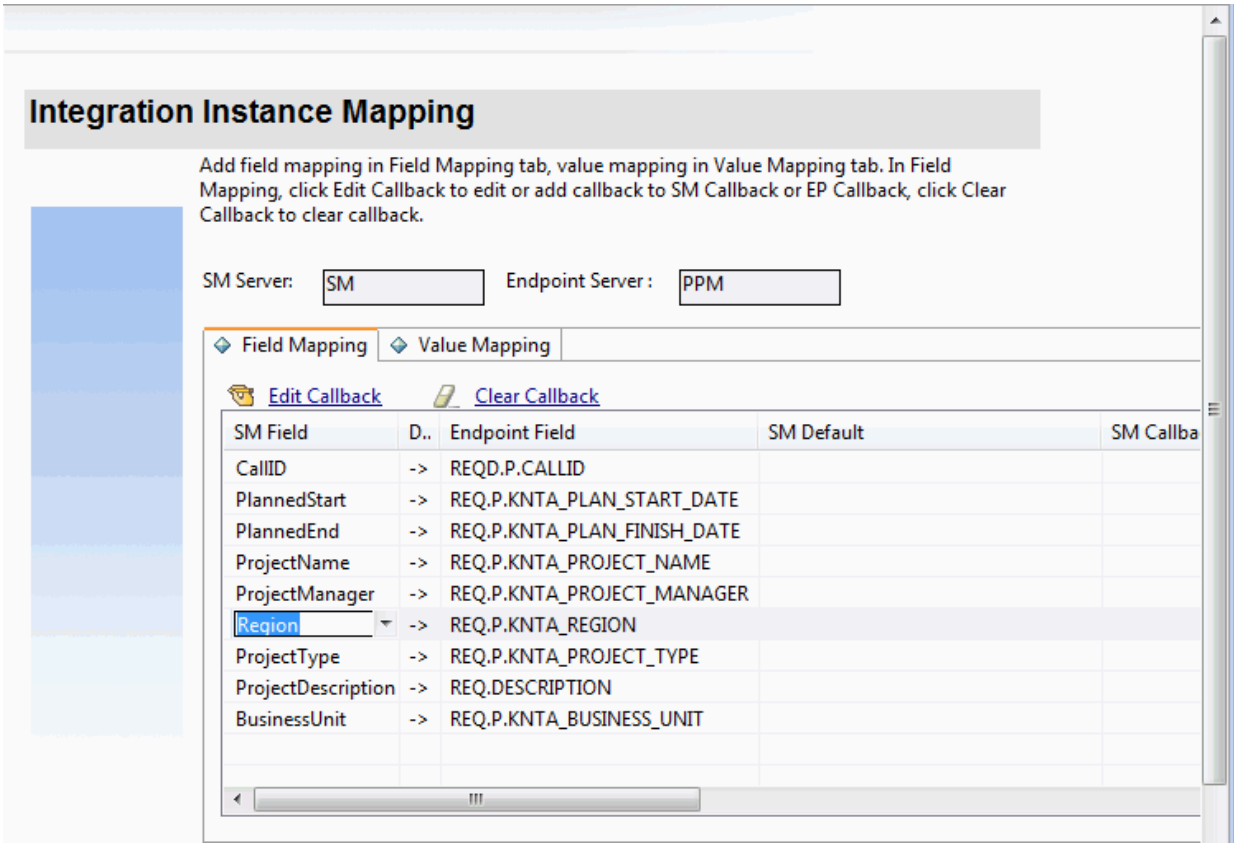
7. Click **Next**.
8. Click **Next** again. The Integration Instance Fields page displays.
9. In the **SM Fields** tab, add the created fields (for example, **Region**) to the Project Proposal Catalog Item.



10. Click the **Endpoint Fields** tab, and add PPM fields that map to the fields added to the Project Proposal Catalog Item. For example, **REQ.P.KNTA_REGION** is a field in PPM mapping to **Region** of the Service Manager Project Proposal Catalog Item.



11. Click **Next**.
12. In the Field Mapping pane, map the fields of the Service Manager Project Proposal Catalog Item to PPM Project Proposal fields.



Note

The **CallID** field is mandatory. PPM Center uses the mapping field **REQD.P.CALLID** to synchronize the status back to Service Manager.

For more information about configuring field mappings or value mappings, see Service Manager Integration Suite (SMIS).

1.5. Application Lifecycle Management (ALM)

Application Lifecycle Management (ALM) empowers organizations to manage the core application lifecycle and grants application teams the crucial visibility and collaboration needed for the predictable, repeatable, and adaptable delivery of modern applications. The SM-ALM integration provides a bi-directional interface to exchange defects and requirements between Service Manager (SM) and ALM. The overall intent of this integration is as follows.

- Leverages Service Manager's position as the service provider and application with the most intimate contact with the customer.
- Efficiently reconciles and resolves the demands placed within Service Manager.
- Tightly couples the demands placed within Service Manager with the ALM requirement management and defect management capabilities within Service Manager.

Defects and Requirements Exchange with Service Manager includes the following scenarios:

- Synchronize Service Manager Changes with ALM defects (SM Change → ALM Defect)
- Synchronize Service Manager Changes with ALM requirements (SM Change → ALM Requirement)
- Synchronize Service Manager Problems with ALM defects (SM Problem ↔ ALM Defect)

This section describes how to configure Service Manager and ALM for integration and how to test the synchronization. The target readers include Consultants and Application Administrators who must set up and maintain ALM to ensure that the synchronization meets all user organization procedural requirements. Based on the assumption that the reader is an experienced user of either Service Manager or ALM (but not necessarily of both), this section describes only the basics of both Service Manager and ALM.

Sample use case

The following typical use cases shed some light on how you can use this integration in your organization.

SM Change → ALM Defect

A business owner creates a Normal Change request in Service Manager and later the Normal Change enters the **Build and Test** phase. On the **ALM** tab of the Change request, this business owner selects the **Change To Defect** action and then selects the ALM project. After saving these updates, the system synchronizes the Change request to a defect in ALM. Developers and QA personnel can begin working on the defect in ALM, in which the key information is synchronized from Service Manager to ALM.

During the QA process, key information is synchronized between ALM and SM. The integration administrator has the responsibility of determining the key information and specifying the information in the field mapping (using the Integration Manager) in order for the business owner to view updated (scheduled) information in SM. The information includes the Change request's title, description, severity, and so on.

For a complete use case, see Synchronize Service Manager Change to ALM defect.

SM Change → ALM Requirement

The requirement synchronization feature of this integration allows requirements found during the Change Management process to be systematically tracked by SM and ALM.

A business owner creates a Normal Change request in Service Manager and later the Normal Change enters the **Build and Test** phase. On the **ALM** tab of the Change request, this business owner selects the **Change To Requirement** action and then selects the ALM project. After saving these updates, the system synchronizes the Change request to a defect in ALM. Developers and QA personnel can begin working on the defect in ALM, in which the key information is synchronized from Service Manager to ALM.

During the QA process, key information is synchronized between ALM and SM. The integration administrator has the responsibility of determining the key information and specifying the information in the field mapping (using the Integration Manager) in order for the business owner to view updated (scheduled) information in SM.

For a complete use case, see Synchronize Service Manager Change to ALM requirement.

SM Problem → ALM Defect

After a Problem is created, if the CPE engineer determines that there is a bug with the Problem after analyzing it, and the bug fixing work needs to be tracked, the CPE engineer triggers/initiates the creation of the ALM CR ticket. On the **ALM** tab of the Problem record, the CPE engineer selects the **Problem To Defect** action and then selects the ALM project. After saving these updates, the system synchronizes the Problem record to a defect in ALM.

For a complete use case, see Synchronize Service Manager Problem to ALM defect.

ALM Defect → SM Problem

The business process for defect management in ALM supports the creation of Problems in SM based on information in ALM. However, in the current solution, the integration can only create a Problem in SM from a defect in ALM.

Related topics

Defects and Requirements Exchange (SM ↔ ALM)

1.5.1. Prerequisites

User role: Service Manager System Administrator

In earlier Service Manager versions, when setting up defects and requirements exchange between Application Lifecycle Management (ALM) and Service Manager (SM), you need to install the middleware of ALM Synchronizer with its add-in. As of version 9.80, the SM-ALM integration is achievable through Integration Manager (SMIS) alone in Service Manager. As a prerequisite, before setting up the new SM-ALM integration solution, new customers and existing customers need to take different actions as follows.

New customers

If you have not configured the SM-ALM integration before, complete the following tasks to set up the new SM-ALM integration.

1. Configure the settings in ALM
2. Configure the settings in Service Manager
3. Manage mappings

Existing customers

If you have already worked with the SM-ALM integration with ALM Synchronizer and add-in before, here are some hints.

- Service Manager no longer needs Web Service configuration and the original configuration is unaffected.
- Refer to the original field mappings in the ALM Synchronizer and reconfigure these mappings in SMIS.
- To reduce the impact, refer to Add custom fields and then create new fields in ALM, such as **SM ID**, **SM Table**, and **Push to SM Problem**.

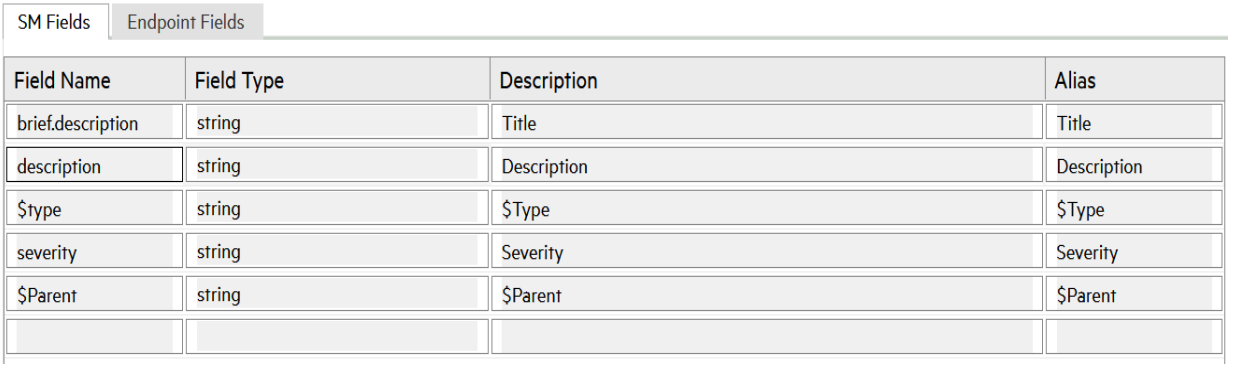
Suppose you have synchronized some Change and Problem records before but the synchronization is not completed yet. When switching to the new integration solution, follow the steps below if you hope to continue synchronizing these Change and Problem records.

1. Sign in to your Service Manager server as a system administrator.
2. Complete the three tasks as described in the *New customers* section above to set up the new SM-ALM integration.

   Note: When configuring field mapping for SM Change to ALM requirement synchronization, the out-of-the-box value for **Endpoint Default** is 3 for the **$Type** field in SM. This value means that the default requirement type is **Functional** after synchronization. You need to match the requirement type by yourself.

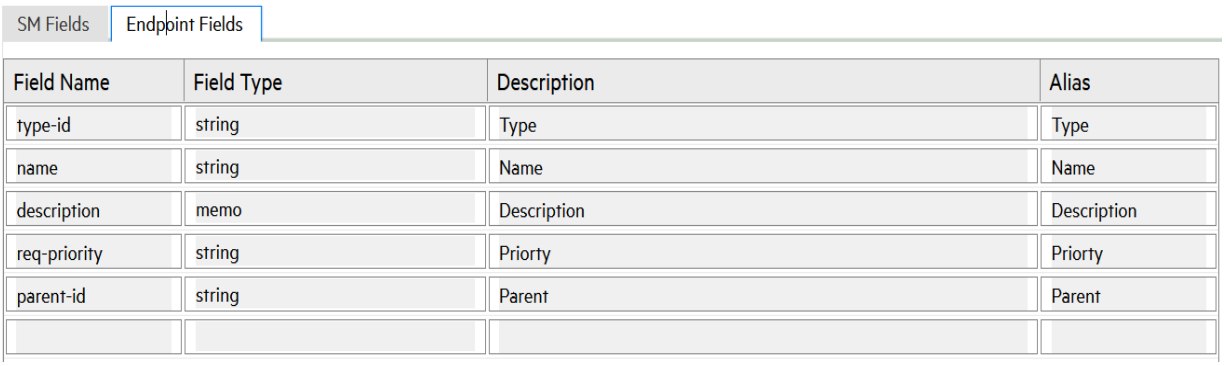
SM Field	Direction	Endpoint Field	SM Default	Endpoint Default	Value Mapping Group	Description
$Type	→	Type		3		

In addition, requirements in ALM are under the **Requirements** root folder. To synchronize requirements to another root folder, add a new endpoint field `parent-id` for ALM and then set its value to ID of the folder to be synchronized. See the following sample screenshots:



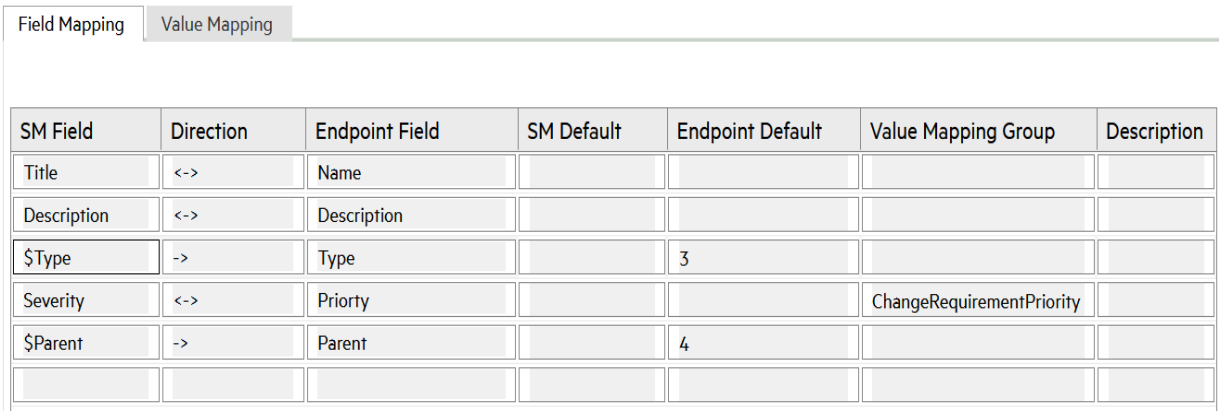
SM Fields | Endpoint Fields

Field Name	Field Type	Description	Alias
brief.description	string	Title	Title
description	string	Description	Description
$type	string	$Type	$Type
severity	string	Severity	Severity
$Parent	string	$Parent	$Parent



SM Fields | Endpoint Fields

Field Name	Field Type	Description	Alias
type-id	string	Type	Type
name	string	Name	Name
description	memo	Description	Description
req-priority	string	Priorty	Priorty
parent-id	string	Parent	Parent



Field Mapping | Value Mapping

SM Field	Direction	Endpoint Field	SM Default	Endpoint Default	Value Mapping Group	Description
Title	<->	Name				
Description	<->	Description				
$Type	->	Type		3		
Severity	<->	Priorty			ChangeRequirementPriority	
$Parent	->	Parent		4		

3. Create a ScriptLibrary and then add the following scripts.

```
var createContext = lib.AlmMigration.createContext;
var migrate = lib.AlmMigration.migrate;

function run() {
    // lib.OctaneTask.setLogLevel('debug');

    // TODO change to the SMIS instance id for ALM project
    var smisIntId = 1;
    var context = createContext(smisIntId);

    // TODO change the find query to match the Change records which
needs to migrate.
    var query1 = 'active=true and qcintegration.type>0 and not
null(qcintegration.id) and qcintegration.project=:project';
    lib.c.$('!cm3r', true).find(query1,
'ALM/HP/SM').each(function(item){
        migrate(context, item);
    });

    // TODO change the find query to match the Problem records
which needs to migrate.
    var query2 = 'open=true and qcintegration.type>0 and not
null(qcintegration.id) and qcintegration.project=:project';
    lib.c.$('!rootcause', true).find(query2,
'ALM/HP/SM').each(function(item){
        migrate(context, item);
    });

    context.logout();
}

run();
```

The scripts above perform these steps.

1. Create a context using the SMIS instance ID. Update it with your SM-ALM SMIS instance ID.
2. Query the Change records in the current Project that still needs synchronization with the fields in ALM. Modify the query and the `ALM/HP/SM` parameter in the query as required.
3. Call a migration method to migrate each Change record.
    1. Assign a value to the ALM Action field in each Change record.
    2. Create AlmInfo and AlmInfoMore records for the Change record.
    3. Update the SM ID and SM Table fields in ALM.

    4. Continue to follow *step 2* and *step 3* to perform similar operations on the Problem records.

4. Click the **Execute** button.

Note: The legacy **QC Integration** tab still exists after migration and some legacy fields have duplicate values with the new fields. To avoid duplication and misleading, you need to set the following fields to read-only or invisible.

- In Service Manager: **Server/Domain/Project**, **Synchronize with QC**, and **Forward to QC** on the legacy **QC Integration** tab
- In ALM: **Synchronize with SM Problem**

1.5.2. Configure the settings in ALM

User role: ALM Project Administrator

Complete the configurations below when setting up the integration between Service Manager and ALM. Ensure that you have working knowledge about ALM before you proceed. Detailed instructions about ALM administration are outside the scope of Service Manager documentation and are hence not covered. For more information, see the ALM documentation.

Set up API access

You need to set up API access for Service Manager and grant Service Manager registered access keys (Client ID and a Client secret) so that it can access ALM. In the SM-ALM integration, Service Manager uses the Client ID and Client secret for authentication when synchronizing information with ALM.

Follow these steps:

1. Visit https://*<ALM server name>*:*<port>*/qcbin and then click **API Key**. Sign in to ALM as a project administrator.
2. Click **API ACCESS**, and then enter an appropriate name for the access.
3. Select a user who has the create and update privileges for defects and requirements in the corresponding domain and project, and then enter a description if needed. Also, this user needs to be an independent user dedicated to SM-ALM integration and you should not use this user for other purposes.
4. Click **Create**. A dialog displays with a Client ID and an API key secret.
   Click **Copy** to copy these keys to the clipboard so you can use them when configuring Service Manager that needs to access ALM.
5. Click **Close**. The access ID is added to the grid and its status is active.

Add custom fields

You need to manually add the following user-defined fields (UDFs) to ALM.

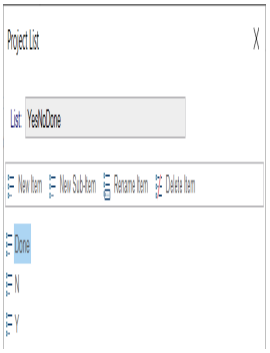
1. The **SM ID** field. The defects and requirements in ALM display the corresponding Service Manager ticket ID in this field.
2. The **SM Table** field. The defects and requirements in ALM use this field to store the name of the SM module table and the SMIS instance ID (for example, cm3r-1).
3. The **Push to SM Problem** field. You need to select the action **Y** in this field before pushing a defect to a new Problem in SM. ALM also displays the **Done** status in this field after a new Problem is created in SM.

See the following screenshot as a sample display of these fields in ALM:

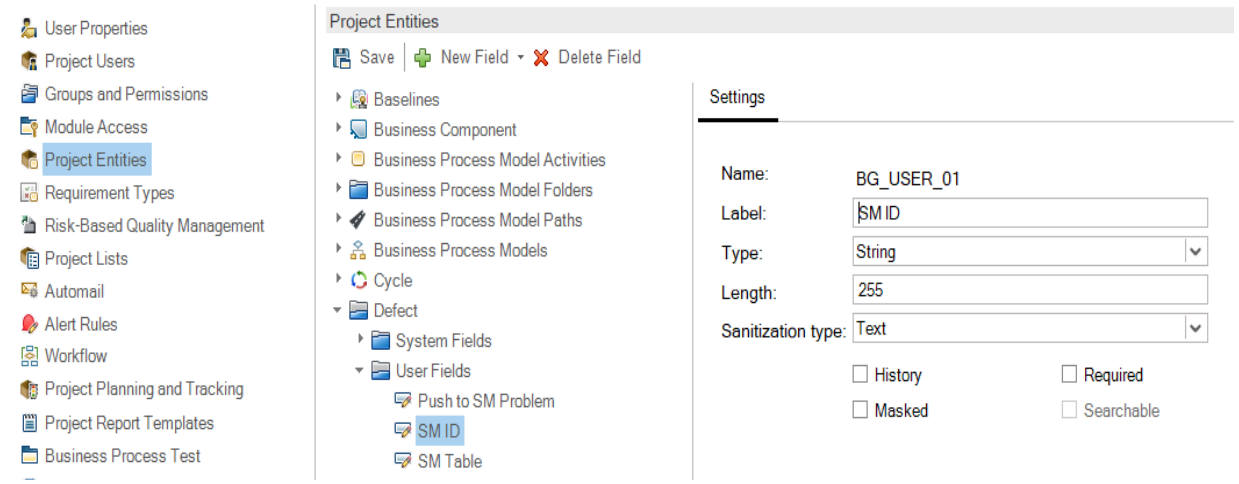
Defect ID	SM ID	Push to SM Problem	Description	Severity
3	C10066		New item req...	2-Medium
4	PM10001		Recently, us...	1-Low
5	PM10004	Done	Outlook serv...	3-High

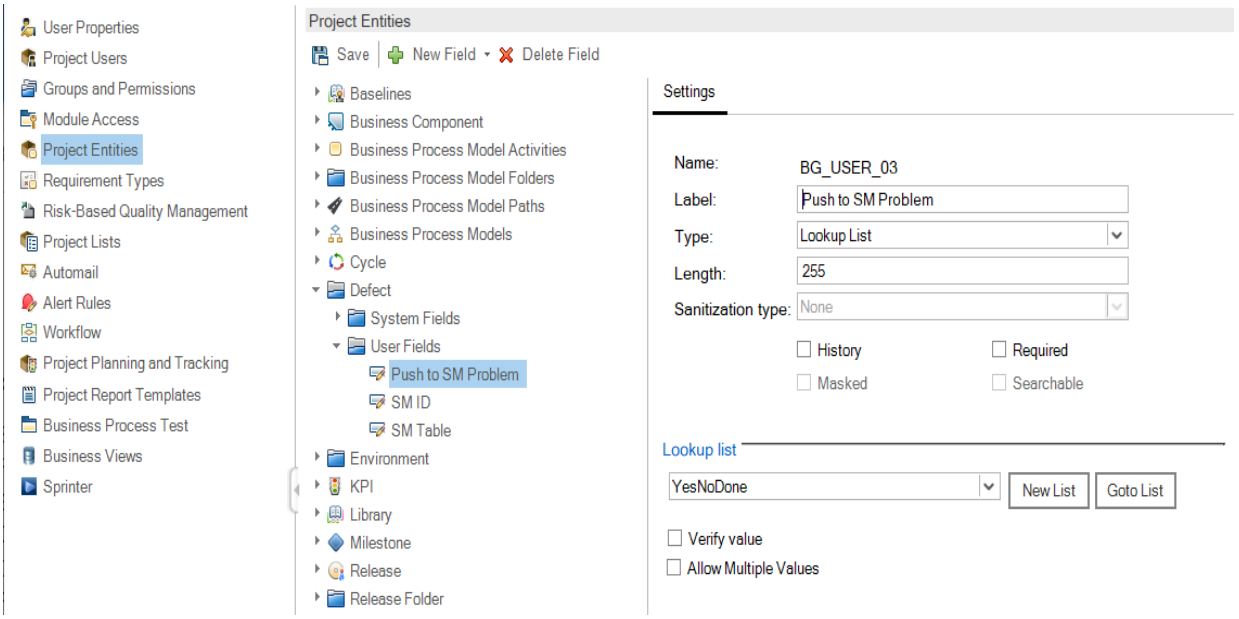
Follow these steps:

1. Sign in to ALM as a project administrator.
2. Click **Settings** > **Customize**. The **Application Lifecycle Management - Project Customization** module opens.
3. Click **Project Entities** > **Defect** > **User Fields** > **New Field**, and then add the following fields for the defect entity in Project Entities (*XX*, *XY*, and *XZ* are sequential numbers auto-generated by ALM).

Name	Label	Type	Lookup List
BG_USER_*XX*	SM ID	String	---
BG_USER_*XY*	SM Table	String	---
BG_USER_*XZ*	Push to SM Problem	Lookup List	Click **New List** and then create a list named YesNoDone. Click **New Item** and then add **Y**, **N**, and **Done** to the project list. 

See the following screenshots as examples:

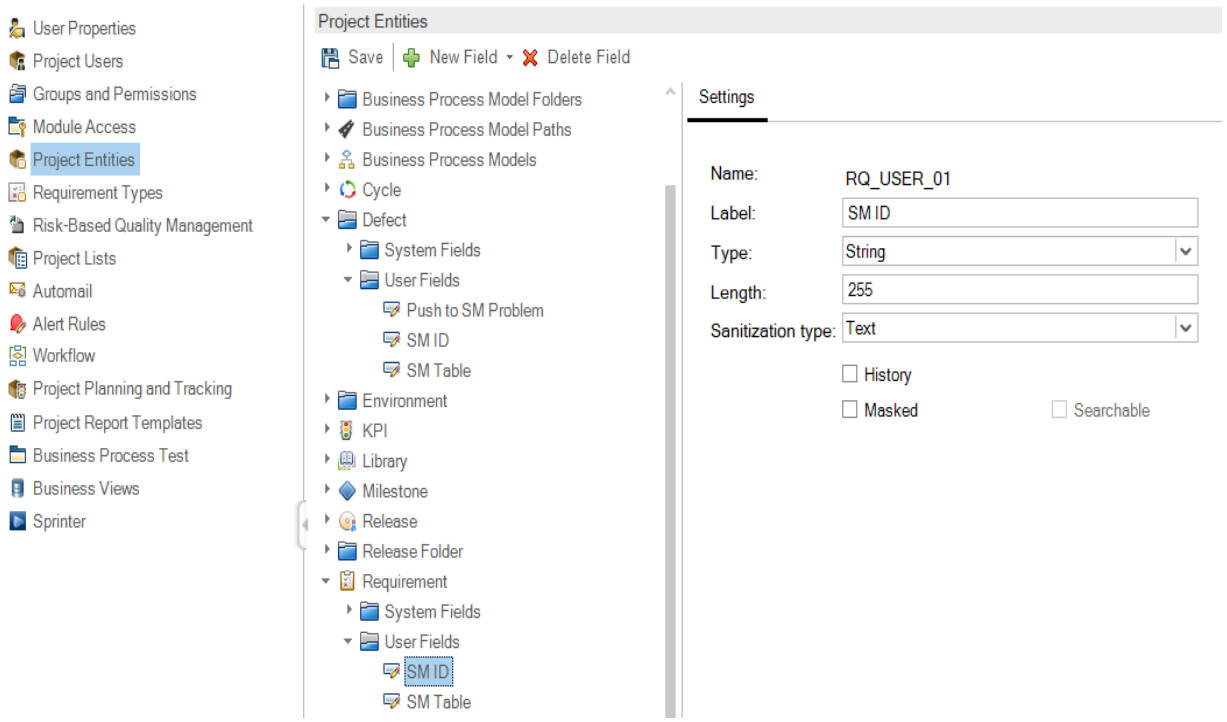




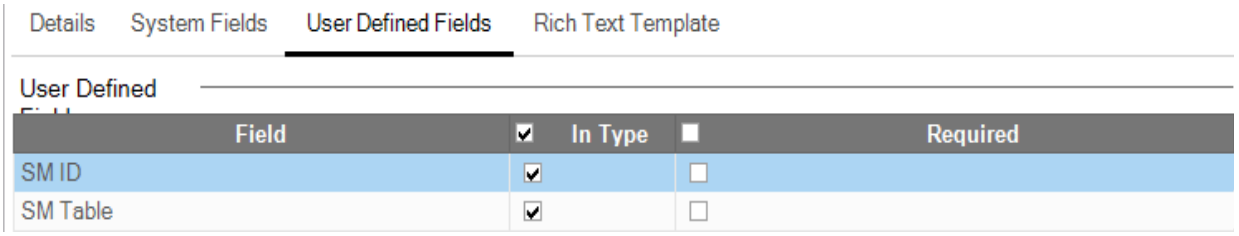
4. Click **Requirement** > **User Fields** > **New Field**, and then add the SM ID and SM Table fields for the requirement entity in Project Entities.

Name	Label	Type
RQ_USER_*XX*	SM ID	String
RQ_USER_*XY*	SM Table	String

See the following screenshots as an example:



5. Click **Save** to save your changes.
6. To functionalize the newly-added fields for the requirement entity, click **Requirement Types** > **Functional** > **User Defined Fields**, and then select **In Type** for both new fields.



   If you have other user-defined fields to be synchronized, select **In Type** for these fields as well.

   Note: The **Functional** requirement type is an example only. You can select another requirement type as required.

7. Click **Save** to save your changes.

Add rules to restrict the writing permission to the custom fields

You can define a rule that triggers actions in ALM when certain conditions are met. In this task, you need to manually add rules to restrict the writing permission to the **SM ID**, **SM Table**, and **Push to SM Problem** user-defined fields. Both the **SM ID** and **SM Table** fields are read-only to all ALM users and can only be updated by the background API. The **Push to SM Problem** field is read-only when the **SM ID** and **SM Table** fields have values.

Follow these steps:

1. Sign in to ALM as a project administrator.
2. Click **Settings** > **Customize**. The **Application Lifecycle Management - Project Customization** module opens.
3. Click **Workflow** > **Script Editor**. The **Script Editor** window opens.
4. Click **Defects module script** > **Bug_New**, and then update the subroutine as follows.

```
Sub Bug_New
  On Error Resume Next

  On Error GoTo 0

Bug_Fields.Field("BG_USER_01").IsReadOnly=True
Bug_Fields.Field("BG_USER_02").IsReadOnly=True
Bug_Fields.Field("BG_USER_02").IsVisible=False

If Not Bug_Fields.Field("BG_USER_01").IsNull then
Bug_Fields.Field("BG_USER_03").IsReadOnly=True
End If

End Sub
```

`BG_USER_01`, `BG_USER_02`, and `BG_USER_03` are the corresponding field names for **SM ID**, **SM Table**, and **Push to SM Problem** fields for ALM defects.

5. Click **Defects module script** > **Bug_MoveTo**, and then update the subroutine as follows.

```
Sub Bug_MoveTo
  On Error Resume Next

  On Error GoTo 0

Bug_Fields.Field("BG_USER_01").IsReadOnly=True
Bug_Fields.Field("BG_USER_02").IsReadOnly=True
Bug_Fields.Field("BG_USER_02").IsVisible=False

If Not Bug_Fields.Field("BG_USER_01").IsNull then
Bug_Fields.Field("BG_USER_03").IsReadOnly=True
End If

End Sub
```

6. Click **Requirements module script** > **Req_New**, and then update the subroutine as follows.

```
Sub Req_New
  On Error Resume Next

  On Error GoTo 0

Req_Fields.Field("RQ_USER_01").IsReadOnly = True
Req_Fields.Field("RQ_USER_02").IsReadOnly = True
Req_Fields.Field("RQ_USER_02").IsVisible=False
End Sub
```

   `RQ_USER_01` and `RQ_USER_02` are the corresponding field names for **SM ID** and **SM Table** fields for ALM requirements.

7. Click **Requirements module script** > **Req_MoveTo**, and then update the subroutine as follows.

```
Sub Req_MoveTo
  On Error Resume Next

  On Error GoTo 0

Req_Fields.Field("RQ_USER_01").IsReadOnly = True
Req_Fields.Field("RQ_USER_02").IsReadOnly = True
Req_Fields.Field("RQ_USER_02").IsVisible=False
End Sub
```

8. Click **Save** to save your changes.

1.5.3. Configure the settings in Service Manager

User role: Service Manager System Administrator

You must complete the following tasks from your Service Manager system to support the integration.

Add an integration instance

To set up ALM integration, you must add and enable an instance of the ALM integration in Service Manager Integration Manager.

To add and enable an ALM integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager** to open Integration Instance Manager.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **SMALM** from the Integration Template list, and then select **Import Mapping**.
4. Click **Next**. The Integration Instance Information page opens.
5. Complete the following settings on the Integration Instance Information page as necessary.

Setting	Value	Description
Name (required)	User-defined.	Name of the integration instance. Default: SMALM. **Note:** To integrate multiple ALM projects, you can add more integration instances as necessary. Make sure to use a unique name for each SM-ALM integration instance.
Version (required)	1.0	The version number of the integration.
Interval Time (s) (required)	User-defined.	The interval time that the integration background schedule runs. For example, if you set the value to 300, the background process tries to execute the SMIS integration task every 5 minutes. Default: 300.
Max Retry Times	User-defined.	Maximum allowed number of retries if the background scheduler fails to run. Default: 5. **Note:** The ALM integration doesn't use this option.

Setting	Value	Description
Log Level	Select a level from the drop-down list.	Level of diagnostic information that the Service Manager server logs to the log file directory. The possible log levels are DEBUG, INFO (default), WARNING, ERROR, and OFF. Default: INFO.
Log File Directory	Specify a directory.	This is a directory that exists on the Service Manager server host where log files are stored. The log includes information for receiving and processing messages. For example: c:\ALM_logs. If this field is empty, the system uses the Service Manager sever log (sm.log) for ALM integration.

Setting	Value	Description
Shared Scheduler	User-defined.	The scheduler that the integration instance uses to schedule background processes. Multiple integration instances can use a shared scheduler. When you create an integration template, you can define a default shared scheduler for the instances that are based on the template. All instances that are based on this template can use the same shared scheduler. However, if you create an instance that uses a different scheduler, SMIS creates a new scheduler accordingly. If this field is empty, the system uses <ALM integration name + instance ID> for ALM integration.
Run at system startup	True or false.	Automatically enable or disable the integration instance when the Service Manager server starts.
Description	User-defined.	Description of the integration instance.

6. Click **Next**. The Integration Instance Parameters page opens.
7. Complete the following settings on the **ALM Project** tab.

Setting	Value	Description
Domain	Paste the ALM domain.	The ALM domain.
Project	Paste the ALM project.	The ALM project.
Project Alias	User-defined.	A unique alias of the ALM project. Later, when setting up an SM-ALM synchronization, you need to select this alias of your ALM project on the ALM tab in a Change or a Problem record. Default: ALM-Default-Project. **Note:** After cloning an SM-ALM integration instance in SMIS, make sure to update this field so that the project alias is unique.

Setting	Value	Description
Base URL	Paste the base URL of the ALM server.	The base URL of the ALM server. The URL has the following format: https://*<ALM server name>*:*<port>* Replace *<ALM server name>* with the IP address or fully-qualified domain name (FQDN) of the ALM server, and replace *<port>* with the communications port the ALM server uses. **Note:** To set up an HTTPS connection, make sure to add the ALM server root CA certificate to the SM trust store so as to verify the connection. See the *Add the ALM root CA certificate to the SM trust store* section below.
Client ID	Paste a client ID string.	The client ID you received in ALM when setting up API access for Service Manager.
Client Secret	Paste a client secret string.	The client secret string you received in ALM when setting up API access for Service Manager.
Version Enabled	False	Select this checkbox if version control is enabled in an ALM project.

8. Complete the following settings on the **Defect** tab as necessary.

Option	Description
SM ID Field	The ALM field that displays the corresponding Service Manager ticket ID. For example, if the SM ID field name for ALM defects is BG_USER_01, the value for ALM Field for SM ID is user-01.
SM Table Field	The ALM field that displays the corresponding Service Manager table name. For example, if the SM Table field name for ALM defects is BG_USER_02, the value for ALM Field for SM Table is user-02.
Push to SM Problem Field	The ALM field that defines whether to push an ALM defect to SM and create a Problem record. For example, if the Push to SM Problem field name for ALM defects is BG_USER_03, the value for ALM Field for Push to SM Problem is user-03.
Exclude Status	Review and update the ALM defect status for the selected options. ALM stops updating a defect if it is in any of the listed status.
Enable SM Change to ALM Defect	Select whether to enable the synchronization from SM Change to ALM Defect.
Enable SM Problem to ALM Defect	Select whether to enable the synchronization between SM Problem and ALM Defect.

9. Complete the following settings on the **Requirement** tab as necessary.

Option	Description
SM ID Field	The ALM field that displays the corresponding Service Manager ticket ID. For example, if the SM ID field name for ALM requirements is RQ_USER_01, the value for ALM Field for SM ID is user-01.
SM Table Field	The ALM field that displays the corresponding Service Manager table name. For example, if the SM Table field name for ALM requirements is RQ_USER_02, the value for ALM Field for SM Table is user-02.
Enable SM Change to ALM Requirement	Select whether to enable the synchronization from SM Change to ALM Requirement.

10. Click **Verify Connection** to verify the connection settings.
11. Click **Next**. The Integration Instance Fields page opens and displays the out-of-the-box fields available for both SM and ALM of the integration. You can review, update, or add more fields as required.

    Or, if you have selected more than one module in *step 8* and *step 9*, the Select Module page opens. You can either double-click a module or select a module from the list and then click **Open Field Mapping** to open the Integration Instance Fields page. Otherwise, if you have selected only one module in *step 8* or *step 9*, the Integration Instance Fields page opens directly.

    For your selected module, the system displays the out-of-the-box fields available for both SM and ALM of the integration. You can review, update, or add more fields as required.
12. Click **Next**. The Integration Instance Fields page opens. Review, update, or add more fields to the **SM Fields** tab and the **Endpoint Fields** tab as necessary.
13. Click **Next**. The Integration Instance Field Mappings page opens.
14. Review, update, or add more fields to the **Field Mapping** tab and the **Value Mapping** tab as necessary. For more information, see Manage mappings.

    To map fields:

1. Select an SM field and an ALM endpoint field.
2. Select a direction in the **Direction** column.
3. (Optional) If an SM field to be mapped has multiple value selections (such as Impact), specify a name in the **Value Mapping Group** column and then configure the value mappings on the **Value Mapping** tab.

To map values:

1. Enter a value mapping group name as you have specified on the **Field Mapping** tab.
2. Enumerate the value mapping pairs between an SM field and an ALM endpoint field.
3. Set the condition to true.

15. Click **Finish**. Service Manager creates the instance.
16. Enable the integration instance. A processor for the ALM integration instance now runs in the background.



Note

If your ALM has multiple projects to be integrated, you can add more integration instances as necessary.

Add the ALM root CA certificate to the SM trust store

If you are setting up an HTTPS connection, follow these steps to import the ALM server root CA certificate to the SM trust store.

1. Export ALM root CA certification to a file. For example, almca.crt.
2. Import almca.crt to the SM trust store by using the keytool in your JDK. See the following sample commands.

```
keytool -import -trustcacerts -alias almca -file almca.crt -
keystore cacerts -storepass changeit
```

3. Copy cacerts to the `<SM_SERVER_HOME>/RUN` folder.
4. Update sm.ini by adding the following lines:

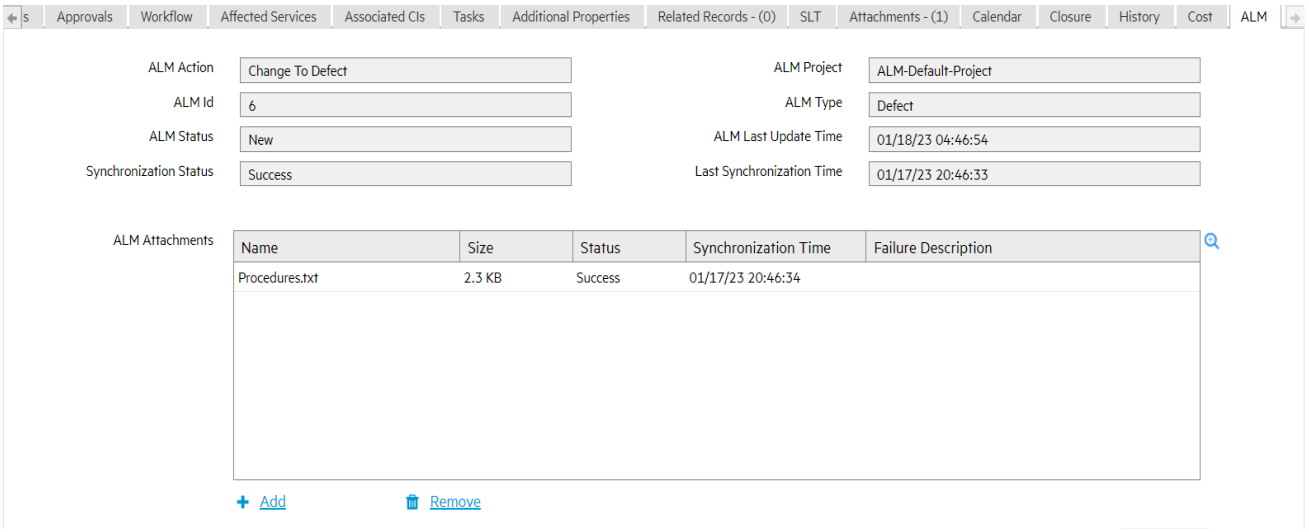
```
truststoreFile:cacerts
truststorePass:changeit
```

5. Restart the SM server.

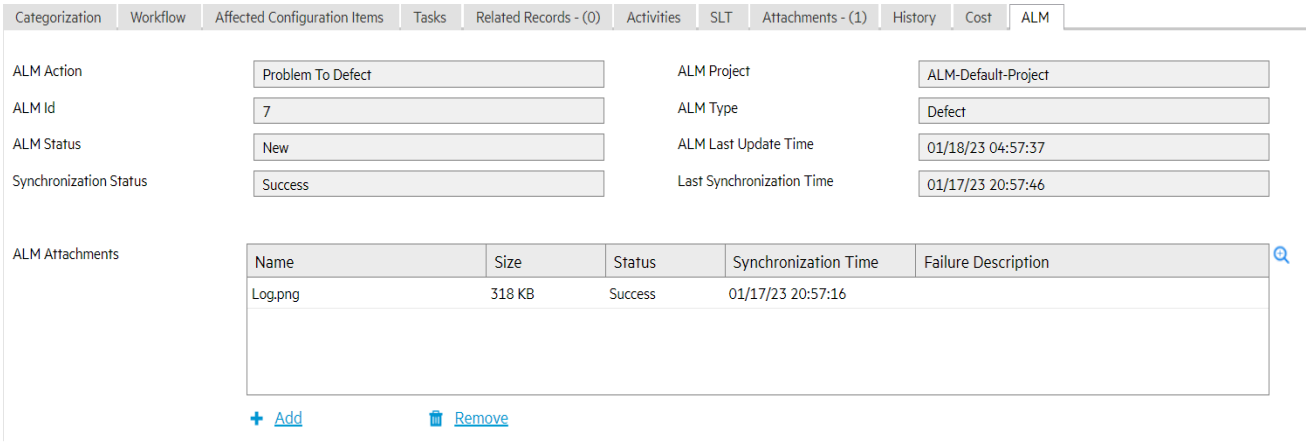
(Optional) Tailor the integration

The out-of-the-box SM-ALM integration synchronizes a Normal Change request that is in the **Build and Test** phase in Service Manager to an ALM defect/requirement. It also synchronizes a Service Manager Problem record to an ALM defect. You can tailor the integration to work with more Change categories and more Change/Problem workflow phases so as to increase the usability and flexibility of this integration. All you need to do is to add the **ALM** tab to a target workflow phase of a Change request regardless of the Change request's category, or add the **ALM** tab to a target workflow phase of a Problem.

In the SM-ALM integration, Service Manager uses the **ALM** tab in a Change/Problem record to define the ALM synchronization action and ALM project. This tab also displays a series of synchronization information, including ALM Id, ALM Type, ALM Status, ALM Last Update Time, and so on. Meanwhile, you can also select the attachments of a record on the **ALM** tab so as to synchronize them to ALM. The following screenshot illustrates the ALM tab after Service Manager synchronizes a Change record to an ALM Defect.



The following screenshot illustrates the ALM tab after Service Manager synchronizes a Problem record to an ALM Defect.



To display the ALM tab in any workflow phase of a Change request or a Problem, you need to manually add this tab to the corresponding form of that phase. Refer to the following steps for some guidelines.

For Changes:

1. Click **Tailoring** > **Forms Designer**.
2. Search for the corresponding form of a phase. For example, to display the ALM tab when a Change enters the Validation phase, search for the **chm.normal.registration** form.
3. Add a Notebook Tab widget to your Change format, and then set the following property of the Notebook Tab widget.

Property	Value
Visible Condition	[$G.alm.change.module.enabled]

4. Add a subform to the Notebook Tab widget, and then set the following property of the subform control.

Property	Value
Format	alm.info

5. Save your changes.

For Problems:

1. Click **Tailoring** > **Forms Designer**.
2. Search for the corresponding form of a phase. For example, to display the ALM tab when a Problem enters the Resolution phase, search for the **pbm.problem.resolution** form.
3. Add a Notebook Tab widget to your Problem format, and then set the following property of the Notebook Tab widget.

Property	Value
Visible Condition	[$G.alm.problem.module.enabled]

4. Add a subform to the Notebook Tab widget, and then set the following property of the subform control.

Property	Value
Format	pbm.alm.info

5. Save your changes.

1.5.4. Manage mappings

User role: Service Manager System Administrator, ALM Project Administrator

In the Service Manager and ALM integration, a mapping consists of field mapping and value mapping.

Field Mapping

The **Field Mapping** tab defines the mapping between ALM fields and SM fields.

The following table describes the columns on this tab.

Column Name	Description
SM Field	The name of a field in Service Manager, and can be selected from the list.
Direction	Indicates the mapping direction of a field mapping, and has only three values to select from. • →: from SM Field to ALM Field. • ←: from ALM Field to SM Field. • ↔: in both directions.
Endpoint Field	The name of a field in ALM. You can select a field from the list.
SM Default	Defines the default value of the SM Field if there is no value or there is a constant in the corresponding ALM Field when mapping an ALM Field to an SM Field. **Note** If the field is of date type, the value's format must resemble either `2023-03-18T00:00:00Z` or `2023-03-18T00:00:00+08:00`. If the field is of boolean type, the value can be `true` or `false`.

Column Name	Description
Endpoint Default	Defines the default value of an ALM Field if there is no value or there is a constant in the corresponding SM Field when mapping an SM Field to an ALM Field. **Note** If the field is of date type, the value's format must resemble `2023-03-18`. If the field is a boolean type, the value can be `Y` or `N`.
Value Mapping Group	The name of a value mapping group to which an ALM Field and an SM Field belong. A value mapping group contains a collection of enumerated ALM and SM value pairs. The Service Manager value and ALM value will be mapped according to the configuration on the **Value Mapping** tab.
Description	Enter a text description for this field mapping.

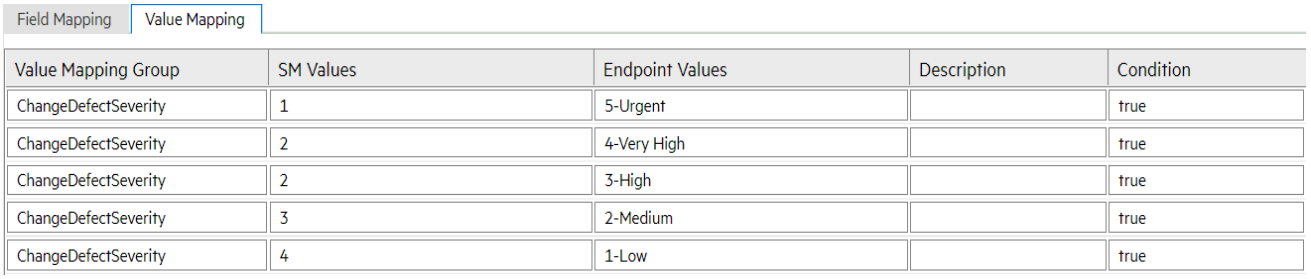
Value Mapping

The **Value Mapping** tab defines the value mapping between ALM and SM for those fields that have multiple values.

The following table describes the columns on this tab.

Column Name	Description
Value Mapping Group	The name of a value mapping group. A value mapping group contains a collection of enumerated ALM/SM value pairs.
SM Values	The values of the Service Manager fields.
Endpoint Values	The values of the ALM fields.
Description	Description of the value mapping.
Condition	SMIS enables you to configure conditions for value mappings. Set the value to `true` so that the value mapping defined in this entry is always effective.

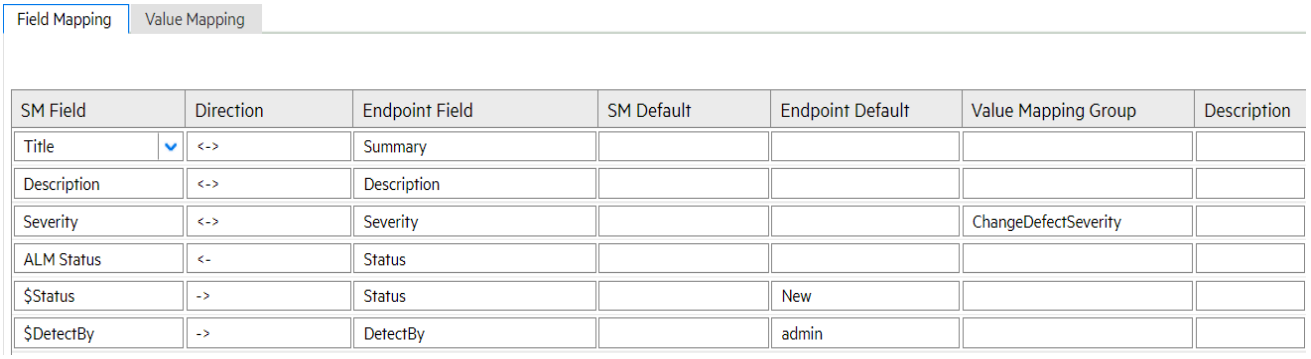
For example, below is a value mapping named **ChangeDefectSeverity**, and there are five mapping entries for this value mapping group.



Field Mapping | Value Mapping

Value Mapping Group	SM Values	Endpoint Values	Description	Condition
ChangeDefectSeverity	1	5-Urgent		true
ChangeDefectSeverity	2	4-Very High		true
ChangeDefectSeverity	2	3-High		true
ChangeDefectSeverity	3	2-Medium		true
ChangeDefectSeverity	4	1-Low		true

The name **ChangeDefectSeverity** can be filled in the **Value Mapping Group** column on the **Field Mapping** tab.



Field Mapping | Value Mapping

SM Field	Direction	Endpoint Field	SM Default	Endpoint Default	Value Mapping Group	Description
Title	<->	Summary				
Description	<->	Description				
Severity	<->	Severity			ChangeDefectSeverity	
ALM Status	<-	Status				
$Status	->	Status		New		
$DetectBy	->	DetectBy		admin		

This means when mapping the Severity field between a Service Manager Change and an ALM Defect, if the value of the Severity field in Service Manager is 3, the value `3` will be mapped to `medium` for the Severity field in ALM, and vice versa.

1.5.5. Verify the integration

User role: Service Manager system administrator

> **Note**
>
> The synchronization process for attachments and new updates is unidirectional and incremental. That is, you can only synchronize attachments and new updates from SM to ALM. Meanwhile, if you remove attachments and new updates from SM after synchronization, the attachments and comments in ALM are not affected.

SM Change → ALM defect/requirement

The following example verifies the integration by synchronizing an SM Change to an ALM defect/requirement.

1. Synchronize an SM Change to an ALM defect/requirement.
    1. Navigate to Change Management, and then open a Normal Change record that is in the **Build and Test** phase.
    2. Click the **ALM** tab.
    3. To synchronize an SM Change to an ALM defect, select **Change To Defect** in the **ALM Action** drop-down list, and then select your ALM project (project alias) in the **ALM Project** drop-down list.

       To synchronize an SM Change to an ALM requirement, select **Change To Requirement** in the **ALM Action** drop-down list, and then select your ALM project (project alias) in the **ALM Project** drop-down list.
    4. Click **Save** and then wait for the background schedule to synchronize the Change record to ALM. After synchronization, the ALM Id, ALM Type, ALM Status, ALM Last Update Time, Synchronization Status, and Last Synchronization Time fields on the **ALM** tab have read-only values that are populated automatically.

Field	Description
ALM Id	The defect/requirement ID in ALM.
ALM Type	Defect/Requirement.
ALM Status	The defect's or requirement's status in ALM.
ALM Last Update Time	The time when the defect/requirement is created in ALM for the first-time synchronization. **Note:** For the later synchronizations, this field displays the last time when the defect/requirement is updated by a user in ALM.
Synchronization Status	The synchronization status.
Last Synchronization Time	The last synchronization time between SM and ALM.

Note: A requirement doesn't have status in ALM. Hence, the ALM Status field is not available for requirement synchronization.

2. Update some fields, add attachments, and then perform synchronization again from the SM Change to the ALM defect/requirement.
    1. Open the Change record again and then click the **Change Details** tab. Update the title, description, and severity information.
    2. Click the **Activities** tab. Select **Update to external service** in the **New Update Type** drop-down list, and then enter some notes in the **New Update** field.
    3. Click the **Attachments** tab and add some attachments to the Change record. Next, click the **ALM** tab and then click **Add** to select the attachments from the current Change record for ALM synchronization.
    4. Click **Save** and then wait for the next synchronization.
3. View the synchronization result in the ALM defect/requirement.

    Sign in to the ALM Desktop Client and open the defect/requirement record. Check that your updates to the title, description, and severity in the SM Change record are synchronized to the corresponding fields in the ALM defect/requirement record. Meanwhile, your notes in the SM **New Update** field now appear as comments in the ALM

defect/requirement record. Your selected attachments in the SM Change record now appear as the ALM defect/requirement record's attachments as well.

For complete use cases, see Synchronize Service Manager Change to ALM defect and Synchronize Service Manager Change to ALM requirement.

SM Problem → ALM defect

The following example verifies the integration by synchronizing an SM Problem to an ALM defect.

1. Synchronize an SM Problem to an ALM defect.
    1. Navigate to Problem Management, and then open a problem record.
    2. Click the **ALM** tab.
    3. To synchronize an SM Problem to an ALM defect, select **Problem To Defect** in the **ALM Action** drop-down list, and then select your ALM project (project alias) in the **ALM Project** drop-down list.
    4. Click **Save** and then wait for the background schedule to synchronize the Problem record to ALM. After synchronization, the ALM Id, ALM Type, ALM Status, ALM Last Update Time, Synchronization Status, and Last Synchronization Time fields on the **ALM** tab have read-only values that are populated automatically.

Field	Description
ALM Id	The defect ID in ALM.
ALM Type	Defect.
ALM Status	The defect's status in ALM.
ALM Last Update Time	The time when the defect is created in ALM for the first-time synchronization. **Note:** For the later synchronizations, this field displays the last time when the defect is updated by a user in ALM.
Synchronization Status	The synchronization status.
Last Synchronization Time	The last synchronization time between SM and ALM.

2. Update some fields, add attachments, and then perform synchronization again from the SM Problem to the ALM defect.

    1. Open the Problem record again and then update the title and description information.

    2. Click the **Activities** tab. Select **Update to external service** in the **New Update Type** drop-down list, and then enter some notes in the **New Update** field.

    3. Click the **Attachments** tab and add some attachments to the Problem record. Next, click the **ALM** tab and then click **Add** to select the attachments from the current Problem record for ALM synchronization.

    4. Click **Save** and then wait for the next synchronization.

3. View the synchronization result in the ALM defect.

    Sign in to the ALM Desktop Client and then open the defect record. Check that your updates to the title and description in the SM Problem record are synchronized to the corresponding fields in the ALM defect record. Meanwhile, your notes in the SM **New Update** field now appear as comments in the ALM defect record. Your selected attachments in the SM Problem record now appear as the ALM defect record's attachments as well.

For a complete use case, see Synchronize Service Manager Problem to ALM defect.

ALM defect → SM Problem

The following example verifies the integration by synchronizing an ALM defect to Service Manager and creating an SM Problem.

1. Create an SM Problem from an ALM defect.

    1. Sign in to the ALM Desktop Client and create a new defect record. Fill in the required fields, and then select **Y** in the **Push to SM Problem** drop-down list.

    2. Click **Submit** and then wait for the background schedule to synchronize the defect record to Service Manager. After synchronization, Service Manager creates a new Problem record.

    3. In Service Manager, click the **ALM** tab and then check that the ALM Action, ALM Project, ALM Id, ALM Type, ALM Status, ALM Last Update Time, Synchronization Status, and Last Synchronization Time fields have read-only values that are populated automatically.

    4. In ALM, check that the **Push to SM Problem** drop-down list in the defect record becomes read-only and displays **Done**. The SM ID and SM Table fields have read-only values that are populated automatically.

2. Update some fields and then perform synchronization again from the ALM defect to the SM Problem.

1. Open the defect record again and then update the summary, description, and severity information.
2. Save your updates and then wait for the next synchronization.

3. View the synchronization result in the SM Problem.

   In Service Manager, check that your updates to the summary, description, and severity information in the ALM defect record are synchronized to the corresponding fields in the Problem record.

1.6. Universal CMDB (UCMDB)

Universal CMDB (UCMDB) is a Configuration Management database for enterprise IT organizations to capture, document, and store information about configuration items (CIs), service dependencies, and relationships that support business services. When integrated with a Service Manager system, Universal CMDB stores the actual state of CIs and CI relationships, while the Service Manager system stores the expected or managed state of the CIs. Service Manager tracks any change requests, baselines, and historical updates CIs in the Universal CMDB.

The integration is on the basis of an integration adapter. The following sections describe how to establish a connection between UCMDB and Service Manager. Hence, this integration synchronizes configuration items (CIs) and CI relationships between the two systems and triggers appropriate actions in Service Manager.

1.6.1. Introduction

This section provides an overview of the Universal CMDB (UCMDB) - Service Manager (SM) integration (also referred to as the Universal CMDB (UCMDB) integration or UCMDB-SM integration in the Service Manager documentation).

Who should read the SM-UCMDB integration Documentation

Supported adapters

Purpose of the integration

How CI information is synchronized between UCMDB and Service Manager

Discovery Event Manager (DEM)

1.6.1.1. Who should read the SM-UCMDB integration documentation

The documentation is intended for a system implementer or system administrator who will be establishing and maintaining a connection between the UCMDB and Service Manager systems. The documentation assumes that you have administrative access to both systems.

1.6.1.2. Supported adapters

Service Manager can integrate with UCMDB through the following adapters:

- An XSLT-based adapter (ServiceManagerAdapter9.41)

  Note

  This adapter is supported for UCMDB 10.01 or later.

- An enhanced generic adapter (ServiceManagerEnhancedAdapter9.41)

  Note

  This adapter is based on the generic adapter framework introduced in UCMDB 10.20, and is therefore supported for UCMDB 10.20 or later only.

1.6.1.3. Purpose of the integration

An integration between Universal CMDB (UCMDB) and Service Manager enables you to share information about the actual state of a configuration item (CI) between your UCMDB system and a Service Manager system. CIs commonly include IT services, hardware, and software. Any organization that wants to implement the best practices Configuration Management and Change Management ITIL processes can use this integration to verify that CIs actually have the attribute values the organization has agreed to support.

You can use this integration to automate the creation of Service Manager change or incident records to update or roll back CIs that have unexpected attribute values. Service Manager allows you to programmatically define what actions you want to take whenever a CI's actual state doesn't match the expected state as defined in the CI record.

The integration offers several different ways for users to view CI actual state information:

- By default, the integration automatically updates the managed fields of Service Manager CI records as part of the regular UCMDB synchronization schedule. You can choose the option to configure the integration to automatically create change or incident records instead.
- A Service Manager user can view the current actual state of a CI by looking at the Actual State section in the CI record. When you open the Actual State section, Service Manager makes a web services request to UCMDB and displays all CI attributes the request returns. Service Manager only makes the web service call when you open this section.
- A Service Manager user can use the **View in UCMDB UI** option to view the current CI attributes from UCMDB.

1.6.1.3.1. Supported use cases

This section describes use cases that are supported by the UCMDB-SM integration. The supported use cases provide the core business processes that are enabled by the UCMDB-SM integration.

The following main business use cases are supported by the UCMDB-SM integration:

- Planned Change: A change created in Service Manager (SM) through the formal SM change process.
- Unplanned Change: A change or incident that occurred in SM and doesn't conform to the formal SM change process.
- Retrieving SM Incident, Change, and Problem record information: The ability to view SM Incident, Change and Problem record information in UCMDB.
- Actual State: The ability for SM users to view the actual state of UCMDB CIs.
- View CIs in UCMDB: The ability for SM users to view SM CI information in UCMDB or in UCMDB UI.
- Primary CI History in UCMDB: The ability for SM users to view UCMDB change history of the primary CI of a Problem record in SM for root cause investigation.
- Support of Automated Service Modeling (ASM) in UCMDB Browser: The ability that enables the integration to push consumer-provider relationships that are automatically discovered by ASM to SM, and enables SM users to access the ASM functionality of UCMDB Browser.
- Centralized VI visualization and impact analysis: The ability for SM users to access, directly from the SM user interface, the CI Visualization and Impact Analysis functionalities of UCMDB Browser.

All of the use cases provide important capabilities that enable the user to perform ITIL (IT Infrastructure Library) processes. The ITIL processes refer to a set of best practices that define and outline how organizations should manage their IT.

1.6.1.3.1.1. Enabling ITIL processes

By activating CI push from UCMDB to SM the user facilitates ITIL processes such as Incident, Problem and Change Management in SM.

SM utilizes the data pushed from UCMDB in the following modules:

- Incident Management: the Service Desk operator (SD Agent) selects the “Service” and the “Affected CI” for the specific Incident record.
- Problem Management: the SD agent selects the “Service” and the “Primary CI” for the specific Problem record.
- Change Management: the SD agent selects the “Service” and the “Affected CI(s)” for the specific Change record.

In each of the previously mentioned ITIL processes, SM utilizes CI information for Service, Affected CIs and Primary CIs that all originate in UCMDB.

1.6.1.3.1.2. Managing planned Changes

The purpose of the “planned Change” use case is to provide IT organizations a formal process by which changes to the IT infrastructure are introduced after thorough review and analysis. This is performed according to the “Change Management” process defined in ITIL v3.

A “planned Change” is initiated by the SM user through the formal “Change Management” process module in SM. This is followed by the actual change implementation.

The actual changes are discovered by a discovery tool such as DDMA, and then updated in UCMDB and the relevant modifications are pushed to SM. Once the user has validated the change, the user closes the relevant planned change in SM.

1.6.1.3.1.3. Managing unplanned Changes

The purpose of the “unplanned Change” use case is to provide IT organizations a formal process by which all changes that occur to the IT infrastructure are both logged and conventionalized through the organizations formal approval process.

An “unplanned Change” is a change that's recognized by a Discovery tool such as DDMA. The change is first updated and visible in UCMDB and then the data is pushed to SM. SM recognizes the change and as a result an “Incident” or “Change” record is generated.

These Changes are seen also in the SM “Pending Changes” section in the Configuration Item form, once approved they are moved to the SM “Historic Changes” section.

1.6.1.3.1.4. Retrieving Service Manager record information

Retrieving record information from within UCMDB provides all Software applications users with access to this information by using UCMDB's federation capabilities and supporting APIs. These applications include Asset Manager (AM), Operations Orchestration (OO), and so on.

SM record data is accessed from within UCMDB using UCMDB federation capabilities. SM record data includes Incident, Problem and Change records as well as a key set of their attributes.

UCMDB enables users to create reports or views that combine the federated record data from SM with CI information from UCMDB.

1.6.1.3.1.5. Retrieving the actual state of UCMDB CIs

By default, Service Manager only stores and displays the expected or managed state of CIs. The information Service Manager displays in the Configuration Management form is essentially the definitive list of attributes that the CI should have. However, the actual state of the CI may differ from the expected state.

The purpose of “Actual State” is to enable SM users insight into CIs’ current state as detected by “Discovery Tools” and populated in UCMDB. This state provides up-to-date information that may vary from the information displayed in SM both in content and in scope.

The “Actual State” of the CI is displayed in SM in order to enable the user to validate the current state of the CI that resides in UCMDB or in another data repository.

SM users retrieve the Actual State of CIs from UCMDB or additional data sources by viewing the CI's Actual State section in the SM Configuration Item form.

To view the actual state of the CI, you must first create an integration to a Universal CMDB server. The Universal CMDB server periodically discovers the actual state of CIs and records the actual state in the Configuration Management database. Service Manager accesses the actual state information by using a Web services connection. Service Manager sends the CI ID to the Universal CMDB server and receives a full list of the attributes for that CI. Service Manager displays the CI attributes in the Actual State section of the Configuration Management form.

If a Service Manager CI doesn't have a matching CI in the Universal CMDB server, Service Manager doesn't display the Actual State section. For example, you may track office furnishing CIs in Service Manager that can't be discovered and tracked in Universal CMDB.

To retrieve the actual state of a configuration item in Service Manager, follow these steps:

1. Click **Configuration Management** > **Resources** > **Search CIs**.
2. Use search or advanced search to find one or more records.
3. Select the CI with the actual state that you want to see.
4. From the CI detail form, open the **Actual State** section. Service Manager displays the actual state of the CI, as well as its UCMDB Id value.



Note

The **Actual State** section is visible only if your system has an active integration to a Universal CMDB server and the CI has a matching entry in the Configuration Management database. Service Manager doesn't display the **Actual State** section if there is no matching CI in the Universal CMDB server.

1.6.1.3.1.6. Accessing SM CI information in UCMDB

When integrated with UCMDB UI, Service Manager (SM) displays a **View in UCMDB UI** button and enables SM users to view CI information in UCMDB UI. You can click the **View in UCMDB UI** button to open the UCMDB UI in the context of the current CI.

1.6.1.3.1.7. Retrieving UCMDB primary CI change history in SM

When integrated with UCMDB UI, Service Manager displays a **Primary CI History in UCMDB** section in a Problem record whose primary CI is from UCMDB synchronization. You can view the CI changes on that primary CI for root cause investigation.

For information about how to enable an integration with UCMDB UI, see How to add UCMDB connection information.

To view the primary CI change history, follow these steps:

1. Sign in to Service Manager.
2. Navigate to **Problem Management**, and perform a search to open a Problem record whose Primary CI is from UCMDB synchronization.
3. Click the **Primary CI History in UCMDB** tab.

1.6.1.3.1.8. Support of Automated Service Modeling (ASM)

Service Discovery in UCMDB UI enables you to configure and run activities to discover your business services and applications. The discovery process starts from a CI that represents the URL of a specified service, and continues only with other CIs that can potentially be related to the service. This functionality is also referred to as Automated Service Modeling (ASM).

Note

ASM is supported only for UCMDB 10.20 or later.

Starting with Service Manager version 9.41, the Service Manager adapters support data push of consumer-provider relationships that are discovered by ASM. The adapters provide an ASM TQL query for push.

Note

Only consumer-provider relationships directly pointing from a Business Element CI to a Running Software CI can be pushed. Consumer-provider relationships between a Business Element CI and a Cluster CI can't be pushed.

Additionally, Service Manager can integrate with ASM so that SM users can access ASM in the context of an affected service in SM. This ability enables SM users to see the tree structure of a service CI and facilitates service modeling in SM.

Note

To enable ASM support, you still need to manually tailor related forms in Service Manager. For more information, see the *How to Enable ASM Support* topic.

1.6.1.3.1.9. Centralized CI visualization and impact analysis

Once Service Manager is integrated with UCMDB and UCMDB Browser, Service Manager users can access UCMDB UI directly from within Service Manager to visualize CI relationships and simulate the impact in the context of an affected service or configuration item (CI) in Service Manager.

If light-weight single sign-on (LW-SSO) is disabled for the integration, Service Manager users are prompted with UCMDB Browser login page; however, the integration provides a **Use a dedicated user account** option, which enables you to configure a dedicated UCMDB UI user account so that users bypass UCMDB UI login page.

Additionally, UCMDB Browser CI visualization and impact graphs are displayed according to the user's SM login language, which is English by default.

Important

To view UCMDB UI CI visualization and impact graphs embedded in Service Manager, users are recommended to use the latest Firefox or Chrome.

CI visualization

Only if an **SMtoUCMDB** integration instance is enabled in Service Manager (**Tailoring** > **Integration Manager**) and the **Show UCMDB UI CI Visualization widget** option is enabled, Service Manager uses the Get Related widget from UCMDB UI to visualize CI relationships. This enables users to use the same CI visualization on both product sides. For more information about the Get Related widget in UCMDB UI, see UCMDB Browser Help Center (version 4.10 or later).

Note

- For CIs that aren't synchronized to UCMDB, for example, non-discoverable CIs, to show CI visualization using the embedded Get Related UCMDB UI widget, it is recommended to set up an appropriate TQL script in UCMDB and then synchronize this kind of CIs to UCMDB.
- If there is no **SMtoUCMDB** integration instance enabled or the **Show UCMDB UI CI Visualization widget** option is disabled in the integration instance, Service Manager uses its own CI visualization.

Service Manager users can access UCMDB UI Get Related widget as described in the following table.

Module	How to access
Incident	1. Open an incident record. 2. Click the Smart Indicator icon for the following fields: ◦ **Primary Affected Service** ◦ **Affected CI** 3. On the Context Details form, click **Visualize CI**. The following figure shows an example CI visualization graph from UCMDB browser.
Interaction	1. Open an interaction record. 2. Click the Smart Indicator icon for the **Affected Service** field. 3. On the Context Details form, click **Visualize CI**.
Configuration	1. Open a configuration item record. 2. Click the **Relationship Graph** tab, or click **More** > **Visualize CI**. The following figure shows an example CI visualization graph from UCMDB browser.
Change or Change Task	1. Open a Change or Change Task record. 2. Click **More** > **Visualize CI**. 3. Select the affected service or an affected CI of the change record, and then click **Finish**.

Impact analysis

If an **SMtoUCMDB** integration instance is enabled in Service Manager (**Tailoring** > **Integration Manager**) and the **Show UCMDB UI Impact widget** option is enabled, Service Manager users

can access the Impact Simulation widget of UCMDB UI, as described in the following table. Pay attention to the following:

- The Impact Simulation widget uses a default impact severity that's defined in the **SMtoUCMDB** integration instance. The out-of-box value is Critical. For more information, see How to add UCMDB connection information.
- UCMDB UI uses blue lines and black lines to indicate different types of relationships. For more information, refer to the *Service Modeling* section in the *Universal CMDB Browser Help Center*.



Note

If there is no **SMtoUCMDB** integration instance enabled or if the **Show UCMDB UI Impact widget** option is disabled in the integration instance, the **Visualize Impact** button and **Run Impact Analysis** icon and menu option described in the following table aren't displayed.

Module	How to access
Incident	1. Open an incident record. 2. Click the Smart Indicator icon for the following fields: ◦ **Primary Affected Service** ◦ **Affected CI** 3. On the Context Details form, click **Visualize Impact** .
Interaction	1. Open an interaction record. 2. Click the Smart Indicator icon for the **Affected Service** field. 3. On the Context Details form, click **Visualize Impact**.

Module	How to access
Change or Change Task	1. Open a Change or Change Task record. 2. Click the **Run Impact Analysis** icon for the **Service** field or click **More** > **Run Impact Analysis**. 3. If a **Next** button appears, click **Next** to view the next impact analysis graph. **Note** Because a Change or Change Task record may have a number of affected CIs, the Impact Simulation result may need to be displayed on more than one page depending on the **Maximum Trigger CIs per Impact Graph** setting that's defined in the integration instance. If more than one page is needed, a **Next** button appears at the top of each impact graph page before the last one.

1.6.1.3.2. Core features

This section explains the rudimentary concepts behind the Federation, Push, and Population features as they pertain to the integration.

Topics in this section include:

Push

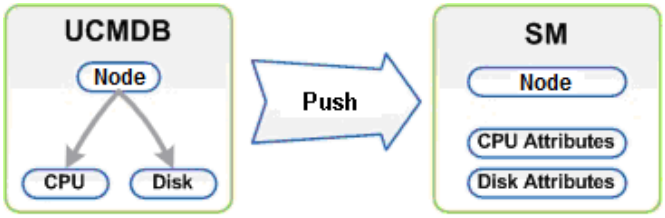
Federation

Population

1.6.1.3.2.1. Push

UCMDB can automatically discover most types of CIs available in Service Manager. This integration enables you to push these types of CIs from UCMDB to Service Manager.

The following figure shows how data is pushed from UCMDB to Service Manager (SM). The data is physically pushed (copied) from UCMDB to SM. Once the data is physically located in SM, the data is utilized by the SM user that consumes this information in various SM processes.



Note

CI Type and Attribute Push

Only information that's physically present in UCMDB can be pushed to SM.

1.6.1.3.2.2. Federation

With the federation feature, UCMDB pulls various ticket information (for example, Incident, Problem, Change, and Change Task ticket information) from SM. This enables users to see ticket information in UCMDB as ticket CIs that are connected to the relevant Nodes.

Note

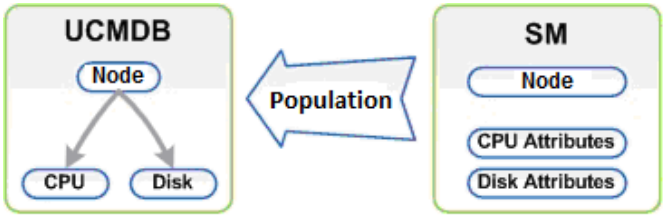
Change Task federation is supported only when the ServiceManagerEnhancedAdapter9.41 is used.

When data is federated (reflected or mirrored) from SM to UCMDB, the data is not physically present in UCMDB, instead it is passed over to UCMDB through Web Services.

1.6.1.3.2.3. Population

You can also use this integration to populate those types of CIs that UCMDB can't automatically discover or CIs that have been created in Service Manager before you have a UCMDB system deployed.

Population is the reverse of Push. The following figure shows how data is populated from SM to UCMDB. One SM CI record with multiple attributes is transferred to UCMDB as multiple CI records.



1.6.1.4. How CI information is synchronized between UCMDB and Service Manager

This section explains how CI information is transferred between the UCMDB and Service Manager systems.

This section includes:

- CI information usage
- High-level components of the integration
- Relationships between integration components
- What information is stored in UCMDB
- What information is stored in Service Manager

1.6.1.4.1. CI display name field in Service Manager

Prior to version 9.41, Service Manager used the CI Identifier field (logical.name field in the device table) to store Configuration Item (CI) names, and used the CI Name field (id field in the device table) to store CI identification numbers. When CIs were pushed from UCMDB to Service Manager, their names in UCMDB were synchronized to the logical.name field in Service Manager, and duplicate CI names were renamed because logical.name values must be unique in Service Manager.

Starting with version 9.41, Service Manager uses the logical.name field to store CI identification numbers, and a new field named Display Name (display.name in the device table) to store CI names. The original CI Name field (id field in the device table) is removed from the CI form (but still kept in the dbdict). CI display names must follow these rules:

- Duplicate Display Names are always allowed for CIs that are created from UCMDB.
- Duplicate Display Names aren't allowed for CIs that are created from Service Manager, unless they belong to different CI types.



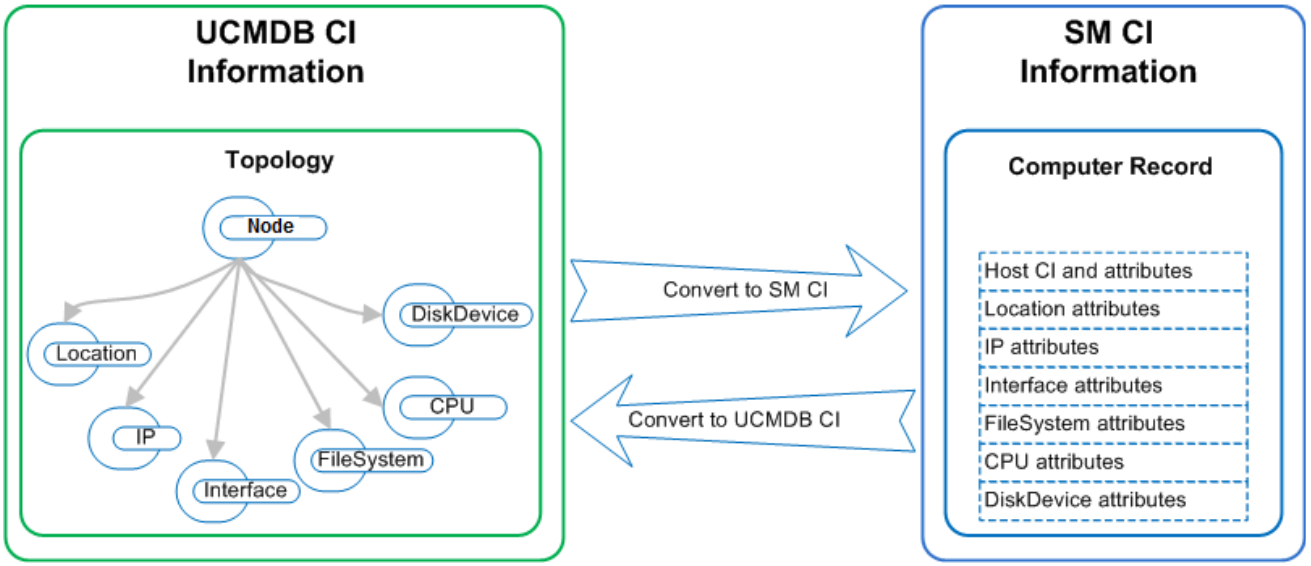
Note

Because Service Manager 9.41 or later allows duplicate CI names (as long as the CIs belong to different CI types), the automatic CI renaming mechanism is removed since Service Manager 9.41. After upgrading to Service Manager 9.41 or later, if you still need to push CI data to the CI Identifier (logical.name) field in Service Manager, you need to enable logical.name support first and make sure that each value to be pushed is unique. Otherwise, Service Manager can't create the CIs.

1.6.1.4.2. CI information usage

When referring to the concept of CI information it is important to make the distinction between a UCMDB CI and a Service Manager (SM) CI. The UCMDB model represents a topology that contains a number of CI types and relationships.

The UCMDB topology can be represented in Service Manager as a single entity. Multiple CIs from UCMDB and their attributes are merged into a single record in SM and the relevant UCMDB attributes are mapped to their appropriate counterparts in the SM record.



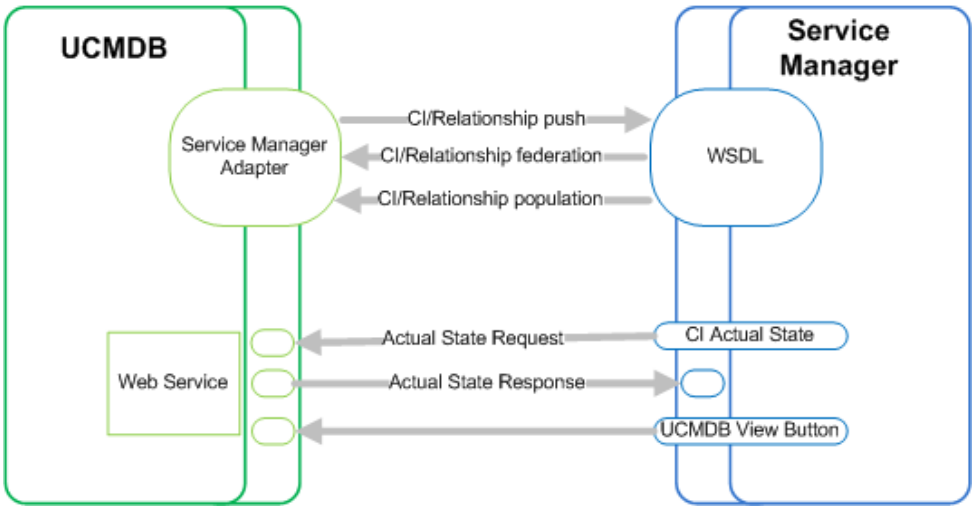
The above figure shows the correlation between the UCMDB topological model and its representation of the Computer Instance together with its parallel representation in SM. The SM computer CI contains all of the UCMDB information that's passed through the integration.

In the push flow, in the UCMDB topological view several CIs such as Node, IP, Interface, Location, File System, CPU, Disk Device and their Relationships are converted into a single SM computer record with the IP, MAC Address and Location, File System, CPU, and Disk Device attributes.

In the population flow, the conversion is reversed.

1.6.1.4.3. High-level components of the integration

The following diagram shows the high-level components of the UCMDB integration, and illustrates the interactions between UCMDB and Service Manager.

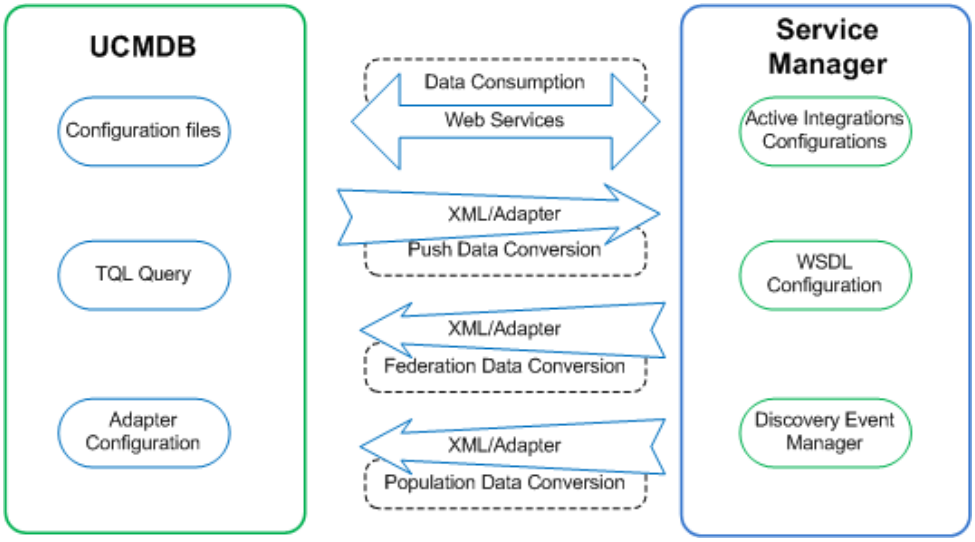


1.6.1.4.4. Relationships between integration components

The following figure illustrates the relationships between the Service Manager Adapter components in UCMDB and the associated components in Service Manager.

The Service Manager Adapter includes configuration files, which are used to map UCMDB entities to their counterparts in Service Manager during data push, as well as map Service Manager CIs to UCMDB entities during population.

The configuration files utilize UCMDB queries that define a superset of data relevant for the integration.



1.6.1.4.5. What information is stored in UCMDB

Your UCMDB system stores the actual state of CIs and CI relationships as CI attributes. Typically, UCMDB uses one or more integrations and discovery mechanisms (feeders) to automatically detect CI attribute values. The UCMDB-SM integration only uses a subset of the CI attributes available in a UCMDB system.

1.6.1.4.6. What information is stored in Service Manager

Your Service Manager system stores the managed or expected state of CIs and CI relationships as attribute values in a CI record. To be part of the integration, a CI attribute in your UCMDB system must map to a managed field in the Service Manager CI record. You can add, remove, or update the managed fields that are part of the integration by tailoring the Service Manager web services that manage the integration.

Service Manager runs according to a set of rules that define what actions you want the system to take whenever a CI's actual state doesn't match the expected state as defined in the CI record. You define these rules from the Discovery Event Manager (DEM) in Service Manager where you can do the following:

- Automatically update a CI record to match the attribute values listed in the actual state (This is the default behavior)
- Automatically create a change record to review the differences between the actual state and the managed state
- Automatically create an incident record to review the differences between the actual state and the managed state

1.6.1.5. Discovery Event Manager

The Discovery Event Manager tool provides you with information about configuration items (CIs) in use by your organization. Discovery Event Manager collects data from associated Web services, such as the Universal Configuration Management Database (UCMDB) for enterprise IT organizations. UCMDB captures, documents, and stores information about CIs, service dependencies, and relationships that support business services. The Discovery Event Manager tool takes the information captured by UCMDB and compares the actual state of each incoming CI record (both existing and new) to the managed field state of the CI record in Service Manager.

If the actual state of an incoming UCMDB CI record differs from the managed field state of the CI record in Service Manager, the Discovery Event Manager tool works with Configuration Management and Change Management to perform any required changes to the incoming CI record according to the rules you have set in the Discovery Event Manager tool.

Related topics

- Discovery Event Manager managed fields
- Discovery Event Manager rules
- Discovery Event Manager rule options
- Add a managed field in Discovery Event Manager
- Add a rule in Discovery Event Manager
- Customize Changes in Discovery Event Manager
- Customize Incidents in Discovery Event Manager
- View, modify, or delete a managed field in Discovery Event Manager
- View or modify rules in Discovery Event Manager

1.6.1.5.1. Discovery Event Manager rules

Discovery Event Manager (DEM) rules define DEM's actions when the actual state of an incoming configuration item (CI) record differs from the CI's managed state in Service Manager.

During initial configuration of Discovery Event Manager, you start with a basic set of rules for Change Management. You can refine those rules as necessary. Each rule identifies a series of checks that Discovery Event Manager processes whenever Service Manager receives a CI record from Web services. Depending on the rule, Discovery Event Manager may add or delete the record, open a change, log the information, or update the record's status.

Service Manager opens a change or incident based on the settings defined in the `populateChange` or `populateIncident` function in the `discoveryEvent` ScriptLibrary record. You can override the default settings by writing a custom JavaScript on the **Change Customization** or **Incident Customization** tab.

Related topics

- Discovery Event Manager
- Discovery Event Manager managed fields
- Discovery Event Manager rule options
- Add a managed field in Discovery Event Manager
- Add a rule in Discovery Event Manager
- Customize changes in Discovery Event Manager
- Customize incidents in Discovery Event Manager
- View, modify, or delete a managed field in Discovery Event Manager
- View, modify, or delete rules in Discovery Event Manager

1.6.1.5.1.1. Discovery Event Manager rule options

Rules help to automate the process of managing incoming configuration item (CI) records. When you have rules set up, you can manage whether a record is added, updated, or deleted to the CI records in Service Manager. The Discovery Event Manager tool checks the incoming CI records and determines what to do when the actual state of the CI records doesn't match the managed state of the CI records in Service Manager. When Discovery Event Manager finds the rule that applies to an incoming CI record type, the server checks the rule, and then updates the record according to the rules you have set up.

Example: If a user's machine has 4 GB of RAM added and the Discovery Event Manager tool discovers that the actual state of the CI record doesn't match the Service Manager managed state for that CI record type, the Discovery Event Manager tool opens an unplanned change. This then gives the Change Manager the opportunity to review the CI record and determine what tasks to complete.

Actions are required in the following cases:

- Records that don't exist.
- Records that contain unexpected data.
- Records are marked for deletion.

The following options are available when configuring rules to meet your business needs:

- **Action if matching record doesn't exist:** If a CI record doesn't exist in Service Manager.
  - **Add the record:** (Default) When the information received from Web services through Discovery Event Manager doesn't bring up a matching record in Service Manager, add the record.
  - **Add the record, and set dependency as true**: This option is available only for synchronization of CI relationship data. Service Manager will add the CI relationship record and enable outage dependency for the record by checking its **Outage Dependency** checkbox and setting its number of dependent downstream CIs to 1.
  - **Open an Incident:** Open an incident to investigate a new CI record that currently doesn't exist in Service Manager to determine if it is compliant with Service Manager.
  - **Open a Change:** Open an unplanned change to review the new CI record, because it currently doesn't exist in Service Manager. This change gives the Service Manager Administrator an opportunity to deny the Change Management request and back it out by using the change management process or to accept the actual state of the new CI record and assign tasks accordingly.
- **Action if record exists but unexpected data discovered:** Changes to the information in an existing CI record raise a flag for the Discovery Event Manager tool. Some of the information is not part of the managed state in Service Manager. The unexpected data in the CI record must be logged or reviewed.

  - **Open a Change:** (Default) Open an unplanned change to review the actual state of the CI record. This change gives the Service Manager Administrator an opportunity to deny the Change Management request and back it out by using the change management process, or to accept the actual state of the existing CI record and assign tasks accordingly.
  - **Log Results and update record:** Log the results of the actual state of the CI record, and then update the record.
  - **Open an Incident:** Open an Incident to investigate the actual state of a CI record and determine what actions must be performed or initiated to bring the record into compliance with Service Manager.
- **Action if record is to be deleted:** If an external event specifies that the record needs to be deleted.
  - **Delete record:** (Default for CI relationship records) This option is available for synchronization of both CI and CI relationship records. Service Manager automatically deletes the record.
  - **Open an Incident**: This option is available only for synchronization of CI relationship records. Service Manager opens an incident to investigate the deleted record and determines which actions must be performed or initiated to bring the record into compliance with Service Manager.
  - **Open a Change**: This option is available only for synchronization of CI relationship records. Service Manager opens an unplanned change to review the deleted record. The change allows someone to investigate whether the deleted record is compliant with your business practices. If the record is compliant, the change can be approved. If the record is not compliant, then the change can be denied and the record added back to the system.
  - **Update record to the selected status:** (Default) This option is available only for synchronization of CI records. Service Manager updates the status of the CI record to a value selected from the drop-down list (for example, **Retired/Consumed**), instead of deleting the record permanently.
  - **Open an Incident to update record to the selected status:** This option is available only for synchronization of CI records. Service Manager opens an incident to update the record's status to a value selected from the drop-down list (for example,**Retired/Consumed**). Once the incident has been closed, Service Manager automatically updates the CI record to the selected status.
  - **Open a Change to update record to the selected status:** This option is available only for synchronization of CI records. Service Manager opens an unplanned change to update the CI record's status to a value selected from the drop-down list (for example, **Retired/Consumed**). The change allows someone to investigate whether the requested status change is compliant with your business practices. Once the change has been approved and closed, Service Manager automatically changes the CI record to the selected status. If the change has been denied, Service Manager makes no changes to the CI record.

Related topics

Discovery Event Manager
Discovery Event Manager change open process
Discovery Event Manager managed fields
Discovery Event Manager rules
Add a managed field in Discovery Event Manager
Add a rule in Discovery Event Manager
Customize changes in Discovery Event Manager
Customize incidents in Discovery Event Manager
View, modify, or delete a managed field in Discovery Event Manager
View, modify, or delete rules in Discovery Event Manager

1.6.1.5.1.2. Add a rule in Discovery Event Manager

Applies to User roles: System Administrator

Rules are the core of Discovery Event Manager processing. Based on your organization's requirements, you can refine the basic set of rules that are configured with the Discovery Event Manager tool by adding rules.

To add a rule:

1. Click **Tailoring** > **Web Services** > **Discovered Event Manager Rules**. The Discovery Event Manager form opens.
2. Click **New**. The new rule form opens.
3. Enter the new rule name.
4. Select a table to associate with the rule from the **Table Name** list, and then click **Next**.
5. Enter the condition for the rule. The system adds the rule to the records table.

   Note

   • The condition must ensure that only one rule applies when processing the web service request.
   • An empty condition evaluates to true by default.

6. Click **Save**.
7. Click **OK**.

Related topics

- Discovery Event Manager
- Discovery Event Manager change open process
- Discovery Event Manager managed fields
- Discovery Event Manager rules
- Discovery Event Manager rule options
- Add a managed field in Discovery Event Manager
- Customize changes in Discovery Event Manager
- Customize incidents in Discovery Event Manager
- View, modify, or delete a managed field in Discovery Event Manager
- View, modify, or delete rules in Discovery Event Manager

1.6.1.5.1.3. View, modify, or delete rules in Discovery Event Manager

Applies to User roles: System Administrator

Rules help you to automate the change control process and update the incoming configuration item (CI) records to comply with the CI record fields in Service Manager. As you reevaluate your organization's requirements, you may view the rules that are set up and make changes as you see fit. In addition, when the existing rules settings for a CI type are no longer valid, you can replace the existing rules settings with a new set of rules.

Warning: Make sure you delete the rules for the CI ID type you want to delete. If you delete the wrong set of rules, you have to add the CI ID type and set up the rules for each action as necessary.

To view or modify existing rules:

1. Click **Tailoring** > **Web Services** > **Discovered Event Manager Rules**. The Discovery Event Manager form opens.
2. Click **Search** to retrieve a list of CI ID types.
3. Select the ID type for the rules you want to view.
4. Select the **Rules** tab. The existing rules settings for the selected CI type are displayed.
5. If you want to edit the rule, do the following:
    - Make the necessary changes. For example, if you choose to select a different action step for records that don't exist, make your change.
    - Click **Save**.
6. Click **OK**.

To delete a set of rules:

1. Click **Tailoring** > **Web Services** > **Discovered Event Manager Rules**. The Discovery Event Manager form opens.
2. Click **Search** to retrieve a list of CI ID types.
3. Select the CI ID type, and then select the **Rules** tab. Existing rules settings for the selected CI type are displayed.
4. Delete the rules for this CI type as required.
5. Click **OK**.

Related topics

- Discovery Event Manager
- Discovery Event Manager managed fields
- Discovery Event Manager rules
- Discovery Event Manager rule options
- Add a managed field in Discovery Event Manager
- Add a rule in Discovery Event Manager

- Customize changes in Discovery Event Manager
- Customize incidents in Discovery Event Manager
- View, modify, or delete a managed field in Discovery Event Manager

1.6.1.5.1.4. Customize Changes in Discovery Event Manager

Applies to User roles: System Administrator

You may have an incoming configuration item (CI) record that isn't compliant with the managed state of that CI record in Service Manager go through the Change Management process. If this is your best practice, you can customize how you want the Discovery Event Manager tool to process those incoming CI records.

To customize the way changes are handled within Discovery Event Manager:

1. Click **Tailoring** > **Web Services** > **Discovered Event Manager Rules**. The Discovery Event Manager form opens.
2. Click **Search** to retrieve a list of Configuration Item (CI) ID types.
3. Select the ID type for the customization you want to make, and then select the **Change Customization** tab.
4. Enter the customized script you want to override the default values set by the `discoveryEvent` ScriptLibrary record (You may refer to the change record as "change").
5. Click **Save**.
6. Click **OK**.

Related topics

- Discovery Event Manager
- Discovery Event Manager managed fields
- Discovery Event Manager rules
- Discovery Event Manager rule options
- Add a managed field in Discovery Event Manager
- Add a rule in Discovery Event Manager
- Customize Incidents in Discovery Event Manager
- View, modify, or delete a managed field in Discovery Event Manager
- View, modify, or delete rules in Discovery Event Manager

1.6.1.5.1.5. Customize Incidents in Discovery Event Manager

Applies to User roles: System Administrator

You may have an incident logged when an incoming configuration item (CI) record isn't compliant with the managed state of a CI record in Service Manager. If this is your best practice, you can customize the rules you set for automatically opening an incident record.

To customize the JavaScript for opening an incident record:

1. Click **Tailoring** > **Web Services** > **Discovered Event Manager Rules**. The Discovery Event Manager form opens.
2. Click **Search** to retrieve a list of Configuration Item (CI) ID types.
3. Select the ID type for the customization you want to make to incidents, and then select the **Incident Customization** tab.
4. Enter the customized script you have created to override the default values set by the `discoveryEventScriptLibrary` record (You may refer the incident record as "incident").
5. Click **Save**.
6. Click **OK**.

Related topics

- Discovery Event Manager
- Discovery Event Manager managed fields
- Discovery Event Manager rules
- Discovery Event Manager rule options
- Add a managed field in Discovery Event Manager
- Add a rule in Discovery Event Manager
- Customize Changes in Discovery Event Manager
- View, modify, or delete a managed field in Discovery Event Manager
- View, modify, or delete rules in Discovery Event Manager

1.6.1.5.2. Discovery Event Manager managed fields

Managed fields are key fields in the Service Manager configuration item (CI) record types that the Discovery Event Manager tool uses to validate the incoming CI records from Web services. By default, the fields of the incoming CI records should match the key fields in the Service Manager CI records.

Suppose Discovery Event Manager has discovered some discrepancies between the actual state of the incoming CI record fields and the Service Manager managed fields. In this case, Change Management handles these discrepancies by default. Rules define what to do with a CI record to make it compliant with the CI record types in Service Manager.

Related topics

- Discovery Event Manager
- Discovery Event Manager rules
- Discovery Event Manager rule options
- Add a managed field in Discovery Event Manager
- Add a rule in Discovery Event Manager
- Customize changes in Discovery Event Manager
- Customize incidents in Discovery Event Manager
- View, modify, or delete a managed field in Discovery Event Manager
- View, modify, or delete rules in Discovery Event Manager

1.6.1.5.2.1. Add a managed field in Discovery Event Manager

Managed fields are key fields in Service Manager configuration item (CI) record types. They are important to helping the Discovery Event Manager tool to discover differences between the data in the incoming CI records from Web services versus what Service Manager is expecting to receive in those records. If the incoming records don't match the managed key fields in Service Manager, the rules set in the Discovery Event Manager tool determine what will happen to those incoming CI records.

For example: If the joinbizservice table has three fields in the Managed Fields tab in Discovery Event Manager and some of the expected information is different or missing from any of those fields, the Discovery Event Manager tool will deal with that incoming CI record that's not compliant with the Service Manager managed fields for that CI record type.

If you determine that a necessary field is not included in the existing discovery process, you can add a managed field to the applicable table using the settings on the **Managed Fields** tab.



Note

If a managed field doesn't exist in an incoming CI record, the Discovery Event Manager tool is not able to find discrepancies based on that field.



Tip

The following steps describe how to manually add a managed field. If you want to automatically add multiple or all fields from a table as managed fields, you can click the **Load Fields** button on the **Managed Fields** tab.

To add a managed field:

1. Click **Tailoring** > **Web Services** > **Discovered Event Manager Rules**. The Discovery Event Manager rules form is displayed.
2. Click **Search** to retrieve a list of CI ID types.
3. Select the ID type for the fields you want to view, and then select the **Managed Fields** tab. You can see which fields are being managed for this CI type.
4. To select a new field you want managed, do the following:

    a. Select the drop-down arrow in the next blank **Field Name** field. You will see the list of fields related to this CI type.

    b. Select the field you want to add to the **Managed Fields** tab for this CI type.

c. For a field that's part of an array of structures, choose a **Structure** and specify the field's **Index** value. For example, the CI type computer contains array elements, such as ports, printers, and scanners. Choose an array element within the structure, and then choose the corresponding index number.

d. Click **Save**.

5. Repeat this process to add a managed field for another ID type.
6. Click **OK**.

Related topics

Discovery Event Manager
Discovery Event Manager managed fields
Discovery Event Manager rules
Discovery Event Manager rule options
Add a rule in Discovery Event Manager
Customize changes in Discovery Event Manager
Customize incidents in Discovery Event Manager
View, modify, or delete a managed field in Discovery Event Manager
View, modify, or delete rules in Discovery Event Manager

1.6.1.5.2.2. View, modify, or delete a managed field in Discovery Event Manager

Applies to User roles: System Administrator

Managed fields are important to ensure that the Discovery Event Manager tool can process incoming configuration item (CI) data from Web services. When the inventory of your organization changes, you can add or update the CI types based on your new inventory requirements. You can also review the managed fields for existing CI types to decide if the necessary fields exist and add, modify, or delete managed fields.

Warning: If you are deleting a field, make sure you select the field you want to delete. If you delete the wrong field, you must add the field back into the Managed Fields tab.

To view, modify, or delete a managed field:

1. Click **Tailoring** > **Web Services** > **Discovered Event Manager Rules**. The Discovery Event Manager form opens.
2. Click **Search** to retrieve a list of CI ID types.
3. Select the ID type for the fields you want to view, and then select the **Managed Fields** tab. The system displays a list of managed fields for the selected CI type.
4. Select the field you want to edit or delete. Make any necessary changes or delete a field that's no longer valid. Click **Save**.
5. Click **OK**.

Related topics

- Discovery Event Manager
- Discovery Event Manager managed fields
- Discovery Event Manager rules
- Discovery Event Manager rule options
- Add a managed field in Discovery Event Manager
- Add a rule in Discovery Event Manager
- Customize changes in Discovery Event Manager
- Customize incidents in Discovery Event Manager
- View, modify, or delete rules in Discovery Event Manager

1.6.1.5.3. Discovery Event Manager change open process

You can use the Discovery Event Manager tool to automate the change open process. When a change is needed to address issues for review or tasks to be performed, the change open process begins. The change goes through phases that drive the unplanned change through the necessary tasks to update the managed fields of a CI or to determine if the change is necessary or desired. To see an Unplanned Change flow diagram, see the related topics.

The change phases are as follows:

Discovery Assessment phase: Assess whether or not an unplanned change is allowed. Comments can be added and updated in this phase, as needed. The CCB Approver must confirm that the change is desired.

- If the change is approved, update the managed field and close the change.
- If the change is denied, move it to the Discovery Backout phase to undo the change.

Discovery Backout phase: Back out a change after determining that the change is unnecessary or desired. Comments can be added and updated in this phase, as needed, including updates to modified data. If the backout of the change is complete, move it to the Discovery Verification phase.

- If the denied change is verified, close the change.
- If the denied change is not verified, return to the Discovery Backout phase.

Discovery Implementation phase: The change requires further consideration before it is closed.

- If the backout of the change fails verification, return to the Discovery Implementation phase for further consideration.
- If verification of the change succeeds, close the change.

Discovery Verification phase: Verify that the change is successfully implemented.

- If verification of the change fails, return to the Discovery Implementation phase.
- If verification of the change succeeds, close the change.

Related topics

Discovery Event Manager
Discovery Event Manager managed fields
Discovery Event Manager rules
Discovery Event Manager rule options
Add a managed field in Discovery Event Manager
Add a rule in Discovery Event Manager
Customize changes in Discovery Event Manager

Customize incidents in Discovery Event Manager
View, modify, or delete a managed field in Discovery Event Manager
View, modify, or delete rules in Discovery Event Manager

1.6.2. Using the ServiceManagerEnhancedAdapter9.41 adapter

This section describes how to implement an integration between Service Manager and UCMDB by using the ServiceManagerEnhancedAdapter9.41 adapter.

How to upgrade your integration

Integration setup

Multi-tenancy (Multi-company) setup

Standards and best practices

Tailoring the integration

1.6.2.1. How to upgrade your integration

This section describes the upgrade steps required to use the ServiceManagerEnhancedAdapter9.41 adapter with Service Manager, including steps to adapt the integration to the logical name solution introduced in Service Manager 9.41 and steps to migrate from an XSLT-based adapter to a generic adapter.

The logical name solution contains significant CI related code changes. For example, prior to version 9.41, Service Manager used the **logical.name** field in the **device** table to store CI names. The logical.name field must be unique in the Service Manager system, but UCMDB allows duplicate CI names. As a result, CIs may have to be renamed when they are pushed from UCMDB to Service Manager. Starting with version 9.41, Service Manager stores CI names in the **display.name** field in the **device** table, and stores unique CI identification numbers in the **logical.name** field. The **display.name** field allows duplicate values if they are pushed from UCMDB, and requires unique values for CIs that are created from Service Manager (unless they're not of the same CI type).

Before you proceed to the upgrade steps described in this section, make sure you have a good understanding of the logical name solution and its impact on the SM-UCMDB integration. For details, see the Service Manager Logical Name Solution white paper.

Perform the following tasks to upgrade your integration to use the ServiceManagerEnhancedAdapter9.41 adapter.

Task 1. Close All SM unplanned Changes and Incidents created for CI modifications

Task 2. Upgrade Service Manager

(Optional) Task 3. Copy data to the joinnode table

Task 4. Update the DEM rules in Service Manager

Task 5. Enable the Restful API for custom CI types in Service Manager

Task 6. Upgrade UCMDB

Task 7. Upgrade the configurations for custom CI types in UCMDB

Task 8. Upgrade field mapping

Task 9. Reconfigure an integration point

Task 10. Run a full push to resynchronize CI names

Task 11. Restore the DEM rules in Service Manager

1.6.2.1.1. Task 1. Close all SM unplanned Changes and Incidents created for CI modifications

Before you upgrade Service Manager (SM), close all unplanned Change and Incident records that were created due to CI modifications. After the system upgrade, you will run a full data push to resynchronize CI names from UCMDB to the **display.name** field in the Service Manager **device** table.

1.6.2.1.2. Task 2. Upgrade Service Manager

If you want to use the ServiceManagerAdapter9.41 adapter for the integration, you must upgrade Service Manager to version 9.41 or later, which uses display.name instead of logical.name for field mapping between UCMDB and Service Manager.

Note

The upgrade includes the SM server, Windows and web clients, applications, and Mobility client. If you upgrade only the server and clients and your applications version is still earlier than 9.41, you can only use an old adapter rather than any of the Service Manager 9.41 adapters.

For information about how to upgrade Service Manager, see Upgrade.

1.6.2.1.3. (Optional) Task 3. Copy data to the joinnode table

Service Manager uses the **joinnode** table to replace the **mainframe**, **networkcomponents**, **storage**, and **computer** tables. If you are upgrading from Service Manager 9.31 or earlier and want to migrate to the joinnode-based data mappings, you need to copy data to the **joinnode** table.

Note

If you want to use the original tables instead of joinnode, this task is not needed. Instead, you need to modify the data mappings of your Service Manager adapter in UCMDB.

To copy your data to the joinnode table, follow these steps:

1. Sign in to Service Manager as a system administrator.
2. Open Database Manager
3. Enter cidatacopy in the Table field, and then click **Search**. The Copy CI Type Data form opens.
4. Click **Search**. A list of records is displayed: **joincomputer**, **joinmainframe**, **joinnetworkcomponents**, and **joinstorage**.
5. Select each record from the list, update or add source/target fields as needed, and click the **Copy Data** button. One of the following messages is displayed:
   - A message that resembles “<XXX> records were successfully copied, and <YYY> records were ignored within xxx ms”, where XXX and YYY represent the numbers of records copied and ignored, respectively.
   - “Configuration validation failed, and no records were copied. Please check your Source Fields/Target Fields settings and run Copy Data again.”
6. If the error message is displayed, correct your Source Fields/Target Fields settings, and click **Copy Data** again until all of your records have been successfully processed.

1.6.2.1.4. Task 4. Update the DEM rules in Service Manager

Service Manager 9.41 or later uses an updated CI form which may rename CI names in the **logical.name** field. Hence, you may want to correct these CI names and update the CI Display Name (**display.name**) field values in SM by using the new field mapping.

To do this, you need to run a full data push to resynchronize the CIs to Service Manager. Before doing so, you must update the Discovered Event Manager (DEM) rules to avoid creating unwanted Change or Incident records due to the CI name changes.

1. Sign in to Service Manager as a System Administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Click **Search**.
4. For each DEM rule record, select **Log Results and update record** for **Action if record exists but unexpected data discovered**.

1.6.2.1.5. Task 5. Enable the Restful API for custom CI types in Service Manager

If your existing UCMDB-SM integration environment uses any custom CI types, you need to update the External Access Definition of each custom CI type in Service Manager to enable its RESTful API for using a generic adapter. A Service Manager generic adapter requires the ucmdbIntegration RESTful APIs to work.

The following table describes the parameters that you need to update in each External Access Definition in Service Manager (**System Administration** > **Tailoring** > **Web Services** > **External Access Definition**).

Parameter	Description	Example Value
RESTful Enabled	Enables the RESTful API (must be set to true to enable the RESTful API)	true
Resource Collection Name	Specifies a unique name for the resource collection. The following convention is recommended: <WS Object Name>+'s'	ucmdbATMs
Resource Name	Specifies a name for the resource. The following convention is recommended: <WS Object Name>	ucmdbATM
Unique Keys	Unique keys of the relevant device table	logical.name
Max Records Returned in Query	Maximum number of returned records in one query	1000
Resource Collection Actions - POST	Action to invoke for a POST request on the resource collection	Create
Resource Actions - POST	Action to invoke for a POST request on the resource	Create
Resource Actions - PUT	Action to invoke for a PUT request on the resource	Update
Resource Actions - DELETE	Action to invoke for a DELETE request on the resource	Delete

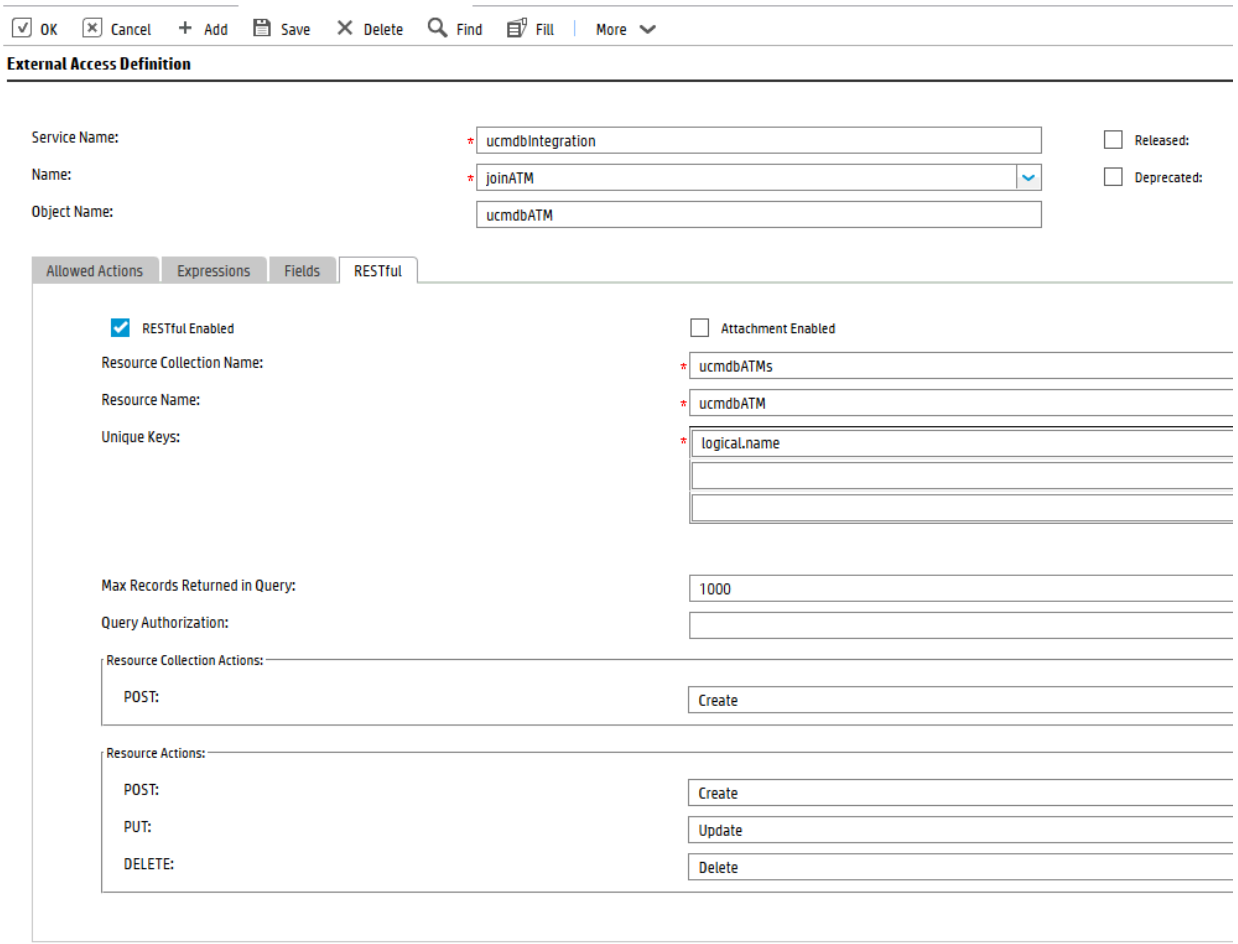
To access these parameters, follow these steps:

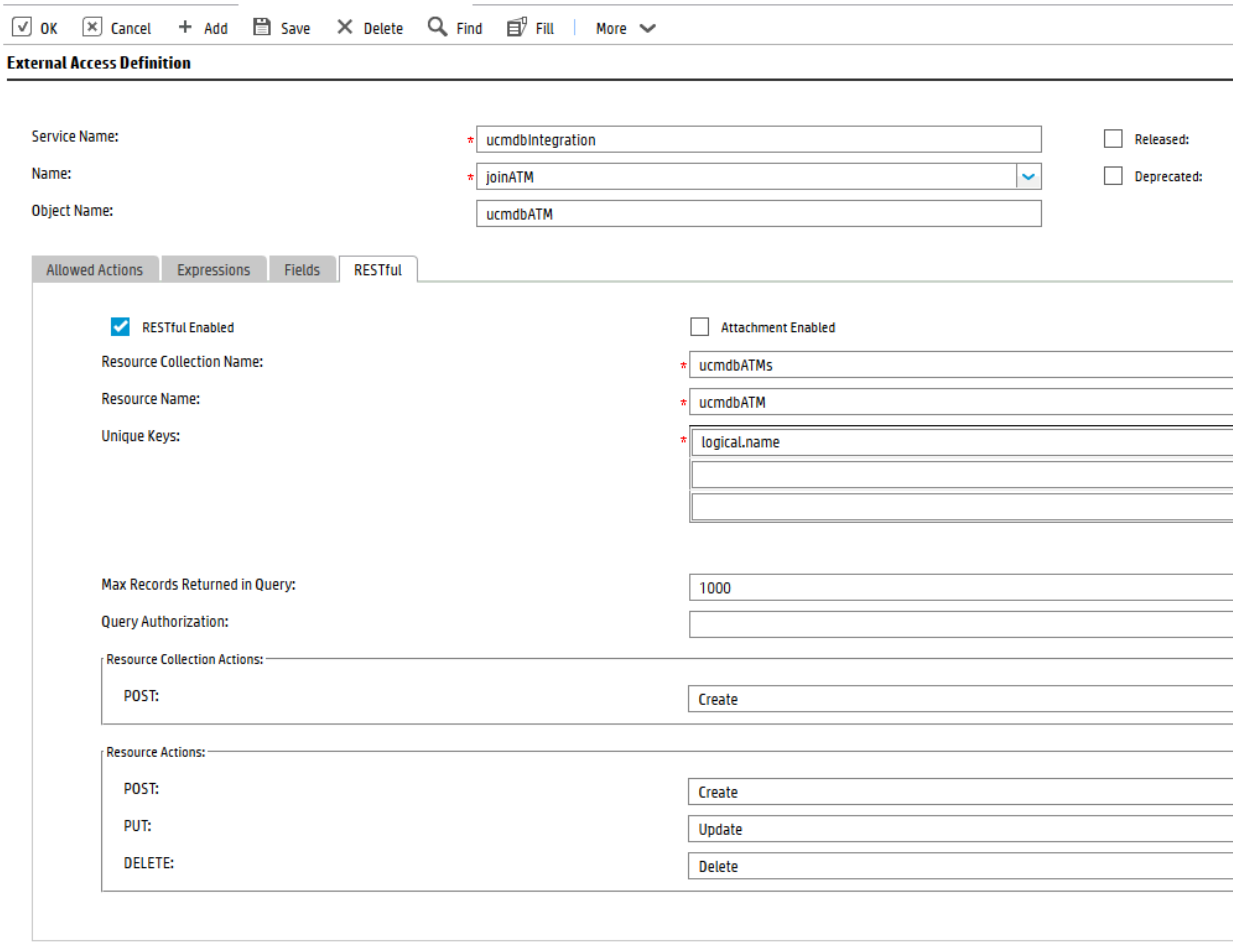
1. Open the External Access Definition for the custom CI type.
    1. Click **Tailoring > Web Services** > **Web Service Configuration** to open the External Access Definition form.
    2. In the Service Name field, type ucmdbIntegration.

3. In the Name field, select the table for the custom CI type.
4. In the Object Name field, type the relevant web service object name.
5. Click **Search**.

2. Click the **Restful** tab, and update the parameters.

   As an example, the following figure shows the external access definition of a custom CI type named ATM Machine.




OK
Cancel
Add
Save
Delete
Find
Fill
More
External Access Definition
Service Name:
ucmdbIntegration
Released:
Name:
joinATM
Deprecated:
Object Name:
ucmdbATM
Allowed Actions
Expressions
Fields
RESTful
RESTful Enabled
Attachment Enabled
Resource Collection Name:
ucmdbATMs
Resource Name:
ucmdbATM
Unique Keys:
logical.name
Max Records Returned in Query:
1000
Query Authorization:
Resource Collection Actions:
POST:
Create
Resource Actions:
POST:
Create
PUT:
Update
DELETE:
Delete

1.6.2.1.6. Task 6. Upgrade UCMDB

Service Manager 9.41 or later can work with XSLT-based adapters and generic adapters specifically designed for Service Manager 9.41 or later:

- ServiceManagerAdapter9.41 for UCMDB 10.01, 10.11, 10.20, 10.21, or 10.30

  Note

  These are four XSLT-based adapters, one for each of the UCMDB versions.

- ServiceManagerEnhancedAdapter9.41 for UCMDB 10.20, 10.21, or 10.30

  Note

  These are two generic adapters based on the UCMDB generic framework, one for each of UCMDB 10.20 and 10.21.

For more information, see Integration Requirements.

1.6.2.1.7. Task 7: Update the configurations for custom CI types in UCMDB

The configurations for all out-of-the-box CI types in UCMDB 10.20 are already based on the Service Manager Enhanced Generic Adapter. However, you still need to perform the following steps to manually update the configurations for your custom CI types in UCMDB.

Step 1. Convert the mapping scripts from XSLT to XML and Groovy.

The old Service Manager adapters use XSLT scripts for field mapping between UCMDB and SM. Most of the mapping scripts were developed for the four typical mapping scenarios described in the following table.

Scenario	Notes
One to One field mapping	• One field in one side is mapped to one field in the other side • The value is synchronized between UCMDB and SM directly (without value conversion)
One to Many field mapping	• One field in one side is mapped to many fields in the other side • The value of the one field is calculated based on the values of many fields
Many to Many field mapping	• Multiple fields are defined as a child CI type of the relevant CI type • A single record has multiple child instances
Value Conversion	• The value of one field is converted to another value based on a specific algorithm • Value conversion can happen on all kinds of field mappings (one to one, one to many, and many to many)

The Service Manager Enhanced Generic Adapter uses XML and Groovy mapping scripts. Old mapping scripts for the out-of-the-box CI types have been converted to XML and Groovy by default. However, you still need to convert your existing custom scripts to XML and Groovy.

To convert a custom mapping script, you need to identify its mapping scenario (One to One, One to Many, Many to Many, or Value Conversion) first; then you can convert the script by referring to the following sample scripts for your specific scenario.

Tip

In all mapping scenarios, you can use the Visual Mapping tool to generate an XML script skeleton. Compared with the use of an old XSLT adapter, this is much easier as you can drag and drop the mapped fields to generate the script instead of writing every code line from scratch. For details about how to use XML and Groovy scripts in the UCMDB integration framework, see the *Universal CMDB 10.20 Developer Reference Guide.*

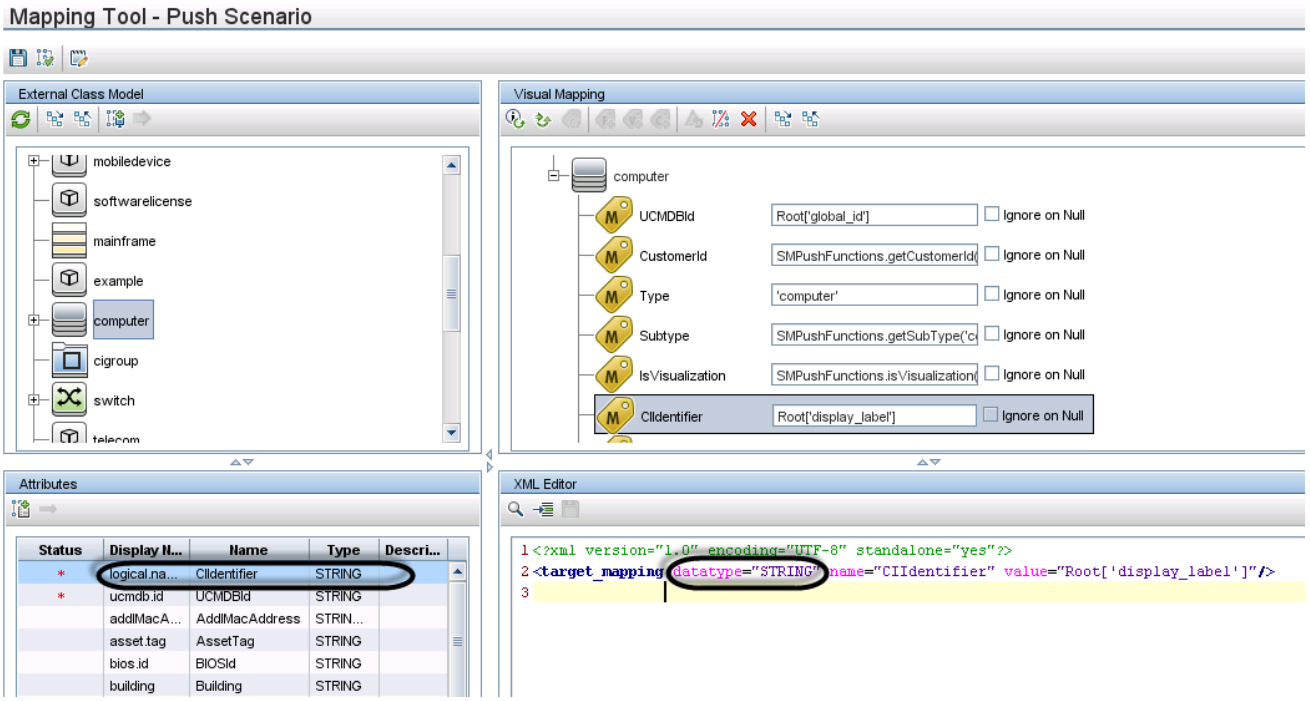
Tip

The Service Manager Enhanced Generic Adapter has four out-of-the-box Groovy scripts, which you can use as a reference:

- SMUtils.groovy: defines common methods used for push, population, and federation.
- SMPushFunctions.groovy: defines the methods used in the out-of-the-box push mapping scripts.
- SMPopulateFunctions.groovy: defines the methods used in the out-of-the-box population mapping scripts.
- SMFederationFunctions.groovy: defines the methods used in the out-of-the-box federation mapping scripts.

Additionally, the SMFederationConverter.groovy script defines value conversions for federation.

Field mapping must use the correct data type for each attribute. It is convenient to determine the data type for an attribute by using the Visual Mapping tool. The following figure illustrates the data type for the CIIdentifier attribute. You can either find the data type from the External Class Model Attributes pane or simply drag and drop the attribute from that pane to the Visual Mapping area to automatically get the correct data type.



One to One Field Mapping

Sample Script in XSLT:

```
<Type>computer</Type>
```

Or

```
<UCMDBId><xsl:value-of select="@id"/></UCMDBId>
```

Or

```
<xsl:for-each select="@display_label">
<DisplayName><xsl:value-of select="."/></DisplayName>
```

```
</xsl:for-each>
```

Sample Script in XML (Groovy is not required in this scenario):

```
<target_mapping datatype="STRING" name="DisplayName"
value="Root['display_label']"/>
```

Where: The **datatype** must be the correct data type defined in Service Manager for the attribute. You can find the data type from the Visual Mapping tool interface.

One to Many Field Mapping

Sample Script in XSLT:

```
<xsl:variable name="prefix" select="'Value&gt;'"/>
<xsl:variable name="suffix" select="'&lt;/Value'"/>
<Subtype>
<xsl:choose>
<xsl:when
test="contains(@node_role,concat($prefix,'desktop',$suffix))">Desktop</
xsl:when>
<xsl:when test="@os_family">
<xsl:value-of select="@os_family"/>
</xsl:when>
<xsl:otherwise>Server</xsl:otherwise>
</xsl:choose>
</Subtype>
```

Sample Script in XML:

```
<target_mapping datatype="STRING" name="Subtype"
value="SMPushFunctions.getSubType('computer',Root['node_role'],Root['os
_family'])"/>
```

Sample Script in Groovy:

You must develop a groovy function to implement the mapping logic. Usually the relevant fields in the counterpart system will be used as the parameters of the Groovy function.

Many to Many Field Mapping

Sample Script in XSLT:

```
<xsl:for-each select="cpus">
<cpu>
<xsl:for-each select="cpu">
<cpu>
<CpuID><xsl:value-of select="@cpu_id"/></CpuID>
<CpuName><xsl:value-of select="@name"/></CpuName>
<CpuClockSpeed>
<xsl:value-of select="@cpu_clock_speed"/>
</CpuClockSpeed>
</cpu>
</xsl:for-each>
</cpu>
</xsl:for-each>
```

Sample Script in XML:

```
<for-each-source-entity count-index="i" source-entities="Root.Cpu">
<target_entity name="cpu">
<target_mapping datatype="STRING" name="CpuID" value="Root.Cpu[i]
['cpu_id']"/>
<target_mapping datatype="STRING" name="CpuName" value="Root.Cpu[i]
['name']"/>
<target_mapping datatype="STRING" name="CpuClockSpeed"
value="Root.Cpu[i]['cpu_clock_speed']"/>
</target_entity>
</for-each-source-entity>
```

Where: <*source-entities*> (**Root Cpu**) comes from the local TQL query structure, while <*target_entity name*> (**cpu**) comes from the <cpu> tag in the XSLT file.



Note

Groovy scripts aren't required for this scenario.

Value Conversion

Sample Script in XSLT:

```
<xsl:if test="@customer_id">
<xsl:variable name="ucmdbCustomerId" select="@customer_id"/>
<xsl:variable name="tenantMappingEntry"
select="document('SM_MT_mapping.xml')/list['TenantMapping']/entry[@ucmd
b=$ucmdbCustomerId]"/>
<xsl:choose>
<xsl:when test="$tenantMappingEntry">
<CustomerId><xsl:value-of select="$tenantMappingEntry/@sm"/>
</CustomerId>
</xsl:when>
<xsl:otherwise>
<CustomerId><xsl:value-of select="@customer_id"/></CustomerId>
</xsl:otherwise>
</xsl:choose>
</xsl:if>
```

Or

```
<xsl:if
test="contains(@node_role,concat($prefix,'virtualized_system',$suffix))
">
<IsVisualization>true</IsVisualization>
</xsl:if>
```

Sample Script in XML:

```
<target_mapping datatype="STRING" name="CustomerId"
value="SMPushFunctions.getCustomerId(CustomerInformation)"/>
```

Or

```
<target_mapping datatype="BOOLEAN" name="IsVisualization"
value="SMPushFunctions.isVisualization(Root['node_role'])"/>
```



Note

Usually the relevant field in the counterpart system will be used as one of the parameters of the Groovy function.

Sample Scripts in Groovy:

```
/**
* Priority Mapping
* [uCMDB value : SM value ]
*/
private static final def PriorityMapping = [
"1_critical":"1",
"2_high":"2",
"3_average":"3",
"4_low":"4"];


/**

* Convert the Priority value from uCMDB to SM
*
* @param priority uCMDB Priority value
* @return SM Priority value
*/
public static String convertPriority(String priority, DataAdapterLogger
log) {
return convertEnumValue(priority,"Priority");
}
```

Step 2. Update the configuration files.

Update the files as described in the following table.

File	Description
icon.properties	This configuration file defines the UCMDB icons that will be displayed in the mapping tool for Service Manager CI Types. If you have defined any custom CI types in SM, you can update this configuration file to assign them appropriate icons. For more information, see the *Universal CMDB Data Flow Management Guide*.
sm.properties	This is the integration adapter configuration file. If you have modified the out-of-the-box settings in your old sm.properties file (for example, the number of concurrent threads), you need to update this configuration file accordingly for those options that are still valid in the Service Manager Enhanced Generic Adapter. For more information, see How to update the integration adapter configuration file (sm.properties).
smFedConf.xml	This configuration file is used for federation and is introduced by the Service Manager Enhanced Generic Adapter. If you have any custom logic for the federation feature, such as custom fields for federation CIs, you need to update this configuration file.

File	Description
smPopConf.xml	This configuration file is used for population and replaces the old smPopConfFile.xml configuration file. If you have any custom logic for the population feature, such as custom query conditions, you need to update this configuration file. For more information, see the following topics: What is the purpose of the <container> element in the population configuration file (smPopConf.xml)? No TQL query configured in smPopConf.xml
smPushConf.xml	This configuration file is used for relationship data push and replaces the old smSyncConfFile.xml configuration file. For more information about this file, see the following topics: How to map a relationship type query to the Service Manager Web Service object Query not configured in smPushConf.xml

Step 3. Enable Push, Population, and Federation for CI types.

Once you have completed the tasks described above, you must edit your integration point to add Push and Population jobs for the CI types, as well as enable Federation for supported CI types. For details, see the following topics:

How to create an integration point in UCMDB

How to define data push jobs in UCMDB

How to define population jobs in UCMDB

How to add an attribute of a supported CI type for federation

1.6.2.1.8. Task 8. Upgrade field mapping

The out-of-box mapping files in the Service Manager 9.41 adapters are already updated to align with the logical name solution. You need to make similar updates to your own field mapping.

> **Note**
>
> In Service Manager 9.41 or later, the original CI Name field (the **id** field in the **device** table) is removed from the Configuration Item detail form.

Upgrading the field mappings includes the following steps.

Step 1. Upgrade your data push field mapping

To update your CI push mapping files, follow these steps:

1. Switch the mapping field from CIIdentifier to DisplayName.

   For example, from

   `<target_mapping name="CIIdentifier" datatype="STRING" value="Root['display_label']"/>`

   to:

   `<target_mapping name="DisplayName" datatype="STRING" value="Root['display_label']"/>`

2. (Optional) Simplify the business logic for DisplayName.

   If you have customized the original CIIdentifier field values to append more fields so that they are unique and not renamed by Service Manager, you can remove such business logic. This is because the DisplayName field in Service Manager now supports duplicate values.

Step 2. Upgrade your population field mapping

You need to make the following changes to your population field mapping files:

1. Change `CIIdentifier` to `DisplayName`.
2. Change `CIName` to `CIIdentifier`.
3. Make sure that the **global.id** field is included in the mapping files. For example:

   `<target_mapping datatype="STRING" name="global_id" value="bizservice['UCMDBId']"/>`

For the Relationship and Running Software population mapping files, perform the following additional steps:

1. Remove all upstream and downstream `name` mapping entries from all relationship population mapping files. The following are two example `name` mapping entries:

```
<target_mapping name="name" datatype="STRING"
value="cirelationship1to1['upstreamci.logical.name']"/>
<target_mapping name="name" datatype="STRING"
value="cirelationship1to1['downstreamci.logical.name']"/>
```

2. Remove the `name` mapping entry (the entry in bold) from the container node in the running software population mapping file.

```
<for-each-source-entity count-index="i" source-
entities="runningsoftware.computer">
   <target_entity name="Node">
   <target_mapping name="name" datatype="STRING"
value="runningsoftware.computer[i]['CIIdentifier']"/>
   <target_mapping name="global_id" datatype="STRING"
value="runningsoftware.computer[i]['UCMDBId']"/>
   </target_entity>
</for-each-source-entity>
```

Step 3. Upgrade your federation configuration files

Make the following updates to your federation configuration files:

- Update the smFedConf.xml file to change one groovy method to **convertArrayOrString**, as shown below:

```
<property ucmdbName="reference_number" smName="IncidentID"
valueConverter="convertArrayOrString"/>
```

- Add the following two functions to the SMFederationConverter.groovy file.

```
/**
* Convert arraylist to a string
*
* @param to be converted
* @return converted string value
*/
public static String convertArrayString(def arrStr,
DataAdapterLogger log){
def strList=''
for (s in arrStr) {
strList+='"'+s+'"'+',';
}
if (strList.size()&gt;0)
strList=strList.substring(0, strList.length()-1);
return '{' + strList +'}';
}

public static String convertArrayOrString(def str,
DataAdapterLogger log){
if (str instanceof String)
return convertString(str, log);
else
return convertArrayString(str, log);
}
```

Step 4. Configure the UCMDB JMX console

Configure the UCMDB JMX console, as described in the following steps.



Note

This feature enables the UCMDB global ID solution for Service Manager. It is available only in UCMDB 10.21 or later. For this reason, a known issue (QCCR1E122493) exists in UCMDB versions earlier than 10.21: If two CIs with the same display name are pushed to SM and then updated in SM with a new display name, when you run a population job to synchronize the CIs back to UCMDB, the two CIs are merged in UCMDB and therefore have the same ucmdb.id value in SM.

1. Go to the JMX Console: http://<UCMDB server name>:<port>/jmx-console/.
2. Enter **UCMDB:service=Settings Services** in the search box, and press Enter.
3. Click **setSettingValue** to go to this section.
4. In the **customerID** field, enter your customer ID.
5. In the **name** field, enter **reconciliation.match.attributes**.
6. In the **value** field, enter **global_id**.

7. Click **Invoke**.

   If the configuration is successfully completed, the following message is displayed:

   ```
   Setting [reconciliation.match.attributes] value changed successfully to [global_id]
   ```

1.6.2.1.9. Task 9. Reconfigure an integration point

You need to set up an integration point to use the ServiceManagerEnhancedAdapter9.41 adapter, and activate the integration point. For details, see How to create an integration point in UCMDB.

Once this adapter is successfully enabled, you should be able to open one of the out-of-box mapping files in the Visual Mapping tool interface without any problems. For details on how to use the Visual Mapping tool, refer to the *Universal CMDB Data Flow Management Guide*.

1.6.2.1.10. Task 10. Run a Full Push to resynchronize CI names

Due to the logical name solution that was introduced in Service Manager 9.41, once you have upgraded to Service Manager 9.41 or later, you must run a full data push to resynchronize CI names to the CI display name field in Service Manager. When performing this step, you need to use the new integration point that you reconfigured.

To resynchronize CIs, follow these steps:

1. Run a Full Push in UCMDB.
2. Check the CI data in Service Manager.
    1. Perform a search to find the CIs.
    2. Verify that the Display Name field values of the CIs have been updated correctly.

> **Note**
>
> The display names of Retired CIs aren't updated, because they are already removed from UCMDB.

1.6.2.1.11. Task 11. Restore the DEM rules in Service Manager

After running a full data push to resynchronize CI names to SM, you must restore the Discovered Event Manager (DEM) rules to their original state. Hence, these DEM rules can function as usual.

To restore the DEM rules, follow these steps:

1. Sign in to Service Manager as a System Administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Click **Search**.
4. In each DEM rule, for **Action if record exists but unexpected data discovered**, restore the setting to the original value (for example, **Open a Change** or **Open an Incident**).

1.6.2.2. Integration setup

Before implementing the integration in your production environment, you can set up the integration in a test environment using the out-of-the-box integration configurations. This section describes the basic integration setup tasks without any tailoring or multi-tenancy configurations.

Integration requirements

Integration setup overview

Service Manager setup

How to create an integration user account

How to add UCMDB connection information

Universal CMDB setup

How to create an integration point in UCMDB

Populating UCMDB with Service Manager CI data

How to define population jobs in UCMDB

View Service Manager CI data in UCMDB

How to schedule CI population jobs

Pushing UCMDB CI data to Service Manager

How to define data push jobs in UCMDB

How to schedule data push jobs

How to view UCMDB CI data in Service Manager

How to view the Change History of the Primary CI of a Problem record

Federating SM ticket data to UCMDB

Federation queries

Examples of using federation

Tip

Before you proceed to implement the integration in your production environment, you can refer to the following chapters for further information:

- Multi-tenancy (Multi-company) setup, which describes how you set up the integration in multi-tenancy mode.
- Standards and best practices, which describes best practices for implementing the integration and also provides Frequently-Asked-Questions information.
- Tailoring the integration, which describes how you can tailor the integration to better suit your business needs.
- Troubleshoot UCMDB integration, which provides information on troubleshooting data push and population issues.

1.6.2.2.1. Integration requirements

This section describes the integration requirements when using the ServiceManagerEnhancedAdapter9.41 adapter.

Supported product versions

The supported product versions for using this adapter are listed in the following table.

Important

If using UCMDB 11.0, CP 26 is required to integrate UCMDB with Service Manager or other applications.

Product	Versions
Service Manager	9.41 or later
UCMDB	10.20 or later
UCMDB UI	Compatible versions for UCMDB

Note: To support the embedding of UCMDB UI's CI Visualization and Impact Visualization widgets in Service Manager, the following product versions are required:

- Service Manager 9.50 or later
- UCMDB UI 4.10 or later

Setting up required components

You must set up the following required components to establish an integration between UCMDB and Service Manager.

- Universal CMDB installation

  Add a UCMDB Probe for the population feature if you don't already have one.

- Service Manager installation

  Add the UCMDB URL to the SMtoUCMDB integration instance in Service Manager Integration Manager.

- Network connection between the Universal CMDB and Service Manager systems.

For instructions on installing and configuring your systems, see the UCMDB and Service Manager documentation.

Deploying an appropriate adapter

To use a Service Manager 9.41 adapter, all of your Service Manager server, clients, and applications must be version 9.41 or later. You need to verify your Service Manager applications version and then find the correct adapter.

To to this, follow these steps:

1. Sign in to Service Manager as a system administrator.
2. Enter db in the command line, and press Enter.
3. Enter scversion in the Table field, and then click **Search**.
4. Click **Search** again.
5. Check the Application Version value that's displayed.
6. If your applications version is earlier than 9.41, use the ServiceManagerAdapter9.41 adapter.
7. If you are running Service Manager applications 9.41 or later, use the ServiceManagerEnhancedAdapter9.41 adapter.

Configure one UCMDB as the global ID generator



Tip

- If UCMDB acts as the global ID generator, the newly-created CI on this UCMDB will generate a valid global_id attribute.
- If UCMDB doesn't act as the global ID generator, make sure that the CI to be pushed has a valid global_id attribute. Otherwise, the push to the remote Service Manger may fail.
- For the UCMDB cluster, it is required that only one of the UCMDBs acts as the global ID generator, and the rest UCMDBs will synchronize the global ID within this cluster. If there are two or more UCMDBs act as the global ID generators, potential duplicated global IDs may cause identification problems and synchronization failure.

Follow these steps:

1. Launch the Web browser and enter the following address:

   http://<RTSM server>:21212/jmx-console

   Use the user name sysadmin and the password you set for the administrator during the installation.

2. Click **UCMDB:service=Multiple CMDB Instances Services** to open the JMX MBEAN View page.

3. Click one of the following methods and enter values as required:

Method	Description
setAsGlobalIdGenerator	Specifies that the CMDB will act as the global ID generator for all locally existing scopes
setAsGlobalIdGeneratorForScopes	Specifies the scopes for which global IDs will be generated
setAsNonGlobalIdGenerator	Stops the CMDB from acting as the global ID generator for all scopes

4. Click **Invoke**.



Note

If you want to check which scopes are currently set or which UCMDB/RTSM instance is acting as the CMS, use the **getGlobalIdGeneratorScopes** method.

1.6.2.2.2. Integration setup overview

The integration requires setup on both the UCMDB and Service Manager systems.

This task includes the following steps:

1. Set up the Service Manager system.

   See Service Manager setup.

2. Set up the UCMDB system.

   See Universal CMDB setup.

3. Run the UCMDB population jobs to synchronize CIs to UCMDB.

   See Populating UCMDB with Service Manager CI data.

4. Run the UCMDB data push jobs to transfer CIs to Service Manager.

   See Populating UCMDB with Service Manager CI data.

1.6.2.2.3. Service Manager setup

You must complete the following tasks from your Service Manager system to support the integration.

1. Create a dedicated integration user account in Service Manager.

   See How to create an integration user account.

2. Add the UCMDB connection information to the **SMtoUCMDB** integration instance configured in Integration Manager.

   See How to add UCMDB connection information.

1.6.2.2.3.1. How to create an integration user account

This integration requires an administrator user account for UCMDB to connect to Service Manager. The user account must already exist in both UCMDB and Service Manager.



Note

The Service Manager Enhanced Generic Adapter requires the Service Manager `ucmdbIntegration` RESTful APIs to work. For this reason, the integration user account must have the **RESTful API** capability word in Service Manager.

To create a dedicated integration user account in Service Manager:

1. Sign in to Service Manager as a system administrator.
2. Type `contacts` in the Service Manager command line, and press **ENTER**.
3. Create a new contact record for the integration user account.
    1. In the Contact Name field, type a name. For example, `UCMDBIntUser`.
    2. Click **Add**, and then **OK**.
4. Type `operator` in the Service Manager command line, and press **ENTER**.
5. In the Login Name field, type the username of an existing system administrator account, and click **Search**.

    Service Manager displays the system administrator account.
6. Create a new user account based on the existing one.
    1. Change the Login Name to the integration account name that you want (for example, **ucmdb**).
    2. Type a Full Name. For example, UCMDB.
    3. In the Contact ID field, click the **Fill** icon and select the contact record that you have just created.
    4. Click **Add**.
    5. Select the **Security** tab, and change the password.
    6. Select the **Startup** tab, and add the **RESTful API** capability word for the operator.
    7. Click **OK**.

Service Manager creates the integration user account. Later, you will need to add this user account in UCMDB, and then specify this user account in the Credentials ID field when creating an integration point in UCMDB. See How to create an integration point in UCMDB.

1.6.2.2.3.2. How to add UCMDB connection information

UCMDB-SM integration requires the UCMDB connection information to obtain CI attribute information from the UCMDB system, and display it in the Actual State section in the Service Manager configuration item form; it also requires UCMDB Browser or UCMDB Gateway connection information to retrieve data from UCMDB Browser or UCMDB Gateway and display the data to Service Manager users.

Caution

If you don't specify the correct connection information, an error, instead of correct CI related data, is displayed to Service Manager users.

This task includes the following steps:

1. Sign in to Service Manager as a system administrator.
2. Click **Tailoring** > **Integration Manager**.
3. Click **Add**.
4. In the **Integration Template** field, select **SMtoUCMDB**.

   Some fields in the form are automatically populated with a value. The empty fields can be ignored.
5. (Optional) Update the **Name**, **Log Level**, and **Description** fields as needed.
6. On the **UCMDB Server Configuration** tab, provide a value for each field in the following table.

   Important

   When specifying UCMDB URLs, use the fully qualified domain name (FQDN) rather than the IP address of the server.

Field	Description
UCMDB webservice URL	Enter the URL to the Universal CMDB web service API. The URL has the following format: https://*<UCMDB server name>*:*<port>*/axis2/services/ucmdbSMService Replace *<UCMDB server name>* with the fully-qualified domain name (FQDN) of your UCMDB server, and replace *<port>* with the communications port your UCMDB server uses. **Note** If you are setting up an HTTPS connection, make sure to add the UCMDB root CA certificate to the SM truststore. See the *Add the UCMDB root CA certificate to the SM truststore* section below.
User Id	Enter the user name required to manage CIs on the UCMDB system. For example, enter **admin**, which is the out-of-the-box administrator user name.
Password	Enter the password required to manage CIs on the UCMDB system. For example, enter **admin**, which is an out-of-the-box password for the **admin** user account.
Customer Id	Enter the Customer Id in Universal CMDB Server. For example, enter 1, which is an out-of-the-box customer id in UCMDB Server.

Field	Description
Multi-tenant UCMDB webservice URL	This is the URL that Service Manager uses to synchronize company records with UCMDB when running in multi-company mode. This URL should use the following format: https://*<UCMDB server name>*:*<port>*/axis2/services/UcmdbManagementService Replace *<UCMDB server name>* with the fully-qualified domain name (FQDN) of your UCMDB server, and replace *<port>* with the communications port your UCMDB server uses. **Note** ◦ This field will appear when the Service Manager is running in multi-company mode ◦ If you are setting up an HTTPS connection, make sure to add the UCMDB root CA certificate to the SM truststore. See the *Add the UCMDB root CA certificate to the SM truststore* section below.
UserId Password	This is UCMDB account (user name/password) that has the privilege to add/delete company records. **Note**: These fields will appear when the Service Manager is running in multi-company mode.

7. Click **Verify Connection** to verify that your UCMDB Server configuration is correct.
8. On the **UCMDB Browser Configuration** tab, configure each field or option as described in the following table.

Field / Option	Default value	Description
UCMDB Browser URL		Enter your UCMDB Browser URL in the following format: https://*<UCMDB Browser server name>*:*<port>*/ucmdb-browser Replace <UCMDB Browser server name> and <port> with the fully-qualified domain name (FQDN) and communications port of UCMDB Browser server host, respectively. **Important** When specifying UCMDB Browser URL, use the FQDN rather than the IP address of the server host.
UCMDB Server Hostname	Default Client	Enter your UCMDB Server's IP or Hostname. For embedded UCMDB Browser, use default value. For standalone UCMDB Browser, use the same value you defined for <host_name> in ucmdb_browser_config.xml file in UCMDB Browser.

Field / Option	Default value	Description
Show UCMDB Browser Impact widget	false (disabled)	(Optional) Select this option if you want to embed UCMDB Browser Impact widget in Service Manager. **Tip** This option shows or hides the **Impact Widget** tab on this page.
Show UCMDB Browser CI Visualization widget	false (disabled)	(Optional) Select this option if you want to embed the UCMDB Browser CI Visualization widget in Service Manager. **Tip** This option shows or hides the **CI Visualization Widget** tab on this page.

Field / Option	Default value	Description
Show UCMDB Browser in a New Browser Page	false (disabled)	Select this option if you want to display the UCMDB Browser CI Visualization widget on a new browser page. This option works only after you select either **Show UCMDB Browser Impact widget** or **Show UCMDB Browser CI Visualization widget**. **Caution** If the number of CIs exceeds the value as specified in **Maximum Trigger CIs per Impact Graph**, the CIs graph displays the first page only.

Field / Option	Default value	Description
Use a dedicated user account	false (disabled)	(Optional) Select this option if LW-SSO is disabled for the integration and you want to configure a dedicated UCMDB Browser user account for users to view UCMDB Browser data from Service Manager. This option enables users to access UCMDB Browser data directly from the Service Manager user interface without the need to enter a user name and password, and is therefore useful when LW-SSO is disabled for the integration. For example, this dedicated user account is used when users view an embedded UCMDB Browser widget (such as CI Visualization and Impact Simulation) from a Service Manager record or when users click the **View in UCMDB Browser** button from a CI record in Service Manager.

Field / Option	Default value	Description
		Note This option takes effect only when LW-SSO is disabled for the integration. If you have enabled both LW-SSO and this option for the integration, LW-SSO is used. **Tip** This option shows or hides the **Dedicated User** tab on this page.

Field / Option	Default value	Description
Impact Widget tab > **Maximum Trigger CIs per Impact Graph**	100	This is the maximum number of CIs that Service Manager is allowed to send to UCMDB Browser as trigger CIs when running impact simulation. For example, if the number of affected service and CIs of a Change record exceeds this value, when users click **Run Impact Analysis** for the Change record, the impact analysis result is displayed on multiple pages, each of which except the last one contains a **Next** button on the top. Users can click **Next** to view the next impact simulation graph. **Caution** The value of this field must not be greater than 100.

Field / Option	Default value	Description
Impact Widget tab > **Impact Bundle**		This is a rule bundle that's defined in UCMDB Server and will be used for impact analysis. If you leave this field blank, the **List of impact rule bundles** setting in UCMDB will be used. To access this setting in UCMDB, go to **Administration** > **Infrastructure Settings Manager**, and choose **UCMDB Browser Settings**. An empty value of this setting means all rule bundles will be used.
Impact Widget tab > **Default Impact Severity**	critical	This is the default impact severity for impact analysis. You can set it to **critical**, **high**, **medium**, **low**, or **none**. **Note** A severity of "none" will be used when this field is left blank.

Field / Option	Default value	Description
CI Visualization Widget tab > **Filter TQL for CI Visualization**		This is a filter TQL query that's used to retrieve CI relationship graphs from UCMDB Browser. A filter TQL query is used to restrict the CI types and relationship types that you want to display on each relationship graph.

Field / Option	Default value	Description
Dedicated User tab > **User Id**		Create a dedicated user account in UCMDB Browser and grant appropriate permissions to this account. Enter the user name in this field. **Caution:** This user account must be granted only limited permissions that are just enough to view the Impact Simulation, Get Related, and History widgets and to access CIs. For information about how to specify which widgets are displayed to a user and how to authorize CI access for a user, refer to the *UCMDB Browser Installation and Configuration Guide*. **Caution:** To protect your privacy, we recommend enabling LW-SSO configuration instead of using the user id and password. See How to enable Lightweight Single Sign-On (LW-SSO) configuration for more information.

Field / Option	Default value	Description
Dedicated User tab > **Encrypted Password**		Obtain an encrypted value of the password of the dedicated user account, and enter the value in this field. To obtain an encrypted value for the password of a UCMDB Browser user account, follow these steps: 1. Open UCMDB URL in your browser. 2. Click **JMX Console** from the login page. 3. Sign in to the JMX Console as an administrator. 4. Search for **getencryptedPasswordForURL** and then open this section. 5. In the **User Name** and **Password** fields, enter the user name and password of the user account that you want to use. 6. Click **Invoke**. An encrypted value of the password is returned. **Caution:** To protect your privacy, we recommend enabling LW-SSO configuration instead of using the user id and password. See How to enable Lightweight Single Sign-On (LW-SSO)

Field / Option	Default value	Description
		configuration for more information.

9. On the **UCMDB Gateway Configuration** tab, configure each field or option as described in the following table.

> **Important**
>
> The UCMDB Gateway configuration invalidates the UCMDB Browser configuration. After configuring the UCMDB Gateway setting, the CI visualization graph and the Impact visualization graph that are previously displayed by the UCMDB Browser are still available in Service Manager. However, you still need to perform UCMDB Browser configuration if you need to view CI history.

Field / Option	Default value	Description
UCMDB Gateway URL		Enter your UCMDB Gateway URL in the following format. https://*<server host:port>*/cms-gateway Replace *<server host>* and *<port>* with the fully-qualified domain name (FQDN) and communications port of UCMDB Gateway server host, respectively.

Field / Option	Default value	Description
		Note If you are setting up an HTTPS connection, make sure to add the UCMDB Gateway root CA certificate to the SM truststore. See the *Add the UCMDB Gateway root CA certificate to the SM truststore* section below. Meanwhile, if you are connecting to the UCMDB Gateway with SSL, you also need to configure the UCMDB Gateway truststore for the web tier. See the *Configure the UCMDB Gateway truststore for the web tier* section below.

Field / Option	Default value	Description
User Id		Enter the name of the UCMDB Gateway integration user. This user must have access to the data on all tenants.
Password		Enter the password of the UCMDB Gateway integration user.

10. Click **Verify Connection** to verify that your UCMDB Gateway configuration is correct.
11. Click **Finish**.

    Service Manager creates an SMtoUCMDB integration instance.
12. Go to **Integration Manager**, and then click the SMtoUCMDB integration instance.
13. Click **Next** until the **Initialize** link becomes active, and then click the link.

    > **Note**
    >
    > This step creates records in the **cirelationship1to1** table so that CI relationships can be properly populated to UCMDB.

14. Click **Finish** to go back to Integration Manager, select the SMtoUCMDB integration instance, and then click **Enable** to enable the integration instance.

Important

When using the embedded UCMDB Browser, for the Impact Simulation Widget and Get Related Widget to work, perform the following steps:

1. Go to **JMX console > UCMDB:service=Settings Services > setGlobalSettingValue**.
2. In the **name** field, enter **ui.framing.host**.
3. In the **value** field, enter the desired Service Manager web tier host address. For example, `frame-ancestors https://<IP>:<port>` or `frame-ancestors https://<web tier FQDN>:<port>`.
4. Click **Invoke**.
5. Restart the UCMDB server.

When using UCMDB Browser standalone, for the Impact Simulation Widget and Get Related Widget to work, perform the following steps:

1. Open the **%WEB_SERVER_HOME%\webapps\ucmdb-browser\WEB-INF\web.xml** file.
2. Locate the following lines:

```
<filter>
    <filter-name>ClickjackingFilterSameOrigin</filter-
name>
    <filter-
class>com.hp.ucmdb_browser.server.web.ClickjackingFilter</
filter-class>
    <init-param>
        <param-name>mode</param-name>
        <param-value>SAMEORIGIN</param-value>
    </init-param>
</filter>
<filter-mapping>
        <filter-name>ClickjackingFilterSameOrigin</filter-
name>
        <url-pattern>/*</url-pattern>
</filter-mapping>
```

3. In the section above, replace the following line:

```
<param-value>SAMEORIGIN</param-value>
```

with

```
<param-value>ALLOW_FROM <Service Manager URL></param-
value>
```

For example,

```
<param-value>ALLOW_FROM
https://SMservername.domain.com</param-value>
```

4. Restart the UCMDB Browser.

Add the UCMDB root CA certificate to the SM truststore

Follow these steps to add the UCMDB root CA to the SM server by using the certificate generation toolkit.

1. Copy the UCMDB server root certification file cmsroot.crt from the UCMDB server installation directory (for example, `C:\UCMDB\UCMDBServer\conf\security\`) to the `cert-utils/certs` folder.
2. Import cmsroot.crt to the SM truststore. See the following sample commands.

   Non-FIPS mode:

   ```
   import-cert.bat --alias cmsroot --file certs/cmsroot.crt --keystore
   certs/cacerts --storepass changeit
   ```

   FIPS mode:

   ```
   import-cert.bat --alias cmsroot --file certs/cmsroot.crt --keystore
   certs/cacerts --storetype bcfks --storepass changeit
   ```

3. Copy cacerts to the `<SM_SERVER_HOME>/RUN` folder.
4. Update sm.ini by adding the following lines:

   ```
   truststoreFile:cacerts
   truststorePass:changeit
   ```

5. Restart the SM server.

Add the UCMDB Gateway root CA certificate to the SM truststore

Follow these steps to add the UCMDB Gateway root CA to the SM server by using the certificate generation toolkit.

1. Copy the UCMDB Gateway root CA certification file cmsrootca.crt from the UCMDB Gateway installation directory (for example, `C:\cms-gateway-11.6.9.59-windows\cms-gateway-installer\ssl`) to the `cert-utils/certs` folder.
2. Import cmsrootca.crt to the SM truststore. See the following sample commands.

   Non-FIPS mode:

   ```
   import-cert.bat --alias cmsrootca --file certs/cmsrootca.crt --
   keystore certs/cacerts --storepass changeit
   ```

   FIPS mode:

   ```
   import-cert.bat --alias cmsrootca --file certs/cmsrootca.crt --
   keystore certs/cacerts --storetype bcfks --storepass changeit
   ```

3. Copy cacerts to the `<SM_SERVER_HOME>/RUN` folder.
4. Update sm.ini by adding the following lines:

   ```
   truststoreFile:cacerts
   truststorePass:changeit
   ```

5. Restart the SM server.

Configure the UCMDB Gateway truststore for the web tier

Follow these steps to configure the UCMDB Gateway truststore for the web tier by using the certificate generation toolkit.

1. Copy the UCMDB Gateway root CA certification file cmsrootca.crt from the UCMDB Gateway installation directory (for example, `C:\cms-gateway-11.6.9.59-windows\cms-gateway-installer\ssl`) to the `cert-utils/certs` folder.
2. Import cmsrootca.crt to the UCMDB Gateway truststore. See the following sample commands.

   Non-FIPS mode:

   ```
   import-cert.bat --alias cmsrootca --file certs/cmsrootca.crt --
   keystore certs/cmsrootca.p12 --storepass <your password for the CMS
   truststore>
   ```

FIPS mode:

```
import-cert.bat --alias cmsrootca --file certs/cmsrootca.crt --
keystore certs/cmsrootca.bcfks --storetype bcfks --storepass <your
password for the CMS
truststore>
```

3. Copy cmsrootca.p12 (or cmsrootca.bcfks) to the `<customize-folder>/config` folder if you have configured the `customize-folder` parameter, or to the `<WEBTIER>\WEB-INF` folder if you haven't configured the `customize-folder` parameter.
4. Edit the `<customize-folder>/config/webtier.properties` file if you have configured the `customize-folder` parameter, or edit the `<WEBTIER>/WEB-INF/webtier.properties` file if you haven't configured the `customize-folder` parameter. For example:

   ```
   cms.truststore=<customize-folder>/config/cmsrootca.p12
   cms.truststore.password=changeit
   ```

   Or:

   ```
   cms.truststore=/WEB-INF/cmsrootca.p12
   cms.truststore.password=changeit
   ```

   For more information about `cms.truststore` and `cms.truststore.password`, see Web parameter:cms.truststore and Web parameter:cms.truststore.password, respectively.
5. Restart the web application server.

1.6.2.2.4. Universal CMDB setup

You must complete the following task from your UCMDB system to support the integration:

Create an integration point between UCMDB and Service Manager. See How to create an integration point in UCMDB.

1.6.2.2.4.1. How to create an integration point in UCMDB

A default UCMDB installation already includes the **ServiceManagerEnhancedAdapter9.41** package. To use the integration package, you must create an integration point that lists the connection properties for the integration.

> **Caution**
>
> For data population, this integration supports the use of only one probe for your Service Manager system. Don't run population jobs on different probes by setting up multiple integration points with different probes for your Service Manager system. The integration allows only one probe for one Service Manager system.

To create an integration point, follow these steps:

1. Sign in to UCMDB as an administrator.
2. Add the integration user account that you created in Service Manager.
    1. Click **Security** > **Users and Groups**.
    2. Click the **New User** button  .
    3. For User Name and Password, type the user name and password that you created in Service Manager. See How to create an integration user account.
    4. Click **Next**, and then select **Admin** from the Role List.
    5. Click **Finish** to add the integration user account.
3. Navigate to **Data Flow Management > Integration Studio**. UCMDB displays a list of existing integration points.
4. Click the **New Integration Point** icon.

    UCMDB displays a New Integration Point properties window.
5. Complete the integration and adapter property fields as described in the following table.

Field name	Required?	Description
Integration Name	Yes	Type the name (unique key) of the integration point. For example, **sm_integration**.
Integration Description	No	Type a description for the integration point.
Adapter	Yes	Select **OpenText Products** > **Service Manager** > **ServiceManagerEnhancedAdapter9.41**.
Is Integration Activated	Yes	Enable this option to indicate the integration point is active.
Hostname/IP	Yes	Type the hostname or IP address of the Service Manager server. For example, localhost.
Port	Yes	Type the communications port of the Service Manager server. For example, 13080.

Field name	Required?	Description
URL Override	No	This field value (if any) supersedes the previous Hostname/IP and Port settings descriptions. Use this field If you want UCMDB to connect to Service Manager in any combinations of the following ways: ◦ Connect to Service Manager over HTTPS only. When using HTTPS, you may need to add the Service Manager server certificate to the UCMDB JVM Trusted Stores. See the *Add the Service Manager server certificate to UCMDB when using HTTPS connection* section below. ◦ Connect to Service Manager over both HTTP and HTTPS ◦ Connect to multiple Service Manager server nodes (horizontally scaled environment) ◦ Connect to one single Service Manager server node through multiple ports (vertically scaled environment)

Field name	Required?	Description
		For more information, see Push in clustered environments. Type one or more Service Manager web services URLs (separated by a semicolon) in this field. Each URL must use this format: `http(s)://<hostname>:<port>/SM/9/rest` The following are two example values of this field: ◦ https://localhost:13443/SM/9/rest ◦ http://localhost:13080/SM/9/rest; https://localhost:13443/SM/9/rest; http://smfpe04:13080/SM/9/rest
Credentials ID	Yes	Click **Generic Protocol**, click the **Add** icon to add the integration user account that you created, and then select it. This account must exist in both Service Manager and UCMDB. See How to create an integration user account.

Field name	Required?	Description
Data Flow Probe	Yes	Select the name of the Data Flow Probe used to run population jobs. You should have already added the data flow probe for the integration after installing UCMDB. See Integration requirements.

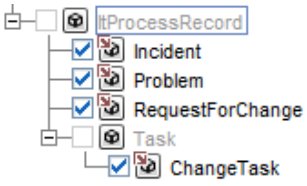
6. Click **Test Connection** to make sure that a connection is successfully established.
7. Click **OK**.

   The system creates the integration point and displays its details.
8. Click the **Federation** tab, and complete the following configuration.

   Tip

   To configure a direct link to the Service Manager system with the context of the selected incident, problem, or change request, see Configure a direct link to Service Manager for a selected Problem, Incident, or Change Request.

   1. In Supported and Selected CI Types, select the following CI types as needed from **ItProcessRecord**:
      - Incident
      - Problem
      - Request for Change
      - Change Task

   2. For each CI type you selected, select **Retrieve CIs of selected CI Type** for **CI Type Retrieval Mode**.
9. Click the **Population** and **Data Push** tabs to view the default integration job details.

> **Note**
>
> UCMDB creates several default population and data push jobs when creating an integration point. If needed, you can create a new job for the integration point. For information about creating integration jobs, see How to Define Data Push Jobs in UCMDB and How to Define Population Jobs in UCMDB.

10. Click the **Save Integration Point** icon.

Add the Service Manager server certificate to UCMDB when using HTTPS connection

Suppose you are working with HTTPS connection between UCMDB and Service Manager. If the Service Manager server uses a certificate signed by a known certificate authority, you need to type one or more Service Manager web services HTTPS URLs in the **URL Override** field.

If not, add the Service Manager server certificate to the local UCMDB JVM Trusted Stores as follows:

1. Enable SSL on the Service Manager server. For more information, see Secure Sockets Layer (SSL) encryption.
2. Copy the certificate `smrootca.crt` generated in the previous step to `<UCMDB Server>\bin\jre\bin`.
3. Back up the following files:
   - `<UCMDB Server>\bin\jre\lib\security\cacerts`
   - `<UCMDB Server>\integrations\conf\security\ProbeTrustStore.jk`
4. Open a command line window and run the following commands to import the certificates copied in *step 2*.

```
cd <UCMDB Server>\bin\jre\bin
keytool -import -keystore <UCMDB
Server>\bin\jre\lib\security\cacerts -trustcacerts -storepass
<truststore pass> -alias smrootca -file smrootca.crt
keytool -import -keystore <UCMDB
Server>integrations\conf\security\ProbeTrustStore.jks -trustcacerts
-storepass <truststore pass> -alias smrootca -file smrootca.crt
```

> **Note**
>
> The default truststore pass of **cacerts** is `changeit`, and the default truststore pass of **ProbeTrustStore.jks** is `logomania`. If the default password for **ProbeTrustStore.jks** doesn't work, refer to Change probe keystore and truststore passwords to reset probe keystore and truststore passwords.

5. At the command line prompt "Trust this certificate?", enter **yes**.
6. Restart the UCMDB service and the UCMDB Integration service.
7. (Optional) If you have installed UCMDB Data Flow Probe, continue to follow *step 2* to *step 5* to import the certificates to `<UCMDB DataFlowProbe>\bin\jre\lib\security\cacerts` and then restart the Data Flow Probe service.
8. Open the Integration Point properties window. Under Adapter properties, fill the **URL Override** field with `https://<hostname>:13443/SM/9/rest` and then save your changes.

1.6.2.2.5. Populating UCMDB with Service Manager CI data

In addition to pushing CI data from UCMDB to Service Manager, this integration also supports the population of CI data (including CIs and CI relationships) from Service Manager to UCMDB. The integration can then update the list of CIs in UCMDB if new CIs or new attribute values are found in Service Manager. The population of data from Service Manager to UCMDB is defined in the Integration Studio in UCMDB. You can manually run the population jobs, however We recommend that you schedule these jobs to keep your CIs and CI attributes up to date.

This task includes the following steps:

1. Define CI/CI Relationship population jobs in UCMDB.

   See How to define population jobs in UCMDB.

2. View the transferred CI/CI Relationship data in UCMDB.

   See View Service Manager CI data in UCMDB.

3. Schedule CI population jobs to keep CIs and CI attributes up to date.

   See How to schedule CI population jobs.

1.6.2.2.5.1. How to define population jobs in UCMDB

A CI or relationship population job copies certain types of CIs or relationships from Service Manager to UCMDB.

To define a CI or CI relationship population job, follow these steps:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**. UCMDB displays a list of existing integration points.
3. Open the integration point that you created for Service Manager.
4. Click the **Population** tab, and add a new job as follows.



Note

UCMDB creates several default population and data push jobs when creating an integration point. The following table lists the default population jobs and their queries. If needed, you can create, update or remove queries for each job.

Queries for CI / CI relationship population

Integration Job	Queries for CI / CI relationship population
SM Configuration Item Population job	Out-of-the-box, the following queries are available for this job, which populates UCMDB with CI records from Service Manager: ◦ **SM Business Service Population 2.0**: Populates UCMDB with CIs of the bizservice type. ◦ **SM RunningSoftware Population 2.0**: Populates UCMDB with CIs of the RunningSoftware type. ◦ **SM Computer Population**: Populates UCMDB with CIs of the computer type. ◦ **SM CLIP Down Time Population 2.0**: Populates UCMDB with CIs of the planned outage type.

Integration Job	Queries for CI / CI relationship population
SM Relations Population job	Out-of-the-box, the following queries are defined for this job, which populates UCMDB with CI Relationship records from Service Manager: ◦ **SM Biz To Biz With Containment 2.0**: Populates UCMDB with CI relationships in which a bizservice CI contains another. ◦ **SM Biz To Biz With Usage 2.0**: Populates UCMDB with CI relationships in which a bizservice CI uses another. ◦ **SM Biz To Computer With Containment 2.0**: Populates UCMDB with CI relationships in which a bizservice CI contains a computer CI. ◦ **SM Biz To Computer With Usage 2.0**: Populates UCMDB with CI relationships in which a bizservice CI uses a computer CI. ◦ **SM Computer To Computer With Connects 2.0**: Populates UCMDB with CI relationships in which a computer CI connects to another. ◦ **SM Biz To Software With Containment 2.0**: Populates UCMDB with CI relationships in which a bizservice CI contains a software CI. ◦ **SM Biz To Software With Usage 2.0**: Populates UCMDB with CI relationships in which a bizservice CI uses a software CI. ◦ **SM Computer Composition Software 2.0**: Populates UCMDB

Integration Job	Queries for CI / CI relationship population
	with CI relationships in which a computer CI connects to a software CI. ◦ **SM CI Connection Down Time CI 2.0**: Populates UCMDB with CI relationships in which a CI connects to a down time CI.

1. Click the **New Integration Job** icon .
2. Type a Name for the integration job. For example, **CI_Population_Job1**.
3. Click the **Add Query** icon to add existing queries to the job (see the **Queries for CI / CI relationship population** table).
4. Select the **Allow Integration Job to delete removed data** checkbox for the query.
5. Click **OK** to save the job.

5. Run the job manually to see if the integration job works fine.
   1. To populate UCMDB with all relevant data for the job, click the icon.
   2. To populate UCMDB with only CI data changes since the job last ran, click the icon.
6. Wait for the job to complete, and click the **Refresh** icon multiple times as needed until the job completes.

> **Note**
>
> When the job completes, the job status becomes one of the following: Succeeded, Passed with failures, or Failed.

7. Click the **Statistics** tab to view the results. If the job failed, click the **Query Status** tab and **Job Errors** tab for more information.
8. Click **OK**.

If the job completes successfully, you can view the transferred CI data in UCMDB and schedule the job so that it can run automatically.

1.6.2.2.5.2. View Service Manager CI data in UCMDB

After a population job is successfully completed, you can search for the Service Manager CI records in UCMDB, and verify that their attributes are correctly populated.

The Service Manager **Display Name** field is populated to the **Name** field on the Configuration Item Properties pane in UCMDB.

Note

To see the entire attribute mappings of a CI type, you can open its population XML file (for example, **SM Business Service Population.xml**), where the UCMDB attribute field names and the mapped Service Manager web service field caption names are defined. For more information, see Tailoring the integration.

1.6.2.2.5.3. How to schedule CI population jobs

You can schedule CI population jobs to match the discovery/maintenance schedule of your Service Manager feeders. For example, if your Service Manager feeders send CI data updates on a daily schedule, then the population jobs must also run on a daily schedule. By using a matching schedule you can ensure that your UCMDB system always has the most current CI data.

This task includes the following steps:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**. UCMDB displays a list of integration points.
3. Open the integration point for Service Manager.
4. Click the **Population** tab, and select a population job from the list.
5. Click the **Edit Integration Job** icon.
6. Select the **Scheduler enabled** option.
7. Select the scheduling options that you want to use. For example, select Repeat every: **Day** and Ends: **Never**.
8. Select a Time Zone.
9. Click **OK**.

1.6.2.2.6. Pushing UCMDB CI data to Service Manager

The integration requires a one-time transfer of CIs from UCMDB to Service Manager to populate the Service Manager system with CIs. The integration will then update the list of CIs in Service Manager when UCMDB discovers new CIs or new attribute values. The integration accomplishes the push of CI data using data push jobs in the UCMDB system. We recommend that you schedule these jobs to keep your CIs and CI attributes up-to-date.

This task includes the following steps:

1. Define CI/CI Relationship data push jobs.

   See How to define data push jobs in UCMDB.

2. View the CI/CI Relationship data pushed from UCMDB.

   See How to view UCMDB CI data in Service Manager.

3. Schedule data push jobs to keep CI/CI Relationship data up to date.

   See How to schedule data push jobs.

1.6.2.2.6.1. How to define data push jobs in UCMDB

Data push jobs copy CI or CI Relationship records from your UCMDB system to your Service Manager system.

To define a CI or CI relationship push job, follow these steps:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management > Integration Studio**. UCMDB displays a list of existing integration points.
3. Select the Integration Point that you created for Service Manager. For example, **sm_integration**.
4. Click the **Data Push** tab.
5. Follow these steps to add a new data push job:
    1. Click the **New Integration Job** icon  .

> **Note**
>
> UCMDB creates a default data push job when creating an integration point. The following table lists the default data push job and its queries. If needed, you can create, update, or remove queries for the push job. To access these out-of-the-box queries for push, go to **Modeling** > **Modeling Studio** > **Resources**, select **Queries** for Resource Type, and then navigate to **Root** > **Integration** > **Service Manager** > **Push**. For information about tailoring data push queries, see How to create a query to synchronize the CI type.

Queries for CI / CI Relationship Push

Integration job	Queries
Integration job	Queries

Integration job	Queries
SM Push job	Out-of-the-box, the following queries are available for this job, which pushes CI records from UCMDB to Service Manager: ▪ **SM Mainframe Push 2.0**: pushes CIs of the mainframe type. ▪ SM Business Element Push 2.0: pushes CIs of the business application, and infrastructure service types ▪ **SM Network Component Push 2.0**: pushes CIs of the network component type. ▪ **SM Running Software Push 2.0**: pushes CIs of the running software type. ▪ **SM Business Service Push 2.0**: pushes CIs of the business service type. ▪ **SM Computer Push 2.0**: pushes CIs of the computer type. ▪ **SM Storage Push 2.0**: pushes CIs of the storage type. ▪ **SM Switch Push 2.0**: pushes CIs of the switch type. ▪ **SM Net Printer Push 2.0**: pushes CIs of the net printer type. ▪ **SM Cluster Push 2.0**: pushes CIs of the cluster type. ▪ **SM Mobile Device Push 2.0**: pushes CIs of the mobile device type. ▪ **SM Local Printer Push 2.0**: pushes CIs of the local printer type. Out-of-the-box, the following queries are available for this job, which pushes CI Relationship records from UCMDB to Service Manager:

Integration job	Queries
	▪ **SM Layer2 Topology Relations Push 2.0**: pushes compound CI relationships between nodes. ▪ **SM Business Service Relations Push 2.0**: pushes CI relationships whose upstream CI type is business service. ▪ **SM CRG Relations Push 2.0**: pushes CI relationships whose upstream CI type is cluster. ▪ **SM Node Relations Push 2.0**: pushes direct CI relationships whose upstream CI type is node.

2. In the **Name** field, type a unique name for the job. For example, **CI_Push_Job1**.
3. Click the **Add Query** icon to add existing queries to the job.
4. Select the **Allow Integration Job to delete removed data** option for each query.
5. Click **OK** to save the job.

6. Run the job manually to verify that the integration job works fine.

> **Caution**
>
> If you have a huge amount of CI data in your UCMDB system, and this is your first time to push CI /CI Relationship data to Service Manager, it's advisable to select the **Add the record** option instead of **Open a change** or **Open an incident** for **Action if matching record doesn't exist** in each Discovery Event Manager Rules definition. Failure to do so may cause unnecessary performance problems. For details, see How to add Discovery Event Manager rules.

1. To push all relevant data for the job, click the icon.
2. To push only the updated data since the job last ran, click the icon.

> **Tip**
>
> You can stop a running push job by pressing the **Stops the selected job** icon .

7. Wait for the job to complete, and click the **Refresh** icon multiple times as needed until the job completes.

> **Note**
>
> When the job completes, the job status becomes one of the following depending on the results: `Completed successfully`, `Completed`, or `Failed`.

8. Click the **Statistics** tab to view the results; if any errors occur click the **Query Status** tab and **Job Errors** tab for more information.
9. Click **OK**.

If the job completes successfully, you can view the UCMDB CI data in Service Manager, and schedule the job so that it can run automatically.

1.6.2.2.6.2. How to schedule data push jobs

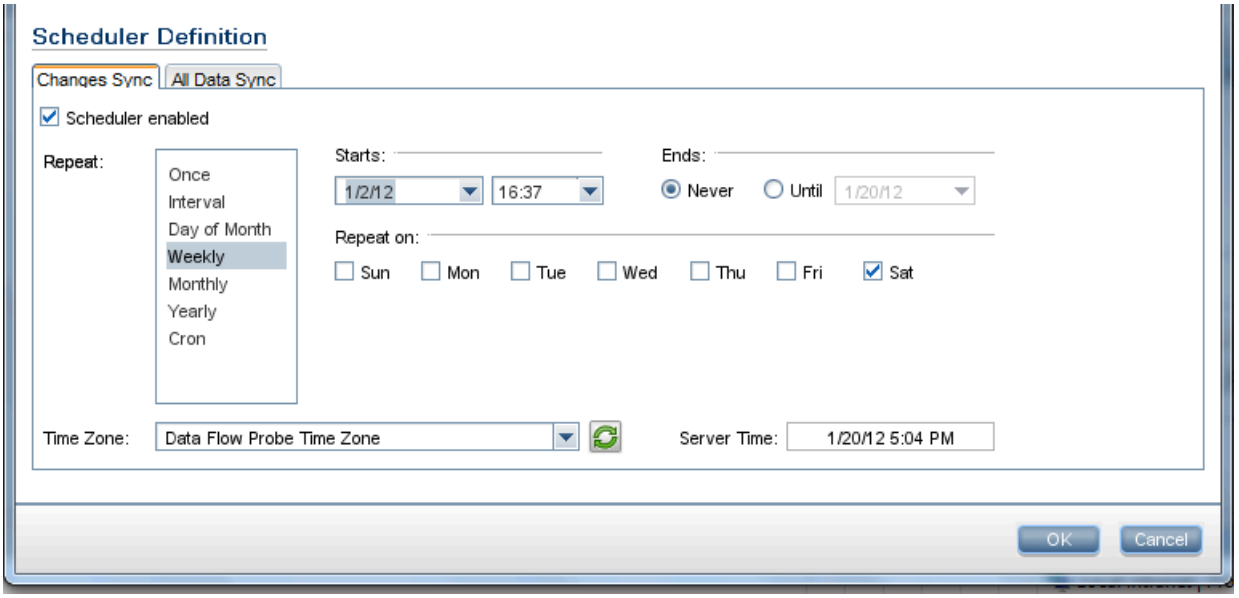
It's a best practice to schedule the data push jobs to match the discovery schedule of your Service Manager feeders. For example, if your Service Manager feeders send CI data updates on a daily schedule, the data push jobs must also run on a daily schedule. By using a matching schedule you can ensure that your Service Manager system always has the most current CI data.

UCMDB allows you to schedule updates directly from a data push job. This task includes the following steps:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**. UCMDB displays a list of integration points.
3. Select the integration point that you created for Service Manager. For example, **SM Integration**.
4. Click the **Data Push** tab.
5. Select a push job. For example, **SM Configuration Item Push Job 2.0**.
6. Click the **Edit Integration Job** icon .

> **Tip**
>
> UCMDB allows you to define two different schedules for two types of data push: Delta Sync, and Full Sync. For recommendations on push scheduling, see Push scheduling recommendations.

7. Define a schedule for Delta Sync. Click the **Delta Synchronization** tab. Select the **Scheduler enabled** option. Select the scheduling options that you want to use.



8. Click the **Full Synchronization** tab, and select the scheduling options that you want to use.

9. Click **OK** to save the data push job.
10. Continue to follow *step 6* to *step 9* for the rest of data push jobs of the integration point.
11. Save the integration point.

1.6.2.2.6.3. How to view UCMDB CI data in Service Manager

After a push job is successfully completed, you can search for and verify the pushed CI/CI relationship data in Service Manager.

CI records pushed from UCMDB contains a **View in UCMDB UI** button, which enables you to access the UCMDB UI to view the CI information.

Note

- You need to specify the UCMDB UI URL in the System Information Record in SM to display the **View in UCMDB UI** button.
- The UCMDB UI is a lightweight UI designed for simple access to UCMDB configuration information. This is a tool for searching, locating, and consuming data that relates to configuration. It's an optional add-on to UCMDB. For more information, refer to the CMS documentation.

To view UCMDB CI data in Service Manager:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Configuration Management** > **Search CIs**.
3. Open a CI record pushed from UCMDB.
4. If the **View in UCMDB UI** button is available, click this button to view the CI record in the UCMDB UI.
5. Open the **Actual State** section.

   Service Manager sends a web services request to UCMDB and displays all CI attributes that the request returns.

   Note

   The web services request uses the UCMDB web service URL and account (for example, admin/admin) defined in the **SMtoUCMDB** integration instance in Service Manager. See How to add UCMDB connection information.

1.6.2.2.6.4. Retrieving UCMDB primary CI change history in SM

When integrated with UCMDB UI, Service Manager displays a **Primary CI History in UCMDB** section in a Problem record whose primary CI is from UCMDB synchronization. You can view the CI changes on that primary CI for root cause investigation.

For information about how to enable an integration with UCMDB UI, see How to add UCMDB connection information.

To view the primary CI change history, follow these steps:

1. Sign in to Service Manager.
2. Navigate to **Problem Management**, and perform a search to open a Problem record whose Primary CI is from UCMDB synchronization.
3. Click the **Primary CI History in UCMDB** tab.

1.6.2.2.7. Federating Service Manager Ticket Data to UCMDB

Federation doesn't physically copy data from Service Manager (SM) to UCMDB; it only retrieves SM data for displaying in UCMDB. Out-of-the-box, the UCMDB-SM integration supports federation for the following external CI types in UCMDB: Incident, Problem, RequestForChange, and ChangeTask. If you have enabled these CI types for federation when creating your integration point, in UCMDB you can retrieve Incident, Problem, Change, and Change Task record data from SM.

Out-of-the-box, each of the CI types (Incident, Problem, Change, and Change Task) supports federation of a subset of their attributes in Service Manager; however, the integration can federate more attributes if tailored as such.

This section includes:

- Federation Queries
- Examples of Using Federation
- How to Add an Attribute of a Supported CI Type for Federation

1.6.2.2.7.1. Federation Queries

Federation uses queries to determine what data to retrieve from Service Manager. To retrieve specific ticket data from Service Manager, you need to create a TQL query first.

The ServiceManagerEnhancedGenericAdapter9-x provides a bunch of sample federation queries, which you can use as a reference. They are available from the following path in UCMDB: **Modeling** > **Modeling Studio** > **Resources** > **Integration** > **Service Manager** > **Federation**.

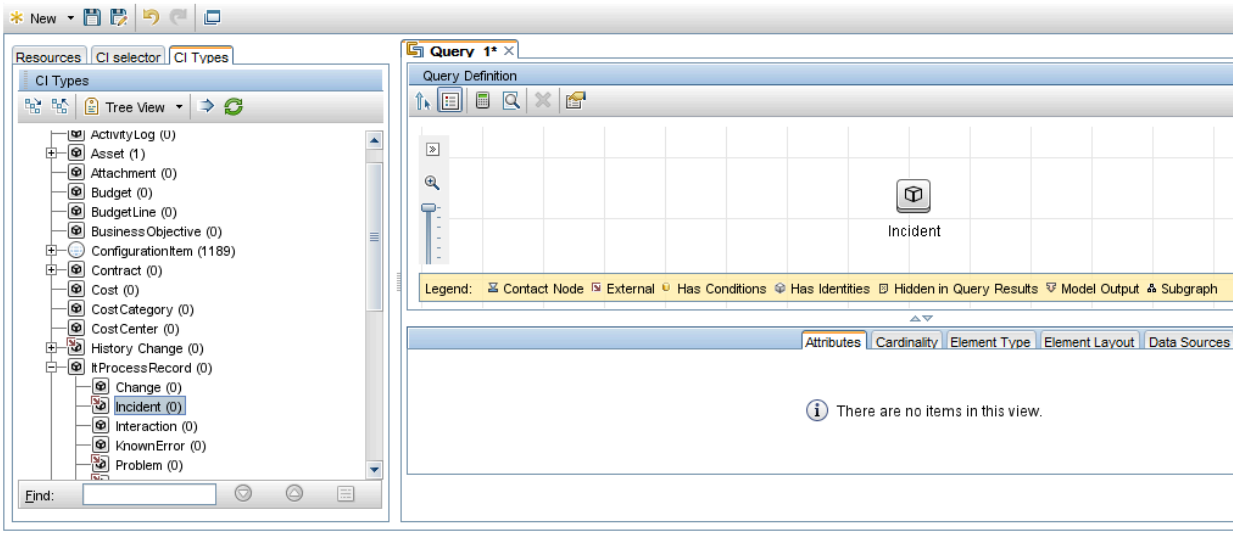
1.6.2.2.7.2. Examples of using federation

You can use the federation feature in many different ways. The following are only examples of using the feature:

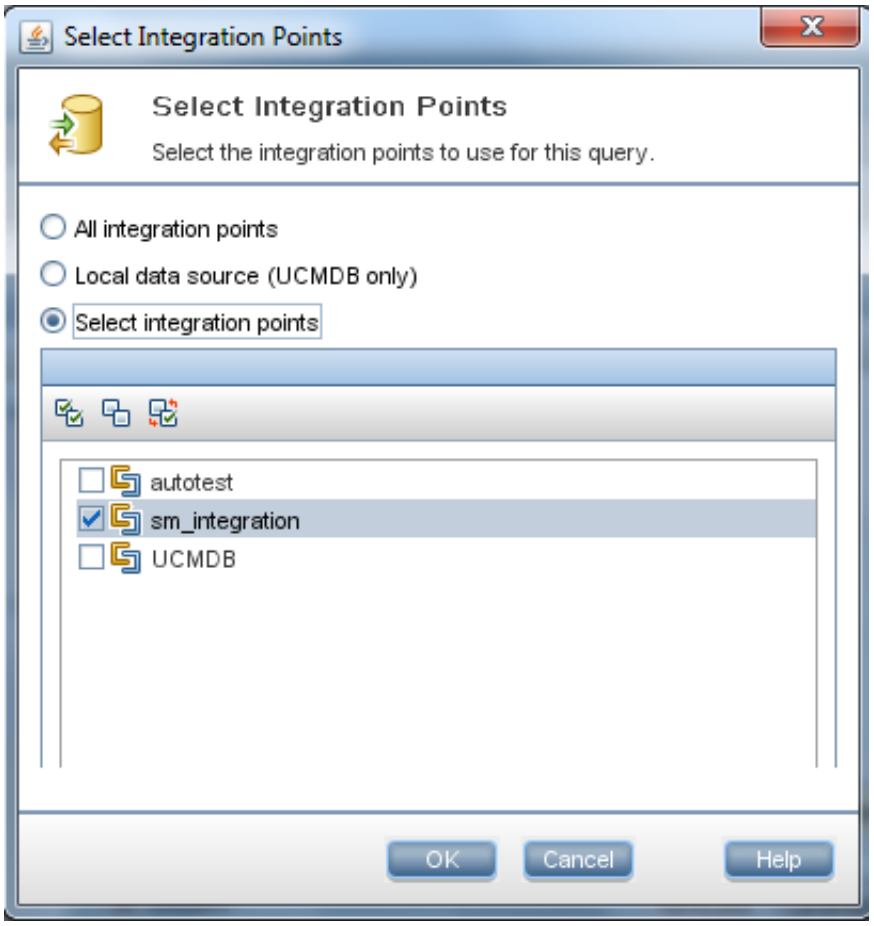
Example 1: Federate all SM Incident tickets

This example illustrates how you retrieve information of all Incident records that exist in Service Manager.

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio** > **Resources**.
3. For Resource Type, select **Queries** from the list.
4. Click **New** > **Query**.
5. On the **CI Types** tab, navigate to **ItProcessRecord** > **Incident**, and drag it to the query pane on the right side.



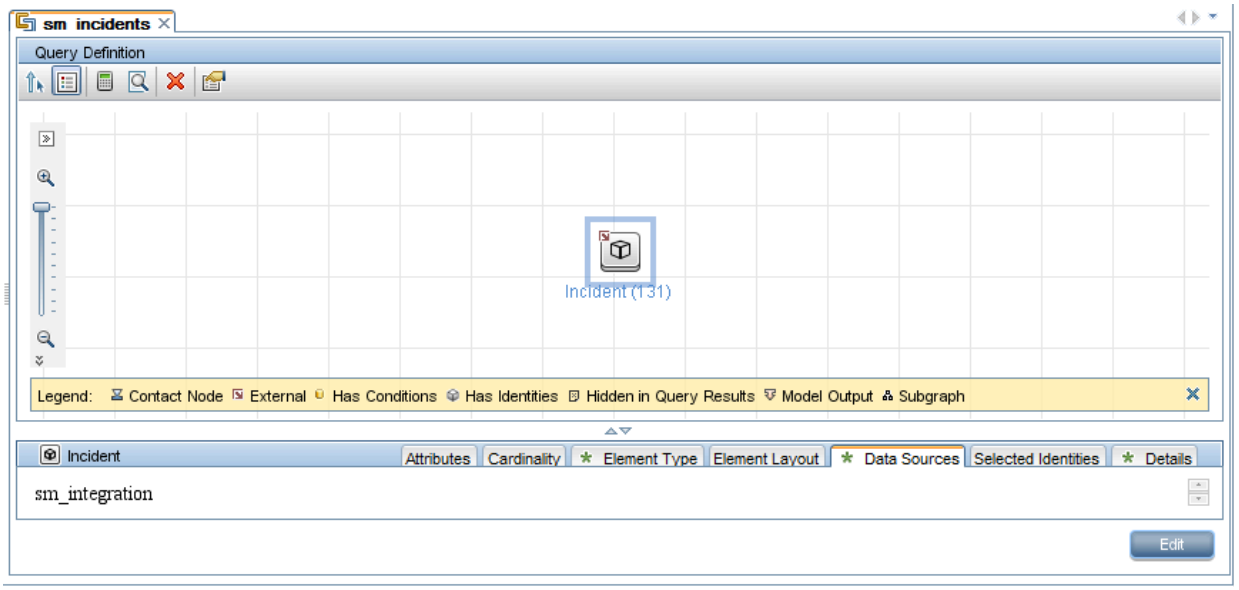
6. Specify Service Manager as the data source for the Incident query node.
    1. Select the Incident query node, click the **Data Sources** tab on the lower right pane, and then click **Edit**.
    2. Select the **Select integration points** option, and then select your integration point name (for example, **sm_integration**). Click **OK**.



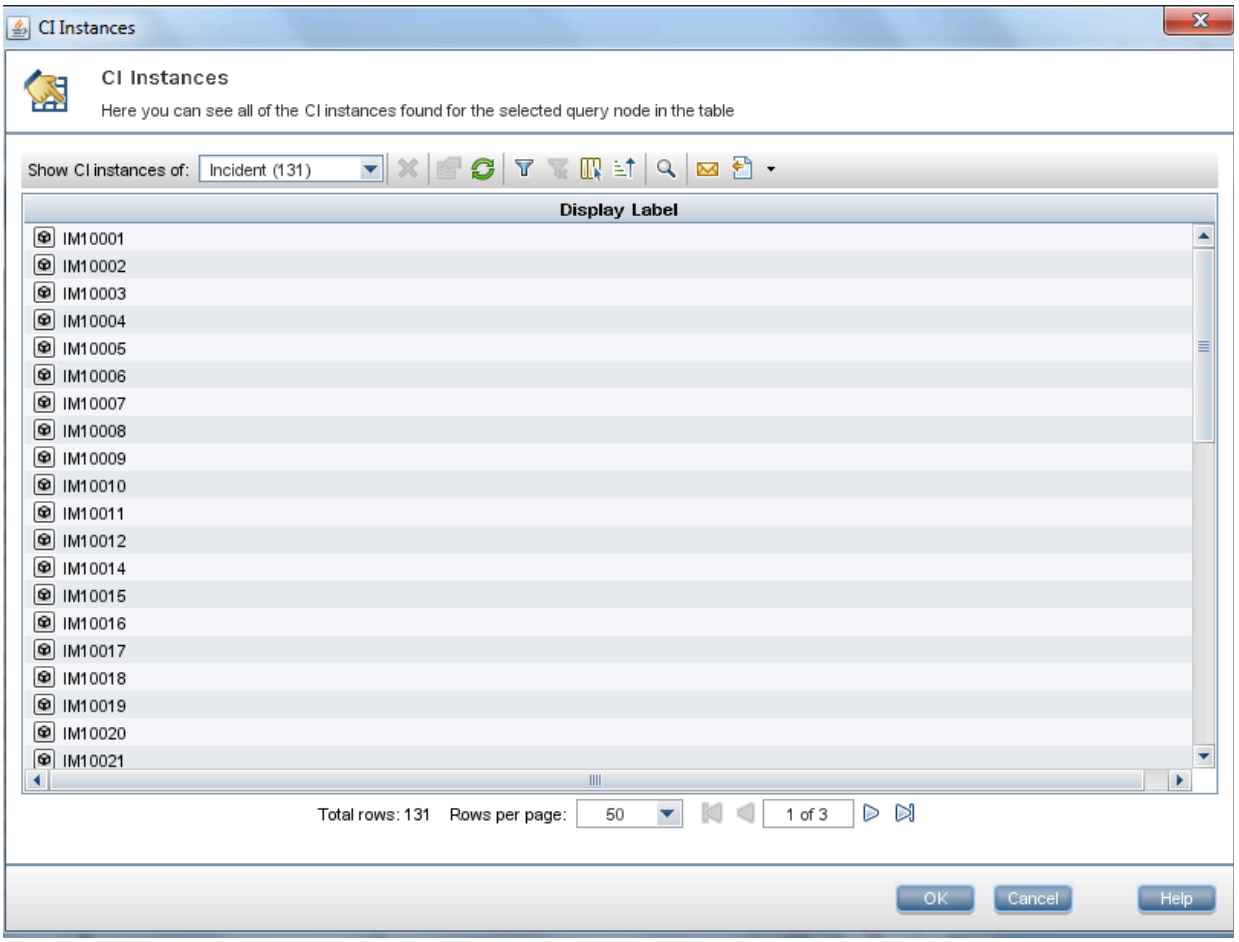
7. Click the **Save** icon , and then type a query name and select a location in which to save the query (for example, select the **Root** folder).

8. Select the Incident query node, and then click the **Calculate Query Result Count** icon .

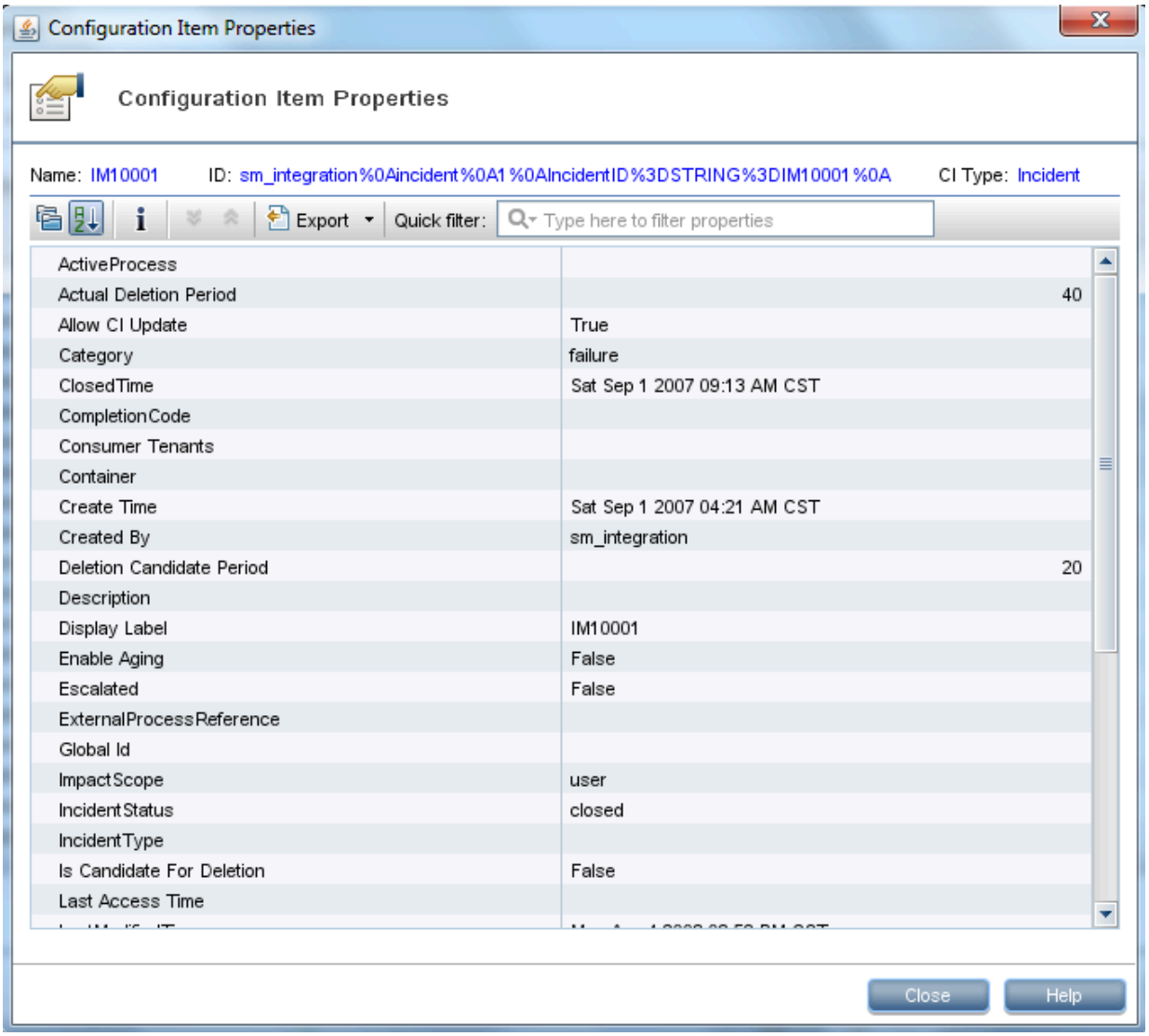
   UCMDB returns the query result count. For example, the following figure shows that there are a total of 116 Incident records in Service Manager.



9. Right-click the Incident query node, and select **Show Element Instances**. UCMDB displays a list of all Incident records that exist in Service Manager.



10. Select an Incident record from the list, and click the **Properties** icon to view its details.

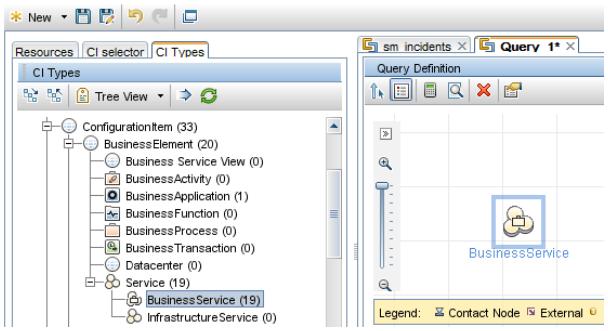


Example 2: Federate SM Incident records that affect a UCMDB Business Service CI

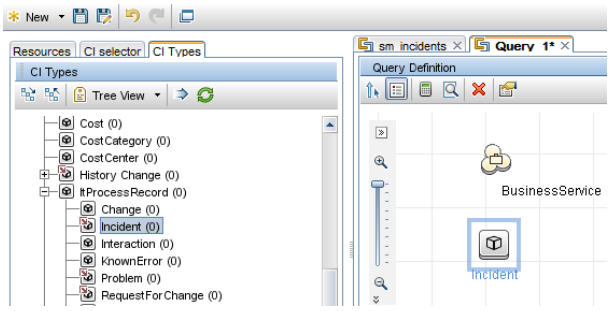
The following example illustrates how to federate a list of Service Manager Incident records whose Affected Service or Affected CI field contains a UCMDB Business Service CI.

In this example, IM10005 in SM has an affected service named **bizservice1**, which was pushed from UCMDB.

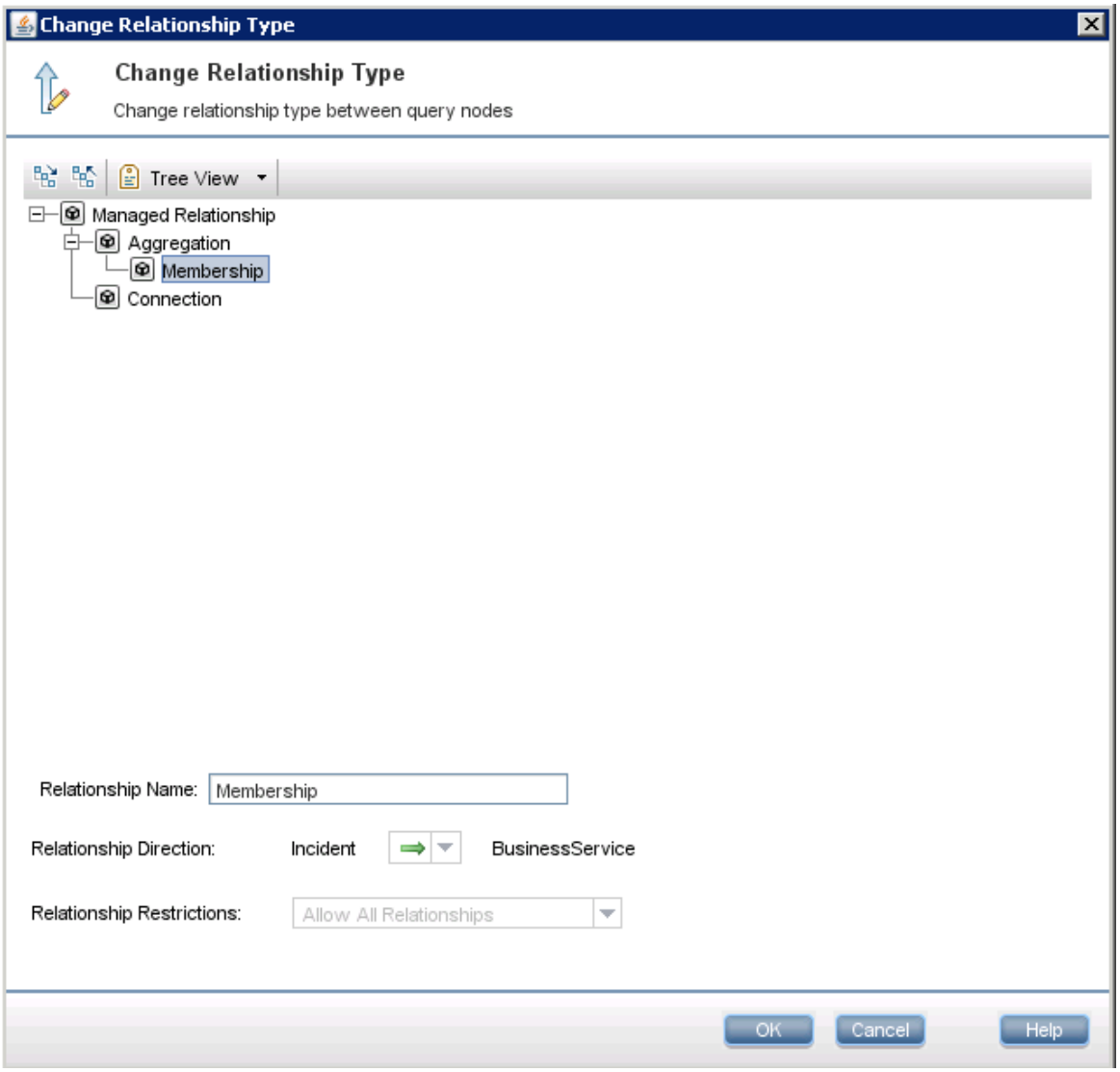
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio** > **Resources**.
3. For Resource Type, select **Queries** from the list.
4. Click **New** > **Query.**
5. On the **CI Type** tab, navigate to **ConfigurationItem** > **BusinessElement** > **Service** > **BusinessService**, and drag it to the query pane on the right side.



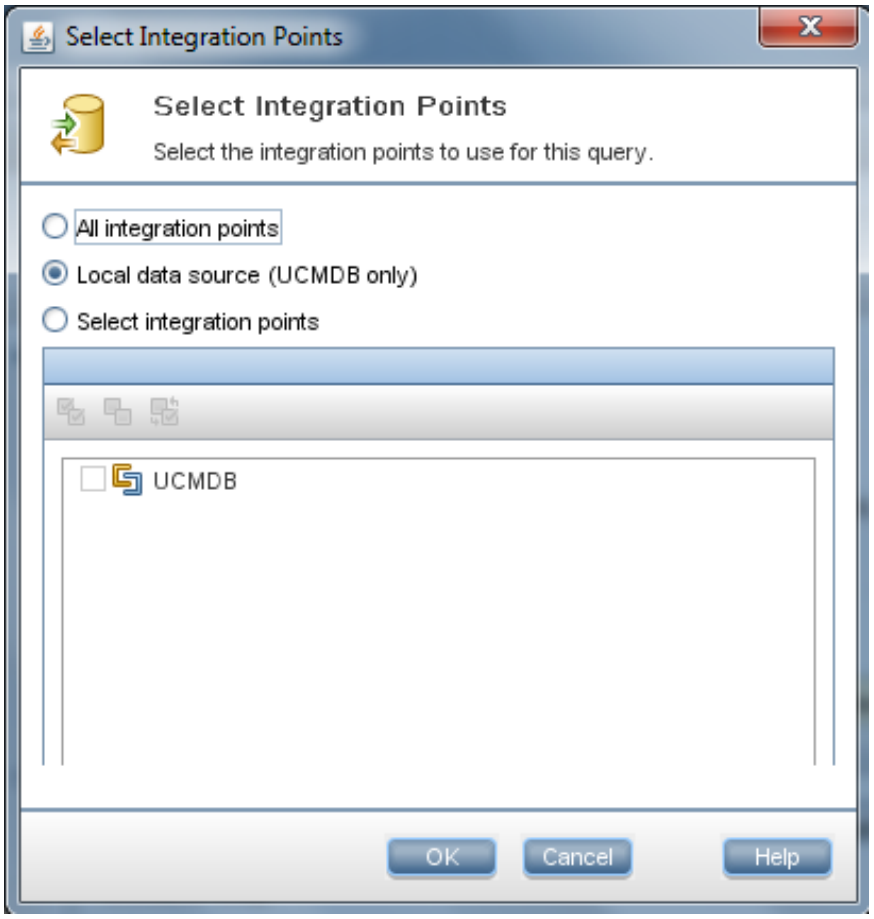
6. Navigate to **ItProcessRecord** > **Incident**, and drag the icon to the query pane.



7. Click the **Create Relationship** icon  .
8. Select the Incident query node, and drag the arrow from this node to the BusinessService node to create a regular relationship between the nodes.
    1. Select **Regular Relationship**, and click **OK**.
    2. Select **Membership**, and optionally enter a relationship name (for example, **Membership**). Click **OK**.

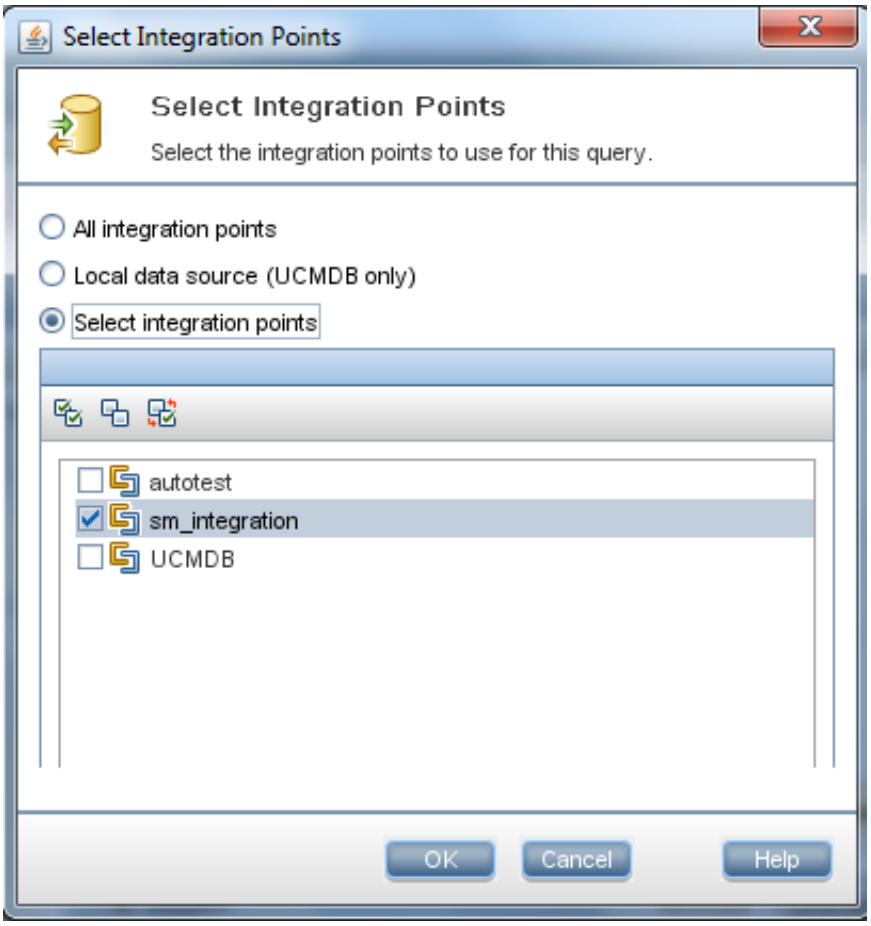


9. Specify UCMDB as the data source for the BusinessService query node. To do this, follow these steps:
    1. Select the **BusinessService** query node.
    2. On the lower right pane, click the **Data Sources** tab and then click **Edit**.
    3. Make sure that the **Local data source (UCMDB only)** option is selected.

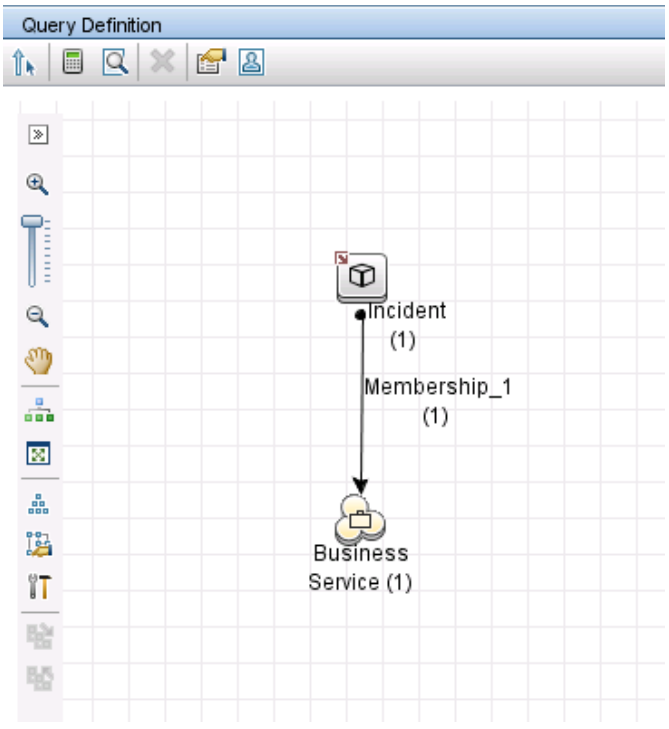


4. Click **OK**.

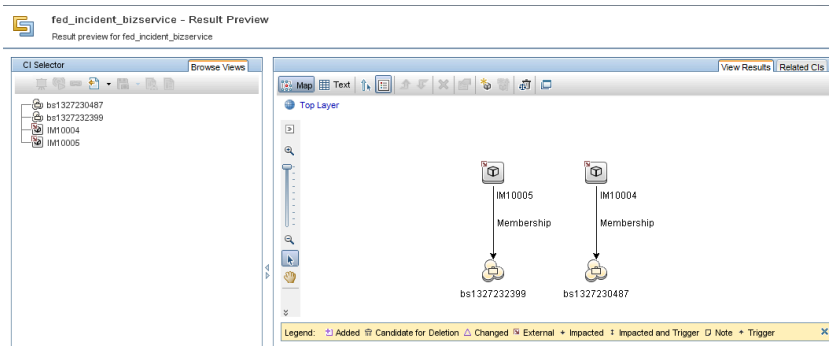
10. Repeat the steps above to specify your integration point as the data source for the Incident query node (for example, **sm_integration**).



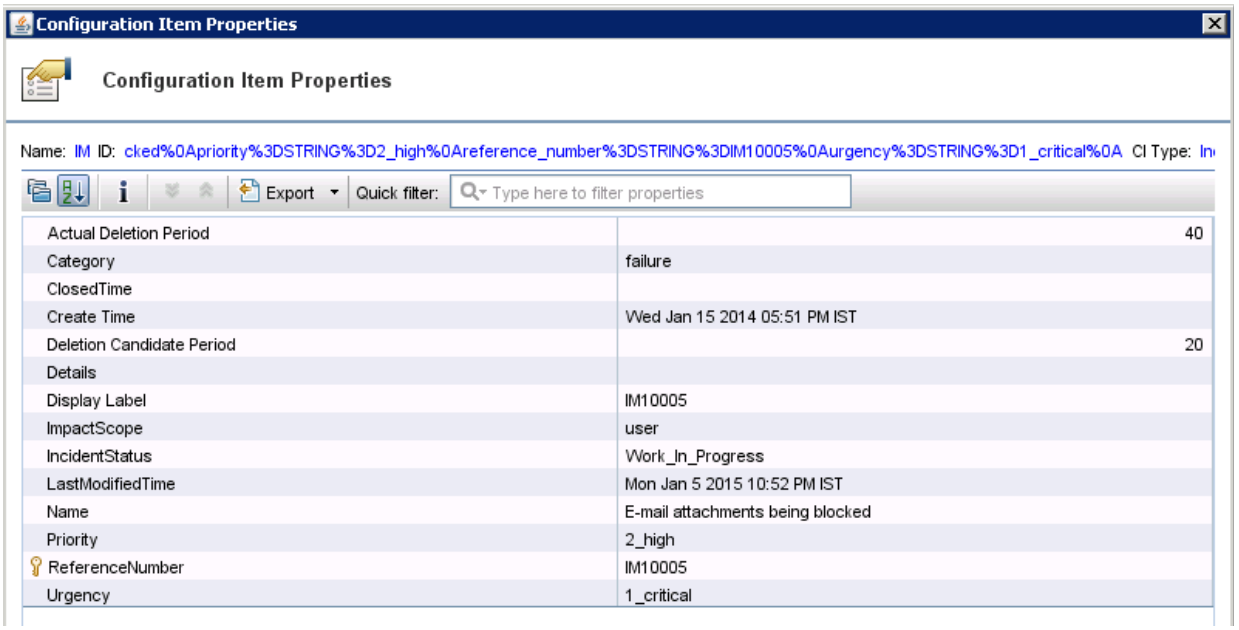
11. Click the **Calculate Query Result Count** icon . The number of SM Incidents and the number of their affected UCMDB Business Service CIs are displayed.



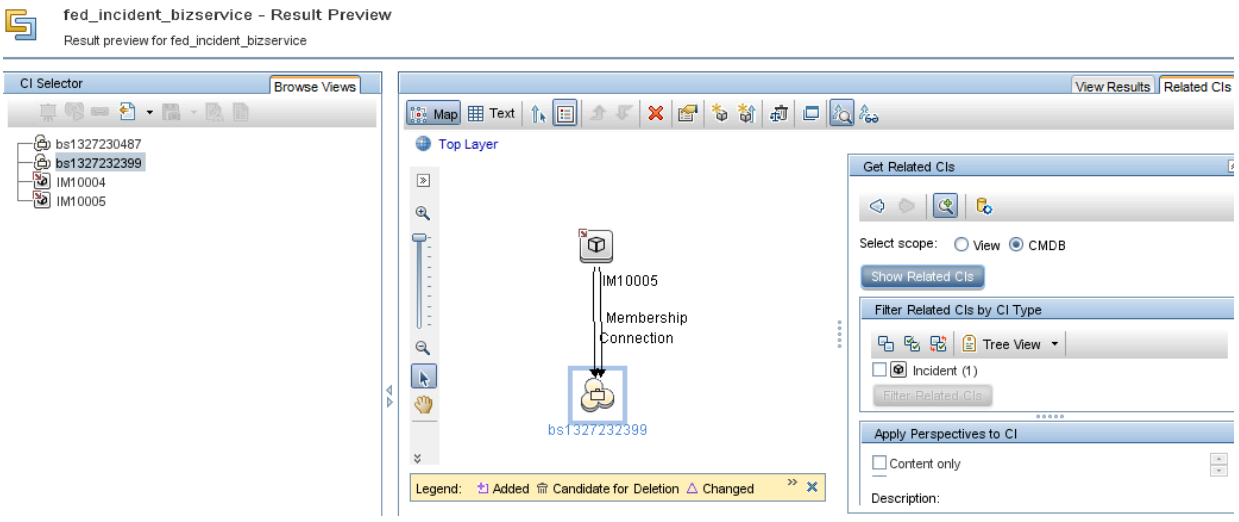
12. Click the **Preview** icon to view the query results .



13. Select each SM Incident record from either the CI Selector pane or the query pane, and click the **Properties** icon to view its details  .



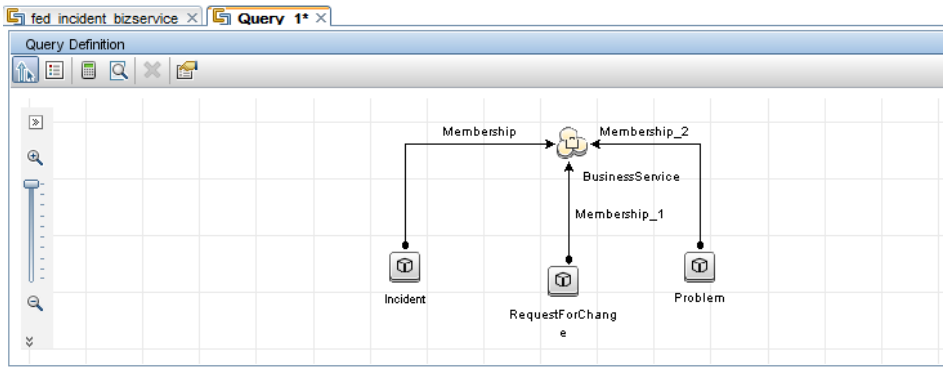
14. Select each UCMDB CI record from the CI Selector pane, and click the **Properties** icon to view its details.



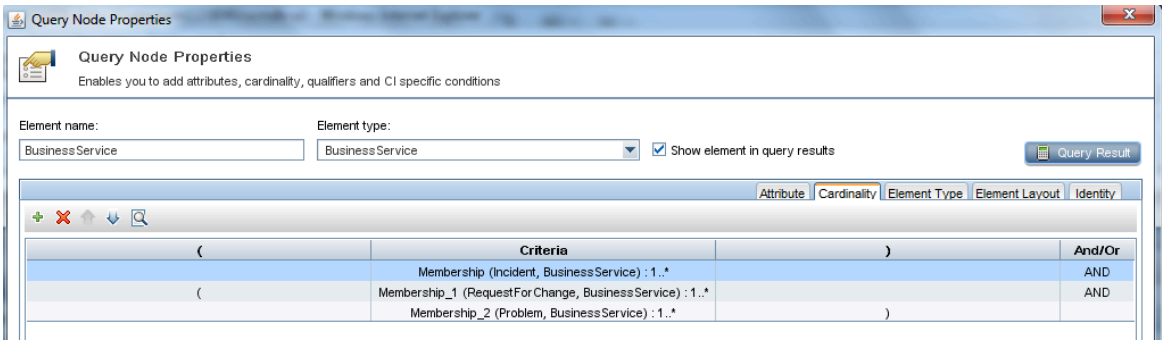
Example 3: Federate Incident, Change, and Problem record data from Service Manager for UCMDB CIs

The following example illustrates how to retrieve information of Incident, Change and Problem records in Service Manager that affect a UCMDB Business Service CI.

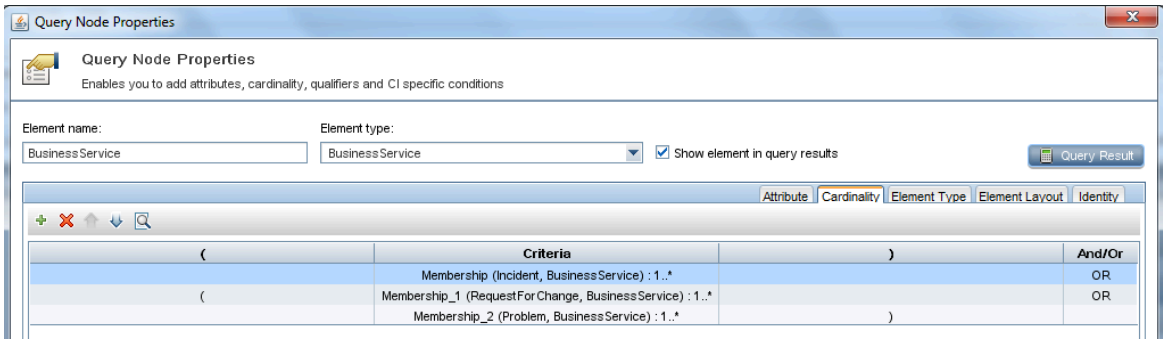
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio** > **Resources**.
3. For Resource Type, select **Queries** from the list.
4. Click **New** > **Query**.
5. On the CI Type tab, go to **ConfigurationItem** > **BusinessElement** > **Service** > **BusinessService**, and drag it to the query pane on the right side.
6. Go to **ItProcessRecord**, and drag **Incident**, **Problem**, and **RequestForChange** to the query pane.
7. Click the **Create Relationship** icon to create regular relationships between the BusinessService node and the other nodes, as shown in the following figure.



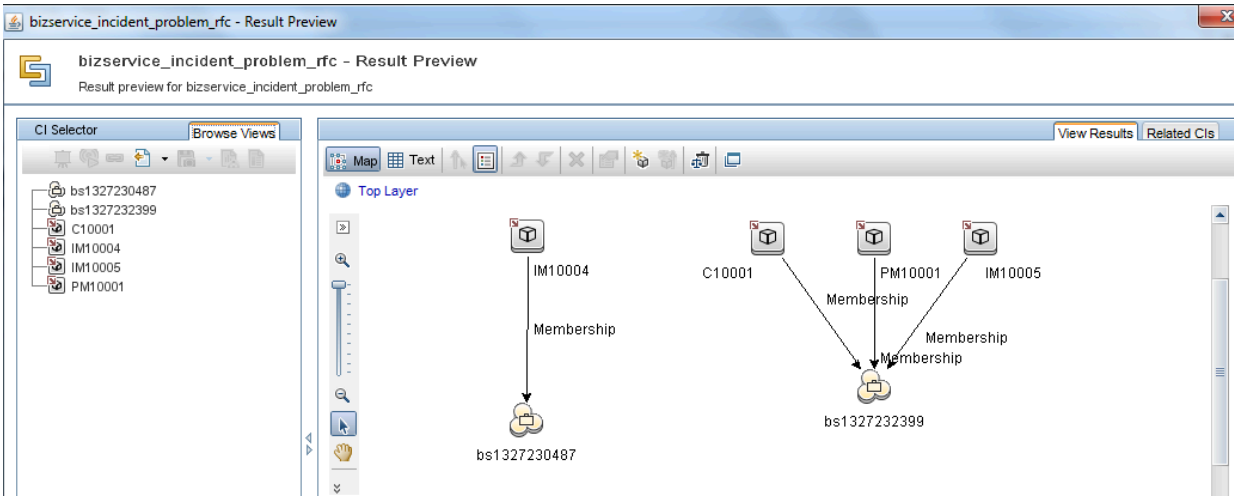
8. Select each node and click the **Data Sources** tab to specify a data source for each node.
   1. For the BusinessService node, specify **UCMDB** as the data source.
   2. For the Incident, Problem, and RequestForChange nodes, specify your integration point as the data source (**sm_integration** in this example).
9. Save the query.
10. Optionally, edit the BusinessService node properties as needed.
    1. Select the BusinessServic node, and click **Edit** on the lower right pane.
    2. Click the **Cardinality** tab. The default Cardinality setting is displayed.



    3. If you wish, change either or both of the **AND** operators to **OR**. This will change the filter criteria and therefore the query results.



11. Click the **Preview** icon to view the query results.

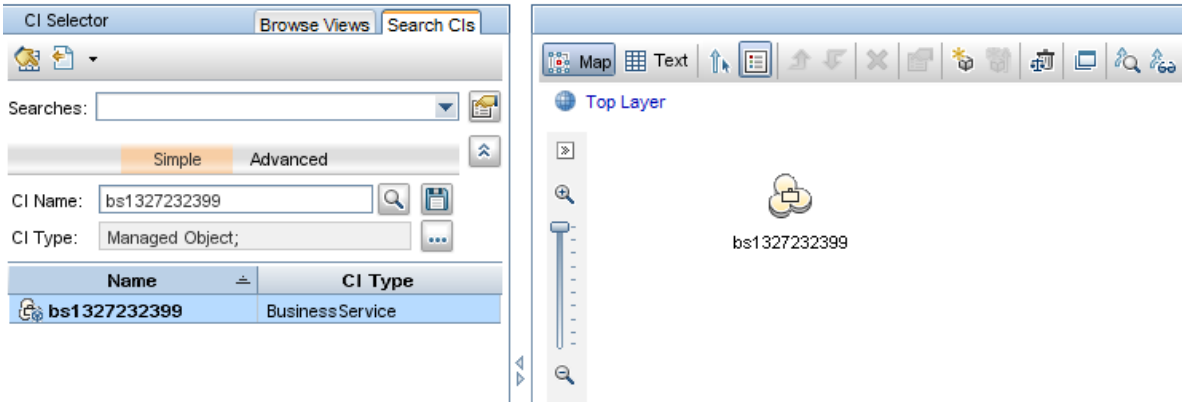


12. Select each SM Incident record from either the CI Selector pane or the query pane, and click the **Properties** icon to view its details.
13. Select each UCMDB CI record from either the CI Selector pane or the query pane, and on the **Related CIs** tab click **Show Related CIs** to view its related CIs in both SM and UCMDB.

Example 4: Retrieve Service Manager records related to a UCMDB CI

The following example illustrates how to retrieve Service Manager records related to a UCMDB CI by using the Get Related CIs functionality.

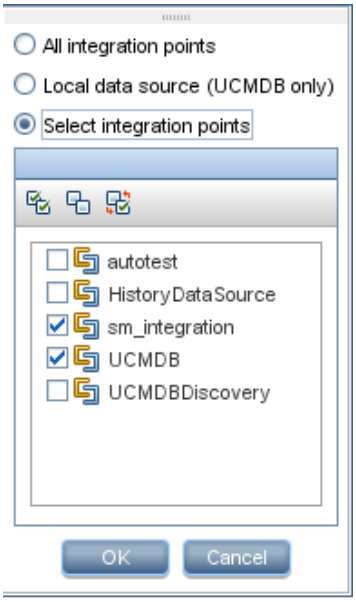
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **IT Universe Manager**.
3. On the **Search CIs** tab, search for a UCMDB CI that has related records in Service Manager. For example, enter bizservice1 in the CI Name field, click **Search**, and then double-click the CI to open it.



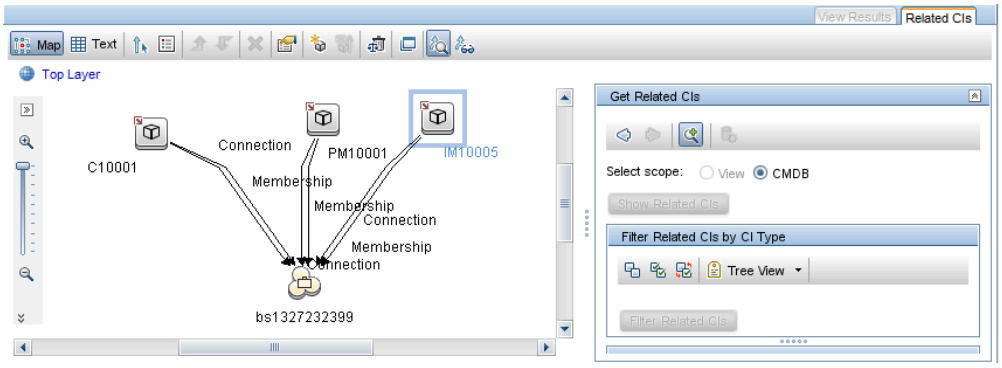
4. If the Get Related CIs pane is not displayed, click the **Show Get Related CIs pane** icon .

5. Click the **Select target Integration Points for related CIs** icon .

6. Select the **Select integration points** option, and then select both **UCMDB** and your integration point. Click **OK**.



7. Click **Show Related CIs**. The CI's related SM records and UCMDB CIs are displayed.



8. Select each SM record from the query pane, and click the **Properties** icon to view its details.

1.6.2.3. Multi-tenancy (Multi-company) setup

The UCMDB-SM Integration supports a multi-tenancy configuration in which both the Service Manager and UCMDB systems track Configuration Items (CIs) and Configuration Item Relationships (CIRs) by company ID. In a multi-tenancy configuration, you can tailor the integration so that each tenant only sees and works with the CIs and CIRs that match their company ID. Multi-tenancy is intended for managed service providers (MSPs) who wish to offer Configuration Management as a service to multiple tenants.

Multi-tenancy (Multi-company) support

Implementing multi-tenancy in the UCMDB-SM integration

Mandanten SM security layer

What multi-tenant information is stored in UCMDB

What multi-tenant information is stored in Service Manager

Synchronization of company records

Tenant-specific Discovery Event Manager (DEM) rules

Multi-tenancy requirements

Setting up the multi-tenancy integration in UCMDB

How to install a separate data flow probe for each tenant

How to start tenant-specific data flow probes

How to configure IP ranges for tenant-specific data flow probes

Setting up the multi-tenancy integration in Service Manager

How to start the schedule process

How to configure the Service Manager system information record

How to add tenant-specific UCMDB user ID and password values

How to add UCMDB customer ID values to existing companies

How to synchronize existing companies from Service Manager to UCMDB

How to view whether company information is in UCMDB

How to resynchronize an existing company with UCMDB

How to inactivate a synchronized company

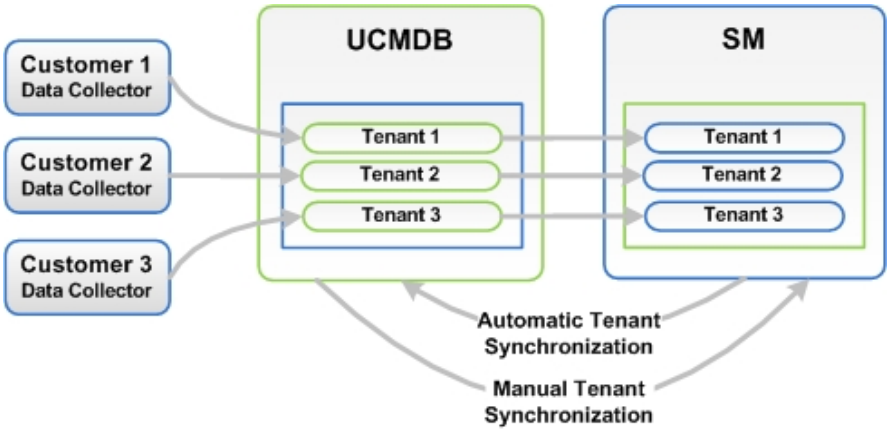
How to reactivate an inactive company

How to add tenant-specific DEM rules

1.6.2.3.1. Multi-tenancy (Multi-company) support

Multi-tenancy is when a single instance of software runs on a server, serving multiple client organizations (also referred to as tenants). Multi-tenancy contrasts with a multi-instance architecture where separate software instances or hardware systems are set up for different client organizations.

When implementing a multi-tenant architecture, a software application is designed to virtually partition its data and configuration so that each client organization works with a customized virtual application instance. The following figure illustrates an example multi-tenant integration deployment.



Every tenant configured in UCMDB works with the relevant tenant in SM. If UCMDB did not configure tenants, the tenant configuration must be activated in order to transfer the configuration from SM to UCMDB automatically. This function is performed once only by the system administrator.

In the event that UCMDB tenant configuration already exists and the SM configuration doesn't exist, SM tenants must be manually configured according to the UCMDB configuration.

1.6.2.3.1.1. Implementing multi-tenancy in the UCMDB-SM integration

SM stores the company records that describe each tenant in the multi-tenant configuration. The Service Manager system is the definitive source for company records and pushes all new company IDs to the UCMDB system creating the equivalent entity in UCMDB.

SM tracks the company ID of each CI and relationship in a multi-tenant configuration. CI records inherit the company ID of the UCMDB feeder that discovered them. Relationship records inherit the company ID of the parent CI in the relationship.

1.6.2.3.1.2. Mandanten SM security layer

Mandanten is an SM software layer that's used to filter the customer ID from the CI information. SM uses the Mandanten to ensure that operators only see CI and relationship records where the CI's company ID matches the operator's company ID. If the view is restricted with Mandanten, then Service Manager also restricts the view to all other related records such as change requests and incidents.

1.6.2.3.1.3. What multi-tenant Information is stored in UCMDB

Your UCMDB system stores a company ID attribute for each CI and CIR. The company ID determines what adapter and synchronization schedule your UCMDB system uses to update CI data. Each CI and relationship record can only have one company ID. The UCMDB system obtains a company ID from the Service Manager system.

If more than one tenant (company) shares the same CI, each tenant has their own unique CI record describing the CI. In effect, the UCMDB system creates multiple CI records to track one managed asset. Each tenant's CI record is unique to that tenant and lists the company's unique company ID.

1.6.2.3.1.4. What multi-tenant information is stored in Service Manager

Your Service Manager stores the company records that describe each tenant in the multi-tenant configuration. The Service Manager system is the definitive source of company IDs and pushes new and updated information to your UCMDB system.

Service Manager tracks the company ID of each CI and relationship in a multi-tenant configuration. CI records inherit the company ID of the UCMDB feeder that discovered them. Relationship records inherit the company ID of the parent CI in the relationship.

In a best practices implementation, Service Manager uses Mandanten to ensure that operators only see CI and relationship records where the CI's company ID matches the operator's company ID. If you restrict the view with Mandanten, then Service Manager also restricts the view to all other related records such as change requests and incidents.

1.6.2.3.1.5. Synchronization of company records

If your system meets all the conditions for multi-tenancy support, Service Manager creates a schedule record to push the company ID of the company record to your UCMDB system. Service Manager uses the following rules to determine whether to push the company ID to your UCMDB system.

Conditions where Service Manager synchronizes company ID with UCMDB

Conditions	Tenant information synchronized?	Schedule record created and action taken in UCMDB
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You create a new company record in Service Manager	Yes	Synch Company with UCMDB - *<UCMDB Company ID>* • Add new company ID
• UCMDB-SM integration **enabled** • You update an existing company record that has not been synchronized with UCMDB • Multi-company mode **enabled** in Service Manager	Yes	Synch Company with UCMDB - *<UCMDB Company ID>* • Add new company ID
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You disable the option to show a company in multi-company lists on a company synchronized with UCMDB	Yes	Inactivate Company with UCMDB - *<UCMDB Company ID>* • Inactivate existing company ID

Conditions	Tenant information synchronized?	Schedule record created and action taken in UCMDB
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You select the option to resynchronize with UCMDB on an existing company record	Yes	Synch Company with UCMDB - *<UCMDB Company ID>* • Add new company ID
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You enable the option to show a company in multi-company lists for an inactivated company	Yes	Synch Company with UCMDB - *<UCMDB Company ID>* • Reactivate company ID
• UCMDB-SM integration **disabled** • Multi-company mode **enabled** in Service Manager • You update an existing company record that has already been synchronized with UCMDB	No	None

Conditions	Tenant information synchronized?	Schedule record created and action taken in UCMDB
• UCMDB-SM integration **disabled** • Multi-company mode **enabled** in Service Manager • You create a new company record in Service Manager	No	None
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You disable the option to show a company in multi-company lists on a company not synchronized with UCMDB	No	None
• UCMDB-SM integration **enabled** • Multi-company mode **disabled** in Service Manager • You create a new company record in Service Manager	No	None

UCMDB Customer ID

When you enable the multi-tenancy integration, Service Manager displays a new field in each company record called UCMDB Customer ID. In order to synchronize a company record with UCMDB, you must first provide a value for this field. After you provide a UCMDB Customer ID value this field becomes read-only. You can't change a company's UCMDB Customer ID after you set it.

This field only accepts numeric data up to ten characters long. Service Manager requires the field value to be a unique positive whole number. You can't enter duplicate values or use decimals, negative numbers, or zero.

Your UCMDB system automatically uses the UCMDB customer ID of 1 when running in single tenant mode. You can reuse this default value in your multi-tenant implementation by assigning a Service Manager company to have this UCMDB customer ID value. Out-of-the-box, no Service Manager company has the UCMDB customer ID of 1.

UCMDB User ID and Password

When you enable the multi-tenancy integration, Service Manager displays two new fields in each company record called UCMDB UserId and UCMDB Password. These fields allow you to specify the connection information you want Service Manager to use when requesting information for the Actual State section. Any user name and password you enter in these fields must be valid for your UCMDB system.

The user name and password you provide in the Company Information record takes precedence over the user name and password you provide in the System Information record. This allows managed service providers to control access to the UCMDB system on a tenant-by-tenant basis. If you don't provide a company-specific UCMDB user name and password, Service Manager uses the credentials you provided in the System Information record.

Company Code

The multi-tenancy integration requires that each company record has a unique Company Code (company field) value. Since Company Code is a required field, your existing company records should already have Company Code values. However, you should ensure that each company record has a unique Company Code value.

When using Service Manager Enhanced Generic Adapter for UCMDB-SM integration, you must define your company codes in the ServiceManagerEnhancedAdapter9.41/mappings/scripts/SMUtils.groovy file using the following format (where a comma is used to separate mapping entries):

```
["<UCMDB Company 1 Code>":"<SM Company 1 Code>", "<UCMDB Company 2 Code>":"<SM Company 2 Code>"]
```

For example: `["1":"microfocus", "2":"sap"]`



Caution

You should not change the Company Code value after you have enabled the multi-tenancy integration because this will cause your Service Manager data to become out of synch.

CI Reconciliation Rules

CIs that are pushed from UCMDB are identified only by their global.id value in UCMDB. By default, only the ucmdb.id field in the **device** table is used in CI reconciliation (the company.id field is not used).

You can add reconciliation rules manually through **Tailoring** > **Web Services** > **DEM Reconciliation Rules**. For details, see How to add DEM Reconciliation Rules.

Company information pushed to CI and CI relationship records

When you enable the multi-tenancy integration, Service Manager inserts the SM Company Code value in CI and relationship records during data push. Service Manager uses the UCMDB Customer ID to look up the matching SM Company Code value.

Company information replicated to Incident records

When you enable the multi-tenancy integration and select the option to create incidents when UCMDB discovers new, updated, or deleted CIs, Service Manager inserts the SM Company Code value in the incident record during replication. Service Manager uses the UCMDB Customer ID to look up the matching SM Company Code value.

Schedule records

Service Manager uses the **problem** schedule process to manage the synchronization of company IDs to your UCMDB system. You can manually enable the **problem** schedule process from the System Status form.

When the synchronization criteria are met as described in Table *Conditions where Service Manager synchronizes company ID with UCMDB*, Service Manager creates a “Synch Company with UCMDB - *<UCMDB Company ID>*” schedule record (for example, “Synch Company with UCMDB - 1234567890”). If you inactivate a company, Service Manager creates an “Inactivate Company with UCMDB - *<UCMDB Company ID>*” schedule record (for example, “Inactivate Company with UCMDB - 1234567890”). The **problem** schedule process processes the new schedule record on the next background process iteration.

If your Service Manager system can't connect to your UCMDB system for some reason, it will reschedule the company synchronization at the next scheduled interval (the out-of-the-box interval is 5 minutes). The problem schedule process updates the schedule record with the status rescheduled. If the Service Manager system receives any other error message while connecting to the UCMDB system, it updates the schedule record with the status “application failed due to error - check msglog for possible messages.”

1.6.2.3.1.6. Tenant-specific Discovery Event Manager (DEM) rules

You can implement the condition field function in order to create SM DEM rules that are specific to a particular tenant in a multi-tenancy UCMDB-SM integration.

Tenant rules vary according to SM tenant configuration requirements, for each record information type pushed from UCMDB to SM different tenants can configure different DEM tenant rules.

Each tenant can have its own set of unique requirements and therefore may implement different processes through the integration.

One tenant may require the addition of CIs directly to SM while another tenant may require opening changes for each CI.

The following table shows a sample set of DEM rules that illustrate how to accomplish this.

Tenant-specific DEM rules

DEM rule ID	Action on new CI	Condition
ucmdbNode_advantage	Add CI	company in $L.file="advantage"
ucmdbNode_hp	Create change	company in $L.file="HP"

Note

When creating DEM rules, be sure to create separate DEM rules for each tenant.

Multi-tenancy use cases

The following table describes the necessary actions to perform in various deployment situations to address multi-tenancy issues.

Multi-tenancy use cases

Deployment Integration Type	Description
UCMDB with multi-tenancy rules SM without multi-tenancy rules	When implementing a UCMDB-SM deployment that has existing multi-tenancy rules in UCMDB and doesn't have multi-tenancy rules configured in SM, the user creates multi-tenancy rules in SM manually and according to the rules in UCMDB.
SM with multi-tenancy rules UCMDB without multi-tenancy rules	When implementing a UCMDB-SM deployment that has existing multi-tenancy rules in configured SM and doesn't have multi-tenancy rules configured in UCMDB, the user creates multi-tenancy rules in UCMDB manually as well as according to the rules previously configured in SM.
UCMDB without multi-tenancy rules SM without multi-tenancy rules	When implementing a UCMDB-SM deployment that doesn't have multi-tenancy rules configured in UCMDB or in SM, the user configures the rules in SM. During the configuration process using the SM multi-tenancy wizard, the user can create a corresponding tenancy configuration in UCMDB. By creating corresponding tenancy configurations in SM the user also creates a corresponding tenant in UCMDB.

1.6.2.3.2. Multi-tenancy requirements

Your system must meet the following requirements in order for the integration to support multi-tenancy.

- Integration enabled between UCMDB and Service Manager
- Multi-company mode enabled on the Service Manager system
- Problem schedule process running on the Service Manager system

For additional information about the multi-tenancy integration, you can visit Software Support Online or refer to the Service Manager Help Center.

1.6.2.3.3. Setting up the multi-tenancy integration in UCMDB

You need to perform the following tasks in UCMDB to set up the multi-tenancy integration.

1. Install a separate data flow probe for each tenant the integration will support.

   See How to install a separate data flow probe for each tenant.

2. Start tenant-specific data flow probes.

   See How to start tenant-specific data flow probes.

3. Configure IP address ranges for tenant-specific data flow probes.

   See How to configure IP ranges for tenant-specific data flow probes.



Caution

Multi-tenancy is unsupported for population when the integration uses the Service Manager Enhanced Generic Adapter.

1.6.2.3.3.1. How to install a separate data flow probe for each tenant

If you plan to support a multi-tenant configuration, you must install a separate data probe for each tenant. Out-of-the-box, the UCMDB installer only installs one data flow probe and service.

To install additional data flow probes and start them from your operating system command prompt:

1. Sign in to the host of your UCMDB system as an administrator.
2. Insert the Universal CMDB Setup Windows DVD into the system disc drive.
3. Start the Data Flow Probe installer (HPUCMDB_DataFlowProbe_x.xx.exe).
4. Follow the on-screen instructions to complete the wizard, but use the following values for each data flow probe you wish to install.
    1. Type a unique path for each installation folder.
    2. Use the same UCMDB application server address for each data flow probe.
    3. Type a valid data flow probe address.
    4. Type a unique name for each data flow probe identifier.
    5. Create a unique customer Data Flow Probe domain for each probe (Clear the Use Default CMDB Domain option).
    6. Use the same probe gateway and probe manager settings for each probe (for example, use combined or separate processes).

    See the *Universal CMDB Deployment Guide* for complete installation instructions.
5. Repeat *step 3* and *step 4* for each data flow probe you wish to install.
6. Open the probe's `DiscoveryProbe.properties` file in a text editor. By default, this file is located in the following folder:

    `<UCMDB installation folder>\<data flow probe installation folder>\conf`

    For example, `C:\UCMDB\DataFlowProbe\conf`.

    Note

    The <data flow probe installation folder> must be unique for each tenant.
7. Edit the following properties in the configuration file.

Discovery Probe properties set for each tenant

Property	Value
serverName	Verify the name of the UCMDB server
customerId	Type the customer ID for the tenant this data flow probe supports
appilog.collectors.probe.name	Verify the probe name is unique such as server + tenant ID
applilog.collectors.domain	Verify the domain name of the data flow probe
appilog.collectors.local.ip	Verify the data flow probe gateway name
appilog.collectors.probe.ip	Verify the data flow probe manager name
appilog.collectors.rmi.port	Type a unique port for each probe
appilog.collectors.rmi.gw.port	Type a unique port for each probe
.port	Type a unique port for each probe
.port	Type a unique port for each probe
appilog.collectors.ProbeUseSpecific RMIPortFrom	Type a unique port for each probe or type 0 to have the system automatically select it
appilog.collectors.bigBrother.port	Type a unique port for each probe

8. Save the configuration file.
9. Repeat *step 6* to *step 8* for the data flow probe of each tenant.

1.6.2.3.3.2. How to start tenant-specific data flow probes

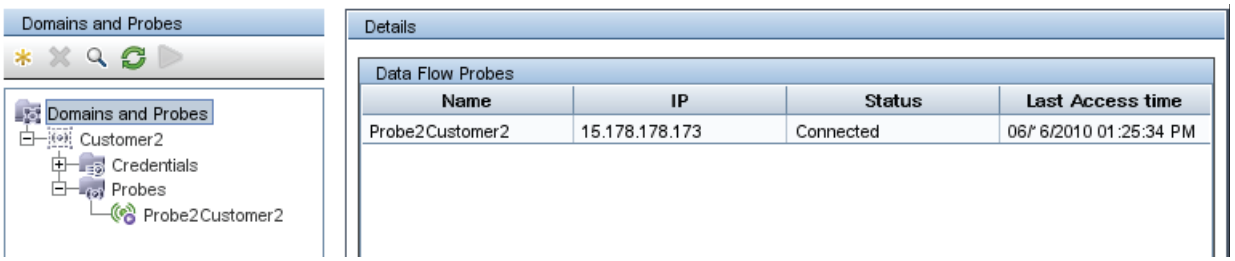
To start tenant-specific data flow probes:

1. Open the OS command prompt and navigate to the probe's bin folder. For example, `C:\UCMDB\DataFlowProbe1\bin`.
2. Type gateway console.
3. Repeat *step 1* to *step 2* for each data flow probe you want to start.

1.6.2.3.3.3. How to configure IP ranges for tenant-specific data flow probes

To configure IP ranges for tenant-specific data flow probes:

1. Sign in to UCMDB as an administrator using the company ID of the tenant whose data flow probe you want to configure.
2. Navigate to **Data Flow Management** > **Data Flow Probe Setup**.
3. Expand the data flow probe domain containing the probe you want to start. For example, **Customer2**.
4. Expand the Probe node and select the data flow probe you want to start. For example, **Probe2Customer2**.



5. Click the **Add IP range** icon .
6. Type an IP range you want the Data Flow Probe to scan. Optionally, add any IP ranges you want to exclude.
7. Click **OK** to save the IP range.
8. Continue to follow *step 1* to *step 7* for each data flow probe you want to configure.

1.6.2.3.4. Setting up the multi-tenancy integration in Service Manager

You need to perform the following tasks in Service Manager to set up the multi-tenancy integration.

Multi-tenancy support is an optional feature of the integration intended for Managed Service Providers (MSPs) who want to offer Configuration Management as a service to their tenants. In a multi-tenancy configuration, each CI and CIR record has a corresponding company ID. Out-of-the-box, Service Manager allows all operators to view CI data regardless of the company ID. If you wish to restrict access to CI data by company ID, you must enable Mandanten and use the company ID field as a restricting query. See the Service Manager help for more information about multi-company mode and Mandanten.

You must complete the following tasks from your Service Manager system to enable multi-tenancy support for the integration.

1. Start the process schedule.

   See How to start the schedule process .

2. Configure the Service Manager System Information Record and Integration Manager.

   See How to configure the Service Manager System Information Record and Integration Manager.

3. Add tenant-specific UCMDB ID and password values to company records (optional).

   See How to add tenant-specific UCMDB user ID and password values.

4. Add UCMDB Customer ID values to existing company records.

   See How to add UCMDB customer ID values to existing companies.

5. Synchronize existing company records with UCMDB.

   See How to synchronize existing companies from Service Manager to UCMDB.

6. Verify that Service Manager synchronized company records with UCMDB (optional).

   See How to view whether company information is in UCMDB.

7. Resynchronize existing company records with UCMDB (as needed).

   See How to resynchronize an existing company with UCMDB.

8. Inactivate company records you don't want to be part of the integration (as needed).

   See How to inactivate a synchronized company.

9. Reactivate inactive company records you want to be part of the integration (as needed).

See How to reactivate an inactive company

10. Add tenant-specific DEM rules.

    See How to add tenant-specific DEM rules.

1.6.2.3.4.1. How to start the schedule process

This integration needs the **problem** schedule process to synchronize company records from Service Manager to UCMDB. Make sure that the process gets started before you synchronize company records.

To start the **problem** schedule process:

1. Sign in to Service Manager as a system administrator.
2. From the System Navigator, click **System Status**.

   Service Manager displays a list of the currently started schedules.
3. Click the **Refresh Display** button to refresh the list.
4. If the **problem** schedule process isn't in the list, perform the following steps:

   1. Click the **Start Scheduler** button.
   2. Double-click the **problem** schedule process.

   Service Manager displays a message, indicating that the **problem** schedule process gets started.

1.6.2.3.4.2. How to configure the Service Manager System Information Record and Integration Manager

To enable the integration to support multi-tenancy, you must enter additional information in the Service Manager System Information Record.

To enter additional information in the Service Manager System Information Record, follow these steps:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Miscellaneous** > **System Information Record**.
3. Click the **General** tab.
4. Enable the **Run in Multi-Company Mode** option and then click **Save**.

   Service Manager displays the message: Information record updated.
5. Click **Tailoring** > **Integration Manager**.
6. Click **Add**.
7. In the **Integration Template** field, select **SMtoUCMDB**.

   Some fields in the form are automatically populated with a value. You can ignore the empty fields.
8. (Optional) Update the **Name**, **Log Level**, and **Description** fields as needed.
9. Configure the following fields on the **UCMDB Server Configuration** tab.
   1. In the **UCMDB web service URL** field, type the URL to the synchronize CIs web service API. The URL has the following format:

      `http://<UCMDB server name>:<port>/axis2/services/ucmdbSMService`

      Replace `<UCMDB server name>` with the host name of your UCMDB server, and replace `<port>` with the communications port your UCMDB server uses.
   2. In the **User Id** and **Password** fields, type the user credentials required to manage CIs on the UCMDB system. For example, the out-of-the-box administrator credentials are **admin/admin**.
   3. In the **Multi-tenant web service URL** field, type the URL to the synchronize company IDs web service API. The URL has the following format:

      `http://<UCMDB server name>:<port>/axis2/services/UcmdbManagementService`

Replace `<UCMDB server name>` with the hostname of your UCMDB server, and replace `<port>` with the communications port your UCMDB server uses.

4. Type the user name and password required to synchronize company IDs on the UCMDB system. For example, the out-of-the-box system administrator credentials for UCMDB 9.x are **sysadmin/sysadmin**.

10. Click **Finish**.
11. Sign out of the Service Manager system, and sign in again with an administrator account.
12. Click **System Status** > **Display Options** > **All Tasks**.
13. Type k in the Command field next to the **problem** schedule process and click **Execute Commands**. Wait a few minutes for the problem schedule process to close.
14. Click **Start Scheduler**.
15. Double-click the **problem** schedule process. The system now supports multi-tenancy for UCMDB.

1.6.2.3.4.3. How to start the process schedule

This integration needs the process schedule to synchronize company records from Service Manager to UCMDB. Make sure to start the process before you synchronize company records.

To start the process schedule:

1. Sign in to Service Manager as a system administrator.
2. From the System Navigator, click **System Status**.

   Service Manager displays a list of the currently started schedules.
3. Click the **Refresh Display** button to refresh the list.
4. If the problem schedule isn't in the list, do the following:

   1. Click the **Start Scheduler** button.
   2. Double-click the **process** schedule.

   A message displays that indicates the **process** schedule has started.

1.6.2.3.4.4. How to add tenant-specific UCMDB user ID and password values

You can enter a tenant-specific UCMDB user name and password for Service Manager to use when requesting information for the Actual State section. If you enter no credentials, Service Manager uses the credentials in the System Information Record for all tenants.



Note

Any credentials you entered in the company record take precedence over credentials you entered in the System Information Record. The **UCMDB UserId** and **UCMDB Password** fields are available only when you have enabled the multi-tenancy integration.

To add tenant-specific UCMDB user name and password:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Type the user name you want this company to use to connect to UCMDB in the **UCMDB UserId** field.
6. Type the password for the UCMDB user name in the **UCMDB Password** field.
7. Click **Save**.
8. Repeat *step 3* through *step 7* for each company you want to enter credentials for.

1.6.2.3.4.5. How to add UCMDB customer ID values to existing companies

You can use the following steps to add a UCMDB Customer ID value to your existing Service Manager company records.

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Type a numeric value in the UCMDB Customer ID field for this company.
6. Click **Save**.
7. Service Manager prompts to confirm that you want to synchronize the record with UCMDB. Click **Yes** if you want to synchronize the company now, or click **No** if you want synchronize the company later.
8. Click **Next** to go to the next company in the record list.
9. Repeat *step 5* through *step 8* for each company in the record list.

1.6.2.3.4.6. How to synchronize existing companies from Service Manager to UCMDB

Your Service Manager system may already contain company records that you want to use with the multi-tenancy integration.

If you update any field in a company record that has not yet been synchronized to UCMDB, Service Manager prompts whether you want to synchronize the company to UCMDB.

Note

Service Manager will not prompt you to synchronize the company record if you have disabled the option to show the company in multi-company lists, or if there is a pending schedule record associated with the company. See How to inactivate a synchronized company for more information.

This task includes the following steps:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Select a company record to update.
6. Update the company record.
7. Click **Save**. Service Manager prompts to confirm that you want to synchronize the record with UCMDB.

   Note

   Service Manager saves the company record regardless of your synchronization choice.

1.6.2.3.4.7. How to view whether company information is in UCMDB

When you enable the multi-tenancy integration, Service Manager displays a read-only field in each company record that lists whether SM has synchronized the UCMDB Customer ID with your UCMDB system.



Note

The UCMDB Customer ID field is visible only when you enable the multi-tenant UCMDB integration.

To view whether company information is in UCMDB:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Review the status of the **Synched with UCMDB** field.

   If the checkbox has a check mark, Service Manager has already synchronized the company ID with your UCMDB system. If the checkbox is clear, Service Manager has yet to add this company to your UCMDB system.

   Note

   For more information about synchronization failures, see Schedule records.

1.6.2.3.4.8. How to resynchronize an existing company with UCMDB

Service Manager provides you a means to resynchronize company records with your UCMDB system in case you lose UCMDB data for some reason. For example, you might intentionally remove UCMDB data during integration testing, or you might need to recover data after a disaster. You can force Service Manager to synchronize companies with your UCMDB system with the **Re-synch with UCMDB** option.

To resynchronize an existing company with UCMDB:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Select a company record to synchronize.
6. Click the **Re-synch** button next to the **Synched with UCMDB?** checkbox.



Note

The **Re-synch** button is available only from company records that have already been synchronized with UCMDB and have the **Synched with UCMDB** checkbox checked.

If your UCMDB system already has a company with this ID value, it will ignore the resynchronization request. Service Manager will also ignore a resynchronization request if there is an existing schedule record to resynchronize the company with UCMDB. In this case, it displays the message `A schedule record has already been added to re-synch this company with UCMDB` .

1.6.2.3.4.9. How to inactivate a synchronized company

After you have synchronized a company record with UCMDB you can no longer delete the record. Instead, you can inactivate a company record, which causes the UCMDB system to cease all further CI updates for the company. Any existing CI data for the company remains in the UCMDB system associated with the inactive UCMDB Customer ID, but both the company and any associated CIs will no longer be visible from the UCMDB system.

To inactivate a synchronized company:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Select a company record to inactivate.
6. Select **No** from Show Company in Multi-Company Lists.
7. Click **Save**.
8. If this company was previously synchronized with UCMDB, Service Manager prompts you to confirm the inactivation.
9. Click **Yes** to confirm the inactivation or **No** to cancel your changes.

1.6.2.3.4.10. How to reactivate an inactive company

You can reactivate any inactive companies on your Service Manager system to include them in the multi-tenancy integration. You must also synchronize the company with UCMDB for UCMDB to process any CI updates for this company.

To reactivate an inactive company:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Select a company record to reactivate.
6. Select **Yes** from **Show Company in Multi-Company Lists**.
7. Click **Save**. Service Manager prompts you to reactivate the company with UCMDB.
8. Click **Yes**. Service Manager creates a schedule record to reactivate the company.

1.6.2.3.4.11. How to add tenant-specific DEM rules

You can use the condition field to create DEM rules that are specific to a particular tenant in a multi-tenancy UCMDB-SM integration. For example, one tenant may want to add CIs directly to Service Manager while another tenant may want to open changes for each CI. The following sample DEM rules illustrate how to accomplish this.

Tenant-specific DEM rules

DEM rule Id	Action on new CI	Condition
ucmdbNode_advantage	Add CI	`company in $L.file=”advantage”`
ucmdbNode_hp	Create change	`company in $L.file=”HP”`

Tip

It is a best practice to create a separate DEM rule for each tenant.

1.6.2.3.4.12. How to add tenant-specific UCMDB user ID and password values

You can provide a tenant-specific UCMDB username and password for Service Manager to use when requesting information for the Actual State section. If you provide no credentials, Service Manager uses the credentials in the System Information Record for all tenants.

Note

Any credentials you provide in the company record take precedence over credentials you provide in the System Information Record. The UCMDB UserId and UCMDB Password fields are available only when you have enabled the multi-tenancy integration.

To add tenant-specific UCMDB user name and password:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Type the user name you want this company to use to connect to UCMDB in the UCMDB UserId field.
6. Type the password for the UCMDB user name in the UCMDB Password field.
7. Click **Save**.
8. Continue to follow *step 3* through *step 7* for each company for which you want to provide credentials.

1.6.2.3.4.13. How to add UCMDB customer ID values to existing companies

You can use the following steps to add a UCMDB Customer ID value to your existing Service Manager company records.

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Type a numeric value in the UCMDB Customer ID field for this company.
6. Click **Save**.
7. Service Manager prompts to confirm that you want to synchronize the record with UCMDB. Click **Yes** if you want to synchronize the company now, or click **No** if you want synchronize the company later.
8. Click **Next** to go to the next company in the record list.
9. Continue to follow *step 5* through *step 8* for each company in the record list.

1.6.2.3.4.14. How to Synchronize existing companies from Service Manager to UCMDBs

Your Service Manager system may already contain company records that you want to use with the multi-tenancy integration.

If you update any field in a company record and Service Manager hasn't yet synchronized it to UCMDB, the system prompts whether you want to synchronize the company to UCMDB.

> **Note**
>
> Service Manager will not prompt you to synchronize the company record if you have disabled the option to show the company in multi-company lists, or if there is a pending schedule record associated with the company. See How to inactivate a synchronized company for more information.

This task includes the following steps:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Select a company record to update.
6. Update the company record.
7. Click **Save**. Service Manager prompts you to confirm that you want to synchronize the record with UCMDB.

   > **Note**
   >
   > Service Manager saves the company record regardless of your synchronization choice.

1.6.2.3.4.15. How to view whether company information is in UCMDB

Suppose you have enabled the multi-tenancy integration. Service Manager displays a read-only field in each company record that indicates whether the system has synchronized the UCMDB Customer ID with your UCMDB system.

> **Note**
>
> The UCMDB Customer ID field is visible only when you enable the multi-tenant UCMDB integration.

To view whether company information is in UCMDB:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Review the status of the **Synched with UCMDB** field.

   If you select this checkbox, Service Manager has already synchronized the company ID with your UCMDB system. If you don't select the checkbox, Service Manager has yet to add this company to your UCMDB system.

   > **Note**
   >
   > For more information about synchronization failures, see Schedule records.

1.6.2.3.4.16. How to resynchronize an existing company with UCMDB

Service Manager provides you a means to resynchronize company records with your UCMDB system in case you lose UCMDB data for some reason. For example, you might intentionally remove UCMDB data during integration testing, or you might need to recover data after a disaster. You can force Service Manager to synchronize companies with your UCMDB system with the **Re-synch with UCMDB** option.

To resynchronize an existing company with UCMDB:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Select a company record to synchronize.
6. Click the **Re-synch** button next to the **Synched with UCMDB** checkbox.



Note

The **Re-synch** button is available only from company records that have already been synchronized with UCMDB and have the **Synched with UCMDB** checkbox checked. If your UCMDB system already has a company with this ID value, it will ignore the resynchronization request. Service Manager will also ignore a resynchronization request if there is an existing schedule record to resynchronize the company with UCMDB. In this case, it displays the message A schedule record has already been added to re-synch this company with UCMDB.

1.6.2.3.4.17. How to inactivate a synchronized company

After you have synchronized a company record with UCMDB you can no longer delete the record. Instead, you can inactivate a company record, which causes the UCMDB system to stop all further CI updates for the company. Any existing CI data for the company still remains in the UCMDB system associated with the inactive UCMDB Customer ID. However, both the company and any associated CIs will no longer be visible from the UCMDB system.

To inactivate a synchronized company:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Select a company record to inactivate.
6. Select **No** from **Show Company in Multi-Company Lists**.
7. Click **Save**.
8. If Service Manager has synchronized this company with UCMDB, the system prompts you to confirm the inactivation.
9. Click **Yes** to confirm the inactivation or **No** to cancel your changes.

1.6.2.3.4.18. How to reactivate an inactive company

You can reactivate any inactive companies on your Service Manager system to include them in the multi-tenancy integration. You must also synchronize the company with UCMDB for UCMDB to process any CI updates for this company.

To reactivate an inactive company:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **System Administration** > **Base System Configuration** > **Companies**.
3. Type the search criteria you want to use to find company records. For example, leave the search form blank to search all company records.
4. Click **Search**.
5. Select a company record to reactivate.
6. Select **Yes** from **Show Company in Multi-Company Lists**.
7. Click **Save**. Service Manager prompts you to reactivate the company with UCMDB.
8. Click **Yes**. Service Manager creates a schedule record to reactivate the company.

1.6.2.3.4.19. How to add tenant-specific DEM rules

You can use the condition field to create DEM rules that are specific to a particular tenant in a multi-tenancy UCMDB-SM integration. For example, one tenant may want to add CIs directly to Service Manager while another tenant may want to open changes for each CI. The following sample DEM rules illustrate how to accomplish this.

Tenant-specific DEM rules

DEM rule Id	Action on new CI	Condition
ucmdbNode_advantage	Add CI	`company in $L.file="advantage"`
ucmdbNode_hp	Create change	`company in $L.file="HP"`

Tip

It is a best practice to create a separate DEM rule for each tenant.

1.6.2.4. Standards and best practices

This section provides best practices for using the integration.

UCMDB-SM configuration best practices

Centralized CI type management

Bi-directional data synchronization recommendations

Push scheduling recommendations

Push in clustered environments

Initial load configurations

How to configure Differential or Delta Load DEM rules

Fault detection and recovery for push

How to enable Lightweight Single Sign-On (LW-SSO) configuration

Frequently asked questions

When is a new CI created in Service Manager?

Can I analyze the reason for a CI deletion in SM?

How do I monitor relationship changes Between UCMDB and SM?

What kinds of relationships are pushed from UCMDB to SM?

What is a root CI node?

What is a root relationship?

What is the "Virtual-Compound" relationship type used in a UCMDB-SM integration query?

When do I need the population feature?

Can I populate physically deleted CIs from SM to UCMDB?

How do I keep the outage dependency setting of a CI Relationship in SM?

How do I create an XML configuration file?

How do I use the Load Fields button to add multiple managed fields?

What is the purpose of the <container> element in the population configuration file (smPopConf.xml)?

Can I populate sub-item deletions?

What will happen if a population job fails or succeeds with warnings?

1.6.2.4.1. UCMDB-SM configuration best practices

This section provides best practices and recommendations for successfully implementing this integration in various environments. This section provides you with valuable understandings and techniques that will enhance the UCMDB-SM integration as well as solve common problems by providing solutions and workarounds to these issues. These practices and recommendations may vary slightly according to each implementation, as the specific system requirements and settings alter per system environment.

This section includes:

- Bi-directional data synchronization recommendations
- Push scheduling recommendations
- Push in clustered environments
- Initial load configurations
- How to configure Differential or Delta Load DEM rules
- Fault detection and recovery for push
- How to enable Lightweight Single Sign-On (LW-SSO) configuration

1.6.2.4.1.1. Centralized CI type management

The Service Manager Enhanced Generic Adapter allows centralized CI type management. Centralized CI type management enables you to manage CI types from UCMDB without the need to do much tailoring work on the Service Manager (SM) side. You can then synchronize CIs from UCMDB to SM through data push.

Managing Out-Of-the-Box CI Types

Service Manager and UCMDB provide out-of-the-box CI types, as well as mapping files for them. To modify an out-of-the-box CI type (for example, adding new attributes according to your business needs), you only need to open the Visual Mapping tool, manually add new attributes to this CI type, and then apply the update without leaving UCMDB. The new attribute will be automatically added to the relevant web service object on the SM side.

Managing New Custom CI Types

To add a new CI type according to your business needs, you only need to add a new CI type and create a TQL query in UCMDB, and then create a matching SM CI type by using the Visual Mapping Tool without the need to leave UCMDB.

When you create a new SM CI type in the UCMDB Visual Mapping Tool interface, all relevant SM objects (DBDICT, JoinDef, Web Service API, DEM Rule, and so on) for the new CI type are automatically created on the SM side, except the format and icon (the new CI type uses the **configurationItem** format by default).



Note

The population feature enables you to synchronize CIs from SM to UCMDB. For information about the circumstances under which you need to use the population feature, see When do I need the population feature?.

Visual Mapping tool

The UCMDB generic adapter framework comes with a Visual Mapping tool, which provides a graphic user interface for you to configure value and field mappings for complex data push or population configurations.

The following table describes the panes of this graphic user interface.

Pane	Description
Local Query	Displays a hierarchical tree structure of the local TQL query in the CMDB.
Local Query Attributes	This is the Attributes pane for the Local Query pane. It displays attributes of a local integration TQL query.
Visual Mapping	Allows you to establish mapping for items you select from the Local Query pane and the External Class Model pane through a drag and drop.
XML Editor	Allows to you edit XML mapping files in a text editor.
External Class Model	Displays a hierarchical tree structure of the external class model. For the UCMDB-SM integration, this pane displays supported CI types from the Service Manager (SM) side.
External Class Model Attributes	The Attributes pane for the External Class Model pane displays attributes of an external class model. For the UCMDB-SM integration, this pane displays attributes from the SM side.

1.6.2.4.1.2. Bi-directional data synchronization recommendations

UCMDB-SM integration supports bi-directional data synchronization between UCMDB and Service Manager (SM). We recommend that you follow the following best practices to avoid unnecessary problems due to improper use of the data push and population features:

- For CIs/CI Relationships that UCMDB can automatically discover, use UCMDB as the data source. Do not make changes to them in Service Manager; instead, always let UCMDB discover their changes and push the changes to SM.
- For CIs/CI Relationships that UCMDB can't automatically discover, use SM as the data source. Do not make changes to them in UCMDB; instead, always make changes to them in SM and populate the changes to UCMDB.
- For CIs/CI Relationships that are already created in SM and UCMDB can automatically discover, run a one-time population to synchronize them to UCMDB, and then use UCMDB as their data source.

Caution

Problems like the following may occur if you don't follow these best practices:

Problem 1:

[Population Adapter] After CIs/CI Relationships are pushed from UCMDB, if you directly make changes in SM to these records without ever populating them back to UCMDB first, the changes may not be populated to UCMDB.

Workaround:

Changing these UCMDB records in SM is not recommended; however if you need to do so you can do the following to solve this issue: After the records are pushed to SM, populate them back to UCMDB first before making any changes to them in SM. This way the changes can then be populated to UCMDB.

Problem 2:

[Population Adapter] After a Composition relationship between a Node CI (node 1) and Running Software CI is pushed to SM, if you change the upstream CI of the relationship from node 1 to node 2 in SM and then run a change population to populate this change, the Running Software CI will be removed in UCMDB.

Workaround:

It is recommended that you remove the running software in UCMDB and create a new one instead of directly replacing the container of the running software in SM. If you can't avoid doing so, do the following:

After you change the upstream CI of the relationship from node 1 to node 2, don't directly run the change population. Follow these steps to avoid this issue:

1. Update the Running Software CI in SM (or simply save it to mark it as updated).
2. Run a Running Software CI change population. This will create a new Composition relationship between node 2 and this Running Software CI.
3. Run a delta population to synchronize the relationship change to UCMDB. The relationship between node 1 and the Running Software CI will be removed, and the new relationship created in step 2 will remain.

If you have run a delta population after changing the upstream CI of the relationship from node 1 to node 2, and as a result the Running Software CI has been removed in UCMDB, follow these steps to solve this issue:

1. Update the Running Software CI in SM (or simply save it to mark it as updated).
2. Run a Running Software CI change population. This will create the Running Software CI, and a new Composition relationship between node 2 and this Running Software CI.

1.6.2.4.1.3. Push scheduling recommendations

Push jobs are run using two main methods, the first of which is manually executing the push job and the second is scheduling the push job.

All push jobs can potentially produce a strain on the UCMDB and SM systems; We recommend that you adhere to the following guidelines.

Scheduler time frames

It is important for you to understand the function of the Scheduler “time frame” concept. Running push jobs creates an increase in system activity and may affect application responsiveness. In order to enable users to effectively interact with applications We recommend the following guidelines:

In order to reduce system strain, schedule the UCMDB to SM push to run at non-peak usage hours, preferably when system usage is at a minimum.

Scheduler frequency

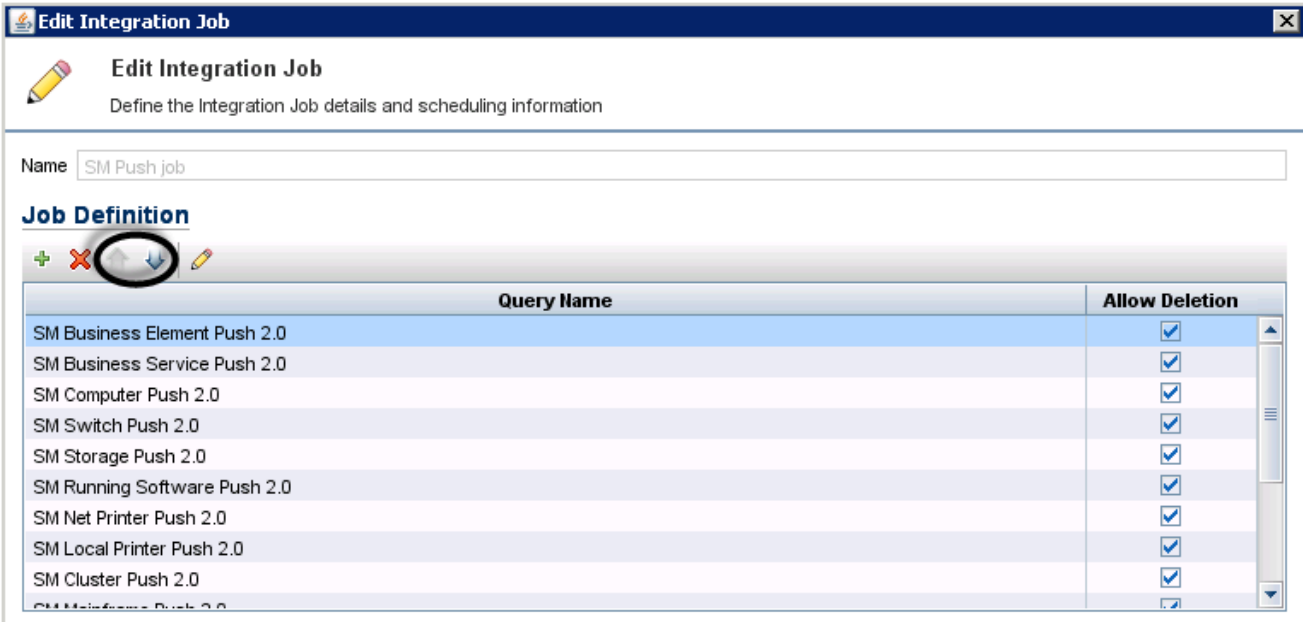
It is important to be aware of the business requirements when configuring the schedule frequency. The scheduler frequency depends on infrastructure environment changes that must be synchronized between UCMDB and SM.

Define the scheduling frequency based on the business requirements for consuming up-to-date CI information. Most implementations require a daily update. When scheduling small IT systems that are prone to frequent changes, the scheduling frequency may need to be increased.

Push Job dependencies

UCMDB Push Jobs don't support dependencies between each other. Each “Push Job” is considered a separate task and users can't define job dependencies. For example, that one job is dependent on another or upon completion before the next job is run.

It is important that both CI queries and their dependent Relationship queries exist in the same Job in order to avoid relationships not being pushed to Service Manager. You can change the position of each query in the list to define an appropriate execution sequence. See the following figure for an example.


Edit Integration Job
Edit Integration Job
Define the Integration Job details and scheduling information
Name
SM Push job
Job Definition
Query Name
Allow Deletion
SM Business Element Push 2.0
SM Business Service Push 2.0
SM Computer Push 2.0
SM Switch Push 2.0
SM Storage Push 2.0
SM Running Software Push 2.0
SM Net Printer Push 2.0
SM Local Printer Push 2.0
SM Cluster Push 2.0

1.6.2.4.1.4. Push in clustered environments

A clustered SM environment is comprised of multiple servlets running in parallel with a load balancer that dispatches user requests to any available servlet. You must configure the UCMDB-SM integration to point to a specific servlet and not to the SM loadBalancer. In order to perform this, you must first create a dedicated web service listener.

Dedicated Web Services

A Service Manager system configured for vertical or horizontal scaling uses a load balancer to redirect client connection requests to an available SM process. However, most Web Services clients can't handle a redirect request and will fail if they use the SM load balancer as the endpoint URL.

We recommend creating one or more SM processes dedicated to Web Services requests. The user must configure the relevant external web service clients to connect directly to the dedicated Service Manager processes.

Step-by-step cluster configuration process

This section describes the steps to configure a cluster environment for the integration.

How to configure web clients

To configure the relevant external web clients:

1. Stop the Service Manager service.
2. Open the `sm.cfg` file, and create a dedicated SM process to listen for Web Services requests using the `-debugnode` parameter.

   The following entries create a dedicated process listening on ports 13085 and 13445.

   ```
   01 sm -httpPort:13080 -loadbalancer
   02 sm -httpPort:13081 -httpsPort:13443
   03 sm -httpPort:13083 -httpsPort:13444
   04 sm -httpPort:13085 -httpsPort:13445 -debugnode
   ```

   Explanation

   The code excerpt illustrates the various settings for each of the SM process listeners (web services) that enable SM clients to connect to the SM service.

   Line 01 defines the load balancer port (13080).

   Lines 02 and 03 define the SM ports to which non-dedicated SM clients are redirected by the SM load balancer.

   Line 04 defines the debugnode port that's utilized by the dedicated SM clients.

Note

Debugnode parameter

The debugnode parameter tells the SM load balancer not to forward any client connection requests to this Service Manager process. Only clients that directly connect to the process can access this port.

How to configure the debugnode

To configure the debugnode:

1. Start the SM service.
2. Configure any external web service clients to connect directly to the SM processes running in debugnode. When performing an integration using UCMDB, the Service Manager Adapter for SM should be configured to connect to the debugnode port.

   For example, for normal connections set the endpoint URL to:

   `http://<fully qualified host name>:13085/SM/9/rest/<Service Name>`

   and for SSL-encrypted connections set the URL to:

   `https://<fully qualified host name>:13445/SM/9/rest/<Service Name>`

   These clients may include UCMDB (for push purposes), Connect-It, and additional applications.

Connecting to multiple SM processes

If you want to improve performance, you can connect to multiple Service Manager processes. The integration supports both Service Manager vertical and horizontal load balancer environments.

You can create more than one SM process that's dedicated to Web Services requests, and configure the URL Override field of the integration point with the dedicated SM processes. This field value (if any) overrides the Hostname/IP and Port settings.

The following is an example value of this field, which connects two SM processes:

`http://<fully qualified host name1>:13080/SM/9/rest;http://<fully qualified host name2>:13082/SM/9/rest`

1.6.2.4.1.5. Initial load configurations

Before the configuration process can begin, you must first assess the amount of CI and relationship data to be transferred from UCMDB to SM and ascertain the iteration process that's required based on the volume.

You must first assess whether or not all of the data can be pushed in a single iteration. This is ascertained by the amount of data that's included in the push queries and the amount of time you have to push this data.

Note

The performance data presented in this document is based on tests that were performed in the lab and are provided for reference only. The integration performance may significantly differ in your environment depending on your hardware configuration.

Push performance in a single-threaded environment

The Push of 22,500 UCMDB root CIs (roots in queries) and/or Relationships in a single-threaded environment takes about an hour and is performed in a linear fashion. See the following table.

Performance data in a single-threaded environment

Number of root CIs/CI Relationships Pushed per Hour	Multi-Threading Settings in sm.properties

22,500	`number.of.concurent.sending.threads=1`
	`min.objects.for.concurent.sending=50`
	`number.of.chunks.per.thread=3`
	`recommended.min.cis.per.chunk=50`

To view or edit the sm.properties file in UCMDB, navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41** > **Configuration Files** > **sm.properties**.

Number of Root CIs and Relationships/22,500

The push time (in hours) in any given environment is calculated as follows:

If the push of a single planned query has the potential of breaching the permitted time frame, the data must be divided into several queries. Each query must be pushed individually.

This query division is performed by creating several queries, each with different node conditions that enable data filtering. Once all queries are pushed for the first time, the Initial Load process is complete.



Note

When applying node conditions to the various Service Manager queries, you must make sure that all of the information is included in the queries, so that all relevant data is copied to Service Manager.

Implementing multi-threading

In order to improve performance, the Service Manager adapter utilizes multiple threads for the push of CI and Relationship data to SM. The following section explains these settings and how to configure them for maximum performance.

The multi-threading configuration is defined in the sm.properties file on the UCMDB server. To view or edit the file in UCMDB, navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41** > **Configuration Files** > **sm.properties**.

The following are example multi-threading definitions in the sm.properties file:

```
01 number.of.concurrent.sending.threads=6
02 min.objects.for.concurrent.sending=50
03 number.of.chunks.per.thread=3
04 recommended.min.cis.per.chunk=50
```

Explanation

The code excerpt illustrates the relevant multi threading settings on the UCMDB server.

- Line 01 defines the number of parallel threads UCMDB will open to SM for CI push. Setting this parameter to 1 disables multi-threading, while a values of 2 or higher enables multi-threading.
- Line 02 defines the minimum number of SM objects needed to use multiple threads as opposed to a single thread.
- Line 03 defines the number of chunks per thread. This number multiplied by the number of threads gives you the total number of CI data chunks.
- Line 04 defines the recommended minimum number of CIs per CI data chunk.

The total number of chunks = number.of.chunks.per.thread * number.of.concurrent.sending.threads

The integration implements a queue mechanism as follows:

The data passed from UCMDB to SM is divided into equal chunks, and these chunks are placed in a queue.

Each available thread pulls the next chunk from the queue until all threads are available. Once this process has completed, the push is complete.

The mechanism is designed to minimize idle time of each thread. As each thread processes its chunk in parallel, some threads may finish before others and it is inefficient for them to wait for each other.

Caution

Defining too many threads

It is ineffective to over-increase the number of threads as this causes the SM server to overload. In enterprise environments where the SM server processing the push data is very robust the number of threads can be increased to 10 and in some cases even 20; however, you must take into account that increasing the number of threads raises CPU usage on the SM server during push, which may reduce application performance.

Push performance in multi-threaded environments

The push of 60,000 UCMDB root CIs (roots in queries) and/or Relationships in an out-of-the-box multi-threaded environment takes about an hour and is performed in a linear fashion. See the following table.

Performance Data in an Out-Of-The-Box Multi-Threaded Environment

Number of Root CIs/Relationships Pushed per Hour	Multi-Threading Settings in sm.properties (Default)

60,000	`number.of.concurrent.sending.threads=6`
	`min.objects.for.concurrent.sending=50`
	`number.of.chunks.per.thread=3`
	`recommended.min.cis.per.chunk=50`

The push time (in hours) in any given environment is calculated as follows:

Number of Root CIs and Relationships/60,000

Push performance in multiple SM processes environments

The Push of 190,000 UCMDB root CIs (roots in queries) and/or Relationships in a multi-threaded environment with multiple SM processes takes about an hour and is performed in a linear fashion. See the following table:

Performance data in a multiple SM processes environment

Number of root CIs/Relationships pushed per hour	SM processes	Multi-threading settings in sm.properties (default)

160,000	2 server hosts, with each host running 4 processes	`number.of.concurrent.sending.threads=60`
		`min.objects.for.concurrent.sending=50`
		`number.of.chunks.per.thread=3`
		`recommended.min.cis.per.chunk=50`
		
		Data Push Chunk Size = 4000 (in Integration Settings in UCMDB)

For more information about defining multiple SM processes for the integration, see How to create an integration point in UCMDB.

The push time (in hours) in any given environment is calculated as follows:

Number of Root CIs and Relationships/160,000

How to set up SM DEM rules for initial loads

SM Discovered Event Manager Rules (DEM Rules) enable the user to define the appropriate action to take for each event type that's reported to SM.

Each CI and relationship record pushed from UCMDB to SM is analyzed against the existing SM records and open Change requests. SM rules define the appropriate action to be taken for each type of CI data update that's sent to SM.

To view or update the SM Discovered Event Manager Rules:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Press **ENTER** or click the **Search** button.

   A list of all the Discovered Event Manager Rules is displayed. Each rule is usually linked to a CI Type or a subset of CIs of the same type.
4. Click the individual CI Discovered Event Manager Rule to view its details.

To set up DEM Rules for initial loads:



Tip

When performing an Initial Loadwe recommend setting the SM Discovered Event Manager Rules to add newly reported CIs as described below. This minimizes the “noise” of an Initial Load, which could potentially create tens of thousands of Changes or Incidents.

For each of the Discovered Event Manager Rules, perform the following steps:

1. Select the relevant Discovered Event Manager Rule.
2. Go to the **Action if matching record doesn't exist** section, select the **Add the record** option.
3. In the **Action if record doesn't exist but unexpected data discovered** section, select the **Log Results and Update Record** option.
4. In the **Action if record is to be deleted** section, select the **Delete Record** option.
5. Save the Discovered Event Manager Rule record.

1.6.2.4.1.6. How to configure Differential or Delta Load DEM rules

Tip: Once the Initial Load or Data Load of the CI data has completed, it's advisable to apply Differential or Delta Load settings. These settings apply to all data loaded from UCMDB to SM. These loads send only updates regarding modifications discovered in the IT infrastructure from UCMDB to SM.

To set up the SM Discovered Event Manager (DEM) Rules for Differential or Delta Loads:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Press Enter or click the **Search** button.

   Service Manager displays a list of all the Discovered Event Manager Rules.
4. For each of the Discovered Event Manager Rules, perform the following steps:
    1. Select the relevant Discovered Event Manager Rule.
    2. In the **Action if matching record doesn't exist** section, select the appropriate action required for each newly detected CI. If uncertain, select the **Add the record** option.
    3. In the **Action if record does exist but unexpected data discovered** section, select the appropriate action for each CI that was already modified, resulting in an unexpected or wrong result. The recommended best practice is to select the **Open a Change** option.
    4. In the **Action if record is to be deleted** section, select the appropriate action required for each CI that was already removed. The recommended best practice is to select the **Delete Record** option for CI Relationships, and select the **Update record to the selected status** option for CIs.
    5. Save the Discovered Event Manager Rule record.

1.6.2.4.1.7. Fault detection and recovery for push

Universal CMDB has provided a fault detection and recovery mechanism since version 9.05: individual CI failures no longer cause the entire push to fail, and you can review all failed CIs in the Universal CMDB studio and then re-push them.

1.6.2.4.1.8. How to enable Lightweight Single Sign-On (LW-SSO) configuration

You can enable LW-SSO for the integration so that users can directly view UCMDB CI records from the Service Manager web client by clicking the **View in UCMDB UI** button, without providing a UCMDB UI username and password.



Note

LW-SSO is unsupported for the Service Manager Windows client.

To enable LW-SSO for the integration:

1. For each Service Manager user account that needs LW-SSO, create a user account in UCMDB with the same username. The passwords in the two systems can be different.
2. Enable LW-SSO in the Service Manager Web tier. For details, see Configure LW-SSO on the Service Manager Web tier.
3. Enable LW-SSO in UCMDB. For details, see the *Enable login to UCMDB with LW-SSO* topic in the Universal Discovery and CMDB Help Center.

1.6.2.4.2. Frequently asked questions

This section provides answers to frequently asked questions about the UCMDB-SM integration.

This section includes:

- When is a new CI created in Service Manager?
- Can I analyze the reason for a CI deletion in SM?
- How do I monitor relationship changes between UCMDB and SM?
- What kinds of relationships are pushed from UCMDB to SM?
- What is a root CI node?
- What is a root relationship?
- What's the “Virtual-Compound” relationship type used in a UCMDB-SM integration query?
- When do I need the population feature?
- Can I populate physically deleted CIs from SM to UCMDB?
- How do I keep the outage dependency setting of a CI relationship in SM?
- How do I create an XML configuration file?
- How do I use the Load Fields button to add multiple managed fields?
- What is the purpose of the <container> element in the population configuration file (smPopConf.xml)?
- Can I populate sub-item deletions?
- What happens if a population job failed or completed?

1.6.2.4.2.1. When is a new CI created in Service Manager?

CIs are created in SM under the following circumstances:

- A CI is manually added to SM through the Configuration Management module or the Request module.
- UCMDB reports a newly discovered CI according to the following:
  - When a new CI is reported and the Discovered Event Manager Rules are set to **Add the Record**.
  - When a new CI is reported, the Discovered Event Manager Rules are set to **Open an Incident** and the Incident has been closed.
  - When a new CI is reported, the Discovered Event Manager Rules are set to **Open a Change** and the Change has been verified.

1.6.2.4.2.2. Can I analyze the reason for a CI deletion in SM?

No.

SM opens a change request on the deleted CI and includes the following information:

“Delete event for CI “CI Name” triggered by discovery”.

Workaround

An SM change request doesn't contain a description of the reason for deletion, however it is possible to extract specific information about CI deletions from the UCMDB History Database. UCMDB data provides information about the user or the discovery pattern that initiated the CI deletion.

1.6.2.4.2.3. How do I monitor relationship changes between UCMDB and SM?

To understand the relationship change in SM, a distinction must be made between the various types of Relationship Changes:

- The second endpoint of the relationship has Changed, so instead of CI X being linked to CI Y through a relationship, now CI X is related to CI Z.
- An attribute of the relationship has changed.

The first type of Relationship change is supported by the UCMDB-SM integration, therefore, such “Relationship Changes” can either invoke CI relationship updates, or perform the creation of Incidents or Changes, which are then reviewed and monitored.

The second is also supported, but it's not covered out-of-the-box; you can configure the Universal CMDB query to expose such attributes of the relationship, and configure the Service Manager WSDL to expose the mapped field, and then configure the adapter mapping configuration in the XML and Groovy. However such a Relationship Attribute Change can't perform the creation of Incidents or Changes, and only supports invoking CI relationship updates directly.

1.6.2.4.2.4. What kinds of relationships are pushed from UCMDB to SM?

Any kinds of relationships are pushed from UCMDB to SM under the following conditions:

- The relationship appears in a Push Query located in the **Service Manager** > **Push** folder in the UCMDB Query Manager.
- The relationship is named **Root** in the Push Query.
- The relationship is mapped to an appropriate target in SM in the UCMDB configuration files (XML and Groovy files).

The out-of-the-box relationships that are pushed from UCMDB to SM are relationships between two CIs such as:

- Between Business Services and Applications;
- Between Business Service and Host;
- Between an Application and a Network Component; or
- Between Host, Network Components and Printers.

1.6.2.4.2.5. What is a Root CI node?

A Root node is a TQL query node that represents the CI type that's created through push to SM from the TQL query structure. The rest of the TQL query structure contains information that can be incorporated within the Root CI type and is used to enrich the record in SM with additional information and or attributes.

1.6.2.4.2.6. What is a Root relationship?

A Root relationship is a relationship within a query and created in SM through push. It represents a relationship between two Root CIs. Only the relationships marked with Root are pushed to SM.

1.6.2.4.2.7. What's the “Virtual-Compound” relationship type used in a UCMDB-SM integration query?

When more than two UCMDB CI entities are connected in series, the “Virtual-Compound” represents the relationship between the first and last entities. This is a virtual relationship, as no physical representation exists.

The “Virtual-Compound” relationship type is a relationship that links two CI type entities that have a logical relationship. See the following figure.



Explanation

The illustration shows an example of a Virtual-Compound relationship. The relationship in SM is created directly between the Host and the Business Service.

1.6.2.4.2.8. When do I need the population feature?

You need the population feature under any of the following circumstances:

- You have done modeling in SM, especially when you are in the planning and design phases, and you want your models to be reflected in UCMDB;
- You want to implement the UCMDB-SM integration, however you have already invested in your SM CMDB and don't want to lose that investment;
- You want to continue to maintain some parts of the SM CMDB while maturing your UCMDB/Discovery implementation.

1.6.2.4.2.9. Can I populate physically deleted CIs from SM to UCMDB?

No.

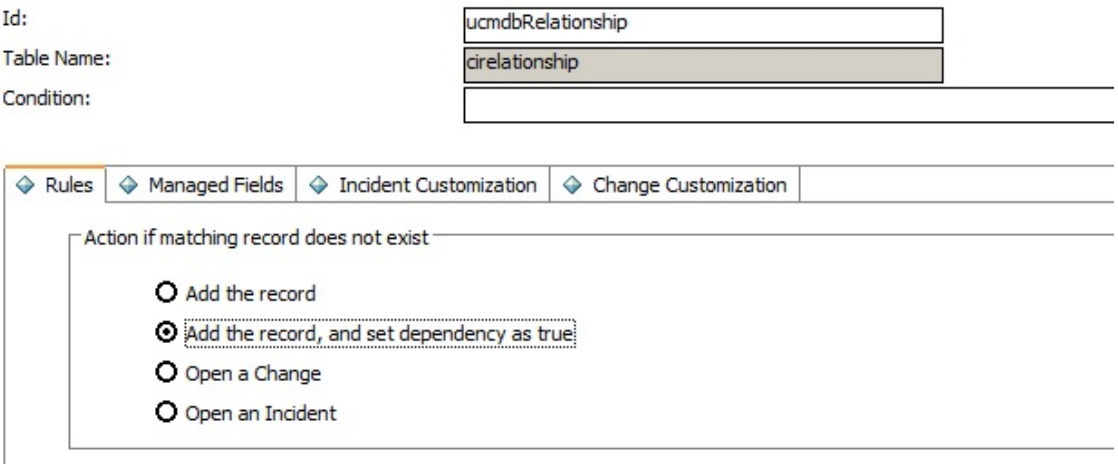
Physical deletions of CIs are allowed in SM, but SM can't get such “deletion changes” and the population feature will not synchronize such changes to UCMDB.

Physical deletions of CIs can be considered as exceptions, which occur only after you create CIs by mistake. Normally, you delete a CI by setting its status to something like **Retired/Consumed**. If such CIs have been populated to UCMDB, it is your responsibility to remove them manually from UCMDB.

1.6.2.4.2.10. How do I keep the outage dependency setting of a CI relationship in SM?

Out-of-the-box, CI relationships that are pushed from UCMDB to SM don't have outage dependency information by default. If you need such information, you can set the Discovered Event Manager (DEM) rule of the CI Relationship Web Services Description Language (WSDL) as follows.

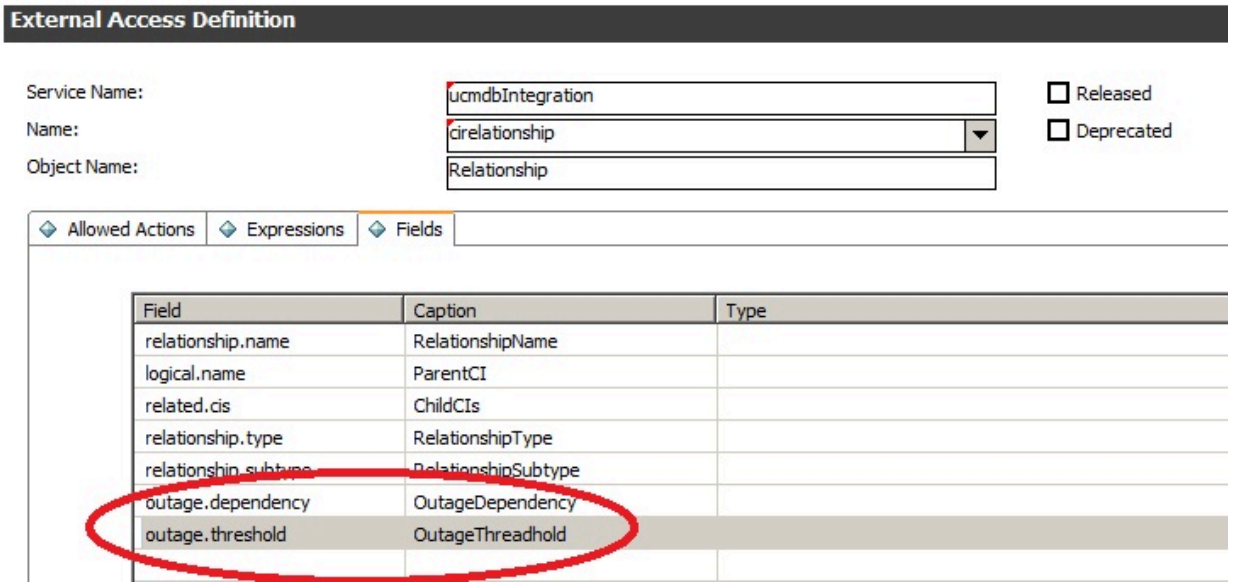
1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Open the **ucmdbRelationship** record.
4. On the Rules tab, select **Add the record, and set dependency as true**.



   This will set the Outage Dependency of each CI Relationship to true, and set the number of dependent downstream CIs to 1 (because UCMDB supports only one-to-one relationships).

Suppose you want to set outage dependency only for some relationships. For example, if you want to configure outage dependency for relationships that start from Business Service, you can configure the adapter configuration file (XML) and WSDL definition. Or, you can also configure outage dependency per relationship type (UCMDB query).

1. In the WSDL definition, expose fields **outage.dependency** and **outage.threshold**.



2. In the XML file, set the exposed outage fields. For example, if you want to set the outage dependency to true and threshold to 1 for Business Service relationships, you simply need to change the XML mapping file **SM Business Service Relations Push 2.0.xml**. In the XML mapping file, use the following `OutageDependency` and `OutageThreadhold` settings:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<integration>
<info>
<source name="UCMDB" vendor="HP" version="10.20"/>
<target name="SM" vendor="HP" version="9.40"/>
</info>
<import>
<scriptFile path="mappings.scripts.SMPushFunctions"/>
</import>
<target_entities>
<source_instance root-element-name="Root_directly" query-
name="EA_SM Business Service Relations Push">
<target_entity name="Relationship">
<target_mapping datatype="STRING" name="RelationshipType"
value="SMPushFunctions.getDisplayName(Root_directly['element_type']
,ClassModel)"/>
<target_mapping datatype="STRING" name="ParentCI"
value="SMPushFunctions.getEndId(OutputCI.getExternalId().getEnd1Id(
))"/>
<target_mapping datatype="STRING_LIST" name="ChildCIs" value="
[SMPushFunctions.getEndId(OutputCI.getExternalId().getEnd2Id())]"/>
<target_mapping datatype="BOOLEAN" name="OutageDependency"
value="true"/>
<target_mapping datatype="NUMBER" name="OutageThreadhold"
value="1"/>
</target_entity>
</source_instance>
</target_entities>
</integration>
```

1.6.2.4.2.11. How do I create an XML configuration file?

You create an XML configuration file in Adapter Management. You can copy the content of an existing XML configuration file to the new file and then make necessary edits.

To create an XML configuration file:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management**. **ServiceManagerEnhancedAdapter9.41** > **Configuration Files**.
3. Click the **Create new resource** icon  .
4. Select **New Configuration File**.
5. Enter a name for the file. The file name should use this format: `<AdapterID>/mappings/<Synch Type>/<filename>` . For example, `ServiceManagerEnhancedAdapter9.41/mappings/push/SM Computer Push 2.0.xml` .
6. Click **OK**. The system displays a message and asks if you want to open the file in the Visual Mapping tool editor.
7. Click **Yes** or **No** to continue.

   UCMDB creates the new XML configuration file in the Configuration Files folder of the adapter.
8. Copy the content of an existing XML configuration file to the new file.
9. Make necessary edits to the new file.



Caution

Invalid XML

When removing XML elements from an XML file, keep in mind that the remaining elements must constitute a valid XML file. The system will use this file to translate the UCMDB Query Definition.

1.6.2.4.2.12. How do I use the Load Fields button to add multiple managed fields?

Service Manager stores a list of managed fields in the **ucmdbIntegration** web service, which consists of a number of web services objects. You can add more managed fields to DEM Rules so that Service Manager can monitor changes in more CI attributes in UCMDB and trigger the actions defined in relevant DEM Rules.

You can manually add managed fields that are exposed in associated WSDL definitions to DEM Rules; however, you can use the **Load Fields** button to automatically (and therefore correctly) add managed fields to DEM Rules.

1. Click the **Managed Fields** tab of the DEM Rule.
2. Click the **Load Fields** button.
3. If the table (in the Table Name field) of the DEM rule record has only one WSDL definition associated to it, all fields exposed in the WSDL definition are immediately added to the Managed Fields list.

   A message is displayed: `<XX> new fields loaded.`
4. If the table has more than one WSDL definition associated to it, the Managed Fields Importing wizard opens, and a list of WSDL definitions (ucmdbIntegration web service objects) is displayed.
   1. Select one or more objects, and click **Next**. All new fields that can be added from the selected web service objects are displayed.
   2. If you want to add all of the fields, click **Finish**; if you want to ignore some of them, change their Action value from **Add** to **Ignore**, and then click **Finish**.

      A message is displayed: `<XX> new fields loaded.`
5. Save the DEM Rule record.

1.6.2.4.2.13. What is the purpose of the <container> element in the population configuration file (smPopConf.xml)?

Out-of-the-box, the smPopConf.xml file has a `container` element.

```
<tql name="SM RunningSoftware Population 2.0"
citype="running_software">
<request  type="Retrieve" dataType="ci"
resourceCollectionName="ucmdbRunningSoftwares"
resourceName="ucmdbRunningSoftware"
basicQueryCondition="type=&quot;runningsoftware&quot;"
fullQueryCondition="istatus~=&quot;Retired/Consumed&quot;"
changedCreationQueryCondition="created.by.date&gt;'{fromDate}' and
istatus~=&quot;Retired/Consumed&quot;"
changedUpdateQueryCondition="created.by.date&lt;='{fromDate}' and
devicemodtime&gt;'{fromDate}' and
istatus~=&quot;Retired/Consumed&quot;"
changedDeletionQueryCondition="devicemodtime&gt;'{fromDate}' and
istatus=&quot;Retired/Consumed&quot;"/>
<container tql="SM Computer Population 2.0"
keyFields="CIIdentifier"
linkTql="SM Computer Composition Software 2.0"
linkRetrieveCondition="downstreamci.logical.name=&quot;$$$&quot; and
upstreamci.type=&quot;computer&quot; and
downstreamci.type=&quot;runningsoftware&quot; and
relationship.subtype=&quot;Composition&quot;"
linkRetrieveConditionKey="CIIdentifier"
linkValueFields="upstreamci.logical.name"/>
</tql>
```

- In UCMDB, RunningSoftware CIs must exist together with a Root Container (Node); however, Service Manager allows RunningSoftware CIs without a Node.
- The integration adapter synchronizes CIs and Relationships separately; when populating a RunningSoftware CI, the integration has no chance to check if a relationship exists between the CI and a Node.

With the `<container>` element, the integration populates RunningSoftware CIs together with a container.

1.6.2.4.2.14. Can I populate sub-item deletions?

Yes.

Service Manager and UCMDB store CI information in different data structures, and therefore one SM CI may be synchronized to UCMDB as several CIs. For example, during population, an SM computer CI record is synchronized to a Node CI in UCMDB, and the computer CI's attributes to CIs such as IP, Interface, Location, etc (which are referred to as sub-items of the Node CI.). In this case, the Node CI is the root CI.

The integration allows you to populate sub-item deletions to UCMDB. For example, if you delete the IP Address attribute value of a computer, the corresponding IP CI record in UCMDB will be deleted too.

1.6.2.4.2.15. What will happen if a population job fails or succeeds with warnings?

When a population job failed

The failure prevents the remaining population tasks from running. The next job run will start from the last Success time. If pagination occurs (that is, the tasks are divided into multiple pages), the tasks will run again and again within the first page from the last “Success” time (once the end of the first page is reached, no new tasks will be executed).

When a population job completed

When the status of a job is "completed" (but not "completed successfully"), warnings occurred. A warning doesn't prevent the remaining population tasks from running. The next job run will run all tasks again starting from the last Success time. If pagination occurs (the tasks are divided into multiple pages), the tasks on all pages will be re-run (including those that successfully completed last time).

1.6.2.5. Tailoring the integration

You can tailor the UCMDB-SM integration to meet your business needs by adding or removing managed CI types, attributes, and relationship types. This section describes the integration architecture and tailoring options for data push, population, and federation.

Integration architecture

Integration class model

Integration queries

Service Manager Web Services

Service Manager reconciliation rules

Service Manager Discovery Event Manager rules

Integration tailoring options

How to enable ASM support

How to update the integration adapter configuration file (sm.properties)

How to add DEM reconciliation rules

How to add Discovery Event Manager rules

How to add a CI relationship type to the integration for data push

How to add a CI attribute to the integration for data push

How to add a CI type to the integration for data push

How to add a CI type, attribute, or relationship type to the integration for population

How to enable or disable UCMDB ID pushback for a CI type

How to add an attribute of a supported CI type for federation

How to enable the automatic deletion of CI relationships from UCMDB

1.6.2.5.1. Integration architecture

Before you tailor the integration, you should understand how the following components of the out-of-the-box integration work.

- Integration class model
- Integration queries
- Service Manager Web Services
- Service Manager reconciliation rules
- Service Manager Discovery Event Manager rules

1.6.2.5.1.1. Integration class model

UCMDB 9.x or later no longer uses a private class model of CI types to manage integration CIs, as was required in prior versions. Instead, the integration uses the standard UCMDB managed objects and maps them to Service Manager CI types and attributes with queries and transformation files.

1.6.2.5.1.2. Integration queries

This section describes out-of-the-box queries used for data push, Actual State, federation, and population.

Queries for push

For the push feature, the integration uses a collection of queries to gather CI attribute information from Universal CMDB and send it to the Service Manager system.

To access the out-of-the-box data push queries, navigate to **Modeling** > **Modeling Studio**, select **Queries** for Resource Type, and then navigate to the **Root** > **Integration** > **Service Manager** > **Push** folder.

If you want to change what CI types, Relationship types or attributes are part of the integration, you must also edit the integration queries to support your updated CI types, CI Relationship types and attributes.



Note

The Service Manager Enhanced Generic Adapter has been introduced since UCMDB version 10.20. This adapter can coexist with an old XSLT adapter. In order to distinguish its queries from those of an XSLT adapter, all queries of the enhanced adapter are suffixed with **2.0**.

Query name	Description
SM Local Printer Push 2.0	This query gathers CI attributes from printer CIs. It also gathers related CI attributes from the Node CI type.
SM Net Printer Push 2.0	This query gathers CI attributes from Node CI types whose NodeRole contains “printer.” It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Mainframe Push 2.0	This query gathers CI attributes from the following Node CI types: Mainframe Logical Partition, and Mainframe CPC. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Mobile Device Push 2.0	This query gathers CI attributes from Node CI types whose NodeRole contains “pda_handheld.” It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.

Query name	Description
SM Network Component Push 2.0	This query gathers CI attributes from Node CI types whose NodeRole contains router, adsl_modem, appletalk_gateway, bandwith_manager, cable_model, csu_dsu, ethernet, fddi, firewall, hub, kvm_switch, load_balancer, multicast_enabled_router, nat_router, token_ring, undefined_network_component, voice_gateway, voice_switch, or vpn_gateway. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Cluster Push 2.0	This query gathers CI attributes from the following Node CI type: ClusterResourceGroup. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, Location, and Cluster.
SM Computer Push 2.0	This query gathers CI attributes from the node CI type with NodeRole containing “desktop”, “server”, “virtualized_system” or not set. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.

Query name	Description
SM Storage Push 2.0	This query gathers CI attributes from the Node CI type whose NodeRole contains san_switch, san_gateway, san_router, or whose Display Name is Storage Array. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Running Software Push 2.0	This query gathers CI attributes from Running Software CIs.
SM Switch Push 2.0	This query gathers CI attributes from the Node CI type whose NodeRole contains atm_switch, frame_relay_switch, or lan_switch. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Service Element Push 2.0	This query gathers CI attributes from the Business Element CI type.
SM Business Service Push 2.0	This query gathers CI attributes from the Business Service CI type.
SM Layer2 Topology Relations Push 2.0	This query gathers relationships between the following components: Two or more nodes. The query includes compound relationships because the relationships can extend through a group.

Query name	Description
SM Business Service Relations Push 2.0	This query gathers relationships between the following components: • Business Service and Running Software CIs • Business Service and Node CIs • Two or more Business Services The query includes compound relationships because the relationships can extend through a group.
SM CRG Relations Push 2.0	This query gathers relationships between the following components: Node and Cluster Resource Group CIs. The query includes compound relationships because the relationships can extend through a group.
SM Node Relations Push 2.0	This query gathers relationships between the following components: • Node and Printer CIs • Node and RunningSoftware CIs The root class of the relationship is composition.

Query name	Description
SM ASM Relations Push 2.0	This query gathers ASM discovered consumer-provider relationships that point from a Business Element CI to a Running Software CI. **Note** This query doesn't support consumer-provider relationships between a Business Element CI and a Cluster CI. **Note** To work correctly, this query requires the ASM Enhanced package to be deployed in UCMDB. This package is already included in UCMDB Content Pack (CP) 17.

Queries for Actual state

Out-of-the-box, the queries in the following table are used for retrieving CI information from UCMDB to the Actual State section of the Service Manager Configuration Item (CI) form. Service Manager retrieves CI Actual State information by calling a UCMDB web service that retrieves CI data according to these queries.

The queries are located in the **Integration** > **SM Query** folder in the UCMDB Modeling Studio.

Query name	Description
localPrinterExtendedData	This query gathers real-time extended information from Printer CIs in UCMDB.
applicationExtendedData	This query gathers real-time extended information from RunningSoftware CIs in UCMDB.
businessServiceExtendedData	This query gathers real-time extended information from business service CIs in UCMDB.
hostExtendedData	This query gathers real-time extended information (such as Asset, Party, Location, LogicalVolume, WindowsService, Printer, InstalledSoftware, FileSystem, IPAddress, Interface, DiskDevice, and Cpu) from the node CI type in UCMDB.

Queries for federation

Each generic adapter stores the out-of-box federation queries in the **Integration** > **Service Manager** > **Federation** folder in the Modeling Studio.

For UCMDB 10.20, the generic adapter has defined a large number of out-of-box federation queries. These queries are for the out-of-the-box CI Types in UCMDB. Due to a technical limitation in UCMDB 10.20, you still need to define static federation queries for each of your custom CI types. For information on how to define static federation queries, see the *Achieving Data Federation Using the Generic Adapter* section in the *Universal CMDB Developer Reference Guide*.

Starting with UCMDB 10.21, you don't need to create additional static queries. The following out-of-box static federation queries are used for all CI types:

- SM Incident 2.0
- SM Incident To CI 2.0
- SM Problem 2.0
- SM Problem To CI 2.0
- SM RFC 2.0
- SM RFC To CI 2.0

Queries for population

For CI/CI Relationship population, the integration uses the following queries to save CI/CI Relationship attribute information to UCMDB.

Query Name	Description
SM Business Service Population 2.0	Defines the CI store structure of business service CIs.
SM Business Application Population 2.0	Defines the CI store structure of Application Service CIs.
SM Infrastructure Service Population 2.0	Defines the CI store structure of Infrastructure Service CIs.
SM Running Software Population 2.0	Defines the CI store structure of running software CIs.
SM Computer Population 2.0	Defines the CI store structure of computer CIs.
SM CLIP Down Time Population 2.0	Defines the CI store structure of ScheduledDowntime CIs.
SM Biz Containment Biz 2.0	This query defines the CI store structure of CI relationships in which a bizservice CI contains another.
SM Biz Usage Biz 2.0	Defines the CI store structure of CI relationships in which a bizservice CI uses another.
SM Biz Containment Computer 2.0	This query defines the CI store structure of CI relationships in which a bizservice CI contains a computer CI.
SM Biz Containment Software 2.0	Defines the CI store structure of CI relationships in which a bizservice CI contains a RunningSoftware CI.
SM Biz Usage Computer 2.0	Defines the CI store structure of CI relationships in which a bizservice CI uses a computer CI.
SM Biz Usage Software 2.0	Defines the CI store structure of CI relationships in which a bizservice CI uses a RunningSoftware CI.

Query Name	Description
SM Computer Connects Computer 2.0	Defines the CI store structure of CI relationships in which a computer CI connects to another.
SM Computer Composition Software 2.0	Defines the CI store structure of CI relationships in which a RunningSoftware CI is contained within a computer CI and the RunningSoftware CI can't exist without the container.
SM CI Connection Down Time CI 2.0	Defines the CI store structure of CI relationships in which a ScheduledDowntime CI connects to an affected CI.

Query requirements

The integration requires that any custom queries you create meet certain formatting conditions. Any queries that you want to include in the integration must meet these conditions:

- To query CIs, a query must contain one CI type labeled Root. The Root node is the main CI that UCMDB synchronizes. All other CIs are contained CIs of the Root CI.
- To query relationships, a query must contain one or more relationships labeled Root.
- A query must contain only the Root CI and CIs that are directly connected to it. The Root CI is always the top node of the query hierarchy.
- A query layout can't have cycles.
- If a query synchronizing relationships has cardinality, it must be the cardinality `1...*`. Additional cardinality entries must have a `OR` condition between them.
- If you want the integration to only synchronize specific CIs, you must configure the condition on the query to filter such CIs.

1.6.2.5.1.3. Service Manager Web Services

Service Manager uses Web Services messages to get and receive CI information from your UCMDB system. Out-of-the-box, UCMDB sends more CI attribute information than the Service Manager system actually manages. Service Manager users can view all of the CI attribute information the UCMDB system sends from the Actual State section of the CI record.

Service Manager publishes several web services for use by the UCMDB-SM integration. The UCMDB system uses the web services to map UCMDB CI types and CI attributes to Web Services objects that the Service Manager system recognizes. You can add UCMDB CI types or CI attributes that you want Service Manager to manage by using the Visual Mapping tool, and the tool will automatically update one or more of these Web Services to define web service objects.

Managed fields



Note

Managed fields are used only for the data push feature.

A Service Manager managed field is a field where the system compares the CI attribute value in the incoming UCMDB web services message to the value in a Service Manager CI record. If the values in the web services message don't match those in the CI record, Service Manager runs a Discovery Event Manager (DEM) rule to determine what action to take. The DEM rule determines which of the fields that are published as web services objects are fields managed by the integration. Only value changes in managed fields trigger the DEM rule.

The **ucmdbIntegration** web service consists of a set of web services objects, each of which defines a list of web service fields. Out-of-the-box, the integration uses only part of them (see the *Mappings between Service Manager web service objects, tables, and DEM rules* table), some of them (along with their relevant DEM Rules) are deprecated and removed (see the **Deprecated ucmdbIntegration web service objects for data push** table), and some are used for population or federation (see the **ucmdbIntegration web service objects used for population or federation** table).

Mappings between Service Manager web service objects, tables, and DEM rules

This web service object	Publishes fields from this table	And uses this DEM rule ID
Relationship	cirelationship	ucmdbRelationship
ucmdbRunningSoftware	device	ucmdbRunningSoftware
ucmdbBusinessService	joinbizservice	ucmdbBusinessService
ucmdbNode	joinnode	ucmdbNode

Deprecated ucmdbIntegration web service objects for data push

This web service object	Publishes fields from this table	Replaced with (object)
ucmdbApplication	device	ucmdbRunningSoftware
ucmdbComputer	ucmdbComputer	ucmdbNode
ucmdbNetwork	joinnetworkcomponents	ucmdbNode
ucmdbPrinter	joinofficeelectronics	ucmdbNode

ucmdbIntegration web service objects used for population or federation

This web service object	Publishes fields from this table	And is used for	Requires a DEM Rule?
cirelationship1to1	cirelationship1to1	Population	No
ucmdbIDPushBack	device	Population	No
UcmdbChange	cm3r	Federation	No
UcmdbChangeTask	cm3t	Federation	No
UcmdbIncident	probsummary	Federation	No
UcmdbProblem	rootcause	Federation	No

The following sections list the fields published as web services objects used for data push (see the *Mappings between Service Manager web service objects, tables, and DEM rules* table) and indicate whether or not they are managed fields in an out-of-the-box Service Manager system. You can use this reference to determine if you need to publish a field as a web service object, and also if you need to create a DEM rule for the object.

Object Name: Relationship

Service Manager publishes the following fields from the cirelationship table:

Web service and managed fields of the Relationship object

Field published as web service object	Caption used in web service messages	Is the field a managed field?
relationship.name	RelationshipName	
logical.name	ParentCI	
related.cis	ChildCIs	Yes
relationship.subtype	RelationshipType	

Object Name: ucmdbRunningSoftware

Service Manager publishes the following fields from the device table:

Web service and managed fields of the ucmdbRunningSoftware object

Field published as web service object	Caption used in web service messages	Is the field a managed field?
ucmdb.id	UCMDBId	
ci.name	ApplicationName	
type	Type	
subtype	Subtype	
company	CompanyId	
logical.name	CIIdentifier	
product.version	ProductVersion	
vendor	Vendor	
version	Version	
ci.name	ServiceName	Yes
sm.device.display.name	DisplayName	Yes

Object Name: ucmdbBusinessService

Service Manager publishes the following fields from the joinbizservice table:

Web service and managed fields of the ucmdbBusinessService object

Field published as web service object	Caption used in web service messages	Is the field a managed field?
ucmdb.id	UCMDBId	
ci.name	ServiceName	Yes
type	Type	
subtype	Subtype	
company	CustomerId	
logical.name	CIIdentifier	
vendor	ServiceProvider	Yes
sm.device.display.name	DisplayName	Yes

Object Name: ucmdbNode

Service Manager publishes the following fields from the joinnode table.

Web service and managed fields of the ucmdbNode object

Field published as web service object	Caption used in web service messages	Is the field a managed field?
ucmdb.id	UCMDBId	
type	Type	
subtype	Subtype	
company	CustomerId	
logical.name	CIIdentifier	
default.gateway	DefaultGateway	Yes
network.name	DNSName	Yes
building	Building	Yes
room	Room	Yes
floor	Floor	Yes
location	Location	
addlIPAddr[addlIPAddress]	AddlIPAddress	Yes
addlIPAddr[addlSubnet]	AddlSubnet	Yes
addlMacAddress	AddlMacAddress	Yes
bios.id	BIOSId	Yes
operating.system	OS	Yes
os.version	OSVersion	Yes
physical.mem.total	PhysicalMemory	Yes
serial.no.	SerialNo	
vendor	Vendor	
cpu[cpu.id]	CpuID	
cpu[cpu.name]	CpuName	

Field published as web service object	Caption used in web service messages	Is the field a managed field?
cpu[cpu.clock.speed]	CpuClockSpeed	
file.system[mount.point]	MountPoint	
file.system[disk.type]	DiskType	
file.system[file.system.type]	FilesystemType	
file.system[disk.size]	DiskSize	
asset.tag	AssetTag	
machine.name	HostName	Yes
disk.device[model.name]	ModelName	
disk.device[disk.vendor]	DiskVendor	
disk.device[disk.name]	DiskName	
sm.device.display.name	DisplayName	Yes
isVisualization	IsVisualization	
istatus	AssetStatus	

Note

The caption for the istatus field must be AssetStatus. Do not update this caption. Otherwise, the data push and data population may fail.

1.6.2.5.1.4. Service Manager reconciliation rules

Service Manager reconciliation rules allow the integration to identify CI records in your Service Manager system that match CIs in your UCMDB system. Service Manager attempts to reconcile CI records with every push of CI attributes from your UCMDB system. The integration uses the following workflow to match UCMDB CIs with Service Manager CIs.

1. The UCMDB system sends a web service message containing the latest CI attribute data to Service Manager.
2. Service Manager scans the web service message for the CI ucmdb.id value.

   Note

   The ucmdb.id field is displayed in the Actual State Section of the CI form, with a label of **Ucmdb ID**.

3. Service Manager searches for an existing CI record that has the same ucmdb.id value.
4. If Service Manager finds a CI that has the ucmdb.id value, no reconciliation is needed. Service Manager compares the UCMDB CI attributes to the Service Manager managed fields and runs the appropriate Discovery Event Manager (DEM) rules as needed to update the CI record in Service Manager.
5. If Service Manager can't find a CI that has the ucmdb.id value, it searches for a DEM reconciliation rule record that's defined for the CI type.
6. If no DEM reconciliation rule record is found, Service Manager creates the CI record according to the appropriate DEM rules.
7. If a DEM reconciliation rule record is found, Service Manager evaluates the rules and appends (ucmdb.id=NULL or ucmdb.id~=NULL and istatus="XXXXXX") to the reconciliation rules in the backend, where the istatus value is retrieved from the DEM rule for deletion (for example, **Retired/Consumed**). If no istatus value is found in the DEM rule, Service Manager only appends ucmdb.id=NULL to the reconciliation rules in the backend.
8. If no matching CI record is found by the reconciliation rules, Service Manager creates the CI record according to the appropriate DEM rules.
9. If a matching CI record is found by the reconciliation rules, Service Manager updates the CI record according to the appropriate DEM rules

Performance implications

Because Service Manager attempts to reconcile CIs with every push, the number of reconciliation fields you have will affect the integration's performance. The more reconciliation rules you have, the more searches Service Manager must perform to match CIs. To improve the performance of reconciliation searches, you should choose reconciliation fields that are unique keys of the underlying Service Manager table. For example, if you want to reconcile CI

records in the **device** table, use the logical.name field as a reconciliation field because it is a unique key. For information about how to create a reconciliation rule, see How to add DEM reconciliation rules.

Dependence on DEM rules

Service Manager uses the **Action if matching record doesn't exist** DEM rule whenever it can't reconcile CIs. You must review the DEM settings and decide if they meet your business standards prior to the initial push of CIs from UCMDB to Service Manager. For example, you can have Service Manager create a change request for every CI in the initial CI push by selecting the **Open a Change** option.

1.6.2.5.1.5. Service Manager Discovery Event Manager rules

You only need to create Discovery Event Manager (DEM) rules if you want to accomplish any of the following custom actions.

Change the conditions under which a DEM rule runs

Service Manager will run a DEM rule only if the condition field evaluates to true. Out-of-the-box, no DEM rule has a condition statement that restricts when the rule runs, and all the integration DEM rules will always run by default.

You can update a DEM rule's condition statements if you want to restrict when Service Manager runs your DEM rules. For example, adding the following condition to the ucmdbNode DEM rule restricts the rule to desktop CIs.

```
subtype in $L.file="Desktop"
```

You can also use the condition field to create multiple DEM rules that apply to the same table name. For example, the following DEM rules both apply to the joinnode table.

DEM rules using different conditions to affect the same table

DEM rule Id	Table Name	Condition
ucmdbNode	joinnode	`subtype in $L.file~="Desktop"`
ucmdbDesktop	joinnode	`subtype in $L.file="Desktop"`

Typically, you will only need to add conditions if your business processes require the integration to take different actions with certain CI types or SLAs.

Change the action the DEM rule takes

Out-of-the-box, the integration DEM rules take the following actions:

- Add a CI record when the UCMDB data doesn't match an existing Service Manager CI record
- Open a Change or log results and update a CI record when the UCMDB CI attribute data doesn't match the CI attribute data in the Service Manager CI record
- Delete a CI record when the UCMDB data specifies that the CI has been deleted

You can change the integration DEM rules to meet your business processes. For example, you could use the ucmdbNode DEM rule to open a change when the integration finds a non-desktop CI with unexpected data, and use the ucmdbDesktop DEM rule to log results and update the record when the integration finds a desktop CI with unexpected data.

Caution

If you want to use the Change Management verification and Change Management validation features of the integration, your DEM rules must use the **Open a Change** option for the "Action if record exists but unexpected data discovered" event.

Create custom JavaScript to open Change or Incident records

Service Manager uses the **discoveryEvent** JavaScript to create CI names and to set the values of required fields when opening change or incident records. Out-of-the-box, the script uses the following default values.

Default values to create a new CI

You can update the createCIName and populateNewCI functions to set the following CI values.

Default values used to create a new CI

CI attribute	Default value defined in discoveryEvent
record.logical_name	System generated ID number
record.assignment	AUTO
record.istatus	Installed
record.os_name	Value in record.operating_system

Default values to create a new change

You can update the populateChange function to set the following change values.

Default values used to create a new change

CI attribute	Default value defined in discoveryEvent
change.category	Unplanned Change
change.reason	Value in reason
change.initial_impact	3
change.severity	3
change.coordinator	Change.Coordinator
change.requested_by	discovery
change.status	initial

Default values to create a new incident

You can update the populateIncident function to set the following incident values.

Default values used to create a new incident

CI attribute	Default value defined in discoveryEvent
incident.category	incident
incident.subcategory	hardware
incident.product_type	missing or stolen
incident.assignment	Hardware
incident.initial_impact	3
incident.severity	3
incident.logical_name	Value of id
incident.site_cateogry	C
incident.contact_name	ANALYST, INCIDENT
incident.affected_item	MyDevices

1.6.2.5.2. Integration tailoring options

The integration offers the following tailoring options:

How to enable ASM support

How to update the integration adapter configuration file (sm.properties)

How to add DEM reconciliation rules

How to add Discovery Event Manager rules

How to add a CI relationship type to the integration for data push

How to add a CI attribute to the integration for data push

How to add a CI type to the integration for data push

How to add a CI type, attribute, or relationship type to the integration for population

How to enable or disable UCMDB ID pushback for a CI Type

How to add an attribute of a supported CI type for federation

How to enable the automatic deletion of CI relationships from UCMDB

1.6.2.5.2.1. How to enable ASM support

As of version 9.41, Service Manager supports the Automated Service Modeling (ASM) functionality in the UCMDB UI.

ASM can automatically discover Service CIs and the consumer-provider relationships between the components of services. The Service Discovery process builds dependency graphs from the consumer-provider components that make up a service.

Note

ASM is supported only for UCMDB 10.20 or later. To use ASM, the ASM Enhanced package must be deployed in UCMDB.

You can tailor Service Manager to integrate with Automated Service Modeling (ASM) of the UCMDB Browser. See the following instructions.

How to integrate with ASM

ASM enables Service Manager users to view the accurate service trees that are discovered by ASM and facilitates service modeling in Service Manager.

For example, to integrate the Service Manager Change Management module with ASM, you can add a control that calls the **us.launch.external** RAD application by passing the following URL:

`http://[UCMDB browser server name]:[port]/ucmdb_browser.jsp?username=<username>&password=<password>&server=Default%20Client&locale=en#view=[Service Name];tab=service-modeling`

Where:

- [UCMDB browser server name], [port], [username] and [password] are the UCMDB server host name, communications port, integration account user name and password, respectively. These parameters are configured in the UCMDB integration section in the System Information Record.
- [Service Name] is the display name of current service CI.

1.6.2.5.2.2. How to update the integration adapter configuration File (sm.properties)

The integration uses a properties file (sm.properties) as a configuration file of the adapter. Out-of-the-box, this file has been set up based on best practices, so usually you can keep the default parameter values. Optionally, you can update the parameter values to better suit your needs.

To update the `sm.properties` file:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41** > **Configuration Files**.
3. Click the properties configuration file: `sm.properties`.
4. Update the parameter values as needed. For a list of the parameters, see the following table.

Parameters in the sm.properties file

Parameter	Default value	Comment
timeout.minutes	10	The integration connection timeout value (in minutes)

Parameter	Default value	Comment
number.of.concurrent.sending.threads	6	The number of concurrent threads used for the data push feature ◦ 1: Disabled ◦ 2 or higher: Enabled **Note** If you are connecting to multiple Service Manager instances to improve the CI data push performance (see the URL Override configuration in How to create an integration point in UCMDB), you are recommended to increase this value for optimized performance. For example, set it to 12 if you are connecting to two Service Manager instances.

Parameter	Default value	Comment
min.objects.for.concurrent.sending	50	The minimum number of Service Manager objects that's required to use concurrent sending instead of single thread sending **Note** It is used for the push feature.
number.of.chunks.per.thread	3	The number of chunks per thread used for the push feature Total of number of chunks = number.of.chunks.per.thread * number.of.concurrent.sending.threads

Parameter	Default value	Comment
recommended.min.cis.per.chunk	50	The minimum allowed number of CIs per CI data chunk. This value is used to mitigate the risk of making your data chunks too small. For example, suppose your system uses the following settings: ◦ number.of.chunks.per.thread=3 ◦ number.of.concurrent.sending.threads=3 ◦ recommended.min.cis.per.chunk=50 When pushing 1000 CIs, the number of CIs in each chunk is calculated as 1000/3/3. The calculated value is greater than the recommended.min.cis.per.chunk value, so the calculated value is used. However, when pushing 270 CIs, the calculated value is less than the recommended.min.cis.per.chunk value, so the calculated value is used.
max.running.hours.for.multi.threaded	20	The maximum number of hours before a push request times out in a multi-threaded environment.

Parameter	Default value	Comment
number.of.cis.per.request	1000	The maximum number of objects retrieved from Service Manager by ID **Caution** It is used for the population and federation features. Do not set it to a number greater than 1000 in case the request has a 64K limit.
federation.request.pagination.switch	on	
federation.request.max.size	2000	size of query conditions for federation
federation.isin.max.count	500	size of isin{} size
use.global.id	true	This parameter is obsolete for the generic adapter starting with SM9.41.

Parameter	Default value	Comment
type.of.expand.enum	2	It configures the value mapping rule for the UCMDB enum type ◦ 0: The feature will be disabled ◦ 1: The enum type will expand to “{value}” ◦ 2: The enum type will expand to “{index}-{value}” **Note** It is used for the push feature.
op.pagination.switch	on	It indicates if pagination (client driven) is enabled ◦ on: Enabled. ◦ off: Disabled. **Note** It is used for the population feature.

Parameter	Default value	Comment
pop.pagination.recordcount	1000	The maximum number of records displayed on each page when pagination is enabled. **Note** It is used for the population feature.
pop.createci.key	sm_id	The UCMDB field of a CI record that stores the Service Manager CI ID. **Note** It is used for UCMDB ID Push Back by the population feature.
ucmdbid.pushback.request	UpdateucmdbID PushBackRequest	The web service request for pushing the UCMDB ID back to Service Manager. **Note** It is used for the population feature.

Parameter	Default value	Comment
check.sm.connections	false	It indicates whether to check the SOAP connections to Service Manager instances before running a job. You can enable it under any of the following circumstances: ◦ Your Service Manager is running in High Availability mode (with load balancing), and you want to connect UCMDB to multiple Service Manager instances. ◦ You want UCMDB to not run a job when no integration connections are available, rather than run the job and then report a failure.

1.6.2.5.2.3. How to add DEM reconciliation rules

It is possible that your Service Manager system already contains CI records that match CIs in your UCMDB system. Rather than add duplicate CI records to your Service Manager system, you can configure Service Manager to reconcile CI records between the two systems based on specified Discovery Event Manager (DEM) reconciliation rules.

A DEM reconciliation rule record enables you to specify what Service Manager queries you want to use to determine whether an existing CI record matches a CI in a UCMDB system. An administrator typically specifies reconciliation rules prior to starting the integration to the UCMDB system so that Service Manager will not create duplicate CI records.

When performing CI reconciliation between Service Manager and UCMDB, Service Manager uses the **ucmdb.id** field to query the device table or a join table to determine whether an Update or Create operation is needed. The reconciliation process is as follows:

1. The UCMDB system sends a web service message containing the latest CI attribute data to Service Manager.
2. Service Manager scans the web service message for the CI ucmdb.id value.

   Note

   The ucmdb.id field is displayed in the Actual State Section of the CI form, with a label of **Ucmdb ID**.

3. Service Manager searches for an existing CI record that has the same ucmdb.id value.
4. If Service Manager finds a CI that has the ucmdb.id value, no reconciliation is needed. Service Manager compares the UCMDB CI attributes to the Service Manager managed fields and runs the appropriate Discovery Event Manager (DEM) rules as needed to update the CI record in Service Manager.
5. If Service Manager can't find a CI that has the ucmdb.id value, it searches for a DEM reconciliation rule record that's defined for the CI type.
6. If no DEM reconciliation rule record is found, Service Manager creates the CI record according to the appropriate DEM rules.
7. If a DEM reconciliation rule record is found, Service Manager evaluates the rules and appends (ucmdb.id=NULL or ucmdb.id~=NULL and istatus="XXXXXX") to the reconciliation rules in the backend. The istatus value is retrieved from the DEM rule for deletion (for example, **Retired/Consumed**). If no istatus value is found in the DEM rule, Service Manager only appends ucmdb.id=NULL to the reconciliation rules in the backend.
8. If no matching CI record is found by the reconciliation rules, Service Manager creates the CI record according to the appropriate DEM rules.

9. If a matching CI record is found by the reconciliation rules, Service Manager updates the CI record according to the appropriate DEM rules

Multi-company mode

By default, Service Manager uses only the ucmdb.id field for CI reconciliation and the company.id field is not used. You can manually add DEM reconciliation rules.

Using join tables for reconciliation

When setting reconciliation rules, if the CI type you are reconciling has a joindef definition (as defined in the devtype table), use the join table name instead of the device table. For example, if you want to reconcile computer CIs, use the joinnode table instead of the device table.

Sequence of reconciliation

By default, Service Manager executes the reconciliation rules in their listed order. To change the order in which Service Manager reconciles CIs, you can add a numeric value to the **Sequence** field.

Steps to add a DEM reconciliation rule record

To create a DEM reconciliation rule record, follow these steps:

1. Click **Tailoring** > **Web Services** > **DEM Reconciliation Rules**.

   Service Manager displays the DEM Reconcile Record form.

2. In the **Table** field, select a Service Manager table against which you want to run reconciliation queries.

   Note

   Only tables that are specified in CI type records are displayed in the drop-down list. You can create only one DEM reconciliation record for one table.

3. Click **New**.

4. Enter expressions in the **Expression** column, and then enter a sequence number for each expression in the **Sequence** column. For example, you can add the following expressions for the **device** table. In this example, Service Manager looks for a matching record first based on the logical.name field, and then based on the sm.device.display.name field.

Expression	Sequence	Notes
logical.name=logical.name in $L.file	1	Checks to see if the CI Identifier matches the logical.name value of an existing CI in Service Manager. If no matching record is found, go to the next expression.
sm.device.display.name=sm.device.display.name in $L.file	2	Checks to see if the CI Display Name matches the display.name value of an existing CI in Service Manager.

5. Click **Add**.

   Service Manager creates the DEM reconciliation rule record.

1.6.2.5.2.4. How to add Discovery Event Manager rules

Service Manager uses Discovery Event Manager (DEM) to define the actions the system should perform when the actual state of an incoming configuration item (CI) record differs from the managed state of a CI record in Service Manager. The DEM rules allow you to define whether the Service Manager system adds, updates, or deletes CI records based on incoming UCMDB data.

For CI records only, the DEM rules also allow you to define how Service Manager should handle duplicate logical names.

To access DEM rules in Service Manager, navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**, and then click **Search** to view existing rules or click **New** to create new rules.

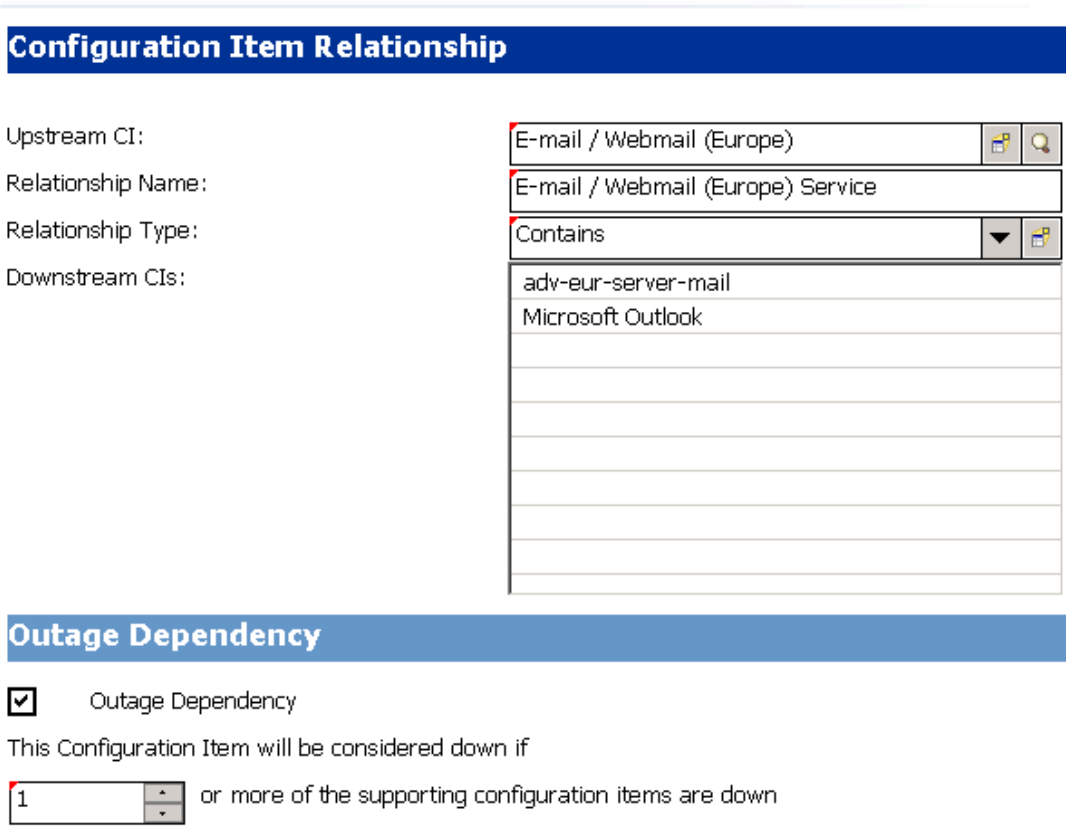
DEM rules

Service Manager offers the following rules options:

Action if the matching record doesn't exist

This is the action you want Service Manager to take if it can't find a matching CI record.

- **Add the record:**(Default) Service Manager will add a CI record when it can't find a matching record. See How to add DEM reconciliation rules to define what fields Service Manager uses to match CI records.
- **Add the record, and set dependency as true**: This option is available only for synchronization of CI relationship data. Service Manager adds the CI relationship record and enables outage dependency for the record by doing the following:
  - Checks the Outage Dependency checkbox;
  - Sets the number of dependent downstream CIs to 1. This is because UCMDB only supports one-to-one CI relationships.



- **Open an Incident:**Service Manager opens an incident for someone to review the new CI record. The incident enables someone to investigate whether the new CI record is compliant with your business practices.
- **Open a Change:**Service Manager opens an unplanned change for someone to review the new CI record. The change allows you to investigate whether the new CI record is compliant with your business practices. If the CI record is compliant, the change can be approved. If the CI record is not compliant, then the change can be denied and the CI record removed. The change record lists both the current and proposed attribute values.

Action if record exists but unexpected data discovered

This is the action you want Service Manager to perform if it doesn't find a matching CI attribute value.

- **Open a Change:** (Default) Service Manager opens an unplanned change to review the actual state of the CI record. The change allows someone to investigate whether the new attribute value is compliant with your business practices. If the value is compliant, the change can be approved. If the value is not compliant, then the change can be denied and the CI attribute value reverted to its managed state.
- **Log Results and update record:**Service Manager logs the results of the actual state of the CI record, and then updates the CI record.
- **Open an Incident:**Service Manager opens an Incident to investigate the actual state of a CI record and determines what actions must be performed or initiated to bring the record into compliance with Service Manager.

Action if record is to be deleted

This is the action you want Service Manager to perform if an external event specifies that the record needs to be deleted.

- **Delete record:** (Default for CI Relationship records) This option is available for synchronization of both CI and CI Relationship records. Service Manager automatically deletes the CI/CI Relationship record.
- **Open an Incident:** This option is available only for synchronization of CI Relationship records. Service Manager opens an incident to investigate the deleted record and determines which actions must be performed or initiated to bring the record into compliance with Service Manager.
- **Open a Change:** This option is available only for synchronization of CI Relationship records. Service Manager opens an unplanned change to review the deleted record. The change allows someone to investigate whether the deleted record is compliant with your business practices. If the record is compliant, the change can be approved. If the record is not compliant, then the change can be denied and the record added back to the system.
- **Update record to the selected status:** (Default for CI records) This option is available only for synchronization of CI records. Service Manager updates the status of the CI record to a value selected from the drop-down list (for example, **Retired/Consumed**), rather than deletes the record permanently.

  > **Note**
  >
  > Values available from the drop-down list are defined in the **ICM Status** global list.

- **Open a Change to update record to the selected status:** This option is available only for synchronization of CI records. Service Manager opens an unplanned change to update the CI record's status to a value selected from the drop-down list (for example, **Retired/Consumed**). The change allows someone to investigate whether the requested status change is compliant with your business practices. Once the change has been approved and closed, Service Manager automatically changes the CI record to the selected status. If the change has been denied, Service Manager makes no changes to the CI record.
- **Open an Incident to update record to the selected status:** This option is available only for synchronization of CI records. Service Manager opens an incident to update the record's status to a value selected from the drop-down list (for example, **Retired/Consumed**). Once the incident has been closed, Service Manager automatically updates the CI record to the selected status.

CI attributes displayed in Change and Incident records

Service Manager displays a Change Details section on the corresponding change or a CMDB Changes section on the corresponding incident when you configure DEM to open either

change records or incident records when it discovers CI attribute changes through the UCMDB-SM integration. Service Manager only displays a tab for CI attributes when the UCMDB-SM integration is enabled and you have defined a rule in the Discovery Event Manager to create a change or incident record when a CI is added, updated, or deleted.

Both the Change Details and CMDB Changes sections display the current CI attribute values alongside the actual attribute values discovered by UCMDB. You can use this information to approve or deny a change or escalate an incident to the proper assignment group.

Searching for Change and Incident records opened by the integration

You can use the following search criteria to find change and incident records opened by the UCMDB-SM integration.

Search options available for change and incident records

Record type	Search option available
Change	Search for records with the **unplanned change** category.
Incident	Search for records using the **Generated by UCMDB Integration** option.

1.6.2.5.2.5. How to add a CI relationship type to the integration for data push

Suppose you have added a new CI type to the integration and have created relationships between it and other CI types in UCMDB. For each of these relationship types, you need to perform the following tasks so that UCMDB can push each type of relationship to Service Manager.

As an example, the following steps illustrate how you add a relationship type named **Ownership** (between the **Cost** and **CostCategory** CI types) to the integration for data push.

> **Note**
>
> These steps assume that you have already added the **Cost** and **CostCategory** CI types to the integration.

1. Create a query to push the relationship type.

   See *How to create a query to push a relationship type* below.

2. Map the relationship query to a Service Manager Web Service object.

   See *How to map a relationship type query to the Service Manager Web Service object* below.

3. Create an XML configuration file for the new relationship type.

   See *How to create an XML configuration file for a relationship type* below.

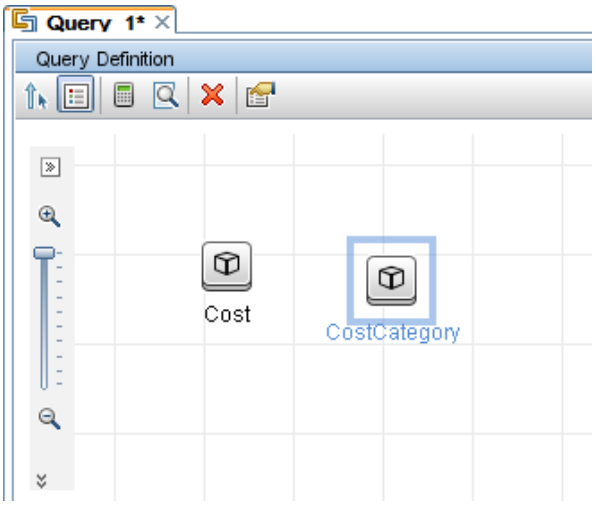
How to create a query to push a relationship type

Once you have created a relationship between two CI types, you must create a query to push the relationship to Service Manager.

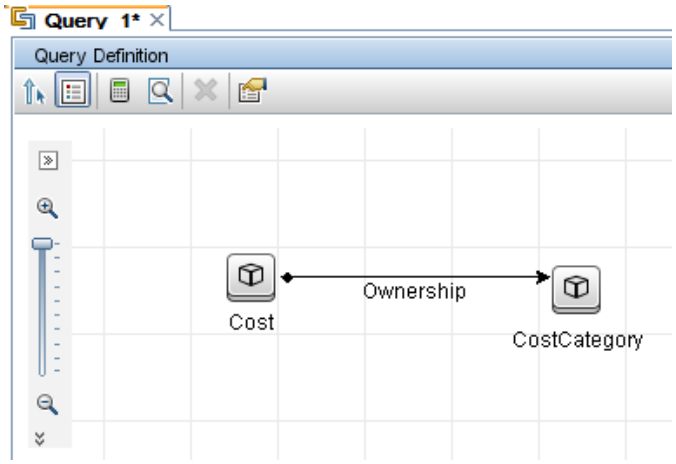
> **Note**
>
> Any query you create must conform to the Query requirements.

To create a new query called **Cost CostCategory Ownership Relations** for Ownership relationships between the **Cost** and **CostCategory** CI types:

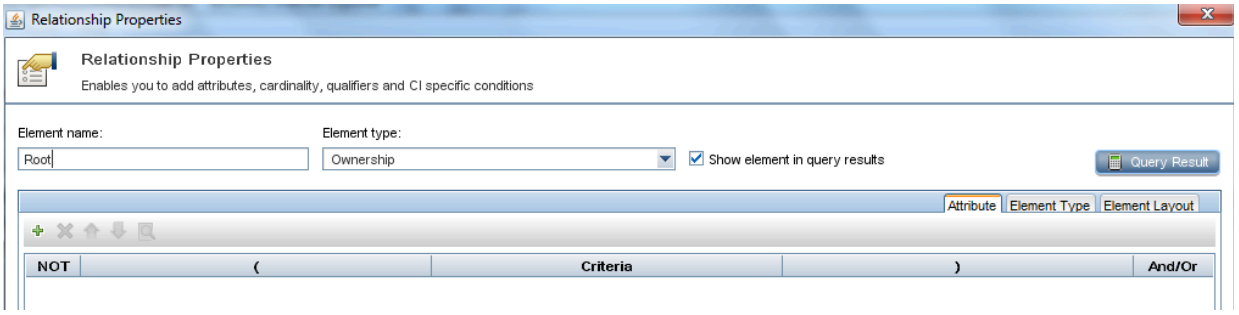
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio**.
3. Click **New** > **Query**. The system displays the Query Definition pane.
4. From the CI Type Selector, drag the **Cost** and **CostCategory** CI types to the query pane.

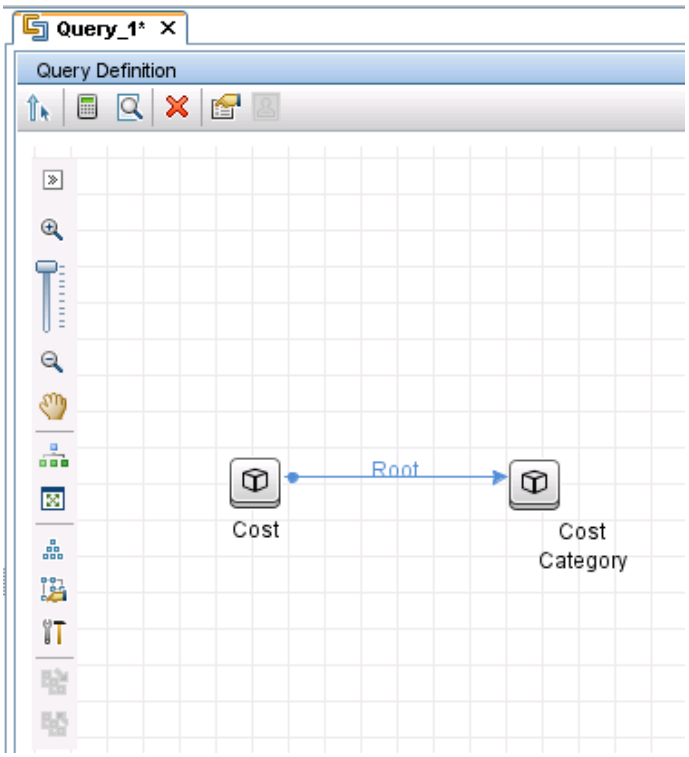


5. Create an Ownership relationship from **Cost** to **CostCategory**.
    1. Click the **Create Relationship** icon.
    2. Select the **Cost** node, and drag the arrow from it to the **CostCategory** node.
    3. Select **Regular Relationship**, and click **OK**.
    4. Select **Connection** > **Ownership**, enter Ownership for Relationship Name, and click **OK**. The system creates an Ownership relationship between the CI types.



6. Right-click the relationship arrow, and select **Relationship Properties**.
7. Change the element name from **Ownership** to **Root** (or a name starting with Root_), and then click **OK**.





8. Click the **Save** icon, and save the query as described in the following.
    1. Enter a query name. For example, `Cost CostCategory Ownership Relations Push`.
    2. Select the **Integration** > **Service Manager > Push** folder.
    3. Click **OK**.

The query is now created. You are ready to add this query to the push configuration file (`smPushConf.xml`).

How to map a relationship type query to the SM Web Service object

Once you have created a query for a relationship type, you need to map the query to the Relationship web service object in Service Manager as described in the following steps.

1. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41** > **Configuration Files**.
2. Click the **smPushConf.xml** file.
3. Add a mapping entry by copying an existing one for relationship push. For example, add the following mapping entry.

```
<tql name="SM Layer2 Topology Relations Push 2.0"
resourceCollectionName="Relationships" resourceName="Relationship"
/>
```

4. Change the TQL query name to the name of the query you created for the relationship type. For example, `Cost CostCategory Ownership Relations Push`.

```
<tql name="Cost CostCategory Ownership Relations Push"
resourceCollectionName="Relationships" resourceName="Relationship"
/>
```

5. Click **Save** to save the configuration file.

How to create an XML configuration file for a relationship type

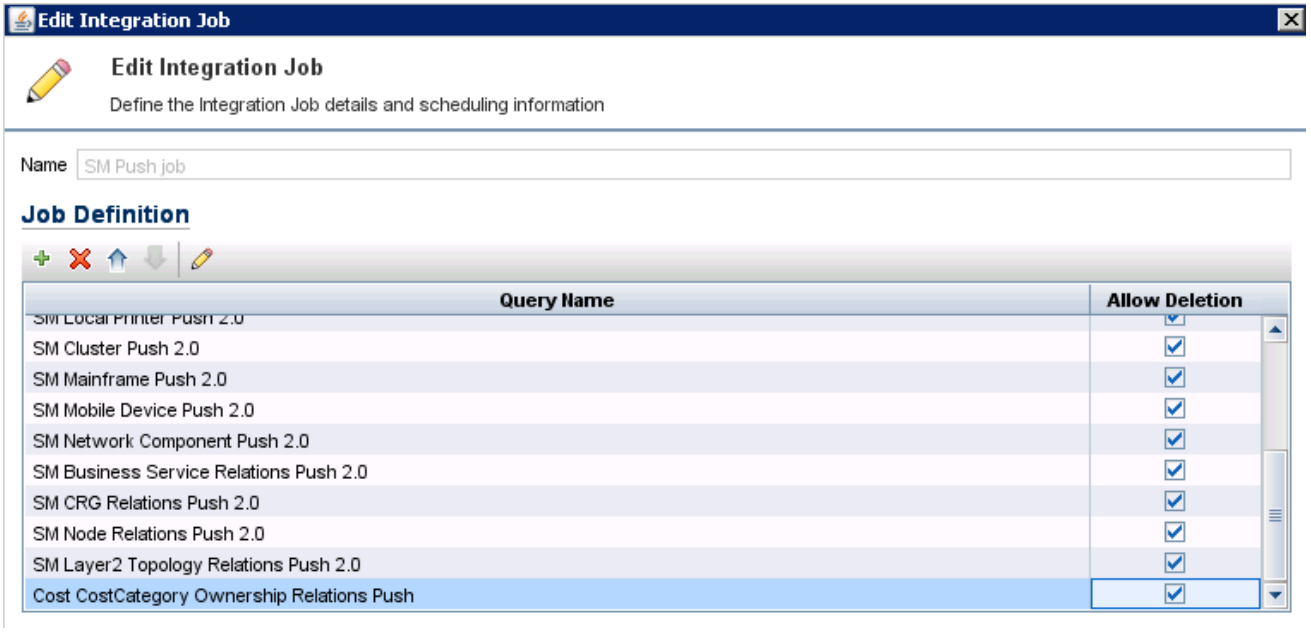
To create an XML configuration file for a relationship type:

1. Sign in to UCMDB with an administrator account.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41**, and select the adapter.
3. Click the **Create New Resource** icon.
4. Select **New Configuration File**.
5. Enter the full file name: `<AdapterID>/mappings/push/<filename>`. For example, `ServiceManagerEnhancedAdapter9.41/mappings/push/SM Cost CostCategory Ownership.xml`.
6. Click **OK** to save to the new file. The new file appears in the list of configuration files.
7. Open the new file in the XML Editor of the Visual Mapping interface.
8. Copy the content from an existing mapping file to overwrite the content of the new file in the XML Editor pane, and update the query name to the name of the query you created (`Cost CostCategory Ownership Relations Push`).

```
<?xml version="1.0" encoding="UTF-8"?>
<integration xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="../mappings_schema.xsd">
<info>
<source name="UCMDB"     version="10.20" vendor="HP"/>
<target name="SM"       version="9.40"    vendor="HP"/>
</info>
<import>
<scriptFile path="mappings.scripts.SMPushFunctions"/>
</import>
<!--
Push Cost to Cost Category Relations.
-->
<target_entities>
<source_instance query-name="Cost CostCategory Ownership Relations
Push" root-element-name="Root">
<target_entity name="Relationship">
<target_mapping name="RelationshipType" datatype="STRING"
value="SMPushFunctions.getDisplayName(Root['element_type'],ClassMod
el)"/>
<target_mapping name="ParentCI" datatype="STRING"
value="SMPushFunctions.getEndId(OutputCI.getExternalId().getEnd1Id(
))"/>
<target_mapping name="ChildCIs" datatype="STRING_LIST" value="
[SMPushFunctions.getEndId(OutputCI.getExternalId().getEnd2Id())]"/>
</target_entity>
</source_instance>
</target_entities>
</integration>
```

9. Click the **Save** icon to save the new configuration file.

Now, you have added the new relationship type to the integration. Next, you need to add the new relationship query to a data push job, as shown in the following figure. For detailed steps, see How to add a custom query to an integration job.



Caution

Before you run the relationship push job, make sure you have already added the relevant CI types (**Cost** and **CostCategory** in this example) to the integration and pushed the relevant CIs to Service Manager.

1.6.2.5.2.6. How to add a CI attribute to the integration for data push

You can use the following steps to add a CI attribute to the integration.

1. Does the CI attribute already exist in the UCMDB class model?

   Yes. Go to *Step 3*.

   No. Go to *Step 2*.

2. Add the CI attribute to the UCMDB class model.

   See *How to add the CI attribute to the UCMDB class model* below.

3. Add the CI attribute to the query layout.

   See *How to add a CI attribute to the query layout* below.

4. Add a web service field for the Service Manager CI type.

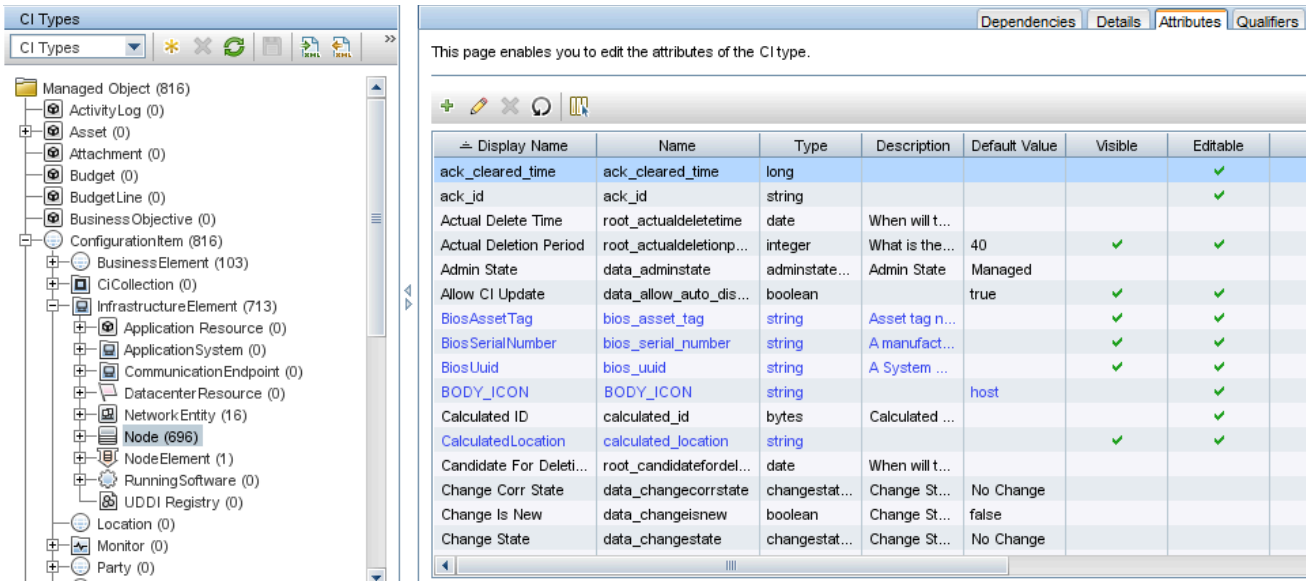
   See *How to add a Web Service field for the Service Manager CI type* below.

5. Map the CI attribute to the Service Manager web service field.

   See *How to map the CI attribute to a Service Manager Web Service field* below.

How to add the CI attribute to the UCMDB class model

The integration only uses a subset of the CI attributes available from your UCMDB system. Out-of-the-box, the integration consists of CI attributes that are typically managed from a Service Manager system such as host name and host DNS name. Before creating a new UCMDB CI attribute, you should determine if there are any existing CI attributes in your UCMDB system that provide the data you want. In most cases, there is an existing attribute tracking the data that you want to add to the integration. For example, the Node CI type contains many attributes that you can add to the integration.



The following steps illustrate how to add a new CI attribute to an existing CI type. This is not the expected typical scenario. Typically, you would add an existing CI attribute to the integration, which means you don't need to perform these steps.

Note

The integration doesn't require any special steps to add a CI attribute to the UCMDB class model. You can use the standard CI attribute creation procedures to add a CI attribute. For more information on CI attribute creation, see the UCMDB Help Center.

As an example, the following steps will add an attribute named **comments** to the Business Service CI type.

To add a CI attribute to the UCMDB class model:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **CI Type Manager**.
3. Select the CI type to which you want to add a new CI attribute from the CI Types navigation tree. For example, **ConfigurationItem** > **BusinessElement** > **BusinessService**.
4. Click the **Attributes** tab.
5. Click the **Add** icon.

   The Add Attribute window opens.
6. In Attribute Name, type the unique name you want to use for the new CI attribute. For example, comments.

> **Caution**
>
> The name can't include any of the following characters: ‘ / \ [ ] : | < > + = ; , ? *.

7. In Display Name, type the name that you want UCMDB to display in the interface. For example, Comments.
8. (Optional) In Description, type a description of the new CI attribute.
9. In Attribute Type, select **Primitive** or **Enumeration/List** depending on the data type of the attribute. For example, select **Primitive** and select **string**.
10. In Value Size, type the maximum character length that the attribute can have. For example, 300.
11. Select the **Enable default value** option, and enter a default value. Or, don't select this option to leave the default value blank.
12. Click **OK** to save the attribute.
13. Click the **Save** icon  to save attribute changes to the CI type.

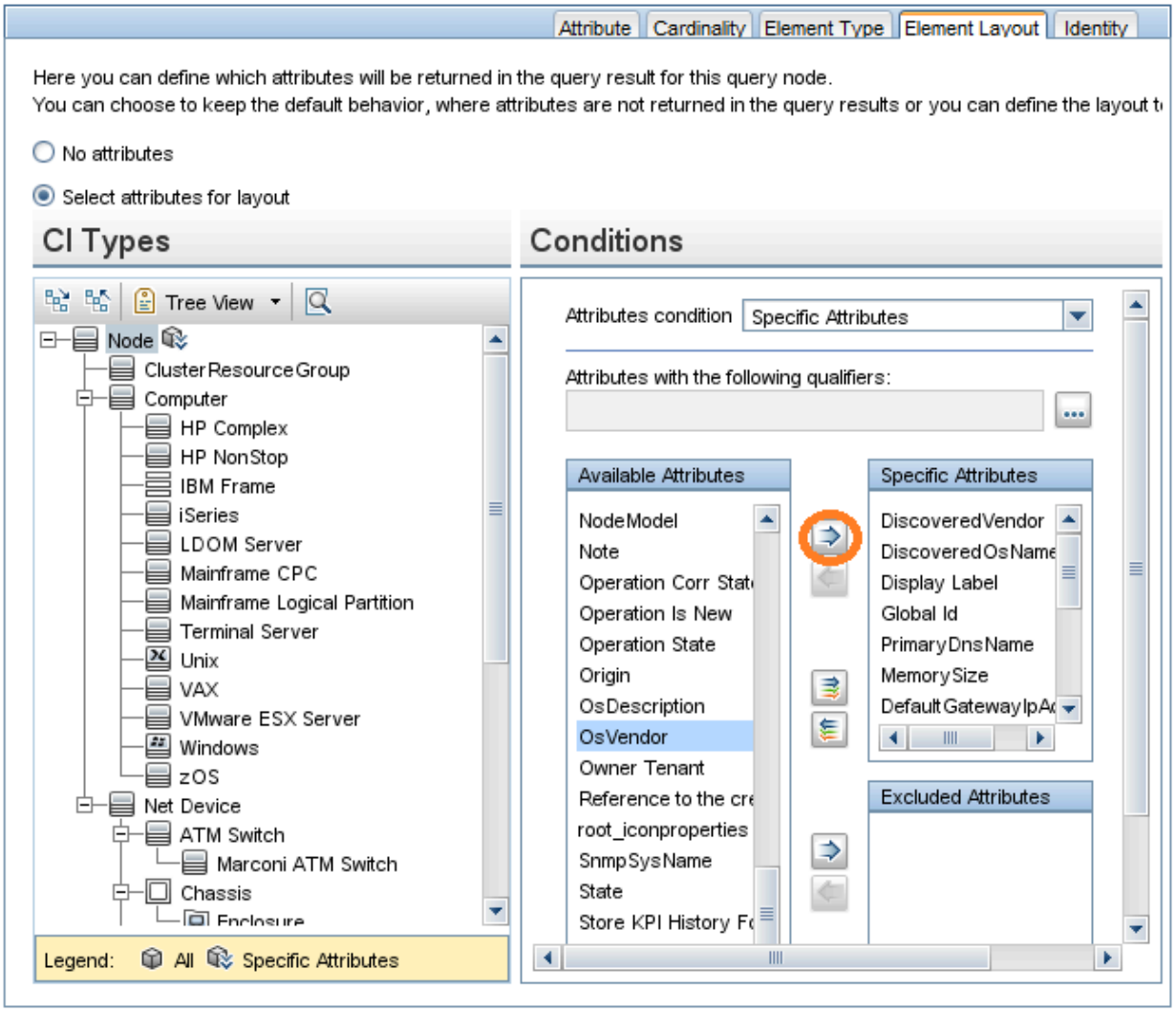
Now, the attribute has been added to the CI type. Next, you need to add the attribute to a query that synchronizes the CI type. See *How to add a CI Attribute to the query layout* below.

How to add a CI attribute to the query layout

To add an existing CI attribute to the integration, you must add this attribute to the layout setting from the query that synchronizes the CI type. You must know which CI type contains the CI attributes that you want to add to the integration.

To add a CI attribute to the query layout:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio**.
3. For Resource Type, select **Queries**.
4. From the Queries navigation tree, click **Integration** > **Service Manager**.
5. Select the query that manages the CI type whose attributes you want to add to the integration. For example, **Push > SM Computer Push 2.0**. UCMDB displays the TQL layout of the query.
6. Select the node from the query layout that contains the CI attribute that you want to add to the integration. For example, **Root**.
7. Right-click the node and select **Query Node Properties**. The Query Node Properties window opens.
8. Select the CI type (**Database** in this example), and click the **Element Layout** tab.
9. Select the CI attribute that you that want to include in the integration from the **Available Attributes** list, and click the **Add** icon to add it to **Specific Attributes** list. For example, **Comments**.



10. Click **OK** to save the node properties.
11. Click the **Save** icon to save the query.

Now, the CI attribute has been added to the relevant query layout. Next, you need to add a Service Manager web service field that will be mapped to this UCMDB CI attribute.

How to add a Web Service field for the Service Manager CI type

The integration uses only a subset of the CI attributes available from your Service Manager (SM) system. CI attributes must be mapped between UCMDB and SM. Before creating a new SM CI attribute that will be mapped to the UCMDB CI attribute you added previously, you must determine if there are any existing CI attributes in your Service Manager system that provide the data you want. In most cases, there is an existing attribute tracking the data that you want to add to the integration. For example, the Computer CI type contains many attributes that you can add to the integration.

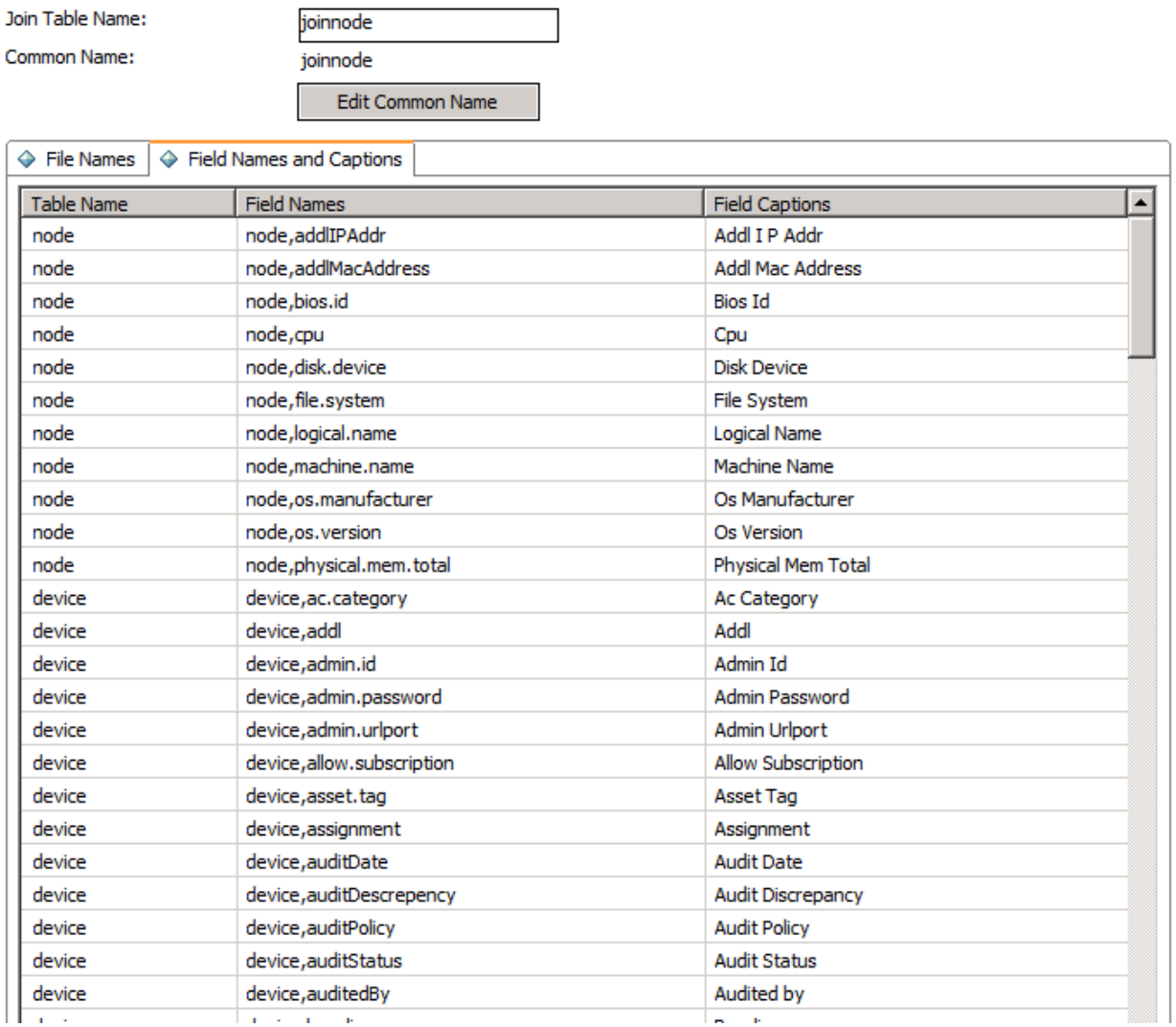


Table Name	Field Names	Field Captions
node	node,addlIPAddr	Addl I P Addr
node	node,addlMacAddress	Addl Mac Address
node	node,bios.id	Bios Id
node	node,cpu	Cpu
node	node,disk.device	Disk Device
node	node,file.system	File System
node	node,logical.name	Logical Name
node	node,machine.name	Machine Name
node	node,os.manufacturer	Os Manufacturer
node	node,os.version	Os Version
node	node,physical.mem.total	Physical Mem Total
device	device,ac.category	Ac Category
device	device,addl	Addl
device	device,admin.id	Admin Id
device	device,admin.password	Admin Password
device	device,admin.urlport	Admin Urlport
device	device,allow.subscription	Allow Subscription
device	device,asset.tag	Asset Tag
device	device,assignment	Assignment
device	device,auditDate	Audit Date
device	device,auditDescrepency	Audit Discrepancy
device	device,auditPolicy	Audit Policy
device	device,auditStatus	Audit Status
device	device,auditedBy	Audited by

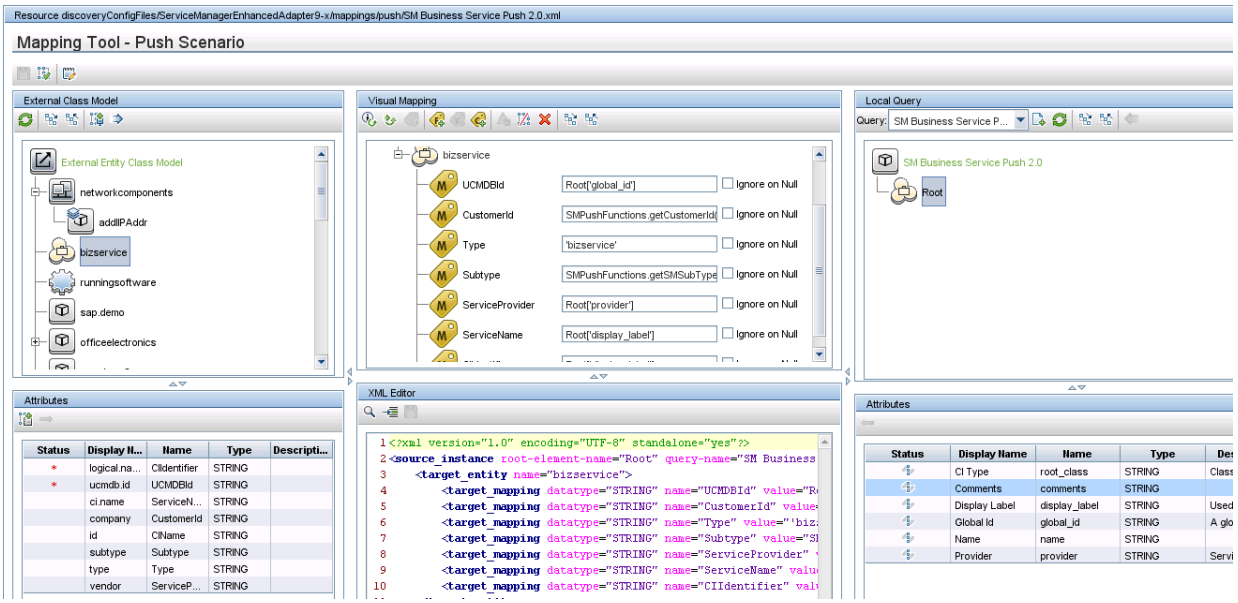
If you decide to use an existing attribute instead of creating a new one, you can directly modify the web service definition in Service Manager to expose that attribute, and then map this SM attribute to the UCMDB attribute by using the UCMDB Visual Mapping tool (see *How to map the CI attribute to a Service Manager Web Service field* below).

If you decide to add a new attribute, follow the steps below. The steps vary depending on the data type of the attribute: simple (string, for example), or complex (array of structure or structure).

Add a simple attribute to the SM CI type

As an example, the following steps illustrate how to add a simple attribute (**comments**) to an existing CI type (Business Service).

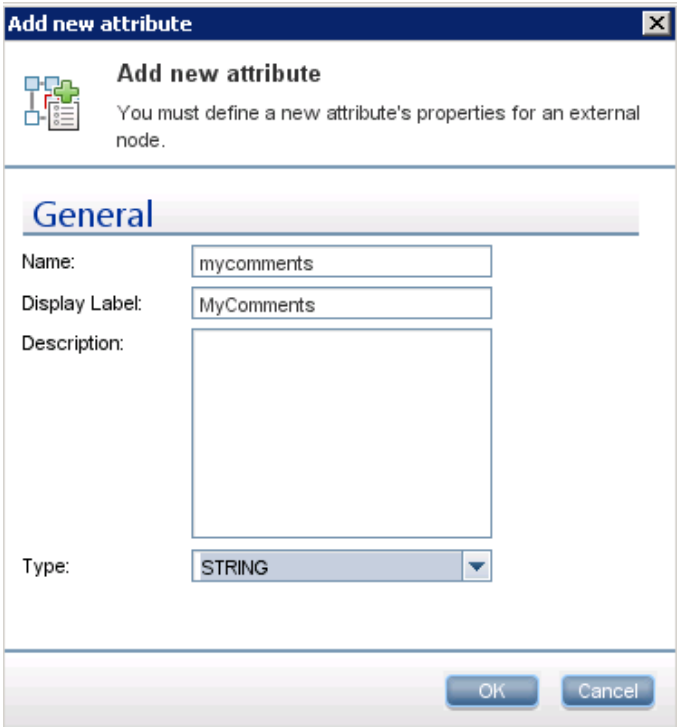
1. Sign in to UCMDB as a system administrator.
2. Click **Data Flow Management** > **Adapter Management**, and then select **ServiceManagerEnhancedAdapter 9-x** to open the corresponding XML mapping file (**SM Business Service 2.0.xml** in this example) with the Visual Mapping tool editor.
3. On the External Class Model pane, select the CI type (**bizservice**).



4. On the left-side Attributes pane (which displays the fields that are retrieved from the Service Manager web service object for the Business Service CI type), click the **Add New Attribute to Selected External Node** icon.

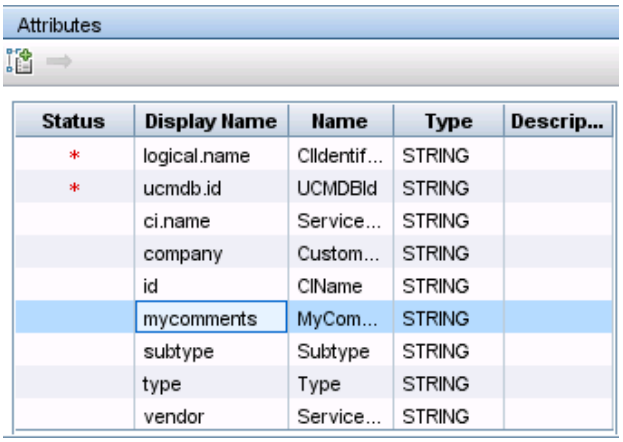
5. Type a name and display label for the new attribute, and click **OK**.

> **Caution**
>
> If the you enter a name that duplicates or is part of an existing field name in the attribute table of the Service Manager CI type or in the **device** table, an error will occur indicating the attribute could not be created.

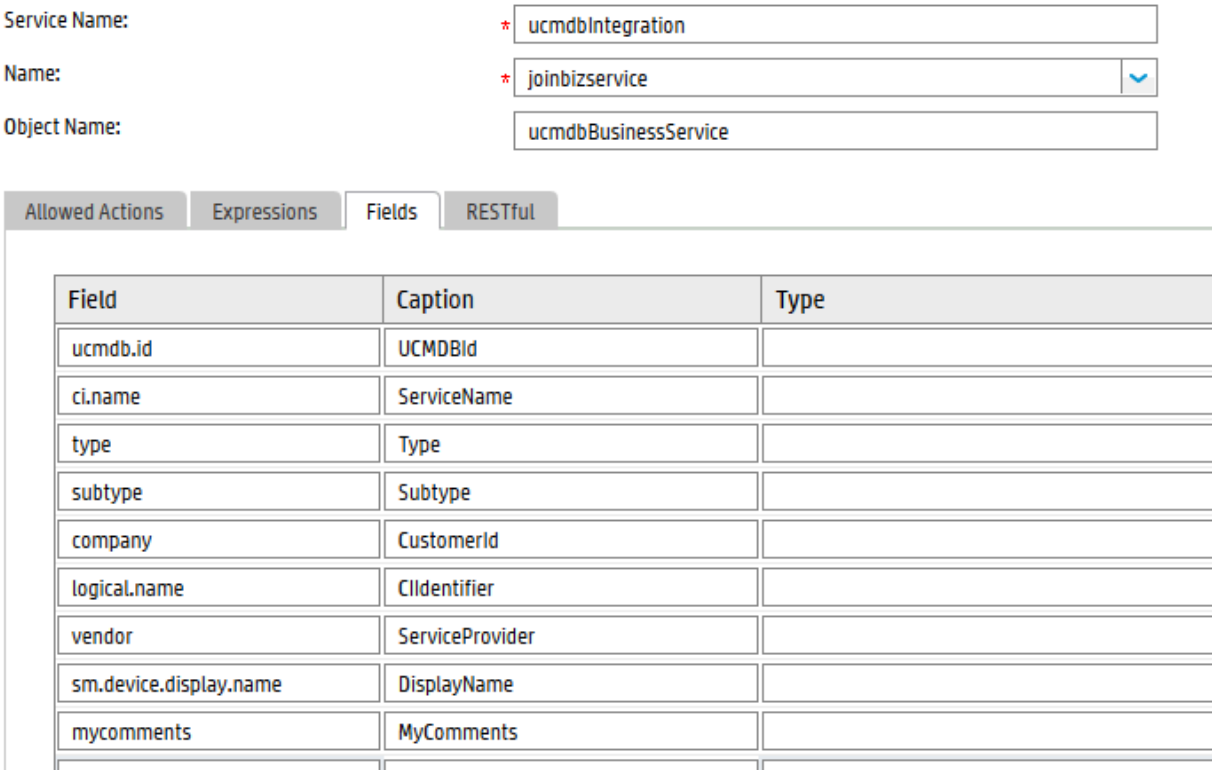


6. Click OK again to confirm the attribute creation.

The new attribute (**mycomments**) appears in the Service Manager attribute list.



At the same time, UCMDB invokes the Service Manager Web Service API to add the new attribute, expose it in the Web Service API and set it as a managed field. You can open the web service object in Service Manager to verify the new attribute (**Tailoring** > **Web Services** > **Web Service Configuration**).



Note

The caption for the istatus field must be AssetStatus. Do not update this caption. Otherwise, the data push and data population may fail.

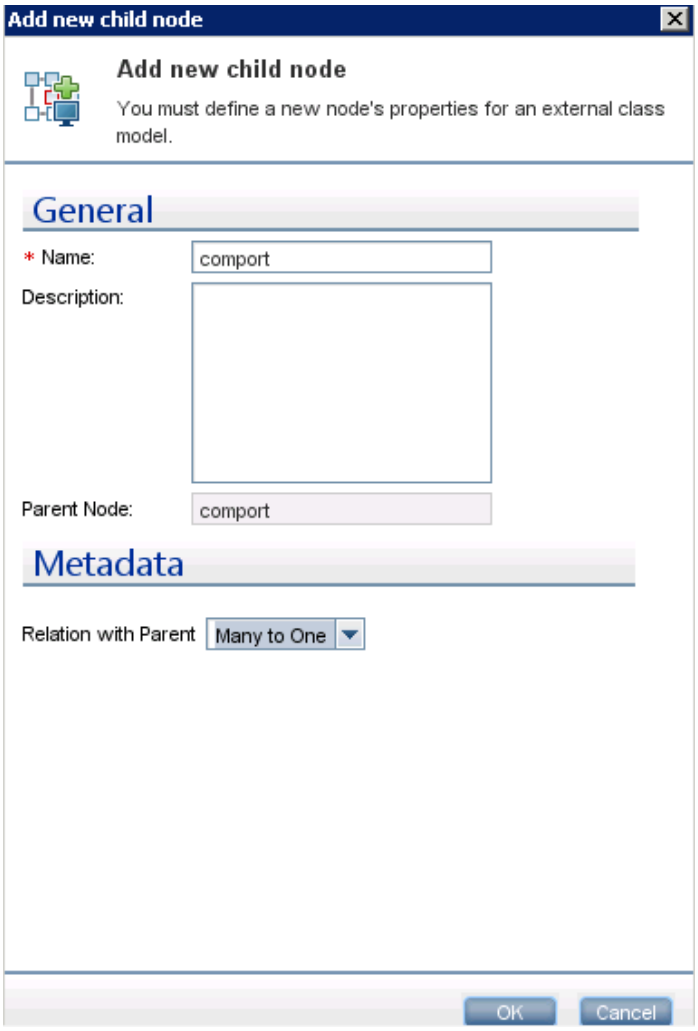
7. Save the configuration file.

Add an array of structure or structure to the CI type

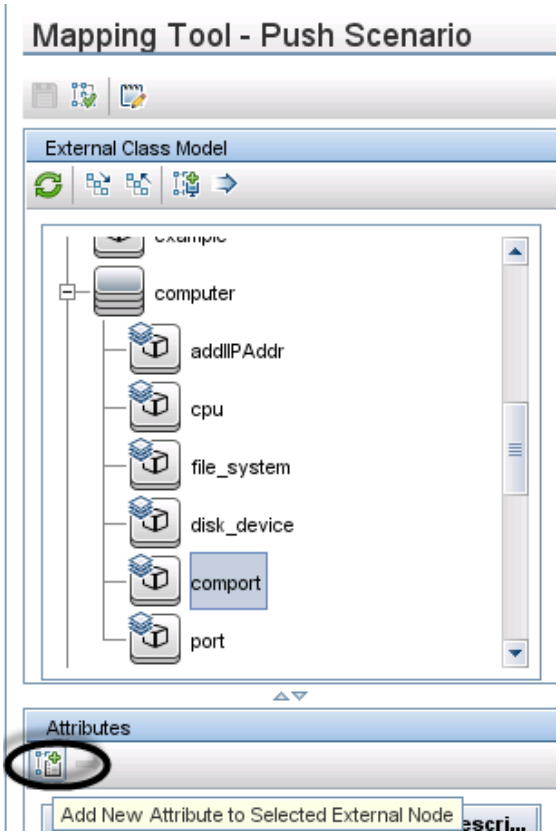
Service Manager uses an Array of Structure or Structure to store complex attributes, for example, the ports of a Computer CI. You can also create such kind of complex attributes through the Visual Mapping tool in UCMDB.

The following steps illustrate how to add an attribute named **comport** to the Computer CI type.

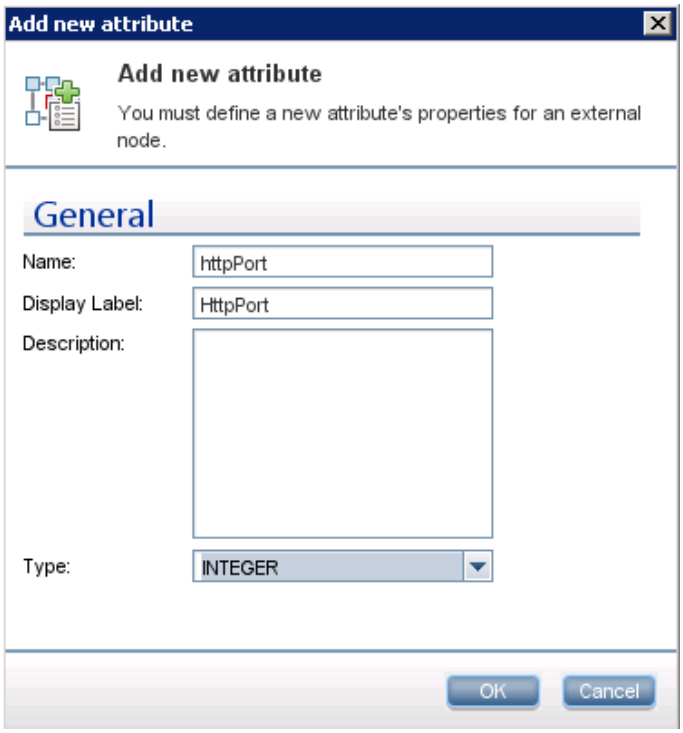
1. Sign in to UCMDB as a system administrator.
2. Open the corresponding XML mapping file (**SM Computer Push 2.0.xml** in this example), with the Visual Mapping tool editor.
3. Select the CI type (**computer**), and click the **Add New CI Type to External Class Model** icon.
4. In the child CI type creation dialog that appears, enter a name and description for the attribute, select **Many to One** for **Relation with Parent** if the attribute data type is array of structure or select **One to One** if the data type is structure.



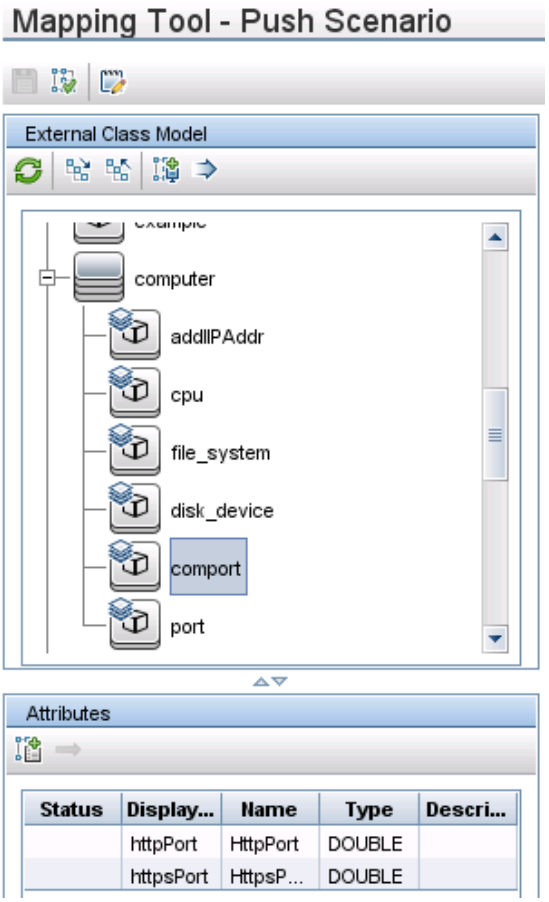
5. Click **OK**. The attribute is created on the Service Manager side.
6. Select the new attribute in the External Class Model pane, and click the **Add New Attribute to Selected External Node** icon.



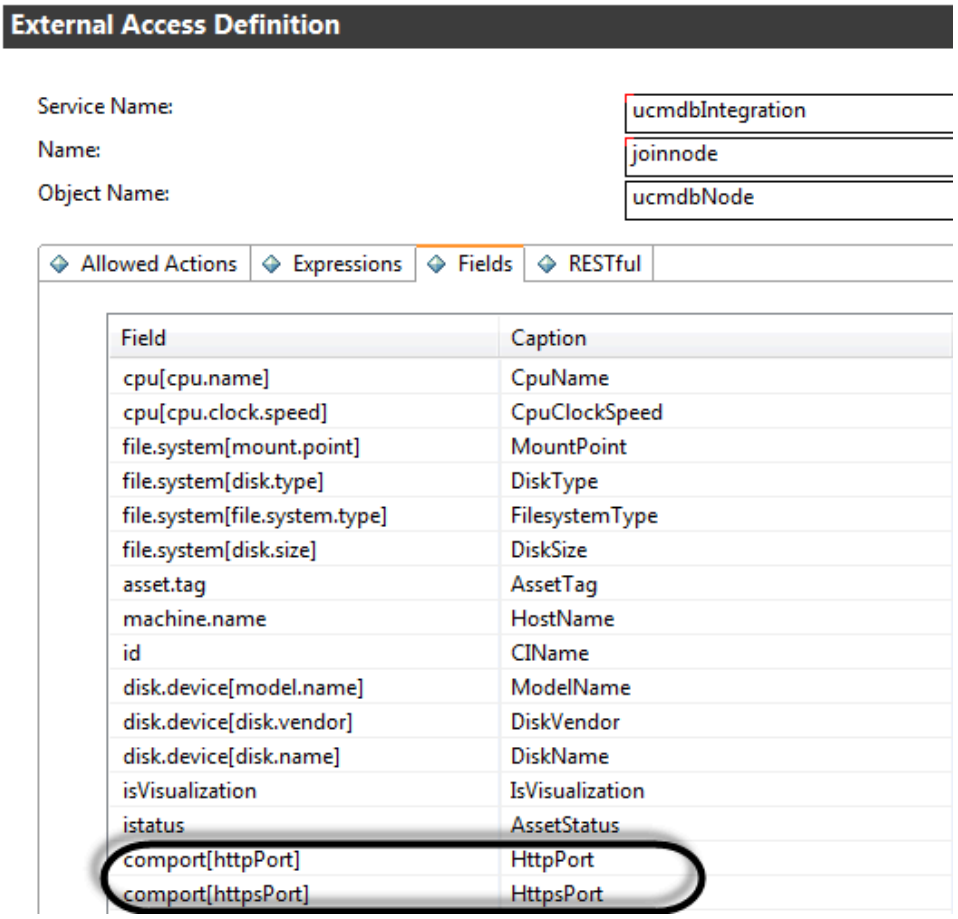
7. Add a field (**httpPort**) to the array of structure, and click **OK**. Click **OK** again to confirm the attribute creation.



8. Repeat the previous two steps to add more fields to the array of structure as needed.



The attribute is also automatically added to the corresponding web service object in Service Manager.



9. Save the configuration file.

Next, you need to map this Service Manager CI type attribute to the one you added previously in UCMDB.

How to map the CI attribute to a Service Manager Web Service field

The integration uses an adapter to transform UCMDB CI attributes to web services objects recognized by Service Manager. The adapter in turn specifies what XML transformation files the integration should use to convert UCMDB queries into properly formatted Service Manager web services messages.

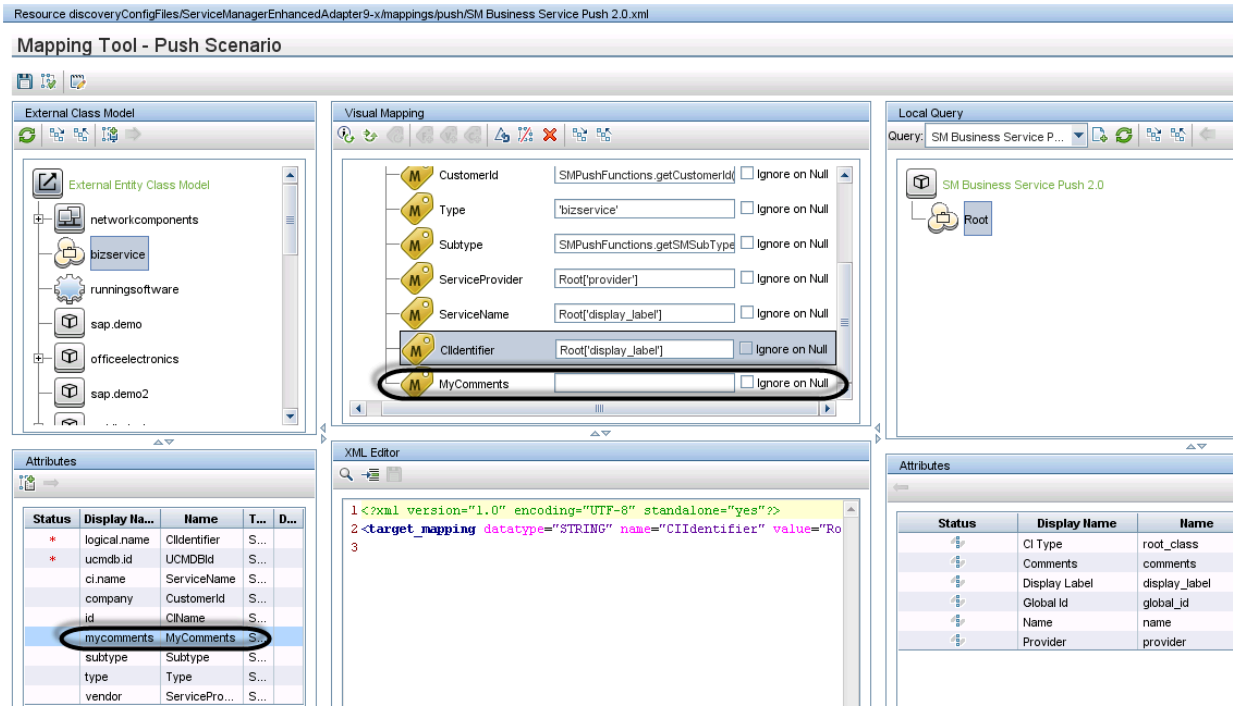
Out-of-the-box, each integration query has a corresponding XML configuration file that maps to a particular CI type in UCMDB. In addition, each attribute for which you enabled calculation requires its own entry in the XML configuration file. Without an XML transformation entry, Service Manager can't receive any CI attribute updates from your UCMDB system.

If you want to add a new attribute to the integration, you must edit the XML configuration file for the parent CI type and add an entry for the CI attribute. For information about which CI types each query manages, see Queries for push.

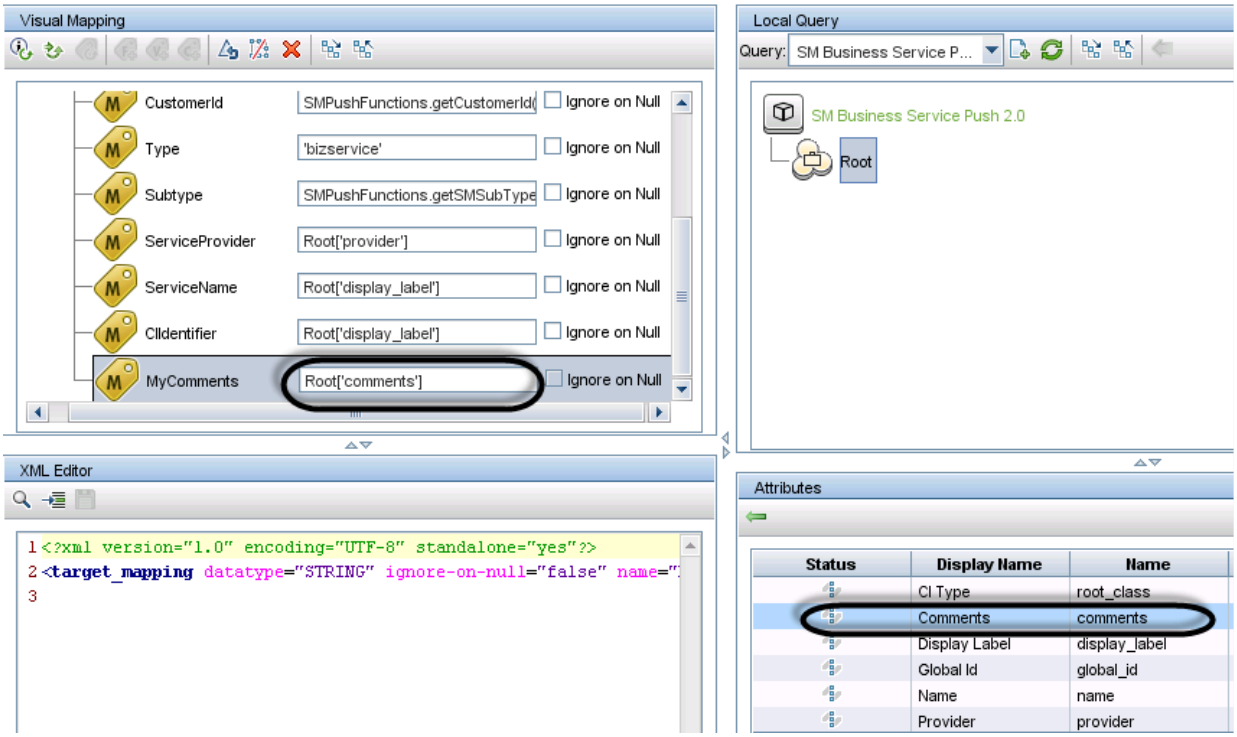
The following steps illustrate how to map UCMDB CI attribute **Comments** (which you added previously) to the **MyComments** Service Manager attribute (which you created in the previous step).

To map a UCMDB CI attribute to an SM CI attribute:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41** > **Configuration Files**.
3. Double-click the XML configuration file that manages the parent CI type of your CI attribute. For example, open **SM Business Service Push 2.0.xml** to add an attribute to the **SM Business Service Push 2.0** query.
4. Select the relevant CI type in the External Class Model pane (**bizservice** in this example), and drag and drop the attribute (**mycomments**) from the left-side Attributes pane into the Visual Mapping pane.



5. Select the relevant Node in the Local Query panel (**Root** in this example), and drag and drop the attribute (**comments** in this case) into the mapping entry that you created in the previous step.



6. Save the XML configuration file. The **Comments** attribute in UCMDB is now mapped to the **MyComments** attribute in Service Manager.

> **Note**
>
> When you create or edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

1.6.2.5.2.7. How to add a CI type to the integration for data push

You can use the following steps to add a CI type to the integration.

1. Does the CI type already exist in the UCMDB class model?

   Yes. Go to *Step 3*.

   No. Go to *Step 2*.

2. Add the CI type to the UCMDB class model.

   See *How to add the CI type to the UCMDB class model* below.

3. Create a query to synchronize the CI type.

   See *How to create a query to synchronize the CI type* below.

4. Add the CI type's attributes to the query layout.

   See *How to add the CI type's attributes to the query layout* below.

5. Add the CI type to Service ManagerService Manager.

   See *How to add the CI type to Service Manager* below.

6. Map the CI type's attributes to web service fields.

   See *How to map the CI type's attributes to Web Service fields* below.

7. Add custom queries to integration data push jobs.

   See *How to add a custom query to an integration job* below.

How to add the CI type to the UCMDB class model

Before creating a new UCMDB CI type, you should determine if there are any existing CI types in your UCMDB system that provide the CI attributes you want. In most cases, you can create links to one or more existing CI types to create a new logical CI type for use by the integration.

The following steps illustrate how to create a new CI type called SM RDBMS based on an existing CI type called database.



Note

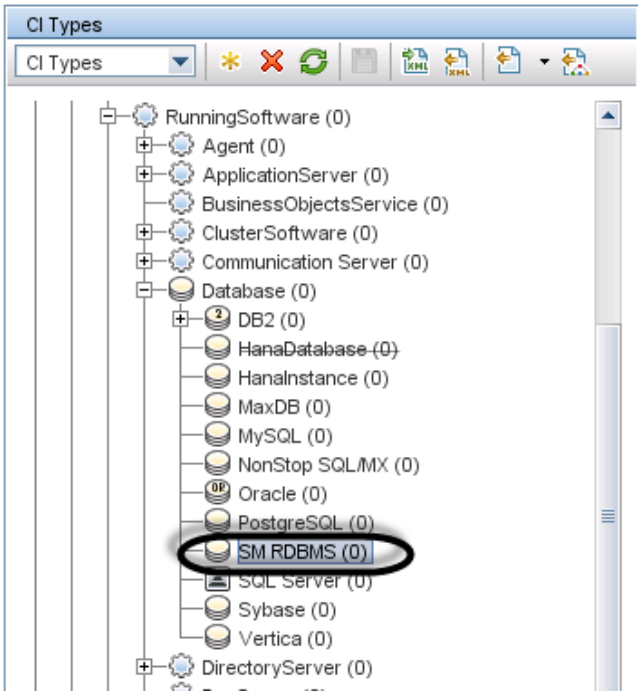
The integration doesn't require any special steps to add a CI type to the UCMDB class model. You can use the standard CI type creation procedures to add a CI type. For more information on CI type creation, see the UCMDB Help Center.

To add a CI type to the UCMDB class model:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **CI Type Manager**.
3. Select the base CI type you want to use for your new CI type from the CI Types navigation tree: **Managed Object** > **ConfigurationItem** > **Infrastructure Element** > **Running Software** > **Database**.
4. Click the **New** icon  .

   The Create Configuration Item Type window opens.
5. In Name, type the unique name you want to use for the new CI type. For example, sm_rdbms.

   > **Caution**
   >
   > The name can't include any of the following characters: ‘ / \ [ ] : | < > + = ; , ? *.
6. In Display Name, type the name you want UCMDB to display in the interface. For example, SM RDBMS.
7. In Description, type a description of the new CI type. This is an optional field. For example, Hosts running relational databases.
8. In Base CI Type, verify that the proper base CI type is selected. Your new CI type will inherit the attributes of the base CI type you select here. For example, **Database**.
9. Click **Next**. The wizard displays a list of CI attributes from the base CI type.
10. Add, edit, or remove CI attributes as needed for the new CI type. For example, accept the default attributes inherited from **Database**.
11. Click **Next**. The wizard displays a list of qualifiers from the base CI type.
12. Add or remove qualifiers as needed for the new CI type. For example, accept the default qualifiers.
13. Click **Next**. The wizard displays a list of icons associated with the CI type.
14. Select the icons associated with this CI type. For example, accept the default abstract class icon.
15. Click **Next** to add any menu item properties or label definitions as needed. For example, accept the default settings from the base CI type.
16. Click **Finish** to create the CI type.

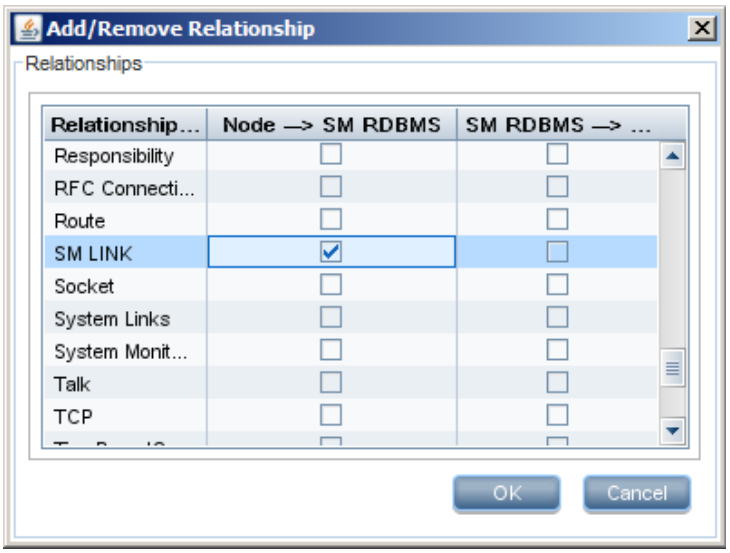


17. Select your new CI type from the tree. For example, **SM RDBMS**.
18. Browse to an existing CI type you want to link to, and control-click it to add it to your selection. For example, **Node**.

> **Note**
>
> Choose an existing CI type that has the attributes that you want to be part of your new logical CI type.

19. Right-click one of the selected CI types, and click **Add/Remove Relationship**. The Relationships window opens.
20. Create an SM Link relationship from the existing CI type to the new CI type. For example, from **Node** to **SM RDBMS**.

> **Note**
>
> You need to create a new SM Link relationship if it doesn't exist.



21. Click **OK** to create the relationship.
22. Click the **Save** icon to save the CI type.

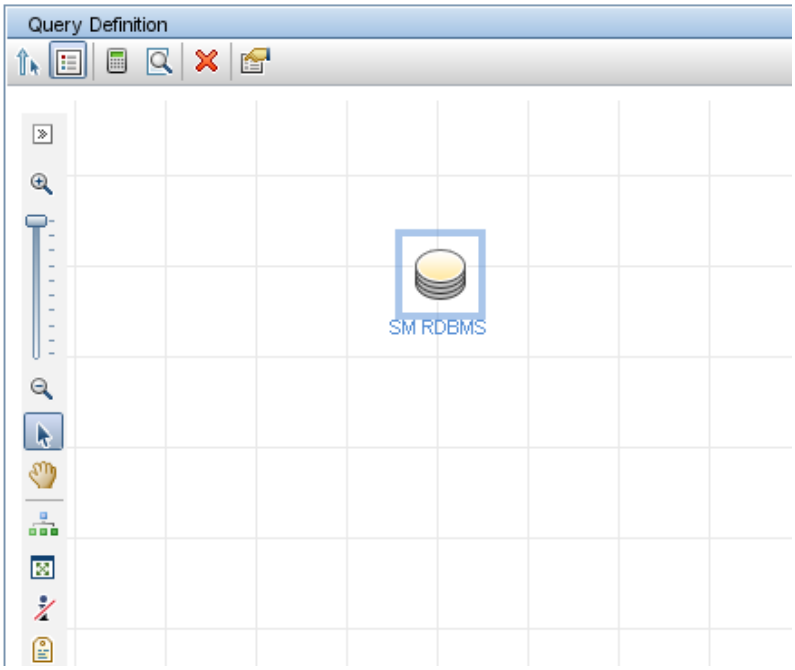
How to create a query to synchronize the CI type

The integration uses queries to gather CI attribute values and pass them to your Service Manager system. You must create a query for any CI type you add to the integration. Any query you create must conform to the Query requirements.

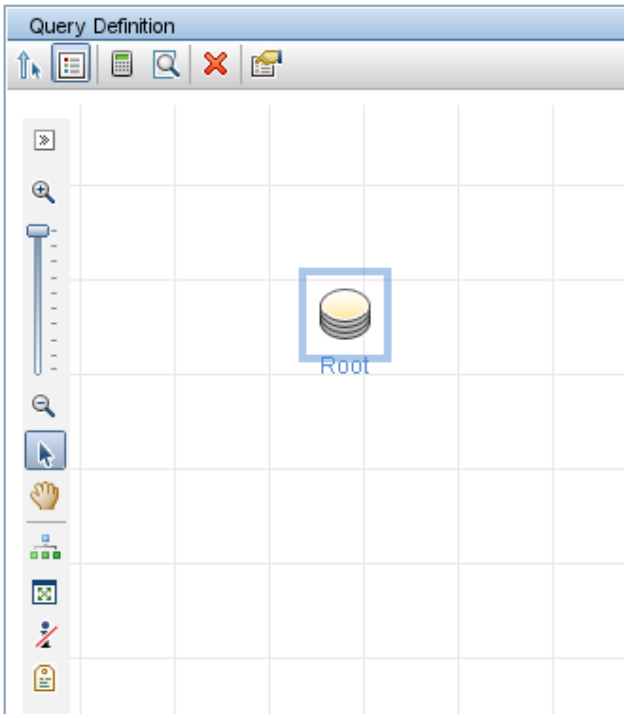
The following steps illustrate how to create a new query named **rdbmsData** for the **SM RDBMS** CI type described in previous sections.

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio**.
3. From the Queries navigation tree, click **Integration** > **Service Manager**.
4. Right-click **Service Manager**, and select the **New** > **Query**.

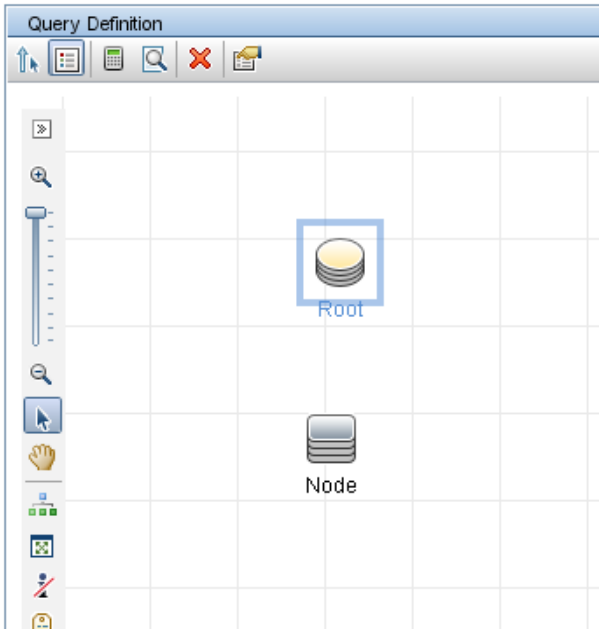
   The Query Definition window opens.
5. Find the CI type that will be the root node of your query from the CI Type Selector. This CI type is typically the one that provides the most attributes for the CI. For example, **Managed Object** > **ConfigurationItem** > **InfrastructureElement** > **RunningSoftware** > **Database** > **SM RDBMS.**
6. Drag the root CI type from the CI Type Selector and drop it into the empty Editing pane. UCMDB displays the icon of the CI type.



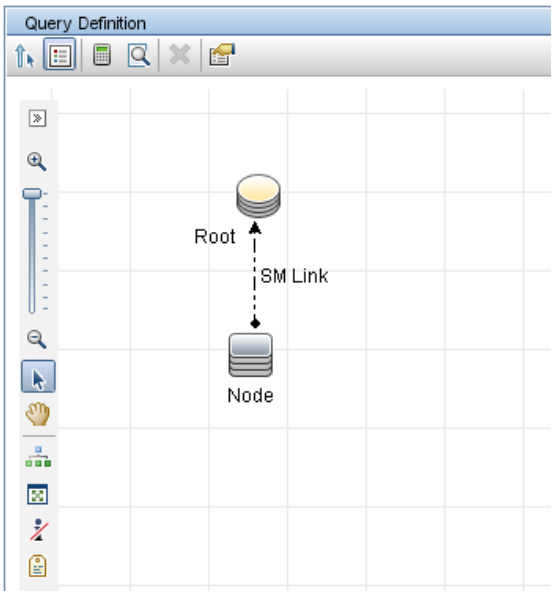
7. Right-click the CI type icon, and select **Query Node Properties**. The Node Properties window opens.
8. Change the Element Name to **Root**.
9. Click **OK** to save the node properties.



10. Find any additional CI types you want to add to the query from the CI Type Selector. These CI types typically provide additional CI attributes. For example, **Managed Object** > **ConfigurationItem** > **Infrastructure Element** > **Node**.
11. Drag the additional CI type from the CI Type Selector and drop it into the Query Definition pane. UCMDB displays the icon of the additional CI type.



12. Create relationships between the Root CI type and the additional CI types as needed. For example, create an SM Link between **Root** and **Node**.
    1. Select **Root** and control-click the additional CI type. For example, **Node**.
    2. Right-click one of the selected items, and click **Add Relationship**. The Add Relationship window opens.
    3. Type a Relationship Name. For example SM Link.
    4. Select the relationship direction.
    5. Click **OK** to add the relationship.



13. Repeat step 10 to step 12 for each additional CI type you want to add to the query. For example, SM RDBMS doesn't need any additional CI types.
14. Click the **Save** icon to save the query.
    1. In Query Name, type the unique name you want to use for the new query. For example, rdbmsData.

2. In the folder tree, select the folder in which you want to save the query. For example, **Root** > **Integration** > **Service Manager** > **Push**.
3. Click **OK**. UCMDB adds your new query to the query list.

How to add the CI type's attributes to the query layout

To add a CI attribute to the integration, you must enable the calculation layout setting from the query that synchronizes the CI type. Because you must enable calculation for each attribute you want to add to the integration, you should be familiar with the integration CI types and their attributes.

The following steps illustrate how to enable calculation for attributes of the **SM RDBMS** CI type described in previous sections.

To add a CI type's attributes to the query layout:

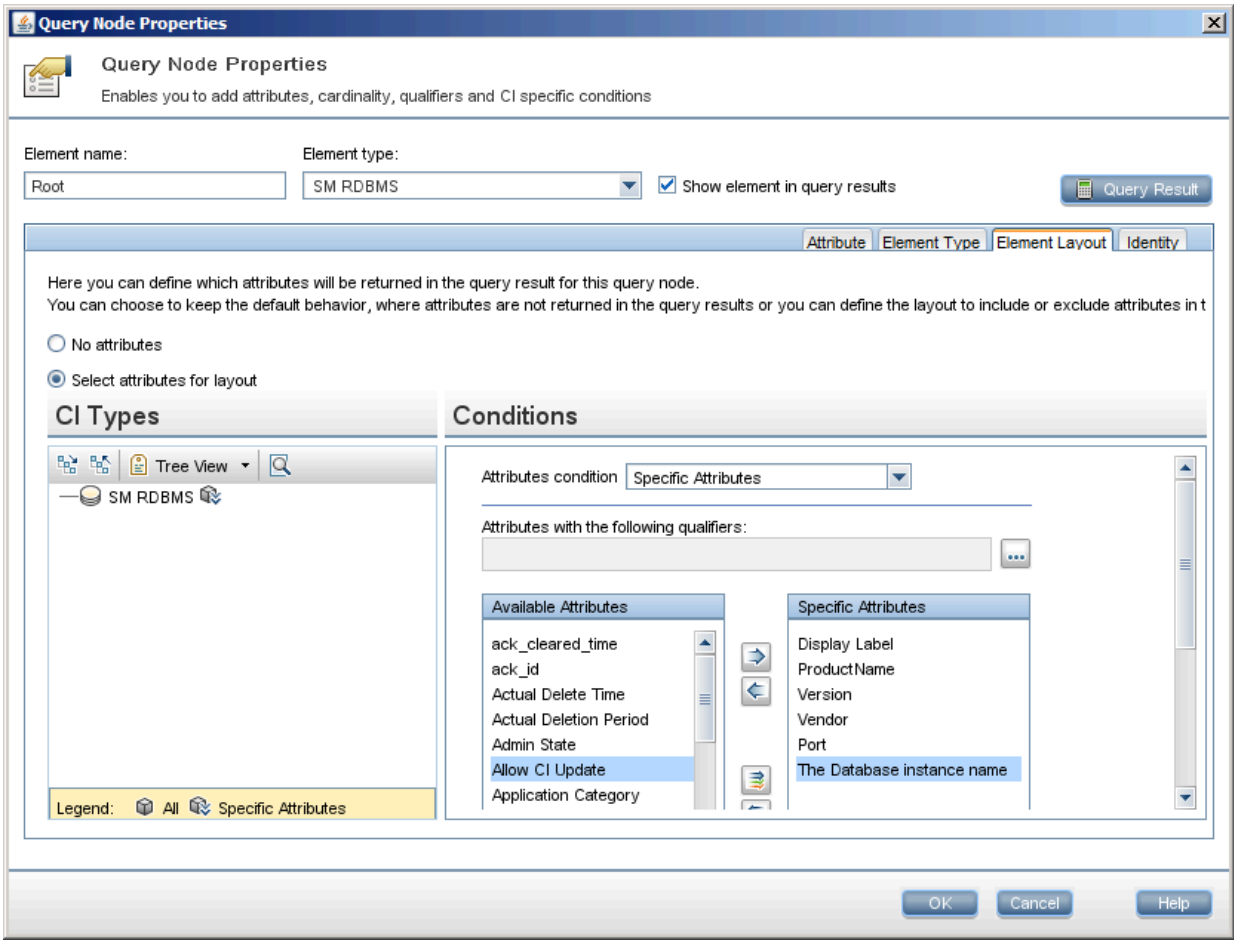
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio**.
3. From the Queries navigation tree, click **Integration > Service Manager**.
4. Double-click the query that manages the CI type whose attributes you want to add to the integration. For example, **rdbmsData**. UCMDB displays the TQL layout of the query.
5. Right-click the **Root** node from the query layout, and then select **Query Node Properties**. The Query Node Properties window opens.



Caution

Your integration query must contain a node called **Root**. See Query requirements for more information.

6. Click the **Element Layout** tab, and select the **Select attributes for layout** option.
7. Select **Specific Attributes** from the **Attributes condition** list, and from the Available Attributes list select the attributes you want and move them to the Specific Attributes list. For example, add the **Product Name**, **Application Version Description**, **The Database Instance Name**, and **Port** attributes.



8. Click **OK** to save the query node properties.
9. Select any additional node that contains the attributes you want to add to the integration. For example, **Node**.
10. Repeat *step 4* through *step8* for each additional node.
11. Click **OK** to save the query node properties.
12. Repeat step 9 to step 13 for each additional node that contains the attributes you want to add to the integration.
13. Click the **Save** icon to save the query .

How to add the CI type to Service Manager

Before creating a new Service Manager CI type, you should determine if there are any existing CI types in your Service Manager system that provide the CI attributes you want. In most cases, you can reuse the existing CI types for integration.

You can add a new CI type to Service Manager by using the Visual Mapping tool in UCMDB, which means you don't have to sign out of UCMDB and then sign in to Service Manager.

The following steps illustrate how to create a new CI type called **RDBMS**.

Note

This example is provided only as an illustration of the steps. The best practice is to reuse the existing Service Manager CI type **RunningSoftware** to map with UCMDB CI type **SM RDBMS**.

1. Sign in to UCMDB as an administrator.
2. Open an existing XML mapping file (**SM Computer Push 2.0.xml**, for example) with the Visual Mapping tool editor.
3. Select the Root node in the External Class Model panel, and click the **Add New CI Type to External Class Model** icon.
4. Enter parameter values as follows.

   Name: rdbms

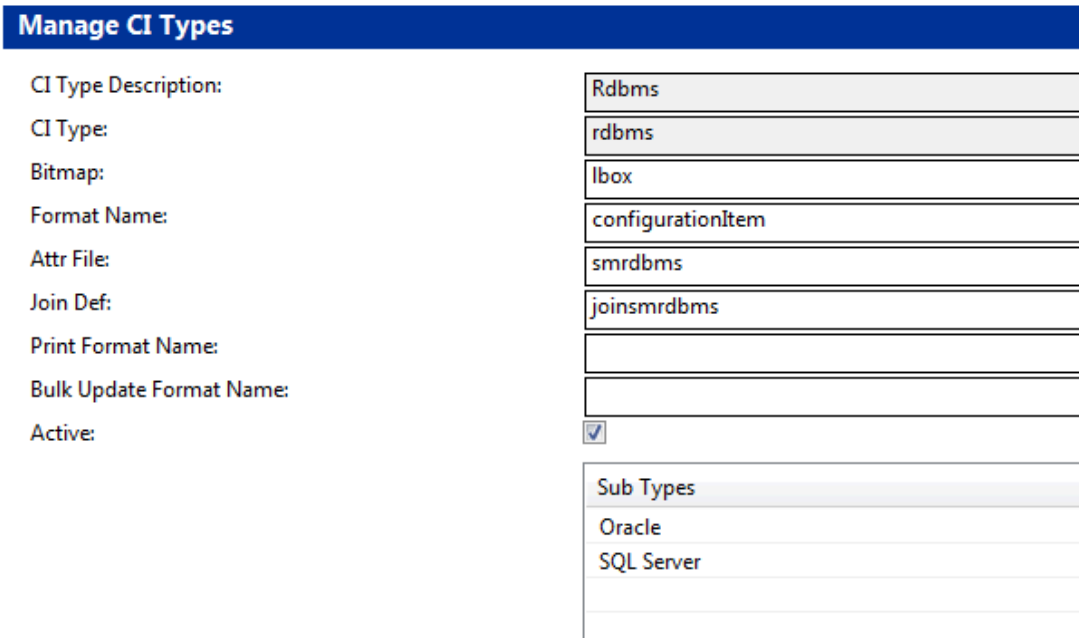
   Description: CI type for RDBMS

   table: smrdbms

   subtype: Oracle, SQL Server



5. Click **OK**.
6. Click **OK** again to confirm the CI type creation. The CI type is automatically created in Service Manager, as shown in the following figure.



What happens on the Service Manager (SM) side?

All relevant SM objects (DBDICT, JoinDef, Web Service API, DEM Rule, and so on) for the new CI type are automatically created on the SM side, except the format. The new CI type will use **configurationItem** as the default format.

If you want to use custom formats in SM for the new CI type, you have to create them manually. You can create the formats in Forms Designer based on existing view forms and bulk update forms. To access Forms Designer in Service Manager, type fd in the command line or navigate to **Tailoring** > **Forms Designer**. For more information about creating forms in Service Manager, see Tailoring best practices.

How to map the CI type's attributes to Web Service fields

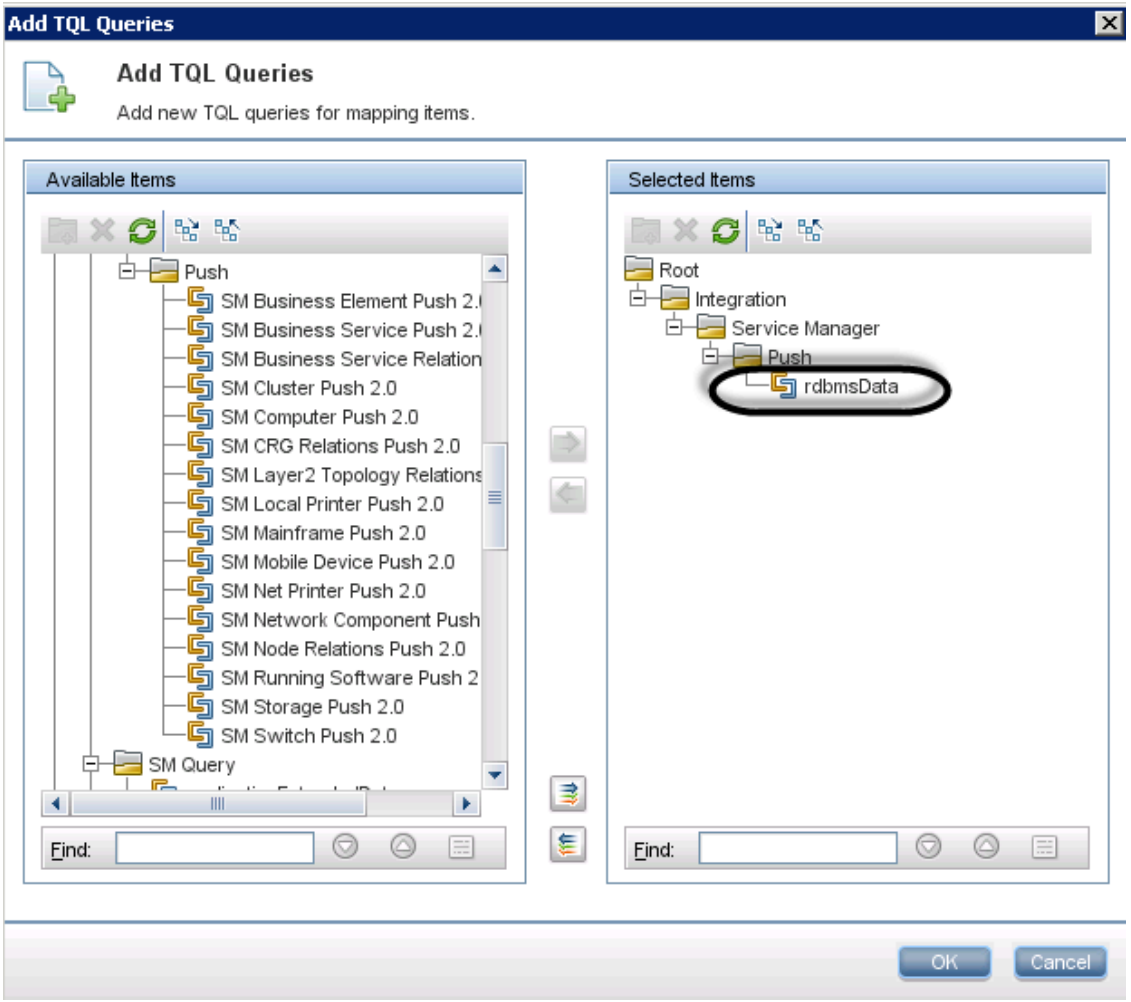
The integration uses the Service Manager Adapter to transform UCMDB CI attributes to web services objects recognized by Service Manager. The Service Manager Adapter uses XML configuration files to convert UCMDB queries into a properly formatted Service Manager web services messages. Out-of-the-box, each integration query has a corresponding XML configuration file. In addition, each attribute you enable for synchronization from advanced layout settings requires its own entry in the XML configuration file.

If you want to add a CI type to the integration, you must create a matching XML configuration file that defines how the Service Manager Adapter transforms each CI type into a Service Manager web service object. See Integration queries for information about the CI types each query manages.

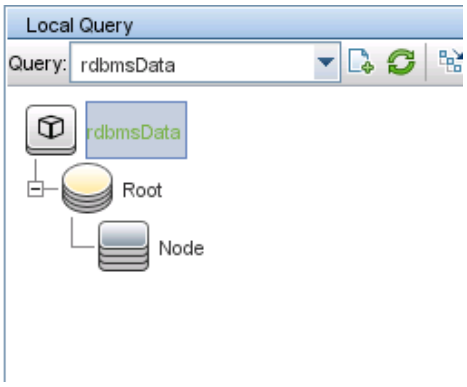
The following steps illustrate how to create an XML configuration file for the rdbmsData query described in previous sections.

To map a CI type's attributes to web service fields:

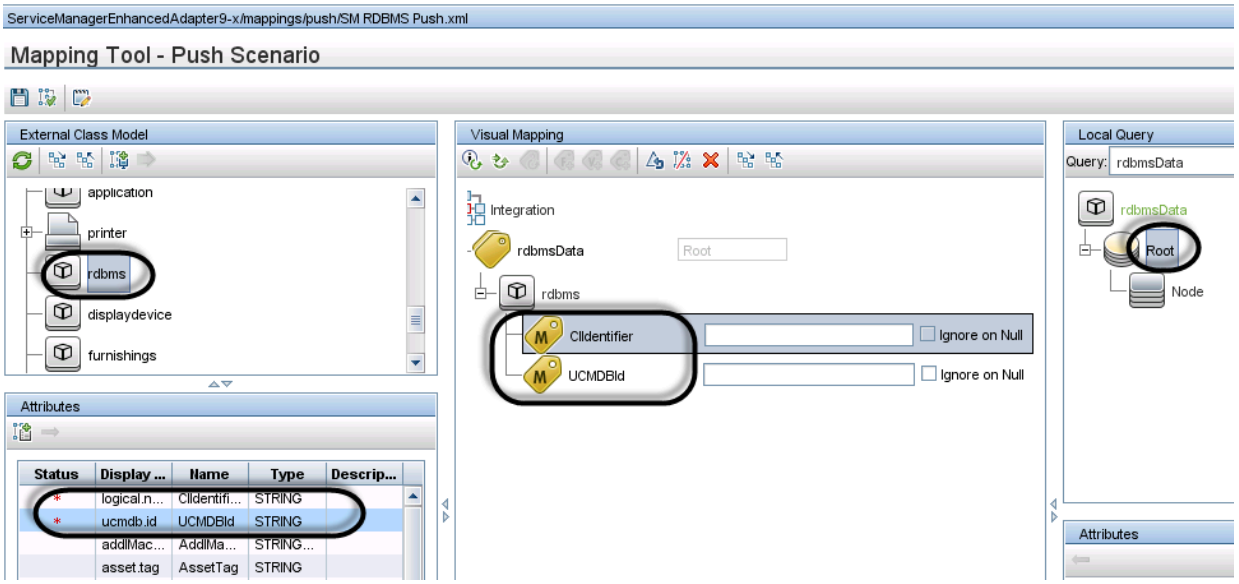
1. Sign in to UCMDB with an administrator account.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41**, and select the adapter.
3. Click the **Create New Resource** icon .
4. Select **New Configuration File**.
5. Enter the full file name: <AdapterID>/mappings/push/<filename>. For example, ServiceManagerEnhancedAdapter9.41/mappings/push/SM RDBMS Push.xml.
6. Click **OK**. The mapping file is created.
7. Double-click the new mapping file to open it with the Visual Mapping tool editor.
8. In the Local Query pane, click the **Add TQL Queries** icon to add the **rdbmsData** query.



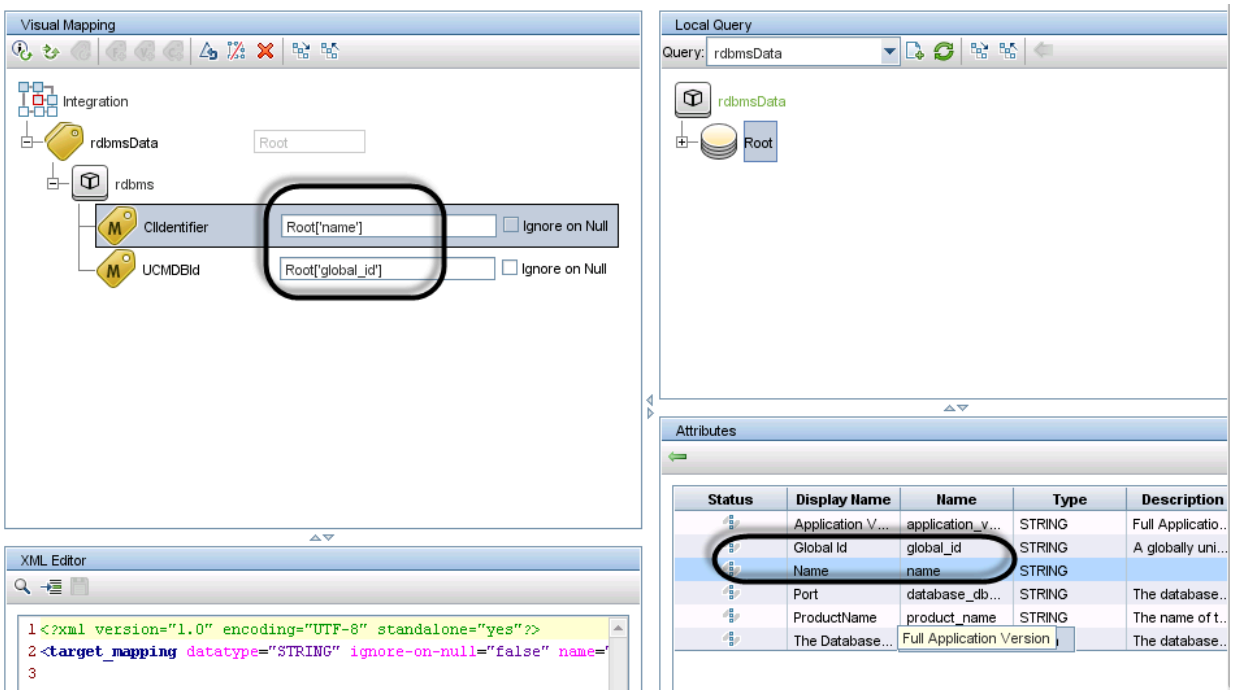
9. Click **OK**.



10. Drag and drop the node “Root” from the Local Query pane into the mapping area.
11. Drag and drop the node “RDBMS” from the External Class Model pane into the mapping area.
12. Drag and drop SM attributes you want into the mapping area.



13. Drag and drop the relevant UCMDB attributes into the mapping area.



14. Save the new mapping file.

> **Note**
>
> When you create or edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

Using Groovy scripts in XML Configuration Files

The Service Manager Enhanced Generic Adapter uses XML and Groovy mapping scripts for four field mapping scenarios: One to One, One to Many, Many to Many, and Value Conversion. Groovy scripts have to be used for complex scenarios. For more information, see Task 7: Update the configurations for custom CI types in UCMDB.

How to add a custom query to an integration job

In order for the integration to synchronize your custom CI types, attributes, and relationships between your Service Manager and UCMDB systems, you must add your custom queries to a data push or population job.

The following steps illustrate how to add a custom query named rdbmsData, which you created previously (see *How to create a query to synchronize the CI type*), to a data push job. The steps for adding a population query to a population job are similar.

To add a custom query to a data push or population job:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**.
3. Click the name of your Service Manager integration point. For example, **sm_integration**.

4. Click the **Data Push** tab.

> **Note**
>
> To add a query to a population job, click the **Population** tab instead.

5. Click the name of the integration job. For example, **SM Configuration Item Push job**.
6. Click the **Edit** icon  .
7. Click the **Add** icon. 
8. Click **Integration** > **Service Manager** > **Push** > **rdbmsData**.

> **Note**
>
> For population, navigate to **Integration** > **Service Manager** > **Population**, and click the population query instead.

9. Click **OK** to add the custom query.
10. Enable the **Allow Deletion** option for the query to allow the integration job to delete removed data.
11. Click **OK** to close the Update Job Definition window.

1.6.2.5.2.8. How to add a CI type, attribute, or relationship type to the integration for population

You can find all out-of-the-box mapping files for Population by navigating to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41** > **ServiceManagerEnhancedAdapter9.41/mappings/population/<*Name of Mapping File*>**.

With the Visual Mapping tool, you can add a CI Type, CI attribute, or relationship to the integration for Population by following similar steps to those for data push. Additionally, you also need to add your population queries to a population job. Refer to the following steps for data push:

How to add a CI attribute to the integration for data push

How to add a CI type to the integration for data push

How to add a CI relationship type to the integration for data push

1.6.2.5.2.9. How to enable or disable UCMDB ID pushback for a CI type

When a new CI is created in UCMDB, a UCMDB ID value is assigned to the CI. When CIs are synchronized from Service Manager (SM) to UCMDB through population, the UCMDB ID Pushback feature can push their UCMDB ID values back to SM. The UCMDB ID field is then used as a flag to indicate if a CI has already been synchronized to UCMDB through population.

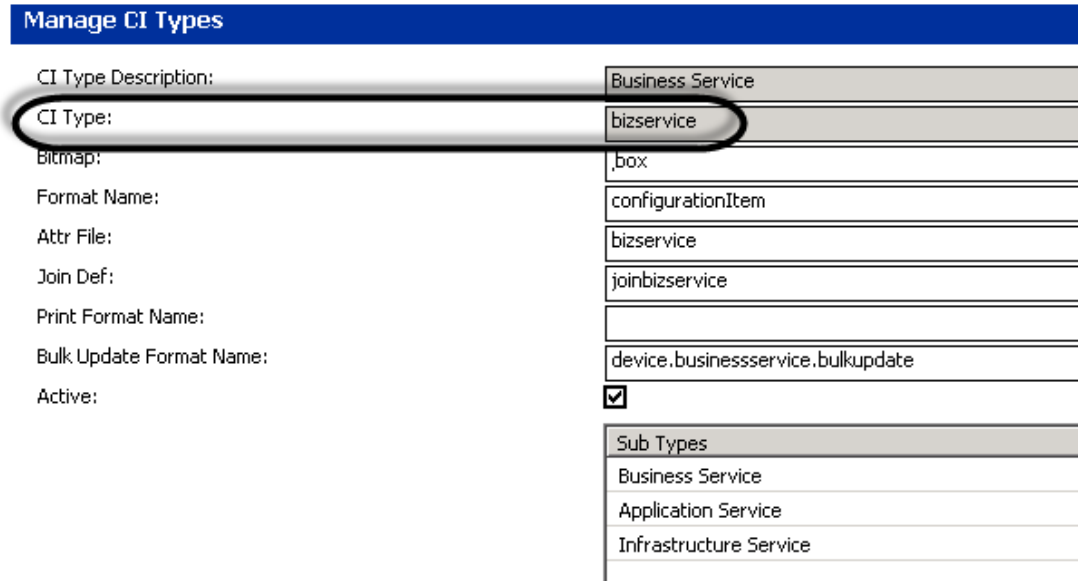
There are also cases when you don't want to enable the UCMDB ID pushback feature. For example, the scheduled_downtime CI type in UCMDB doesn't physically exist in SM. Instead, the integration retrieves Scheduled Downtime information from several entities in SM and then synchronizes the information to scheduled_downtime CIs in UCMDB. For this reason, the UCMDB ID Pushback feature is disabled by default for scheduled_downtime CIs; otherwise a pushback error will occur during population.

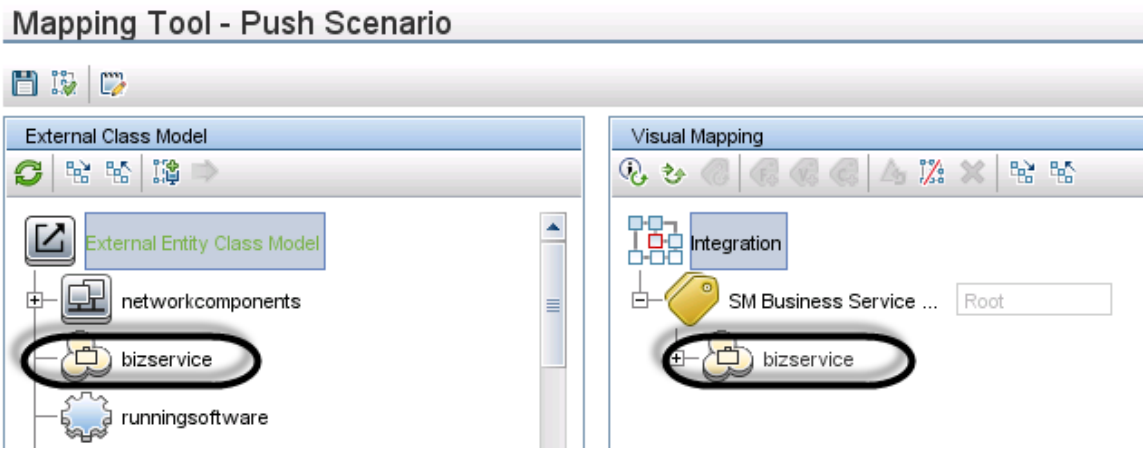
To enable UCMDB ID pushback for a CI type, follow these steps:

1. Add the following entries to your population configuration file for the CI type (for example, **My Business Service Population.xml**).

```
<target_mapping name="sm_id" datatype="STRING"
value="bizservice['CIName']"/>
<target_mapping name="global_id" datatype="STRING"
value="bizservice['UCMDBId']"/>
```

> **Note**
>
> In this example, **bizservice** is the CI Type display name defined in Service Manager and also the external class model name displayed in the UCMDB Visual Mapping interface, as shown in the following figures.





2. Save the XML configuration file.
3. Open the smPopConf.xml file, and make sure that the UCMDB ID Pushback setting for the CI type is either not present or is set to true:

```
<pushback>
   <type name="business_service" enable="true"/>
</pushback>
```

   Where: <type name> is the name of the CI type in UCMDB.

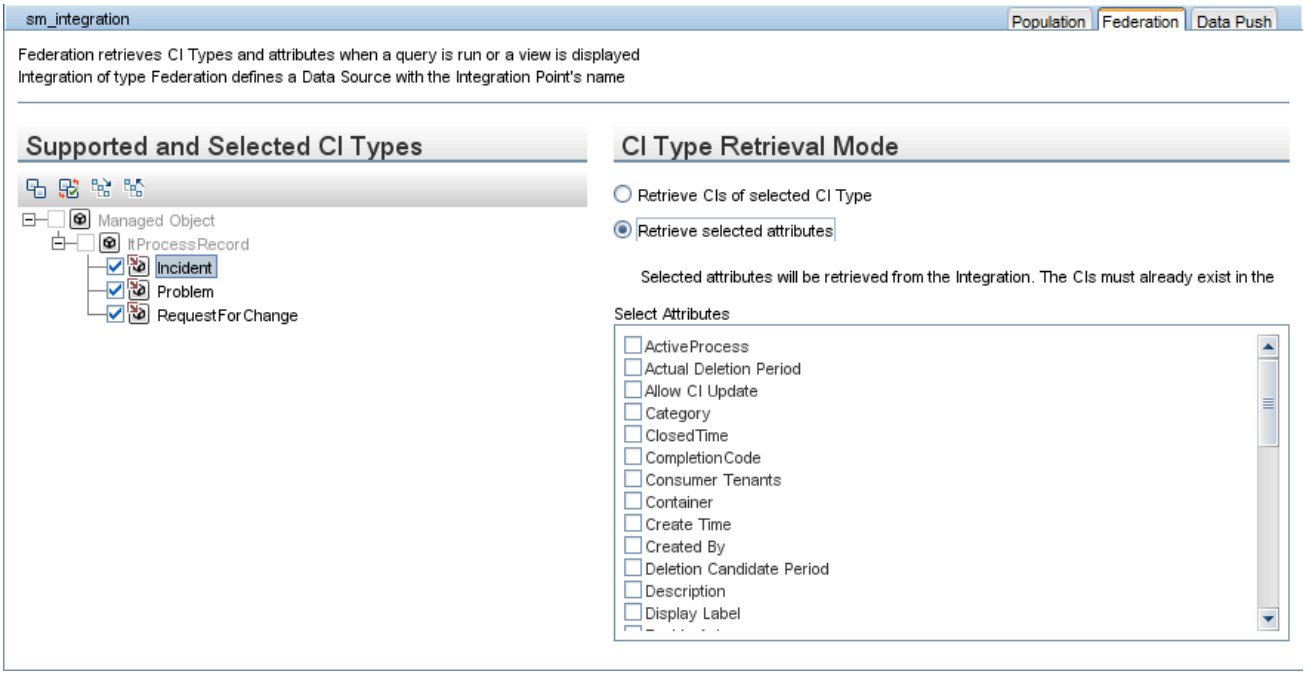
To disable UCMDB ID Pushback feature for a CI type, follow these steps:

1. Open the smPopConf.xml file.
2. In the <pushback> section, insert one line for the CI type:

```
<config>
<pushback>
<type name="scheduled_downtime" enable="false"/>
<type name="business_service" enable="false"/>
</pushback>
<mapping>
<tql name="SM Business Service Population 2.0">
...
</tql>
…
</mapping>
</config>
```

1.6.2.5.2.10. How to add an attribute of a supported CI type for federation

Out-of-the-box, the integration supports federation for three external CI types in UCMDB: **Incident**, **Problem**, and **RequestForChange**. For each of the supported CI types, there is a list of attributes in UCMDB that you can map to Service Manager web service objects for federation. The following figure shows the out-of-the-box UCMDB CI attributes available for the Incident CI type.



For example, to add an SM Incident attribute for federation, you need to expose the field in the SM **UcmdbIncident** web service object and then map it to an appropriate UCMDB attribute. If one doesn't already exist, you need to create it in UCMDB first.

The following figure shows the fields that are exposed in the `UcmdbIncident` web service object in Service Manager.

External Access Definition

Service Name: ucmdbIntegration — Released

Name: probsummary — Deprecated

Object Name: UcmdbIncident

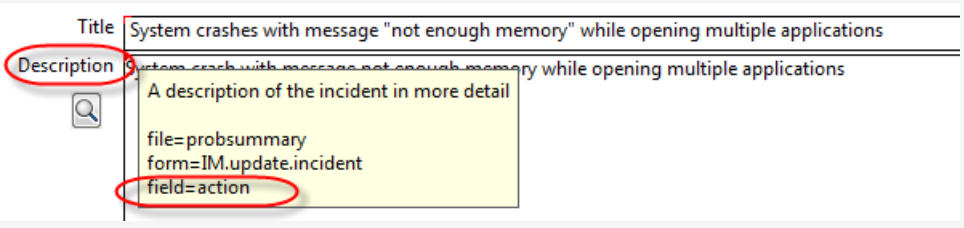
Allowed Actions | Expressions | Fields

Field	Caption	Type
logical.name	ConfigurationItem	
number	IncidentID	
brief.description	BriefDescription	
subcategory	Category	
severity	Urgency	
open.time	OpenTime	DateTimeType
update.time	UpdatedTime	DateTimeType
close.time	ClosedTime	DateTimeType
problem.status	IMTicketStatus	
affected.item	Service	
priority.code	PriorityCode	
initial.impact	ImpactScope	

You can expose more fields to federate more Incident attributes to UCMDB. As an example, the following describes how to add the **action** field in the Service Manager `probsummary` (Incident) file for federation, by mapping it to a new UCMDB attribute named `details`.

Note

On the Incident form in Service Manager, the **action** field has a **Description** label. SM uses this field to describe the Incident record in more detail. See the following figure as an example.



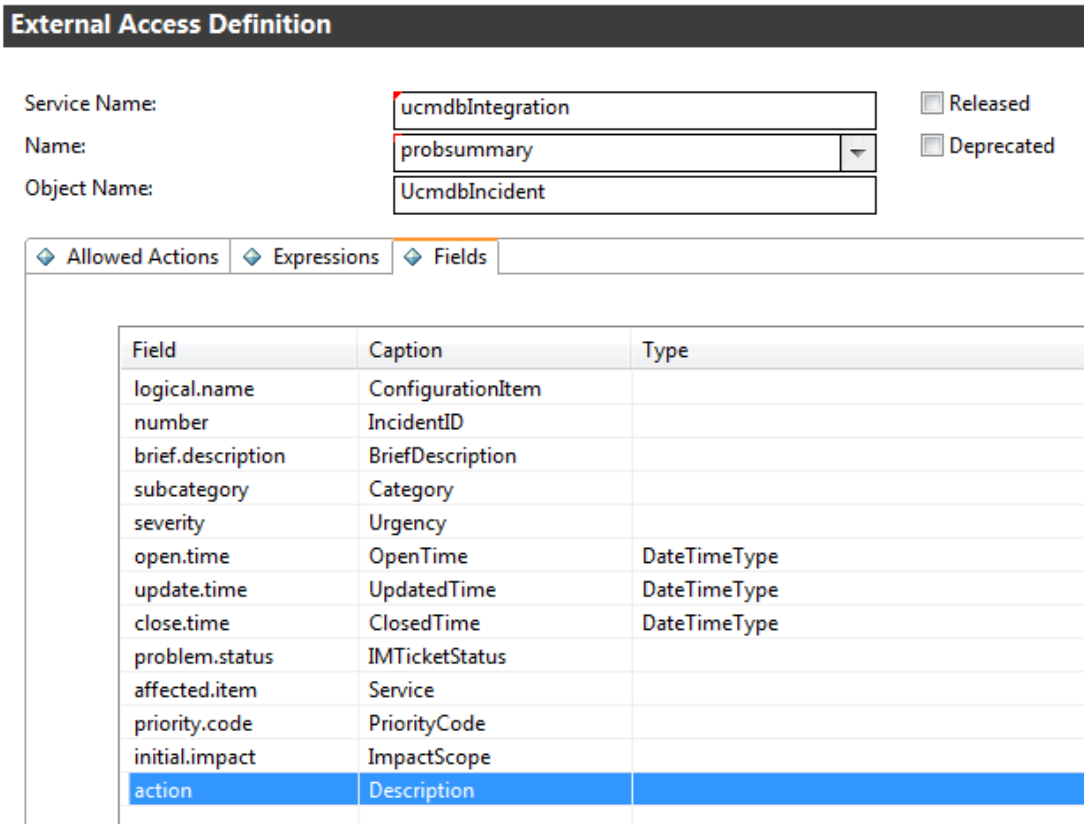
To add an attribute of a supported CI type for federation:

1. **Add the SM attribute to its web service object.**

   The following example describes how to expose the SM **action** field of Incident in **UCMDBIncident** Web Service object.

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Web Services** > **Web Service Configuration**.
3. Enter the following field values, and then click **Search**.
   - Service Name: `ucmdbIntegration`
   - Name: `probsummary`

   Service Manager displays the `UCMDBIncident` web service object.
4. On the **Fields** tab, add the following row:
   - Field: action
   - Caption: Description



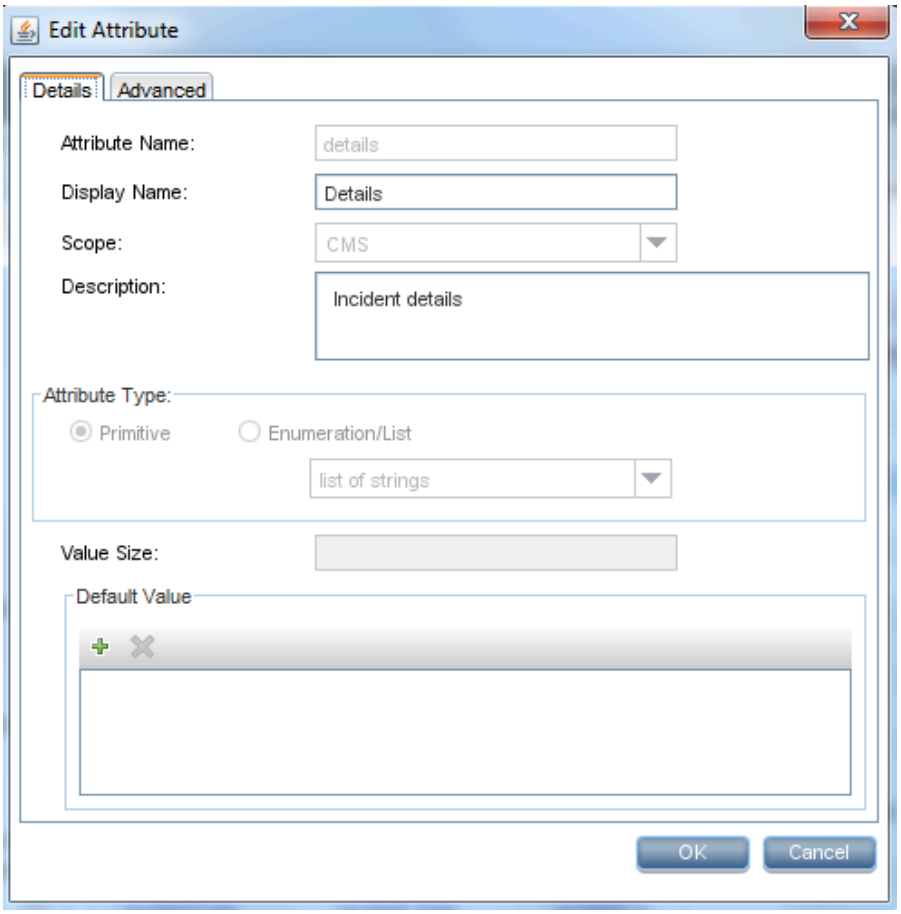
5. Save the web service object.

2. **Map the SM attribute to a UCMDB attribute.**

   The following example describes how to map the SM **action** attribute to a new UCMDB attribute named **details**.

   1. Sign in to UCMDB as an administrator.
   2. Navigate to **Modeling** > **CI Type Manager**.
   3. Navigate to **ItProcessRecord** > **Incident**, and open its properties pane.
   4. Click the **Add** icon to add a new attribute named **details** to the Incident CI type.

- name: details
- Display Name: Details
- Description: Incident details
- Attribute Type: select **Primitive** > **String**
- Value Size: 500



5. Save the Incident CI type record.
6. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerEnhancedAdapter9.41** > **Configuration Files**.
7. Edit the relevant federation mapping file (`ServiceManagerEnhancedAdapter9.41/mappings/federation/SM Incident 2.0.xml`) to add a mapping entry for the new attribute.

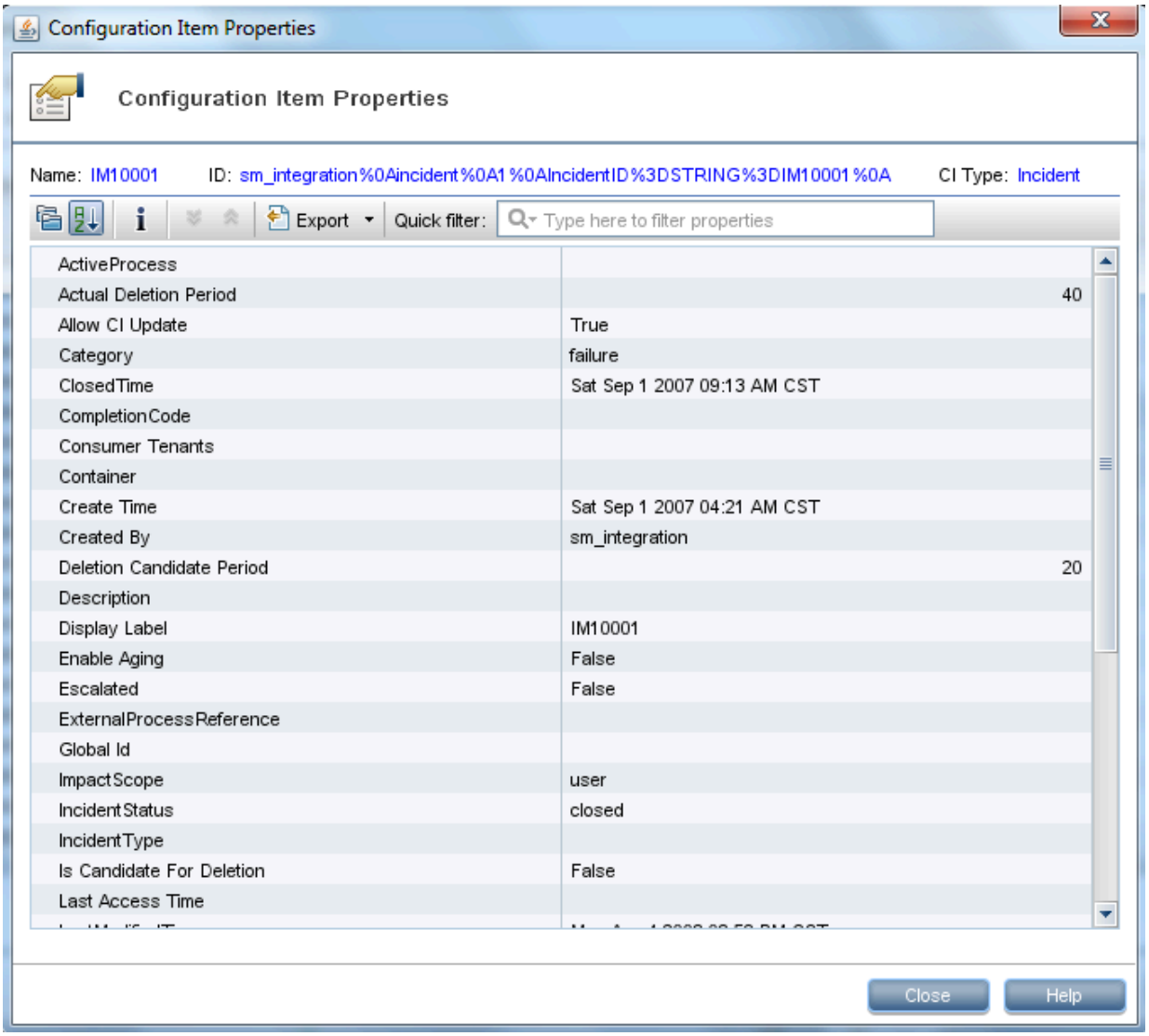
```
<target_entities>
<source_instance query-name="SM Incident 2.0" root-element-
name="ucmdbIncident">
<target_entity name="incident">
<target_mapping name="reference_number"     datatype="STRING"
value="ucmdbIncident['IncidentID']"/>
<target_mapping name="name"                 datatype="STRING"
value="ucmdbIncident['BriefDescription']"/>
<target_mapping name="priority"             datatype="STRING"
value="SMFederationFunctions.getEnumValue(ucmdbIncident['Priori
tyCode'],'Priority')"/>
<target_mapping name="incident_status"
datatype="STRING"
value="SMFederationFunctions.firstLetterToLowerAndReplaceSpaceW
ithUnderscore(ucmdbIncident['IMTicketStatus'])"/>
<target_mapping name="category"             datatype="STRING"
value="SMFederationFunctions.replaceSpaceWithUnderscore(ucmdbIn
cident['Category'])"/>
<target_mapping name="closed_time"            datatype="DATE"
value="SMFederationFunctions.convertDate(ucmdbIncident['ClosedT
ime'])"/>
<target_mapping name="create_time"            datatype="DATE"
value="SMFederationFunctions.convertDate(ucmdbIncident['OpenTim
e'])"/>
<target_mapping name="last_modified_time"     datatype="DATE"
value="SMFederationFunctions.convertDate(ucmdbIncident['Updated
Time'])"/>
<target_mapping name="impact_scope"         datatype="STRING"
value="SMFederationFunctions.getEnumValue(ucmdbIncident['Impact
Scope'],'ImpactScope')"/>
<target_mapping name="urgency"
datatype="STRING"
value="SMFederationFunctions.getEnumValue(ucmdbIncident['Urgenc
y'],'Urgency')"/>
<target_mapping name="details" datatype="STRING"
value="ucmdbIncident['Description']"/>
</target_entity>
</source_instance>
</target_entities>
```

8. Click **Save** to save the file.

Now you have added the Description (field name: action) attribute of SM Incident to the integration for federation. You can run an Incident federation query in the UCMDB

Modeling Studio to see if the SM Description data is normally federated. For information about how to run a federation query, see Examples of Using Federation.

The following figure shows an example where SM has federated the Description of an Incident record to UCMDB as **Details**.



1.6.2.5.2.11. How to enable the automatic deletion of CI relationships from UCMDB

This task explains how to configure an SM-UCMDB integration so that CI relationships that are deleted from Service Manager are automatically removed from UCMDB after you next run a population job. This functionality requires configuration of the Service Manager Enhanced Adapter in UCMDB, and also of Service Manager if you are using Service Manager 9.60 or an earlier version.

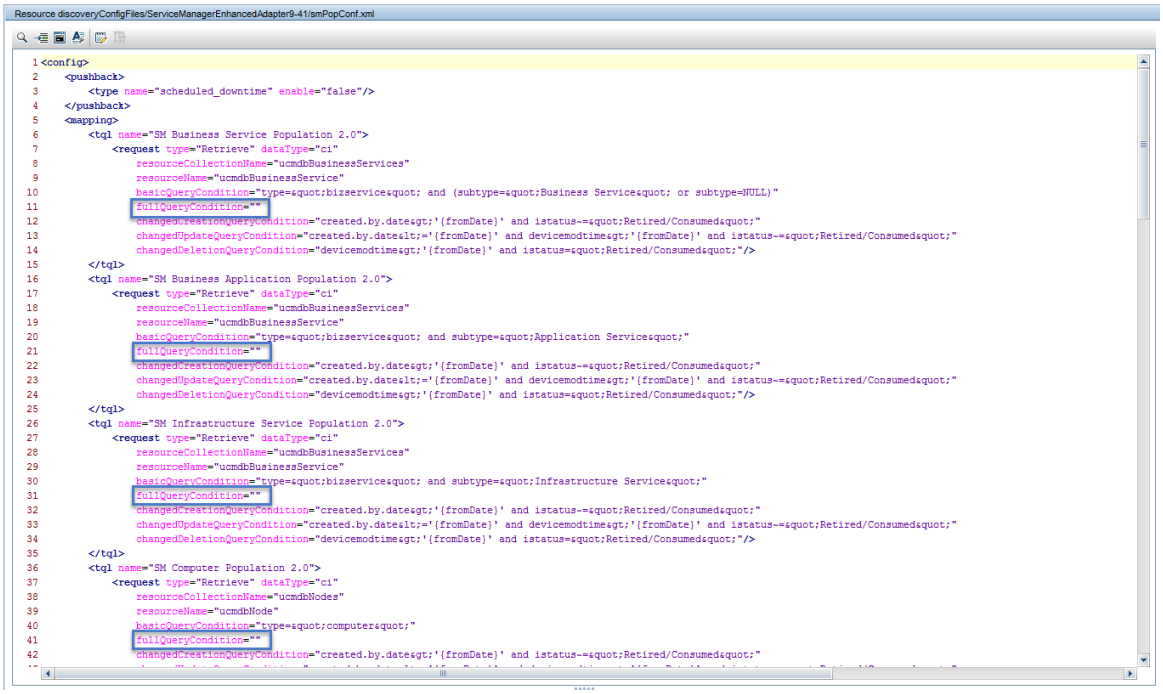
Configure the Service Manager enhanced adapter



Note

Before you begin, we recommend that you back up your existing adapter. To do this, open the Admin UI, go to **Administrator** > **Package Manager**, right-click **ServiceManagerEnhancedAdapter9-41**, and then select **Export Package**.

1. Stop the SM-UCMDB integration.
2. Turn on the switch that enables CI relationship deletion.
    1. In the Admin UI, go to **Data Flow Management** > **Adapter Management**, expand **Service Manager Enhanced Adapter 9-41**, expand **Configuration Files**, and then double-click **Service Manager Enhanced Adapter 9-41/sm.properties**.
    2. Change the value of the **pop.only.use.full.pop.switch** field to **true** (Default: **false**), and then click **Save**.
3. Modify the **smPopConf.xml** file.
    1. Go to **Data Flow Management** > **Adapter Management**, expand **Service Manager Enhanced Adapter 9-41**, expand **Configuration Files**, and then double-click **Service Manager Enhanced Adapter 9-41/smPopConf.xml**.
    2. Set the value of the **fullQueryCondition** field in every `<tql></tql>` section to `""`, with the exception of the following two sections:
        - "SM CLIP Down Time Population 2.0"
        - "SM CI Connection Down Time CI 2.0"



3. Click **Save**.
4. Restart the integration.

Configure Service Manager

> **Important**
>
> Perform these steps only if you are using Service Manager 9.60 or an earlier version.

1. In Service Manager, go to **Tailoring** > **Web Services** > **Web Service Configuration**, and then search for "ucmdbIntegration" in the **Service Name** field.
2. Select **ucmdbBusinessService** in the **Object Name** list.
3. In the **External Access Definition** pane, select the **Fields** tab, add a new field called "istatus", set the new field's caption to "AssetStatus", and then click **Save**.
4. Select **ucmdbRunningSoftware** in the **Object Name** list.
5. In the **External Access Definition** pane, select the **Fields** tab, add a new field called "istatus", set the new field's caption to "AssetStatus", and then click **Save**.

Troubleshooting

If CI relationships aren't removed from UCMDB after you have configured UCMDB and Service Manager, run the UCMDBPopulationHelper script check the consistency of the data in the cirelationship1to1 table against the data in the CI device table.

The script checks the endpoint CIs of the relationship record from the cirelationship1to1 table, and will remove the relationships if the endpoint CIs were already retired on a set date.

Important

The UCMDBPopulationHelper script is not included in Service Manager 9.60 or in earlier versions. If you are running one of these versions, you must add it manually. to do this, follow these steps:

1. Open the Service Manager ScriptLibrary. To do this, type "sl" in the command box, and then press Enter.
2. In the **Name** field, type "UCMDBPopulationHelper".
3. In the **Package** field, select **Integrations**.
4. Copy the following code into the input field:

```
function deleteRelation1to1ByDevice(startDate) {
        var deviceSql = 'istatus="Retired/Consumed"';
        if(startDate) {
        deviceSql += ' and sysmodtime
>=\''+startDate+'\'';
        }

        lib.c.$('device',
SCFILE_READONLY).setFields('logical.name').select(deviceSq
l).
iterate(function(d){
        var logicalName =
lib.StringUtil.escapeAllStr(d.logical_name)
        var sql = '(upstreamci.logical.name="' +
logicalName + '" or
downstreamci.logical.name="'+logicalName+'") and
status="In Used"';

lib.c.$('cirelationship1to1').select(sql).iterate(function
(item){
        item.doDelete();
        });
        });
}
deleteRelation1to1ByDevice('01/07/10 22:15:37');
```

Note

Replace the date in the last line of this script with the date for which you are having issues.

5. Select **Add**.

To run the UCMDBPopulationHelper script, follow these steps:

1. Open the Service Manager ScriptLibrary. To do this, type "sl" in the command box, and then press Enter.
2. Type "UCMDBPopulationHelper" in the **Name** field, and then press Enter.
3. Select **Compile**, and then select **Execute**.

1.6.3. Using the ServiceManagerAdapter9.41 adapter

This section describes how to implement an integration between the Service Manager and UCMDB by using the ServiceManagerAdapter9.41 adapter.

How to upgrade yourintegration

Integration setup

Multi-tenancy (Multi-company) setup

Standards and best practices

Tailoring the integration

1.6.3.1. How to upgrade your integration

The logical name solution contains significant CI related code changes. For example, prior to version 9.41, Service Manager used the **logical.name** field in the **device** table to store CI names. The logical.name field must be unique in the Service Manager system, but UCMDB allows duplicate CI names. As a result, CIs may have to be renamed when they are pushed from UCMDB to Service Manager. Starting with version 9.41, Service Manager stores CI names in the **display.name** field in the **device** table, and stores unique CI identification numbers in the **logical.name** field. The **display.name** field allows duplicate values if they are pushed from UCMDB, and requires unique values for CIs that are created from Service Manager (unless they're not of the same CI type).

Before you proceed to the upgrade steps described in this section, make sure you have a good understanding of the logical name solution and its impact on the SM-UCMDB integration. For details, see the Service Manager Logical Name Solution white paper.

Perform the following steps to upgrade your integration to use the ServiceManagerAdapter9.41 adapter.

Task 1. Close all unplanned Changes and Incidents related to CI modifications in Service Manager

Before you upgrade to Service Manager (SM) 9.41 or later, close all unplanned change and incident records that were created due to CI modifications. After upgrading to version 9.41 or later, you will run a full data push to resynchronize CI names from UCMDB to the **display.name** field in the Service Manager **device** table.

Task 2. Upgrade Service Manager

If you want to use the ServiceManagerAdapter9.41 adapter for the integration, you must upgrade Service Manager to version 9.41 or later, which uses display.name instead of logical.name for field mapping between UCMDB and Service Manager.



Note

The upgrade includes the SM server, Windows and web clients, applications, and Mobility client. If you upgrade only the server and clients and your applications version is still earlier than 9.41, you can only use an old adapter rather than any of the Service Manager 9.41 adapters.

Task 3. Copy data to the joinnode table

Starting with version 9.32, Service Manager uses the **joinnode** table to replace the **mainframe**, **networkcomponents**, **storage**, and **computer** tables. If you are upgrading from Service Manager 9.31 or earlier and want to migrate to the joinnode-based data mappings, you need to copy data to the **joinnode** table.

> **Note**
>
> If you want to use the original tables instead of joinnode, this task is not needed. Instead, you need to modify the data mappings of your Service Manager adapter in UCMDB.

To copy your data to the joinnode table, follow these steps:

1. Sign in to Service Manager as a system administrator.
2. Open Database Manager
3. Enter cidatacopy in the Table field, and then click **Search**. The Copy CI Type Data form opens.
4. Click **Search**. A list of records is displayed: **joincomputer**, **joinmainframe**, **joinnetworkcomponents**, and **joinstorage**.
5. Select each record from the list, update or add source/target fields as needed, and click the **Copy Data** button. One of the following messages is displayed:
   - A message that resembles `<XXX> records were successfully copied, and <YYY> records were ignored within xxx ms`, where XXX and YYY represent the numbers of records copied and ignored, respectively.
   - `Configuration validation failed, and no records were copied. Please check your Source Fields/Target Fields settings and run Copy Data again.`
6. If the error message is displayed, correct your Source Fields/Target Fields settings, and click **Copy Data** again until all of your records have been successfully processed.

Task 4. Update the DEM rules in Service Manager

Service Manager 9.41 or later uses an updated CI form, so you may want to correct the CI names in the logical.name field that have been renamed, and update the CI Display Name (display.name) field values in Service Manager by using the new field mapping.

To do this, you need to run a full data push to resynchronize the CIs to Service Manager. Before doing so, you must update the DEM rules to prevent unwanted Change or Incident records from being created due to the CI name changes.

1. Sign in to Service Manager as a System Administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Click **Search**.
4. For each DEM rule record, select **Log Results and update record** for **Action if record exists but unexpected data discovered**.

Task 5. Upgrade UCMDB

Service Manager 9.41 can work with the following XSLT-based adapters and generic adapters specifically designed for Service Manager 9.41:

- ServiceManagerAdapter9.41 for UCMDB 10.01, 10.11, 10.20, or 10.21

> **Note**
>
> These are four XSLT-based adapters, one for each of the UCMDB versions.

- ServiceManagerEnhancedAdapter9.41 for UCMDB 10.20 or 10.21

> **Note**
>
> These are two generic adapters based on the UCMDB generic framework, one for each of UCMDB 10.20 and 10.21.

For more information, see Integration Requirements.

Task 6. Upgrade field mapping

The out-of-box mapping files in the Service Manager 9.41 adapters are already updated to align with the logical name solution. You need to make similar updates to your own field mapping.

> **Note**
>
> - In Service Manager 9.41 or later, the original CI Name field (id in the device table) is removed from the Configuration Item detail form in Service Manager.
> - No updates are need for CI Relationship push mapping files or federation mapping files.

Perform the following steps to upgrade your own field mapping.

Step 1. Upgrade your data push field mapping

To update your CI push mapping files, follow these steps:

1. Switch the mapping field from CIIdentifier to DisplayName.

   For example, update `<CIIdentifier><xsl:value-of select="."/></CIIdentifier>` to:

   `<DisplayName><xsl:value-of select="."/></DisplayName>`

2. (Optional) Simplify the business logic for DisplayName.

   If you have customized the original CIIdentifier value to append more fields so that it is unique and not renamed by Service Manager, you can remove such business logic. This is because the DisplayName field in Service Manager now supports duplicate values.

   The following is an example mapping for the out-of-box Cluster CI Push.

```
 <CIIdentifier>
<xsl:choose>
<xsl:when test="ip_addresss/ip_address/@authoritative_dns_name">
<xsl:for-each select="ip_addresss">
<xsl:for-each select="ip_address">
<xsl:sort select="@authoritative_dns_name"/><xsl:value-of
select="@authoritative_dns_name"/>_</xsl:for-each>
</xsl:for-each>
</xsl:when>
<xsl:otherwise>
<xsl:for-each select="ip_addresss">
<xsl:for-each select="ip_address">
<xsl:sort select="@name"/><xsl:value-of select="@name"/>_</xsl:for-
each>
</xsl:for-each>
</xsl:otherwise>
</xsl:choose>
</xsl:variable>
<xsl:value-of select="$combinIP"/><xsl:value-of
select="@display_label"/>
</xsl:when>
<xsl:otherwise><xsl:value-of select="@display_label"/>
</xsl:otherwise>
</xsl:choose>
</CIIdentifier>
```

You can simplify the example above as shown in the following:

```
<DisplayName>
<xsl:choose>
<xsl:when test="clusters/cluster/@display_label">
<xsl:value-of select="clusters/cluster/@display_label"/>
</xsl:when>
<xsl:otherwise><xsl:value-of select="@display_label"/>
</xsl:otherwise>
</xsl:choose>
</DisplayName>
```

Step 2. Upgrade your population field mapping

For population, in the field mapping files you need to make the following changes:

1. Map the "name" attribute to "DisplayName":

```
<attribute name="name" type="String"><xsl:value-of select="DisplayName"/>
</attribute>
```

2. Map the "sm_id" attribute to "CIIdentifier":

```
<attribute name="sm_id" type="String"><xsl:value-of select="CIIdentifier"/>
</attribute>
```

3. Add the following line to map the "global.id" attribute to "UCMDBId":

```
<attribute name="global_id" type="String"><xsl:value-of select="UCMDBId"/>
</attribute>
```

Step 3. Configure the UCMDB JMX console, as described in the following steps.

Note

This feature enables the UCMDB global ID solution for Service Manager. It is available only in UCMDB 10.21 or later. For this reason, a known issue (QCCR1E122493) exists in UCMDB versions earlier than 10.21: If two CIs with the same display name are pushed to Service Manager and then updated in Service Manager with a new display name, when you run a population job to synchronize the CIs back to UCMDB, the two CIs are merged in UCMDB and therefore have the same ucmdb.id value in Service Manager.

1. Go to the JMX Console: http://<UCMDB server name>:<port>/jmx-console/.
2. Enter **UCMDB:service=Settings Services** in the search box, and then press Enter.
3. Click **setSettingValue** to go to this section.
4. In the **customerID** field, enter your customer ID.
5. In the **name** field, enter **reconciliation.match.attributes**.
6. In the **value** field, enter **global_id**.
7. Click **Invoke**.

   If the configuration is successfully completed, the following message is displayed:

```
Setting [reconciliation.match.attributes] value changed successfully to [global_id]
```

Task 7. Reconfigure an integration point

You need to set up an integration point to use the ServiceManagerAdapter9.41 adapter, and activate the integration point. For details, see How to create an integration point in UCMDB.

Task 8. Run a full push to resynchronize CI names

Due to the logical name solution that was introduced in Service Manager 9.41, once you have upgraded to Service Manager 9.41 or later, you must run a full data push to resynchronize CI names to the CI display name field in Service Manager. When performing this step, you need to use the new integration point that you reconfigured.

To resynchronize CIs, follow these steps:

1. Run a Full Push in UCMDB.
2. Check the CI data in Service Manager. Perform a search to find the CIs. Verify that the Display Name field values of the CIs have been updated correctly.

> **Note**
>
> The display names of Retired CIs aren't updated, because they are already removed from UCMDB.

Task 9. Restore the DEM rules in Service Manager

After you have run a full data push to resynchronize CI names to Service Manager, you must restore the DEM rules to their original state so that they can function as usual.

To restore the DEM rules, follow these steps:

1. Sign in to Service Manager as a System Administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Click **Search**.
4. In each DEM rule, for **Action if record exists but unexpected data discovered**, restore the setting to the original value (for example, **Open a Change** or **Open an Incident**).

1.6.3.2. Integration setup

Before implementing the integration in your production environment, you can set up the integration in a test environment using the out-of-the-box integration configurations. This section describes the basic integration setup tasks without any tailoring or multi-tenancy configurations.

Integration requirements

Integration setup overview

Service Manager setup

How to create an integration user account

How to add UCMDB connection information

Universal CMDB setup

How to create an integration point in UCMDB

How to update the time zone and date format for the integration adapter

Populating UCMDB with Service Manager CI data

How to define population jobs in UCMDB

View Service Manager CI data in UCMDB

How to schedule CI population jobs

Pushing UCMDB CI data to Service Manager

How to define data push jobs in UCMDB

How to schedule data push jobs

How to view UCMDB CI data in Service Manager

How to view the Change History of the primary CI of a Problem record

Federating SM ticket data to UCMDB

Federation queries

Examples of using federation

Tip

Before you proceed to implementing the integration in your production environment, you can refer to the following sections for further information:

- Multi-tenancy (Multi-company) setup, which describes how you set up the integration in multi-tenancy mode.
- Standards and best practices, which describes best practices for implementing the integration and also provides Frequently-Asked-Questions information.
- Tailoring the integration, which describes how you can tailor the integration to better suit your business needs.
- Troubleshoot UCMDB integration, which provides information on troubleshooting data push and population issues.

1.6.3.2.1. Integration requirements

This section describes the integration requirements when using the ServiceManagerAdapter9.41 adapter.

Supported product versions

The supported product versions for using this adapter are listed in the following table.



Important

If using UCMDB 11.0, CP 26 is required to integrate UCMDB with Service Manager or other applications.

Product	Versions
Service Manager	9.41 or later
UCMDB	10.20 or later
UCMDB UI	Compatible versions for UCMDB

To support the embedding of UCMDB UI's CI Visualization and Impact Visualization widgets in Service Manager, the following product versions are required:

- Service Manager 9.50 or later
- UCMDB UI 4.10 or later

Setting up required components

You must set up the following required components to establish an integration between UCMDB and Service Manager.

- Universal CMDB installation

  Add a UCMDB Probe for the population feature if you don't already have one.

- Service Manager installation

  Add the UCMDB URL in SMIS configuration. See How to add UCMDB connection information.

- Network connection between the Universal CMDB and Service Manager systems.

Deploying an appropriate adapter

To use the ServiceManagerAdapter9.41 adapter, all of your Service Manager server, clients, and applications must be version 9.41 (or later). You need to verify your Service Manager applications version and then find the correct adapter.

To do this, follow these steps:

1. Sign in to Service Manager as a system administrator.
2. Enter db in the command line, and press Enter.
3. Enter scversion in the Table field, and then click **Search**.
4. Click **Search** again.
5. Check the Application Version value that's displayed.
6. If your applications version is earlier than 9.41, use the ServiceManagerAdapter9.41 adapter.
7. If you are running Service Manager applications 9.41 or later, use the ServiceManagerEnhancedAdapter9.41 adapter.

Configure one UCMDB as the global ID generator



Tip

- If UCMDB acts as the global ID generator, the newly-created CI on this UCMDB will generate a valid global_id attribute.
- If UCMDB doesn't act as the global ID generator, make sure that the CI to be pushed has a valid global_id attribute. Otherwise, the push to the remote Service Manger may fail.
- For the UCMDB cluster, it is required that only one of the UCMDBs acts as the global ID generator, and the rest UCMDBs will synchronize the global ID within this cluster. If there are two or more UCMDBs act as the global ID generators, potential duplicated global IDs may cause identification problems and synchronization failure.

Follow these steps:

1. Launch the Web browser and enter the following address:

   http://<RTSM server>:21212/jmx-console

   Use the user name sysadmin and the password you set for the administrator during the installation.

2. Click **UCMDB:service=Multiple CMDB Instances Services** to open the JMX MBEAN View page.

3. Click one of the following methods and enter values as required:

Method	Description
setAsGlobalIdGenerator	Specifies that the CMDB will act as the global ID generator for all locally existing scopes
setAsGlobalIdGeneratorForScopes	Specifies the scopes for which global IDs will be generated
setAsNonGlobalIdGenerator	Stops the CMDB from acting as the global ID generator for all scopes

4. Click **Invoke**.



Note

If you want to check which scopes are currently set or which UCMDB/RTSM instance is acting as the CMS, use the **getGlobalIdGeneratorScopes** method.

1.6.3.2.2. Integration setup overview

The integration requires setup on both the UCMDB and Service Manager systems.

This task includes the following steps:

1. Set up the Service Manager system.

   See Service Manager setup.

2. Set up the UCMDB system.

   See Universal CMDB setup.

3. Run the UCMDB population jobs to synchronize CIs to UCMDB.

   See Populating UCMDB with Service Manager CI data.

4. Run the UCMDB data push jobs to transfer CIs to Service Manager.

   See Populating UCMDB with Service Manager CI data.

1.6.3.2.3. Service Manager setup

You must complete the following tasks from your Service Manager system to support the integration.

1. Create a dedicated integration user account in Service Manager.

   See How to create an integration user account.

2. Add the UCMDB connection information to the system information record.

   See How to add UCMDB connection information.

1.6.3.2.3.1. How to create an integration user account

This integration requires an administrator user account for UCMDB to connect to Service Manager. This user account must already exist in both UCMDB and Service Manager.

To create a dedicated integration user account in Service Manager:

1. Sign in to Service Manager as a system administrator.
2. Type contacts in the Service Manager command line, and press **ENTER**.
3. Create a new contact record for the integration user account.
    1. In the Full Name field, type a full name. For example, `UCMDBIntUser`.
    2. In the Contact Name field, type a name. For example, `UCMDBIntUser`.
    3. Click **Add**, and then **OK**.
4. Type `operator` in the Service Manager command line, and press **ENTER**.
5. In the Login Name field, type the username of an existing system administrator account, and click **Search**.

    Service Manager displays the system administrator account.
6. Create a new user account based on the existing one.
    1. Change the Login Name to the integration account name you want (for example, **ucmdb**).
    2. Type a Full Name. For example, UCMDB.
    3. In the Contact ID field, click the **Fill** button and select the contact record you have just created.
    4. Click **Add**.
    5. Select the **Security** tab, and change the password.
    6. Click **OK**.

Service Manager creates the integration user account. Later you will need to add this user account in UCMDB, and then specify this user account in the Credentials ID field when creating an integration point in UCMDB. See How to create an integration point in UCMDB.

1.6.3.2.3.2. How to add UCMDB connection information

UCMDB-SM integration requires the UCMDB connection information to obtain CI attribute information from the UCMDB system, and display it in the Actual State section in the Service Manager configuration item form; it also requires UCMDB Browser or UCMDB Gateway connection information to retrieve data from UCMDB Browser or UCMDB Gateway and display the data to Service Manager users.

Caution

If you don't specify the correct connection information, an error, instead of correct CI related data, is displayed to Service Manager users.

This task includes the following steps:

1. Sign in to Service Manager as a system administrator.
2. Click **Tailoring** > **Integration Manager**.
3. Click **Add**.
4. In the **Integration Template** field, select **SMtoUCMDB**.

   Some fields in the form are automatically populated with a value. The empty fields can be ignored.
5. (Optional) Update the **Name**, **Log Level**, and **Description** fields as needed.
6. On the **UCMDB Server Configuration** tab, provide a value for each field in the following table.

   Important

   When specifying UCMDB URLs, use the fully qualified domain name (FQDN) rather than the IP address of the server.

Field	Description
UCMDB webservice URL	Enter the URL to the Universal CMDB web service API. The URL has the following format: https://*<UCMDB server name>*:*<port>*/axis2/services/ucmdbSMService Replace *<UCMDB server name>* with the fully-qualified domain name (FQDN) of your UCMDB server, and replace *<port>* with the communications port your UCMDB server uses. **Note** If you are setting up an HTTPS connection, make sure to add the UCMDB root CA certificate to the SM truststore. See the *Add the UCMDB root CA certificate to the SM truststore* section below.
User Id	Enter the user name required to manage CIs on the UCMDB system. For example, enter **admin**, which is the out-of-the-box administrator user name.
Password	Enter the password required to manage CIs on the UCMDB system. For example, enter **admin**, which is an out-of-the-box password for the **admin** user account.

Field	Description
Multi-tenant UCMDB webservice URL	This is the URL that Service Manager uses to synchronize company records with UCMDB when running in multi-company mode. This URL should use the following format: https://*<UCMDB server name>*:*<port>*/axis2/services/UcmdbManagementService Replace *<UCMDB server name>* with the fully-qualified domain name (FQDN) of your UCMDB server, and replace *<port>* with the communications port your UCMDB server uses. **Note** If you are setting up an HTTPS connection, make sure to add the UCMDB root CA certificate to the SM truststore. See the *Add the UCMDB root CA certificate to the SM truststore* section below.
UserId Password	This is UCMDB account (user name/password) that has the privilege to add/delete company records.

7. Click **Verify Connection** to verify that your UCMDB Server configuration is correct.
8. On the **UCMDB Browser Configuration** tab, configure each field or option as described in the following table.

Field / Option	Default value	Description
UCMDB Browser URL		Enter your UCMDB Browser URL in the following format: https://*<UCMDB Browser server name>*:*<port>*/ucmdb-browser Replace <UCMDB Browser server name> and <port> with the fully-qualified domain name (FQDN) and communications port of UCMDB Browser server host, respectively. **Important** When specifying UCMDB Browser URL, use the FQDN rather than the IP address of the server host.

Field / Option	Default value	Description
Show UCMDB Browser Impact widget	false (disabled)	(Optional) Select this option if you want to embed UCMDB Browser Impact widget in Service Manager. **Tip** This option shows or hides the **Impact Widget** tab on this page.
Show UCMDB Browser CI Visualization widget	false (disabled)	(Optional) Select this option if you want to embed the UCMDB Browser CI Visualization widget in Service Manager. **Tip** This option shows or hides the **CI Visualization Widget** tab on this page.

Field / Option	Default value	Description
Show UCMDB Browser in a New Browser Page	false (disabled)	Select this option if you want to display the UCMDB Browser CI Visualization widget on a new browser page. This option works only after you select either **Show UCMDB Browser Impact widget** or **Show UCMDB Browser CI Visualization widget**. **Caution** If the number of CIs exceeds the value as specified in **Maximum Trigger CIs per Impact Graph**, the CIs graph displays the first page only.

Field / Option	Default value	Description
Use a dedicated user account	false (disabled)	(Optional) Select this option if LW-SSO is disabled for the integration and you want to configure a dedicated UCMDB Browser user account for users to view UCMDB Browser data from Service Manager. This option enables users to access UCMDB Browser data directly from the Service Manager user interface without the need to enter a user name and password, and is therefore useful when LW-SSO is disabled for the integration. For example, this dedicated user account is used when users view an embedded UCMDB Browser widget (such as CI Visualization and Impact Simulation) from a Service Manager record or when users click the **View in UCMDB Browser** button from a CI record in Service Manager.

Field / Option	Default value	Description
		Note This option takes effect only when LW-SSO is disabled for the integration. If you have enabled both LW-SSO and this option for the integration, LW-SSO is used. **Tip** This option shows or hides the **Dedicated User** tab on this page.

Field / Option	Default value	Description
Impact Widget tab > **Maximum Trigger CIs per Impact Graph**	100	This is the maximum number of CIs that Service Manager is allowed to send to UCMDB Browser as trigger CIs when running impact simulation. For example, if the number of affected service and CIs of a Change record exceeds this value, when users click **Run Impact Analysis** for the Change record, the impact analysis result is displayed on multiple pages, each of which except the last one contains a **Next** button on the top. Users can click **Next** to view the next impact simulation graph. **Caution** The value of this field must not be greater than 100.

Field / Option	Default value	Description
Impact Widget tab > **Impact Bundle**		This is a rule bundle that's defined in UCMDB Server and will be used for impact analysis. If you leave this field blank, the **List of impact rule bundles** setting in UCMDB will be used. To access this setting in UCMDB, go to **Administration** > **Infrastructure Settings Manager**, and choose **UCMDB Browser Settings**. An empty value of this setting means all rule bundles will be used.
Impact Widget tab > **Default Impact Severity**	critical	This is the default impact severity for impact analysis. You can set it to **critical**, **high**, **medium**, **low**, or **none**. **Note** A severity of "none" will be used when this field is left blank.

Field / Option	Default value	Description
CI Visualization Widget tab > **Filter TQL for CI Visualization**		This is a filter TQL query that's used to retrieve CI relationship graphs from UCMDB Browser. A filter TQL query is used to restrict the CI types and relationship types that you want to display on each relationship graph.

Field / Option	Default value	Description
Dedicated User tab > **User Id**		Create a dedicated user account in UCMDB Browser and grant appropriate permissions to this account. Enter the username in this field. **Caution:** This user account must be granted only limited permissions that are just enough to view the Impact Simulation, Get Related, and History widgets and to access CIs. For information about how to specify which widgets are displayed to a user and how to authorize CI access for a user, refer to the *UCMDB Browser Installation and Configuration Guide*. **Caution:** To protect your privacy, we recommend enabling LW-SSO configuration instead of using the user id and password. See How to enable Lightweight Single Sign-On (LW-SSO) configuration for more information.

Field / Option	Default value	Description
Dedicated User tab > **Encrypted Password**		Obtain an encrypted value of the password of the dedicated user account, and enter the value in this field. To obtain an encrypted value for the password of a UCMDB Browser user account, follow these steps: 1. Open UCMDB URL in your browser. 2. Click **JMX Console** from the login page. 3. Sign in to the JMX Console as an administrator. 4. Search for **getencryptedPasswordForURL** and then open this section. 5. In the **User Name** and **Password** fields, enter the user name and password of the user account that you want to use. 6. Click **Invoke**. An encrypted value of the password is returned. **Caution:** To protect your privacy, we recommend enabling LW-SSO configuration instead of using the user id and password. See How to enable Lightweight Single Sign-On (LW-SSO) configuration for more information.

9. On the **UCMDB Gateway Configuration** tab, configure each field or option as described in the following table.



Important

The UCMDB Gateway configuration invalidates the UCMDB Browser configuration. After configuring the UCMDB Gateway setting, the CI visualization graph and the Impact visualization graph that are previously displayed by the UCMDB Browser are still available in Service Manager. However, you still need to perform UCMDB Browser configuration if you need to view CI history.

Field / Option	Default value	Description
UCMDB Gateway URL	https://<server host:port>/cms-gateway	Enter your UCMDB Gateway URL. Replace <server host> and <port> with the fully-qualified domain name (FQDN) and communications port of UCMDB Gateway server host, respectively.

Field / Option	Default value	Description
		Note If you are setting up an HTTPS connection, make sure to add the UCMDB Gateway root CA certificate to the SM truststore. See the *Add the UCMDB Gateway root CA certificate to the SM truststore* section below. Meanwhile, If you are connecting to the UCMDB Gateway with SSL, you also need to configure the UCMDB Gateway truststore for the web tier. See the *Configure the UCMDB Gateway truststore for the web tier* section below.

Field / Option	Default value	Description
User Id		Enter the name of the UCMDB Gateway integration user. This user must have access to the data on all tenants.
Password		Enter the password of the UCMDB Gateway integration user.

10. Click **Verify Connection** to verify that your UCMDB Gateway configuration is correct.
11. Click **Finish**.

    Service Manager creates an SMtoUCMDB integration instance.
12. Go to **Integration Manager**, and then click the SMtoUCMDB integration instance.
13. Click **Next** until the **Initialize** link becomes active, and then click the link.

    > **Note**
    >
    > This step creates records in the **cirelationship1to1** table so that CI relationships can be properly populated to UCMDB.

14. Click **Finish** to go back to Integration Manager, select the SMtoUCMDB integration instance, and then click **Enable** to enable the integration instance.

Important

When using the embedded UCMDB Browser, for the Impact Simulation Widget and Get Related Widget to work, perform the following steps:

1. Go to **JMX console > UCMDB:service=Settings Services > setGlobalSettingValue**.
2. In the **name** field, enter **ui.framing.host**.
3. In the **value** field, enter the desired Service Manager web tier host address. For example, `frame-ancestors https://<IP>:<port>` or `frame-ancestors https://<web tier FQDN>:<port>`.
4. Click **Invoke**.
5. Restart the UCMDB server.

When using UCMDB Browser standalone, for the Impact Simulation Widget and Get Related Widget to work, perform the following steps:

1. Open the **%WEB_SERVER_HOME%\webapps\ucmdb-browser\WEB-INF\web.xml** file.
2. Locate the following lines:

```
<filter>
    <filter-name>ClickjackingFilterSameOrigin</filter-
name>
    <filter-
class>com.hp.ucmdb_browser.server.web.ClickjackingFilter</
filter-class>
    <init-param>
        <param-name>mode</param-name>
        <param-value>SAMEORIGIN</param-value>
    </init-param>
</filter>
<filter-mapping>
        <filter-name>ClickjackingFilterSameOrigin</filter-
name>
        <url-pattern>/*</url-pattern>
</filter-mapping>
```

3. In the section above, replace the following line:

```
<param-value>SAMEORIGIN</param-value>
```

with

```
<param-value>ALLOW_FROM <Service Manager URL></param-
value>
```

For example,

```
<param-value>ALLOW_FROM
https://SMservername.domain.com</param-value>
```

4. Restart the UCMDB Browser.

Add the UCMDB root CA certificate to the SM truststore

Follow these steps to add the UCMDB root CA to the SM server by using the certificate generation toolkit.

1. Copy the UCMDB server root certification file cmsroot.crt from the UCMDB server installation directory (for example, `C:\UCMDB\UCMDBServer\conf\security\`) to the `cert-utils/certs` folder.
2. Import cmsroot.crt to the SM truststore. See the following sample commands.

   Non-FIPS mode:

   ```
   import-cert.bat --alias cmsroot --file certs/cmsroot.crt --keystore
   certs/cacerts --storepass changeit
   ```

   FIPS mode:

   ```
   import-cert.bat --alias cmsroot --file certs/cmsroot.crt --keystore
   certs/cacerts --storetype bcfks --storepass changeit
   ```

3. Copy cacerts to the `<SM_SERVER_HOME>/RUN` folder.
4. Update sm.ini by adding the following lines:

   ```
   truststoreFile:cacerts
   truststorePass:changeit
   ```

5. Restart the SM server.

Add the UCMDB Gateway root CA certificate to the SM truststore

Follow these steps to add the UCMDB Gateway root CA to the SM server by using the certificate generation toolkit.

1. Copy the UCMDB Gateway root CA certification file cmsrootca.crt from the UCMDB Gateway installation directory (for example, `C:\cms-gateway-11.6.9.59-windows\cms-gateway-installer\ssl`) to the `cert-utils/certs` folder.
2. Import cmsrootca.crt to the SM truststore. See the following sample commands.

   Non-FIPS mode:

   ```
   import-cert.bat --alias cmsrootca --file certs/cmsrootca.crt --
   keystore certs/cacerts --storepass changeit
   ```

   FIPS mode:

   ```
   import-cert.bat --alias cmsrootca --file certs/cmsrootca.crt --
   keystore certs/cacerts --storetype bcfks --storepass changeit
   ```

3. Copy cacerts to the `<SM_SERVER_HOME>/RUN` folder.
4. Update sm.ini by adding the following lines:

   ```
   truststoreFile:cacerts
   truststorePass:changeit
   ```

5. Restart the SM server.

Configure the UCMDB Gateway truststore for the web tier

Follow these steps to configure the UCMDB Gateway truststore for the web tier by using the certificate generation toolkit.

1. Copy the UCMDB Gateway root CA certification file cmsrootca.crt from the UCMDB Gateway installation directory (for example, `C:\cms-gateway-11.6.9.59-windows\cms-gateway-installer\ssl`) to the `cert-utils/certs` folder.
2. Import cmsrootca.crt to the UCMDB Gateway truststore. See the following sample commands.

   Non-FIPS mode:

   ```
   import-cert.bat --alias cmsrootca --file certs/cmsrootca.crt --
   keystore certs/cmsrootca.p12 --storepass <your password for the CMS
   truststore>
   ```

FIPS mode:

```
import-cert.bat --alias cmsrootca --file certs/cmsrootca.crt --
keystore certs/cmsrootca.bcfks --storetype bcfks --storepass <your
password for the CMS
truststore>
```

3. Copy cmsrootca.p12 (or cmsrootca.bcfks) to the `<customize-folder>/config` folder if you have configured the `customize-folder` parameter, or to the `<WEBTIER>\WEB-INF` folder if you haven't configured the `customize-folder` parameter.
4. Edit the `<customize-folder>/config/webtier.properties` file if you have configured the `customize-folder` parameter, or edit the `<WEBTIER>/WEB-INF/webtier.properties` file if you haven't configured the `customize-folder` parameter. For example:

   ```
   cms.truststore=<customize-folder>/config/cmsrootca.p12
   cms.truststore.password=changeit
   ```

   Or:

   ```
   cms.truststore=/WEB-INF/cmsrootca.p12
   cms.truststore.password=changeit
   ```

   For more information about `cms.truststore` and `cms.truststore.password`, see Web parameter:cms.truststore and Web parameter:cms.truststore.password, respectively.
5. Restart the web application server.

1.6.3.2.4. Universal CMDB Setup

You must complete the following tasks from your UCMDB system to support the integration.

1. Create an integration point between UCMDB and Service Manager.

   See How to Create an Integration Point in UCMDB.

2. Update the configuration files of the adapter.

   See How to Update the Time Zone and Date Format for the Integration Adapter.

1.6.3.2.4.1. How to create an integration point in UCMDB

A default UCMDB installation already includes the **ServiceManagerAdapter9.41** package. To use the integration package, you must create an integration point listing the connection properties for the integration.

Caution

For the data population, this integration supports the use of only one probe for your Service Manager system. Don't run population jobs on different probes by setting up multiple integration points with different probes for your Service Manager system. This integration allows only one probe for one Service Manager system.

To create an integration point:

1. Sign in to UCMDB as an administrator.
2. Add the integration user account that you created in Service Manager.
    1. Click **Security** > **Users and Groups**.
    2. Click the **New User** button .
    3. For User Name and Password, type the user name and password you created in Service Manager. See How to create an integration user account.
    4. Click **Next**, and then in the Role List select **Admin**.
    5. Click **Finish** to add the integration user account.
3. Navigate to **Data Flow Management > Integration Studio**. UCMDB displays a list of existing integration points.
4. Click the **New Integration Point** button .

    UCMDB displays a New Integration Point properties window.
5. Complete the integration and adapter property fields as described in the following table.

Integration and adapter properties

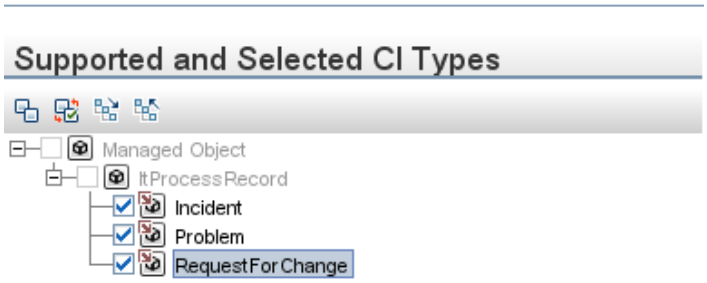
Field name	Required?	Description
Integration Name	Yes	Type the name (unique key) of the integration point. For example, **sm_integration**.
Integration Description	No	Type a description of the current integration point.
Adapter	Yes	Select **OpenText Products** > **Service Manager** > **ServiceManagerAdapter9.41**.
Is Integration Activated	Yes	Enable this option to indicate the integration point is active.
Hostname/IP	Yes	Type the hostname or IP address of the Service Manager server. For example, localhost.
Port	Yes	Type the communications port of the Service Manager server. For example, 13080.

Field name	Required?	Description
URL Override	No	This field value (if any) supersedes the previous Hostname/IP and Port settings descriptions. Use this field If you want UCMDB to connect to Service Manager in any combinations of the following ways: ◦ Connect to Service Manager over HTTPS or over both HTTP and HTTPS ◦ Connect to multiple Service Manager server nodes (horizontally scaled environment) ◦ Connect to one single Service Manager server node through multiple ports (vertically scaled environment) For more information, see Push in clustered environments. Type one or more Service Manager web services URLs (separated by a semicolon) in this field. Each URL must use this format: `http(s)://<hostname>:<port>/sc62server/ws` The following are two example values of this

Field name	Required?	Description
		field: • https://localhost:13443/sc62server/ws • http://localhost:13080/sc62server/ws; https://localhost:13443/sc62server/ws; http://smfpe04:13080/sc62server/ws
Credentials ID	Yes	Click **Generic Protocol**, click the **Add** button to add the integration user account you created, and then select it. This account must exist in both Service Manager and UCMDB. See How to create an integration user account.
Data Flow Probe	Yes	Select the name of the Data Flow Probe used to run population jobs. You should have already added the data flow probe for the integration after installing UCMDB. See Integration requirements.

6. Click **Test Connection** to make sure the connection is successfully created.
7. Click **OK**.

   The system creates the integration point and displays its details.
8. Click the **Federation** tab, and complete the following configuration.
   1. In Supported and Selected CI Types, select the following CI types as needed from **Managed Object > ItProcessRecord**:
      - Incident
      - Problem
      - Request for Change



2. For each CI type you selected (Incident, Problem, or Request for Change), in CI Type Retrieval Mode select **Retrieve CIs of selected CI Type**.

9. Click the **Population** and **Data Push** tabs, to view the default integration job details.

> **Note**
>
> UCMDB creates several default population and data push jobs when creating an integration point. If needed, you can create a new job for the integration point. For information about creating integration jobs, see How to define data push jobs in UCMDB and How to define population jobs in UCMDB.

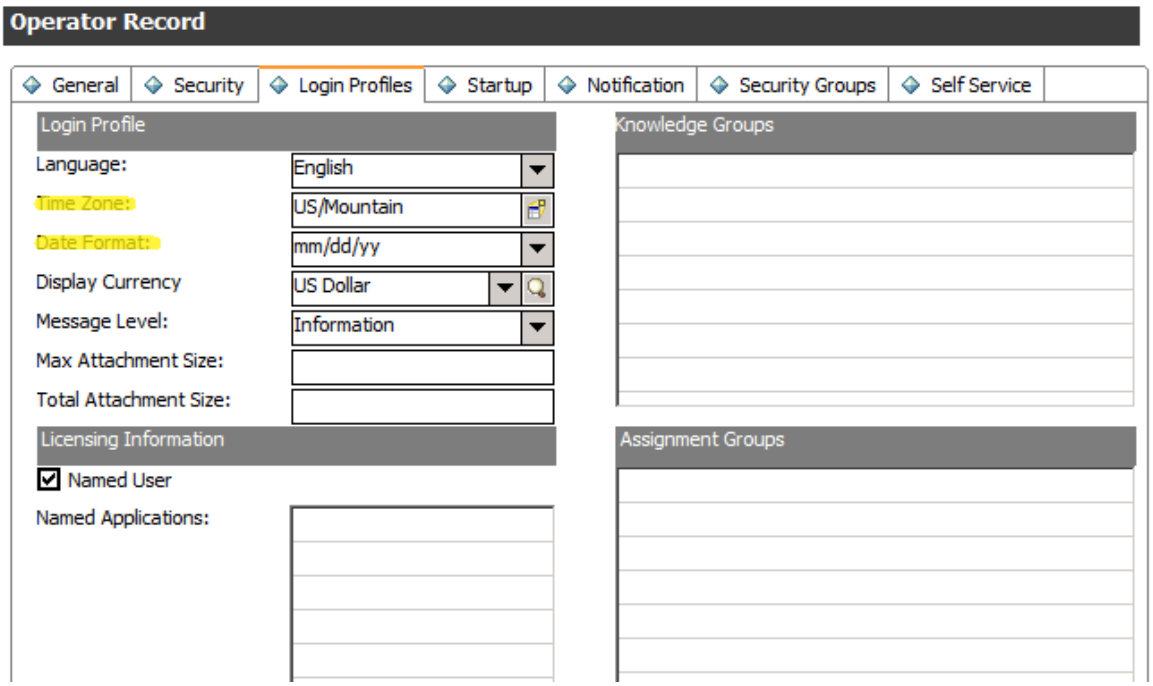
10. Click the **Save Integration Point** icon .

1.6.3.2.4.2. How to update the time zone and date format for the integration adapter

The integration uses an integration user account to connect UCMDB to Service Manager. You specified this user account in the Credentials ID setting when creating an integration point. See How to create an integration point in UCMDB. You must make sure that this integration user account uses the same time zone and date format in both Service Manager and UCMDB. To do so, you define the same time zone and date format in the integration adapter configuration file and in the integration user's operator record in Service Manager.

To update the time zone and date format for the integration adapter:

1. Check or change the time zone and date format of the integration user in Service Manager.
    1. Sign in to Service Manager as a system administrator.
    2. Navigate to **System Administration** > **Ongoing Maintenance** > **Operators**. The Operators form opens.
    3. For Login Name, type the integration user name (for example, `ucmdb`) you set in the integration point (the **Credentials ID** field).
    4. Click **Search**. Service Manager displays the record of the operator.
    5. Click the **Login Profiles** tab.
    6. View or change the operator's Time Zone and Data Format field values.



2. Set the time zone and date format for the adapter in UCMDB.
    1. Sign in to UCMDB as an administrator.
    2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files** > **serviceDeskConfiguration.xml**.

3. At the bottom of the file, locate the elements `<date_pattern>` and `<time_zone>`, and update the values according to the integration user's time zone/date format setting in Service Manager.



Note

The date pattern and time zone are in Java format pattern used by `java.text.SimpleDateFormat`. Out-of-the-box, the date format and time zone values in `serviceDeskConfiguration.xml` are: `MM/dd/yy HH:mm:ss`, and `US/Mountain`.

1.6.3.2.5. Populating UCMDB with Service Manager CI data

In addition to pushing CI data from UCMDB to Service Manager, this integration also supports the population of CI data (including CIs and CI relationships) from Service Manager to UCMDB. The integration can then update the list of CIs in UCMDB if new CIs or new attribute values are found in Service Manager. The population of data from Service Manager to UCMDB is defined in the Integration Studio in UCMDB. You can manually run the population jobs, however, we recommend that you schedule these jobs to keep your CIs and CI attributes up to date.

This task includes the following steps:

1. Define CI/CI Relationship population jobs in UCMDB.

   See How to define population jobs in UCMDB.

2. View the transferred CI/CI Relationship data in UCMDB.

   See View Service Manager CI data in UCMDB.

3. Schedule CI population jobs to keep CIs and CI attributes up to date.

   See How to schedule CI population jobs.

1.6.3.2.5.1. How to define population jobs in UCMDB

A CI/CI relationship population job copies certain types of CIs/CI relationships from Service Manager to UCMDB.

To define a CI or CI relationship population job:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**. UCMDB displays a list of existing integration points.
3. Open an integration point.
4. Click the **Population** tab, and add a new job as follows.

> **Note**
>
> UCMDB creates several default population and data push jobs when creating an integration point. The following table lists the default population jobs and their queries. If needed, you can create, update or remove queries for each job. For information about tailoring population queries, see Create a query to populate the CI type.

Queries for CI / CI relationship population

Integration Job	Queries for CI / CI relationship population
SM Configuration Item Population job	Out-of-the-box, the following queries are available for this job, which populates CI records from Service Manager to UCMDB: ◦ **SM Business Service Population**: Populates CIs of the bizservice type. ◦ **SM RunningSoftware Population**: Populates CIs of the RunningSoftware type. ◦ **SM Computer Population**: Populates CIs of the computer type.
SM Relations Population job	Out-of-the-box, the following queries are defined for this job, which populates CI Relationship records from Service Manager to UCMDB: ◦ **SM Biz To Biz With Containment**: Populates CI relationships in which a bizservice CI contains another. ◦ **SM Biz To Biz With Usage**: Populates CI relationships in which a bizservice CI uses another. ◦ **SM Biz To Computer With Containment**: Populates CI relationships in which a bizservice CI contains a computer CI. ◦ **SM Biz To Computer With Usage**: Populates CI relationships in which a bizservice CI uses a computer CI. ◦ **SM Computer To Computer With Connects**: Populates CI relationships in which a computer CI connects to another.

1. Click the **New Integration Job** button  .
2. Type a Name for the integration job. For example, **CI_Population_Job1**.

3. Click the **Add Query** button  to add existing queries to the job (see the above table).
4. Select the **Allow Integration Job to delete removed data** checkbox for the query.
5. Click **OK** to save the job.

5. Run the job manually to see if the integration job works properly.
   1. To populate all relevant data for the job, click the  button.
   2. To populate only CI data changes since the job last ran, click the  button.
6. Wait for the job to complete, and click the **Refresh** button multiple times as needed until the job is completed.

> **Note**
>
> When the job is completed, the job status becomes one of the following: Succeeded, Passed with failures, or Failed.

7. Click the **Statistics** tab to view the results, and if the job failed, click the **Query Status** tab and **Job Errors** tab for more information.
8. Click **OK**.

If the job is completed successfully, you can view the transferred CI data in UCMDB and schedule the job so that it can run automatically.

1.6.3.2.5.2. View Service Manager CI data in UCMDB

After a population job is successfully completed, you can search for the Service Manager CI records in UCMDB, and verify that their attributes are correctly populated.

The Service Manager **Display Name** field is populated to the **Name** field on the Configuration Item Properties pane in UCMDB.

Note

To see the entire attribute mappings of a CI type, you can open the CI type's population XSLT file (for example, business_service_population.xslt) and the root population xslt file (cmdb_root_attributes_population.xslt), where the UCMDB attribute field names and the mapped Service Manager web service field caption names are defined. For more information, see Tailoring the integration.

1.6.3.2.5.3. How to schedule CI population jobs

You can schedule CI population jobs to match the discovery/maintenance schedule of your Service Manager feeders. For example, if your Service Manager feeders send CI data updates on a daily schedule, then the population jobs should also run on a daily schedule. By using a matching schedule you can ensure that your UCMDB system always has the most current CI data.

This task includes the following steps:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**. UCMDB displays a list of integration points.
3. Open an integration point.
4. Click the **Population** tab, and select a population job from the list.
5. Click the **Edit Integration Job** button.
6. Select the **Scheduler enabled** option.
7. Select the scheduling options you want to use. For example, select Repeat every: **Day** and Ends: **Never**.
8. Select a Time Zone.
9. Click **OK**.

1.6.3.2.6. Pushing UCMDB CI data to Service Manager

The integration requires a one-time transfer of CIs from UCMDB to Service Manager to populate the Service Manager system with CIs. The integration will then update the list of CIs in Service Manager when UCMDB discovers new CIs or new attribute values. The integration accomplishes the push of CI data using data push jobs in the UCMDB system. We recommend that you schedule these jobs to keep your CIs and CI attributes up to date.

This task includes the following steps:

1. Define CI/CI Relationship data push jobs.

   See How to define data push jobs in UCMDB.

2. View the CI/CI Relationship data pushed from UCMDB.

   See How to view UCMDB CI data in Service Manager.

3. Schedule data push jobs to keep CI/CI Relationship data up to date.

   See How to schedule data push jobs.

1.6.3.2.6.1. How to define data push jobs in UCMDB

Data push jobs copy CI or CI Relationship records from your UCMDB system to your Service Manager system.

To define a CI or CI relationship push job:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management > Integration Studio**. UCMDB displays a list of existing integration points.
3. Select the Integration Point you created for Service Manager. For example, **sm_integration**.
4. Click the **Data Push** tab.
5. Follow these steps to add a new data push job:



Note

UCMDB creates a default data push job when creating an integration point. The following table lists the default data push job and its queries. If needed, you can create, update or remove queries for the push job. To access these out-of-the-box queries for push, go to **Modeling** > **Modeling Studio** > **Resources**, select **Queries** for Resource Type, and then navigate to **Root** > **Integration** > **SM Sync** > **9.xx**. For information about tailoring data push queries, see How to create a query to synchronize the CI type.

Queries for CI / CI relationship push

Integration job	Queries

Integration job	Queries
SM Push job	Out-of-the-box, the following queries are available for this job, which pushes CI/CI Relationship records from UCMDB to Service Manager: • **SM Mainframe Push**: pushes CIs of the mainframe type. • **SM Network Component Push**: pushes CIs of the network component type. • **SM Running Software Push**: pushes CIs of the running software type. • **SM Business Service Push**: pushes CIs of the business service type. • **SM Computer Push**: pushes CIs of the computer type. • **SM Storage Push**: pushes CIs of the storage type. • **SM Switch Push**: pushes CIs of the switch type. • **SM Net Printer Push**: pushes CIs of the net printer type. • **SM Cluster Push**: pushes CIs of the cluster type. • **SM Mobile Device Push**: pushes CIs of the mobile device type. • **SM Local Printer Push**: pushes CIs of the local printer type. Out-of-the-box, the following queries are available for this job, which pushes CI Relationship records from UCMDB to Service Manager: • **SM Layer2 Topology Relations Push**: pushes compound CI relationships between nodes. • **SM Business Service Relations Push**: pushes CI relationships whose upstream CI type is business service. • **SM CRG Relations Push**: pushes CI relationships whose upstream CI

Integration job	Queries
	type is cluster. ◦ **SM Node Relations Push**: pushes direct CI relationships whose upstream CI type is node.

1. Click the **New Integration Job** button  .
2. In the **Name** field, type a unique name for the job. For example, **CI_Push_Job1**.
3. Click the **Add Query** button  to add existing queries to the job.
4. Select the **Allow Integration Job to delete removed data** option for each query.
5. Click **OK** to save the job.

6. Run the job manually to verify that the integration job works fine.



Caution

If you have a huge amount of CI data in your UCMDB system, and this is your first time to push CI /CI Relationship data to Service Manager, it's advisable to select the **Add the record** option instead of **Open a change** or **Open an incident** for **Action if matching record doesn't exist** in each Discovery Event Manager Rules definition. Otherwise, unnecessary performance problems might occur. For details, see Add Discovery Event Manager rules.

1. To push all relevant data for the job, click the  button.
2. To push only the updated data since the job last ran, click the  button.



Tip

You can stop a running push job by pressing the **Stops the selected job** button .

7. Wait for the job to complete, and click the **Refresh** button multiple times as needed until the job completes.



Note

When the job completes, the job status becomes one of the following depending on the results: `Succeeded` , `Passed with failures` , or `Failed` .

8. Click the **Statistics** tab to view the results; if any errors occur click the **Query Status** tab and **Job Errors** tab for more information.
9. Click **OK**.

If the job completes successfully, you can view the UCMDB CI data in Service Manager, and schedule the job so that it can run automatically.

1.6.3.2.6.2. How to schedule data push jobs

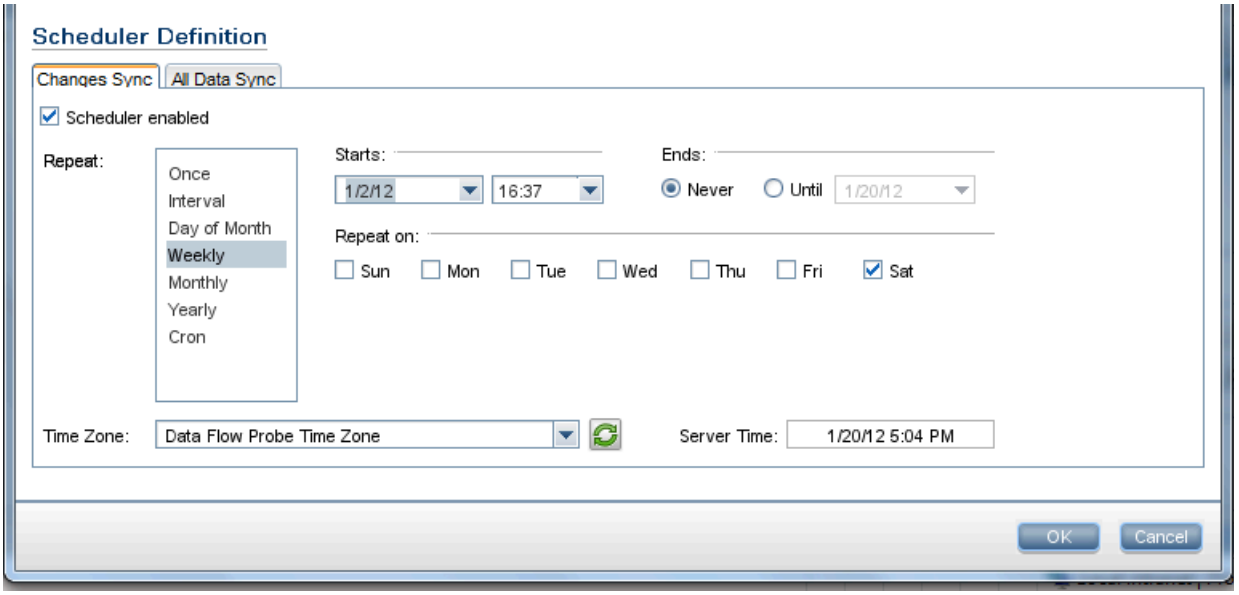
It's a best practice to schedule the data push jobs to match the discovery schedule of your Service Manager feeders. For example, if your Service Manager feeders send CI data updates on a daily schedule, the data push jobs should also run on a daily schedule. By using a matching schedule you can ensure that your Service Manager system always has the most current CI data.

UCMDB allows you to schedule updates directly from a data push job. This task includes the following steps:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**. UCMDB displays a list of integration points.
3. Select the integration point you created for the UCMDB - SM integration. For example, **SM Integration**.
4. Click the **Data Push** tab.
5. Select a push job. For example, **SM Configuration Item Push Job**.
6. Click the **Edit Integration Job** button  .

> **Tip**
>
> UCMDB allows you to define two different schedules for two types of data push: Changes Sync, and All Data Sync. For recommendations on push scheduling, see Push Scheduling recommendations.

7. Define a schedule for Changes Sync.
    1. Click the **Changes Sync** tab.
    2. Select the **Scheduler enabled** option.
    3. Select the scheduling options you want to use.



8. Click **All Data Sync** tab, and select the scheduling options you want to use.
9. Click **OK** to save the data push job.
10. Continue to follow *step 6* to *step 9* for the rest of data push jobs of the integration point.
11. Save the integration point.

1.6.3.2.6.3. How to view UCMDB CI data in Service Manager

After a push job is successfully completed, you can search for and verify the pushed CI/CI relationship data in Service Manager.

CI records pushed from UCMDB contains a **View in UCMDB UI** button, which enables you to access the UCMDB UI to view the CI information.

Note

- To display the **View in UCMDB UI** button, you need to specify the UCMDB UI URL in the System Information Record in SM.
- The UCMDB UI is a lightweight UI designed for simple access to UCMDB configuration information. This is a tool for searching, locating, and consuming data that relates to configuration. It's an optional add-on to UCMDB. For more information, refer to the CMS documentation.

To view UCMDB CI data in Service Manager:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Configuration Management** > **Search CIs**.
3. Open a CI record pushed from UCMDB.
4. If the **View in UCMDB UI** button is available, click this button to view the CI record in the UCMDB UI.
5. Open the **Actual State** section.

   Service Manager sends a web services request to UCMDB and displays all CI attributes that the request returns.

   Note

   The web services request uses the **UCMDB webservice URL** and account (for example, admin/admin) defined in the **SMtoUCMDB** integration instance in Service Manager. See How to add UCMDB connection information.

1.6.3.2.7. Federating Service Manager ticket data to UCMDB

Federation doesn't physically copy data from Service Manager (SM) to UCMDB; it only retrieves SM data for displaying in UCMDB. Out-of-the-box, the UCMDB-SM integration supports federation for the following external CI types in UCMDB: Incident, Problem, and RequestForChange. If you have enabled these CI types for federation when creating your integration point, in UCMDB you can retrieve Incident, Problem, and Change record data from SM.

Out-of-the-box, each of the CI types (Incident, Problem, and Change) supports federation of a subset of their attributes in Service Manager; however, the integration can federate more attributes if tailored as such.

This section includes:

- Federation queries
- Examples of using federation
- How to add an attribute of a supported CI type for federation

1.6.3.2.7.1. Federation queries

Federation uses queries to determine what data to retrieve from Service Manager. To retrieve specific ticket data from Service Manager, you need to create a query first. Out-of-the-box, sample federation queries are available from UCMDB: **Modeling** > **Modeling Studio** > **Resources** > **View** > **Service Desk**.

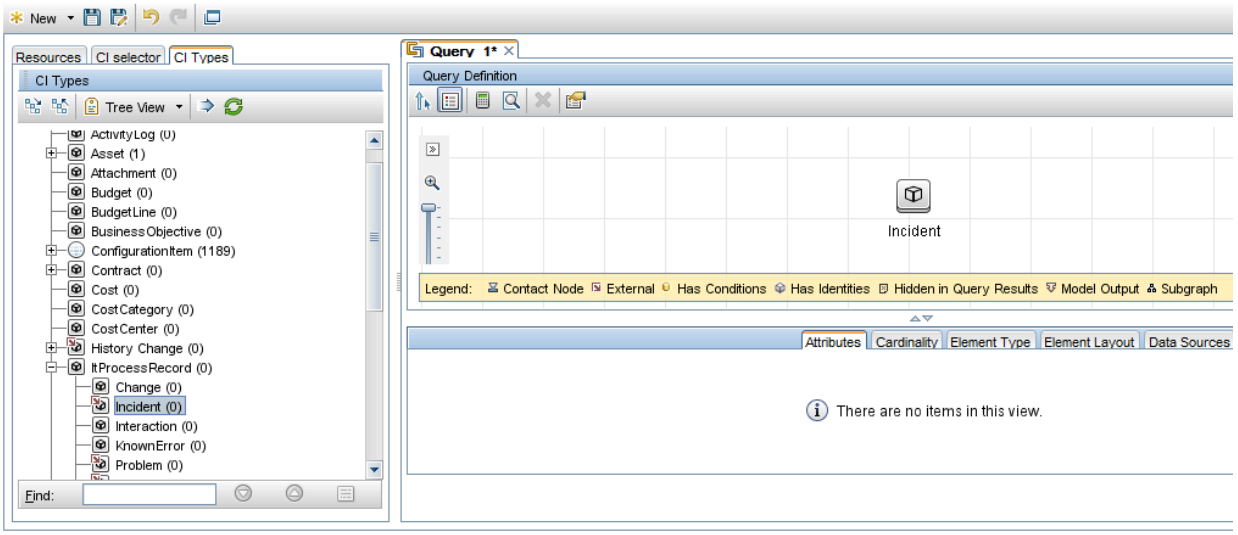
1.6.3.2.7.2. Examples of using federation

You can use the federation feature in many different ways. The following are only examples of using the feature:

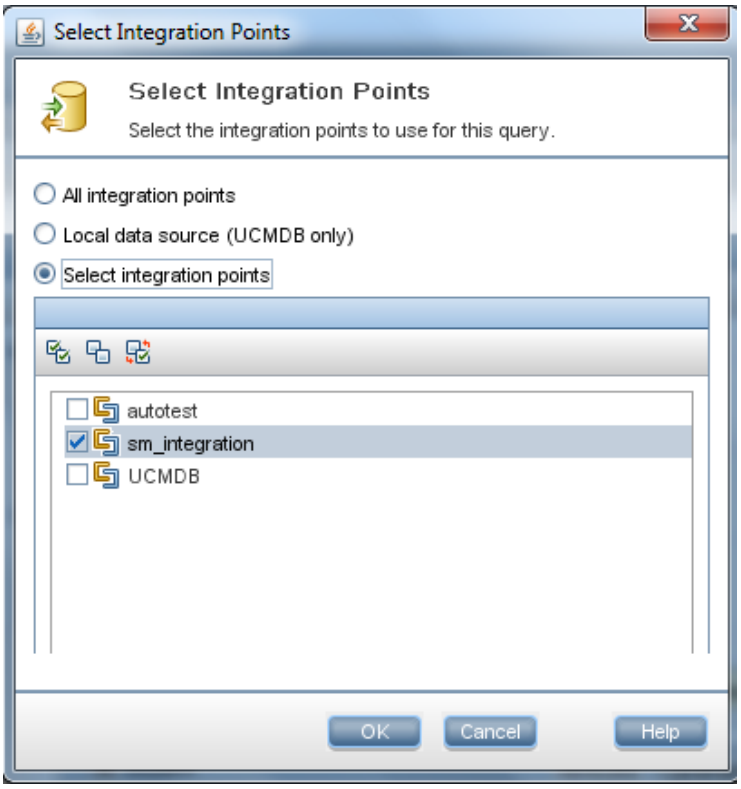
Example 1: Federate all SM Incident tickets

This example illustrates how you retrieve information of all Incident records that exist in Service Manager.

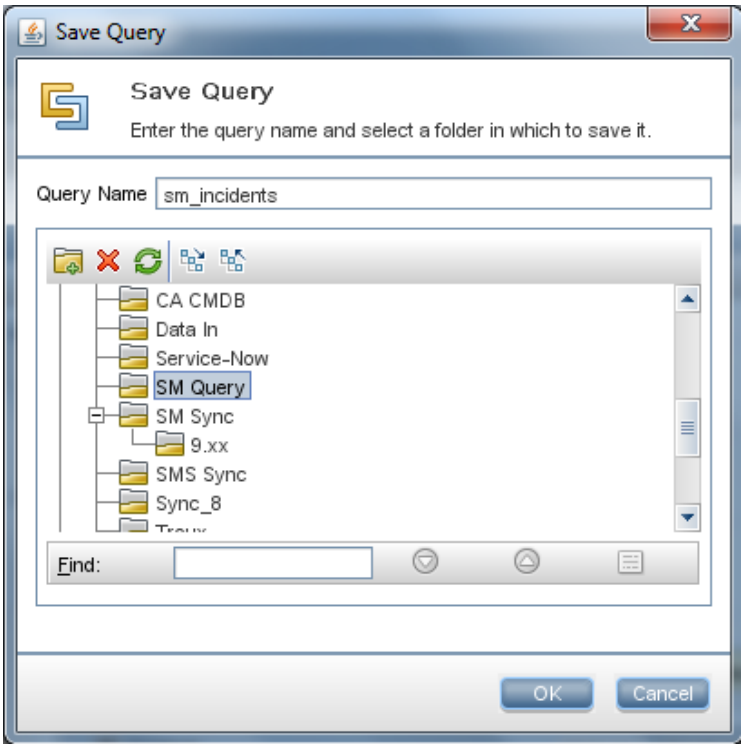
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio** > **Resources**.
3. For Resource Type, select **Queries** from the list.
4. Click **New** > **Query.**
5. On the CI Types tab, go to **ItProcessRecord** > **Incident**, and drag it to the query pane on the right side.



6. Specify Service Manager as the data source for the Incident query node.
    1. Select the Incident query node, click the **Data Sources** tab on the lower right pane, and then click **Edit**.
    2. Select the **Select integration points** option, and then select your integration point name (for example, **sm_integration**). Click **OK**.

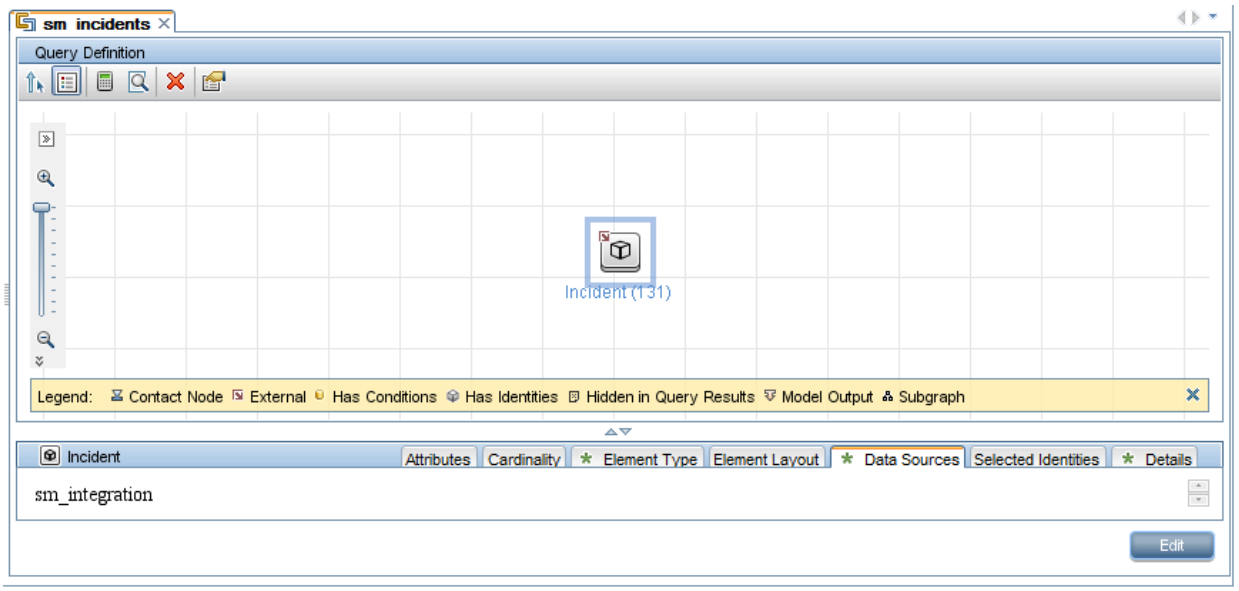


7. Click the **Save** button , and then type a query name and select a location in which to save the query (for example, select the **Root** > **Integration** > **SM Query** folder).

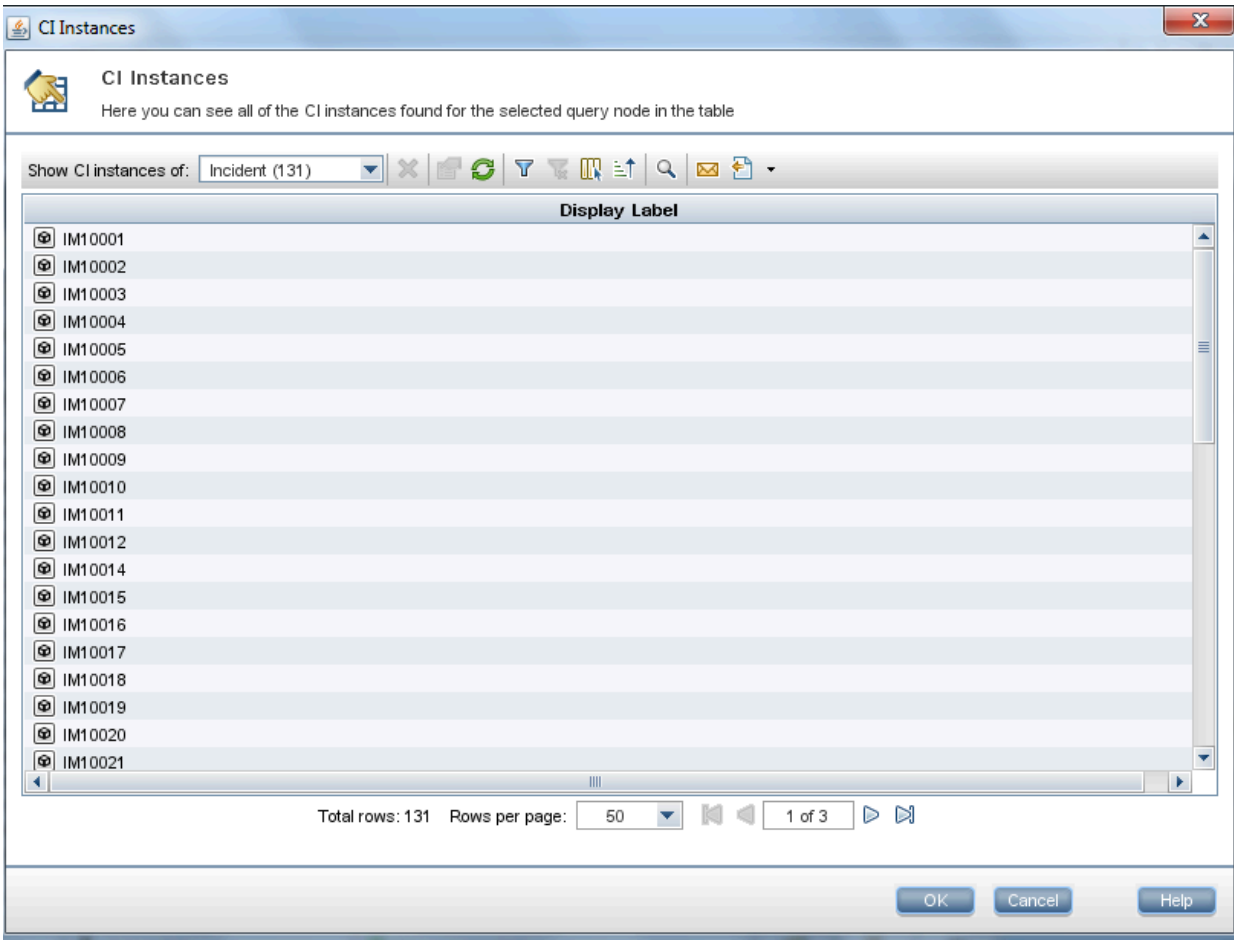


8. Select the Incident query node, and then click the **Calculate Query Result Count** button .

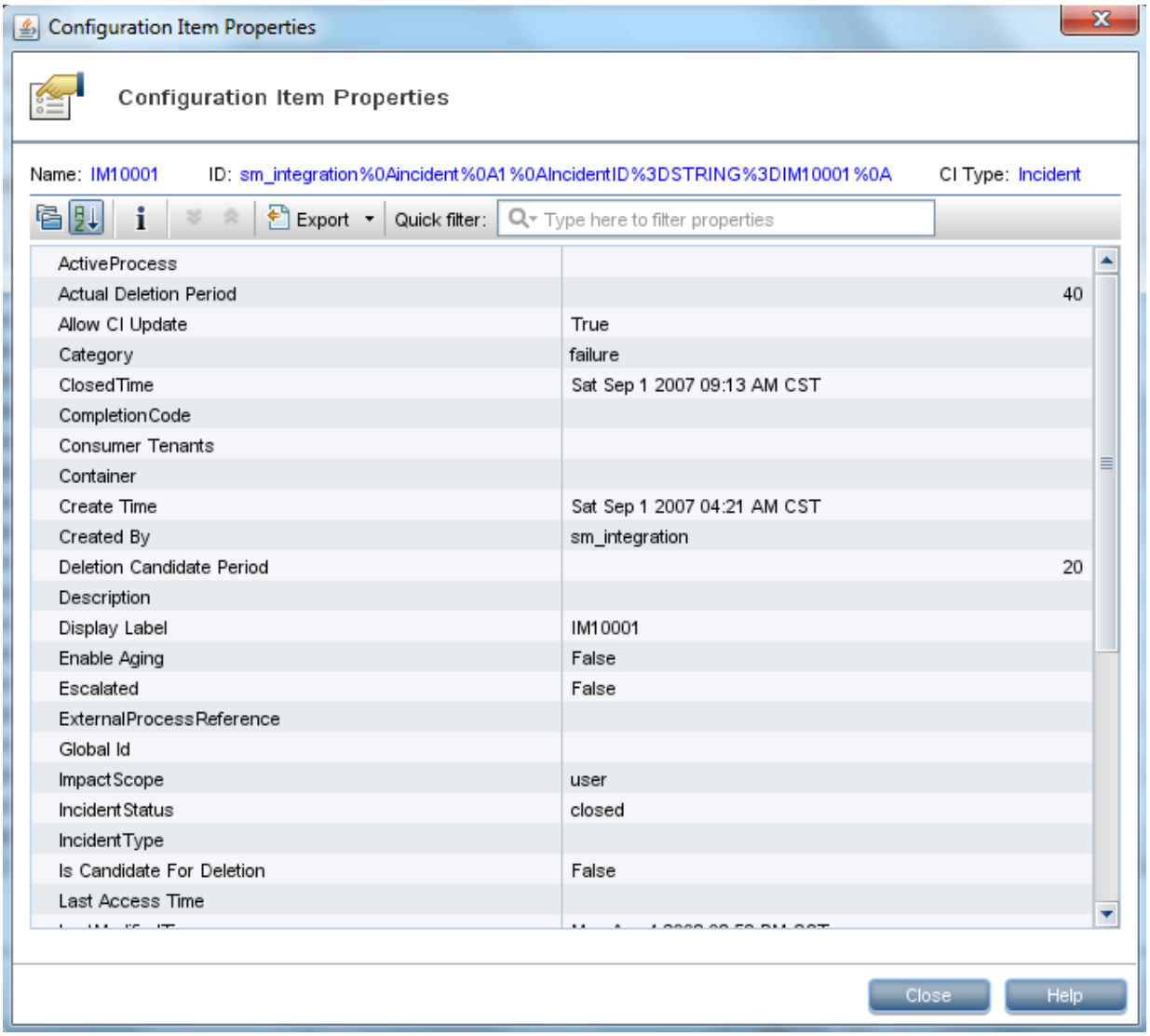
   UCMDB returns the query result count. For example, the following figure shows that there are a total of 131 Incident records in Service Manager.



9. Right-click the Incident query node, and select **Show Element Instances**. UCMDB displays a list of all Incident records that exist in Service Manager.



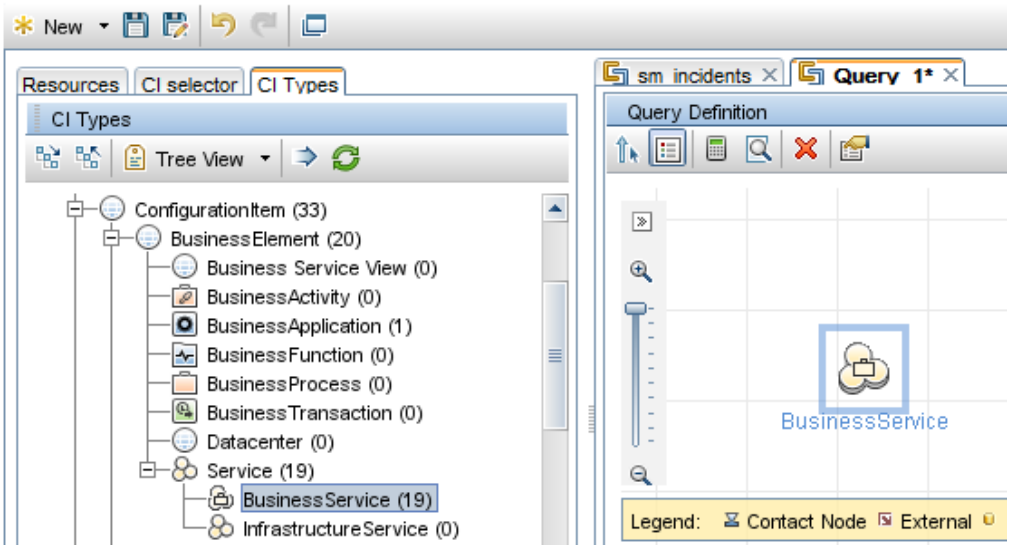
10. Select an Incident ticket from the list, and click the **Properties** button  to view its details.



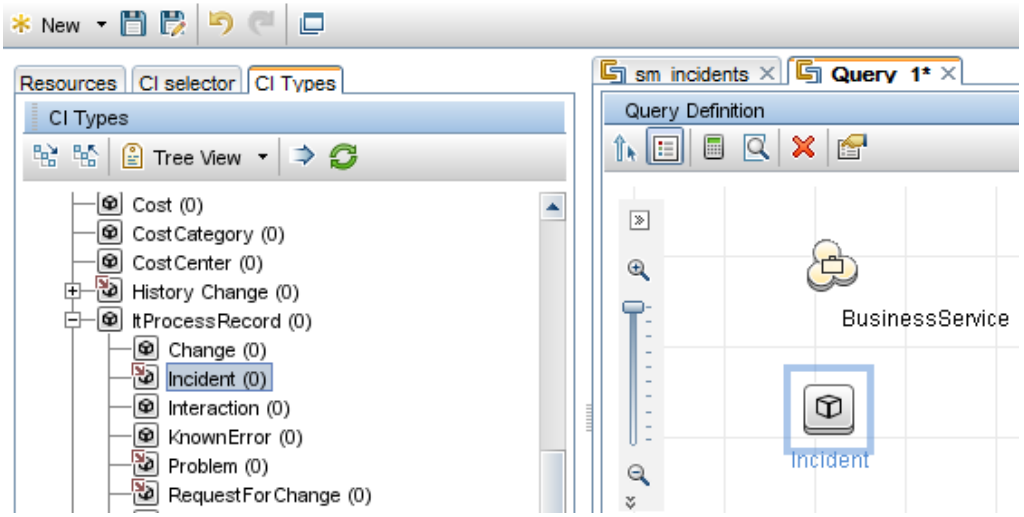
Example 2: Federate SM Incident tickets that affect a UCMDB Business Service CI

The following example illustrates how to federate a list of Service Manager Incident records whose Affected Service or Affected CI field contains a UCMDB Business Service CI.

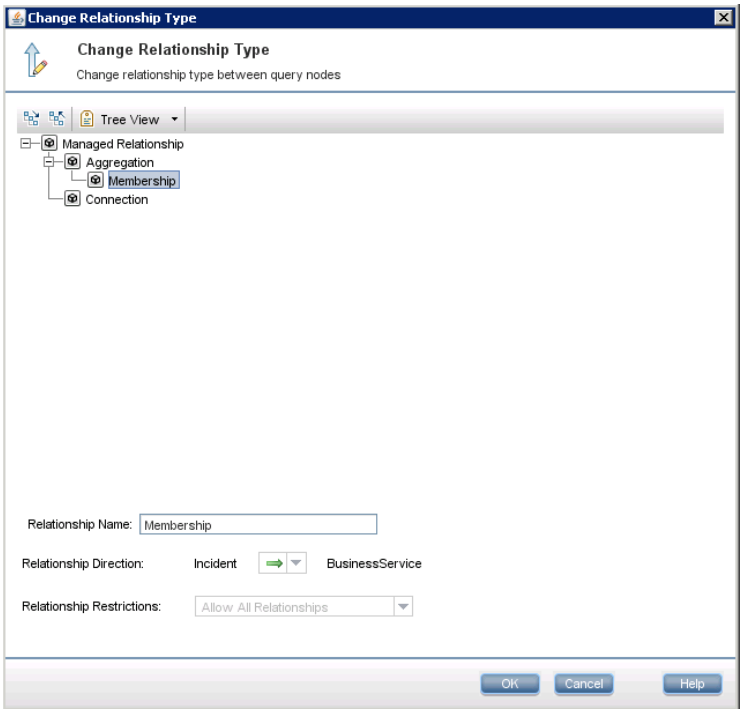
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio** > **Resources**.
3. For Resource Type, select **Queries** from the list.
4. Click **New** > **Query.**
5. On the CI Type tab, go to **ConfigurationItem** > **BusinessElement** > **Service** > **BusinessService**, and drag it to the query pane on the right side.



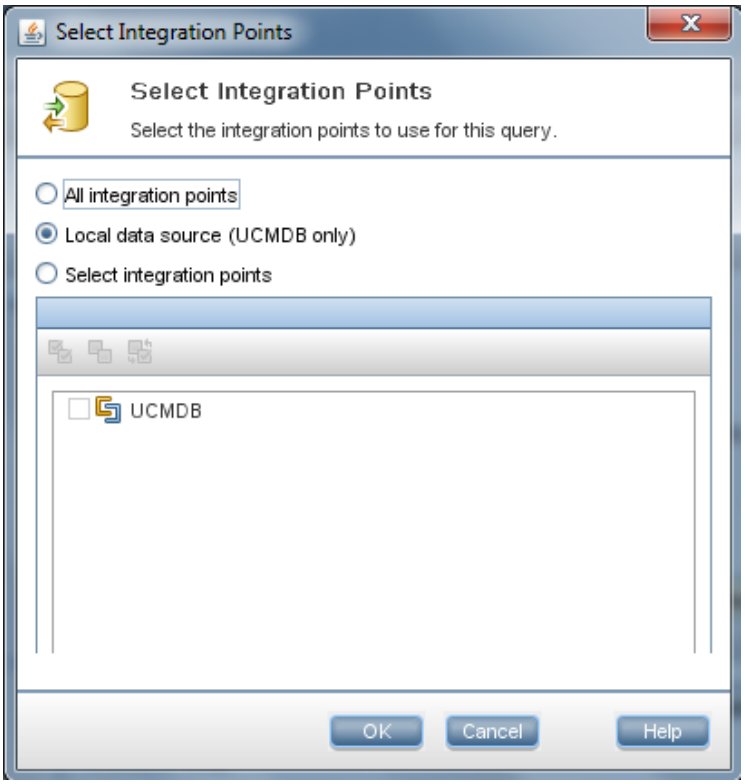
6. Go to **ItProcessRecord** > **Incident**, and drag the icon to the query pane.



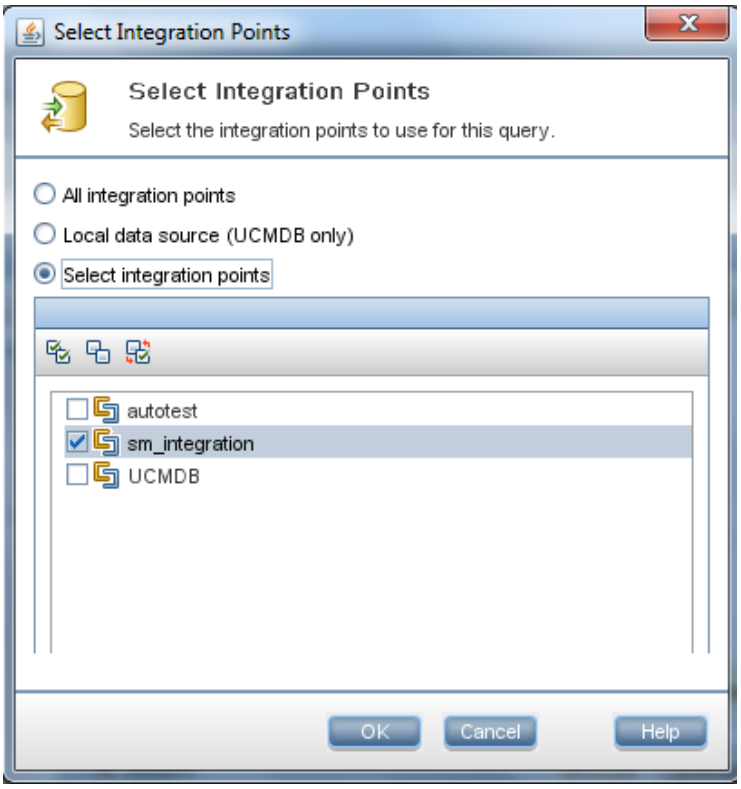
7. Click the **Create Relationship** button .
8. Select the Incident query node, and drag the arrow from this node to the BusinessService node to create a regular relationship between the nodes.
    1. Select **Regular Relationship**, and click **OK**.
    2. Select **Membership**, and optionally enter a relationship name (for example, **Membership_1**). Click **OK**.



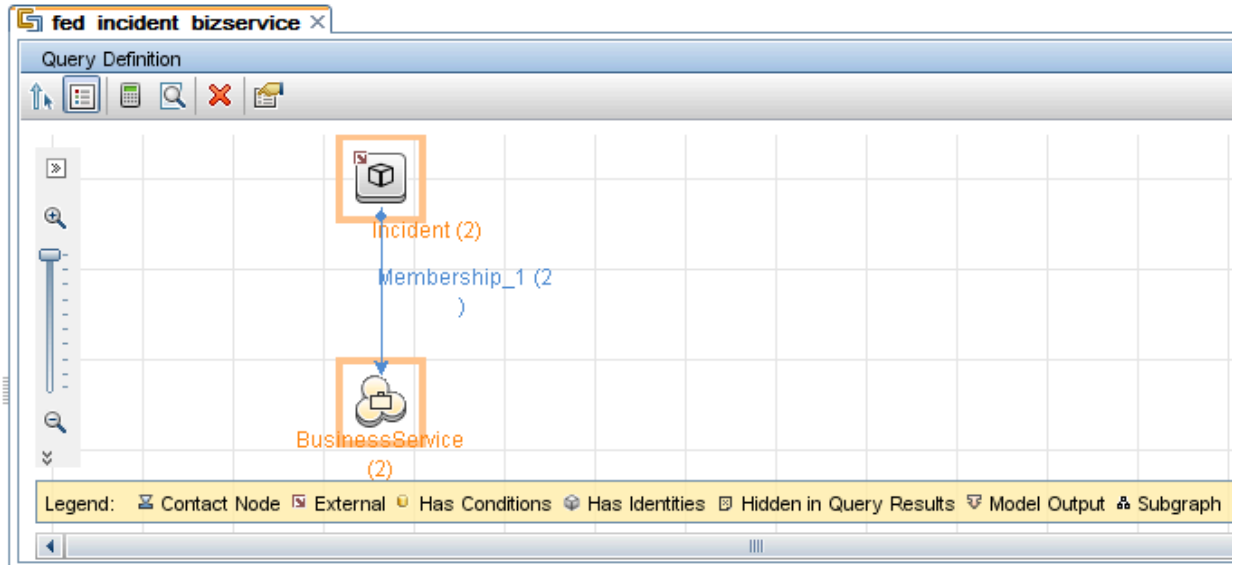
9. Specify UCMDB as the data source for the BusinessService query node.
    1. Select the BusinessService query node.
    2. On the lower right pane, click the **Data Sources** tab and then click **Edit**.
    3. Make sure that the **Local data source (UCMDB only)** option is selected.
    4. Click **OK**.



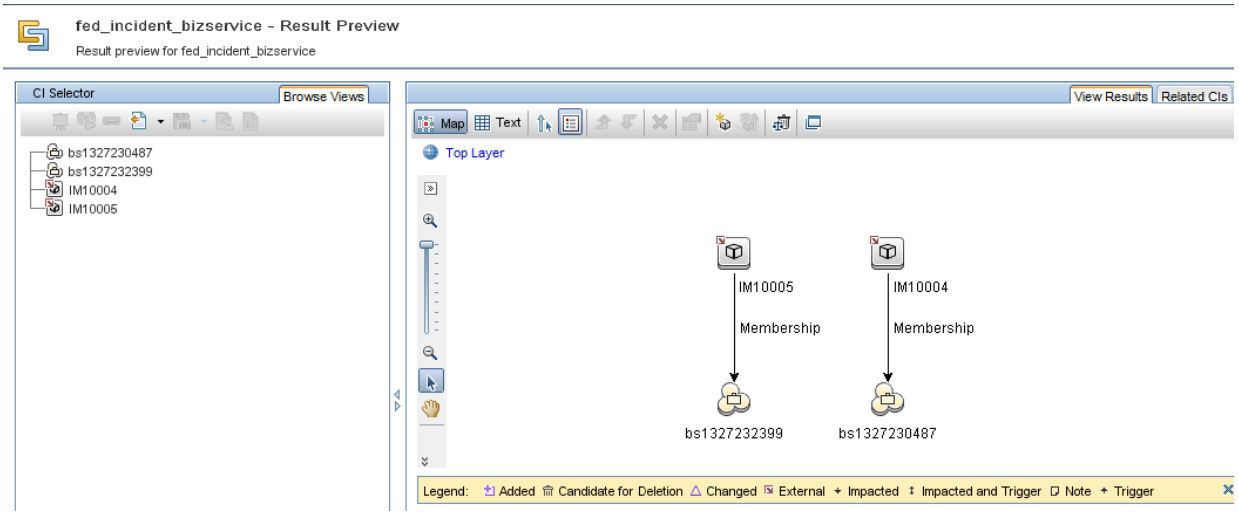
10. Similarly, specify your integration point as the data source for the Incident query node (for example, **sm_integration**).



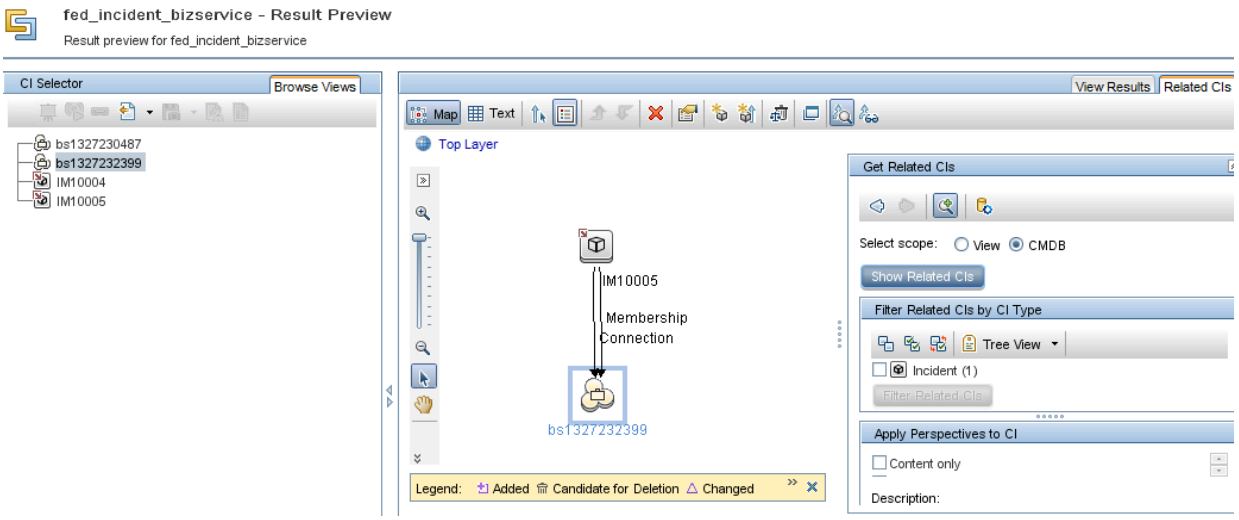
11. Click the **Calculate Query Result Count** button . The number of SM Incidents and the number of their affected UCMDB Business Service CIs are displayed.



12. Click the **Preview** button to view the query result .



13. Select each SM Incident record from either the CI Selector pane or the query pane, and click the **Properties** button to view its details  .
14. Select each UCMDB CI record from either the CI Selector pane or the query pane, and on the **Related CIs** tab click **Show Related CIs**.

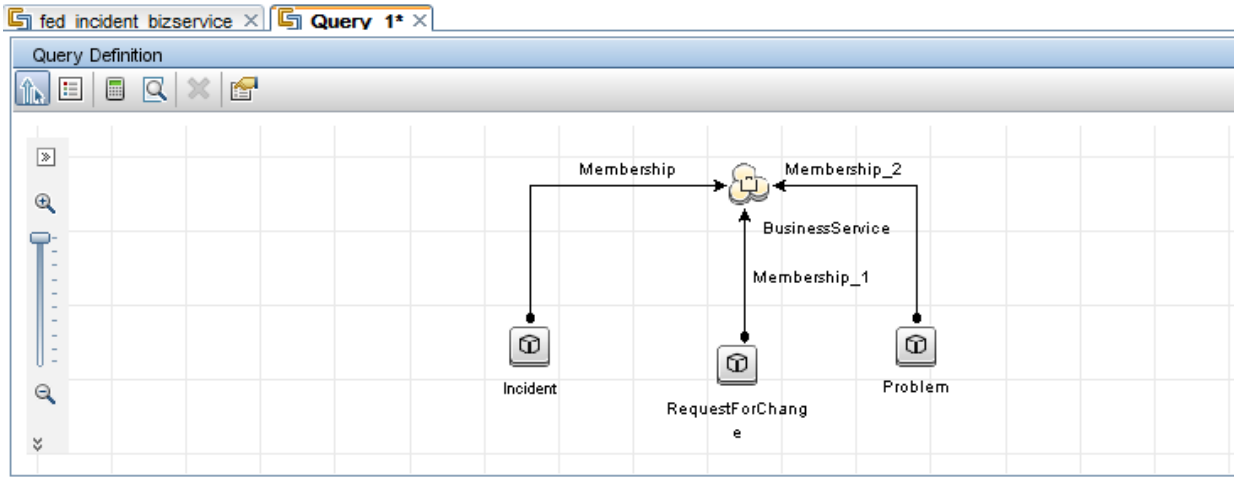


Example 3: Federate SM Incident, Change, and Problem ticket data of UCMDB CIs

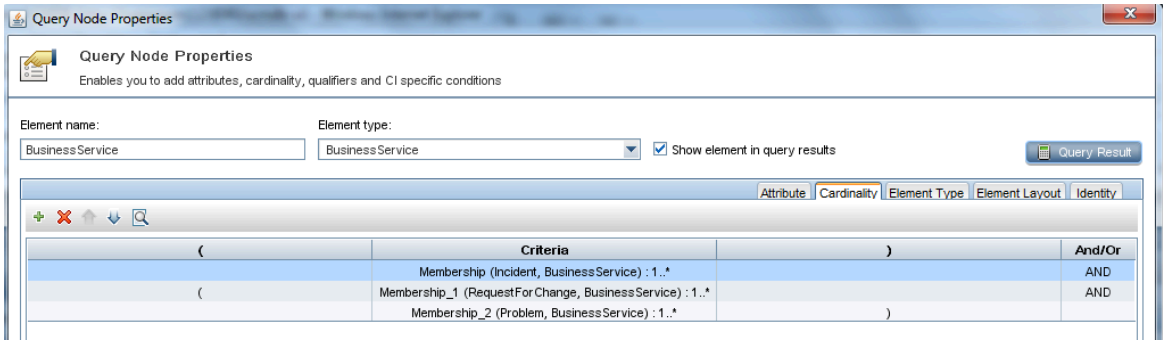
The following example illustrates how you retrieve information of SM Incident, Change and Problem tickets that affect a UCMDB Business Service CI.

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio** > **Resources**.
3. For Resource Type, select **Queries** from the list.
4. Click **New** > **Query.**
5. On the CI Type tab, go to **ConfigurationItem** > **BusinessElement** > **Service** > **BusinessService**, and drag it to the query pane on the right side.
6. Go to **ItProcessRecord**, and drag **Incident**, **Problem**, and **RequestForChange** to the query pane.

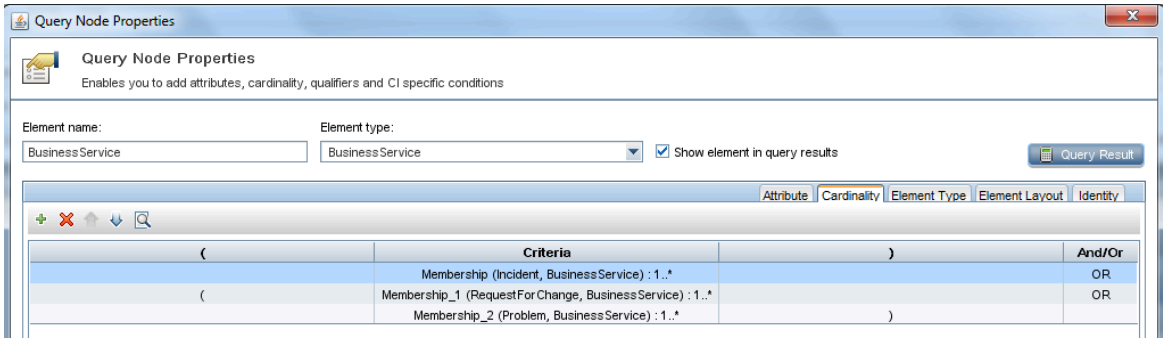
7. Click the **Create Relationship** button to create regular relationships between the BusinessService node and the other nodes as shown in the following figure.



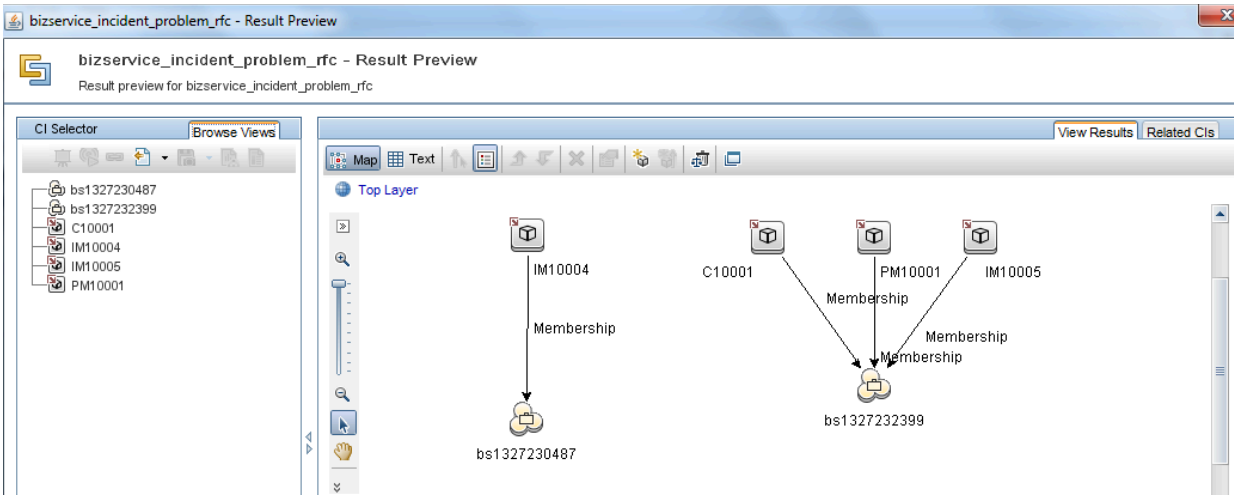
8. For the BusinessService node, specify UCMDB as the data source.
9. For the Incident, Problem, and RequestForChange nodes, specify your integration point as the data source.
10. Save the query.
11. Optionally, edit the BusinessService node properties as needed.
    1. Select the BusinessServic node, and click **Edit** on the lower right pane.
    2. Click the **Cardinality** tab. The default Cardinality setting is displayed.



    3. If you wish, change either or both of the **AND** operators to **OR**.
    4. This will change the filter criteria and therefore the query result.



12. Click the **Preview** button to view the query result.

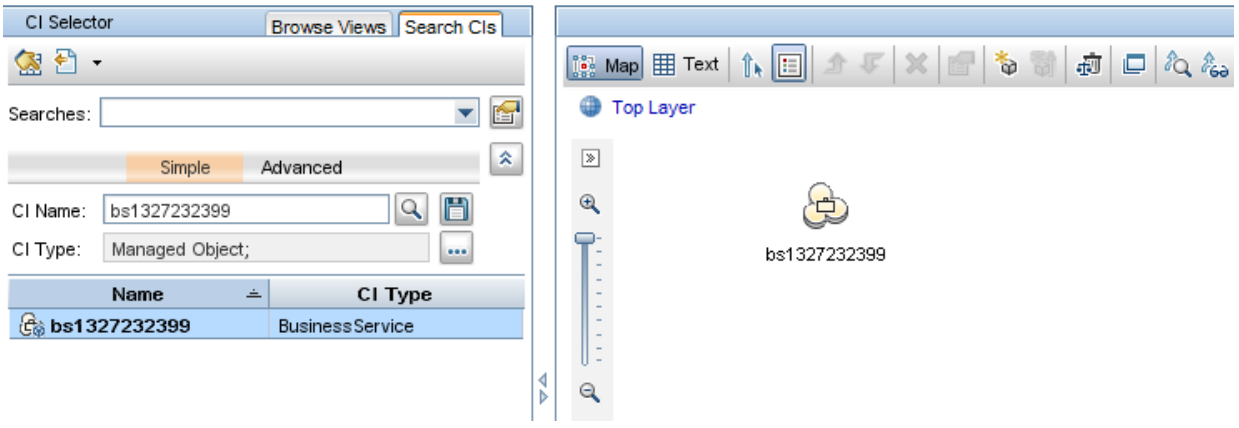


13. Select each SM Incident record from either the CI Selector pane or the query pane, and click the **Properties** button to view its details.
14. Select each UCMDB CI record from either the CI Selector pane or the query pane, and on the **Related CIs** tab click **Show Related CIs** to view its related CIs in both SM and UCMDB.

Example 4: Get related SM ticket data of a UCMDB CI

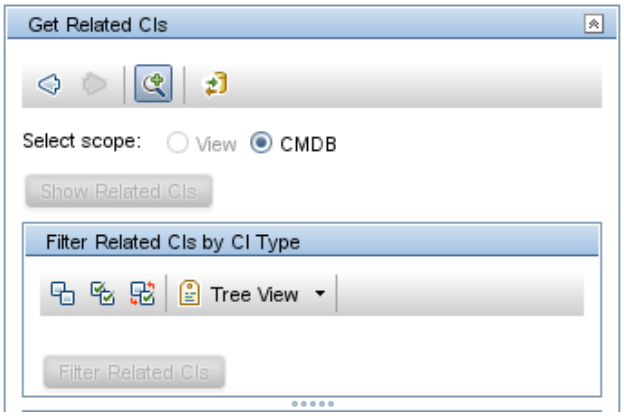
The following example illustrates how you retrieve SM ticket data related to a UCMDB CI by using the Get Related CIs functionality.

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **IT Universe Manager**.
3. On the **Search CIs** tab, search for a UCMDB CI that has associated ticket(s) in Service Manager. For example, enter bs1327232399 in the CI Name field, click **Search**, and double-click the CI to open it.



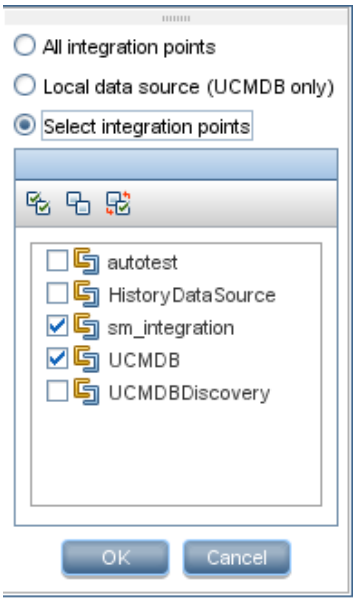
4. Click the **Show Get Related CIs** pane button .

   The Get Related CIs pane is displayed.

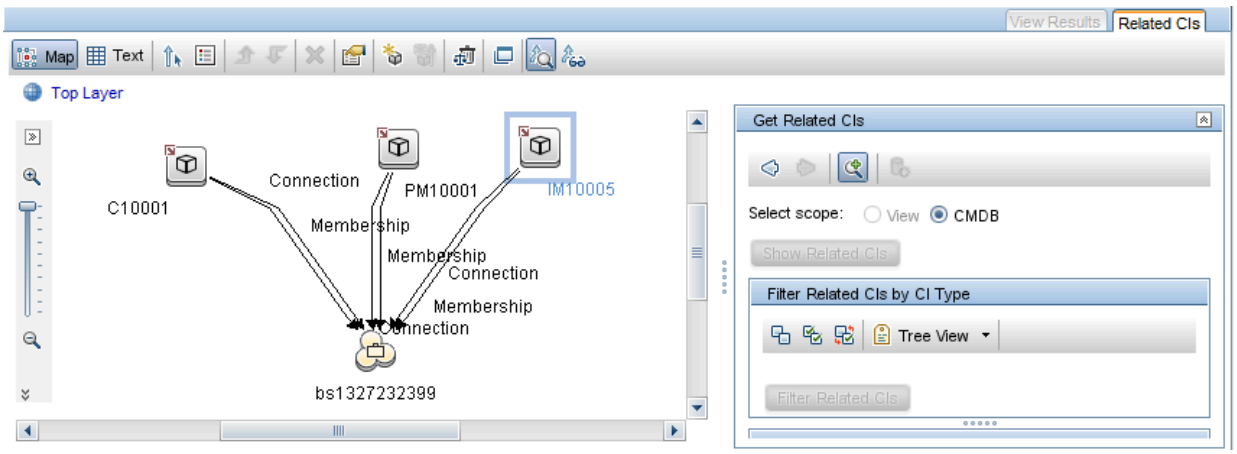


5. Click the **Select target Integration Points for related CIs** button .

6. Select the **Select integration points** option, and then select both **UCMDB** and your integration point. Click **OK**.



7. Click **Show Related CIs**. The CI's related SM tickets and UCMDB CIs are displayed.



8. Select each SM ticket record from the query pane, and click the **Properties** button to view its details.

1.6.3.3. Multi-tenancy (Multi-company) setup

The UCMDB-SM Integration supports a multi-tenancy configuration in which both the Service Manager and UCMDB systems track Configuration Items (CIs) and Configuration Item Relationships (CIRs) by company ID. In a multi-tenancy configuration, you can tailor the integration so that each tenant only sees and works with the CIs and CIRs that match their company ID. Multi-tenancy is intended for managed service providers (MSPs) who wish to offer Configuration Management as a service to multiple tenants.

Multi-tenancy (Multi-company) support

Multi-tenancy requirements

Setting up the multi-tenancy integration in UCMDB

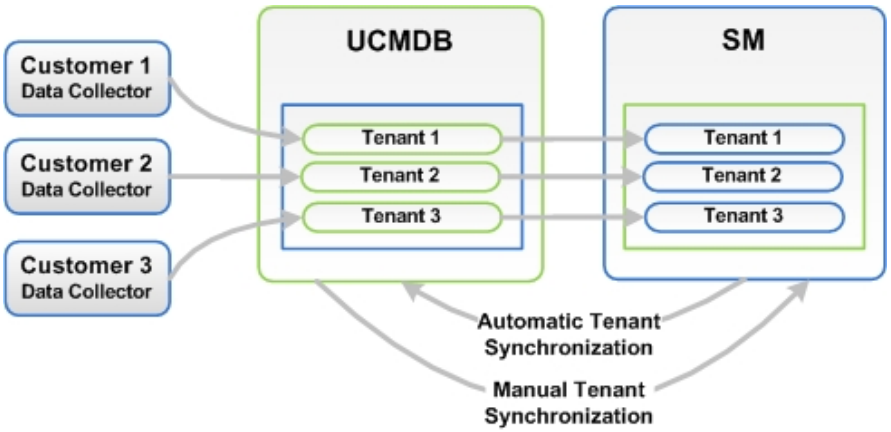
Setting up the multi-tenancy integration in Service Manager

1.6.3.3.1. Multi-tenancy (Multi-company) support

Multi-tenancy is when a single instance of software runs on a server, serving multiple client organizations (also referred to as tenants).

Multi-tenancy is contrasted with a multi-instance architecture where separate software instances or hardware systems are set up for different client organizations.

When implementing a multi-tenant architecture, a software application is designed to virtually partition its data and configuration so that each client organization works with a customized virtual application instance. The following figure illustrates an example of multi-tenant integration deployment.



Explanation

Every tenant configured in UCMDB works with the relevant tenant in SM. If UCMDB did not configure tenants, the tenant configuration must be activated in order to transfer the configuration from SM to UCMDB automatically. This function is performed once only by the system administrator.

In the event that UCMDB tenant configuration already exists and the SM configuration doesn't exist, SM tenant must be manually configured according to the UCMDB configuration.

1.6.3.3.1.1. Implementing multi-tenancy in the UCMDB-SM integration

SM stores the company records that describe each tenant in the multi-tenant configuration. The Service Manager system is the definitive source for company records and pushes all new company IDs to the UCMDB system creating the equivalent entity in UCMDB.

SM tracks the company ID of each CI and relationship in a multi-tenant configuration. CI records inherit the company ID of the UCMDB feeder that discovered them. Relationship records inherit the company ID of the parent CI in the relationship.

1.6.3.3.1.2. Mandanten SM security layer

Mandanten is an SM software layer that's used to filter the customer ID from the CI information. SM uses the Mandanten to ensure that operators only see CI and relationship records where the CI's company ID matches the operator's company ID. If the view is restricted with Mandanten, then Service Manager also restricts the view to all other related records such as change requests and incidents.

1.6.3.3.1.3. What multi-tenant Information is stored in UCMDB

Your UCMDB system stores a company ID attribute for each CI and CIR. The company ID determines what adapter and synchronization schedule your UCMDB system uses to update CI data. Each CI and relationship record can only have one company ID. The UCMDB system obtains a company ID from the Service Manager system.

If more than one tenant (company) shares the same CI, each tenant has their own unique CI record describing the CI. In effect, the UCMDB system creates multiple CI records to track one managed asset. Each tenant's CI record is unique to that tenant and lists the company's unique company ID.

1.6.3.3.1.4. What multi-tenant information is stored in Service Manager

Your Service Manager stores the company records that describe each tenant in the multi-tenant configuration. The Service Manager system is the definitive source of company IDs and pushes new and updated information to your UCMDB system.

Service Manager tracks the company ID of each CI and relationship in a multi-tenant configuration. CI records inherit the company ID of the UCMDB feeder that discovered them. Relationship records inherit the company ID of the parent CI in the relationship.

In a best practices implementation, Service Manager uses Mandanten to ensure that operators only see CI and relationship records where the CI's company ID matches the operator's company ID. If you restrict the view with Mandanten, then Service Manager also restricts the view to all other related records such as change requests and incidents.

1.6.3.3.1.5. Synchronization of company records

If your system meets all the conditions for multi-tenancy support, Service Manager creates a schedule record to push the company ID of the company record to your UCMDB system. Service Manager uses the following rules to determine whether to push the company ID to your UCMDB system.

Conditions where Service Manager synchronizes company ID with UCMDB

Conditions	Tenant information synchronized?	Schedule record created and action taken in UCMDB
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You create a new company record in Service Manager	Yes	Synch Company with UCMDB - *<UCMDB Company ID>* • Add new company ID
• UCMDB-SM integration **enabled** • You update an existing company record that has not been synchronized with UCMDB • Multi-company mode **enabled** in Service Manager	Yes	Synch Company with UCMDB - *<UCMDB Company ID>* • Add new company ID
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You disable the option to show a company in multi-company lists on a company synchronized with UCMDB	Yes	Inactivate Company with UCMDB - *<UCMDB Company ID>* • Inactivate existing company ID

Conditions	Tenant information synchronized?	Schedule record created and action taken in UCMDB
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You select the option to resynchronize with UCMDB on an existing company record	Yes	Synch Company with UCMDB - *<UCMDB Company ID>* • Add new company ID
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You enable the option to show a company in multi-company lists for an inactivated company	Yes	Synch Company with UCMDB - *<UCMDB Company ID>* • Reactivate company ID
• UCMDB-SM integration **disabled** • Multi-company mode **enabled** in Service Manager • You update an existing company record that has already been synchronized with UCMDB	No	None

Conditions	Tenant information synchronized?	Schedule record created and action taken in UCMDB
• UCMDB-SM integration **disabled** • Multi-company mode **enabled** in Service Manager • You create a new company record in Service Manager	No	None
• UCMDB-SM integration **enabled** • Multi-company mode **enabled** in Service Manager • You disable the option to show a company in multi-company lists on a company not synchronized with UCMDB	No	None
• UCMDB-SM integration **enabled** • Multi-company mode **disabled** in Service Manager • You create a new company record in Service Manager	No	None

UCMDB customer ID

When you enable the multi-tenancy integration, Service Manager displays a new field in each company record called UCMDB Customer ID. In order to synchronize a company record with UCMDB, you must first provide a value for this field. After you provide a UCMDB Customer ID value this field becomes read-only. You can't change a company's UCMDB Customer ID after you set it.

This field only accepts numeric data up to ten characters long. Service Manager requires the field value to be a unique positive whole number. You can't enter duplicate values or use decimals, negative numbers, or zero.

Your UCMDB system automatically uses the UCMDB customer ID of 1 when running in single tenant mode. You can reuse this default value in your multi-tenant implementation by assigning a Service Manager company to have this UCMDB customer ID value. Out-of-the-box, no Service Manager company has the UCMDB customer ID of 1.

UCMDB user ID and password

When you enable the multi-tenancy integration, Service Manager displays two new fields in each company record called UCMDB UserId and UCMDB Password. These fields allow you to specify the connection information you want Service Manager to use when requesting information for the Actual State section. Any user name and password you enter in these fields must be valid for your UCMDB system.

The user name and password you provide in the Company Information record takes precedence over the user name and password you provide in the System Information record. This allows managed service providers to control access to the UCMDB system on a tenant-by-tenant basis. If you don't provide a company-specific UCMDB user name and password, Service Manager uses the credentials you provided in the System Information record.

Company code

The multi-tenancy integration requires that each company record has a unique Company Code (company field) value. Since Company Code is a required field, your existing company records should already have Company Code values. However you should ensure that each company record has a unique Company Code value.



Caution

You should not change the Company Code value after you have enabled the multi-tenancy integration because this will cause your Service Manager data to become out of synch.

CI reconciliation rules

CIs that are pushed from UCMDB are identified only by their global.id value in UCMDB. By default, only the ucmdb.id field in the **device** table is used in CI reconciliation (the company.id field is not used).

You can add reconciliation rules manually through **Tailoring** > **Web Services** > **DEM Reconciliation Rules**. For details, see How to add DEM reconciliation rules.

Company information pushed to CI and CI relationship records

When you enable the multi-tenancy integration, Service Manager inserts the SM Company Code value in CI and relationship records during data push. Service Manager uses the UCMDB Customer ID to look up the matching SM Company Code value.

Company information replicated to Incident records

When you enable the multi-tenancy integration and select the option to create incidents when UCMDB discovers new, updated, or deleted CIs, Service Manager inserts the SM Company Code value in the incident record during replication. Service Manager uses the UCMDB Customer ID to look up the matching SM Company Code value.

Schedule records

Service Manager uses the problem schedule processor to manage the synchronization of company IDs to your UCMDB system. You can manually enable the problem schedule processor from the System Status form.

When the synchronization criteria are met as described in Table *Conditions where Service Manager synchronizes company ID with UCMDB*, Service Manager creates a “Synch Company with UCMDB - *<UCMDB Company ID>*” schedule record (for example, “Synch Company with UCMDB - 1234567890”). If you inactivate a company, Service Manager creates a “Inactivate Company with UCMDB - *<UCMDB Company ID>*” schedule record (for example, “Inactivate Company with UCMDB - 1234567890”). The problem schedule processor processes the new schedule record on the next background process iteration.

If your Service Manager system can't connect to your UCMDB system for some reason, it will reschedule the company synchronization at the next scheduled interval (the out-box interval is 5 minutes). The problem schedule processor updates the schedule record with the status rescheduled. If the Service Manager system receives any other error message while connecting to the UCMDB system, it updates the schedule record with the status “application failed due to error - check msglog for possible messages.”

1.6.3.3.1.6. Tenant-specific Discovery Event Manager (DEM) rules

You can implement the condition field function in order to create SM DEM rules that are specific to a particular tenant in a multi-tenancy UCMDB-SM integration.

Tenant rules vary according to SM tenant configuration requirements, for each record information type pushed from UCMDB to SM different tenants can configure different DEM tenant rules.

Each tenant can have its own set of unique requirements and therefore may implement different processes through the integration.

One tenant may require the addition of CIs directly to SM while another tenant may require opening changes for each CI.

The following table shows a sample set of DEM rules that illustrate how to accomplish this.

Tenant-specific DEM rules

DEM rule ID	Action on new CI	Condition
ucmdbNode_advantage	Add CI	company in $L.file="advantage"
ucmdbNode_hp	Create change	company in $L.file="HP"

Note

When creating DEM rules, be sure to create separate DEM rules for each tenant.

Multi-tenancy functional use cases

The following table describes the necessary actions to perform in various deployment situations to address multi-tenancy issues.

Multi-tenancy use cases

Deployment Integration Type	Description
UCMDB with multi-tenancy rules SM without multi-tenancy rules	When implementing a UCMDB-SM deployment that has existing multi-tenancy rules in UCMDB and doesn't have multi-tenancy rules configured in SM, the user creates multi-tenancy rules in SM manually and according to the rules in UCMDB.
SM with multi-tenancy rules UCMDB without multi-tenancy rules	When implementing a UCMDB-SM deployment that has existing multi-tenancy rules in configured SM and doesn't have multi-tenancy rules configured in UCMDB, the user creates multi-tenancy rules in UCMDB manually as well as according to the rules previously configured in SM.
UCMDB without multi-tenancy rules SM without multi-tenancy rules	When implementing a UCMDB-SM deployment that doesn't have multi-tenancy rules configured in UCMDB or in SM, the user configures the rules in SM. During the configuration process using the SM multi-tenancy wizard, the user can create a corresponding tenancy configuration in UCMDB. By creating corresponding tenancy configurations in SM the user also creates a corresponding tenant in UCMDB.

1.6.3.3.2. Multi-tenancy requirements

Your system must meet the following requirements in order for the integration to support multi-tenancy.

- Integration enabled between UCMDB and Service Manager
- Multi-company mode enabled on the Service Manager system
- Problem schedule process running on the Service Manager system

For additional information about the multi-tenancy integration, you can visit Software Support Online or refer to the Service Manager Help Center.

1.6.3.3.3. Setting up the multi-tenancy integration in UCMDB

You need to perform the following tasks in UCMDB to set up the multi-tenancy integration.

1. Install a separate data flow probe for each tenant the integration will support.

   See *How to install separate data flow probes for each tenant* below.

2. Start tenant-specific data flow probes.

   See *How to start tenant-specific data flow probes* below.

3. Configure IP address ranges for tenant-specific data flow probes.

   See *How to configure IP ranges for tenant-specific data flow probes* below.

4. Configure multi-tenancy for population.

   See *How to configure multi-tenancy for population* below.

How to install separate data flow probes for each tenant

If you plan to support a multi-tenant configuration, you must install a separate data probe for each tenant. Out-of-the-box, the UCMDB installer only installs one data flow probe and service.

To install additional data flow probes and start them from your operating system command prompt:

1. Sign in to the host of your UCMDB system as an administrator.
2. Insert the Universal CMDB Setup Windows DVD into the system disc drive.
3. Start the Data Flow Probe installer (HPUCMDB_DataFlowProbe_x.xx.exe).
4. Follow the on-screen instructions to complete the wizard, but use the following values for each data flow probe you wish to install.

   1. Type a unique path for each installation folder.
   2. Use the same UCMDB application server address for each data flow probe.
   3. Type a valid data flow probe address.
   4. Type a unique name for each data flow probe identifier.
   5. Create a unique customer Data Flow Probe domain for each probe (Clear the Use Default CMDB Domain option).
   6. Use the same probe gateway and probe manager settings for each probe (for example, use combined or separate processes).

   See the *Universal CMDB Deployment Guide* for complete installation instructions.

5. Repeat *step 3* and *step 4* for each data flow probe you wish to install.

6. Open the probe's `DiscoveryProbe.properties` file in a text editor. By default, this file is located in the following folder:

   `<UCMDB installation folder>\<data flow probe installation folder>\conf`

   For example, `C:\UCMDB\DataFlowProbe\conf`.

   > **Note**
   >
   > The <data flow probe installation folder> must be unique for each tenant.

7. Edit the following properties in the configuration file.

Discovery Probe properties set for each tenant

Property	Value
serverName	Verify the name of the UCMDB server
customerId	Type the customer ID for the tenant this data flow probe supports
appilog.collectors.probe.name	Verify the probe name is unique such as server + tenant ID
applilog.collectors.domain	Verify the domain name of the data flow probe
appilog.collectors.local.ip	Verify the data flow probe gateway name
appilog.collectors.probe.ip	Verify the data flow probe manager name
appilog.collectors.rmi.port	Type a unique port for each probe
appilog.collectors.rmi.gw.port	Type a unique port for each probe
.port	Type a unique port for each probe
.port	Type a unique port for each probe
appilog.collectors.ProbeUseSpecific RMIPortFrom	Type a unique port for each probe or type 0 to have the system automatically select it
appilog.collectors.bigBrother.port	Type a unique port for each probe

8. Save the configuration file.
9. Repeat *step 6* to *step 8* for each tenant's data flow probe.

How to start tenant-specific data flow probes

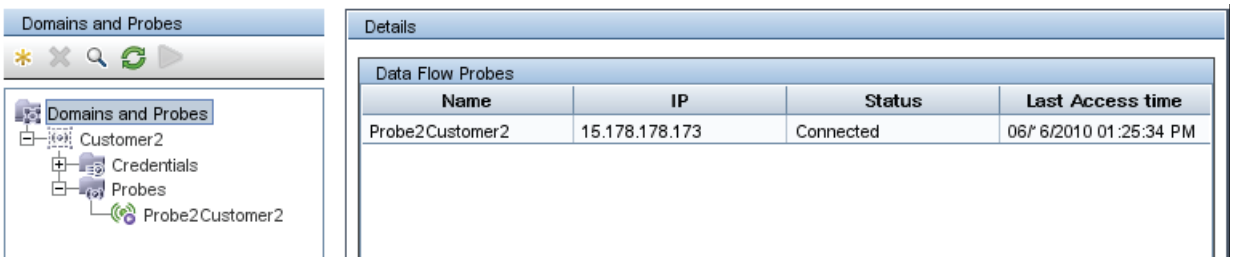
To start tenant-specific data flow probes:

1. Open the OS command prompt and navigate to the probe's bin folder. For example, `C:\UCMDB\DataFlowProbe1\bin`.
2. Type gateway console.
3. Repeat *step 1* to *step 2* for each data flow probe you want to start.

How to configure IP ranges for tenant-specific data flow probes

To configure IP ranges for tenant-specific data flow probes:

1. Sign in to UCMDB as an administrator using the company ID of the tenant whose data flow probe you want to configure.
2. Navigate to **Data Flow Management** > **Data Flow Probe Setup**.
3. Expand the data flow probe domain containing the probe you want to start. For example, **Customer2**.
4. Expand the Probe node and select the data flow probe you want to start. For example, **Probe2Customer2**.



5. Click the **Add IP range** button .
6. Type an IP range you want the Data Flow Probe to scan. Optionally, add any IP ranges you want to exclude.
7. Click **OK** to save the IP range.
8. Repeat step 1 to step 7 for each data flow probe you want to configure.

How to configure multi-tenancy for population

To support multi-tenancy for population, you need to configure the basicQueryCondition setting for each query defined in the population configuration file (`smPopConfFile.xml`).

To configure multi-tenancy for population:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files**.

3. Click the `smPopConfFile.xml` file.
4. For each query defined in this file, update the basicQueryCondition setting by adding the following:

   `and company= "{customerId}"`

   For example:

   `basicQueryCondition="type="bizservice"and company= "{customerId}""`
5. Save the configuration file.

> **Note**
>
> When you create or edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

1.6.3.3.4. Setting up the multi-tenancy integration in Service Manager

You need to perform the following tasks in Service Manager to set up the multi-tenancy integration.

Multi-tenancy support is an optional feature of the integration intended for Managed Service Providers (MSPs) who want to offer Configuration Management as a service to their tenants. In a multi-tenancy configuration, each CI and CIR record has a corresponding company ID. Out-of-the-box, Service Manager allows all operators to view CI data regardless of the company ID. If you wish to restrict access to CI data by company ID, you must enable Mandanten and use the company ID field as a restricting query. See the Service Manager help for more information about multi-company mode and Mandanten.

You must complete the following tasks from your Service Manager system to enable multi-tenancy support for the integration.

1. Start the process schedule.

   See How to start the process schedule.

2. Configure the Service Manager System Information Record and Integration Manager.

   See How to configure the Service Manager System Information Record and Integration Manager.

3. Add tenant-specific UCMDB ID and password values to company records (optional).

   See How to add tenant-specific UCMDB user ID and password values.

4. Add UCMDB Customer ID values to existing company records.

   See How to add UCMDB customer ID values to existing companies.

5. Synchronize existing company records with UCMDB.

   See How to synchronize existing companies from Service Manager to UCMDB.

6. Verify that Service Manager synchronized company records with UCMDB (optional).

   See How to view whether company information is in UCMDB.

7. Resynchronize existing company records with UCMDB (as needed).

   See How to resynchronize an existing Ccompany with UCMDB.

8. Inactivate company records you don't want to be part of the integration (as needed).

   See How to inactivate a synchronized company.

9. Reactivate inactive company records you want to be part of the integration (as needed).

See How to reactivate an inactive company

10. Add tenant-specific DEM rules.

    See How to add tenant-specific DEM rules.

1.6.3.4. Standards and best practices

This section provides best practices for using the integration.

UCMDB-SM configuration best practices

Bi-directional data synchronization recommendations

Push scheduling recommendations

Push in clustered environments

Initial load configurations

How to configure differential/delta load DEM rules

Fault detection and recovery for push

How to enable Lightweight Single Sign-On (LW-SSO) configuration

Frequently asked questions

When is a new CI created in Service Manager

Can I analyze the reason for a CI deletion in SM

How do I monitor relationship changes between UCMDB and SM

What kinds of relationships are pushed from UCMDB to SM

What's a root CI node

What's a root relationship

What's the "friendlyType" specified in an XSLT file

What's the "Virtual-Compound" relationship rype used in a UCMDB-SM integration query

When do I need the population feature

Can I populate physically deleted CIs from SM to UCMDB

How do I keep the outage dependency setting of a CI relationship in SM

How do I create an XSL transformation file

How do I use the Load Fields button to add multiple managed fields

What's the purpose of the <container> element in a population XSLT file

Can I populate sub-item deletions

What will happen if a population job fails or succeeds with warnings

Known issues and limitations

1.6.3.4.1. UCMDB-SM configuration best practices

This section provides best practices and recommendations for successfully implementing this integration in various environments. This section provides you with valuable understandings and techniques that will enhance the UCMDB-SM integration as well as solve common problems by providing solutions and workarounds to these issues. These practices and recommendations may vary slightly according to each implementation, as the specific system requirements and settings alter per system environment.

This section includes:

- Bi-directional data synchronization recommendations
- Push scheduling recommendations
- Push in clustered environments
- Initial load configurations
- How to configure differential/delta load DEM rules
- Fault detection and recovery for push
- How to enable Lightweight Single Sign-On (LW-SSO) configuration

1.6.3.4.1.1. Bi-directional data synchronization recommendations

UCMDB-SM integration supports bi-directional data synchronization between UCMDB and Service Manager (SM). We recommend that you follow the following best practices to avoid unnecessary problems due to improper use of the data push and population features:

- For CIs/CI Relationships that UCMDB can automatically discover, use UCMDB as the data source. Do not make changes to them in Service Manager, and always let UCMDB discover their changes and push the changes to SM.
- For CIs/CI Relationships that UCMDB can't automatically discover, use SM as the data source. Do not make changes to them in UCMDB, and always make changes to them in SM and populate UCMDB with the changes.
- For CIs/CI Relationships that are already created in SM and UCMDB can automatically discover, run a one-time population to synchronize them to UCMDB, and then use UCMDB as their data source.

Caution

Problems like the following may occur if you don't follow these best practices.

Problem 1:

[Population Adapter] After CIs/CI Relationships are pushed from UCMDB, if you directly make changes in SM to these records without ever populating them back to UCMDB first, the changes may not be populated to UCMDB.

Workaround:

Changing these UCMDB records in SM is not recommended; however if you need to do so you can do the following to solve this issue: After the records are pushed to SM, populate them back to UCMDB first before making any changes to them in SM. This way the changes can then be populated to UCMDB.

Problem 2:

[Population Adapter] After a Composition relationship between a Node CI (node 1) and Running Software CI is pushed to SM, if you change the upstream CI of the relationship from node 1 to node 2 in SM and then run a change population to populate this change, the Running Software CI will be removed in UCMDB.

Workaround:

It is recommended that you remove the running software in UCMDB and create a new one instead of directly replacing the container of the running software in SM. If you can't avoid doing so, do the following:

After you change the upstream CI of the relationship from node 1 to node 2, don't directly run the change population. Follow these steps to avoid this issue:

1. Update the Running Software CI in SM (or simply save it to mark it as updated).
2. Run a Running Software CI change population. This will create a new Composition relationship between node 2 and this Running Software CI.
3. Run a change population to synchronize the relationship change to UCMDB. The relationship between node 1 and the Running Software CI will be removed, and the new relationship created in step 2 will remain.

If you have run the change population after changing the upstream CI of the relationship from node 1 to node 2, and as a result the Running Software CI has been removed in UCMDB, follow these steps to solve this issue:

1. Update the Running Software CI in SM (or simply save it to mark it as updated).
2. Run a Running Software CI change population. This will create the Running Software CI, and a new Composition relationship between node 2 and this Running Software CI.

1.6.3.4.1.2. Push scheduling recommendations

Push jobs are run using two main methods, the first method is by manually executing the push job and the second is by scheduling the push job.

All push jobs can potentially produce a strain on the UCMDB and SM systems therefore; We recommend that you adhere to the following guidelines:

Scheduler time frames

It is important for you to understand the function of the Scheduler “time frame” concept. Running push jobs creates an increase in system activity and may affect application responsiveness. In order to enable users to effectively interact with applications We recommend the following guidelines:

In order to reduce system strain, schedule the UCMDB to SM push to run at non-peak usage hours, preferably when system usage is at a minimum.

Scheduler frequency

It is important to be aware of the business requirements when configuring the schedule frequency. The scheduler frequency depends on infrastructure environment changes that must be synchronized between UCMDB and SM.

Define the scheduling frequency based on the business requirements for consuming up-to-date CI information. Most implementations require a daily update. When scheduling small IT systems that are prone to frequent changes, the scheduling frequency may need to be increased.

Push Job dependencies

UCMDB Push Jobs don't support dependencies between each other. Each “Push Job” is considered a separate task and users can't define job dependencies. For example, that one job is dependent on another or upon completion before the next job is run.

It is important that both CI queries and their dependent Relationship queries exist in the same Job in order to avoid relationships not being pushed to Service Manager. UCMDB always pushes the CI queries before their dependent Relationship queries.



Note

Suffix usage

The out-of-the-box job for CI push uses queries that end with the “Push” suffix, for example, “SM Computer Push”.

Queries that create relationships between the CIs uses an extra “Relations” suffix, for example, “SM Node Relations Push”.

1.6.3.4.1.3. Push in clustered environments

A clustered SM environment is comprised of multiple servlets running in parallel with a load balancer that dispatches user requests to any available servlet. You must configure the UCMDB-SM integration to point to a specific servlet and not to the SM loadBalancer. In order to perform this, you must first create a dedicated web service listener.

Dedicated Web Services

A Service Manager system configured for vertical or horizontal scaling uses a load balancer to redirect client connection requests to an available SM process. However, most Web Services clients can't handle a redirect request and will fail if they use the SM load balancer as the endpoint URL.

We recommend creating one or more SM processes dedicated to Web Services requests. The user must configure the relevant external web service clients to connect directly to the dedicated Service Manager processes.

Step-by-step cluster configuration process

How to configure web clients

To configure the relevant external web clients:

1. Stop the Service Manager service.
2. Open the `sm.cfg` file, and create a dedicated SM process to listen for Web Services requests using the `-debugnode` parameter.

   The following entries create a dedicated process listening on ports 13085 and 13445.

   ```
   01 sm -httpPort:13080 -loadbalancer
   02 sm -httpPort:13081 -httpsPort:13443
   03 sm -httpPort:13083 -httpsPort:13444
   04 sm -httpPort:13085 -httpsPort:13445 -debugnode
   ```

   Explanation

   The code excerpt illustrates the various settings for each of the SM process listeners (web services) that enable SM clients to connect to the SM service.

   Line 01 defines the load balancer port (13080).

   Lines 02 and 03 define the SM ports to which non-dedicated SM clients are redirected by the SM load balancer.

   Line 04 defines the debugnode port that's utilized by the dedicated SM clients.

Note

Debugnode parameter

The debugnode parameter tells the SM load balancer not to forward any client connection requests to this Service Manager process. Only clients that directly connect to the process can access this port.

How to configure the debugnode

To configure the debugnode:

1. Start the SM service.
2. Configure any external web service clients to connect directly to the SM processes running in debugnode. When performing an integration using UCMDB, the UCMDBService Manager Adapter for SM should be configured to connect to the debugnode port.

   For example, for normal connections set the endpoint URL to:

   `http://<fully qualified host name>:13085/SM/7/<Service Name>`

   and for SSL-encrypted connections set the URL to:

   `https://<fully qualified host name>:13445/SM/7/<Service Name>` .

   These clients may include UCMDB (for push purposes), Connect-It, and additional applications.

Connecting to multiple SM processes

If you want to have better performance, you can connect to multiple Service Manager processes. The integration supports in both Service Manager vertical or horizontal load balancer environment.

You can create more than one SM processes dedicated to Web Services requests, and configure the field URL Override of integration point with the dedicated SM processes. This field value (if any) overrides the Hostname/IP and Port settings.

The following is an example value of this field, which connects two SM processes:

`http://<fully qualified host name1>:13080/SM/7/ws;http://<fully qualified host name2>:13082/SM/7/ws`

1.6.3.4.1.4. Initial load configurations

Before the configuration process can begin, you must first assess the amount of CI and relationships data to be transferred from UCMDB to SM and ascertain the iteration process that's required based on the volume.

You must first assess whether all of the data can be pushed in a single iteration. This is ascertained by the amount of data that's included in the push queries and the amount of time you have to push this data.

Note

The performance data presented in this document is based on tests that were performed in the lab and is provided for reference only. The integration performance may significantly differ in your environment depending on your hardware configuration.

Push performance in a single-threaded environment

The Push of 22,500 UCMDB root CIs (roots in queries) and/or Relationships in a single-threaded environment takes about an hour and is performed in a linear fashion. See the following table:

Performance data in a single-threaded environment

Number of root CIs/CI Relationships pushed per hour	Multi-threading settings in sm.properties

22,500	`number.of.concurent.sending.threads=1`
	`min.objects.for.concurent.sending=50`
	`number.of.chunks.per.thread=3`

To view or edit the sm.properties file in UCMDB, navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files** > **sm.properties**.

Number of Root CIs and Relationships/22,500

The push time (in hours) in any given environment is calculated as follows:

If the push of a single planned query has the potential of breaching the permitted time frame the data must be divided into several queries. Each query must be pushed individually.

This query division is performed by creating several queries, each with different node conditions that enable data filtering. Once all queries are pushed for the first time, the Initial Load process is complete.



Note

When applying node conditions to the various SM Sync Queries, you must make sure that all of the information is included in the queries, so that all relevant data is copied to SM.

Implementing multi-threading

In order to improve performance, the Service Manager Adapter utilizes multiple threads for the push of CI and Relationship data to SM. The following section explains these settings and how to configure them for maximum performance.

The multi-threading configuration is defined in the sm.properties file on the UCMDB server. To view or edit the file in UCMDB, navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files** > **sm.properties**.

The following are example multi-threading definitions in the sm.properties file:

```
01 number.of.concurent.sending.threads=6
02 min.objects.for.concurent.sending=50
03 number.of.chunks.per.thread=3
```

Explanation

The code excerpt illustrates the relevant multi-threading settings on the UCMDB server.

- Line 01 defines the number of parallel threads UCMDB will open to SM for CI push. Setting this parameter to 1 disables multi-threading, while a values of 2 or higher enables multi-threading.
- Line 02 defines the minimum number of SM objects needed to use multiple threads as opposed to a single thread.
- Line 03 defines the number of chunks per thread. This number multiplied by the number of threads gives you the total number of CI data chunks.
- The total number of chunks = number.of.chunks.per.thread * number.of.concurent.sending.threads

The integration implements a queue mechanism as follows:

The data passed from UCMDB to SM is divided into equal chunks, and these chunks are placed in a queue.

Each available thread pulls the next chunk from the queue until all threads are available. Once this process has completed, the push is complete.

The mechanism is designed to minimize the idle time of each thread. As each thread processes its chunk in parallel, some threads may finish before others and it is inefficient for them to wait for each other.

Caution

Defining too many threads

It is ineffective to over-increase the number of threads as this causes the SM server to overload. In enterprise environments where the SM server processing the push data is very robust the number of threads can be increased to 10 and in some cases even 20; however, you must take into account that increasing the number of threads raises CPU usage on the SM server during push, which may reduce the application performance.

Push performance in multi-threaded environments

The push of 60,000 UCMDB root CIs (roots in queries) and/or Relationships in an out-of-the-box multi-threaded environment takes about an hour and is performed in a linear fashion. See the following table:

Performance data in an out-of-the-box multi-threaded environment

Number of root CIs/Relationships pushed per hour	Multi-threading settings in sm.properties (default)

60,000	`number.of.concurent.sending.threads=6`
	`min.objects.for.concurent.sending=50`
	`number.of.chunks.per.thread=3`

The push time (in hours) in any given environment is calculated as follows:

Number of Root CIs and Relationships/60,000

Push performance in multiple SM processes environments

The Push of 190,000 UCMDB root CIs (roots in queries) and/or Relationships in a multi-threaded environment with multiple SM processes takes about an hour and is performed in a linear fashion. See the following table:

Performance data in a multiple SM processes environment

Number of root CIs/Relationships pushed per hour	SM processes	Multi-threading settings in sm.properties (default)

190,000	2 server hosts, with each host running 3 processes	`number.of.concurent.sending.threads=90`
		`min.objects.for.concurent.sending=50`
		`number.of.chunks.per.thread=3`

For more information about defining multiple SM processes for the integration, see How to create an integration point in UCMDB.

The push time (in hours) in any given environment is calculated as follows:

Number of Root CIs and Relationships/190,000

How to set up SM DEM rules for initial loads

SM Discovered Event Manager Rules (DEM Rules) enable the user to define the appropriate action to take for each event type that's reported to SM.

Each CI and relationship record pushed from UCMDB to SM is analyzed against the existing SM records and open Change requests. SM rules define the appropriate action to be taken for each type of CI data update sent to SM.

To view or update the SM Discovered Event Manager Rules:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Press **ENTER** or click the **Search** button.

   A list of all the Discovered Event Manager Rules is displayed. Each rule is usually linked to a CI Type or a subset of CIs of the same type.
4. Click on the individual CI Discovered Event Manager Rule to view its details.

To set up DEM Rules for initial loads:



Tip

When performing “Initial Loads”, we recommend setting the SM Discovered Event Manager Rules to add newly reported CIs as described below. This minimizes the “noise” of an Initial Load that may potentially create tens of thousands of Changes/Incidents.

For each of the Discovered Event Manager Rules, perform the following steps:

1. Select the relevant Discovered Event Manager Rule.
2. Go to the “Action if matching record doesn't exist” section, select the **Add the record** option.
3. In the “Action if record doesn't exist but unexpected data discovered” section, select the **Log Results and Update Record** option.
4. In the “Action if record is to be deleted” section, select the **Delete Record** option.
5. Save the Discovered Event Manager Rule record.

1.6.3.4.1.5. How to configure differential/delta load DEM rules

Tip: Once the Initial Load or Data Load of the CI data has completed, it's advisable to apply Differential/Delta Load settings. These settings apply to all data loaded from UCMDB to SM. These loads send only updates regarding modifications discovered in the IT infrastructure from UCMDB to SM.

To set up the SM Discovered Event Manager (DEM) Rules for Differential/Delta Loads:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Press Enter or click the **Search** button.

   Service Manager displays a list of all the Discovered Event Manager Rules.
4. For each of the Discovered Event Manager Rules, perform the following steps:
    1. Select the relevant Discovered Event Manager Rule.
    2. In the **Action if matching record doesn't exist** section, select the appropriate action required for each newly detected CI. If uncertain, select the **Add the record** option.
    3. In the **Action if record does exist but unexpected data discovered** section, select the appropriate action for each CI that was already modified, resulting in an unexpected or wrong result. The recommended best practice is to select the **Open a Change** option.
    4. In the **Action if record is to be deleted** section, select the appropriate action required for each CI that was already removed/deleted. The recommended best practice is to select the **Delete Record** option for CI Relationship, and select the **Update record to the selected status** option for CI.
    5. Save the Discovered Event Manager Rule record.

1.6.3.4.1.6. Fault detection and recovery for push

Universal CMDB provides a fault detection and recovery mechanism since version 9.05: individual CI failures no longer cause the entire push to fail, and you can review all failed CIs in the Universal CMDB studio and then re-push them.

1.6.3.4.1.7. How to enable Lightweight Single Sign-On (LW-SSO) configuration

You can enable LW-SSO for the integration so that users can directly view UCMDB CI records from the Service Manager web client by clicking the **View in UCMDB UI** button, without providing a UCMDB UI username and password.



Note

LW-SSO is unsupported for the Service Manager Windows client.

To enable LW-SSO for the integration:

1. For each Service Manager user account that needs LW-SSO, create a user account in UCMDB with the same username. The passwords in the two systems can be different.
2. Enable LW-SSO in the Service Manager Web tier. For details, see Configure LW-SSO on the Service Manager Web tier.
3. Enable LW-SSO in UCMDB. For details, see the *Enable login to UCMDB with LW-SSO* topic in the Universal Discovery and CMDB Help Center.

1.6.3.4.2. Frequently asked questions

This section provides answers to frequently asked questions about the UCMDB-SM integration.

This section includes:

- When is a new CI created in Service Manager
- Can I analyze the reason for a CI deletion in SM
- How do I monitor relationship changes Between UCMDB and SM
- What kinds of relationships are pushed from UCMDB to SM
- What's a root CI node
- What's a root relationship
- What's the “friendlyType” specified in an XSLT file
- What's the “Virtual-Compound” relationship type used in a UCMDB-SM integration query
- When do I need the population feature?
- Can I populate physically deleted CIs from SM to UCMDB
- How do I keep the outage dependency setting of a CI relationship in SM
- How do I create an XSL transformation file
- How do I use the Load Fields button to add multiple managed fields
- What's the purpose of the <container> element in a population XSLT file
- Can I populate sub-item deletions
- What will happen if a population job fails or succeeds with warnings

1.6.3.4.2.1. When is a new CI created in Service Manager?

CIs are created in SM under the following circumstances:

- A CI is manually added to SM through the Configuration Management module or the Request module.
- UCMDB reports a newly discovered CI according to the following:
  - When a new CI is reported and the “Discovered Event Manager Rules” are set to “Add the Record”.
  - When a new CI is reported, the “Discovered Event Manager Rules” are set to “Open an Incident” and the Incident has been closed.
  - When a new CI is reported, the “Discovered Event Manager Rules” are set to “Open a Change” and the Change has been verified.

1.6.3.4.2.2. Can I analyze the reason for a CI deletion in SM?

No.

SM opens a change request on the deleted CI and includes the following information:

“Delete event for CI “CI Name” triggered by discovery”.

Workaround

An SM change request doesn't contain a description of the reason for deletion, however it is possible to extract specific information about CI deletions from the UCMDB “History Database”. UCMDB data provides information about the user or the discovery pattern that initiated the CI deletion.

1.6.3.4.2.3. How do I monitor relationship changes between UCMDB and SM?

To understand the relationship change in SM, a distinction must be made between the various types of Relationship Changes:

- The second endpoint of the relationship has changed, so instead of CI X being linked to CI Y through a relationship, now CI X is related to CI Z.
- An attribute of a relationship has changed.

The first type of Relationship change is supported by the UCMDB-SM integration, therefore, such “Relationship Changes” can either invoke CI relationship updates or perform the creation of Incidents or Changes, which are then reviewed and monitored.

The second is also supported, but it's not covered out-of-the-box; you can configure the Universal CMDB query to expose such attributes of the relationship, and configure the Service Manager WSDL to expose the mapped field, and then configure the adapter mapping configuration in the XSLT. However, such Relationship Attribute Changes can't perform the creation of Incidents or Changes, and only supports invoking CI relationship updates directly.

1.6.3.4.2.4. What kinds of relationships are pushed from UCMDB to SM?

Any kinds of “Relationships” are pushed from UCMDB to SM under the following conditions:

- The relationship appears in a “Push Query” located in the “SM Sync” folder in the UCMDB Query Manager.
- The relationship is named “Root” in the Push Query.
- The relationship is mapped to an appropriate target in SM in the UCMDB configuration files (XML and XSLT files).

The out-of-the-box relationships that are pushed from UCMDB to SM are relationships between two CIs such as:

- Between Business Services and Applications;
- Between Business Service and Host;
- Between an Application and a Network Component; or
- Between Host, Network Components and Printers.

1.6.3.4.2.5. What's a Root CI Node

A “Root” Node is a TQL Node that represents the CI type that's created through push to SM from the TQL structure. The rest of the TQL structure contains information that can be incorporated within the “Root” CI type and is used to enrich the record in SM with additional information and or attributes.

1.6.3.4.2.6. What's a Root relationship

A “Root” relationship is a relationship within a query and created in SM through push. It represents a Relationship between two Root CIs. Only the relationships marked with “Root” are pushed to SM.

1.6.3.4.2.7. What's the "friendlyType" specified in an XSLT file

friendlyType is another name for the UCMDB “CI Type”. These “friendly-types” are usually pushed into the SM Device subtype field/attribute.

1.6.3.4.2.8. What's the “Virtual-Compound” relationship type used in a UCMDB-SM integration query?

When more than two UCMDB CI entities are connected in series, the “Virtual-Compound” represents the relationship between the first and last entities. This is a virtual relationship, as no physical representation exists.

The “Virtual-Compound” relationship type is a relationship that links two CI type entities that have a logical relationship. See the following figure:



Explanation

The illustration shows an example of a Virtual-Compound relationship. The relationship in SM is created directly between the “Host” and the “Business Service”.

1.6.3.4.2.9. When do I need the population feature?

You need the population feature under any of the following circumstances:

- You have done modeling in SM, especially when you are in the planning and design phases, and you want your models to be reflected in UCMDB;
- You want to implement the UCMDB-SM integration, however you have already invested in your SM CMDB and don't want to lose that investment;
- You want to continue to maintain some parts of the SM CMDB while maturing your UCMDB/Discovery implementation.

1.6.3.4.2.10. Can I populate physically deleted CIs from SM to UCMDB?

No.

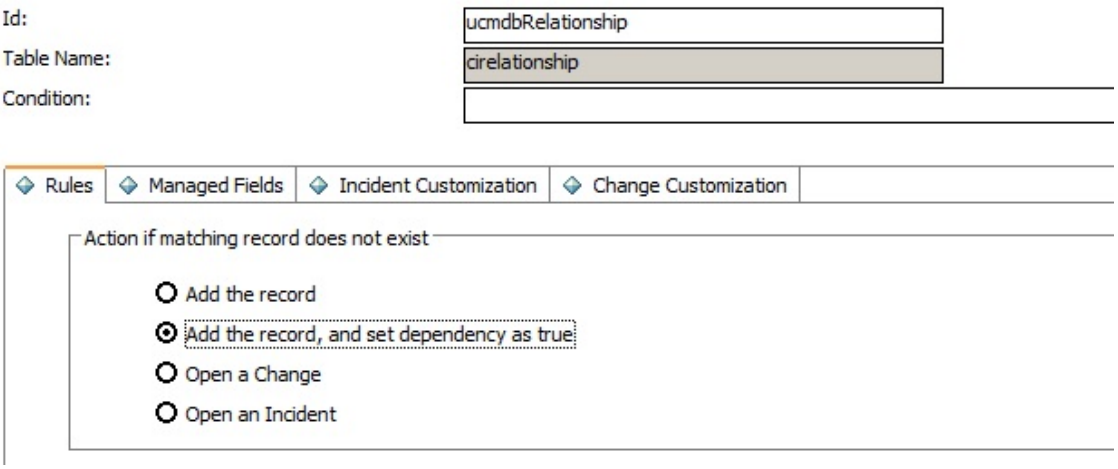
Physical deletions of CIs are allowed in SM, but SM can't get such “deletion changes” and the population feature will not synchronize such changes to UCMDB.

Physical deletions of CIs can be considered as exceptions, which only occur after you create CIs by mistake. Normally, you delete a CI by setting its status to something like “Retired/Consumed”. If such CIs have been populated to UCMDB, it is your responsibility to remove them manually from UCMDB.

1.6.3.4.2.11. How do I keep the outage dependency setting of a CI relationship in SM?

Out-of-the-box, CI relationships that are pushed from UCMDB to SM don't have outage dependency information by default. If you need such information, you can set the Discovered Event Manager (DEM) rule of the CI Relationship Web Services Description Language (WSDL) as follows:

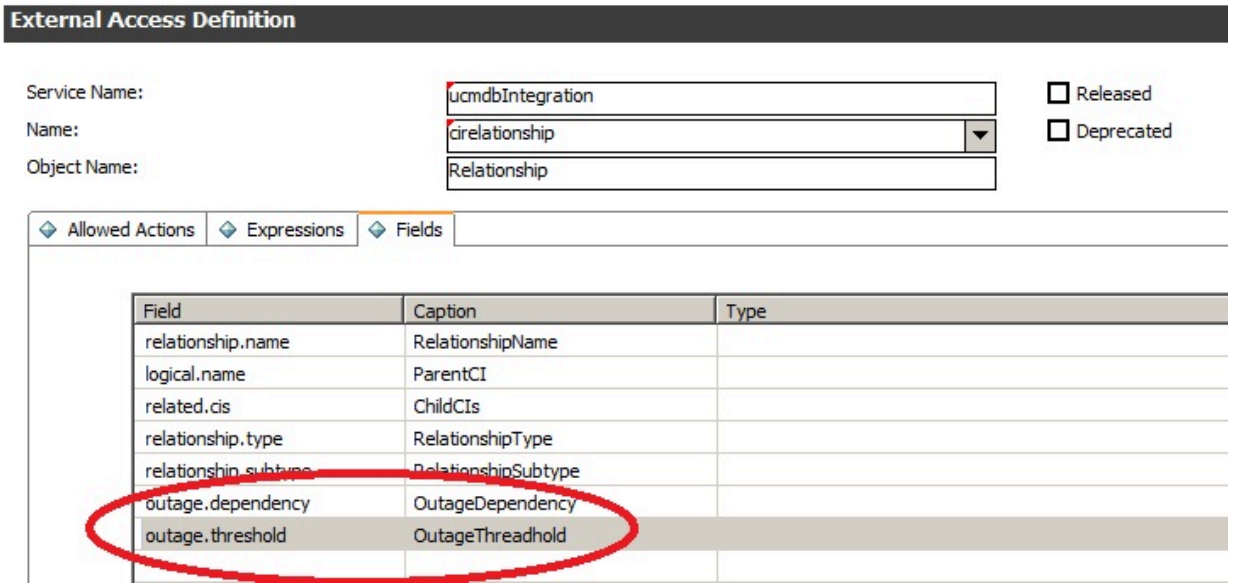
1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. Open the **ucmdbRelationship** record.
4. On the Rules tab, select **Add the record, and set dependency as true**.



   This will set the Outage Dependency of each CI Relationship to true, and set the number of dependent downstream CIs to 1 (because UCMDB only supports one-to-one relationships).

Suppose you want to set outage dependency only for some relationships. For example, if you want to configure outage dependency for relationships that starts from Business Service, you can configure the adapter configuration file (`XSLT`) and WSDL definition. You can also configure outage dependency per relationship type (UCMDB query).

1. In the WSDL definition, expose fields **outage.dependency** and **outage.threshold**.



2. In the `XSLT` file, set the exposed outage fields. For example, if you want to set the outage dependency to true and threshold to 1 for Business Service relationships, you simply need to change the mapping of query and `XSLT` in the `smSyncConfFile.xml` file:

```
<tql name="applicationRelationsData"
xslFile="bizservice_relations.xslt">
        <request type="Create" name="CreateRelationship"/>
        <request type="Update" name="UpdateRelationship"/>
        <request type="Delete" name="DeleteRelationship"/>
</tql>
```

3. In the `bizservice_relations.xslt` file, use the following `OutageDependency` and `OutageThreadhold` settings:

```
<xsl:stylesheet version='1.0'
xmlns:xsl='http://www.w3.org/1999/XSL/Transform'>
   <xsl:template match="/relation">
      <model>
         <keys/>
            <instance>
               <ParentCI><xsl:value-of select="@parentID"/>
</ParentCI>
                  <ChildCIs>
                     <ChildCIs><xsl:value-of select="@childID"/>
</ChildCIs>
                  </ChildCIs>
                  <RelationshipType>Logical</RelationshipType>
                  <RelationshipSubtype><xsl:value-of
select="@friendlyType"/></RelationshipSubtype>
                  <OutageDependency>true</OutageDependency>
                  <OutageThreadhold>1</OutageThreadhold>
            </instance>
         </model>
      </xsl:template>
</xsl:stylesheet>
```

1.6.3.4.2.12. How do I create an XSL transformation file

You create an XSL transformation file in Adapter Management. You can copy the content of an existing XSL transformation file to the new file and then make necessary edits.

To create an XSL transformation file:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management**. **ServiceManagerAdapter9.41** > **Configuration Files**.
3. Click the **Create new resource** button  .
4. Select **New Configuration File**.
5. Enter a name for the file. The file name should use this format: <AdapterID>/<filename>. For example: ServiceManagerAdapter9.41/test_relation_population.xslt .
6. In the Package field, select the adapter name. For example, ServiceManagerAdapter9.41 .
7. Click **OK**. The system displays a file extension warning dialog.
8. Click **Yes** to continue.

   UCMDB creates the new XSL transformation file in the Configuration Files folder of the adapter. For example, **ServiceManagerAdapter9.41 > Configuration Files > ServiceManagerAdapter9.41/test_relation_population.xslt**.
9. Copy the content of an existing XSL transformation file to the new file. For example, for population you can copy the content of an out-of-the-box population XSL transformation file.
10. Make necessary edits to the new file.

> **Caution**
>
> **Invalid XML**
>
> When removing XSL elements from an XSLT file, keep in mind that the remaining XML should be a valid XML file. The system uses these remaining XML to translate the UCMDB Query Definition.

Sample usages of XSLT

For your reference, the following are some samples of using XSLT functionalities in this integration.

Using choose to set CI subtypes:

```
<Type>switch</Type>
<xsl:variable name="prefix" select="'Value&gt;'"/>
<xsl:variable name="suffix" select="'&lt;/Value'"/>
<Subtype>
  <xsl:choose>
    <xsl:when
test="contains(@node_role,concat($prefix,'atm_switch',$suffix))">ATM
Switch</xsl:when>
    <xsl:when
test="contains(@node_role,concat($prefix,'frame_relay_switch',$suffix))
">Frame Relay Switch</xsl:when>
    <xsl:when
test="contains(@node_role,concat($prefix,'lan_switch',$suffix))">Lan
Switch</xsl:when>
    <xsl:otherwise><xsl:value-of select="@friendlyType"/>
</xsl:otherwise>
  </xsl:choose>
</Subtype>
```

Getting the `substrings` from a string:

```
<xsl:variable name="calculatedLocation" select="@calculated_location"/>
<Building>
  <xsl:value-of select="substring-after($calculatedLocation,'
Building:')"/>
</Building>
<Floor>
  <xsl:value-of select="substring-before(substring-
after($calculatedLocation,'Floor:'),' Building:')"/>
</Floor>
<Room>
  <xsl:value-of select="substring-before(substring-
after($calculatedLocation,'Room:'),' Floor:')"/>
</Room>
```

Reading the value mappings from an XML file:

The `SM_CIT_Subtype_list.xml` file defines the value mappings for subtypes:

```
<lists>
  <list name="CIType_bizservice">
    <entry ucmdb="BusinessApplication" sm="Application Service"/>
    <entry ucmdb="BusinessService" sm="Business Service"/>
    <entry ucmdb="InfrastructureService" sm="Infrastructure Service"/>
  </list>
</lists>
```

The `business_service_push.xslt` file uses this mapping definition XML file:

```
<xsl:variable name="CIlists"
select="document('SM_CIT_Subtype_list.xml')/lists"/>
<xsl:variable name="CIT" select="@bdmType"/>
<xsl:for-each select="$CIlists/list[@name='CIType_bizservice']">
  <Subtype><xsl:value-of select="entry[@ucmdb=$CIT]/@sm"/></Subtype>
</xsl:for-each>
```



Tip

For more information about `XSL` transformations, visit the `https://www.w3schools.com/xml/` site.

1.6.3.4.2.13. How do I use the Load Fields button to add multiple managed fields?

Service Manager stores the list of managed fields in the **ucmdbIntegration** web service, which consists of a number of web services objects. You can add more managed fields to DEM Rules so that Service Manager can monitor changes in more CI attributes in UCMDB and trigger the actions defined in relevant DEM Rules.

You can manually add managed fields that are exposed in associated WSDL definitions to DEM Rules; however, you can use the **Load Fields** button to automatically (and therefore correctly) add managed fields to DEM Rules.

1. Click the **Managed Fields** tab of the DEM Rule.
2. Click the **Load Fields** button.
3. If the table (in the Table Name field) of the DEM rule record has only one WSDL definition associated to it, all fields exposed in the WSDL definition are immediately added to the Managed Fields list.

   A message displays: `<XX> new fields loaded.`
4. If the table has more than one WSDL definition associated to it, the Managed Fields Importing wizard opens, and a list of WSDL definitions (ucmdbIntegration web service objects) is displayed.
   1. Select one or more objects, and click **Next**.
   2. All new fields that can be added from the selected web service objects are displayed.
   3. If you want to add all of the fields, click **Finish**; if you want to ignore some of them, change their Action value from **Add** to **Ignore**, and then click **Finish**.

      A message displays: `<XX> new fields loaded.`
5. Save the DEM Rule record.

1.6.3.4.2.14. What's the purpose of the <container> Element in a population XSLT file

Out-of-the-box, in the `runningsoftware_population.xslt` file, there is a `<container>` element:

```
<link direction="incomming" linkType="composition">
     <ci class="node">
         <container tql="SM Computer Population"
keyFields="CIIdentifier">
             <linkTql>SM Computer To Software With
Composition</linkTql>
             <linkRetrieveCondition>downstreamci.logical.name=&amp;quot;
<xsl:value-of select="CIIdentifier"/>
&amp;quot;</linkRetrieveCondition>

<linkValueFields>upstreamci.logical.name</linkValueFields>
          </container>
      </ci>
</link>
```

- In UCMDB, RunningSoftware CIs must exist together with a Root Container (Node). However, Service Manager allows RunningSoftware CIs without a Node.
- The current integration adapter synchronizes CIs and Relationships separately. When populating a RunningSoftware CI, the integration has no chance to check if a relationship exists between the CI and a Node.

With the `<container>` element, the integration populates RunningSoftware CIs together with a container.

1.6.3.4.2.15. Can I populate sub-item deletions?

Yes.

Service Manager and UCMDB store CI information in different data structures, and therefore one SM CI may be synchronized to UCMDB as several CIs. For example, during population, an SM computer CI record is synchronized to a Node CI in UCMDB, and the computer CI's attributes to CIs such as IP, Interface, Location, etc (which are referred to as sub-items of the Node CI.). In this case, the Node CI is the root CI.

The integration allows you to populate sub-item deletions to UCMDB (for example, if you delete the IP Address attribute value of a computer, the corresponding IP CI record in UCMDB will be deleted too). To do so, you need to specify a root in the CI type's population xslt file, using parameter isRoot="true". For example, for Computer CIs, the "node" should be specifed as the root.

Out-of-the-box, only the computer_population.xslt file requires the "node" CI be specified as the root, as shown below:

```
<ci class="node" isRoot="true">
```

1.6.3.4.2.16. What will happen if a population job fails or succeeds with warnings?

When a population job fails:

The failure prevents the remaining population tasks from running. The next job run will start from the last Success time. If pagination occurs (that is, the tasks are divided into multiple pages), the tasks will run again and again within the first page from the last “Success” time (once the end of the first page is reached, no new tasks will be executed).

When a population job succeeds with warnings:

A warning doesn't prevent the remaining population tasks from running. The next job run will run all tasks again starting from the last Success time. If pagination occurs (the tasks are divided into multiple pages), the tasks on all pages will be re-run (including those that were successfully completed last time).

1.6.3.4.3. Known issues and limitations

The following table lists the known issues and limitations of this integration.

Known Issues and Limitations

Global ID	Known Issue/Limitation	Workaround
QCCR1E72246	[Population Adapter] Cannot populate deleted CIs in a full population. While you run a full population, if there are CIs that have been deleted since the last job run in Service Manager, the deleted CIs in Service Manager will not be populated to UCMDB. However, the data push feature first pushes deleted CIs while running a full push.	In UCMDB, manually delete all CIs that were populated from SM by using appropriate filter criteria and then run a full population. **Warning**: You must be very careful when using this workaround.

Global ID	Known Issue/Limitation	Workaround
QCCR1E72222	NodeRole is case-sensitive when mapping Node's NodeRole in UCMDB to Node's Subtype in SM. Steps to reproduce this issue: 1. Create a Node in UCMDB. 2. Set the Node's NodeRole property to "Printer". 3. Run a data push in UCMDB to push Node CIs. 4. Check the CI's attributes in SM. **Result**: The CI's subtype is "Node", which should be "Net Printer". When the NodeRole is "printer", the result is correct.	Use lower-case names for NodeRole (printer).

Global ID	Known Issue/Limitation	Workaround
QCCR1E72327	When you run a full population for the first time to synchronize a large amount of CIs, if some of the CIs have an invalid attribute value (for example, IP address), the population job will ignore such CIs and run to completion with a Failed status and with errors logged on the Job Errors tab; In addition, the last job execution time will not be logged because the job is run for the first time but fails. As a result, after fixing the invalid attribute values in Service Manager, you will not have a chance to re-populate the failed CIs through a change (delta) population. This is because when you run a change population, a full population will be executed instead (because the last job execution time is null).	Not available at this time.

Global ID	Known Issue/Limitation	Workaround
QCCR1E72511	If you set the `use.global.id parameter` in the adapter's `sm.properties` file to True, the federation feature doesn't work.	Set the parameter to False to solve this issue. • When you deploy the adapter on the UCMDB standalone server (which is defined as a global-id generator by default), globalId and ucmdbId are the same thing. For this reason, setting this parameter to false can meet all of your needs satisfied by setting the parameter to true. • When you deploy the adapter on a non-standalone UCMDB (for example, APM's RTSM), setting this parameter to true is unsupported. In addition, we don't recommend integrating the non-standalone CMDB (for example, APM's RTSM) with Service Manager.
QCCR1E73004	Double quotes in UCMDB CI names (for example, "laptop - 003") are removed when the CIs are synchronized to SM.	Not available at this time.

1.6.3.5. Tailoring the integration

You can tailor the Universal CMDB integration to meet your business needs by adding or removing managed CI types, attributes, and relationship types. This section describes the integration architecture and tailoring options for data push, population, and federation.

Integration architecture

Integration class model

Integration queries

Service Manager Web Services

Service Manager reconciliation rules

Service Manager Discovery Event Manager rules

Integration tailoring options

How to enable ASM support

How to update the integration adapter configuration file (sm.properties)

How to add DEM reconciliation rules

Add Discovery Event Manager rules

How to add a CI attribute to the integration for data push

How to add a CI type to the integration for data push

How to add a CI type's relationship types to the integration for data push

How to add a CI attribute to the integration for population

How to add a CI type to the integration for population

How to add a CI type's relationship types to the integration for population

How to customize UCMDB ID pushback for a CI type

How to add custom queries to integration population jobs

How to add an attribute of a supported CI type for federation

1.6.3.5.1. Integration architecture

Before you tailor the integration, you should understand how the following components of the out-of-the-box integration work.

- Integration class model
- Integration queries
- Service Manager Web Services
- Service Manager reconciliation rules
- Service Manager Discovery Event Manager rules

1.6.3.5.1.1. Integration class model

UCMDB 9.x or later no longer uses a private class model of CI types to manage integration CIs, as was required in prior versions. Instead, the integration uses the standard UCMDB managed objects and maps them to Service Manager CI types and attributes with queries and transformation files.

1.6.3.5.1.2. Integration queries

This section describes out-of-the-box queries used for data push, Actual State, and population.

Queries for push

For the push feature, the integration uses a collection of queries to gather CI attribute information from Universal CMDB and send it to the Service Manager system.

To access the out-of-the-box data push queries, navigate to **Modeling** > **Modeling Studio**, select **Queries** for Resource Type, and then navigate to the **Root** > **Integration** > **SM Sync** > **9.xx** folder.

If you want to change what CI/Relationship types or attributes are part of the integration, you must also edit the integration queries to support your updated CI/CI Relationship types and attributes.

Out-of-the-box queries for data push

Query name	Description
SM Running Software Push	This query gathers CI attributes from Running Software CIs.
SM Business Service Push	This query gathers CI attributes from business service CIs.
SM Business Service Relations Push	This query gathers relationships between the following components: • Business service and Running Software CIs • Business service and node CIs • Two or more business services The query includes compound relationships because the relationships can extend through a group.
SM Computer Push	This query gathers CI attributes from the node CI type with NodeRole containing “desktop”, “server”, “virtualized_system” or not set. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Switch Push	This query gathers CI attributes from the node CI type with NodeRole containing “atm_switch”, “frame_relay_switch”, or “lan_switch”. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.

Query name	Description
SM Storage Push	This query gathers CI attributes from the node CI type with NodeRole containing “san_switch”, “san_gateway”, “san_router” or exact CI type equal to “storage Array”. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Net Printer Push	This query gathers CI attributes from the node CI type with NodeRole containing “printer”. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Mainframe Push	This query gathers CI attributes from the node CI type with exact CI type equal to “Mainframe Logical Partition”, or “Mainframe CPC”. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Mobile Device Push	This query gathers CI attributes from the node CI type with NodeRole containing “pda_handheld”. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.

Query name	Description
SM Network Component Push	This query gathers CI attributes from the node CI type with NodeRole containing “router”, “adsl_modem”, “appletalk_gateway”, “bandwith_manager”, “cable_model”, “csu_dsu”, “ethernet”, “fddi”, “firewall”, “hub”, “kvm_switch”, “load_balancer”, “multicast_enabled_router”, “nat_router”, “token_ring”, “undefined_network_component”, “voice_gateway”, “voice_switch”, or “vpn_gateway”. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, and Location.
SM Cluster Push	This query gathers CI attributes from the node CI type with the exact CI type equal to “ClusterResourceGroup”. It also gathers related CI attributes from the following CI types through containers and links: IPAddress, Interface, CPU, FileSystem, DiskDevice, Location, and Cluster.
SM Node Relations Push	This query gathers relationships between the following components: • Node and Printer CIs • Node and RunningSoftware CIs The root class of the relationship is composition.

Query name	Description
SM CRG Relations Push	This query gathers relationships between the following components: Node and Cluster Resource Group CIs. The query includes compound relationships because the relationships can extend through a group.
SM Layer2 Topology Relations Push	This query gathers relationships between the following components: Two or more nodes. The query includes compound relationships because the relationships can extend through a group.
SM Local Printer Push	This query gathers CI attributes from printer CIs. It also gathers related CI attributes from the Node CI type.

Query name	Description
SM ASM Relations Push	This query gathers ASM discovered consumer-provider relationships that point from a Business Element CI to a Running Software CI. **Note** This query doesn't support consumer-provider relationships between a Business Element CI and a Cluster CI. **Note** To work correctly, this query requires the ASM Enhanced package to be deployed in UCMDB. This package is already included in UCMDB Content Pack (CP) 17.

Queries for Actual State

Out-of-the-box, the following queries (see the table below) are used for retrieving CI information from UCMDB to the Actual State section of the Service Manager Configuration Item (CI) form. Service Manager retrieves CI Actual State information by calling a UCMDB web service that retrieves CI data according to these queries.

The queries are located in the **Integration** > **SM Query** folder in the UCMDB Modeling Studio.

Out-of-the-box queries for Actual State

Query name	Description
localPrinterExtendedData	This query gathers real-time extended information from Printer CIs in UCMDB.
applicationExtendedData	This query gathers real-time extended information from RunningSoftware CIs in UCMDB.
businessServiceExtendedData	This query gathers real-time extended information from business service CIs in UCMDB.
hostExtendedData	This query gathers real-time extended information (such as Asset, Party, Location, LogicalVolume, WindowsService, Printer, InstalledSoftware, FileSystem, IPAddress, Interface, DiskDevice, and Cpu) from the node CI type in UCMDB.

Queries for population

For CI/CI Relationship population, the integration doesn't need queries to save CI/CI Relationship attribute information to Universal CMDB; however in the population configuration file the following query names are mapped to relevant population XSL transformation files (see the following table).

Queries for population

Query name	Description
SM Business Service Population	This query defines the CI store structure of business service CIs.
SM Computer To Computer With ConnectM Running Software Population	This query defines the CI store structure of running software CIs.
SM Computer Population	This query defines the CI store structure of computer CIs.
SM Biz To Biz With Containment	This query defines the CI store structure of CI relationships in which a bizservice CI contains another.
SM Biz To Biz With Usage	This query defines the CI store structure of CI relationships in which a bizservice CI uses another.
SM Biz To Computer With Containment	This query defines the CI store structure of CI relationships in which a bizservice CI contains a computer CI.
SM Biz To Computer With Usage	This query defines the CI store structure of CI relationships in which a bizservice CI uses a computer CI.
SM Biz To Software With Containment	This query defines CI store structure of CI relationships in which a bizservice CI contains a RunningSoftware CI.
SM Biz To Software With Usage	This query defines CI store structure of CI relationships in which a bizservice CI uses a RunningSoftware CI.
SM Computer To Computer With Connects	This query defines the CI store structure of CI relationships in which a computer CI connects to another.
SM Computer To Software With Composition	This query defines the CI store structure of CI relationships in which a RunningSoftware CI is contained within a computer CI and the RunningSoftware CI can't exist without the container.

Query name	Description
CLIP Down Time Population	This query defines the CI store structure of ScheduledDowntime CIs.
CI To Down Time CI With Connection	This query defines the CI store structure of CI relationships in which a ScheduledDowntime CI connects to an affected CI.

Query requirements

The integration requires that any custom queries you create meet certain formatting conditions. Any queries that you want to include in the integration must meet these conditions:

- To query CIs, a query must contain one CI type labeled **Root**. The Root node is the main CI that the UCMDB synchronizes. All other CIs are contained CIs of the Root CI.
- To query relationships, a query must contain one or more relationships labeled **Root**.
- A query must contain only the Root CI and CIs that are directly connected to it. The Root CI is always the top node of the TQL hierarchy.
- A TQL graph can't have cycles.
- If a query synchronizing relationships has cardinality, it must be cardinality **1...***. Additional cardinality entries must have an OR condition between them.
- If you only want the integration to synchronize specific CIs, you must configure the condition on the query to filter such CIs.

1.6.3.5.1.3. Service Manager Web Services

Service Manager uses web services messages to get and receive CI information from your UCMDB system. Out-of-the-box, UCMDB sends more CI attribute information than the Service Manager system actually manages. Service Manager users can view all of the CI attribute information the UCMDB system sends from the Actual State section of the CI record.

Service Manager publishes several web services for use by the UCMDB-SM integration. The UCMDB system uses the web services to map UCMDB CI types and CI attributes to web services objects the Service Manager system recognizes. If you add UCMDB CI types or CI attributes that you want Service Manager to manage, then you must update one or more of these web services to define them as web services objects. See Service Manager Web Services for more information about publishing web services.

Managed fields



Note

Managed fields are used only for the data push feature.

A Service Manager managed field is a field where the system compares the CI attribute value in the incoming UCMDB web services message to the value in a Service Manager CI record. If the values in the web services message don't match those in the CI record, Service Manager runs a Discovery Event Manager (DEM) rule to determine what action to take. The DEM rule determines which of the fields that are published as web services objects are fields managed by the integration. Only value changes in managed fields trigger the DEM rule.

The **ucmdbIntegration** web service consists of a set of web services objects, each of which defines a list of web service fields. Out-of-the-box, the integration uses only part of them (see the *Mappings between Service Manager web service objects, tables, and DEM rules* table), some of them (along with their relevant DEM Rules) are deprecated and removed from Service Manager (see the *Deprecated ucmdbIntegration web service objects for data push* table), and some are used for population or federation (see the *ucmdbIntegration web service objects used for population or federation* table).

Mappings between Service Manager web service objects, tables, and DEM rules

This web service object	Publishes fields from this table	And uses this DEM rule ID
Relationship	cirelationship	ucmdbRelationship
ucmdbRunningSoftware	device	ucmdbRunningSoftware
ucmdbBusinessService	joinbizservice	ucmdbBusinessService
ucmdbNode	joinnode	ucmdbNode

Deprecated ucmdbIntegration web service objects for data push

This web service object	Publishes fields from this table	Replaced with (object)
ucmdbApplication	device	ucmdbRunningSoftware
ucmdbComputer	ucmdbComputer	ucmdbNode
ucmdbNetwork	joinnetworkcomponents	ucmdbNode
ucmdbPrinter	joinofficeelectronics	ucmdbNode

ucmdbIntegration web service objects used for population or federation

This web service object	Publishes fields from this table	And is used for	Requires a DEM Rule?
cirelationship1to1	cirelationship1to1	Population	No
ucmdbIDPushBack	device	Population	No
UcmdbChange	cm3r	Federation	No
UcmdbChangeTask	cm3t	Federation	No
UcmdbIncident	probsummary	Federation	No
UcmdbProblem	rootcause	Federation	No

The following sections list the fields published as web services objects used for data push (see the *Mappings between Service Manager web service objects, tables, and DEM rules* table) and indicate whether they are managed fields in an out-of-the-box Service Manager system. You can use this reference to determine if you need to publish a field as a web service object, and also if you need to create a DEM rule for the object.

Object Name: Relationship

Service Manager publishes the following fields from the cirelationship table:

Web service and managed fields of the Relationship object

Field published as web service object	Caption used in web service messages	Is the field a managed field?
relationship.name	RelationshipName	
logical.name	ParentCI	
related.cis	ChildCIs	Yes
relationship.subtype	RelationshipType	

Object Name: ucmdbRunningSoftware

Service Manager publishes the following fields from the device table:

Web service and managed fields of the ucmdbRunningSoftware object

Field published as web service object	Caption used in web service messages	Is the field a managed field?
ucmdb.id	UCMDBId	
ci.name	ApplicationName	
type	Type	
subtype	Subtype	
company	CompanyId	
logical.name	CIIdentifier	Yes
product.version	ProductVersion	
vendor	Vendor	
version	Version	
ci.name	ServiceName	Yes
sm.device.display.name	DisplayName	

Object Name: ucmdbBusinessService

Service Manager publishes the following fields from the joinbizservice table:

Web service and managed fields of the ucmdbBusinessService object

Field published as web service object	Caption used in web service messages	Is the field a managed field?
ucmdb.id	UCMDBId	
ci.name	ServiceName	Yes
type	Type	
subtype	Subtype	
company	CustomerId	
logical.name	CIIdentifier	
vendor	ServiceProvider	Yes
sm.device.display.name	DisplayName	Yes

Object Name: ucmdbNode

Service Manager publishes the following fields from the joinnode table:

Web service and managed fields of the ucmdbNode object

Field published as web service object	Caption used in web service messages	Is the field a managed field?
ucmdb.id	UCMDBId	
type	Type	
subtype	Subtype	
company	CustomerId	
logical.name	CIIdentifier	
default.gateway	DefaultGateway	Yes
network.name	DNSName	Yes
building	Building	Yes
room	Room	Yes
floor	Floor	Yes
location	Location	
addlIPAddr[addlIPAddress]	AddlIPAddress	Yes
addlIPAddr[addlSubnet]	AddlSubnet	Yes
addlMacAddress	AddlMacAddress	Yes
bios.id	BIOSId	Yes
operating.system	OS	Yes
os.version	OSVersion	Yes
physical.mem.total	PhysicalMemory	Yes
serial.no.	SerialNo	
vendor	Vendor	
cpu[cpu.id]	CpuID	
cpu[cpu.name]	CpuName	

Field published as web service object	Caption used in web service messages	Is the field a managed field?
cpu[cpu.clock.speed]	CpuClockSpeed	
file.system[mount.point]	MountPoint	
file.system[disk.type]	DiskType	
file.system[file.system.type]	FilesystemType	
file.system[disk.size]	DiskSize	
asset.tag	AssetTag	
machine.name	HostName	Yes
disk.device[model.name]	ModelName	
disk.device[disk.vendor]	DiskVendor	
disk.device[disk.name]	DiskName	
sm.device.display.name	DisplayName	Yes
isVisualization	IsVisualization	
istatus	AssetStatus	

1.6.3.5.1.4. Service Manager reconciliation rules

Service Manager reconciliation rules allow the integration to identify CI records in your Service Manager system that match CIs in your UCMDB system. Service Manager attempts to reconcile CI records with every push of CI attributes from your UCMDB system. The integration uses the following workflow to match UCMDB CIs with Service Manager CIs.

1. The UCMDB system sends a web service message containing the latest CI attribute data to Service Manager.
2. Service Manager scans the web service message for the CI ucmdb.id value.

   Note

   The ucmdb.id field is displayed in the Actual State Section of the CI form, with a label of **Ucmdb ID**.

3. Service Manager searches for an existing CI record that has the same ucmdb.id value.
4. If Service Manager finds a CI that has the ucmdb.id value, no reconciliation is needed. Service Manager compares the UCMDB CI attributes to the Service Manager managed fields and runs the appropriate Discovery Event Manager (DEM) rules as needed to update the CI record in Service Manager.
5. If Service Manager can't find a CI that has the ucmdb.id value, it searches for a DEM reconciliation rule record that's defined for the CI type.
6. If no DEM reconciliation rule record is found, Service Manager creates the CI record according to the appropriate DEM rules.
7. If a DEM reconciliation rule record is found, Service Manager evaluates the rules and appends (ucmdb.id=NULL or ucmdb.id~=NULL and istatus="XXXXXX") to the reconciliation rules in the backend, where the istatus value is retrieved from the DEM rule for deletion (for example, **Retired/Consumed**). If no istatus value is found in the DEM rule, Service Manager only appends ucmdb.id=NULL to the reconciliation rules in the backend.
8. If no matching CI record is found by the reconciliation rules, Service Manager creates the CI record according to the appropriate DEM rules.
9. If a matching CI record is found by the reconciliation rules, Service Manager updates the CI record according to the appropriate DEM rules.

Performance implications

Because Service Manager attempts to reconcile CIs with every push, the number of reconciliation fields you have will affect the integration's performance. The more reconciliation rules you have, the more searches Service Manager must perform to match CIs. To improve the performance of reconciliation searches, you should choose reconciliation fields that are unique keys of the underlying Service Manager table. For example, if you want to reconcile CI

records in the device table, use the logical.name field as a reconciliation field because it is a unique key. See How to add DEM reconciliation rules to create a reconciliation rule.

Dependence on DEM rules

Service Manager uses the “Action if matching record doesn't exist” DEM rule whenever it can't reconcile CIs. You should review the DEM settings and decide if they meet your business standards prior to the initial push of CIs from UCMDB to Service Manager. For example, you can have Service Manager create a change request for every CI in the initial CI push by selecting the “Open a change” option.

1.6.3.5.1.5. Service Manager Discovery Event Manager rules

You only need to create Discovery Event Manager (DEM) rules if you want to accomplish any of the following custom actions:

Change the conditions under which a DEM rule runs

Service Manager will run a DEM rule only if the condition field evaluates to true. Out-of-the-box, no DEM rule has a condition statement that restricts when the rule runs, and all the integration DEM rules will always run by default.

You can update a DEM rule's condition statements if you want to restrict when Service Manager runs your DEM rules. For example, adding the following condition to the ucmdbNode DEM rule restricts the rule to desktop CIs.

```
subtype in $L.file="Desktop"
```

You can also use the condition field to create multiple DEM rules that apply to the same table name. For example, the following DEM rules both apply to the joinnode table.

DEM rules using different conditions to affect the same table

DEM rule Id	Table Name	Condition
ucmdbNode	joinnode	`subtype in $L.file~="Desktop"`
ucmdbDesktop	joinnode	`subtype in $L.file="Desktop"`

Typically, you will only need to add conditions if your business processes require the integration to take different actions with certain CI types or SLAs.

Change the action the DEM rule takes

Out-of-the-box, the integration DEM rules take the following actions:

- Add a CI record when the UCMDB data doesn't match an existing Service Manager CI record
- Open a Change or log results and update a CI record when the UCMDB CI attribute data doesn't match the CI attribute data in the Service Manager CI record
- Delete a CI record when the UCMDB data specifies that the CI has been deleted

You can change the integration DEM rules to meet your business processes. For example, you could use the ucmdbNode DEM rule to open a change when the integration finds a non-desktop CI with unexpected data, and use the ucmdbDesktop DEM rule to log results and update the record when the integration finds a desktop CI with unexpected data.

Caution

If you want to use the Change Management verification and Change Management validation features of the integration, your DEM rules must use the Open a Change option for the “Action if record exists but unexpected data discovered” event.

Update the list of managed fields for a CI type

If you add CI attributes to your UCMDB system that you want to include in the integration, you must also create matching managed fields in Service Manager. Each managed field must have a corresponding web services object definition in order to receive CI attribute updates from your UCMDB system. See How to add a CI attribute to the integration for data push and How to add a CI type to the integration for data push for information on how to add managed fields.

Create custom JavaScript to open Change or Incident records

Service Manager uses the **discoveryEvent** JavaScript to create CI names and to set the values of required fields when opening change or incident records. Out-of-the-box, the script uses the following default values.

Default values to create a new CI

You can update the createCIName and populateNewCI functions to set the following CI values.

Default values used to create a new CI

CI attribute	Default value defined in discoveryEvent
record.logical_name	System generated ID number
record.assignment	AUTO
record.istatus	Installed
record.os_name	Value in record.operating_system

Default values to create a new change

You can update the populateChange function to set the following change values.

Default values used to create a new change

CI attribute	Default value defined in discoveryEvent
change.category	Unplanned Change
change.reason	Value in reason
change.initial_impact	3
change.severity	3
change.coordinator	Change.Coordinator
change.requested_by	discovery
change.status	initial

Default values to create a new incident

You can update the populateIncident function to set the following incident values.

Default values used to create a new incident

CI attribute	Default value defined in discoveryEvent
incident.category	incident
incident.subcategory	hardware
incident.product_type	missing or stolen
incident.assignment	Hardware
incident.initial_impact	3
incident.severity	3
incident.logical_name	Value of id
incident.site_cateogry	C
incident.contact_name	ANALYST, INCIDENT
incident.affected_item	MyDevices

1.6.3.5.2. Integration tailoring options

The integration offers the following tailoring options.

How to enable ASM support

How to update the integration adapter configuration file (sm.properties)

How to add DEM reconciliation rules

Add Discovery Event Manager rules

How to add a CI attribute to the integration for data push

How to add a CI type to the integration for data push

How to add a CI type's relationship types to the integration for data push

How to add a CI attribute to the integration for population

How to add a CI type to the integration for population

How to add a CI type's relationship types to the integration for population

How to customize UCMDB ID pushback for a CI type

How to add custom queries to integration population jobs

How to add an attribute of a supported CI type for federation

1.6.3.5.2.1. How to enable ASM support

As of version 9.41, Service Manager supports the Automated Service Modeling (ASM) functionality in the UCMDB UI.

ASM can automatically discover Service CIs and the consumer-provider relationships between the components of services. The Service Discovery process builds dependency graphs from the consumer-provider components that make up a service.



Note

ASM is supported only for UCMDB 10.20 or later. To use ASM, the ASM Enhanced package must be deployed in UCMDB.

You can tailor Service Manager to integrate with Automated Service Modeling (ASM) of the UCMDB Browser. See the following instructions.

How to integrate with ASM

ASM enables Service Manager users to view the accurate service trees that are discovered by ASM and facilitates service modeling in Service Manager.

For example, to integrate the Service Manager Change Management module with ASM, you can add a control that calls the **us.launch.external** RAD application by passing the following URL:

```
http://[UCMDB browser server name]:[port]/ucmdb_browser.jsp?username=<username>&password=<password>&server=Default%20Client&locale=en#view=[Service Name];tab=service-modeling
```

Where:

- [UCMDB browser server name], [port], [username] and [password] are the UCMDB server host name, communications port, integration account user name and password, respectively. These parameters are configured in the UCMDB integration section in the System Information Record.
- [Service Name] is the display name of current service CI.

1.6.3.5.2.2. How to update the integration adapter configuration File (sm.properties)

The integration uses a properties file (sm.properties) as a configuration file of the adapter. Out-of-the-box, this file has been set up based on best practices. Usually you can keep the default parameter values. Optionally, you can update the parameter values to better suit your needs.

To update the `sm.properties` file:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files**.
3. Click the properties configuration file: `sm.properties`.
4. Update the parameter values as needed. For a list of the parameters, see the following table.

Parameters in the sm.properties file

Parameter	Default value	Comment
timeout.minutes	10	The integration connection timeout value (in minutes).

Parameter	Default value	Comment
number.of.concurent.sending.threads	6	The number of concurrent threads used for the data push feature. ◦ 1: Disabled ◦ 2 or higher: Enabled **Note** If you are connecting to multiple Service Manager instances to improve the CI data push performance (see the URL Override configuration in How to create an integration point in UCMDB), you are recommended to increase this value for optimized performance. For example, set it to 12 if you are connecting to two Service Manager instances.

Parameter	Default value	Comment
min.objects.for.concurent.sending	50	The minimum number of Service Manager objects that's required to use concurrent sending instead of single thread sending. **Note** It is used for the push feature.
number.of.chunks.per.thread	3	The number of chunks per thread used for the push feature.
number.of.cis.per.request	1000	The maximum number of objects retrieved from Service Manager by ID. **Note** It is used for the population and federation features. **Do not set it to a number greater than 1000 in case the request has a 64K limit.**

Parameter	Default value	Comment
use.global.id	true	If set to true, globalId is used in the integration. If set to false, ucmdbId is used in the integration. For more information about the differences between UcmdbID and GlobalID, see the UCMDB documentation.
type.of.expand.enum	2	It configures the value mapping rule for the UCMDB enum type. ◦ 0: The feature will be disabled ◦ 1: The enum type will expand to “{value}” ◦ 2: The enum type will expand to “{index}-{value}” **Note** It is used for the push feature.

Parameter	Default value	Comment
use.type.label	true	It configures whether the generated source XML uses the real CI/Relationship type directly or uses the label that's defined in the query. If set to false, the real type is used directly instead of the label.
op.pagination.switch	on	It indicates if pagination (client driven) is enabled. ◦ on: Enabled. ◦ off: Disabled. **Note** It is used for the population feature.
pop.pagination.recordcount	1000	The maximum number of records that are displayed on each page when pagination is enabled. **Note** It is used for the population feature.

Parameter	Default value	Comment
pop.createci.key	sm_id	The UCMDB field of a CI record that stores the Service Manager CI ID. **Note** It is used for the population feature.
ucmdbid.pushback.request	UpdateucmdbID PushBackRequest	The web service request for pushing the UCMDB ID back to Service Manager. **Note** It is used for the population feature.
ucmdbid.pushback.xslt	ucmdbid_pushback.xslt	The xslt configuration file for pushing the UCMDB ID back to Service Manager. **Note** It is used for the population feature.

Parameter	Default value	Comment
check.sm.connections	false	It indicates whether to check the SOAP connections to Service Manager instances before running a job. You can enable it under any of the following circumstances: ◦ Your Service Manager is running in High Availability mode (with load balancing), and you want to connect UCMDB to multiple Service Manager instances. ◦ You want UCMDB to not run a job when no integration connections are available, rather than run the job and then report a failure.

1.6.3.5.2.3. How to add DEM reconciliation rules

It is possible that your Service Manager system already contains CI records that match CIs in your UCMDB system. Rather than add duplicate CI records to your Service Manager system, you can configure Service Manager to reconcile CI records between the two systems based on specified Discovery Event Manager (DEM) reconciliation rules.

A DEM reconciliation rule record enables you to specify what Service Manager queries you want to use to determine whether an existing CI record matches a CI in a UCMDB system. An administrator typically specifies reconciliation rules prior to starting the integration to the UCMDB system so that Service Manager will not create duplicate CI records.

When performing CI reconciliation between Service Manager and UCMDB, Service Manager uses the **ucmdb.id** field to query the device table or a join table to determine whether an Update or Create operation is needed. The reconciliation process is as follows:

1. The UCMDB system sends a web service message containing the latest CI attribute data to Service Manager.
2. Service Manager scans the web service message for the CI ucmdb.id value.

   Note

   The ucmdb.id field is displayed in the Actual State Section of the CI form, with a label of **Ucmdb ID**.

3. Service Manager searches for an existing CI record that has the same ucmdb.id value.
4. If Service Manager finds a CI that has the ucmdb.id value, no reconciliation is needed. Service Manager compares the UCMDB CI attributes to the Service Manager managed fields and runs the appropriate Discovery Event Manager (DEM) rules as needed to update the CI record in Service Manager.
5. If Service Manager can't find a CI that has the ucmdb.id value, it searches for a DEM reconciliation rule record that's defined for the CI type.
6. If no DEM reconciliation rule record is found, Service Manager creates the CI record according to the appropriate DEM rules.
7. If a DEM reconciliation rule record is found, Service Manager evaluates the rules and appends (ucmdb.id=NULL or ucmdb.id~=NULL and istatus="XXXXXX") to the reconciliation rules in the backend. The istatus value is retrieved from the DEM rule for deletion (for example, **Retired/Consumed**). If no istatus value is found in the DEM rule, Service Manager only appends ucmdb.id=NULL to the reconciliation rules in the backend.
8. If no matching CI record is found by the reconciliation rules, Service Manager creates the CI record according to the appropriate DEM rules.

9. If a matching CI record is found by the reconciliation rules, Service Manager updates the CI record according to the appropriate DEM rules

Multi-company mode

By default, Service Manager uses only the ucmdb.id field for CI reconciliation and the company.id field is not used. You can manually add DEM reconciliation rules.

Using join tables for reconciliation

When setting reconciliation rules, if the CI type you are reconciling has a joindef definition (as defined in the devtype table), use the join table name instead of the device table. For example, if you want to reconcile computer CIs, use the joinnode table instead of the device table.

Sequence of reconciliation

By default, Service Manager executes the reconciliation rules in their listed order. To change the order in which Service Manager reconciles CIs, you can add a numeric value to the **Sequence** field.

Steps to add a DEM reconciliation rule record

To create a DEM reconciliation rule record, follow these steps:

1. Click **Tailoring** > **Web Services** > **DEM Reconciliation Rules**.

   Service Manager displays the DEM Reconcile Record form.

2. In the **Table** field, select a Service Manager table against which you want to run reconciliation queries.

   Note

   Only tables that are specified in CI type records are displayed in the drop-down list. You can create only one DEM reconciliation record for one table.

3. Click **New**.

4. Enter expressions in the **Expression** column, and then enter a sequence number for each expression in the **Sequence** column. For example, you can add the following expressions for the **device** table. In this example, Service Manager looks for a matching record first based on the logical.name field, and then based on the sm.device.display.name field.

Expression	Sequence	Notes
logical.name=logical.name in $L.file	1	Checks to see if the CI Identifier matches the logical.name value of an existing CI in Service Manager. If no matching record is found, go to the next expression.
sm.device.display.name=sm.device.display.name in $L.file	2	Checks to see if the CI Display Name matches the display.name value of an existing CI in Service Manager.

5. Click **Add**.

   Service Manager creates the DEM reconciliation rule record.

1.6.3.5.2.4. Add Discovery Event Manager rules

Service Manager uses the Discovery Event Manager (DEM) to define which actions the system should perform when the actual state of an incoming configuration item (CI) record differs from the managed state of a CI record in Service Manager. The DEM rules allow you to define whether the Service Manager system adds, updates, or deletes CI records based on incoming UCMDB data.

For CI records only, the DEM rules also allow you to define how Service Manager should handle duplicated logical names.

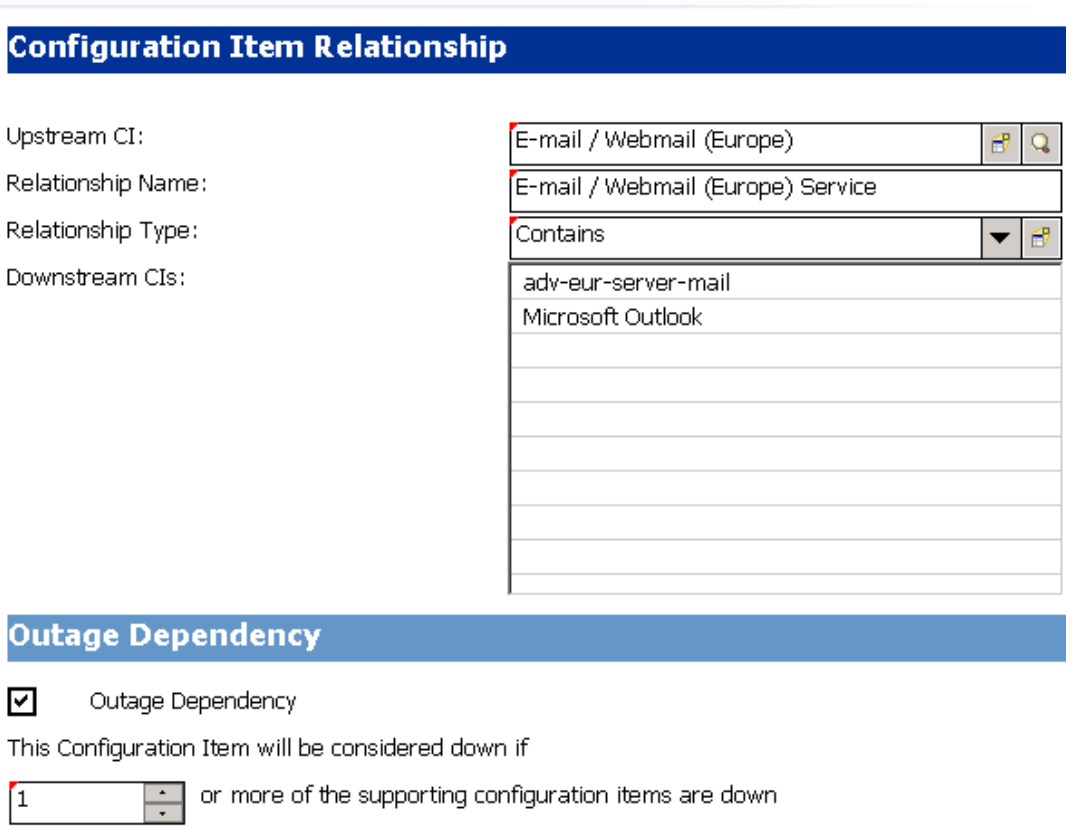
DEM rules

Service Manager offers the following rules options:

Action if a matching record doesn't exist

You may want Service Manager to take this action if it can't find a matching CI record.

- **Add the record:**(Default) Service Manager will add a CI record when it can't find a matching record. See How to add DEM reconciliation rules to define what fields Service Manager uses to match CI records.
- **Add the record, and set dependency as true**: This option is available only for synchronization of CI relationship data. Service Manager adds the CI relationship record and enables outage dependency for the record by doing the following:
  - Checks the Outage Dependency checkbox;
  - Sets the number of dependent downstream CIs to 1. This is because UCMDB only supports one-to-one CI relationships.



- **Open an Incident:**Service Manager opens an incident for someone to review the new CI record. The incident enables someone to investigate whether the new CI record is compliant with your business practices.
- **Open a Change:**Service Manager opens an unplanned change for someone to review the new CI record. The change allows you to investigate whether the new CI record is compliant with your business practices. If the CI record is compliant, the change can be approved. If the CI record is not compliant, then the change can be denied and the CI record removed. The change record lists both the current and proposed attribute values.

Action if a record exists but unexpected data discovered

This is the action you want Service Manager to perform if it doesn't find a matching CI attribute value.

- **Open a Change:** (Default) Service Manager opens an unplanned change to review the actual state of the CI record. The change allows someone to investigate whether the new attribute value is compliant with your business practices. If the value is compliant, the change can be approved. If the value is not compliant, then the change can be denied and the CI attribute value reverted to its managed state.
- **Log Results and update record:** Service Manager logs the results of the actual state of the CI record, and then updates the CI record.
- **Open an Incident:**Service Manager opens an Incident to investigate the actual state of a CI record and determines what actions must be performed or initiated to bring the record into compliance with Service Manager.

Action if a record is to be deleted

This is the action you want Service Manager to perform if an external event specifies that the record needs to be deleted.

- **Delete record:** (Default for CI Relationship records) This option is available for synchronization of both CI and CI Relationship records. Service Manager automatically deletes the CI/CI Relationship record.
- **Open an Incident:** This option is available only for synchronization of CI Relationship records. Service Manager opens an incident to investigate the deleted record and determines which actions must be performed or initiated to bring the record into compliance with Service Manager.
- **Open a Change:** This option is available only for synchronization of CI Relationship records. Service Manager opens an unplanned change to review the deleted record. The change allows someone to investigate whether the deleted record is compliant with your business practices. If the record is compliant, the change can be approved. If the record is not compliant, then the change can be denied and the record added back to the system.
- **Update record to the selected status:** (Default for CI records) This option is available only for synchronization of CI records. Service Manager updates the status of the CI record to a value selected from the drop-down list (for example, **Retired/Consumed**), rather than deletes the record permanently.

  Note

  Values available from the drop-down list are defined in the **ICM Status** global list.

- **Open a Change to update record to the selected status:** This option is available only for synchronization of CI records. Service Manager opens an unplanned change to update the CI record's status to a value selected from the drop-down list (for example, **Retired/Consumed**). The change allows someone to investigate whether the requested status change is compliant with your business practices. Once the change has been approved and closed, Service Manager automatically changes the CI record to the selected status. If the change has been denied, Service Manager makes no changes to the CI record.
- **Open an Incident to update record to the selected status:** This option is available only for synchronization of CI records. Service Manager opens an incident to update the record's status to a value selected from the drop-down list (for example, **Retired/Consumed**). Once the incident has been closed, Service Manager automatically updates the CI record to the selected status.

CI attributes displayed in Change and Incident records

Service Manager displays a Change Details section on the corresponding change or a CMDB Changes section on the corresponding incident when you configure DEM to open either

change records or incident records when it discovers CI attribute changes through the UCMDB-SM integration. Service Manager only displays a tab for CI attributes when the UCMDB-SM integration is enabled and you have defined a rule in the Discovery Event Manager to create a change or incident record when a CI is added, updated, or deleted.

Both the Change Details and CMDB Changes sections display the current CI attribute values alongside the actual attribute values discovered by UCMDB. You can use this information to approve or deny a change or escalate an incident to the proper assignment group.

Searching for Change and Incident records opened by the integration

You can use the following search criteria to find change and incident records opened by the UCMDB-SM integration.

Search options available for change and incident records

Record type	Search option available
Change	Search for records with the category **unplanned change**.
Incident	Search for records using the **generated by the UCMDB integration** option.

1.6.3.5.2.5. How to add a CI attribute to the integration for data push

You can use the following steps to add a CI attribute to the integration.

1. Does the CI attribute already exist in the UCMDB class model?

   Yes. Go to *Step 3*.

   No. Go to *Step 2*.

2. Add the CI attribute to the UCMDB class model.

   See *How to add the CI attribute to the UCMDB class model* below.

3. Add the CI attribute to the query layout.

   See *How to add a CI attribute to the query layout* below.

4. Add the CI attribute to the Service Manager table.

   See *How to add the CI attribute to the Service Manager table* below.

5. Create a web service field to support the CI attribute.

   See *How to Create a Web Service field to support the CI attribute* below.

6. Add a managed field to monitor changes in the CI attribute.

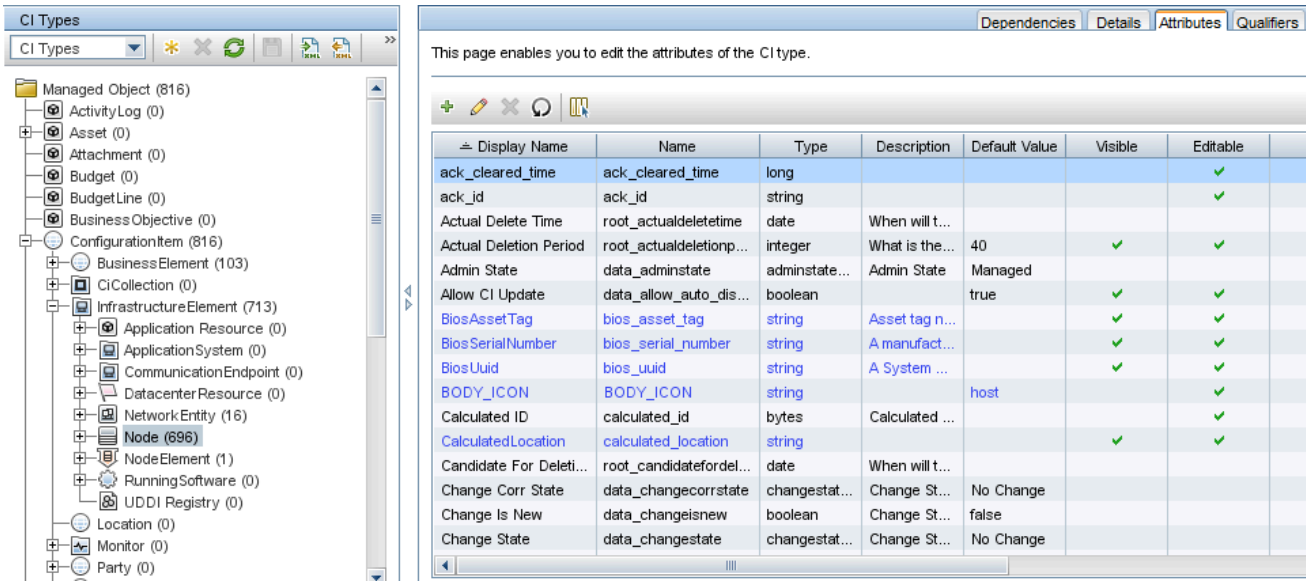
   See *How to add a managed field to support the CI attribute* below.

7. Map the CI attribute to a web service field.

   See *How to map the CI attribute to a Web Service field* below.

How to add the CI attribute to the UCMDB class model

The integration only uses a subset of the CI attributes available from your UCMDB system. Out-of-the-box, the integration consists of CI attributes that are typically managed from a Service Manager system such as host name and host DNS name. Before creating a new UCMDB CI attribute, you should determine if there are any existing CI attributes in your UCMDB system that provide the data you want. In most cases, there is an existing attribute tracking the data you want to add to the integration. For example, if you review the attributes of the Node CI type, you see that there are many attributes available to be added to the integration.



The following steps illustrate how to add a new CI attribute to an existing CI type. This scenario is not the expected typical case. Typically, you would add an existing CI attribute to the integration.

Note

The integration doesn't require any special steps to add a CI attribute to the UCMDB class model. You can use the standard CI attribute creation procedures to add a CI attribute. For more information on CI attribute creation, see the *Universal CMDB CI Attribute Customization Guide*.

To add a CI attribute to the UCMDB class model:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **CI Type Manager**.
3. Select the CI type to which you want to add a new CI attribute from the CI Types navigation tree. For example, **ConfigurationItem** > **InfrastructureElement** > **RunningSoftware** > **Database**.
4. Click the **Attributes** tab.
5. Click the **Add** button.

   The Add Attribute window opens.
6. In Attribute Name, type the unique name you want to use for the new CI attribute. For example, database_owner.

Caution

The name can't include any of the following characters: ‘ / \ [ ] : | < > + = ; , ? *.

7. In Display Name, type the name you want UCMDB to display in the interface. For example, Database Owner.
8. In Description, type a description of the new CI attribute. This is an optional field. For example, a System user who owns the database.
9. In Attribute Type, select either Primitive or Enumeration/List. For example, select **Primitive** and select **string**.
10. In Value Size, type the maximum character length the attribute can have. For example, 300.
11. In Default Value, type the value to be used when no other value is available. For example, leave the default value blank.
12. Click **OK** to save the attribute.
13. Click the **Save** button  to save attribute changes to the CI type.

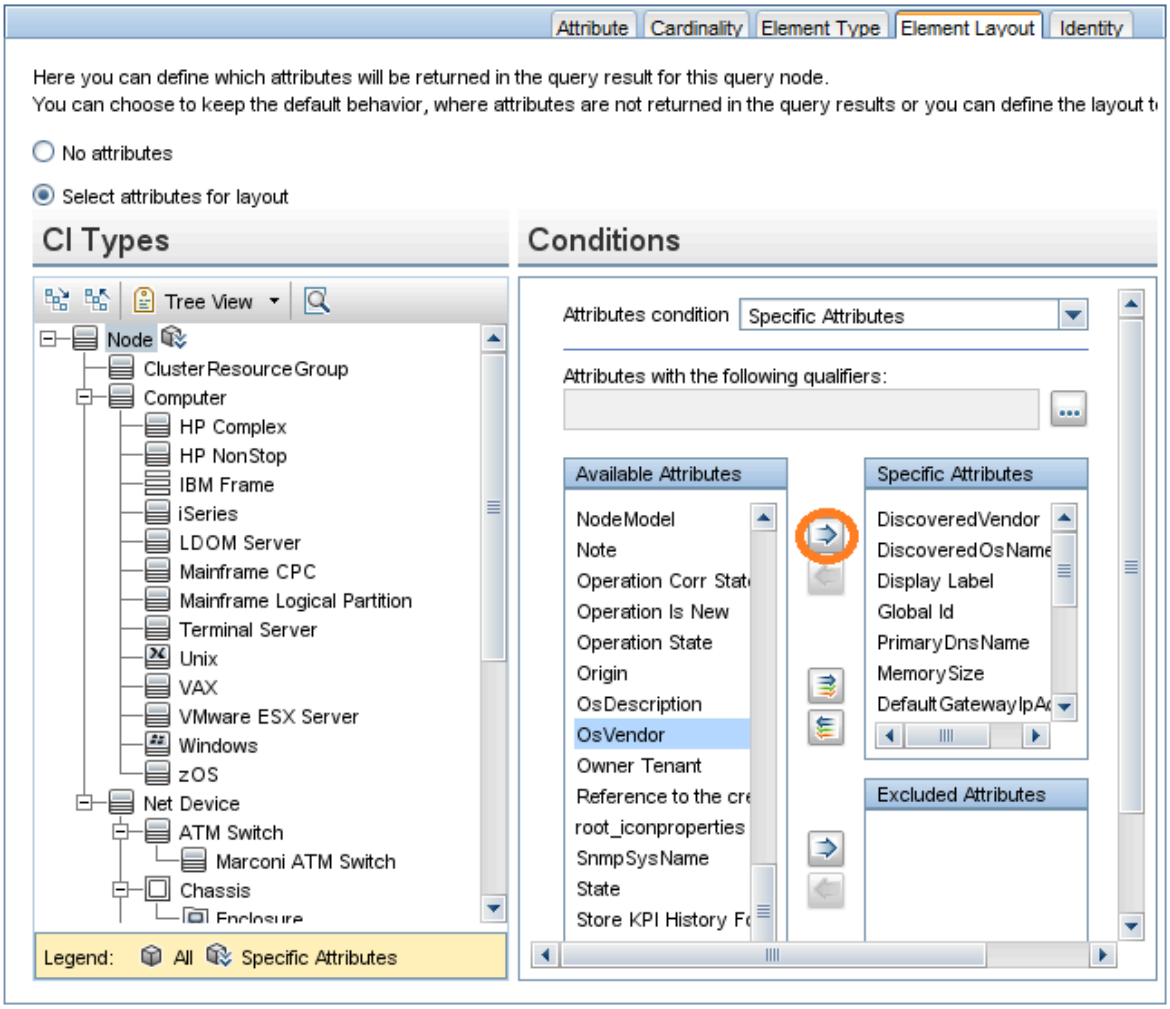
How to add a CI attribute to the query layout

To add a CI attribute to the integration, you must add this attribute to the layout setting from the query that synchronizes the CI type. You must know what CI type contains the CI attributes you want to add to the integration.

Keep a list of the attributes that you enable, because you will need to create a matching XSL transformation for each attribute you enable.

To add a CI attribute to the query layout:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio**.
3. For Resource Type, select **Queries**.
4. From the Queries navigation tree, click **Integration** > **SM Sync** > **9.xx**.
5. Select the query that manages the CI type whose attributes you want to add to the integration. For example, **SM Computer Push**. UCMDB displays the TQL graph for the integration query.
6. Select the node from the query layout that contains the CI attribute you want to add to the integration. For example, **Root**.
7. Right-click the node and select **Query Node Properties**. The Query Node Properties window opens.
8. Click **Element layout tab**. The Layout Settings tab opens.
9. Select the CI attribute you want to include in the integration from the **Available Attributes** list, and click the **Add** button to add it to **Specific Attributes** list. For example, **OsVendor**.



10. Click **OK** to save the node properties.
11. Click the **Save** button to save the query.

How to add the CI attribute to the Service Manager table

The integration uses only a subset of the CI attributes available from your Service Manager system. Before creating a new Service Manager CI attribute, you should determine if there are any existing CI attributes in your Service Manager system that provide the data you want. In most cases, there is an existing attribute tracking the data you want to add to the integration. For example, if you review the attributes of the Computer CI type, you see that there are many attributes available to be added to the integration.

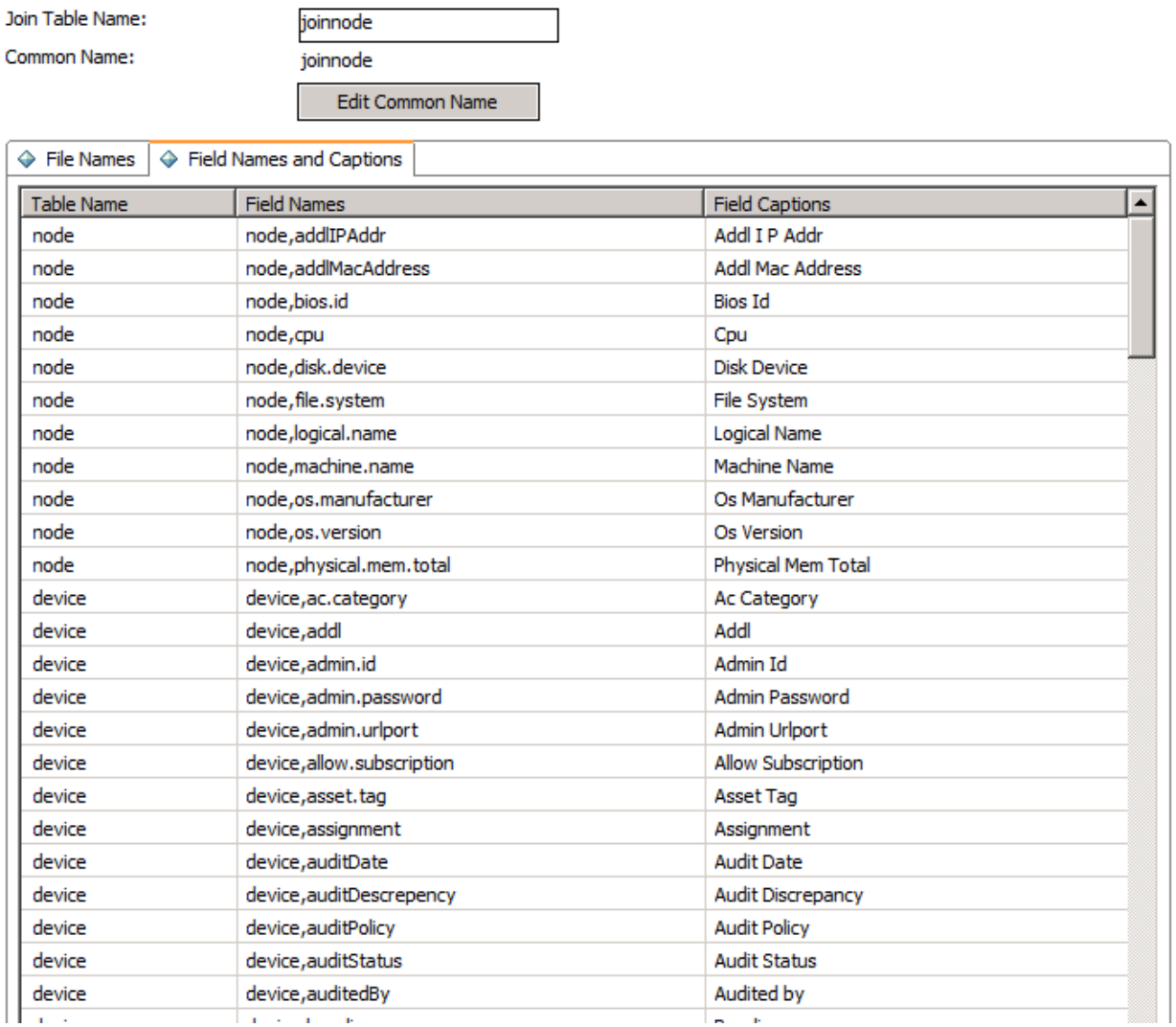


Table Name	Field Names	Field Captions
node	node,addlIPAddr	Addl I P Addr
node	node,addlMacAddress	Addl Mac Address
node	node,bios.id	Bios Id
node	node,cpu	Cpu
node	node,disk.device	Disk Device
node	node,file.system	File System
node	node,logical.name	Logical Name
node	node,machine.name	Machine Name
node	node,os.manufacturer	Os Manufacturer
node	node,os.version	Os Version
node	node,physical.mem.total	Physical Mem Total
device	device,ac.category	Ac Category
device	device,addl	Addl
device	device,admin.id	Admin Id
device	device,admin.password	Admin Password
device	device,admin.urlport	Admin Urlport
device	device,allow.subscription	Allow Subscription
device	device,asset.tag	Asset Tag
device	device,assignment	Assignment
device	device,auditDate	Audit Date
device	device,auditDescrepency	Audit Discrepancy
device	device,auditPolicy	Audit Policy
device	device,auditStatus	Audit Status
device	device,auditedBy	Audited by

The following steps illustrate how to add a new CI attribute to an existing CI type.

The integration doesn't require any special steps to add a CI attribute to the Service Manager table. You can use the standard table attribute creation procedures to add a CI attribute. For more information on table attribute creation, see Service Manager Help Center and *Service Manager Tailoring Best Practices*.

To add a CI attribute to the Service Manager table:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Database Dictionary**.
3. In File Name, type the name of the table where you want to add the new CI attribute. For example, node.
4. Click the **Search** button  .

   The node dbdict record opens.
5. Click the **Fields** tab.
6. Click the **New Field/Key** button.

The Add Attribute window opens.

7. In Name, type the name you want to use for the new CI attribute. For example, os.manufacturer.

   Caution

   The name can't include any of the following characters: ' / \ [ ] : | < > + = ; , ? *.

8. In Type, select a type from the list. For example, select **character**.
9. Click the **Add Field** button to save the attribute.
10. Click the **OK** button to save attribute changes to the table.

How to create a Web Service field to support the CI attribute

UCMDB uses the Service Manager ucmdbIntegration web service to send CI data. This web service publishes the objects that match the out-of-the-box CI types and CI attributes provided by the UCMDB-SM integration. For a list of the out-of-the-box web service fields and their mappings to Service Manager tables, see Managed fields.

If you add a CI attribute to the integration on your UCMDB system, you must create a corresponding web service field on your Service Manager system to receive the incoming CI data from UCMDB. Each web service field must map to a valid Service Manager table and column.

To create a web service field for the OSVendor attribute described in previous sections:

1. Sign in to Service Manager with an administrator account.
2. Navigate to **Tailoring** > **Web Services** > **WSDL Configuration**. The External Access Definition form opens.
3. For Service Name, type **ucmdbIntegratio**n.
4. Click **Search**. Service Manager displays a record list of the objects that make up the ucmdbIntegration web service.
5. Select an existing web service object to which you want to add the CI attribute. For example, select **ucmdbNode**.
6. Click the **Fields** tab. Service Manager displays the fields published as web service fields.
7. Select an empty row in the Fields list.
8. For Field, select the Service Manager column name where you want to store the incoming CI attribute values. For example, **os.manufacturer**.

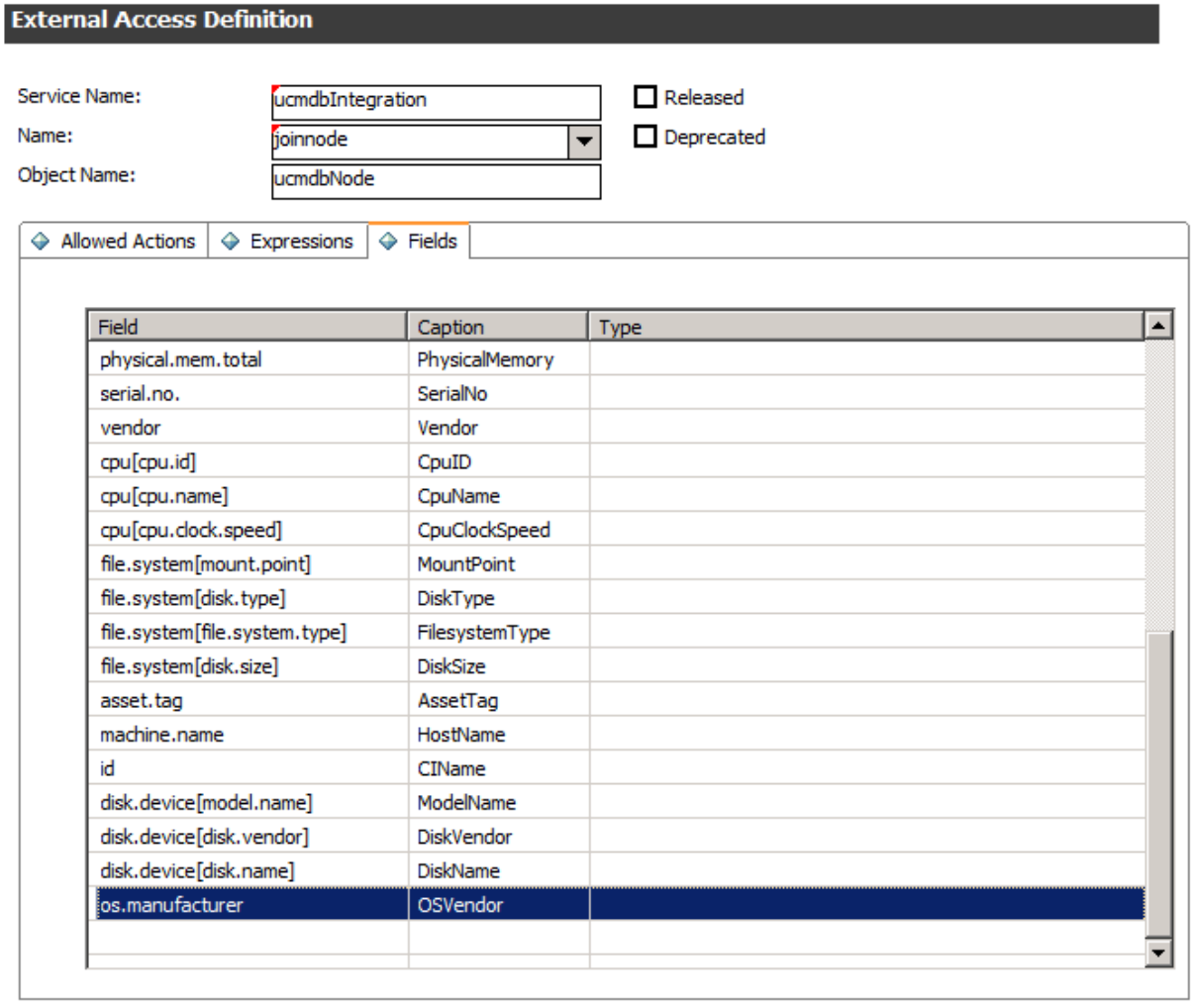
> **Note**
>
> Service Manager displays the fields from all join tables associated with the table listed in the Name field. For example, for joinnode Service Manager displays the fields from the device and node tables.

9. For Caption, type the name you want Service Manager to use when publishing the field as a web service field. For example, **OSVendor**.

> **Caution**
>
> The Caption name must match the object name you listed in the XSL transformation file in Universal CMDB, or Service Manager will not receive any CI updates from your Universal CMDB system.

10. Click **Save**.

    The new web service field is available immediately; you don't need to restart the Service Manager system.



How to add a managed field to support the CI attribute

In order for a CI attribute you add to the integration to trigger the automated Change Management validation and verification processes, you must add a managed field for the CI attribute. Service Manager managed fields are part of the Discovery Event Manager Rules. For a list of fields that trigger Change Management validation and verification, see Service Manager Discovery Event Manager rules.

To add a managed field to the integration:

1. Sign in to Service Manager with an administrator account.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**. Service Manager displays a record search/creation form.
3. Click **Search** to display a list of all Discovery Event Manager rules.
4. **Select the rule ID that matches the web services object where you mapped the incoming CI attribute**. For example, **ucmdbNode**. See *How to create a Web Service field to support the CI attribute*.

   Service Manager displays the rules for this web service object.
5. Click the **Managed Fields** tab. Service Manager displays the list of fields that trigger Change Management validation and verification.
6. Select an empty row in the Managed Fields list.
7. For Field Name, select the caption name of the Service Manager column that you previously selected to store the incoming CI attribute values. For example, **Os Manufacturer**.

   Note

   Service Manager displays the fields from all join tables associated with the table listed in the Table Name field. For example, joinnode displays fields from the **device** and **computer** tables.

   Tip

   If you want to add all fields that are exposed in the WSDL definition, you can click the **Load Fields** button. For more information, see How do I use the Load Fields button to add multiple managed fields.
8. If the field you use to store the incoming CI attribute is an array of structure, use the Structure field to select the name of the array of structure where the column can be found. For example, **Os Manufacturer** is a primitive character field and therefore doesn't need to identify an array of structure name.
9. If the field you use to store the incoming CI attribute is an array of structure, use the Index field to select the index number that identifies the column in the array of structure. For

example, **Os Manufacturer** is a primitive character field and therefore doesn't need to identify an array of structure index.

10. Click **Save**.

How to map the CI attribute to a Web Service field

The integration uses an adapter to transform UCMDB CI attributes to web services objects recognized by Service Manager. The adapter in turn specifies what XSL transformation files the integration should use to convert UCMDB queries into properly formatted Service Manager web services messages.

Out-of-the-box, each integration query has a corresponding XSL transformation file that maps to a particular CI type in UCMDB. In addition, each attribute for which you enabled calculation requires its own entry in the XSL transformation file. Without an XSL transformation entry, Service Manager can't receive any CI attribute updates from your UCMDB system.

If you want to add a new attribute to the integration, you must edit the XSL transformation file for the parent CI type and add an entry for the CI attribute. For information about which CI types each query manages, see Queries for push. In order to create a proper XSL mapping, you must be familiar with the service and object names Service Manager publishes as web services. For information on publishing tables and columns as web service objects, see *Service Manager Web Services* available from the Service Manager help.

The following steps illustrate how to map a UCMDB CI attribute called host_vendor to a Service Manager web service object called OSVendor.

To map a CI attribute to a web service field:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files**.
3. Double-click the XSL transformation file that manages the parent CI type of your CI attribute. For example, open `computer_push.xslt` to add an attribute to the **SM Computer Push** query.
4. Find the element that defines the Service Manager table name where the integration will store CI attribute values. For example, the element `<file.device>` will store CI attributes in the Service Manager device table.
5. Within the table naming element (<file.device>), you will see an element of the following format that defines how to transform each UCMDB CI attribute into a web service object:

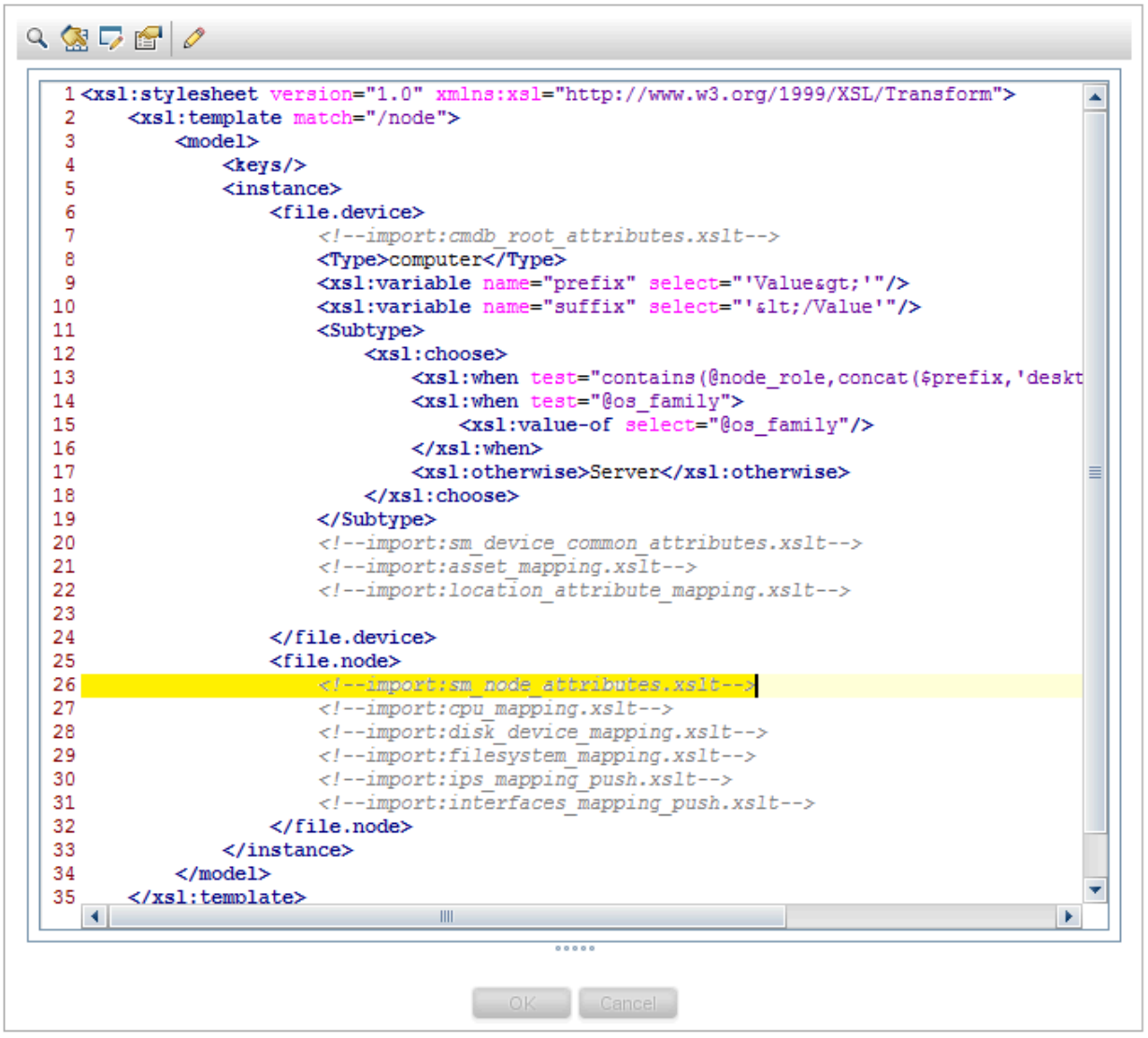
```
<xsl:for-each select="@CI_attribute_name">
   <SMAttributeName><xsl:value-of select="."/></SMAttributeName>
</xsl:for-each>
```

`@CI_attribute_name` is the name of attribute in the UCMDB system.

`SMAttributeName` is the name of a web service attribute published by the Service Manager system.

> **Caution**
>
> The web service attribute name is case-sensitive.

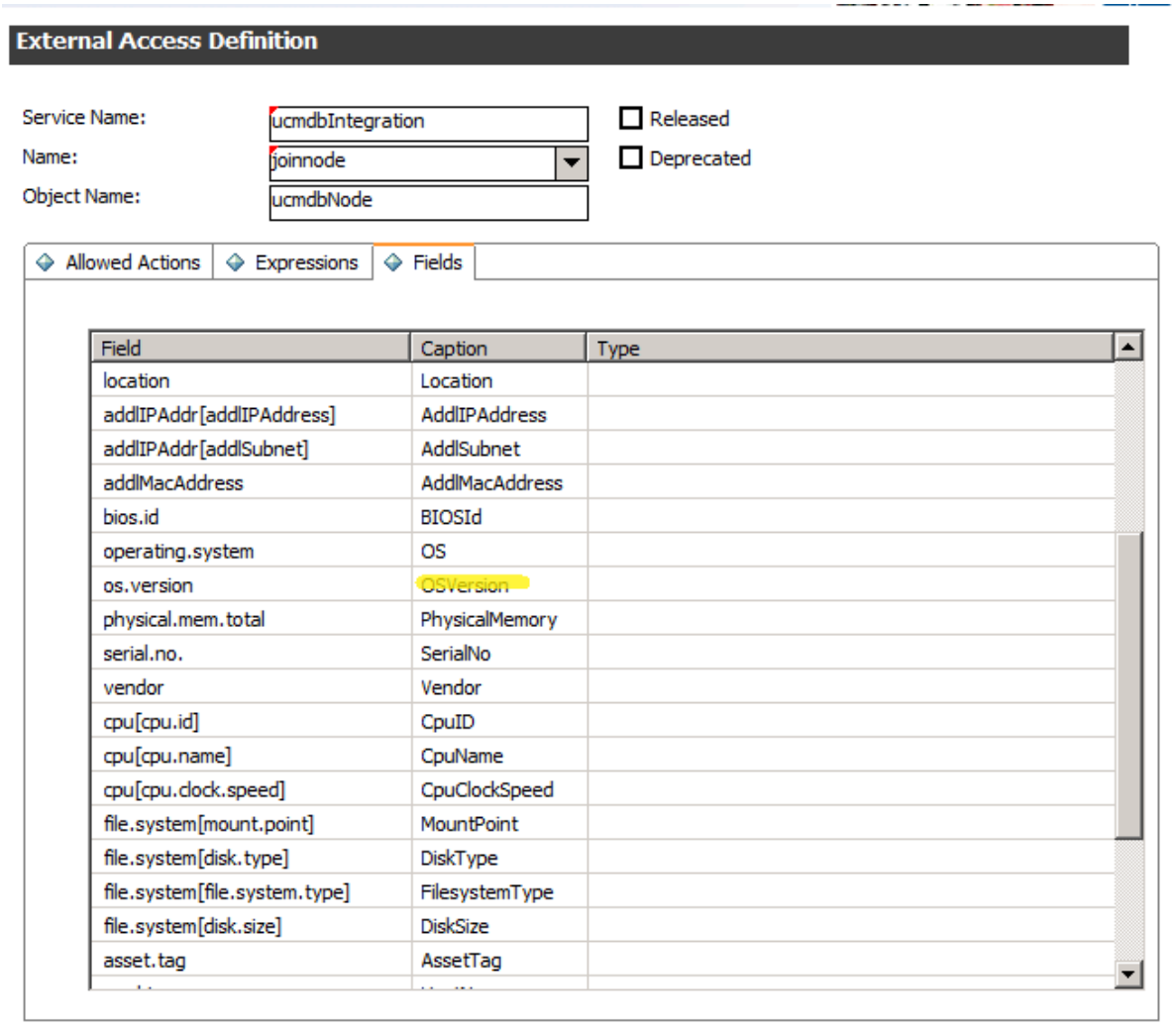
CI attributes in the `computer_push.xslt` XSL transformation file are illustrated in the following screenshot:



CI attributes in the `sm_node_attributes.xslt` XSL transformation file are illustrated in the following screenshot:

```
<xsl:for-each select="@name">
    <HostName>
        <xsl:value-of select="."/>
    </HostName>
</xsl:for-each>
<xsl:for-each select="@discovered_os_version">
    <OSVersion>
        <xsl:value-of select="."/>
    </OSVersion>
</xsl:for-each>
<xsl:for-each select="@memory_size">
    <PhysicalMemory>
        <xsl:value-of select="."/>
    </PhysicalMemory>
</xsl:for-each>
<xsl:for-each select="@bios_serial_number">
    <BIOSId>
        <xsl:value-of select="."/>
    </BIOSId>
</xsl:for-each>
```

Matching CI attributes in the `ucmdbNode` web service are illustrated in the following screenshot:



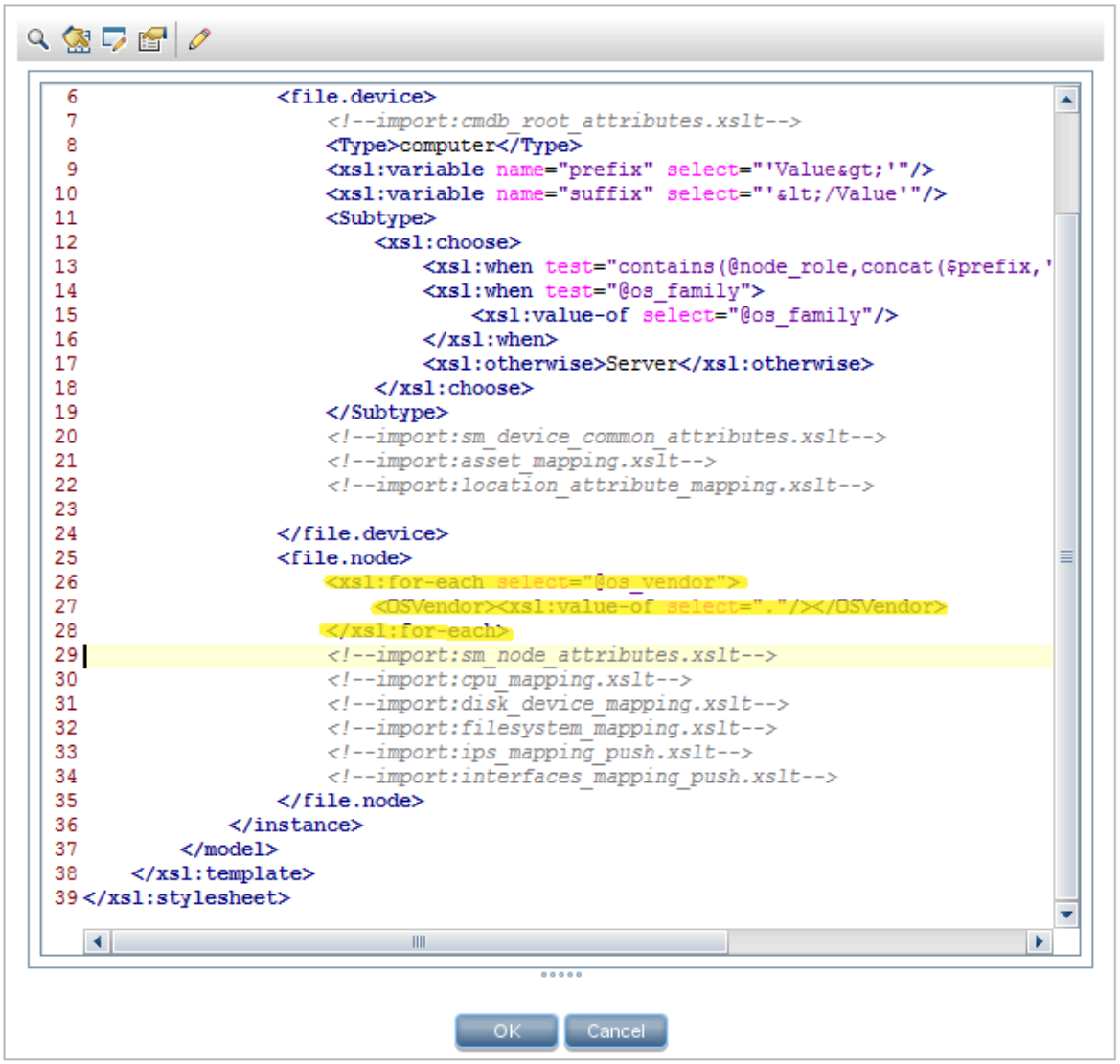
To push a custom string list value (the attribute type is `string_list`) from UCMDB to an array field in SM, you need to add the following line to web service expressions.

```
$L.void=jscall("discoveryEvent.formatUcmdbStringList", $L.file, "sm
field name")
```

6. Copy an existing XSL transformation element to use it as a template to create a new transformation entry.
7. Paste the new XSL transformation element within the proper table naming element. For example, `<file.node>`.
8. Update the CI attribute name and web service object name within the new element to match the attribute you want to add to the integration. For example, create the following XSL transformation element to add an attribute to the integration.

   ```
   <xsl:for-each select="@os_vendor">
   <OSVendor><xsl:value-of select="."/></OSVendor>
   </xsl:for-each>
   ```

   New attribute in the `computer_push.xslt` XSL transformation file is illustrated in the following screenshot:



9. Save the XSL transformation file.

Note

When you create/edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

1.6.3.5.2.6. How to add a CI type to the integration for data push

You can use the following steps to add a CI type to the integration.

1. Does the CI type already exist in the UCMDB class model?

   Yes. Go to *Step 3*.

   No. Go to *Step 2*.

2. Add the CI type to the UCMDB class model.

   See *How to add the CI type to the UCMDB class model* below.

3. Add CI attributes to the CI type as needed.

   See How to add a CI attribute to the integration for data push.

4. Create a query to synchronize the CI type.

   See *How to create a query to synchronize the CI type* below.

5. Add the CI type's attributes to the query layout.

   See *How to add the CI type's attributes to the query layout* below.

6. Add the CI type to Service ManagerService Manager.

   See *How to add the CI type in Service Manager* below.

7. Create web service fields to support the CI type.

   See *How to create Web Service fields to support the CI type* below.

8. Add managed fields to support the CI type.

   See *How to add managed fields to support the CI type* below.

9. Map the CI type's query to an XSL transformation file.

   See *How to map the CI type's query to an XSL transformation file* below.

10. Map the CI type's attributes to web service fields.

    See *How to map the CI type's attributes to Web Service fields* below.

11. Add custom queries to integration data push jobs.

    See *How to add custom queries to data push jobs* below.

How to add the CI type to the UCMDB class model

Before creating a new UCMDB CI type, you should determine if there are any existing CI types in your UCMDB system that provide the CI attributes you want. In most cases, you can create links to one or more existing CI types to create a new logical CI type for use by the integration.

The following steps illustrate how to create a new CI type called SM RDBMS based on an existing CI type called database.

Note

The integration doesn't require any special steps to add a CI type to the UCMDB class model. You can use the standard CI type creation procedures to add a CI type. For more information on CI type creation, see the *Universal CMDB CI Attribute Customization Guide*.

To add a CI type to the UCMDB class model:

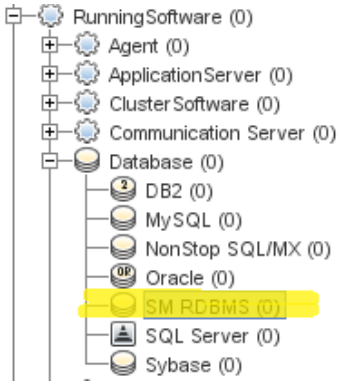
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **CI Type Manager**.
3. Select the base CI type you want to use for your new CI type from the CI Types navigation tree: **Managed Object** > **ConfigurationItem** > **Infrastructure Element** > **Running Software** > **Database**.
4. Click the **New** button  .

   The Create Configuration Item Type window opens.
5. In Name, type the unique name you want to use for the new CI type. For example, sm_rdbms.

   Caution

   The name can't include any of the following characters: '' / \ [ ] : | < > + = ; , ? *.
6. In Display Name, type the name you want UCMDB to display in the interface. For example, SM RDBMS.
7. In Description, type a description of the new CI type. This is an optional field. For example, Hosts running relational databases.
8. In Base CI Type, verify that the proper base CI type is selected. Your new CI type will inherit the attributes of the base CI type you select here. For example, **Database**.
9. Click **Next**. The wizard displays a list of CI attributes from the base CI type.
10. Add, edit, or remove CI attributes as needed for the new CI type. For example, accept the default attributes inherited from Database.
11. Click **Next**. The wizard displays a list of qualifiers from the base CI type.
12. Add or remove qualifiers as needed for the new CI type. For example, accept the default qualifiers.
13. Click **Next**. The wizard displays a list of icons associated with the CI type.

14. Select the icons associated with this CI type. For example, accept the default abstract class icon.
15. Click **Next** to add any menu item properties or label definitions as needed. For example, accept the default settings from the base CI type.
16. Click **Finish** to create the CI type.

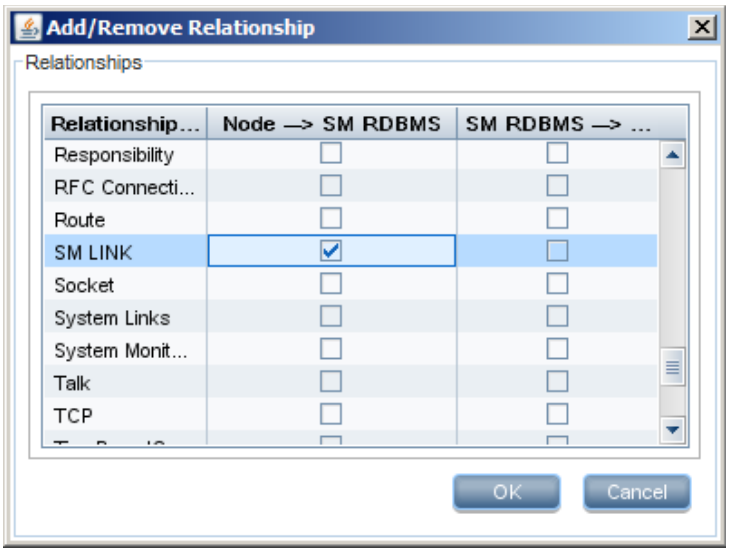


17. Select your new CI type from the tree. For example, **SM RDBMS**.
18. Browse to an existing CI type you want to link to, and control-click it to add it to your selection. For example, **Node**.

> **Note**
>
> Choose an existing CI type that has the attributes that you want to be part of your new logical CI type.

19. Right-click one of the selected CI types, and click **Add/Remove Relationship**. The Relationships window opens.
20. Create an SM Link relationship from the existing CI type to the new CI type. For example, from **Node** to **SM RDBMS**.

> **Note**
>
> You need to create a new SM Link relationship if it doesn't exist.



21. Click **OK** to create the relationship.
22. Click the **Save** button to save the CI type.

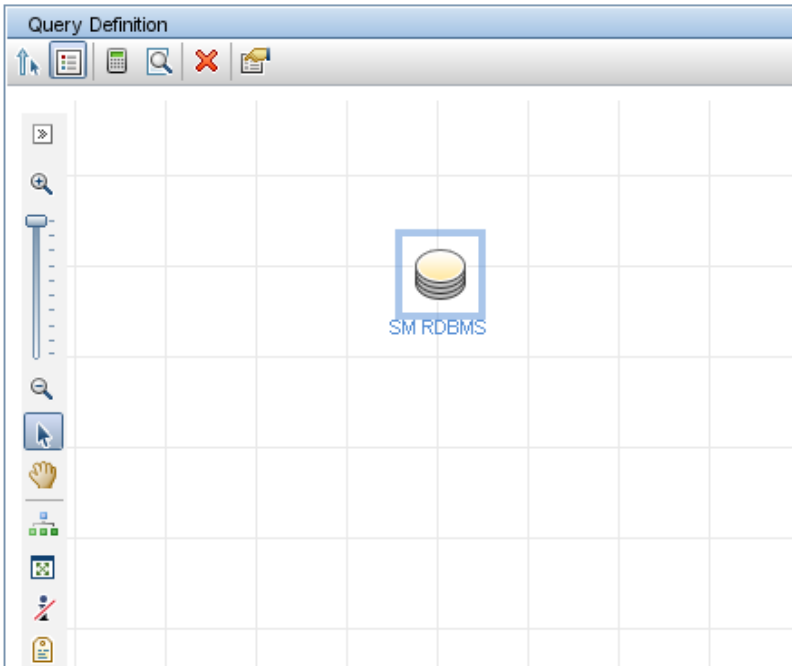
How to create a query to synchronize the CI type

The integration uses queries to gather CI attribute values and pass them to your Service Manager system. You must create a query for any CI type you add to the integration. Any query you create must conform to the Query requirements.

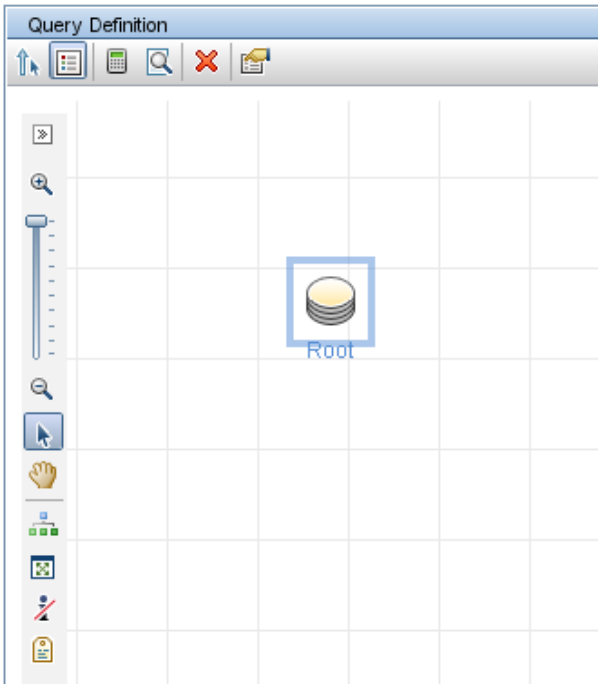
The following steps illustrate how to create a new query called rdbmsData for the SM RDBMS CI type described in previous sections.

1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio**.
3. From the Queries navigation tree, click **Integration** > **SM Sync**.
4. Right-click **SM Sync**, and select the **New** > **Query**.

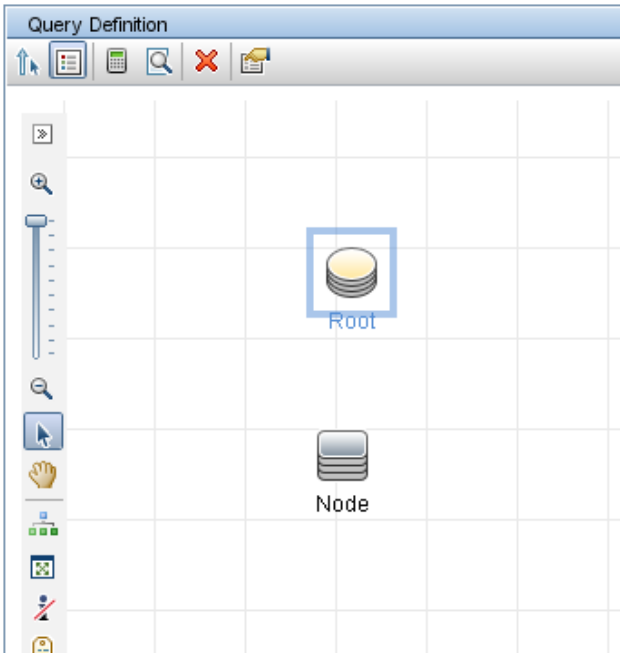
   The Query Definition window opens.
5. Find the CI type that will be the root node of your query from the CI Type Selector. This CI type is typically the one that provides the most attributes for the CI. For example, **Managed Object** > **ConfigurationItem** > **InfrastructureElement** > **RunningSoftware** > **Database** > **SM RDBMS.**
6. Drag the root CI type from the CI Type Selector and drop it into the empty Editing pane. UCMDB displays the icon of the CI type.



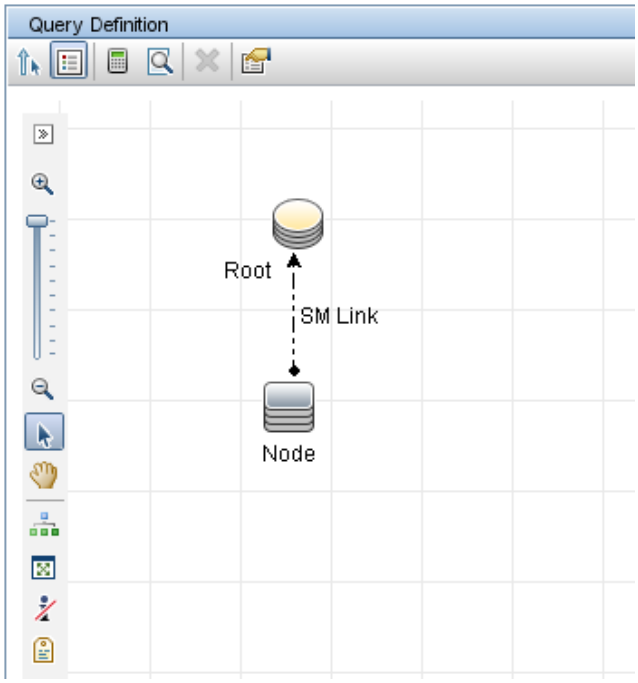
7. Select the CI type, and click **Edit** from the Information Pane. The Node properties window opens.
8. Change the Element name to **Root**.
9. Click **OK** to save the node properties.



10. Find any additional CI types you want to add to the query from the CI Type Selector. These CI types typically provide additional CI attributes. For example, **Managed Object** > **ConfigurationItem** > **Infrastructure Element** > **Node**.
11. Drag the additional CI type from the CI Type Selector and drop it into the empty Editing pane. UCMDB displays the icon of the additional CI type.



12. Create relationships between the Root CI type and the additional CI types as needed. For example, create an SM Link between **Root** and **Node**.
    1. Select **Root** and control-click the additional CI type. For example, **Node**.
    2. Right-click one of the selected items, and click **Add Relationship**. The Add Relationship window opens.
    3. Select **SM Link**.
    4. Type a Relationship Name. For example SM Link.
    5. Click **OK** to add the relationship.
13. Repeat step 10 to step 12 for each additional CI type you want to add to the query. For example, SM RDBMS doesn't need any additional CI types.



14. Click the **Save** button  to save the query.

15. In Query Name, type the unique name you want to use for the new query. For example, rdbmsData.
16. In Description, type a description of the new query. This is an optional field. For example, Query for hosts running relational databases.
17. In the folder tree, select the folder in which you want to save the query. For example, **Root** > **Integration** > **SM Sync**.
18. Click **OK**. UCMDB adds your new query to the Queries list.

How to add the CI type's attributes to the query layout

To add a CI attribute to the integration, you must enable the calculation layout setting from the query that synchronizes the CI type. Because you must enable calculation for each attribute you want to add to the integration, you should be familiar with the integration CI types and the CI attributes that they contain.



Tip

Keep a list of the attributes that you enable, because you will need to create a matching XSL transformation for each one.

The following steps illustrate how to enable calculation for attributes of the SM RDBMS CI type described in previous sections.

To add a CI type's attributes to the query layout:

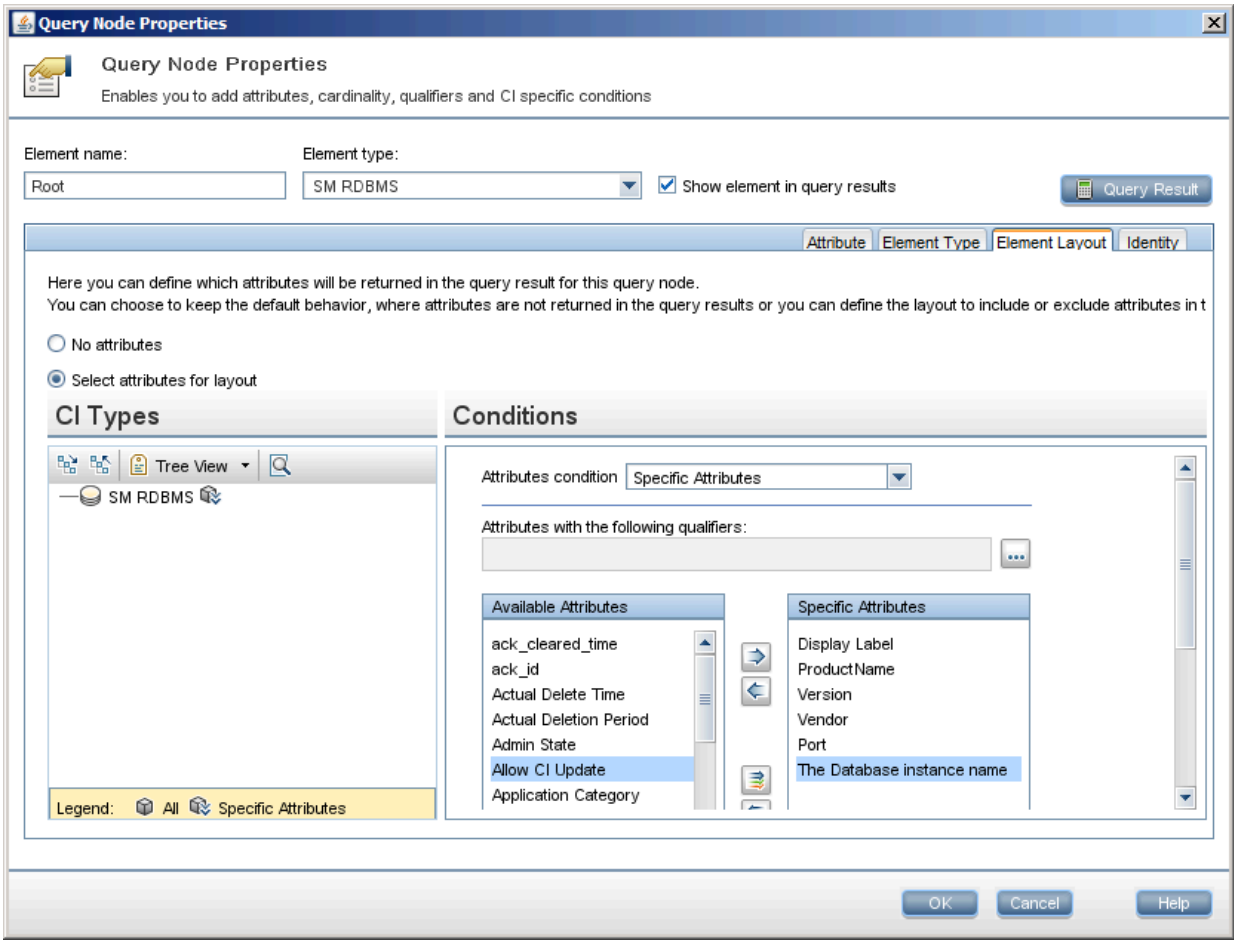
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio**.
3. From the Queries navigation tree, click **Integration > SM Sync**.
4. Select the query that manages the CI type whose attributes you want to add to the integration. For example, **rdbmsData**. UCMDB displays the query for the integration.
5. Select the **Root** node from the query layout, and then click the **Edit** button from the Information Pane. The Node properties window opens.



Caution

Your integration query must contain a node called Root. See Query requirements for more information.

6. Click the **Element Layout** tab, and select the option **Select attributes for layout**.
7. Select **Specific Attributes** from the **Attributes condition** list, and from the Available Attributes list select each CI attribute you want to add to the Specific Attributes list. For example, select the Product Name, Application Version Description, Vendor, Version, Description, The Database Instance Name, and Port attributes.



8. Click **OK** to save the query node properties.
9. Select any additional nodes that contain CI attributes you want to add to the integration. For example, **Node**.
10. Click the **Edit** button from the Information Pane. The Node properties window opens.
11. Click **Element Layout** tab, and select the option **Select attributes for layout**.
12. Select **Specific Attributes** in Attributes condition, and from the Available Attributes list select each CI attribute you want to add to the Specific Attributes list. For example, select attributes for the OS Vendor, and name attributes.
13. Click **OK** to save the query node properties.
14. Repeat step 9 to step 13 for each additional node that contains CI attributes that you want to add to the integration.
15. Click the **Save** button to save the query .

How to add the CI type in Service Manager

Before creating a new Service Manager CI type, you should determine if there are any existing CI types in your Service Manager system that provide the CI attributes you want. In most cases, you can reuse the existing CI types for the integration.

The integration doesn't require any special steps to add a CI type to Service Manager. You can use the standard CI type creation procedures to add a CI type. For more information on CI type creation, see the Service Manager online help.

To add a new CI type in Service Manager:

1. Create a table for storing the specific attributes of this new CI type.
2. Create a join definition to join the device table.
3. Create an erddef definition that defines a relationship between the two tables.
4. Create a view form and a bulk update form for the new CI type.
5. Add the CI type.

The following steps illustrate how to create a new CI type called **RDBMS**.

> **Note**
>
> This example is provided only as an illustration of the steps. The best practice is to reuse the existing Service Manager CI type **RunningSoftware** to map with UCMDB CI type **SM RDBMS**.

To create a table in Service Manager:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Database Dictionary**.
3. In File Name, type the table name you want to add. For example, rdbms.
4. Click the **New** button  .
5. Click the **Fields** tab.
6. Click the **New Field/Key** button. The Add Field window opens.
7. In Name, type a field name you want to add. For example, logical.name, which is mandatory for joining the **device** table.

   > **Caution**
   >
   > The name can't include any of the following characters: ' / \ [ ] : | < > + = ; , ? *.

8. In Type, select a type from the list. For example, select **character**.
9. Click the **Add Field** button  to save the attribute.
10. Repeat step 7 to step 9 for each attribute you wish to add. For example, dbinstance and port.
11. Click the **Keys** tab.
12. Place the cursor on the first line of the structure, and click the **New Field/Key** button. The Add Key window opens.
13. In Type, select **primary** from the list.
14. In Fields list, type the name of a field that you want to use as the primary key. For example, logical.name.
15. Click **Add Key** button  to save the key.
16. Click the **OK**button to save attribute changes to the table.

To create a join definition in Service Manager:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Database Manager**.
3. In Table, type the table name joindefs.
4. Click the **Search** button  .
5. In Join Table Name, type a name for the join definition. For example, joinRDBMS.
6. In File Names, select the names of the tables to join. For example, **device** and **rdbms**.
7. Click **Add** to save the join definition.

To create an ERD definition in Service Manager:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Database Manager**.
3. In Table, type the table name erddef.
4. Click the **Search** button  .
5. In First Filename, type the name of the first table of the join definition. For example, device.
6. In Second Filename, type the name of the second table of the join definition. For example, rdbms.
7. In Relationship type, select a value from the list. For example, One to One.
8. In Field Names from First Filename, add the unique field name of first table. For example, logical.name.
9. In Field Names from Second Filename, add the unique field name of the second table. For example, logical.name.
10. Click the **Add** button to save the ERD definition.

To create forms for view and bulk update in Service Manager:

Create a view form named **configurationItemRDBMS**, and a bulk update form named **device.rdbms.bulkupdate**.

You can create them in Forms Designer based on existing view forms and bulk update forms. To view the form names of an existing CI type in Service Manager, click **Configuration Management** > **Resources** > **CI Types** > **Search**, and then open the CI type record.

To access Forms Designer in Service Manager, type fd in the command line or go to **Tailoring** > **Forms Designer**.

For more information about creating forms in Service Manager, see the Service Manager online help and *Tailoring Best Practices*.

To add a CI type to Service Manager:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Configuration Management** > **Administration** > **Add New CI Type**, and click **Next**.

3. In the CI Type Name field, type a descriptive name for the new CI type. For example, RDBMS.
4. In the CI Type field, type a name for the new CI type. For example, rdbms.
5. Click **Next**.
6. In View Form, type the name of the view form you created for the new CI type. For example, configurationItemRDBMS.
7. In Bulk Update Form, type the name of the bulk update form you created for the new CI type. For example, device.rdbms.bulkupdate.
8. Click **Next**.
9. In Attribute File, select the table you created for the CI type. For example, **rdbms**.
10. Click **Next**.
11. Click **Next** to keep the default setting for Fields Specific to the Attribute File.
12. In the Join Def Record field, select the join definition you created for this CI type. For example, **joinRDBMS**.
13. Click **Next**.
14. In the Subtypes field, add necessary subtypes for the CI type. For example, Oracle and SQL Server.
15. Click **Next**.
16. Check the **Activate CI Type** checkbox.
17. Click **Next** to save the new CI type.

How to create Web Service fields to support the CI type

In order to add a CI type to the integration, you must create a Service Manager web service object for each CI attribute for which you created an XSL transformation on the UCMDB system. Service Manager uses the web service object to determine which Service Manager table and column to store the incoming CI attribute values.

The following steps illustrate how to create a web service object necessary to support the SM RDBMS CI type described in previous sections.



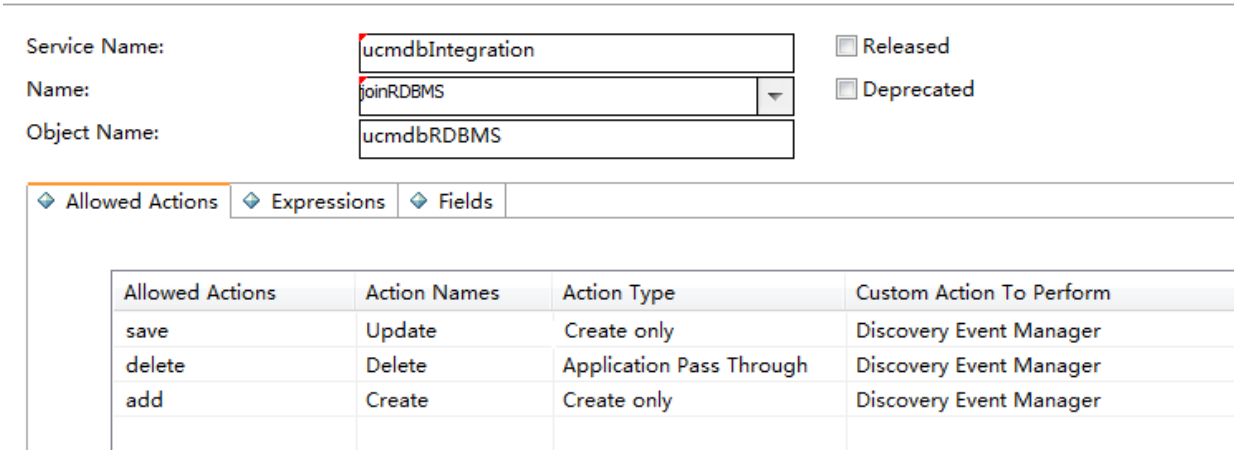
Note

This example of creating a new web service object (ucmdbRDBMS) is provided only as an illustration of the steps. The best practice is to reuse the existing Service Manager web service object ucmdbRunningSoftware to map with Universal CMDB CI type SM RDBMS.

To create web service fields to support your new CI type:

1. Sign in to Service Manager with an administrator account.
2. Navigate to **Tailoring** > **Web Services** > **WSDL Configuration**.
3. In Service Name, type ucmdbIntegration.
4. In Name, select the name of the join file you have created for the new CI type. For example, **joinRDBMS**.
5. In Object Name, type a name. For example, ucmdbRDBMS.

6. Click the **Allowed Actions** tab, and specify the actions as shown in the following figure.



> **Caution**
>
> For UCMDB integration WSDL configurations, be sure to use the "Create only" action type for the "add" and "save" actions, and "Application Pass Through" for the "delete" action. For more information about the action types, see *Service Manager Web Services*.

7. Click **Add** to create the WSDL configuration.
8. Click the **Fields** tab, select fields from the list and type a caption for each of them, as shown in the following table.

Field	Caption
ucmdb.id	UCMDBId
ci.name	ApplicationName
type	Type
subtype	SubType
company	CustomerId
sm.device.display.name	DisplayName
description	Description

9. Click **Save** to save WSDL configuration changes.

   The new web service fields are now available to the integration.

How to add managed fields to support the CI type

In order for your custom CI type to trigger the automated Change Management validation and verification processes, you must add a managed field for each CI attribute within your CI type. Service Manager managed fields are part of the Discovery Event Manager Rules. For a list of fields that trigger Change Management validation and verification, see Service Manager Discovery Event Manager rules.

The following steps illustrate how to add the managed fields for the SM RDBMS CI type described in the previous sections.

To add managed field to support your CI type:

1. Sign in to Service Manager as a system administrator.
2. Navigate to **Tailoring** > **Web Services** > **Discovered Event Manager Rules**.
3. In ID, type the ID you want to create for the new Discovered Event Manager Rule. For example, ucmdbRDBMS.
4. In Table Name, select the table or join definition you want to associate to the new Discovered Event Manager Rule. For example, **joinRDBMS**.
5. Click **New**.
6. Click **Next**.
7. Click the **Managed Fields** tab.



Note

The list of fields you will add here will trigger Change Management validation and verification.

8. Select an empty row in the Managed Fields list.
9. For Field Name, select the caption names of the Service Manager fields that you previously selected to store the incoming CI attribute values. See *How to create Web Service fields to support the CI type*.



Note

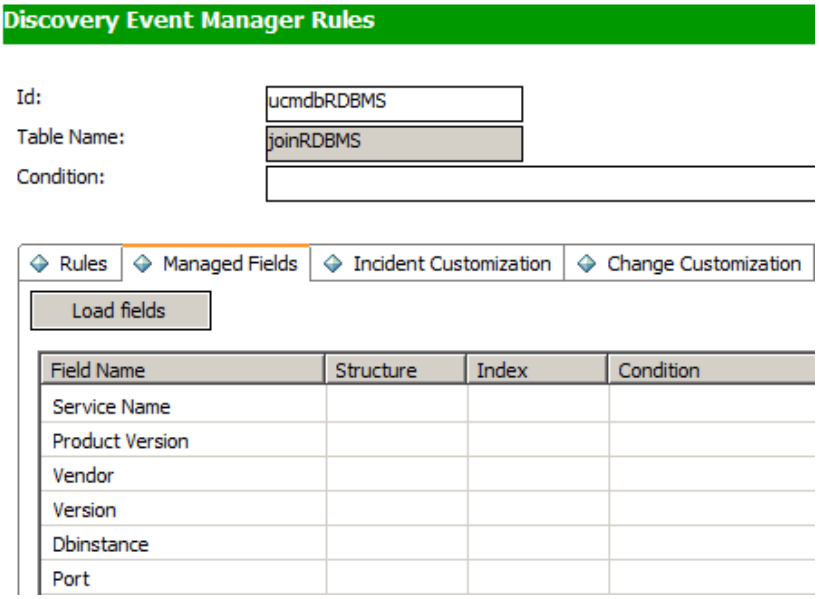
Service Manager displays the fields from all join tables associated with the table specified in the Table Name field. For example, for the **joinRDBMS** table, the fields from the **device** and **rdbms** tables are available from the Field Name list.



Tip

If you want to add all fields that are exposed in the WSDL definition, you can click the **Load Fields** button. For more information, see How do I use the Load Fields button to add multiple managed fields.

10. If the field you use to store the incoming CI attribute is an array of structure, use the Structure field to select the name of the array of structure where the column can be found. For example, Vendor is a primitive character field and therefore doesn't need to identify an array of structure name.
11. If the field you use to store the incoming CI attribute is an array of structure, use the Index field to select the index number that identifies the column in the array of structure. For example, Vendor is a primitive character field and therefore doesn't need to identify an array of structure index.
12. Click **Save**.



How to map the CI type's query to an XSL transformation file

The integration uses a configuration file called `smSyncConfFile.xml` to map each Universal CMDB query to an XSL transformation file. In order for custom queries to be part of the integration, you must add a mapping entry for each query in the configuration file.

The following steps illustrate mapping the query rdbmsData described in previous sections to the Service ManagerucmdbRDBMS web service.

Tip

When adding a query name in the `smSyncConfFile.xml` file, you can use a wildcard (an asterisk) in the query name. This is helpful in the debugging phase when you may have updated an out-of-the-box query and saved it as several query names. For example, if you have saved the <Query_name> query to <Query_name>_1, and <Query_name>_2, you can specify the query name as <Query_name>* in the configuration file, and the integration will automatically use this mapping entry on all of the three queries.

Note

Out-of-the-box, all query names in the `smSyncConfFile.xml` file are suffixed with a wildcard (an asterisk).

To map a query to an XSL transformation file:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files**.
3. Click the `smSyncConfFile.xml` file.
4. Add a query mapping element to the file by copying an existing one. A query mapping element uses the following format:

```
<tql name="Query" xslFile="XSL_File">
<!-- Description of mapping -->
<request type="Create" name="Create_web_service"/>
<request type="Update" name="Update_web_service"/>
<request type="Delete" name="Delete_web_service"/>
</tql>
```

`Query` is the name of the UCMDB query you created.

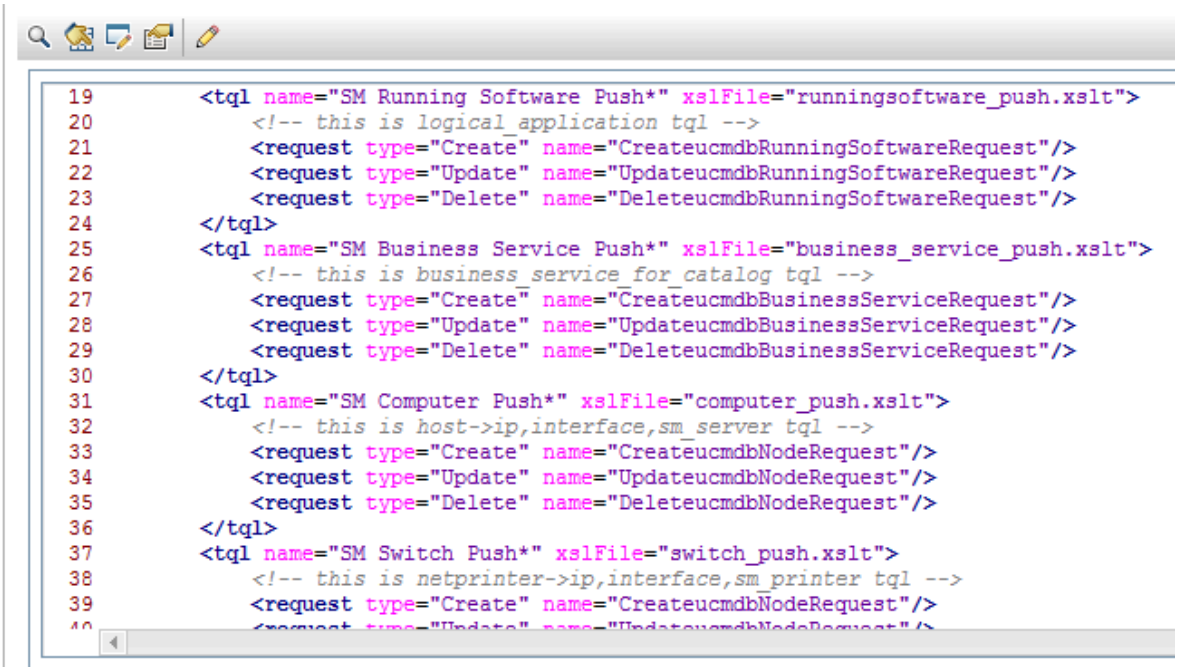
`XSL_File` is the name of the XSL transformation file the integration will use to map Universal CMDB attributes to Service Manager web service fields.

`Create_web_service` is the name of the Service Manager web service you want to the integration to use to create CIs from this query.

`Update_web_service` is the name of the Service Manager web service you want to the integration to use to update CIs in this query.

`Delete_web_service` is the name of the Service Manager web service you want to the integration to use to delete CIs from this query.

See the following screenshot for an excerpt of `smSynchConfFile.xml`:



```
19        <tql name="SM Running Software Push*" xslFile="runningsoftware_push.xslt">
20            <!-- this is logical_application tql -->
21            <request type="Create" name="CreateucmdbRunningSoftwareRequest"/>
22            <request type="Update" name="UpdateucmdbRunningSoftwareRequest"/>
23            <request type="Delete" name="DeleteucmdbRunningSoftwareRequest"/>
24        </tql>
25        <tql name="SM Business Service Push*" xslFile="business_service_push.xslt">
26            <!-- this is business_service_for_catalog tql -->
27            <request type="Create" name="CreateucmdbBusinessServiceRequest"/>
28            <request type="Update" name="UpdateucmdbBusinessServiceRequest"/>
29            <request type="Delete" name="DeleteucmdbBusinessServiceRequest"/>
30        </tql>
31        <tql name="SM Computer Push*" xslFile="computer_push.xslt">
32            <!-- this is host->ip,interface,sm_server tql -->
33            <request type="Create" name="CreateucmdbNodeRequest"/>
34            <request type="Update" name="UpdateucmdbNodeRequest"/>
35            <request type="Delete" name="DeleteucmdbNodeRequest"/>
36        </tql>
37        <tql name="SM Switch Push*" xslFile="switch_push.xslt">
38            <!-- this is netprinter->ip,interface,sm_printer tql -->
39            <request type="Create" name="CreateucmdbNodeRequest"/>
```

5. Add or update query mapping elements for each query you want to add to the integration. For example, the following query creates a mapping between the `rdbmsData` query and the `rdbms_push.xslt` file.

```
<tql name="rdbmsData" xslFile="rdbms_push.xslt">
<!-- this is database tql -->
<request type="Create" name="CreateucmdbRDBMSRequest"/>
<request type="Update" name="UpdateucmdbRDBMSRequest"/>
<request type="Delete" name="DeleteucmdbRDBMSRequest"/>
</tql>
```

6. Save the configuration file.

Note

When you create/edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

How to map the CI type's attributes to Web Service fields

The integration uses the Service Manager Adapter to transform UCMDB CI attributes to web services objects recognized by Service Manager. The Service Manager Adapter uses XSL transformation files to convert UCMDB queries into a properly formatted Service Manager web services messages. Out-of-the-box, each integration query has a corresponding XSL transformation file. In addition, each attribute you enable for synchronization from Advanced layout settings requires its own entry in the XSL transformation file.

If you want to add a CI type to the integration, you must create a matching XSL transformation file that defines how the Service Manager Adapter transforms each CI type into a Service Manager web service object. See Integration queries for information about which CI types each query manages. In order to create a proper XSL mapping, you must be familiar with the service and object names Service Manager publishes as Web services. See the Service Manager help for information on publishing tables and columns as Web service objects.

The following steps illustrate creating an XSL transformation file for the rdbmsData query described in previous sections.

To map a CI type's attributes to web service fields:

1. Sign in to UCMDB with an administrator account.
2. Navigate to **Data Flow Management** > **Adapter Management**.
3. Click the **Create New Resource** button  .
4. Select **New Configuration File**.
5. Select the ServiceManagerAdapter9.41 package.
6. Enter the full file name: <AdapterID>/<filename>. For example, ServiceManagerAdapter9.41/rdbms_push.xslt.
7. Copy the content of an existing XSL transformation file (for example, `runningsoftware_push.xslt`) to the new XSL transformation file.
8. Find the CI type definition element in the new file. The CI type definition element uses the following format:

   ```
   <xsl:template match="/CI_type_name">
   ```

   `CI_type_name` is the name of CI type in the UCMDB system.

   The following is an example from the `runningsoftware_push.xslt` file:

   ```
   <xsl:template match="/running_software">
   ```

9. Update the CI type name to match the CI type you want to add to the integration. For example, create the following CI type definition element to add the database CI type to the integration.

   ```
   <xsl:template match="/sm_rdbms">
   ```

10. Add or update table naming elements as needed. By default, UCMDB sends CI attribute data to the Service Manager device table. If you want to send CI attributes to one of the join tables of device, you must add an element to specify the table name using the format <file.table_name>. For example, you don't need to specify an additional jointable to define a database CI type since Service Manager doesn't use a separate jointable to manage database CI types.

11. Find the elements that transform UCMDB CI attributes into Service Manager web service fields. The CI attribute transformation elements use the following format:

```
<xsl:for-each select="@CI_attribute_name">
<WSFieldName><xsl:value-of select="."/></WSFieldName>
</xsl:for-each>
```

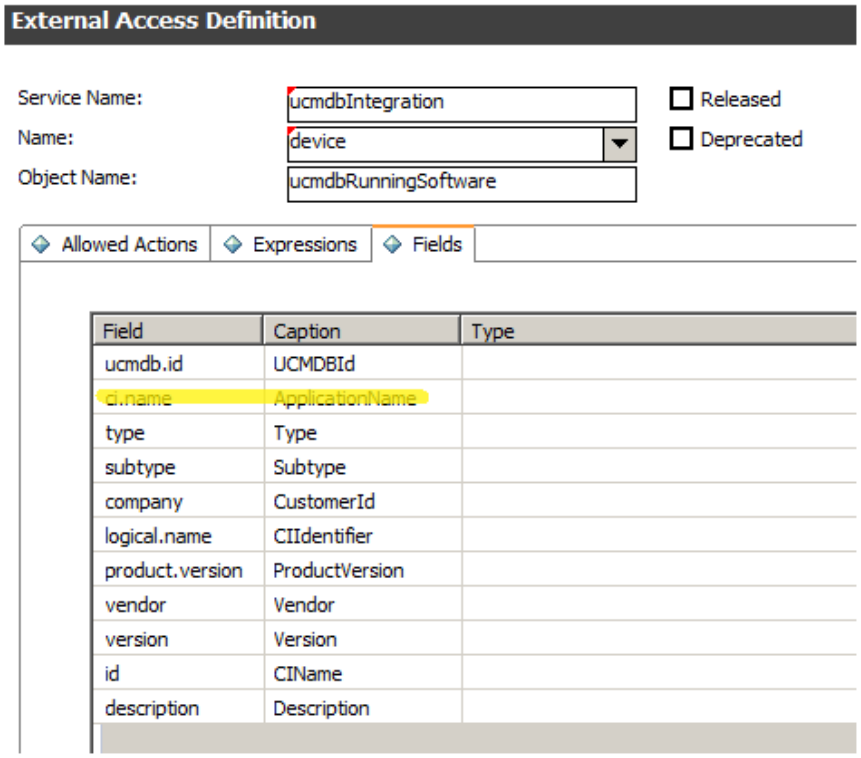
`@CI_attribute_name` is the name of the attribute in the UCMDB system.

`WSFieldName` is the name of a web service field published by the Service Manager system.

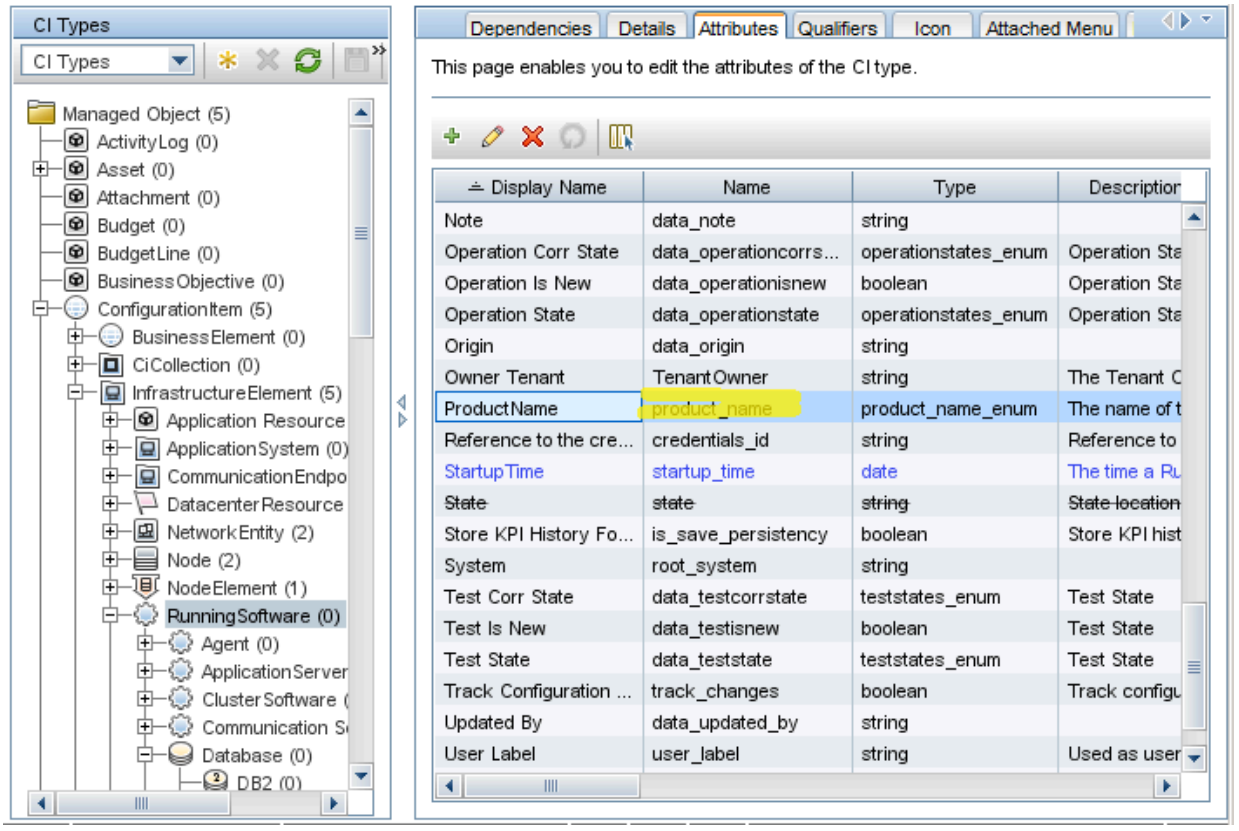
The following figures show an example for CI attributes in `runningsoftware_push.xslt`:

```
            <!--import:cmdb_root_attributes.xslt-->
             <Type>running_software</Type>
             <xsl:variable name="CIT" select="@bdmType"/>
             <Subtype><xsl:value-of select="$CIT"/></Subtype>
             <xsl:for-each select="$CIlists/list[@name='CIType_runningsoftware']">
                 <Subtype><xsl:value-of select="entry[@ucmdb=$CIT]/@sm"/></Subtype>
             </xsl:for-each>
             <xsl:variable name="fullDNSName" select="nodes/node/@primary_dns_name"/>
             <xsl:for-each select="@display_label">
                 <CIIdentifier><xsl:value-of select="$fullDNSName"/>_<xsl:value-of sele
ct="."/></CIIdentifier>
             </xsl:for-each>
             <xsl:for-each select="@product_name">
                 <ApplicationName><xsl:value-of select="."/></ApplicationName>
             </xsl:for-each>
             <xsl:for-each select="@application_version">
                 <ProductVersion><xsl:value-of select="."/></ProductVersion>
             </xsl:for-each>
             <xsl:for-each select="@vendor">
                 <Vendor><xsl:value-of select="."/></Vendor>
             </xsl:for-each>
             <xsl:for-each select="@version">
                 <Version><xsl:value-of select="."/></Version>
             </xsl:for-each>
         </instance>
     </model>
 </xsl:template>
```

Refer to the following screenshot to map CI attributes in the `ucmdbRunningSoftware` web service object:



Refer to the following screenshot to map CI attributes in the UCMDB `RunningSoftware` CI type:



12. Add or update CI attribute transformation elements for each CI attribute you want to add to the integration. For example, create the following XSL transformation elements for the database CI type.

Sample XSL transformation elements for database CIs

UCMDBattribute	Sample transformation elements
port	`<xsl:for-each select="@port"> <Port><xsl:value-of select="."/></Port> </xsl:for-each>`
database_dbsid	`<xsl:for-each select="@database_dbsid"> <DBInstance><xsl:value-of select="."/></DBInstance> </xsl:for-each>`
description	`<xsl:for-each select="@description"> <Description> <xsl:value-of select="."/> </Description> </xsl:for-each>`

New attribute mappings in `rdbms_push.xslt` are illustrated in the following screenshot:

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:template match="/sm_rdbms">
        <model>
            <keys/>
            <instance>
               <!--import:cmdb_root_attributes.xslt-->
                <Type>rdbms</Type>
                <xsl:variable name="CIT" select="@bdmType"/>
                <xsl:for-each select="@bdmType">
                    <Subtype><xsl:value-of select="."/></Subtype>
                </xsl:for-each>
                <xsl:for-each select="@display_label">
                    <CIIdentifier><xsl:value-of select="."/></CIIdentifier>
                </xsl:for-each>
                <xsl:for-each select="@product_name">
                    <ApplicationName><xsl:value-of select="."/></ApplicationName>
                </xsl:for-each>
                <xsl:for-each select="@application_version">
                    <ProductVersion><xsl:value-of select="."/></ProductVersion>
                </xsl:for-each>
                <xsl:for-each select="@vendor">
                    <Vendor><xsl:value-of select="."/></Vendor>
                </xsl:for-each>
                <xsl:for-each select="@version">
                    <Version><xsl:value-of select="."/></Version>
                </xsl:for-each>
                <xsl:for-each select="@port">
                    <Port><xsl:value-of select="."/></Port>
                </xsl:for-each>
                <xsl:for-each select="@database_dbsid">
                    <DBInstance><xsl:value-of select="."/></DBInstance>
                </xsl:for-each>
                <xsl:for-each select="@description">
                    <Description><xsl:value-of select="."/></Description>
                </xsl:for-each>
            </instance>
        </model>
    </xsl:template>
</xsl:stylesheet>
-- SELECT --                                                    9          35,32
```

13. Save the new XSL transformation file.

> **Note**
>
> When you create/edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

How to add custom queries to data push jobs

In order for the integration to send your custom CI types and attributes to your Service Manager system, you must add your custom queries to the data push job between your Changes data store and your Service Manager data store. The following steps illustrate how to add an custom query named rdbmsData, which is described in the previous sections.

To add custom queries to a data push job:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**.
3. Click the name of your Service Manager integration point. For example, **SM Integration**.
4. Click the **Data Push** tab.
5. Click the name of your data push job. For example, **SM Configuration Item Push job**.
6. Click the **Edit** button  .
7. Click the **Add** button. 
8. Click **Integration** > **SM Sync** > **rdbmsData**.
9. Click **OK** to add the custom query.
10. Enable the **Allow Integration Job to delete removed data** option for the query.
11. Click **OK** to close the Update Job Definition window.

1.6.3.5.2.7. How to add a CI type's relationship types to the integration for data push

Once you have added a new CI type to the integration and have created relationships between it and other CI types in UCMDB, for each of these relationship types you need to perform the following tasks so that UCMDB can push the relationships to Service Manager.

As an example, the following steps illustrate how you add a relationship type named Ownership (between the Cost and CostCategory CI types) to the integration for data push. These steps assume that you have already added the Cost and CostCategory CI types to the integration and have created an Ownership relationship between them in UCMDB.

1. Add a mapping entry for each relationship type in the push relationship mapping definition file.

   See *How to add a push mapping entry for each relationship type of the CI type* below.

2. Create a query to push relationships of the CI type.

   See *How to create a query to push each relationship type of the CI type* below.

3. Map the relationship query to an XSL transformation file in the push configuration file.

   See *How to map each relationship type query to an XSL transformation file* below.

How to add a push mapping entry for each relationship type of the CI type

For data push, the `SM_CIT_Subtype_list.xml` file defines how UCMDB relationship types are mapped to SM ones.

> **Note**
>
> This XML file can be found from **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files**.

If this file doesn't contain a mapping entry for a new relationship type, you need to add an entry for it.

To add a push mapping entry for the Ownership relationship type:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files**.
3. Double-click the `SM_CIT_Subtype_list.xml` file.
4. Go to the `RelationshipType` list section, and add a mapping entry for Ownership.

```
<list name="RelationshipType">

<!--ucmdb is link display lable, ucmdbType is link type, sm is the
relationship type.
compound link will use root_* as the lable -->

    <entry ucmdb="Aggregation" ucmdbType="aggregation"
sm="Aggregation"  />

    ...

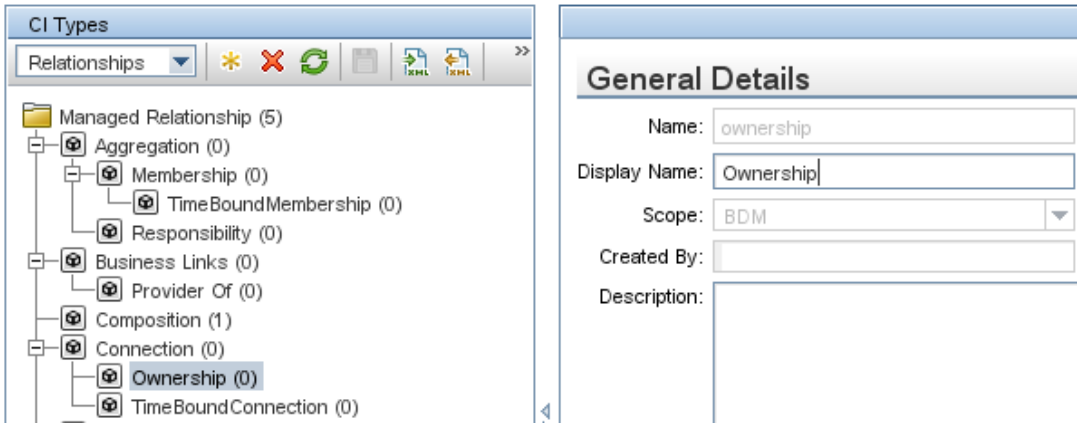
<entry ucmdb="Ownership"  ucmdbType="ownership"  sm="Ownership" />

    ...

</list>
```

Where:

ucmdb: the display name of the UCMDB relationship type. Name and Display Name of Ownership are illustrated in the following screenshot:



5. Click **OK** to save the file.

How to create a query to push each relationship type of the CI type

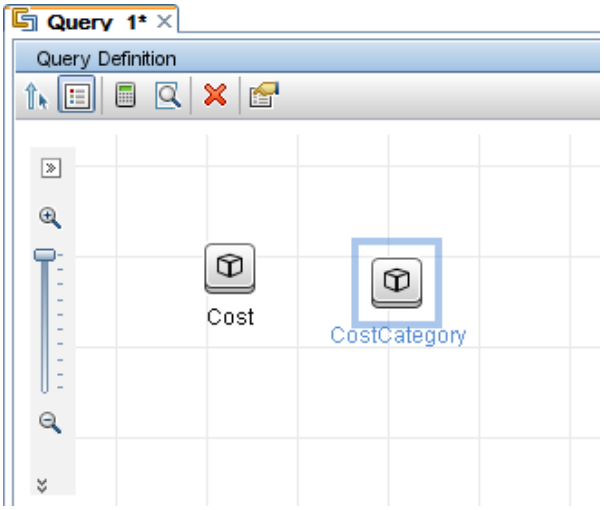
Once you have created CI Relationship types for the new CI type, you must create a query for each relationship type to push it to Service Manager.

Note

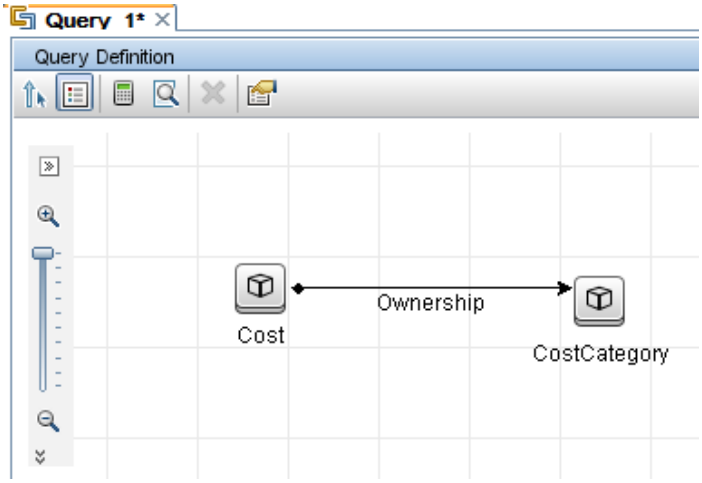
Any query you create must conform to the Query requirements.

To create a new query called cost_costcategory_ownership_relation_push for Ownership relationships between the Cost and CostCategory CI types:

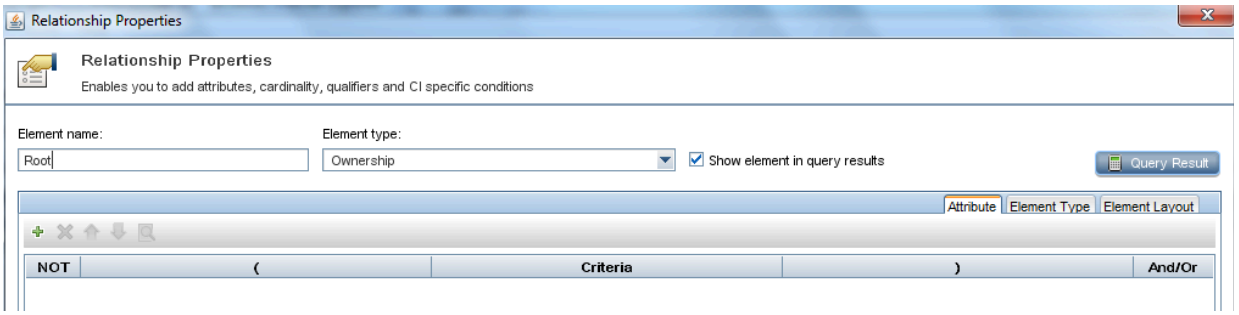
1. Sign in to UCMDB as an administrator.
2. Navigate to **Modeling** > **Modeling Studio**.
3. Click **New** > **Query**. The Query Definition pane displays.
4. From the CI Type Selector, drag the Cost and CostCategory CI types to the query pane.



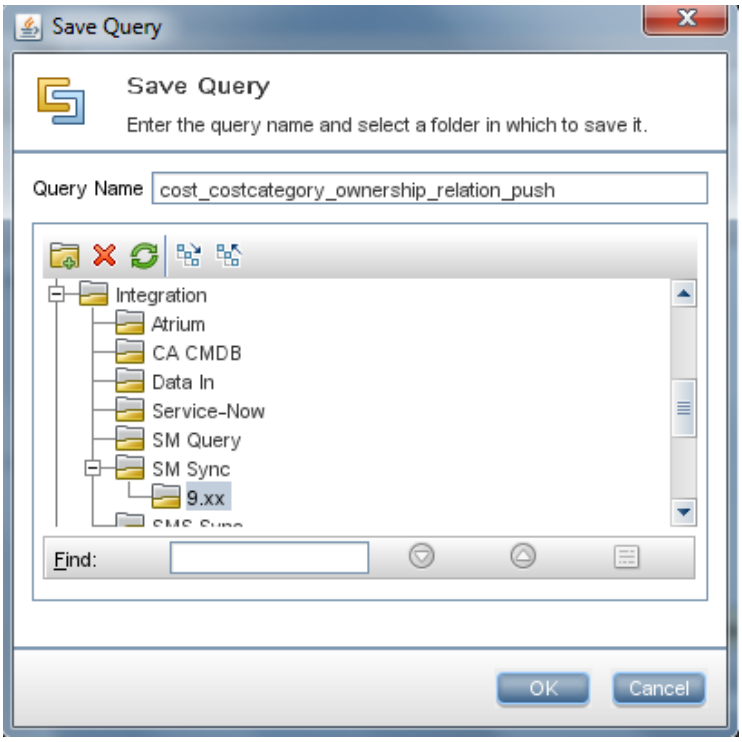
5. Create an Ownership relationship from Cost to CostCategory.
    1. Click the **Create Relationship** button.
    2. Click the **Cost** node, and drag the arrow from it to the CostCategory node.
    3. Select **Regular Relationship**, and click **OK**.
    4. Select **Connection** > **Ownership**, and click **OK**. An Ownership relationship is created between the CI types.



6. Right-click the relationship arrow, and select **Relationship Properties**.
7. Change the element name from **Ownership** to **Root** (or a name starting with "Root_"), and then click **OK**.



8. Click the **Save** button, and save the query as described in the following.
    1. Enter a query name. For example, cost_costcategory_ownership_relation_push.
    2. Select the **Integration** > **SM Sync** > **9.xx** folder.
    3. Click **OK**.



The query is now created. You are ready to map this query to an XSL transformation file.

How to map each relationship type query to an XSL transformation file

Once you have created a query for a relationship type, you need to map the query to an XSL transformation file as described in the following steps.

Tip

Out-of-the-box, there is a common XSL transformation file (`common_relations.xslt`), which is used for pushing all types of CI relationships. For this reason, you don't need to create a new XSL transformation file; instead, you only need to map the new relationship query to this existing XSLT file.

1. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files**.
2. Click the `smSyncConfFile.xml` file.
3. Add a query mapping entry by copying an existing one for relationship push. For example, copy the following query mapping entry.

```
<tql name="SM Layer2 Topology Relations Push*"
xslFile="common_relations.xslt">
            <request type="Create" name="CreateRelationship"/>
            <request type="Update" name="UpdateRelationship"/>
            <request type="Delete" name="DeleteRelationship"/>
</tql>
```

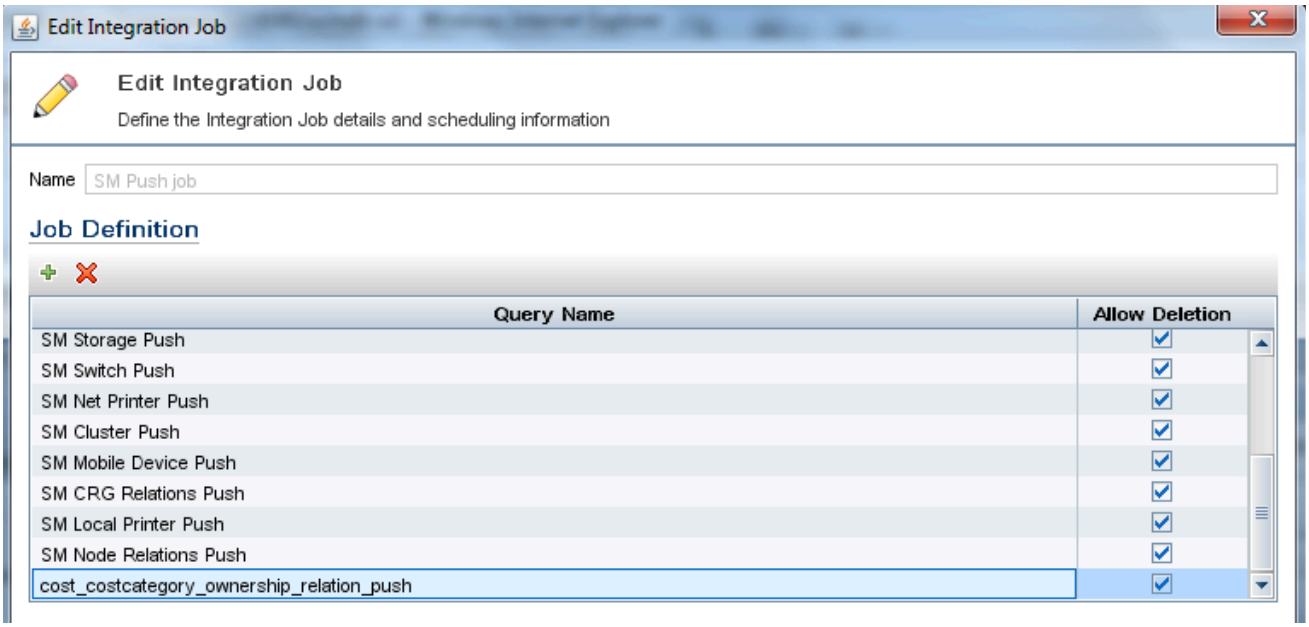
4. Change the query name to the name of the query you created for the relationship type. For example, cost_costcategory_ownership_relation_push.

```
<tql name="cost_costcategory_ownership_relation_push"
xslFile="common_relations.xslt">
            <request type="Create" name="CreateRelationship"/>
            <request type="Update" name="UpdateRelationship"/>
            <request type="Delete" name="DeleteRelationship"/>
 </tql>
```

5. Click **OK** to save the configuration file.

Now, you have added the new relationship type to the integration. Next, you need to add the new relationship query to a data push job (see the following figure and How to add custom queries to data push jobs).

Refer to the following screenshot to add a new relationship query to a data push job.


Edit Integration Job
Edit Integration Job
Define the Integration Job details and scheduling information
Name
SM Push job
Job Definition
Query Name
Allow Deletion
SM Storage Push
SM Switch Push
SM Net Printer Push
SM Cluster Push
SM Mobile Device Push
SM CRG Relations Push
SM Local Printer Push
SM Node Relations Push
cost_costcategory_ownership_relation_push

1.6.3.5.2.8. How to Add a CI Attribute to the Integration for Population

To add a CI attribute to the integration for population, perform the following tasks:

1. Create a web service field to support the CI attribute.

   See *Create a Web Service field to support the CI attribute* below.

2. Map the CI attribute to the web service field.

   See *How to map the CI attribute to the Web Service field* below.

Create a Web Service field to support the CI attribute

UCMDB uses the Service Manager ucmdbIntegration web service to retrieve CI data from Service Manager. This web service publishes the objects that match the out-of-the-box CI types and CI attributes provided by the UCMDB integration.

If you want to populate an additional CI attribute from Service Manager to your Universal CMDB system, you must create a corresponding web service field on your Service Manager system to provide the CI data from Service Manager. Each web service field must map to a valid Service Manager table and column.

For the steps of creating a web service field, see How to create a Web Service field to support the CI attribute.

How to map the CI attribute to the Web Service field

The integration uses an adapter to transform Service Manager web service fields to Universal CMDB CI attributes. The adapter in turn specifies what XSL transformation files the integration should use to convert Service Manager web services messages into a properly formatted Universal CMDB CI and/or relationship.

Out-of-the-box, each integration query has a corresponding XSL transformation file that maps to a particular CI type in Universal CMDB. Without an XSL transformation entry, Universal CMDB can't receive any CI attribute updates from your Service Manager system.

Unlike the Push feature, you don't need to create real queries for Population on the UCMDB server.

If you want to add a new attribute to the integration, you must edit the XSL transformation file for the parent CI type and add an entry for the CI attribute. For information about which CI types each population query manages, see Integration queries. In order to create a proper XSL mapping, you must be familiar with the service and object names that Service Manager publishes as Web services. For information on publishing tables and columns as Web service fields, see the Service Manager help.

The following steps illustrate how to map a UCMDB CI attribute called host_vendor to a Service Manager web service field called OSVendor.

To map a CI attribute to a web service field:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management**.
3. Navigate to the Service Manager Adapter configuration files path: `ServiceManagerAdapter9.41 > Configuration Files`.
4. Click the XSL transformation file that manages the parent CI type of your CI attribute. For example, open computer_population.xslt to add an attribute to the **SM Computer Population** query.
5. Find the element that defines the name of the Universal CMDB CI Type where the integration will store CI attribute values. For example, the element `<ci class="node">` will store CI attributes in the Universal CMDB Node CI Type.
6. Within the ci naming element (`<ci class="node">`), you will see an element of the following format that defines how to transform each web service field into an Universal CMDB CI attribute:

   ```
   <attribute name="UCMDB_CI_attribute_name"
   type="UCMDB_CI_attribute_type" ignoreCIIfEmpty="true">
   <xsl:value-of select="SMAttributeName "/></attribute>
   ```

   `UCMDB_CI_attribute_name` is the name of attribute in the Universal CMDB system.

   `UCMDB_CI_attribute_type` is the type of attribute of the Universal CMDB system which this integration supports. Currently the following types are supported: String, StringList, Integer, Long, Double, Boolean, IPAddress, Date, Float, and IntList.

   `ignoreCIIfEmpty` is a parameter that specifies whether or not to ignore the CI during population if this attribute has an empty value (true: ignore; false: not ignore).

   Note

   - A StringList is a list of strings separated by a semicolon (;). For example, str1;str2;str3. An IntList is a list of integers separated by a semicolon (;). For example, 1;2;3.
   - For information about time zone and date format configuration of the Date type, see How to update the time zone and date format for the integration adapter.

   `SMAttributeName` is the name of a web service attribute published by the Service Manager system.

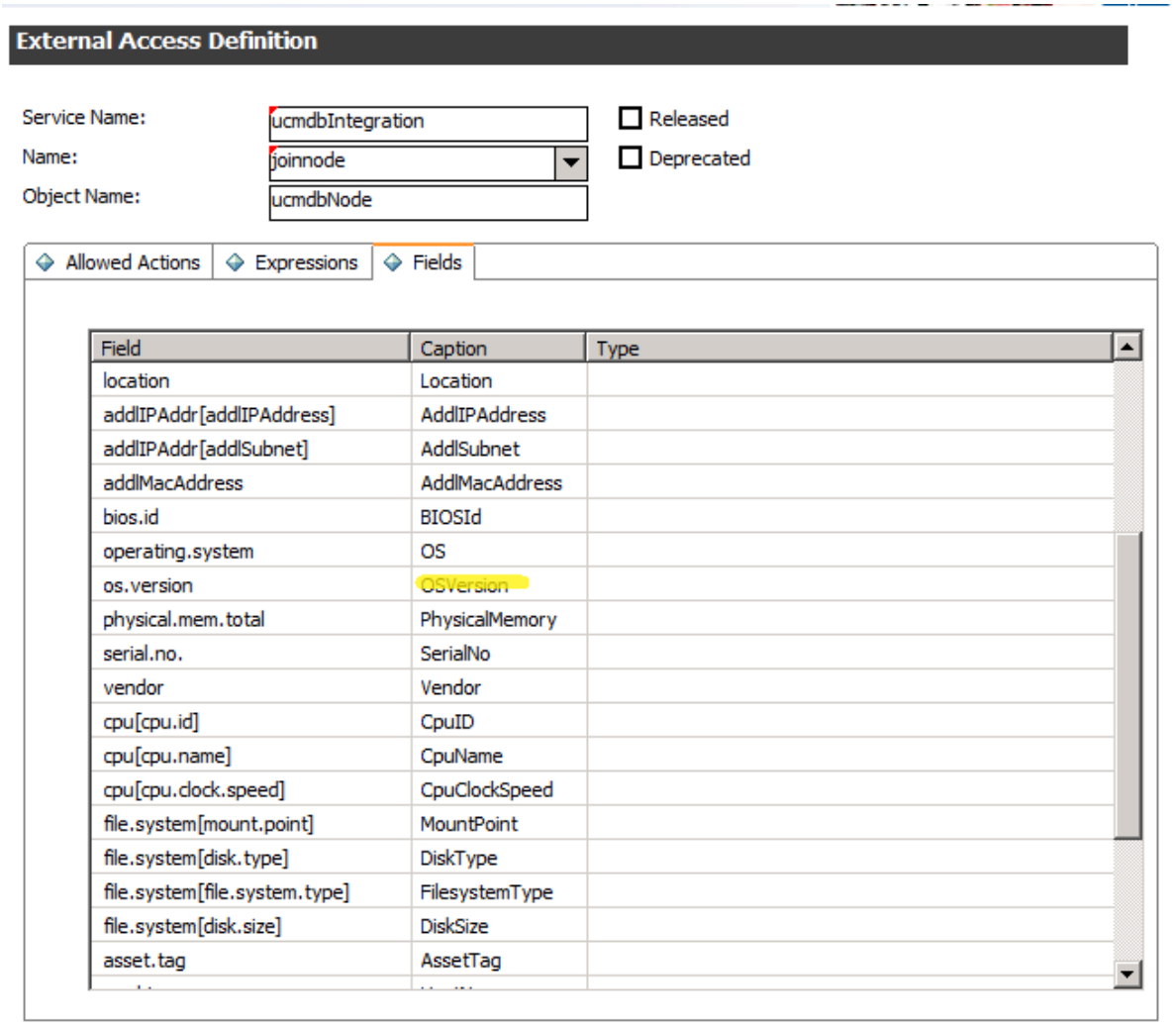
   CI attributes in the `computer_population.xslt` file are illustrated in the following screenshot:

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:template match="/RetrieveucmdbNodeListResponse">
        <topology>
            <xsl:for-each select="instance">
                <xsl:variable name="building" select="file.device/Building"/>
                <xsl:variable name="floor" select="file.device/Floor"/>
                <xsl:variable name="room" select="file.device/Room"/>

                <ci class="node">
                    <!--import:cmdb_root_attributes_population.xslt-->
                    <attribute name="primary_dns_name" type="String"><xsl:value-of select="file.device/DNSName"/></attribute>
                    <attribute name="calculated_location" type="String"><xsl:value-of select="concat('Building:', $building,
                    <attribute name="discovered_os_name" type="String"><xsl:value-of select="file.device/OS"/></attribute>
                    <attribute name="default_gateway_ip_address" type="String"><xsl:value-of select="file.device/DefaultGatewa
                    <attribute name="discovered_os_version" type="String"><xsl:value-of select="file.node/OSVersion"/></attrib
                    <attribute name="memory_size" type="Integer"><xsl:value-of select="file.node/PhysicalMemory"/></attribute>
                    <!--import:ips_mapping_population.xslt-->
                    <!--import:interfaces_mapping_population.xslt-->
                </ci>
            </xsl:for-each>
        </topology>
    </xsl:template>
</xsl:stylesheet>

```

Refer to the following screenshot to match CI attributes in the `ucmdbNode` web service object:



7. Copy an existing XSL transformation element to use it as a template to create a new transformation entry.
8. Paste the new XSL transformation element within the proper table naming element. For example, <ci class="node">.

9. Update the CI attribute name and web service field name within the new element to match the attribute you want to add to the integration. For example, create the following XSL transformation element to add the `os_vendor` attribute to the integration.

```
<attribute name="os_vendor" type="String"><xsl:value-of
select="file.node/OSVendor"/></attribute>
```

New attribute in the `computer_population.xslt` file is illustrated in the following screenshot:



```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:template match="/RetrieveucmdbNodeListResponse">
        <topology>
            <xsl:for-each select="instance">
                <xsl:variable name="building" select="file.device/Building"/>
                <xsl:variable name="floor" select="file.device/Floor"/>
                <xsl:variable name="room" select="file.device/Room"/>

                <ci class="node">
                    <!--import:cmdb_root_attributes_population.xslt-->
                    <attribute name="primary_dns_name" type="String"><xsl:value-of select="file.device/DNSName"/></attribute>
                    <attribute name="calculated_location" type="String"><xsl:value-of select="concat('Building:', $building,
                    <attribute name="discovered_os_name" type="String"><xsl:value-of select="file.device/OS"/></attribute>
                    <attribute name="default_gateway_ip_address" type="String"><xsl:value-of select="file.device/DefaultGatewa
                    <attribute name="discovered_os_version" type="String"><xsl:value-of select="file.node/OSVersion"/></attril
                    <attribute name="memory_size" type="Integer"><xsl:value-of select="file.node/PhysicalMemory"/></attribute>
                    <attribute name="os_vendor" type="String"><xsl:value-of select="file.node/OSVendor"/></attribute>
                    <!--import:ips_mapping_population.xslt-->
                    <!--import:interfaces_mapping_population.xslt-->
                </ci>
            </xsl:for-each>
        </topology>
    </xsl:template>
</xsl:stylesheet>

```

10. Save the XSL transformation file.

When you create/edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

1.6.3.5.2.9. How to add a CI type to the integration for population

You can use the following steps to add a CI type to the integration for the population.

1. Create a query to populate the CI type.

   See *Create a query to populate the CI type* below.

2. Map the CI type's query to an XSL transformation file.

   See *How to map the CI type's query to an XSL transformation file* below.

3. Map the CI type's attributes to web service fields.

   See *How to map the CI type's attributes to Web Service fields* below.

Create a query to populate the CI type

Unlike for push, for population the integration doesn't require you to create custom queries in Universal CMDB to save CI attribute values.

The population feature only needs the smPopConfFile.xml file and population XSL transformation files to synchronize CI/CI Relationship types and attributes; however for each CI/CI Relationship type you still need to define a query mapping in the smPopConfFile.xml file, and the query doesn't necessarily have to exist in UCMDB. It is simply a query name, which will appear in the query list when you add queries to a population job.

We still recommend you create queries to help you better understand what CI types or attributes are part of the population. For information on how to create a query in UCMDB, see How to create a query to synchronize the CI type.

How to map the CI type's query to an XSL transformation file

The integration uses a configuration file called **smPopConfFile.xml** to map each Universal CMDB query to an XSL transformation file. In order for custom queries to be part of the integration, you must add a mapping entry for each query in the configuration file.

To map the query rdbmsData described in previous sections to the Service ManagerucmdbRDBMS web service:

To map a query to an XSL transformation file:

1. Sign in to UCMDB with an administrator account.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files** > **smPopConfFile.xml**.

3. Add a query mapping element by copying an existing one. A query mapping element uses the following format:

```
<tql name="Query" xslFile="XSL_File">
            <request type="Retrieve"
   dataType="Data_Type"
       retrieveFileList="Retrieve_SM_Tables"
       retrieveKeysQueryName="Retrieve_keys_web_service"
       retrieveListQueryName="Retrieve_Objects_web_service"
       ballQueryCondition="Full_Query_Condition"
       changedUpdateQueryCondition="Changed_Update_Condition"
       changedDeletionQueryCondition="Changed_Deletion_Condition"/>
</tql>
```

`Query` is a query name. The query doesn't have to exist in UCMDB.

`XSL_File` is the name of the XSL transformation file that the integration will use to map Service Manager web service fields to Universal CMDB attributes.

`Data_Type` is the type of the object retrieved from Service Manager. The possible values are `ci` and `relationship`.

`Retrieve_SM_Tables` is the table name list in the Service Manager web service configuration; if you define the WSDL on a join definition, you need to list all the table names of the join definition to which the fields of XSLT mapping belongs; if you define the WSDL on a simple table, simply leave Retrieve_SM_Tables empty. You can also check this by opening the WSDL URL: `http://<SM server>:<port>/SM/7/<object name>.wsdl`. For example, out-of-the-box, the instance type of ucmdbNode is as shown in the following figure, so the Retrieve_SM_Tables value is `file.device, file.node`.

SM tables of the `ucmdbNode` instance type in the `ucmdbNode` WSDL are illustrated in the following screenshot:

```
- <xs:complexType name="ucmdbNodeInstanceType">
  - <xs:sequence>
    - <xs:element name="file.device">
      - <xs:complexType>
        - <xs:complexContent>
          - <xs:extension base="cmn:StructureType">
            - <xs:sequence>
                <xs:element minOccurs="0" name="CIIdentifier" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Vendor" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="DNSName" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="SerialNo" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Location" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Type" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="CIName" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Subtype" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Building" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Floor" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Room" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="AssetTag" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="OS" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="CustomerId" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="DefaultGateway" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="UCMDBId" nillable="true" type="cmn:StringType" />
              </xs:sequence>
            </xs:extension>
          </xs:complexContent>
        </xs:complexType>
      </xs:element>
    - <xs:element name="file.node">
      - <xs:complexType>
        - <xs:complexContent>
          - <xs:extension base="cmn:StructureType">
            - <xs:sequence>
              - <xs:element minOccurs="0" name="addlIPAddr">
                - <xs:complexType>
                  - <xs:complexContent>
                    - <xs:extension base="cmn:ArrayType">
```

`Retrieve_keys_web_service` is the name of the Service Manager web service that you want the integration to use to retrieve a CI Key list from Service Manager.

`Retrieve_Objects_web_service` is the name of the Service Manager web service that you want the integration to use to retrieve a CI Object list from Service Manager.

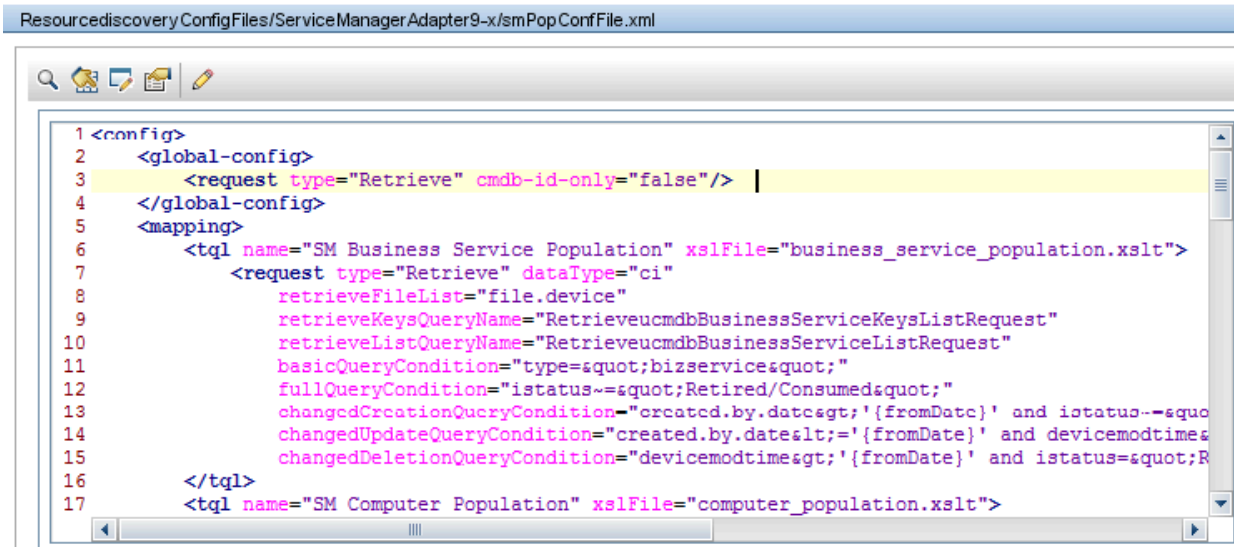
`Basic_Query_Condition` is an internal Query Condition of Service Manager that you want the integration to use to retrieve a CI list from Service Manager for a CI type; it is used as the basic condition for a full or changes population.

`Full_Query_Condition` is an additional internal Query Condition of Service Manager. Along with the Basic_Query_Condition, it is used to retrieve a list of all CIs from Service Manager for a full population.

`Changed_Update_Condition` is an additional internal Query Condition of Service Manager; along with the Basic_Query_Condition, it is used to retrieve a list of updated CIs from Service Manager since the last job execution; it includes both updated CIs and newly created CIs. It is used for a changes population.

`Changed_Deletion_Condition` is an additional internal Query Condition of Service Manager; along with the Basic_Query_Condition, it is used to retrieve a list of deleted CIs from Service Manager since the last job execution. It is used for a changes population.

The following figure shows an excerpt of the `smPopConfFile.xml` :

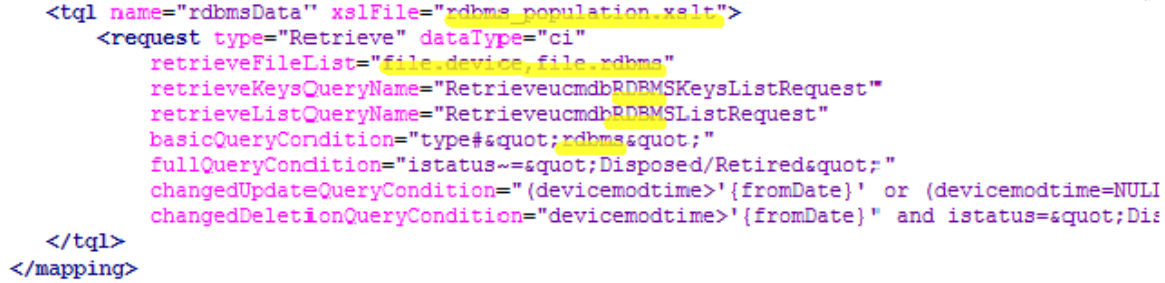


4. Add or update query mapping elements for each query you want to add to the integration.

   For example, the following query creates a mapping between the `rdbmsData` query and the `rdbms_population.xslt` file.

```
<tql name="rdbmsData" xslFile="rdbms_population.xslt">
    <request type="Retrieve" dataType="ci"
        retrieveFileList="file.device,file.rdbms"
        retrieveKeysQueryName="RetrieveucmdbRDBMSKeysListRequest"
        retrieveListQueryName="RetrieveucmdbRDBMSListRequest"
        basicQueryCondition="type#&quot;rdbms&quot;"
        fullQueryCondition="istatus~=&quot;Disposed/Retired&quot;"
        changedUpdateQueryCondition="(devicemodtime>'{fromDate}' or
(devicemodtime=NULL and created.by.date>'{fromDate}')) and
istatus~=&quot;Disposed/Retired&quot;"
        changedDeletionQueryCondition="devicemodtime>'{fromDate}'
and istatus=&quot;Disposed/Retired&quot;"/>
</tql>
```

   The following figure shows the above-mentioned query mapping elements in the `smPopConfFile.xml` file:



   The following figure shows an excerpt of the `ucmdbRDBMS` WSDL for your reference:

```
- <xs:complexType name="ucmdbRDBMSKeysType">
  - <xs:sequence>
      <xs:element minOccurs="0" name="CIIdentifier" nillable="true" type="cmn:StringType" />
    </xs:sequence>
    <xs:attribute name="query" type="xs:string" use="optional" />
    <xs:attribute name="updatecounter" type="xs:long" use="optional" />
  </xs:complexType>
- <xs:complexType name="ucmdbRDBMSInstanceType">
  - <xs:sequence>
    - <xs:element name="file.device">
      - <xs:complexType>
        - <xs:complexContent>
          - <xs:extension base="cmn:StructureType">
            - <xs:sequence>
                <xs:element minOccurs="0" name="CIIdentifier" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Vendor" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Type" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="CIName" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Description" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="Version" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="SubType" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="CustomerId" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="ApplicationName" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="UCMDBId" nillable="true" type="cmn:StringType" />
                <xs:element minOccurs="0" name="ProductVersion" nillable="true" type="cmn:StringType" />
              </xs:sequence>
            </xs:extension>
          </xs:complexContent>
        </xs:complexType>
      </xs:element>
    - <xs:element name="file.rdbms">
      - <xs:complexType>
        - <xs:complexContent>
          - <xs:extension base="cmn:StructureType">
            - <xs:sequence>
                <xs:element minOccurs="0" name="DBInstance" nillable="true" type="cmn:StringType" />
```

5. Save the configuration file.

> **Note**
>
> When you create/edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

How to map the CI type's attributes to Web Service fields

The integration uses the Service Manager Adapter to transform Service Manager web services objects to Universal CMDB CI attributes. The Service Manager Adapter uses XSL transformation files to convert a properly formatted Service Manager web services messages into Universal CMDB CI. Out-of-the-box, each integration query has a corresponding XSL transformation file. In addition, each attribute you want to populate to Universal CMDB requires its own entry in the XSL transformation file.

If you want to add a CI type to the integration, you must create a matching XSL transformation file that defines how the Service Manager Adapter transforms each Service Manager web service field into a CI type. In order to create a proper XSL mapping, you must be familiar with the service and object names that Service Manager publishes as Web services. For information on publishing tables and columns as Web service fields, see *Service Manager Web Services*.

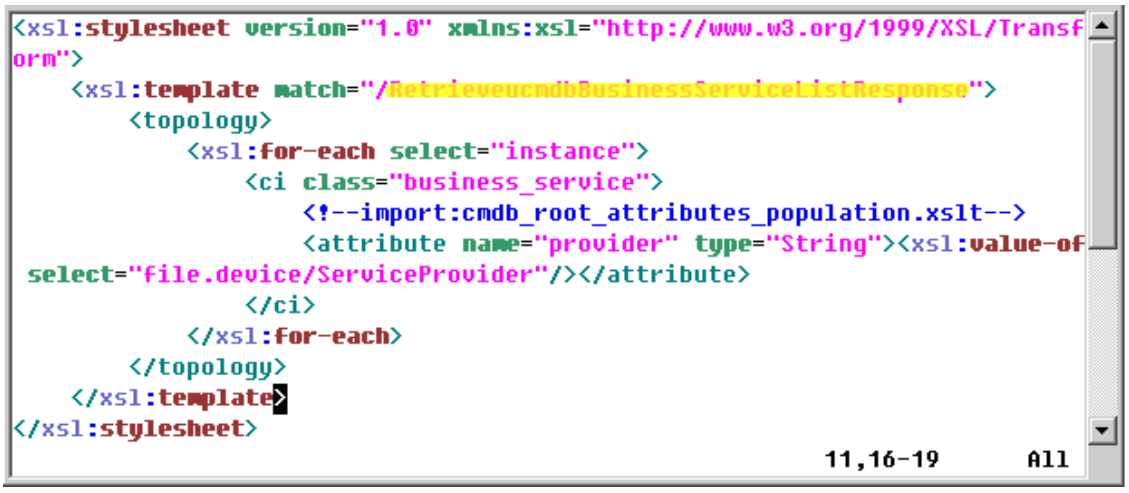
To create an XSL transformation file for the SM RDBMS CI type, which is described in previous sections:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management**.
3. Click the **Create new resource** button .
4. Select **New Configuration File**.
5. Select the ServiceManagerAdapter9.41 package.
6. Enter the full file name using this format: <AdapterID>/<filename>. For example, `ServiceManagerAdapter9.41/rdbms_population.xslt`.
7. Copy the content of an existing XSL transformation file to use it as a template to create the new transformation file. For example, copy the content of `business_service_population.xslt` to the new file.
8. Find the web service response name definition element, which uses the following format:

   ```
   <xsl:template match="/SM_WEBSERVICE_RESPONSE_NAME">
   ```

   `SM_WEBSERVICE_RESPONSE_NAME` is the name of web service response in the Service Manager system.

   The following figure shows an example of Web Service response definition in `business_service_population.xslt`:

9. Update the web service response name to match the response name that you want to add to the integration. For example, you can create the following CI type definition element to add the database response of retrieve object list to the integration.

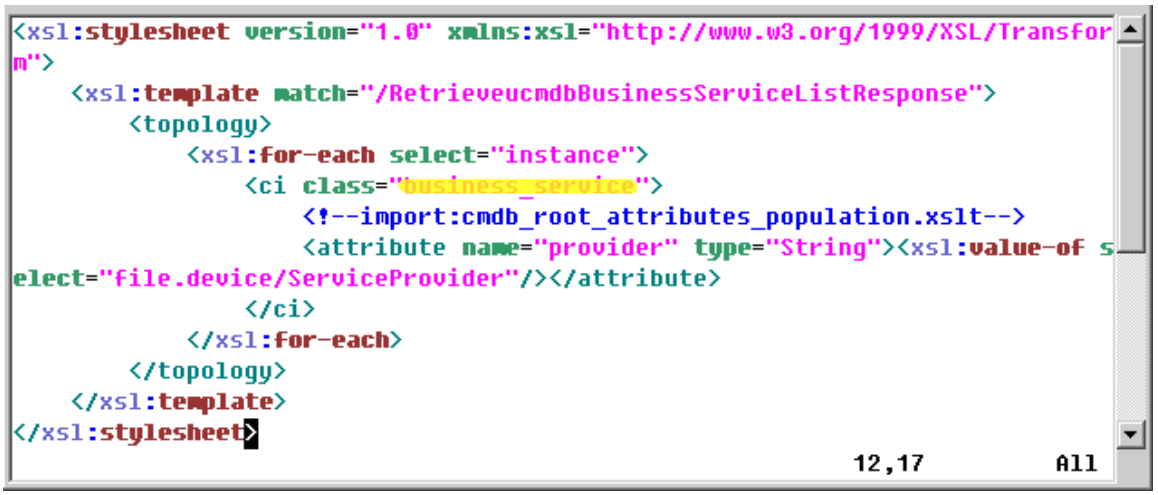
   ```
   <xsl:template match="/RetrieveucmdbRDBMSListResponse">
   ```

10. Find the Universal CMDB CI type definition element, which uses the following format:

```
<ci class="UCMDB_CI_TYPE_NAME">
```

`UCMDB_CI_TYPE_NAME` is the CI type name in the Universal CMDB System.

The following figure shows an example CI type definition in `business_service_population.xslt` :



11. Update the Universal CMDB CI type definition name to match the name you want to add to the integration. For example, create the following CI type definition element to add the database response of retrieve object list to the integration.

```
<ci class="sm_rdbms">
```

> **Note**
>
> Do not use any variables in the CI type definition name. The CI type definition name must be a fixed value.

12. Find the elements that transform Service Manager web service fields into Universal CMDB CI attributes. The CI attribute transformation elements use the following format:

```
<attribute name="UCMDB_CI_attribute_name"
type="UCMDB_CI_attribute_type" ignoreCIIfEmpty="true">
<xsl:value-of select="SMAttributeName "/></attribute>
```

`UCMDB_CI_attribute_name` is the name of the CI attribute in the Universal CMDB system.

`UCMDB_CI_attribute_type` is the type of the CI attribute in the Universal CMDB system that this integration supports. Currently the following types are supported: String, StringList, Integer, Long, Double, Boolean, IPAddress, Date, Float, and IntList.

`ignoreCIIfEmpty` is a parameter that specifies whether or not to ignore the CI during population if this attribute has an empty value (true: ignore; false: not ignore).

Note

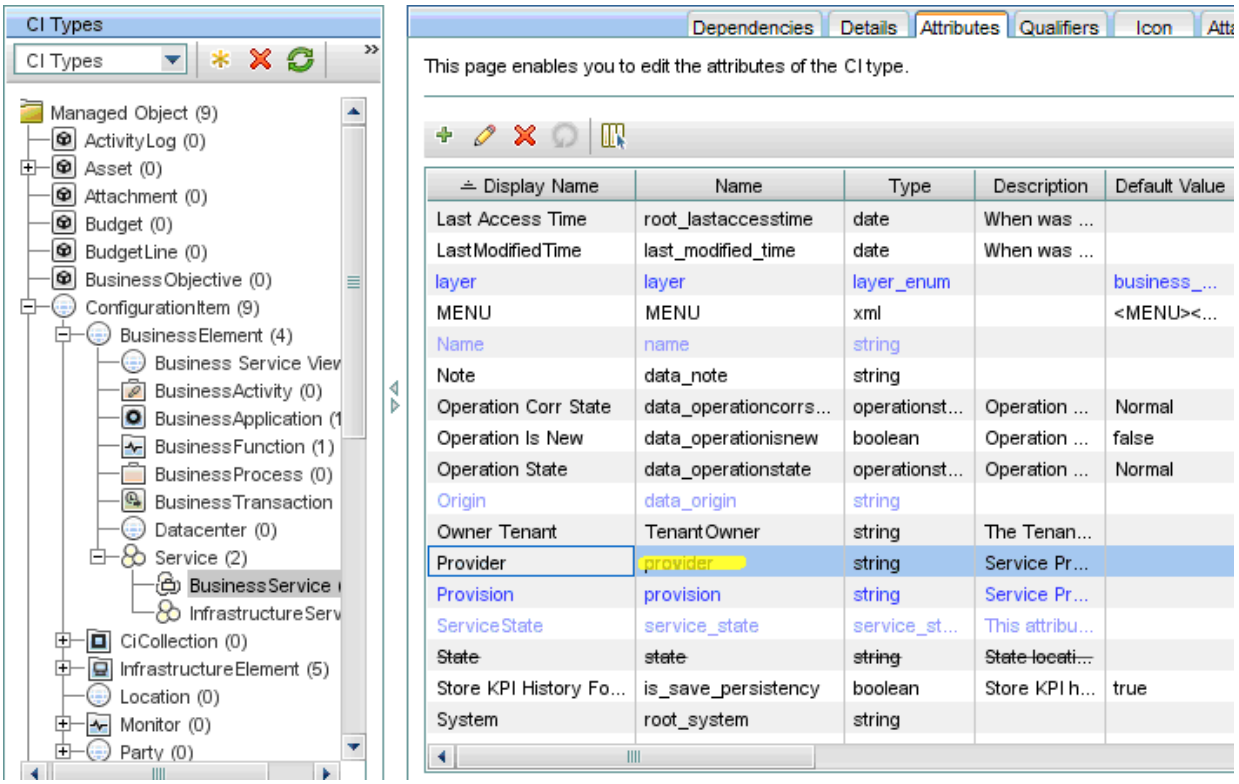
- A StringList is a list of strings separated by a semicolon (;). For example, str1;str2;str3. An IntList is a list of integers separated by a semicolon (;). For example, 1;2;3.
- For information about setting a time zone and date format for the Date type, see How to Update the Time Zone and Date Format for the Integration Adapter.

`SMAttributeName` is the name of a web service attribute published by the Service Manager system.

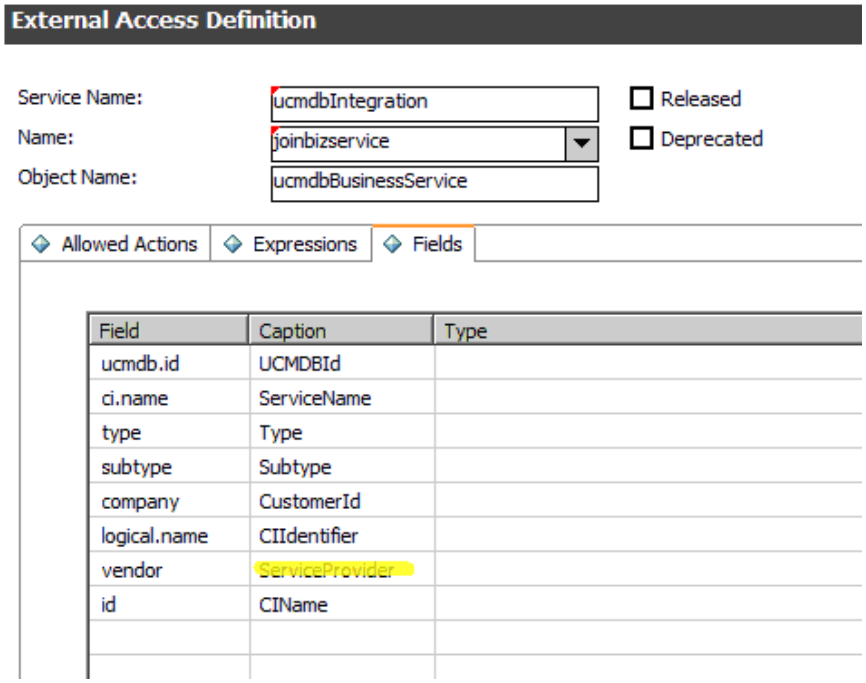
See the following figure for an example of CI attributes in `business_service_population.xslt`:

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transf
orm">
    <xsl:template match="/RetrieveucmdbBusinessServiceListResponse">
        <topology>
            <xsl:for-each select="instance">
                <ci class="business_service">
                    <!--import:cmdb_root_attributes_population.xslt-->
                    <attribute name="provider" type="String"><xsl:value-of
 select="file.device/ServiceProvider"/></attribute>
                </ci>
            </xsl:for-each>
        </topology>
    </xsl:template>
</xsl:stylesheet>
                                                          11,16-19      All
```

Refer to the following screenshot to match CI attributes in the Universal CMDB `BusinessService` CI type:



Refer to the following screenshot to match CI attributes in the Service Manager `ucmdbBusinessService` web service:



13. In the SM RDBMS population example, there are no sub items of a CI. If you want to populate the sub items of a CI, for example, to populate the IP addresses of a computer together with the computer CI, you need to add a "link" element under the "ci" element. A link transformation element uses the following format:

```
<link direction="Link_Direction" linkType="UCMDB_Link_Type">
    <ci class="UCMDB_SUB_CI_TYPE_NAME">
      <attribute name="UCMDB_CI_attribute_name"
type="UCMDB_CI_attribute_type">
<xsl:value-of select="SMAttributeName "/></attribute>
      <attribute name="UCMDB_CI_attribute_name2"
type="UCMDB_CI_attribute_type2">
<xsl:value-of select="SMAttributeName 2"/></attribute>
   </ci>
   </xsl:for-each>
</link>
```

`Link_Direction` is the direction between a parent CI and sub CI. The supported directions are:

- outgoing: The link direction is from an upstream CI to the current downstream CI (for example, from node to ip_address).
- incoming: The link direction is from a downstream CI to the current upstream CI.

`UCMDB_Link_type` is the relationship type in the Universal CMDB system.

The following figures show the out-of-the-box configurations for populating the Computer CI together with its IP addresses:

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:template match="/RetrieveucmdbNodeListResponse">
        <topology>
            <xsl:for-each select="instance">
                <xsl:variable name="building" select="file.device/Building"/>
                <xsl:variable name="floor" select="file.device/Floor"/>
                <xsl:variable name="room" select="file.device/Room"/>

                <ci class="node">
                    <!--import:cmdb_root_attributes_population.xslt-->
                    <attribute name="primary_dns_name" type="String"><xsl:value-
of select="file.device/DNSName"/></attribute>
                    <attribute name="calculated_location" type="String"><xsl:val
ue-of select="concat('Building:', $building, 'Floor:', $floor, 'Room:', $room)"/
></attribute>
                    <attribute name="discovered_os_name" type="String"><xsl:valu
e-of select="file.device/OS"/></attribute>
                    <attribute name="default_gateway_ip_address" type="String"><
xsl:value-of select="file.device/DefaultGateway"/></attribute>
                    <attribute name="discovered_os_version" type="String"><xsl:v
alue-of select="file.node/OSVersion"/></attribute>
                    <attribute name="memory_size" type="Integer"><xsl:value-of s
elect="file.node/PhysicalMemory"/></attribute>
                    <!--import:cpu_mapping_population.xslt-->
                    <!--import:disk_device_mapping_population.xslt-->
                    <!--import:filesystem_mapping_population.xslt-->
                    <!--import:ips_mapping_population.xslt-->
                    <!--import:interfaces_mapping_population.xslt-->
                </ci>
            </xsl:for-each>
        </topology>
    </xsl:template>
</xsl:stylesheet>
                                                              16,52         All
```

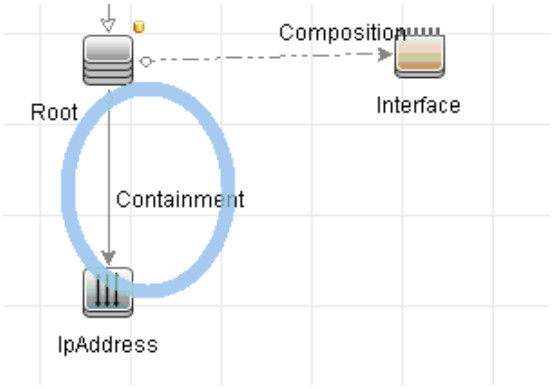
Link definition in the import xslt file (ips_mapping_population.xslt) is illustrated in the following screenshot:

```
<link direction="outgoing" linkType="containment">
    <xsl:for-each select="file.node/addlIPAddr/addlIPAddr">
        <ci class="ip_address">
            <attribute name="name" type="String"><xsl:value-of select
="AddlIPAddress"/></attribute>
            <attribute name="data_externalid" type="String"><xsl:valu
e-of select="AddlIPAddress"/></attribute>
            <attribute name="ip_netmask" type="String"><xsl:value-of
select="AddlSubnet"/></attribute>
        </ci>
    </xsl:for-each>
</link>
                                                          9,7           All
```

Refer to the following screenshot to match CI attributes and links in the Service Manager WSDL(http://<SM server>:<port>/SM/7/ucmdbNode.wsdl):

```
- <xs:complexType name="ucmdbNodeInstanceType">
  - <xs:sequence>
    - <xs:element name="file.device">
      - <xs:complexType>
        - <xs:complexContent>
          + <xs:extension base="cmn:StructureType">
          </xs:complexContent>
        </xs:complexType>
      </xs:element>
    - <xs:element name="file.node">
      - <xs:complexType>
        - <xs:complexContent>
          - <xs:extension base="cmn:StructureType">
            - <xs:sequence>
              - <xs:element minOccurs="0" name="addlIPAddr">
                - <xs:complexType>
                  - <xs:complexContent>
                    - <xs:extension base="cmn:ArrayType">
                      - <xs:sequence>
                        - <xs:element maxOccurs="unbounded" minOccurs="0" name="addlIPAddr">
                          - <xs:complexType>
                            - <xs:complexContent>
                              - <xs:extension base="cmn:StructureType">
                                - <xs:sequence>
                                    <xs:element minOccurs="0" name="AddlIPAddress" nillable="true" type="cmn:StringType" />
                                    <xs:element minOccurs="0" name="AddlSubnet" nillable="true" type="cmn:StringType" />
                                  </xs:sequence>
                                </xs:extension>
                              </xs:complexContent>
                            </xs:complexType>
                          </xs:element>
```

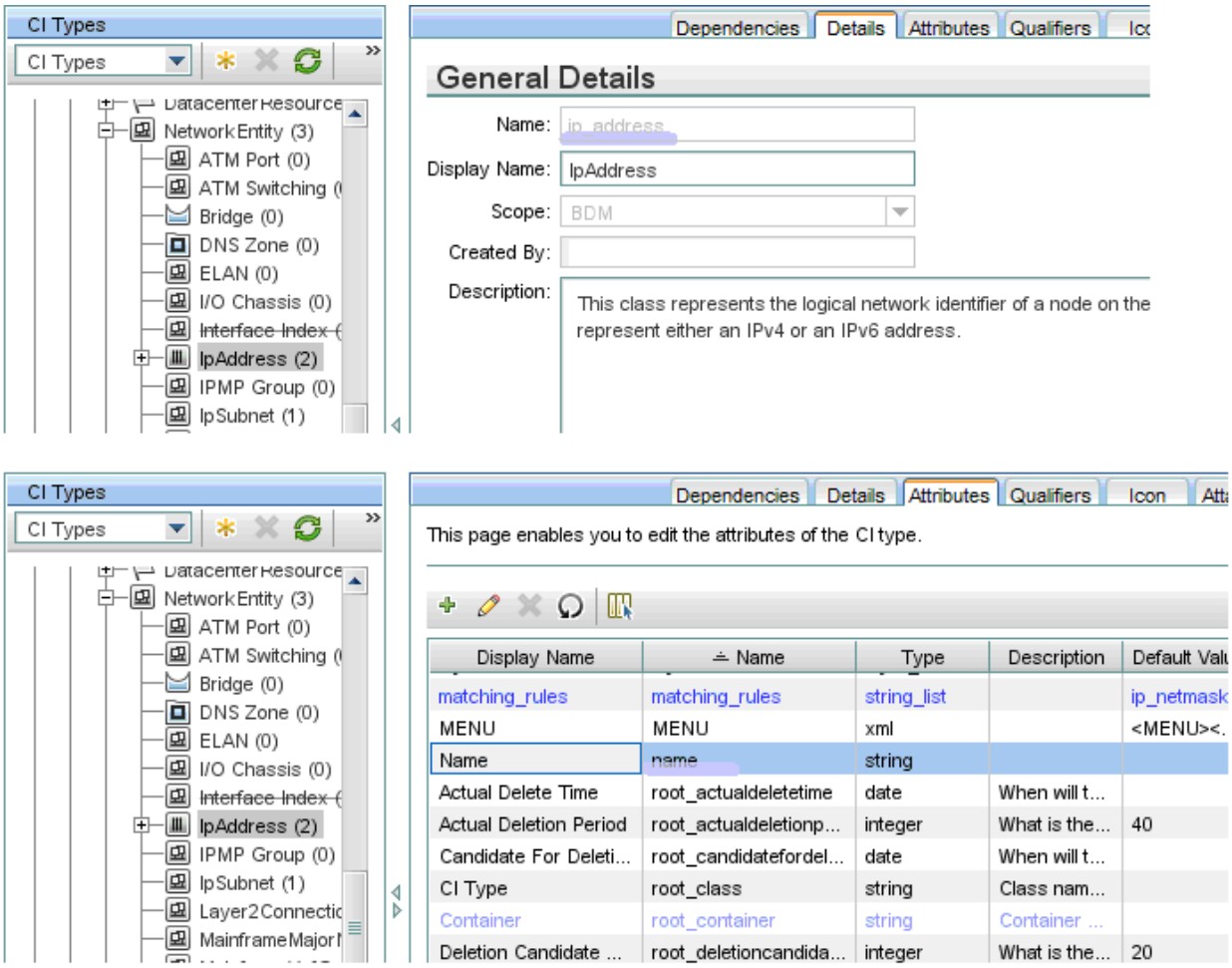
Refer to the following screenshot to match CI links in UCMDB at the TQL level:



Note

The figure above is provided only to give you a graphic view of how relevant CIs are stored in UCMDB. You don't need to create the query, since the population feature doesn't require it.

Refer to the following screenshot to match CI type and attributes in the UCMDB CI Type (`IpAddress`):



14. Add or update the CI attribute transformation elements for each CI attribute you want to add to the integration. For example, you can use the following XSL transformation elements for the database CI type.

Sample XSL transformation elements for database CIs

UCMDB attribute	Sample transformation elements
discovered_product_name	`<attribute name="discovered_product_name" type="String"><xsl:value-of select="file.device/CIIdentifier"/></attribute>`
product_name	`<attribute name="product_name" type="String"><xsl:value-of select="file.device/ApplicationName"/></attribute>`
application_version	`<attribute name="application_version" type="String"><xsl:value-of select="file.device/ProductVersion"/></attribute>`
vendor	`<attribute name="vendor" type="String"><xsl:value-of select="file.device/Vendor"/></attribute>`
version	`<attribute name="version" type="String"><xsl:value-of select="file.device/Version"/></attribute>`

UCMDB attribute	Sample transformation elements
dbinstance	`<attribute name="dbinstance" type="String"> <xsl:value-of select="file.rdbms/DBInstance"/></attribute>`
port	`<attribute name="port" type="String"> <xsl:value-of select="file.rdbms/Port"/> </attribute>`
description	`<attribute name="description" type="String"> <xsl:value-of select="file.rdbms/Description"/></attribute>`

Attribute mappings in `rdbms_population.xslt` are illustrated in the following screenshot:

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:template match="/RetrieveucmdbRDBMSListResponse">
        <topology>
            <xsl:for-each select="instance">
                <ci class="sm_rdbms">
                    <!--import:cmdb_root_attributes_population.xslt-->
                    <attribute name="discovered_product_name" type="String"><xsl
:value-of select="file.device/CIIdentifier"/></attribute>
                    <attribute name="product_name" type="String"><xsl:value-of s
elect="file.device/ApplicationName"/></attribute>
                    <attribute name="application_version" type="String"><xsl:val
ue-of select="file.device/ProductVersion"/></attribute>
                    <attribute name="vendor" type="String"><xsl:value-of select=
"file.device/Vendor"/></attribute>
                    <attribute name="version" type="String"><xsl:value-of select
="file.device/Version"/></attribute>
                    <attribute name="dbinstance" type="String"><xsl:value-of sel
ect="file.rdbms/DBInstance"/></attribute>
                    <attribute name="port" type="String"><xsl:value-of select="f
ile.rdbms/Port"/></attribute>
                    <attribute name="description" type="String"><xsl:value-of se
lect="file.rdbms/Description"/></attribute>
                </ci>
            </xsl:for-each>
        </topology>
    </xsl:template>
</xsl:stylesheet>
~
-- SELECT --                                    8          14,109-124     All
```

15. Save the XSL transformation file.



Note

When you create/edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

1.6.3.5.2.10. How to add a CI type's relationship types to the integration for population

Once you have added a new CI type to the integration for population, you need to add the new CI type's relationship types to the integration. For each relationship type, perform the following tasks.

As an example, the following steps describe how to add the Ownership relationship type to the integration; these steps assume that you have already added the Cost and CostCategory CI types to the integration for population.

1. Create an XSL transformation file to map each relationship type's attributes to web service objects.

   See *How to map each relationship type's attributes to Web Service objects* below.

2. Add a query mapping for each relationship type to the population configuration file.

   See *How to define a query mapping for each relationship type* below.

How to map each relationship type's attributes to Web Service objects

This example illustrates how to create an XSL transformation file to map the attributes of the Ownership relationship type to web service objects.

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management**.
3. Click the **Create new resource** button  .
4. Select **New Configuration File**.
5. Select the ServiceManagerAdapter9.41 package.
6. Enter the full file name using this format: <AdapterID>/<filename>. For example, `ServiceManagerAdapter9.41/cost_to_costcategory_population.xslt` .
7. Click **Yes** to ignore the file extension warning. The file is added to the Configuration Files folder.
8. Copy the content of an existing relationship population XSLT file (for example, `computer_to_computer_connects_population.xslt`) to the new XSLT file.

   Note

   In the next step, you will update the `<ci class>` names and `linkType` value.

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:template match="/Retrievecirelationship1to1ListResponse">
        <topology>
            <xsl:for-each select="instance">
                <ci class="node">
                    <attribute name="name" type="String">
<xsl:value-of select="upstreamci.logical.name"/></attribute>
                    <attribute name="sm_id" type="String">
<xsl:value-of select="upstreamci.id"/></attribute>
                    <link direction="outgoing" linkType="tcp">
                        <ci class="node">
                            <attribute name="name" type="String">
<xsl:value-of select="downstreamci.logical.name"/></attribute>
                            <attribute name="sm_id" type="String">
<xsl:value-of select="downstreamci.id"/></attribute>
                        </ci>
                    </link>
                </ci>
            </xsl:for-each>
        </topology>
    </xsl:template>
</xsl:stylesheet>
```

9. Change the <ci class> names to `cost` and `cost_category`, and linkType to `ownership`.

```
...
<ci class="cost">
                    <attribute name="name" type="String">
<xsl:value-of select=
"upstreamci.logical.name"/></attribute>
                    <attribute name="sm_id" type="String">
<xsl:value-of select=
"upstreamci.id"/></attribute>
                    <link direction="outgoing"
linkType="ownership">
                        <ci class="cost_category">
...
```

`ci class`: the name (not display name) of the each CI type involved in the relationship. It should be the Name field value on the General Details tab of the CI type definition.

`linkType`: the name of the relationship type. It should be the Name field value on the General Details tab of the relationship type definition.

> **Note**
>
> In this example, you don't need to change the link direction (outgoing). This is because the relationship (Ownership) direction is from cost to cost_category, that is, from a <ci class> outside of the <link> element to a <ci class> inside it).

10. Save the XSL transformation file.

How to define a query mapping for each relationship type

For each relationship type of the new CI type, you need to define a query mapping in the `smPopConfFile.xml` file.

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files** > **smPopConfFile.xml**.
3. In the `smPopConfFile.xml` file, add a query mapping for the relationship type, by copying an existing query mapping element.

> **Note**
>
> In the next step, you will update the highlighted values with values of the new relationship type.

```
<tql name="SM Biz To Biz With Usage"
xslFile="businessservice_to_businessservice_usage_population.xslt">
   <request type="Retrieve" dataType="relationship"

retrieveKeysQueryName="Retrievecirelationship1to1KeysListRequest"

retrieveListQueryName="Retrievecirelationship1to1ListRequest"
       basicQueryCondition="upstreamci.type=&quot;bizservice&quot;
and
downstreamci.type=&quot;bizservice&quot; and
relationship.subtype=&quot;Usage&quot;"
       fullQueryCondition="status~=&quot;Removed&quot;"

changedCreationQueryCondition="create.datetime&gt;'{fromDate}' and
status~=&quot;Removed&quot;"

changedUpdateQueryCondition="created.datetime&lt;='{fromDate}' and
update.datetime&gt;'{fromDate}' and status~=&quot;Removed&quot;"

changedDeletionQueryCondition="update.datetime&gt;'{fromDate}' and
status=&quot;Removed&quot;"/>
</tql>
```

4. Update the TQP mapping element with the values of the new relationship type.



Note

For the query name, you can enter any descriptive name as you like (for example, SM Cost to CostCategory with Ownership). This query doesn't really exist, since population doesn't require it.

The upstreamci.type, downstreamci.type, and relationship.subtype values are defined in the cirelationship1to1 table in Service Manager.

```
<tql name="SM Cost to CostCategory with Ownership"
xslFile="cost_to_costcategory_population.xslt">
   <request type="Retrieve" dataType="relationship"

retrieveKeysQueryName="Retrievecirelationship1to1KeysListRequest"

retrieveListQueryName="Retrievecirelationship1to1ListRequest"
       basicQueryCondition="upstreamci.type=&quot;cost&quot; and
downstreamci.type=&quot;costcategory&quot; and
relationship.subtype=&quot;Ownership&quot;"
       fullQueryCondition="status~=&quot;Removed&quot;"

changedCreationQueryCondition="create.datetime&gt;'{fromDate}' and
status~=&quot;Removed&quot;"

changedUpdateQueryCondition="created.datetime&lt;='{fromDate}' and
update.datetime&gt;'{fromDate}' and status~=&quot;Removed&quot;"

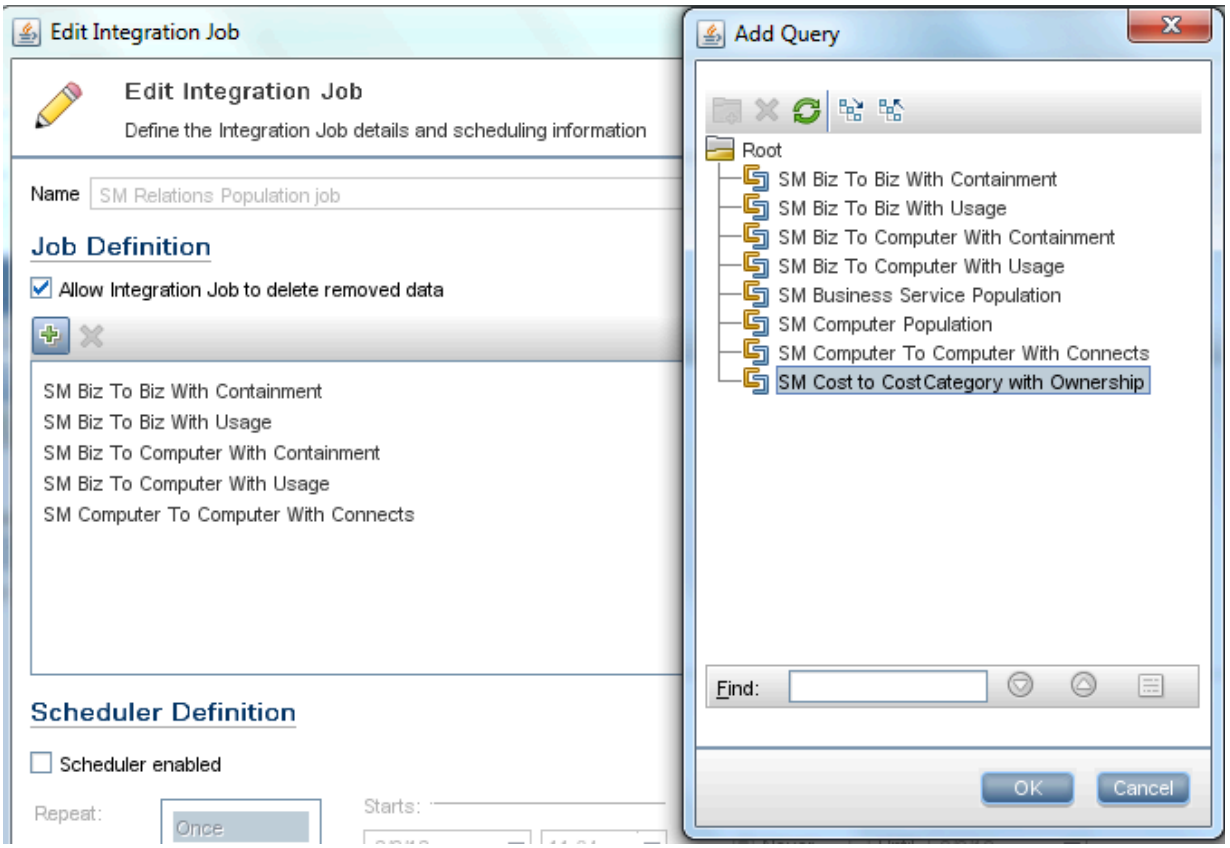
changedDeletionQueryCondition="update.datetime&gt;'{fromDate}' and
status=&quot;Removed&quot;"/>
</tql>
```

5. Save the population configuration file.

> **Note**
>
> When you create/edit and then save a configuration file in Adapter Management, UCMDB automatically restarts the adapter with the new configuration file.

Now the relationship type is added to the integration for population. Next, you need to add the relationship query name you specified in the population configuration file (in this example, SM Cost to CostCategory with Ownership) to a relationship population job, so that the integration can populate this type of relationships to UCMDB. See the following figure and also How to add custom queries to integration population jobs.

Refer to the following screenshot to add a new relationship query name to a relationship population job:


Edit Integration Job
Edit Integration Job
Define the Integration Job details and scheduling information
Name
SM Relations Population job
Job Definition
Allow Integration Job to delete removed data
SM Biz To Biz With Containment
SM Biz To Biz With Usage
SM Biz To Computer With Containment
SM Biz To Computer With Usage
SM Computer To Computer With Connects
Scheduler Definition
Scheduler enabled
Repeat:
Once
Starts:
Add Query
Root
SM Biz To Biz With Containment
SM Biz To Biz With Usage
SM Biz To Computer With Containment
SM Biz To Computer With Usage
SM Business Service Population
SM Computer Population
SM Computer To Computer With Connects
SM Cost to CostCategory with Ownership
Find:
OK
Cancel

1.6.3.5.2.11. How to customize UCMDB ID pushback for a CI type

Out-of-the-box, UCMDB pushes the UCMDB ID of each CI type back to Service Manager during population, by calling a Service Manager web service (`ucmdbIDPushBack`) based on the `ucmdbid.pushback.request` and `ucmdbid.pushback.xslt` settings in the `sm.properties` file in UCMDB. See How to update the integration adapter configuration file (sm.properties).

To better suit your business needs, you can make the following tailorings to the ucmdb id pushback feature, using the <idPushbackConfigurations> element in the `ServiceDeskConfiguration.xml` file:

- How to disable the UCMDB ID pushback feature for a specific CI type
- How to define a custom pushback Web Service and xslt file for a specific CI type



Note

For a specific UCMDB class (CI type), its definitions in the `ServiceDeskConfiguration.xml` file supercede the following global settings in the `sm.properties` file:

- idPushbackEnable: In the `sm.properties` file, this setting is not present; however, out-of-the-box, it is set to true for all CI types.
- idPushbackRequest: In the `sm.properties` file, its out-of-the-box value is UpdateucmdbIDPushBackRequest.
- idPushbackXSLT: In the `sm.properties` file, the out-of-the-box value is ucmdbid_pushback.xslt.

How to disable the UCMDB ID pushback feature for a specific CI type

You may want to disable the pushback feature for certain UCMDB CI types, for example, UCMDB classes that are mapped to a sub-item type (IP Address, CPU, etc.) in Service Manager. Doing so can aviod unnecessary system overload.

To disable the pushback feature for a UCMDB CI type (class), use this format:

```
<idPushbackConfiguration ucmdbClassName="<ucmdbClassName>"
idPushbackEnable="false"/>
```

Where, `<ucmdbClassName>` is the name of the UCMDB CI type.

Here are the out-of-the-box settings in the `ServiceDeskConfiguration.xml` file:

```
<idPushbackConfigurations>
        <idPushbackConfiguration ucmdbClassName="interface"
idPushbackEnable="false"/>
    <idPushbackConfiguration ucmdbClassName="cpu"
idPushbackEnable="false"/>
  <idPushbackConfiguration ucmdbClassName="disk_device"
idPushbackEnable="false"/>
  <idPushbackConfiguration ucmdbClassName="file_system"
idPushbackEnable="false"/>
  <idPushbackConfiguration ucmdbClassName="ip_address"
idPushbackEnable="false"/>
</idPushbackConfigurations>
```

To enable the pushback feature for a CI type, set `idPushbackEnable="true"` for it.

How to define a custom pushback Web Service and xslt file for a specific CI type

There are occasions when you need to define a custom pushback web service and XSLT for a specific CI type (class), for example, when a CI type you want to populate is not stored in the device table in Service Manager. This is because the out-of-the-box ucmdbIDPushBack web service is based on the device table.

For example, you have tailored the integration in order to populate the Functional Group from Service Manager to UCMDB. If the Functional Group in Service Manager is not stored in the device table, you can define a custom web service and XSLT like the following:

```
<idPushbackConfigurations>
       <idPushbackConfiguration ucmdbClassName="functional_group"
idPushbackEnable="true"
idPushbackRequest="UpdateucmdbIDPushBackForFunctionalGroupRequest"
idPushbackXSLT="ucmdbid_pushback_functionalgroup.xslt"/>
......
</idPushbackConfigurations>
```



Note

To support your pushback customizations, you need to create the specified custom web service record in Service Manager and XSLT file in UCMDB.

1.6.3.5.2.12. How to add custom queries to integration population jobs

To send your custom SM web service object and fields to your UCMDB system, you must add your custom queries to the population job between your SM data store and your UCMDB data store. The following steps illustrate how to add the custom `rdbmsData` query described in previous sections.

To add custom queries to population job definitions:

1. Sign in to UCMDB as an administrator.
2. Navigate to **Data Flow Management** > **Integration Studio**.
3. Double-click the name of your Service Manager integration point. For example, **SM Integration**.
4. Click the **Population** tab.
5. Select a population job. For example, **SM Configuration Item Population job**.
6. Click the **Edit Integration Job** button  .
7. Click the **Add** button  .

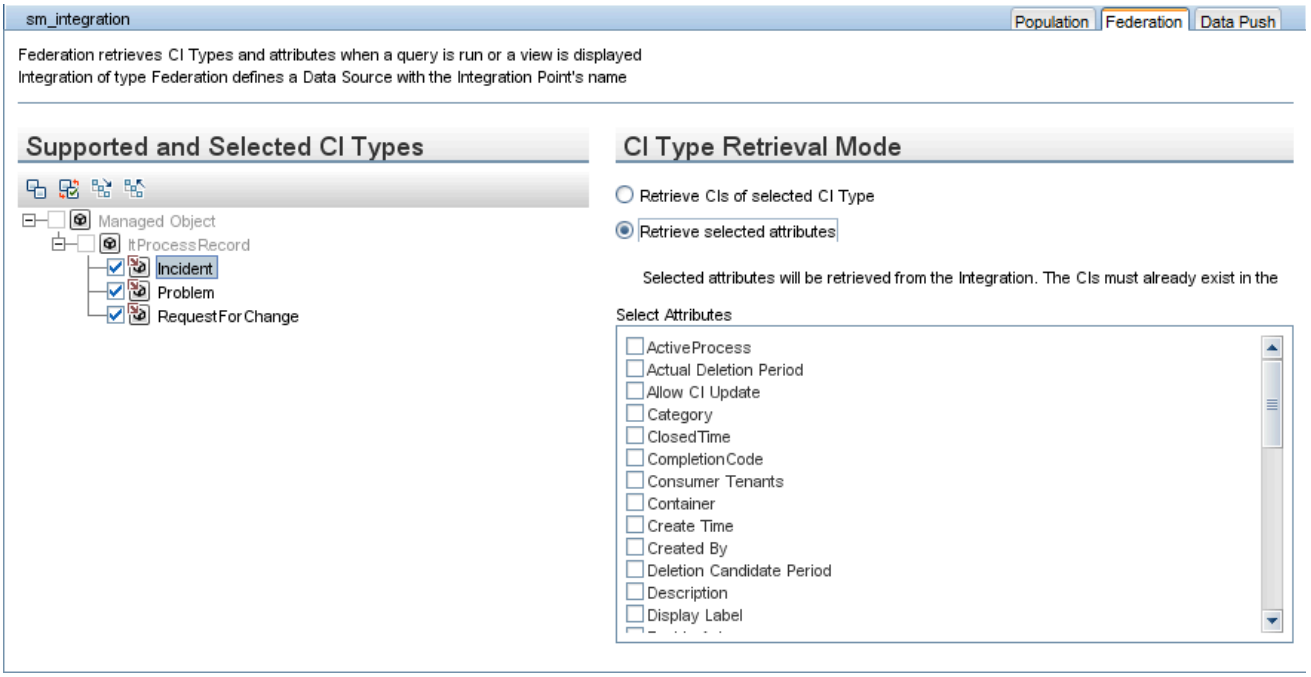
   The query names configured in the `smPopConfFile.xml` file are listed.
8. Click **Root** > **rdbmsData**.
9. Click **OK** to add a custom query.
10. Click **OK** to close the Update Job Definition window.

1.6.3.5.2.13. How to add an attribute of a supported CI type for federation

Out-of-the-box, the integration supports federation for three external CI types in UCMDB: Incident, Problem, and RequestForChange.

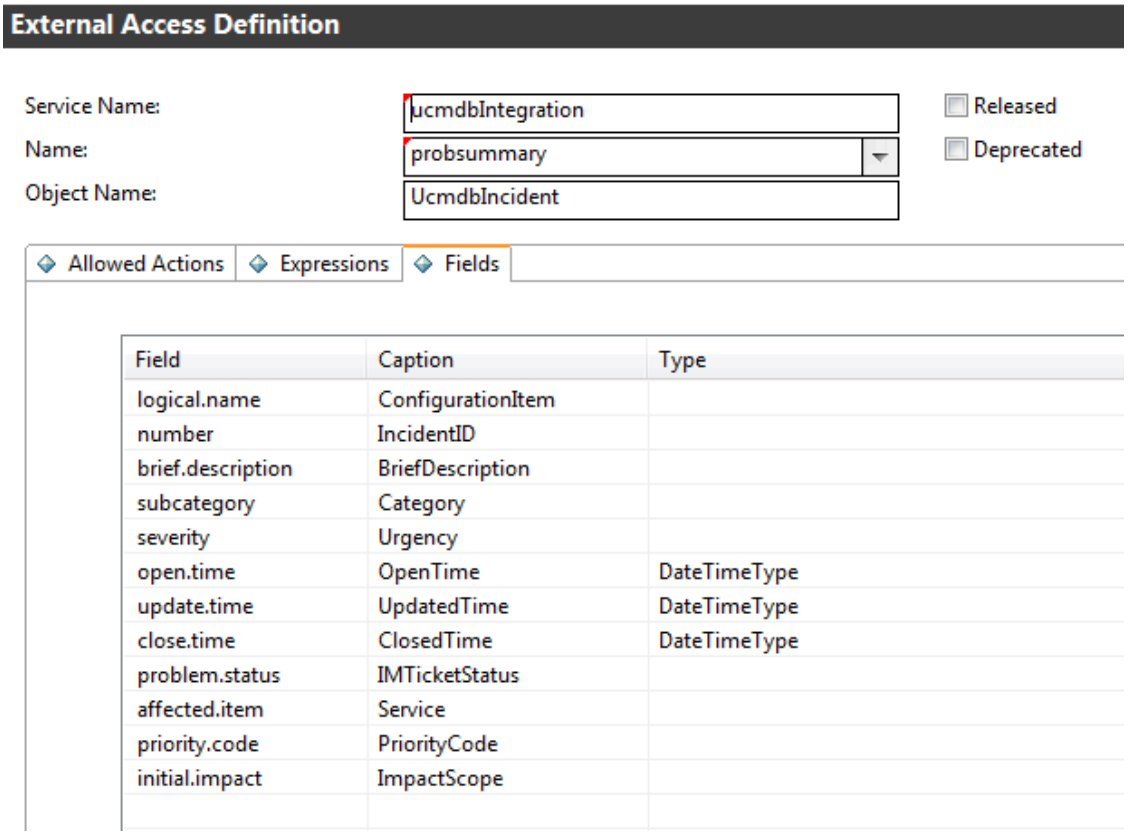
For each of the supported CI types, there is a list of attributes in UCMDB that you can map to Service Manager web service objects for federation. The following figure shows the out-of-the-box UCMDB CI attributes available for the Incident CI type.

Incident CI attributes supported for federation are illustrated in the following screenshot:



For example, to add an SM Incident attribute for federation, you need to expose the field in the SM UcmdbIncident web service object and then map it to an appropriate UCMDB attribute (if one doesn't already exist, you need to create it in UCMDB first).

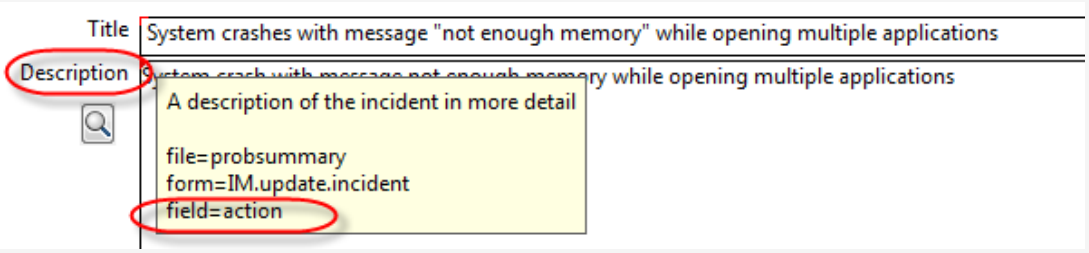
The following figure shows the fields that are exposed in the `UcmdbIncident` web service object in Service Manager:



You can expose more fields so that more Incident attributes can be federated to UCMDB. As an example, the following describes how to add the **action** field in the Service Manager probsummary (Incident) file for federation, by mapping it to a new UCMDB attribute named **details**.

Note

On the Incident form in Service Manager, the **action** field is labeled **Description**, which describes the incident ticket in more detail. See the following figure.



To add an attribute of a supported CI type for federation:

1. **Add the SM attribute to its web service object.**

   The following example describes how to expose the SM **action** field of Incident in UCMDBIncident web service object.

   1. Sign in to Service Manager as a system administrator.
   2. Navigate to **Tailoring** > **Web Services** > **WSDL Configuration**.

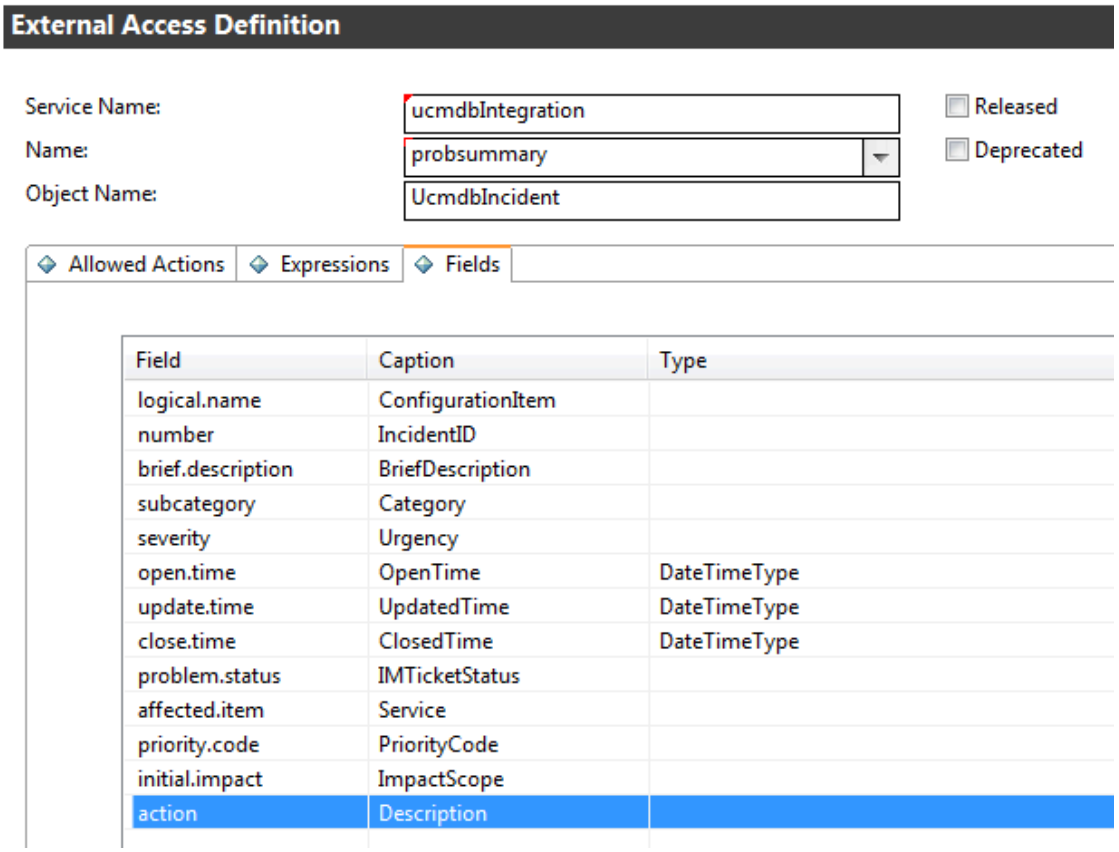
3. Enter the following field values, and then click **Search**.

   - Service Name: ucmdbIntegration
   - Name: probsummary

   UCMDBIncident web service object is displayed.

4. On the Fields tab, add the following row:

   - Field: action.
   - Caption: Description

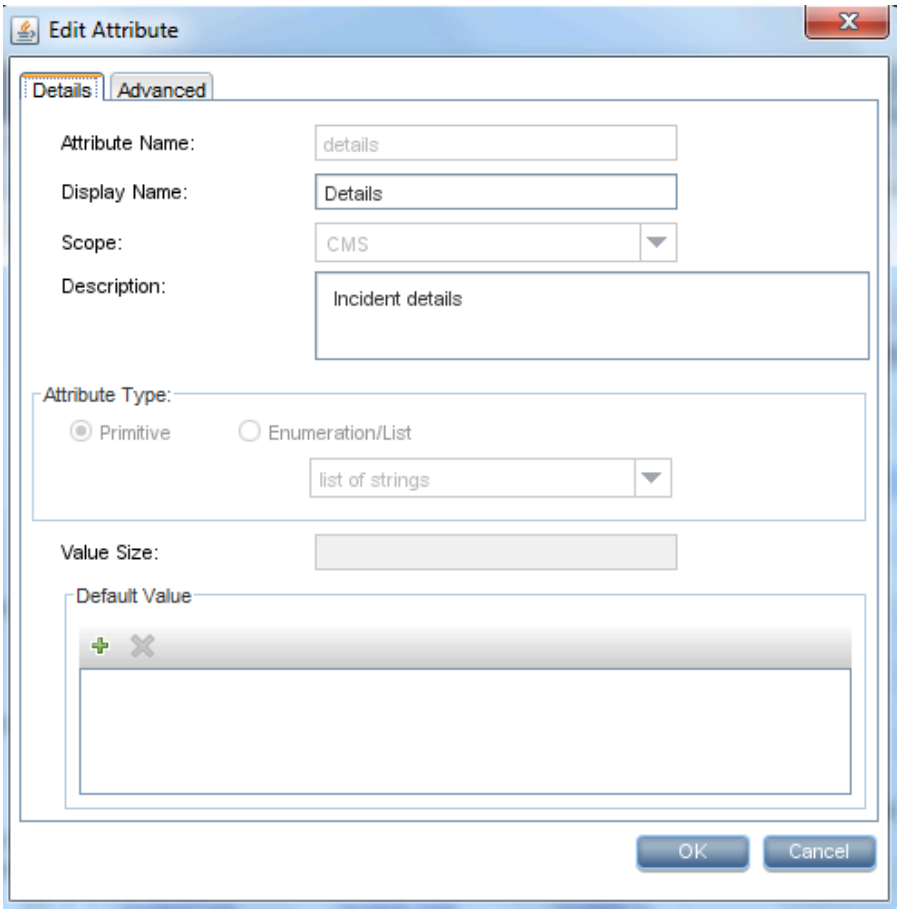


5. Save the web service object.

2. **Map the SM attribute to a UCMDB attribute.**

   The following example describes how to map the SM **action** attribute to a new UCMDB attribute named **details**.

   1. Sign in to UCMDB as an administrator.
   2. Navigate to **Modeling** > **CI Type Manager**.
   3. Browse to **ItProcessRecord** > **Incident**, and open its properties pane.
   4. Click the **Add** button to add a new attribute named **details** to the Incident CI type.



- name: details
- Display Name: Details
- Description: Incident details
- Attribute Type: Primitive > List of strings (this is because the **action** field in SM is an array)

5. Save the Incident CI type record.
6. Navigate to **Data Flow Management** > **Adapter Management** > **ServiceManagerAdapter9.41** > **Configuration Files**.
7. Click the `ServiceDeskConfiguration.xml` file.
8. Add a mapping entry for the **details** attribute in the Incident attributeMappings section, as shown in the following.

```
<ucmdbClassConfiguration ucmdbClassName="incident">
    <attributeMappings>
        <attributeMapping ucmdbAttributeName="reference_number"
serviceDeskAttributeName="IncidentID"/>
        <attributeMapping ucmdbAttributeName="name"
serviceDeskAttributeName="BriefDescription"/>
         ...
        <attributeMapping ucmdbAttributeName="incident_status"
serviceDeskAttributeName="IMTicketStatus"
converterClassName="com.mercury.topaz.fcmdb.adapters.
serviceDeskAdapter.converter.PropertyValueConverterFirstLetter
ToUpperAndReplaceUnderscoreWithSpace"
reversedConverterClassName="com.mercury.topaz.fcmdb.adapters.
serviceDeskAdapter.converter.PropertyValueConverterFirstLetter
ToLowerAndReplaceSpaceWithUnderscore"/>
         ...
       <attributeMapping ucmdbAttributeName="urgency"
serviceDeskAttributeName="Urgency"/>
<attributeMapping ucmdbAttributeName="details"
serviceDeskAttributeName="Description"/>
    </attributeMappings>
```

Tip

The attribute mapping entry uses the following format:

```
<attributeMapping ucmdbAttributeName="details"
serviceDeskAttributeName="Description"/>
```

Where:

`ucmdbAttributeName` is the UCMDB attribute name in the Incident CI type definition to which you want to map the SM attribute;

`serviceDeskAttributeName` is the field caption you defined in the SM web service object.

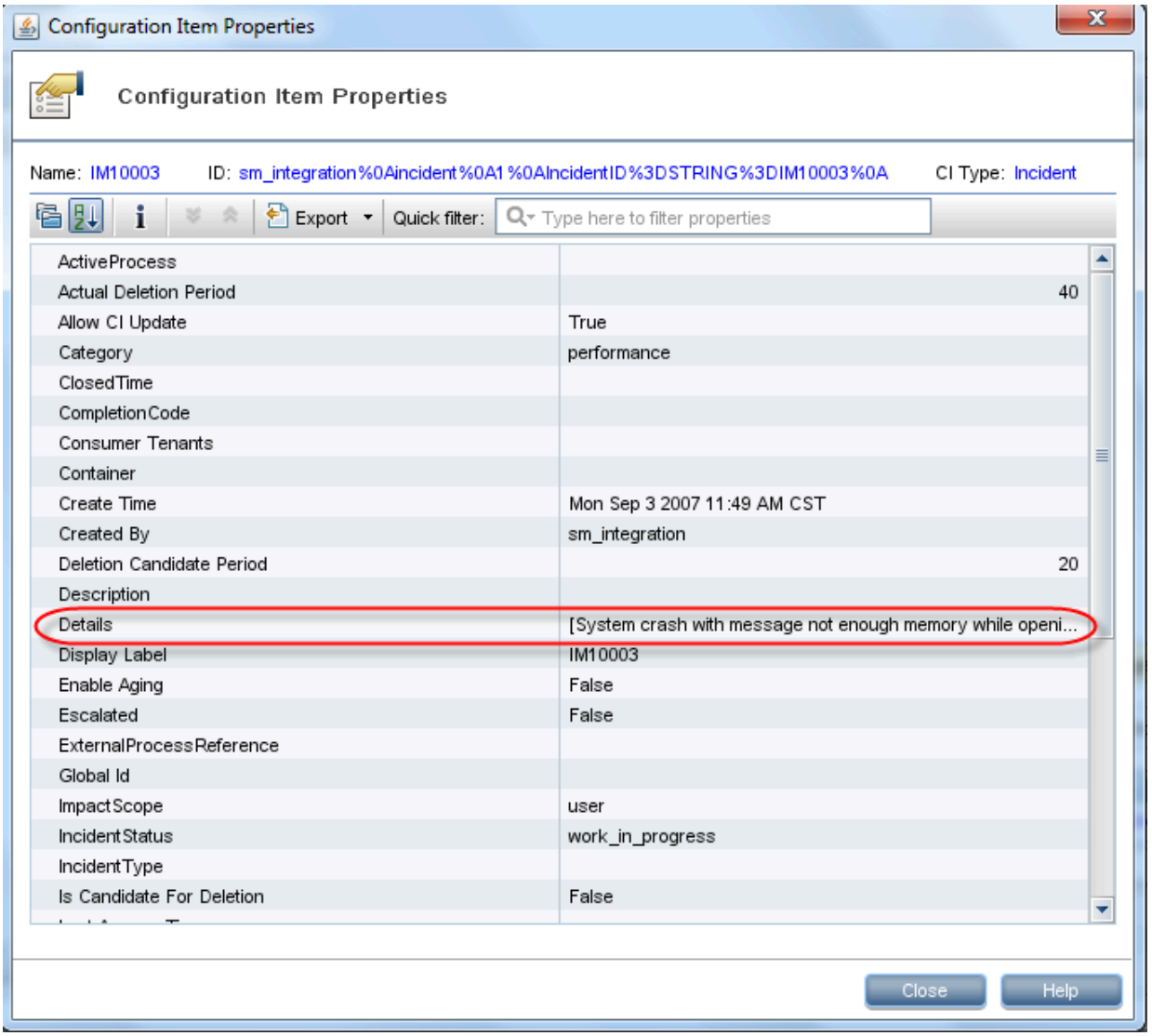
For an SM attribute (for example, problem.status) that's a drop-down list, the attribute mapping uses the following format (you only need to change the attribute names):

```
<attributeMapping ucmdbAttributeName="incident_status"
serviceDeskAttributeName="IMTicketStatus"
converterClassName="com.mercury.topaz.fcmdb.adapters.
serviceDeskAdapter.converter.PropertyValueConverterFir
stLetter
ToUpperAndReplaceUnderscoreWithSpace"
reversedConverterClassName="com.mercury.topaz.fcmdb.ad
apters.
serviceDeskAdapter.converter.PropertyValueConverterFir
stLetter
ToLowerAndReplaceSpaceWithUnderscore"/>
```

9. Click **OK** to save the file.

Now the Description (field name: action) attribute of SM Incident has been added to the integration for federation. You can run an Incident federation query in the UCMDB Modeling Studio to see if the SM Description data is properly federated. For details, see Examples of using federation.

The following figure shows an example where the Description of an SM incident ticket has been federated to UCMDB as Details:


Configuration Item Properties
Configuration Item Properties
Name: IM10003 ID: sm_integration%0Aincident%0A1%0AIncidentID%3DSTRING%3DIM10003%0A CI Type: Incident
Export
Quick filter:
Type here to filter properties
ActiveProcess
Actual Deletion Period 40
Allow CI Update True
Category performance
ClosedTime
CompletionCode
Consumer Tenants
Container
Create Time Mon Sep 3 2007 11:49 AM CST
Created By sm_integration
Deletion Candidate Period 20
Description
Details [System crash with message not enough memory while openi...
Display Label IM10003
Enable Aging False
Escalated False
ExternalProcessReference
Global Id
ImpactScope user
IncidentStatus work_in_progress
IncidentType
Is Candidate For Deletion False
Close
Help

1.6.4. CMS UI

The UCMDB UI is a lightweight web-based client to access UCMDB data. The UCMDB UI provides a simple and intuitive search for Configuration Items (CIs) in UCMDB and displays important data in the context of the selected CI. It is an ideal tool for providing quick access to specific CI information. For each CI, relevant data is presented and gathered into information widgets (for example, Properties, Environment, and Impact Simulation widgets). Data is presented by default in a Preview mode, with the option to view more comprehensive data in an Expanded mode.

How to integrate Service Manager with the UCMDB UI

You enable an integration to the UCMDB UI when enabling an integration to UCMDB. For details, see How to add UCMDB connection information.

Supported use cases

When integrated with UCMDB UI, Service Manager provides the following out-of-box features.

Viewing CI information in the UCMDB UI

A **View in UCMDB UI** button is available in the Configuration Item form. This button is displayed in each CI record that's synchronized from UCMDB. When a user clicks this button a CI, a UCMDB UI login screen is displayed; after the user logs in with a UCMDB UI account (username/password), the UCMDB UI opens in the context of this CI.

Retrieving primary CI change history from the UCMDB UI

The Change History widget from the UCMDB UI is displayed in a problem record whose primary CI is synchronized from UCMDB. This widget is provided in the **Primary CI History in UCMDB** tab on the Problem form. Users can view the CI changes on that primary CI for root cause investigation.



Note

For accessibility support of the embedded UCMDB widget, refer to the UCMDB UI documentation.

Support of Automated Service Modeling (ASM)



Note

Automated Service Modeling requires the UCMDB ASM Enhanced package to be deployed in UCMDB.

Service Manager supports the ASM functionality in the UCMDB UI. Service CIs and consumer-provider relationships that are discovered by ASM can be pushed to Service Manager;

additionally, you can tailor Service Manager such that Service Manager users can access ASM in the context of a business service, and access Impact Simulation in the context of an affected CI.

Support of ASM enables Service Manager users to view the accurate service trees that are discovered by ASM and facilitates service modeling in Service Manager.

For details, see Universal CMDB.

Related topics

Universal CMDB

1.6.5. CMS Gateway

UCMDB Gateway consists of new high-performance APIs based on the GraphQL technology. It allows dynamic, high-performance, and highly backward-compatible API calls into more than one system to merge the responses in one JSON payload. As of version 9.72, Service Manager implements integration with the UCMDB Gateway to display the CI Topology graph and the CI Impact Service graph. This integration uses the UCMDB Gateway API to access the data and embedded the graphic widgets for CI Topology and Impact representation.

How to integrate Service Manager with the UCMDB Gateway

You can enable integration to the UCMDB Gateway when enabling integration to UCMDB. For details, see How to add UCMDB connection information.

Supported use cases

When integrated with the UCMDB Gateway, Service Manager provides the following out-of-the-box features.

Visualizing CI

Service Manager displays the CI Topology widget. You can visualize the CIs that are pushed from UCMDB on a Configuration Item form, a Change form, an Incident form, or an Interaction form.

Visualizing Impact

Service Manager displays the Show Impact widget. You can visualize impacts that are pushed from UCMDB on a Change form, an Incident form, or an Interaction form.

Related topics

Universal CMDB

1.7. Operations Orchestration (OO)

Operations Orchestration (OO) software automates simple tasks, such as auto archiving, and complex tasks, such as disaster recovery planning. It provides the means to automate processes that include managing and provisioning a virtual infrastructure. The OO flows communicate and document procedures, decreasing dependencies on individuals or groups. Refer to the Operations Orchestration (OO) documentation for more information.

When integrated with Service Manager, OO shares information between monitoring and automation systems and the help desk.

- Web client users have access to OO flows from Knowledge Management, where they can view, add, update, or delete OO flows.
- Incident Management processes are enhanced by linking Knowledge documents with OO flows
- Implement automated deployment of changes by linking OO flows to change records.
- Perform automated implementation of change tasks by linking OO flows to change task records.
- Implement automated fulfillment of request tasks by linking OO flows to request task records.

Prerequisites:

- System Administrators must add, configure, and enable an instance of this integration in Integration Manager (SMIS).
- Secure Sockets Layer (SSL) is advisable for communication between the servers.
- Optionally, you can use Lightweight Single Sign-On (LW-SSO) to bypass the sign in prompts.

Related topics

- Integration Manager
- Operations Orchestration integration setup
- Automated resolution of incidents
- Automated deployment of changes
- Add an Operations Orchestration integration
- Enable LW-SSO for the Operations Orchestration integration
- Enable SSL connection from Service Manager to Operations Orchestration
- Manage OO flows from Knowledge Management
- Add OO flow links to a knowledge document
- Remove OO flow links from a knowledge document
- Launch OO flows from an incident
- View OO flow execution results from an incident
- Add OO flow links to a new change record
- Add OO flow links to an existing change record
- Remove OO flow links from a change record

- Update OO flow links in a change record
- Manually launch change flows
- Automatically launch change flows upon approval
- Automatically launch change flows on a schedule (for an existing change)
- Automatically launch change flows on a schedule (for an existing change)
- View execution results of change flows

1.7.1. Operations Orchestration integration setup

To set up the OO integration in your environment, you need to complete the following tasks:

- Add an Operations Orchestration integration

  This task adds and enables an OO integration in the Integration Manager (SMIS). It specifies all parameter values required to set up the integration.

- Enable SSL connection from Service Manager to Operations Orchestration

  This task enables SSL connection between the Service Manager and OO servers. The Service Manager server acts as a trusted client connecting to the OO server. This task creates a root CA and self-signed certificate in the OO server and then imports them into Service Manager.

- Enable LW-SSO for the Operations Orchestration integration

  This task is optional but strongly recommended. Enabling LW-SSO for the integration allows Service Manager users to bypass the sign in prompts when viewing OO flow execution results from within Service Manager.

Note that configuring Service Manager web service integrations in a network environment using a firewall or proxy server requires additional configuration. For details, see Web Services with a proxy server.

Related topics

- Integration Manager
- Operations Orchestration (OO)
- Add an Operations Orchestration integration
- Manage OO flows from Knowledge Management
- Add OO flow links to a knowledge document
- Remove OO flow links from a knowledge document
- Launch OO flows from an incident
- View OO flow execution results from an incident
- Add OO flow links to a new change record
- Add OO flow links to an existing change record
- Remove OO flow links from a change record
- Update OO flow links in a change record
- Manually launch change flows
- Automatically launch change flows upon approval
- Automatically launch change flows on a schedule (for an existing change)
- Automatically launch change flows on a schedule (for an existing change)
- View execution results of change flows

1.7.1.1. Add an Operations Orchestration integration

Applies to User roles: System Administrator

To use the Service Manager to OO integration, you must add and enable an instance of this integration in Integration Manager. Optionally, you can enable Lightweight Single Sign-On (LW-SSO) in OO and in Service Manager to bypass the log-in prompts.

To add and enable a Service Manager to OO integration instance:

1. Sign in to the Service Manager Windows or Web client.
2. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
3. Click **Add**. The Integration Template Selection wizard opens.
4. Select **SMOO** from the Integration Template list. Ignore the **Import Mapping** checkbox, which has no effect on this integration.
5. Click **Next**. The Integration Instance Information page opens.
6. Complete the following fields as necessary.

Field	Value	Description
Name (required)	User-defined. Default: `SMOO`.	Name of the integration instance.
Version (required)	1.1	Version number of the integration.
Interval Time (s) (required)	User-defined. For example: `600`.	Polling interval (in seconds) for Operations Orchestration flow synchronization.
Max Retry Times (required)	Enter 0 (zero) for this field.	Maximum allowed number of retries if the background scheduler fails to run. SMOO doesn't use this value.
SM Server	Enter a name that identifies your Service Manager server host. For example, `sm_host1`.	Display name for your Service Manager server host.
Endpoint Server	Enter a name that identifies your Operations Orchestration server host. For example, `oo_host1`.	Display name for your Operations Orchestration server host.
Log Level (required)	Select a level from the drop-down list. Default: `INFO`.	Level of diagnostic information that the Service Manager server logs to the log file directory. The possible log levels are: DEBUG, INFO (default), WARNING, ERROR, and OFF.

Field	Value	Description
Log File Directory (required)	Specify a directory. For example: `c: \smoologs`.	This is a directory that exists on the Service Manager server host where log files of the Service Manager to OO integration are stored. The log files are generated in the following format: `SMOO-<yyyy><mm><dd>.log` (for example, `SMOO-20100328.log`) **Note** **Note:** If you specify a directory that doesn't exist, the log files will get lost; if you don't specify a directory, the Service Manager to OO integration will not work.
Run at system startup	Selected/not selected	Automatically enable/disable the integration instance when the Service Manager server is started.
Description	Operations Orchestration flows linked to Service Manager.	Description of the integration instance.

7. Click **Next**. The Integration Instance Parameters page opens.

8. Click the **General Parameters** and **Secure Parameters** tabs, and modify the parameter values as shown in the table below:

Parameter	Value	Example
oo.server.url	Server address of Operations Orchestration Central: `https://[servername]:[port]` . Note that [servername] should be the fully qualified domain name (FQDN) of the Operations Orchestration server host. **Note** SSL is recommended for communication between the servers.	`https://oo.microfocus.com:8443` **Note** If you are using OO 10.x, mark the **Enable Authentication** checkbox to ensure the integration API can pass the authentication .

Parameter	Value	Example

oo.user.name	User name of the Operations Orchestration user account that the Service Manager server uses to access Operations Orchestration Central to synchronize and launch OO flows.
	
	Note
	If you use domain user accounts to sign in to OO, you need to specify a domain user account by using the DOMAIN\user name format (for example, Internal\admin).
	Otherwise, Service Manager will fail to connect to the OO server.
		`admin`
		
		or
		
		`Internal\admin`

Parameter	Value	Example
	Note For security reason, we recommend that you only grant the necessary rights to the user account that's used in the integration.	
oo.password	Password of oo.user.name.	admin
basepath.delimiter	Delimiter between multiple basepaths. The default delimiter is a semicolon (;).	;
basepath	basepath1;bathpath2;... The paths are separated by the basepath delimiter. Only the OO flows under the basepath and its sub-folders will be synchronized from OO.	/Library/ITIL/Change Management;/Library/ITIL/Incident Management
Accept-Language	REST Request Language Setting	en
http.conn.timeout	Http Connection Timeout setting (seconds)	30
http.rec.timeout	Http Receive Timeout setting (seconds)	30

Parameter	Value	Example
OOKM	Set to true/false to enable/disable the visibility of this integration instance in Knowledge Management (default: true).	`true` or `false`
OOCM	Set to true/false to enable/disable the visibility of the OO integration instance in Change Management (default: true).	`true` or `false`

9. Click **Next** twice and then click **Finish**. Leave the Integration Instance Mapping and Integration Instance Fields settings blank. This integration doesn't use these settings.

   Service Manager creates the instance. You can edit, enable, disable, or delete it in Integration Manager.

10. Enable the integration instance.
11. (Optional) Configure LW-SSO in Operations Orchestration and configure LW-SSO in Service Manager.

Related topics

Integration Manager
Integration Instance Information fields
Enable SSL connection from Service Manager to Operations Orchestration
Operations Orchestration (OO)
Operations Orchestration flow synchronization rules
Enable LW-SSO for the Operations Orchestration integration
Configure LW-SSO in the Service Manager Web tier
Enable or disable an integration instance
Monitor integration instance status

1.7.1.2. Enable SSL connection from Service Manager to Operations Orchestration

Applies to User roles: System Administrator

The Service Manager to OO integration requires that Secure Sockets Layer (SSL) be enabled in the Operations Orchestration and Service Manager servers to ensure data security. Before using this integration, you must complete the following procedures:

1. Configure SSL in Operations Orchestration.
2. Configure SSL in Service Manager.

Note

- The following procedures are provided as examples, assuming that you have not changed the security configurations of Operations Orchestration and Service Manager since they were installed. You may need to adjust the procedures depending on your specific security configurations in Operations Orchestration and Service Manager.
- To perform the following procedures, you must have OpenSSL installed on your Operations Orchestration Central host. In addition, you must have a Java platform installed on the Operations Orchestration and Service Manager hosts.
- In the following procedures, <OO_HOME> represents the Operations Orchestration home directory, and <SM_HOME> represents the Service Manager home directory.

Configure SSL in Operations Orchestration

Configure SSL in Operations Orchestration 9.x

To configure SSL in Operations Orchestration 9.x, perform the following tasks.

Task 1: Back up keystores in Operations Orchestration

Before you start configuring SSL in Operations Orchestration, make a backup copy of existing keystores (if any) in the following directories:

- <OO_HOME>\Central\conf
- <OO_HOME>\RAS\Java\Default\webapp\conf
- <OO_HOME>\Scheduler\conf
- <OO_HOME>\Studio\conf

> **Note**
>
> If your Operations Orchestration system doesn't contain any keystores in these folders, skip this task.

Task 2: Configure SSL in Operations Orchestration Central

You can use any pair of public/private certificates based on your specific certificate requirements, security policies, and so on. This section provides an example of using OpenSSL and the standard Java keytool to generate a self-signed certificate.

To configure SSL in Operations Orchestration Central:

1. Stop the RSCentral, RSScheduler and RSJRAS services.
2. Generate a private/public key pair for Root Certificate Authority.
    1. Change to the following directory: `<OO_HOME>\Central\conf`.
    2. Run the following command to create the private key:

        `openssl genrsa -aes256 -out <RSA_private_key_file> 2048`

        For example: `openssl genrsa -aes256 -out cakey.pem 2048`
    3. When prompted, enter a password phrase you want to use to protect your authority's private key file (`cakey.pem`). For example, `CAKeyPassword`.

        > **Note**
        >
        > Remember this password phrase. You will be asked to enter it again later when you use the Java keytool to generate a request (see step 3).

    4. Run the following command to create the public key:

        `openssl req -new -key <RSA_private_key_file> -x509 -days 1095 -out mycacert.pem`

        For example, `openssl req -new -key cakey.pem -x509 -days 1095 -out mycacert.pem`.
    5. Enter other required information that will be incorporated into your certificate request.

        > **Note**
        >
        > To create a unique `.pem` file, provide a unique Organization Name (for example, `org1`). When asked for a Common Name, enter the fully qualified domain name (FQDN) of the Operations Orchestration host.

3. Use the Java keytool to generate a request.

    1. Run the following command:

        `keytool -genkey -keyalg RSA -alias sm -keystore rc_keystore -storepass <store password for rc_keystore> -keypass <key password for rc_keystore> -dname "CN=<OO_server_FQDN>, OU=<Organization Unit>, O=<Organization>, L=<Location>, ST=<State or Province>,C=<Country>"`

        For example: `keytool -genkey -keyalg RSA -alias sm -keystore rc_keystore -storepass bran507025 -keypass bran507025 -dname "CN=<hostname.domainname>, OU=MyOrganizationUnit, O=MyCompany, L=MyCity, ST=MyState,C=MyCountry"`

        Note

        The default value for both `<store password for rc_keystore>` and `<key password for rc_keystore>` is: **bran507025**.

        Note

        We recommend that the `keyalg` parameter uses a value of `RSA` rather than the default of `DSA`. Doing so allows your TLS communications to use the stronger ECDHE cipher suites which aren't vulnerable to Logjam attacks (CVE-2015-4000).

    2. Run the following command:

        `keytool -certreq -keystore rc_keystore -alias sm -storepass <store password for rc_keystore> -file <CRS file>`

        For example: `keytool -certreq -keystore rc_keystore -alias sm -storepass bran507025 -file req.crs`

    3. Run the following command:

        `openssl x509 -req -days 1095 -in <CRS file> -CA mycacert.pem -CAkey <RSA_private_key_file> -CAcreateserial -out smcert.pem`

        For example, `openssl x509 -req -days 1095 -in req.crs -CA mycacert.pem -CAkey cakey.pem -CAcreateserial -out smcert.pem`

    4. When prompted, type the password phrase you entered in step 2 to protect your authority's private key file (`cakey.pem`). For example, `CAKeyPassword`.

4. Import the root CA and self-signed certificate to `rc_keystore`.

    1. Run the following command:

`keytool -import -v -alias rootca -keystore rc_keystore -storepass <store password for rc_keystore> -file mycacert.pem`

For example: `keytool -import -v -alias rootca -keystore rc_keystore -storepass bran507025 -file mycacert.pem`

The command window prompts the certificate information, such as Owner, Issuer, Serial number, Valid period, Certificate fingerprints, and Extensions.

2. When you are prompted to confirm that this certificate should be trusted, type: `y`.

   The command window prompts: "Certificate was added to keystore".

3. Run the following command:

   `keytool -import -v -alias sm -keystore rc_keystore -storepass <store password for rc_keystore> -file smcert.pem`

   For example: `keytool -import -v -alias sm -keystore rc_keystore -storepass bran507025 -file smcert.pem`

   The command window prompts: "Certificate reply was installed in keystore".

Task 3: Configure SSL in Operations Orchestration RAS

1. Change to the following directory: `<OO_HOME>\RAS\Java\Default\webapp\conf\`.
2. Copy the generated root CA `mycacert.pem` and self-signed certificate `smcert.pem` from `<OO_HOME>\Central\conf` to the current directory.
3. Import the root CA `mycacert.pem` and self-signed certificate `smcert.pem` to `ras_keystore.jks`. See step 4 in *Configure SSL in Operations Orchestration Central*.

Task 4: Configure SSL in Operations Orchestration Scheduler

1. Change to the following directory: `<OO_HOME>\Scheduler\conf\`.
2. Copy the generated root CA `mycacert.pem` and self-signed certificate `smcert.pem` from `<OO_HOME>\Central\conf` to the current directory.
3. Import the root CA `mycacert.pem` and self-signed certificate `smcert.pem` to `rc_keystore`. See step 4 in *Configure SSL in Operations Orchestration Central*.

Task 5: Start the Operations Orchestration services

Start the RSCentral, RSScheduler and RSJRAS services.

Task 6: Configure SSL in Operations Orchestration Studio

1. Change to the following directory: <OO_HOME>\Studio\conf\.
2. Copy the generated root CA mycacert.pem and self-signed certificate smcert.pem from <OO_HOME>\Central\conf to the current directory.
3. Import the root CA mycacert.pem and self-signed certificate smcert.pem to rc_keystore. See step 4 in *Configure SSL in Operations Orchestration Central*.

Configure SSL in Operations Orchestration 10.x

To configure SSL in Operations Orchestration 10.x, perform the following tasks.

Task 1: Configure Central SSL Server Certificate with FQDN

You can generate a self-signed certificate using the Keytool utility.

1. Stop Central and back up the original key.store file, located in *<installation dir>*/central/var/security/key.store.
2. Open a command line in *<installation dir>*/central/var/security.
3. Delete the existing server certificate from the Central key.store file, using the following command:

   ```
   keytool -delete -alias tomcat -keystore key.store -storepass changeit
   ```

4. Generate a self-signed certificate, using the following command:

   ```
   keytool -genkey -alias tomcat -keyalg RSA -keypass changeit -keystore <path/for/new/Keystore> -storepass changeit -storetype pkcs12 -dname "CN=<CENTRAL_FQDN>, OU=<ORGANIZATION_UNIT>, O=<ORGANIZATION>, L=<LOCALITY>, C=<COUNTRY>"
   ```

   > **Note**
   >
   > If you don't enter a path for generating the new keystore, it is created in the folder where you entered the command, for example *<installation dir>*/central/var/security.

5. Import the self-signed certificate to the Central key.store file, using the following command:

   ```
   keytool -v -importkeystore -srckeystore new/path/created/Keystore -srcstoretype PKCS12 -srcstorepass changeit -destkeystore .key.store -deststoretype JKS -deststorepass changeit
   ```

6. Start Central.

Task 2: Configure SSL on OO Central

1. Stop the **Operations Orchestration Central** service.
2. Search for the keytool.exe file installed on your machine and append its location to the *Path* variable in your system environment.
3. Open a command line in *<installation dir>*/central/var/security.
4. Run the following command:

   ```
   keytool.exe -export -alias tomcat -file "xxx\oo10-certificate.cer" -keystore "%OO_Home%\central\var\security\key.store" -storepass changeit
   ```

> **Note**
>
> Later, when configuring SSL in Service Manager, you will import oo10-certificate.cer into the Service Manager trust store file.

5. Start the **Operations Orchestration Central** service.

Configure SSL in Service Manager

Once you have successfully configured SSL in Operations Orchestration, you are ready to configure SSL in Service Manager.

Configure SSL for Service Manager to connect to OO 9.x

To configure SSL for Service Manager to connect to OO 9.x, perform the following steps.

1. Create a trust store for Service Manager.
    1. Change to the following directory: `<SM_HOME>/Server/RUN`.
    2. Copy the generated `mycacert.pem` and `smcert.pem` from `<OO_HOME>\Central\conf` to `<SM_HOME>/Server/RUN`.
    3. Run the following command:

        `keytool -import -v -alias rootca -keystore <keystore_file> -storepass <store password for keystore_file> -file mycacert.pem`

        For example: `keytool -import -v -alias rootca -keystore smtrust -storepass smoointabc123 -file mycacert.pem`

        The command window displays the certificate information.
    4. When the command window prompts: "Trust this certificate?", type `y`.

        The command window prompts: "Certificate was added to keystore".
    5. Run the following command:

        `keytool -import -v -alias sm -keystore <keystore_file> -storepass <store password for keystore_file> -file smcert.pem`

        For example: `keytool -import -v -alias sm -keystore smtrust -storepass smoointabc123 -file smcert.pem`

        > **Note**
        >
        > In this example, the trust store file name is `smtrust`, and its store password is `smoointabc123`. You will add this information to `sm.ini` in the next step.

2. Add the following lines to `sm.ini`:

```
#
# Certificates
#
truststoreFile:<keystore_file>
truststorePass:<store password for keystore_file>
```

For example:

```
#
# Certificates
#
truststoreFile:smtrust
truststorePass:smoointabc123
```

3. Restart the Service Manager server so that your configuration takes effect.

Configure SSL for Service Manager to connect to OO 10.x

To configure SSL for Service Manager to connect to OO 10.x, perform the following steps.

1. Stop the Service Manager Server service.
2. Copy oo10-certificate.cer into a directory on the Service Manager server host.

> **Note**
>
> This is the certificate you created when configuring SSL in OO.

3. Search for the keytool.exe file and append its location to the *Path* variable in the system environment.
4. Open a CMD window under **%SM_home%\Server\RUN**.
5. Import the OO certificate into the Service Manager trust store file, using the following command:

```
keytool.exe -import -alias xxx -file "xxx\oo10-certificate.cer" -
keystore smtrust -storepass smoointabc123
```

6. Answer Y when prompted. The confirmation message **Certificate was added to keystore** appears.
7. Verify smtrust was created under *<SM_home>*\Server\RUN.
8. Append the following lines to the sm.ini file under the above location:

```
# Certificates
truststoreFile:smtrust
truststorePass:smoointabc123
```

9. Start the Service Manager Server service.

Related topics

Integrations
Integration Manager
Integration Instance Information fields
Operations Orchestration (OO)
Add an Operations Orchestration integration
Enable or disable an integration instance
Enable LW-SSO for the Operations Orchestration integration
Configure LW-SSO in the Service Manager Web tier

1.7.1.3. Enable LW-SSO for the Operations Orchestration integration

Applies to User Roles: System Administrator

If you have enabled Lightweight Single Sign-On (LW-SSO) in both Service Manager and Operations Orchestration (OO), users who have logged on to Service Manager are allowed to sign on to Operations Orchestration through the web tier without providing a username and password.

To enable LW-SSO for the OO integration, complete the following tasks:

- Configure LW-SSO in the Service Manager Web tier
- Configure LW-SSO in Operations Orchestration (OO)

Related topics

Using LW-SSO with integrations
Operations Orchestration (OO)
Operations Orchestration integration setup
Add an Operations Orchestration integration
Enable SSL connection from Service Manager to Operations Orchestration

1.7.2. Manage OO flows from Knowledge Management

User roles: Users with the **SysAdmin** or **programmer** capability word

You can view, create, update, and delete Operations Orchestration (OO) flows from the **Operations Orchestration Flows** menu in Knowledge Management.

> **Note**
>
> To manage OO flows, you must first set up and enable an OO integration instance in Integration Manager (SMIS).

To manage Operations Orchestration flow records in Service Manager:

1. Sign in to the Service Manager web client.
2. Click **Knowledge Management** > **Configuration** > **Operations Orchestration Flows**.

   The Operations Orchestration Flow window opens.
3. Click **Search**.

   If a Service Manager to OO integration instance is correctly configured and enabled in Integration Manager (SMIS), Service Manager displays a list of OO flow records.

   > **Note**
   >
   > Service Manager populates the list of OO flow records from the `OOFlow` table under **System Definition**. These OO flow records come from the following sources.
   >
   > - Automatically synchronized from OO (according to the base path specified in the SM - OO integration instance)
   > - Manually added in SM (for example, an individual OO flow that's not under the base path or its subfolders)

   You can also type a full or partial display name in the Display Name field, and click **Search** to view one or more specific OO flow records.
4. Select a record from the list to view its details.

   The details of the selected OO flow are displayed. For the description of each field on the OO flow detail form, see Operations Orchestration flow detail form fields.
5. To add a new OO flow:
   1. Click **New**.
   2. Complete the fields of the OO flow record.

> **Note**
>
> You can only add an OO flow that has a unique UUID.

3. Click **OK**.

6. To update an existing OO flow:
   1. Select the OO flow.
   2. Update the details.
   3. Click **Save & Exit**.

> **Note**
>
> If the OO flow gets updated in Operations Orchestration after you update it in Service Manager, the integration will discard your updates at the next synchronization.

7. To delete an OO flow:
   1. Select the OO flow.
   2. Click **Delete**.
   3. Click **Yes** to confirm the deletion.

Related topics

- Operations Orchestration (OO)
- Operations Orchestration flow detail form fields
- Remove OO flow links from a knowledge document
- Launch OO flows from an incident
- View OO flow execution results from an incident

1.7.2.1. Operations Orchestration flow synchronization rules

The SM to OO integration synchronizes flows from Operations Orchestration (OO) to Service Manager according to the basepaths that are configured in Integration Manager (Service Manager Integration Suite). The following describes the synchronization rules, using an example **basepath** value: **/Library/ITIL/Change Management; /Library/How Do I flows**.

Rule 1: Only the Operations Orchestration flows under the basepath or its sub-folders are synchronized.

Operations Orchestration Flow Path	Synchronize?
/Library/ITIL/Change Management/Network/Compare Device ACLs	Yes
/Library/ITIL/Change Management/Servers/Block NAS Task	Yes
/Library/How Do I flows/How do I: Iterate through a list	Yes
/Library/ITIL/Incident Management/Servers/Server Health Check	No

Rule 2: If an Operations Orchestration flow is removed in Operations Orchestration, it will also be removed in Service Manager when it is under the basepath or its sub-folders.

Operations Orchestration Flow Path Deleted in Operations Orchestration	Remove in Service Manager?
/Library/ITIL/Change Management/Network/Compare Device ACLs	Yes
/Library/ITIL/Change Management/Servers/Block NAS Task	Yes
/Library/How Do I flows/How do I: Iterate through a list	Yes
/Library/ITIL/Incident Management/Servers/Server Health Check	No

Rule 3: If an Operations Orchestration flow is updated in Operations Orchestration, it will also be updated in Service Manager when it is under the basepath or its sub-folders.

Operations Orchestration Flow Path Updated in Operations Orchestration	Update in Service Manager?
/Library/ITIL/Change Management/Network/Compare Device ACLs	Yes
/Library/ITIL/Change Management/Servers/Block NAS Task	Yes
/Library/How Do I flows/How do I: Iterate through a list	Yes
/Library/ITIL/Incident Management/Servers/Server Health Check	No

Note

If an Operations Orchestration flow (under the basepath or its sub-folders) is updated in Service Manager, the updates will not be discarded until the Operations Orchestration flow is updated in Operations Orchestration.

Related topics

Integration Manager
Integration Instance Information fields
Enable SSL connection from Service Manager to Operations Orchestration
Operations Orchestration (OO)
Operations Orchestration flow detail form fields
Add an Operations Orchestration integration
Enable LW-SSO for the Operations Orchestration integration
Configure LW-SSO in the Service Manager Web tier
Enable or disable an integration instance
Monitor integration instance status

1.7.2.2. Operations Orchestration flow detail form fields

You can view the details of an Operations Orchestration (OO) flow from **Knowledge Management** > **Configuration** > **Operations Orchestration Flows**. The following table lists the fields contained in the OO flow detail form.

Note

In the table below, an OO field refers to a field from Operations Orchestration, and the integration instance synchronizes its value from Operations Orchestration. A custom field is only for Service Manager (has no corresponding field in Operations Orchestration), and the integration uses this field when launching the OO flow.

Field	Value	Comments
UUID	A universally unique identifier automatically assigned to the OO flow when it gets created in OO.	An OO field.
Path	Full path of the OO flow. Consists of the path and name of the OO flow. For example: `/Library/ITIL/Change Management/Network/Compare Device ACLs`.	An OO field.
Display Name	Name of the OO flow (defined by the user) in Service Manager.	A custom field. When the integration synchronizes an OO flow from OO, this field displays the name of the OO flow. However, you can change the name as you like.
Parameter Name	Name of an input for the operation(s) of the OO flow. The person running the flow can enter this name or set it to a specific value. The system can also obtain this name from information gathered by another step of the OO flow.	An OO field.
Required?	**true** or **false**	An OO field. Indicates whether the parameter is obligatory or not.
Secure?	**true** or **false**	Indicates whether the field is a security field or not. The value of a security field will display as a string of asterisks.

Field	Value	Comments
Description	Full description of the OO flow, including the steps/operations, inputs, responses, and returns of the OO flow.	An OO field.

Related topics

- Operations Orchestration (OO)
- Operations Orchestration flow synchronization rules
- Manage OO flows from Knowledge Management
- Remove OO flow links from a knowledge document
- Launch OO flows from an incident
- View OO flow execution results from an incident

1.7.2.3. Add OO flow links to a knowledge document

User roles: Only users that have the permissions to update a knowledge document can add Operations Orchestration (OO) flow links to it.



Note

Before you perform the following steps, make sure that a Service Manager to OO integration (SMOO) instance is enabled in Integration Manager (SMIS).

To add OO flow links to a new knowledge document:

1. Sign in to the Service Manager web client.
2. Click **Knowledge Management** > **Contribute Knowledge**.
3. Select a document type. The Contribute Knowledge window opens.
4. Select the **OO Flow Links** tab.
5. In the **Select Path** field, select a path, and click **Add Link**.

   The OO Flow Link Detail form is displayed.
6. In the Parameters section, complete the fields in the following table as necessary.

   In the table below, an OO field refers to a field from Operations Orchestration, and the integration instance synchronizes its value from Operations Orchestration; a custom field is only for Service Manager (has no corresponding field in Operations Orchestration), and will be used when launching the OO flow.

   Note

   In some cases, you may need to add additional parameter names and values that are required to launch the OO flow.

Field	Value	Description
Parameter Name	Name of an input for the operation(s) of the OO flow. Can be entered by the person running the flow; or set to a specific value; or obtained from information gathered by another step of the OO flow.	An OO field.
Required?	**true** or **false**	An OO field. Indicates whether the parameter is required or not.
Mapped CI Field	Select one from the drop-down list.	A custom field. Useful CI fields to launch the OO flow. The CI field value will be populated when launching the OO flow. The CI fields listed are defined in global list "OO Fields".
Default Value	Default value of the parameter.	A custom field. If there is no Mapped CI Field configured for the parameter, or no value in the Mapped CI Field, the default value will be populated when launching the OO flow. **Note:** This field is not the same field in OO.

7. Click **Add**. A message displays, stating the OO flow link is added.
8. Click **OK**. The OO flow appears in the Links table, and the OO Flow UUID displays as a link.
9. (Optional) If you want to update the OO flow, click the UUID link. Update the parameters and then click **OK**.

   Note: If you don't click **OK** to save and exit, your updates to the Mapped CI Field and Default Value fields might be lost.

10. Repeat the steps above to add more OO flow links to the knowledge document.

To add OO flow links to an existing knowledge document:



Note

You can only add OO flow links to published documents and draft documents (not to retired documents).

1. Sign in to the Service Manager web client.
2. Click **Knowledge Management** > **Published Documents** or **Draft Documents**.
3. Search for and open a published document, or directly select a draft document from the list.
4. Click **Edit**. The Contribute Knowledge window opens.
5. Select the **OO Flow Links** tab.
6. Add OO flow links to the knowledge document, by following steps 3 to 9 described above for a new knowledge document.

Related topics

Integration Manager
Operations Orchestration (OO)
Automated resolution of incidents
Add an Operations Orchestration integration
Enable or disable an integration instance
Remove OO flow links from a knowledge document
Launch OO flows from an incident

1.7.2.4. Remove OO flow links from a knowledge document

User roles: Only users that have the permissions to update a knowledge document can remove Operations Orchestration (OO) flow links from the knowledge document.



Note

Before you perform the following steps, make sure to enable a Service Manager to OO integration instance in Integration Manager (SMIS).

To remove OO flow links from a knowledge document:

1. Sign in to the Service Manager Web client.
2. Click **Knowledge Management** > **Published Documents** or **Draft Documents**.
3. Search for and open a published document, or directly select a draft document from a list of draft documents.
4. Click **Edit**. The Contribute Knowledge window opens.
5. Select the **OO Flow Links** tab.
6. Select an OO flow link from the OO Flow Path table, and click **Remove Link**.

   Service Manager removes the OO flow link from the OO Flow Path table. At the same time, a message displays, stating that the OO flow is no longer linked to the knowledge document.
7. Repeat step 5 to remove more OO flow links from the knowledge document.

Related topics

- Operations Orchestration (OO)
- Operations Orchestration flow synchronization rules
- Operations Orchestration flow detail form fields
- Manage OO flows from Knowledge Management
- Add OO flow links to a knowledge document
- Launch OO flows from an incident
- View OO flow execution results from an incident

1.7.3. Automated resolution of incidents

When integrated with Service Manager, OO shares information between monitoring and automation systems and the help desk. Incident Management processes are enhanced by linking Knowledge documents with OO flows, allowing technicians to triage, diagnose, and resolve incidents more quickly and efficiently.

Web client users can add or remove OO flows links for a knowledge document, execute flows from related knowledge documents for an incident; and view OO flow execution results attached to an incident as historic activities.

Related topics

Integration Manager
Automated deployment of changes
Add an Operations Orchestration integration
Manage OO flows from Knowledge Management
Add OO flow links to a knowledge document
Remove OO flow links from a knowledge document
Launch OO flows from an incident
View OO flow execution results from an incident

1.7.3.1. Launch OO flows from an incident

When searching knowledge from an incident, you can launch Operations Orchestration (OO) flows linked to a knowledge document. Only users who have the permissions to edit a knowledge document can launch the OO flows linked to it.

> **Note**
>
> This feature is available only when a Service Manager to OO integration instance is enabled in Integration Manager (SMIS) and the Search Engine is installed. To work with this feature, you must enable IDOL first.

To launch OO flows from an incident:

1. Sign in to the Service Manager Web client.
2. From **Incident Management**, search for an incident record and open the record.
3. Search for knowledge documents by using Smart Search.
4. Select the knowledge document from which you want to launch one or more associated OO flows. The knowledge document opens.
5. Click **Execute OO Flow**.
   - If there is no OO flow linked to the document, an error message is displayed: "No related OO flows found."

     You can select to click **Edit** to add OO flow links to the document, and then click **Execute OO Flow** again. You can also select to close the document window to exit.
   - If there is only one OO flow linked to the document, the **Set Parameters** page opens.
   - If there are multiple OO flows linked to the document, the **Select an OO Flow** page opens. Select an OO flow from the list. The **Set Parameters** page opens.

   > **Note**
   >
   > - If the selected OO flow has a parameter with a Mapped CI Field (for example, **Ip Address**) and the Affected CI of the incident has a corresponding value (for example, an IP address) for this Mapped CI Field, this affected CI field value from the incident will be automatically populated as the value of this OO flow parameter. If the mapped CI field is an array type field, the first value of the array will be populated.
   > - If the selected OO flow has a parameter without a Mapped CI Field, the value specified in the Default Value field of this parameter will be populated.
6. Provide all the required parameter values for the selected OO flow.

> **Note**
>
> You may need to add additional parameter names and provide their values in some cases.

7. (Optional) If you want the OO flow to be executed asynchronously, select **Asynchronous**.
8. Click **Next**. The selected OO flow is launched, and you are returned to the Select an OO Flow page.

> **Note**
>
> If you don't want to launch more OO flows, click **Finish** instead of **Next** to launch the selected OO flow and to return to the knowledge document detail page.

9. Continue to launch more OO flows from the list on the Select an OO Flow page.
10. Click **Finish**.

    Once an OO flow execution is complete, a dialog box opens, stating that the execution is done and asking if you want to view the execution result.

    - Click **Yes** to view the execution result in Operations Orchestration.

      If LW-SSO is configured in both the Service Manager web client and Operations Orchestration, you are directed to the execution result in Operations Orchestration. If LW-SSO is not configured, an Operations Orchestration login page opens; after you sign in to Operations Orchestration, the execution result is displayed.

    - Click **No** to exit the dialog box.

Related topics

Operations Orchestration (OO)
Automated resolution of incidents
Manage OO flows from Knowledge Management
Add OO flow links to a knowledge document
Remove OO flow links from a knowledge document
View OO flow execution results from an incident

1.7.3.2. View OO flow execution results from an incident

Suppose you have searched for a knowledge document from an incident. After launching an Operations Orchestration (OO) flow from this knowledge document, you can view the OO flow execution result from the list of Historic Activities of the incident.

To view an OO flow execution result from an incident:

1. Sign in to the Service Manager Web client.
2. From **Incident Management**, search for an incident record and open it.
3. In the **Activities** section, select an activity of the **OO Response** type from the activities list.

   The Activity Log - Incident Management page opens, displaying the execution details of an OO flow launched from this incident.
4. Click **Open Report**.

   If you have configured Lightweight Single Sign-On (LW-SSO) in both the Service Manager web client and Operations Orchestration, the system directs you to the execution result in Operations Orchestration. If you haven't configured LW-SSO, an Operations Orchestration login page opens, and the execution result displays after you log in.

Related topics

- Operations Orchestration (OO)
- Operations Orchestration flow synchronization rules
- Operations Orchestration flow detail form fields
- Manage OO flows from Knowledge Management
- Add OO flow links to a knowledge document
- Remove OO flow links from a knowledge document
- Launch OO flows from an incident

1.7.4. Automated deployment of changes

Service Manager 9.30 or later provides an enhanced integration with Operations Orchestration (OO) that allows users to link requests for change (RFCs) to OO flows. This enables users to implement automated deployment of changes by manually or automatically launching OO flows.

Before a change moves to the Change Approval phase, you can link OO flows to the Change. Hence, Service Manager performs each of the change deployment steps and returns the results. These OO flows originated from out-of-the-box content in OO intended to support common Change use cases. These use cases including Database Script Execution, Service/Server Restart, Running Batch Files, etc.

To run the automated flow, select the relevant OO flows and complete all required parameters of the flows. These parameters can be either fixed or references to attributes of the Request for Change (RFC) or Configuration Items (CIs) in the RFC. You also need to select when and how to trigger the Change flows - upon Change approval, based on a schedule or manually. When a Change moves to the predefined phase, you can manually launch the OO flows from within Service Manager. Or, the OO flows are automatically executed based on the trigger conditions you set.



Note

The following change categories don't support this feature.

- Subscription
- Unplanned Change

Related topics

- Operations Orchestration (OO)
- Add OO flow links to a new change record
- Add OO flow links to a new change record
- Add OO flow links to an existing change record
- Update OO flow links in a change record
- Remove OO flow links from a change record
- Manually launch change flows
- Automatically launch change flows upon approval
- Automatically launch change flows on a schedule (for a new change)
- Automatically launch change flows on a schedule (for an existing change)
- View execution results of change flows

1.7.4.1. Add OO flow links to a new change record

User roles: Change Manager, Change Coordinator, and Release Manager

To add OO flow links to a *new* Change record:

1. Sign in to the Service Manager Web client.
2. Click **Change Management** > **Changes** > **Create New Change**.
3. Select a change category other than **Subscription** and **Unplanned Change**.

   Note

   Do not select any of these categories, because the OO Flow Links feature is unavailable for a change record with these change categories.

4. Complete the form with all required and optional information.
5. Go to the OO Flow Links section, select a path in the Select Path field, and click **Add Link**. The OO Flow Link Detail form is displayed.
6. In the **Sequence No** field, type a number. This number defines the order in which the OO flow is automatically launched. Each OO flow link must have a unique Sequence No.
7. In the **Target CIs** field, enter the target Configuration Items that will be used when the OO flow is launched.

   Note

   If you are planning to launch the flow manually, you can skip this step.

8. In the Parameters section, complete the fields listed in the following table as necessary.

Field	Value	Description
Parameter Name	Name of an input for the operation(s) of the OO flow; can be entered by the user running the flow, or set to a specific value, or obtained from information gathered by another step of the OO flow	An OO field
Required?	`true` or `false`	An OO field that indicates whether the parameter is required or not
Mapped Change Field	Select one from the drop-down list.	A custom field This is a field from the change record used to launch the OO flow. The Change field value is populated when the OO flow is launched. The Change fields listed are defined in the "OO RFC Fields" global list.

Field	Value	Description
Mapped CI Field	Select one from the drop-down list.	A custom field Useful CI field to launch the OO flow. If there is no Mapped Change Field configured for the parameter, or there is no Mapped Change Field value, the CI field value will be populated when launching the OO flow. The CI fields listed are defined in "OO Fields" global list.
Default Value	Default value of the parameter	A custom field If neither Mapped Change Field nor Mapped CI Field is configured for the parameter, or if no value is available in Mapped Change Field or Mapped CI Field, the default value is used when the OO flow is launched. **NOTE:** This field is not the same field in OO.

9. Click **Add**. The OO flow link is added.
10. Click **OK**. You are returned to the Change form.



Note

If you don't click **OK** to save and exit, your updates to the Mapped CI Field and Default Value fields might be lost.

11. Repeat the steps above to add more links as necessary.
12. Click **Save** to save the change record.

Related topics

Automated deployment of changes
Add OO flow links to an existing change record
Update OO flow links in a change record
Remove OO flow links from a change record
Manually launch change flows
Automatically launch change flows upon approval
Automatically launch change flows on a schedule (for a new change)
Automatically launch change flows on a schedule (for an existing change)
View execution results of change flows

1.7.4.2. Add OO flow links to an existing change record

User roles: Change Manager, Change Coordinator, and Release Manager

> **Note**
>
> You can update OO flow links only in a change record of a category other than either of the following:
>
> - Subscription
> - Unplanned Change

To add OO flow links to an *existing* change record:

1. Sign in to the Service Manager Web client.
2. Open an existing change record that meets the conditions previously mentioned.
3. Go to the OO Flow Links section, select a path in the Select Path field, and click **Add Link**. The OO Flow Link Detail form is displayed.
4. In the **Sequence No** field, type a number. This number defines the order in which the OO flow is automatically launched. Each OO flow link must have a unique Sequence No.
5. In the **Target CIs** field, enter the target Configuration Items that will be used when the OO flow is launched.

   > **Note**
   >
   > If you are planning to launch the flow manually, you can skip this step.

6. In the Parameters section, complete the fields listed in the following table as necessary.

Field	Value	Description
Parameter Name	Name of an input for the operation(s) of the OO flow; can be entered by the user running the flow, or set to a specific value, or obtained from information gathered by another step of the OO flow	An OO field
Required?	`true` or `false`	An OO field that indicates whether the parameter is required or not
Mapped Change Field	Select one from the drop-down list.	A custom field This is a field from the change record used to launch the OO flow. The Change field value is populated when the OO flow is launched. The Change fields listed are defined in the "OO RFC Fields" global list.

Field	Value	Description
Mapped CI Field	Select one from the drop-down list.	A custom field Useful CI field to launch the OO flow. If there is no Mapped Change Field configured for the parameter, or there is no Mapped Change Field value, the CI field value will be populated when launching the OO flow. The CI fields listed are defined in "OO Fields" global list.
Default Value	Default value of the parameter	A custom field If neither Mapped Change Field nor Mapped CI Field is configured for the parameter, or if no value is available in Mapped Change Field or Mapped CI Field, the default value is used when the OO flow is launched. **Note** This field is not the same field in OO.

7. Click **Add**. The OO flow link is added.
8. Click **OK**. You are returned to the Change form.

Note

If you don't click **OK** to save and exit, your updates to the Mapped CI Field and Default Value fields might be lost.

9. Repeat the steps above to add more links as necessary.
10. Click **Save** to save the change record.

Related topics

Automated deployment of changes
Add OO flow links to a new change record
Update OO flow links in a change record
Remove OO flow links from a change record
Manually launch change flows
Automatically launch change flows upon approval
Automatically launch change flows on a schedule (for a new change)
Automatically launch change flows on a schedule (for an existing change)
View execution results of change flows

1.7.4.3. Update OO flow links in a change record

User roles: Change Manager, Change Coordinator, and Release Manager

> **Note**
>
> You can update OO flow links only in a Change record of a category other than either of the following:
>
> - Subscription
> - Unplanned Change

To update OO flow links in a change record:

1. Sign in to the Service Manager web client.
2. Open an existing change record that meets the conditions mentioned at the beginning of this topic.
3. In the OO Flow Links section, click the **Seq No** of an OO flow link that you want to update. The OO Flow Link Detail form opens.
4. Update the fields as necessary, and then click **Save**.
5. Click **Cancel**. You are returned to the Change form.
6. Click **Save** to save the change record.

Related topics

- Automated deployment of changes
- Add OO flow links to a new change record
- Add OO flow links to an existing change record
- Remove OO flow links from a change record
- Manually launch change flows
- Automatically launch change flows upon approval
- Automatically launch change flows on a schedule (for a new change)
- Automatically launch change flows on a schedule (for an existing change)
- View execution results of change flows

1.7.4.4. Remove OO flow links from a change record

User roles: Change Manager, Change Coordinator, and Release Manager

To remove OO flow links from a change record

1. Sign in to the Service Manager Web client.
2. Open the change record from which you want to remove OO flow links.
3. In the OO Flow Links section, click the **Seq No** of an OO flow link that you want to remove. The OO Flow Link Detail form opens.
4. Click **Delete**, and then click **YES** to confirm the deletion.

   You are returned to the Change form with the OO flow link removed.
5. Click **Save** to save the change record.

Related topics

- Automated deployment of changes
- Add OO flow links to a new change record
- Add OO flow links to an existing change record
- Update OO flow links in a change record
- Manually launch change flows
- Automatically launch change flows upon approval
- Automatically launch change flows on a schedule (for a new change)
- Automatically launch change flows on a schedule (for an existing change)
- View execution results of change flows

1.7.4.5. Manually launch change flows

User roles: Change Manager, Change Coordinator, and Release Manager

Once a change record is approved and advances to the pre-defined phase, you can select to manually launch the OO flows linked to the change record.

Note

You can update OO flow links only in a change record of a category other than either of the following:

- Subscription
- Unplanned Change

To manually launch OO flows from a change record:

1. Sign in to the Service Manager Web client.
2. Open an existing change record that meets the conditions previously mentioned.
3. Click **More**, and then select **Execute OO Flow**.
    - If there is no OO flow linked to the change, an error message appears: "No related OO flows found."
    - If there is only one OO flow linked to the change, the Select Affected CI for OO Flow page opens.
    - If there are multiple OO flows linked to the change, the Select an OO Flow page opens. Select an OO flow from the table. The Select affected CI for OO flow page opens.
4. Select an affected CI from the table or click **Skip** to go to the next page if there are no CIs related to this OO flow. The Set Parameters page opens.
    - If the selected OO flow has a parameter with a Mapped Change Field (for example, Impact) and the change has a corresponding value for this Mapped Change Field, this Impact field value from the change will be automatically populated as the value of this OO flow parameter. If the Mapped Change Field is an array type field, the values of the array will be combined and separated by commas (,).
    - If the selected OO flow has a parameter with a Mapped CI Field (for example, Ip Address) and the Affected CI of the change has a corresponding value (for example, an IP address) for this Mapped CI Field, this affected CI field value from the change will be automatically populated as the value of this OO flow parameter. If the mapped CI field is an array type field, the values of the array will be combined and separated by commas (,).
    - If the selected OO flow has a parameter without a Mapped Change Field or Mapped CI Field, the value specified in its Default Value field will be populated.

5. Provide all the required parameter values for the selected OO flow.

   > **Note**
   >
   > In some cases, you may need to add additional parameter names and specify their values.

6. (Optional) If you want the OO flow to be executed asynchronously, select **Asynchronous**.
7. Click **Next**. The selected OO flow is launched, and you are returned to the Select an OO Flow page when the change is linked to more than one OO flow.

   > **Note**
   >
   > If you don't want to launch more OO flows, click **Finish** instead of **Next** to launch the selected OO flow and return to the change form.

8. Continue to launch more OO flows from the list on the Select an OO Flow page.
9. Click **Finish**.
10. Once an OO flow execution is complete, a dialog box opens, stating that the execution is done and asking if you want to view the execution result.
    - Click **Yes** to view the execution result in Operations Orchestration.

      If LW-SSO is configured in both the Service Manager Web client and Operations Orchestration, the execution result in Operations Orchestration is displayed without any login prompt. If LW-SSO is not configured, an Operations Orchestration log-in page opens; after you sign in to Operations Orchestration, the execution result is displayed.
    - Click **No** to close the dialog box.

Related topics

Automated deployment of changes
Automated deployment of changes
Add OO flow links to a new change record
Enable LW-SSO for the Operations Orchestration integration
Add OO flow links to a new change record
Add OO flow links to an existing change record
Update OO flow links in a change record
Remove OO flow links from a change record
Automatically launch change flows upon approval
Automatically launch change flows on a schedule (for a new change)
Automatically launch change flows on a schedule (for an existing change)
View execution results of change flows

1.7.4.6. Automatically launch change flows upon approval

User roles: Change Manager, Change Coordinator, and Release Manager

You can set a change record so that the linked OO flows are executed automatically once it gets approved and moved to the predefined phase.

1. Sign in to the Service Manager Web client.
2. Open a change in the Change Assessment & Planning phase (or in the Assess or the Plan and Design phase if the change has a category of Release Management).
3. In the OO Flow Links section, select **When approved** as the Automation Type.
4. Click **Save** to save the change record.
5. Once an IT agent approves the change, click **Close Phase** to move the change to the predefined phase.

   Service Manager creates a scheduler of OO Flow Automation and starts it in 50 seconds, and the alert stage is **SMOO Auto-Approval**.

   The OO flows are automatically executed, and all the execution reports are saved in an activity log with an operator of **smoo-automation**.

Related topics

- Automated deployment of changes
- Add OO flow links to a new change record
- Add OO flow links to an existing change record
- Update OO flow links in a change record
- Remove OO flow links from a change record
- Manually launch change flows
- Automatically launch change flows on a schedule (for a new change)
- Automatically launch change flows on a schedule (for an existing change)
- View execution results of change flows

1.7.4.7. Automatically launch change flows on a schedule (for a new Change)

User roles: Change Manager, Change Coordinator, and Release Manager

You can select to trigger the automatic execution of Change flows at a scheduled time: either a specific date and time, or a future time based on a field value of a Change, such as Planned Start, Planned End, and Requested End Date. Once the Change is approved and advances to the pre-defined phase, the OO flows linked to this Change record will be automatically launched at the scheduled time.

To set an automatic execution time for OO flows of a new Change:

1. Sign in to the Service Manager Web client.
2. From Change Management, click **Create New Change**.
3. Select a Change category other than **Subscription** and **Unplanned Change**.

   Note

   The OO Flow Links feature is unavailable for a Change record with any of these Change categories.

4. Complete the form with all required and optional information.
5. In the OO Flow Links section, do either of the following:

   Select **Use a fixed date** as the Automation Type. The **Scheduled Time** field is displayed. Select a date and specify a time, for example, `06/04/10 10:00:00`.

   Note

   The date/time format is determined by your profile date/time format.

   Or

   Select **Use a field in the record + interval** as the Automation Type. The **Change Field** and **Interval** fields are displayed. Select a field in the Change Field list as the base time, and then enter a value in the Interval field using the `+/-ddd hh:mm:ss` format. Note that a space is required between `ddd` and `hh`, and you can omit the plus sign (`+`) for positive intervals.

   For example, if you select **Requested End Date** in Change Field and enter `-1 02:30:10` in Interval, the OO flows of the Change will be automatically launched one day, two hours, thirty minutes, and ten seconds before the Requested End Date.

Note

- The automation execution time should be a future time between the Planned Start and Planned End.
- The automation execution time you selected is invalidated now. Instead, the time will be validated when the Change moves to the pre-defined phase. If the validation fails, you will receive a warning and the Change can't advance to implementation. In this case, you have to reset the execution time. To do so, click **Cancel** and move the Change back to a phase prior to Change Approval (or back to **Assess** or **Plan and Design**), reset the automation execution time, and then go through the Change phases again.

Related topics

Automated deployment of Changes
Add OO flow links to a new Change record
Add OO flow links to an existing Change record
Update OO flow links in a Change record
Remove OO flow links from a Change record
Manually launch Change flows
Automatically launch Change flows upon approval
Automatically launch Change flows on a schedule (for an existing Change)
View execution results of Change flows

1.7.4.8. Automatically launch change flows on a schedule (for an existing change)

User roles: Change Manager, Change Coordinator, and Release Manager

> **Note**
>
> You can update OO flow links only in a change record of a category other than either of the following:
>
> - Subscription
> - Unplanned Change

To set an automatic execution time for OO flows of an existing change:

1. Sign in to the Service Manager Web client.
2. Open an existing change record that meets the conditions previously mentioned.
3. In the OO Flow Links section, do either of the following:

   Select **Use a fixed date** as the Automation Type. The **Scheduled Time** field is displayed. Select a date and specify a time, such as `06/04/10 10:00:00`.

   > **Note**
   >
   > The date/time format is determined by your profile date/time format.

   Or

   Select **Use a field in the record + interval** as the Automation Type. The **Change Field** and **Interval** fields are displayed. Select a field in the Change Field list as the base time, and then enter a value in the Interval field using the `+/-ddd hh:mm:ss` format. Note that a space is required between `ddd` and `hh`, and you can omit the plus sign (`+`) for positive intervals.

   For example, if you select **Requested End Date** in Change Field and enter `-1 02:30:10` in **Interval**, the OO flows of the change will be automatically launched one day, two hours, thirty minutes, and ten seconds before the Requested End Date.

Note

- The automation execution time should be a future time between the Planned Start and Planned End.
- The automation execution time you selected is invalidated now. Instead, the time will be validated when the change moves to the pre-defined phase. If the validation fails, you will receive a warning and the change can't advance to implementation. In this case, you have to reset the execution time. To do so, click **Cancel** and move the change back to a phase prior to Change Approval (or back to **Assess** or **Plan and Design**), reset the automation execution time, and then go through the change phases again.

Related topics

Automated deployment of changes
Add OO flow links to a new change record
Add OO flow links to an existing change record
Update OO flow links in a change record
Remove OO flow links from a change record
Manually launch change flows
Automatically launch change flows upon approval
Automatically launch change flows on a schedule (for a new change)
View execution results of change flows

1.7.4.9. View execution results of change flows

User roles: Change Manager, Change Coordinator, and Release Manager

OO flow execution results are saved in Service Manager in different ways depending on how they're executed: manually or automatically.

To view OO flow execution results from a change record:

1. Sign in to the Service Manager Web client.
2. From Change Management, search for a change record and open it.
3. In the Activities section, do one of the following:
   - Select an activity of the **OO Response** type from the activities list. The Activity Log - Change Management page opens, displaying the execution details of an OO flow manually launched from this change.
   - Select an activity that has a type of **OO Response** and an operator of **smoo-automation**. The execution results of all the OO flows automatically executed are saved in this log as separate links.
4. Click **Open Report**.

   Note

   For automatically executed OO flows, clicking **Open Report** opens only the report link of the first OO flow. You can view the reports of the other OO flows directly in OO.

   If you have configured Lightweight Single Sign-On (LW-SSO) in both the SM web client and Operations Orchestration, SM displays execution results in Operations Orchestration without any login prompt. If you haven't configured LW-SSO, an Operations Orchestration login page opens, and displays the execution result after you sign in.

Related topics

- Automated deployment of changes
- Add OO flow links to a new change record
- Add OO flow links to an existing change record
- Update OO flow links in a change record
- Remove OO flow links from a change record
- Manually launch change flows
- Automatically launch change flows upon approval
- Automatically launch change flows on a schedule (for a new change)
- Automatically launch change flows on a schedule (for an existing change)

1.7.5. Automated implementation of change tasks

Service Manager provides an enhanced integration with Operations Orchestration (OO) that allows users to link change tasks to OO flows. This enables users to implement automated fulfillment of change tasks by manually or automatically launching OO flows.

You can add OO flow links to the "Automation" tasks planned in change models. Depending on your configuration, when the change tasks are opened in change records, the OO flows are automatically executed or you can manually launch the OO flows. Or, you can add an OO flow to a change task directly from a change record via record level task planner.



Note

In the out-of-the-box environment, this feature is only supported for the tasks in the Automation task category.

Related topics

- Operations Orchestration (OO)
- Add an Operations Orchestration integration
- Enable OO flow automation for change tasks
- Add an OO flow link to a change task
- Execute an OO flow from a change task

1.7.5.1. Enable OO flow automation for change tasks

Applies to User Roles: System Administrators

To enable OO flow automation for change tasks, follow these steps:

1. Make sure that the Operations Orchestration integration is set up. For details, see Add an Operations Orchestration integration.
2. Navigate to **Tailoring** > **Process Designer** > **Configuration** > **Task Planner Mapping**, and then click **Search**.
3. Locate and open the "cm3r" task planner mapping record, which contains the status mapping information between change task and task planner.
4. Select the **Enable OO Flow for Automation** checkbox.
5. Click **Save**.
6. Navigate to **Miscellaneous** > **System Status**, click the **Start Scheduler** button, and then click **AutomationTaskUpdate** to start this schedule. When this schedule is started, the status of the change task can be automatically updated after the OO flow execution.

Related topics

Automated implementation of change tasks
Add an Operations Orchestration integration
Add an OO flow link to a change task
Execute an OO flow from a change task

1.7.5.2. Add an OO flow link to a change task

User roles: Change Manager, Change Coordinator

Before you can add an OO flow link to a change task, make sure that you have already enabled the OO flow automation for change tasks. For details, see Enable OO flow automation for change tasks.

You can add OO flow links to the change tasks by planning tasks in change models or directly in change records. Then, the changes will contain the change tasks associated with the OO flows that you defined.

To add an OO flow link to a change task in a change model, follow these steps:

> **Note**
>
> If you want to add an OO flow link directly to a change task from a change record instead of change model, open the task planner from the change record.

1. Sign in to the Service Manager Web client.
2. Click **Change Management** > **Configuration** > **Change Models**.
3. Search for and open the change model record that you want to modify.
4. Go to the **Task Information** section, and then click **OO Flow Configuration**. The OO Flow Configuration form is displayed.
5. Select an OO flow in the **Select Path** field, and then click **Add Link**. The OO Flow Details form is displayed.
6. Type a name for the OO flow link.
7. Select the **Execute Automatically** checkbox if you want to execute the OO flow automatically. Otherwise, clear the checkbox so that you can manually execute the OO flow by clicking **Execute OO Flow** from the change task records.
8. In the **General Parameters** section and the **Secure Parameters** section, complete the fields listed in the following table for each parameter as necessary.

Field	Value	Description
Parameter Name	Name of an input for the operations of the OO flow; can be entered by the user running the flow, or set to a specific value, or obtained from information gathered by another step of the OO flow	An OO field.
Required?	`true` or `false`	An OO field that indicates whether the parameter is required or not.
Default Value	Default value of the parameter	A custom field. If the Value field is not configured for the parameter, or if no value is available in the Value field, the default value is used when the OO flow is launched. **Note** This field is not the same field in OO.
Value Type	Available Types are: • Additional Properties • Parent File • Current File • User Options • JavaScript Select one from the drop-down list.	A custom field. The value type defines where the OO flow parameter value can be retrieved when the OO flow is launched.

Field	Value	Description
Value	The available fields based on the selected value type. Select one from the drop-down list.	A custom field. The value of the selected field will be used to launch the OO flow. **Note** If Join Table is set to true, this "Value" is used to match the "Join Table Field" to identify the corresponding value for the specified "Select Field", which is then used to launch the OO flow.
Join?	`true` or `false`	A custom field. If it is set to true, Join Table name is required.
Join Table	All available table names. Select one from the drop-down list.	A custom field. Select the join table name. For example, if the value type is "Parent File" (in change task planner, the Parent File is "change") and the value is "Affected Service", when the Join Table is selected, you may select the Join Table Name as "device".

Field	Value	Description
Join Table Field	Available fields based on the selected Join Table. Select one from the drop-down list.	A custom field. The field list of the selected Join Table. The value of this field should match the value of the "Value" field. For example, "CI Identifier" should be selected in Join Table Field if "Affected Service" is selected as the "Value" of the Parent File value type.
Select Field	A field other than the Join Table Field in the Join Table. Select one from the drop-down list.	A custom field. The value of the selected field will be used to launch the OO flow when Join Table is used.

Note

If you want the change task to be performed on all Affected Configuration Items of the change record when the OO flow is launched, click the parameter links to configure the **Value Type** field as Parent File and the **Value** field as Affected Configuration Item.

9. In the **Secure Parameters** section, complete the fields for security parameters as necessary.

Field	Value	Description
Field Name	Name of the security parameter	An OO field.
Required?	`true` or `false`	An OO field that indicates whether the parameter is required or not.
Default Value	Default value of the parameter	A custom field. If the Value field is not configured for the parameter, or if no value is available in the Value field, the default value is used when the OO flow is launched. **Note** This field is not the same field in OO.

10. Click **Save and Exit**, and then go back to the **Task Information** section in the change model.
11. Click **Edit** to open the task planner.
12. Select or create a change task that's in the "Automation" task category.
13. In the **OO Flow Link** drop-down list, select the OO flow link that you added.
14. Click **Save**. The OO flow link is now added to the change task of the change model. The changes that use this change model will contain the change task with the OO flow link that you defined.

Related topics

Automated implementation of change tasks
Add an Operations Orchestration integration
Enable OO flow automation for change tasks
Execute an OO flow from a change task

1.7.5.3. Execute an OO flow from a change task

Security roles: Change Coordinator, Change Analyst

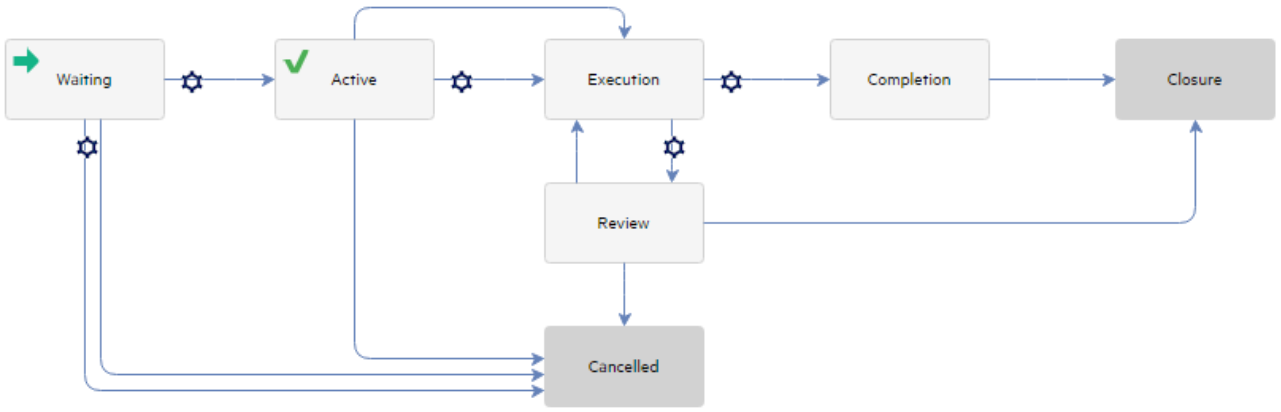
Suppose you have selected the **Execute Automatically** checkbox in the OO flow link configuration. Once a change task linked to that OO flow moves to the Execution phase, the associated OO flow will execute automatically. Otherwise, you need to manually execute the OO flow from the change task record.

To manually launch an OO flow from a change record, follow these steps:

1. Sign in to the Service Manager Web client.
2. Click **Change Management** > **Search Tasks**, and then search for and open the change task that requires a manual execution of the OO flow link.
3. From the change task record, click the **Execute OO Flow** button.
4. Verify the OO flow configuration, and then click **OK** to execute the OO flow.

When the OO flow is successfully executed, the change task moves to the **Completion** phase automatically. If additional properties are defined, the change analyst or change coordinator needs to manually fill in the OO execution results in the additional properties. Hence, the remaining tasks can benefit from the output of the current automation task.

If the OO flow execution fails, the change task moves to the **Review** phase automatically. After review, you can launch the OO flow again manually, or you can cancel or close the change task.



To view the execution result of the OO flow, follow these steps:

1. Open a change task record.
2. From the **Activities** tab, you can find the OO execution result shown as activity logs.
3. Click the **Show Report** button if you want to check the execution result in OO.

Related topics

Automated implementation of change tasks
Add an Operations Orchestration integration

Enable OO flow automation for change tasks
Add an OO flow link to a change task

1.7.6. Automated fulfillment of request tasks

Service Manager 9.40 or later provides an enhanced integration with Operations Orchestration (OO) that allows users to link request tasks to OO flows. This enables users to implement automated fulfillment of request tasks by manually or automatically launching OO flows.

You can add OO flow links to the "Automation" tasks planned in request models. Then, when the request tasks are opened in request records, the OO flows are automatically executed or you can manually launch the OO flows, depending on the configuration of the OO flow links. Or, you can add an OO flow to a request task directly from a request record.



Note

In the out-of-the-box environment, this feature is only supported for the tasks in the Automation category.

Related topics

- Operations Orchestration (OO)
- Add an Operations Orchestration integration
- Enable OO flow automation for request tasks
- Add an OO flow link to a request task
- Execute an OO flow from a request task

1.7.6.1. Enable OO flow automation for request tasks

User roles: System Administrators

To enable OO flow automation for request tasks, follow these steps:

1. Make sure that the Operations Orchestration integration is set up. For details, see Add an Operations Orchestration integration.
2. Navigate to **Tailoring** > **Process Designer** > **Configuration** > **Task Planner Mapping**, and then click **Search**.
3. Locate and open the "request" task planner mapping record, which contains the status mapping information between request task and task planner.
4. Select the **Enable OO Flow for Automation** checkbox.
5. Click **Save**.
6. Navigate to **Miscellaneous** > **System Status**, click the **Start Scheduler** button, and then click **AutomationTaskUpdate** to start this schedule. When this schedule is started, the status of the request task can be automatically updated after the OO flow execution.

Related topics

Automated fulfillment of request tasks
Add an Operations Orchestration integration
Add an OO flow link to a request task
Execute an OO flow from a request task

1.7.6.2. Add an OO flow link to a request task

User roles: Request Manager, Request Coordinator

Before you can add an OO flow link to a request task, make sure that you have already enabled the OO flow automation for request tasks. For details, see Enable OO flow automation for request tasks.

You can add OO flow links to the request tasks by planning tasks in request models or directly in request records. Then, the requests will contain the request tasks associated with the OO flows that you defined.

To add an OO flow link to a request task in a request model, follow these steps:



Note

If you want to add an OO flow link directly to a request task from a request record instead of request model, open the task planner from the request record.

1. Sign in to the Service Manager Web client.
2. Click **Request Fulfillment** > **Configuration** > **Request Models**.
3. Search for and open the request model record that you want to modify.
4. Go to the **Task Information** section, and then click **OO Flow Configuration**. The OO Flow Configuration form is displayed.
5. Select an OO flow in the **Select Path** field, and then click **Add Link**. The OO Flow Details form is displayed.
6. Type a name for the OO flow link.
7. Select the **Execute Automatically** checkbox if you want to execute the OO flow automatically. Otherwise, clear the checkbox so that you can manually execute the OO flow by clicking **Execute OO Flow** from the request task records.
8. In the **General Parameters** section and the **Secure Parameters** section, complete the fields listed in the following table for each parameter as necessary.

Field	Value	Description
Parameter Name	Name of an input for the operations of the OO flow; can be entered by the user running the flow, or set to a specific value, or obtained from information gathered by another step of the OO flow	An OO field.
Required?	`true` or `false`	An OO field that indicates whether the parameter is required or not.
Default Value	Default value of the parameter	A custom field. If the Value field is not configured for the parameter, or if no value is available in the Value field, the default value is used when the OO flow is launched. **Note** This field is not the same field in OO.
Value Type	Available Types are: • Additional Properties • Parent File • Current File • User Options • JavaScript Select one from the drop-down list.	A custom field. The value type defines where the OO flow parameter value can be retrieved when the OO flow is launched.

Field	Value	Description
Value	The available fields based on the selected value type. Select one from the drop-down list.	A custom field. The value of the selected field will be used to launch the OO flow. **Note** If Join Table is set to true, this "Value" is used to match the "Join Table Field" to identify the corresponding value for the specified "Select Field", which is then used to launch the OO flow.
Join Table?	true or false	A custom field. If it is set to true, Join Table Name is required.
Join Table Name	All available table names. Select one from the drop-down list.	A custom field. Select the join table name. For example, if the value type is "Parent File" (in request task planner, the Parent File is "request") and the value is "Affected Service", when the Join Table is selected, you may select the Join Table Name as "device".

Field	Value	Description
Join Table Field	Available fields based on the selected Join Table Name. Select one from the drop-down list.	A custom field. The field list of the selected Join Table. The value of this field should match the value of the "Value" field. For example, "CI Identifier" should be selected in Join Table Field if "Affected Service" is selected as the "Value" of the Parent File value type.
Select Field	A field other than the Join Table Field in the Join Table. Select one from the drop-down list.	A custom field. The value of the selected field will be used to launch the OO flow when Join Table is used.

9. In the **Secure Parameters** section, complete the fields for security parameters as necessary.

Field	Value	Description
Field Name	Name of the security parameter	An OO field.
Required?	`true` or `false`	An OO field that indicates whether the parameter is required or not.
Default Value	Default value of the parameter	A custom field. If the Value field is not configured for the parameter, or if no value is available in the Value field, the default value is used when the OO flow is launched. **Note** This field is not the same field in OO.

10. Click **Add**. The OO flow link is added.
11. Click **Save and Exit**, and then go back to the **Task Information** section in the request model.
12. Click **Edit** to open the task planner.
13. Select or create a request task that's in the "Automation" task category.
14. In the **OO Flow Link** drop-down list, select the OO flow link that you added.
15. Click **Save**. The OO flow link is now added to the request task of the request model. The requests that use this request model will contain the request task with the OO flow link that you defined.

Related topics

Automated fulfillment of request tasks
Add an Operations Orchestration integration
Enable OO flow automation for request tasks
Execute an OO flow from a request task

1.7.6.3. Execute an OO flow from a request task

Security roles: Request Coordinator, Request Analyst

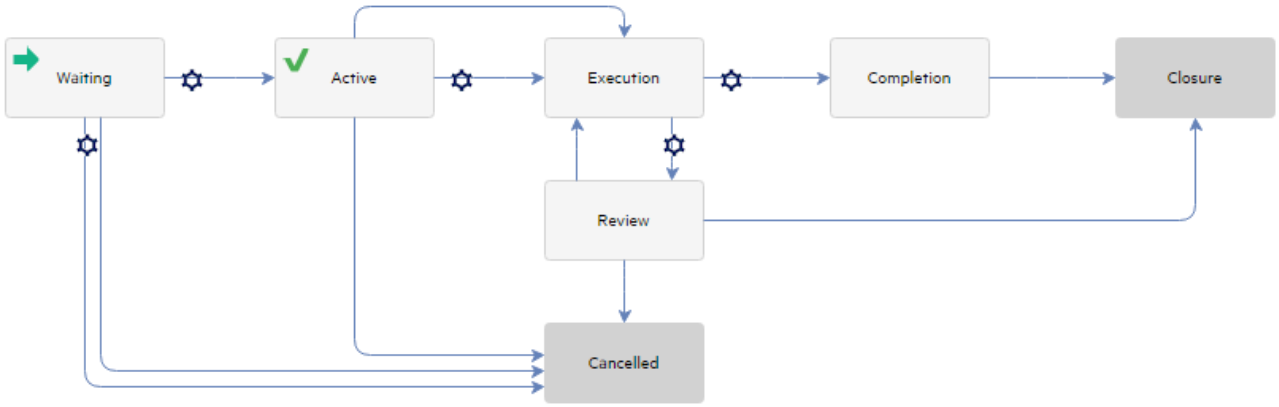
Suppose you have selected the **Execute Automatically** checkbox in the OO flow link configuration. Once a request task linked to that OO flow moves to the Active phase, the associated OO flow will execute automatically. Otherwise, you need to manually execute the OO flow from the request task record.

To manually launch an OO flow from a request record, follow these steps:

1. Sign in to the Service Manager Web client.
2. Click **Request Fulfillment** > **Search Request Tasks**, and then search for and open the request task that requires a manual execution of the OO flow link.
3. From the request task record, click the **Execute OO Flow** button.
4. Verify the OO flow configuration, and then click **OK** to execute the OO flow.

When the OO flow is successfully executed, the request task moves to the **Completion** phase automatically. If additional properties are defined, the request analyst or request coordinator needs to manually fill in the OO execution results in the additional properties. Hence, the remaining tasks can benefit from the output of the current automation task.

If the OO flow execution fails, the request task moves to the **Review** phase automatically. After review, you can launched the OO flow manually again, or you can cancel or close the request task.



To view the execution result of the OO flow, follow these steps:

1. Open a request task record.
2. From the **Activities** tab, you can find the OO execution result shown as activity logs.
3. Click the **Show Report** button if you want to check the execution result in OO.

Related topics

Automated fulfillment of request tasks
Add an Operations Orchestration integration
Enable OO flow automation for request tasks
Add an OO flow link to a request task

1.7.7. Example: launching automated implementation for change and request tasks

This example describes how to launch automated change task implementation and request task fulfillment when Service Manager is integrated with Operations Orchestration (OO). In this example, automated implementation is launched for tasks related to the following two out-of-box service items:

- Restart Tomcat Service
- Password Reset

Note

The Restart Tomcat Service and the Password Reset items used in this example are only provided in Service Manager 9.51 and later versions. Two OO flows are configured for these two items respectively in the out-of-box Service Manager.

Precondition

Before you start, make sure that necessary content packs are deployed in your Operations Orchestration client. If not, follow these steps to deploy the content packs.

1. Sign in to Operations Orchestration as an Administrator.
2. Click **Content Management** on the left-hand panel.
3. Click the **Content Pack** tab.
4. Click the **Add** icon to add the contents.
5. Go to <OO Installation folder>/content, and then select the following content files of which the extensions end with .jar:
   - oo10-base-cp-1.8.0.jar
   - oo10-sm-cp-1.0.3.jar
   - oo-microsoft-office-365-cp-1.0.1.jar
   - oo-saw-cp-2.0.0.jar
   - oo-servicenow-cp-1.0.0.jar
   - oo10-request-fulfillment-cp-2.0.3.jar
6. Click **OK**.
7. Click **Save**.

Add an Operations Orchestration instance in Service Manager

To add an Operations Orchestration instance in Service Manager, see Add an Operations Orchestration integration.

Enable OO flow automation for change tasks

To enable OO flow automation for change tasks, see Enable OO flow automation for change tasks.

Start the AutomationTaskUpdate backend schedule

1. From the System Navigator, click **Miscellaneous** > **System Status** > **Start Scheduler**.
2. Select AutomationTaskUpdate to start the schedule.

When this schedule is started, the status of the change task can be automatically updated after the OO flow execution.

Launch automated task implementation

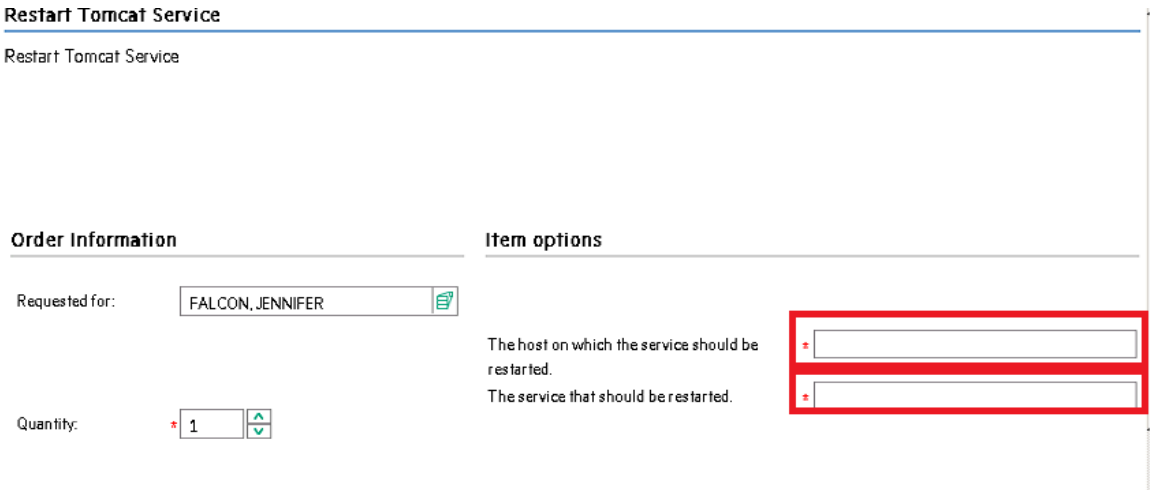
Launch automated implementation for change tasks related to the Restart Tomcat Service item

To launch automated implementation for change tasks related to Restart Tomcat Service, follow these steps:

1. Open the change task for which you want to launch automated implementation.

   If there are no change tasks related to Restart Tomcat Service, order the Restart Tomcat Service item from Service Catalog to create a change record first, then generate a change task from the change record:

   1. From the Service Manager System Navigator, click **Service Catalog** > **Order from Catalog**.
   2. The Service Catalog Entries page opens.
   3. Search for and then open the **Restart Tomcat Service** item.
   4. Specify the fields on the form.



   5. Click **Add to Cart**.
   6. Click **View Cart/Checkout**, and then open the Restart Tomcat Service item.
   7. Specify the fields and then click **Submit**. Remember the ID of the request for later use, for example, SD10219.

8. Sign in to Service Manager as SDApprover using another session. From the System Navigator, click **Service Catalog** > **Search Request** to search for the request created, for example, SD10219.
9. The request should be in the Pending Approval status. Approve the request.
10. Switch back to the session where you submitted the order and wait for a few seconds. Then, there should be a change record in the **Related Records** tab. Click the related record ID, for example, C10061.
11. In the change record form, move the change to **Execution** Phase. Then, click the **Task** tab. A task should be found there.

2. Click the task ID to open the task details page. The phase should be in Execution.
3. Wait for a few minutes for the OO task to complete. The task should go to the Completion phase if service is restarted successfully or the Review phase if not.

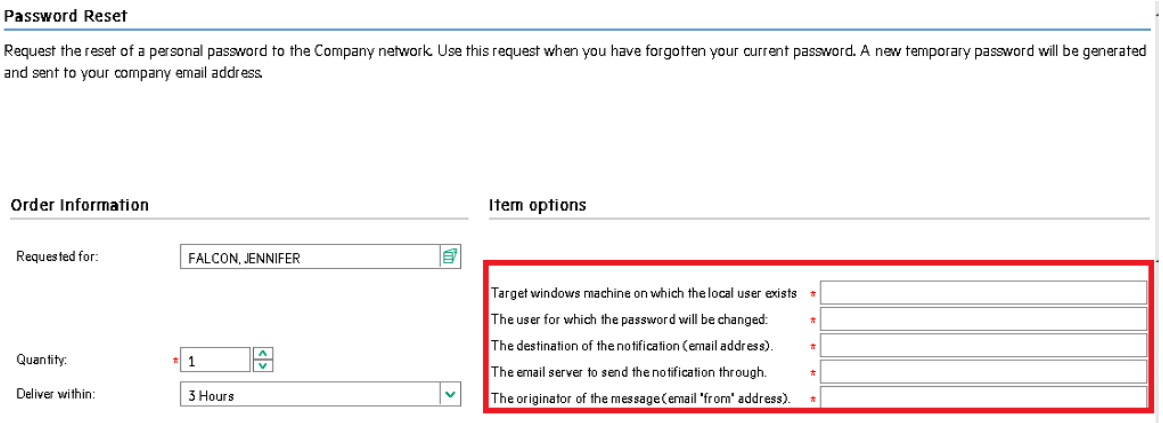
Launch automated implementation for request tasks related to the Password Reset item

To launch automated implementation for request tasks related to Password Reset, follow these steps:

1. Open the request task for which you want to launch automated implementation.

   If there are no request tasks related to Password Reset, order the Password Reset item from Service Catalog to create a request record first, then generate a request task from the request record:

   1. From the Service Manager System Navigator, click **Service Catalog** > **Order from Catalog**.
   2. The Service Catalog Entries page opens.
   3. Search for and then open the **Password Reset** item.
   4. Specify the fields on the form, such as host name, user name, and user mail address.



5. Click **Add to Cart**.
6. Click **View Cart/Checkout**, and then open the **Password Reset** item.

7. Specify the fields and then click **Submit**. Remember the ID of the request for later use, for example, SD10220.
8. Sign in to Service Manager as SDApprover using another session. Click **Service Catalog** > **Search Request** to search for the request created, for example, SD10220.
9. The request should be in Pending Approval status. Approve the request.
10. Switch back to the session where you submitted the order and wait for a few seconds. Then, there should be a request record in the **Related Records** tab. Click the related record ID, for example, RF10016.
11. In the request record form, move the change to Fulfillment Phase. Then, click the **Task** tab. A task should be found there.

2. Click the task ID to open the task details page. The phase should be in Execution.
3. Wait for a few minutes for the OO task to complete. The task should go to the Completion phase if the password is reset successfully or the Review phase if not. An email with new password will be sent to the user once the password is reset.

1.8. Application Performance Management (APM)

Application Performance Management (APM) is an end-to-end management solution portfolio that integrates network, server, application, and business transaction monitoring to help improve IT operations efficiency while delivering high-quality services. APM Operations Manager i (OMi) optimizes event management, identifies cause/symptom relationships between events, and reduces operational expenditures. Refer to the Application Performance Management (APM) documentation for more information.

- Incident Exchange (OMi - SM) integration
- APM Business Impact Report (BIR)
- SM - APM downtime synchronization

Related topics

- Integrations
- Integration Manager
- Incident Exchange (OMi - SM) integration
- Incident Exchange (OMi - SM) integration setup
- APM Business Impact Report (BIR)
- APM Business Impact Report integration setup
- SM - APM downtime synchronization setup
- Add an integration instance for each Operations Manager i (OMi) server
- Enable LW-SSO for the Incident Exchange (OMi - SM) integration
- Add an APM Business Impact Reports integration
- Enable LW-SSO for the Business Impact Report (BIR) integration
- Add an instance in Service Manager Integration Suite (SMIS)
- Tailor Service Manager to handle phase change
- Set up integration in APM
- Set up integration in Universal CMDB

1.8.1. Incident Exchange (OMi - SM) integration

The Incident Exchange (OMi - SM) integration is a bidirectional integration between incident records in Service Manager and events in Application Performance Management (APM) Operations Management provided by the Operations Manager i (OMi) license.

This integration requires configurations on both product sides. For details, see Incident Exchange (OMi - SM) integration setup.



Note

Service Manager can integrate with multiple APM OMi servers. For details, see Add an integration instance for each Operations Manager i (OMi) server.

Service Manager can accept RESTful-based requests from OMi to create incidents in Service Manager, based on events information in OMi. When Service Manager accepts an incident creation request from a remote OMi server, it creates an incident record and automatically assigns it to an existing group based on certain field values. Incident Management users can view the details of the related OMi event by clicking the **View OMi Event** menu option from the incident record. See View related OMi event details from an incident.

When an Incident Management user makes any changes to the incident record, Service Manager automatically synchronizes the changes to the corresponding event in OMi, by sending an update request to the OMi RESTful web service interface. In the event of a synchronization failure, a queuing mechanism will re-synchronize the changes. See Synchronization of incident changes back to Operations Manager i (OMi).

System administrators can configure global settings that determine whether and when incident records opened from OMi events can be automatically closed. However, Incident Management users can mark individual OMi incident records as eligible or ineligible for automatic closure. See Configure automatic closure for OMi incidents and Mark an incident for automatic closure.

Related topics

Application Performance Management (APM)
Incident Exchange (OMi - SM) integration setup
Synchronization of incident changes back to Operations Manager i (OMi)
Add an integration instance for each Operations Manager i (OMi) server
Enable LW-SSO for the Incident Exchange (OMi - SM) integration
Configure automatic closure for OMi incidents
Mark an incident for automatic closure
View related OMi event details from an incident
Change the default assignment group for OMi incidents

1.8.1.1. Incident Exchange (OMi - SM) integration setup

The Incident Exchange (OMi - SM) integration requires the following configuration tasks be completed on the Service Manager and Application Performance Management (APM) systems.

1. Create user accounts for the Incident Exchange (OMi - SM) integration.

   This task creates a user account on each product side for the two systems to connect to each other and to synchronize data.

2. Configure an event forwarding rule in Operations Manager i (OMi) .

   This task configures a rule for the OMi server to forward events to the Service Manager server.

3. Configure the Service Manager server as a connected server in Operations Manager i (OMi).

   Perform this task in APM OMi. If you need to integrate Service Manager with more than one OMi server, perform this task on each of the OMi servers.

4. Enable incident drill-down from Operations Manager i (OMi) Event Browser.

   This task configures the Service Manager web tier in the **sm:ServiceManagerAdapter** script in OMi.

5. Configure SSL for the Incident Exchange (OMi - SM) integration .

   This task is needed if your OMi server requires HTTPS connections. If SSL is not configured in this case, changes on incidents that are created from OMi will not be able to be synchronized back to OMi.

6. Configure the Instance Count in the SMOMi integration template.

   This task is needed only when you have more than one OMi server. The Instance Count setting defines the allowed number of SMOMi integration instances in Service Manager (default: 1).

7. Add an integration instance for each Operations Manager i (OMi) server.

   This task creates and enables an instance of this integration in Integration Manager (SMIS). A separate integration instance is required for each OMi server.

8. Enable LW-SSO for the Incident Exchange (OMi - SM) integration.

   Lightweight Single Sign-On (LW-SSO is) optional but recommended for the Incident Exchange (OMi - SM) integration. This task includes enabling LW-SSO in the Service Manager server and Web tier, as well as in APM.

9. (Optional) Configure automatic closure for OMi incidents.

This task is optional. By default, automatic closure is disabled for OMi incidents. System administrators can enable automatic closure and further configure under what conditions OMi incidents can be automatically closed.

10. (Optional) Change the default assignment group for OMi incidents.

    This task is optional. When created, OMi incidents are automatically assigned with an assignment group based on their certain field values and a predefined default assignment group. System Administrators can change the default assignment group setting (**Application**).

Related topics

Integrations
Integration Manager
Application Performance Management (APM)
APM Business Impact Report (BIR)
Synchronization of incident changes back to Operations Manager i (OMi)
Add an integration instance for each Operations Manager i (OMi) server
Enable LW-SSO for the Incident Exchange (OMi - SM) integration
Configure LW-SSO in the Service Manager Web tier
Edit an integration instance
Enable or disable an integration instance
Configure automatic closure for OMi incidents
Mark an incident for automatic closure
View related OMi event details from an incident
Change the default assignment group for OMi incidents

1.8.1.1.1. Create user accounts for the Incident Exchange (OMi - SM) integration

Applies to User roles: System Administrator

The Incident Exchange (OMi - SM) integration is bidirectional. Synchronizing events and incidents between the Service Manager and APM OMi systems requires you to set up integration accounts for the two systems to access each other.

1. Create an operator record with system administration privileges in Service Manager. For details, see Creating operator records.

   This is the user account that the OMi server uses to access Service Manager and to forward events to Service Manager.

   Later, when configuring the Service Manager server as a connected server in OMi, you need to specify this operator's the login name and password on the **Outgoing Connection** tab. See Configure the Service Manager server as a connected server in Operations Manager i (OMi).

2. Create a user account with system administration privileges on each APM OMi server. For details, see the APM documentation.

   This is the user account that Service Manager uses to access the OMi server and to synchronize incident changes back to the OMi server.

   Later, when configuring the Service Manager server as a connected server in OMi, you must specify this user's login name as the **Name** of the Service Manager server on the **General** tab. Next, specify this user's password as the **Password** on the **Incoming Connection** tab. See Configure the Service Manager server as a connected server in Operations Manager i (OMi).

   Also, you need to specify the same user account when adding an **SMOMi** integration instance for each OMi server (the **username** and **Password** parameters). See Add an integration instance for each Operations Manager i (OMi) server.

Related topics

- Integration Manager
- Incident Exchange (OMi - SM) integration
- Incident Exchange (OMi - SM) integration setup
- Add or delete an integration instance
- Edit an integration instance
- Enable or disable an integration instance
- Enable LW-SSO for the Incident Exchange (OMi - SM) integration

1.8.1.1.2. Configure the Service Manager server as a connected server in OMi

Synchronizing changes between Application Performance Management (APM) Operations Management events and Service Manager incidents requires configuring a Connected Server within OMi to identify the target Service Manager server instance correctly.

To configure the Service Manager server as a target-connected server, perform the following steps:

1. Sign in to Application Performance Management as a system administrator.
2. Navigate to the Connected Servers manager in the Operations Management user interface:

   Admin > **Operations Management** > **Setup** > **Connected Servers**
3. Click the **New** button to open the **Create New Server Connection** dialog box.
4. In the **Display Name** field, enter a name for the Service Manager server.

   The **Name** field is filled automatically. If the auto-completed value is not the user name of the APM user account you created for Service Manager to access the OMi server, change the value to the correct user name. See Create user accounts for the Incident Exchange (OMi - SM) integration.

   Make a note of the Name of the new target server. You need to provide it later on as the **username** when configuring the Service Manager server to communicate with the OMi server. See Add an integration instance for each Operations Manager i (OMi) server.

   Optionally, enter a description for the new target server.

   Make sure that you check the **Active** checkbox.

   Click **Next**.
5. Select **External Event Processing** to choose the server type suitable for an external incident manager like Service Manager.

   Click **Next**.
6. Enter the **Fully Qualified DNS Name** of the target Service Manager server.

   Click **Next**.
7. In the **Integration Type** dialog box, you can choose between using a Groovy script adapter, or the Event Synchronization Web Service.

   1. As a Service Manager Groovy script adapter is provided for integrating with Service Manager, select **Call Script Adapter**.

2. In the **Script Name** field, select **sm:ServiceManagerAdapter**.
3. Click **Next**.

8. In the **Outgoing Connection** dialog, enter the credentials (user name, password, and port number) to connect to the Service Manager server and forward events to that server.
   1. In the **User Name** and **Password** fields, enter the Service Manager user credentials you created for the integration. See Create user accounts for the Incident Exchange (OMi - SM) integration.
   2. Repeat the password entry in the **Password (Repeat)** field.
   3. In the **Port** field, enter the communications port of the Service Manager server.

      The Service Manager server configuration file (sm.ini) defines the HTTP and HTTPS ports. Enter the HTTP port when Service Manager is running in HTTP mode, or enter the HTTPS port when it is running in secure HTTP mode.

      Tip

      Clicking **Set default port** automatically populates the **Port** field with the default port (**13080** for http or **13443** for secure http). However, your actual Service Manager ports may differ from the default values.

   4. Select the Use Secure HTTP checkbox if the Service Manager server uses secure HTTP (SSL) mode. If it uses HTTP mode, make sure the checkbox is not checked.
   5. If the **Use Secure HTTP** checkbox is selected, download and install a copy of the Service Manager server's SSL certificate by clicking the **Retrieve from Server** link, or **Import from File** if the certificate is available in a local file.
   6. Make sure that the **Enable Synchronize and Transfer Control** checkbox is checked.

      When the **Enable Synchronize and Transfer Control** flag is set, an Operations Management operator is then able to transfer ownership of the event to the target connected server. If the flag is not set, then the option **Synchronize and Transfer Control** doesn't appear in the list of forwarding types when configuring forwarding rules.

      Also, note that if the **Enable Synchronize and Transfer Control** flag is not set for any target-connected server, the **Transfer Control to** option doesn't appear at all in the Event Browser context menu.

      If a specific server is configured without the **Enable Synchronize and Transfer Control** flag set, then that server is unavailable in the Event Browser context menu as a server to which you can transfer ownership.

   7. Click **Test Connection**. A **Success** or **ERROR** hyperlink appears.

      Click the link to get a more detailed message.

8. Click **Next**.

9. If, in addition to automatically generating Service Manager incidents from OMi events, you want to also be able to drill-down into Service Manager, you need to specify the fully qualified DNS name and port of the Service Manager web application server (for example, Tomcat).

   > **Note**
   >
   > To enable incident drill-down to Service Manager, you must have the Service Manager web tier deployed on a web application server.

   In the **Event Drilldown** dialog box, configure the **Fully Qualified DNS Name** and **Port** of the web application server where the Service Manager web tier is deployed.

   > **Note**
   >
   > If you don't specify a server in the **Event Drilldown** dialog box, it is assumed that the web tier is deployed on the same machine as the Service Manager server.

   Click **Next**.

10. The next thing to do is to enable event changes to be synchronized back from Service Manager to OMi. For this, you need to provide credentials for the Service Manager server to access the OMi server in the **Incoming Connection** dialog box.

    1. Select the **Accept event changes from external processing server** checkbox.

       > **Note**
       >
       > If **Enable Synchronize and Transfer Control** was previously selected, the **Accept event changes from external processing server** option is assumed, and can't be disabled.

    2. Enter the password of the user account that you created for the Service Manager server to access the OMi server. See Create user accounts for the Incident Exchange (OMi - SM) integration.

    3. Make sure the **User Name** (auto-generated) matches the one of the user account you created. If not, click the **General** tab, and change the **Name** field to the correct value.

    4. Click **Finish**. The target Service Manager server appears in the list of Connected Servers.

Related topics

Integration Manager
Incident Exchange (OMi - SM) integration
Incident Exchange (OMi - SM) integration setup
Add or delete an integration instance
Edit an integration instance
Enable or disable an integration instance
Enable LW-SSO for the Incident Exchange (OMi - SM) integration

1.8.1.1.3. Configure an event forwarding rule in OMi

Suppose you have configured the Service Manager server as a connected server in Application Performance Management (APM) Operations Management i (OMi). You need to configure an event forwarding rule for the OMi server to forward events to Service Manager.

1. Sign in to APM as a system administrator.
2. Navigate to **Admin** > **Operations Management** > **Event Automation**.
3. Click the **New Item** button.
4. In the **Display Name** field, enter a name for the rule. For example, `smserver1`.
5. In the **Event Filter** field, select **Critical**.
6. In the **Target Servers** field, select the Service Manager server you configured as a connected server, and then click the **Add** button.

   The system displays details of the target server.
7. In the **Forwarding Type** field, select **Synchronize and Transfer Control**.
8. Click **OK**.

   The **Event Forwarding Rules** section displays the forwarding rule.

Related topics

- Integration Manager
- Incident Exchange (OMi - SM) integration
- Incident Exchange (OMi - SM) integration setup
- Add or delete an integration instance
- Edit an integration instance
- Enable or disable an integration instance
- Enable LW-SSO for the Incident Exchange (OMi - SM) integration

1.8.1.1.4. Enable incident drill-down from OMi Event Browser

Suppose you have configured the Service Manager server as a connected server in Application Performance Management (APM) Operations Management i (OMi). If you want to be able to drill down to Service Manager incidents from the OMi Event Browser, you need to configure the Service Manager web tier in the **sm:ServiceManagerAdapter** script in OMi.

To configure the Service Manager web tier name in the **sm:ServiceManagerAdapter** script in OMi, follow these steps:

1. Sign in to APM as a system administrator.
2. Navigate to **Admin** > **Operations Management** > **Setup** > **Connected Servers**.
3. Click the **Manage Scripts** icon in the toolbar.
4. Click the **sm:ServiceManagerAdapter** field, and then click the **Edit** icon.
5. On the **Script** tab, locate the following string in the script:

   ```
   private static final String SM_WEB_TIER_NAME
   ```

6. Change the value of the parameter to the name of your Service Manager web tier. For example, if your web tier file name is **sm-9.xx.war**, change the parameter value to:

   ```
   private static final String SM_WEB_TIER_NAME=sm-9.xx
   ```

7. Click **OK** to save the script.

Related topics

- Integration Manager
- Incident Exchange (OMi - SM) integration
- Incident Exchange (OMi - SM) integration setup
- Add or delete an integration instance
- Edit an integration instance
- Enable or disable an integration instance
- Enable LW-SSO for the Incident Exchange (OMi - SM) integration

1.8.1.1.5. Configure SSL for the Incident Exchange (OMi - SM) integration

Applies to User roles: System Administrator

If you have configured Application Performance Management (APM) to accept HTTPS connections only, you must configure SSL for the integration. If you don't do so, the integration can't synchronize changes on an incident that's created from OMi back to APM/OMi.

> **Note**
>
> The following steps describe how you do so by using the built-in key tool in Service Manager, and the file paths are for Windows only. Be sure to change the file paths if your Service Manager system is running on Linux.

To configure SSL for the integration, follow these steps:

1. Import the APM root certificate to the Service Manager server trusted keystore.

   The following is an example of the command line:

   `<SM Install path>\server\RUN\jre\bin\keytool -import -alias myCA -file <.pem file of your APM root certificate> -keystore <SM Install path>\Server\RUN\jre\lib\security\cacerts -storepass <changeit>`

   Where: `changeit` is the default password of the trusted keystore. Change it to your own password if you have changed it.

2. Add the following parameters to the Service Manager server configuration file (`<SM install path>\Server\RUN\sm.ini`):

   `truststoreFile:<SM install path>\Server\RUN\jre\lib\security\cacerts`

   `truststorePass:<changeit>`

3. Restart the Service Manager Server service.

Related topics

- Application Performance Management (APM)
- SM - APM downtime synchronization
- Tailor Service Manager to handle phase change
- Set up integration in APM
- Set up integration in Universal CMDB
- Verify the SM - APM downtime synchronization setup

1.8.1.1.6. Configure the Instance Count in the SMOMi integration template

Applies to User roles: System Administrator

Service Manager can integrate with more than one APM OMi server. However, the integration allows only one OMi server by default. If you need to integrate Service Manager with more than one OMi server, you need to configure the Instance Count setting in the **SMOMi** integration template, as described below.

1. Sign in to Service Manager as a system administrator.
2. Type db in the command line and press Enter.
3. In the **Table** field, type `SMISRegistry`, and click **Search**.

   The Service Manager Integration Suite (SMIS) integration template form opens.
4. Click **Search**.

   A list of SMIS integration templates opens.
5. Select **SMOMi** from the list.
6. In the **Instance Count** field, change the value of 1 to the number of OMi servers that you want to integrate with Service Manager. For example, if you need two OMi servers, change the value to 2.
7. Click **Save**.

Related topics

Application Performance Management (APM)
Incident Exchange (OMi - SM) integration
Configure automatic closure for OMi incidents
Mark an incident for automatic closure
Change the default assignment group for OMi incidents

1.8.1.1.7. Add an integration instance for each OMi server

Applies to User roles: System Administrator

Once you have completed your configuration in Application Performance Management Operations Management, and have changed the Instance Count setting in the SMOMi integration template (which is needed only when you have multiple OMi servers), you are ready to add and enable a separate integration instance in Service Manager for each OMi server. For example, if you have two OMi servers, you must configure two SMOMi integration instances.

Support of multiple OMi servers

As of version 9.34, Service Manager can integrate with multiple OMi servers. This is implemented through the Instance Count setting and the **omi.mgr.id** parameter in the SMOMi integration template.

By default, the SMOMi integration template supports only one integration instance. If you have multiple OMi servers, before you proceed, make sure you have already updated the Instance Count setting in the SMOMi integration template. For details, see Configure the Instance Count in the SMOMi integration template.

When using the **omi.mgr.id** parameter, keep the following in mind:

- If you have only one OMi server (and hence need only one SMOMi integration instance), you must either correctly configure this parameter or clear the entire row of this parameter (both the parameter name and value) in the SMOMi integration instance, otherwise the integration will not work.
- If you have multiple OMi servers (and hence need multiple SMOMi integration instances), you must correctly configure this parameter in all SMOMi integration instances. Only those correctly configured integration instances will work. If none of the SMOMi integration instances are correctly configured, none of them will work.
- Users can view the OMi event details from an OMi incident record only when you specify the **omi.mgr.id** parameter correctly. If the value you specify in the corresponding SMOMi integration instance doesn't match the Universally Unique Identifier (UUID) which is automatically generated in the OMi server for the target Service Manager server and stored in the Incident record, users will not see the **View OMi Event** option from the Incident record.

To add and enable an Incident Exchange (OMi-SM) integration instance:

1. Sign in to Service Manager as a system administrator.
2. Click **Tailoring** > **Integration Manager**.
3. Click **Add**.

The Integration Template Selection wizard opens.

4. Select **SMOMi** from the Integration Template list.

> **Note**
>
> Ignore the **Import Mapping** checkbox, which has no effect on this integration.

5. Click **Next**.

6. Complete the integration instance information:

   • Modify the **Name** and **Version** fields to the exact values you need.

   • In the **Interval Time (s)** field, enter a value. For example, 600. If an OMi opened incident fails to be synchronized back to OMi, Service Manager will retry the failed task at the specified interval (for example, 600 seconds).

   • In the **Max Retry Times** field, enter a value. For example, 10. This is the maximum allowed number of retries for each failed task.

   • (Optional) In the **SM Server** field, specify a display name for the Service Manager server host. For example, my_Local_SM.

   • (Optional) In the **Endpoint Server** field, specify a display name for the APM server host. For example, my_APM_1.

   • (Optional) In the **Log File Directory** field, specify a directory where log files of the integration will be stored. This must be a directory that already exists on the Service Manager server host.

   • (Optional) In the **Log Level** field, change the log level from INFO (default) to another level. For example, **WARNING**.

   • (Optional) If you want this integration instance to be automatically enabled when the Service Manager Server service is started, select **Run at system startup**.

7. Click **Next**. The Integration Instance Parameters page opens.

8. On the **General Parameters** tab, complete the following fields as necessary:

Field	Sample Value	Description
omi.server.url	http://<servername>:opr-gateway/rest/synchronization/event	This is the URL address of the OMi server's RESTful web service. Replace <servername> with the fully qualified domain name of your OMi server.
http.conn.timeout	30	The HTTP connection timeout setting in seconds. **Note** The out-of-box value is 30 (seconds), and 15 (seconds) is used if this field is empty.
http.rec.timeout	30	The HTTP receive timeout setting in seconds. **Note** The out-of-box value is 30 (seconds), and 15 (seconds) is used if this field is empty.

Field	Sample Value	Description
http.send.timeout	30	The HTTP send timeout setting in seconds. **Note** The out-of-box value is 30 (seconds), and 15 (seconds) is used if this field is empty.
sm.mgr.id	55436DBE-F81E-4799-BA05-65DE9404343B	The Universally Unique Identifier (UUID) automatically generated for this instance of Service Manager. **Note** This field is automatically completed each time when you add an SMOMi integration instance. Do not change it, otherwise the integration will not work properly.

Field	Sample Value	Description
omi.reference.prefix	urn:x-hp:2009:opr:	The prefix of the BDM External Process Reference field, which will be present in incoming synchronization requests from the OMi server. **Note** This field is automatically completed and has a fixed value. Do not change it.
sm.reference.prefix	urn:x-hp:2009:sm:	The prefix of the BDM External Process Reference field, which will be present in outgoing synchronization requests from Service Manager. **Note** This field is automatically completed and has a fixed value. Do not change it.

Field	Sample Value	Description
omi.eventdetail.baseurl	http://<servername>/opr-console/opr-evt-details.jsp?eventId=	The basic URL address of the event detail page in OMi. Replace <servername> with the fully qualified domain name of your OMi server.

9. On the **General Parameters** and **Secure Parameters** tabs, enter three parameter values that you specified when configuring the Service Manager server as a connected server in APM OMi. The following table lists the parameters, whose values you can copy from your APM OMi server.

   To copy the parameter values from APM OMi, follow these steps:

   a. Sign in to APM as a system administrator.

   b. Navigate to **Admin** > **Operations Management** > **Setup** > **Connected Servers**.

   c. Locate your Service Manager server configuration entry and double-click anywhere on the entry pane.

   d. On the **General** tab, copy the **ID** string at the bottom into the **omi.mgr.id** field in Service Manager.

   e. On the **Incoming Connection** tab, copy the **User Name** and **Password** to the **username** and **Password** fields in Service Manager, respectively.

Field	Sample Value	Description
omi.mgr.id (on the **General Parameters** tab)	f3832ff4-a6b9-4228-9fed-b79105afa3e4	The Universally Unique Identifier (UUID) automatically generated in OMi for the target Service Manager server.

Field	Sample Value	Description
		Note This parameter was introduced to support multiple OMi servers. Service Manager uses the UUID to identify from which OMi server an incident was opened. Be aware that if you delete the connected server configuration for the Service Manager server in OMi and then recreate the same configuration, OMi generates a new UUID. You need to reconfigure the integration instance by changing the old UUID to the new one.

Field	Sample Value	Description
		Tip If you have only one OMi server, you can simply remove this parameter (remove both the parameter name and value) from the integration instance. See the *Support of multiple OMi servers* section above.
username omi.mgr.id (on the **General Parameters** tab)	SM_Server	This is the user name that the Service Manager server uses to synchronize incident changes back to the OMi server.
Password (on the **Secure Parameters** tab)	SM_Server_Password	This is the password that the Service Manager server uses to synchronize incident changes back to the OMi server.

10. Click **Next** twice, and then click **Finish**.

Note

Leave the Integration Instance Mapping and Integration Instance Fields settings blank. This integration doesn't use these settings.

Service Manager creates the instance. You can edit, enable, disable, or delete it in Integration Manager.

11. Enable the integration instance.
12. If you have multiple OMi servers, repeat the steps above for the rest of your OMi servers.

Next, you can optionally enable Lightweight Single Sign-On (LW-SSO) in both APM and Service Manager so that users can bypass the log-in prompts. For details, see Enable LW-SSO for the Incident Exchange (OMi - SM) integration.

Related topics

Integration Manager
Incident Exchange (OMi - SM) integration
Incident Exchange (OMi - SM) integration setup
Add or delete an integration instance
Edit an integration instance
Enable or disable an integration instance
Enable LW-SSO for the Incident Exchange (OMi - SM) integration

1.8.1.1.8. Enable LW-SSO for the Incident Exchange (OMi - SM) integration

Applies to User roles: System Administrator

Lightweight Single Sign-On (LW-SSO) is optional but recommended for the Incident Exchange (OMi - SM) integration. You have different LW-SSO configuration choices depending on your needs. The following describes how LW-SSO can be used in the Incident Exchange (OMi - SM) integration workflow.

When OMi creates an incident from an OMi event record

OMi creates an incident from an OMi event record by sending RESTful-based requests to Service Manager. The incident ID is then stored in the event record.

LW-SSO is NOT needed in this process. A dedicated Service Manager user account was specified when configuring the Service Manager integration in OMi. OMi uses this dedicated user account when calling the Service Manager RESTful Web Service to create the incident.

When an OMi user views the incident details

The user can log in to Service Manager and view the incident details using the incident ID stored in the event record. For more information, refer to the APM documentation.

If the user wants to view the incident details by clicking the incident link from the event record, LW-SSO can be used; otherwise a Service Manager login prompt will appear.

LW-SSO is optional for this process. To enable LW-SSO for this process, configure LW-SSO in both the Service Manager server and Web tier (because the server needs to trust the Web tier), as well as in APM.

- Configure LW-SSO in the Service Manager server
- Configure LW-SSO in the Service Manager Web tier
- Configure LW-SSO in Application Performance Management (APM)

When Service Manager synchronizes the OMi incident status back to OMi

When a user has updated the OMi incident, Service Manager calls the OMi server's RESTful Web Service to update the incident changes to the OMi event record.

LW-SSO is NOT needed in this process. A dedicated OMi user account was specified when the Incident Exchange (OMi - SM) integration was set up in SMIS, and Service Manager uses this user account when calling the OMi server's RESTful Web Service to synchronize the incident status back to the OMi event record.

When a user views the event details from the OMi incident

The user clicks the **View OMi Event** option from the incident to view the event details.

LW-SSO is optional for this process. If you enable LW-SSO in the Service Manager Web tier and in APM, the APM login prompt is bypassed.

- Configure LW-SSO in the Service Manager Web tier
- Configure LW-SSO in Application Performance Management (APM)

Related topics

Application Performance Management (APM)
Incident Exchange (OMi - SM) integration
Incident Exchange (OMi - SM) integration setup
Using LW-SSO with integrations
Add an integration instance for each Operations Manager i (OMi) server
View related OMi event details from an incident

1.8.1.1.9. Configure automatic closure for OMi incidents

Applies to User Roles: System Administrator

OMi incidents can be automatically closed after a predefined amount of time since they were last updated (or resolved if they have not been updated after being resolved).

The workflow is as follows:

1. An incident is opened from OMi.
2. If the **Schedule Condition** is met, the system creates a schedule record for the incident. The schedule record will expire at a future time based on the **Calc Expression**.
3. A user updates the incident and saves the changes.
4. The **Reset alerts if** expression on the **Alerts** tab of the **probsummary** object definition is evaluated. If it evaluates to true, the Expiration time of the schedule record is updated based on the Calc Expression. By default, the expiration time of the schedule record is updated only when the incident has a category of **incident**.
5. When the schedule record expires, the **Alert Condition** is evaluated. If it evaluates to true, the incident is automatically closed.

Complete the following tasks o enable automatic closure for OMi incidents.

1. Configure the global settings in the Incident Management Environment record
2. Configure the alert definition that determines when an incident should be closed
3. Configure alert information in the **probsummary** object

Configure the global settings in the Incident Management Environment record

1. Click **System Administration** > **Ongoing Maintenance** > **Environment Records** > **Incident Management Environment**.
2. Change the following settings as necessary.

Field	Value
Close Incident Automatically?	This option disables or enables the automatic closure of OMi incidents at the global level. • If this option is not selected, no incidents will be automatically closed. • If this option is selected, incidents will be automatically closed under specified conditions. Default: Not selected
Closure Code	This value will be copied to the **Closure Code** field of incidents when they are automatically closed. Default: Automatically Closed
Solution	This description will be appended to the end of the **Solution** field of incidents when they are automatically closed. Default: This incident which belongs to OMi has been closed automatically.

3. Click **Save**.
4. Restart the Service Manager server.



Note

If you have made any changes to any of the configuration options in the Incident Management Environment record, the Service Manager server must be restarted for the changes to take effect.

Configure the alert definition that determines when an incident should be closed

Note

The **alert** and **problem** processes must be running to enable the successful closure of OMi incidents.

1. Click **Tailoring** > **Document Engine** > **Alerts**.
2. In the Alert Name field, enter: **OMI Auto-Close**.
3. Click **Search**. The OMI Auto-Close alert definition detail form opens.

Caution

These fields in the alert definition are used to control automatic closure of OMi incidents. You can change the default values of these fields. However, you must be aware of the risk that automatic closure of OMi incidents will not work properly if the **Schedule Condition** and **Alert Condition** fields aren't configured correctly.

Field	Value
Schedule Condition	This expression is used to determine if an incident should be scheduled for automatic closure. Default: `jscall("SMOMi.isAutoCloseAndResolved")`. An incident is scheduled for automatic closure when the following conditions are met. • The **Close Incident Automatically?** option in the Incident Management Environment record is selected. • In the incident record, the **Do not close this incident automatically** option is not selected. • The incident has a status of **Resolved**.
Alert Condition	This expression is evaluated when an incident is about to be automatically closed. If it evaluates to true, the incident is closed. Default: `jscall("SMOMi.isAutoCloseEnabled")`. An incident is closed when the following conditions are met. • The **Close Incident Automatically?** option in the Incident Management Environment record is selected. • In the incident record, the **Do not close this incident automatically** option is not selected.

Field	Value
Calc Expression	This expression is used to determine how much time will elapse before an incident is automatically closed. Default: `$L.alert.time=update.time in $L.file+'7 00:00:00'` . The default value means the amount of time elapsed is equal to seven days since the incident was last updated.

Configure alert information in the probsummary object

The OMi autoclose alert definition is configured to only be used by OMi incidents. The closure time is reset each time the incident is updated. If the closure time is reached without the incident being updated then Service Manager will automatically close the incident.

1. Click **Tailoring** > **Document Engine** > **Objects**.
2. In the **File name** field, enter **probsummary** and press ENTER. The **probsummary** object definition is displayed.
3. Select the **Alerts** tab.

   The **Reset alerts if** expression is used to reset the automatic closure time of OMi incidents.

   Default: `category in $L.file="incident" and not null(1 in external.process.reference in $L.file)` .

Related topics

Application Performance Management (APM)
Incident Exchange (OMi - SM) integration
Mark an incident for automatic closure
View related OMi event details from an incident
Change the default assignment group for OMi incidents

1.8.1.1.10. Change the default assignment group for OMi incidents

Applies to User Roles: System Administrator

Service Manager can accept REST based requests from APM Operations Manager i (OMi) to create incidents based on events information in OMi. An incident opened from an OMi event is automatically assigned to an existing group based on the following field values, listed from the highest to lowest priority:

- The **Affected Service** of the incident
- The **Category** of the incident
- The **Affected CI** of the incident

However, if none of these field values is available, the system assigns the incident to a default group named **Application**. If necessary, you can use BTO Data Model (BDM) Mapping Management to change this default group setting as follows:

1. Navigate to **System Administration** > **Ongoing Maintenance** > **BDM Mapping Management**. The BDM mapping configuration search page opens.
2. Enter **incident** in the BDM Name field, select **1.1** in the **Version** field, and then click **Search**. The BDM mapping record **incident** opens. The Incident Exchange (OMi - SM) integration uses this BDM mapping record when creating an incident from an OMi event.
3. Select the **Field Mapping** tab, scroll down to the **assignment** field in the SM Object Field column, and click the **SM Callback** field in the same row.
4. Change **Application** in the following code to the name of another assignment group:

```
4)  A default assignment group if no other criteria is met
        if( ! $result ) {
                $result = "Application";
        }
```

5. Click **Save**. The default assignment group is now changed.

Related topics

Application Performance Management (APM)
Incident Exchange (OMi - SM) integration
BDM Mapping Management

1.8.1.2. Synchronization of incident changes back to OMi

After OMi opens an incident in Service Manager, Service Manager will synchronize the incident changes back to OMi.

Operations Manager i (OMi) can forward an event record to Service Manager as an incident by calling a Service Manager Web Service. The incident ID is then stored in the event record.

When a user has updated the incident opened from OMi, Service Manager calls an OMi server RESTful Web Service to update the incident changes to the OMi event record.

If Service Manager fails to synchronize the incident changes back to OMi for some reasons (for example, because of a network problem), Service Manager behaves as follows:

- Displays a warning message, indicating that the incident failed to be synchronized to OMi.
- Saves the failed task in the SMIS task queue, and retries the task to re-synchronize the changes to OMi based on an interval time and a maximum retry times configured when adding the Incident Exchange (OMi - SM) integration in SMIS. When the re-synchronization is successfully completed, the failed task is removed from the task queue.

System Administrators can monitor failed tasks, and reset their retry times or rerun expired tasks. For more information, see Monitor failover tasks.

Related topics

Integration Manager
Incident Exchange (OMi - SM) integration
Incident Exchange (OMi - SM) integration setup
Add an integration instance for each Operations Manager i (OMi) server
Enable LW-SSO for the Incident Exchange (OMi - SM) integration
View related OMi event details from an incident
Monitor failover tasks

1.8.1.3. Working with the Incident Exchange (OMi - SM) integration

Once the integration is set up, Service Manager Incident Management users with the right permissions can view event details from OMi incidents and mark individual OMi incidents as eligible or ineligible for automatic closure (if their system administrator has enabled automatic closure for OMi incidents).

Related topics

Integration Manager
Incident Exchange (OMi - SM) integration
Incident Exchange (OMi - SM) integration setup
Add an integration instance for each Operations Manager i (OMi) server
Enable LW-SSO for the Incident Exchange (OMi - SM) integration
View related OMi event details from an incident
Monitor failover tasks

1.8.1.3.1. View related OMi event details from an incident

If all of the following conditions are met, you can view the related Operations Manager i (OMi) event details from an OMi incident:

- You are accessing the incident through the Service Manager standard Web client (not the employee self-service (ESS) interface).
- One or more SMOMi integration instances are set up and enabled in Integration Manager.
- You are also a Application Performance Management (APM) user who has been granted the permission **Events assigned to user** including the required actions.

To view related OMi event details from an OMi incident:

1. Sign in to the Service Manager Web client.
2. From **Incident Management**, search for the incident record created from OMi.
3. Click **More** and then select **View OMi Event**.

   Note

   The **View OMi Event** option displays only when the **omi.mgr.id** parameter in the corresponding SMOMi integration instance is set correctly.

   If Lightweight Single Sign-On (LW-SSO) is enabled in both the Service Manager Web client and Application Performance Management (APM), the OMi event detail page opens in a new browser window, displaying the details of the related event in OMi. If LW-SSO is disabled, a APM login page opens, and the related OMi event detail page displays after you sign in to APM.

Related topics

Application Performance Management (APM)
Incident Exchange (OMi - SM) integration
Configure automatic closure for OMi incidents
Mark an incident for automatic closure
Change the default assignment group for OMi incidents

1.8.1.3.2. Mark an incident for automatic closure

Applies to User Roles: Incident Coordinator

You can mark an OMi incident as eligible or ineligible for automatic closure after a predefined amount of time since it was last updated.

To mark an incident as ineligible or eligible for automatic closure:

1. From **Incident Management**, search for an incident record opened from OMi.
2. Select or deselect the **Do not close this incident automatically** checkbox to mark this incident as ineligible or eligible for automatic closure.

> **Note**
>
> By default, this checkbox isn't selected.

Related topics

- Application Performance Management (APM)
- Incident Exchange (OMi - SM) integration
- Configure automatic closure for OMi incidents
- View related OMi event details from an incident
- Change the default assignment group for OMi incidents

1.8.2. APM Business Impact Report (BIR)

Application Performance Management (APM) includes a report that you can use to help evaluate the impact of incidents on your business. A Business Impact report displays information about how a CI impacts the Business Services it belongs to. Data about the affected Business Service, Application, and Business Process CIs includes KPI data, over-time data, and SLA data. For example, if a host CI has critical status, you can use the report to display the status of the Business Service CIs to which the host CI is attached.

Incident Management users can launch an impact report from an incident in context with the incident's affected configuration item (CI). Service Desk Agents can validate the updated status of the business impact to categorize and prioritize the incident accordingly.

This integration also supports multi-tenancy. System Administrators can enable multi-tenancy for the integration.



Note

- This integration requires that both Service Manager and APM be integrated with Universal CMDB (UCMDB) so that configuration items (CIs) can be synchronized between the servers. For details about how to configure an integration from Service Manager or APM to UCMDB, see Universal CMDB (UCMDB) and refer to the Application Performance Management (APM) documentation.
- Only one instance of this integration is allowed.

Prerequisites:

- Before end-users can use this integration, System Administrators must add and enable an instance of this integration in Integration Manager (SMIS).
- Optionally, System Administrators can use Lightweight Single Sign-On (LW-SSO) to bypass the log-in prompts.

Related topics

Integration Manager
Application Performance Management (APM)
APM Business Impact Report integration setup
Add a APM Business Impact Reports integration
Enable LW-SSO for the Business Impact Report (BIR) integration
Enable multi-tenancy for the APM Business Impact Report integration
Launch a Business Impact Report from an incident

1.8.2.1. APM Business Impact Report integration setup

To set up the APM Business Impact Report (BIR) integration in your environment, you need to complete the following tasks:

1. Add a APM Business Impact Reports integration

   This task creates and enables an instance of this integration in the Integration Manager (SMIS).

2. Enable LW-SSO for the Business Impact Report (BIR) integration

   LW-SSO is optional but highly recommended for this integration. This task includes enabling LW-SSO in both the Service Manager Web tier and APM. Enabling LW-SSO for the integration enables Incident Management users to bypass the login prompts when launching a APM business impact report from an incident.

3. Enable multi-tenancy for the APM Business Impact Report integration

   Perform this task only when your Service Manager environment is running in **multi-company mode**.

Related topics

Application Performance Management (APM)
APM Business Impact Report (BIR)
Launch a Business Impact Report from an incident

1.8.2.1.1. Add a Business Impact Report integration

Applies to User Roles: System Administrator

To use the Service Manager to BIR integration, you must add and enable an instance of this integration in Integration Manager. Optionally, you can enable Lightweight Single Sign-On (LW-SSO) in APM and in Service Manager to bypass the log-in prompts.

To add and enable a Service Manager to BIR integration instance:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **SMBIR** from the Integration Template list. Ignore the **Import Mapping** checkbox, which has no effect on this integration.

   Note

   Only one instance of this integration is allowed. If an instance of this integration already exists in Integration Manager, the **SMBIR** template is unavailable. You have to delete the existing integration instance before you can add a new one.

4. Click **Next**. The Integration Instance Information page opens.
5. Update the following fields:

   Note

   Only **Name** and **Version** are required fields. This integration doesn't use the **Interval Time (s)** and **Max Retry Times** fields as it is UI-based.

Field	Value
Name	(Required) The name of the integration instance. Default: `SMBIR`
Version	(Required) The version of the integration template. Default: `1.0`
SM Server	The name of the Service Manager server machine.
Endpoint Server	The name of the APM Server machine.
Log Level	Select one from: DEBUG, INFO (default), WARNING, ERROR, and OFF.
Log File Directory	A directory on the Service Manager Server machine in which log files will be stored.
Description	If you want, modify the default description of the instance.
Run at system startup	Select this checkbox only if you want this instance to be automatically enabled when the Service Manager Server is started.

6. Click **Next**. The Integration Instance Parameters page opens.
7. On the **General Parameters** tab, replace "APM_host" in the **baseurl** parameter with the hostname of the real APM server.
8. Click **Next** twice and then click **Finish**. Leave the Integration Instance Mapping and Integration Instance Fields settings blank. This integration doesn't use these settings.

   Service Manager creates the instance. You can edit, enable, disable, or delete it in Integration Manager.
9. Enable the integration instance.
10. (Optional) Configure LW-SSO in APM and then configure LW-SSO in Service Manager

Related topics

Integration Manager
Application Performance Management (APM)
APM Business Impact Report (BIR)
Incident Exchange (OMi - SM) integration
Enable LW-SSO for the Incident Exchange (OMi - SM) integration
Configure LW-SSO in the Service Manager Web tier
Edit an integration instance
Enable or disable an integration instance
Launch a Business Impact Report from an incident

1.8.2.1.2. Enable LW-SSO for the Business Impact Report (BIR) integration

If you have enabled LW-SSO for the Business Impact Report (BIR) integration, you can launch a business impact report in APM from a Service Manager incident.

To enable LW-SSO for the BIR integration, complete the following tasks:

- Configure LW-SSO in the Service Manager Web tier
- Configure LW-SSO in Application Performance Management (APM)

Related topics

Application Performance Management (APM)
APM Business Impact Report (BIR)
Launch a Business Impact Report from an incident

1.8.2.1.3. Enable multi-tenancy for the APM Business Impact Report integration

Applies to User roles: System Administrator

The APM Business Impact Report integration provides multi-tenancy support. It allows an Incident Management user to launch APM Business Impact Report (BIR) based on the UCMDB Customer ID of the user's company.

To enable multi-tenancy for this integration, you need to complete the following tasks:

1. Enable multi-tenancy in Service Manager.
2. Synchronize companies between Service Manager and UCMDB.
3. Synchronize companies between APM and UCMDB.

Suppose you have enabled multi-tenancy for the integration and a user launches a Business Impact Report from an incident. Based on the Customer ID of the company record retrieved from the user's contact information, the system displays BIR in the incident's related Business Service or Affected CI.



Note

Note: You can click the Find icon for a user's **Contact ID** field to find information about the company associated to the user.

To enable multi-tenancy in Service Manager:

1. Sign in to Service Manager.
2. Go to **System Administration** >**Base System Configuration** > **Miscellaneous** > **System Information Record**.
3. In the **General** tab, select **Run in Multi-Company Mode**.
4. In the **Active Integrations** tab, do the following:
    - Select **Universal CMDB**.
    - In the **Multi-tenant UCMDB webservice URL** field, type the URL to the multi-tenant UCMDB Web Service API. The URL has the following format:

      `<UCMDB server name>:<port>/axis2/services/ucmdbManagementService`

      Replace *<UCMDB server name>* with the hostname of your UCMDB server, and replace *<port>* with the communications port your UCMDB server uses.
    - In the **UserId** and **Password** fields, type the user ID and password of a UCMDB user account that has permission to access the multi-tenant UCMDB Web Service API.

> **Note**
>
> The **UCMDB webservice URL**, **UserID**, and **Password** fields aren't required for enabling multi-tenancy.

5. Log off and then log back on to Service Manager so that the configuration takes effect.

To synchronize a company between Service Manager and UCMDB:

1. Sign in to **System Administration** > **Base System Configuration** > **Companies**.
2. Open an existing company record or create a new one.
    1. To open an existing company record, click **Search** and select the record from the list.
    2. To create a new company record, type a `Customer ID`, `Company Code`, and `Company Name` for the new record, and then click **Add**.
3. In the **Company** tab, complete the following fields:
    - In the **Customer ID** field, type a unique ID number. For example, 2.
    - In the **Customer Code** field, type a unique code name.
    - In the **Show Company in Multi-Company Lists** field, select **Yes**.
4. Click **OK**.
5. When prompted to confirm if you want to synchronize this company with UCMDB, select **Yes**.

To synchronize a company between APM and UCMDB:

For information about how to synchronize a company between APM and UCMDB, refer to the APM documentation.

Related topics

- Integrations
- Integration Manager
- APM Business Impact Report (BIR)
- Integration Instance Information fields
- Add a APM Business Impact Reports integration
- Edit an integration instance
- Enable or disable an integration instance
- Enable LW-SSO for the Incident Exchange (OMi - SM) integration
- Configure LW-SSO in the Service Manager Web tier

1.8.2.2. Launch a Business Impact Report from an incident

Applies to User roles: Service Desk Agent

> **Note**
>
> Make sure to enable an **SMBIR** integration instance in **Integration Manager** before performing the steps below.

To launch a Business Impact Report from an incident:

1. Sign in to the Service Manager Web client.
2. From **Incident Management**, search for an incident record and open it.
3. Click **More** and then select **Launch Business Impact Report**.

   If you have enabled LW-SSO in both SM and APM:

   The report displays the APM's summary information about business CIs that are impacted by the Incident's affected CI.

   If you have disabled LW-SSO:

   An APM login page displays, and the business impact summary information displays after you log on.

Related topics

- Integration Manager
- Application Performance Management (APM)
- APM Business Impact Report (BIR)
- Incident Exchange (OMi - SM) integration
- Add a APM Business Impact Reports integration
- Edit an integration instance
- Enable or disable an integration instance
- Enable LW-SSO for the Incident Exchange (OMi - SM) integration
- Configure LW-SSO in the Service Manager Web tier
- Enable multi-tenancy for the APM Business Impact Report integration

1.8.3. SM-APM downtime synchronization

The downtime entity plays a central role in many of the IT processes that are executed during the normal operations of an IT group. Failing to correctly manage this entity across all IT disciplines can:

- Cause misunderstandings about the state of a CI and as a result the state of a business service.
- Increase false positive events.
- Increase the mean time to restoration (MTTR).
- And overall negatively influence the perception of the business users about the SLA they get from IT.

The synchronization of downtime information between Service Manager and APM is to avoid the above situations.

Application Performance Management (APM) includes a module that you can use to synchronize the downtime which is generated in Service Manager. The runtime service model (RTSM) will maintain the downtime as a scheduled downtime CI with its related configuration items. If there is Universal CMDB (UCMDB) deployed between Service Manager and RTSM, the integration will synchronize downtime information through UCMDB between the servers.

The Service Manager and APM integration supports two use cases of downtime management:

- Synchronize scheduled downtimes from Service Manager RFCs and tasks to APM in order to suppress events.

  For details about Service Manager configuration and adapter setup in APM or UCMDB for downtime management, see SM-APM downtime synchronization setup.

- Event management of APM to synchronize scheduled downtimes from APM to Service Manager incidents for the Service Desk's attention.

  For details about how to create and close incidents in Service Manager from APM Operations Manager i (OMi), see Configure automatic closure for OMi incidents and Change the default assignment group for OMi incidents.



Note

Note: The incoming incident created from APM downtime does impact CI Availability in Service Manager.

Related topics

Integration Manager
Application Performance Management (APM)
Add an instance in Service Manager Integration Suite (SMIS)

Tailor Service Manager to handle phase change
Set up integration in APM
Set up integration in Universal CMDB
Verify the SM-APM downtime synchronization setup

1.8.3.1. SM-APM downtime synchronization setup

Prerequisite:

- To enable downtime synchronization between Service Manager and APM, you need to apply the latest Content Package and update to UCMDB or to RTSM.
- The SLA scheduler needs to be started in the **System Status** form.
- Make sure the related CI which downtime is scheduled on has already been synchronized between Service Manager and Universal CMDB.

To set up the SM-APM downtime synchronization in your environment, you need to complete the following tasks:

- Add an instance in Service Manager Integration Suite (SMIS)

  To set up the integration from Service Manager to RTSM, this task creates the SMBSM_DOWNTIME instance in Service Manager Integration Suite (SMIS). Note that additional setup is required on the APM OMi side for integration from APM to Service Manager. For details, refer to the *Business Service Management Operations Manager i Extensibility Guide*.

- Tailor Service Manager to handle phase change

  In Change Management module, authorized users can manually change the phase of a change record. This will affect the validation of related scheduled downtime. This task tailors Service Manager to keep the scheduled downtime accurate after users manually changed the phase of related change.

- Set up integration in APM

  If you don't have UCMDB, perform this task to set up a new integration in RTSM to proactively pull the scheduled downtime from Service Manager.

- Set up integration in Universal CMDB

  This task sets up a new integration in Universal CMDB to proactively pull the scheduled downtime from Service Manager.

- Verify the SM-APM downtime synchronization setup

  Perform this task to check if you have successfully set up your SM-APM downtime synchronization.

Note

Notes:

1. For Changes/Tasks that have final approval phases defined in Service Manager Integration Suite (SMIS), the downtimes will be synchronized after the Changes/Tasks get final approval.
2. Only downtimes that end at a future time will be synchronized.
3. Select the **Configuration Item(s) Down** checkbox when scheduling downtimes in Changes/Tasks.

Related topics

Application Performance Management (APM)
SM-APM downtime synchronization
Add an instance in Service Manager Integration Suite (SMIS)
Tailor Service Manager to handle phase change
Set up integration in APM
Set up integration in Universal CMDB
Verify the SM-APM downtime synchronization setup

1.8.3.1.1. Add an instance in Service Manager Integration Suite (SMIS)

Applies to User roles: System Administrator

To set up the integration from Service Manager to RTSM, you must add an instance of this integration in the Service Manager Integration Suite (SMIS). Note that additional setup is required on the APM OMi side for integration from APM to Service Manager. For details, refer to the *Business Service Management Operations Manager i Extensibility Guide*.

To add the SMBSM_DOWNTIME instance:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **SMBSM_DOWNTIME** from the Integration Template list. Ignore the **Import Mapping** checkbox, which has no effect on this integration.
4. Click **Next**. The Integration Instance Information page opens.
5. Do the following:

   • Modify the **Name** and **Version** fields to the exact values you need.

   • In the **Interval Time(s)** field, enter a value based on your business needs in regard to downtime exchange frequency. Note that a short interval time can be safe because the next scheduled task will not start until the previous task is completed and the interval time passed.

   • In the **Max Retry Times** field, enter a value. This is the maximum allowed number of retries (for example, 10) for each failed task.

   • In the **Log File Directory** field, specify a directory where log files of the integration will be stored. This must be a directory that already exists on the Service Manager server. By default, logging message is output to `sm.log`.

   • (Optional) In the **SM Server** field, specify a display name for the Service Manager server host. For example, my_local_SM.

   • (Optional) In the **Endpoint Server** field, specify a display name for the APM server host. For example, my_APM_1.

   • (Optional) In the **Log Level** field, change the log level from INFO (default) to another level. For example, WARNING.

   • (Optional) If you want this integration instance to be automatically enabled when the Service Manager Server service is started, select **Run at system startup**.

6. Click **Next**. The Integration Instance Parameters page opens.

7. On the **General Parameters** tab, complete the following fields as necessary:

Name	Category	Value	Description
WithdrawDowntime	General	true/false	Set this value to `true`: When authorized users are manually changing the phase of a change record that has a "valid" outage, a window will open and provide choices of withdrawing the outage. Set this value to `false`: The pop-up window is disabled. This operation may cause some unapproved planned downtimes to be synchronized to APM. By default, this value is set to `true`.
Category or workflow (Process Designer) name of changes	Change	The final approval phase for changes	Set the final approval phase for downtime, which is the indication of valid downtime information.

Name	Category	Value	Description
Category or workflow (Process Designer) name of tasks	Task	The final approval phase for tasks	Set the final approval phase for downtime, which is the indication of valid downtime information.
sm.host	General	<sm server name >	Set the Service Manager server hostname or DNS name to compose the External Process Reference and the Reference Number of Scheduled Downtime CI in UCMDB. **Note** **Note:** Do not include a colon in this field. Other wise, the logic will be broken .

Name	Category	Value	Description
sm.reference.prefix	General	urn:x-hp:2009:sm	Set the prefix to compose the External Process Reference of Scheduled Downtime CI in UCMDB. **Note** **Note:** This field has a fixed value. Do not change it.

Notes:

• Type category or workflow name of change/task in the **Name** column. This value is case-sensitive and it must match the record in Service Manager database.

• Set the value to `Change` for changes in the **Category** column. Similarly, set the value to `Task` for tasks.

• Type the final approval phase in the **Value** column. This value is case-sensitive and it must match the record in Service Manager database. You can separate multiple phases by semicolons, which must be the English character.

• Detailed information will be displayed in the integration log if user input of categories/phases for the changes/tasks is not correct or the category and phase pair doesn't exist in the database.

• For Change Management categories which don't have approval phase, the downtime integration will treat its downtime information as final approved once created. You don't need to define any phases in SMIS parameters.

• For the category or workflow name of the changes and the tasks, the integration will ignore all the final phases defined for the redundant category or workflow.

8. Click **Next** twice and then click **Finish**. Leave the Integration Instance Mapping and Integration Instance Fields settings blank. This integration doesn't use these settings.

   Service Manager creates the instance. You can edit, enable, disable, or delete it in Integration Manager.

9. Enable the integration instance. SMIS will validate all the final phases you filled in the Integration Instance Parameters page and print warning messages if there are errors.

Related topics

Application Performance Management (APM)
SM-APM downtime synchronization
Tailor Service Manager to handle phase change
Set up integration in APM
Set up integration in Universal CMDB
Verify the SM-APM downtime synchronization setup

1.8.3.1.2. Tailor Service Manager to handle phase change

Applies to User roles: System Administrator, Implementer

In the Service Manager Change Management module, authorized users can manually change the phase of a change record. If the phase is changed to the one prior to the final approval phase in the SMBSM_DOWNTIME instance, the system will check if there are existing planned downtimes that have been set to Ready. If such downtimes exist, a window will open and provide two options for the corresponding planned downtimes:

- Click **Yes** to withdraw the corresponding planned downtimes from UCMDB. The changes or tasks need to be approved again to synchronize with UCMDB at another time.
- Click **No**. There will be no change to the planned downtimes even if the actual status of the changes or tasks aren't approved.



Note

Note: To disable the pop-up window when withdrawing the planned downtimes, you need to set the `WithdrawDowntime` parameter to `false` in the SMBSM_DOWNTIME instance. This operation may cause some unapproved planned downtimes to be synchronized to APM.

With Process Designer (PD) Content Pack 2 applied in Service Manager, you can tailor the process to transit changes or tasks from one phase that's after the final approval phase in the SMBSM_DOWNTIME instance to another that's prior to the final approval phase.To withdraw the related planned downtime for this kind of transition, you need to add a rule set for the transition in the Closed Loop Incident Process (CLIP) solution. Refer to the following steps:

1. Go to the target workflow that needs tailoring.
2. Select the transition that moves a phase from before the final approval phase to after the final approval phase.
3. In the Rule Sets section, click **Add** and select the **clip.downtime.withdraw** rule set.
4. Click **OK** to save the workflow.

Related topics

Application Performance Management (APM)
SM-APM downtime synchronization
Add an instance in Service Manager Integration Suite (SMIS)
Set up integration in APM
Set up integration in Universal CMDB
Verify the SM-APM downtime synchronization setup

1.8.3.1.3. Set up integration in APM

If you have Universal CMDB (UCMDB) between Service Manager and APM, you can use UCMDB to maintain downtime CIs that are populated from Service Manager, and APM will consume them through UCMDB. There is no need to set up an integration in APM (RTSM).

If you don't have UCMDB between Service Manager and APM, you need to set up a new integration in APM (RTSM) to proactively pull the scheduled downtime from Service Manager.

Being an instance of UCMDB, RTSM embeds in APM and performs the expected functions of a CMDB. RTSM reconciles and stores configuration items (CIs) that represent the IT environment components and helps organizations understand the relationships between these components and track their configuration.

Make sure you have applied the latest content package and update to RTSM. For instructions on how to set up a downtime integration in RTSM, refer to the *Data Flow Management Guide*.

Related topics

- Application Performance Management (APM)
- SM-APM downtime synchronization
- Add an instance in Service Manager Integration Suite (SMIS)
- Tailor Service Manager to handle phase change
- Set up integration in Universal CMDB
- Verify the SM-APM downtime synchronization setup

1.8.3.1.4. Set up integration in Universal CMDB

This task sets up a new integration in Universal CMDB (UCMDB) to proactively pull the scheduled downtime from Service Manager. Make sure you have applied the latest Content Package and update to your UCMDB.

To set up the Universal CMDB adapter:

1. Sign in to your UCMDB system as an administrator.
2. Create an integration point that connects to your Service Manager server. For example, SM980. Make sure this integration point uses the latest **ServiceManager Adapter 9-x**.
3. Click **Managers** > **Data Flow Management** > **Integration Studio**. UCMDB displays a list of integration points.
4. Select **sm931** in the Integration Point list. Make sure the **SM Configuration Items Population job** and the **SM Relations Population job** run first.
5. Create two integration jobs in the integration point.
    1. Create a new job including the `CLIP Down Time Population` job definition. Under **Scheduler Definition**, select the **Scheduler enabled** checkbox and set the Repeat Interval to 1 Minute. Click **OK** to save the job.
    2. Create another new job including the `CI To Down Time CI With Connection` job definition. Under **Scheduler Definition**, select the **Scheduler enabled** checkbox and set the Repeat Interval to 1 Minute. Click **OK** to save the job.

    Pay attention to the running order and run the `CLIP Down Time Population` job first. You can set the two jobs as schedule-based and set the schedule interval according to your needs.
6. After the two jobs have finished successfully, go back to the **Modeling** section and check the CIs with the type **ScheduledDowntime**. Make sure their relationships are created.



Note

Note: If no related CIs exist in UCMDB when creating relationships, the population will fail or succeed with a warning. To disable the warning, remove the downtime CI that doesn't have related CIs in UCMDB.

Related topics

Application Performance Management (APM)
SM-APM downtime synchronization
Add an instance in Service Manager Integration Suite (SMIS)
Tailor Service Manager to handle phase change
Set up integration in APM
Verify the SM-APM downtime synchronization setup

1.8.3.1.5. Verify the SM-APM downtime synchronization setup

Applies to User roles: System Administrator

When you have set up the Downtime integration, you can perform the following tasks to see if you have successfully set up your downtime synchronization.

Task 1: Open a new change of a category that has the final approval phase defined in SMIS

1. Click **Change Management** > **Changes** > **Create New Change**.
2. Select **Hardware** for example.
3. In the Affected CI field, set the name of the CI to be synchronized. For example: `adv-afr-desk-101`.
4. Set Scheduled Downtime Start and Scheduled Downtime End to a future time.
5. Select the **Configuration Item(s) Down** checkbox.
6. Set other required fields.
7. Click **Save&Exit**.

Task 2: Approve the change at the final approval phase

1. Click **Change Management** > **Changes** > **Search Change** and search for the change opened in Task 1.
2. Move the Change to the Change Approval phase.
3. Sign in to Service Manager with user account `Change.Approver`.
4. Search for the change and approve it.

Task 3: Create new format for the intClipDownTime table

1. Click **Tailoring** > **Forms Designer**.
2. Create a new format for the intClipDownTime table by using the Form Wizard.
3. Add all fields to this format.

Task 4: Check the corresponding intClipDownTime record

1. From Database Manager, open the format of the intClipDownTime table.
2. Click **Search** to see the record created for this downtime.
3. Check the External Status field:

External Status values	Description
`NULL`	The downtime is waiting for final approval, or the scheduler has not proceeded this record yet.
`0` (Canceled)	The downtime is canceled before being implemented.
`1` (Ready)	The downtime has been approved and is ready to be synchronized to UCMDB or APM (RTSM).
`2` (Withdrawn)	The downtime is approved first and then the approval is retracted (withdrawn).

Note

Notes:

1. Only downtime records with External Status `1` can be synchronized.
2. If the External Status is not `1`, wait some time for background schedulers SLA and SMBSM_DOWNTIME to process this record.

Task 5: Populate downtime from Service Manager to UCMDB

1. From UCMDB, run the `CLIP Down Time Population` job and the `CI To Down Time CI With Connection` job in a fixed order.
2. Search for the `adv-afr-desk-101` CI in UCMDB. Check that a corresponding Scheduled Downtime CI is created, and a relationship between the Scheduled Downtime CI and the affected CI is created.

Note

Note: If any of the aforementioned tasks fails, refer to SM-APM downtime synchronization setup and check if all the system prerequisites are met.

Related topics

Integration Manager
Application Performance Management (APM)
Add an instance in Service Manager Integration Suite (SMIS)
Tailor Service Manager to handle phase change

Set up integration in APM
Set up integration in Universal CMDB

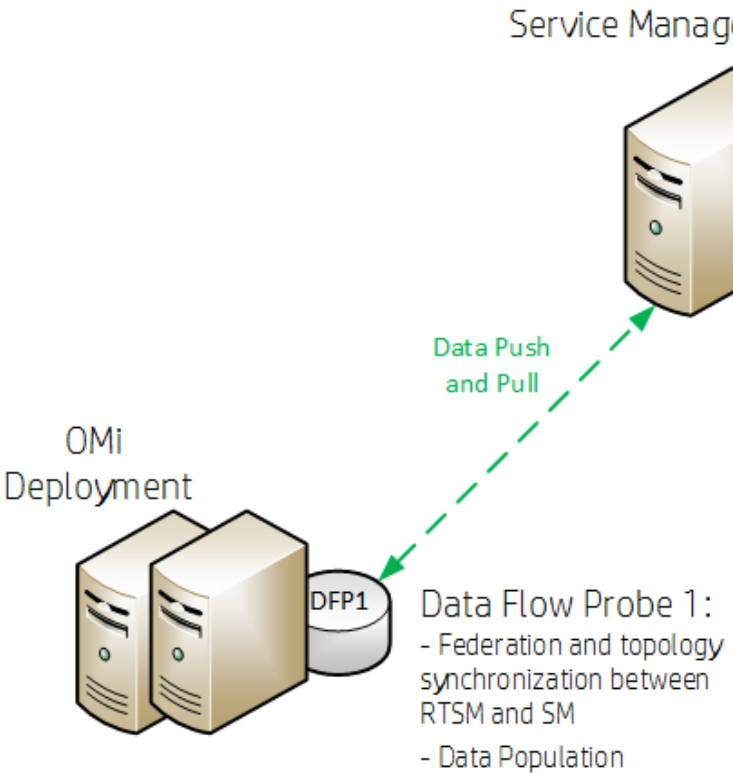
1.9. Operations Bridge Manager (OBM)

Note: Operations Bridge Manager (OBM) was known as OMi before version 2018.05.

There are two variations of integrating Service Manager (SM) with OBM. One case uses the RTSM contained in OBM as the CMDB, and needs only one data flow probe (DFP1), which is installed in the OBM deployment. The other case uses a UCMDB, and needs two data flow probes (DFP1 installed on the UCMDB server and DFP2 in the OBM deployment).

Point-to-point integration

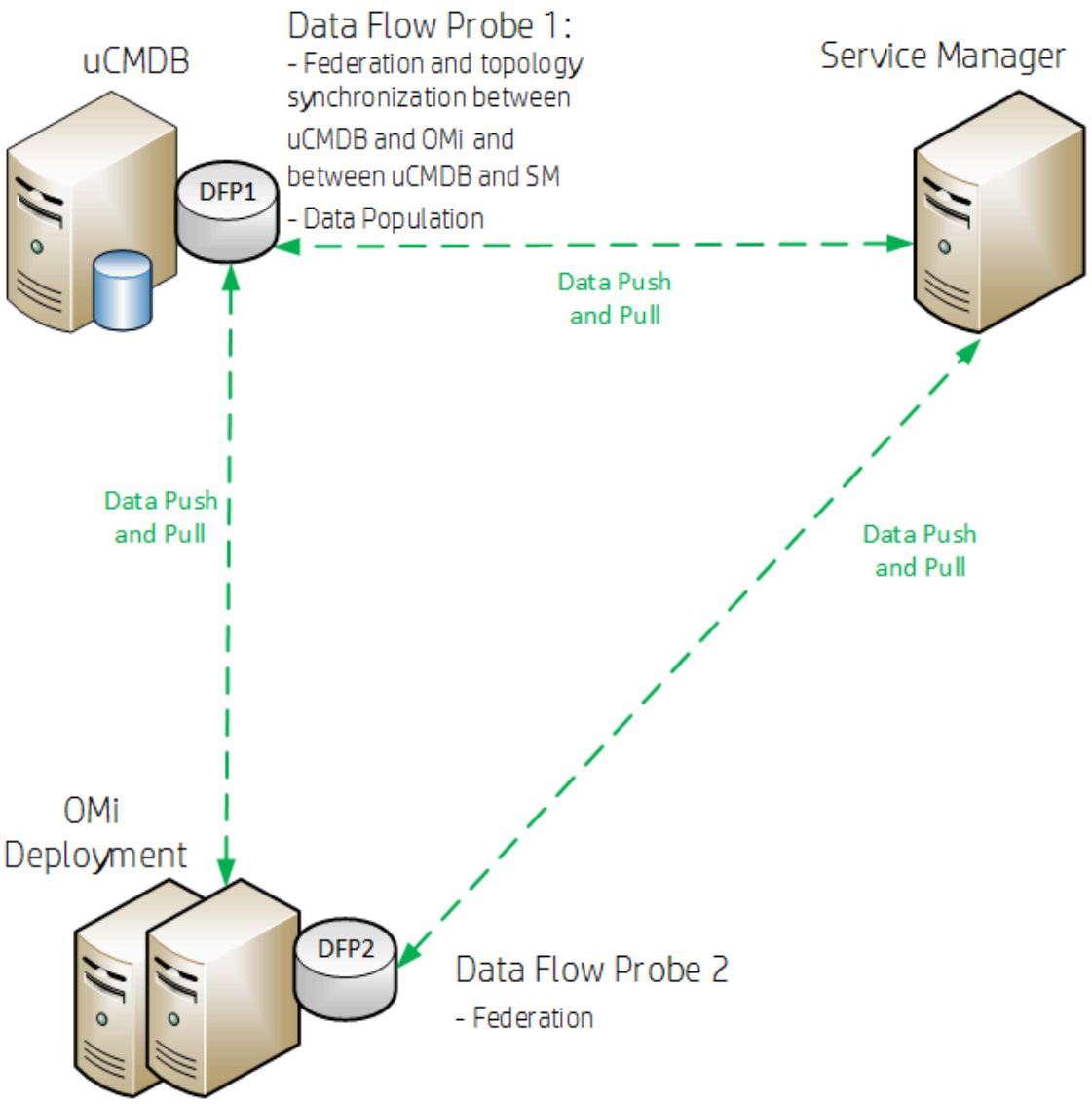
OBM is integrated with Service Manager directly, using OBM's Run Time Service Model (RTSM) as CMDB, as shown in the following figure.



We recommend installing the Data Flow Probe 1 (DFP1) in the OBM deployment.

Integration using a Universal Configuration Management Database (UCMDB)

OBM is set up using an external UCMDB, as shown in the following figure.



We recommend installing the Data Flow Probe 1 (DFP1) on the UCMDB server and the Data Flow Probe 2 (DFP2) in the OBM deployment.

Data flow probes

Two different data flow probes need to be installed. They have different purposes.

- DFP1 is needed for the following:
  - Populating the RTSM with CIs (Data Population)
  - Federation
  - Topology synchronization (CIs) between RTSM and SM in the case of a point-to-point integration

    OR
  - Topology synchronization (CIs) between UCMDB and OBM and between UCMDB and SM in case of using an external UCMDB

- DFP2 is needed for federation only.

Prerequisites

If you are using a standalone CMDB, you need to do the following before continuing with the OBM-SM integration:

- Set up the integration between OBM and UCMDB.
- Integrate UCMDB with SM to synchronize CIs from UCMDB to SM.

For details, see Universal CMDB.

Integration options

- **Incident exchange between SM and OBM**. OBM enables you to forward events from OBM to SM. Forwarded events and subsequent event changes are synchronized back from SM to OBM. You can also drill down from OBM events to SM incidents. For details, see Incident Exchange (OBM - SM) integration.
- **Downtime exchange between OBM and SM.** OBM enables you to forward downtimes (also known as outages) from OBM to SM, and from SM to OBM. The downtime defined in OBM is directed to SM as an incident, and vice versa. For details, see Downtime Exchange between OBM and Service Manager.
- **View planned changes and incident details in Service Health.** This integration enables you to view planned changes and incident details in the Changes and Incidents tab on the 360° View page in Service Health. For details, see *OBM - Service Manager integration overview*.
- The **Business Impact Report** integration is part of the Closed Loop Incident Process (CLIP) solution. When deployed as part of the OBM solution, Incident Management users can launch an impact report from an incident in context with the incident's affected CI. Service desk agents can validate the updated status of the business impact to categorize and prioritize the incident accordingly.

  For details, see the CLIP page in the APM Business Impact Report (BIR).

Note

- **Service Manager Query Security**: If you have set up an integration from OBM to SM, there is a CI context menu that enables you to access SM from OBM Service Health. This drill-down option is unavailable if you have enabled Service Manager query security.
- **Troubleshooting Multiple Domains**: If OBM and SM are in different domains, and you are using Internet Explorer as your browser, you may need to add the domains to the list of allowed domains in the Privacy tab (**Internet Options > Privacy > Sites**).

1.9.1. Incident Exchange (OBM - SM) integration

The Incident Exchange (OBM - SM) integration is a bidirectional integration between incident records in Service Manager and events in Operations Bridge Manager (OBM).

This integration requires configurations on both product sides. For details, see Incident Exchange (OBM - SM) integration setup.



Note

Service Manager can integrate with multiple OBM servers. For details, see Add an integration instance for each OBM server.

Service Manager can accept requests that based on RESTful from OBM to create Incidents in Service Manager, based on events information in OBM. When Service Manager accepts an incident creation request from a remote OBM server, it creates an incident record and automatically assigns it to an existing group based on certain field values. Incident Management users can view the details of the related OBM event by clicking the **View OBM Event** menu option from the incident record. See Drill down to the OBM event details from an incident.

When an Incident Management user makes any changes to the incident record, Service Manager sends an update request to the OBM RESTful web service interface. Hence, it automatically synchronizes the changes to the corresponding event in OBM. In the event of a synchronization failure, a queuing mechanism will synchronize the changes again. See Synchronization of incident changes back to Operations Bridge Manager (OBM).

System administrators can configure global settings to decide whether and when incident records opened from OBM events can be automatically closed. However, Incident Management users can mark individual OBM incident records as eligible or ineligible for automatic closure. See Configure automatic closure for OBM incidents and Mark an incident for automatic closure.

Related topics

- Incident Exchange (OBM - SM) integration setup
- Synchronization of incident changes back to Operations Bridge Manager (OBM)
- Add an integration instance for each OBM server
- Enable LW-SSO for the Incident Exchange (OBM - SM) integration
- Configure automatic closure for OBM Incidents
- Mark an incident for automatic closure
- Drill down to the OBM event details from an incident
- Change the default assignment group for OBM incidents

1.9.1.1. Incident Exchange (OBM - SM) integration setup

The Incident Exchange (OBM-SM) integration requires you to complete the following configuration tasks on the Service Manager and Operations Bridge Manager (OBM) systems.

Note

Configuring Service Manager web service integrations in a network environment using a firewall or proxy server requires additional configuration. For details, see Web Services with a proxy server.

1. Create user accounts for the Incident Exchange (OBM - SM) integration.

   This task creates a user account on each product side for the two systems to connect to each other and synchronize data.

2. Configure an event forwarding rule in OBM.

   This task configures a rule for the OBM server to forward events to the Service Manager server.

3. Configure the Service Manager server as a connected server in OBM.

   This task configures the connection settings for the target Service Manager server. If you need to integrate Service Manager with more than one OBM server, perform this task on each of the OBM servers.

4. (Optional) Add custom attributes and map to SM fields.

   This task adds your own custom attributes in the Groovy script selected for the SM server and maps the attributes to appropriate fields in SM.

5. Enable event drill-down from Service Manager into OBM

   This task sets up Service Manager operators as valid users with appropriate permissions in OBM. These permissions are required for the users to perform event drill-down from Service Manager to OBM.

6. Enable incident drill-down from the OBM Event Browser.

   This task configures the Service Manager web tier in the **sm:ServiceManagerAdapter** script in OBM.

7. (Optional)Configure SSL for the Incident Exchange (OBM - SM) integration .

   This task is needed if your OBM server requires HTTPS connections. If SSL is not configured in this case, changes on incidents that are created from OBM will not be able to be synchronized back to OBM.

8. Configure the Instance Count in the SMOBM integration template.

   This task is needed only when you use more than one OBM server. The Instance Count setting defines the allowed number of SM-OBM integration instances in Service Manager (default: 1).

9. Add an integration instance for each OBM server.

   This task creates and enables an instance of this integration in Integration Manager (SMIS). A separate integration instance is required for each OBM server.

10. (Optional) Enable LW-SSO for the Incident Exchange (OBM - SM) integration.

    Lightweight Single Sign-On (LW-SSO) is optional but recommended for the Incident Exchange (OBM - SM) integration. This task includes enabling LW-SSO in the Service Manager server and Web tier, as well as in OBM.

11. (Optional) Configure automatic closure for OBM incidents.

    By default, automatic closure is disabled for OBM incidents. System administrators can enable automatic closure and further configure under what conditions OBM incidents can be automatically closed.

12. (Optional) Change the default assignment group for OBM incidents.

    When created, OBM incidents are automatically assigned with an assignment group based on their certain field values and a predefined default assignment group. System Administrators can change the default assignment group setting (**Application**).

13. Test the connection

    This task sends an event to the OBM server that matches the filter you defined (in our example filter, the severity value is Critical), and then verifies that the event is forwarded to Service Manager as expected.

14. (Optional) Synchronize attributes

    Not all attributes are synchronized back from SM to OBM by default. This task changes the out-of-box behavior regarding which attributes are synchronized upon change.

Related topics

Integrations
Integration Manager
Synchronization of incident changes back to Operations Bridge Manager (OBM)
Add an integration instance for each OBM server
Enable LW-SSO for the Incident Exchange (OBM - SM) integration
Configure automatic closure for OBM incidents
Mark an incident for automatic closure
Drill down to the OBM event details from an incident
Change the default assignment group for OBM incidents

1.9.1.1.1. Create user accounts for the Incident Exchange (OBM-SM) integration

Applies to User roles: System Administrator

The Incident Exchange (OBM - SM) integration is bidirectional. Synchronizing events and incidents between the Service Manager and OBM systems requires you to set up integration accounts for the two systems to access each other.

1. Create an operator record with system administration privileges in Service Manager. For details, see Creating operator records.

   This is the user account that the OBM server uses to access Service Manager and to forward events to Service Manager.

   Later, when configuring the Service Manager server as a connected server in OBM, you need to specify this operator's login name and password on the **Outgoing Connection** tab. See Configure the Service Manager server as a connected server in OBM.

2. Create a user account with system administration privileges on each OBM server. For details, see the OBM online help.

   This is the user account that Service Manager uses to access the OBM server and to synchronize incident changes back.

   Later, when configuring the Service Manager server as a connected server in OBM, you must specify this user's login name as the **Name** of the Service Manager server on the **General** tab. Next, specify this user's password as the **Password** on the **IncOBMng Connection** tab. See Configure the Service Manager server as a connected server in OBM.

   Also, you need to specify the same user account when adding an SM - OBM integration instance for each OBM server (the **username** and **Password** parameters). See Add an integration instance for each OBM server.

Related topics

Add an integration instance for each OBM server
Configure the Service Manager server as a connected server in OBM

1.9.1.1.2. Configure the Service Manager server as a connected server in OBM

To synchronize changes between OBM events and Service Manager incidents, you need to configure a connected server within OBM to correctly identify the target Service Manager server instance.

To configure the Service Manager server as a target connected server, perform the following steps:

1. Log on to OBM as a system administrator.
2. Navigate to the Connected Servers user interface:

   Administration > **Setup and Maintenance** > **Connected Servers**
3. Click the **New** button, and then select **External Event Processing** to open the **Create New Server Connection** dialog box.
4. In the **Display Name** field, enter a descriptive name for the Service Manager server.

   In the **Name** field, enter the user name of the OBM user account you created for Service Manager to access the OBM server. See Create user accounts for the Incident Exchange (OBM - SM) integration.

   Make a note of the Name of the new target server. You need to provide it later on as the **username** when configuring the Service Manager server to communicate with the OBM server. See Add an integration instance for each OBM server.

   Optionally, enter a description for the new target server.

   Make sure that the **Active** checkbox is selected.

   Click **Next**.
5. In the Server Properties pane, enter the following information:

   Fully Qualified DNS Name: The fully qualified domain name of the target Service Manager server.

   CI Type: Select **Service Manager System** from the list.

   Keep the Advanced Delivery Options as default.

   Click **Next**.
6. In the **Integration Type** dialog box, you can choose between using a Groovy script adapter, or the Event Synchronization Web Service.

   1. As a Service Manager Groovy script adapter is provided for integrating with Service Manager, select **Call Script Adapter**.

2. In the **Script Name** field, select **sm:ServiceManagerAdapter**.
3. Click **Next**.

7. In the **Outgoing Connection** dialog, enter the credentials (user name, password, and port number) to connect to the Service Manager server and to forward events to that server.
   1. In the **User Name** and **Password** fields, enter the Service Manager user credentials you created for the integration. See Create user accounts for the Incident Exchange (OBM - SM) integration.
   2. Repeat the password entry in the **Verify Password** field.
   3. In the **Port** field, enter the communications port of the Service Manager server.

      The Service Manager server configuration file (sm.ini) defines the http and http ports. Enter the http port when Service Manager is running in http mode, or enter the https port when it is running in secure http mode.

      Tip

      If you click **Set default port**, the **Port** field is automatically populated with the default port (**13080** for http or **13443** for secure http). However, your actual Service Manager ports may differ from the default values.

   4. If the Service Manager server uses secure http (SSL) mode, select the **Use Secure HTTP** checkbox. If it uses http mode, make sure the checkbox is not checked.
   5. If the **Use Secure HTTP** checkbox is selected, download and install a copy of the Service Manager server's SSL certificate by clicking the link **Retrieve from Server**, or **Import from File** if the certificate is available in a local file.
   6. Make sure that the **Enable Synchronize and Transfer Control** checkbox is checked.

      When the **Enable Synchronize and Transfer Control** flag is set, an Operations Management operator is then able to transfer ownership of the event to the target connected server. If the flag is not set, then the option **Synchronize and Transfer Control** doesn't appear in the list of forwarding types when configuring forwarding rules.

      Also, note that if the **Enable Synchronize and Transfer Control** flag is not set for any target connected server, the **Transfer Control to** option doesn't appear at all in the Event Browser context menu.

      If a specific server is configured without the **Enable Synchronize and Transfer Control** flag set, then that server is unavailable in the Event Browser context menu as a server to which you can transfer ownership.

   7. Click **Test Connection**. A **Success** or **ERROR** hyperlink appears.

If an error appears, click the link to get a more detailed message. Fix the problems and try again until the connection is successful.

8. Click **Next**.

8. If, in addition to automatically generating Service Manager incidents from OBM events, you want to also be able to drill-down into Service Manager, you need to specify the fully qualified DNS name and port of the Service Manager web application server (for example, Tomcat).

   Note

   To enable incident drill-down to Service Manager, you must have the Service Manager web tier deployed on a web application server.

   In the **Event Drilldown** dialog box, configure the **Fully Qualified DNS Name** and **Port** of the web application server on which the Service Manager web tier is deployed.

   Note

   If you don't specify a server in the **Event Drilldown** dialog box, it is assumed that the web tier is deployed on the same host as the Service Manager server.

   If the web application server communicates with the browser over SSL (secure HTTP), select the **Use Secure HTTP** checkbox.

   Click **Next**.

9. The next thing to do is to enable event changes to be synchronized back from Service Manager to OBM. For this you need to provide credentials for the Service Manager server to access the OBM server in the **IncOBMng Connection** dialog box.

   1. Select the **Accept event changes from external processing server** checkbox.

      Note

      If **Enable Synchronize and Transfer Control** was previously selected, the **Accept event changes from external processing server** option is assumed, and can't be disabled.

   2. Enter the password of the OBM user account that you created for the Service Manager server to access the OBM server. See Create user accounts for the Incident Exchange (OBM - SM) integration.

   3. Make sure the **User Name** (auto-generated) matches the one of the OBM user account you created. If not, click the **General** tab, and change the **Name** field to the correct value.

4. Click **Finish**. The target Service Manager server now appears in the list of Connected Servers.

Related topics

Integration Manager
Incident Exchange (OBM - SM) integration
Incident Exchange (OBM - SM) integration setup
Add or delete an integration instance
Edit an integration instance
Enable or disable an integration instance
Enable LW-SSO for the Incident Exchange (OBM - SM) integration

1.9.1.1.3. Add custom attributes and map to SM fields

You can add your own custom attributes in the Groovy script selected for the SM server in the Connected Servers pane. Next, map these custom attributes to the appropriate fields in SM. For details about groovy scripting, see the Operations Bridge Manager help center.

You can also change how attributes are mapped from OBM to SM. You can configure the mapping in the BTO Data Model (BDM) Mapping Manager in SM:

System Administration > **Ongoing Maintenance** > **BDM Mapping Management**

For details about mapping attributes, see BDM Mapping Management.

1.9.1.1.4. Configure an event forwarding rule in OBM

Suppose you have configured the Service Manager server as a connected server in Application Performance Management (APM) Operations Bridge Manager (OBM). Next, you need to configure an event forwarding rule for the OBM server to forward events to Service Manager.

1. Sign in to OBM as a system administrator.
2. Navigate to **Administration** > **Event Processing** > **Automation** > **Event Forwarding**.
3. Click the **New Item** button.
4. In the **Display Name** field, enter a name for the rule. For example, `smserver1`.
5. Optionally, enter a description for the rule.
6. In the **Event Filter** field, click the **Manage Filters** button.

   Click **New**, and then select **New Simple Filter** or **New Advanced Filter**.

   Follow the screen prompts to configure the rule. For example, for Severity, select **Critical**.

   Click **OK**.
7. In the **Target Servers** field, select the Service Manager server you configured as a connected server, and then click the **Add** button.

   The details of the target server are displayed.
8. Click **OK**.

   The **Event Forwarding Rules** list displays the forwarding rule.

Related topics

Integration Manager
Incident Exchange (OBM - SM) integration
Incident Exchange (OBM - SM) integration setup
Enable event drill-down from Service Manager into OBM

1.9.1.1.5. Enable event drill-down from Service Manager into OBM

An OBM administrator must configure the operators as valid users with appropriate permissions in OBM. Hence, these operators are able to perform event drill-down from SM into the OBM user interface using a URL launch of the Event Browser,

> **Note**
>
> Without valid user names, or if a user doesn't have the required viewing permissions, trying to perform a URL launch of the OBM Event Browser from SM results in an empty browser window.

User account requirements

- If you have configured SM to use Single Sign-On (SSO) authentication, set up each user in OBM with the *same* user name that's used by the SM operator to sign in to SM and perform the URL call. (The password of each OBM user can be any string, but not empty.) After successfully logging on to SM, the OBM users can launch the OBM Event Browser without further authentication.

  For details on setting up SSO, see **System Administration > Integrations > Service Manager integration methods and tools > Integration Manager > Using LW-SSO with integrations** in the SM online help.

- If SM isn't configured to use SSO authentication, set up each user with the *same* user name that's used by the SM operator and specify a valid password. The users are required to enter their user name and password when launching the OBM Event Browser.

Required user permissions

You must grant the permission `Events assigned to user` including the required actions to each OBM user.

To do this, follow these steps:

1. Select **Administration** > **Users** > **Users, Groups, and Roles**.
2. Select a role or create a new one. In the Permissions section, go to the **Operations Console** category, select **Events** and specify the actions users can perform on **Events assigned to user**.
3. You can optionally grant the permission to view events not assigned to each user.

Related topics

Integration Manager
Incident Exchange (OBM - SM) integration

Incident Exchange (OBM - SM) integration setup
Enable incident drill-down from the OBM Event Browser

1.9.1.1.6. Enable incident drill-down from the OBM Event Browser

Suppose you have configured the Service Manager server as a connected server in Operations Bridge Manager (OBM). If you want to be able to drill down to Service Manager incidents from the OBM Event Browser, you need to perform the following tasks.

1. Configure the Service Manager web tier in the `sm:ServiceManagerAdapter` script in OBM
2. Set the `queryhashcode` parameter in the Service Manager server configuration file

Configure the Service Manager web tier in the `sm:ServiceManagerAdapter` script in OBM

To do this, follow these steps:

1. Sign in to OBM as a system administrator.
2. Navigate to **Administration** > **Setup and Maintenance** > **Connected Servers**.
3. Click the **Manage Scripts** icon in the toolbar.
4. Click the **sm:ServiceManagerAdapter** script, and then click the **Edit** icon.
5. On the **Script** tab, locate the following string in the script:

   ```
   private static final String SM_WEB_TIER_NAME
   ```

6. Change the value of this parameter to the name of your Service Manager web tier. For example, if your web tier file name is **sm-9.xx.war**, change the parameter value to:

   ```
   private static final String SM_WEB_TIER_NAME = 'sm-9.xx'
   ```

7. Click **OK** to save the script.

Set the `queryhashcode` parameter in the Service Manager server configuration file

To do this, follow these steps:

1. Stop the Service Manager server.
2. Add the following parameter to the server configuration file (`sm.ini`):

   `queryhashcode:<host>:<port>`

   Where: `<host>:<port>` must be the same as the Service Manager hostname and port number that are configured in the Service Manager web tier configuration file (`web.xml`).

3. Restart the Service Manager server.

Related topics

Integration Manager
Incident Exchange (OBM - SM) integration
Incident Exchange (OBM - SM) integration setup
Configure SSL for the Incident Exchange (OBM - SM) integration

1.9.1.1.7. Configure SSL for the Incident Exchange (OBM - SM) integration

Applies to User roles: System Administrator

When Operations Bridge Manager (OBM) or Service Manager is configured to accept HTTPS connections only, you must configure SSL for the integration. If you don't do so, changes on an incident that's created from OBM can't be synchronized back to OBM.

> **Note**
>
> The following steps describe how you do so by using the built-in keytool in Service Manager, and the file paths are for Windows only. Be sure to change the file paths accordingly if your Service Manager system is running on Linux.

If OBM is configured to accept HTTPS connections only, do the following:

1. Import the OBM root certificate to the Service Manager server trusted keystore.

   The following is an example of the command line:

   `<SM Install path>\server\RUN\jre\bin\keytool -import -alias myCA -file <.pem file of your APM root certificate> -keystore <SM Install path>\Server\RUN\jre\lib\security\cacerts -storepass <changeit>`

   Where: *changeit* is the default password of the trusted keystore. Change it to your own password if you changed it previously.

2. Add the following parameters to the Service Manager server configuration file (<*SM install path*>\Server\RUN\sm.ini):

   `truststoreFile:<SM install path>\Server\RUN\jre\lib\security\cacerts`

   `truststorePass:<changeit>`

3. Restart the Service Manager Server service.

For bidirectional SSL, you need to enable SSL in Service Manager first, and then import the SM certs to the OBM trusted keystore. Follow these steps:

1. Enable SSL on the Service Manager server. For more information, see Secure Sockets Layer (SSL) encryption.
2. Run the following command to export the SM certs.

   `<SM Install path>\server\RUN\jre\bin\keytool -export -alias smserver -file sm-certificate.crt -keystore <SM Install path>\server\RUN\smserver.p12 -storepass <serverkeystore>`

3. Run the following command to import the SM certs to the OBM trusted keystore.

```
opr-cert-mgmt.bat -import smserver sm-certificate.crt
```

Related topics

Configure the Instance Count in the SMOMi integration template

1.9.1.1.8. Configure the Instance Count in the SMOMi integration template

Applies to User roles: System Administrator

Service Manager can integrate with more than one Operations Bridge Manager (OBM) server. However, this integration only allows one OBM server by default. If you need to integrate Service Manager with more than one OBM server, you need to configure the Instance Count setting in the **SMOMi** integration template, as described below.

1. Sign in to Service Manager as a system administrator.
2. Type db in the command line, and press Enter.
3. In the **Table** field, type `SMISRegistry`, and click **Search**.

   The SMIS integration template form opens.
4. Click **Search**.

   A list of SMIS integration templates opens.
5. Select **SMOMi** from the list.
6. In the **Instance Count** field, change the value of 1 to the number of OBM servers that you want to integrate with Service Manager. For example, if you need two OBM servers, change the value to 2.
7. Click **Save**.

Related topics

Incident Exchange (OBM - SM) integration
Configure automatic closure for OBM incidents
Mark an incident for automatic closure
Change the default assignment group for OBM incidents

1.9.1.1.9. Add an integration instance for each OBM server

Applies to User roles: System Administrator

Once you have completed your configuration in Operations Bridge Manager (OBM), and have changed the Instance Count setting in the SMOMi integration template (which is needed only when you have multiple OBM servers), you are ready to add and enable a separate integration instance in Service Manager for each OBM server. For example, if you have two OBM servers, you must configure two SMOMi integration instances.

Support of multiple OBM servers

As of version 9.34, Service Manager can integrate with multiple OBM servers. This is implemented through the Instance Count setting and the **omi.mgr.id** parameter in the SMOMi integration template.

By default, the SMOMi integration template supports only one integration instance. If you have multiple OBM servers, before you proceed, make sure you have already updated the Instance Count setting in the SMOMi integration template. For details, see Configure the Instance Count in the SMOMi integration template.

When using the **omi.mgr.id** parameter, keep the following in mind:

- If you have only one OBM server (and hence need only one SMOMi integration instance), you must either correctly configure this parameter or clear the entire row of this parameter (both the parameter name and value) in the SMOMi integration instance, otherwise the integration will not work.
- If you have multiple OBM servers (and hence need multiple SMOMi integration instances), you must correctly configure this parameter in all SMOMi integration instances. Only those correctly configured integration instances will work. If none of the SMOMi integration instances are correctly configured, none of them will work.
- Users can view the OBM event details from an OBM incident record only when you specify the **omi.mgr.id** parameter correctly. If the value you specify in the corresponding SMOMi integration instance doesn't match the Universally Unique Identifier (UUID) which is automatically generated in the OBM server for the target Service Manager server and stored in the Incident record, users will not see the **View OMi Event** option from the Incident record.

To add and enable an Incident Exchange (OBM - SM) integration instance:

1. Sign in to Service Manager as a system administrator.
2. Click **Tailoring** > **Integration Manager**.
3. Click **Add**.

   The Integration Template Selection wizard opens.

4. Select **SMOMi** from the Integration Template list.

> **Note**
>
> Ignore the **Import Mapping** checkbox, which has no effect on this integration.

5. Click **Next**.
6. Complete the integration instance information:
   - Modify the **Name** and **Version** fields to the exact values you need.
   - In the **Interval Time (s)** field, enter a value. For example: 600. If an OBM opened incident fails to be synchronized back to OBM, Service Manager will retry the failed task at the specified interval (for example, 600 seconds).
   - In the **Max Retry Times** field, enter a value. For example: 10. This is the maximum allowed number of retries for each failed task.
   - (Optional) In the **SM Server** field, specify a display name for the Service Manager server host. For example, my_Local_SM.
   - (Optional) In the **Endpoint Server** field, specify a display name for the APM server host. For example, my_APM_1.
   - (Optional) In the **Log File Directory** field, specify a directory where log files of the integration will be stored. This must be a directory that already exists on the Service Manager server host.
   - (Optional) In the **Log Level** field, change the log level from INFO (default) to another level. For example, **WARNING**.
   - (Optional) If you want this integration instance to be automatically enabled when the Service Manager Server service is started, select **Run at system startup**.
7. Click **Next**. The Integration Instance Parameters page opens.
8. On the **General Parameters** tab, complete the following fields as necessary:

Field	Sample Value	Description
omi.server.url	http://<servername>:<port>/opr-gateway/rest/synchronization/event	This is the URL address of the OBM server's RESTful web service. Replace <servername> and <port> with the fully qualified domain name and port number of your OBM server.
http.conn.timeout	30	The HTTP connection timeout setting in seconds. **Note** The out-of-box value is 30 (seconds), and 15 (seconds) is used if this field is empty.
http.rec.timeout	30	The HTTP receive timeout setting in seconds. **Note** The out-of-box value is 30 (seconds), and 15 (seconds) is used if this field is empty.

Field	Sample Value	Description
http.send.timeout	30	The HTTP send timeout setting in seconds. **Note** The out-of-box value is 30 (seconds), and 15 (seconds) is used if this field is empty.
sm.mgr.id	55436DBE-F81E-4799-BA05-65DE9404343B	The Universally Unique Identifier (UUID) automatically generated for this instance of Service Manager. **Note** This field is automatically completed each time when you add an SMOMi integration instance. Do not change it, otherwise the integration will not work properly.

Field	Sample Value	Description
omi.reference.prefix	urn:x-hp:2009:opr:	The prefix of the BDM External Process Reference field, which will be present in incoming synchronization requests from the OBM server. **Note** This field is automatically completed and has a fixed value. Do not change it.
sm.reference.prefix	urn:x-hp:2009:sm:	The prefix of the BDM External Process Reference field, which will be present in outgoing synchronization requests from Service Manager. **Note** This field is automatically completed and has a fixed value. Do not change it.

Field	Sample Value	Description
omi.eventdetail.baseurl	http://<servername>:<port>/opr-web/eventDetails/app?eventId=	The basic URL address of the event detail page in OBM. Replace <servername> with the fully qualified domain name of your OBM server.

9. On the **General Parameters** and **Secure Parameters** tabs, enter three parameter values that you specified when configuring the Service Manager server as a connected server in APM OBM. The following table lists the parameters, whose values you can copy from your APM OBM server.

   To copy the parameter values from APM OBM, follow these steps:

   1. Sign in to APM as a system administrator.
   2. Navigate to **Admin** > **Operations Management** > **Setup** > **Connected Servers**.
   3. Locate your Service Manager server configuration entry and double-click anywhere on the entry pane.
   4. On the **General** tab, copy the **ID** string at the bottom into the **omi.mgr.id** field in Service Manager.
   5. On the **Incoming Connection** tab, copy the **User Name** and **Password** to the **username** and **Password** fields in Service Manager, respectively.

Field	Sample Value	Description
omi.mgr.id (on the **General Parameters** tab)	f3832ff4-a6b9-4228-9fed-b79105afa3e4	The Universally Unique Identifier (UUID) automatically generated in OBM for the target Service Manager server.

Field	Sample Value	Description
		Note This parameter was introduced to support multiple OBM servers. Service Manager uses the UUID to identify from which OBM server an incident was opened. Be aware that if you delete the connected server configuration for the Service Manager server in OBM and then recreate the same configuration, OBM generates a new UUID. You need to reconfigure the integration instance by changing the old UUID to the new one.

Field	Sample Value	Description
		Tip If you have only one OBM server, you can simply remove this parameter (remove both the parameter name and value) from the integration instance. See *Support of multiple* OBM *servers*.
username omi.mgr.id (on the **General Parameters** tab)	SM_Server	This is the user name that the Service Manager server uses to synchronize incident changes back to the OBM server.
Password (on the **Secure Parameters** tab)	SM_Server_Password	This is the password that the Service Manager server uses to synchronize incident changes back to the OBM server.

10. Click **Verify Connection**.

    If everything is fine, the following message is displayed: `Verification succeeded`.

11. Click **Next** twice, and then click **Finish**.

> **Note**
>
> Leave the Integration Instance Mapping and Integration Instance Fields settings blank. This integration doesn't use these settings.

Service Manager creates the instance. You can edit, enable, disable, or delete it in Integration Manager.

12. Enable the integration instance.
13. If you have multiple OBM servers, repeat the steps above for the rest of your OBM servers.

Next, you can optionally enable Lightweight Single Sign-On (LW-SSO) in both OBM and Service Manager so that users can bypass the log-in prompts. For details, see Enable LW-SSO for the Incident Exchange (OBM - SM) integration.

Related topics

Integration Manager
Incident Exchange (OBM - SM) integration
Incident Exchange (OBM - SM) integration setup
Enable LW-SSO for the Incident Exchange (OBM - SM) integration

1.9.1.1.10. Enable LW-SSO for the Incident Exchange (OBM - SM) integration

Applies to User roles: System Administrator

Lightweight Single Sign-On (LW-SSO) is optional but recommended for the Incident Exchange (OBM - SM) integration. You have different LW-SSO configuration choices depending on your needs. The following describes how LW-SSO can be used in the Incident Exchange (OBM - SM) integration workflow.

When OBM creates an incident from an OBM event record

OBM creates an incident from an OBM event record by sending RESTful-based requests to Service Manager. The incident ID is then stored in the event record.

LW-SSO is NOT needed in this process. A dedicated Service Manager user account was specified when configuring the Service Manager integration in OBM. OBM uses this dedicated user account when calling the Service Manager RESTful Web Service to create the incident.

When an OBM user views the incident details

The user can log in to Service Manager and view the incident details using the incident ID stored in the event record.

If the user wants to view the incident details by clicking the incident link from the event record, LW-SSO can be used; otherwise a Service Manager login prompt will appear.

LW-SSO is optional for this process. To enable LW-SSO for this process, configure LW-SSO in both the Service Manager server and Web tier (because the server needs to trust the Web tier), as well as in OBM.

When Service Manager synchronizes the OBM incident status back to OBM

When a user has updated the OBM incident, Service Manager calls the OBM server's RESTful Web Service to update the incident changes to the OBM event record.

LW-SSO is NOT needed in this process. A dedicated OBM user account was specified when the Incident Exchange (OBM - SM) integration was set up in SMIS, and Service Manager uses this user account when calling the OBM server's RESTful Web Service to synchronize the incident status back to the OBM event record.

When a user views the event details from the OBM incident

The user clicks the **View OBM Event** option from the incident to view the event details.

LW-SSO is optional for this process. If you enable LW-SSO in the Service Manager Web tier and in OBM, the OBM login prompt is bypassed.

Related topics

Incident Exchange (OBM - SM) integration
Incident Exchange (OBM - SM) integration setup
Using LW-SSO with integrations
Add an integration instance for each OBM server
Drill down to the OBM event details from an incident

1.9.1.1.11. Configure automatic closure for OBM incidents

Applies to User Roles: System Administrator

OBM incidents can be automatically closed after a predefined amount of time since they were last updated (or resolved if they have not been updated after being resolved).

The workflow is as follows:

1. An incident is opened from OBM.
2. If the **Schedule Condition** is met, the system creates a schedule record for the incident. The schedule record will expire at a future time based on the **Calc Expression**.
3. A user updates the incident and saves the changes.
4. The **Reset alerts if** expression on the **Alerts** tab of the **probsummary** object definition is evaluated. If it evaluates to true, the Expiration time of the schedule record is updated based on the Calc Expression. By default, the expiration time of the schedule record is updated only when the incident has a category of **incident**.
5. When the schedule record expires, the **Alert Condition** is evaluated. If it evaluates to true, the incident is automatically closed.

To enable automatic closure for OBM incidents:

1. Configure the global settings in the Incident Management Environment record.
    1. Click **System Administration** > **Ongoing Maintenance** > **Environment Records** > **Incident Management Environment**.
    2. Change the following settings as necessary.

Field	Value
Close Incident Automatically?	This option disables or enables the automatic closure of OBM incidents at the global level. ▪ If this option is not selected, no incidents will be automatically closed. ▪ If this option is selected, incidents will be automatically closed under specified conditions. Default: Not selected
Closure Code	This value will be copied to the **Closure Code** field of incidents when they are automatically closed. Default: Automatically Closed
Solution	This description will be appended to the end of the **Solution** field of incidents when they are automatically closed. Default: This incident which belongs to OBM has been closed automatically.

3. lick **Save**.
4. Restart the Service Manager server.

> **Note**
>
> If you have made any changes to any of the configuration options in the Incident Management Environment record, the Service Manager server must be restarted for the changes to take effect.

2. Configure the alert definition that determines when an incident should be closed.

> **Note**
>
> The **alert** and **problem** processes must be running to enable the successful closure of OBM incidents.

1. Click **Tailoring** > **Document Engine** > **Alerts**.
2. In the Alert Name field, enter: **OBM Auto-Close**.
3. Click **Search**. The OBM Auto-Close alert definition detail form opens.

> **Caution**
>
> These fields in the alert definition are used to control automatic closure of OBM incidents. You can change the default values of these fields. However, you must be aware of the risk that automatic closure of OBM incidents will not work properly if the **Schedule Condition** and **Alert Condition** fields aren't configured correctly.

Field	Value
Schedule Condition	This expression is used to determine if an incident should be scheduled for automatic closure. Default: `jscall("SMOBM.isAutoCloseAndResolved")`. An incident is scheduled for automatic closure when the following conditions are met. - The **Close Incident Automatically?** option in the Incident Management Environment record is selected. - In the incident record, the **Do not close this incident automatically** option is not selected. - The incident has a status of **Resolved**.
Alert Condition	This expression is evaluated when an incident is about to be automatically closed. If it evaluates to true, the incident is closed. Default: `jscall("SMOBM.isAutoCloseEnabled")`. An incident is closed when the following conditions are met. - The **Close Incident Automatically?** option in the Incident Management Environment record is selected. - In the incident record, the **Do not close this incident automatically** option is not selected.

Field	Value
Calc Expression	This expression is used to determine how much time will elapse before an incident is automatically closed. Default: `$L.alert.time=update.time in $L.file+'7 00:00:00'` . The default value means the amount of time elapsed is equal to seven days since the incident was last updated.

3. Configure alert information in the **probsummary** object.

   The OBM autoclose alert definition is configured to only be used by OBM incidents. The closure time is reset each time the incident is updated. If the closure time is reached without the incident being updated then Service Manager will automatically close the incident.

   1. Click **Tailoring** > **Document Engine** > **Objects**.
   2. In the **File name** field, enter **probsummary** and press ENTER. The **probsummary** object definition is displayed.
   3. Select the **Alerts** tab.

      The **Reset alerts if** expression is used to reset the automatic closure time of OBM incidents.

      Default: `category in $L.file="incident" and not null(1 in external.process.reference in $L.file)` .

Related topics

Incident Exchange (OBM - SM) integration
Mark an incident for automatic closure
Drill down to the OBM event details from an incident
Change the default assignment group for OBM incidents

1.9.1.1.12. Change the default assignment group for OBM incidents

Applies to User roles: System Administrator

Service Manager can accept REST based requests from Operations Bridge Manager (OBM) to create incidents based on events information in OBM. An incident opened from an OBM event is automatically assigned to an existing group based on the following field values, listed from the highest to lowest priority:

- The **Affected Service** of the incident
- The **Category** of the incident
- The **Affected CI** of the incident

However, if none of these field values is available, the system assigns the incident to a default group named **Application**. If necessary, you can change this default group setting by using BTO Data Model (BDM) Mapping Management as follows:

1. Navigate to **System Administration** > **Ongoing Maintenance** > **BDM Mapping Management**. The BDM mapping configuration search page opens.
2. Enter **incident** in the BDM Name field, select **1.1** in the **Version** field, and then click **Search**. The BDM mapping record **incident** opens. The Incident Exchange (OBM - SM) integration uses this BDM mapping record when creating an incident from an OBM event.
3. Select the **Field Mapping** tab, scroll down to the **assignment** field in the SM Object Field column, and click the **SM Callback** field in the same row.
4. Change **Application** in the following code to the name of another assignment group:

```
4)  A default assignment group if no other criteria is met
        if( ! $result ) {
                $result = "Application";
        }
```

5. Click **Save**. The default assignment group is now changed.

Related topics

Incident Exchange (OBM - SM) integration
BDM Mapping Management

1.9.1.1.13. Test the connection

To test the connection, send an event to the server hosting OBM that matches the filter you defined (in the example filter, the severity value is `Critical`). Next, verify that OBM forwards the event to SM as expected.

To test the connection, do the following:

1. On the Gateway Server system running OBM, open an Event Browser.
2. On the system running OBM, open a command prompt and change to the following directory:

   `<OBM_HOME>\opr\support`
3. Send an event using the following command:

   `sendevent -s critical -t test111-1`
4. Verify that the event appears in the OBM Event Browser.
5. Select the **Forwarding** tab.
6. In the **External Id** field, you should see a valid SM incident ID.
7. Verify that the incident appears in the Incident Management in Service Manager:

   If the event drill-down connection is correctly configured, click the hyperlink created with the incident ID. A browser window opens, which takes you directly to the incident in Service Manager.

   If you haven't configured the event drill-down connection, do the following:

   1. In the Forwarding tab in the OBM Event Browser, copy or note the incident ID from the External Id field.
   2. In the Service Manager user interface, navigate to **Incident Management** > **Search Incidents**.
   3. Paste or enter the incident ID in the Incident Id field.
   4. Click the **Search** button. This takes you to the incident in the Incident Details.
8. Close the incident in Service Manager.
9. Verify that Service Manager synchronizes the change in the state of the incident (it's now `closed`) back to OBM. You shouldn't be able to see the closed event in SM in the active Event Browser, but it should now be in the History Browser.

Related topics

Incident Exchange (OBM - SM) integration
Synchronize attributes

1.9.1.1.14. Synchronize attributes

Not all attributes are synchronized back from SM to OBM by default. When the SM incident is initially created from an OBM event, event attributes are mapped to the corresponding SM incident attribute. Out of the box, after the initial incident creation, whenever the incident or event subsequently changes, only a subset of the changed event and incident attributes are synchronized. The following describes how to customize the list of attributes to synchronize upon change.

Unidirectional synchronization: OBM to SM

OBM transfers the following attributes to SM when the event was initially created, and you have configured the transfer of control of the event in the Connected Servers manager.

These attributes support bidirectional synchronization, but are disabled out-of-the-box.

- Title
- Severity
- Priority
- Operator: the operator assigned to the event who forwarded the event
- Category
- Subcategory
- Related CI

OBM event annotations are synchronized to SM activity log and there is no back synchronization from SM to OBM.

Bidirectional synchronization

Attributes that support bidirectional synchronization between OBM and SM are:

- Description
- Lifecycle state (the state is only updated when the state changes to closed)
- Solution
- Contents under the Forwarding tab in the Event Details

Attribute synchronization using Groovy scripts

To change the out-of-the-box behavior regarding which attributes are updated, you can specify this in the Groovy script used on the OBM side for synchronization or incident creation. In the Groovy script, you can specify which fields are updated in SM, and which fields are updated in OBM. You can also specify custom attributes in the Groovy script.

For more information, see Tips for customizing groovy scripts.

Related topics

Incident Exchange (OBM - SM) integration
Tips for customizing groovy scripts

1.9.1.2. Tips for customizing groovy scripts

This section provides some tips about customizing Groovy scripts. It contains a few selected examples of what you can customize. To see further items that can be modified, see the configuration section of a Groovy script.

In the configuration section of the Groovy script, you can define and modify the attributes that are to be synchronized between OBM and SM. The configuration section of the Groovy script also contains the default value mappings for lifecycle state, severity, and priority. You can also modify these, and it is possible to define the mappings for in-going and out-going requests differently.

More advanced configuration can be done in other parts of the Groovy script if required.

The beginning and the end of the configuration section of the Groovy script is marked as follows:

```
//
// configuration section to customize the Groovy script
// BEGIN
...
...
//
// configuration section to customize the Groovy script
// END
```



Note

Modifications to Groovy scripts aren't overwritten by patches and hotfixes. Your customized version of a script will remain after an update or a patch. If you want to use the newer version of a script, make a copy of your version, revert back to the predefined version, and then reapply your changes.

The mapping from OBM to SM is compliant to BDM 1.1 incident web service specifications. The mapping of the BDM 1.1 incident web service to SM is specified in SM in the BDM Mapping Manager. For more information about the BDM Mapping Manager, see the BDM Mapping Manager section of the Service Manager online help.

Controlling attribute synchronization

You can control how updates to certain attributes are synchronized between OBM and SM by setting some Boolean variables to `true` or `false`.

Examples:

- `SyncAllProperties` variable. By default, it is `false`. If you set it to `true`, all properties will be synchronized in both directions. The other variables will be ignored.
- `private static final SyncTitleToSMOnUpdate = false;`

  This line of the Groovy script disables the synchronization of changes to the title made in OBM to SM.
- `private static final Boolean SyncTitleToOPROnUpdate = false;`

  This line of the Groovy script disables the synchronization changes to the title made in SM to OBM.

The title is a required attribute in SM, and it is set, independently of the flags above, using the title given in OBM during the creation of the incident.

Mapping OPR Lifecycle States to BDM Lifecycle States

Individual OPR event state and SM incident status changes may be selected for synchronization. Out of the box, only the "closed" state is synchronized in both directions. To change this behavior, add the desired states to the appropriate list, `SyncOPRStatesToSM` or `SyncSMStatusToOPR`.

Examples:

- `private static final Set SyncOPRStatesToSM = ["closed", "in_progress", "resolved"]`
- `private static final Set SyncSMStatusToOPR = ["closed", "resolved"]`

In the example, the OPR event lifecycle states `closed`, `in_progress`, and `resolved` are synchronized to the SM incident status, and SM incident statuses `closed` and `resolved` are synchronized to the OPR event state.



Note

The special state `"*"` denotes all states, so to synchronize all OPR event states to the SM incident status property, specify the following:

`private static final Set SyncOPRStatesToSM = ["*"]`

Additionally, two maps are used to specify the mapping of the OPR event lifecycle state to the BDM incident status. The maps are named `MapOPR2SMStatus` and `MapSM2OPRState`. Out of the box, all possible states have a mapping.

Examples:

- `private static final Map MapOPR2SMStatus = ["open": "open", "in_progress": "work-in-progress", "resolved": "resolved", "closed": "closed"]`
- `private static final Map MapSM2OPRState = ["accepted": "open", "assigned": "open", "open": "open", "reopened": "open",`

"pending-change": "in_progress", "pending-customer": "in_progress", "pending-other": "in_progress",

"pending-vendor": "in_progress", "referred": "in_progress", "suspended": "in_progress",

"work-in-progress": "in_progress", "rejected": "resolved", "replaced-problem": "resolved",

"resolved": "resolved", "cancelled": "resolved", "closed": "closed"]

Avoiding Errors with Large TQL Queries

If the Groovy script executes a TQL query that produces a large number of results, an error message appears informing you about the TQL query result exceeding the size limit. As a consequence, the integration event is not sent. It is possible, however, to increase this limit by modifying the value of the `tql.compound.link.max.visited.objects` setting.



Note

To check the default value of the `tql.compound.link.max.visited.objects` setting, from the JMX console, select **UCMDB:service=Settings Services**, and then locate the **showSettingsByCategory** method and enter **TQL Settings** as the category name.

To modify the value of the `tql.compound.link.max.visited.objects` setting, follow these steps:

1. From the JMX console, select **UCMDB:service=Settings Services**.
2. Click **setSettingValue**.
3. Enter `tql.compound.link.max.visited.objects` as the name of the setting you want to modify and a new value for it.



Caution

Increasing the value of the `tql.compound.link.max.visited.objects` setting also increases the load on the RTSM. Therefore, be sure to carefully consider how much to increase this value.

Syntax Errors

If you get a syntax error when customizing your Groovy scripts, you will get an event in the event browser with a detailed description of the error. In addition, you may view the `opr-event-sync-adapter.log` log file for information about how to resolve the error. You can find this log file at the following location:

`<Gateway Server root directory>/log/opr-event-sync-adapter.log`

Related topics

Incident Exchange (OBM - SM) integration
Synchronize attributes

1.9.1.3. Synchronization of incident changes back to Operations Bridge Manager (OBM)

After OBM opens an incident in Service Manager, Service Manager will synchronize the incident changes back to OBM.

Operations Bridge Manager (OBM) can forward an event record to Service Manager as an incident by calling a Service Manager Web Service. The incident ID is then stored in the event record.

When a user has updated the incident opened from OBM, Service Manager calls an OBM server RESTful Web Service to update the incident changes to the OBM event record.

If Service Manager fails to synchronize the incident changes back to OBM for some reasons (for example, because of a network problem), Service Manager behaves as follows:

- Displays a warning message, indicating that Service Manager fails to synchronize the incident to OBM.
- Saves the failed task in the Service Manager Integration Suite (SMIS) task queue, and retries the task to synchronize the changes again to OBM based on an interval time and a maximum retry times configured when adding the Incident Exchange (OBM - SM) integration in SMIS. When the synchronization is successfully completed, the system removes the failed task from the task queue.

System Administrators can monitor failed tasks, and reset their retry times or rerun expired tasks. For more information, see Monitor failover tasks.

Related topics

- Integration Manager
- Incident Exchange (OBM - SM) integration
- Incident Exchange (OBM - SM) integration setup
- Add an integration instance for each OBM server
- Enable LW-SSO for the Incident Exchange (OBM - SM) integration
- Drill down to the OBM event details from an incident
- Monitor failover tasks

1.9.1.4. Working with the Incident Exchange (OBM - SM) integration

After setting up the integration, Service Manager Incident Management users with the right permissions can perform the following tasks.

- View event details from OBM incidents
- Mark individual OBM incidents as eligible or ineligible for automatic closure (if their system administrator has enabled automatic closure for OBM incidents)

Related topics

- Integration Manager
- Incident Exchange (OBM - SM) integration
- Incident Exchange (OBM - SM) integration setup
- Add an integration instance for each OBM server
- Enable LW-SSO for the Incident Exchange (OBM - SM) integration
- Drill down to the OBM event details from an incident
- Monitor failover tasks

1.9.1.4.1. Drill down to the OBM event details from an incident

If all of the following conditions are met, you can view the related Operations Bridge Manager (OBM) event details from an OBM incident:

- You are accessing the incident through the Service Manager standard Web client (not the employee self-service (ESS) interface).
- One or more SMOMi integration instances are set up and enabled in Integration Manager.
- You are also an OBM user who has been granted the permission **Events assigned to user** including the required actions.

To drill down to the OBM event details from an OBM incident, follow these steps:

1. Sign in to the Service Manager Web client.
2. From **Incident Management**, search for the incident record created from OBM.
3. Click **More** and then select **View OMi Event**.

   Note

   The **View OMi Event** option is displayed only when the **omi.mgr.id** parameter in the corresponding SMOMi integration instance is set correctly.

   If Lightweight Single Sign-On (LW-SSO) is enabled in both the Service Manager Web client and OBM, the OBM event detail page opens in a new browser window, displaying the details of the related event in OBM. If LW-SSO is disabled, an OBM login page opens, and the related OBM event detail page is displayed after you sign in to OBM.

Related topics

Incident Exchange (OBM - SM) integration
Add an integration instance for each OBM server

1.9.1.4.2. Mark an incident for automatic closure

Applies to User roles: Incident Coordinator

You can mark an OBM incident as eligible or ineligible for automatic closure after a predefined amount of time since it was last updated.

To mark an incident as ineligible or eligible for automatic closure:

1. From **Incident Management**, search for an incident record opened from OBM.
2. Select or deselect the **Do not close this incident automatically** checkbox to mark this incident as ineligible or eligible for automatic closure.

> **Note**
>
> By default, this checkbox isn't selected.

Related topics

Incident Exchange (OBM - SM) integration
Configure automatic closure for OBM incidents
Drill down to the OBM event details from an incident
Change the default assignment group for OBM incidents

1.9.2. Downtime exchange between OBM and Service Manager

Operations Bridge Manager (OBM) enables you to forward downtimes (also known as outages) from OBM to Service Manager (SM), and from SM to OBM. The downtime defined in OBM is converted to an incident in SM, and vice versa.

Integration overview

The downtime integration between OBM and SM includes information exchanges in both of the following directions:

- **SM > OBM.** When you create a downtime RfC (request for change) in SM, the RfC includes the CI that's under change and a start and end date/time of the downtime. If you don't want to waste effort with false alarms in your operations center, and don't want to have these times included in service availability reports, you can set up the integration so that these RfCs are translated to downtimes in OBM.

  In this scenario, you install a data flow probe on the CMDB:

  - If you use OBM's RTSM as CMDB, install the data flow probe provided on the OBM installation media.
  - If you use UCMDB as CMDB, install the data flow probe as described in the UCMDB documentation. In this case, the synchronization must also be done from UCMDB to OBM, additionally to synchronizing SM with UCMDB. The RfC creates a planned downtime CI in the CMDB, and the data flow probe DFP1 sends the planned downtime CI to OBM to create a downtime.

- **OBM > SM.** When you define downtimes using OBM , the help desk should be aware of such operational downtimes: After you set up the integration, downtimes in OBM trigger events, which create corresponding incidents in SM.

  In this scenario, when a downtime starts, OBM generates an event. Using the event forwarding mechanism, the event generates an incident in SM. When the downtime ends, an event is sent to close the downtime incident.

A single downtime can be defined on more than one CI. In the case of OBM > SM, a separate event is sent for each CI in downtime.

Prerequisites

Supported platforms

To set up the downtime integration, you must meet the following prerequisites:

- Service Manager 9.31 and higher.
- UCMDB 10.01 or higher with content pack 12 or higher.

- Before deploying the adapter, verify that CP11 or higher is installed. If it's not, install the content pack.
- If the adapter is installed on the RTSM, and the adapter is working behind a reverse proxy, the DPS must have the correct certificates installed to send requests to the reverse proxy.

If you are using a UCMDB as a CMS, make sure that the CMS integration is set up. When it is set up, it serves as the global ID generator.

Global ID generator

If you are using OBM's RTSM you need to configure the RTSM to be the global ID generator, to enable the downtime integration:

1. Access the following location with your browser: http://<DPS name>:21212/jmx-console/HtmlAdaptor?action=inspectMBean&name=UCMDB:service=Multiple CMDB Instances Services
2. In the **setAsGlobalIdGenerator()** method, fill the **customerID** parameter with the value of **1**, and click **Invoke**.

Step 1: Send OBM downtime events to SM

To enable OBM to send downtime definitions to SM, follow these steps:

1. Access the following location in OBM:

   Administration > Setup and Maintenance > Infrastructure Settings > Foundations > Downtime

2. Change the value of the **Downtime Send Event** parameter to **true**.
3. Restart your OBM services on all Gateway Servers and Data Processing Servers.

This procedure generates events in OBM. After performing it, make sure you edit and enable the **Automatically forward "downtime started" and "downtime ended" events to Trouble Ticket System** event forwarding rule to forward downtime-start and downtime-end events to the SM server that should be specified in the alias connected server called "Trouble Ticket System". For details on event forwarding and connected servers, see the *OBM Administration Guide*.

Downtime events use the following formats:

- Downtime Start

Event field	OBM Downtime
Severity	Normal
Category	Downtime Notification
Title	Downtime for <CI Type><Affected CI Name>started at <Downtime Start Time>
Key	<OBM Downtime ID>:<Affected CI ID>:downtime-start
SubmitCloseKey	False
OutageStartTime	<Downtime Start Time>
OutageEndTime	<Downtime End Time>
CiName	<Affected CI Name>
CiId	<Affected CI Global ID>
CiHint	UCMDB:<Affected CI ID>
HostHint	UCMDB:<Related Host ID>
EtiHint	downtime:start

- Downtime End

Event field	OBM Downtime
Severity	Normal
Category	Downtime Notification
Title	Downtime for <CI Type><Affected CI Name> ended at < Downtime End Time>
Key	<OBM Downtime ID>:<Affected CI ID>:downtime-stop
SubmitCloseKey	true
CloseKeyPattern	<OBM Downtime ID>:<Affected CI ID>:downtime-start
EtiHint	downtime:end
LogOnly	true

Step 2: Integrate SM downtimes with OBM

To enable downtimes defined in SM to be sent to OBM, again, you need to distinguish between the two cases of where the CMDB is:

If you are using OBM's RTSM as CMDB, no further steps are required. See also Point to point integration.

If you are using an external UCMDB, you need to install the DFP2 in the OBM deployment. See also Integration using a Universal Configuration Management Database (UCMDB)

Important:

- Following the initial integration, a large amount of data may be communicated from SM to OBM. It is highly recommended that you perform this procedure during off-hours, to prevent negative impact on system performance.
- The integration consists of two parts: SM > CMS/UCMDB, and CMS/UCMDB > OBM adapter. You should configure both parts of the integration as one flow, without a significant time lag between setting up the two parts. If you set up the SM > CMSUCMDB part, and then wait a long time before setting up the CMS/UCMDB > OBM adapter part, the number of downtimes communicated to OBM initially may be extremely high.

Note

- The following procedure doesn't describe the SM > CMS/UCMDB connection setup. SM should be configured to create its CIs in the CMS. This procedure connects the adapter between the CMS/UCMDB and OBM.
- The default job synch frequency is one minute.

Create a new integration point as follows:

1. Create the integration point credentials:
    1. Access the Data Flow Probe Setup on your OBM or UCMDB depending on whether you use OBM's RTSM or use a CMS.

        Note

        You don't need a probe to perform this integration. Nevertheless you create credentials using the Data Flow Probe Setup tab.

    2. Click **Add domain or probe**, and enter a name and description of your choice.
    3. Expand the submenus and select **HTTP protocol**.
    4. Click the **+** sign (**Add new connection details**) and enter the OBM Gateway hostname, Port 80, and the OBM username and password. Leave the **Trust** fields blank. When you are done, click **OK** to save the credentials.
2. Create a new integration point:
    1. If you have OBM, do the following on your OBM. If you use a CMS, do the following on your CMS:

        Navigate to **Administration** > **RTSM Administration** > **Data Flow Management** > **Integration Studio**

    2. Click **New Integration Point**, enter a name and description of your choice, and select **BSMDowntimeAdapter/SM scheduled Downtime Integration into BSM**.
    3. Enter the following information for the adapter: OBM Gateway hostname and port, the integration point credentials you just created, communication protocol, and the context root (if you have a non-default context root).
    4. Click **OK**, then click the **Save** button above the list of the integration points.
3. You can use the **Statistics** tab in the lower pane to track the number of downtimes that are created or updated. By default, the integration job runs every minute. If a job has failed, you can open the **Query Status** tab and double-click the failed job to see more details on the error.

If there is an authentication error, verify the OBM credentials entered for the integration point.

If you receive an unclear error message with error code, this generally indicates a communication problem. Check the communication with OBM. If no communication problem is found, restart the **MercuryAS** process.

A failed job will be repeated until the problem is fixed.

1.9.2.1. Downtime Exchange integration setup

To set up the downtime exchange integration between Operations Bridge Manager (OBM) and Service Manager (SM), you need to perform the following tasks.

Note

Configuring Service Manager web service integrations in a network environment using a firewall or proxy server requires additional configuration. For details, see Web Services with a proxy server.

1. Enable OBM to send downtime events to Service Manager.
2. Integrate Service Manager downtime with OBM.
3. Add an integration instance in Service Manager.
4. Tailor Service Manager to handle phase change.
5. Verify the OBM - SM downtime synchronization setup.

Related topics

OBM - Service Manager integration overview
Downtime Exchange between OBM and Service Manager

1.9.2.1.1. Enable OBM to send downtime events to Service Manager

Downtime events use the formats listed in the following tables.

Downtime Start

Event field	OBM Downtime
Severity	Normal
Category	Downtime Notification
Title	Downtime for <CI Type><Affected CI Name>started at <Downtime Start Time>
Key	<OBM Downtime ID>:<Affected CI ID>:downtime-start
SubmitCloseKey	False
OutageStartTime	<Downtime Start Time>
OutageEndTime	<Downtime End Time>
CiName	<Affected CI Name>
CiId	<Affected CI Global ID>
CiHint	UCMDB:<Affected CI ID>
HostHint	UCMDB:<Related Host ID>
EtiHint	downtime:start

Downtime End

Event field	OBM Downtime
Severity	Normal
Category	Downtime Notification
Title	Downtime for <CI Type><Affected CI Name> ended at < Downtime End Time>
Key	<OBM Downtime ID>:<Affected CI ID>:downtime-stop
SubmitCloseKey	true
CloseKeyPattern	<OBM Downtime ID>:<Affected CI ID>:downtime-start
EtiHint	downtime:end
LogOnly	true

To enable OBM to send downtime definitions to Service Manager (SM), follow these steps:

1. Access the following location in OBM:

   Administration > Setup and Maintenance > Infrastructure Settings > Foundations > Downtime

2. Change the value of the **Downtime Send Event** parameter to **true**.
3. Restart your OBM services on all Gateway Servers and Data Processing Servers.

This procedure generates events in OBM. After performing it, make sure you edit and enable the **Automatically forward "downtime started" and "downtime ended" events to Trouble Ticket System** event forwarding rule to forward downtime-start and downtime-end events to the SM server that should be specified in the alias connected server called "Trouble Ticket System". For details on event forwarding and connected servers, see the *OBM Administration Guide*.

1.9.2.1.2. Integrate Service Manager downtime with OBM

To enable SM to send the defined downtime to OBM, again, you need to distinguish between the two cases of where the CMDB is:

If you are using OBM's RTSM as CMDB, no further steps are required. See also Point to point integration.

If you are using an external UCMDB, you need to install DFP2 in the OBM deployment. See also Integration using a Universal Configuration Management Database (UCMDB)

Important:

- Following the initial integration, SM may communicate a large amount of data to OBM. It's highly recommended that you perform this procedure during off-hours, to prevent a negative impact on system performance.
- The integration consists of two parts: SM > CMS/UCMDB, and CMS/UCMDB > OBM adapter. You should configure both parts of the integration as one flow, without a significant time lag between setting up the two parts. If you set up the SM > CMS/UCMDB part, and then wait a long time before setting up the CMS/UCMDB > OBM adapter part, the number of downtime communicated to OBM initially may be extremely high.

Note

- The following procedure doesn't describe the SM > CMS/UCMDB connection setup. You should configure SM to create its CIs in the CMS. This procedure connects the adapter between the CMS/UCMDB and OBM.
- The default job synchronization frequency is one minute.

Create a new integration point as follows:

1. Create the integration point credentials:
    1. Access the Data Flow Probe Setup on OBM or UCMDB depending on whether you use the OBM's RTSM or use a CMS.

        Note

        You don't need a probe to perform this integration. Nevertheless you create credentials using the Data Flow Probe Setup tab.

    2. Click **Add domain or probe**, and enter a name and description of your choice.
    3. Expand the submenus and select **HTTP protocol**.

4. Click the **+** sign (**Add new connection details**) and enter the OBM Gateway hostname, Port 80, and the OBM username and password. Leave the **Trust** fields blank. When you are done, click **OK** to save the credentials.

2. Create a new integration point:

    1. Access the Integration Studio on your OBM or UCMDB depending on whether you use OBM or use a CMS.
    2. Click **New Integration Point**, enter a name and description of your choice, and select **BSMDowntimeAdapter/SM scheduled Downtime Integration into BSM**.
    3. Enter the following information for the adapter: OBM Gateway hostname and port, the integration point credentials you just created, communication protocol, and the context root (if you have a non-default context root).
    4. Click **OK**, then click the **Save** button on top of the integration points list.

3. You can use the **Statistics** tab in the lower pane to track the number of downtime that are created or updated. By default, the integration job runs every minute. If a job has failed, you can open the **Query Status** tab and double-click the failed job to see more details on the error.

    If there is an authentication error, verify the OBM credentials entered for the integration point.

    If you receive an unclear error message with error code, this generally indicates a communication problem. Check the communication with OBM. If there is no communication problem, restart the **MercuryAS** process.

    The system repeats a failed job until the problem become fixed.

1.9.2.1.3. Add an integration instance in Service Manager

Applies to User roles: System Administrator

To set up the downtime exchange integration, you must add an instance of this integration in the Service Manager Integration Suite (SMIS).

To add the SMBSM_DOWNTIME instance, follow these steps:

1. Click **Tailoring** > **Integration Manager**. Integration Instance Manager opens.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **SMBSM_DOWNTIME** from the Integration Template list. Ignore the **Import Mapping** checkbox, which has no effect on this integration.
4. Click **Next**. The Integration Instance Information page opens.
5. Do the following:
    - Modify the **Name** and **Version** fields to the exact values you need.
    - In the **Interval Time(s)** field, enter a value based on your business needs in regard to downtime exchange frequency. Note that a short interval time can be safe because the next scheduled task will not start until the previous task is completed and the interval time passed.
    - In the **Max Retry Times** field, enter a value. This is the maximum allowed number of retries (for example, 10) for each failed task.
    - In the **Log File Directory** field, specify a directory where log files of the integration will be stored. This must be a directory that already exists on the Service Manager server. By default, logging message is output to `sm.log`.
    - (Optional) In the **SM Server** field, specify a display name for the Service Manager server host. For example, my_local_SM.
    - (Optional) In the **Endpoint Server** field, specify a display name for the APM server host. For example, my_APM_1.
    - (Optional) In the **Log Level** field, change the log level from INFO (default) to another level. For example, WARNING.
    - (Optional) If you want this integration instance to be automatically enabled when the Service Manager Server service is started, select **Run at system startup**.
6. Click **Next**. The Integration Instance Parameters page opens.
7. On the **General Parameters** tab, complete the following fields as necessary:

Name	Category	Value	Description
WithdrawDowntime	General	true/false	Set this value to `true`: When authorized users are manually changing the phase of a change record which has 'valid' outage, a window will open and provide choices of withdrawing the outage. Set this value to `false`: The pop-up window is disabled. This operation may cause some unapproved planned downtimes be synchronized to APM. By default, this value is set to `true`.
Category or workflow (Process Designer) name of changes	Change	The final approval phase for changes	Set the final approval phase for downtime, which is the indication of valid downtime information.

Name	Category	Value	Description
Category or workflow (Process Designer) name of tasks	Task	The final approval phase for tasks	Set the final approval phase for downtime, which is the indication of valid downtime information.
sm.host	General	<sm server name >	Set the Service Manager server host name or DNS name to compose the External Process Reference and the Reference Number of Scheduled Downtime CI in UCMDB. **Note** **Note:** Do not include a colon in this field. Other wise, the logic will be broken .

Name	Category	Value	Description
sm.reference.prefix	General	urn:x-hp:2009:sm	Set the prefix to compose the External Process Reference of Scheduled Downtime CI in UCMDB. **Note** **Note:** This field has a fixed value. Do not change it.

Note

Notes:

1. Type category or workflow name of change/task in the **Name** column. This value is case-sensitive and it must match the record in Service Manager database.
2. Set the value to `Change` for changes in the **Category** column. Similarly, set the value to `Task` for tasks.
3. Type the final approval phase in the **Value** column. This value is case-sensitive and it must match the record in Service Manager database.You can separate multiple phases by semicolons, which must be the English character.
4. Detailed information will be displayed in the integration log when the following errors occur:
   - User input of categories/phases for the changes/tasks is not correct.
   - The category and phase pair doesn't exist in the database.
5. For Change Management categories which don't have approval phase, the downtime integration will treat its downtime information as final approved once created. You don't need to define any phases in SMIS parameters.
6. For the category or workflow name of the changes and the tasks, the integration will ignore all the final phases defined for the redundant category or workflow.

8. Click **Next** twice and then click **Finish**. Leave the Integration Instance Mapping and Integration Instance Fields settings blank. This integration doesn't use these settings.

   Service Manager creates the instance. You can edit, enable, disable, or delete it in Integration Manager.

9. Enable the integration instance. SMIS will validate all the final phases you filled in the Integration Instance Parameters page and print warning messages if there are errors.

Related topics

Downtime Exchange between OBM and Service Manager
Tailor Service Manager to handle phase change
Verify the OBM-SM downtime synchronization setup

1.9.2.1.4. Tailor Service Manager to handle phase change

Applies to User roles: System Administrator, Implementer

In the Service Manager Change Management module, authorized users can manually change the phase of a change record. If the phase is changed to the one prior to the final approval phase in the SMBSM_DOWNTIME instance, the system will check if there are existing planned downtimes that have been set to Ready. If such downtimes exist, a window will open and provide two options for the corresponding planned downtimes:

- Click **Yes** to withdraw the corresponding planned downtimes from UCMDB. The changes or tasks need to be approved again to synchronize with UCMDB at another time.
- Click **No**. There will be no change to the planned downtimes even if the actual status of the changes or tasks aren't approved.



Note

To disable the pop-up window when withdrawing the planned downtimes, you need to set the `WithdrawDowntime` parameter to `false` in the SMBSM_DOWNTIME instance. This operation may cause some unapproved planned downtimes to be synchronized to Operations Bridge Manager (OBM).

With Process Designer (PD) Content Pack 2 or later applied in Service Manager, you can tailor the process to transit changes or tasks from one phase that's after the final approval phase in the SMBSM_DOWNTIME instance to another that's prior to the final approval phase.To withdraw the related planned downtime for this kind of transition, you need to add a rule set for the transition in the Closed Loop Incident Process (CLIP) solution.

To do this, follow these steps:

1. Go to the target workflow that needs tailoring.
2. Select the transition that moves a phase from before the final approval phase to after the final approval phase.
3. In the Rule Sets section, click **Add** and select the **clip.downtime.withdraw** rule set.
4. Click **OK** to save the workflow.

Related topics

Add an integration instance in Service Manager
Verify the OBM-SM downtime synchronization setup

1.9.2.1.5. Verify the OBM-SM downtime synchronization setup

Applies to User roles: System Administrator

When you have set up the Downtime Exchange integration, you can perform the following tasks to see if you have successfully set up your downtime synchronization.

Task 1: Open a new change of a category that has the final approval phase defined in SMIS

1. Click **Change Management** > **Changes** > **Create New Change**.
2. Select **Hardware** for example.
3. In the Affected CI field, set the name of the CI to be synchronized. For example: `adv-afr-desk-101`.
4. Set Scheduled Downtime Start and Scheduled Downtime End to a future time.
5. Select the **Configuration Item(s) Down** checkbox.
6. Set other required fields.
7. Click **Save & Exit**.

Task 2: Approve the change at the final approval phase

1. Click **Change Management** > **Changes** > **Search Change** and search for the change opened in Task 1.
2. Move the Change to the Change Approval phase.
3. Log on to Service Manager with user account `Change.Approver`.
4. Search for the change and approve it.

Task 3: Create new format for the intClipDownTime table

1. Click **Tailoring** > **Forms Designer**.
2. Create a new format for the intClipDownTime table by using the Form Wizard.
3. Add all fields to this format.

Task 4: Check the corresponding intClipDownTime record

1. From Database Manager, open the format of the intClipDownTime table.
2. Click **Search** to see the record created for this downtime.
3. Check the External Status field:

External Status values	Description
NULL	The downtime is waiting for final approval, or the scheduler has not proceeded this record yet.
0 (Canceled)	The downtime is canceled before being implemented.
1 (Ready)	The downtime has been approved and is ready to be synchronized to UCMDB or OBM (RTSM).
2 (Withdrawn)	The downtime is approved firstly and then the approval is retracted (withdrawn).

Note

Notes:

1. Only downtime records with External Status `1` can be synchronized.
2. If the External Status is not `1`, wait some time for background schedulers SLA and SMBSM_DOWNTIME to process this record.

Task 5: Populate downtime from Service Manager to UCMDB

1. From UCMDB, run the `CLIP Down Time Population` job and the `CI To Down Time CI With Connection` job in a fixed order.
2. Search for the `adv-afr-desk-101` CI in UCMDB. Check that a corresponding Scheduled Downtime CI is created, and a relationship between the Scheduled Downtime CI and the affected CI is created.

Note

Note: If any of the aforementioned tasks fails, refer to and check if all the system prerequisites are met.

Related topics

Integration Manager
Add an integration instance in Service Manager
Tailor Service Manager to handle phase change

1.10. Service Management Automation X (SMAX)

Service Manager provides a Case Exchange framework to exchange data between Service Management Automation X (SMAX) and Service Manager. The Case Exchange framework enables you to define Service Manager as an external system with which information will be exchanged. Hence, Service Manager has the ability to create and update records in SMAX as well as to relate those records to records in Service Manager.

Use cases

The integration with SMAX is designed to support use cases involving a Line of Business IT organization that's supported by a Central IT organization. It also supports use-cases involving Central IT and other IT suppliers.

The integration enables you to exchange incidents and requests between SMAX and Service Manager. Incident or request records created in SMAX can be assigned to a group that has been registered to a Service Manager system. This sends the record to Service Manager for investigation and resolution. Additional information that's added to the record, including comments and attachments, is also exchanged with Service Manager. Updates done in Service Manager, including activities and attachments, are sent to SMAX. It's also possible to create an incident or request in Service Manager and exchange it with SMAX. In this case as well, updates on either side are exchanged with the other system.

The exchanged record includes only the data necessary to create the incident or request in the Service Manager system. The workflow phases and statuses of the record in the two systems are independent of each other. Instead, the integration uses the concept of Operations. Operations are additional (bi-directional) information provided on the update of a record. They tell the receiving system what needs to be done with the record, and the system can determine which workflow steps or data modifications need to be implemented.

Defining possible operations for the actual exchange reduces the need to perform complex workflow mapping between the two systems. The operations are interpreted by the receiving system and may be mapped to their existing workflow and business rules without affecting the sending system.

Set up the integration

Implement the Case Exchange framework

For details, see General procedure.

Configure the integration

For detailed instructions, see Enable Case Exchange with SMAX.

Related topics

SMAX integration with Service Manager (SM)

1.11. Asset Manager

When ITAM and ITSM applications share common reference data related to the management and support of IT-related Configuration Items (CIs) and these references are not available through the CMDB, the data must be reconciled through other methods. Examples of these references include a variety of common entities, such as employees who use CIs and the supervisors responsible for managing them; the operating business units (that is, departments) and physical locations where "in-service" CIs are deployed; the suppliers and service providers of hardware and software Assets, supporting Application or Service CIs. The list continues. A simple way to categorize this wide array of references is "People, Places and Things."

In the case of Asset Manager (AM) and Service Manager (SM), the two repositories utilize "synchronization type" replication to maintain the integrity of these shared common references. Connect-It synchronizes the "People, Places and Things" between AM and SM and provides bi-directional mapping scenarios to perform all insert and update-related transactions in the target application. (Delete transactions must be managed within each respective application for audit and data integrity purposes.) The following table describes related scenarios depending on what will be synchronized and in which direction.

Integration direction	Integration technology	Replicated Data
From Asset Manager to Service Manager	Connect-It	• **amsm-ppt.scn:** • Companies • Vendors • Locations • Departments • Contacts • Models • Stock rooms • Links between CIs and Contact, Model, Vendor, Location, Department • **amsm-ci-ppt-link.scn:** • Links between CIs and Contact, Model, Vendor, Location, Department
From Service Manager to Asset Manager	Connect-It	• **smam-ppt.scn:** Companies • Vendors • Locations • Departments • Contacts • Models • Stock rooms • **smam-wo.scn:** • Changes and tasks

Note: This integration doesn't synchronize "IT Equipment" CIs between Asset Manager and Service Manager, such as desktops, laptops, servers, network components, and so on.

After synchronization, this integration provides Asset Manager (AM) users with the Service Manager (SM) interface that can be used to carry out requests. With this integration, you can:

- Describe in AM the catalog to publish in SM self-service
- Export AM models as categories in the SM catalog

- Export AM requests, products, and off-catalog products to the SM catalog
- Create requests using the SM catalog self-service
- View the status of your requests in AM

Set up the integration

For detailed instructions, see the Asset Manager Help Center.

Related topics

Reference Data Synchronization via Connect-It (AM ↔ SM)
Employee Self-Service Catalog for Asset Manager (AM - SM)

1.12. ALM Octane

ALM Octane is a web-based application lifecycle management platform that enables teams to collaborate easily, manage the product delivery pipeline, and visualize the impact of changes. In the integration between Service Manager (SM) and ALM Octane in your release lifecycle, Service Manager acts as the service provider and tightly couples defect/feature management in ALM Octane with the Change Management capabilities in SM. ALM Octane provides REST API and enables the users to retrieve and perform CRUD (create, read, update, delete) operations on the entities available in the Octane graphical user interface. The record creation and update between Service Manager and ALM Octane are synchronized through Integration Manager (SMIS) tasks in Service Manager.

This integration synchronizes SM Change to ALM Octane defect/feature for record creation. The following scenarios are included:

- SM Change → ALM Octane defect
- SM Change → ALM Octane feature

After record creation, this integration implements bidirectional record updates seamlessly between SM Change and ALM Octane defect/feature.

The overall intent of this integration is:

- Leverages SM's position as the service provider and application with the most intimate contact with the customer
- Efficiently tracks, reconciles, and resolves the demands placed within SM

Sample use case

The following typical use cases shed some light on how you can use this integration in your organization.

SM Change → ALM Octane defect

A business owner creates a Normal Change request in Service Manager and later the Normal Change enters the **Build and Test** phase. On the **Octane** tab of the Change request, this business owner selects the **Change To Defect** action, and then selects the ALM Octane workspace. After saving these updates, the system synchronizes the Change request to a defect in ALM Octane. Developers and QA personnel can begin working on the defect in ALM Octane, in which the key information is synchronized from Service Manager to ALM Octane.

The integration administrator has the responsibility of determining the key information and specifying the information in the field mapping (using the Integration Manager) in order for the business owner to view updated (scheduled) information in Service Manager. The information includes the Change request's title, description, severity, Octane Phase, and so on.

For complete use cases, see Synchronize Service Manager Change to ALM Octane defect.

SM Change → ALM Octane feature

A business owner creates a Normal Change request in Service Manager and later the Normal Change enters the **Build and Test** phase. On the **Octane** tab of the Change request, this business owner selects the **Change To Feature** action, and then selects the ALM Octane workspace. After saving these updates, the system synchronizes the Change request to a feature in ALM Octane. Developers and QA personnel can begin working on the feature in ALM Octane, in which the key information is synchronized from Service Manager to ALM Octane.

Similar to the SM Change → ALM Octane defect use case, the integration administrator has the responsibility of determining the key information and specifying the information in the field mapping (using the Integration Manager) in order for the business owner to view updated (scheduled) information in Service Manager. The information includes the Change request's title, description, severity, Octane Phase, and so on.

For complete use cases, see Synchronize Service Manager Change to ALM Octane feature.

1.12.1. Configure the settings in ALM Octane

User role: ALM Octane Administrator

See the following prerequisites that you must follow when setting up the integration between Service Manager and ALM Octane. Ensure that you have working knowledge about ALM Octane before you proceed.

Detailed instructions about ALM Octane administration are outside the scope of the OpenText product suite and are hence not covered. For more information, see the ALM Octane documentation.

Set up API access

You need to set up API access for Service Manager and grant Service Manager registered access keys (Client ID and a Client secret) so that it can access ALM Octane. In the SM-Octane integration, Service Manager uses the Client ID and Client secret for authentication when synchronizing information with ALM Octane.

Follow these steps:

1. Sign in to ALM Octane as a system administrator.
2. Click **Settings** > **Spaces** > **API ACCESS**, and then click **+ API Access**.
3. Enter an appropriate name for the access.
4. (Optional) Set an expiration date for the API key, and then enter a description if needed.
5. Select the predefined roles of **API** and other user roles as required (for example, **Team Member**) for Service Manager to use when accessing ALM Octane.
   Select the relevant workspaces for each role. If additional relevant workspaces are created later, you will need to manually assign them.
6. Click **Add**. A dialog displays with a Client ID and a Client secret.
   Click **Copy** to copy these keys to the clipboard so you can use them when configuring Service Manager that needs to access ALM Octane.
7. Click **OK**. The access ID is added to the grid and its status is active.

Add custom fields

You need to manually add the **sm_id** and the **sm_table** user-defined fields (UDFs) to ALM Octane so that the defects in ALM Octane can display the corresponding Service Manager Change ticket ID and the name of the module table (cm3r). See the following screenshot as a sample display of these two fields in ALM Octane:

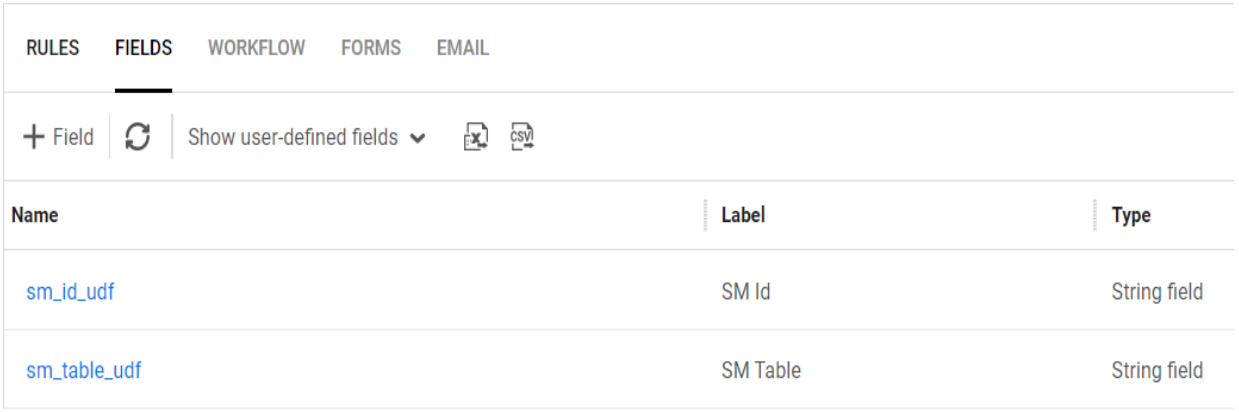


Follow these steps:

1. Sign in to ALM Octane as a system administrator.
2. Click **Settings** >**Spaces**, and then select the shared space or the workspace where you want to create user-defined fields.
3. Click the **ENTITIES** tab.
4. In the Entities list on the left side of the pane, select **Defect**.
5. Click the **FIELDS** tab, and then click **+ Field** to add a user-defined field.
6. Enter the following relevant values.

Name	Description
Name	A unique name that APIs will use to access this field. No uppercase letters or punctuation allowed. ALM Octane automatically adds the suffix _udf to the name of the field. Enter **sm_id** in this field. **Note:** UDFs can only be defined in English even if ALM Octane has been localized for a specific language. Labels, however, can be specified in other languages.
Label	A unique name for the field as you want it displayed in the UI. You can enter English, non-English, and special characters.
Type	Select **String field** from the drop-down list.
Description	(Optional) Enter a description for the user-defined field. **Note:** The description text remains the same for all ALM Octane display languages.

7. Click **Add**. The **sm_id_udf** field is available in ALM Octane just like other predefined system fields.
8. Continue to follow *step 5* and *step 6* to add the **sm_table_udf** field. See the following screenshot as an example:



9. In the Entities list on the left side of the pane, select **Feature**.
10. Continue to follow *step 5* to *step 8* to add both the **sm_id_udf** field and the **sm_table_udf** field for ALM Octane features.

Add a rule to restrict the writing permission to the custom fields

You can define a rule that triggers actions in ALM Octane when certain conditions are met. In this task, you need to manually add a rule to restrict the writing permission to both the **sm_id** and the **sm_table** user-defined fields. Both fields are read-only to all ALM Octane users and can be updated by the background API only.

Follow these steps:

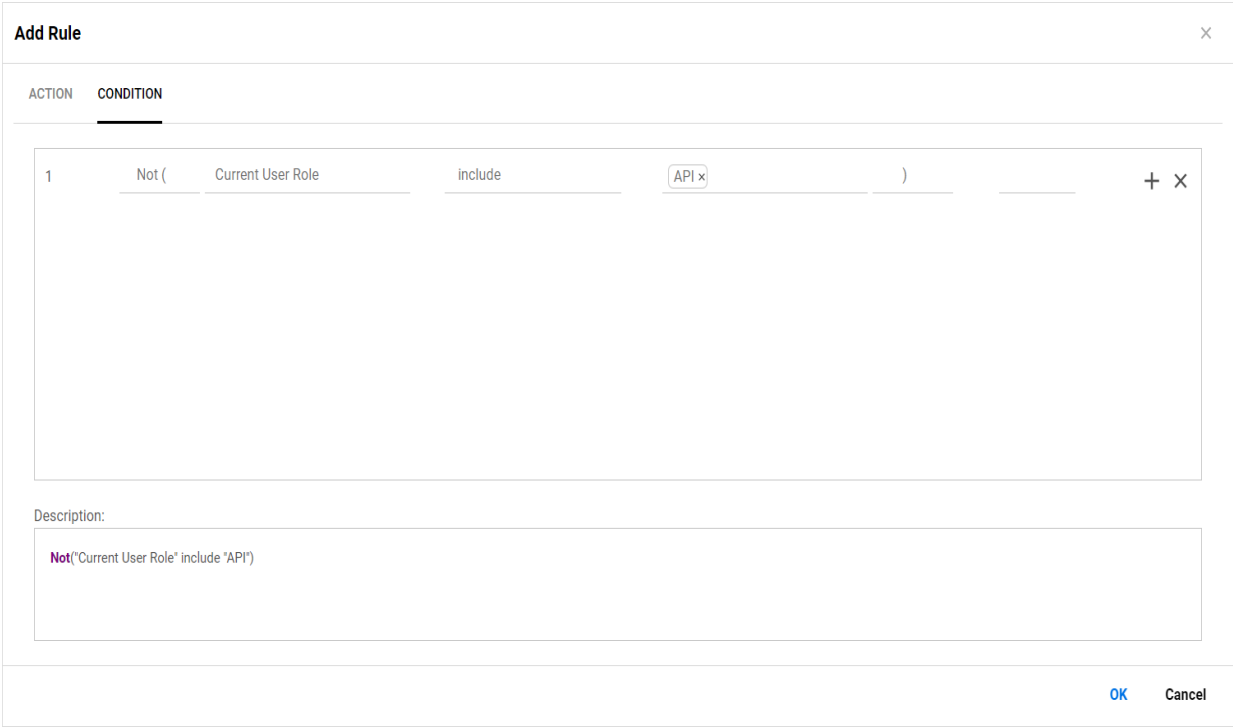
1. Sign in to ALM Octane as a system administrator.
2. Click **Settings** >**Spaces**, and then select the shared space or the workspace where you want to create user-defined fields.
3. Click the **ENTITIES** tab.
4. In the Entities list on the left side of the pane, select **Defect**.
5. On the **RULES** tab, click **+** to add a rule.
6. On the **ACTION** tab, enter the following relevant values.

Name	Description
Status	Status of the rule.
Action	When conditions are met, the rule's action is performed. Select **Make read-only** from the drop-down list.
Field	Select both the **sm_id** and the **sm_table** fields.
Remark	(Optional) Enter a remark that describes the rule.

7. On the **CONDITION** tab, apply the following condition.

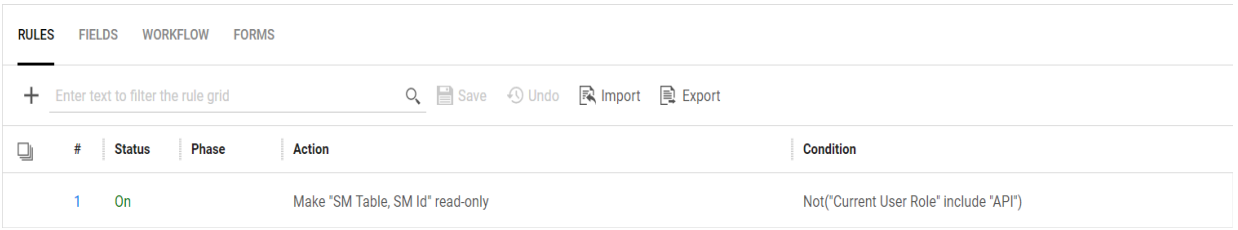
```
Not("Current User Role" include "API")
```

See the following screenshot as an example:



In this example, both fields are read-only to all ALM Octane users and can be updated by the background API only. You can add your own rule as required to restrict the writing permission to these two fields.

8. Click **OK**. The new rule is available in ALM Octane just like other predefined rules. See the following screenshot as an example:



9. In the Entities list on the left side of the pane, select **Feature**.
10. Continue to follow *step 5* to *step 8* to add the rule for both the **sm_id** and the **sm_table** fields for ALM Octane features.

1.12.2. Configure the settings in Service Manager

User role: Service Manager System Administrator

You must complete the following tasks from your Service Manager system to support the integration.

Add an integration instance

To set up ALM Octane integration, you must add and enable an instance of the ALM Octane integration in Service Manager Integration Manager.

To add and enable an ALM Octane integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager** to open Integration Instance Manager.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **SMOctane** from the Integration Template list, and then select **Import Mapping**.
4. Click **Next**. The Integration Instance Information page opens.
5. Complete the following settings on the Integration Instance Information page as necessary.

Setting	Value	Description
Name (required)	User-defined.	Name of the integration instance. Default: SMOctane. **Note:** If your ALM Octane has multiple workspaces to be integrated, you can add more integration instances as necessary. Make sure to use a unique name for each SM-Octane integration instance.
Version (required)	1.0	The version number of the integration.
Interval Time (s) (required)	User-defined.	The interval time that the integration background schedule runs. For example, if you set the value to 300, the background process tries to execute the SMIS integration task every 5 minutes. Default: 60.
Max Retry Times	User-defined.	Maximum allowed number of retries if the background scheduler fails to run. Default: 5. **Note:** The ALM Octane integration doesn't use this option.

Setting	Value	Description
Log Level	Select a level from the drop-down list.	Level of diagnostic information that the Service Manager server logs to the log file directory. The possible log levels are DEBUG, INFO (default), WARNING, ERROR, and OFF. Default: INFO.
Log File Directory	Specify a directory.	This is a directory that exists on the Service Manager server host where log files are stored. The log includes information for receiving and processing messages. For example: c:\Octane_logs. If this field is empty, the system uses the Service Manager sever log (sm.log) for ALM Octane integration.

Setting	Value	Description
Shared Scheduler	User-defined.	The scheduler that the integration instance uses to schedule background processes. Multiple integration instances can use a shared scheduler. When you create an integration template, you can define a default shared scheduler for the instances that are based on the template. All instances that are based on this template can use the same shared scheduler. However, if you create an instance that uses a different scheduler, SMIS creates a new scheduler accordingly. If this field is empty, the system uses <Octane integration name + instance ID> for ALM Octane integration.
Run at system startup	True or false.	Automatically enable or disable the integration instance when the Service Manager server starts.
Description	User-defined.	Description of the integration instance.

6. Click **Next**. The Integration Instance Parameters page opens.
7. Complete the following settings on the **Workspace** tab as necessary.

Setting	Value	Description
Name	User-defined.	Name of the workspace.
Sharespace	Paste the sharespace ID.	The ID of the sharespace. For example, if the ALM Octane URL is https://*<Octane server name>*:*<port>*/ui/?p=1001/1002#/product-overview/hierarchy/defects, the ID of the sharespace is 1001.
Workspace	Paste the workspace ID.	The ID of the workspace. For example, if the ALM Octane URL is https://*<Octane server name>*:*<port>*/ui/?p=1001/1002#/product-overview/hierarchy/defects, the ID of the workspace is 1002.

Setting	Value	Description
Base URL	Paste the base URL of the ALM Octane server.	The base URL of the ALM Octane server. The URL has the following format: https://*<Octane server name>*:*<port>* Replace *<Octane server name>* with the IP address or fully-qualified domain name (FQDN) of the ALM Octane server, and replace *<port>* with the communications port the ALM Octane server uses. **Note** To set up an HTTPS connection, make sure to add the ALM Octane server root CA certificate to the SM trust store so as to verify the connection. See the *Add the ALM Octane root CA certificate to the SM trust store* section below.

Setting	Value	Description
Client Id	Paste a client ID string.	The client ID you received when setting up API access for Service Manager.
Client Secret	Paste a client secret string.	The client secret string you received when setting up API access for Service Manager.
Ahead of Octane Time (s)	User-defined.	The time difference between the Service Manager server and the ALM Octane server. The value can be either positive or negative. For example, if the SM server time is 12:02:00 and the ALM Octane server time is 12:00:00, the value is 120.

8. Select the options on the **Modules** tab as necessary. Review and update the Octane phases under the selected options. Service Manager stops updating an ALM Octane defect or feature if it is in any of the listed phases.

Option	Value	Description
Enable SM Change to Octane Defect	User-selected.	Select whether to enable the synchronization from SM Change to ALM Octane Defect.
Enable SM Change to Octane Feature	User-selected.	Select whether to enable the synchronization from SM Change to ALM Octane Feature.

9. Click **Verify Connection** to verify the connection settings.
10. Click **Next**. The Integration Instance Fields page opens and displays the out-of-the-box fields available for both SM and ALM Octane of the integration. You can review, update, or add more fields as required.

11. If you have selected more than one module in *step 8*, the Select Module page opens. You can either double-click a module or select a module from the list and then click **Open Field Mapping** to open the Integration Instance Fields page. Otherwise, if you have selected only one module in *step 8*, the Integration Instance Fields page opens directly.

    For your selected module, the system displays the out-of-the-box fields available for both SM and ALM Octane of the integration. You can review, update, or add more fields as required.

12. Click **Next**. The Integration Instance Field Mappings page opens.

13. Review, update, or add more fields to the **Field Mapping** tab as necessary. For more information, see Manage mappings.

    To map fields:

    1. Select an SM field and an ALM Octane endpoint field.
    2. Select a direction in the **Direction** column.
    3. (Optional) If an SM field to be mapped has multiple value selections (such as Impact), specify a name in the **Value Mapping Group** column and then configure the value mappings on the **Value Mapping** tab.

14. Review, update, or add more fields to the **Value Mapping** tab as necessary. For more information, see Manage mappings.

    To map values:

    1. Enter a value mapping group name as you have specified on the **Field Mapping** tab.
    2. Enumerate the value mapping pairs between an SM field and an ALM Octane endpoint field.
    3. Set the condition to true.

15. Click **Finish**. Service Manager creates the instance.

16. Enable the integration instance. A processor for the ALM Octane integration instance now runs in the background.



Note

If your ALM Octane has multiple workspaces to be integrated, you can add more integration instances as necessary.

Add the ALM Octane root CA certificate to the SM trust store

If you are setting up an HTTPS connection, follow these steps to add the ALM Octane server root CA certificate to the SM trust store by using the certificate generation toolkit.

1. Export ALM Octane root CA certification to a file. For example, octaneca.crt.

2. Import octaneca.crt to the SM trust store. See the following sample commands.

   Non-FIPS mode:

   ```
   import-cert.bat --alias octaneca --file certs/octaneca.crt --
   keystore certs/cacerts --storepass changeit
   ```

   FIPS mode:

   ```
   import-cert.bat --alias octaneca --file certs/octaneca.crt --
   keystore certs/cacerts --storetype bcfks --storepass changeit
   ```

3. Copy cacerts to the `<SM_SERVER_HOME>/RUN` folder.

4. Update sm.ini by adding the following lines:

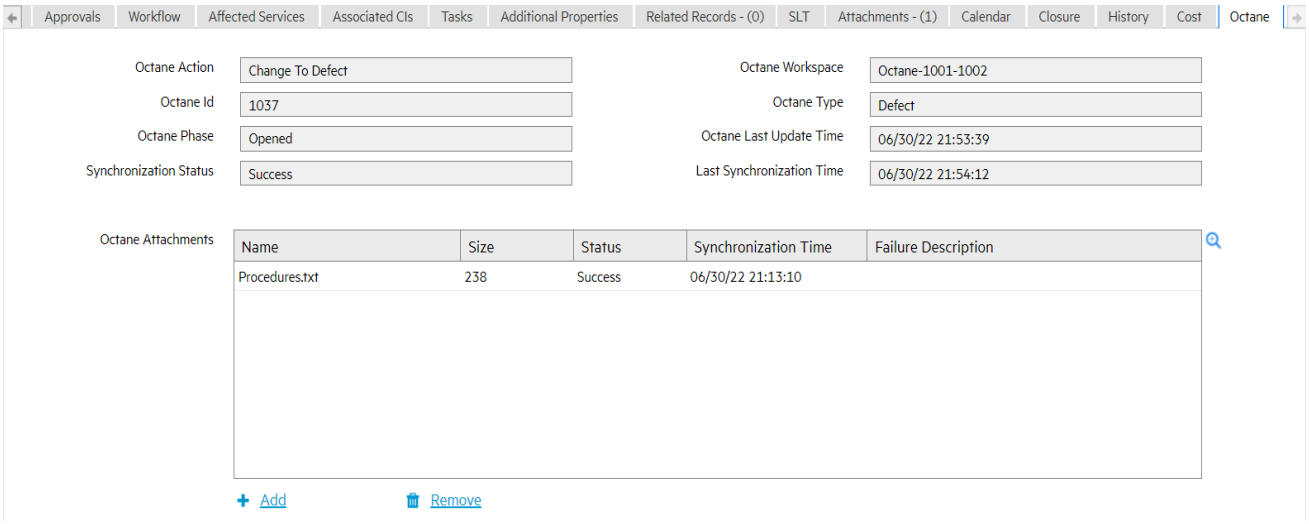
   ```
   truststoreFile:cacerts
   truststorePass:changeit
   ```

5. Restart the SM server.

(Optional) Tailor the integration for more Change categories and workflow phases

The out-of-the-box SM-Octane integration synchronizes a Normal Change request that is in the **Build and Test** phase in Service Manager to an ALM Octane defect/feature. You can tailor the integration to work with more Change category and more workflow phases so as to increase the usability and flexibility of this integration. Regardless of the Change request's category, all you need to do is to add the **Octane** tab to a target workflow phase of a Change request.

In the SM-Octane integration, Service Manager uses the **Octane** tab in a Change record to define the Octane synchronization action and Octane workspace. This tab also displays a series of synchronization information, including Octane Id, Octane Type, Octane Phase, Octane Last Update Time, and so on. Meanwhile, you can also select the attachments of a record on the **Octane** tab so as to synchronize them to Octane. The following screenshot illustrates the Octane tab after Service Manager synchronizes a Change record to an ALM Octane Defect.



To display the Octane tab in any workflow phase of a Change request, you need to manually add this tab to the corresponding form of that phase. Refer to the following steps for some guidelines.

1. Click **Tailoring** > **Forms Designer**.
2. Search for the corresponding form of a phase. For example, to display the Octane tab when a Change enters the Validation phase, search for the **chm.normal.registration** form.
3. Add a Notebook Tab widget to your Change format, and then set the following property of the Notebook Tab widget.

Property	Value
Visible Condition	[$G.octane.enable]

4. Add a subform to the Notebook Tab widget, and then set the following property of the subform control.

Property	Value
Format	octane.info

5. Save your changes.

1.12.3. Manage mappings

User role: Service Manager System Administrator, ALM Octane System Administrator

In the Service Manager and ALM Octane integration, a mapping consists of field mapping and value mapping.

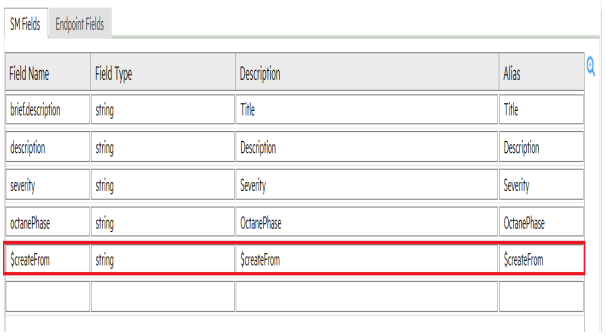
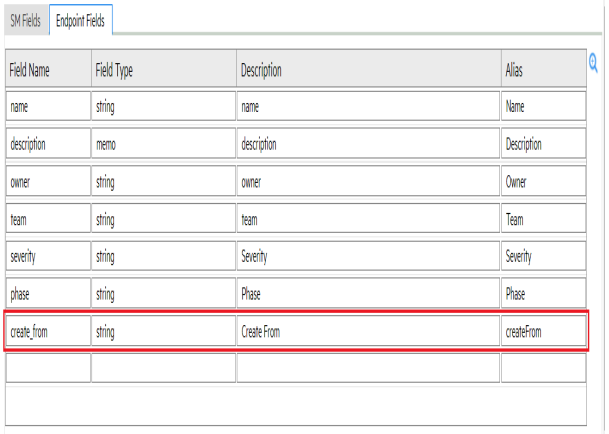
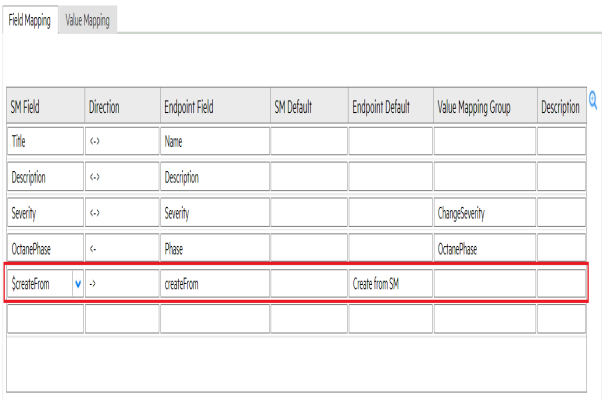
Field Mapping

The **Field Mapping** tab defines the mapping between ALM Octane fields and SM fields.

Field Mapping columns

The following table describes the columns on the **Field Mapping** tab.

Column Name	Description
SM Field	The name of a field in Service Manager, and can be selected from the list.
Direction	Indicates the mapping direction of a field mapping, and has only three values to select from. • →: from SM Field to ALM Octane Field. • ←: from ALM Octane Field to SM Field. • ↔: in both directions.
Endpoint Field	The name of a field in ALM Octane, and can be selected from the list.
SM Default	Defines the default value of the SM Field if there is no value or there is a constant in the corresponding ALM Octane Field when mapping an ALM Octane Field to an SM Field. **Note** If the value is a date type, the format must resemble either `2022-06-18T00:00:00Z` or `2022-06-18T00:00:00+08:00`.

Column Name	Description
Endpoint Default	Defines the default value of an ALM Octane Field if there is no value or there is a constant in the corresponding SM Field when mapping an SM Field to an ALM Octane Field. **Example 1:** Suppose you have defined the **create_from** field in Octane and then configure the field mapping as follows.  SM Fields | Endpoint Fields Field Name | Field Type | Description | Alias briefdescription | string | Title | Title description | string | Description | Description severity | string | Severity | Severity octanePhase | string | OctanePhase | OctanePhase $createFrom | string | $createFrom | $createFrom  SM Fields | Endpoint Fields Field Name | Field Type | Description | Alias name | string | name | Name description | memo | description | Description owner | string | owner | Owner team | string | team | Team severity | string | Severity | Severity phase | string | Phase | Phase create_from | string | Create From | createFrom  Field Mapping | Value Mapping SM Field | Direction | Endpoint Field | SM Default | Endpoint Default | Value Mapping Group | Description Title | <-> | Name Description | <-> | Description Severity | <-> | Severity | ChangeSeverity OctanePhase | <- | Phase | OctanePhase $createFrom | -> | createFrom | Create from SM ALM Octane automatically fills the **Create From** field with the value of **Create from SM**

Column Name	Description
	when creating this defect from an SM Change. **Example 2:** If you enter a constant `$detectedBy` in **SM Field** and then enter `{"type":"workspace_user","id":"1001"}` in **Endpoint Default**, ALM Octane automatically fills the **Detected by** field with the name of a workspace user whose ID is 1001 when creating this defect from an SM Change. **Note** If the value is a date type, the format must resemble either `2022-06-18T00:00:00Z` or `2022-06-18T00:00:00+08:00`.
Value Mapping Group	The name of a value mapping group to which an ALM Octane Field and an SM Field belong. A value mapping group contains a collection of enumerated ALM Octane and SM value pairs. The Service Manager value and ALM Octane value will be mapped according to the configuration on the **Value Mapping** tab.
Description	Enter a text description for this field mapping.

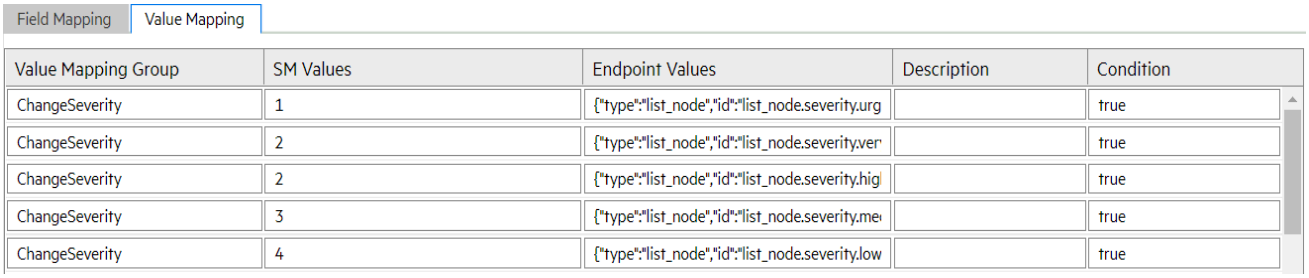
Value Mapping

The **Value Mapping** tab defines the value mapping between ALM Octane and SM for those fields that have multiple values.

The following describes the columns on this tab.

Column Name	Description
Value Mapping Group	The name of a value mapping group. A value mapping group contains a collection of enumerated ALM Octane/SM value pairs.
SM Values	The values of the Service Manager fields.
Endpoint Values	The values of the ALM Octane fields. For more information about how to get the endpoint values, see the *Get the endpoint values in ALM Octane* section below.
Description	Description of the value mapping.
Condition	SMIS enables you to configure conditions for value mappings. You can set the **Condition** field of a value mapping entry to one of the following strings: • `true` : The value mapping defined in this entry is always effective. • `false` : The value mapping defined in this entry is always ineffective. • A string that uses placeholder: The value mapping defined in this entry is effective only when the condition is met. For more information about how to use place holder, see Use placeholders. For more information about how to configure condition, see Configure a condition.

For example, below is a value mapping named **ChangeSeverity**, and there are five mapping entries for this value mapping group.



The name **ChangeSeverity** can be filled in the **Value Mapping Group** column on the **Field Mapping** tab.

Field Mapping | Value Mapping

SM Field	Direction	Endpoint Field	SM Default	Endpoint Default	Value Mapping Group	Description
Title	<->	Name				
Description	<->	Description				
Severity	<->	Severity			ChangeSeverity	
OctanePhase	<-	Phase			OctanePhase	

This means when mapping the Severity field between a Service Manager Change and an ALM Octane Defect, if the value of the Severity field in Service Manager is 3, the value `3` will be mapped to `medium` for the Severity field in ALM Octane, and vice versa.

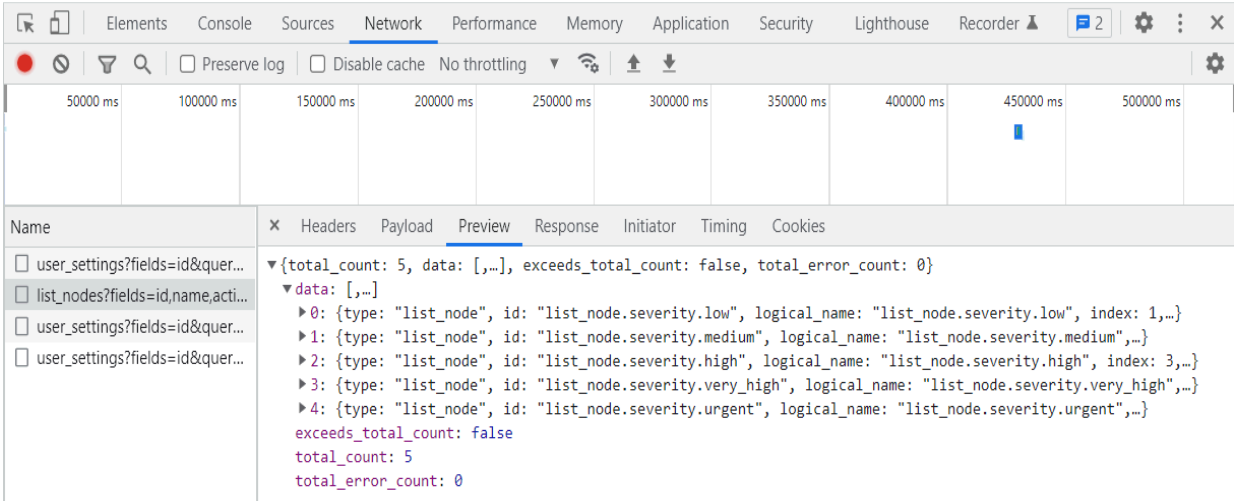
Get the endpoint values in ALM Octane

The following example uses Google Chrome as an example. Follow these steps to get the endpoint values in ALM Octane:

1. Click **Settings** > **Spaces** > **LISTS**, and then select a list. For example, Severity.

   ALM Octane lists contain values for list type fields. For example, the Severity list contains a list of possible severity values, including Low, Medium, High, Very High, and Critical.

2. Open the **Web Developer Tools** in your browser, and then click the **Network** tab.
3. In the **Name** list on the left side of the pane, select the name that begins with `list_nodes?fields=...`.
4. Click the **Preview** tab on the right side of the pane, and then expand the **data** node to display all data on the list. See the following screenshot as an example:



   The value for the EndPoint Values field includes both the `type` and `id` sections. For example, `{"type":"list_node","id":"list_node.severity.urgent"}`.

1.12.4. Verify the integration

User role: Service Manager system administrator

SM Change → ALM Octane defect/feature

The following example verifies the integration by synchronizing an SM Change to an ALM Octane defect/feature.

1. Synchronize an SM Change to an ALM Octane defect/feature.
    1. Navigate to Change Management, and then open a Normal Change record that is in the **Build and Test** phase.
    2. Click the **Octane** tab.
    3. To synchronize an SM Change to an ALM Octane defect, select **Change To Defect** in the **Octane Action** drop-down list, and then select your ALM Octane workspace in the **Octane Workspace** drop-down list.

       To synchronize an SM Change to an ALM Octane feature, select **Change To Feature** in the **Octane Action** drop-down list, and then select your ALM Octane workspace in the **Octane Workspace** drop-down list.
    4. Click **Save** and then wait for the background schedule to synchronize the Change record to ALM Octane. After synchronization, the Octane Id, Octane Type, Octane Phase, Octane Last Update Time, Synchronization Status, and Last Synchronization Time fields on the **Octane** tab have read-only values that are populated automatically.

Field	Description
Octane Id	The defect ID in Octane.
Octane Type	Defect.
Octane Phase	The defect's or feature's phase in Octane.
Octane Last Update Time	The time when the defect/feature is created in ALM Octane for the first-time synchronization. **Note:** For the later synchronizations, this field displays the last time when the defect/feature is updated by a user in ALM Octane.
Synchronization Status	The synchronization status.
Last Synchronization Time	The last synchronization time between SM and ALM Octane.

2. Update some fields, add attachments, and then perform synchronization again from the SM Change to the ALM Octane defect/feature.
    1. Open the Change record again and then click the **Change Details** tab. Update the title, description, and severity information.
    2. Click the **Activities** tab. Select **Update to external service** in the **New Update Type** drop-down list, and then enter some notes in the **New Update** field.
    3. Click the **Attachments** tab, and then add some attachments to the Change record. Next, click the **Octane** tab and then click **Add** to select the attachments from the current Change record for Octane synchronization.
    4. Click **Save** and then wait for the next synchronization.
3. View the synchronization result in the ALM Octane defect/feature.
    1. Click the **View in Octane** button. The browser opens a new tab and then displays the ALM Octane login page.
    2. Sign in to ALM Octane and the system opens the defect/feature record automatically. Check that your updates to the title, description, and severity in the SM Change record are synchronized to the corresponding fields in the ALM Octane defect/feature record. Meanwhile, your notes in the SM **New Update** field now

appear as comments in the ALM Octane defect/feature record. Your selected attachments in the SM Change record now appear as the ALM Octane defect/feature record's attachments as well.

Note

The synchronization process for attachments and new updates is unidirectional and incremental. That is, you can only synchronize attachments and new updates from SM to ALM Octane only. Meanwhile, if you remove attachments and new updates from SM after synchronization, the attachments and comments in ALM Octane are not affected.

For a complete use case, see Synchronize Service Manager Change to ALM Octane defect and Synchronize Service Manager Change to ALM Octane feature.

1.13. Microsoft Teams

Microsoft Teams is a chat-based workspace in Microsoft 365 that provides built-in access to team-specific resources. The integration between Service Manager and Microsoft Teams is a simplified collaboration model that not only enables people to exchange information in a rapid way but also provides automated procedures to create and manage tickets.

Compared to traditional collaboration ways, the Teams integration provides the following highlights:

- Ensure speedy resolution of critical issues by allowing IT agents to open Teams conversations directly from a major Incident.
- Reduce manual work, improve productivity and efficiency by automating the creation and management of Service Manager tickets directly in Microsoft Teams through an agent bot.
- Reduce IT support costs by providing knowledge article links and automating the creation of Service Desk tickets through a virtual agent.

Sample use cases

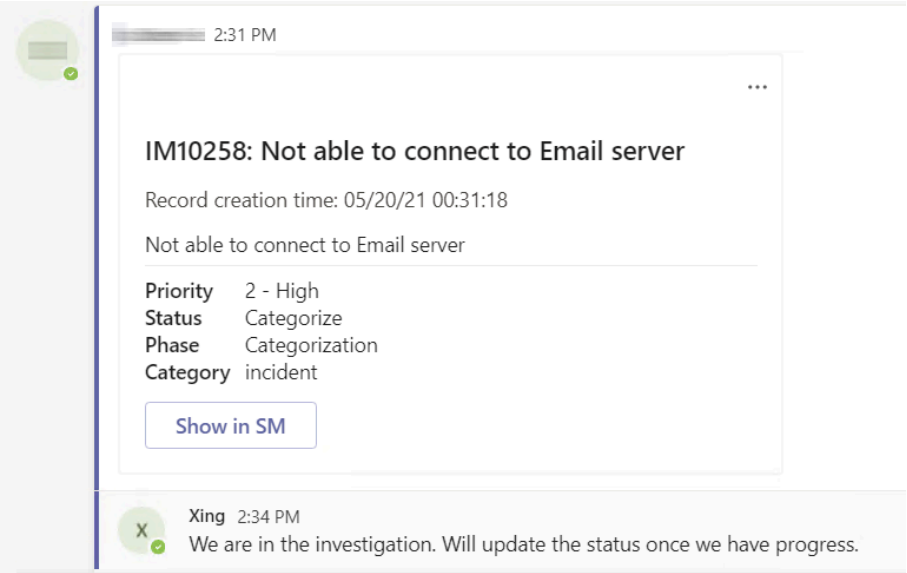
The following typical use cases shed some light on how you can use this integration in your organization. Note that this isn't an exhaustive list, and the integration supports many more use cases.

Major Incident

This use case allows the agents to collaborate on Teams with war room experience when handling a major Incident and ensures speedy resolution of critical issues.

This integration enables an IT agent to initiate a Teams conversation from a major Incident record and helps this agent directly contact the right people. By viewing the Incident details (such as Incident ID, Incident Title, and Description), stakeholders including the Incident Coordinator and Incident Manager and other IT agents can collaborate on its resolution in real-time in the Teams channel. In the meantime, the system synchronizes all discussions to the Incident record, which allows other Incident analysts to reuse the solutions later.

Sample screenshot:



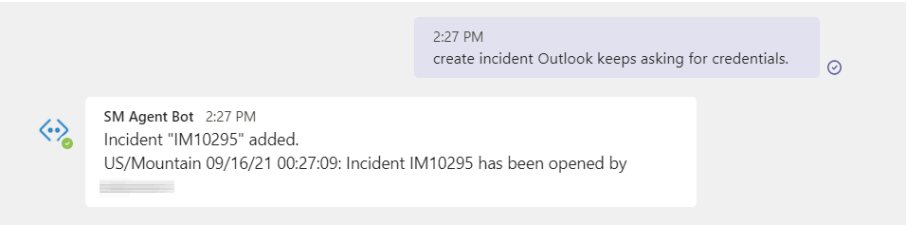
For a complete use case, see Collaborate on Teams with war room experience.

Agent bot

This use case improves productivity by allowing Incident stakeholders to ask an embedded agent bot in Microsoft Teams to open Service Manager tickets.

This integration enables an IT agent to initiate a Teams conversation with the agent bot. After sending a command to the bot, this agent can directly open an Incident record from Teams. By following the Incident workflows, the IT agent can update, resolve, and close this Incident - all through a conversation with the bot in Microsoft Teams without logging on to Service Manager. In the meantime, the system tracks all operations on the **Activity** tab for later reference.

Sample screenshot:



For a complete use case, see Interact with Service Manager by using agent bot.

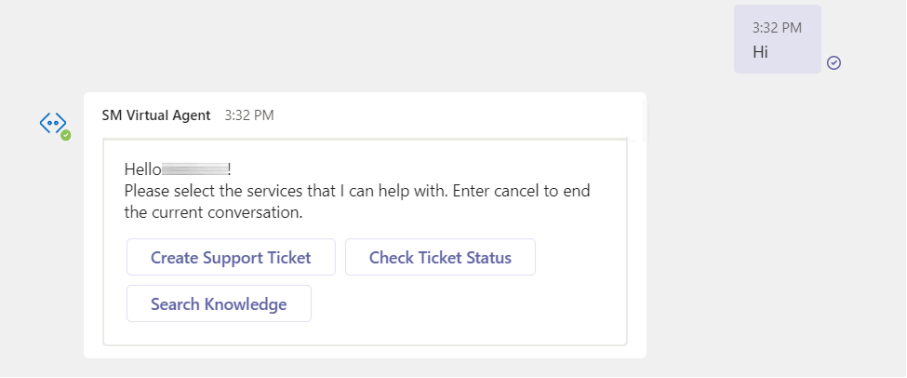
Virtual agent

This use case reduces IT support costs by allowing a Service Manager employee self-service (ESS) user to view knowledge articles and then open a Service Desk ticket through an embedded virtual agent in Microsoft Teams.

This integration enables an ESS user to initiate a Teams conversation with the virtual agent. After describing an issue, the virtual agent responds with several knowledge articles after performing a Smart Search in Service Manager. The ESS user can browse these knowledge articles so as to find a solution. In case none of these articles serve as the best solution, this

user can ask the virtual agent to create a Service Desk ticket. Later, this user can check the Service Desk ticket's status through the virtual agent as well.

Sample screenshot:

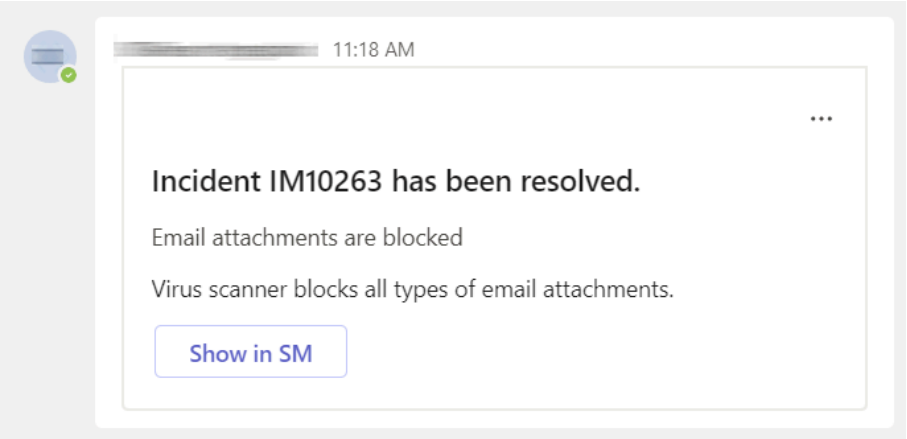


For a complete use case, see Interact with Service Manager by using virtual agent.

Teams notification

This use case allows Service Manager to automatically route a new notification to Microsoft Teams as a timely reminder when an IT agent updates a record or requires further communication. For example, when the status of an Incident becomes Resolved, the contact of this Incident will receive a message on Teams that claims `Incident IM##### has been resolved`.

Sample screenshot:



Note

Descriptions in this section are mainly based on Microsoft Teams version 1.5.00 at the time of this writing. As Teams evolves, we may not provide immediate support to the latest Teams features and certain functions of this integration that are related to these new features may be affected. We will provide continuous improvement to this integration to keep pace with the Teams evolvements.

Related topics

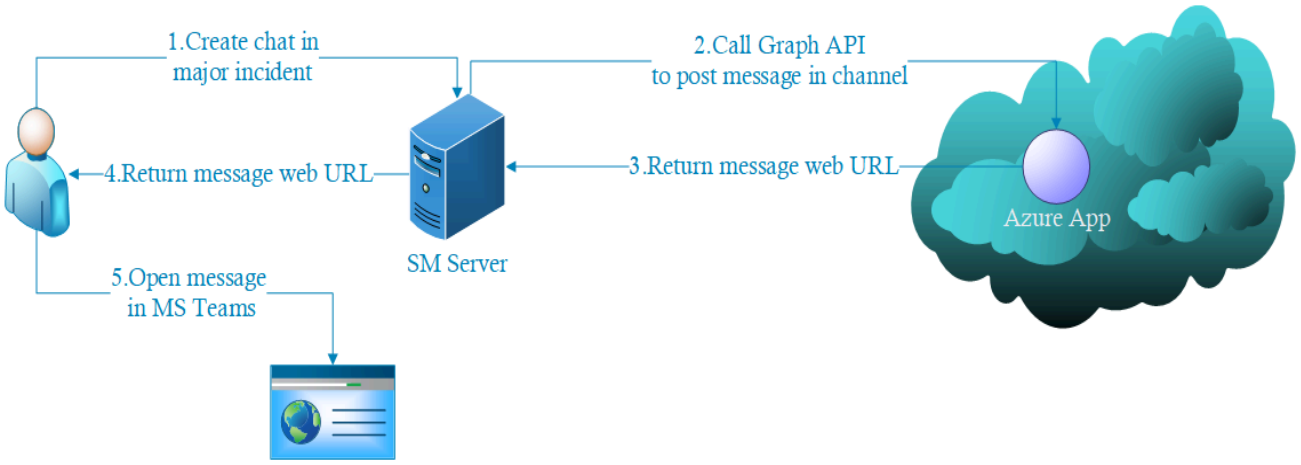
Microsoft Teams Integration
Troubleshoot Teams integration

1.13.1. Set up the integration for war room experience

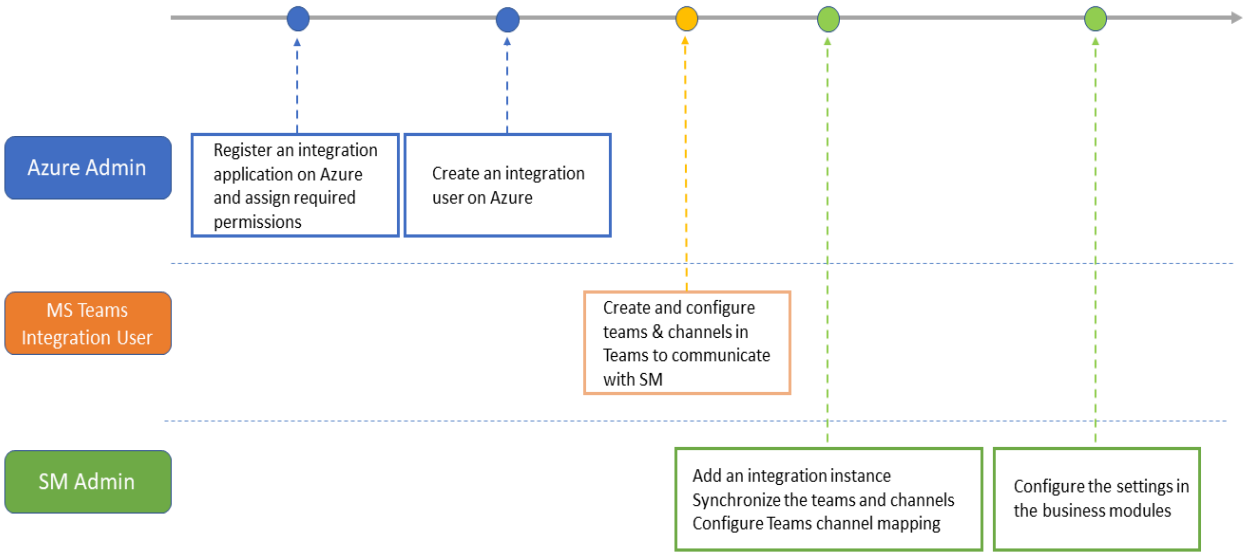
By setting up the integration for war room experience, IT agents can collaborate in real-time (or anytime) in Teams channels when handling a major Incident, an Interaction, a Problem, a Change, or a Request.

We use Microsoft Graph which exposes REST APIs and client libraries to access data on Microsoft Teams. After an IT agent creates a conversation in a supported Service Manager record, the system calls Microsoft Graph API through your registered Azure application to create a message and then returns the message's web URL to the Service Manager server. After that, the Service Manager server opens the message's web URL in Microsoft Teams and exposes the conversation to the IT agent.

See the following illustration for Teams chat creation workflow:



The following content describes how to set up the integration between Service Manager and Microsoft Teams. The following diagram highlights the steps.



Configure the settings on Azure

User role: Azure Administrator

See the following for prerequisites that you may need to follow when setting up the integration between Service Manager and Microsoft Teams. Ensure that you have working knowledge about the Microsoft 365 Developer Program and Azure Entra admin center before you proceed.

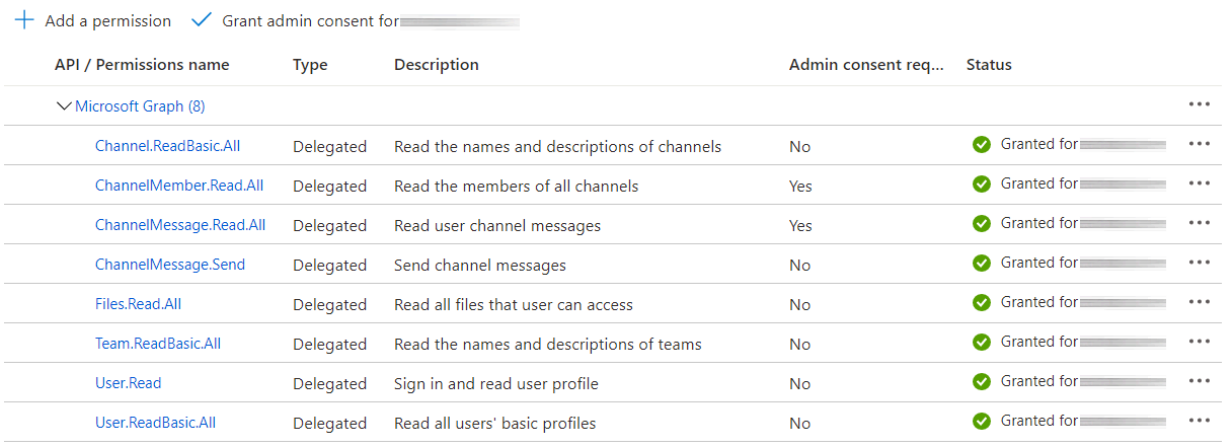
Detailed instructions about Microsoft Azure cloud services are outside the scope of the OpenText product suite and are hence not covered. For more information, see Azure documentation.

Register an application

Register a new Entra ID Application. Here are some guidelines for your reference when registering an Application on Azure:

1. Sign in to the Azure Entra admin center with an Azure account that has the Global administrator role.
2. Enter a name for your Teams integration. Under Supported Account Types, select **Accounts in this organizational directory only**.
3. Under Advanced settings, set **Allow public client flows** to **No**.
4. Copy and save the values of the Application (client) ID and Directory (tenant) ID for later use.
5. Click **App Registrations** > ***<Name of your application>*** > **Certificates & secrets** to add a new client secret.

   Copy and save the values of the client secret for later use.
6. If you decide to use **Authorization Code** as **Authorization Grant Type**, you need to click **App Registrations** > ***<Name of your application>*** **> Manage > Authentication** and then add `https://<SM Webtier Host>/<Context Path>/oauth/callback` as the **Web Redirect URIs**. Where `<SM Webtier Host>/<Context Path>` should match the Web Server URL defined in System Information Record. For example: `https://myserver.mydomain.com:myport/webtier/oauth/callback`
7. Click **App Registrations** > ***<Name of your application>*** > **API permissions** and then add the following permissions.



+ Add a permission ✓ Grant admin consent for [redacted]

API / Permissions name	Type	Description	Admin consent req...	Status
Microsoft Graph (8)				
Channel.ReadBasic.All	Delegated	Read the names and descriptions of channels	No	Granted for [redacted]
ChannelMember.Read.All	Delegated	Read the members of all channels	Yes	Granted for [redacted]
ChannelMessage.Read.All	Delegated	Read user channel messages	Yes	Granted for [redacted]
ChannelMessage.Send	Delegated	Send channel messages	No	Granted for [redacted]
Files.Read.All	Delegated	Read all files that user can access	No	Granted for [redacted]
Team.ReadBasic.All	Delegated	Read the names and descriptions of teams	No	Granted for [redacted]
User.Read	Delegated	Sign in and read user profile	No	Granted for [redacted]
User.ReadBasic.All	Delegated	Read all users' basic profiles	No	Granted for [redacted]

Note

- You must grant admin consent to the **ChannelMessage.Read.All** permission, which synchronizes the conversation messages between Teams and Service Manager. If you don't need to synchronize messages, you can skip this permission. For more information about the message synchronization options in Service Manager, see *Add an integration instance*.
- You must grant admin consent to the **ChannelMember.Read.All** permission, which determines the owner of the private channels. If you don't have any private channels, you can skip this permission.

Create an integration user

The Teams integration uses Authorization Code or Resource Owner Password Credentials (ROPC) to call Graph API background, where **Authorization Code** allows an application to sign in the user by token, while **ROPC** allows an application to sign in the user by directly handling their password. Here are some guidelines for your reference when creating users on Azure:

1. Sign in to the Azure Entra admin center with the Azure account you use when registering an application.
2. Click **Users** to create a standalone Azure Entra ID user with the type of **Member**. This user is the integration user to invoke Microsoft Graph API. Alternatively, you can use the Azure account user directly.
3. Add Microsoft Teams license to this user. This user isn't able to sign in to Teams without this license.

Note

If you select **Resource Owner Password Credentials** as **Authorization Grant Type**, you must disable security defaults on Azure and disable multifactor authentication (MFA) for the integration user.

Otherwise, you need to select **Authorization Code** as **Authorization Grant Type**.

For detailed instructions, see Security defaults in Microsoft Entra ID.

Create teams and channels in Teams

User role: Teams Integration User

This is a prerequisite for this configuration. The integration user must be the default owner of the teams and channels that you want to use for the communication with Service Manager.

1. For the General channel, click **Manage channel**. On the **Channel settings** tab, select **Only owners can post messages**.
2. For the other public channels, click **Manage channel**. On the **Channel settings** tab, set **Channel moderation** to **On**. When you turn on moderation for a channel, only the team owner (the Teams integration user) can start new posts in that channel and regular users in the channel can only reply to existing posts and can't create new posts.

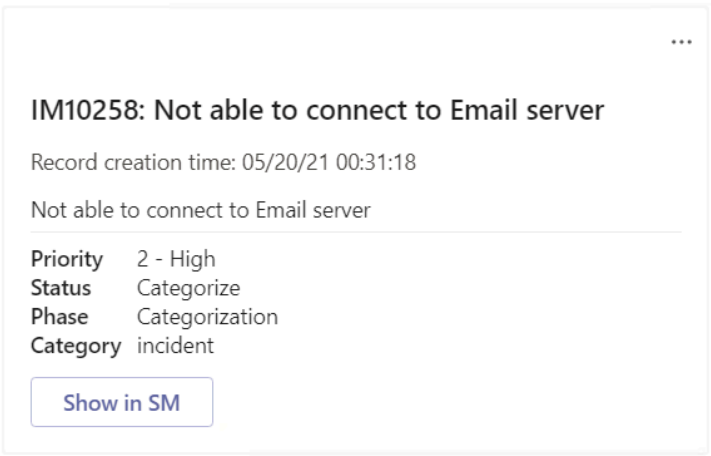
For detailed instructions, see Microsoft Teams documentation.

Configure the settings in Service Manager

User role: Service Manager System Administrator

Functionalize the Show in SM button

When a conversation starts, the system displays a **Show in SM** button in the default channel messages of a record. The conversation participants can click this button and then sign in to the Service Manager web tier to access the record on a new browser tab. See the following screenshot as an example:



As a prerequisite, you need to enable query hash and then add web server URL to make the **Show in SM** button work.

Enable query hash

Enable query hash by adding the *querysecurity* and *queryhashcode* parameters to the sm.ini file. For example:

```
querysecurity:1
queryhashcode:<hostname.com>:13080
```

Note that you need to provide the same hostname and communication port as above for the *queryhashcode* parameter in web.xml. For example:

```
<param-name>serverHost</param-name>
<param-value>hostname.com</param-value>

<param-name>serverPort</param-name>
<param-value>13080</param-value>
```

For more information, see Parameter: querysecurity and Parameter: queryhashcode.

Add the web server URL

Follow these steps:

1. Click **System Administration** > **Base System Configuration** > **Miscellaneous** > **System Information Record**.
2. Click the **Active Integrations** tab, and then type the web server URL in the Web Server URL field. For example, https://<hostname.com>:<port>/webtier/index.do

   Note that you must use the server's Fully Qualified Domain Name (FQDN) instead of the IP address in the web server URL.
3. Click **Save** and **OK**.
4. Restart the Service Manager server.

Add an integration instance

To set up Teams integration, you must add and enable an instance of the Teams integration in Integration Manager.

To add and enable a Teams integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager** to open Integration Instance Manager.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **MicrosoftTeams** from the Integration Template list.
4. Click **Next**. The Integration Instance Information page opens.
5. Complete the following settings in the Integration Instance Information page as necessary.

Setting	Value	Description
Name (required)	User-defined.	Name of the integration instance. Default: MicrosoftTeams.
Version (required)	1.0	Version number of the integration.
Interval Time (s) (required)	User-defined.	The interval time that the integration background schedule runs. For example, if you set the value to 300, the background process tries to execute the SMIS integration task every 5 minutes. In Teams integration, the synchronized task consists of several types, including message, attachment, and image. If a user posts a reply with an attachment, the background process can't synchronize both the reply and attachment in one SMIS task. The background process synchronizes the reply (raw text) first and then detects the embedded attachment information. Hence, it creates another SMIS task to synchronize the attachment in the next round. Default: 300.

Setting	Value	Description
Max Retry Times (required)	User-defined.	Maximum allowed number of retries if the background scheduler fails to run. Default: 5.
Log Level	Select a level from the drop-down list.	Level of diagnostic information that the Service Manager server logs to the log file directory. The possible log levels are DEBUG, INFO (default), WARNING, ERROR, and OFF. Default: INFO.
Log File Directory	Specify a directory.	This is a directory that exists on the Service Manager server host where log files are stored. The log includes information for receiving and processing messages. For example: c:\teams_logs. If this field is empty, the system uses the Service Manager sever log (sm.log) for Teams integration.

Setting	Value	Description
Shared Scheduler	User-defined.	The scheduler that the integration instance uses to schedule background processes. Multiple integration instances can use a shared scheduler. When you create an integration template, you can define a default shared scheduler for the instances that are based on the template. All instances that are based on this template can use the same shared scheduler. However, if you create an instance that uses a different scheduler, SMIS creates a new scheduler accordingly. If this field is empty, the system uses <Teams integration name + instance ID> for Teams integration.
Run at system startup	True or false.	Automatically enable or disable the integration instance when the Service Manager server starts.
Description	User-defined.	Description of the integration instance.

6. Click **Next**. The Integration Instance Parameters page opens.
7. Complete the following settings on the **Connectivity** tab as necessary.

Basic Configuration

Setting	Value	Description
Authorization Grant Type	Authorization Code (default value) Resource Owner Password Credentials	**Authorization Code** is the most common grant type, where the authorization server returns a single-use authorization code to the client. The client then exchanges the code for an access token. If you select **Authorization Code** as **Authorization Grant Type**, you may click **Get Access Token** to generate the authorization token. **Resource Owner Password Credentials** is an grant type where the application exchanges the user's username and password for an access token. This grant type suits clients highly trusted by the user, such as their computer operating system or a highly privileged application.
User Name	User-defined.	Name of the integration user you have registered to invoke Microsoft Graph API. For example: xxx@xxx.onmicrosoft.com

Setting	Value	Description
Password	User-defined.	Password of the integration user. This field is only required when you select **Resource Owner Password Credentials** as **Authorization Grant Type**.
Application (Client) ID	Paste the Application (Client) ID.	The Application (Client) ID you received when registering the application.
Directory (Tenant) ID	Paste the Directory (Tenant) ID.	The Directory (Tenant) ID you received when registering the application.
Client Secret	Paste a client secret string.	A secret string that the application uses to provides its identity when requesting a token.

Proxy Configuration

Setting	Value	Description
Proxy Host	Specify an IP address or domain name.	If you need to use a proxy server to connect to the Microsoft Graph API from Service Manager server, type the FQDN or IP address of the proxy server in this field.
Proxy Port	Specify a port number.	The port number of the proxy server.
Proxy Username	Type an account name.	The user name to connect to the proxy server.
Proxy Password	Type a password.	The password to connect to the proxy server.

Click **Get Access Token** to generate the authorization token if you selected **Authorization Code** as **Authorization Grant Type**. Once you click **Get Access Token**, a new browser tab is open to access MS Azure portal. After you select your account and click **Accept**, the

"Successfully obtained the access token" message is displayed which indicates that the token is successfully obtained.
Click **Verify Connection** to verify the connection settings.

8. Complete the following settings on the **Options** tab as necessary.

Basic Options

Setting	Value	Description
Message Post Template	User-defined.	Function name of the JavaScript that Service Manager uses to post messages to Teams. Default: MicrosoftTeamsTemplate.generateMessage. **Tip** You can search for the MicrosoftTeamsTemplate JavaScript in Service Manager and then create customized templates. For more information, see Adaptive Cards.

Setting	Value	Description
Synchronize Message	True or false.	Select whether to synchronize the Teams conversation messages, stickers, and user-pasted images to Service Manager or not. **Note** Service Manager doesn't synchronize the sticker or the user-posted image whose size exceeds the limitation defined in Service Manager. Moreover, Service Manager stops synchronizing the attachments if the total attachment size exceeds the limitation.

Setting	Value	Description
Synchronize Attachment	True or false.	Select whether to synchronize the attachments in Teams conversations to Service Manager or not. This option is available only after you select **Synchronize Message**. **Note** Service Manager doesn't synchronize the attachment which size exceeds the limitation defined in Service Manager. Moreover, Service Manager stops synchronizing the attachments if the total attachment size exceeds the limitation.

Setting	Value	Description
Synchronize Interval Time (min)	User-defined.	Interval (in minutes) for Teams conversation synchronization. This option works in conjunction with **Interval Time**. For example, if you set the value of **Interval Time** to 300, you need to wait 5 minutes for attachment and image synchronization after a reply is synchronized. The system performs a check every 5 minutes till the accumulated time reaches 60 minutes. Then, a new round of message synchronization starts. Default: 60. In the customer's environment, we suggest setting this value to 60 or larger.
Hide Participant Name	True or false.	Select whether to hide the participants' names in the conversation history to protect their personal identification information (PII) or not. In Teams, all conversation participants have their distinctive names. However, if you select this checkbox, the **Conversation** tab in Service Manager displays the value you specified in **General Display Name** instead of all participants' names.

Setting	Value	Description
General Display Name	User-defined.	Enter a general name for all participants. This option is available only after you select **Hide Participant Name**. Default: Microsoft Teams.

9. Click **Finish**. Service Manager creates the instance.
10. Enable the integration instance. A processor for the Teams integration instance now runs in the background.

Synchronize the teams and channels

To synchronize the existing teams and channels to Service Manager, follow these steps:

1. Click **System Administration** > **Ongoing Maintenance** > **Collaboration** > **Teams Settings** to open the Teams synchronization wizard.
2. Select the team to synchronize, and then click **OK**. The Teams Team Information page opens.

   The system populates the Display Name, Team ID, and Description fields automatically. A team is active by default. If you clear the **Active** checkbox, all channels under the current team become inactive and you cannot configure the channel mappings for this team.

   In the Channels list, you can find the existing channels in the Team. Click a channel to open the Channel Information page on which you can view the channel information.

   If you need to synchronize another Team, click **New** on the Teams Team Information page.



Tip

After you add, update, or remove the teams or channels in Teams, make sure to wait a few minutes and then click the **Synchronize** button to synchronize the latest teams and channels information to Service Manager. If you run into an error, wait a few minutes as well. This is a limitation of Microsoft Graph API.

Configure Teams channel mapping

Channel mapping is an important step for connecting the conversations over the records with the channels in Teams. To direct conversations to an appropriate Teams channel, you must configure a series of channel mapping between Service Manager and Teams.

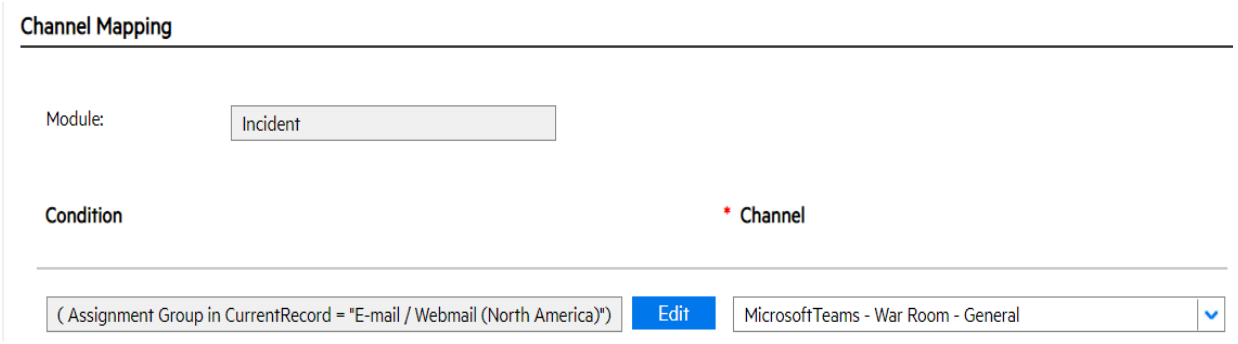
To configure Teams channel mapping, follow these steps:

1. Click **System Administration** > **Ongoing Maintenance** > **Collaboration** > **Teams Settings** to open the Teams synchronization wizard.
2. Click **Channel Mapping**. Select a business module from the **Module** drop-down list, and then click **Add**. The Channel Mapping page opens.

   > **Note**
   >
   > Out-of-the-box, the **Module** drop-down list consists of Incident, Interaction, Problem, Change, and Request. To add channel mappings to another record type (such as Change Task), you need to manually add this module and its related scmessage to the **Microsoft Teams Modules** global list.

3. Click **Edit** to build a condition or set of conditions, and then select a channel from the Channel list.

   See the following screenshot as an example:

   In this example, the system directs conversations over the major Incidents with the assignment group of E-mail / Webmail (North America) to the General channel under the War Room team for the MicrosoftTeams integration instance.

   Add more conditions as necessary and make sure that the conditions are reasonable. If a record meets multiple conditions, the system directs conversations according to the sequence of the mappings.

4. You can click **Move Up** (or press Alt+F8) or **Move Down** (or press Alt+F9) to arrange the mappings. To remove a mapping, select this mapping and then click **Remove** (or press Alt+F7).
5. Click **Save** and **OK**.

Note

- After you add, update, or remove the channels in Teams, make sure to validate the channel mappings in Service Manager.
- Make sure that the IT agents have access to the corresponding Teams channels especially when you update a record that affect channel mappings. Otherwise, the IT agents may fail to start a conversation when clicking the **Discuss on Teams** button. The error message may resemble the following:

  We can't find that channel due to connectivity issues. Please try again later.
- If you clear the **Active** checkbox of a channel that already has a channel mapping, the corresponding channel mapping entry displays this channel as **(Empty)**.

(Optional) Index conversation history

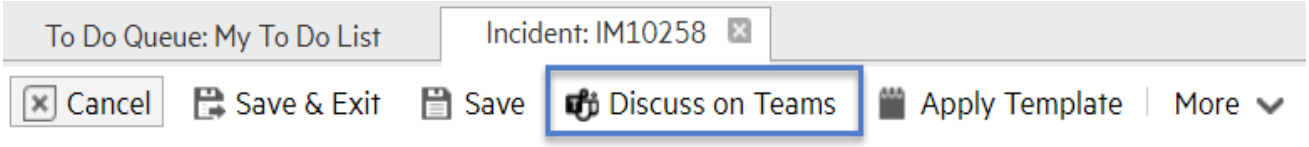
If you are working with Smart Analytics, you can index the Teams conversation history in the knowledgebases of the corresponding business modules and then search for the keywords in the conversation history by using Smart Search.

To allow the system to index the Teams conversation history, follow these steps:

1. Click **System Administration** > **Ongoing Maintenance** > **Smart Analytics** > **Smart Search** to open the Smart Search Configuration page.
2. Click to open the library of a business module in the Knowledgebase list. For example, Incident Library.
3. On the Knowledgebase Maintenance page, select the **Index Chat History** checkbox.
4. Click **Save** and **OK**.
5. Sign out and then sign in to Service Manager again.

Display the Discuss on Teams button

To start a Teams conversation, you can click the **Discuss on Teams** button in a record. See the following screenshot as an example:



This button is available for out-of-the-box Incident workflows. However, if you work with customized Incident workflows, you need to add a new ruleset to display this button.

To display the **Discuss on Teams** button in a major Incident, follow these steps:

1. Click **Incident Management** > **Configuration** > **Workflows** to display a list of workflows.
2. Open your Incident workflow, and then click **Edit Workflow Properties**.

3. Click the **Workflow Based Actions** tab, and then add the msteams.chat ruleset as follows:

ID	Action	Location	Action Condition
Discuss on Teams	Discuss on Microsoft Teams	Tray	Expression: jscall("MicrosoftTeamsConversation.isShowConversationAction",$L.file) and nullsub(major.incident in $L.file, false)=true and $L.mode~="add"

4. Save your changes.

To display the **Discuss on Teams** button in a Change record, follow these steps:

1. Click **Incident Management** > **Configuration** > **Workflows** to display a list of workflows.
2. Open your Change workflow, and then click **Edit Workflow Properties**.
3. Click the **Workflow Based Actions** tab, and then add the msteams.chat ruleset as follows:

ID	Action	Location	Action Condition
Discuss on Teams	Discuss on Microsoft Teams	Tray	*<Define your expression>*

4. Save your changes.

To display the **Discuss on Teams** button in other record types, follow similar steps as described above.

Display the conversation history

To display the conversation history in a record, follow these steps to update the related workflows.

1. Click **Tailoring** > **Process Designer** > **Workflows** to display a list of workflows.
2. Open your Incident, Interaction, Problem, Change, or Request workflow, and then click **Edit Workflow Properties**.
3. Click the **Workflow Based Rule Sets** tab > **Initialization** tab > **Add**, and then add the msteams.init.vars ruleset.
4. Click the **Workflow Based Actions** tab, and then add the **msteams.refresh.conversation** ruleset as follows:

ID	Action	Location	Action Condition
Refresh Teams Conversation	Refresh Teams Conversation	Button	3039 This value may vary depending on your workflow.

5. Save your changes.

Add the Conversation tab

The system uses the **Conversation** tab to display conversation history. See the following screenshot as an example:



Follow these steps to add the **Conversation** tab to a major Incident:

1. Click **Tailoring** > **Forms Designer**.
2. Search for the **im.incident.subform.msteamsconversation** subform, and then set the Button ID property of the **Go** button to the same value as the option ID of the **msteams.refresh.conversation** ruleset.
3. Search for your Incident format. If you are working with the out-of-the-box Incident formats, search for the following formats:
   - im.incident.categorization
   - im.incident.investigation
   - im.incident.review
   - im.incident.recovery
   - im.incident.closure
4. Add a Notebook Tab widget to your Incident format, and then set the following properties:

Property	Value
Caption	Conversation
Visible	Yes
Visible Condition	[$msteams.conversation.show]=true

5. Add the **im.incident.subform.msteamsconversation** subform to the Notebook Tab widget.
6. Save your changes.

Follow these steps to add the **Conversation** tab to other record types:

1. Click **Tailoring** > **Forms Designer**.
2. Search for the corresponding subform as described below, and then set the Button ID property of the **Go** button to the same value as the option ID of the **msteams.refresh.conversation** ruleset.

Module	Subform
Change	chm.subform.msteamsconversation
Interaction	sd.interaction.subform.msteamsconversation
Problem	pbm.problem.subform.msteamsconversation
Request	rm.request.subform.msteamsconversation



Note

Out-of-the-box, the system provides Teams conversation subforms to Incident, Interaction, Problem, Change, and Request. To add the Conversation tab to another record type (such as Change Task), you need to manually create a subform similar to the out-of-the-box ones and then define the **HTML-Generating Script** and **Input** properties in the HTML Viewer control on the subform.

3. Search for the format of the record type. Add a Notebook Tab widget to the format of the record type, and then set the following properties.

Property	Value
Caption	Conversation
Visible	Yes
Visible Condition	[$msteams.conversation.show]=true

4. Add the subform you have updated in *step 2* to the Notebook Tab widget.
5. Save your changes.
6. Continue to follow *step 3* to *step 5* with other formats of the record type.

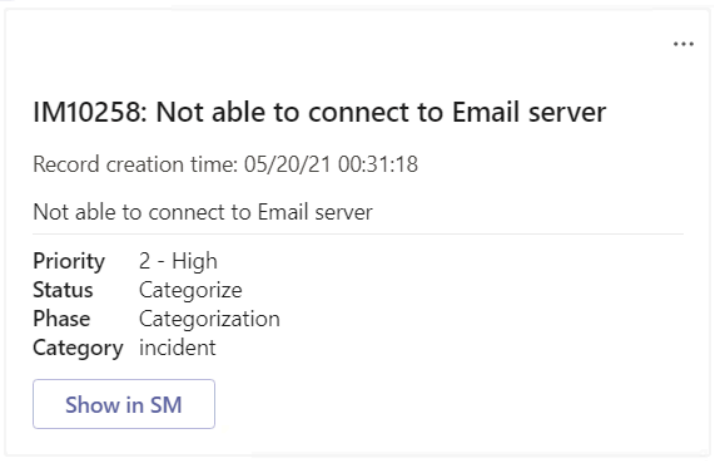
Inactivate the messages to a closed record

After you close a record, you can still post messages in the Teams channel. If you hope to stop synchronizing messages to the closed record, follow these steps to inactivate the messages:

1. Click **Tailoring** > **Process Designer** > **Workflows** to display a list of workflows.
2. Open your Incident, Interaction, Problem, Change, or Request workflow, and then click the **Closure** phase.
3. Click the **Rule Sets** tab > **On enter** tab > **Add**, and then add the **msteams.message.close** ruleset.
4. Save your changes.

Customize the message post template

After you click the **Discuss on Teams** button from an Incident, an Interaction, a Problem, a Change, or a Request record in Service Manager, the system uses a default message post template to send a channel message to Teams. This channel message displays the **Show in SM** button together with the description, priority, status, and other related information of this record. See the following screenshot as an example:



>

However, if you start a conversation from another record type (such as Change Task), the default channel message may display the **Show in SM** button only. In this case, you need to access the Service Manager Script Library and then create a customized message post template based on the **MicrosoftTeamsTemplate** JavaScript to include other information for this record type.

(Optional) Configure Teams integration for multiple SMIS instances

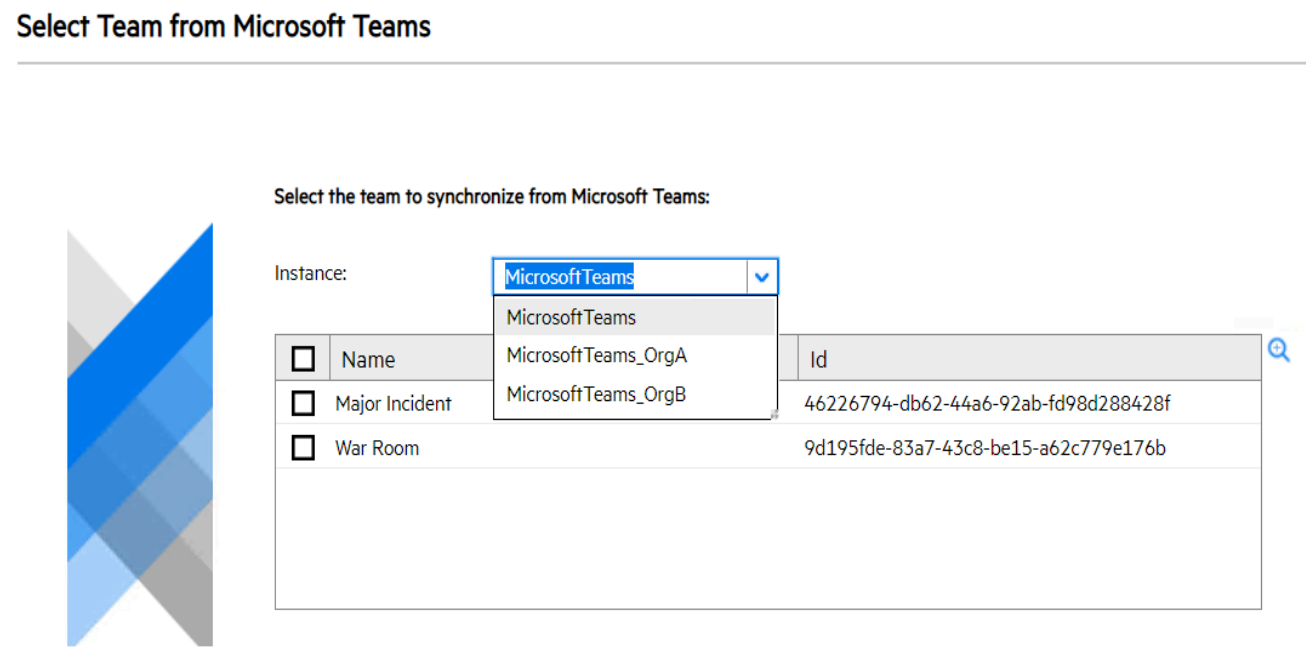
If your Service Manager system operates with folder entitlement or Mandanten file security, you can also configure the Teams integration to support these security features by setting up more than one SM-Teams integration instance. As a result, IT agents can start Teams conversations from the records they are currently associating.

Follow these steps to configure Teams integration to support multiple SMIS instances:

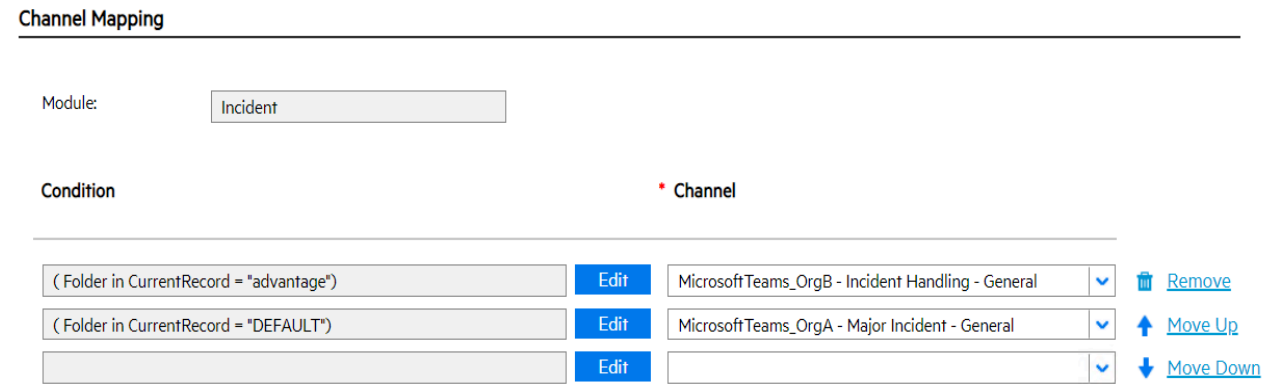
1. From **Database Manager**, open the **SMISRegistry** table.
2. Click **Search** and then select the **MicrosoftTeams** SMIS registry.
3. On the **General** tab, set the value for the **Instance Count** field to a number greater than 1.

4. Click **Save** and **OK**.
5. Sign out and then sign on to Service Manager again.

After you configure Teams integration and set up several SMIS integration instances, you need to synchronize the Teams and channels for each SMIS integration instance. The synchronization wizard now additionally displays the **Instance** field and enables you to select the integration instance. See the following screenshot as an example:



Next, configure channel mapping accordingly and associate the records with the corresponding Teams channels under different SMIS instances. See the following screenshot as an example:



This example assumes that the system administrator has enabled folder entitlement. The system directs conversations over major Incidents in the **advantage** folder to the General channel under the Incident Handling team for the MicrosoftTeams_OrgB integration instance, whereas the conversations over major Incidents in the **DEFAULT** folder go to the General channel under the major Incident team for the MicrosoftTeams_OrgA integration instance.

Verify the integration

User role: Service Manager System Administrator

This example uses a major Incident record to verify the integration.

1. Navigate to Incident Management, and then open an existing major Incident record.
2. Click the **Discuss on Teams** button, and then check that a default channel message about this Incident record arrives at the Teams channel.

   Note: If you update the Incident's priority, status, or phase information after the default channel message appears in the Teams channel, this information on the default channel message won't be updated.
3. Send a reply to this message in Teams.
4. Click the **Show in SM** button in the default channel messages, and then sign in to the Service Manager web tier to access the record on a new browser tab.
5. Check that your reply appears on the **Conversation** tab of the Incident record, and the reply submitter is your MS Teams user name. If you have selected **Hide Participant Name** in SMIS, the system displays the value you specified in **General Display Name** instead of your actual MS Teams user name.

   If a conversation consists of the following items, the system automatically synchronizes these items to Service Manager as well:

   - Table, text formats, emojis, and giphys provided by Teams

     These items are available on the **Conversation** tab
   - URLs

     This item is available on the **Conversation** tab
   - Images posted by users and stickers provided by Teams

     Service Manager displays images and stickers on the **Conversation** tab and saves them to the **Attachments** tab with the `Teams_` prefix in file names. For example, `Teams_xxxx.png`.
   - User attachments that follow the type and size policy of Service Manager

     Service Manager displays the names of these attachments on the **Conversation** tab and saves these attachments to the **Attachments** tab.

Important

The system doesn't synchronize the following items from a Teams conversation to Service Manager.

- Reactions in a conversation message
- Meetings (provided by the Teams **Schedule a meeting** app)
- Cards including but not limited to praise badges and approvals (provided by the Teams **Praise** app and **Approvals** app, respectively). In this case, Service Manager displays blank messages on the **Conversation** tab after synchronization.
- Replies sent by the bot

Note

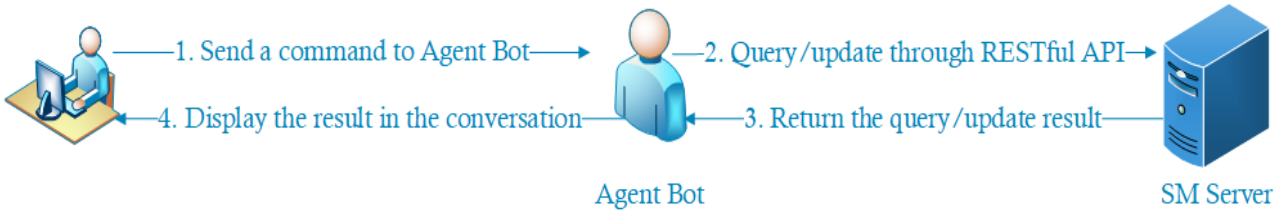
- If an IT agent deletes a message from Teams, the corresponding message is deleted from Service Manager as well after synchronization. However, if a message with an attachment, an image, or a sticker is removed from Teams, the attachment in the removed message is not affected. The attachment, image, or sticker is still available on the Attachments tab of the Incident.
- If an attachment with the same name (for example, `abc.txt`) appears more than once in the replies to the same record in Teams, Service Manager synchronizes this file to the Attachment tab with an accumulating number in the file name. For example, `abc (1).txt`, `abc (2).txt`, and so on.

For a complete use case, see Collaborate on Teams with war room experience.

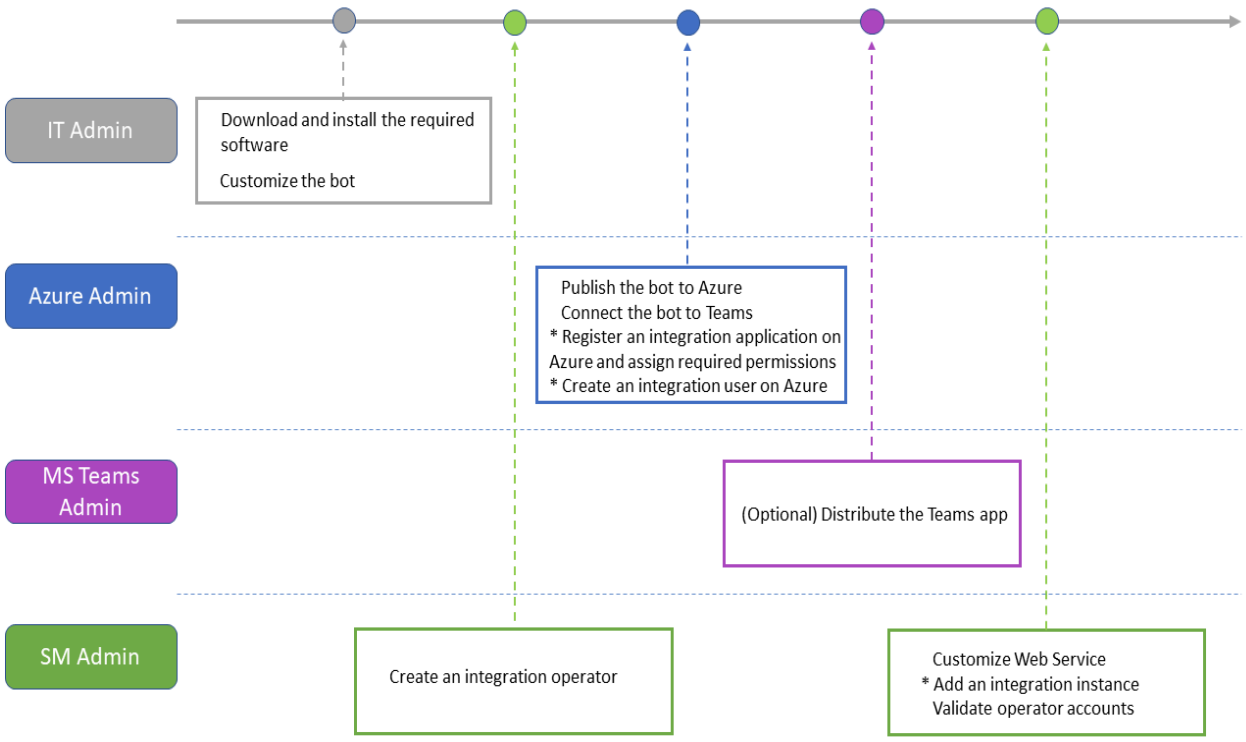
1.13.2. Set up agent bot

By embedding agent bot to Microsoft Teams, Service Manager IT agents can create Incidents and Interactions, update, resolve, or close Incidents - all through a conversation in Microsoft Teams without signing in to Service Manager. After an IT agent starts a Teams conversation with the bot and sends a command, the bot calls RESTful API to query or update the record on the Service Manager server. After that, the Service Manager server returns the query or update result to the bot and hence the bot exposes this result to the IT agent in the Teams conversation.

See the following illustration for the bot conversation creation workflow:



The following content describes how to set up the bot. The following diagram highlights the steps.



Note: Skip the steps with asterisks (*) if you have set up the integration between Service Manager and Microsoft Teams for war room experience.

Prepare the bot locally

User role: IT Administrator

The Service Manager 9.80 release Teams integration package (sm9.80.*xxxx*_TeamsIntegration.zip) includes an out-of-the-box agent bot that is developed based on Microsoft Bot Framework Composer. Extract the package to get the **AgentBot** folder and then open this folder with Bot Framework Composer. You can test the bot functions first, and then either make customization to this bot or create a new bot based on the default one.

Note

Don't rename the **AgentBot** folder. Otherwise, the bot will not work.

Download and install the required software

As a prerequisite, you need to download and install Bot Framework Composer and the related software for the testing, customization, or creation of your bot.

(Optional) Customize and localize the bot

Localize the bot

The out-of-the-box bot talks to users in different languages that Service Manager supports. However, if you need the bot to respond to a language that is unofficially supported by Service Manager, follow these steps:

1. Open the out-of-the-box bot with Bot Framework Composer.
2. Update language settings and add the target language to the bot. See Multilingual support for detailed instructions.
3. Localize the response strings of the bot to the target language. Open the **Common** file under **Bot Responses**, and then localize all response strings in `#allsmmessages`.

Customize the bot

You may hope to make customizations to the out-of-the-box bot by adding more actions, updating the dialogs, and so on. For detailed instructions, see the Bot Framework Composer documentation.

Before customizing the out-of-the-box bot, we commend backing up all response strings to maintain consistency of the bot's behavior among the localized languages. Here are some guidelines for your reference:

1. Open the out-of-the-box bot with Bot Framework Composer.
2. Open the **Common** file under **Bot Responses**, and then back up all response strings in `#allsmmessages`. You need to switch between languages to backed up the responses for each language.
3. Remove all languages except for English.
4. Make customizations to the English version of the bot by adding more actions, updating the dialogs, and so on.
5. Add the supported languages back. By doing so, Composer creates a copy of your bot's content with the name of each additional language. Hence, your customizations maintain

consistency among the languages.
6. Switch between languages, open the **Common** file, and then copy the backups of the response strings to `#allsmmessages` respectively.

Create an integration operator in Service Manager

User role: Service Manager System Administrator

This integration requires a user account for the bot to connect to Service Manager. You need to create a dedicated integration user account with the following settings in Service Manager.

- **Security Roles**:

  Service Desk: **view**, **new**
  Incident: **view**, **new**, **update**, **close**

- **Execute Capabilities**:

  Need **RESTful API**.

You can use this integration operator for both agent bot and virtual agent.

Prepare the bot on Azure

User role: Azure Administrator

Publish the bot to Azure

For detailed instructions about how to publish the bot to Azure and then test the published bot, see Publish a bot to Azure.

After publishing the bot to Azure, sign in to the Azure portal and follow these steps to update settings for the integration user and the Service Manager server Web Service URL.

1. Select the **App Service** of the bot.
2. Click **Settings** > **Configuration**, and then click **Application settings**.
3. Click **New application setting** and then add **SM_USERNAME**, **SM_PASSWORD**, and **SM_ENDPOINT**, respectively.

Name	Value
SM_USERNAME	The user name of the SM integration operator.
SM_PASSWORD	The SM login password of the SM integration operator.
SM_ENDPOINT	The actual Service Manager server Web Service URL in your production environment. This URL **MUST** be a public IP address so that the bot can access the Service Manager server. For example, `http://<servername>:<port>/SM/9/rest/`.

For detailed instructions about how to configure common settings for web apps, see Configure app settings.

Connect the bot to Teams

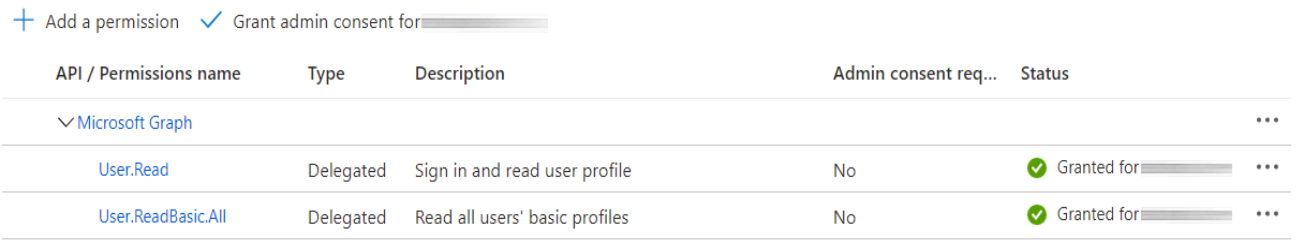
Add the bot to Microsoft Teams as one of the apps. For detailed instructions, see Connect a bot to Microsoft Teams.

Register an application

Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. Also, skip this step if you have registered an application when setting up the virtual agent.

If you haven't done so, register a new Entra ID Application for the bot. See Register an application for more information.

Add the following API permissions since the bot needs to query the user's information.



Create an integration user on Azure

Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. Also, skip this step if you have created an integration user on Azure when setting up the virtual agent.

If you haven't done so, see Create an integration user for more information. Note that this user doesn't require a Teams license.

(Optional) Distribute the bot

User role: Teams Administrator

> **Note**
>
> To interact with the agent bot, you can skip this step and use the **bot embed code** instead. Copy the HTTPS part of the code that is shown in the **Get bot embed code** dialog and then paste this address in the browser. After you open this URL in Microsoft Teams, you should be able to see the bot listed as a contact. For detailed instructions, see Connect a bot to Microsoft Teams.

Now you can distribute your bot to the IT operators or anyone who requires to use it by uploading the bot in Teams, publish the bot to your org, or publishing the bot to the Teams store. Hence, you can send messages to the bot in Microsoft Teams and then receive the bot's responses in Teams conversation. For detailed instructions, see Distribute your Microsoft Teams app.

> **Important**
>
> In this release, use the bot in one-on-one conversations only. When publishing the bot, open the manifest.json file of the bot and then set the `scopes` to `scopes": ["personal"]`. For more information about bots' schema, see Reference: Manifest schema for Microsoft Teams.
>
> Don't add the bot to a team or a channel because it may bring chaos.

Configure the settings in Service Manager

User role: Service Manager System Administrator

To make the bot work with Service Manager Incident and Interaction records, you need to configure the following settings in Service Manager.

1. Customize Web Service.

   Click **Tailoring** > **Web Services** > **Web Service Configuration** to open the External Access Definition page.

   To the default primary affected service and the default category when adding an Incident, search for the **MSTeamsIncident** object, click the **Expressions** tab, and then add expressions as required. For example:

```
// set default primary affected service to "Applications" when add
if ($L.mode="add" and null(affected.item in $L.file)) then
(affected.item in $L.file="CI1001030")

// set default category when add
if ($L.mode="add" and null(category in $L.file)) then (category in
$L.file=nullsub(jscall("settings.getSettingValue", "Incident",
"default.category"), "incident"))
```

To set the default category when adding an Interaction, search for the **MSTeamsInteraction** object, click the **Expressions** tab, and then add expressions as required. For example:

```
// set default category when add
if ($L.mode="add" and null(category in $L.file)) then (category in
$L.file=nullsub(jscall("settings.getSettingValue", "Service Desk",
"default.category"), "incident"))
```

2. Add an integration instance.

   Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. Also, skip this step if you have added an integration instance when setting up the virtual agent.

   If you haven't done so, see Add an integration instance for detailed instructions. For the settings in the **Basic Configuration** section on the **Connectivity** tab, consult the Azure administrator for the bot integration user information, the bot Application (Client) ID, Directory (Tenant) ID, and the Client Secret.

3. Validate operator accounts with Teams login IDs.

   Usually, an IT agent signs in Teams with an email address. To validate that an IT agent is an authorized operator in Service Manager, either the operator's login name or email must be the same as the IT agent's Teams login ID.

Verify the integration

User role: Service Manager System Administrator

1. Sign in to Teams and then start a conversation with the bot.
2. Send the following commands to the bot one by one.



Note

These commands use regular expressions to parse the parameters entered by the user. For special requirements, you can modify the corresponding triggers of these commands in Bot Framework Composer.

Command	Description
help	List all available commands.
help interaction	List detailed information for Interaction commands.
help incident	List detailed information for Incident commands.

get incident <ID> [field]	Get the details of an Incident. <ID> is the record ID with an Incident's prefix, and [field] is the caption of a field defined in Web Services.
	Example commands:
	`get incident IM10265`
	`get incident IM10265 Impact`

create incident description	Create an Incident. Example command: `create incident Outlook keeps asking for credentials`
create interaction description	Create an Interaction. Example command: `create interaction Cannot add attachment to OneNote Shared directory`

Command	Description

update incident <ID> field1:"value1" [,field2:"value2",...]	Update one or more field values of an Incident.
	
	For each field, use the field caption defined in the **Incident** web service definition or the field name in the **probsummary** dbdict record, and enclose the field value in double quotation marks. If you want to update multiple fields, separate them with a comma.
	
	If the field caption or the field name consists of special characters (for example, a dot), enclose the field caption in double quotation marks as well.
	
	Note: Use ISO standard format for the Datetime field.
	
	Example command:
	
	`update incident IM10265 Status:"Work In Progress",Assignee:"Incident.Coordinator"`
	
	`update incident IM10265 ClosedTime:"2021-09-14T02:39:52+00:00"`
	
	`update incident IM10265 "brief.description":"New Title"`

resolve incident <ID>	Resolve an Incident with a solution. Example command: `resolve incident IM10265` The bot will prompt you to enter a solution for this Incident.

Command	Description
close incident <ID>	Close the incident specified by <ID>. Example command: `close incident IM10265` The bot will prompt you to select a closure code for this Incident.
cancel	Cancel your operation.
version	Display version information of the bot.

After sending each command that relates to Incident and Interaction records, sign in to Service Manager to verify the result. The system tracks all operations on the **Activity** tab.

For a complete use case, see Interact with Service Manager by using agent bot.

Important

The Service Manager employee self-service (ESS) users are not authorized to interact with Service Manager by using agent bot.

1.13.3. Set up virtual agent

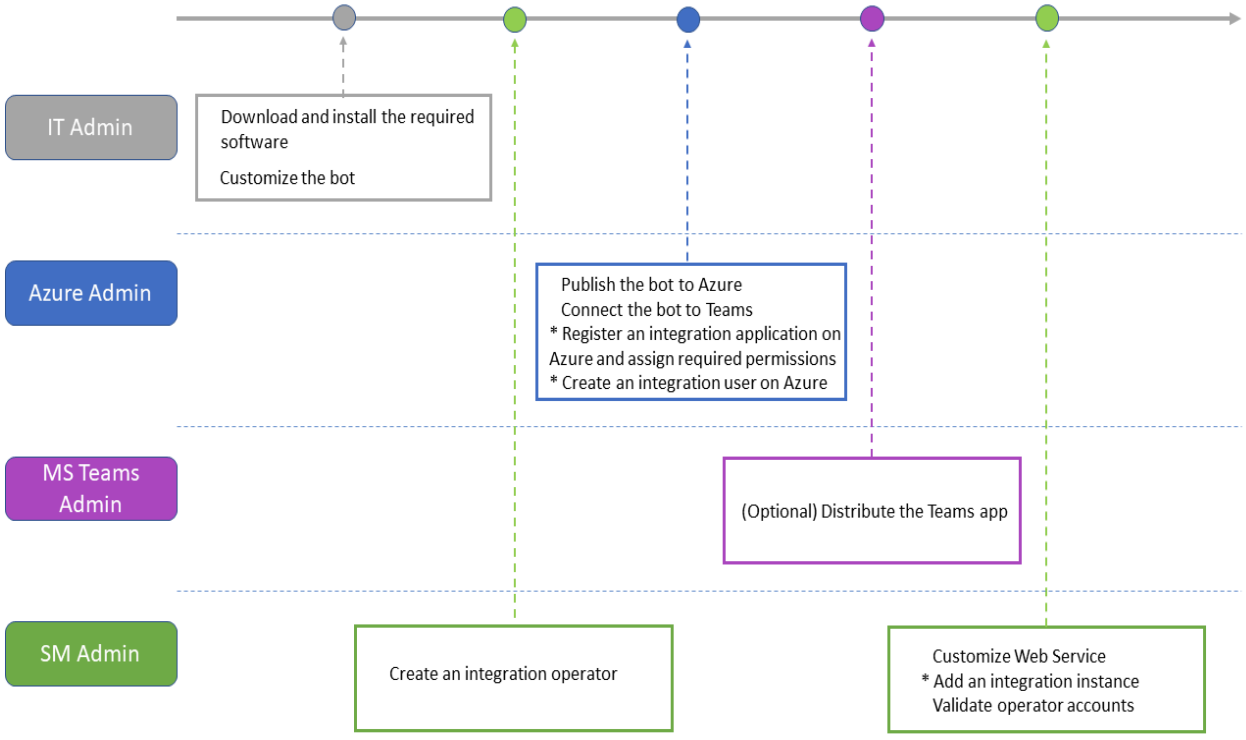
Being embedded in Microsoft Teams, the virtual agent is a bot that is contextually aware of a user's questions and gives relevant answers to a conversation in a natural, human-like manner. When a Service Manager employee self-service (ESS) user starts a conversation in Teams and describes an issue, the virtual agent leverages Smart Search and then retrieves solutions from a variety of resources, including Service Manager Service Catalog items and Knowledge Management (KM) documents. The ESS user can also create Interactions and check Interactions' status by using the options provided by the virtual agent - all through a conversation in Microsoft Teams without signing in to Service Manager.

After an ESS user starts a Teams conversation with the virtual agent, the virtual agent calls RESTful API to send the content of the conversation to the Service Manager server. As a result, the Service Manager server returns a valid response or knowledge search results to the virtual agent and hence the virtual agent exposes this response or results to the ESS user in the Teams conversation.

See the following illustration for the virtual agent conversation creation workflow:



The following content describes how to set up the virtual agent. The following diagram highlights the steps.



Note: Skip the steps with asterisks (*) if you have set up the integration between Service Manager and Microsoft Teams for war room experience.

Prepare the virtual agent locally

User role: IT Administrator

The Service Manager 9.80 release Teams integration package (sm9.80.*xxxx*_TeamsIntegration.zip) includes an out-of-the-box virtual agent that is developed based on Microsoft Bot Framework Composer. Extract the package to get the **VirtualAgent** folder and then open this folder with Bot Framework Composer. You can test the functions first, and then either make customization to this virtual agent or create a new virtual agent based on the default one.



Note

Don't rename the **VirtualAgent** folder. Otherwise, the virtual agent will not work.

Download and install the required software

As a prerequisite, you need to download and install Bot Framework Composer and the related software for the testing, customization, or creation of your virtual agent.

(Optional) Customize the virtual agent

You may hope to make customizations to the out-of-the-box virtual agent by adding more actions, updating the dialogs, and so on. For detailed instructions, see the Bot Framework Composer documentation.

Create an integration operator in Service Manager

User role: Service Manager System Administrator

This integration requires a user account for the virtual agent to connect to Service Manager. You need to create a dedicated integration user account with the following settings in Service Manager.

- **Security Roles**:

  Service Desk: **view**, **new**

- **Execute Capabilities**:

  Need **RESTful API**.

Skip this step if you have created an integration operator when setting up the agent bot.

Prepare the virtual agent on Azure

User role: Azure Administrator

Publish the virtual agent to Azure

For detailed instructions about how to publish the virtual agent to Azure and then test the published bot, see Publish a bot to Azure.

After publishing the virtual agent to Azure, sign in to the Azure portal and follow these steps to update settings for the integration user and the Service Manager server Web Service URL.

1. Select the **App Service** of the virtual agent.
2. Click **Settings** > **Configuration**, and then click **Application settings**.
3. Click **New application setting** and then add **SM_USERNAME**, **SM_PASSWORD**, and **SM_ENDPOINT**, respectively.

Name	Value
SM_USERNAME	The user name of the SM integration operator.
SM_PASSWORD	The SM login password of the SM integration operator.
SM_ENDPOINT	The actual Service Manager server Web Service URL in your production environment. This URL **MUST** be a public IP address so that the bot can access the Service Manager server. For example, `http://<servername>:<port>/SM/9/rest/base/virtualAgent`.

For detailed instructions about how to configure common settings for web apps, see Configure app settings.

Connect the virtual agent to Teams

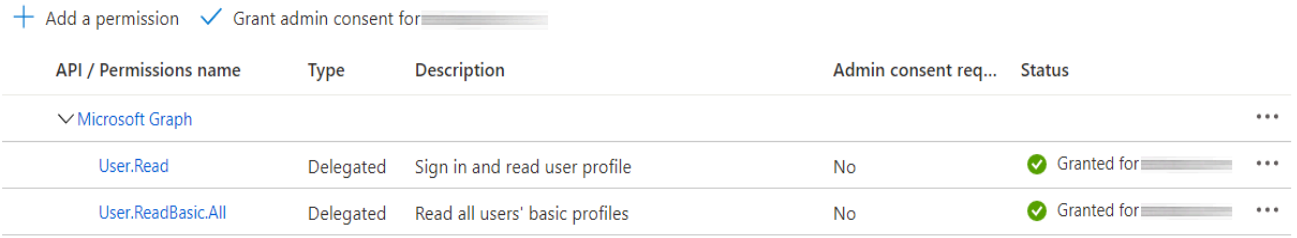
Add the virtual agent to Microsoft Teams as one of the apps. For detailed instructions, see Connect a bot to Microsoft Teams.

Register an application

Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. Also, skip this step if you have registered an application when setting up the agent bot.

If you haven't done so, register a new Entra ID Application for the virtual agent. See Register an application for more information.

Add the following API permissions since the virtual agent needs to query the user's information.



Create an integration user on Azure

Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. Also, skip this step if you have created an integration user on Azure when setting up the agent bot.

If you haven't done so, see Create an integration user for more information. Note that this user doesn't require a Teams license.

(Optional) Distribute the virtual agent

User role: Teams Administrator

Note

To interact with the virtual agent, you can skip this step and use the **bot embed code** instead. Copy the HTTPS part of the code that is shown in the **Get bot embed code** dialog and then paste this address in the browser. After you open this URL in Microsoft Teams, you should be able to see the virtual agent listed as a contact. For detailed instructions, see Connect a bot to Microsoft Teams.

Now you can distribute your virtual agent to the IT operators or anyone who requires to use it by uploading the bot in Teams, publish the bot to your org, or publishing the bot to the Teams store. Hence, you can send messages to the bot in Microsoft Teams and then receive the bot's responses in Teams conversation. For detailed instructions, see Distribute your Microsoft Teams app.

Important

In this release, use the virtual agent in one-on-one conversations only. When publishing the bot, open the manifest.json file of the bot and then set the `scopes` to `scopes":["personal"]`. For more information about virtual agents' schema, see Reference: Manifest schema for Microsoft Teams.

Don't add the virtual agent to a team or a channel because it may bring chaos.

Configure the settings in Service Manager

User role: Service Manager System Administrator

To make the virtual agent work with Service Manager Incident and Interaction records, you need to configure the following settings in Service Manager.

1. Add an integration instance.

   Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. Also, skip this step if you have added an integration instance when setting up the agent bot.

   If you haven't done so, see Add an integration instance for detailed instructions. For the settings in the **Basic Configuration** section on the **Connectivity** tab, consult the Azure administrator for the virtual agent integration user information, the virtual agent Application (Client) ID, Directory (Tenant) ID, and the Client Secret.

2. Validate operator accounts with Teams login IDs.

   Usually, an ESS user signs in Teams with an email address. To validate that an ESS user is an authorized operator in Service Manager, either the operator's login name or email must be the same as the ESS user's Teams login ID.

3. Configure service portal information.

   When an ESS user enters a keyword and searches for knowledge articles, the Smart Search results returned by the virtual agent may include links to Catalog Items that are related to the keyword. An ESS user can click this kind of link to access the corresponding Catalog Items in the service portal and then place an order. As a prerequisite, you need to select the right portal depending on which portal is being used for ESS users in your organization. To do this, follow these steps:

   1. Go to **System Administration** > **Base System Configuration** > **Miscellaneous** > **System Information Record**.
   2. Select the **Active Integrations** tab.
   3. In the **SM Portal** field, select the right portal from the drop-down list. In the **Service Portal URL** field, specify the portal URL for your portal.
   4. Click **Save** and **OK**.
   5. Restart the Service Manager server.

Verify the integration

User role: Service Manager System Administrator

1. Sign in to Teams and then start a conversation with the virtual agent.

2. The virtual agent can respond to the following commands in this release.

Command	Description
help	List all available commands.
cancel	Cancel your operation.
version	Display version information of the virtual agent.
setlanguage	Prompt the user to select a language.

3. Send a message to the virtual agent in Teams. The virtual agent may display up to three language selections and prompt you to select a language.

   The language selections are based on the following language settings:

   - The Service Manager operator's language setting in Contact Information and in Operator Record (The language setting in Contact Information has higher priority)
   - The Service Manager system language setting in System Information Definition
   - Teams environment language

   When the languages from the above sources are different, the user needs to select a language as prompted. Otherwise, When the languages from the above sources are the same, the virtual agent uses this language by default without prompting for a language selection.

   Note

   If any language from the above sources is unofficially supported by the out-of-the-box Service Manager system, the virtual agent still lists this language as a user selection. However, when a user selects this language but the system administrator hasn't localized Service Manager into this language yet, the virtual agent continues displaying English in the conversation.

4. After you confirm the language selection, the virtual agent responds with these services: **Create Support Ticket**, **Check Ticket Status**, and **Search Knowledge**.

Service	Description
Create Support Ticket	Click this option and then describe your issue to create a Service Desk Interaction.
Check Ticket Status	Click this option to check the status of the Service Desk tickets that relate to you. The virtual agent lists the unclosed tickets submitted in the last 30 days, and you can also check other tickets related to you. You must be Service Recipient, Alternate Contact, or Primary Contact of these tickets. The number of Service Desk tickets that can be displayed by the virtual agent is up to 50.
Search Knowledge	Click this option and then enter keywords to start Smart Search.

For a complete use case, see Interact with Service Manager by using virtual agent.

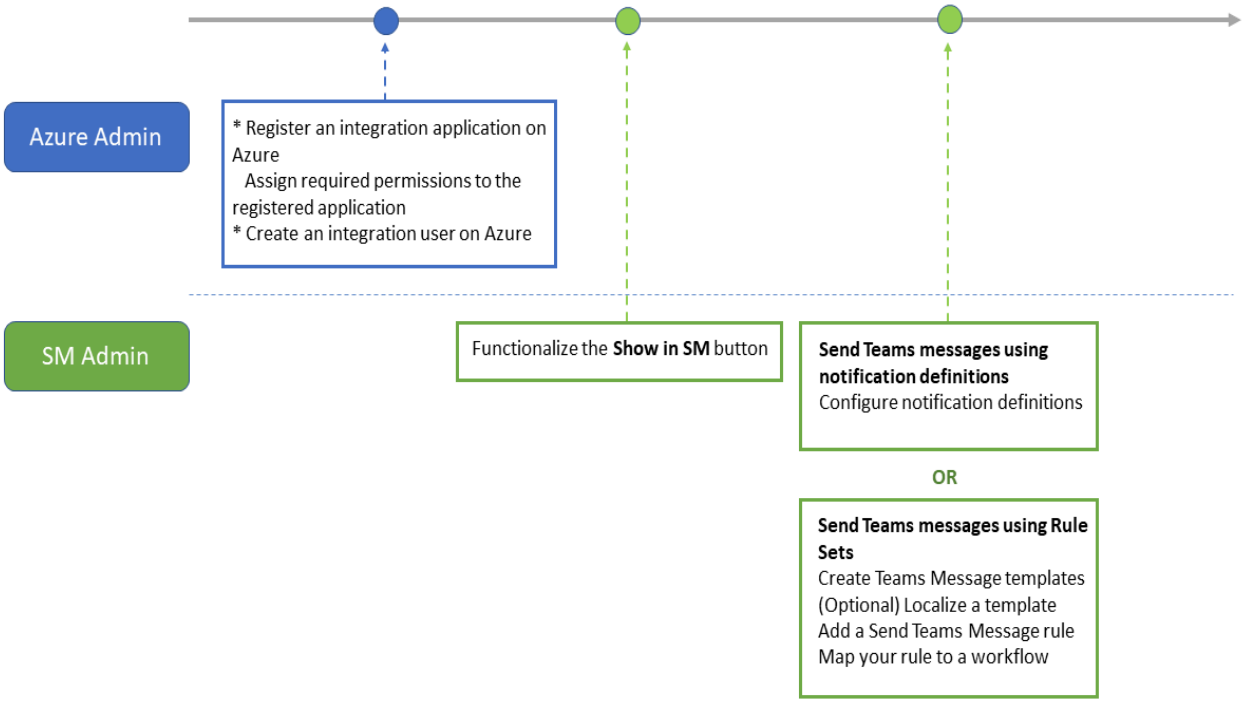
Note

If you ignore the **Yes** and **No** options and click an option again after functioning, the name of this option displays in the conversation. For example, the virtual agent returns Smart Search results after you click **Search Knowledge**. At this time, if you ignore the **Yes** and **No** options and click **Search Knowledge** again, the text **Search Knowledge** appears in the conversation.

1.13.4. Set up Teams notification

By setting up Teams notification, when an IT agent updates a record or requires further communication, Service Manager automatically routes a new notification to Microsoft Teams as a timely reminder. For example, when the status of an Incident becomes Resolved, the contact of this Incident will receive a message from the integration user on Teams that claims **Incident IM##### has been resolved**.

The following content describes how to set up Teams notification. The diagram below highlights the steps.



Note: Skip the steps with asterisks (*) if you have set up the integration between Service Manager and Microsoft Teams for war room experience.

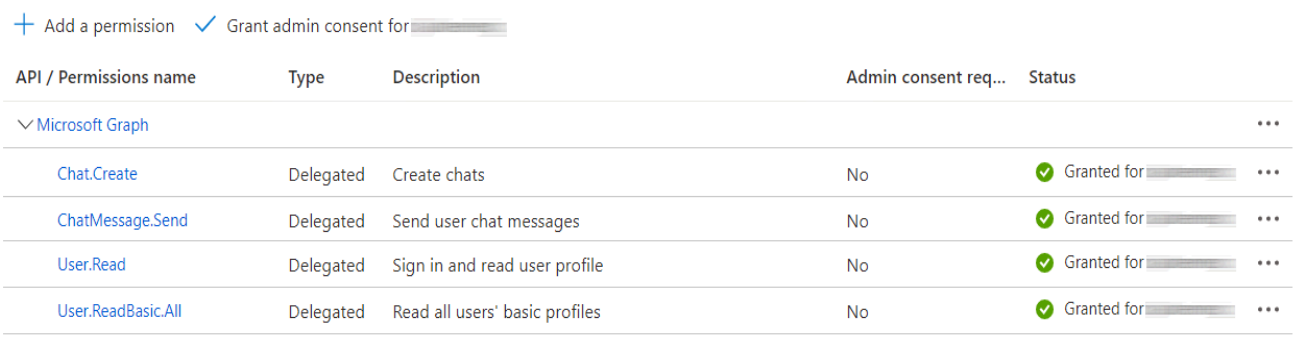
Register an application on Azure

User role: Azure Administrator

Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. Also, skip this step if you have registered an application when setting up the agent bot or virtual agent.

If you haven't done so, register a new Entra ID Application for the Teams notification function. See Register an application for more information.

Add the following four delegated permissions since Microsoft Teams needs to create a chat and then send messages.



Create an integration user on Azure

User role: Azure Administrator

Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. Also, skip this step if you have created an integration user on Azure when setting up the agent bot or virtual agent.

If you haven't done so, see Create an integration user for more information. Note that this user requires a Teams license.

Note

If you use the account of an actual user in your organization as the Azure integration user account instead of creating a new one, this actual user account can't send a Teams message to itself.

Configure the settings in Service Manager

User role: Service Manager System Administrator

You can send notifications through Teams messages by using either Service Manager rule sets or notification definitions.

Functionalize the Show in SM button

Skip this step if you have set up the integration between Service Manager and Microsoft Teams for war room experience. If you haven't done so, see Functionalize the Show in SM button for more information about this setting.

Send Teams messages using Service Manager Rule Sets

Follow these tasks to send Teams messages using Service Manager rule sets.

Task 1: Create Teams Message templates

Service Manager provides an out-of-the-box Teams message template **Incident Resolved** and the system uses this template when sending a notification through a Teams message if an Incident's status becomes Resolved. To send Teams messages from more Service Manager modules, you can use this template as a basis and create your own ones.

To create a custom Teams message template, follow these steps:

1. Click **Tailoring** > **Notifications** > **Teams Message Templates**.
2. Search for and select the out-of-the-box **Incident Resolved** template as the base of the new template.
3. On the **General** tab, specify a name for the new template in the **Name** field.
4. Select a module from the **Module** drop-down list.
5. Update the **Title** field if necessary.
6. In the **Content** text box, design the message body of your Teams message. To reference a variable or expression in the message, enclose your variable or expression in braces, such as `{$RECORD.action}`.
7. In the **Subtitle** text box, design the subtitle of your Teams message. To reference a variable or expression in the message, enclose your variable or expression in braces, such as `{$RECORD.brief_description}`.
8. Click **Add** to create the new template.

A template is active by default. If you clear the **Active** checkbox, this Teams message template becomes inactive and is unavailable when configuring a send Teams messages rule in Task 3.

(Optional) Task 2: Localize a template

To create a language version of a Teams message template, follow these steps:

1. Click **Tailoring** > **Notifications** > **Teams Message Templates**.
2. Use the search or advanced search to find the template that you want to localize.
3. On the **General** tab, select a language in the **Language** field for the localized template.
4. Keep the name in the **Name** field as is, because Teams message template records with the same name are considered language versions of one template.
5. In the **Title** field, **Subtitle** field, and **Content** text box, replace the original text with the appropriate translation and make sure all the variables are in the correct positions.
6. Click **Add** to create the localized template.

Task 3: Add a Send Teams Message rule

Service Manager provides an out-of-the-box rule set **im.notification.resolved** for the system to send a Teams message to the contact of an Incident on Teams when this Incident's status becomes Resolved. To send notifications from more Service Manager modules and from a variety of records status, you can use the **im.notification.resolved** rule set as a basis and create your own rule set.

For detailed instructions about how to add a rule for sending Teams messages, see Add a Send Teams Message rule.

Task 4: Map your rule to a workflow

The out-of-the-box rule set **im.notification.resolved** is mapped to **Transition: Recovery - Review** in the out-of-the-box Incident workflow. After creating your rule for sending Teams messages, use the default Incident workflow as a reference to map your rule set to the corresponding workflows as required.

Send Teams messages using notification definitions

Follow these steps to send Teams messages using Service Manager notification definitions.

1. Click **Tailoring** > **Notifications** > **Notifications**. The **Notification Definition** form opens.
2. Search for and select an existing notification definition as required.
3. On the **Message** tab, add the following configuration.

Msg Class	Msg No.	Arguments	Condition	Format	Notify Method	Recipient(s)	Group File	Group Area	Subgroup
msteams	message	jscall("MicrosoftTeamsTemplate.generateMessageFromTemplate","message template name",$L.file)	true		teams	operator()			

In this example:

- **Arguments:** You need to replace the second argument of the jscall function with the name of the template that you want to use for this notification. For example,jscall(

"MicrosoftTeamsTemplate.generateMessageFromTemplate", "Incident Resolved", $L.file).
- **Condition:** You may need to update this column with other valid expressions.
- **Recipient(s):** You may need to update this column with other valid values. This column supports operator groups as well. For example:

Recipient(s)	Group File
assignment in $L.file	assignment

4. Click **Save** and **OK**.

Verify the integration

User role: Service Manager System Administrator

This example uses the out-of-the-box Teams messages template, rule set, and an Incident record to verify the integration.

1. Navigate to Incident Management, and then open an existing Incident record that is not in the Closure phase.
2. Record the email address of the Contact Person in the Incident record.
3. Sign in to Teams by using the Contact Person's email address.
4. In Service Manager, update the Incident's Status to **Resolved**.
5. The integration user sends a Teams message and claims **Incident *xxx* has been resolved**.

1.14. Atlassian Jira

Jira Software is a powerful work management tool for all kinds of use cases, from requirements and test case management to agile software development. The SM-Jira integration seamlessly incorporates Service Manager Incident/Change Management into Jira capabilities of bug/new feature tracking and improves the efficiency of agile teams. The overall intent of this integration is as follows.

- Automates bidirectional ticket information flow and eliminates dependency on manual updates between Service Manager and Jira
- Provides real-time visibility of ticket creation and status updates and ensures complete traceability of the tickets
- Increases collaboration across multiple Service Manager teams and Jira DevOps through the sharing of comments, attachments, and statuses in both applications
- Reconciles and resolves customer issues more efficiently and leverages the best functionalities of the delivery ecosystem

Out-of-the-box, this integration includes the following scenarios.

- Synchronize Service Manager Incident with Jira bug (SM Incident → Jira bug)
- Synchronize Service Manager Change with Jira new feature (SM Change → Jira new feature)

This section describes how to configure Service Manager and Jira for integration and how to test the synchronization. The target readers include Consultants and Application Administrators who must set up and maintain Jira to ensure that the synchronization meets all user organization procedural requirements. Based on the assumption that the reader is an experienced user of either Service Manager or Jira (but not necessarily of both), this section describes only the basics of Service Manager and Jira.

Sample use case

The following typical use cases shed some light on how you can use this integration in your organization.

SM Incident → Jira bug

A business owner creates an Incident record in Service Manager. On the **Jira** tab of the Incident record, this business owner selects the **Incident To Bug** action, and then selects the Jira project. After saving these updates, the system synchronizes the Incident record to a bug in Jira. Developers and QA personnel can begin working on the bug in Jira with the key information synchronized from Service Manager.

The integration administrator has the responsibility of determining the key information and specifying the information in the field mapping (using the Integration Manager) for the business owner to view updated (scheduled) information in Service Manager. The information includes but isn't limited to the Incident record's title, description, severity, and Jira Phase.

For a complete use case, see Synchronize Service Manager Incident to Jira bug.

SM Change → Jira new feature

A business owner creates a Normal Change request in Service Manager. On the **Jira** tab of the Change request, this business owner selects the **Change To New Feature** action, and then selects the Jira project. After saving these updates, the system synchronizes the Change request to a feature in Jira. Developers and QA personnel can begin working on the feature in Jira with the key information synchronized from Service Manager.

Similar to the SM Incident → Jira bug use case, the integration administrator has the responsibility of determining the key information and specifying the information in the field mapping (using the Integration Manager) for the business owner to view updated (scheduled) information in Service Manager. The information includes but isn't limited to the Change request's title, description, severity, and Jira Phase.

For a complete use case, see Synchronize Service Manager Change to Jira new feature.

1.14.1. Configure the settings in Jira

User role: Jira Project Administrator

See the following prerequisites that you must follow when setting up the integration between Service Manager and Jira. Ensure that you have working knowledge of Jira before you proceed.

Detailed instructions about Jira administration are outside the scope of the OpenText product suite and are hence not covered. For more information, see the Jira documentation.

Set up API Token or Personal Access Token

In Jira Cloud, you need to set up API access for Service Manager and grant Service Manager a registered API token so that it can access Jira;

In Jira DataCenter, you need to set up Personal Access Token for Service Manager and grant Service Manager a registered Personal Access Token so that it can access Jira DataCetner.

In the integration, Service Manager uses the API token or Personal Access Token for authentication when synchronizing information with Jira.

Follow these steps to set up API Token on Jira Cloud:

1. Sign in to Jira as a project administrator.
2. Click **Your profile and settings** > **Manage account**, and then click **Security**.
3. Click **Create and manage API tokens** under API token, and then click **Create API token**.
4. Enter an appropriate label for the token, and then click **Create**.
5. Click **Create**. A dialog displays with a new API token.
   Click **Copy** to copy this token to the clipboard so you can use it when configuring Service Manager that needs to access Jira.
6. Click **Close**. The system adds the API token to the list.

Follow these steps to set up Personal Access Token on Jira Data Center:

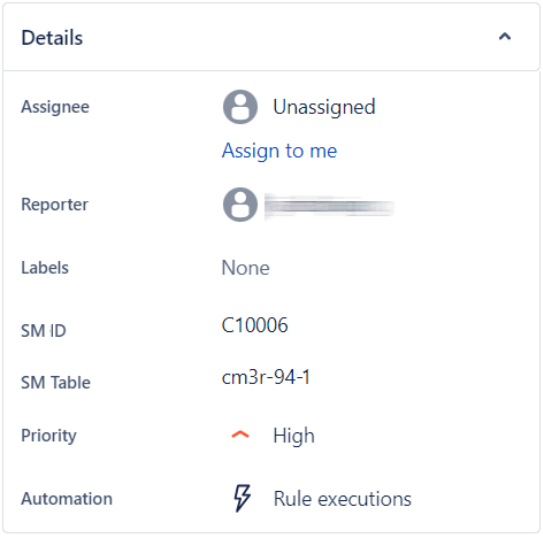
1. Sign in to Jira as a project administrator.
2. Select your profile picture at the top right corner of the screen, and then choose **Profile**.
3. Once you access your profile, select **Personal Access Tokens** in the left-hand menu.
4. Select **Create token**.
5. Give your new token a name.
6. Optionally, for security reasons, you can set your token to automatically expire after a set number of days. The EXPIRES SOON status appears 5 days before the actual expiration

date. Once the token expires, you can't reactivate it. You must create a new token.

7. Click **Create**.
8. Your personal access token is created. Copy the **token** and store it in a safe space. Once you click **Close** you won't be able to see your Token again.

Add custom fields

You need to manually add the **SM ID** and the **SM Table** fields to Jira. Hence, the synchronized records in Jira can display the corresponding Service Manager ticket ID and the name of the module table (for example, `cm3r`). See the following screenshot as a sample display of these two fields in Jira:



Follow these steps to add custom fields:

1. Sign in to Jira as a project administrator.
2. Click **Settings** > **Issues**, and then click **Custom fields** under **FIELDS**.
3. Click **Create custom field**, and the **Select a Field Type** window opens.
4. Select **Text Field (read only)**, and then click **Next**.
5. Enter a name for the custom field (for example, **SM ID**), and then click **Create**.
6. Associate the SM ID field with the appropriate screens of your project. Select the required screens in the list, and then click **Update**.
7. Continue to follow *step 2* to *step 6* to add the **SM Table** field and associate it with the appropriate screens.
8. Obtain the ID of the **SM ID** and **SM Table** fields.

   Locate the SM ID field in the Custom fields list and then click the **More** (•••) icon to the right of this field. Select the **Edit details** option and then observe the URL in your browser. This URL contains the ID of the field. For example, `https://<Jira server`

name>.atlassian.net/secure/admin/EditCustomField!default.jspa?id=10009 . In this case, the ID of the **SM ID** field is 10009. Record this ID for later use when configuring Service Manager Integration Suite (SMIS) settings in Service Manager.

Continue to follow this step and obtain the ID of the **SM Table** field.

Tip: To hide the custom fields, go to your project's **Project settings** and then edit the issue's layout.

1.14.2. Configure the settings in Service Manager

User role: Service Manager System Administrator

You must complete the following tasks from your Service Manager system to support the integration.

Add an integration instance

To set up Jira integration, you must add and enable an instance of the Jira integration in Service Manager Integration Manager.

To add and enable a Jira integration instance, follow these steps:

1. Click **Tailoring** > **Integration Manager** to open Integration Instance Manager.
2. Click **Add**. The Integration Template Selection wizard opens.
3. Select **SMJira** from the Integration Template list, and then select **Import Mapping**.
4. Click **Next**. The Integration Instance Information page opens.
5. Complete the following settings on the Integration Instance Information page as necessary.

Setting	Value	Description
ID	System-generated.	ID of the integration instance. Copy this ID for later use.
Name (required)	User-defined.	Name of the integration instance. Default: SMJira. **Note:** To integrate multiple Jira projects, you can add more integration instances as necessary. Make sure to use a unique name for each SM-Jira integration instance.
Version (required)	1.0	The version number of the integration.
Interval Time (s) (required)	User-defined.	The interval time that the integration background schedule runs. For example, if you set the value to 300, the background process tries to execute the SMIS integration task every 5 minutes. Default: 60.
Max Retry Times	User-defined.	Maximum allowed number of retries if the background scheduler fails to run. Default: 5. **Note:** The Jira integration doesn't use this option.

Setting	Value	Description
Log Level	Select a level from the drop-down list.	Level of diagnostic information that the Service Manager server logs to the log file directory. The possible log levels are **DEBUG**, **INFO**, **WARNING**, **ERROR**, and **OFF**. Default: INFO.
Log File Directory	Specify a directory.	This is a directory that exists on the Service Manager server host where log files are stored. The log includes information for receiving and processing messages. For example: c:\Jira_logs. If this field is empty, the system uses the Service Manager sever log (`sm.log`) for Jira integration.

Setting	Value	Description
Shared Scheduler	User-defined.	The scheduler that the integration instance uses to schedule background processes. Multiple integration instances can use a shared scheduler. When you create an integration template, you can define a default shared scheduler for the instances that are based on the template. All instances that are based on this template can use the same shared scheduler. However, if you create an instance that uses a different scheduler, SMIS creates a new scheduler accordingly. If this field is empty, the system uses <Jira integration name + instance ID> for Jira integration.
Run at system startup	True or false.	Automatically enable or disable the integration instance when the Service Manager server starts.
Description	User-defined.	Description of the integration instance.

6. Click **Next**. The Integration Instance Parameters page opens.
7. Complete the following settings on the **Connectivity** tab as necessary.

Setting	Value	Description
Deploy Type	Specify Jira deploy type	The type that Jira is deployed: Data Center or Cloud
Base URL	Paste the base URL of the Jira server.	The base URL of the Jira server. The URL has the following format: https://*<Jira server name>*:*<port>* For example, https://*<registered site>*.atlassian.net:443. Replace *<Jira server name>* with the IP address or fully-qualified domain name (FQDN) of the Jira server, and replace *<port>* with the communications port the Jira server uses.
Email / Username	Type an email address if Jira is deployed on Cloud; Type the username if Deploy Type is Jira Data Center	The email address or user name that's used for the Jira account.
Token	Paste a token string.	The token you received when setting up for Service Manager. If Deploy Type is Cloud, paste yor API Token; If Deploy Type is DataCenter, paste your Personal Access Token.

Setting	Value	Description
Number of threads	Specify the number of threads.	The number of threads in a SMIS instance. You can adjust the number of threads if a SMIS instance needs to process many tasks. For more information, see Service Manager Jira Software (Jira) integration deployment sizing. The minimum number is 1 and the maximum number is 4. For example, if the number of threads is 4, there are 4 background processes to process the task of this instance. All tasks distribute evenly to different background processes. Default: 1.
Proxy Host	Specify an IP address or domain name.	If you need to use a proxy server to connect to the Jira server from the Service Manager server, type the FQDN or IP address of the proxy server in this field.
Proxy Port	Specify a port number.	The port number of the proxy server.
Proxy User Name	Type an account name.	The user name that's used to connect to the proxy server.
Proxy Password	Type a password.	The password that's used to connect to the proxy server.

8. Click **Verify Connection** to verify the connection settings.
9. Complete the following settings on the **Modules** tab as necessary.

Option	Description
Jira Project	The key of the Jira project.
Project Alias	A unique alias of the Jira project. Later, when setting up an SM-Jira synchronization, you need to select this alias of your Jira project on the **Jira** tab in a Change or an Incident record. **Note:** After cloning an SM-Jira integration instance in SMIS, make sure to update this field so that the project alias remains unique.
SM Table Field	The Jira field that displays the corresponding Service Manager table name. For example, if the **SM Table** field's ID in Jira is 10008, the value here is customfield_10008. For more information about how to get the ID of a custom field, see Add custom fields.
SM ID Field	The Jira field that displays the corresponding Service Manager ticket ID. For example, if the **SM ID** field's ID in Jira is 10009, the value here is customfield_10009. For more information about how to get the ID of a custom field, see Add custom fields.
Exclude Status	Review and update the Jira record's status for the selected options. Jira stops updating a bug or a new feature if it's in any of the listed statuses.

Option	Description
Enable Incident Module	Select whether to enable the synchronization from SM Incident to Jira. The out-of-the-box SM-Jira integration synchronizes a Service Manager Incident to a Jira bug.
Enable Change Module	Select whether to enable the synchronization from SM Change to Jira. The out-of-the-box SM-Jira integration synchronizes a Service Manager Change to a Jira new feature.

10. Click **Next**. The Integration Instance Fields page opens and displays the out-of-the-box fields available for both SM and Jira of the integration. You can review, update, or add more fields as required.
11. If you have selected more than one module in *step 9*, the Select Module page opens. You can either double-click a module or select a module from the list and then click **Open Field Mapping** to open the Integration Instance Fields page. Otherwise, if you have selected only one module in *step 9*, the Integration Instance Fields page opens directly.

    For your selected module, the system displays the out-of-the-box fields available for both SM and Jira of the integration. You can review, update, or add more fields as required.
12. Click **Next**. The Integration Instance Field Mappings page opens.
13. Review, update, or add more fields to the **Field Mapping** tab as necessary. For more information, see Manage mappings.

    To map fields:

    1. Select an SM field and a Jira endpoint field.
    2. Select a direction in the **Direction** column.
    3. (Optional) If an SM field to be mapped has multiple value selections (such as Urgency), specify a name in the **Value Mapping Group** column and then configure the value mappings on the **Value Mapping** tab.
14. Review, update, or add more fields to the **Value Mapping** tab as necessary. For more information, see Manage mappings.

    To map values:

    1. Enter a value mapping group name as you have specified on the **Field Mapping** tab.
    2. Enumerate the value mapping pairs between an SM field and a Jira endpoint field.
    3. Set the condition to true.
15. Click **Finish**. Service Manager creates the instance.

16. Enable the integration instance. A processor for the Jira integration instance now runs in the background.

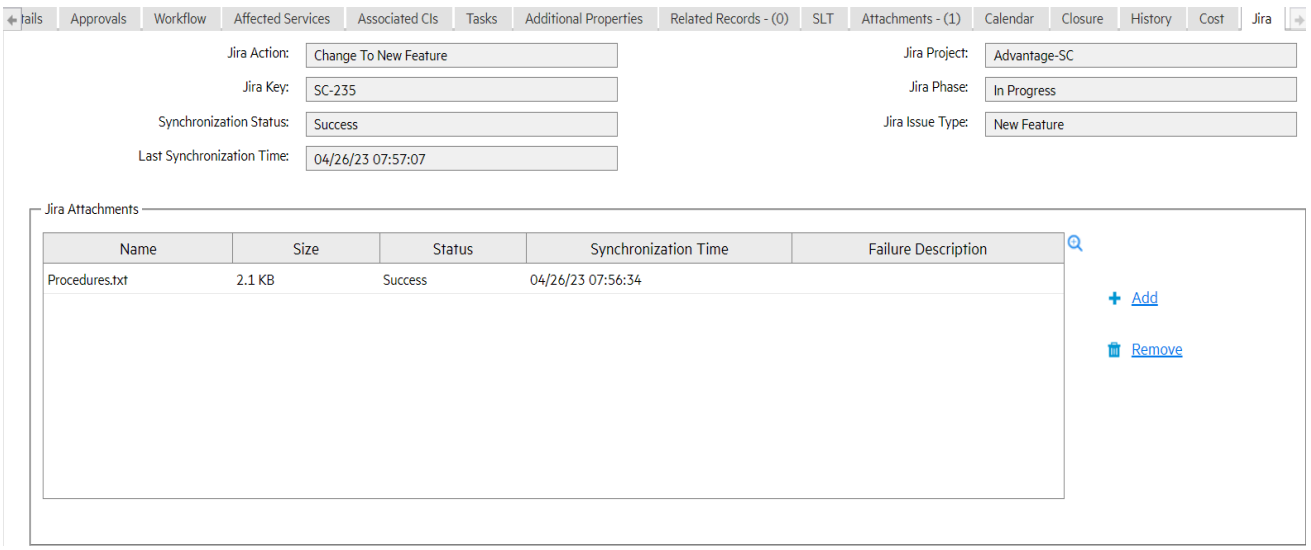
(Optional) Tailor the integration for more user scenarios

The out-of-the-box SM-Jira integration synchronizes a Service Manager Incident to a Jira bug. Meanwhile, it also synchronizes a Normal Change request that's in the **Build and Test** phase in Service Manager to a Jira new feature. To increase the usability and flexibility of this integration, you can tailor the integration to work with more user scenarios.

Scenario 1: Use Jira integration with more Change categories

Regardless of the Change request's category, all you need to do is to add the **Jira** tab to a target workflow phase of a Change request.

In the SM-Jira integration, Service Manager uses the **Jira** tab in a Change record to define the Jira synchronization action and Jira project. This tab also displays a series of synchronization information, including but isn't limited to the Jira Key, Synchronization Status, and Jira Issue Type. Meanwhile, you can also select the attachments of a record on the **Jira** tab so as to synchronize them to Jira. The following screenshot illustrates the Jira tab after Service Manager synchronizes a Change record to a Jira new feature.



To display the Jira tab in any workflow phase of a Change request, you need to manually add this tab to the corresponding form of that phase. Next, add the corresponding rule sets to the target workflow phase. Refer to the following steps for some guidelines.

1. Click **Tailoring** > **Forms Designer**.
2. Search for the corresponding form of a phase. For example, to display the Jira tab when a Change enters the Validation phase, search for the **chm.normal.registration** form.
3. Add a Notebook Tab widget to your Change format, and then set the following property of the Notebook Tab widget.

Property	Value
Visible Condition	[$smJira]

4. Add a subform to the Notebook Tab widget, and then set the following property of the subform control.

Property	Value
Format	chm.subform.jira

5. Save your changes.

Scenario 2: Synchronize a Normal Change request to Jira improvement

You may hope to expand the SM-Jira integration to cover more synchronization cases, such as synchronizing a Normal Change request that's in the **Build and Test** phase to Jira improvement. In this case, you need to update the Jira issue types that the Change module supports. Refer to the following steps for some guidelines.

1. Click **Tailoring** > **Script Library**.
2. Search for the **SMJiraCustomerConfig** JavaScript.
3. Update `function supportJiraTypes(module)` as follows.

```
function supportJiraTypes(module) {
    if (module == "cm3r") {
        vars.$jiraActions = ["Improvement"];
        vars.$jiraActionsDisplay = ["Change To Improvement"];
    } else if (module == "probsummary") {
        vars.$jiraActions = ["Bug"];
        vars.$jiraActionsDisPlay = ["Incident To Bug"];
    }
}
```

   Hence, the **Change To Improvement** option appears in the **Jira Action** drop-down list on the Jira tab. You can select this option manually when synchronizing a Normal Change request that's in the Build and Test phase to Jira improvement.

4. For the system to select **Change To Improvement** option automatically when synchronizing a Normal Change request that's in the Build and Test phase to Jira improvement, update `function changeAutoSync(file)` as follows.

```
function changeAutoSync(file) {
    var assignmentGroup = file["assign.dept"];
    var projects =
jiraUtil.getAllEnabledInstance(funcs.filename(file));
    var isEnable = isInstanceEnable(projects);
    if (isEnable && assignmentGroup == "Application") {
        vars.$jiraProject = projects[0];
        vars.$jiraAction = "Improvement";
    }
}
```

1.14.3. Manage mappings

User role: Service Manager System Administrator, Jira Project Administrator

In the Service Manager and Jira integration, a mapping consists of field mapping and value mapping.

Field Mapping

The **Field Mapping** tab defines the mapping between Jira fields and SM fields.

Field Mapping columns

The following table describes the columns on the **Field Mapping** tab.

Column Name	Description
SM Field	The name of a field in Service Manager, and you can select a value from the list.
Direction	Indicates the mapping direction of a field mapping, and has only three values to select from. • →: from SM Field to Jira Field. • ←: from Jira Field to SM Field. • ↔: in both directions.
Endpoint Field	The name of a field in Jira. You can select a field from the list.
SM Default	Defines the default value of the SM Field if there is no value or there is a constant in the corresponding Jira Field when mapping a Jira Field to an SM Field. **Note:** If the value is a date type, the format must resemble either `2022-06-18T00:00:00Z` or `2022-06-18T00:00:00+08:00`.
Endpoint Default	Defines the default value of a Jira Field if there is no value or there is a constant in the corresponding SM Field when mapping an SM Field to a Jira Field. **Note:** Jira time types include `date` and `datetime`, and the SM-Jira integration supports synchronization between SM date and Jira `datetime`. If the value is a date type, the format must resemble either `2022-06-18T00:00:00Z` or `2022-06-18T00:00:00+08:00`. However, if the value in Jira is of date type, Jira and SM need to be in the same time zone.

Column Name	Description
Value Mapping Group	The name of a value mapping group to which a Jira Field and an SM Field belong. A value mapping group contains a collection of enumerated Jira and SM value pairs. The Service Manager value and Jira value will map according to the configuration on the **Value Mapping** tab. **Note:** You must configure a value mapping group if the type of an SM Field is different from that of a Jira Field.
Description	Enter a text description for this field mapping.

Value Mapping

The **Value Mapping** tab defines the value mapping between Jira and SM for those fields that have multiple values.

The following describes the columns on this tab.

Column Name	Description
Value Mapping Group	The name of a value mapping group. A value mapping group contains a collection of enumerated Jira/SM value pairs.
SM Values	The values of the Service Manager fields.
Endpoint Values	The values of the Jira fields.
Description	Description of the value mapping.
Condition	SMIS enables you to configure conditions for value mappings. You can set the **Condition** field of a value mapping entry to one of the following strings: • `true`: The value mapping defined in this entry is always effective. • `false`: The value mapping defined in this entry is always ineffective. • A string that uses placeholder: The value mapping defined in this entry is effective only when the condition is met. For more information about how to use placeholders, see Use placeholders. For more information about how to configure a condition, see Configure a condition.

For example, below is a value mapping named **IncidentPriority**, and there are five mapping entries for this value mapping group.

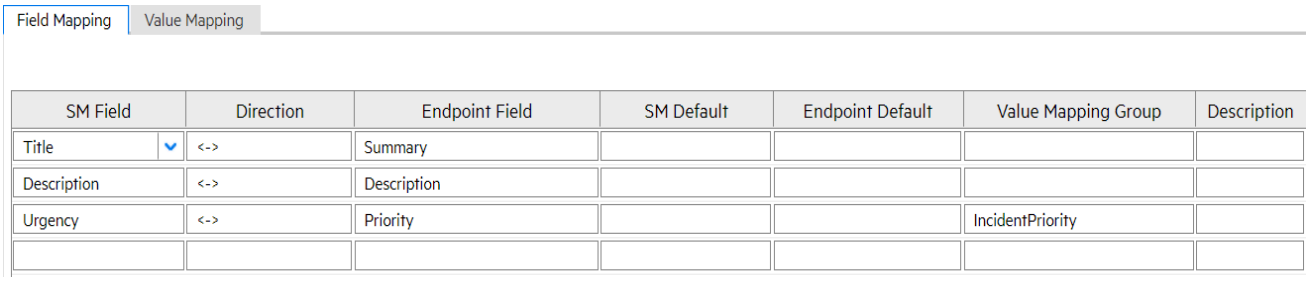
Field Mapping | Value Mapping

Value Mapping Group	SM Values	Endpoint Values	Description	Condition
IncidentPriority	1	{"id":"1"}		true
IncidentPriority	2	{"id":"2"}		true
IncidentPriority	3	{"id":"3"}		true
IncidentPriority	4	{"id":"4"}		true
IncidentPriority	4	{"id":"5"}		true

Note: In Endpoint Values, if the string is of object type, the value must be in a `{"id":"<id number>"}` format. See the *Example: Add value mappings for an SM field of Boolean type*

section below for more information about how to get the endpoint values in Jira.

You can fill the name **IncidentPriority** in the **Value Mapping Group** column on the **Field Mapping** tab.



This means when mapping the Priority field between a Service Manager Incident and a Jira bug, if the value of the Urgency field in Service Manager is `3`, the value `3` maps to `3` for the Priority field in Jira, and vice versa.

Example: Add value mappings for an SM field of Boolean type

The following steps provide some guidelines about how to manage the mapping for an SM field of boolean type.

1. On the Field Mapping tab, fill in the name of the boolean field and define the synchronization direction.
2. Define a custom field in Jira.
    1. Click **Settings** > **Issues**, and then click **Custom fields** under **FIELDS**.
    2. Click **Create custom field**, and the **Select a Field Type** window opens.
    3. Select **Select List (single choice)**, and then click **Next**.
    4. Enter a name for the custom field (for example, **Booleantest**), add two options **True** and **False**, and then click **Create**.
3. Fill in the Endpoint Field column with Booleantest.
4. In Jira, locate the Booleantest field in the Custom fields list and then click the **More** (•••) icon to the right of this field. Select the **Contexts and default value** option and then click **Edit Options**.

    Click **Edit** for the **True** option and then observe the URL in your browser. This URL contains a value of the option. For example, `https://<Jira server name>.atlassian.net/secure/admin/EditCustomFieldOptions!edit.jspa?fieldConfigId=xxxx&atl_token=xxxx&selectedValue=10113`. In this case, the value of the **True** option is 10113. Record this value.

    Similarly, find and record the value of the **False** option. For example, 10114.

5. Specify a name in the **Value Mapping Group** column and then configure the value mapping as follows on the **Value Mapping** tab.

Value Mapping Group	SM Values	Endpoint Values
<Value mapping group name>	true	{"id":"10113"}
<Value mapping group name>	false	{"id":"10114"}

6. Save your changes.

1.14.4. Verify the integration

User role: Service Manager system administrator

The following synchronization tasks describe how to verify the SM-Jira integration. Pay attention to the following notes before proceeding with these tasks.

1. **Attachments and new updates synchronization**

   The synchronization process for attachments and new updates is unidirectional and incremental. That is, you can only synchronize attachments and new updates from Service Manager to Jira only. Meanwhile, if you remove attachments and new updates from Service Manager after synchronization, the attachments and comments in Jira aren't affected.

2. **Formatting synchronization**

   New Update

   You can enter notes in the New Update field with standard HTML formatting. However, when synchronizing the New Update field from Service Manager to Jira, the HTML formatting of the notes is lost by design and these notes appear as plain text in Jira comments.

   Description

   Out-of-the-box, the synchronization process for the Description field is bidirectional. It's notable that your input in the Description field in Service Manager is plain text, whereas the Description field in Jira supports formatting. By design, Service Manager doesn't process the formatting from Jira when synchronizing the Description field. As a result, descriptions with formatting from Jira may have unreadable or data loss issues after synchronization. Hence, keep your input in the Description field in Jira as plain text. If you configure other formatted fields in Jira as the Endpoint Field in field mapping, keep the content in such fields in plain text as well.

SM Incident → Jira bug

The following example verifies the integration by synchronizing an SM Incident to a Jira bug.

1. Synchronize an SM Incident to a Jira bug.
   1. Navigate to Incident Management, and then open an Incident whose status isn't **Closed**.
   2. Click the **Jira** tab. The background synchronization tasks start after filling in the Jira synchronization action and the Jira project. There are two ways: manually or automatically.

      To automatically fill the Jira synchronization action and the Jira project, check that the **SMJiraCustomerConfig** JavaScript exists in Script Library. The automatic

Incident synchronization function in this script may resemble the following.

```
/**
* Update the conditions for automatic synchronization of
Incident
* vars.$jiraProject: The integration instance ID
* vars.$jiraAction: The type of the Jira issue
* Example: If the assignment of the Incident is "Application",
 *          The current Incident record will be synchronized to
Jira's SMBT project as a Bug.
*/
function incidentAutoSync(file) {
    var assignment = file["assignment"];
    var projects =
jiraUtil.getAllEnabledInstance(funcs.filename(file));
    var isEnable = isEnstanceEable(projects);
    if (isEnable && assignment == "Application") {
        vars.$jiraProject = projects[0];
        vars.$jiraAction = "Bug";
    }
}
```

You can update the parameters for `isEnable && assignment`, `vars.$jiraProject`, and `vars.$jiraAction` as required to redefine the conditions for automatic Incident synchronization.

Otherwise, manually select **Incident To Bug** in the **Jira Action** drop-down list, and select the Jira project (project alias) in the **Jira Project** drop-down list. Save this Incident to commence the synchronization process.

3. After Service Manager runs the integration background process, Jira creates a bug. In Service Manager, the Jira Key, Jira Phase, Synchronization Status, Jira Issue Type, and Last Synchronization Time fields on the **Jira** tab have read-only values that are populated automatically.

Field	Description
Jira Key	The bug's ID in Jira.
Jira Phase	The bug's status in Jira.
Synchronization Status	The synchronization status.
Jira Issue Type	Bug.
Last Synchronization Time	The last synchronization time between SM and Jira.

2. Update some fields, add attachments, and then perform synchronization again from the SM Incident to the Jira bug.
    1. Open the Incident record again and then update the title, description, and urgency information.
    2. Click the **Activities** tab. Select **Update to external service** in the **Update Type** drop-down list, and then enter some notes in the **Update** field.
    3. Click the **Attachments** tab and add some attachments to the Incident record. Save your updates.
    4. Click the **Jira** tab and then click **Add** to select the attachments from the current Incident record for Jira synchronization.
    5. Click **Save** and then wait for the next synchronization.
3. View the synchronization result in the Jira bug.

    Click **View in Jira**. Sign in to Jira and open the bug record. Check that your updates to the title, description, and urgency in the SM Incident record are synchronized to the corresponding fields in the Jira bug record. Meanwhile, your notes in the SM **Update** field now appear as comments in the Jira bug record. Your selected attachments in the SM Incident record now appear as the Jira bug record's attachments as well.

For a complete use case, see Synchronize Service Manager Incident to Jira bug.

SM Change → Jira new feature

The following example verifies the integration by synchronizing an SM Change to a Jira new feature.

1. Synchronize an SM Change to a Jira new feature.
    1. Open the workflow of a Normal Change and click the **Build And Test** phase. Make sure that the **chm.jira.enter** rule set exists in **On enter** and the **chm.jira.assignproject** rule set exists in **On update**.

2. Navigate to Change Management, and then open a Normal Change record that's in the **Build and Test** phase.

3. Click the **Jira** tab. The background synchronization tasks start after filling in the Jira synchronization action and the Jira project. There are two ways: manually or automatically.

   To automatically fill the Jira synchronization action and the Jira project, check that the **SMJiraCustomerConfig** JavaScript exists in Script Library. The automatic Change synchronization function in this script may resemble the following.

   ```
   /**
   * Update the conditions for automatic synchronization of Change
   * vars.$jiraProject: The integration instance ID
   * vars.$jiraAction: The type of the Jira issue
   * Example: If the assignmentGroup of the Change is
   "Application",
    *          The current Change record will be synchronized to
   Jira's SC project as a New Feature
   */
   function changeAutoSync(file) {
       var assignmentGroup = file["assign.dept"];
       var projects =
   jiraUtil.getAllEnabledInstance(funcs.filename(file));
       var isEnable = isEnstanceEable(projects);
       if (isEnable && assignmentGroup == "Application") {
           vars.$jiraProject = projects[0];
           vars.$jiraAction = "New Feature";
       }
   }
   ```

   You can update the parameters for `isEnable && assignment`, `vars.$jiraProject`, and `vars.$jiraAction` as required to redefine the conditions for automatic Change synchronization.

   Otherwise, manually select **Change To New Feature** in the **Jira Action** drop-down list, and select the Jira project (project alias) in the **Jira Project** drop-down list. Save this Change to commence the synchronization process.

4. After Service Manager runs the integration background process, Jira creates a new feature. The Jira Key, Jira Phase, Synchronization Status, Jira Issue Type, and Last Synchronization Time fields on the **Jira** tab have read-only values that are populated automatically.

Field	Description
Jira Key	The new feature's ID in Jira.
Jira Phase	The new feature's status in Jira.
Synchronization Status	The synchronization status.
Jira Issue Type	New feature.
Last Synchronization Time	The last synchronization time between SM and Jira.

2. Update some fields, add attachments, and then perform synchronization again from the SM Change to the Jira new feature.
    1. Open the Change record again and then update the title and description information.
    2. Click the **Activities** tab. Select **Update to external service** in the **New Update Type** drop-down list, and then enter some notes in the **New Update** field.
    3. Click the **Attachments** tab and add some attachments to the Change record. Save your updates.
    4. Click the **Jira** tab and then click **Add** to select the attachments from the current Change record for Jira synchronization.
    5. Click **Save** and then wait for the next synchronization.
3. View the synchronization result in the Jira new feature.

    Click **View in Jira**. Sign in to Jira and then open the new feature record. Check that your updates to the title and description in the SM Change record are synchronized to the corresponding fields in the Jira new feature record. Meanwhile, your notes in the SM **New Update** field now appear as comments in the Jira new feature. Your selected attachments in the SM Change record now appear as the Jira new feature record's attachments as well.

For a complete use case, see Synchronize Service Manager Change to Jira new feature.

© Copyright 2026 Open Text

For more info, visit https://docs.microfocus.com